

Advanced Windows Debugging. Русский перевод

Русский перевод книги Advanced Windows Debugging.

Больше крутых материалов в моём телеграм-канале [@grogrigs](#)

Марио Хевардт
Даниэль Прават

Upper Saddle River, NJ - Boston - Indianapolis - San Francisco
New York - Toronto - Montreal - London - Munich - Paris - Madrid
Cape Town - Sydney - Tokyo - Singapore - Mexico City

Сведения об издании

Многие обозначения, используемые производителями и продавцами для различения своих продуктов, заявлены как товарные знаки.

Там, где такие обозначения встречаются в этой книге и издателю было известно о заявке на товарный знак, эти обозначения напечатаны с начальными заглавными буквами или полностью заглавными буквами.

Авторы и издатель проявили тщательность при подготовке этой книги, но не дают никаких явных или подразумеваемых гарантий любого рода и не принимают на себя ответственности за ошибки или упущения.

Не принимается ответственность за случайные или косвенные убытки, связанные с использованием информации или программ, содержащихся здесь, либо возникшие в результате такого использования.

Издатель предлагает отличные скидки на эту книгу при заказе в количестве для оптовых закупок или специальных продаж, которые могут включать электронные версии и/или специальные обложки и содержимое, адаптированные под ваш бизнес, цели обучения, маркетинговый фокус и интересы брендинга.

За дополнительной информацией обращайтесь:

U.S. Corporate and Government Sales
(800) 382-3419
corpsales@pearsoned.com

Для продаж за пределами США обращайтесь:

International Sales
international@pearsoned.com

Посетите нас в Web: www.awprofessional.com

Библиографическая запись Library of Congress

Hewardt, Mario.
Advanced windows debugging / Mario Hewardt, Daniel Pravat.
p. cm.
Includes index.
ISBN 0-321-37446-0 (pbk. : alk. paper)
1. Microsoft Windows (Computer file)
2. Operating systems (Computers) - Management.
3. Debugging in computer science.
I. Pravat, Daniel. II. Title.
QA76.76.063H497 2007
005.4'46 - dc22
2007030163

Copyright (c) 2008 Pearson Education, Inc.

Все права защищены.

Напечатано в Соединенных Штатах Америки.

Эта публикация защищена авторским правом, и перед любым запрещенным воспроизведением, хранением в системе поиска или передачей в любой форме и любыми средствами, электронными, механическими, фотокопированием, записью или аналогичными способами, необходимо получить разрешение издателя.

По вопросам разрешений пишите:

Pearson Education, Inc.
Rights and Contracts Department
501 Boylston Street, Suite 900
Boston, MA 02116
Fax (617) 671 3447

Этот материал может распространяться только в соответствии с условиями Open Publication License, v1.0 или более поздней версии.

Последняя версия на момент публикации доступна по адресу

<http://www.opencontent.org/openpub/>.

ISBN-13: 978-0-321-37446-2

ISBN-10: 0-321-37446-0

Текст напечатан в Соединенных Штатах на переработанной бумаге в Edwards Brothers, Ann Arbor, Michigan.

Первое издание: октябрь 2007.

Редакционная группа

Роль	Имя
Editor-in-Chief	Karen Gettman
Acquisitions Editor	Joan Murray
Senior Development Editor	Chris Zahn
Managing Editor	Gina Kanouse
Copy Editor	Rhonda Tinch-Mize
Indexer	Brad Herriman
Proofreader	Karen A. Gill
Editorial Assistant	Kim Boedigheimer
Cover Designer	Chuti Prasertsith
Composition	TnT Design

Посвящения

Моей жене Пии, чья поддержка, терпение и ободрение помогли сделать эту книгу реальностью.
Моей семье, которая учила и поощряла меня следовать моим мечтам и увлечениям.

Марио Хевардт

Клаудии, Алексис и Эдварду.

Даниэль Прават

Предисловие

У программного обеспечения есть одна цель: упрощать.

Если существует рабочий процесс, который можно оптимизировать или автоматизировать, данные, которые можно хранить или обрабатывать эффективнее, программное обеспечение берется за эту работу.

Упрощая, программное обеспечение не должно вносить ненужную сложность, и поэтому оно должно устанавливаться с минимальным участием пользователя, бесшовно интегрировать службы и данные из других приложений и множества источников, а также быть устойчивым к изменениям в своей программной и аппаратной среде.

По большей части программное обеспечение просто волшебным образом работает.

Однако, стремясь упростить опыт конечных пользователей и администраторов, программное обеспечение становится все более и более сложным.

Будь то объем данных, с которыми оно работает, количество приложений, с которыми оно взаимодействует, степень внутреннего параллелизма или API, которые оно прямо и косвенно импортирует из программного стека, на котором выполняется, большая часть кажущейся простоты программного обеспечения скрывает мир тонких временных зависимостей, зависимостей и предположений, проходящих между слоями программного обеспечения, часто через разные приложения и даже компьютеры.

Просто определить, какой компонент виноват, не говоря уже о том, почему это произошло, когда проблема проявляется как сбой в библиотеке, бессмысленное сообщение об ошибке или зависание, часто оказывается пугающей задачей.

Причина, по которой вы читаете эту книгу, состоит в том, что вы разрабатываете, тестируете или поддерживаете программное обеспечение, а значит сталкиваетесь с поломками в его бесчисленных движущихся частях, которые вам поручено расследовать до корневой причины и, возможно, исправить.

Успех в этом деле означает максимально быстрое и эффективное определение источника проблемы, а это требует знания, чем смотреть, где смотреть и как смотреть.

Иными словами, успех означает знание того, какие инструменты находятся в вашем распоряжении, какие из них наиболее эффективны для определенного класса отказов, и как применять возможности и функциональность инструмента, чтобы быстро сузить поиск источника проблемы.

Обучение устранению неполадок и отладке приложений Windows на рабочем месте по большей части было единственным вариантом, но когда вы отлаживаете отказ приложения, знание одного малоизвестного инструмента или специфичной для сценария команды отладчика может означать разницу между мгновенным пониманием проблемы и часами или даже днями безуспешной охоты за ней.

Именно поэтому такая книга многократно окупает себя.

Advanced Windows Debugging берет объединенные знания и годы практического опыта не только Марио и Даниэля, но также Microsoft Customer Support Services и команд разработки Windows-продукта и инструментов, и кладет их вам под руку.

Нет более авторитетного места, чтобы узнать, например, как диспетчер кучи Windows влияет на поведение переполнений буфера или какую команду расширения отладчика следует использовать для устранения зависаний DCOM.

Я отлаживаю собственные приложения Windows и драйверы устройств уже более десяти лет, но, просматривая рукопись, узнал о новых методиках, инструментах и командах отладчика, с которыми раньше не сталкивался и которым уже нашел применение.

Мы все зарабатываем свою оплату и репутацию не тем, как мы отлаживаем, а тем, насколько быстро и точно мы это делаем.

Отлаживаете ли вы приложения Windows уже много лет или только начинаете, Марио и Даниэль хорошо снаряжают вас для ваших экспедиций по охоте за ошибками.

Удачной охоты!

Марк Руссинович

Technical Fellow, подразделение Platform and Services

Microsoft Corporation

Вступление

Не так давно мы вспоминали одну действительно сложную проблему, с которой столкнулись на работе.

Команда Quality Assurance выполняла стресс-тесты нашего продукта, и каждые четыре или пять дней аварийное завершение снова поднимало свою уродливую голову.

Конечно, мы отладили сбой настолько далеко, насколько считали возможным, и провели обширные ревью кода, пытаясь во всем разобраться, но, увы, информации все равно было недостаточно, чтобы добраться до сути.

После нескольких недель бесплодных попыток мы начали искать альтернативные подходы.

Во время случайного разговора в коридоре кто-то вскользь упомянул инструмент под названием gflags.

Никогда прежде не слышав об этом инструменте, мы решили провести исследование и выяснить, как он может помочь нам добраться до сути нашего сбоя.

К сожалению, процесс обучения оказался несколько трудным.

Во-первых, найти информацию об инструменте оказалось настоящим испытанием.

В справочной документации, поставляемой с инструментами, было множество отличной информации, но было трудно понять, как на самом деле начать.

Мы быстро поняли, что без базового руководства у нас мало надежды суметь использовать инструмент.

Естественно, мы решили спросить человека, который случайно упомянул инструмент, знает ли он какую-либо документацию или подсказки.

Он дал нам несколько кратких описаний инструмента и, возможно еще важнее, имена других людей, которые много работали с этим инструментом.

За этим последовала серия долгих и поучительных разговоров, и постепенно базовая идея инструмента начала становиться на место.

Добрались ли мы когда-нибудь до сути сбоя?

Да, добрались.

Фактически включение правильного инструмента во время выполнения стресс-тестов указало на проблему с такой точностью, что для нахождения и исправления неправильного кода понадобился всего час ревью.

Если бы мы с самого начала знали об этом инструменте и о том, как им пользоваться, мы сэкономили бы несколько недель работы.

С этого момента мы посвятили довольно много времени дальнейшему пониманию инструментов и того, как они могут помогать при устранении неполадок в неправильно работающем коде.

За прошедшие годы отладчики и инструменты Windows созрели, выросли и стали все более мощными.

Количество возможностей экономии времени, доступных сейчас, действительно поражает воображение.

Не менее поразительно и то, что спустя несколько лет нативные отладчики и инструменты все еще относительно неизвестны разработчикам.

Те немногие разработчики, которые узнают о существовании этих инструментов, должны пройти через такой же болезненный процесс обучения, через который мы прошли годы назад.

Нам повезло иметь роскошь работать с инженерами Microsoft (некоторые из которых писали эти инструменты), но без этой роскоши многие полные надежд разработчики упираются в тупик и так и не могут воспользоваться преимуществами инструментов.

Эта печальная проблема нехватки учебных материалов также оказалась прекрасной возможностью для решения, и так родилась идея этой книги.

Ключ к тому, чтобы дать разработчикам необходимые знания, - предоставить центральное хранилище краткой информации, полностью объясняющей все тонкости отладочных инструментов и процессов.

Книга, которую вы держите в руках, служит этим ключом и является конечным результатом трех лет написания и более чем пятнадцати лет совместного отладочного опыта.

Мы надеемся, что вам понравится читать эту книгу так же, как нам понравилось ее писать, и что она откроет дверь в действительно удивительный мир высокоэффективного устранения неполадок и отладки программного обеспечения.

Знание того, как использовать инструменты и методики, описанные в этой книге, является критически важной частью работы специалиста по вычислительной технике и может научить вас очень эффективно устранять некоторые из самых сложных проблем в программном обеспечении.

Для кого эта книга?

Краткий ответ на этот вопрос: для любого, кто участвует в любой стороне разработки программного обеспечения и имеет сильное желание узнать, что на самом деле происходит глубоко внутри Windows.

Хотя техническая природа книги может заставить вас поверить, что ее содержание предназначено только для продвинутых системных инженеров, это совершенно не так.

Один из ключевых пунктов этой книги - устранение магии.

По разным причинам многие инженеры программного обеспечения верят, что между программным обеспечением, над которым они работают, и операционной системой существует магическая связь. Когда возникает проблема, требующая анализа компонентов операционной системы (таких как RPC/COM или диспетчер кучи Windows), это предвзятое представление о магии мешает им заглянуть внутрь Windows, чтобы получить больше информации, потенциально способной помочь решить проблему.

Чтобы эффективно использовать эту книгу, вам придется научиться избавляться от этого предвзятого представления и действительно принять настрой, что за кулисами нет никакой магии. Основные компоненты Windows следует рассматривать как расширение вашего продукта, а не как отдельный и магический слой.

В конце концов, это всего лишь код, часть которого просто написана другими людьми.

Если вы сможете перестроить свое мышление и принять это, вы сделаете первые шаги к овладению искусством отладки Windows.

Разработчики программного обеспечения

Любой, от низкоуровневого системного разработчика до высокоуровневого RAD-разработчика, получит пользу от чтения этой книги.

Предпочитаете ли вы писать программное обеспечение для Windows на языке ассемблера или с использованием .NET Framework, в ней есть масса полезной информации об инструментах и методиках, лежащих в основе отладки Windows.

За прошедшие годы у нас было несколько обсуждений с высокоуровневыми RAD-разработчиками, которые утверждали, что действительно не видят необходимости изучать эти низкоуровневые темы.

В конце концов, красота написания кода на более высоком уровне состоит в том, что все низкоуровневые тонкости абстрагируются и скрываются от разработчика.

Мы полностью согласны.

Однако наше утверждение состоит в том, что, хотя абстрактное программирование позволяет разработчику не сосредотачиваться на низкоуровневых деталях, оно не устраняет необходимости знать, как на самом деле работает абстракция.

Суть этого утверждения проста.

То, с чем вы работаете, действительно является именно этим - абстракцией.

Использование этой абстракции в проекте, для которого она не подходит, может вызвать серьезные проблемы в вашем программном обеспечении; и в таком случае без твердого понимания того, как работает абстракция, это может означать разницу между своевременной поставкой продукта и сдвигом даты выпуска на несколько месяцев.

Другой ключевой фактор при рассмотрении освоения отладчиков и инструментов Windows связан с отладкой живых производственных серверов.

Хотя следует приложить все усилия, чтобы исправить ошибки до поставки продукта, все мы знаем, что некоторые ошибки могут проскользнуть сквозь щели.

Когда эти ошибки проявляются после выпуска, их отслеживание может стать настоящей головной болью.

Клиенты, которые сталкиваются с ошибками на живых производственных серверах, обычно очень чувствительны к простоям и изменениям конфигурации, что делает невозможной установку сложного пакета отладчика.

Debugging Tools for Windows, с другой стороны, обеспечивает живую отладку без изменения конфигурации сервера и без требований к установке.

Короче говоря, он позволяет клиентам сохранять первозданный сервер во время процесса устранения неполадок.

Инженеры Quality Assurance

Так же как разработчики программного обеспечения найдут информацию в этой книге полезной в повседневных задачах, ее найдут полезной и инженеры Quality Assurance.

Quality Assurance обычно выполняет набор тестов для любого тестируемого компонента.

За это время может проявиться любое количество ошибок.

Будь то повреждения памяти, утечки ресурсов или зависания, знание того, какую расширенную инструментализацию включить во время тестового прогона, может резко сократить время, необходимое для анализа корневой причины.

Например, представьте, что Quality Assurance поручено стресс-тестирование службы авторизации кредитных карт.

Одна из целей состоит в том, что служба должна быть способна пережить одну неделю непрерывной и одновременной нагрузки клиентскими запросами.

На шестой день служба начинает сообщать ошибки для всех клиентских запросов.

В этот момент вызывают разработчиков, ответственных за службу, чтобы проанализировать проблему.

Им не требуется много времени, чтобы понять, что у сервера закончилась память, предположительно из-за небольшой утечки памяти, накапливающейся со временем. Однако после шести дней накопившихся утечек выяснение источника утечки становится гораздо более сложной задачей, которая может занять дни отладки и ревью кода. Если бы правильная расширенная инструментализация была включена во время выполнения этих тестов, время, необходимое для анализа утечки, могло бы быть значительно сокращено.

Инженеры поддержки продукта

Примерно так же, как Quality Assurance использует отладчики и инструменты Windows, чтобы сделать анализ корневых причин эффективнее, это могут делать и инженеры поддержки продукта. Поддержка продукта сталкивается со многими из тех же проблем, что и Quality Assurance и разработчики программного обеспечения, изо дня в день.

Ключевое отличие, однако, состоит в ограничениях среды, в которой они работают.

Ограничения могут включать отсутствие полного доступа к серверу, на котором проявляются проблемы, ограниченное количество времени, доступного для устранения неполадок сервера, ограниченный доступ к исходному коду клиента и другие вопросы.

Информация, представленная в этой книге, даст инженерам поддержки продукта большой запас средств при решении этих трудных проблем.

Знание того, как отлаживать проблемы клиентов с минимальным временем простоя и минимальными изменениями конфигурации системы, позволяет инженерам поддержки продукта гораздо эффективнее и ненавязчивее собирать необходимые данные, чтобы добраться до сути проблемы.

Если есть желание, найдется и способ

Не должно удивлять, что материал, представленный в этой книге, имеет крайне техническую природу.

Мы не собираемся убеждать вас, что для получения пользы от книги вам не нужно ничего знать о внутреннем устройстве Windows, потому что простая правда состоит в том, что знать нужно.

Как и в любой технически ориентированной книге, предполагается определенный объем знаний.

Любознательность и желание учиться

Во время написания этой книги мы осознали, что некоторые области Windows, о которых мы писали, воспринимались нами как должное.

Конечно, большую часть времени мы знали, что эти области работают определенным образом, но не знали точно, что заставляет их работать именно так.

Мы могли бы просто принять тот факт, что они просто работают, но любопытство взяло над нами верх (как обычно и бывает).

Мы потратили довольно много времени на исследование тем и попытки соединить точки.

Конечным результатом стало более глубокое понимание Windows, которое, в свою очередь, позволило нам эффективнее отлаживать проблемы.

Базовый принцип изучения чего угодно состоит в том, что должно быть желание учиться.

В зависимости от вашего опыта часть высокоуровневого материала книги может казаться пугающей.

Примите это ощущение, и вы окажетесь в более сильной позиции для полного усвоения и понимания содержания этой книги.

Если у вас есть желание учиться и большая любознательность, вы уже на пути к тому, чтобы стать экспертом по отладке Windows.

C/C++

Весь примерный код в книге написан на C/C++, и поэтому требуется хорошее понимание языка, а также его объектной раскладки.

Если некоторые концепции языка в книге вам незнакомы и вы хотите освежить навыки C/C++, мы рекомендуем следующие книги:

- The C++ Programming Language (3rd Edition), Bjarne Stroustrup, Boston: Addison-Wesley, 2000.
- Inside the C/C++ Object Model, Stanley B. Lippman, Reading, MA: Addison-Wesley, 1996.

Внутреннее устройство Windows

Эта книга посвящена продвинутой отладке Windows, и поэтому части книги посвящены описанию внутреннего устройства нескольких неотъемлемых компонентов Windows (например, диспетчера кучи, RPC, подсистемы безопасности).

Наше намерение состоит не в том, чтобы полностью объяснить все аспекты этих компонентов, а скорее в том, чтобы дать краткое, но глубокое резюме того, как компонент функционирует применительно к иллюстрируемым сценариям отладки.

Если вы хотите продвинуть свои знания внутреннего устройства Windows еще дальше, мы настоятельно рекомендуем прочитать:

- Microsoft Windows Internals, Fourth Edition: Microsoft Windows Server 2003, Windows XP, and Windows 2000, Mark E. Russinovich and David A. Solomon. Redmond, WA: Microsoft Press, 2004.

Организация

Книга состоит из трех основных частей.

В этом разделе мы даем краткое описание содержания каждой главы.

Часть I. Обзор

Часть I закладывает основу.

Она дает обзор инструментов и отладчиков и позволяет познакомиться с фундаментальными принципами отладчиков.

Даже если вы уже знакомы с отладчиками Windows, мы настоятельно рекомендуем вам как минимум бегло просмотреть эти главы, поскольку они содержат массу ценной информации.

Глава 1, "Introduction to the Tools", дает высокоуровневое введение в инструменты, используемые на протяжении книги.

Подробно рассматриваются такие темы, как расположения для загрузки, инструкции установки и сценарии использования.

Глава 2, "Introduction to the Debuggers", знакомит читателя с основами отладчиков Windows.

Рассматриваются базовые концепции, такие как какие отладчики доступны, как их использовать и как их настраивать.

Глава 3, "Debuggers Uncovered", дает более глубокое рассмотрение отладчиков пользовательского режима.

Приведена минималистичная реализация отладчика, а также рассматриваются более продвинутые темы, такие как работа механизма диспетчеризации исключений.

Глава 4, "Managing Symbol and Source Files", обсуждает, как поддерживать две самые критически важные части информации во время отладки: файлы символов и исходные файлы.

Она дает краткое описание того, что такое серверы символов и исходного кода, как использовать их совместно с отладчиками и как эффективно управлять ими, настраивая серверы символов и поддерживая серверы исходного кода для вашей организации.

Часть II. Прикладная отладка

Фокус части II состоит в том, чтобы дать читателю возможность анализировать распространенные ошибки программирования с помощью отладчиков Windows.

Каждая глава в этом разделе сосредоточена на конкретной категории проблем, такой как повреждение памяти, утечки памяти и RPC/COM.

Каждая глава начинается с обзора задействованных компонентов Windows, за которым следуют один или несколько сценариев, иллюстрирующих распространенные ошибки программирования в этой области.

За исключением глав 5 и 6, главы части II являются самостоятельными и могут читаться в любом порядке.

Главы 5, "Memory Corruption Part I - Stacks", и 6, "Memory Corruption Part II - Heaps", подробно рассматривают очень распространенную проблему, ежедневно преследующую разработчиков: повреждения памяти.

Глава 5 сосредоточена на повреждениях стека, а глава 6 - на повреждениях кучи.

Каждая глава начинается с объяснения общей концепции рассматриваемого типа памяти (стек и куча), за которым следует ряд распространенных сценариев, при которых может возникать повреждение.

Каждый сценарий имеет связанный примерный код и пошаговое описание процесса, используемого при отладке и анализе корневой причины.

Глава 7, "Security", обсуждает распространенные проблемы, связанные с безопасностью, которые часто проявляются во время разработки.

Довольно часто разработчики сталкиваются с ситуациями, когда API возвращает код ошибки отказа в доступе без какой-либо более глубокой информации, что затрудняет понимание или отслеживание источника ошибки.

Эта глава покажет несколько связанных с безопасностью примеров кода и то, как использовать отладчики и подходящие инструменты, чтобы добраться до сути проблемы.

Глава 8, "Interprocess Communication", полностью сосредоточена на отладке межпроцессного взаимодействия.

Возможно, самым используемым протоколом межпроцессного взаимодействия в Windows, но также самым магическим, является RPC/LPC.

Знание того, как устранять неполадки этого важного компонента, имеет первостепенное значение при работе с большинством приложений.

С помощью отладчиков эта глава покажет, как отслеживать идентичность, анализировать отказы RPC и многое другое.

Глава 9, "Resource Leaks", подробно рассматривает очень распространенную проблему современного программного обеспечения: утечки ресурсов.

Самая распространенная форма утечек ресурсов связана с памятью, но ею не ограничивается. Другие примеры включают разделы реестра, файловые дескрипторы и так далее.

Эта глава рассматривает проблему утечек ресурсов, показывая ряд сценариев и связанный примерный код, а также то, как использовать отладчики и инструменты для их эффективного отслеживания.

Глава 10, "Synchronization", обсуждает тему зависаний приложений и то, как наиболее эффективно использовать отладчики для отслеживания проблем синхронизации, таких как взаимоблокировки и конкуренция за блокировки.

Рассматривается ряд различных сценариев синхронизации со связанными сеансами отладки, которые дают глубокий взгляд на процесс анализа.

Часть III. Расширенные темы

Часть III - это расширенный раздел, состоящий из глав, которые обсуждают такие темы, как посмертная отладка, 64-битная отладка, основы Windows Vista и многое другое.

Цель этих глав - не дать исчерпывающее рассмотрение каждой области, а скорее предоставить достаточно основ, чтобы читатель мог начать работу с объясняемой темой.

Глава 11, "Writing Custom Debugger Extensions", говорит о пользовательских расширениях отладчика.

Хотя отладчики Windows содержат чрезвычайно мощный набор команд и инструментов, бывают моменты, когда вы хотите автоматизировать определенные аспекты собственных сеансов отладки приложения.

Эта глава подробно описывает, как работает модель расширяемости отладчиков, и описывает пример пользовательского расширения отладчика.

Глава 12, "64-Bit Debugging", вводит базовые концепции отладки 64-битных архитектур.

Обсуждаются базовые концепции, такие как трассировки стека, вызовы функций и передача параметров, чтобы читатель мог начать отладку этих мощных архитектур.

Глава 13, "Postmortem Debugging", обсуждает посмертную отладку, невероятно полезный способ устранения неполадок, когда нет возможности отлаживать проблему в точке возникновения.

Это очень распространенная форма отладки после поставки продукта, когда проблемы проявляются на стороне клиента.

Глава 14, "Power Tools", обсуждает два мощных инструмента, которые можно использовать для автоматизации процесса отладки.

Первый инструмент называется DebugDiag и предоставляет отличный способ автоматизировать отладку утечек ресурсов.

Другой инструмент - команда analyze, автоматизирующая начальный процесс анализа отказа.

Глава 15, "Windows Vista Fundamentals", подробно рассматривает некоторые основы Windows Vista.

С появлением платформы Windows нового поколения некоторые аспекты операционной системы резко изменились, и в этой главе изложены некоторые ключевые изменения.

Необходимые инструменты

Все инструменты, необходимые для полноценного использования этой книги, доступны для бесплатной загрузки.

Новый Windows Drivers Kit содержит полную командную среду разработки C/C++ и отличный набор связанных инструментов разработки.

Примерный код

Как инженеры программного обеспечения, мы проводим большую часть времени в поисках главного сокровища - написания идеального кода.

Во время написания этой книги перед нами стояла почти противоположная задача: необходимость писать не совсем идеальный код, чтобы иллюстрировать распространенные ошибки программирования.

Примерный код структурирован для достижения одной цели: представить примеры распространенных ошибок программирования в самой короткой и краткой форме, чтобы не загрязнять базовый принцип рассматриваемой ошибки программирования.

Чтобы удовлетворить цель коротких и кратких примеров, нам иногда приходилось придумывать примеры, а не использовать реальные.

Хотя примерный код "выдуман", он служит для имитации реальных примеров, и были приложены все усилия, чтобы пример оставался верным рассматриваемой проблеме.

Весь примерный код написан на C/C++.

Мы выбрали этот язык по двум простым причинам:

- C/C++ преимущественно используется в разработке для Windows.
- Чтобы не затемнять обсуждаемые отладочные концепции высокоуровневыми абстракциями, мы выбрали язык, который наиболее распространен и в то же время наиболее близок к ядру.

Весь примерный код скомпилирован и протестирован с использованием Windows Drivers Kit.

WDK был выбран, чтобы читатели могли наслаждаться изучением искусства отладки Windows без необходимости покупать полный комплект разработчика.

Исходный код предполагает среду Unicode, и поэтому вызовы Win32 API, как они видны в отладчике, будут иллюстрироваться с использованием Unicode-версии API.

Например, примерный код может показывать вызов API CreateProcess, но при работе в отладчике будет использоваться API CreateProcessW.

API, показанный в отладчике, предваряется именем модуля, реализующего API.

Один пример - API CreateProcessW, реализованный в kernel32.dll.

Часто требуется указывать и имя модуля, и имя API, разделенные символом !

(kernel32!CreateProcessW).

Весь примерный код и двоичные файлы доступны на Web-сайте книги

(<http://www.advancedwindowsdebugging.com>).

Помимо доступности исходного кода и двоичных файлов, сайт выступает сервером символов и исходного кода для двоичных файлов книги.

Когда вы пробуете сеансы отладки, проиллюстрированные в книге, нет необходимости загружать все символы для двоичных файлов; вместо этого направьте путь символов ваших отладчиков прямо на сервер символов книги, и вы сможете отлаживать с удаленными символами.

Исходные тексты также извлекаются source servers с Web-сайта книги.

Чтобы обеспечить согласованный учебный опыт, двоичные файлы на Web-сайте книги собраны как неоптимизированные checked-релизы для архитектуры x86 с использованием платформы Windows XP.

Мы выбрали Windows XP как общий знаменатель из-за ее широкого распространения.

Если вы решите собрать примеры самостоятельно, используя другую целевую платформу, в отладочном выводе могут быть небольшие различия.

Чтобы собрать примеры самостоятельно, просто откройте окно сборки WDK и введите следующую команду из каталога, содержащего makefile:

```
build /ZCc
```

Если компилируемый исходный код требует дополнительных шагов, эти шаги будут описаны в главе, обсуждающей примерный код.

На протяжении книги предполагается, что все двоичные файлы загружены с Web-сайта и скопированы на локальный жесткий диск (с сохранением структуры папок) в следующее расположение: C:\AWDBIN, а исходные тексты загружены в папку C:\AWD.

Соглашения

Код, действия командной строки и описания синтаксиса в книге отображаются моноширинным шрифтом.

Многие примеры и пошаговые разборы в этой книге показывают большой объем того, что известно как debug spew.

Debug spew просто означает вывод, который отладчик отображает в результате некоторого действия пользователя.

Обычно этот debug spew состоит из информации, показанной в очень компактной и краткой форме. Чтобы эффективно ссылаться на фрагменты этих данных и облегчить вам чтение, используются полужирный и курсивный шрифты.

Кроме того, все, что набрано полужирным в debug spew, указывает команды, которые вы будете вводить.

Следующий пример иллюстрирует механизм.

```
0:000> ~*kb
. 0 Id: 924.a18 Suspend: 1 Teb: 7ffdf000 Unfrozen
ChildEBP RetAddr Args to Child
0007fb1c 7c93edc0 7ffdf000 7ffd4000 00000000 ntdll!DbgBreakPoint
0007fc94 7c921639 0007fd30 7c900000 0007fce0 ntdll!LdrpInitializeProcess+0xffa
0007fd1c 7c90eac7 0007fd30 7c900000 00000000 ntdll!_LdrpInitialize+0x183
00000000 00000000 00000000 00000000 00000000 ntdll!KiUserApcDispatcher+0x7
0:000> dd 0007fd30
0007fd30 00010017 00000000 00000000 00000000
0007fd40 00000000 00000000 00000000 ffffffff
0007fd50 ffffffff f735533e f7368528 ffffffff
0007fd60 f73754c8 804eddf9 8674f020 85252550
0007fd70 86770f38 f73f4459 b2f3fad0 804eddf9
0007fd80 b30dccd1 852526bc b30e81c1 855be944
0007fd90 85252560 85668400 85116538 852526bc
0007fda0 852526bc 00000000 00000000 00000000
```

В этом примере ожидается, что вы введете ~*kb в сеансе отладки.

Результат ввода этой команды показывает несколько строк, где наиболее критическая часть информации - 0007fd30.

Далее следует ввести показанную команду dd 0007fd30, чтобы получить больше информации о ранее выделенном числе 0007fd30.

Предполагается, что все инструменты, используемые в этой книге, запускаются из папки установки. Например, если отладчики Windows установлены в папку C:\Program Files\Debugging Tools for Windows, командная строка для запуска windbg.exe будет показана как:

```
C:\>windbg
```

Поддерживаемые версии Windows

Для полноценного использования этой книги требуется Windows XP или более поздняя версия. Весь примерный код и сценарии отладки выполнялись на Windows XP SP2 или Windows Server 2003 SP1, в зависимости от требований конкретного сценария.

Обратите внимание, что пакеты обновлений или даже отдельные исправления могут изменить результат различных команд, хотя эти изменения не повлияют на общий итог того, что иллюстрируется сеансом отладки.

Глава 15, "Windows Vista Fundamentals", охватывает самые важные изменения, внесенные в Windows Vista, и включает сеансы отладки, которые должны выполняться на машине под управлением Windows Vista.

Кроме того, все примеры и сеансы отладки выполнялись с использованием 32-битной версии Windows.

Примеры, используемые в главе 12, "64-Bit Debugging", выполнялись с использованием 64-битной версии Windows XP.

Поддержка

Хотя были предприняты все попытки сделать эту книгу на 100% точной, ошибки, без сомнения, будут найдены.

Если вы обнаружите ошибку в этой книге, можете связаться с нами, используя любой из следующих ресурсов:

Email: marioh@advancedwindowsdebugging.com or
daniel@advancedwindowsdebugging.com.

Кроме того, форум обсуждения книги по адресу <http://www.advancedwindowsdebugging.com> отслеживается и может использоваться для сообщения об ошибочной информации.

По мере внесения исправлений они будут публиковаться в разделе errata на Web-сайте.

Благодарности

Написание технической книги - это крупномасштабная работа, гораздо более существенная, чем мы изначально ожидали.

Как авторы, мы предоставили сырой материал и первый черновик книги, но на протяжении проекта множество людей делилось своими мыслями и экспертизой, чтобы сделать эту книгу достойной времени, потраченного на ее чтение.

Спасибо всем членам команды Addison Wesley, особенно Elizabeth Peterson, Jana Jones, Curt Johnson, Joan Murray и Gina Kanouse.

Chris Zahn также сыграл важную роль в редактировании книги и исправлении нашего самобытного синтаксиса.

Как и в любой технической публикации, техническая точность имеет первостепенное значение. Нам повезло, что замечательные инженеры (многие из них владеют конкретными технологическими областями, обсуждаемыми в книге) посмотрели материал и дали обратную связь.

Благодарим Mark Russinovich, Ivan Brugiolo, Pat Styles, Pavel Lebedynskiy, Daniel Mihai, Doug Ellis, Cristi Vlasceanu, Adrian Marinescu, Saji Abraham, Kamen Moutafov, Kinshuman Kinshumann, Bob Wilton, Raymond McCollum, Viorel Mititean, Andy Cheung, Saar Picker, Drew Bliss, Jason Cunningham, Adam Edwards, Jen-Lung Chiu, Alain Lissor и Brandon Jiang.

Особая благодарность Mark Russinovich не только за рецензирование книги, но и за написание предисловия.

Выдающийся объем работ Марка хорошо известен среди разработчиков программного обеспечения и оказал большое влияние на нас и бесчисленное множество других инженеров.

Ivan Brugiolo также сыграл важную роль в рецензировании и предоставлении глубокой обратной связи.

Ivan был невероятно щедр на свое свободное время, делясь знаниями, которые существенно повысили ценность этой книги.

Мы также хотим выразить благодарность Alexandra Hewardt за проектирование и реализацию Web-страницы книги.

Об авторах



© www.BrookeClark.com

Mario Hewardt - senior design engineer в Microsoft Corporation, который последние девять лет много работал в области разработки Windows на системном уровне.

На протяжении пяти выпусков Windows (начиная с Windows 98) он работал преимущественно в области управления серверами и рабочими столами, сосредотачивая большую часть своего времени на обеспечении надежности, устойчивости и безопасности продукта.



Photo by Eduard Koller

Daniel Pravat - senior design engineer в Microsoft Corporation, активно участвовавший в выпуске нескольких компонентов Windows в нескольких релизах Windows.

До прихода в Microsoft он разрабатывал телекоммуникационное программное обеспечение для серверов компьютерной телефонии.

Он ожидает, что все программные приложения будут надежными, предсказуемыми и эффективными.

Часть I. Обзор

Глава 1. Введение в инструменты

О важности правильного проектирования программного обеспечения и инженерных принципов написано множество книг и статей. Одни публикации выдерживают сбалансированный подход между методологией и практикой, другие сосредоточены главным образом на методологии. Книги о важности объектно-ориентированного проектирования и программирования, шаблонов проектирования или модульного программирования - отличные примеры методологий, которые помогают нам писать более качественное программное обеспечение. Несомненно, правильные программные методологии являются предпосылкой всех успешных программных проектов. Однако они не единственные факторы успеха программного обеспечения. Как бы хорошо, по нашему мнению, мы ни умели проектировать программы и каким бы точным ни считали свое планирование, в процессе разработки нас всегда преследуют загадочные проблемы. Напряженные графики, сложные взаимодействия компонентов и унаследованный код - лишь некоторые причины, по которым практически невозможно предвидеть и решить все проблемы одним только применением хороших методологий разработки. Помимо методологий, мы должны знать, как устранять сложные неполадки экономно по времени и затратам.

В этой главе представлены бесценные инструменты, которые окажут большую помощь в процессе поиска и устранения неполадок, а также помогут сократить время и деньги, затрачиваемые на работу с широким кругом распространенных проблем. Многие проблемы, обсуждаемые в этой книге, вызывают у разработчиков разочарование из-за своей сложности. Даже если разработчик представляет, как вручную подойти к конкретной проблеме, затраты на ее отслеживание обычно очень велики. Многие разработчики не знают, что помощь существует: она приходит в виде замечательных наборов инструментов, помогающих находить и решать множество проблем такого рода. Эти инструменты не только помогают решать проблемы, но и делают это очень эффективно.

В этой главе дается введение в инструменты, используемые на протяжении всей книги. Каждый инструмент рассматривается подробно; описание включает важные сведения, такие как типовые сценарии применения, места загрузки и вводную информацию о том, как эти инструменты выполняют свою работу. Описания инструментов не являются исчерпывающими источниками для всех возможных сценариев применения; скорее, они служат высокоуровневыми обзорами. Каждый из перечисленных инструментов используется в других частях книги, чтобы показать его применение при решении реальной проблемы. Эту главу можно рассматривать как введение в набор инструментов, которое дополняется практическими сценариями их применения в последующих главах книги.

Обратите внимание, что инструменты, описанные в этой главе, являются последними версиями каждого инструмента, доступными на момент написания. К тому времени, когда вы будете читать эту главу, могли выйти более новые версии. Это не создает проблемы, поскольку общее поведение инструментов обычно остается прежним.

Leak Diagnosis Tool

Параметр	Значение
Сценарии применения	Обнаружение утечек памяти
Текущая версия	1.25
Место загрузки	ftp://ftp.microsoft.com/PSS/Tools/Developer Support Tools/LeakDiag
Механизм анализа	Файлы журналов

Leak Diagnosis Tool (LeakDiag) - это инструмент, используемый в процессе обнаружения утечек памяти. Он значительно выходит за рамки базовых возможностей, которые показывают только, сколько памяти утекло из процесса: LeakDiag предоставляет подробные сведения, такие как точная трассировка стека, приведшая к выделению памяти, и статистика выделений.

Процесс установки LeakDiag тривиален. Загрузите leakdiag125.msi из указанного места загрузки и используйте параметры установки по умолчанию. По умолчанию приложение устанавливается в C:\LEAKDIAG и может работать в двух режимах. В частности, у него есть версия для командной строки и версия с графическим пользовательским интерфейсом (GUI). Версия командной строки называется ldcmd.exe, а GUI-версия - leakdiag.exe. Обе можно запустить из командной строки или через кнопку Start, выбрав All Programs, LeakDiag.

LeakDiag включает надмножество возможностей UMDH.exe (см. далее раздел "UMDH") в том смысле, что UMDH способен показывать только выделения, поступающие из стандартного диспетчера кучи. LeakDiag расширяет эту функциональность, включая не только стандартные выделения из кучи, но и выделения COM (внешние и внутренние), выделения виртуальной памяти и многое другое. В целом текущая версия LeakDiag поддерживает шесть различных распределителей:

- Virtual Allocator
- Heap Allocator [DEFAULT]
- MPHeap Allocator
- COM AllocatorCoTaskMem
- COM Private Allocator
- C Runtime Allocator

Способность LeakDiag поддерживать все эти распределители делает его очень гибким инструментом для обнаружения утечек памяти. Еще одно существенное отличие от большинства других инструментов обнаружения утечек памяти заключается в том, как LeakDiag собирает сведения об операциях с памятью. Вместо того чтобы полагаться на поддержку операционной системы для записи трассировок стека выделений памяти, LeakDiag использует технологию Microsoft Detours для перехвата вызовов распределителей памяти. Благодаря этому LeakDiag устраняет необходимость включать поддержку трассировки стека в операционной системе.

На рисунке 1.1 показан стартовый экран GUI-версии LeakDiag. Интерфейс LeakDiag состоит из двух основных разделов: списка всех запущенных процессов и доступных распределителей памяти с соответствующими кнопками действий. Чтобы начать отслеживание выделений памяти, просто выберите один из запущенных процессов, а затем распределитель памяти, который хотите отслеживать. Нажмите кнопку Start, затем кнопку Log. Воспроизведите утечку памяти и снова нажмите кнопку Log. Когда закончите отслеживание, нажмите кнопку Stop. LeakDiag выводит всю информацию в журнальные файлы в формате XML. По умолчанию журнальные файлы записываются в C:\LeakDiag\logs, а имена журналов LeakDiag назначает сам, чтобы гарантировать уникальное имя файла для каждого запуска.

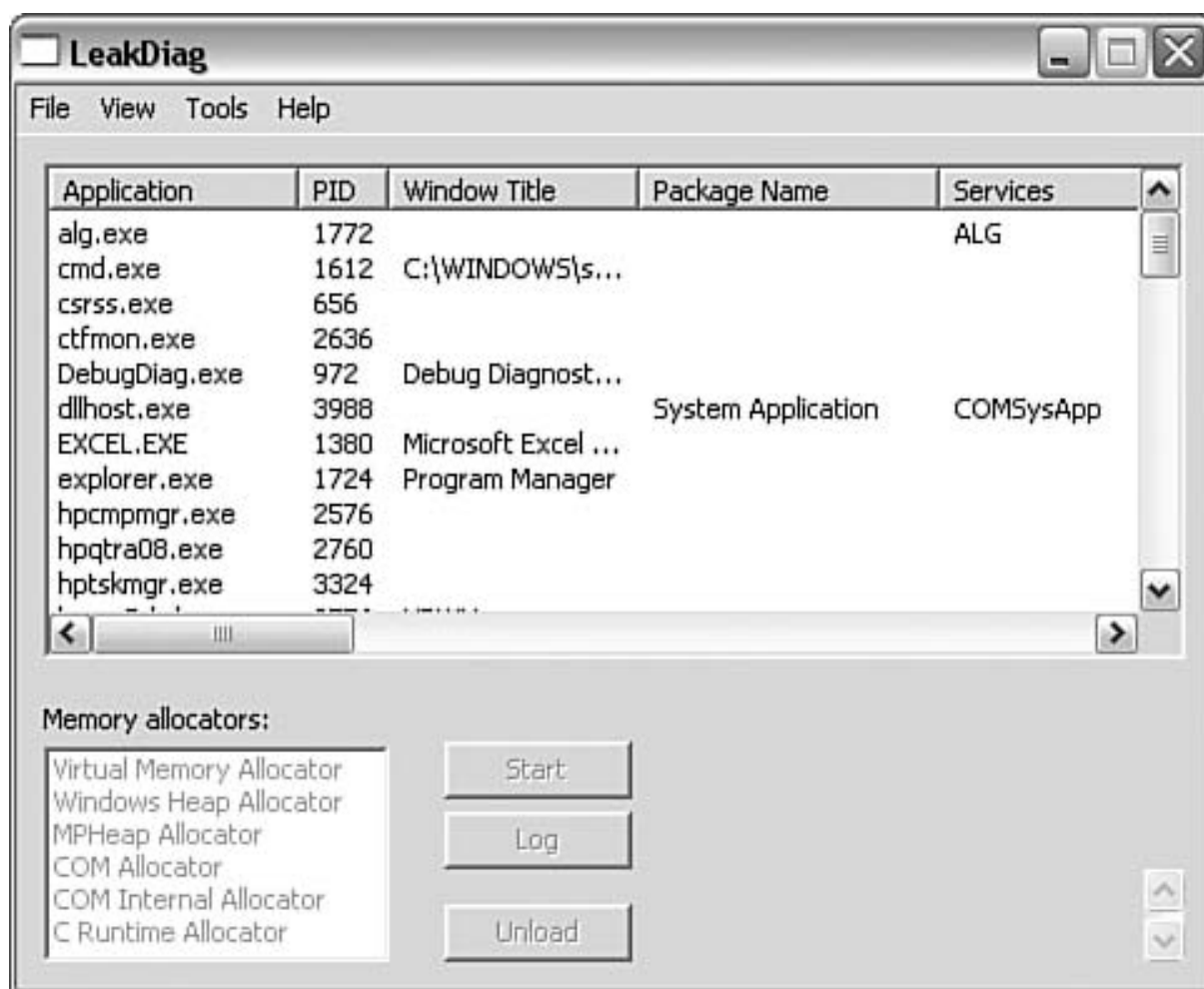


Рисунок 1.1. Стартовый экран LeakDiag

Как и большинство инструментов обнаружения утечек памяти, LeakDiag работает на основе сравнения снимков. Делая снимки всех выделений памяти через регулярные интервалы, LeakDiag способен вычислять разницу между снимками, чтобы описать выделения, которые еще не были освобождены (потенциальные утечки). Кнопка Log - это механизм, с помощью которого вы делаете такие снимки.

У LeakDiag есть несколько параметров, позволяющих настроить поведение по умолчанию. Выбрав пункт Options в меню Tools, вы увидите диалоговое окно Options, показанное на рисунке 1.2.

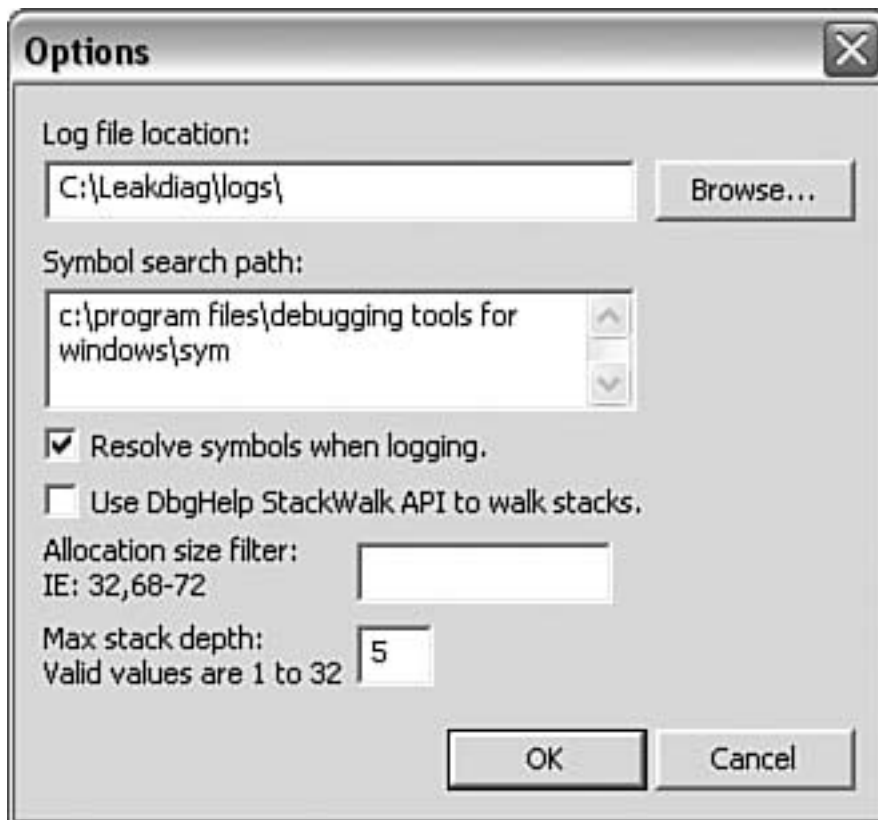


Рисунок 1.2. Диалоговое окно Options LeakDiag

В диалоговом окне Options можно изменить расположение журнальных файлов, а также указать путь к символам. Как и в большинстве инструментов трассировки стека, для того чтобы LeakDiag мог создавать полезные трассировки стека, требуются правильные символы. Если путь к символам указан неверно или символы неправильные, для каждого кадра трассировки стека будут видны только адреса. При этом запись трассировок стека - дорогостоящая операция, способная заметно изменить скорость выполнения. Более того, иногда скорость выполнения может измениться настолько, что утечка памяти вообще не проявится (если она связана с параллелизмом и/или проблемами синхронизации по времени). К счастью, существует также флажок, позволяющий отключить разрешение символов во время журналирования. Фильтр размера выделения позволяет указать диапазон размеров выделения, которые нужно отслеживать. Наконец, глубина стека позволяет задать количество кадров на одну трассировку стека, выводимых в журнальный файл.

Подробное описание режима командной строки LeakDiag, а также формата журнального файла см. в главе 9, "Утечки ресурсов", где мы используем LeakDiag для анализа и точного выявления реальной утечки памяти.

Библиотека Microsoft Detours. Microsoft Detours - новаторское решение проблемы инструментирования и/или улучшения существующего кода на двоичном уровне. Исторически инструментирование и/или улучшение кода означало простое изменение исходного кода и повторную компиляцию. Однако в современном мире коммерческой разработки вы редко (если вообще когда-либо) имеете доступ к исходному коду компонента или продукта. Microsoft Detours позволяет перехватывать двоичные функции и предоставлять собственную detour-функцию, которая может либо полностью заменить исходную функцию, либо добавить некоторый код, а затем вызвать исходную функцию (через trampoline). Этот кажущийся волшебным прием выполняется путем замены первых нескольких инструкций исходной функции безусловным переходом к новой функции. Важно понимать, что этот процесс происходит во время выполнения и не сохраняется постоянно; по сути это означает, что можно перенаправлять разные экземпляры одного и того же

приложения независимо друг от друга.

Дополнительные сведения о Microsoft Detours см. по адресу
<http://research.microsoft.com/sn/detours>.

Debugging Tools for Windows

Параметр	Значение
Сценарии применения	Набор отладчиков и инструментов
Текущая версия	6.6.0007.5
Место загрузки	http://www.microsoft.com/whdc/ddk/debugging/

Debugging Tools for Windows - это комплексный, свободно доступный пакет, содержащий мощные отладчики и инструменты, которые помогают разработчикам эффективнее выполнять повседневную работу.

В месте загрузки можно выбрать между 32-разрядной и 64-разрядной версиями (Itanium и x64). Установка проста, и express-установки достаточно, чтобы получить все необходимые инструменты. Есть одно предостережение: если вы планируете разрабатывать собственные расширения отладчика (как мы покажем в главе 11, "Написание собственных расширений отладчика"), необходимо выполнить custom-установку и выбрать установку SDK. В таблице 1.1 показаны все инструменты, входящие в этот пакет.

Таблица 1.1

Образ	Описание
agestore.exe	Удобная утилита удаления файлов, удаляющая файлы на основе даты последнего доступа.
cdb.exe	Консольный отладчик пользовательского режима. Практически идентичен NTSD.
dbengprx.exe	Легковесный прокси-сервер, передающий данные между двумя разными машинами.
dbgrpc.exe	Инструмент, используемый для запроса и отображения информации Microsoft Remote Procedure Call (RPC).
dbgrpc.exe	Сервер процессов, используемый для удаленной отладки.
dumpchk.exe	Инструмент, используемый для проверки файла дампа памяти.
gflags.exe	Инструмент настройки, используемый для включения и отключения системного инструментирования.
kd.exe	Отладчик режима ядра.
kdbgctrl.exe	Инструмент, используемый для управления и настройки соединения отладки режима ядра.
kdsrv.exe	Сервер соединений, используемый при отладке режима ядра.
kill.exe	Консольный инструмент для завершения процессов.
logger.exe	Инструмент, который журналирует активность процесса (например, вызовы функций).
logviewer.exe	Инструмент, используемый для просмотра журнальных файлов, созданных logger.exe.

Образ	Описание
ntsd.exe	Консольный отладчик пользовательского режима. Практически идентичен CDB.
remote.exe	Инструмент, используемый для удаленного управления консольными программами.
rtlist.exe	Средство просмотра списка удаленных процессов.
symchk.exe	Инструмент, используемый для проверки файлов символов или загрузки файлов символов с сервера символов.
symstore.exe	Инструмент, используемый для создания и сопровождения хранилища символов.
tlist.exe	Инструмент для перечисления всех запущенных процессов.
umdh.exe	Инструмент, используемый для обнаружения утечек памяти.
windbg.exe	Отладчик пользовательского режима и режима ядра с графическим пользовательским интерфейсом.

Неудивительно, что самый важный инструмент - сам отладчик. Глава 2, "Введение в отладчики", и глава 3, "Отладчики изнутри", посвящены объяснению того, как работают отладчики, как их настроить и как наиболее эффективно использовать.

Введение в инструменты в этой главе подробно описывает наиболее интересные инструменты, которые мы используем на протяжении книги. Когда в месте загрузки для каждого инструмента указано "Part of Debugging Tools for Windows", это означает, что должен быть установлен пакет Debugging Tools for Windows.

Обратите внимание, что на момент написания самой свежей версией была 6.6.0007.5. Вполне возможно, что к тому времени, когда вы будете читать эту книгу, выйдет новая версия отладчиков Windows. Даже в этом случае изменения в выводе отладчика должны быть относительно небольшими, а весь материал книги должен по-прежнему применяться и быть простым для повторения. URL загрузки отладчиков также хранит историю версий отладчика (на две-три версии назад), которые можно скачать. Если вы хотите следовать той же версии, можно загрузить Debugging Tools for Windows, соответствующий версии 6.6.0007.5.

UMDN

Параметр	Значение
Сценарии применения	Обнаружение утечек памяти
Текущая версия	6.0.5457.0
Место загрузки	Входит в Debugging Tools for Windows
Механизм анализа	Файлы журналов

UMDN - еще один инструмент обнаружения утечек памяти, включающий подмножество функциональности LeakDiag. В то время как LeakDiag способен отслеживать память из разных распределителей, UMDN способен отслеживать только память, происходящую из диспетчера кучи. Кроме того, для правильной работы он требует, чтобы в операционной системе была включена трассировка стеков пользовательского режима (см. раздел "Global Flags" в этой главе). В главе 9 показаны примеры использования UMDN для поиска утечек памяти.

Microsoft Application Verifier

Параметр	Значение
Сценарии применения	Общий поиск неполадок приложений
Текущая версия	3.3
Место загрузки	http://www.microsoft.com/downloads/details.aspx?FamilyID=bd02c19c-1250-433c-8c1b-2619bd93b3a2&DisplayLang=en
Механизм анализа	Файлы журналов и отладчик

Каждый серьезный разработчик должен знать об инструменте Application Verifier. Включив Application Verifier для своего процесса, можно обнаружить целый спектр распространенных ошибок программирования. Примеры включают неправильное использование дескрипторов, использование блокировок, пути к файлам и многое другое. Хорошая практика - всегда держать Application Verifier включенным для всех процессов, задействованных во время разработки. При этом некоторые тестовые параметры Application Verifier могут резко изменить скорость выполнения приложения и тем самым привести к тому, что проблемы, связанные с синхронизацией по времени, не проявятся. Одно распространенное решение этой проблемы - всегда держать Application Verifier включенным, а на выбранных контрольных этапах отключать его и снова запускать весь набор тестов, чтобы убедиться, что проблемы синхронизации не являются фактором. Еще один хороший момент для включения Application Verifier - режим исправления ошибок в продукте. Запуск с включенным Application Verifier помогает убедиться, что при исправлении ошибок не внесены регрессии.

Установка Application Verifier проста при использовании параметров установки по умолчанию. После завершения установки можно запустить Application Verifier через кнопку Start, выбрав All Programs, Application Verifier. На рисунке 1.3 показан стартовый экран, который появляется при запуске Application Verifier.

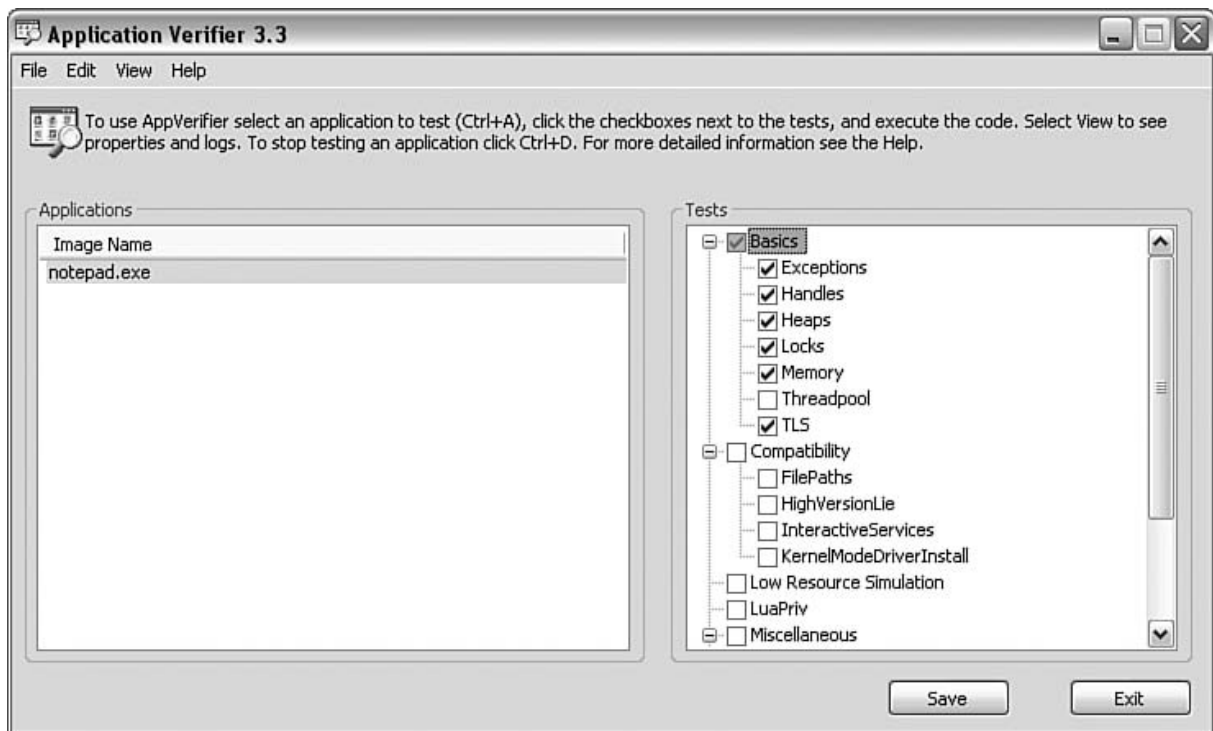


Рисунок 1.3. Стартовый экран Application Verifier

Панель Applications показывает все приложения, для которых сейчас включена проверка. Можно добавить приложения, выбрав пункт Add Application в меню File. Соответственно, приложения можно удалить, выбрав приложение и затем пункт Delete Application в меню File.

Чтобы изменить параметры конкретного приложения, выберите приложение на левой панели и пункт Property Window в меню View. Это добавит раздел свойств в нижнюю часть стартового окна и позволит управлять следующим поведением:

- **Propagate**: управляет тем, будут ли тестовые параметры этого образа распространяться на дочерние процессы. Включение этого свойства приводит к распространению тестовых параметров.
- **AutoClr**: если включено, заставляет Application Verifier отключить все тестовые параметры этого образа после начала его выполнения.
- **AutoDisableStop**: если включено, заставляет Application Verifier сообщать о заданной проблеме только один раз.
- **LoggingWithLocksHeld**: если включено, заставляет Application Verifier журналировать события загрузки и выгрузки DLL. Обратите внимание, что это может вызвать проблемы в приложении, поскольку журналирование требует ввода-вывода, выполняемого во время исполнения пути кода DllMain.

Чтобы получить краткое описание каждого тестового параметра, можно навести указатель мыши на этот параметр и открыть всплывающую подсказку. Всплывающая подсказка также сообщит, требуется ли отладчик для просмотра результатов тестов.

Чтобы получить более подробные сведения или настройки конфигурации для каждого тестового параметра, можно щелкнуть тестовый параметр правой кнопкой мыши и выбрать один из двух вариантов.

- **Properties**: позволяет управлять свойствами выбранного теста. Например, выбор свойств для теста Handles позволяет управлять количеством трассировок, которые будут записываться для отслеживания дескрипторов. Обратите внимание, что выбор Properties доступен не для всех тестовых параметров.

- Verifier Stop Options: позволяет управлять параметрами для выбранного теста. На рисунке 1.4 показан выбор меню verifier stop options при использовании с тестовым параметром Handles.

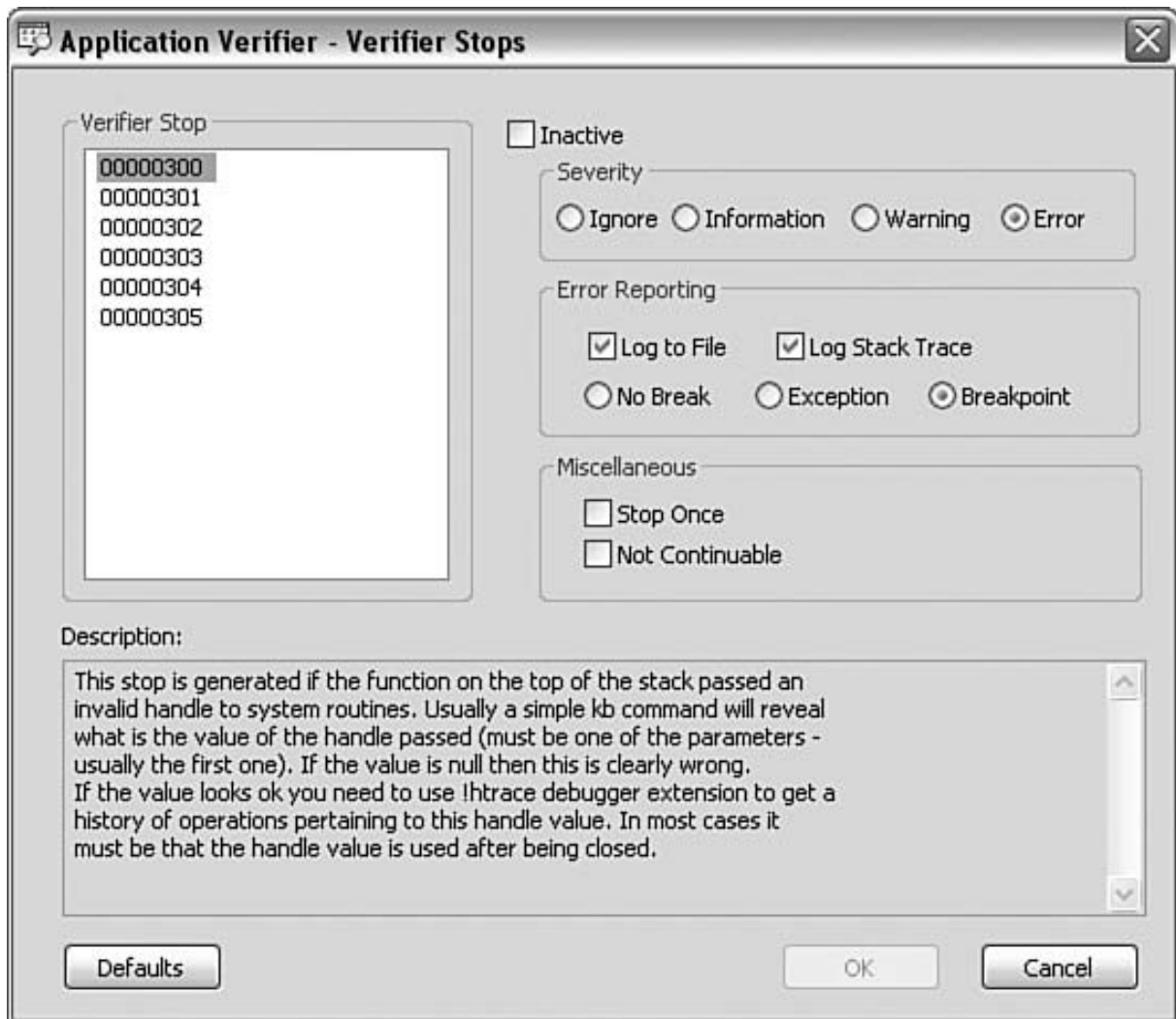


Рисунок 1.4. Параметры остановов Application Verifier

Параметры Application Verifier Stop далее делятся на несколько разделов:

- Раздел Verifier Stop содержит список всех verifier stop, которые способен выполнить тестовый параметр. На рисунке 1.4 раздел Verifier Stop показывает, что при проверке дескрипторов доступны шесть остановов. Все остальные разделы этого окна работают на основе выбранного кода останова.
- Раздел Description дает подробное описание выбранного verifier stop.
- Флажок Inactive управляет тем, активен или неактивен выбранный verifier stop, позволяя управлять детальностью тестового параметра.
- Раздел Severity позволяет управлять тем, насколько серьезным вы считаете код останова. Сделанный выбор напрямую влияет на то, как будет проявляться останов. Например, установка verifier stop 00000300 в Ignore приводит к тому, что при срабатывании этот останов не прерывает выполнение в отладчик.
- Раздел Error Reporting позволяет подробнее управлять тем, что должно произойти при возникновении verifier stop. Флажки управляют выполняемыми действиями журналирования (например, нужно ли записывать сведения в файл), а также тем, нужно ли записывать трассировку стека для останова. Переключатели управляют поведением отладчика при возникновении останова. Можно настроить выполнение точки останова, генерацию исключения или отсутствие прерывания вообще.

- Раздел Miscellaneous управляет частотой останова. Если выбран флажок Stop Once, останов произойдет только при первом обнаружении. Если выбран флажок Non Continuable, отладчик прервется при возникновении останова, и восстановиться после него будет невозможно, что фактически помешает продолжить выполнение процесса.

Следующий раздел стартового экрана (см. рисунок 1.3), панель Tests, показывает все доступные тестовые параметры. Установка флажка включает соответствующий тестовый параметр для выбранного процесса. Непосредственно под панелью Tests находится краткое описание самого тестового параметра.

После того как для приложения включена проверка, можно просто запустить приложение, и Application Verifier будет работать в фоновом режиме. В зависимости от того, как настроен каждый тестовый параметр, существует два основных способа увидеть результаты запуска Application Verifier. Первый способ - просмотреть соответствующий журнальный файл, выбрав пункт Logs в меню View, а затем выбрав интересующий журнал приложения. Важно отметить, что не все тестовые параметры сообщают результаты с помощью журнальных файлов. Некоторые тестовые параметры требуют отладчика, чтобы получить нужные результаты. Чтобы увидеть, какие тестовые параметры требуют отладчика, просто наведите указатель мыши на тестовый параметр и получите контекстную справку. Если тестовый параметр требует отладчика, необходимо запустить приложение под отладчиком, чтобы увидеть результаты.

Когда Application Verifier требует подключенного отладчика, вывод нарушения в общем случае имеет следующий вид:

```
VERIFIER STOP <stop-code>: <process-PID>: <message>
parameter-1: <description>
parameter-2: <description>
parameter-3: <description>
parameter-4: <description>
```

stop-code указывает конкретное произошедшее нарушение, PID показывает идентификатор процесса, в котором произошел сбой, а сообщение дает краткое текстовое описание сбоя. Список параметров зависит от типа выполняемого теста.

Например, следующий вывод показывает нарушение, о котором сообщает Application Verifier при попытке закрыть недопустимый дескриптор:

```
=====
VERIFIER STOP 00000300 : pid 0xFF0: Invalid handle exception for current stack trace.
C0000008 : Exception code.
0007FBD4 : Exception record. Use .exr to display it.
0007FBE8 : Context record. Use .cxr to display it.
00000000 : Not used.
=====
This verifier stop is continuable.
After debugging it use `go` to continue.
=====
```

Использовать GUI-режим для включения тестов для приложения довольно удобно, но иногда необходимо включать тесты автоматизированно. Предположим, продукт, над которым вы работаете, собирается каждую ночь, а автоматические тесты запускаются сразу после завершения сборки. В рамках этого набора тестов группа контроля качества попросила включать Application Verifier во время тестирования. Вместо того чтобы каждый вечер инженер вручную использовал GUI-версию Application Verifier и включал тесты, он может просто написать сценарий, использующий консольную версию для включения тестов. Путь установки application verifier по умолчанию:

```
C:\windows\system32\appverif.exe
```

Если запустить исполняемый файл appverif.exe с ключом /?, вы увидите следующее:

```

Application Verifier 3.3.0045
Copyright (c) Microsoft Corporation. All rights reserved.
Application Verifier Command-Line Usage:
-enable TEST ... -for TARGET ... [-with [TEST.]PROPERTY=VALUE ...]
-disable TEST ... -for TARGET ...
-query TEST ... -for TARGET ...
-configure STOP ... -for TARGET ... -with PROPERTY=VALUE...
-verify TARGET [-faults [PROBABILITY [TIMEOUT [DLL ...]]]]
-export log -for TARGET -with To=XML_FILE [Symbols=SYMBOL_PATH]
[StampFrom=LOG_STAMP] [StampTo=LOG_STAMP] [Log=RELATIVE_TO_LAST_INDEX]
-delete [logs|settings] -for TARGET ...
-stamp log -for TARGET -with Stamp=LOG_STAMP [Log=RELATIVE_TO_LAST_INDEX]
-logtoxml LOGFILE XMLFILE
-installprovider PROVIDERBINARY
Available Tests:
Heaps
Handles
Locks
Memory
TLS
Exceptions
DirtyStacks
LowRes
DangerousAPIs
TimeRollOver
Threadpool
LuaPriv
HighVersionLie
FilePaths
KernelModeDriverInstall
InteractiveServices
PrintAPI
PrintDriver
(For descriptions of tests, run appverif.exe in GUI mode.)
Examples:
appverif -enable handles locks -for foo.exe bar.exe
(turn on handles locks for foo.exe & bar.exe)
appverif -enable heaps handles -for foo.exe -with heaps.full=false
(turn on handles and normal pageheap for foo.exe)
appverif -enable heaps -for foo.exe -with full=true dlls=mydll.dll
(turn on full pageheap for the module of mydll.dll in the foo.exe)
appverif -enable * -for foo.exe
(turn on all tests for foo.exe)
appverif -disable * -for foo.exe bar.exe
(turn off all tests for foo.exe & bar.exe)
appverif -disable * -for *
(wipe out all the settings in the system)
appverif -export log -for foo.exe -with to=c:\sample.xml
(export the most recent log associated with foo.exe to c:\sample.xml)
appverif /verify notepad.exe /faults 5 1000 kernel32.dll advapi32.dll
(enable fault injection for notepad.exe. Faults should happen with
probability 5%, only 1000 msec after process got launched and only
for operations initiated from kernel32.dll and advapi32.dll)

```

Чтобы включить все тесты Application Verifier для заданного исполняемого файла, можно использовать следующую командную строку:

```
appverif.exe -enable * -for myexecutable.exe
```

Помимо включения тестов для заданного приложения, Application Verifier также можно управлять из отладчика. Команда расширения, используемая для управления Application Verifier из отладчика, - !avr.f. Полный список всех доступных тестовых параметров см. в приложении А, "Тестовые параметры Application Verifier".

Global Flags

Параметр	Значение
Сценарии применения	Конфигурация
Текущая версия	6.6.0007.5
Место загрузки	Входит в Debugging Tools for Windows
Исполняемый файл	gflags.exe

Приложение Global Flags (gflags) устанавливается как часть Debugging Tools for Windows, а исполняемый файл (gflags.exe) можно запустить из пути установки по умолчанию. Например, на моей системе для запуска gflags я использовал бы следующую командную строку:

```
c:\>gflags.exe
```

Многие инструменты, используемые в этой книге, для правильной работы полагаются на поддержку Windows. Например, UMDH требует включения параметра Create user mode stack trace database. Global Flags (или gflags) - единый инструмент настройки всех доступных различных параметров.

GUI-режим

Большинство доступных параметров можно включать для всей системы (то есть для всех запущенных процессов) или отдельно для каждого процесса. На рисунке 1.5 показан главный экран gflags.

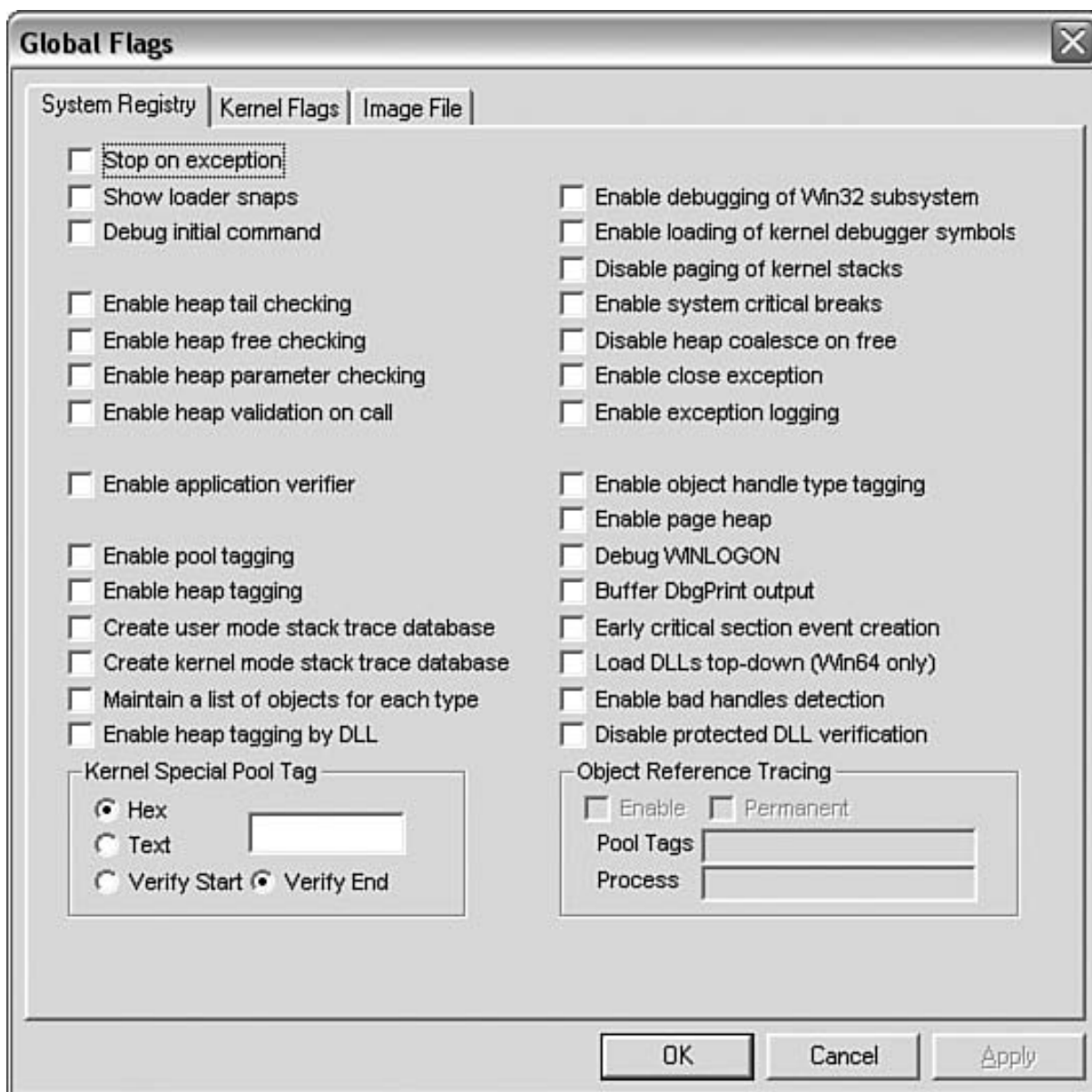


Рисунок 1.5. Вкладки System Registry и Kernel Flags в Global Flags

Вкладка System Registry показывает параметры, доступные в масштабе всей системы, а вкладка Image File - параметры, доступные для отдельного процесса. Если изменить какие-либо общесистемные параметры, обычно требуется перезагрузка. Вкладка Kernel Flags показывает параметры, влияющие только на работающее ядро. Для параметров отдельного процесса процесс должен быть перезапущен, прежде чем параметры вступят в силу.

Поскольку доступные в gflags параметры настраивают различные аспекты операционной системы, где хранятся эти настройки и как они интерпретируются? Ответ: в реестре. В зависимости от того, меняете ли вы общесистемные параметры или параметры отдельного процесса, они хранятся в разных местах реестра:

- Общесистемные параметры:
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\SessionManager\GlobalFlag
- Параметры отдельного процесса: HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\Current Version\Image File Execution Options\<Image File Name>\GlobalFlag

Путь реестра для отдельного процесса имеет несколько интересных свойств. Помимо хранения глобальных флагов в значении GlobalFlag, там могут храниться и другие полезные настройки. Например, если вы пытаетесь отлаживать процесс, который запускаете не напрямую (например,

службу, запускаемую диспетчером управления службами), можно включить отладку этого процесса, задав следующее значение реестра:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\Current Version\Image File Execution  
Options\<Image File Name>\Debugger
```

Можно указать выбранный отладчик, который должен запускаться при старте процесса. В главе 2 мы увидим подробнее, как можно использовать эту возможность.

Вкладка Image File позволяет включать инструментирование для отдельного процесса. На рисунке 1.6 показаны доступные параметры.

Когда вы впервые переходите на эту вкладку, все параметры будут недоступны, пока вы не укажете имя образа в текстовом поле Image и не нажмете клавишу Tab.

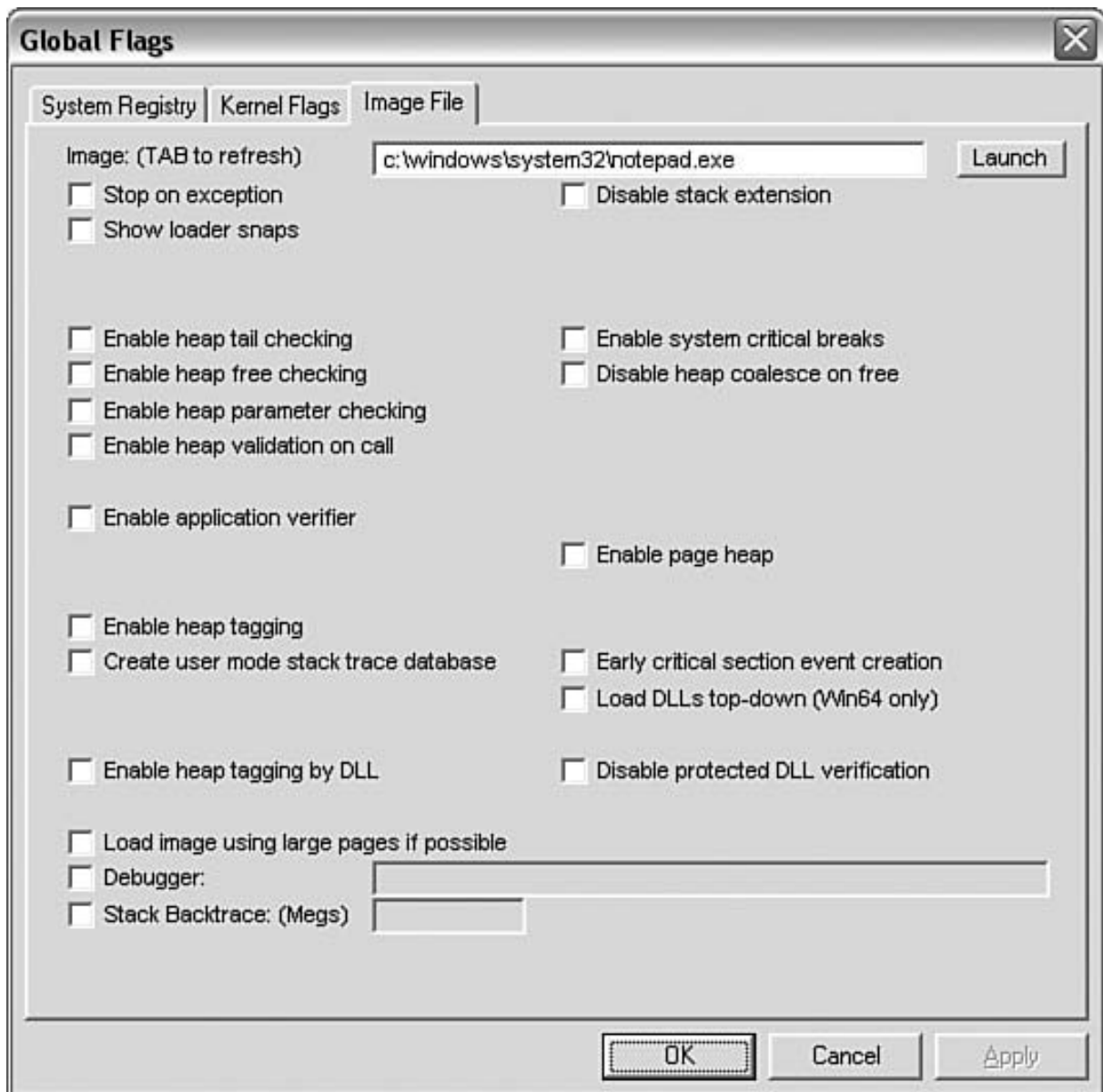


Рисунок 1.6. Вкладка Image File в Global Flags

Режим командной строки

Помимо GUI-режима, gflags можно запускать из командной строки. Параметры, доступные в командной строке, повторяют параметры GUI-режима:

```
usage: GFLAGS [-r [<Flags>]] |  
[-r +spp TAG | -r +spp SIZE | -r -spp |  
[-k [<Flags>]] |  
[-i <ImageFileName> [<Flags>]] |  
[-i <ImageFileName> -tracedb <SizeInMb>] |  
[-p <PageHeapOptions>] (use '-p ?' for help) |
```

Каждый параметр немного подробнее объясняется в следующем списке:

- -r управляет постоянными параметрами для всей системы (аналог вкладки System Registry в GUI-режиме).
- -k управляет текущими параметрами ядра (аналог вкладки Kernel Flags в GUI-режиме).
- -i управляет параметрами для отдельного образа (аналог вкладки Image File в GUI-режиме).
- -p управляет параметрами pageheap (аналог вкладки Verifier в GUI-режиме).

Каждый из приведенных выше ключей может либо отображать текущие настройки для соответствующего ключа, либо изменять настройки в соответствии с указанными флагами. Если нужно просто посмотреть настройки, укажите ключ (например, -i notepad.exe) без флагов. Если нужно включить настройки, флаги можно указать либо как шестнадцатеричное число, либо как сокращение, представляющее параметр gflags. В таблице 1.2 показаны доступные сокращения.

Таблица 1.2

Сокращение	Описание
soe	Stop On Exception
sls	Show Loader Snaps
dic	Debug Initial Command
shg	Stop on Hung GUI
htc	Enable heap tail checking
hfc	Enable heap free checking
hpc	Enable heap parameter checking
hvc	Enable heap validation on call
vrif	Enable application verifier
ptg	Enable pool tagging
htg	Enable heap tagging
ust	Create user mode stack trace database
kst	Create kernel mode stack trace database
otl	Maintain a list of objects for each type
htd	Enable heap tagging by DLL
dse	Disable stack extensions
d32	Enable debugging of Win32 Subsystem
ksl	Enable loading of kernel debugger symbols
dps	Disable paging of kernel stacks
scb	Enable system-critical breaks
dhc	Disable Heap Coalesce on Free

Сокращение	Описание
ece	Enable close exception
eel	Enable exception logging
eot	Enable object handle type tagging
hpa	Enable page heap
dwl	Debug WINLOGON
ddp	Disable kernel mode DbgPrint output
cse	Early critical section event creation
ltd	Load DLLs top-down
bhd	Enable bad handles detection
dpd	Disable protected DLL verification
lpg	Load image using large pages if possible

Чтобы установить конкретный параметр, используйте +<abbreviation>; чтобы снять конкретный параметр, используйте -<abbreviation>. Например, если нужно включить базу данных трассировок стека пользовательского режима для notepad.exe, используйте следующую командную строку:

```
C:\> gflags /i notepad.exe +ust
Current Registry Settings for notepad.exe executable are: 00001000
ust - Create user mode stack trace database
```

Соответственно, если нужно отключить тот же параметр, используйте:

```
C:\> gflags /i notepad.exe -ust
Current Registry Settings for notepad.exe executable are: 00000000
```

Если нужно просто узнать настройки для конкретного образа, используйте:

```
C:\> gflags /i notepad.exe
Current Registry Settings for notepad.exe executable are: 00000000
```

Чтобы посмотреть, какие параметры доступны для pageheap и Application Verifier, можно использовать:

```
C:\> gflags.exe /p /?
```

и

```
C:\> gflags.exe /v /?
```

Последний важный ключ - -tracedb, который позволяет указать размер базы данных трассировок стека. Если в системе достаточно активности, максимальный размер может быть легко достигнут. Этот ключ позволяет настроить размер базы данных.

Мы не будем обсуждать смысл всех различных параметров gflags в этой главе, поскольку это обсуждение предназначено лишь как введение в инструмент. На протяжении части II, "Прикладная отладка", мы будем использовать различные параметры, выставляемые через gflags, чтобы показать, как их можно применять для поиска некоторых действительно интересных и трудных проблем.

Process Explorer

Параметр	Значение
Сценарии применения	Анализ общего состояния системы и процессов
Текущая версия	10.2
Место загрузки	http://www.microsoft.com/technet/sysinternals/ProcessesAndThreads/ProcessExplorer.msp
Исполняемый файл	procexp.exe

Process Explorer - инструмент, первоначально разработанный командой SysInternals, а теперь входящий в Microsoft TechNet. Проще всего описать Process Explorer как мощную альтернативу Windows Task Manager. Он предоставляет подробную информацию обо всех процессах, выполняющихся в системе. Возможности включают:

- Подробные сведения об использовании дескрипторов, включая тип дескриптора и его имя. Он также предоставляет подробную информацию по каждому дескриптору, включая счетчик ссылок, сигнальное состояние и многое другое.
- Мощные возможности поиска позволяют искать дескрипторы по имени или типу во всех процессах либо, как альтернативу, искать любой процесс, в который загружен конкретный файл.
- Подробную информацию о процессе, такую как использование потоков, историю производительности, безопасность и многое другое.

Инструмент настолько мощный, что большинство пользователей, начавших им пользоваться, в итоге уже не возвращаются к традиционному Windows Task Manager. Более того, один из параметров Process Explorer - Replace Task Manager.

Инструмент поставляется в виде zip-файла: нужно просто извлечь его содержимое в выбранное место. Имя исполняемого файла - procexp.exe. На рисунке 1.7 показано, как выглядит Process Explorer при первом запуске.

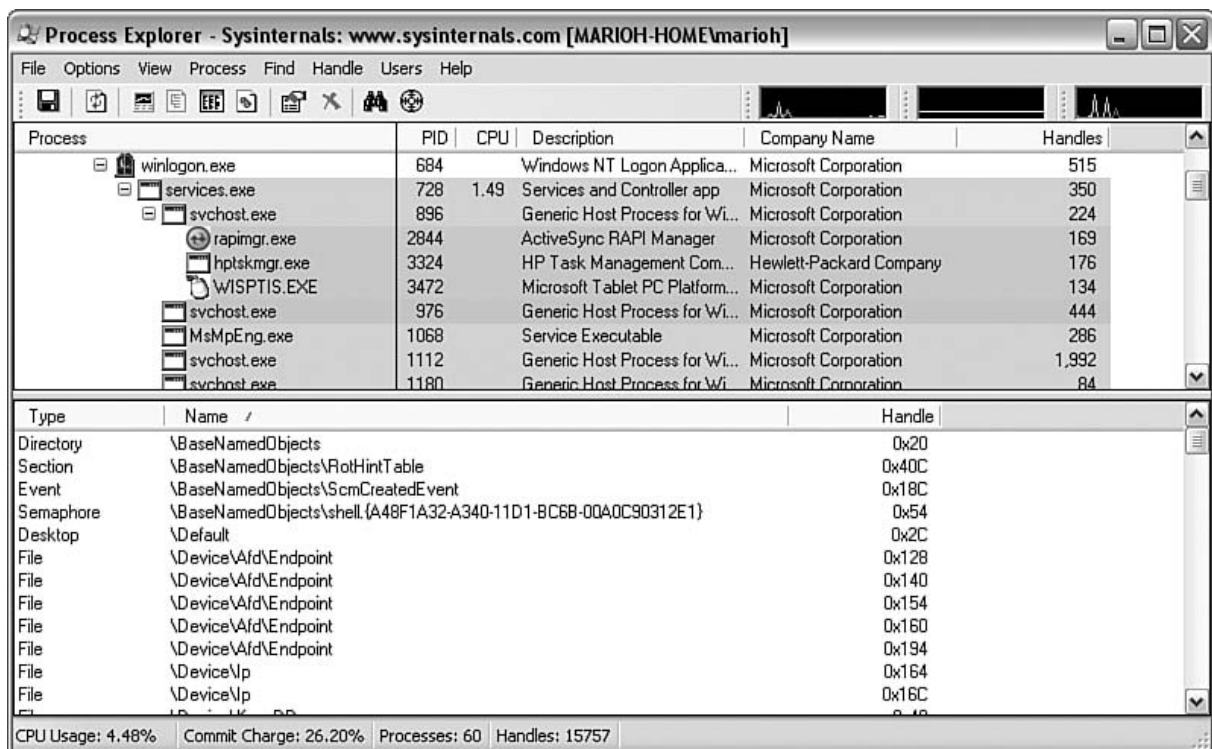


Рисунок 1.7. Главное окно Process Explorer

По умолчанию Process Explorer состоит из двух основных представлений. Верхнее представление перечисляет все процессы, выполняющиеся в системе, а нижнее показывает все дескрипторы, открытые выбранным процессом (а также имя дескриптора). Столбцы верхнего представления можно настроить, щелкнув правой кнопкой мыши строку состояния столбцов и выбрав меню Select Columns. Нижнее представление можно переключить с перечисления дескрипторов на перечисление DLL, выбрав DLLs в меню Lower Pane View меню View.

В главе 9 мы будем использовать Process Explorer, чтобы показать, как этот инструмент помогает в поиске утечек ресурсов.

Process Monitor. Process Monitor, еще один недавно выпущенный инструмент, связан с Process Explorer. Process Monitor - продвинутый инструмент мониторинга, который показывает активность файловой системы, реестра и процессов/потоков. Мы используем этот инструмент в нескольких главах книги. Инструмент бесплатен и доступен для загрузки по адресу <http://www.microsoft.com/technet/sysinternals/utilities/processmonitor.mspx>.

Windows Driver Kits

Параметр	Значение
Сценарии применения	Общая разработка
Текущая версия	WDK 6000
Место загрузки	Можно загрузить с http://www.microsoft.com/whdc/devtools/wdk/betawdk.mspx

Windows Driver Kits (WDK) - мощная и полноценная среда сборки, которую можно использовать для промышленной разработки. Эта среда разработки действительно примечательна, потому что включает большое количество мощных инструментов разработки (включая компилятор и компоновщик) и доступна бесплатно от Microsoft.

WDK поддерживает сборку для всех версий Windows, начиная с Windows XP и вплоть до Windows Vista включительно. Это позволяет разрабатывать для архитектур x86, x64 и Itanium.

Установка WDK проста, и обычно достаточно выбрать параметры по умолчанию. Когда установка начинается, вам будет предложено установить предварительные пакеты (такие как .NET Framework 2.0). После завершения установки этих пакетов можно выбрать установку WDK.

На рисунке 1.8 показаны различные параметры, доступные во время установки.

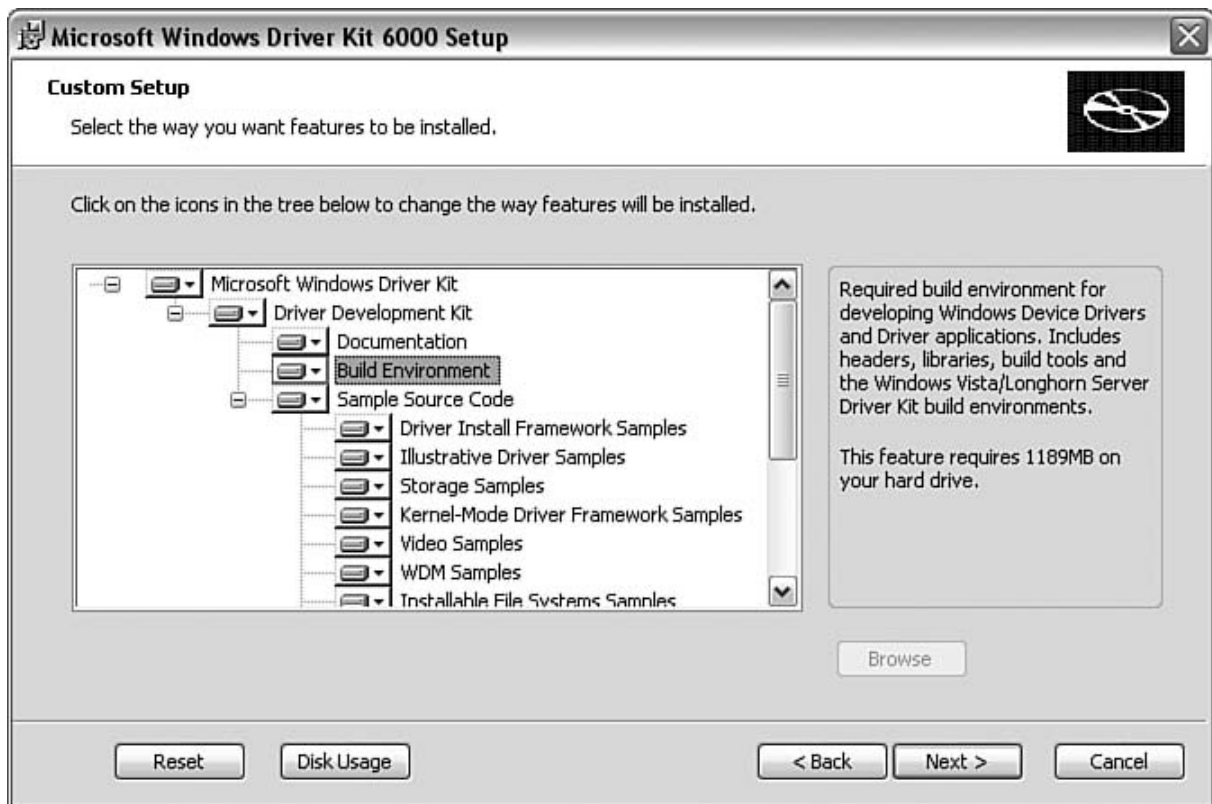


Рисунок 1.8. Параметры установки WDK

По умолчанию будут установлены среда сборки, документация, инструменты и примеры.

Путь установки WDK по умолчанию:

```
%systemdrive%\WINDDK\6000
```

Как упоминалось ранее, узел документации выбран по умолчанию. Если у вас нет ограничений по размеру жесткого диска или вы не знаете WDK досконально, этот выбор всегда следует оставлять.

Наконец, параметр Tools позволяет выбрать конкретные инструменты, которые нужно установить. Большинство инструментов в этой группе очень специфичны для разработчиков драйверов устройств, но некоторые (например, инструменты командной строки для реестра) могут быть очень полезны не только разработчикам драйверов устройств, но и во всех типах разработки.

После завершения установки все, что нужно сделать для начала сборки исходного кода, - открыть окно командной строки WDK через меню Start, All Programs, Windows Driver Kits, WDK 6000, Build Environments и выбрать нужную целевую платформу. Среда сборки WDK бывает двух разновидностей: free и checked. Версия free обычно является финальной версией продукта и содержит сильно оптимизированный код. Версия checked, напротив, используется во время разработки, чтобы упростить процесс поиска неполадок. Checked-версии обычно имеют минимальные оптимизации или не имеют их вовсе, что значительно облегчает отладку кода.

Откройте окно Windows XP Checked x86 Build Environment и перейдите в следующий каталог:

```
C:\AWD\Chapter1
```

Этот каталог содержит пример очень маленького консольного приложения. Чтобы собрать это приложение, введите build /ZC:

```
C:\AWD\Chapter1>build /ZC
BUILD: Adding /Y to COPYCMD so xcopy ops won't hang.
BUILD: Object root set to: ==> objchk_wxp_x86
BUILD: Compile and Link for i386
BUILD: Examining c:\awd\chapter1 directory for files to compile.
BUILD: Compiling (NoSync) c:\awd\chapter1 directory
```

```

Compiling - sample.cpp for i386
BUILD: Linking c:\awd\chapter1 directory
Linking Executable - objchk_wxp_x86\i386\sample.exe for i386
BUILD: Done
2 files compiled
1 executable built

```

Итогом этой успешной компиляции является `sample.exe`, расположенный в:

```
C:\AWDBIN\WinXP.x86.chk
```

Запуск этого примера дает:

```

C:>C:\AWDBIN\WinXP.x86.chk\01sample.exe
Welcome to Advanced Windows Debugging!!!

```

Важное замечание: результирующие выходные каталоги именуются согласно следующему соглашению:

```
obj<flavor>_<platform>_<architecture1>\<architecture2>\<target executable>
```

`flavor` может быть одним из следующих:

- `chk`: соответствует `checked`-сборкам
- `fre`: соответствует `free`-сборкам

`platform` может быть одним из следующих:

- `wnet`: соответствует Windows Server 2003
- `wxp`: соответствует Windows XP

`architecture1` может быть одним из следующих:

- `x86`: соответствует 32-разрядным процессорам Intel
- `amd64`: соответствует 64-разрядным процессорам AMD

Наконец, `architecture2` может быть одним из следующих:

- `I386`: соответствует 32-разрядным процессорам Intel
- `AMD64`: соответствует 64-разрядным процессорам AMD

Все примеры в этой книге собираются с помощью свободно доступного WDK; однако примеры должны корректно собираться и в среде Visual Studio, хотя тестирование в этой среде сборки не выполнялось.

Эта книга не стремится подробно описать каждый аспект WDK, а лишь использует базовый механизм сборки для создания реалистичных примеров сложных отладочных проблем, часто возникающих в мире программного обеспечения. Более подробные сведения о WDK см. в документации.

Ethereal

Параметр	Значение
Сценарии применения	Анализатор сетевых протоколов
Текущая версия	0.99
Место загрузки	http://www.ethereal.com/download.html
Механизм анализа	Сетевые трассы

Ethereal - мощный сетевой анализатор протоколов с открытым исходным кодом, который можно использовать для помощи в поиске неполадок при вызовах между машинами. Ethereal позволяет

захватывать и анализировать данные из живой сети или анализировать ранее созданные файлы захвата.

При установке Ethereum выберите типовой вариант установки.

В главе 8, "Межпроцессное взаимодействие", приведены примеры использования Ethereum для анализа и поиска проблем межпроцессного взаимодействия в вашем коде.

DebugDiag

Параметр	Значение
Сценарии применения	Поиск неполадок процессов (утечки памяти и сбои)
Текущая версия	1.0
Место загрузки	Входит в IIS Diagnostics Toolkit: http://www.microsoft.com/downloads/details.aspx?familyid=9BFA49BC-376B-4A54-95AA-73C9156706E7&displaylang=en
Механизм анализа	Отладчики, файлы журналов

DebugDiag изначально был разработан для помощи в анализе проблем производительности IIS, но его с таким же успехом можно использовать с любым процессом. Он сочетает следующие возможности поиска неполадок:

- Сбор данных о сбоях процессов: подобно отладчикам Windows, DebugDiag подключается к процессу и создает файлы дампа при возникновении сбоя или исключения.
- Утечки памяти: инструмент DebugDiag внедряет DLL в процесс, который должен отслеживаться на наличие утечек, и наблюдает за выделениями памяти во времени. Затем создается дамп, который можно проанализировать, чтобы найти путь кода, вызывающий утечку. В зависимости от шаблона выделений приложения инструмент вычисляет вероятность утечки.
- Мощная расширяемая объектная модель (на основе COM): она предоставляет информацию, необходимую для анализа утечек памяти и сбоев процессов.

При установке IIS Diagnostics Toolkit выберите типовой вариант установки.

Глава 14, "Мощные инструменты", дает примеры использования DebugDiag для помощи в анализе и поиске утечек памяти и сбоев процессов.

Резюме

Инструменты, описанные в этой главе, являются лучшими друзьями разработчика. Вместо того чтобы полагаться на дорогостоящий метод проб и ошибок при продвижении через сложные проблемы, эти бесплатные инструменты не только сократят время, которое вы тратите на выявление и отслеживание трудных ошибок, но и выявят ошибки, которые иначе могли бы не обнаружиться во время тестирования. Учитывая, что эти инструменты доступны бесплатно в виде простых загрузок, не должно быть причин не интегрировать их полностью в процесс разработки (что делает их отличным дополнением к интегрированным средствам разработки). Владение этими инструментами - ключевой компонент высокой эффективности в процессе отладки.

На протяжении оставшейся части этой книги мы покажем, как овладеть этими инструментами, используя их для поиска сложных и распространенных проблем.

Глава 2. Введение в отладчики

Процесс software debugging имеет разные значения в зависимости от programming language, использованного для создания продукта, а также от текущей ситуации и опыта developer. Хотя некоторые developers все еще выполняют debugging с помощью extensive console printouts или анализа verbose logging files, большинство использует специализированный tool: debugger.

Эта глава сосредоточена на Debugging Tools for Windows, свободно доступных от Microsoft Corporation.

В набор входит несколько debuggers, которые мы вскоре опишем.

Почему эти debuggers так важны?

Windows debuggers развиваются параллельно с процессом разработки Windows, поскольку они используются для debugging каждой версии operating system.

В результате они всегда синхронизированы с последней версией operating system или service pack. Поскольку те же tools также используются для debugging предыдущих версий operating systems, разработчики debuggers прилагают большие усилия, чтобы текущие debuggers были совместимы с существующими systems.

Когда конкретная functionality недоступна в более старых operating systems, debuggers fail gracefully. Чтобы понять уровень backward compatibility этих debuggers, достаточно упомянуть, что последние Windows debuggers работают с Windows 9x/Me, Windows NT, Windows 2000, Windows XP, Windows 2003 и Windows Vista.

Другие качества этих debuggers не так очевидны, например extensibility, minimal install и runtime requirements.

Functionality Windows debuggers можно расширять domain specific extensions, выполняющимися одновременно с существующими debugger commands.

Но они также очень гибкие, потому что не требуют local registration, что делает их по-настоящему хору "installable".

Они могут выполняться из любого location, например с USB thumb drive, куда debugger folder был скопирован из другой installation, и требуют очень мало memory.

В параллельном развитии 64-bit family operating systems Windows является первым шагом введения 64-bit computing в mainstream, и многие development companies уже планируют перевод 32-bit applications в 64-bit.

Debugging Tools for Windows предлагает превосходную debugging environment для 64-bit platform.

Все это делает Windows debuggers идеальным набором tools: мощным и пригодным в любой ситуации.

В этой главе мы рассмотрим:

- Основы Windows debuggers
- Как настроить Windows debuggers
- Как работать с symbols и sources
- Basic commands, доступные в Windows debuggers
- Как использовать Windows debugger remotely
- Несколько debugging scenarios

В этой главе используется 02sample.exe, специально подготовленная программа, помогающая познакомиться с Windows debuggers.

Source code и binary для 02sample.exe можно найти в следующих folders:

Source code: C:\AWD\Chapter2
Binary: C:\AWDBIN\winXP.x86.chk\02sample.exe

Debugger Basics

Этот раздел описывает types доступных debuggers, когда использовать каждый debugger и наиболее эффективный способ их применения.

User mode developers являются основной аудиторией этого раздела, даже если некоторые sections содержат references to kernel mode.

Debugger Types

Два basic types debuggers, обсуждаемых здесь, - это user mode и kernel mode debuggers.

User Mode Debuggers

Самая простая форма debugger способна debug single target user mode (UM) process.

User mode debuggers способны examining program state: running threads, memory content, registers и kernel objects, opened in the process space, представляющие debugger target.

Capabilities похожи на то, что способен делать target process, если он может execute code, похожий на code, выполняемый debugger.

User mode debuggers также способны modifying the state: changing thread execution order, changing registers' content и changing memory content, а также получать notifications о special events, происходящих в target process.

Этот scenario обычно известен как live debugging, потому что debugger может interact with debugger target, пока target process выполняется.

User mode debuggers также могут examine dump file, содержащий snapshot заданного process; это также известно как postmortem debugging.

Глава 13, "Postmortem Debugging", подробно описывает разные способы создания user mode dump files.

Поскольку эти snapshots представляют process state, они являются хорошим representation исходного running process и могут успешно использоваться для расследования разных problems с minimal impact on the application.

Debugging Tools for Windows поставляется с тремя user mode debuggers: cdb.exe, ntsd.exe и windbg.exe.

Эти три debugger построены вокруг одного и того же debugger engine, но expose одну и ту же functionality разными способами.

Все три способны debug console applications, а также graphical Windows programs.

Все три можно использовать для source-level debugging, если sources доступны, или для прямого machine-level debugging.

Короткое объяснение каждого поможет решить, какой из них использовать наиболее уместно.

- cdb.exe (CDB) - character-based console program, позволяющая low-level analysis of Windows user-mode memory and constructs.

CDB чрезвычайно powerful для debugging currently running или recently crashed program и проста в setup.

CDB может attach к vital subsystem processes, выполняющимся during early boot phase, например WinLogon или CSRSS, тогда как graphical debugger не работает так рано в boot process,

поскольку graphical subsystem еще не initialized.

Если target application является console application, target будет share console window с CDB.

Чтобы spawn отдельное console window для target console application, используйте command-line option -2.

- `ntsd.exe` (NTSD) идентичен CDB во всех отношениях, кроме того, что при запуске он spawn новое text window.

Точнее, CDB является console application, тогда как NTSD является GUI application, способным create its own console.

Как и CDB, NTSD полностью способен debug both console applications and graphical Windows programs.

Единственный случай, когда они не взаимозаменяемы, - debugging user mode system process.

В этом случае errors или breaks в process могут привести к неправильной работе всех console applications.

В таких случаях можно configure NTSD to run with no console at all.

- `windbg.exe` (WinDbg) - мощный graphical interface debugger с теми же debugging capabilities, что есть в console mode debuggers.

Он расширен для automation routine tasks: examining current call stack, viewing variables, including C++ objects, showing current registers и многого другого.

WinDbg также предоставляет удобный full source-level debugging, когда symbol files properly configured, как мы объясним далее в этой главе.

При startup некоторые settings WinDbg извлекаются из workspaces, которые можно change and save во время debugging session.

Все эти capabilities делают WinDbg preferred tool для interactive debugging user mode applications.

Kernel Mode Debuggers

В отличие от user mode debuggers, kernel debuggers могут inspect computer system as a whole, почти с тем же view, что и system processor.

Для kernel debuggers каждый process или thread - всего лишь collection of data structures.

Memory addresses имеют direct relation with physical memory, installed on the system, a paged out memory недоступна без loading it in physical memory.

Kernel mode debugger может change state of the entire computer и получать notifications о special events.

Эта модель debugging известна как live kernel debugging.

Kernel debuggers главным образом используются device driver developers, но также могут быть очень полезны при debugging user mode applications.

Несколько scenarios, описанных в этой книге, используют kernel mode debuggers, даже если debugged code выполняется полностью in user mode.

Во многом так же, как user mode debuggers могут load user mode dumps, kernel debugger может load kernel mode dumps и выполнять offline debugging existing system или postmortem analysis of the bug checks.

Windows debuggers содержат два basic kernel mode debuggers: `kd.exe` и `windbg.exe`.

- `kd.exe` (KD) - символьный отладчик режима ядра.

Он позволяет выполнять глубокий анализ активности режима ядра в Windows и может использоваться для отладки программ и драйверов режима ядра, отладки приложений пользовательского режима или наблюдения за поведением самой операционной системы.

- `windbg.exe` (WinDbg) также способен выполнять kernel mode debugging.

WinDbg предоставляет full source-level debugging для Windows kernel, kernel-mode drivers, а также user mode applications, running on the system.

Он позволяет debug any application or kernel module в friendly user interface с помощью tracing source code, setting breakpoints based on source content и многого другого.

Отладчики ядра способны отлаживать целевой компьютер, работающий на платформе, отличной от платформы хоста.

Отладчик автоматически определяет платформу, на которой работает цель.

Команды отладчика

Отладчики Windows поддерживают набор команд, изначально реализованных в исполняемом файле; они вводятся без специального префикса в командной строке.

Большинство коротких команд, таких как kP, являются встроенными командами.

Meta-commands - другой set of commands, implemented by the executable file, который начинается с dot (.).

Например, .help - meta-command, отображающий все meta-commands, implemented by debuggers.

Кроме того, Windows debuggers позволяют использовать debugger extension commands.

Extensions добавляют debugger power and flexibility, расширяя range of functions, которые можно execute against debugger target, и упрощая parsing target data and structures.

Extension support enables a model, в котором дополнительные extensions можно добавлять в debugger для component-specific и driver-specific debugging.

Debugger extensions иногда называют "bang" commands, чтобы указать, что все они prefixed with exclamation point (!).

Debugger extension commands используются примерно так же, как standard debugger commands.

Однако, хотя built-in debugger commands являются частью самих debugger binaries, debugger extension commands exposed by DLLs, separated from the debugger.

Ряд debugger extension DLLs поставляется вместе с самими debugging tools.

Syntax, используемый для вызова debugger extension, имеет вид !module.extension [arguments], где module name - имя debugger extension DLL, а extension name - function, exported by that DLL.

Extension function также может accept parameters через arguments в command line.

Эти extension commands вводятся в debugger prompt так же, как другие commands.

Разные DLLs, поставляемые с kernel debugger, предоставляют default kernel and user mode extensions, including kext.dll and exts.dll.

Когда extension вызывается without a module name specified, эти DLLs всегда checked, если только не загружена другая extension DLL, containing that command.

Examples debugger extensions, supported by these DLLs, include !teb для получения thread environment block using a thread from any debugger.

Другой пример - !thread для получения information on the current or a specific thread from kernel mode debugger.

Extension DLL может быть implicitly loaded вызовом function из этой DLL с full !module.extension syntax.

Extension DLL также можно explicitly loaded с помощью debugger command .load, указав full path to the DLL.

После loading все остальные extension functions можно вызывать without specifying the extension DLL, unless the same function is implemented in two loaded extensions.

В этом случае full syntax must be used to resolve the name collision.

Setting Up the Debuggers

Даже в basic usage Windows debuggers предоставляют exceptional and valuable flexibility, одновременно заставляя выбирать между разными options. Этот section подробно описывает options, позволяющие configure debugger для всех cases, представленных в этой книге.

User Mode Debuggers

Debuggers нужны по меньшей мере два key ingredients, чтобы работать на полную мощность: target image, being debugged, и symbol information, associated with that image.

В этом section мы сосредоточимся на setting up debugger target.

Более поздний section "Setting Up and Using the Symbols" показывает, как load associated symbols для debugger target.

Некоторые examples из этого section используют cdb.exe, но они работают аналогично с windbg.exe или ntsd.exe.

В наиболее распространенной ситуации debugger запускает новый process, а target image загружается в newly created process, который становится debugger target.

Используя executable tlist.exe, located in the debugger installation folder, можно увидеть debugger как parent of the debugged process.

Executable name передается в debuggers как parameter, как видно в листинге 2.1.

Command line, starting the debugger, выглядит как cdb 02sample.exe.

Debugger cdb.exe с process identifier 2428 является parent для process 02sample.exe с process identifier 2816.

Листинг 2.1. Вывод всех процессов как task tree

```
C:\> REM tlist with -t parameter displays the process tree
C:\> tlist -t tlist will display the process tree
System Process (0)
System (4)
smss.exe (756)
csrss.exe (836)
winlogon.exe (864)
services.exe (908)
svchost.exe (1080)
svchost.exe (1152)
svchost.exe (1216)
svchost.exe (1348)
svchost.exe (1408)
spoolsv.exe (1748)
svchost.exe (572)
svchost.exe (1688)
lsass.exe (920)
explorer.exe (3552) Program Manager
cmd.exe (2856) C:\WINDOWS\system32\cmd.exe - tlist -t
cdb.exe (2428) cdb 02sample.exe
02sample.exe (2816)
tlist.exe (268)
```

При debugging process, в котором actual process lifetime управляется external entity, один подход состоит в attach debugger к running process.

Section "Debugging Scenarios" ближе к концу этой главы описывает additional options для debugging такого process.

Этот подход используется при debugging Windows services, DCOM servers, IIS filters и т. д.

Листинг 2.2 показывает list of switches, которые можно использовать при attaching to already running process.

Листинг 2.2. Options для attach debugger к running process

```
C:\>cdb -?
cdb version 6.4.0004.3
usage: cdb [options]
Options:
...
<command-line> command to run under the debugger
- equivalent to -G -g -o -p -l -d -pd
[ more]
-p <pid> specifies the decimal process ID to attach to
-pn <name> specifies the name of the process to attach to
-psn <name> specifies the process to attach to by service name
-pv specifies that any attach should be noninvasive
-pvr specifies that any attach should be noninvasive and nonsuspending
...
```

Хотя большинство options, отображаемых command help, self-explanatory, мы подчеркнем несколько helpful parameters, которые стоит использовать при attaching debugger to a running process.

cdb.exe -p <pid> - standard command, используемая, когда process identifier известен.

Если image name известен, как в случае DCOM servers или SCM services, cdb.exe -pn <image name> отлично справляется с finding its process identifier and attaching to it.

Однако если несколько processes started with the same image, command bails out, как показано здесь:

```
C:\>cdb -pn svchost.exe
There is more than one 'svchost.exe' process running. Find the process ID of the
instance you are interested in and use -p <pid>.
```

В этом случае мы находим target process identifier с помощью tlist.exe и используем его как parameter для command cdb -p <pid>.

Специально для service writers, sharing the same host image name, можно указать service name как parameter: cdb -psn <service name>.

Last, but not least, -pv можно использовать со всеми другими options, чтобы attach nonintrusively to a running process.

Это позволяет access process information, даже если к этому process attached another debugger или previous debugger hung: bad extensions, long symbols resolution и т. д.

Листинг 2.3 показывает command line, used to attach nonintrusively to the dnscache service, а также output, generated by the debugger.

Листинг 2.3. Nonintrusive debugging service

```
C:\>cdb.exe -pv -psn Dnscache
...
*** wait with pending attach
Symbol search path is: SRV*c:\symbols*http://msdl.microsoft.com/download/symbols
Executable search path is:
WARNING: Process 1320 is not attached as a debuggee
The process can be examined but debug events will not be received
.....
(528.52c): Wake debugger - code 80000007 (first chance)
eax=0007fc44 ebx=00000000 ecx=7c80999b edx=02160001 esi=00000000 edi=00000068
eip=7c90eb94 esp=0007fc48 ebp=0007fcb0 iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
ntdll!KiFastSystemCallRet:
7c90eb94 c3 ret
```

Debugging session заканчивается, когда debugger target прекращает существовать, или когда вы используете command q (quit) либо command qd (quit and detach).

Последний option оставляет debugger target running.

Пункт меню Exit в меню File WinDbg, то есть key combination ALT+F4, эквивалентен command q.

Распространенный scenario, встречающийся в development centers, - dumping process memory on error и restarting test process.

В этом случае memory dump можно load как active target с помощью command windbg -z <dumpname>.

Листинг 2.4 показывает, как load one dump file, который ранее был generated from a running instance of the notepad.exe process.

Глава 13 описывает multiple ways to create memory dump files и effectively use them.

Листинг 2.4. Debugging memory dump

```
C:\>windbg -z c:\AWDBIN\DUMPS\notepad.dmp
...
Loading Dump File [C:\AWDBIN\DUMPS\notepad.dmp]
User Dump File: Only application data is available
...
.....
eax=7ffdc000 ebx=00000001 ecx=00000002 edx=00000003 esi=00000004 edi=00000005
eip=7c901230 esp=0091ffcc ebp=0091ffff iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=0038 gs=0000 efl=00000246
ntdll!DbgBreakPoint:
7c901230 cc int 3
```

Отладчики ядра

Отладчик ядра обычно выполняется на системе, отличной от отлаживаемой.

Живая отладка режима ядра требует двух компьютеров: хост-компьютера, на котором работает отладчик ядра, и целевого отлаживаемого компьютера, поскольку цель отладчика не может выполнять код, пока она остановлена в отладчике ядра.

Цель отладчика - это система, в которой произошел сбой программного компонента, системной службы, приложения или операционной системы.

Эта система может находиться в нескольких футах от компьютера, на котором выполняется отладчик ядра, или в совершенно другом месте, в зависимости от используемых вариантов подключения.

Цель отладчика также может быть виртуальной машиной, работающей внутри хост-системы.

Отладчик ядра очень гибок.

Он может работать с целевыми компьютерами на платформах x86, Itanium или x64.

Отладчик ядра автоматически определяет целевую платформу.

Операционная система, работающая на хост-компьютере, не обязана быть той же версии, что и операционная система, работающая на цели отладчика.

Однако рекомендуется поддерживать отладчик ядра в актуальном состоянии, чтобы он поддерживал последние версии операционной системы, используемые целью отладчика.

Часть debugging system живет внутри operating system и работает независимо от того, подключен ли kernel debugger к системе.

Поскольку эта portion является integral part of the Windows kernel, kernel debugger не требует установки additional software на debugger target.

Эта functionality configured at boot time.

Например, system enabled for kernel debugging freezes при вводе CTRL-SysReq с PS/2 keyboard.

В этом state kernel debugger может connect to this system and debug it.

На x86 computers running Windows XP kernel debugger можно enable в файле boot.ini, или его можно enable interactively at boot time, выбрав Windows Advanced Option после нажатия key F8 from the boot console, как показано на рисунке 2.1.

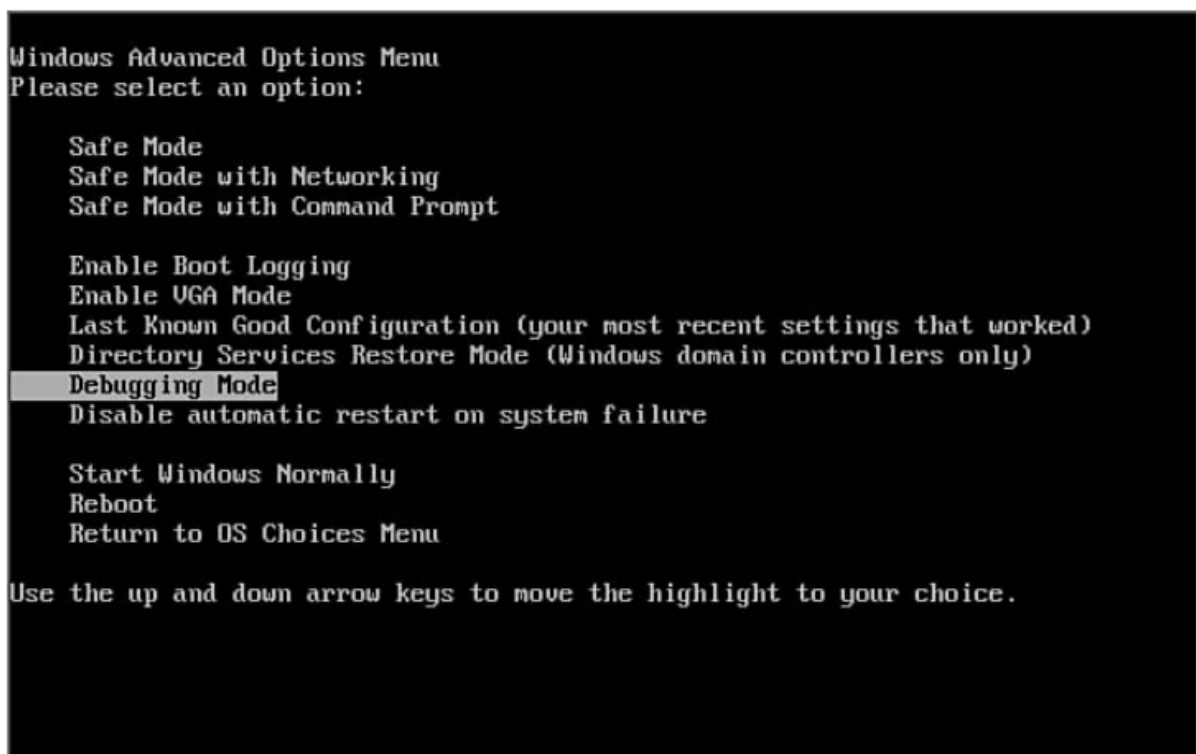


Рисунок 2.1. Windows Advanced Options menu

Ниже показан sample entry с несколькими parameters, controlling kernel debugger, такими как /debug, enabling the debugger, /debugport, representing serial port used by the kernel debugger, и /baudrate, serial port's baud rate.

Полное описание всех available options при changing boot.ini см. в debugger help: help topic Boot parameters to Enable Debugging.

Несмотря на доступную documentation about boot.ini, самый безопасный способ changing configuration files - through bootcfg.exe, поскольку он guarantees correctness of startup parameters. Простой файл boot.ini, который запускает default installation с активным kernel debugger на COM1 port, initialized at 57600 baud rate, показан здесь:

```
[boot loader]
timeout=30
default=multi(0)disk(0)rdisk(0)partition(1)\WINDOWS
[operating systems]
multi(0)disk(0)rdisk(0)partition(1)\WINDOWS="KD" /fastdetect /debug /debugport=COM1
/baudrate=57600
```

Предполагая, что serial cable подключен к serial port COM2 host system, следующую строку можно использовать для запуска kernel debugger через этот port at 57600 baud rate.

```
C:\>windbg -k com:port=COM2,baud=57600
```

Kernel debugger enabled, если в boot.ini найден любой debug parameter, независимо от наличия switch /debug.

Connecting the Kernel Debuggers

В наиболее распространенном случае, на live operating system, kernel debugger connects to the target operating system с помощью serial null-modem cable, но уже доступны и более быстрые способы connect, например IEEE 1394 или USB 2.0 cables.

Сегодня каждое connection является physical connection, represented by a cable, как показано на рисунке 2.2.

Но в ближайшем будущем могут появиться другие connection paradigms, например kernel debugging support over TCP/IP с использованием dedicated networked controller board, который runs independent from the host computer.



Рисунок 2.2. Подключение kernel debugger к target system

Для target computers running Windows XP or higher connection from the debugger to the target computer можно establish с помощью IEEE 1394 (FireWire) cable.

Connection to target computers running Windows Vista or higher может использовать USB 2.0 debug cable connection.

Выбранный connection method определяется available hardware для establishing connection и characteristics target computer.

Дополнительную information about connection options and command line, required to use such a connection, см. в debugger help file: help topic Choosing Kernel Debugging Settings.

Полезен ли kernel debugger, если вы не можете использовать два computers из-за restrictions environment?

В этом случае можно simulate target machine in a virtual machine environment и по меньшей мере получить те же options, что и в two machine setup case.

Сейчас большинство virtualization software products на рынке предлагает free version.

Хотя этот section использует Microsoft Virtual PC как example, такая же functionality доступна во всех virtualization products.

За исключением hardware-specific software, все другие software components могут успешно run and be debugged within a virtual machine.

Эмулятор виртуальной машины виртуализирует последовательный порт, доступный на целевом ПК, в именованный канал в пространстве имен хост-компьютера.

На рисунке 2.3 serial port COM2 Microsoft Virtual PC доступен как named pipe on the host PC с именем `\\.\pipe\pipe2`.

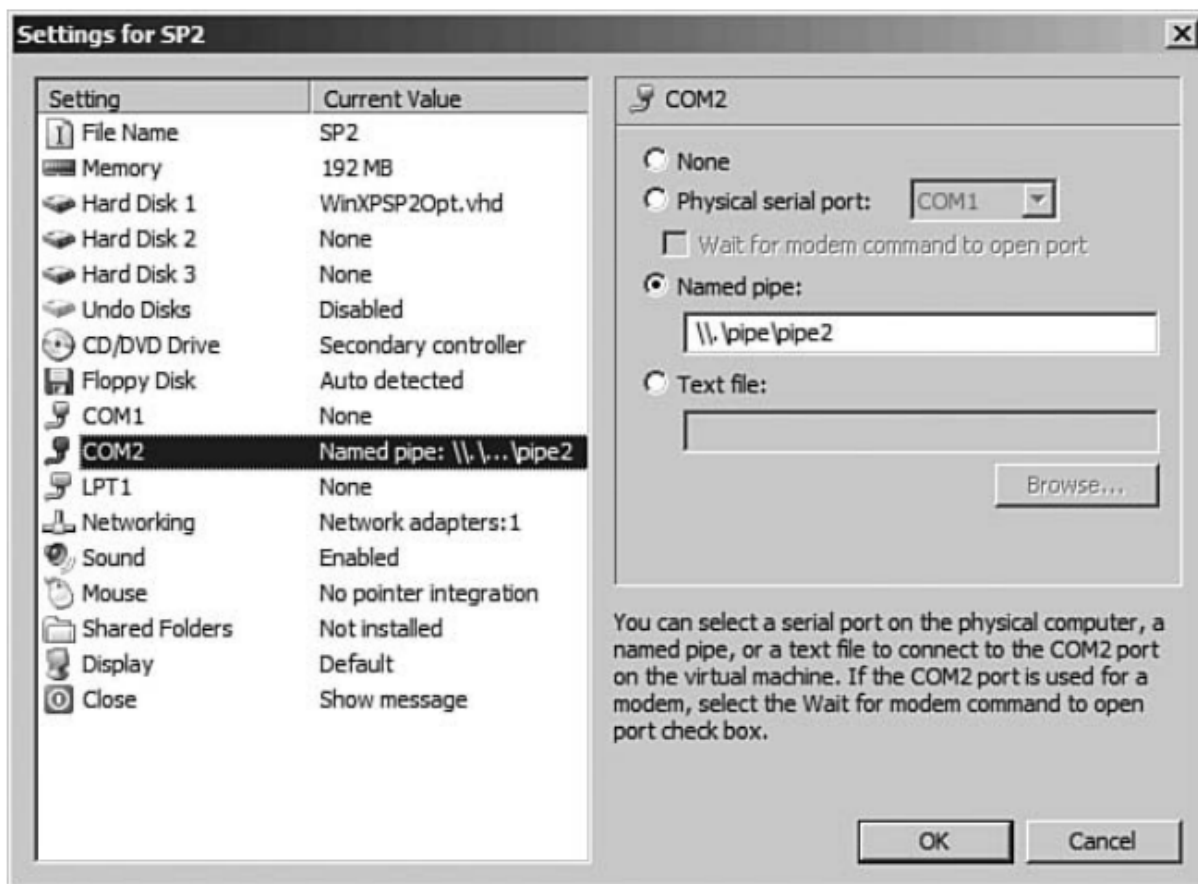


Рисунок 2.3. Enable Virtual PC for kernel debugger

Затем kernel debugger может connect to the virtual machine с settings, shown in Figure 2.3, используя следующую command line:

```
C:\>windbg -k com:pipe,port=\\.\pipe\pipe2
```

Kernel mode debugging session заканчивается, когда debugger target прекращает существовать или когда kernel debugger disconnects from the target с помощью command CTRL+B.

Если debugger target ожидает user input перед disconnecting kernel debugger, system state не меняется до тех пор, пока к нему не подключится новый kernel debugger или пока system не будет restarted.

Пункт меню Exit в меню File WinDbg, то есть key combination ALT+F4, эквивалентен command CTRL+B.

Если использование virtual machine невозможно из-за license constraints, все еще можно получить пользу от kernel debugger in local connection mode, functionality introduced starting with Windows XP. У вас очень limited functionality in controlling the target, но unlimited options to view the machine status. Любую memory write следует very carefully inspect, потому что она может potentially corrupt integrity of the operating system running the kernel debugger.

Как и при любой kernel debugger setup, соответствующая entry boot.ini должна specify flag /debug. Kernel mode debugger может start in local mode с помощью следующей command line:

```
C:\>windbg -kl
```

Kernel mode debugger также может open kernel dump files, generated using the methods described in Chapter 13.

И kd.exe, и windbg.exe могут open kernel dumps, поэтому выбор между ними является personal preference.

windbg.exe recognizes kernel dump file type и starts in kernel mode debugging без requirement for additional command-line parameter.

Следующие command lines могут open mini dump files, captured automatically by the operating system in the %windir%\Minidump folder, а также некоторые manually generated dumps.

```
C:\>kd -z %temp%\full.dmp
C:\>kd -z %windir%\Minidump\Mini091704-01.dmp
C:\>windbg -z %wtemp%\full.dmp
```

Перенаправление отладчика пользовательского режима через отладчик ядра

Одна важная feature kernel debugger - capability control user mode debugger for the kernel debugger session и synchronize user mode debugging session with system activity.

Поскольку system activity frozen, пока вы control user mode debugger, это можно использовать для debug sequences, которые expected to execute in a bound time period, то есть time relative to the system activity.

Так как kernel debugging session already established at system boot time, можно debug processes early in the startup phase или very late in the system shutdown phase, когда interactive console недоступна.

Kernel debugger также дает access to information, unavailable from a user mode session debugger, что делает эту combination самой мощной формой user mode debugging.

Запуская user mode debugger с parameter -d в command line, любой user mode debugger redirects its input and output to a kernel debugger, как в следующем listing:

```
C:\>ntsd -d <Process Path>
C:\>ntsd -d -p <PID>
```

Kernel mode debugger должен быть enabled перед использованием redirection options.

Иначе user mode debugger returns to the command prompt without executing command, passed in as a parameter.

Однако при enabled kernel debugger operating system позволяет low privilege users stop the entire activity, что не всегда desired.

Когда debugger находится в state, в котором он waits for user input, либо at the user mode prompt, либо at the kernel mode prompt, как показано на рисунке 2.4, kernel activity suspended.

Точное состояние ясно определяется по вводу отладчика.

KD показывает user mode prompt как regular user mode debugger, тогда как WinDbg, used as a kernel debugger, показывает prompt as Input> instead of the regular kd> prompt.

Нередко приходится переходить back and forth между kernel mode debugger и user debugger before resolving problems involving interprocess communication.

После entering a new command at the user mode debugger prompt kernel mode debugger dispatches that command to the current user mode debugger and resumes system activity, allowing user mode debugger to perform the command.

Если после executing the command user mode debugger prompts the user, system goes back to the user mode debugger prompt.

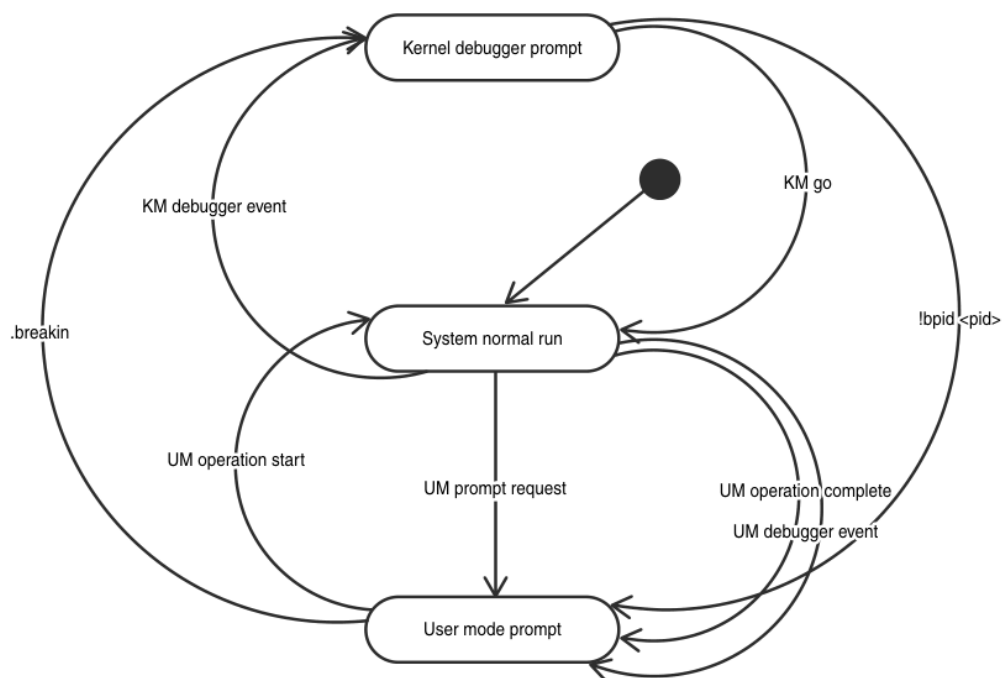


Рисунок 2.4. State transition между kernel mode prompt и user mode prompt

Находясь в user mode prompt state, можно jump to kernel mode prompt state, введя command `.breakin` в user mode debugger.

Отладчик ядра останавливается в контексте процесса отладчика, а не отлаживаемого процесса:

```

0:000> .breakin
.breakin
Break instruction exception - code 80000003 (first chance)
nt!RtlpBreakWithStatusInstruction:
8051ac9c cc int 3
kd> !process -1 0
PROCESS ff7eeb38 SessionId: 0 Cid: 055c Peb: 7ffdf000 ParentCid: 03c8
DirBase: 03983000 ObjectTable: e1a02fb8 HandleCount: 39.
Image: ntsd.exe

```

Эта command requires `SeDebugPrivilege` privileges для самого debugger process и fails with an explicit error, если debugger не выполняется under an account having the debug privilege:

```

0:000> .breakin
.breakin
.breakin requires debug privilege

```

В таких случаях alternative way to go into KD - issue a break с помощью CTRL+C, CTRL+break или CTRL+SysRq после просьбы user mode debugger выполнить что-нибудь long running, например sleep command, как видно в листинге 2.5.

Key combination CTRL+C интерпретируется kernel mode debugger как kernel mode event.

Листинг 2.5. Switching from user mode to kernel mode debugger

```

0:000> .sleep 1000
.sleep 1000
Break instruction exception - code 80000003 (first chance)
*****
*
* You are seeing this message because you pressed either
* CTRL+C (if you run kd.exe) or
* CTRL+BREAK (if you run WinDBG)
* on your debugger machine's keyboard.
*
* THIS IS NOT A BUG OR A SYSTEM CRASH

```

```

*
* If you did not intend to break into the debugger, press the "g" key,*
* then press the "Enter" key now. This message might immediately
* reappear. If it does, press "g" and "Enter" again.
*
*****
nt!DbgBreakPointWithStatus+0x4:
8051ac9c cc int 3
kd>

```

Из kernel mode prompt можно enter the system in normal execution mode, введя любую форму command g.

Если user mode debugger prompts the user, system moves to the user mode prompt.

Transition back into the user mode prompt difficult, когда нет user mode prompt или когда new debugger event, requiring user prompting, был sent to the kernel debugger.

Самый reliable method to regain control of the user mode debugger - использовать utility `breakin.exe`, installed with Debugging Tools for Windows.

`breakin.exe` принимает только один parameter: process identifier of the target process that must be stopped.

В этом случае process identifier - это user mode process, previously started under the user mode debugger.

Command `breakin.exe <pid>` выполняется directly on the target computer being debugged.

Из kernel debugger prompt можно regain user mode debugger prompt с помощью extension command `!bpid <pid>`.

Useful command для suspending user mode debugger - `.sleep <time>`.

Эта command leaves the target system in a normal running state for the specified time interval, во время которого system можно использовать для operations, например copying local symbols или даже attach user mode debugger к another process.

DEFAULT NUMERIC BASE IS IMPORTANT.

Если вы когда-нибудь wondered, почему command `.sleep 1000` ощущается скорее как четыре секунды, чем как одна секунда, нужно отметить, что timeout interpreted according to the current radix used by the debugger; default base is 16.

To KD or Not to KD

Большинство application developers не рассматривает использование kernel debugger, поскольку это кажется unnecessary, если не too complicated.

Мы хотим, чтобы вы рассмотрели cases, в которых kernel debugger является natural way of debugging a particular problem.

Как именно это делать, подробно показано в более позднем section "Debugging Scenarios", а также в некоторых других chapters этой книги.

В таких cases все alternative solutions for debugging the problem обычно являются просто expensive workarounds.

На другом конце spectrum находятся cases, в которых kernel debugging вообще не является option, главным образом потому, что другие components, installed on the system, не могут хорошо работать в его presence.

В эту category можно включить разные products, использующие files, protected by Digital Right Management (DRM) technologies.

Такие products стали commonly used in our lives, чтобы securely store music или protect confidentiality of our files.

К сожалению, products, capable of reading or writing DRM-protected content, не работают с debuggers, including kernel mode debuggers.

Ожидается, что все такие products используют всевозможные anti-debugging tricks и debugging detection mechanisms.

В наиболее распространенном case они просто refuse to work, если detected kernel mode debugger.

В этом case каждый scenario, для которого мы recommend use of a kernel debugger, should instead use an alternative, non-KD method.

На development phase бывают cases, в которых user developed application видит огромный number of failures, когда enabled kernel mode debugger.

В этом case product может содержать special function calls, named asserts, которые break in the debugger for specific parameters.

Эти assert statements были introduced by developers просто to validate their thinking.

Когда assert statement больше не valid in the customer environment и kernel mode debugger enabled, application часто breaks in the kernel debugger.

В этом case correct solution should be tailored to the environment: disabling kernel mode debugger, updating application или removing assert statement.

SECURITY NOTE. Если вы enable kernel debugger на system, shared by multiple users, debugger не будет differentiate между handling breakpoints on low privileged users' processes и breakpoints in processes running under a system or administrator account.

Enabling KD this way, вы позволяете любому user break the system и put the system's service into a nonfunctional state.

Поэтому best practice - disable kernel debugger on production systems.

Теперь мы можем recognize некоторые situations, в которых kernel debugging не является acceptable technique in the toolbox, но не всегда уверены, когда он действительно useful.

Поэтому в более позднем section "Debugging Scenarios" мы раскроем typical situations, в которых kernel debugger extremely useful.

Basic Debugger Tasks

После setting up debugger вы должны увидеть command prompt или debugger window, waiting for your commands.

После entering a new command debugger switches to execution mode, executes command, displaying the results, and switches back into command prompt mode.

Если entered command requires target to execute code, любой debugger event, encountered while executing the command, returns debugger back into command mode.

В следующих sections мы описываем некоторые most used commands и даем brief description resultant output, highlighting the most relevant information from it.

Entering Debugger Commands

В console-based debuggers ntsd.exe, cdb.exe и kd.exe entire console window используется для display results of commands, entered at the command prompt.

В WinDbg output window является special window, identifiable by the Command title.

Window имеет input box at the bottom, используемый для entering commands таким же образом, как в console-based debuggers.

Menu item Command в Tool menu можно использовать для display command windows; alternatively, shortcut Alt+1.

Одно большое advantage GUI interface - capability show multiple views of the debugged process at the same time.

Это устраняет need to enter a new command to display that piece of information и позволяет accept commands from the menu and toolbar.

Все user interface commands имеют corresponding textual command и могут be entered in the command window.

Поскольку command window WinDbg more or less identical to console of any text-based debugger, все examples in this book illustrated using command window commands.

Кроме того, одно из biggest advantages WinDbg over console mode debugger - source mode capabilities.

При proper access to symbol and source files, managed using a process similar to the one described in Chapter 4, "Managing Symbol and Source Files", power of WinDbg fully realized.

User получает debugger, который automatically retrieves source files, shows and synchronizes multiple views into debugger target, while enabling fine control of debugger target using command prompt.

Этот debugger также можно extend with business-specific functionality, как объясняется в Chapter 11, "Writing Custom Debugger Extensions".

Можно использовать любую command из multitude of debugger commands или debugger extensions commands, но ваша goal - resolve a specific problem, и стоит follow some general directions.

Общий рабочий процесс, используемый для разбора сеанса отладки, начинается с определения текущего отладочного окружения и, если возможно, исправления любых проблем с символами.

Следующий шаг - понять, почему отладчик остановился именно там, где остановился, и с доступной информацией создать возможные сценарии, приведшие к текущей остановке.

Имея в виду каждый такой scenario, следует использовать any piece of information from the debugger session, чтобы try to prove that the scenario was really executed.

Если мы find any contradiction, следует go back and try another scenario.

Когда scenario proven by current state of the application, developer идет в source code, finds the problem and fixes it.

В следующем section мы explore basic commands, used to explore application state required in the steps described previously.

Interpreting the Debugger Prompt

Не вводя никаких commands в debugger и просто looking at the debugger prompt, including some of the previous console output, можно выяснить несколько details concerning debugger target.

Мы начнем с examining normal output from a user mode debugger immediately after starting a new process, например `c:\>windbg notepad`.

Output показан в листинге 2.6.

Листинг 2.6. User mode debugger output

```
(2d4.23c): Break instruction exception - code 80000003 (first chance)
eax=7ffdf000 ebx=00000001 ecx=00000002 edx=00000003 esi=00000004 edi=00000005
eip=77f75a58 esp=0084ffcc ebp=0084fff4 iopl=0 nv up ei pl zr na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=0038  gs=0000  efl=00000246
ntdll!DbgBreakPoint:
77f75a58 cc int 3
0:000> vertarget
Windows XP Version 2600 (Service Pack 2) UP Free x86 compatible
Product: WinNt, suite: SingleUserTS
kernel32.dll version: 5.1.2600.2180 (xpsp_sp2_rtm.040803-2158)
Debug session time: Mon May 28 20:21:23.486 2007 (GMT-7)
System Uptime: 2 days 18:44:45.827
Process Uptime: 0 days 0:01:04.402
Kernel time: 0 days 0:00:00.000
User time: 0 days 0:00:00.010
```

```

0:000> .lastevent
Last event: 2d4.23c: Break instruction exception - code 80000003 (first chance)
0:000> ||
. 0 Live user mode: <Local>

```

Первая строка содержит process и thread identifier, generating the last debugger event; debugger events подробнее описаны в Chapter 3, "Debuggers Uncovered".

Они отображаются как (2d4.23c) вместе с event description, break instruction exception, и exception code 80000003.

Отладчик обработал событие на первом шансе, до обычной обработки исключений в пользовательском коде.

Exception handling подробнее рассматривается в Chapter 3.

Эта information не всегда available, но ее следует использовать, если мы можем ее найти.

Register values, displayed on the next few lines, сейчас не столь relevant, за заметным exception of instruction pointer (eip) and stack pointer (esp).

Register structure сообщает об architecture, under which this process runs, например x64 или Itanium.

Сразу после register information находится symbol associated with the address where the last event was raised, вместе с address and instruction at that address.

Как вы увидите в remainder of the book, сама instruction can explain immediate cause of the break.

Последняя часть information from debugger output - command prompt.

Prompt (0:000>) говорит, что мы находимся in the user mode debugger.

Для kernel mode debugger session prompt содержит string kd.

Первое number указывает active target этого debugger, и в большинстве debugging sessions оно будет 0.

Второе number представляет thread "number" потока, raising the debugger event.

DEBUGGING MULTIPLE TARGETS. Не очень widely known fact: Microsoft debuggers способны debug multiple remote systems at the same time.

В этом case debugger изменит prompt и prefix the prompt with the system name as 0:0:000>.

Подробнее об этом можно прочитать в debuggers help в topic "Debugging Targets on Multiple Computers".

Приглашения отладчика ядра раскрывают информацию о выполняющемся окружении и причине остановки.

Использование option 2 программы 02sample.exe in the presence of the kernel debugger causes the whole system to stop.

Листинг 2.7 показывает kernel debugger console output while using the same commands as in the previous listing.

Листинг 2.7. Kernel mode debugger output

```

Break instruction exception - code 80000003 (first chance)
7c901230 cc int 3
kd> vertarget
Windows XP Kernel Version 2600 (Service Pack 2) UP Free x86 compatible
Product: WinNt, suite: TerminalServer SingleUserTS
Built by: 2600.xpsp_sp2_rtm.040803-2158
Kernel base = 0x804d7000 PsLoadedModuleList = 0x8055ab20
Debug session time: Tue May 29 20:47:16.107 2007 (GMT-7)
System Uptime: 0 days 0:11:24.844
kd> .lastevent
Last event: Break instruction exception - code 80000003 (first chance)
debugger time: Tue May 29 20:48:23.671 2007 (GMT-7)
kd> ||
. 0 Remote KD:
KdSrv:Server=@{<Local>},Trans=@{COM:Port=\\.\pipe\pipe1,Baud=19200,Pipe,Timeout=4000,
Resets=2}

```

Первые несколько lines indicate cause of the current break, а amount of information зависит от stop type.

В этом example kernel debugger encountered a break instruction and stopped.

Debugger также сообщает exception code 80000003, generated by the break instruction.

Следующая line содержит address of current instruction pointer, followed by current instruction in assembly language.

64-bit address for the instruction indicates, что current processor runs in 64-bit mode.

В этом case 32-bit address indicates processor executing in 32-bit mode.

Версия и архитектура операционной системы отображаются в ответ на команду `vertarget`.

Debugger uses `kd>` as a prompt, когда debugger target is a single processor system, и `n:kd>` as a prompt, когда debugger target has more than one processor.

Число обозначает номер логического процессора, сгенерировавшего текущее отладочное событие.

Настройка и использование символов

Debugging application break without proper symbols difficult, и шансы discover the problem in that application минимальны.

Неудивительно, что determining accuracy of the symbol information - самый important step in debugging.

Bad symbols могут lead you in wrong directions and create unrealistic hypotheses.

В этом section мы обсуждаем, как использовать symbol files и discover their importance in debugging.

Что такое файлы символов?

Когда applications, libraries, drivers или operating systems built, compile and link procedure создает `.exe`, `.dll`, `.sys` и другие executable files.

Эти files collectively known as binaries or images; та же procedure также создает ряд additional files, known as symbol files.

Чтобы effectively debug target image, вся symbolic information, generated at compile and link time, должна быть available to the debugger.

По разным reasons, ranging from compilation performance to IP protection, Microsoft used several symbol formats, such as Common Object File Format (COFF), CodeView format (CV), and Program Database format (PDB).

Таблица 2.1 представляет некоторые characteristics этих formats.

Таблица 2.1. Different formats, used by Microsoft in the past 10 years

Формат	Встроен в PE-образ	Расширение внешнего файла	Поддерживается WinDbg/NTSD
COFF	Да	.dbg	Да
CV	Да	.dbg	Да
PDB	Нет	.pdb	Да
Символы ядра Windows 9x/Me	Нет	.sym	Нет

Например, early versions of Windows NT использовали symbol files с extension `.dbg`.

Windows 2000 и более ранние versions of Windows NT keep their symbols in files с extensions `.pdb` и `.dbg`.

Windows XP и Windows Server 2003 используют исключительно `.pdb` files.

Symbols for Windows drivers могут follow either model, depending on compiler and linker version used to build them.

Двоичные файлы, сгенерированные инструментами, не соответствующими ни одному из распознаваемых форматов, не могут корректно отлаживаться с помощью отладчиков Windows.

Файлы символов содержат разнообразные данные, не нужные при выполнении двоичных файлов, но необходимые для процесса отладки.

Обычно symbol files содержат:

- Имена и адреса глобальных переменных
- Имена функций, их адреса и сигнатуры
- Данные Frame Pointer Optimization (FPO), помогающие отладчику
- Имена и расположения локальных переменных
- Пути к исходным файлам и номера строк, связанные с каждым символом
- Информация о типах для переменных, структур и так далее

Двоичные файлы становятся меньше благодаря хранению этих файлов символов отдельно.

Однако это означает, что при debugging нужно убедиться, что debugger can access symbol files associated with the target you are debugging.

И interactive debugging, и debugging crash dump files benefit from using correct symbols.

Нужно obtain proper symbols for the code you want to debug and load these symbols into the debugger.

Errors, encountered in binary images running on the customer's site, можно investigate without having all this information available on the customer's site.

Чтобы discourage reverse engineering, generated symbol files, also known as private symbols, usually kept private by the company owning the intellectual property for those binary images.

Однако customer always can use another symbol file, containing restricted set of symbols, called public symbols.

Публичных файлов символов достаточно для пользователей модуля, при этом они не раскрывают внутренние структуры, параметры функций или локальные переменные.

Например, public symbols доступны for download as a whole package для каждой version of the operating system, shipped by Microsoft.

Кроме того, каждый driver, shipped with any version of Windows, has public symbols available in the same download package.

Двоичный файл содержит только указатель на файлы символов, а отладчик загружает публичный или приватный символ в зависимости от доступности.

Если вы хотите see debug information stored in the binary file, utility link.exe, available from within WDK build windows, is the best tool for the task, как показано в листинге 2.8.

Информация о файле символов хранится в разделе debug directory каждого исполняемого модуля.

Листинг 2.8. Использование utility link.exe для поиска debug information, stored in the binary file

```
C:\>link -dump -headers C:\WINDOWS\system32\ntdll.dll
Microsoft (R) COFF/PE Dumper Version 7.10.2179
Copyright (C) Microsoft Corporation. All rights reserved.
Dump of file C:\WINDOWS\system32\ntdll.dll
... other information about the module
Debug Directories
Time      Type Size RVA      Pointer
-----
41107F17 cv   22   0007B6DC 7AAD8 Format: RSDS, {36515FB5-D043-45E4-
91F6-72FA2E2878C0}, 2, ntdll.pdb
41107F17 ( A) 4    0007B6D8 7AAD8 BB030D70
```

Public symbol download packages represent convenient way to get access to all symbol files, если system does not change over time.

Поскольку very common to see one binary file being updated several times between service pack releases, dynamic method of downloading symbols just in time much more useful. Эта functionality provided by a symbol server, described in more detail in the "Symbol Server" section. Сервер символов находит и по требованию загружает файл символов, связанный с отлаживаемым модулем, используя информацию debug directory как ключ к файлу символов.

Путь символов

Как debugger узнает, где взять symbols, required for a specific assembly?

Debugger использует две pieces of information: location of the symbols path, represented as a collection of paths, combined with information stored in module headers, used to validate symbol files.

Каждый path может быть local folder, UNC share или symbol server path, как описано в section "Symbol Server".

In simple form symbol path - это succession of folders, separated by semicolon (;) character, entered in the interactive debugger using following command:

```
0:000>.sympath C:\SymPath1;\mysymbols\symbols
```

Имя файла символов извлекается из CV-записи заголовка образа или создается из имени двоичного файла, когда заголовок недоступен.

Отладчик использует эвристический алгоритм для поиска файла символов по пути символов, проверяя каждый найденный файл символов по информации модуля.

Если matching symbol file не найден, debugger defaults to using symbols exported by the module, как в листинге 2.9.

Commands, used in the listing, будут shortly explained в section "Reloading the Symbols".

Листинг 2.9. Эвристика, используемая отладчиком для поиска файла символов

```
0:000> !sym noisy
noisy mode - symbol prompts off
0:000> !reload -f kernel32.dll
DBGHELP: c:\SymPath\kernel32.pdb - file not found
DBGHELP: c:\SymPath\symbols\dll\kernel32.pdb - file not found
DBGHELP: c:\SymPath\dll\kernel32.pdb - file not found
DBGHELP: C:\WINDOWS\system32\kernel32.pdb - file not found
DBGHELP: kernel32.pdb - file not found
*** ERROR: Symbol file could not be found. Defaulted to export symbols for C:
\WINDOWS\system32\kernel32.dll -
DBGHELP: kernel32 - export symbols
```

Сервер символов

Правильная настройка символов для отладки может быть сложной задачей, особенно когда конкретный модуль выпускался более одного раза.

Она требует знания имен и выпусков всех модулей, загруженных в цель отладчика.

Отладчик должен быть способен найти каждый файл символов, соответствующий выпуску продукта и пакету обновления.

Это может привести к чрезвычайно длинному symbol path, состоящему из длинного списка directories.

Чтобы simplify difficulties associated with coordinating symbol files, можно использовать symbol server. Symbol server позволяет debuggers автоматически получать correct symbol files без указания product names, releases или build numbers.

Symbol server активируется включением определенной text string в symbol path.

Каждый раз, когда debugger needs to load symbols, он вызывает symbol server, чтобы locate appropriate files.

Symbol server locates files в symbol store, который представляет собой collection of symbol files, indexed по combination of parameters, таких как symbol filename, time stamp и image size.

Symbol path к symbol server использует special syntax, который может содержать multiple paths к downstream stores, за которыми следует real address symbol server.

Basic syntax для symbol path:

```
0:000>SRV*[cachei]*toppath
```

String SRV указывает, что path является symbol server path, где toppath представляет address symbol server.

Symbol path может содержать до 10 downstream stores, local или UNC, используемых для cache symbols.

Chain cache stores - удобный метод реализации common caches для remote location с ограниченной bandwidth.

Address symbol server может быть UNC path к symbol server, реализованному на file system share, или URL к symbol server.

Этот path можно combined with другими symbol paths, используя semicolon (;) как separator, чтобы создать symbol search path с доступом ко всем symbols, required in that specific debugging session.

Within a symbol server path, symbol server ищет symbol file в first downstream symbol store и, если находит, loads it из этого location.

При failure он recursively searches каждый symbol store for the file, пока file не будет найден.

Затем debugger caches этот symbol file в previous downstream stores, которые writable.

Поскольку software выполняется на Microsoft Windows operating systems, debugger always should use Microsoft public symbol store, available at <http://msdl.microsoft.com/download/symbols>, как один entry in symbol path.

Также настоятельно рекомендуется, чтобы companies имели strong private symbol management policy.

Глава 4 описывает process создания и поддержания такого symbol store.

В этом случае company-wide private symbol store path будет first entry in symbol path, а за ним, most likely, будет следовать address Microsoft public symbol store.

First downstream store in symbol path должен быть local cache entry, который обычно faster than any other remote store.

Листинг 2.10 показывает examples of symbol paths, указывающих на Microsoft public symbol store, на company symbol store, combined или not combined with downstream store.

Examples используют folder `c:\symbols` как downstream store для более быстрого symbol access.

Обратите внимание, что symbol server paths можно combine with обычными UNC locations, как описано в previous section.

Листинг 2.10. Examples of symbol server paths

```
0:000>.srvpath srv*c:\Symbols*http://msdl.microsoft.com/download/symbols
0:000>.srvpath srv*http://msdl.microsoft.com/download/symbols
0:000>.srvpath srv*c:\symbols*\myserver\mysymbols*http://msdl.microsoft.com/
download/symbols
```

Кэш символов

В previous section вы видели, как debugger uses downstream folders как intermediate caches for symbol files, provided by symbol server.

Caching improves response time всех operations, requiring new symbol file download.

Однако если symbol files are stored in remote share, но not organized as a symbol server, этот caching mechanism использовать нельзя.

Более поздние версии отладчиков устраняют этот недостаток с помощью встроенной поддержки кэширования файлов символов.

Возможность кэширования включается указанием папки кэша в пути символов с помощью специального формата.

Отладчик распознает директиву cache* и рассматривает папку после символа звездочки (*) как расположение кэша.

Все символы, полученные отладчиком из любого пути после директивы кэша, будут кэшироваться независимо от их источника.

Листинг 2.11 uses cache directive to indicate local cache for symbols downloaded from a symbol server or from a symbol share.

Листинг 2.11. Examples of symbol paths with local cache

```
0:000>.srvpath cache*c:\symbols;srv*http://msdl.microsoft.com/download/symbols
0:000>.srvpath cache*c:\symbols;\\farawayserver\symbols;
```

Обслуживание кэша символов

Local cache, created by the mechanism described in the previous sections, не имеет expiration policy и может grow unbound, если target binaries change often.

Хорошая идея - periodically purge cache folder.

Debugging Tools for Windows provides cleanup tool agestore.exe, который может delete all files not accessed after a specific date.

Встроенной справки достаточно, чтобы научиться эффективно им пользоваться.

Листинг 2.12 uses agestore.exe command in list mode to evaluate how many files were not recently used.

Рекомендуется always use this option before actual delete operation, чтобы confirm which files need to be deleted.

Листинг 2.12. Вывод всех файлов символов, не использовавшихся с указанной даты

```
C:\> agestore.exe -date=01-01-2007 -l -s c:\symbols
processing all files last accessed before 01-01-2007 12:00 AM
12-26-2006 9:43 PM c:\symbols\02sample.pdb\5226684770524C77B6D9658E94FEA2F21\
02sample.pdb
12-26-2006 9:43 PM c:\symbols\kernel32.pdb\04B9D5F57B154AA2BDBAB7946947DC4F2\
kernel32.pdb
12-26-2006 9:43 PM
c:\symbols\msvcrt.pdb\8A24BF4B1A05412FB0312AD4CB7867042\msvcrt.pdb
12-26-2006 9:43 PM c:\symbols\ntdll.pdb\C0A498F0036E4D4FB5CBF69005B0F9242\ntdll.pdb
6098944 bytes would be deleted
```

Задание пути символов

При запуске отладчик читает переменные окружения _NT_ALT_SYMBOL_PATH и _NT_SYMBOL_PATH и использует их вместе как путь символов именно в таком порядке.

Если окружение нельзя задать, другой способ задать путь символов с начала сеанса отладки - запустить отладчик с параметром -y.

WinDbg объединяет путь, полученный из workspace, с путем, предоставленным через альтернативные механизмы.

Два sections, shown in Listing 2.13, have the same meaning.

Листинг 2.13. Два способа задания пути символов при запуске отладчика

```
Using the environment
c:\>set _NT_SYMBOL_PATH=c:\symbols
c:\>windbg <image.exe>

Using the command-line parameter
C:\>windbg -y c:\symbols <image.exe>
```

Независимо от метода, использованного для указания пути символов при запуске отладчика, его можно перезаписать в интерактивном режиме.

После перехода отладчика в интерактивный режим существует несколько способов управления путями символов.

Можно set symbol path using .sympath command in one of the following forms.

Важно заметить, что change does not affect symbol files already loaded from previous symbol path.

- 0:000>.sympath <new path> меняет текущий путь символов на новый путь, указанный как аргумент команды; отладчик использует его для загрузки файлов символов. Она перезаписывает существующий путь символов без перезагрузки файлов символов и без удаления уже загруженных символов.
- 0:000>.sympath+ <new path> добавляет указанный новый путь к существующему пути символов.
- 0:000>.sympath отображает и разрешает текущий путь символов. Недоступные пути символов перечисляются в конце вывода; в настоящее время записи сервера символов не разрешаются.

Если смотреть на previous examples using Microsoft symbol store, может возникнуть вопрос, нужно ли memorize such a long URL.

Можно keep it in a file with well-known strings to paste in the debugger console when you need it, но better way - использовать command .symfix.

- 0:000>.symfix <downstream folder> меняет путь символов на публичное хранилище символов Microsoft. Команда принимает downstream-папку, кэшируя все символы, загруженные из публичного хранилища символов Microsoft. В результате этой команды путь символов устанавливается в SRV*downstream folder*http://msdl.microsoft.com/download/symbols.
- 0:000>.symfix+ <downstream folder> добавляет публичное хранилище символов Microsoft к существующему пути символов. Команда принимает downstream-папку, кэшируя все символы, загруженные из публичного хранилища символов Microsoft.

Листинг 2.14 shows typical usage of .sympath and .symfix commands.

Листинг 2.14. Использование команд .sympath и .symfix

```
0:000> .sympath srv*c:\symstore.pri
Symbol search path is: srv*c:\symstore.pri
0:000> .sympath+ c:\PathNotAvailable
Symbol search path is: srv*c:\symstore.pri;c:\PathNotAvailable
WARNING: Inaccessible path: 'c:\PathNotAvailable'
0:000> .sympath
Symbol search path is: srv*c:\symstore.pri;c:\PathNotAvailable
WARNING: Inaccessible path: 'c:\PathNotAvailable'
0:000> .symfix c:\symbols
```

```

0:000> .sympath
Symbol search path is: SRV*c:\symbols*http://msdl.microsoft.com/download/
symbols
0:000> .sympath c:\
Symbol search path is: c:\
0:000> .symfix+ c:\symbols
0:000> .sympath
Symbol search path is: c:\;SRV*c:\symbols*http://msdl.microsoft.com/download/symbols

```

Даже если все показанные примеры используются в отладчике пользовательского режима, те же параметры доступны и для отладчика режима ядра.

Важно отметить, что all paths are relative to where debugger engine runs; this has direct impact in scenarios where user mode debugger is redirected through kernel debugger.

Проверка загруженных модулей и файлов символов

Отладчик загружает символы по мере необходимости при первой попытке разрешить символ внутри указанного модуля.

Если операция загрузки завершается неудачно, отладчик не пытается повторно загрузить модуль. Состояние загрузки символов можно просмотреть с помощью команды `lm` (list modules), одной из самых полезных команд для исследования информации о загруженных модулях.

```
0:000>lm [option] [-a Address] [-m Pattern] [-M Pattern]
```

Общая форма команды имеет множество параметров, но чаще используются лишь некоторые из них.

Этот section включает several examples using `02sample.exe` binary, the book's symbols store, followed by Microsoft public symbols store.

Для ясности путь символов задается с помощью переменной окружения следующим образом:

```

c:\>set _NT_SYMBOL_PATH=CACHE*C:\Symbols;
SRV*http://www.advancedwindowsdebugging.com/symbols/symstore.pri;
SRV*http://msdl.microsoft.com/download/symbols
C:\>windbg C:\AWDBIN\WinXP.x86.chk\02sample.exe

```

Переменная `_NT_SYMBOL_PATH` учитывается большинством инструментов, используемых для отладки программных приложений на платформе Windows.

Тот же путь символов можно задать в любом другом инструменте методами, специфичными для этого инструмента.

Пути символов, показанного в предыдущем листинге, достаточно для загрузки и кэширования всех символов, используемых в примерах книги.

`lm` возвращает информацию обо всех модулях, загруженных в процесс, вместе с диапазоном адресов, используемым модулем, результатами загрузки символов и путем к файлу символов относительно пути символов.

```

0:000> lm
start      end          module name
00400000 00404000    02sample (private pdb symbols)
c:\symbols\02sample.pdb\DE4335BC88FD4EA1A1714350C33B84281\02sample.pdb
76080000 760e5000    msvcp60 (deferred)
77c10000 77c68000    msvcrt (deferred)
7c800000 7c8f4000    kernel32 (deferred)
7c900000 7c9b0000    ntdll (pdb symbols)
c:\symbols\ntdll.pdb\36515FB5D04345E4
91F672FA2E2878C02\ntdll.pdb

```

Команда принимает разные параметры, фильтрующие список обрабатываемых модулей.

Например, `lm 1` обрабатывает только загруженные файлы символов, тогда как `lm *` обрабатывает модули, для которых файл символов не найден.

Команда `lm` также принимает строковый шаблон, используемый для фильтрации модулей, обрабатываемых командами.

Фильтрация по имени модуля задается параметром `m`, а фильтрация по полному пути включается параметром `M`.

Параметры можно комбинировать для получения желаемого поведения, как показано в листинге 2.15.

Листинг 2.15 показывает подробную информацию о модулях, имена которых соответствуют строке `kernel*`.

Обратите внимание, что строка шаблона не включает расширение.

Если расширение введено как часть шаблона, команда не находит указанный модуль.

Листинг 2.15. Отображение информации о загруженном модуле

```
0:000> lm v m kernel*
start      end          module name
7c800000 7c8f4000  kernel32  (export symbols) C:\WINDOWS\system32\kernel32.dll
Loaded symbol image file: C:\WINDOWS\system32\kernel32.dll
Image path: C:\WINDOWS\system32\kernel32.dll
Image name: kernel32.dll
Timestamp:      Wed Aug 04 00:56:36 2004 (411096B4)
Checksum:       000FF848
ImageSize:      000F4000
File version:   5.1.2600.2180
Product Version: 5.1.2600.2180
File flags:     0 (Mask 3F)
File OS:        40004 NT Win32
File type:      2.0 Dll
File date:      00000000.00000000
Translations:   0409.04b0
CompanyName:    Microsoft Corporation
ProductName:     Microsoft<< Windows<< Operating System
InternalName:    kernel32
OriginalFilename: kernel32
ProductVersion:  5.1.2600.2180
FileVersion:     5.1.2600.2180 (xpsp_sp2_rtm.040803-2158)
FileDescription: Windows NT BASE API Client DLL
LegalCopyright:  © Microsoft Corporation. All rights reserved.
```

Несмотря на объем информации, возвращаемой командой `lm`, дополнительную информацию, скрытую в заголовке модуля, можно исследовать с помощью команды расширения `!lmi`.

Эта команда расширения выводит всю информацию `debug directory`, как показано в листинге 2.16.

Листинг 2.16. Отображение заголовков модуля

```
0:000> * !lmi command accepts the module address or module's name
0:000> !lmi ntdll.dll
Loaded Module Info: [ntdll.dll]
Module: ntdll
Base Address: 7c900000
Image Name: ntdll.dll
Machine Type: 332 (I386)
Time Stamp: 411096b4 Wed Aug 04 00:56:36 2004
Size: b0000
Checksum: af2f7
Characteristics: 210e perf
Debug Data Dirs: Type Size VA Pointer
CODEVIEW 22, 7b6dc, 7aad8 RSDS - GUID: (0x36515fb5, 0xd043, 0x
45e4, 0x91, 0xf6, 0x72, 0xfa, 0xe, 0x28, 0x78, 0xc0)
Age: 2, Pdb: ntdll.pdb
CLSID 4, 7b6d8, 7aad8 [Data not mapped]
Image Type: FILE - Image read successfully from debugger.
C:\WINDOWS\system32\ntdll.dll
Symbol Type: PDB - Symbols loaded successfully from symbol server.
ntdll.pdb\36515FB5D04345E491F672FA2E2878C02\ntdll.pdb
Load Report: public symbols , not source indexed
ntdll.pdb\36515FB5D04345E491F672FA2E2878C02\ntdll.pdb
```

В некоторых случаях даже информации, возвращаемой `!lmi`, недостаточно.

Заголовки модуля можно дополнительно исследовать с помощью другого расширения отладчика, `!dh <module address>`, или изучить вне отладчика выбранными вами инструментами.

ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ О МОДУЛЕ. Некоторые ситуации отладки требуют дополнительной информации о двоичных образах.

Например, при отладке переполнения стека легко получить размер стека, используемый потоком. Однако это значение нужно сравнить со стандартным размером резерва стека.

Этот размер, хранящийся в заголовках образа процесса, полезен для понимания того, использует ли поток больше пространства стека, чем предполагал разработчик.

Следующая команда отображает заголовки модуля, подобно инструменту `WDK link.exe`, описанному в листинге 2.8.

```
0:000>!dh <module start address>|<module name> -f
```

Перезагрузка символов

Поскольку использовать недопустимый файл символов хуже, чем не использовать никакой, перезагрузка правильных файлов символов важна.

Базовая команда для исправления символов - `.reload` в сочетании с множеством доступных параметров.

Несмотря на свое имя, команда `.reload` по умолчанию не загружает новые файлы символов.

Команда отбрасывает ранее загруженные файлы символов и полагается на отладчик, который перезагрузит файлы при первой попытке их использовать.

Некоторые распространенные формы команды `.reload`:

- `0:000>.reload` отбрасывает информацию о символах для всех загруженных модулей, возвращая отладчик в исходное состояние.
Любая попытка разрешить символ перезагружает файл символов с диска.
- `0:000>.reload <module>` отбрасывает информацию об указанном модуле.
Любая попытка разрешить символ перезагрузит файл символов с диска.
- `0:000>.reload /f <module>` заставляет отладчик немедленно разрешить и загрузить файл символов, связанный с модулем.
- `0:000>.reload nt` - параметр отладчика режима ядра.
Он перезагружает файл символов, соответствующий текущему ядру Windows NT, что существенно для большинства операций в отладчике режима ядра.
`Command does not work in user mode.`
Команда не работает в пользовательском режиме.
- `0:000>.reload /user` - параметр отладчика режима ядра.
Он перезагружает все файлы символов пользовательского режима для активного процесса.
- `0:000>.reload <module>=start, length` использует явный диапазон адресов модуля.
Все команды, показанные выше, используют информацию, сохраненную в заголовке модуля и в блоке управления процессом (PCB), чтобы получить адресное пространство модуля в памяти и ссылку на файл символов.
Если какая-то информация отсутствует, как бывает при нехватке памяти в системе, можно найти начальный адрес в других источниках, например в журнале сборки или на идентичных работающих системах, и принудительно загрузить символы, указав начальный адрес:

```
0:000>.reload rpcrt4.dll=78000000,86000
```

Это также полезно, если у вас есть адрес модуля, который уже выгружен, и нужно восстановить стек для пути кода в отсутствующем модуле.

- `0:000>.sym noisy` включает подробное журналирование операций с символами.

Когда команда `.reload` завершается неудачно, нужно включить подробный журнал для `.reload`, управляемый командой `.sym`.
`.sym noisy` включает подробное журналирование, после чего любая команда `.reload` показывает все попытки загрузки и результаты их выполнения.

Проверка символов

Без правильных символов хороший разработчик может часами читать исходный код, надеясь понять, почему отладчик показывает бессмысленный стек или почему некоторые переменные имеют совершенно нереалистичные значения.

Невозможно переоценить важность проверки корректности символов.

Но как убедиться, что символы правильные?

Первый вариант - использовать команду `!ml`, чтобы просмотреть возможные предупреждения о файлах символов.

Кроме того, отладчик предоставляет команду расширения, которая может проверить допустимость файла символов по файлу образа.

Эта команда расширения принимает либо адрес внутри загруженного образа, либо имя образа.

Расширение проверяет файл символов, указанный как параметр, или файл символов, уже загруженный отладчиком.

Следующий листинг использует команду расширения для проверки корректности загруженных символов для образа, загруженного по указанному адресу.

```
0:000> !chkSYM 01001b90
02sample.exe
Timestamp: 461001C1
SizeOfImage: 5000
pdb: 02sample.pdb
pdb sig: 52266847-7052-4C77-B6D9-658E94FEA2F2
age: 1
Loaded pdb is +.sympath
SRV\02sample.pdb\5226684770524C77B6D9658E94FEA2F21\02sample.pdb
02sample.pdb
pdb sig: 52266847-7052-4C77-B6D9-658E94FEA2F2
age: 1
MATCH: 02sample.pdb and 02sample.exe
```

Использование символов

Почти каждая команда использует информацию символов напрямую или косвенно, но несколько команд специально предназначены для исследования символов.

Базовая команда для исследования символов - `x`, что означает "examine symbols".

Команда имеет следующий общий синтаксис:

```
0:000>x [options] module!symbols
```

И часть модуля, и часть символов могут содержать подстановочные знаки.

Поддержка подстановочных знаков - мощный инструмент при отладке незнакомого кода, потому что она позволяет угадывать имена функций или глобальных переменных задолго до чтения кода.

Ниже перечислены несколько распространенных вариантов использования команды `x`:

- `0:000>x *!*some*` ищет имя символа, содержащее строку `some` в середине каждого символа внутри каждого файла символов для цели отладчика.
Если символ является экспортируемой функцией, результат содержит как модули, реализующие ее, так и модули, импортирующие ее, с префиксом строки `_imp`:

```
0:000> x *!*NtOpenThreadToken*
77e41348 kernel32!_imp__NtOpenThreadToken = <no type information>
7c821808 ntdll!NtOpenThreadTokenEx = <no type information>
7c8217f8 ntdll!NtOpenThreadToken = <no type information>
```

- 0:000>x module!prefix* учитывает соглашения об именовании в исследовании.
Например, если все глобальные переменные имеют префикс g_, команда x module!g_* перечисляет все глобальные переменные вместе с их текущими значениями:

```
0:000> x kernel32!g_*
77ecdb74 kernel32!g_hModXPSP2Res = <no type information>
...
77e77c80 kernel32!g_DllEntries = <no type information>
```

- 0:000>x /v /t module!symbol может помочь лучше понять содержимое двоичного файла.
Она показывает тип символа и размер в байтах, занимаемый этим объектом или функцией, в порядке возрастания размера.

```
0:000> x /v /t 02sample!*
prv global 00402004 4 02sample!__security_cookie_complement = 0xffff4134
...
prv global 004010a0 4 02sample!__xc_a = <function> *[1]
...
prv func 00401713 11 02sample!__SEH_epilog (void)
prv func 004013fa cc 02sample!wmain (unsigned long, wchar_t **)
...
```

Команды исследования символов не могут работать в полную силу, когда отладчик использует публичный файл символов для образа.

Еще одна полезная команда, хорошо использующая символы, - ln, что означает "list near".

Команда ln показывает символ, связанный с конкретным адресом, если он доступен.

Когда ни один символ точно не соответствует адресу, отладчик возвращает символ, сгенерированный указательной арифметикой на ближайшем к этому адресу символе.

```
0:000> ln 01001b90
(01001b90) 02sample!wmain | (01001bc0) 02sample!AppInfo::AppInfo
Exact matches:
02sample!wmain (unsigned long, wchar_t **)
0:000> ln 01001b90+1
(01001b90) 02sample!wmain+0x1 | (01001bc0) 02sample!AppInfo::AppInfo
```

Точные совпадения очень ценны, хотя к вычисленному совпадению нужно относиться осторожно, особенно когда адрес является частью файла образа, входящего в операционную систему.

Перед выпуском Microsoft использует специальные техники оптимизации исполняемых образов для производительности.

После оптимизации одна функция может быть разбита на несколько секций, расположенных по разным адресам, что отрицательно влияет на указательную арифметику, выполняемую отладчиком.

Оптимизированный для производительности образ можно определить по наличию атрибута perf в характеристиках модуля, как показано в листинге 2.16.

Эта команда очень мощна, когда вы исследуете произвольный фрагмент данных и не знаете, что он представляет.

Если исследуемый адрес является частью стека, скорее всего, вы найдете последовательности из стека вызовов, и ln поможет их идентифицировать.

Если вы исследуете блок кучи, вполне возможно найти фрагменты исходных объектов, что может помочь определить использование блока.

Использование исходных файлов

При отладке программного приложения исходные файлы полезны в двух основных ситуациях: при посстрочном выполнении кода для изучения или проверки его поведения и при создании возможных сценариев, приводящих к сбою приложения.

В обоих случаях требуется доступ к приватным файлам символов, потому что они содержат информацию, связывающую каждый символ с именем исходного файла и строкой, а также расположение всех исходных файлов, использованных для генерации двоичного файла.

Отладчик использует информацию о расположении исходников, сохраненную в файле символов, и пытается найти файлы в разных местах, указанных путем исходников.

WinDbg сохраняет последнее расположение пути исходников в workspace.

Расположение можно перезаписать с помощью ключа командной строки `srcpath`, например `windbg -srcpath <SourcePath>`.

В интерактивном режиме путь исходников можно изменить с помощью команды `.srcpath` или пункта меню `Source File Path` в меню `File`.

При отладке образов на той же системе, где они были скомпилированы, отладчику не нужен никакой путь исходников.

Необработанные файлы символов содержат полные пути к исходным файлам, которые отладчик открывает напрямую.

Путь исходников интерпретируется отладчиками как список путей файлов, разделенных точкой с запятой (;).

Затем отладчик находит исходный файл, расположенный в папке пути исходников, который лучше всего соответствует пути файла, изначально использованному для сборки двоичного файла.

Путь исходников вводится в командном окне отладчика с помощью команды с точкой (.):

```
0:000>.srcpath c:\;\mycompany\sources
Source search path is: c:\;\mycompany\sources
```

Поскольку процесс разрешения исходного файла относительно сложен и зависит от ряда параметров локальной системы, иногда отладчик не может найти или получить доступ к правильному исходному файлу для пути исходников, извлеченного из приватных файлов символов. Отладчик предоставляет подробный режим для процесса поиска правильных файлов исходного кода.

Этот режим управляется другой командой, `.srcnoisy <1|0>`.

Когда он включен, отладчик отображает все расположения, проверенные на наличие исходного файла, а также результат каждой операции.

```
0:000>.srcnoisy 1
Noisy source output: on
0:000>.srcpath e:\;c:\
Source search path is: e:\;c:\
DBGENG: Scan paths for partial path match:
DBGENG: prefix 'c:\awd\chapter2'
DBGENG: suffix 'sample.cpp'
DBGENG: match 'e:' against 'c:\awd\chapter2': 14 (match '')
DBGENG: match 'c:' against 'c:\awd\chapter2': 14 (match '')
DBGENG: Scan paths for partial path match:
DBGENG: prefix 'c:\awd'
DBGENG: suffix 'chapter2\sample.cpp'
DBGENG: match 'e:' against 'c:\awd': 5 (match '')
DBGENG: match 'c:' against 'c:\awd': 5 (match '')
DBGENG: Scan paths for partial path match:
DBGENG: prefix 'c:'
DBGENG: suffix 'awd\chapter2\sample.cpp'
DBGENG: match 'e:' against 'c:': 1 (match '')
DBGENG: match 'c:' against 'c:': -1 (match 'c:')
DBGENG: check 'c:\awd\chapter2\sample.cpp'
DBGENG: found file 'c:\awd\chapter2\sample.cpp'
```

Default source file matching не так strict, как symbol file matching, потому что source information - это просто fully qualified source filename.

Пока source file с тем же name, которое указано в symbol file, найден в source path, debugger loads it. Process работает reasonably well для applications, в которых source files не changed since last compilation.

Глава 4 объясняет, как решить эту problem с помощью source server, который works side by side with source control system, ensuring source correctness.

Debugger interprets source server information, stored in symbol files, когда string SRV* присутствует in source path.

Debugger extracts source file из source store, described in symbol file, и caches it on local system.

For convenience, debugger accepts command .srcfix, которая simply sets source path to SRV*, если exact syntax of source server path забыт.

Процесс загрузки исходного файла с source server показан в следующем листинге:

```
0:000> .srcnoisy 1
Noisy source output: on
0:000> .srcfix
Source search path is: SRV*
DBGENG: Scan srcsrv SRV* for:
DBGENG: '<token>!c:\awd\chapter2\sample.cpp'
DBGENG: found file 'c:\awd\chapter2\sample.cpp'
DBGENG: server path 'SRV*'
DBGENG: local 'http://www.advancedwindowsdebugging.com/sources/AWD/Chapter2/
sample.cpp/VERSION1/sample.cpp'
```

Когда source path является combination of local paths and source server path, debugger uses source server mechanism для всех files, indexed in source server, как described in symbol files.

Debugger uses standard path при matching all other files.

Даже если sources provided by multiple source stores, string SRV* required только один раз in source path.

Similar to symbol path, чтобы simplify process of composing source path, both .srcfix and .srcpath provide alternative syntax: .srcpath+ <srcpath> или .srcfix+, которые append to existing source server path.

Следующий listing показывает example of appending share location to existing source path.

```
0:000> .srcpath+ \\mysources\sources
Source search path is: srv*;\\mysources\sources
```

Exploratory Commands

Как вы уже видели, message, displayed by debugger, очень helpful для понимания, why and where debugger stopped.

Если мы connect to a remote debugger after event has been encountered, мы теряем precious information, которая могла быть previously displayed in the debugger console.

В этом section мы explore a few options, available when trying to understand state in which debugger target stopped and reason for the current stop.

Why Did the Debugger Stop?

Command .lastevent displays information about last debugger event, из-за которого current debugger stopped.

Глава 3 объясняет origin and importance возможных debugger events.

Листинг 2.17 shows sample output, generated by .lastevent command in two cases: after debugger stopped because of user-defined breakpoint and, во втором output, because of operation on inaccessible memory location.

Knowing why debugger stopped can sometimes complete investigation, как в случае initial process breakpoint или process exit breakpoint.

Листинг 2.17. .lastevent output

```
0:000> * after a breakpoint
0:000> .lastevent
Last event: 170c.1464: Hit breakpoint 2
0:000> * after an access violation exception
0:000> .lastevent
Last event: 170c.1464: Access violation - code c0000005 (first chance)
```

Какова целевая система?

Программа, которую вы отлаживаете, ведет себя по-разному в зависимости от операционной системы и установленных на ней обновлений.

Причина не обязательно в том, что program uses feature of one of those releases; operating system mechanism itself can change between releases.

В то же время отладчик и его расширения используют компоненты, реализованные в операционной системе, которые могут вести себя по-разному в разных выпусках, создавая ограничения для самого инструмента отладки.

Поэтому, кроме случая, когда вы отлаживаете компонент, не зависящий от служб операционной системы, вам, скорее всего, нужно знать версию операционной системы, версию отладчика, версию загруженного расширения и так далее.

Команда vertarget является подмножеством команды version, отображающим только версию операционной системы, на которой работает цель отладчика.

Команда version показывает дополнительную информацию об окружении отладчика и командной строке, использованной для запуска сеанса отладки, как показано в листинге 2.18.

Если система использует более одного процессора, первая строка также показывает число активных процессоров; иначе она показывает строку UP, что означает uni processor.

Листинг 2.18. version output from a user mode debugger

```
0:000> version
Windows XP Version 2600 (Service Pack 2) UP Free x86 compatible
Product: WinNt, suite: SingleUserTS
kernel32.dll version: 5.1.2600.3119 (xpsp_sp2_gdr.070416-1301)
Debug session time: Sun Jul 8 14:31:35.259 2007 (GMT-7)
System Uptime: 0 days 0:10:39.826
Process Uptime: 0 days 0:00:04.356
Kernel time: 0 days 0:00:00.030
User time: 0 days 0:00:00.020
Live user mode: <Local>
command line: '"c:\Program Files\Debugging Tools for Windows"\ntsd notepad'
Debugger Process 0x738
dbgeng: image 6.6.0007.5, built Sat Jul 08 13:12:40 2006
[path: c:\Program Files\Debugging Tools for Windows\dbgeng.dll]
dbghelp: image 6.6.0007.5, built Sat Jul 08 13:11:32 2006
[path: c:\Program Files\Debugging Tools for Windows\dbghelp.dll]
DIA version: 60516
Extension DLL search Path:
c:\Program Files\Debugging Tools for Windows\winext;c:\Program Files\Debugging
Tools for Windows\winext\arcade;c:\Program Files\Debugging Tools for
Windows\WINXP;c:\Program Files\Debugging Tools for Windows\pri;c:\Program Files\Debug-
ging Tools for Windows;c:\Program Files\Debugging Tools for
Windows\winext\arcade;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS
\System32\Wbem
```

```

Extension DLL chain:
dbghelp: image 6.6.0007.5, API 6.0.6, built Sat Jul 08 13:11:32 2006
[path: c:\Program Files\Debugging Tools for Windows\dbghelp.dll]
ext: image 6.6.0007.5, API 1.0.0, built Sat Jul 08 13:10:52 2006
[path: c:\Program Files\Debugging Tools for Windows\winext\ext.dll]
exts: image 6.6.0007.5, API 1.0.0, built Sat Jul 08 13:10:48 2006
[path: c:\Program Files\Debugging Tools for Windows\WINXP\exts.dll]
uext: image 6.6.0007.5, API 1.0.0, built Sat Jul 08 13:11:02 2006
[path: c:\Program Files\Debugging Tools for Windows\winext\uext.dll]
ntsdexts: image 6.0.5457.0, API 1.0.0, built Sat Jul 08 13:29:38 2006
[path: c:\Program Files\Debugging Tools for Windows\WINXP\ntsdexts.dll]

```

What Are the Current Register Values?

После того как мы знаем, почему отладчик остановился, в какой операционной системе он работает и какие расширения доступны для наших исследований, пора найти объяснение текущей остановки.

Процесс поиска причины остановки можно сравнить с криминалистической работой по сбору и опросу каждого фрагмента доказательств, который мы можем получить от отладчика.

Мы исследуем все неизвестные элементы и проверяем любое предположение, сделанное нами при расследовании сбоя.

Первый шаг - проверить корректность символов, как описано в разделе о символах.

Если символы некорректны, мы можем легко исправить их, как описано ранее в разделе "Reloading the Symbols".

Команда `r`, что означает register, предоставляет доступ к регистрам процессора.

В простейшей форме она отображает все значения регистров согласно маске регистров, активной в отладчике.

Команда `r` также может загрузить в регистр введенное пользователем значение.

Эта возможность чрезвычайно полезна, когда вы используете отладчик для имитации разных сбоев в выполнении кода, чтобы вызвать разные пути кода.

Например, после вызова выделения памяти функцией `malloc` адрес выделенного блока возвращается из функции в регистре `eax`.

Если заменить это значение нулем, приложение можно проверить на условия нехватки памяти.

Команду отображения можно ограничить одним регистром или даже одним флагом из регистра `eFlags`.

WinDbg предоставляет окно регистров, которое обновляется текущим контекстом каждый раз, когда отладчик останавливается.

В листинге 2.19 команда `r` используется для чтения и записи значений регистров.

Листинг 2.19. Значения регистров с использованием маски регистров по умолчанию

```

0:000> r
eax=00000000 ebx=00000000 ecx=00000000 edx=00000000 esi=7d61cbcf edi=00000000
eip=7d61cbe1 esp=0014fed4 ebp=0014ff0c iopl=0 nv up ei pl nz na po nc
cs=0023  ss=002b  ds=002b  es=002b  fs=0053  gs=002b  efl=00000202
ntdll!NtTerminateProcess+0x12:
7d61cbe1 c20800 ret 8
0:000> * Displaying eax register
0:000> reax
eax=00000000
0:000> * Displaying the overflow flag
0:000> r of
of=0
0:000> * Changing eax register
0:000> reax=1

```

Маска регистров - это битовая маска, управляющая тем, какие регистры отображаются командой `r`.

Команду `rm` можно использовать для отображения текущей маски регистров или для ее изменения согласно потребностям отладки.

В листинге 2.20 показано несколько полезных примеров команды `rm`.

В общем случае для стандартного приложения нас интересуют только целочисленные регистры.

Если приложение активно использует floating point, мы установим маску так, чтобы показывать и эти значения.

При отладке программ, активно использующих Streaming SIMD Extensions, мы можем включить регистры MMX или SSE XMM в вывод с помощью маски регистров.

Листинг 2.20. Изменение маски регистров по умолчанию

```
0:000> * What is the current mask?
0:000> rm
Register output mask is 9:
1 - Integer state (32-bit)
8 - Segment registers
```

Листинг 2.20. Изменение маски регистров по умолчанию (продолжение)

```
0:000 > * What is the meaning of all register mask bits?
0:000 > rm ?
1 - Integer state (32-bit) or
2 - Integer state (64-bit), 64-bit takes precedence
4 - Floating-point state
8 - Segment registers
10 - MMX registers
20 - Debug registers and, in kernel, CR4
40 - SSE XMM registers
0:000 > * Setting the mask to zero (nothing is displayed)
0:000 > rm 0
0:000 > r
ntdll!NtTerminateProcess+0x12:
7d61cbe1 c20800 ret 8
```

Первый вопрос, который мы можем задать, - это значение регистра program counter (также известного как регистр instruction pointer).

Мы также можем спросить, как процессор попал в это место.

Имя регистра instruction pointer зависит от архитектуры процессора, из-за чего случайным пользователям отладчика трудно помнить его имя на всех платформах.

Чтобы преодолеть проблему именования, команда отладчика ввела различные псевдорегистры, специализируемые отладчиком под аппаратную архитектуру.

Например, имя псевдорегистра `$ip` представляет имя регистра instruction pointer в текущей архитектуре цели отладчика.

Псевдорегистры - это символические имена в форме `$name`, распознаваемые отладчиком как переменные, хранящие значения в текущем сеансе отладки.

Отладчик управляет несколькими автоматическими псевдорегистрами, представляющими значения, значимые в текущем сеансе отладки.

Например, псевдорегистр `$ip` совпадает с регистром `eip` процессоров x86 или регистром `rip` процессоров x64; псевдорегистр `$tpid` - это текущий идентификатор процесса (PID).

Отладчик предоставляет еще 20 псевдорегистров общего назначения, именуемых `$t0-$t19`, в текущем сеансе отладки.

Как и имена стандартных регистров, имена псевдорегистров должны экранироваться в выражениях с помощью символов амперсанда (`@`).

Подробный список с описанием каждого псевдорегистра можно найти в отладчике (раздел справки Pseudo-Registers), вместе с их доступностью в различных сценариях отладки.

В оставшейся части этой книги мы будем использовать следующие псевдорегистры настолько часто, насколько возможно:

- `$ip`: регистр instruction pointer; знак точки (.) также вычисляется как текущий instruction pointer. В зависимости от архитектуры процессора `$ip` вычисляется следующим образом:

```
$ip = eip on x86 architecture
$ip = rip on x64 architecture
$ip = iip on Itanium architecture
```

- `$ra`: адрес возврата из текущей функции.
- `$retreg`: основной регистр значения; сразу после возврата из вызова функции он содержит результат функции. В зависимости от архитектуры процессора `$retreg` вычисляется следующим образом:

```
$retreg = eax on x86 architecture
$retreg = rax on x64 architecture
$retreg = ret0 on Itanium architecture
```

- `$csp`: текущий указатель стека; в зависимости от архитектуры процессора `$csp` вычисляется следующим образом:

```
$csp = esp on x86 architecture
$csp = rsp on x64 architecture
$csp = bsp on Itanium architecture
```

- `$proc`: текущий процесс; он содержит адрес блока окружения процесса (PEB) в пользовательском режиме или адрес структуры EPROCESS текущего процесса в отладчике режима ядра.
- `$thread`: текущий поток; он содержит адрес блока окружения потока (TEB) в пользовательском режиме или адрес структуры ETHREAD текущего потока в отладчике режима ядра.
- `$tpid`: идентификатор текущего процесса (PID).
- `$tid`: идентификатор текущего потока (TID).

В листинге 2.21 показано типичное использование псевдорегистра в обычных командах.

Листинг 2.21. Псевдорегистр, используемый при остановке отладчика пользовательского режима (x86)

```
0:000> reip
eip=00401264
0:000> r$ip
$ip=00401264
0:000> ?.
Evaluate expression: 4199012 = 00401264
0:000> reax
eax=00401264
0:000> r$retreg
$retreg=00401264
0:000> r$proc
$proc=7ffde000
```

Листинг 2.21. Псевдорегистр, используемый при остановке отладчика пользовательского режима (x86) (продолжение)

```
0:000> r $peb
$peb=7ffde000
0:000> r$thread
$thread=7ffdd000
0:000> r$teb
$teb=7ffdd000
0:000> ~
. 0 Id: 16f8.16c8 Suspend: 1 Teb: 7ffdd000 Unfrozen
0:000> r$tid
$tid=000016c8
0:000> r$tpid
$tpid=000016f8
0:000> r$t1=0xbaadf00d
0:000> r$t1
$t1=baadf00d
```

What Code Is the Processor Executing Now?

Чтобы выяснить подробности о текущей остановке, мы начнем с анализа секции кода, содержащей сбой, начиная с текущего program counter.

Команда `u`, что означает "unassembly", используется для проверки машинного кода, сгенерированного из исходного кода.

Мы запускаем исполняемый файл `02sample.exe` под отладчиком и выбираем вариант 1, чтобы сгенерировать access violation.

В листинге 2.22 показано окно команд отладчика после использования команды `u` на остановке.

WinDbg предоставляет окно дизассемблирования, которое обновляется ассемблерным кодом по текущему адресу instruction pointer каждый раз, когда отладчик останавливается.

Листинг 2.22. Команда `u`, используемая в отладчике пользовательского режима (x86)

```
0:000> * Unassembly eight instruction as the address current $ip
0:000> u .
02sample!RaiseAV+0xd:
00401264 c6050000000000 mov byte ptr [00000000],0x0
0040126b 8be5 mov esp,ebp
0040126d 5d pop ebp
0040126e c3 ret
...
0:000> * Unassembly the entire function containing the current $ip
0:000> uf .
02sample!RaiseAV:
00401257 8bff mov edi,edi
00401259 55 push ebp
0040125a 8bec mov ebp,esp
0040125c 6a04 push 0x4
0040125e 58 pop eax
0040125f e8cc020000 call 02sample!_chkstk (00401530)
00401264 c6050000000000 mov byte ptr [00000000],0x0
0040126b 8be5 mov esp,ebp
0040126d 5d pop ebp
0040126e c3 ret
0:000> * Unassembly eight instructions prior to the current $ip
0:000> ub .
02sample!RaiseCPP+0x24:
00401255 cc int 3
00401256 cc int 3
02sample!RaiseAV:
00401257 8bff mov edi,edi
00401259 55 push ebp
0040125a 8bec mov ebp,esp
0040125c 6a04 push 0x4
0040125e 58 pop eax
0040125f e8cc020000 call 02sample!_chkstk (00401530)
0:000> * Unassembly two instructions after the current $ip
0:000> u . L2
02sample!RaiseAV+0xd:
00401264 c6050000000000 mov byte ptr [00000000],0x0
0040126b 8be5 mov esp,ebp
0:000> * Unassembly two instructions prior to the current $ip
0:000> ub . L2
02sample!RaiseAV+0x7:
0040125e 58 pop eax
0040125f e8cc020000 call 02sample!_chkstk (00401530)
0:000> * Unassembly ten instructions between $ip and $ip plus ten
0:000> u . .+a
02sample!RaiseAV+0xd:
00401264 c6050000000000 mov byte ptr [00000000],0x0
0040126b 8be5 mov esp,ebp
0040126d 5d pop ebp
```

What Is the Current Call Stack?

Знание текущих значений регистров, текущего исполняемого instruction pointer, а также нескольких инструкций вокруг него помогает нам понять текущий сбой.

Но мы все еще далеки от понимания динамических факторов, участвующих в этом сбое: какой код выполнялся до него, как регистры были изменены другими функциями и многое другое.

Процессор использует области памяти стека, управляемые стековым регистром, чтобы записывать адрес возврата, где выполнение должно продолжиться после завершения текущего вызова функции.

Поскольку каждый процессор управляет стеком по-своему, во всех примерах этой главы мы сосредоточимся на семействе процессоров x86, так как они распространены и легко доступны.

Аспекты, специфичные для 64-битных процессоров, обсуждаются в главе 12, "64-Bit Debugging", которую необходимо изучить перед погружением в 64-битную область.

Стек процессора x86 всегда растет вниз и адресуется регистром указателя стека, называемым esp.

Глава 5, "Memory Corruption Part I - Stacks", подробно объясняет различия между различными соглашениями о вызовах, используемыми в архитектуре процессора x86, и то, как они влияют на выполнение кода.

Эта глава сосредоточена на соглашении о вызове `__stdcall`, так как это соглашение по умолчанию, используемое Windows API.

Этот раздел (и оставшаяся часть книги) игнорирует оптимизацию frame pointer omission (FPO) просто потому, что она не используется в Windows XP SP2 и более поздних операционных системах.

Поскольку FPO-оптимизация делает отладку практически невозможной без символов, текущая рекомендация - полностью избегать ее.

При входе в функцию компилятор создает так называемый stack frame, который поддерживается с помощью регистра frame base pointer ebp.

Пролог функции сохраняет текущее значение ebp в стеке и загружает текущее значение указателя стека, которое будет сохраняться до выполнения эпилога функции.

Внутри функции компилятор адресует входные параметры с помощью положительных смещений относительно frame-based pointer, а локальную переменную, выделенную в функции, - с помощью отрицательных смещений.

Самые простые пролог и эпилог функции показаны здесь:

```
0:000> uf .
02sample!KBTest::Fibonacci_stdcall:
00401760 8bff mov edi,edi
00401762 55 push ebp
00401763 8bec mov ebp,esp
...
004017b3 8be5 mov esp,ebp
004017b5 5d pop ebp
004017b6 c20400 ret 4
```

В эпилоге функции значение ebp перезагружается сохраненным значением, чтобы регистр был сохранен после вызова.

Размещение входных параметров, локальной переменной и базового указателя кадра показано на следующем рисунке.

Перед выполнением вызова функции вызывающий код помещает все параметры функции в стек.

Затем процессор сохраняет адрес, с которого выполнение продолжится при возврате.

Вызванная функция использует стек для сохранения старого ebp и выделяет необходимое пространство для локальной переменной.

Затем регистр ebp используется для доступа к входным параметрам и локальной переменной, как видно в правой части рисунка 2.5.

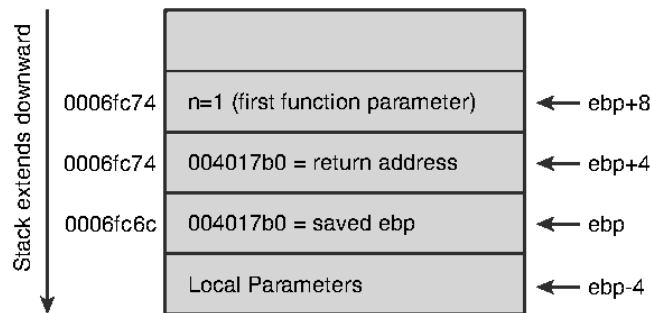


Figure 2.5 Stack content when calling a function following the `__stdcall` convention

Рисунок 2.5. Содержимое стека при вызове функции по соглашению `__stdcall`

Стек вызовов записывает всю цепочку вызовов функций, сделанных текущим потоком, которая привела к вызову текущей функции.

Представление стека начинается с текущей выполняемой функции, отображенной наверху, за которой следует ее вызывающая функция, вызывающая функция вызывающей функции и так далее; каждая точка вызова идентифицируется своим stack frame.

Процесс повторяется, пока отладчик не достигнет последнего stack frame в стеке вызовов или пока внешнее условие, например некорректные символы или недоступный стек, не помешает отладчику дальше декодировать стек.

Неудивительно, что стек текущего сбоя является одним из наиболее используемых фрагментов информации.

Иногда стек потока используется для индексирования и каталогизации программных сбоев.

Команду `k` (`display stack back trace`) можно использовать для анализа текущего стека с помощью символов модулей и форматирования информации согласно дополнительным параметрам, переданным в командной строке.

Как и большинство контекстно-зависимых команд, `k` интерпретирует стек из информации текущего контекста.

WinDbg предоставляет окно `call stack`, которое обновляется каждый раз, когда отладчик останавливается.

Чтобы поэкспериментировать с командами `k`, мы запустим `02sample.exe` под отладчиком и выберем вариант генерации обычного стека вызовов.

Этот вариант рекурсивно вычисляет 32-е число из ряда Fibonacci.

Исходный код функции показан в листинге 2.23.

Листинг 2.23. Исходный код функции `Fibonacci`, реализованной в примере `02sample.exe`

```
#define STOP_ON_DEBUGGER { if (IsDebuggerPresent()) DebugBreak();}
unsigned int Fibonacci(unsigned int n)
{
    switch(n)
    {
        case 0: STOP_ON_DEBUGGER;return 0;
        case 1: return 1;
        default: return Fibonacci(n-1)+Fibonacci(n-2);
    }
}
```

Эта функция включает специальную функциональность, облегчающую ее отладку.

Когда она выполняется под отладчиком пользовательского режима, наша функция `Fibonacci` вызывает `DebugBreak` перед возвратом `F(0)`.

Мы уже обсуждали (в разделе "Setting Up and Using the Symbols"), как устанавливать символы, и предполагаем, что они корректны.

Теперь мы готовы экспериментировать с командами k после того, как программа остановится в отладчике.

В базовой форме команда k показывает максимальное количество фреймов, управляемое командой .kframes; значение по умолчанию равно 20.

Для каждого фрейма команда отображает информацию stack frame в столбце ChildEBP.

В столбце RetAddr она отображает адрес, с которого код начнет выполняться, когда функция вернется, и символ, с которым связана текущая функция, как показано в листинге 2.24.

Листинг 2.24. Отображение стека вызовов

```
0:000> k
ChildEBP RetAddr
0006fcb0 010017eb ntdll!DbgBreakPoint
0006fcc0 01001810 02sample!KBTest::Fibonacci_stdcall+0x2b
0006fcd4 01001802 02sample!KBTest::Fibonacci_stdcall+0x50
...
0006ff2c 0100179c 02sample!KBTest::Fibonacci_stdcall+0x42
0006ff38 01001d93 02sample!Stack+0xc
0006ff50 01001cab 02sample!AppInfo::Loop+0xb3
0006ff5c 01002076 02sample!wmain+0x1b
0006ffa0 76033833 02sample!__wmainCRTStartup+0x102
0006ffac 7734a9bd kernel32!BaseThreadInitThunk+0xe
0006ffec 00000000 ntdll!_RtlUserThreadStart+0x23
```

Каждая функция, скорее всего, получает несколько параметров со значениями, важными для истории выполнения программы.

Команды kr и kP специально предназначены для интерпретации информации каждой функции и отображения типа параметра, имени параметра, а также связанного с ним значения.

kr показывает все параметры в одной строке (см. листинг 2.25), тогда как kP использует отдельную строку для каждого параметра.

Листинг 2.25. Отображение параметров, используемых последними пятью функциями из стека вызовов

```
0:000> * Displays the past five function on the stack with their parameters
0:000> kP 5
ChildEBP RetAddr
0006fcb0 010017ab ntdll!DbgBreakPoint
0006fcc0 010017d0 02sample!KBTest::Fibonacci_stdcall(
unsigned int n = 0)+0x2b
0006fcd4 010017c2 02sample!KBTest::Fibonacci_stdcall(
unsigned int n = 2)+0x50
0006fce8 010017c2 02sample!KBTest::Fibonacci_stdcall(
unsigned int n = 3)+0x42
0006fcfc 010017c2 02sample!KBTest::Fibonacci_stdcall(
unsigned int n = 4)+0x42
```

Поскольку символы функций являются частью private symbols, стек часто содержит функцию без информации о параметрах.

В таких случаях мы можем использовать команду kb, чтобы отобразить первые три параметра, переданные этой функции в стеке.

Используя дополнительную информацию, например сигнатуру функции и ее соглашение о вызове, мы можем интерпретировать, какие параметры действительны для каждой функции.

В листинге 2.26 видно, что реальный параметр показан корректно, тогда как два следующих параметра не имеют смысла в этом стеке, потому что функция имеет только один параметр.

Листинг 2.26. Отображение первых трех параметров, используемых пятью функциями из стека вызовов

```
0:000> kb 5
ChildEBP RetAddr Args to Child
```

```

0006fc6c 004017b0 00000001 00191ffc 00000003 02sample!KBTest::Fibonacci_stdcall+0x5
0006fc80 004017a2 00000003 00191ffc 00000004 02sample!KBTest::Fibonacci_stdcall+0x50
0006fc94 004017a2 00000004 00191ffc 00000005 02sample!KBTest::Fibonacci_stdcall+0x42
0006fca8 004017a2 00000005 00191ffc 00000006 02sample!KBTest::Fibonacci_stdcall+0x42
0006fcbc 004017a2 00000006 00191ffc 00000007 02sample!KBTest::Fibonacci_stdcall+0x42

```

В процессе разработки и тестирования надежных серверов неспособность расширить стек потока в условиях нехватки памяти представляет собой распространенный сбой.

Решение, применяемое в этом случае, заключается в ограничении использования стека размером committed stack путем внимательного отслеживания пространства стека, используемого в каждом stack frame, и его максимального уменьшения.

Использование стека для каждого фрейма можно вычислить, вычитая текущий base frame pointer из base frame pointer одной из функций, вызванных текущей функцией.

Процесс облегчается формой команды k, которая вычисляет и показывает это значение для каждой функции, кроме текущей.

Команда kf принимает те же параметры, что и все остальные формы команды k, и используется в листинге 2.27 для отображения последних пяти функций.

В первом столбце команда отображает размер стека, использованный функцией.

Листинг 2.27. Отображение размера стека, использованного последними пятью функциями из стека вызовов

```

0:000> kf 5
Memory ChildEBP RetAddr
0006fc6c 004017b0 02sample!KBTest::Fibonacci_stdcall+0x5
14 0006fc80 004017a2 02sample!KBTest::Fibonacci_stdcall+0x50
14 0006fc94 004017a2 02sample!KBTest::Fibonacci_stdcall+0x42
14 0006fca8 004017a2 02sample!KBTest::Fibonacci_stdcall+0x42
14 0006fcbc 004017a2 02sample!KBTest::Fibonacci_stdcall+0x42

```

В некоторых случаях доступна только часть стека, и команда отладчика k не может декодировать стек, поскольку адрес, на который указывает текущий base frame pointer ebp, и текущий stack pointer esp недоступны.

В таких случаях вместо нее можно использовать вариант команды k, принимающий значения base frame pointer, stack pointer и instruction pointer.

Самая трудная часть в ручном процессе реконструкции стека - определить хорошую пару значений из области памяти, представляющую корректный stack frame из вызывающего стека.

Один способ найти их - определить последовательность значений, представляющую адрес, указывающий на текущий стек, за которым следует исполняемый адрес.

Каждый адрес может быть потенциальным фреймом, и его следует проверить с помощью команды k.

Операцию следует повторять с другим потенциальным фреймом до тех пор, пока стек не будет правильно отображен и команда k не покажет разумный стек, как в листинге 2.28.

Листинг 2.28. Ручная реконструкция стека с использованием команды k

```

0:000> * Dump the memory block and look for pattern
0:000> dc esp
0006fc6c 0006fc80 004017b0 00000001 00191ffc .....@.....
0006fc7c 00000003 0006fc94 004017a2 00000003 .....@.....
0006fc8c 00191ffc 00000004 0006fca8 004017a2 .....@.
0006fc9c 00000004 00191ffc 00000005 0006fcbc .....
0006fcac 004017a2 00000005 00191ffc 00000006 ..@.....
0006fcbc 0006fcd0 004017a2 00000006 00191ffc .....@.....
0006fccc 00000007 0006fce4 004017a2 00000007 .....@.....
0006fcdc 00191ffc 00000008 0006fcf8 004017a2 .....@.
0:000> * Used saved ebp, the address storing it and the return address
0:000> k = 0006fc80 0006fc6c 004017b0
ChildEBP RetAddr
0006fc80 004017a2 02sample!KBTest::Fibonacci_stdcall+0x50
0006fc94 004017a2 02sample!KBTest::Fibonacci_stdcall+0x42

```

Это распространенный сценарий, встречающийся при отладке чрезвычайно нагруженных систем из отладчика режима ядра, когда только некоторые страницы из стека потока находятся в памяти.

Setting a Code Breakpoint

Отладчик часто используется для проверки выполнения конкретной последовательности кода: либо путем остановки выполнения в начале последовательности, либо когда происходит интересное условие.

Этого можно добиться с помощью команд breakpoint.

Code breakpoints устанавливаются с помощью команды bp, которая принимает в качестве параметров адрес, где нужно установить breakpoint, параметры breakpoint, ограничения breakpoint и строку, содержащую команду, выполняемую при срабатывании breakpoint.

Breakpoint, установленный в отладчике пользовательского режима, может иметь префикс с идентификатором потока; в таком случае отладчик остановится только тогда, когда указанный поток достигнет breakpoint.

В листинге 2.29 показано использование команд breakpoint для установки breakpoint, перечисления всех breakpoints и их удаления.

Листинг 2.29. Использование breakpoints в отладчике пользовательского режима

```
0:000> * Breakpoint only on thread 0 and execute "resp" command
0:000> ~0 bp 02sample!KbTest::Fibonacci "resp"
0:000> * List the breakpoints
0:000> bl
0 e 00401750 0001 (0001) 0:~000 02sample!KbTest::Fibonacci_stdcall "resp"
0:000> g
esp=0006fdc4
eax=00000012 ebx=7ffdf000 ecx=00000011 edx=77c61b78 esi=7c9118f1 edi=00011970
eip=00401750 esp=0006fdc4 ebp=0006fdd4 iopl=0 nv up ei pl nz na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000206
02sample!KbTest::Fibonacci_stdcall:
00401750 8bff mov edi,edi
0:000> * Clear all breakpoints
0:000> bc *
0:000> * Set a breakpoint for all threads to execute "reasp;g"
0:000> bp 02sample!KbTest::Fibonacci "resp;g"
0:000> g
esp=0006fc98
esp=0006fcac
esp=0006fc98
esp=0006fc98
...
```

При создании каждый breakpoint получает числовой идентификатор, который позже можно использовать для внесения изменений в этот breakpoint.

Идентификатор breakpoint, который стал причиной текущей остановки, отладчик показывает сразу после остановки.

WinDbg предоставляет кнопку панели инструментов и окно Breakpoints для управления breakpoints.

Тот же breakpoint можно установить из отладчика режима ядра; главное отличие в том, что он является глобальным для всей системы.

Если область действия breakpoint нужно ограничить конкретным процессом или потоком, адрес структуры EPROCESS или KTHREAD должен быть указан как параметр команды breakpoint.

В листинге 2.30 первый breakpoint устанавливается для всех потоков (и неявно для всех процессов), выполняющихся в системе, тогда как второй ограничен процессом, идентифицированным текущим процессом через псевдорегистр \$proc.

Листинг 2.30. Использование breakpoints в отладчике режима ядра

```

kd> * Breakpoint on ntdll!RtlAllocateHeap will break on each allocation
kd> bp ntdll!RtlAllocateHeap
kd> * Breakpoint limited to the process
kd> bp /p @$proc ntdll!RtlAllocateHeap "!process -1 0;g"
kd> g
PROCESS 811de7f8 SessionId: 0 Cid: 037c Peb: 7ffd9000 ParentCid: 0240
DirBase: 0567b000 ObjectTable: e1781770 HandleCount: 1412.
Image: svchost.exe
kd> bl
0 e 7c9105d4 0001 (0001) ntdll!RtlAllocateHeap
Match process data 811de7f8

```

Команда `bm` является удобным способом установить несколько breakpoints по всем адресам, соответствующим шаблону символа, указанному как параметр.

В листинге 2.31 команда `bm` используется для установки breakpoints для всех методов, реализованных классом `KBTest`.

Когда `private symbols` недоступны для целевого модуля, команда `bm` завершается ошибкой, если мы не переопределим ее поведение с помощью параметра `/a`.

Листинг 2.31. Использование breakpoints в отладчике пользовательского режима

```

0:000> bm 02sample1!*kbtest*
1: 00401860 @"!\"02sample!KBTest::Fibonacci_fastcall"
2: 004017a0 @"!\"02sample!KBTest::Fibonacci_stdcall"
3: 004018d0 @"!\"02sample!KBTest::ObjFibonacci"
4: 00401800 @"!\"02sample!KBTest::Fibonacci_cdecl"
breakpoint 2 redefined
2: 004017a0 @"!\"02sample!KBTest::Fibonacci"

```

Операционная система Windows загружает `dynamic link libraries` тогда, когда они нужны, и нам часто приходится устанавливать breakpoint на модуль, который еще не загружен.

Команда `bu` может установить `deferred breakpoint`, который становится реальным breakpoint, когда модуль, владеющий этим breakpoint, загружается.

Например, следующая строка устанавливает `deferred breakpoint` на функцию инициализации DCOM.

```

0:000> bu ole32!CoInitializeEx

```

Когда модуль, содержащий символ, уже загружен в память, команда `bu` немедленно устанавливает breakpoint по адресу символа.

Поскольку `deferred breakpoints` основаны на символической информации, они сохраняются в `workspaces`, создаваемых WinDbg, и используются в последующих сеансах отладки.

Неудивительно, что `bu` часто используется как предпочтительный способ включения breakpoints.

Команда `bu` работает и с отладчиком режима ядра.

Но для отладчика режима ядра команда устанавливает breakpoints только на модули, загружаемые в пространство ядра.

Поэтому breakpoints пользовательского режима должны устанавливаться с помощью комбинации техник, как вы увидите позже в разделе "Debugging Scenarios".

What Are the Variable Values?

Поскольку все выполнение кода зависит от мгновенных значений всех переменных, используемых в конкретной функции, необходимо знать эти значения, чтобы понимать историю выполнения и прогнозировать дальнейшее выполнение.

Команда `dv` делает именно это, предлагая большой набор параметров для проверки переменных. По смыслу, а иногда и по функциональности, эта команда похожа на команду `x`, используемую для проверки информации о символах.

Чтобы проиллюстрировать функциональность команды `dv`, мы установим breakpoint в member function `Fibonacci_thiscall`, построенной в `02sample.exe` и вызываемой выбором варианта 6. Member function, показанная в следующем листинге, реализует функциональность `Fibonacci`.

```
unsigned int KBTTest::Fibonacci_thiscall(unsigned int n)
{
    m_lastN = n;
    int localN = n + gGlobal.m_ref;
    switch(n)
    {
        case 0: STOP_ON_DEBUGGER; return 0;
        case 1: return 1;
        default:
        {
            return Fibonacci_thiscall(localN-2)+Fibonacci_thiscall(localN-3);
        }
    }
}
```

Функция использует всего четыре переменные: параметр функции с символическим именем `n`; неявный указатель C++ с именем `this`; локальную переменную `localN`; и глобальную переменную `gGlobal`.

В листинге 2.32 показаны различные варианты использования команды `dv` для исследования значений переменных в контексте функции `Fibonacci_thiscall` после того, как выполнение кода было остановлено breakpoint.

Исполняемый файл был скомпилирован без оптимизации, чтобы минимизировать расхождения между C++ кодом и сгенерированным ассемблерным кодом.

Даже когда оптимизация отключена, команда `dv` иногда возвращает пользователю неожиданную информацию.

WinDbg предоставляет окно `Locals`, которое обновляется текущими значениями переменных каждый раз, когда отладчик останавливается.

Листинг 2.32. Использование команды `dv`

```
0:000> * In the simplest form dv displays the local variables
0:000> dv
this = 0x77c146f0
n = 0x20
localN = -1
0:000> * dv can be used to display variables matching a pattern
0:000> dv 02sample!gGlo*
02sample!gGlobal$initializer$ = 0x01002920
02sample!gGlobal = class Global
0:000> * dv /i shows the symbol type (priv) and parameter type
0:000> * on the second column
0:000> dv /i
prv local this = 0x77c146f0
prv param n = 0x20
prv local localN = -1
0:000> * dv /V shows the location where the variable is stored
0:000> dv /V this
0006fee4 @ebp-0x08 this = 0x77c146f0
0:000> * If the variable is not correct, unassemble the function
```

Когда переменная является сложным типом, например структурой данных или классом, команда `dv` показывает только ее адрес.

Однако команда `dt`, что означает `display type`, может интерпретировать блок памяти как тип данных, имя которого передано параметром.

Команда `dt` не требует имени типа данных, если адрес является символическим именем, тип которого известен отладчику.

В листинге 2.33 показано несколько примеров использования команды `dt`.

Команда `dt` также может рекурсивно обрабатывать встроенный объект или массив объектов с правильными параметрами, хорошо описанными в справке отладчика (раздел справки `DT`).

Листинг 2.33. Использование команды dt

```
0:000> * dt interprets this object type when displaying the memory block
0:000> dt this
Local var @ 0x6fee4 Type KBTes*
0x77c146f0
+0x000 __VFN_table : ???
+0x004 m_lastN : ??
Memory read error 0x77c146f0
0:000> * dt uses the data type passed in when displaying the memory block
0:000> dt KBTes 0x006fee4
02sample!KBTes
+0x000 __VFN_table : ???
+0x004 m_lastN : ??
0:000> * dt interpret the object type when displaying the memory block
0:000> dt 02sample!gGlobal
gGlobal
+0x000 m_ref : 1
```

Если вы произвольно проверяете блок heap, вполне возможно найти в первых нескольких позициях символ v-table, указывающий тип C++ объекта, расположенного (или ранее располагавшегося) по этому адресу.

Затем можно использовать информацию о типе для отображения объекта, как показано в следующем листинге, снятом на той же остановке, что и листинг 2.33.

```
0:000> dc @ecx 14
0006fee4 00401504 ffffffff 0006ff90 01002b28 .....(+..
0:000> ln 00401504
(00401504) 02sample!KBTes::`vftable' | (00401508) 02sample!`string'
Exact matches:
0:000> dt KBTes @ecx
02sample!KBTes
+0x000 __VFN_table : 0x00401504
+0x004 m_lastN : -1
```

В листинге 2.32 значение, отображаемое для переменной-указателя `this`, выглядит неправильно, поскольку такое значение обычно зарезервировано для сегментов системного двоичного кода. Глядя на код, можно увидеть, что объект выделен в стеке и должен иметь значение, близкое к текущему stack pointer.

Рассмотрим вывод команды `dv /V this`:

```
0006fee4 @ebp-0x08 this = 0x77c146f0
```

Указатель `this` хранится в позиции стека `0006fee4`, и код функции обращается к нему с помощью frame-based register `@ebp-0x08`.

Значение, сохраненное по этому адресу, на самом деле неверно.

Как такое может быть?

Вызов member function следует соглашению `__thiscall`, означающему, что регистр `ecx` содержит значение указателя `this`.

Позднее значение регистра сохраняется в stack frame функции по адресу `@ebp-0x08`, что означает, что значение становится точным после выполнения функцией следующей инструкции:

```
00401878 894df8 mov dword ptr [ebp-8],ecx
```

Теперь вопрос становится таким: почему компилятор не генерирует лучшие символы для отслеживания расположений локальных переменных?

Попробуйте представить, что произойдет в сильно оптимизированном коде с большим количеством переменных: регистры переиспользуются, а записи в stack frame функции минимизируются.

Это означает, что компилятору пришлось бы генерировать новую ссылку символа для каждой ассемблерной инструкции, затрагивающей переменные.

Это значит, что файлы символов были бы больше.

Этот более крупный файл пришлось бы перемещать и загружать отладчиками во время отладки, а также намного чаще просматривать, что привело бы к плохому пользовательскому опыту при минимальной пользе.

Пока не найдено лучшее решение этой проблемы, вы должны убедиться, что значение переменной корректно, прежде чем продолжать исследование.

Затем можно проверить его с помощью команды `dt`, как в следующем листинге:

```
0:000> dt kbTest @ecx
02sample!KBTest
+0x000 __VFN_table : 0x00401504
+0x000 m_lastN : -1
```

LOCAL VARIABLE VERSUS INPUT PARAMETERS. В общем случае большинство входных параметров можно найти в стеке, и они адресуются с помощью `frame-based parameters` с положительным смещением, например `@ebp+8`.

Локальные параметры доступны через отрицательные смещения, например `@ebp-8`.

Иногда компилятор повторно использует хранилище переменных, что может вызывать трудности при отладке.

How Do You Inspect Memory?

При расследовании проблемы в отладчике нам часто приходится изучать разные блоки памяти, чтобы понять причину проблемы и затем доказать, что сценарий действительно верен.

Поскольку состояние различных объектов сохраняется в памяти, содержимое памяти эквивалентно состоянию объекта.

Команда `display` принимает адрес или диапазон адресов и отображает содержимое, сохраненное по этим адресам, согласно аргументам команды.

Самая распространенная форма команды `display` просто читает, форматирует и отображает данные на основе типов, сохраненных по адресу.

Отладчик не пытается угадать, какие данные хранятся в этом месте, потому что в большинстве случаев он, скорее всего, ошибется.

Пользователь определяет формат, в котором данные должны интерпретироваться.

`display` имеет следующий синтаксис:

```
d[type] [AddressRange]
```

Чтобы проиллюстрировать различные формы этой команды, мы используем тот же `02sample.exe`, но запускаем его с несколькими аргументами командной строки.

Даже если аргументы игнорируются, они все равно передаются функции `main`.

Сигнатура функции является стандартным объявлением `main`, как показано ниже:

```
VOID _cdecl main( ULONG argc, PCHAR argv[] )
```

В листинге 2.34 мы используем несколько форм команды `display` для проверки параметров командной строки, переданных в массиве `argv[]`, после установки `breakpoint` в функции `02sample!wmain`.

Листинг 2.34. Использование команды `d`

```
0:000> bp 02sample!wmain
0:000> g
Breakpoint 0 hit
0:000> * Get the address of argv parameter
0:000> dv /V argv
0006ff68 @ebp+0x0c argv = 0x005f0ea0
0:000> * Dump 4 double words at argv address
0:000> dc 0x005f0ea0 14
```



```

005f0ea0 005f0eb4 005f0efe 005f0f08 005f0f12 .._..._..._..._.
0:000> dd 0x005f0ea0
005f0ea0 005f0eb4 005f0efe 005f0f08 005f0f12
0:000> * Dump one Unicode string
0:000> du 005f0eb4
005f0eb4 "c:\AWDBIN\WinXP.x86.chk\02sample"
005f0ef4 ".exe"
0:000> * Dump one Unicode string as ASCII string

```

Листинг 2.34. Использование команды d (продолжение)

```

0:000> da 005f0eb4
005f0eb4 "c"
0:000> * Dump four bytes as byte array
0:000> db 005f0eb4 l4
005f0eb4 63 00 3a 00 c.:.
0:000> * Dump four bytes in binary format
0:000> * The heading line represent the bit position
0:000> dyb 005f0eb4 l4
76543210 76543210 76543210 76543210
-----
005f0eb4 01100011 00000000 00111010 00000000 63 00 3a 00
0:000> * Dump four double words in binary format
0:000> dyd 005f0eb4 l4
3 2 1 0
10987654 32109876 54321098 76543210
-----
005f0eb4 00000000 00111010 00000000 01100011 003a0063
005f0eb8 00000000 01000001 00000000 01011100 0041005c
005f0ebc 00000000 01000100 00000000 01010111 00440057
005f0ec0 00000000 01001001 00000000 01000010 00490042
0:000> * Dump four float numbers
0:000> df 005f0eb4 l4
005f0eb4 5.3265975e-039 5.9694362e-039 6.2449357e-039 6.7040837e-039
0:000> * Dump four words numbers
0:000> dw 005f0eb4 l4
005f0eb4 0063 003a 005c 0041
0:000> * Dump four float numbers with the character representation
0:000> dW 005f0eb4 l4
005f0eb4 0063 003a 005c 0041 c.:.\.A
0:000> * Dump an invalid memory address
0:000> dc 0 l4
00000020 ???????? ???????? ???????? ???????? ?????????????????

```

В листинге непечатаемые символы отображаются как точки (.).

Это может немного сбивать с толку, когда блок действительно содержит точки.

В других случаях отладчик отображает только поток вопросительных знаков (?), которые представляют, ну... ничего.

Адрес недействителен, и отладчик не может ничего прочесть по этому адресу, потому что адрес не отображен в целевом процессе.

После выбора варианта 6 мы используем thread zero, чтобы показать другие формы этой команды. Следующая форма используется для дампа области памяти, а также для трактовки каждого элемента в памяти как символа и его разрешения.

Есть три формы этой команды, обобщенно называемые командами d*s: dds трактует каждую группу из четырех байтов как символ; dqс трактует каждую группу из восьми байтов как символ; а dрс использует длину, наиболее подходящую для архитектуры отлаживаемого процессора. В листинге 2.35 показан пример использования этой команды для некоторой памяти стека.

Листинг 2.35. Использование команды d*s

```

0:000> dps esp l8
0005fcb4 010017ab 02sample!KBTest::Fibonacci_stdcall+0x2b
0005fcb8 00000001
0005fcbc 00000000
0005fcc0 0006fcd4
0005fcc4 010017d0 02sample!KBTest::Fibonacci_stdcall+0x50

```

Последняя форма похожа на команду `d*s`.

Отладчик проходит по области памяти, рассматривая ее как последовательность 32- или 64-битных указателей, как делает ранее обсуждавшаяся команда `d*s`.

Он использует каждое значение, прочитанное из области памяти, как указатель на другой тип данных, который затем отображается с использованием формата, специфичного для типа.

Не убеждены или не понимаете полезность этого?

В приглашении отладчика, использованном в листинге 2.34, мы применяем этот параметр для отображения массива Unicode strings, представляющих аргументы командной строки цели отладчика.

```
0:000> * Dump an array of UNICODE strings
0:000> dpu 0x005f0ea0 L4
005f0ea0 005f0eb4 "c:\AWDBIN\WinXP.x86.chk\02sample.exe"
005f0ea4 005f0efe "arg1"
005f0ea8 005f0f08 "arg2"
005f0eac 005f0f12 "arg3"
```

Эта форма команды также очень эффективна при работе с неизвестной областью памяти.

Команда `s`, что означает `search`, является еще одной эффективной командой для обнаружения известных значений в памяти цели отладчика.

Команда принимает искомый тип и искомое значение как параметры.

Следующий листинг демонстрирует использование команды `s` для поиска кода исключения в памяти процесса.

Он снят после выбора варианта 1 в `02sample.exe`.

Команда `s` ищет значение `double-word` в первых 265MB виртуального адресного пространства.

```
0:000> * Run the debugger target after the access violation exception
0:000> g
(53a8.4070): Access violation - code c0000005 (!!! second chance !!!)
eax=00000000 ebx=00000000 ecx=01003008 edx=01003008 esi=00000001 edi=0100373c
eip=010016d0 esp=0006ff34 ebp=0006ff38 iopl=0 nv up ei pl nz na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010206
02sample!RaiseAV+0x10:
010016d0 66c7000000 mov word ptr [eax],0 ds:0023:00000000=????
0:000> * Search for the exception code in the first 256Mb of the address space
0:000> s -d 0 L10000000/4 C0000005
0006fc4c c0000005 00000000 00000000 010016d0 .....
0006ff80 c0000005 00000000 0006ff70 0006fb30 .....p...0...
0006ffc8 c0000005 76b25984 76b25984 0006ffb8 .....Y.v.Y.v....
```

Setting a Breakpoint on Access

Не все проблемы можно найти с помощью `code breakpoints`.

Например, есть множество случаев, когда одна позиция памяти изменяется реже, чем вызывается функция, изменяющая данные такого типа, как в случае `API kernel32!HeapFree`.

Нас интересует момент, когда удаляется конкретный блок, и непрактично перехватывать все вызовы и останавливаться только тогда, когда параметр, переданный `API`, соответствует интересующему нас адресу.

Тем не менее блок может быть изменен в результате `buffer overrun`, а не во время выполнения функции.

Проблему в этом сценарии можно эффективно решить только с помощью способности процессора генерировать `breakpoint` при доступе к конкретной позиции памяти.

Эта возможность управляется командой отладчика `ba`, или `breakpoint on access`.

Адрес, отслеживаемый средствами `breakpoint on access`, должен быть выровнен относительно размера данных, отслеживаемого `breakpoint`.

Листинг 2.36 содержит определение класса Global, используемого в 02sample.exe для объявления глобальной переменной gGlobal.

Класс имеет одну member variable, m_ref, которая изменяется каждый раз при выполнении конструктора или деструктора этого класса.

Гипотетически класс используется во многих других местах помимо global static variable, но наша цель - выяснить, какой стек изменяет member m_ref глобальной static variable.

Листинг 2.36. Объявление gGlobal

```
class Global
{
public:
    int m_ref;
    Global():m_ref(1){};
    ~Global()
    {
        m_ref = 0;
    };
} gGlobal;
```

После беглого взгляда на определение класса можно попробовать установить breakpoint на конструктор и деструктор класса Global, предполагая, что мы легко поймем, какой объект изменяется.

Поскольку деструктор вызывается много раз, процесс становится дорогим и склонным к ошибкам.

Однако адрес памяти объекта и, неявно, адрес памяти member m_ref известен в каждом сеансе отладки.

Затем этот адрес используется для установки breakpoint on access, отслеживающего адрес памяти m_ref для операций записи.

Breakpoint установлен для отслеживания четырех байтов, хранящих member m_ref.

В листинге 2.37 показано, как ba может решить проблему одной строкой.

Команда ba требует режим доступа и размер данных, которые будет отслеживать процессор.

Листинг 2.37. Типичное использование команды ba

```
0:000> * Getting the address of the variable to be monitored
0:000> dt gGlobal
+0x000 m_ref : 0
0:000> * Setting a breakpoint when m_ref memory address is changed
0:000> * The processor monitors writes in the four bytes following
0:000> ba w4 gGlobal+0
0:000> bl
0 e 0040301c w 4 0001 (0001) 0:**** 02sample!gGlobal
0:000> g
Breakpoint 0 hit
eax=0040301c ebx=00000000 ecx=0040301c edx=775ec534 esi=00000001 edi=003f2bd0
eip=004018c2 esp=0006fefc ebp=0006fff0 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
02sample!Global::~~Global+0x12:
004018c2 8be5 mov esp,ebp
0:000> * The break is happening after the change happened
0:000> ub . 11
02sample!Global::~~Global+0xc:
004018bc c70000000000 mov dword ptr [eax],0
```

Breakpoint on access работает так же хорошо из отладчика режима ядра.

What Does That Memory Location Contain?

Во время отладки в объектах, а также в стеке, есть множество указателей, для которых мы не можем быстро угадать, что они представляют.

Хотя адреса пространства ядра отличить легче, чем адреса пользовательского режима, нелегко отличить адрес, представляющий стек, от адреса, представляющего блок в heap.

Команда отладчика создала extension command, полезную для решения этой проблемы, доступную как `!address <address>`.

Команда чрезвычайно полезна при отладке пользовательского режима.

Типичный вывод показан в листинге 2.38.

Листинг 2.38. Пример команды расширения отладчика `!address`

```
0:000> !address .
7c900000 : 7c901000 - 0007b000
Type 01000000 MEM_IMAGE
Protect 00000020 PAGE_EXECUTE_READ
State 00001000 MEM_COMMIT
Usage RegionUsageImage
FullPath ntdll.dll
0:000> !address @esp
00030000 : 0006e000 - 00002000
Type 00020000 MEM_PRIVATE
Protect 00000004 PAGE_READWRITE
State 00001000 MEM_COMMIT
Usage RegionUsageStack
Pid.Tid 1124.1568
0:000> !address 00080000
00080000 : 00080000 - 00004000
Type 00020000 MEM_PRIVATE
Protect 00000004 PAGE_READWRITE
State 00001000 MEM_COMMIT
Usage RegionUsageHeap
Handle 00080000
0:000> !address 1000
00000000 : 00000000 - 00010000
Type 00000000
Protect 00000001 PAGE_NOACCESS
State 00010000 MEM_FREE
Usage RegionUsageFree
```

В первый раз параметр команды является адресом кода (текущим адресом выполнения); во второй раз это адрес стека, затем адрес heap и, наконец, недействительный адрес.

Extension command может обрабатывать и другие типы памяти.

Когда адрес не указан, расширение ищет и перечисляет все зоны памяти со всеми доступными подробностями, как показано в листинге 2.39.

После этого оно вычисляет сводку по использованию памяти на основе типа секции, режима доступа и режима совместного использования страниц.

Упрощенный вывод анализа пространства процесса можно увидеть в следующем листинге.

Листинг 2.39. Команда `!address`

```
0:000> !address
00000000 : 00000000 - 00010000
Type 00000000
Protect 00000001 PAGE_NOACCESS
State 00010000 MEM_FREE
Usage RegionUsageFree
...
7ffdf000 : 7ffdf000 - 00001000
Type 00020000 MEM_PRIVATE
Protect 00000004 PAGE_READWRITE
State 00001000 MEM_COMMIT
Usage RegionUsageTeb
Pid.Tid 1124.1568
```

```

...
----- Usage SUMMARY -----
TotSize ( KB) Pct(Tots) Pct(Busy) Usage
1d4000 ( 1872) : 00.09% 32.16% : RegionUsageIsVAD
7fa41000 ( 2091268) : 99.72% 00.00% : RegionUsageFree
266000 ( 2456) : 00.12% 42.20% : RegionUsageImage
40000 ( 256) : 00.01% 04.40% : RegionUsageStack
1000 ( 4) : 00.00% 00.07% : RegionUsageTeb
130000 ( 1216) : 00.06% 20.89% : RegionUsageHeap
0 ( 0) : 00.00% 00.00% : RegionUsagePageHeap
1000 ( 4) : 00.00% 00.07% : RegionUsagePeb
1000 ( 4) : 00.00% 00.07% : RegionUsageProcessParameters
2000 ( 8) : 00.00% 00.14% : RegionUsageEnvironmentBlock
Tot: 7fff0000 (2097088 KB) Busy: 005af000 (5820 KB)
----- Type SUMMARY -----
TotSize ( KB) Pct(Tots) Usage
7fa41000 ( 2091268) : 99.72% : <free>
266000 ( 2456) : 00.12% : MEM_IMAGE
1d4000 ( 1872) : 00.09% : MEM_MAPPED
175000 ( 1492) : 00.07% : MEM_PRIVATE
----- State SUMMARY -----
TotSize ( KB) Pct(Tots) Usage
34e000 ( 3384) : 00.16% : MEM_COMMIT
7fa41000 ( 2091268) : 99.72% : MEM_FREE
261000 ( 2436) : 00.12% : MEM_RESERVE
Largest free region: Base 00405000 - Size 75c7b000 (1929708 KB)

```

Other Exploratory Commands

Еще один распространенный вопрос, который задают пользователи отладчика, - какие параметры командной строки использовались для запуска текущей цели отладчика.

Эта информация хранится в process environment block (PEB) и может быть легко получена с помощью extension command !peb, как показано в листинге 2.40.

Команда интерпретирует PEB, показывая командную строку, расположение всех загруженных DLL, переменные окружения и многое другое.

Листинг 2.40. Получение PEB процесса

```

0:000> !peb
PEB at 7ffdd000
InheritedAddressSpace: No
ReadImageFileExecOptions: No
BeingDebugged: Yes
ImageBaseAddress: 00400000
Ldr 00181ea0
Ldr.Initialized: Yes
Ldr.InInitializationOrderModuleList: 00181f58 . 001821a0
Ldr.InLoadOrderModuleList: 00181ee0 . 00182190
Ldr.InMemoryOrderModuleList: 00181ee8 . 00182198
Base TimeStamp Module
400000 453bf190 Oct 22 15:32:48 2006 C:\AWDBIN\WinXP.x86.chk\02sample.exe
7c900000 411096b4 Aug 04 00:56:36 2004 C:\WINDOWS\system32\ntdll.dll
7c800000 44ab9a84 Jul 05 03:55:00 2006 C:\WINDOWS\system32\kernel32.dll
77c10000 41109752 Aug 04 00:59:14 2004 C:\WINDOWS\system32\msvcrt.dll
76080000 41109751 Aug 04 00:59:13 2004 C:\WINDOWS\system32\msvcpx60.dll
SubSystemData: 00000000
ProcessHeap: 00080000
ProcessParameters: 00020000
WindowTitle: 'C:\AWDBIN\WinXP.x86.chk\02sample.exe'
ImageFile: 'C:\AWDBIN\WinXP.x86.chk\02sample.exe'
CommandLine: 'C:\AWDBIN\WinXP.x86.chk\02sample.exe'
DllPath:
'C:\AWDBIN\WinXP.x86.chk;C:\WINDOWS\system32;C:\WINDOWS\system;C:\WINDOWS;. ;c:\Debug.x
86\winext\arcade;C:\WINDDK\3790~1.183\bin\x86;C:\WINDDK\3790~1.183\bin;C:\WINDDK\3790~
1.183\bin\x86\drvfast\scripts;C:\Perl\bin\;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\
System32\Wbem;'
Environment: 00010000

```

```

=::=:\
=C:= C:\
=ExitCode=00000000
...
OS=Windows_NT
Path=c:\Debug.x86\winext\arcade;C:\WINDDK\3790~1.183\bin\x86;C:\WINDDK\3790~1.183\bin;C:\WINDDK\3790~1.183\bin\x86\drvfast\scripts;C:\Perl\bin\;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;
PATHEXT=.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH
PREFAST_ROOT=C:\WINDDK\3790~1.183\bin\x86\drvfast
...
_NT_TOOLS_VERSION=0x700

```

Extension command !peb зависит от текущего контекста процесса, который можно изменить с помощью одного из параметров, объясненных позже в разделе "Changing the Context".

Еще один фрагмент полезной информации - thread environment block, который можно отобразить с помощью extension command !teb.

Хотя можно отобразить TEB любого потока, указав адрес как параметр extension command, чаще всего команда расширения определяет адрес TEB из текущего потока, как видно в листинге 2.41.

Листинг 2.41. Получение TEB потока

```

0:000> !teb
TEB at 7ffdf000
ExceptionList: 0006ff34
StackBase: 00070000
StackLimit: 0006e000
SubSystemTib: 00000000
FiberData: 00001e00
ArbitraryUserPointer: 00000000
Self: 7ffdf000
EnvironmentPointer: 00000000
ClientId: 000013b4 . 00001184
RpcHandle: 00000000
Tls Storage: 00000000
PEB Address: 7ffdd000
LastErrorValue: 203
LastStatusValue: c0000100
Count Owned Locks: 0
HardErrorMode: 0

```

Extension command !teb зависит от текущего контекста потока, который можно изменить с помощью одного из параметров, объясненных позже в разделе "Changing the Context".

Win32 APIs не всегда возвращают status code вызывающему коду через возвращаемое значение или один из выходных параметров.

На самом деле большинство API сохраняет last error code в заранее выделенном, специфичном для потока месте в thread environment block, к которому программно обращаются с помощью API kernel32!GetLastError.

Значение можно проверить сразу после сбоя API с помощью extension command !gle.

Эта команда извлекает значение и отображает пользователю отформатированную строку.

Команда также отображает последний NTSTATUS error, представляющий ошибку, ранее возвращенную системным API.

```

0:000> !gle
LastErrorValue: (Win32) 0xc0000100 (203) - The system could not find the environment option
that was entered.
LastStatusValue: (NTSTATUS) 0xc0000100 - Indicates the specified environment variable
name was not found in the specified environment block.

```

Команда читает error code из контекстов текущего потока.

Последняя полезная команда в этой категории - простая клавиша <enter> или <CTRL>+M, повторяющая последние введенные команды.

Это полезно только тогда, когда последняя команда изменяет некоторое внутреннее состояние в отладчике, как происходит с командами `d` или `u`, и операция повторяется для следующего блока памяти.

Context-Changing Commands

Следующий набор команд влияет на состояние цели отладчика и обычно используется, чтобы наблюдать цель отладчика в управляемом режиме выполнения или изменить представление, интерпретируемое различными extension commands.

Tracing Code Execution

`t` - это базовая команда, используемая для пошагового выполнения кода, также известного как tracing.

Когда мы трассируем код в `assembly mode`, она проходит по одной ассемблерной инструкции за раз.

Когда отладчик работает в `source mode`, каждый шаг выполняет несколько ассемблерных инструкций, представляющих одну строку в `source mode`.

Режим можно контролировать командой `source option mode`, как видно в следующем листинге:

```
0:000> l+t
Source options are 1:
1/t - Step/trace by source line
0:000> l-t
Source options are 0:
None
```

Глава 3 объясняет механизмы, используемые отладчиком для реализации функциональности tracing в `assembly mode`.

Tracing в `source mode` возможен только в модулях, для которых доступны `private symbols`; иначе отладчик молча переключается в `assembly mode`.

Полезность tracing ограничена случаями, когда нужно внимательно отслеживать изменения регистров или выполнение кода должно войти внутрь вызова метода вместо выполнения его целиком как одного оператора, как видно в следующем листинге:

```
02sample!KBTest::Fibonacci_stdcall+0x4b:
004017ab e8b0ffffff call 02sample!KBTest::Fibonacci_stdcall (00401760)
0:000> t
02sample!KBTest::Fibonacci_stdcall:
00401760 8bff mov edi,edi
```

При трассировке многопоточного приложения любое переключение контекста потока планирует выполнение другого потока на текущем процессоре.

Во время выполнения нового потока отладчик может встретить breakpoint или другое событие, требующее внимания пользователя, и команда может вернуться с другим активным потоком и стеком.

Инженер может предотвратить переключение контекста, добавив к команде `trace` префикс с нужным номером потока.

Например, команда `~.t` выполняет один оператор в текущем потоке, тогда как остальные потоки `suspended`.

SOURCE-LEVEL TRACING VERSUS ASSEMBLY LEVEL TRACING. Многим разработчикам, использующим tracing на уровне исходного кода, очень трудно отлаживать сильно оптимизированный код, потому что отладчик прыгает вперед и назад между строками исходника.

Объяснение кроется в количестве операторов процессора, которые компилятор генерирует для каждой строки исходного кода, и в том, как они перемешиваются с кодом, соответствующим другой строке, чтобы максимально использовать процессор.

В таких случаях переход от source-level debugging к assembly-level debugging возвращает предсказуемость debugging tracing.

Stepping Over a Function Execution

Команда `p` функционально похожа на `trace command` для всех операторов, кроме вызовов функций. Команда `p` рассматривает весь вызов функции как один оператор и выполняет его целиком.

```
0:000> p
02sample!KBTest::Fibonacci_stdcall+0x4b:
004017ab e8b0ffffff call 02sample!KBTest::Fibonacci_stdcall (00401760)
0:000> p
02sample!KBTest::Fibonacci_stdcall+0x50:
004017b0 03c6 add eax,esi
```

При отладке сложного фрагмента кода мы хотим только проверить значение переменной в какой-то важной точке выполнения кода, например там, где код вызывает новую функцию.

В этой точке можно проверить как параметры функции, так и возвращаемые значения после ее выполнения.

`pc` - это команда, выполняющая код целиком до следующего вызова подпрограммы.

Она хорошо комбинируется с `p`, когда важны только результаты функции, или с `t`, когда требуется более осторожная трассировка.

Когда отладчик остановлен прямо перед вызовом функции, можно проверить все параметры, передаваемые функции.

Если необходимо, параметры можно изменить с помощью команд `e` или `r`; обычно это делается для имитации различных сбоев.

```
0:000> t
02sample!wmain:
01001c90 8bff mov edi,edi
0:000> pc
02sample!wmain+0xe:
01001c9e e81d000000 call 02sample!AppInfo::AppInfo (01001cc0)
0:000> p
02sample!wmain+0x13:
01001ca3 8d4dfc lea ecx,[ebp-4]
```

Continuing Code Execution

Когда отладчик ожидает в `command mode`, цель отладчика вообще не меняет свое состояние.

Чтобы возобновить выполнение цели отладчика, пользователь должен явно сказать отладчику продолжить выполнение.

Когда текущая остановка была вызвана исключением и отладчик очистил состояние исключения, продолжение следует выполнять с помощью формы команды, сообщаящей системе, что исключение было обработано.

Очень хорошее описание этих деталей можно найти в главе 3.

`g` - это базовая команда, используемая для освобождения цели отладчика, и она одинаково хорошо работает в отладчике пользовательского режима и режима ядра.

Это, безусловно, самая используемая команда; в простейшей форме она просто безусловно продолжает выполнение цели отладчика.

Вторая по частоте использования форма, `g <address>`, используется, чтобы продолжить выполнение цели отладчика до достижения конкретного адреса, где выполнение останавливается в отладчике.

Команда эквивалентна установке breakpoint, выполнению `debug target` до срабатывания breakpoint и удалению breakpoint.

`gu` - еще одна распространенная форма, используемая для продолжения выполнения цели отладчика до завершения текущей функции и возврата к вызывающему коду.

Команда знает текущий stack pointer, поэтому ее можно использовать для возврата из рекурсивного вызова функции.

В отладчике пользовательского режима все формы `execute command` могут быть направлены на конкретный поток вместо всего процесса.

Когда указан идентификатор потока, все потоки, кроме указанного, замораживаются до тех пор, пока цель отладчика снова не остановится в отладчике.

```
0:000> k3
ChildEBP RetAddr
0006fc64 00401792 02sample!KBTest::Fibonacci_stdcall+0x50
0006fc78 00401792 02sample!KBTest::Fibonacci_stdcall+0x42
0006fc8c 00401792 02sample!KBTest::Fibonacci_stdcall+0x42
0:000> * Execute until returning from the current function
0:000> gu
eax=00000001 ebx=7ffd9000 ecx=00000001 edx=00000000 esi=00000000 edi=00000000
eip=00401792 esp=0006fc70 ebp=0006fc78 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
02sample!KBTest::Fibonacci_stdcall+0x42:
00401792 8bf0 mov esi,eax
0:000> * Unassemble the function to find a good spot to execute to
0:000> u . l4
02sample!KBTest::Fibonacci_stdcall+0x42:
00401792 8bf0 mov esi,eax
00401794 8b5508 mov edx,dword ptr [ebp+8]
00401797 83ea02 sub edx,2
0040179a 52 push edx
0:000> * Execute until 0040179a address is reached
0:000> g 0040179a
eax=00000001 ebx=7ffd9000 ecx=00000001 edx=00000001 esi=00000001 edi=00000000
eip=0040179a esp=0006fc70 ebp=0006fc78 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
02sample!KBTest::Fibonacci_stdcall+0x4a:
0040179a 52 push edx
0:000> * Execute until returning from the current function, freezing all threads but
0.
0:000> ~0 gu
eax=00000002 ebx=7ffd9000 ecx=00000001 edx=00000001 esi=00000000 edi=00000000
eip=00401792 esp=0006fc84 ebp=0006fc8c iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
02sample!KBTest::Fibonacci_stdcall+0x42:
00401792 8bf0 mov esi,eax
```

Все `execute commands`, описанные до сих пор, имеют соответствующие кнопки на панели инструментов WinDbg.

Трассировка и наблюдение за выполнением функции

`wt` - очень полезная команда, которую можно использовать вместо команды `p` для `step over` функции.

Команда получает статистическую информацию о вызванной функции, например какие функции вызываются внутри, сколько раз они вызываются и сколько инструкций процессора выполняется внутри самой функции.

Команда принимает несколько параметров; самым важным является `nesting level` -1.

В листинге 2.42 показан вывод команды wt при выполнении конструктора 02sample!AppInfo::AppInfo.

Листинг 2.42. Trace and watch выполнения функции

```
0:000> g
Breakpoint 2 hit
02sample!wmain:
01001b90 8bff mov edi,edi
0:000> pc
02sample!wmain+0xe:
01001b9e e81d000000 call 02sample!AppInfo::AppInfo (01001bc0)
0:000> wt -11
13 0 [ 0] 02sample!AppInfo::AppInfo
13 instructions were executed in 12 events (0 from other threads)
Function Name Invocations MinInst MaxInst AvgInst
02sample!AppInfo::AppInfo 1 13 13 13
0 system calls were executed
```

Независимо от того, как возобновляется выполнение кода, processor context изменяется каждый раз, когда выполняется новая ассемблерная инструкция. Иногда context должен быть установлен явно, чтобы вычислять значения регистров или локальную переменную.

Changing the Context

Чтобы понять, как должен изменяться context, начнем с определения того, что такое context в разных ситуациях.

Самое распространенное использование термина context относится к набору регистров, представляющих состояние процессора в конкретный момент, известному как register context. Глава 3 описывает использование context применительно к exception dispatching.

Register context на момент генерации исключения сохраняется кодом exception dispatcher в стеке и может использоваться для восстановления значений регистров в момент, когда исключение было raised.

Как найти этот context?

Самый простой способ - получить его из параметров различных функций, используемых в процессе exception dispatching, или выполнить поиск информации context в стеке.

Независимо от того, как найден register context, его можно установить как текущий context с помощью команды .cxr <context address>, как показано ниже.

После выбора варианта генерации access violation exception исследование продолжилось, когда произошло access violation exception.

```
0:000> * Search for full context signature in the first 256Mb of the address space
0:000> s -d 0 L10000000/4 0001003f
0006fc1c 0001003f 00000000 00000000 00000000 ?.....
0:000> * Set the context found at this address
0:000> .cxr 0006fc1c
eax=00000000 ebx=7ffde000 ecx=00401174 edx=77c61b18 esi=7c9118f1 edi=00011970
eip=0040130a esp=0006fee8 ebp=0006fef0 iopl=0 nv up ei pl nz na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000206
02sample!RaiseAV+0x1a:
0040130a c60000 mov byte ptr [eax],0 ds:0023:00000000=??
```

После установки context все команды, зависящие от context, используют эту информацию как основу.

(k показывает стек для текущего context; dv показывает локальную переменную для текущей функции.)

В user mode context, используемый отладчиком для выполнения различных операций, также можно изменить, выбрав поток, отличный от текущего.

Отладчик идентифицирует каждый поток номером потока, который является индексом, начинающимся со значения 0.

Чтобы активировать конкретный поток, нужно использовать номер потока в команде ~<thread index>s.

После изменения все команды выполняются в context нового потока.

Некоторые команды отладчика могут получать префикс с индексом потока, чтобы выполняться в другом thread context без изменения активного потока.

Индекс потока не имеет смысла для приложения.

Приложение знает только идентификаторы потоков, полученные из различных API, которые обычно хранятся в разных местах приложения.

Вместо перечисления всех потоков, поиска индекса потока, соответствующего идентификатору потока, и использования этого индекса для всех команд, связанных с потоками, можно использовать идентификатор потока напрямую.

~~[ThreadIdentifier] - эквивалентная команда, использующая идентификатор потока.

Мы используем тот же пример с вариантом генерации stack overflow, чтобы поэкспериментировать с этими командами, как показано здесь:

```
0:002> ~
0 Id: 16cc.f80 Suspend: 1 Teb: 7ffdf000 Unfrozen
1 Id: 16cc.1248 Suspend: 1 Teb: 7ffde000 Unfrozen
. 2 Id: 16cc.10e4 Suspend: 1 Teb: 7ffdd000 Unfrozen
3 Id: 16cc.111c Suspend: 1 Teb: 7ffdc000 Unfrozen
0:002> * dot sign marks the current thread
0:002> ~0s
eax=0006fec8 ebx=00000000 ecx=0000bd09 edx=7c90eb94 esi=0006fdc8 edi=00000000
eip=7c90eb94 esp=0006fd7c ebp=0006fd9c iopl=0 nv up ei pl zr na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
ntdll!KiFastSystemCallRet:
7c90eb94 c3 ret
0:002> ~~[f80] s
eax=0006fec8 ebx=00000000 ecx=0000bd09 edx=7c90eb94 esi=0006fdc8 edi=00000000
eip=7c90eb94 esp=0006fd7c ebp=0006fd9c iopl=0 nv up ei pl zr na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
ntdll!KiFastSystemCallRet:
7c90eb94 c3 ret
0:000> * # sign is the thread that broke initially in the debugger
0:000> ~
. 0 Id: 16cc.f80 Suspend: 1 Teb: 7ffdf000 Unfrozen
1 Id: 16cc.1248 Suspend: 1 Teb: 7ffde000 Unfrozen
# 2 Id: 16cc.10e4 Suspend: 1 Teb: 7ffdd000 Unfrozen
Id: 16cc.111c Suspend: 1 Teb: 7ffdc000 Unfrozen
0:000> k
ChildEBP RetAddr
0006fd94 77370190 ntdll!KiFastSystemCallRet
0006fd98 77377fdf ntdll!NtRequestWaitReplyPort+0xc
0006fdb8 760416f4 ntdll!CsrClientCallServer+0xc2
0006fea4 760415ef kernel32!GetConsoleInput+0xd2
0006fec4 75e4f529 kernel32!ReadConsoleInputW+0x1a
0006ff04 75e4f5ef msvcrt!_getwch_nolock+0xa8
0006ff38 01001d50 msvcrt!_getwch+0x1d
0006ff50 01001cab 02sample!AppInfo::Loop+0x70
0006ff5c 01002076 02sample!wmain+0x1b
0006ffa0 76033833 02sample!__wmainCRTStartup+0x102
0006ffac 7734a9bd kernel32!BaseThreadInitThunk+0xe
0006ffec 00000000 ntdll!_RtlUserThreadStart+0x23
0:000> * dv command depends on the last .frame command
0:000> .frame 8
08 0006ff5c 01002076 02sample!wmain+0x1b
0:000> dv
argc = 1
argv = 0x001b2d58
appInfo = class AppInfo
```

В предыдущем листинге мы также используем команду `.frame`, которая изменяет context и влияет на то, какие локальные переменные отображаются с помощью команды `dv`.

Команда одинаково хорошо работает в user mode и с отладчиком режима ядра.

Команда `frame` внутренне выполняется WinDbg каждый раз, когда из окна Calls выбирается другая функция.

Когда другой поток выбирается из окна Processes and Threads, текущий context изменяется на этот поток.

Специфичными только для kernel mode являются register contexts, захваченные, когда потоки переходят в kernel mode, и идентифицируемые в каждом стеке потока как trap frames.

Каждый такой captured trap может использоваться как параметр команды `.trap`.

Все команды, используемые после этого, зависят от последнего trap context.

Каждый поток имеет собственное состояние, context которого можно установить как текущий register context независимо от его running state с помощью команды `.thread`.

Это предполагает, что цель отладчика остановлена в отладчике режима ядра, поэтому каждый thread context зафиксирован во времени.

В отладчике режима ядра каждый поток потенциально может быть частью другого процесса.

Отладчику нужна process-specific information, например информация файлов символов, чтобы интерпретировать стек и выполнять различные команды.

Это называется process context.

Если поток, исследуемый пользователем, не находится в том же процессе, который вызвал остановку, process context должен быть переключен на процесс, которому принадлежит поток.

Process context - это page directory, используемая для трансляции виртуальных адресов в физические адреса, необходимые для чтения содержимого виртуального пространства.

User mode symbols загружаются на основе текущего process context и используются до тех пор, пока отладчик не перезагрузит user mode symbols.

В результате каждый раз, когда интересующий нас поток или trap связан с другим процессом, мы должны убедиться, что process context корректен и что загружены user mode symbols, соответствующие текущему процессу.

Следующий листинг использует все эти концепции в сеансе отладчика режима ядра, остановленном в произвольном месте клавишами CTRL+C.

Поток, на котором мы сосредоточились, был выбран из списка потоков, готовых к следующему запуску, отображаемого extension command `!ready`.

```
kd> !ready
Processor 0: Ready Threads at priority 10
THREAD ffb9a020 Cid 037c.04d4 Teb: 7ffa4000 Win32Thread: 00000000 READY
kd> * Setting the current thread, change the active process and reload user mode
symbols
kd> .thread /p /r ffb9a020
Implicit thread is now ffb9a020
Implicit process is now 812532d8
.cache forcedecodeuser done
Loading User Symbols
.....
.....
kd> * Debugger tells that context has been set explicitly
kd> k
*** Stack trace for last set context - .thread/.cxr resets it
ChildEBP RetAddr
f72973f0 806f4070 nt!KiDispatchInterrupt+0x7f
f72973f0 faa0d8c7 hal!HalpDispatchInterrupt2ndEntry+0x1b
f729746c 804f82ae Ntfs!NtfsAllocateFcbTableEntry
...
kd> * Display full thread information
```

```

kd> !thread ffb9a020
THREAD ffb9a020 Cid 037c.04d4 Teb: 7ffa4000 Win32Thread: 00000000 READY
Impersonation token: e1a54278 (Level Impersonation)
Owning Process 812532d8 Image: svchost.exe
Wait Start TickCount 3721769 Ticks: 2 (0:00:00.020)
Context Switch Count 523
UserTime 00:00:00.0260
KernelTime 00:00:06.0329
Win32 Start Address schedsvc!PfSvProcessTraceThread (0x7730a597)
Start Address kernel32!BaseThreadStartThunk (0x7c810856)
Stack Init f7298000 Current f72973dc Base f7298000 Limit f7295000 Call 0
Priority 8 BasePriority 8 PriorityDecrement 0 DecrementCount 16
ChildEBP RetAddr Args to Child
f72973f0 806f4070 00000000 f7297484 faa0d8c7 nt!KiDispatchInterrupt+0x7f
f72973f0 faa0d8c7 00000000 f7297484 faa0d8c7 hal!HalpDispatchInterrupt2ndEntry+0x1b
(TrapFrame @ f72973fc)
f729746c 804f82ae 812943c8 0000001c e13afcc8 Ntfs!NtfsAllocateFcbTableEntry
f7297484 faa3c180 812943c8 f72974c8 0000000c
nt!RtlInsertElementGenericTableFullAvl+0x1f
f7297520 faa3c9ec f7297880 81294100 00004cae Ntfs!NtfsCreateFcb+0x20c
...
kd> * Set the context from a TrapFrame address
kd> .trap f7297100
ErrCode = 00000000
eax=ffbb7201 ebx=f7297228 ecx=ffb9a020 edx=ffb9a020 esi=ffbb71e8 edi=f7297230
eip=804f61b8 esp=f7297174 ebp=f72971e8 iopl=0 nv up ei pl nz na po nc
cs=0008 ss=0010 ds=0894 es=715c fs=7164 gs=7228 efl=00000202
nt!CcPinFileData+0x3ca:
804f61b8 e925abffff jmp nt!CcPinFileData+0x3fc (804f0ce2)
kd> k
*** Stack trace for last set context - .thread/.cxr resets it
ChildEBP RetAddr
f72971e8 8057a5a7 nt!CcPinFileData+0x3ca
f729725c faa34017 nt!CcPinMappedData+0xf4
f729727c faa35045 Ntfs!NtfsPinMappedData+0x4f
...
kd> * Make sure the current process and symbols are correct. .trap does not fix them
kd> .process /p /r 812532d8
Implicit process is now 812532d8
.cache forcedecodeuser done
Loading User Symbols
.....

```

Команда, используемая для проверки локальных переменных, а также стеков, сбрасывается после каждого переключения context.

Когда user mode symbols загружены некорректно, все команды, зависящие от символов, имеют непредсказуемое поведение.

Entering Value

Хотя большинство команд отладчика не являются разрушительными, возможность изменить часть памяти цели отладчика можно считать опасной.

То, что она делает, достаточно ясно: она позволяет изменить содержимое памяти по конкретному виртуальному адресу или по серии адресов.

Чаще всего мы изменяем глобальную переменную, нужную для запуска конкретного изменения в системе, или, возможно, локальную переменную, которая не была правильно инициализирована из-за какой-то ошибки.

Команда имеет несколько форм, которые нужно выбирать согласно типу данных, которые мы хотим изменить; команда eb используется для ввода серии байтов, а серию DWORDs нужно вводить с помощью команды ed.

Следующий листинг демонстрирует использование команды ed для изменения сначала локальной переменной, а затем глобальной переменной.

Следующий листинг снят после выбора варианта 6 в 02sample.exe.

```
0:000> * We want to change the input parameter for testing purposes
0:000> dv /V
0006fc60 @ebp+0x08 n = 0
0:000> * Change a dword variable using its name as address
0:000> ed n 3
0:000> * Change a dword variable using its storage address
0:000> ed @ebp+0x08 5
0:000> dv /V
0006fc60 @ebp+0x08 n = 5
0:000> * Change a dword global variable
0:000> ed kernel32!g_dwLastErrorToBreakOn 5
```

Команда достаточно мощна, чтобы изменить код, выполняемый на цели отладчика.

Хотя это не является распространенной операцией, нам нужно понимать, когда и как ее использовать.

По нашему опыту, самый распространенный случай - чрезмерно активная assert function, которая мешает продолжить конкретную операцию, а время оборота для исправления в исходном коде относительно велико.

В таких случаях мы patch цель отладчика, заменяя assert code серией NOP operations, чтобы код просто пропустил прежний assert.

```
0:000> * After returning from breakpoint we examine the previous instruction
0:000> ub . l1
02sample!KBTest::Fibonacci_stdcall+0x25:
00401785 ff1508104000 call dword ptr [02sample!_imp__DebugBreak (00401008)]
0:000> * DebugBreak call takes 6 bytes that will be replaced with opcode 90
0:000> eb -6 90 90 90 90 90
0:000> ub . L6
02sample!KBTest::Fibonacci_stdcall+0x25:
00401785 90 nop
00401786 90 nop
00401787 90 nop
00401788 90 nop
00401789 90 nop
0040178a 90 nop
```

Вооружившись минимальным набором команд, позволяющих изменять содержимое памяти, любой сеанс отладчика становится легко доступным, потому что он становится управляемым.

В следующем разделе мы описываем некоторые команды без очевидной связи с отладчиком, которые доказали, что экономят драгоценное время отладки.

Other Helper Commands

Не все команды взаимодействуют с целью отладчика, но они все равно предоставляют пользователю полезную функциональность.

Мы перечислим несколько из них вместе с примерами использования.

Одна очень распространенная ситуация, встречающаяся при отладке, - иметь на экране error code и не иметь никакого представления, что он означает.

Extension command !error принимает ошибку и пытается найти message code, связанный с ней.

```
0:000> !error 0x80070005
Error code: (HRESULT) 0x80070005 (2147942405) - Access is denied.
0:000> !error 5
Error code: (Win32) 0x5 (5) - Access is denied.
```

В некоторых случаях невозможно запустить полный GUI только для просмотра значений registry, как бывает при remote debugger sessions.

Решением является еще одна debugger extension command, !dreg, которую можно использовать для исследования значений registry на отлаживаемой машине.

Команда принимает множество параметров, очень хорошо описанных в документации отладчика или самой командой при запуске в help mode:

```
!dreg
```

Поскольку параметры, принимаемые extension command !dreg, длинные, их часто копируют из заметки или предыдущего сеанса отладки.

Нет ничего необычного в том, чтобы иметь файлы, содержащие список команд, используемых каждый раз перед исследованием каждого сеанса отладчика.

```
0:000> !dreg Software\Microsoft\Windows NT\CurrentVersion\AeDebug!*
Value: "Auto" - REG_SZ: "0"
-----
Value: "Debugger" - REG_SZ: "\"C:\WINDOWS\system32\vsjitdebugger.exe" -p %ld -e %ld"
-----
Value: "UserDebuggerHotKey" - REG_DWORD: 0 = 0x00000000
-----
```

При отладке фрагмента кода мы сталкиваемся с задачей выполнить некоторые вычисления, не слишком сложные, но трудные для ручного выполнения.

Встроенный expression evaluator можно вызвать с помощью символа вопроса (?), за которым следует математическое MASM expression, которое нужно вычислить.

Отладчик также предоставляет C++ expression evaluator, вызываемый строкой из двух вопросительных знаков (??).

Использование обоих expression evaluators похоже и предсказуемо, пока не задействованы symbolic names.

Чтобы лучше понять различия, мы рассмотрим информацию об объекте с использованием переменной-указателя this и информацию о стеке, связанную с текущим потоком.

Используемый класс имеет один integer member со смещением 4, как показано ниже:

```
class KBTTest
{
    int m_lastN;
};
```

MASM expression evaluator считает каждый символ равным его адресу памяти; другими словами, каждый символ является указателем.

Чтобы получить значение из этого места, мы должны разыменовать указатель с помощью одного из dereference expressions.

В зависимости от типа указателя для этого должны использоваться разные операторы: poi для размера указателя, специфичного для архитектуры, qwo для quad word pointer, dwo для double-word pointer, wo для word pointer и by для byte pointer.

Далее у нас есть простое выражение, используемое для показа значения member m_lastN, за которым следует выражение для вычисления размера стека текущего потока с помощью MASM expression.

```
0:000>dt this
Local var @ 0x6fee4 Type KBTTest*
0x0006ff20
+0x000 __VFN_table : 0x00401504
+0x004 m_lastN : 32
0:000> ? poi(poi(this)+4)
Evaluate expression: 32 = 00000020
0:000> ?poi(@$teb+4)-poi(@$teb+8)
Evaluate expression: 8192 = 00002000
```

То же вычисление можно выполнить с помощью C++ expression evaluator, который использует информацию о типах для выполнения необходимых indirections.

Обратите внимание, что evaluator понимает тип для каждого значения псевдорегистра.

```
0:000> ?? this->m_lastN
int 32
0:000> ?? int(@$teb->NtTib.StackBase) - int(@$teb->NtTib.StackLimit)
int 8192
```

Наконец, expression evaluator можно использовать для выполнения преобразований чисел в разных системах счисления из десятичного в шестнадцатеричный формат.

```
0:000> ? 0y1010
Evaluate expression: 10 = 0000000a
0:000> ? 0n255
Evaluate expression: 255 = 000000ff
0:000> ? 0xFF
Evaluate expression: 255 = 000000ff
```

Когда нужны более сложные преобразования, пользователь должен использовать команду `.formats`, которая показывает параметр в различных форматах, как показано ниже:

```
0:000> .formats 44444444
Evaluate expression:
Hex: 44444444
Decimal: 1145324612
Octal: 10421042104
Binary: 01000100 01000100 01000100 01000100
Chars: DDDD
Time: Mon Apr 17 18:43:32 2006
Float: low 785.067 high 0
Double: 5.65866e-315
```

Некоторые читатели спрашивают, как они могут запомнить все команды, описанные в этой главе.

Команда отладчика приходит на помощь, предоставляя простой эквивалент клавиши F1 в командной строке - команду `.hh <string>`.

Она запускает справку отладчика в режиме поиска со строкой, уже введенной в поле поиска.

Просто выберите тему, в которой вы не уверены и о которой хотите получить больше информации.

Например, команда `.hh log`, введенная в консоли отладчика, запускает справку на тему, описывающей, как пользователь может вести logs активности отладчика, чтобы позже использовать их как reference.

Множество extensions можно использовать в конкретных ситуациях; проявляйте любопытство к различным commands и extension commands, используемым в других местах этой книги.

Не забудьте проверить веб-сайт этой книги на наличие различных tips и real-life scenarios, которые мы не смогли охватить в этой книге.

Examples

При отладке приложения мы должны сочетать возможности, предоставляемые отладчиком, с нашим знанием цели отладчика, чтобы получить результаты.

Этот раздел показывает несколько распространенных случаев, демонстрирующих возможности таких комбинаций.

Conditional Breakpoints

С каждым breakpoint отладчик принимает команду, которая выполняется каждый раз, когда выполнение цели отладчика запускает этот breakpoint.

Эту возможность можно использовать для создания мощного conditional breakpoint.

Часто у нас есть функция, которая иногда дает сбой, и мы хотим остановить выполнение в этой точке и выполнить дальнейшие исследования.

Этого можно добиться, условно выполняя команду `g`, когда `error condition` не обнаружена после выполнения каждой функции.

В следующем листинге мы устанавливаем breakpoint, выполняющий такие шаги: он выполняет текущую функцию; затем проверяет результат функции; и если результат отличается от значения 1, отладчику предписывается выполнить еще одну команду `g`.

Когда функция возвращает значение 1, отладчик ожидает в `command prompt`.

```
0:000> bp 02sample!KBTest::Fibonacci_stdcall "gu;.if (eax!=1) {g}"
0:000> g
eax=00000001 ebx=00000000 ecx=00000001 edx=0100302c esi=00000001 edi=0100373c
eip=010017c2 esp=0006fccc ebp=0006fcd4 iopl=0 nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000246
02sample!KBTest::Fibonacci_stdcall+0x42:
010017c2 8bf0 mov esi, eax
```

Detecting a Reference Release

Breakpoints on access чрезвычайно полезны, например, чтобы поймать, что удерживает reference на конкретный kernel object.

Когда reference поддерживается user mode process, расследование довольно просто выполнить с помощью tools, таких как Process Explorer, доступных от Microsoft.

Если reference поддерживается kernel component, например antivirus filter driver, ни один tool не способен выяснить, что удерживает эту reference.

В этом случае лучший вариант - предположить, что reference в конечном итоге будет освобождена со временем или при shutdown системы.

Чтобы найти виновника, начните с объекта и выясните адрес object header.

Адрес используется как база для breakpoint on access со смещением 0 при отслеживании object-only reference или со смещением 4 при отслеживании handle reference.

В листинге 2.43 мы отслеживаем последнее освобождение handle, причем handle указывает на process object, представляющий экземпляр `cmd.exe`.

Мы начинаем с использования extension command `!process`, чтобы получить адрес структуры `EPROCESS` для target process.

Затем мы используем extension command `!object`, чтобы получить его header address, который используется для установки breakpoint on access.

Листинг 2.43. Поиск стека, освободившего конкретный handle

```
kd> !process 0 0 cmd.exe
PROCESS ffb1020 SessionId: 0 Cid: 01a4 Peb: 7ffd5000 ParentCid: 05d4
DirBase: 0567e000 ObjectTable: e17c2b60 HandleCount: 30.
Image: cmd.exe
kd> !object ffb1020
Object: ffb1020 Type: (812ee900) Process
ObjectHeader: ffb1008
HandleCount: 1 PointerCount: 8
kd> dt nt!_OBJECT_HEADER ffb1008
+0x000 PointerCount : 8
+0x004 HandleCount : 1
...
kd> ba w4 ffb1008+8
```

```

kd> g
Breakpoint 2 hit
nt!ObpFreeObject+0x16c:
80563f66 5e pop esi
kd> k
ChildEBP RetAddr
fafb3cd0 80563ffe nt!ObpFreeObject+0x16c
fafb3ce8 804e3c55 nt!ObpRemoveObjectRoutine+0xe7
fafb3d0c 8057e5fb nt!ObfDereferenceObject+0x5f
fafb3d24 80563ff6 nt!PspThreadDelete+0xea
fafb3d40 804e3c55 nt!ObpRemoveObjectRoutine+0xdf
fafb3d64 804f9c5c nt!ObfDereferenceObject+0x5f
fafb3d74 804e47fe nt!PspReaper+0x4a
fafb3dac 8057dfed nt!ExpWorkerThread+0x100
fafb3ddc 804fa477 nt!PspSystemThreadStartup+0x34
00000000 00000000 nt!KiThreadStartup+0x16
kd> dt nt!_OBJECT_HEADER ffb1008
+0x000 PointerCount : 0
+0x004 HandleCount : 0
...

```

Remote Debugging

Remote debugging - популярный выбор в сообществе разработчиков, потому что он позволяет иметь высокую плотность систем, доступных для тестирования, без необходимости выделять место для разработчика приложения, которому может понадобиться отлаживать эти системы. Remote debugging дает роскошь использовать личный кабинет со всей книжной полкой вокруг вместо отладки системы при физическом присутствии в удаленном месте.

Remote.exe

Самый простой метод remote debugging - перенаправить потоки консоли отладчика, STDIN и STDOUT, через утилиту remote.exe (раздел справки Remote.exe).

Remote.exe автоматически устанавливается вместе с Debugging Tools for Windows.

Remote.exe использует Windows named pipes для связи между remote server и remote client.

Клиент должен быть authenticated сервером, чтобы иметь возможность подключиться к нему.

Эта утилита не специфична для отладки и может использоваться для удаленного доступа к любой interactive command-line utility, например cmd.exe.

Командная строка, показанная в листинге 2.44, активирует remote server с именем DiskPartRemote, соответствующий консоли, выполняющей команду diskpart.exe.

Затем та же утилита remote.exe используется для подключения к серверу с помощью командной строки, предоставленной remote server при запуске (строка To Connect: в листинге 2.44).

Листинг 2.44. Удаленный доступ к консоли с помощью remote.exe

```

C:\> remote /S "diskpart" DiskPartRemote
*****
***** REMOTE *****
***** SERVER *****
*****

```

Листинг 2.44. Удаленный доступ к консоли с помощью remote.exe (продолжение)

```

To Connect: Remote /C AWD-TEST "DiskPartRemote"
Microsoft DiskPart version 5.1.3565
Copyright (C) 1999-2003 Microsoft Corporation.
On computer: AWD-TEST
DISKPART>

```

Важно отметить, что `remote.exe` использует существующую консоль для запуска командной строки, переданной как параметр, накладывая некоторые ограничения, когда вы хотите породить из нее другой `remote session`.

Например, предположим, что у вас есть доступ к `remote session`, выполняющему `cmd.exe`, и вы хотите создать другой `remote session` для вторичного выполнения `cmd.exe`.

Сначала нужно создать новую консоль с помощью `start` и передать командную строку `remote` как параметр.

В результате получится новый `remote server` для нового процесса с другим именем, тогда как первый `remote` все еще доступен.

Следующий листинг иллюстрирует последовательность команд, необходимую для порождения другого `remote session`.

```
C:\> remote /s "cmd" cmdOrigRemote
*****
***** REMOTE *****
***** SERVER *****
*****
To Connect: Remote /C AWD-TEST "cmdOrigRemote"
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.
C:\>start remote /s "cmd" cmdNewRemote
start remote /s "cmd" cmdNewRemote
C:\>
```

Debug Server

Второй вариант `remote debugging` - встроенная поддержка в отладчике, называемая `debugger server`.

Каждый отладчик имеет возможность отдавать свое управление `remote debugging clients`, используя разные `protocols`, через следующую форму командной строки (раздел справки `Activating a Debugging Server`):

```
<debugger> -server <protocol>:<protocol options> <debugger options>
```

Если отладчик уже запущен, `debugger server` можно запустить в любой момент, введя встроенную команду отладчика `.server`.

Этот вариант имеет преимущество перед командной строкой: можно поддерживать несколько `endpoints` одновременно.

Некоторые примеры использования команды `.server` показаны в листинге 2.45.

Листинг 2.45. Запуск `debugger server`

```
Command form
0:000>.server <protocol>:<protocol options>
Results
0:000> .server npipe:pipe=notepad_%i_debug
Server started. Client can connect with
<path>\<debugger>.exe -remote <options>
0 - Debugger Server - tcp:Port=6000,Server=AWD-TEST
1 - Debugger Server - tcp:Port=6001,Server=AWD-TEST
2 - Debugger Server - npipe:Pipe=notepad_debug,Server=AWD-TEST
3 - Debugger Server - npipe:Pipe=notepad_2112_debug,Server=AWD-TEST
```

`Remote debugger client`, то есть `controller`, может подключиться к `debugging server` с помощью следующей команды (раздел справки `Activating a Debugging Client`):

```
C:\><debugger> -remote <protocol>:<protocol options>
```

Параметр `<debugger>` может быть `WinDbg.exe`, `cdb.exe` или `kd.exe`, тогда как параметр `<protocol>` может быть `npipe`, `tcp`, `spipe`, `ssl` и даже `serial com port`.

Вы будете использовать тот или другой вариант в зависимости от ситуации отладки. Рассмотрим каждый protocol подробнее.

The npipe protocol

Protocol npipe (и его защищенная версия spipe) использует Windows named pipes, управляемые SMB redirector и Named Pipe File System (NPFS).

Клиент должен authenticate к SMB server так же, как любой другой клиент, с помощью системной command-line utility, как показано ниже:

```
net use \\RemoteServer\IPC$
```

Protocol npipe требует, чтобы у пользователей был набор credentials в домене, где работает debugger server.

NOTE. Debugger server может интерпретировать до двух formatting commands, %d или %x, заменяя их идентификатором процесса отладчика и идентификатором потока отладчика.

Эта возможность удобна, когда нужно attach debugger без вмешательства человека и обеспечить уникальность имени.

Например, следующие командные строки разворачиваются, как показано:

```
C:\> ntsd -server npipe:pipe=pid(%d)tid(%d) notepad
C:\> ntsd -server npipe:pipe=pid(%d) notepad
C:\> cdb -QR \\AWD-TEST
Servers on \\AWD-TEST:
Debugger Server - npipe:Pipe=pid(296)tid(608)
Debugger Server - npipe:Pipe=pid(3188)
```

TCP

TCP и его защищенная версия SSL используют TCP/IP stack и лучше всего подходят, когда authentication невозможна или нежелательна.

Debug server позволяет указать конкретный port или разрешить системе выбрать его за вас.

В качестве альтернативы можно указать range, и отладчик выберет первый port из этого range.

```
0:000> * remote using a specified port
0:000>.server tcp:port=5000
0:000> * remote using the first free port
0:000> .server tcp:port=
0:000> * remote using a range and ask the debugger to pick the first one available in
the range
0:000>.server tcp:port=5000:6000
```

Серверы, запущенные в этом случае в системе, были такими.

(Обратите внимание, что команда .servers предлагает ту же функциональность, что и командная строка <debuggers> -QR, но из консоли debugger server.)

```
0:000> .servers
On the client, use <path>\<debugger>.exe -remote <options>
0 - Debugger Server - tcp:Port=5000,Server=AWD-TEST
1 - Debugger Server - tcp:Port=4488,Server=AWD-TEST
2 - Debugger Server - tcp:Port=5001:6000,Server=AWD-TEST
```

Protocol TCP предлагает еще один параметр, clicon=<client_host>, полезный при отладке server behind firewalls, когда debugger client принимает входящее TCP/IP connection.

Следующая строка запускает debugger server и говорит ему попытаться подключиться к AWD-TEST на port 5000, а следующая строка запускает debugger client, ожидающий connection request на port 5000.

```
c:\> ntsd -server tcp:port=5000,clicon=AWD-TEST notepad 2
c:\> ntsd -remote tcp:port=5000,clicon=AWD-TEST
```

Other Commands

Другие полезные команды в remoting scenarios перечислены здесь.
(Некоторые из них уже использовались ранее в главе.)

- `.endsrv <server_id>` останавливает debugger server.
- `.servers` перечисляет debugger servers, запущенные этим отладчиком.
- `.clients` перечисляет текущих подключенных clients.
- `.remote_exit` выходит из текущего debugger client.
- `.echo` полезна для отправки текстовых сообщений другим пользователям, подключенным к тому же debugging session.

Process and Kernel Server

До сих пор вы видели remote debuggers в действии и должны хорошо понимать их и то, как их использовать.

Предыдущие методы требуют оператора с полным доступом к remote system, чтобы найти правильный process identifier, attach debugger в server mode, reattach, если процесс завершается, и так далее.

В некоторых случаях невозможно, чтобы оператор делал все это, и есть лучший способ решить проблему.

Решение представлено stand-alone debugger servers: user mode debug server, известный как process server, реализован в `dbgsvr.exe`; а kernel mode debug server, известный как KD connection server, реализован в `kdsrv.exe`.

Мы подробнее описываем user mode debug server, потому что та же идея применима к kernel mode debug server.

Process server работает на target system и, по сути, не делает ничего, кроме приема команд от remote smart clients.

Принимаемые команды похожи на то, что поддерживает debugger engine, и дают возможность отлаживать процессы на target system так же, как мы отлаживаем локальные процессы.

Process server принимает transport option как параметр, который виден при запросе target system как Remote Process Server.

```
C:\>dbgsvr -t npipe:pipe=smart_um
C:\>cdb -QR 127.0.0.1
Servers on 127.0.0.1:
Remote Process Server - npipe:Pipe=smart_um
```

После запуска process server можно использовать любые user mode debuggers как smart client с помощью параметра `-premove`, за которым следует тот же transport protocol, который использовался для запуска process server.

После transport sequence мы указываем командную строку, используемую отладчиком, так как отладчик будет выполняться локально на target system.

Ниже приведены два примера использования smart client для запуска двух debugging sessions: в первом случае process server запускает новый процесс, а во втором подключается к выполняющемуся процессу.

```
C:\>cdb -premove npipe:server=localhost,pipe=smart_um notepad
C:\>cdb -premove npipe:server=localhost,pipe=smart_um -p PID
```

В отличие от remote server scenarios, smart client выполняет все действия, влияющие на разрешение symbols и source.

Symbol source files доступны напрямую smart clients.

Большинство extensions не знает о smart client environment и работает нормально, за исключением нескольких dedicated commands, наиболее заметной из которых является команда .send_file.

WinDbg ведет себя чрезвычайно интересно, когда запускается в smart client mode без указания debugger target.

Он запускается нормально, но все существующие menu commands, такие как пункт Open Executable или пункт Attach to a process в меню File, работают против remote process server, фактически абстрагируя отношение удаленности.

Если этого недостаточно, любой smart client также можно запустить как debugger server, и он сможет принимать remote connections от ordinary clients.

Эта последняя конфигурация известна как "symbols in the middle scenario", потому что ни debugger operator, ни target system не имеют физического доступа к symbol или source files, но система посередине может иметь к ним доступ.

KD connection server работает так же, за исключением метода передачи connection string, требуемой на стороне server.

Параметр, используемый kernel debugger, чтобы стать smart client, - kdsrv, как показано здесь:

```
C:\>kdsrv -t npipe:pipe=smart_kd
C:\>cdb -QR 127.0.0.1
Servers on 127.0.0.1:
Remote Kernel Debugger Server - npipe:Pipe=smart_kd
C:\>kd -k kdsrv:server=@{npipe:server=localhost,pipe=smart_kd},
trans=@{com:port=com1}
```

Разрешение символов в сценариях удаленной отладки

Успех remote debugging зависит от symbols, доступных отладчику, а иногда и от доступности source code.

Поскольку remote debugging включает server и client, работающие в разных logon sessions, в большинстве случаев на разных компьютерах, очень важно понимать, где и как происходит symbol resolution или как source виден отладчику.

Поскольку symbols загружаются debugger server engine, то есть engine, интерпретирующим symbols и взаимодействующим с image, эти symbol files должны быть видимы и доступны этому debugger server session.

Когда debugger console предоставляется совместно с помощью remote.exe, ясно, что debugger server работает там, где запускается debugger process.

Для альтернативного метода remote debugging, где server запускается командой отладчика -server, debugger server работает там, где работает server.

Если smart client подключен к process server, debugger engine работает на smart client, и symbol files должны быть доступны ему.

Рисунок 2.6 показывает отношение между debugger client, debugger server и расположением symbols.

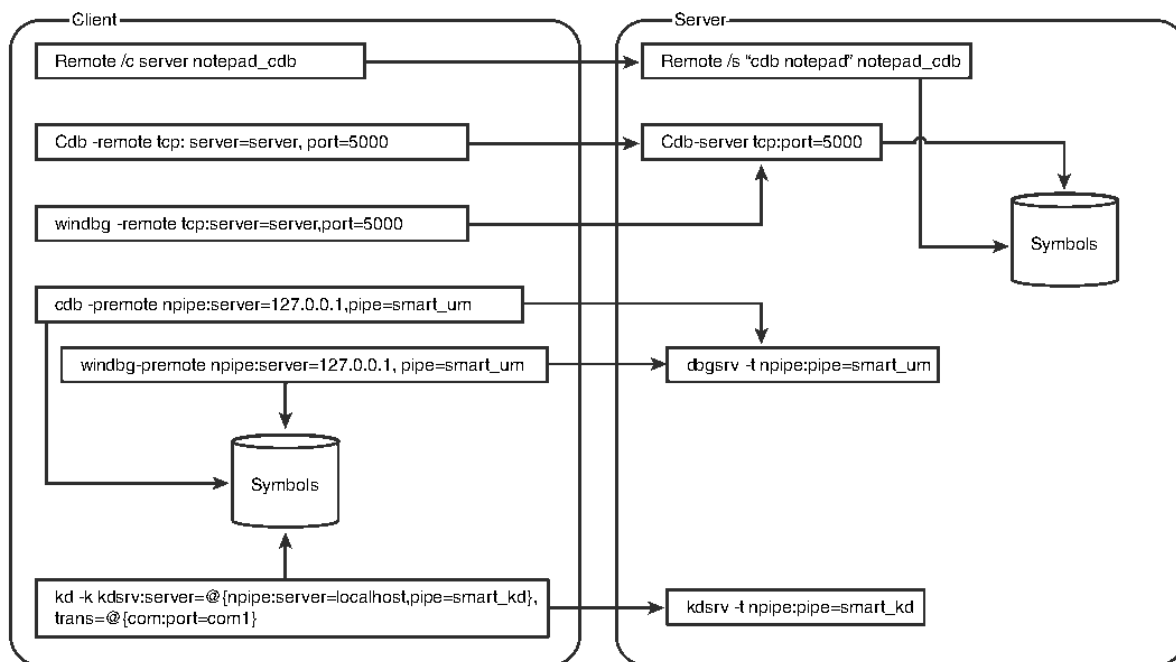


Figure 2.6 Remote debugging and symbol resolution

Рисунок 2.6. Remote debugging и symbol resolution

Когда debugger target развернут на remote server без соответствующего symbol file, а symbol нужен локально, мы должны найти способы сделать его доступным server.

В большинстве случаев мы не можем authenticate remote server к нашему client с помощью .shell net use \\client\ipc\$ /U:user password, потому что это требует ввода password в общую debugger console.

Одно решение - скопировать symbol files в удаленное расположение, видимое server без ввода новых credentials.

Интересный способ объединить все remote capabilities - использовать комбинацию normal clients и smart clients, чтобы отправить symbols на remote box.

Сценарий такой же, как раньше, и client debugger подключен к debug server.

1. Запустите process server из отладчика с помощью команды .shell, используя transport, отличный от того, который используется текущим debugger server.

```
0:000>.shell start dbgsrv.exe -t tcp:port=5001
```

1. Запустите smart client с командой неинтерактивного attach к процессу, который мы сейчас отлаживаем; в этом случае это процесс с PID, равным 3204.

```
C:\>ntsd -premove tcp:server=AWD-TEST1,port=5001 -pvr -p 3204
```

1. Используйте smart client, чтобы разрешить все symbols, необходимые для отладки, и отправить их на server с помощью команды .send_file в symbol path, используемый server. Target path является локальным для remote debugger server.

```
0:000> .send_file -s c:\temp
Copying C:\symbols\02sample.pdb\DE4335BC88FD4EA1A1714350C33B84281\02sample.pdb
(155 KB)
Copying c:\symbols\msvcrt.pdb\62B8BDC3CC194D2992DCFAED78B621FC1\msvcrt.pdb (395
KB).
Copying c:\symbols\kernel32.pdb\75CFE96517E5450DA600C870E95399FF2\kerne132.pdb
(1.52 MB).....
Copying c:\symbols\msvcpr60.pdb\3CF541551\msvcpr60.pdb (489 KB).
Copying c:\symbols\ntdll.pdb\DCE823FCF71A4BF5AA489994520EA18F2\ntdll.pdb (1.16
MB)....
```

1. Вернувшись к исходному отладчику, укажите symbol path на расположение, использованное на шаге 3, и перезагрузите symbols.

```
0:000> !sympath c:\temp
Symbol search path is: c:\temp
0:000> !reload -f
Reloading current modules
.....
0:000> lm1
start end module name
00400000 00404000 02sample (private pdb symbols) c:\temp\02sample.pdb
77ba0000 77bfa000 msvcrt (pdb symbols) c:\temp\msvcrt.pdb
77e40000 77f42000 kernel32 (pdb symbols) c:\temp\kernel32.pdb
780c0000 78121000 msvcpr60 (pdb symbols) c:\temp\msvcpr60.pdb
7c800000 7c8c0000 ntdll (pdb symbols) c:\temp\ntdll.pdb
```

Разрешение исходников в сценариях удаленной отладки

Sources обрабатываются аналогично тому, как обрабатываются symbol files; система, где работает отладчик, должна иметь доступ к source file.

Неудивительно, что WinDbg намного мощнее при работе с source files.

Он поддерживает concept of a local source path, используемый при remote debugging.

Он загружает source file на remote client, который обычно имеет более широкий доступ к source file.

Local source path поддерживается дополнительным набором команд, .lsrspath и .lsrctfix, или использованием checkbox Local в пункте Source File Path меню File.

Debugging Scenarios

Каковы самые распространенные проблемы при использовании Windows debuggers?

Самые трудные ситуации, похоже, возникают тогда, когда невозможно интерактивно управлять lifetime цели отладчика.

В таких случаях отладчик должен запускаться системой, а его конфигурация должна выполняться автоматически.

Когда отладчик запускает debugger target, мы можем запускать debugger target столько раз, сколько нужно, поскольку он полностью управляем.

Что если процесс, который нужно отлаживать, запускается другим приложением, которое нельзя изменить так, чтобы оно запускало процесс под отладчиком?

В этом случае parent application должен быть запущен под отладчиком с параметром -o, который заставляет любой новый процесс, порожденный отлаживаемым приложением, запускаться под тем же отладчиком, как показано здесь:

```
C:\>windbg -o cmd.exe /c notepad.exe
```

Тот же отладчик attach к каждому новому процессу.

Текущий процесс можно переключить с помощью команды process set, |<process number>s.

Current process number становится частью приглашения отладчика, как в следующем листинге:

```
1:001> |
0 id: 1dc8 create name: cmd.exe
. 1 id: f44 child name: notepad.exe
```

Другой параметр, реализованный операционной системой, требует изменений в registry key Image File Execution Option (известном как IFEO).

Registry key IFEO содержит несколько values, влияющих на то, как операционная система запускает executable.

Одно value в соответствующем IFEO key представляет debugger values, содержимое которых используется операционной системой для запуска executable.

В следующем примере Notepad запускается под отладчиком с command-line options -g -G:

```
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution  
Options\notepad.exe]  
"Debugger"="c:\\debug.x86\\ntsd.exe -g -G"
```

В качестве альтернативы прямому изменению registries можно использовать gflags.exe, установленный как часть Debugging Tools for Windows.

Предыдущий IFEO можно установить с помощью следующей command line:

```
C:\>gflags /p /enable notepad.exe /debug "c:\\debug.x86\\ntsd.exe -g -G"
```

После завершения расследования можно откатить изменения в registry следующим образом:

```
C:\>gflags /p /disable notepad.exe
```

После записи этих изменений в registry каждый экземпляр notepad.exe запускается под отладчиком.

Вместо запуска приложения, идентифицированного IFEO key, система запускает отладчик и передает имя приложения как параметр.

Если приложение видно пользователю, отладчик также будет виден.

Если приложение выполняется в noninteractive session, как все services, отладчик запускается, но с ним нельзя действовать, так как он не виден.

Отладка неинтерактивного процесса (службы или COM-сервера)

Хотя IFEO представляет хороший вариант для interactive processes, большинство Win32 services и COM servers выполняются в noninteractive station.

Отладчик, запущенный системой с помощью IFEO, невидим, и нам нужно найти методы подключения к debugger console.

Kernel debugger - лучший вариант в этом сценарии, а самый простой вариант - просто перенаправить debugger console в kernel debugger.

Image file execution option изменяется, как объяснялось ранее, чтобы использовать другую debugger command line, ntsd -d.

```
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\  
myService.exe]  
"Debugger"="c:\\debug.x86\\ntsd.exe -d"
```

В нескольких случаях имя процесса не является хорошим discriminator, как в случае модулей, загружаемых DllHost.exe, и вы хотите иметь возможность отлаживать только свой module.

В этом случае отладчик принимает несколько команд из command line: просит отладчик остановиться на initial breakpoint (не используйте параметр -g), raise exception при загрузке module и продолжить выполнение.

Если shared host никогда не загружает наш module, breakpoint никогда не срабатывает, и система работает нормально.

```
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\  
dllhost.exe]  
"Debugger"="c:\\debug.x86\\ntsd.exe -d -G -c "sxe ld <mymodule>;g""
```

Отладка неинтерактивного процесса (службы или СОМ-сервера) без отладчика ядра

Когда kernel debugger не подключен к target system, систему можно отлаживать с помощью remote capabilities отладчика пользовательского режима.

Debugger в server mode используется как debugger parameter в IFEO.

```
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\
dllhost.exe]
"Debugger"=" c:\debug.x86\ntsd.exe -server tcp:port=6000 -G"
```

Client подключается к debug server после запуска server process с помощью конкретной connection string.

```
C:\>windbg -remote tcp:port=6000,server=localhost
```

Этот метод плохо работает, когда debugger target реализует Windows service и отладчик выходит без предупреждения вскоре после запуска debugging session.

Это стандартное поведение Service Control Manager, также известного как SCM, если service не сообщает ему starting status в течение 30 секунд.

К счастью, этот limit можно изменить, модифицировав один registry setting, как показано здесь:

```
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control
ServicesPipeTimeout = NewTimeoutInMilliseconds
```

Что происходит, если service запускается в системе несколько раз, как в случае процесса dllhost.exe?

Поскольку каждый экземпляр отладчика открывает указанный endpoint, только первый процесс нормально запустится под отладчиком; все остальные экземпляры завершатся ошибкой, когда отладчик попытается открыть endpoint и запустить debugger server.

Решение - отложить инициализацию debugger server до тех пор, пока не будет идентифицирован target process, загружающий этот module.

Возможность указать команду, выполняемую, когда отладчик выводит prompt пользователю, позволяет нам отправить команду остановить выполнение при загрузке конкретной DLL и только затем запустить remote server.

```
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\
dllhost.exe]
"Debugger"="c:\debug.x86\ntsd.exe -d -G -c "sxe ld <mymodule>;g;.server
tcp:port=6000""
```

Все techniques, описанные здесь, можно комбинировать с параметром CLICON, упомянутым в разделе "TCP", чтобы лучше синхронизировать debugger server с debugger client.

Когда несколько процессов используют один и тот же IFEO key и все процессы должны отлаживаться с помощью debugger servers, endpoint должен создаваться динамически, но имена должны быть предсказуемыми.

Имя named pipe может быть autogen отладчиком, как показано в листинге 2.45, с discoverable name, которое позже используется на debugger client.

Следующий листинг представляет registry value, из-за которого каждый процесс dllhost.exe запускает named pipe debugger server с именем pipe \\.\pipe\dllHost_xyz.

```
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\
dllhost.exe]
"Debugger"="c:\debug.x86\ntsd.exe -d -G -c ".server npipe:pipe=dllHost_%i;g""
```

Summary

Windows debuggers - мощные инструменты, которые можно использовать для troubleshooting software problems на протяжении всего software life cycle.

На начальной фазе разработки отладчики используются для проверки корректности кода, обычно при наличии исходного кода.

Позже, после развертывания кода, software developers отлаживают dump files, создаваемые каждый раз, когда приложение падает в системе пользователя.

Благодаря своей гибкости Windows debuggers можно использовать в различных комбинациях и расширять, чтобы максимизировать продуктивность всех инженеров, участвующих в development process.

Чтобы эффективно использовать debugger, пользователь должен хорошо владеть некоторыми basic commands и быть готовым изучать новые commands или options, как того требует текущий debugging scenario.

Следующие главы вводят дополнительные commands по мере необходимости в сценариях глав.

Глава 3. Отладчики изнутри

Пакет Microsoft Debugging Tools for Windows поставляется с очень мощными tools, которые были спроектированы с целью предоставить полный контроль над целью отладчика, удерживая накладные расходы на ее выполнение на минимальном уровне.

Каждая команда, введенная в command window, выполняется без запроса подтверждения, поэтому пользователь полностью отвечает за последствия команды.

Как и с любым tool, чем больше вы о нем знаете, тем выше вероятность, что вы поймете side effects и сможете предсказать конечный результат его применения.

По нашему опыту, мы сталкиваемся со многими ситуациями, когда приложение остановлено в отладчике в одной критической точке, а любое дальнейшее продвижение приложения необратимо меняет состояние цели отладчика.

Потеря debugging session таким образом нежелательна, особенно если failure scenario очень трудно воспроизвести.

В нескольких других случаях отлаживаемый процесс является частью более крупной живой системы, и вы должны понимать влияние отладчика на этот процесс.

Иначе, скорее всего, придется перезапустить service или, в worst-case scenario, внутренние структуры окажутся повреждены, что приведет к непредсказуемому поведению.

Эта глава раскрывает часть "магии", предоставляемой отладчиками, и объясняет лежащий в основе механизм, используемый для предоставления этой магии.

В этой главе подробно описывается взаимодействие между отладчиком и operating system, а также между отладчиком и целью отладчика.

В этой главе мы исследуем:

- как работает отладчик и как он связан с выполнением кода;
- как operating system и debugger target генерируют debugger events, особенно software exceptions;
- как operating system взаимодействует с exception handling code, содержащимся в приложении;
- как отладчик управляет target и чего ожидать от каждого debugger action, введенного пользователем отладчика. Это позволяет тонко настроить debugging technique, подходящую для конкретного debugging scenario.

В этой главе используется файл 03sample.exe, который выполняет basic operations, выполняемые отладчиком, в полностью автоматизированном режиме.

Вместо требования пользовательского ввода перед переходом к следующему шагу pseudo-debugger отображает информацию о текущем состоянии и продолжает работу в заранее настроенном режиме.

Debugger target передается как command-line parameter.

Исходный код и binary расположены в следующих папках:

```
Source code: C:\AWD\Chapter3  
Binary: C:\AWDBIN\WinXP.x86.chk\03sample.exe
```

Пример повторно использует 02sample.exe, представленный в главе 2, "Introduction to the Debuggers", как debugger target.

Внутреннее устройство отладчика пользовательского режима

Как было представлено в главе 2, Microsoft Debugging Tools for Windows содержит несколько user mode debuggers и kernel mode debuggers, все они совместно используют функциональность, частично предоставляемую operating system.

Поскольку user mode debuggers являются основным tool, используемым software engineers для проверки своих предположений о code sequence, проверки корректности algorithms, а также для исследования неожиданных failures в их приложении, эта глава сосредоточена на internals user mode debuggers.

Этот раздел и большая часть текущей главы описывают, как работают user mode debuggers, и подчеркивают, как использовать каждую feature, предоставляемую отладчиками, наиболее эффективным образом.

Поддержка отладчика пользовательского режима со стороны операционной системы

Windows предоставляет небольшой набор Win32 APIs, exposing debugger support, реализованную в operating system.

User mode debuggers объединяют debugger APIs с другими general-purpose Win32 APIs, чтобы предоставить ожидаемую от них функциональность.

Эти Win32 APIs можно сгруппировать в несколько категорий на основе предоставляемой ими функциональности:

- APIs для создания debugger target;
- APIs для обработки debugger events, используемых в debugger loop;
- APIs для проверки и изменения debugger target, используемые при обработке debugger event.

Этот раздел исследует использование каждой группы APIs.

Создание цели отладчика

Live debugging session начинается с создания debugger target.

User mode debuggers могут запустить новый процесс или attach к выполняющемуся процессу, запущенному альтернативными механизмами.

После этого шага этот процесс становится новой debugger target, на которую направлены все дальнейшие действия, выполняемые отладчиком.

Operating system связывает debugger target с текущим отладчиком, и эта связь поддерживается до тех пор, пока debugger target не перестанет существовать или отладчик явно не разорвет эту связь.

Отладчики запускают новые debugger targets, передавая флаг `DEBUG_PROCESS` в вызов API `CreateProcess`, используемый для запуска нового процесса.

Примеры `03sample.exe` создают debugger target с помощью code sequence, показанной в листинге 3.1.

Имя процесса, переданное как второй параметр API `CreateProcess`, является первым command-line parameter, представленным переменной `argv[1]`.

Листинг 3.1. Пример кода, используемый для запуска процесса под user mode debugger

```
STARTUPINFOA startupInfo={0};
startupInfo.cb = sizeof(startupInfo);
PROCESS_INFORMATION processInfo = {0};
BOOL res = CreateProcess(NULL, argv[1], NULL, NULL, FALSE,
    DEBUG_PROCESS, NULL, NULL, &startupInfo, &processInfo);
```

Выполняющийся процесс может в любой момент перейти в debug state, если отладчик попросит operating system начать отладку этого процесса, attaching к нему с помощью API `DebugActiveProcess`.

Независимо от метода, использованного для создания debugger target, attaching к существующему процессу или запуска его с целью отладки, дальнейшее взаимодействие между отладчиком и operating system выполняется одинаково.

Debugger process, подключенный к debugger target таким образом, называется active debugger. У каждой debugger target может быть только один active debugger.

Цикл отладчика

Когда процесс отлаживается, notable operations, встреченные этим процессом, сигнализируются отладчику.

Dynamic library loading и unloading, создание нового потока, завершение потока и exception, thrown кодом или процессором, считаются special events, интересными для отладчиков.

Когда такое event должно быть отправлено отладчику, Windows kernel suspends все threads в процессе, уведомляет active debugger о встрече event и ожидает от него continuation command.

Большую часть времени отладчик ожидает, пока kernel вернет новые данные в ответ на API `WaitForDebugEvent`, данные, сгенерированные только если debugger target встречает одно из special debugging events, описанных ранее.

API `WaitForDebugEvent` возвращает event information в структуру `DEBUG_EVENT`, которая содержит union всех возможных event types, нужных отладчику для дальнейшей интерпретации event.

Пока отладчик исследует структуру `DEBUG_EVENT`, состояние процесса не меняется, так как каждый thread suspended.

После того как event было должным образом интерпретировано и обработано, отладчик возобновляет выполнение debugger target, вызывая API `ContinueDebugEvent`.

В ответ Windows kernel продолжает выполнение процесса, учитывая параметры API `ContinueDebugEvent`.

В зависимости от event type kernel может немедленно dismiss event и отменить его обработку для текущего event, а если event не является exception, возобновить выполнение всех threads с той точки, где они были оставлены при генерации event.

Эта последовательность операций, называемая debugger loop, продолжается до завершения debugging session, либо потому что debugger target больше не существует, либо потому что отладчик detaches от target.

Листинг 3.2 иллюстрирует такой debugger loop.

Листинг 3.2. Стандартный user mode debugger loop

```

for(DWORD endDisposition = DBG_CONTINUE; endDisposition != 0;)
{
    DEBUG_EVENT debugEvent = { 0 };
    WaitForDebugEvent(&debugEvent, INFINITE);
    endDisposition = ProcessEvent(debugEvent);
    ContinueDebugEvent(debugEvent.dwProcessId, debugEvent.dwThreadId, endDisposition);
}

```

Обработка отладочных событий

После того как debugger loop получает новое event, отладчик должен интерпретировать информацию из структуры `DEBUG_EVENT`, возможно передавая управление debugger target инженеру, использующему этот отладчик, перед возвратом в debugger loop.

Листинг 3.3 показывает очень простую processing function, которая игнорирует любую информацию внутри структуры `DEBUG_EVENT` и возвращает `DBG_CONTINUE` для каждого типа event, кроме типа `EXIT_PROCESS_DEBUG_EVENT`, когда она возвращает zero.

Для простоты return code используется и для завершения loop, и как parameter API `ContinueDebugEvent`.

Листинг 3.3. Простая обработка debugger events

```

DWORD ProcessEvent(DEBUG_EVENT& dbgEvent)
{
    switch (dbgEvent.dwDebugEventCode)
    {
        case EXCEPTION_DEBUG_EVENT:
            break;
        case CREATE_THREAD_DEBUG_EVENT:
            break;
        case CREATE_PROCESS_DEBUG_EVENT:
            break;
        case EXIT_THREAD_DEBUG_EVENT:
            break;
        case EXIT_PROCESS_DEBUG_EVENT:
            break;
        case LOAD_DLL_DEBUG_EVENT:
            break;
        case UNLOAD_DLL_DEBUG_EVENT:
            break;
        case OUTPUT_DEBUG_STRING_EVENT:
            break;
        case RIP_EVENT:
            break;
    }
    return DBG_CONTINUE ;
}

```

В следующих разделах несколько cases из switch statement в листинге 3.3 подробно рассматриваются с automated handling code, спроектированным с идеей предоставить разумное default action.

Cases, не описанные в книге, покрыты в `03sample.exe`, и их понимание оставлено читателю как упражнение.

Обратите внимание, что полноценный debugger позволяет пользователю исследовать и изменять состояние debugger target перед вызовом API `ContinueDebugEvent`.

Обработка OUTPUT_DEBUG_STRING_EVENT

Software engineers часто используют debug output commands в своем коде с целью предоставить easy-to-use tracing, требуемый для troubleshooting их кода.

Точный синтаксис различается между языками, но большинство вариантов синтаксиса в итоге вызывает один из предоставляемых Windows debugging APIs, таких как OutputDebugStringA или OutputDebugStringW.

String output, сгенерированный такими способами debugger target, может быть отображен отладчиком с помощью event processing code, похожего на показанный в листинге 3.4.

Структура DEBUG_EVENT содержит структуру OUTPUT_DEBUG_STRING_INFO, которая, в свою очередь, содержит message-specific information.

Member lpDebugStringData содержит адрес, относительный к address space debugger target, строки, которую нужно отобразить, тогда как nDebugStringLength содержит длину этой строки, а fUnicode сообщает, являются ли characters Unicode или ANSI characters.

Код использует handle процесса, где возникло event, чтобы прочитать message из address space debugger target.

Листинг 3.4. Обработка output debug string event

```
case OUTPUT_DEBUG_STRING_EVENT:
//typedef struct _OUTPUT_DEBUG_STRING_INFO {
// LPSTR lpDebugStringData;
// WORD fUnicode;
// WORD nDebugStringLength;
//} OUTPUT_DEBUG_STRING_INFO, *LPOUTPUT_DEBUG_STRING_INFO;
{
    OUTPUT_DEBUG_STRING_INFO& OutputDebug = dbgEvent.u.DebugString;
    WCHAR * msg = ReadRemoteString(hTargetProcessHandle, OutputDebug.lpDebugString-
        Data, OutputDebug.nDebugStringLength, OutputDebug.fUnicode);
    std::wcout << L"OutputDebugStringEvent\nMessage:\t";
    std::wcout << msg << std::endl;
    delete[] msg;
    break;
}
```

Функция ReadRemoteString, использованная в листинге 3.4, является helper function, абстрагирующей character size и string length из структуры OUTPUT_DEBUG_STRING_INFO, построенной вокруг kernel32!ReadProcessMemory.

Она читает строку из address space debugger target и преобразует ее в null-terminated Unicode string, как требуется 03sample.exe.

Реализация ReadRemoteString приведена в листинге 3.5.

Листинг 3.5. Чтение строки заданной длины из пространства debugger target

```
WCHAR *
ReadRemoteString(HANDLE process,LPVOID address,WORD length,BOOL unicode)
{
    WCHAR * msg = new WCHAR[length];
    if (!msg) return NULL;
    memset(msg, 0, sizeof(WCHAR)*(length));
    if ( unicode )
    {
        ReadProcessMemory(process, address ,msg, length*sizeof(WCHAR),NULL);
        return msg;
    }
    else
    {
        CHAR * originalMsg = new CHAR[length];
        if (!originalMsg)
        {
            delete[] msg;
            return NULL;
        }
        memset(originalMsg, 0, sizeof(BYTE)*(length));
```

```

        ReadProcessMemory(process, address ,originalMsg, length,NULL);
        for (WORD i = 0; i < length; i++)
        {
            msg[i] = originalMsg[i];
        }
        delete[] originalMsg;
        return msg;
    }
}

```

После отображения полученной строки в debugger console debugger loop продолжается. Debugger target продолжает выполнение после того, как отладчик снова входит в loop. Эта дополнительная активность, выполняемая debugger target, меняет timing выполнения приложения, что может скрыть или раскрыть race conditions в приложении.

Обработка EXCEPTION_DEBUG_EVENT

Debugger target может генерировать несколько exceptions за весь lifetime; каждый тип exception обрабатывается отладчиком по-разному.

Некоторые exceptions имеют специальное значение для самого отладчика, тогда как другие имеют runtime meaning для debugger target.

Debugger exception handler может быть очень сложным.

Этот раздел раскрывает только основы, необходимые для понимания exception processing, выполняемой отладчиком.

В случае EXCEPTION_DEBUG_EVENT структура DEBUG_EVENT содержит структуру EXCEPTION_DEBUG_INFO, содержащую копию exception information, упакованную как структура EXCEPTION_RECORD в member ExceptionRecord, как описано в листинге 3.6.

Из EXCEPTION_RECORD отладчик получает exception code, адрес, по которому exception была raised, и exception arguments.

Второй member EXCEPTION_DEBUG_EVENT, флаг dwFirstChance, сообщает отладчику, является ли это первым notification об этой exception.

Весь аспект first- versus second-chance (exception) notification подробно рассматривается позже в этой главе.

С точки зрения Windows operating system отладчик должен интерпретировать exception и использовать либо DBG_CONTINUE, либо DBG_EXCEPTION_NOT_HANDLED как parameter для ContinueDebugEvent.

В первом случае Windows предполагает, что exception была должным образом dismissed, условие, вызвавшее exception, больше не присутствует, и выполнение может продолжиться по адресу, вызвавшему exception.

Во втором случае Windows ведет себя так, как будто отладчика вообще нет, и продолжает свою обычную dispatching procedure.

Листинг 3.6 показывает minimal handler, использованный в design примера 03sample.exe, поэтому он не влияет на Windows exception mechanism для большинства exceptions.

Поскольку Windows operating system уведомляет отладчик о других special operations с помощью STATUS_BREAKPOINT exception, наш exception handler возвращает DBG_CONTINUE для таких exceptions.

Листинг 3.6. Обработка exception debug event

```

case EXCEPTION_DEBUG_EVENT:
//typedef struct _EXCEPTION_DEBUG_INFO {
// EXCEPTION_RECORD ExceptionRecord;
// DWORD dwFirstChance;
//} EXCEPTION_DEBUG_INFO;

```



```

std::cout << "ExceptiondebugEvent\nException Code:\t " << std::hex <<
    dbgEvent.u.Exception.ExceptionRecord.ExceptionCode;
std::cout << "\tFirstChance:\t" << dbgEvent.u.Exception.dwFirstChance
    <<std::endl;
switch (dbgEvent.u.Exception.ExceptionRecord.ExceptionCode)
{
case EXCEPTION_BREAKPOINT:
case EXCEPTION_SINGLE_STEP:
    return DBG_CONTINUE;
}
return DBG_EXCEPTION_NOT_HANDLED;

```

Return code из handling routine возвращается Windows как последний parameter API ContinueDebugEvent, имеющий имя dwContinueStatus.

Debugger Events Order

Во временном интервале между моментом, когда debugger loop возвращается из API WaitForDebugEvent, и моментом вызова API ContinueDebugEvent, debugger target не выполняется, и его состояние остается неизменным.

Пока target suspended, полная реализация debugger вошла бы в interactive mode, принимая user commands, и выполняла бы их различными средствами.

В рамках выполнения отладчик может использовать debugger APIs, чтобы узнать больше информации о debugger target и debugger event.

Он может исследовать symbol files, связанные с модулями debugger target, и использовать любой другой Win32 API для предоставления любой функциональности, запрошенной пользователем. Когда команда, введенная в debugger input lines, является execution command, отладчик вызывает ContinueDebugEvent и ожидает следующего event.

Со всей этой информацией и кодом, доступными в примере, пора получить список всех events, сгенерированных debugger target, с помощью нашего 03sample.exe.

Листинг 3.7 содержит console output, сгенерированный запуском примера, который использует xcopy.exe как parameter и debugger target.

Листинг 3.7. Debugger events, сгенерированные простым выполнением процесса (xcopy.exe)

```

C:\>C:\AWDBIN\WinXP.x86.chk\03sample.exe xcopy.exe
DebugEvent from PID.TID=33308.32256
EventType: CreateProcessDebugEvent
PID: 33308
DebugEvent from PID.TID=33308.32256
EventType: LoadDllDebugEvent
Mapped address: 7C900000
ImageName: ntdll.dll
DebugEvent from PID.TID=33308.32256
EventType: LoadDllDebugEvent
Mapped address: 7C800000
ImageName: C:\WINDOWS\system32\kernel32.dll
... More LoadDllDebugEvent ...
DebugEvent from PID.TID=33308.32256
EventType: LoadDllDebugEvent
Mapped address: 77920000
ImageName: C:\WINDOWS\system32\setupapi.DLL
DebugEvent from PID.TID=33308.32256

```

Листинг 3.7. Debugger events, сгенерированные простым выполнением процесса (xcopy.exe) (продолжение)

```

EventType: ExceptiondebugEvent
Exception Code: 80000003 FirstChance: 1
DebugEvent from PID.TID=33308.32256
EventType: LoadDllDebugEvent
Mapped address: 5CB70000

```

```

ImageName: C:\WINDOWS\system32\ShimEng.dll
... More LoadDllDebugEvent ...
DebugEvent from PID.TID=33308.32256
EventType: LoadDllDebugEvent
Mapped address: 5D090000
ImageName: C:\WINDOWS\system32\comctl32.dll
Invalid number of parameters
0 File(s) copied
DebugEvent from PID.TID=33308.32256
EventType: ExitProcessDebugEvent
ExitCode: 4

```

Листинг 3.7 показывает порядок events и заслуживает нескольких комментариев.

Первое event, полученное отладчиком при запуске debugger target, - CreateProcessDebugEvent, за которым следует серия LoadDllDebugEvents, по одному для каждой dynamic library, от которой зависит процесс.

Поскольку LoadDllDebugEvent не генерируется для самого process image, CreateProcessEvent содержит информацию, присутствующую в LoadDllDebugEvent.

Это handle executable file, starting address image, debug info pointers и имя executable image, плюс event-specific information, такая как process handle, handle первого thread или start address.

Event генерируется после того, как module был mapped в process space, и может использоваться для установки breakpoints в process code или для исследования global variables.

После того как все modules mapped в debugger target, debugger target готов к выполнению, и отладчик уведомляется, что процесс готов к запуску.

Это лучшая возможность установить breakpoints до того, как процесс фактически начнет выполняться.

Отладчик уведомляется kernel с помощью STATUS_BREAKPOINT exception (идентифицируемой exception code 0x80000003).

В этой точке sample application 03sample.exe возвращает DBG_CONTINUE, позволяя debugger target начать выполнение процесса.

Выполнение процесса продолжается загрузкой нескольких других dynamic libraries в process space, что генерирует соответствующие LoadDllDebugEvents.

Наконец, процесс выполняет свою задачу, и output объединяется в console output.

После завершения выполнения процесса target генерирует ExitDebugProcessEvent как последнее event перед исчезновением процесса.

Важно понимать порядок debugger events или распознавать ситуации, в которых отладчик не получает event.

Например, когда процесс завершается, отладчик не получает UnloadDllDebugEvent для всех dynamic libraries, все еще загруженных в процесс.

Также очень важно распознавать значение каждого exception и ситуации, в которых Windows operating system raises STATUS_BREAKPOINT exception, чтобы уведомить отладчик о special event.

Зная debugger events и порядок, в котором они принимаются в течение lifetime debugger target, в оставшейся части этой главы мы используем debugger windbg.exe с 02sample.exe как debugger target.

Управление исключениями и событиями из отладчика

Не все events созданы одинаковыми, и не все обрабатываются одинаково.

Windows debuggers перехватывают все debugger events, но то, как эти events обрабатываются отладчиком или как ими управляет пользователь, различается между event types и даже от event к event.

Большинство debugger events являются чистыми notification events, которые отладчик может ignore. Отладчик так и делает: автоматически продолжает выполнение, иногда после вывода краткого описания event.

Отладчик также может остановиться на этом event, если пользователь попросит его сделать это, позволяя пользователю взаимодействовать с системой.

Хотя большинство debugger events, показанных ранее, генерируется Windows operating system независимо от выполнения debugger target, debugger target генерирует debugger exception events как часть normal execution.

Взаимодействие между exception-handling code и debugger спроектировано так, чтобы минимизировать влияние на runtime execution flow и при этом предоставить debugger максимальную гибкость.

Debuggers могут выбирать, обрабатывать exceptions так же, как любое другое debugger event: они могут ignore их, выводить exception information на экран или break into debugger.

Debugger exception event EXCEPTION_DEBUG_EVENT может быть сгенерировано более одного раза для одной и той же exception, как описано далее в этой главе.

Первое возникновение event, называемое first-chance exception, отправляется как debugger aid, тогда как второе event, сгенерированное для той же exception, называемое second-chance exception, означает, что operating system или приложение не может handle эту exception.

Поскольку second-chance exceptions становятся unhandled exceptions, завершающими процесс, важно исследовать и понимать легитимность каждой такой exception и переоценивать желаемое поведение приложения в таких случаях.

Windows operating systems используют механизм structured exception handling (SEH), чтобы propagating exceptions, raised процессором, в kernel и в user mode applications.

Каждый SEH exception type уникально идентифицируется unsigned integer, представляющим exception code, назначаемым ему, когда exception raised в системе.

Exceptions, raised operating system, используют well-known exception codes, определенные разработчиками operating system (exceptions вроде access violation или breakpoint exception).

Другие exceptions, например C++ exceptions, также представлены в системе как structured exceptions с использованием конкретного exception code.

Информация C++ exception управляется runtime, предоставляемым compiler.

Например, C++ exceptions имеют code 0xE06D7363, access violation exceptions имеют code 0xC0000005, а breakpoint exceptions имеют code 0x80000003.

Common exception codes, которые ожидаемо будут использоваться всеми software engineers, разрабатывающими код для Windows, можно найти в headers <ntstatus.h> в WDK как constants, определенные с именами формы STATUS_<NAME>, например:

```
#define STATUS_BREAKPOINT ((NTSTATUS)0x80000003L)
```

Вы можете спросить, почему это важно для любого инженера, отлаживающего Windows code.

Ответ - чтобы использовать tools на максимальной мощности.

Правда в том, что software developers привыкли работать только с symbolic names и игнорировать value за name.

Этот indirection layer между их кодом и operating system изолирует их от изменений в operating system и делает application code легким для чтения и понимания.

Поскольку symbol files не имеют references на исходные symbolic names, debuggers отображают raw numbers, представленные symbolic names в source code.

Так как эта ситуация вряд ли изменится в ближайшем будущем и не меняется для создаваемых сегодня систем, важно познакомиться с некоторыми "magic" numbers, снова и снова встречающимися в этой книге.

Еще важнее понимать, как самостоятельно найти их meaning.

Большинство используемых exception-symbolic names также можно найти в справке отладчика, включая source header или raw value (раздел справки Specific Exception).

Events Alias

Поскольку exception codes трудно запоминать, Windows debuggers имеют сопоставленные с ними friendly aliases, которые можно использовать для управления поведением debugger.

Alias names напоминают exception type и могут использоваться взаимозаменяемо с exception codes в командах, управляющих debugger events.

Например, трудно запоминаемый C++ exception code 0xE06D7363 имеет alias eh, тогда как breakpoint exception code 0x80000003 имеет alias bpe.

DEBUGGER EVENTS AS EXCEPTIONS. Некоторые debugger events на самом деле являются exceptions, raised кодом, реализующим event behavior, как в случае initial breakpoint exceptions или output debug string events.

В таких случаях следует использовать другие hints, например stack, чтобы выяснить break reason.

Inspecting Events Break and Handling

Встроенная команда events-handling, sx, вызванная без parameters, позволяет пользователю проверить event-handling settings, используемые в соответствующем debugging session (см. листинг 3.8).

Вывод команды сгруппирован в три области: events-handling interaction с соответствующим handling mode, затем вторая группа со standard exceptions interaction и handling behavior, и, наконец, user-defined exceptions interaction и handling behavior.

Листинг 3.8. Отображение текущего event-handling state

```
0:000> sx
ct - Create thread - ignore
et - Exit thread - ignore
cpr - Create process - ignore
epr - Exit process - break
ld - Load module - output
ud - Unload module - ignore
ser - System error - ignore
ibp - Initial breakpoint - break
iml - Initial module load - ignore
out - Debuggee output - output
av - Access violation - break - not handled
asrt - Assertion failure - break - not handled
aph - Application hang - break - not handled
bpe - Break instruction exception - break
bpec - Break instruction exception continue - handled
eh - C++ EH exception - break - not handled
clr - CLR exception - second-chance break - not handled
clrn - CLR notification exception - second-chance break - handled
cce - Control-Break exception - break
cc - Control-Break exception continue - handled
cce - Control-C exception - break
cc - Control-C exception continue - handled
dm - Data misaligned - break - not handled
```

Листинг 3.8. Отображение текущего event handling state (продолжение)

```
dbce - Debugger command exception - ignore - handled
gp - Guard page violation - break - not handled
ii - Illegal instruction - second-chance break - not handled
ip - In-page I/O error - break - not handled
dz - Integer divide-by-zero - break - not handled
```

```

iov - Integer overflow - break - not handled
ch - Invalid handle - break
hc - Invalid handle continue - not handled
lsq - Invalid lock sequence - break - not handled
isc - Invalid system call - break - not handled
3c - Port disconnected - second-chance break - not handled
sse - Single step exception - break
ssec - Single step exception continue - handled
sbo - Stack buffer overflow - break - not handled
sov - Stack overflow - break - not handled
vs - Verifier stop - break - not handled
vcpp - Visual C++ exception - ignore - handled
wkd - Wake debugger - break - not handled
wob - WOW64 breakpoint - break - handled
wos - WOW64 single step exception - break - handled
* - Other exception - second-chance break - not handled
Exception option for:
12345678 - break - not handled

```

Adjusting Event Break and Handling

Поскольку exceptions полезны, если мы можем break program execution в момент возникновения event, этот раздел показывает, как управлять поведением debugger из interactive prompt.

В самой общей форме синтаксис этой команды следующий:

```

sx{e|d|i|n} [-c "Cmd1"] [-c2 "Cmd2"] [-h] {Exception|Event|*} [parameter]

```

где:

- `sxe` (set exceptions enable) используется, чтобы включить debugger break на events.
- `sxd` (set exceptions disable) используется, чтобы отключить debugger break на events.

Хотя first-chance exception не останавливает выполнение, second chance останавливается в debugger, и message отображается на экране как обычно для этого конкретного event.

- `sxn` (set exceptions notify) используется, чтобы отключить debugger break (либо first-, либо second-chance exception), но все равно выводить message на экран.

Побочный эффект состоит в том, что debugger входит в continuous loop.

Operating system уведомляет debugger о first chance exception; debugger выводит message и продолжает target execution.

Если handler не найден, debugger получает second-chance notification.

При continuation debugger снова получает first-chance exception, и процесс повторяется до тех пор, пока debugger не получит другое event.

- `sxi` (set exceptions ignore) используется, чтобы полностью "ignore" exception (либо first-, либо second-chance exception); exception handled точно так же, как в случае `sxn`.
- `-c` - parameter, содержащий command, выполняемую, когда debugger получает новое debugger event.

Когда это event является exception event, parameter влияет только на first-chance exception.

Поскольку command выполняется до обработки event отладчиком, она никогда не должна содержать statement `g` (`go`).

- `-c2` - parameter, содержащий command, выполняемую, когда second-chance exception dispatched to debugger. Поскольку command выполняется до обработки event отладчиком, она никогда не должна содержать statement `g` (`go`).
- `Exception|Event|*` представляет event alias, exception alias или exception code, например `ct` для create thread event или `av` для access violation exception.

Вместо alias можно использовать exception code, например `0xC0000005`.

Символ star (*) представляет все остальные exceptions, идентифицируемые exception code, а не alias.

- `parameter` содержит `parameters`, специфичные для `event`. Например, `DllLoadEvent` можно ограничить одной или несколькими `dynamic libraries`, указанными в `parameter`. Чтобы `break application` при загрузке `ole32.dll`, `event DllLoadEvent` должен быть `configured` следующей командой.

```
0:000>sxe ld:ole32.dll
```

- `-h` - `parameter`, который инструктирует `debugger` изменить `handling behavior` вместо `break behavior`. Как описано в начале этой главы, после получения `exception event` `debugger` должен вернуть `handling state operating system`, так называемый `continuation disposition`. Поскольку явной `option` для указания `handling state` не существует, она `inferred` из команды так: `sxe` означает, что `exception handled`; все остальное означает, что `exception не handled` отладчиком.

Другая `interactive command`, `sxr` (`structured exception reset`), должна использоваться для `reset` всех `event breaks` и `handlings` к `default values`.

WHAT IS THE DIFFERENCE BETWEEN SSE AND SSEC? После внимательной проверки всех возможных `events` можно увидеть `exception pairs`, например `sse` (`single step exception`), за которым следует `ssec` (`single-step exception continuation`).

Это разделение не имеет поддержки со стороны `operating system`, интерпретируется только `debugger engine` и создано лишь для того, чтобы легко вывести `break` и `handling state` в `command line` как два разных `events`.

Настройка остановки и обработки событий из GUI WinDbg

Хотя `command window` дает всю гибкость мира, большинство людей предпочитает использовать `WinDbg UI`, чтобы изменить `event break` и `handling state`.

`Options` доступны через выбор пункта `Event Filters` в меню `Debug` из любого `debugging session`, выполненного с помощью `WinDbg`, как показано на рисунке 3.1.

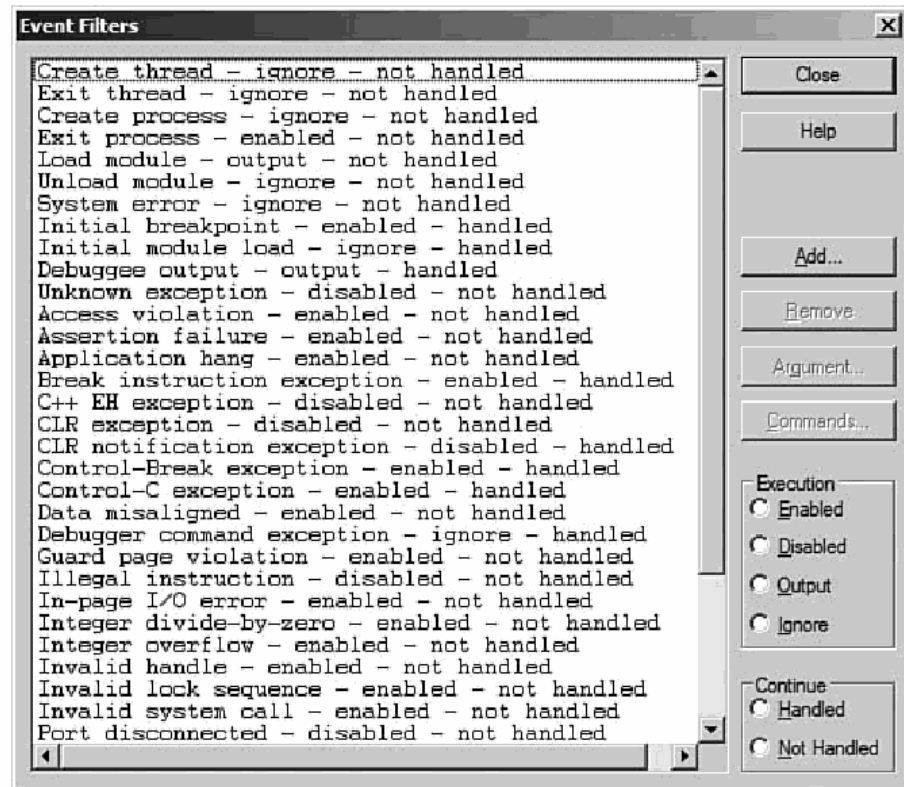


Figure 3.1 WinDbg.exe Event Filters window

Рисунок 3.1. Окно WinDbg.exe Event Filters

Все options, доступные в command line, также доступны в окне Event Filters.

Event command strings (-c и -c2) можно изменить, нажав кнопку Commands, break status можно изменить с помощью radio buttons Execution, handling state можно изменить с помощью radio buttons Continue, а event parameters можно добавить с помощью кнопки Argument.

Команды изменения event break и handling state влияют на event, выбранное из main list.

New exception codes также можно добавить или удалить из main list с помощью кнопок Add и

Remove соответственно, если debugger target использует exception codes, не показанные в этом list.

Настройка стандартной остановки и обработки событий

Знание того, как управлять event break и handling state в interactive mode, позволяет в любой момент подстроить debugging environment под debugging needs.

В некоторых случаях default event-handling settings не подходят для debugging situation.

Например, произвольный module, используемый для управления media licenses в Digital Rights Management (DRM) system, нельзя отлаживать с normal debugger settings, поскольку он использует различные anti-debugging tricks, такие как handled access violations, handled debug breakpoints и так далее.

Неудивительно, что такие anti-debugging tricks используют side effects, внесенные debugger в behavior процесса.

В этом случае software engineer должен использовать другие способы настройки event break и handling defaults под specific debugging needs.

Самый распространенный способ настроить defaults - использовать command-line parameters, описанные в таблице 3.1.

Таблица содержит command-line option и equivalent interactive command, а также описание command.

Таблица 3.1. Command-Line Parameter, сопоставленный с Interactive Commands

Parameter	Interactive Command	Description
-g	sxd ibp	Не останавливаться при process start-up
-G	sxd epr	Не останавливаться при process termination
-xe <event>	sxe <event>	Break on <event> occurrences
-xd <event>	sxd <event>	Не break on <event> occurrences
-xi <event>	sxi <event>	Ignore all <event> occurrences
-xn <event>	sxn <event>	Notify on <event> occurrences
-x	sxd av	Не break on access violation

Чтобы проиллюстрировать mapping между command-line parameters и interactive commands, следующая command line имеет такой effect:

```
C:\>windbg -g -xe ld:kernel32* -xd av <debugger target>
```

- -g disables initial breakpoint.
- -xd av disables access violation breaks.
- -xe ld:kernel32 breaks после mapping kernel32.dll в address space. Library name может содержать wildcards. Например, string ld:msvc* matches все различные versions и flavors C runtime library.

Другой вариант установки initial debugging environment для command-line debugger - initialization file, читаемый debugger при start-up.

Initialization file называется tools.ini, а его folder location указывается environment variable с именем INIT.

Например, чтобы получить то же behavior, что и у предыдущей command line для ntsd.exe, tools.ini должен содержать строки, показанные в листинге 3.9.

Листинг 3.9. Содержимое tools.ini

```
[NTSD]
sxd: av
sxd: ibp
sxe: ld kernel32.dll
```

Debugger WinDbg загружает эти defaults, а также другие runtime parameters, из workspace file, созданного либо явно пользователями, либо неявно при завершении debugger session. Workspaces очень хорошо покрыты в debugger reference (раздел справки Workspaces).

SAVING THE ENVIRONMENT. WinDbg сохраняет последние debugger settings и перезагружает их при начале нового session.

Хотя это не совсем способ управления environment, он предлагает довольно приятный experience для casual debugger users.

Debugger Events

Этот раздел берет несколько events из листинга 3.8, анализирует их в debugger console, отмечает особенности и дает советы по их использованию.

Поскольку следующий раздел посвящен exceptions, здесь основное внимание уделено actionable debugger events: созданию process debug event, выходу process debug event, загрузке DLL debug

event, выгрузке DLL debug event, созданию thread debug event и выходу thread debug event.

Create a Process Event (cpr)

Event cpr, которое не следует путать с initial breakpoint event, automatically handled Windows debuggers.

При необходимости automatic handling можно отключить из debugger command line.

Это event raised до загрузки dynamic libraries, от которых зависит process, в process address space. В этот момент все global variables, требующие explicit initialization, еще не initialized, тогда как plain old data variables заполнены своими default values.

Это первая возможность пользователя debugger выполнить различные commands, например установить breakpoints или unassemble functions в process image.

Это типичное время для включения load notification для dynamic library, от которой зависит process.

Initial Breakpoint Event (ibp)

После загрузки dependent libraries в process система генерирует другое exception, обозначающее initial breakpoint.

Initial breakpoint raised прямо перед началом выполнения процесса.

В этот момент мы можем установить breakpoint в constructor, используемый для initialization одной global variable, или установить breakpoints в любой function, реализованной в process image, например в function main.

Если initial breakpoint не нужен, можно переопределить event handling с помощью command line parameter -g.

Раздел "Debugging Scenarios" в главе 2 содержит хороший пример того, как initial breakpoint можно использовать для облегчения automation tasks.

Следует заметить, что initial breakpoint не выглядит иначе, чем обычный breakpoint, и event должен быть идентифицирован путем проверки stack на текущем breakpoint, как показано в листинге 3.10.

Первые два числа, отображаемые командой .lastevent, являются process identifier и thread identifier, raised event.

Листинг 3.10. Stack trace initial breakpoint для любого процесса, запущенного под debugger

```
0:000> .lastevent
Last event: 13b4.184: Break instruction exception - code 80000003 (first chance)
0:000> k
ChildEBP RetAddr
0007fb1c 7c93edc0 ntdll!DbgBreakPoint
0007fc94 7c921639 ntdll!LdrpInitializeProcess+0xffa
0007fd1c 7c90eac7 ntdll!_LdrpInitialize+0x183
00000000 ntdll!KiUserApcDispatcher+0x7
```

Exit a Process Event (epr)

Перед завершением debugger target отладчик получает последнее notification в форме event epr, event, распознаваемого командой .lastevent.

Команда .lastevent использует event information для отображения process exit code, как показано в листинге 3.11.

Event не handled по умолчанию, но это можно переопределить, запустив debugger с command-line parameter -G.

Листинг 3.11. Последнее event для любого процесса, запущенного под debugger

```

0:000> .lastevent
Last event: 1674.c80: Exit process 0:1674, code 0
0:000> k
ChildEBP RetAddr
0007fde4 7c90e89a ntdll!KiFastSystemCallRet
0007fde8 7c81ca5e ntdll!NtTerminateProcess+0xc
0007fee4 7c81cab6 kernel32!_ExitProcess+0x62
0007fef8 77c39d45 kernel32!ExitProcess+0x14
0007fff04 77c39e78 msvcrt!__crtExitProcess+0x32
0007fff14 77c39e90 msvcrt!_cinit+0xee
0007fff28 01007522 msvcrt!exit+0x12
0007ffc0 7c816d4f notepad!WinMainCRTStartup+0x185
0007fff0 00000000 kernel32!BaseProcessStart+0x23

```

Load a Module Event (ld)

ld генерируется Windows operating system сразу после того, как dynamic library mapped в process memory, но до выполнения library initialization code.

Это единственная возможность установить breakpoints в library initialization code, включая initialization global variables, или понять, почему именно эта library brought into process space. Последнее можно понять, проверив call stack этого event, как показано в листинге 3.12.

Листинг 3.12. Stack trace после загрузки dynamic link library

```

0:000> .lastevent
Last event: 43c.b18: Load module C:\WINDOWS\system32\ShimEng.dll at 5cb70000
0:000> k
ChildEBP RetAddr
0007f72c 7c90dc61 ntdll!KiFastSystemCallRet
0007f730 7c91c3da ntdll!NtMapViewOfSection+0xc
0007f824 7c916071 ntdll!LdrpMapDll+0x330
0007fae4 7c924a07 ntdll!LdrpLoadDll+0x1e9
0007fb10 7c9216b6 ntdll!LdrpLoadShimEngine+0x28
0007fc94 7c921639 ntdll!LdrpInitializeProcess+0x1079
0007fd1c 7c90eac7 ntdll!_LdrpInitialize+0x183
00000000 00000000 ntdll!KiUserApcDispatcher+0x7

```

Unload a Module Event (ud)

Event ud генерируется после того, как dynamic library unmapped из address space в результате вызова FreeLibrary (см. листинг 3.13).

Это event может быть полезно для отслеживания unload order dynamic link library, если это необходимо.

Листинг 3.13. Оценка event ud

```

0:000> .lastevent
Last event: 138c.cbc: Unload module C:\WINDOWS\System32\MSXML3.DLL at 74980000
0:000> k
ChildEBP RetAddr
0007fc28 7c90e96c ntdll!KiFastSystemCallRet
0007fc2c 7c91e7d3 ntdll!NtUnmapViewOfSection+0xc
0007fd1c 7c80aa7f ntdll!LdrUnloadDll+0x31a
0007fd30 77513442 kernel32!FreeLibrary+0x3f
0007fd3c 77513456 ole32!CClassCache::CDllPathEntry::CFinishObject::Finish+0x2f
0007fd50 77530729 ole32!CClassCache::CFinishComposite::Finish+0x1d
0007fe10 7752fd6a ole32!CClassCache::CleanupDlIsForProcess+0x1b2
0007fe14 7752fee4 ole32!ProcessUninitialize+0x37
0007fe28 774fee88 ole32!wCoUninitialize+0x11b
0007fe44 01035966 ole32!CoUninitialize+0x5b
0007ff44 0103caab WMIC!wmain+0x8af
0007ffc0 7c816d4f WMIC!wmainCRTStartup+0x125
0007fff0 00000000 kernel32!BaseProcessStart+0x23

```

Create a Thread Event (ct)

Event ct генерируется при создании нового thread (см. листинг 3.14).

К сожалению, в этом event нет полезной информации, такой как stack creator thread или creator thread identifier.

Тем не менее это event может быть очень полезно для debugging thread lifetime issues в thread pool code.

Однако breakpoint, установленный на вызовы kernel32!CreateThread, часто достаточен для определения execution path, ведущего к созданию thread.

Листинг 3.14. Оценка event ct

```
0:001> .lastevent
Last event: 1494.1220: Create thread 1:1220
0:001> k
ChildEBP RetAddr
0007cea4 00090178 kernel32!BaseThreadStartThunk
WARNING: Frame IP not in any known module. Following frames may be wrong.
0007cea4 00000000 0x90178
```

Exit a Thread Event (et)

Event et генерируется, когда running thread terminated.

Ero stack back-trace дает clues, почему thread terminated.

Например, thread из листинга 3.15 естественно exits, когда это определено idle-detection mechanism thread pool в ole32.dll.

Листинг 3.15. Оценка event et

```
0:003> .lastevent
Last event: 1494.11ac: Exit thread 3:11ac, code 0
0:003> k
ChildEBP RetAddr
011eff50 7c90e8af ntdll!KiFastSystemCallRet
011eff54 7c80cd04 ntdll!NtTerminateThread+0xc
011eff94 7c80cebf kernel32!ExitThread+0x8b
011effa0 774fe45d kernel32!FreeLibraryAndExitThread+0x28
011effb4 7c80b50b ole32!CRpcThreadCache::RpcWorkerThreadEntry+0x34
011effec 00000000 kernel32!BaseThreadStart+0x37
```

Механизм структурированной диспетчеризации исключений

Exception - это event, возникающее во время выполнения кода либо в результате event, встреченного CPU при выполнении кода, такие events известны как hardware exceptions, либо из-за explicit instructions to raise exception, известных как software exceptions.

Hardware exceptions - это механизмы, используемые CPU для сигнализации errors, встреченных при выполнении instruction stream, например при встрече invalid instruction или выполнении breakpoint statement.

Поскольку в коде нет explicit statement для raise exception, compiler documentation часто называет такие hardware exceptions asynchronous exceptions.

С другой стороны, software exceptions raised путем передачи exception information вместе с desired handling mode в user mode API kernel32!RaiseException.

High-level languages, такие как C++ или .NET languages, используют этот механизм для throw exceptions и полагаются на operating system для их корректного dispatch.

Поскольку compilers знают, что statement throw вносит discontinuity в code execution, такие exceptions известны как synchronous exceptions.

Оставшаяся часть этой главы использует 02sample.exe как debugger target.

Пример является collection of bad practices: код обращается к invalid addresses, raises exceptions и не handles их, и так далее.

Каждое такое bad behavior можно выбрать из application menu.

Например, при использовании варианта 3 sample имитирует ситуацию unhandled C++ exception.

Exception Structures

Чтобы сделать exception handling mechanism единообразным во всей operating system, Windows operating systems объединяют обе концепции и обрабатывают все exceptions как structured exceptions, независимо от их source.

Эта uniformity начинается с использования common data structures для передачи exception record information между operating system и exception handlers.

Структура _EXCEPTION_POINTERS, определенная в <winnt.h>, содержит pointer на exception record и еще один pointer на processor context в момент, когда exception была raised, как показано ниже:

```
struct _EXCEPTION_POINTERS {
    EXCEPTION_RECORD *ExceptionRecord,
    CONTEXT *ContextRecord
}
```

EXCEPTION_RECORD определена в <winnt.h> и приведена в листинге 3.16.

Позже та же структура передается operating system отладчику, где информация, сохраненная внутри структуры, используется для интерпретации и представления exception information пользователю.

Листинг 3.16. Структура EXCEPTION_RECORD, определенная в header <winnt.h>

```
typedef struct _EXCEPTION_RECORD {
    DWORD ExceptionCode;
    DWORD ExceptionFlags;
    struct _EXCEPTION_RECORD *ExceptionRecord;
    PVOID ExceptionAddress;
    DWORD NumberParameters;
    ULONG_PTR ExceptionInformation[EXCEPTION_MAXIMUM_PARAMETERS];
} EXCEPTION_RECORD;
```

Поскольку большинство exceptions нефатальны, особенно debugger breakpoint statements, operating system должна capture processor state в exception location, чтобы resume code execution, если ее попросят сделать это.

Processor state хранится в processor architecture-specific structure, называемой exception context, которая содержит все register values и определена в <winnt.h>.

Первый member структуры описывает тип структуры CONTEXT (см. листинг 3.17).

Листинг 3.17. Структура CONTEXT, как определено в MSDN

```
typedef struct _CONTEXT {
    DWORD ContextFlags;
    ...
} CONTEXT,
```

Поле ContextFlags принимает значение из constants, определенных в том же header <winnt.h>. Например, возможные constant values для семейства процессоров x86 показаны в листинге 3.18.

Полный exception context для типичного приложения, выполняющегося на x86 processor, всегда начинается с 0x0001003f, что представляет constant CONTEXT_ALL.

Такой signature очень полезен при поиске stack content и попытке понять meaning конкретного memory block.

Мы можем установить context, распознанный таким образом, как current thread context, чтобы понять, каким был processor state перед raising exception.

Листинг 3.18. Значения x86 context flags

```
#define CONTEXT_i386 0x00010000 // this assumes that i386 and
#define CONTEXT_CONTROL (CONTEXT_i386 | 0x00000001L) // SS:SP, CS:IP, FLAGS,
BP
#define CONTEXT_INTEGER (CONTEXT_i386 | 0x00000002L) // AX, BX, CX, DX, SI,
DI
#define CONTEXT_SEGMENTS (CONTEXT_i386 | 0x00000004L) // DS, ES, FS, GS
#define CONTEXT_FLOATING_POINT (CONTEXT_i386 | 0x00000008L) // 387 state
#define CONTEXT_DEBUG_REGISTERS (CONTEXT_i386 | 0x00000010L) // DB 0-3,6,7
#define CONTEXT_EXTENDED_REGISTERS (CONTEXT_i386 | 0x00000020L) // cpu-specific
extensions
#define CONTEXT_FULL (CONTEXT_CONTROL | CONTEXT_INTEGER | \
CONTEXT_SEGMENTS)
#define CONTEXT_ALL (CONTEXT_CONTROL | CONTEXT_INTEGER | CONTEXT_SEGMENTS |
CONTEXT_FLOATING_POINT | CONTEXT_DEBUG_REGISTERS | CONTEXT_EXTENDED_REGISTERS)
```

Exception Life Cycle

Hardware event принудительно передает processor control от текущей выполняемой program к system routines, которые обрабатывают interrupt events.

Эти routines называются interrupt handlers и устанавливаются operating system.

После того как processor state переключается в kernel mode, kernel сохраняет processor state в trap context, который можно использовать для проверки processor state до transition.

Листинг 3.19 показывает call stack потока сразу после того, как он raised exception.

Process, throwing exceptions, был запущен под user mode debugger с помощью command line windbg.exe 02sample.exe.

Exception raised выбором варианта 3.

Затем process останавливается в debugger, который, в свою очередь, ждет user input.

Thread фактически blocked, пока Windows operating system dispatches exception information to debugger, что мы можем увидеть, используя kernel mode debugger в этом состоянии.

Мы идентифицируем process с помощью extension command !process и extension command !thread, чтобы интерпретировать stack единственного thread процесса.

Листинг 3.19. Exception, dispatched to user mode debugger

```
kd> !process 0 4 02sample.exe
PROCESS ff68a020 SessionId: 0 Cid: 0a7c Peb: 7ffdd000 ParentCid: 0a70
DirBase: 03912000 ObjectTable: e180e158 HandleCount: 7.
Image: 02sample.exe
THREAD ffa7d868 Cid 0a7c.0a78 Teb: 7ffdf000 Win32Thread: 00000000 WAIT
kd> !thread ffa7d868
THREAD ffa7d868 Cid 0a7c.0a78 Teb: 7ffdf000 Win32Thread: 00000000 WAIT: (Executive)
KernelMode Non-Alertable
SuspendCount 1
f7cf3490 SynchronizationEvent
Not impersonating
DeviceMap e19f85a0
Owning Process ff68a020 Image: 02sample.exe
Wait Start TickCount 14796478 Ticks: 1035 (0:00:00:10.364)
Context Switch Count 44
UserTime 00:00:00.0000
KernelTime 00:00:00.0290
Win32 Start Address 02sample!mainCRTStartup (0x0040183d)
```

```

Start Address kernel32!BaseProcessStartThunk (0x7c810867)
Stack Init f7cf4000 Current f7cf3414 Base f7cf4000 Limit f7cf1000 Call 0
Priority 10 BasePriority 8 PriorityDecrement 0 DecrementCount 16
ChildEBP RetAddr Args to Child
f7cf342c 804dc6a6 ffa7d8d8 ffa7d868 804dc6f2 nt!KiSwapContext+0x2e ( )
f7cf3438 804dc6f2 00000000 ffa7d868 f7cf3488 nt!KiSwapThread+0x46

```

Листинг 3.19. Exception, dispatched to user mode debugger (продолжение)

```

f7cf3460 8065879b 00000000 00000000 00000000 nt!KewaitForSingleObject+0x1c2
f7cf3540 80659903 ff68a020 00000000 f7cf3578 nt!DbgkpQueueMessage+0x17c
f7cf3564 8060fed2 f7cf3578 00000001 f7cf3d64 nt!DbgkpSendApiMessage+0x45
f7cf35f0 804fc914 f7cf39d8 00000001 00000000 nt!DbgkForwardException+0x8f
f7cf39b0 804fcbfe f7cf39d8 00000000 f7cf3d64 nt!KiDispatchException+0x1f4
f7cf3d34 804e297d 0006fe48 0006fb64 00000000 nt!KiRaiseException+0x175
f7cf3d50 804df06b 0006fe48 0006fb64 00000001 nt!NtRaiseException+0x31
f7cf3d50 7c81eb33 0006fe48 0006fb64 00000001 nt!KiFastCallEntry+0xf8 (TrapFrame @
f7cf3d64)
0006fe98 77c2272c e06d7363 00000001 00000003 kernel32!RaiseException+0x53
0006fed8 004012c5 0006feec 00401d38 004012b0 msvcrt!_CxxThrowException+0x36
0006fef0 00401471 00011970 7c9118f1 7ffdd000 02sample!RaiseCPP+0x25
0006ff44 0040196c 00000002 00262588 00262a58 02sample!wmain+0xe1
0006ffc0 7c816d4f 00011970 7c9118f1 7ffdd000 02sample!mainCRTStartup+0x12f
0006fff0 00000000 0040183d 00000000 78746341 kernel32!BaseProcessStart+0x23
kd> .trap f7cf3d64
ErrCode = 00000000
eax=0006fe48 ebx=7ffdd000 ecx=00000000 edx=002625b0 esi=0006fed8 edi=0006fed8
eip=7c81eb33 esp=0006fe44 ebp=0006fe98 iopl=0 nv up ei pl nz na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000206
kernel32!RaiseException+0x53:
001b:7c81eb33 5e pop esi
kd> k
*** Stack trace for last set context - .thread/.cxr resets it
ChildEBP RetAddr
0006fe98 77c2272c kernel32!RaiseException+0x53
0006fed8 004012c5 msvcrt!_CxxThrowException+0x36
0006fef0 00401471 02sample!RaiseCPP+0x25
0006ff44 0040196c 02sample!wmain+0xe1
0006ffc0 7c816d4f 02sample!mainCRTStartup+0x12f
0006fff0 00000000 kernel32!BaseProcessStart+0x23

```

Handler использует trap information и, возможно, другую information, полученную от processor, чтобы создать две части information: exception record, описывающую encountered exception, и exception context, содержащий state processor в момент, когда processor encountered эту exception. Обратите внимание, что trap frame information (показанная в первой kernel function из предыдущего stack как TrapFrame), captured в точке transition into kernel mode, может использоваться как context information для команды .trap, как показано в листинге 3.19.

Software exceptions иницируются explicit call в kernel mode с помощью undocumented API ntdll!NtRaiseException, вызываемого public API kernel32!RaiseException. ntdll!NtRaiseException создает exception record и captures process state в exception context. С exception record и exception context kernel готов dispatch exception с помощью exception-dispatching mechanism, похожего на hardware exceptions.

Dispatching process начинается в kernel mode и позже продолжается в user mode или kernel mode, matching mode, active when exception was encountered.

Все exceptions, encountered in kernel mode, должны быть handled; иначе такая exception вызывает bug check (также известный как blue screen errors или BSOD), например:

```
bug check 0x8E: KERNEL_MODE_EXCEPTION_NOT_HANDLED
```

Когда exception information captured, как описано ранее, operating system запускает exception-dispatching routine.

В рамках этой routine Windows operating system выполняет несколько activities:

- пытается вызвать все registered handlers, пока exception не будет handled;

- предоставляет additional functionality, например exception logging;
- в конечном счете решает, что делать с любой unhandled exception.

Эта complex functionality, предоставляемая Windows operating system, выполняется почти silently. Мы используем "almost", потому что exception dispatching относительно expensive по сравнению с normal code execution.

Пока exceptions не raised как часть normal execution flow, overall cost dispatching exception negligible.

Диспетчеризация исключений

Операционная система Windows учитывает наличие отладчика при диспетчеризации исключения, то есть отладчика пользовательского режима, подключенного к процессу, который сгенерировал исключение, или отладчика режима ядра, подключенного к системе, где возникло исключение. Область этого раздела ограничена исключениями, возникающими при выполнении кода пользовательского режима.

Когда операционная система Windows начинает обработку исключений пользовательского режима, она сначала просит отладчик пользовательского режима, подключенный к процессу, если он есть, обработать исключение.

Если отладчик не подключен к процессу, ядро проверяет глобальный флаг, управляющий процессом диспетчеризации, и диспетчеризует исключение в соответствии с этим флагом. Бит 0 nt!NTGlobalFlag управляет поведением диспетчеризации исключений и называется StopOnException (soe).

Когда флаг StopOnException установлен, все исключения, встреченные в процессе, не подключенном к отладчику пользовательского режима, сначала передаются отладчику ядра, подключенному к целевой системе.

Когда флаг не установлен, отладчик режима ядра не вмешивается в код диспетчеризации исключений, если исключение не имеет специального отладочного смысла, например STATUS_BREAKPOINT и STATUS_SINGLE_STEP.

Лучший вариант для расшифровки флагов - команда расширения !gflag, которая расшифровывает содержимое nt!NTGlobalFlag, как показано в листинге 3.20.

Листинг 3.20. Расшифровка глобальных флагов ядра

```
kd> dc nt!NTGlobalFlag l1
80540aec 00000001 ....
kd> !gflag
Current NtGlobalFlag contents: 0x00000001
soe - Stop On Exception
```

Этот флаг, как и все остальные флаги ядра, можно изменить из консоли отладчика.

Флаги также можно изменять с помощью утилиты gflags.exe, устанавливаемой вместе с Debugging Tools for Windows.

Листинг 3.21 показывает пример временного или постоянного включения флага StopOnException с помощью gflags.exe.

Листинг 3.21. Изменение флагов ядра с помощью командной строки gflags.exe

```
c:\> gflags -k +soe
Current Running Kernel Settings are: 00000000
soe - Stop On Exception
c:\> gflags -r +soe
Current Boot Registry Settings are: 00000001
soe - Stop On Exception
```

Однако для более удобной интерактивной работы пользователь может запустить gflags.exe без параметров и изменить флаги ядра в графическом интерфейсе, как показано на рисунке 3.2.

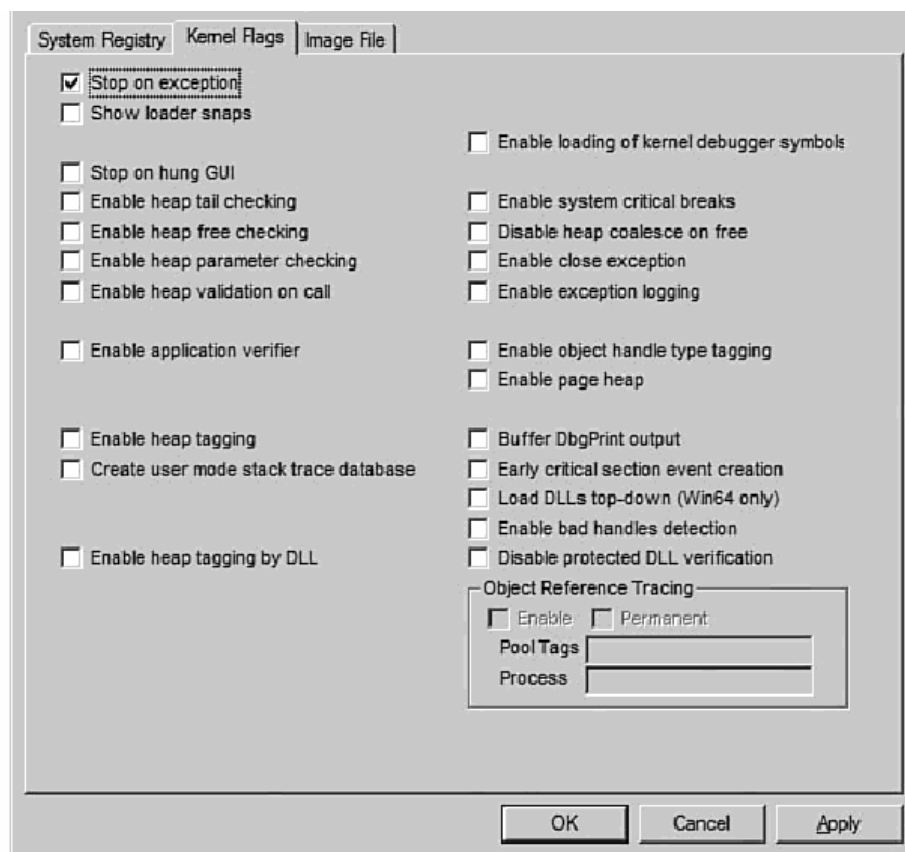


Figure 3.2 Changing kernel flags using GUI gflags.exe

Рисунок 3.2. Изменение флагов ядра с помощью GUI gflags.exe

Независимо от способа изменения флага StopOnException, поведение исключений изменяется одинаково.

Следующий раздел сосредоточен на шагах, которые выполняет ядро для диспетчеризации исключения, также учитывая флаг StopOnException.

Логика, используемая для диспетчеризации исключения пользовательского режима, описана ниже. На рисунке 3.3 эта логика представлена в виде блок-схемы.

Диспетчеризацию исключения пользовательского режима можно кратко описать так:

1. Когда возникает новое исключение, ядро Windows пытается передать его отладчику пользовательского режима, если он доступен.
Если он доступен, поток диспетчеризации исключения продолжается с шага 6.
Если к узлу подключен отладчик ядра, поток диспетчеризации исключения продолжается с шага 2; иначе он продолжается с шага 4.
2. Исключения, имеющие смысл для отладчика, например STATUS_BREAKPOINT или STATUS_SINGLE_STEP, отправляются отладчику ядра как отладочные уведомления.
Когда флаг StopOnException установлен, все остальные исключения также отправляются отладчику ядра как отладочные уведомления; иначе поток диспетчеризации исключения продолжается с шага 4.
Система "заморожена" и ждет ответа на уведомление отладчика ядра.
3. Отладчик ядра исследует исключение и, в зависимости от настроек отладчика, может обработать его.

В этом случае исключение отклоняется, и выполнение кода продолжается с места исключения, когда отладчик ядра отвечает на отладочное уведомление.

Для необработанных исключений поток диспетчеризации продолжается с шага 4.

4. Ядро Windows ищет обработчик исключения, проверяя все функции из стеков вызовов на наличие кадрового обработчика исключений.

Фильтры обработчиков исключений, найденные на этой стадии, вызываются начиная с самой недавней функции из стека, пока один из фильтров не вернет `EXCEPTION_EXECUTE_HANDLER`.

Начиная с Windows XP и Windows Server 2003 разработчик может зарегистрировать дополнительные фильтры, вызываемые до начала процесса поиска, с помощью механизма векторных обработчиков исключений.

Найдя обработчик исключения, ядро начинает откатывать стек выполнения к функции, которой принадлежит обработчик, выполняя все финальные обработчики, зарегистрированные в пройденных функциях; этот процесс называется раскруткой стека.

Наконец, выполнение кода продолжается с обработчика исключения в целевой функции.

5. Что если стек текущего потока не содержит обработчика, способного обработать текущее исключение?

Каждый поток защищает код процедуры встроенным фильтром и обработчиком, предназначенными для обработки всех исключений, не обработанных пользовательским кодом. Этот фильтр, обычно называемый фильтром необработанных исключений, выполняет необходимые шаги для завершения процесса, вызывая `API kernel32!UnhandledExceptionFilter`, когда исключение не обработано.

Логика, используемая фильтрами необработанных исключений, описана в главе 13, "Посмертная отладка".

6. Когда к процессу подключен отладчик пользовательского режима, он получает уведомление об исключении и может обработать его или оставить необработанным в зависимости от настроек отладчика.

Настройки обработки исключений см. в предыдущем разделе "Управление исключениями и событиями из отладчика".

В документации отладчика такое уведомление называется исключением первого шанса.

Обработка исключений, не обработанных отладчиком, продолжается поиском обработчика исключения и раскруткой стека, если это возможно, как в процессе, описанном на шаге 4.

Исключения, обработанные отладчиком пользовательского режима, например `STATUS_BREAKPOINT`, продолжают выполнение кода с места, которое сгенерировало исключение, после всех корректировок, сделанных отладчиком.

7. Если отладчик не обрабатывает исключение и на шаге 6 обработчик не найден, ядро Windows делает вторую попытку передать исключение отладчику; этот процесс уведомления известен как исключение второго шанса.

Если исключение все еще не обработано отладчиком, процесс просто перезапускает последовательность с шага 6, пока исключение не будет обработано.

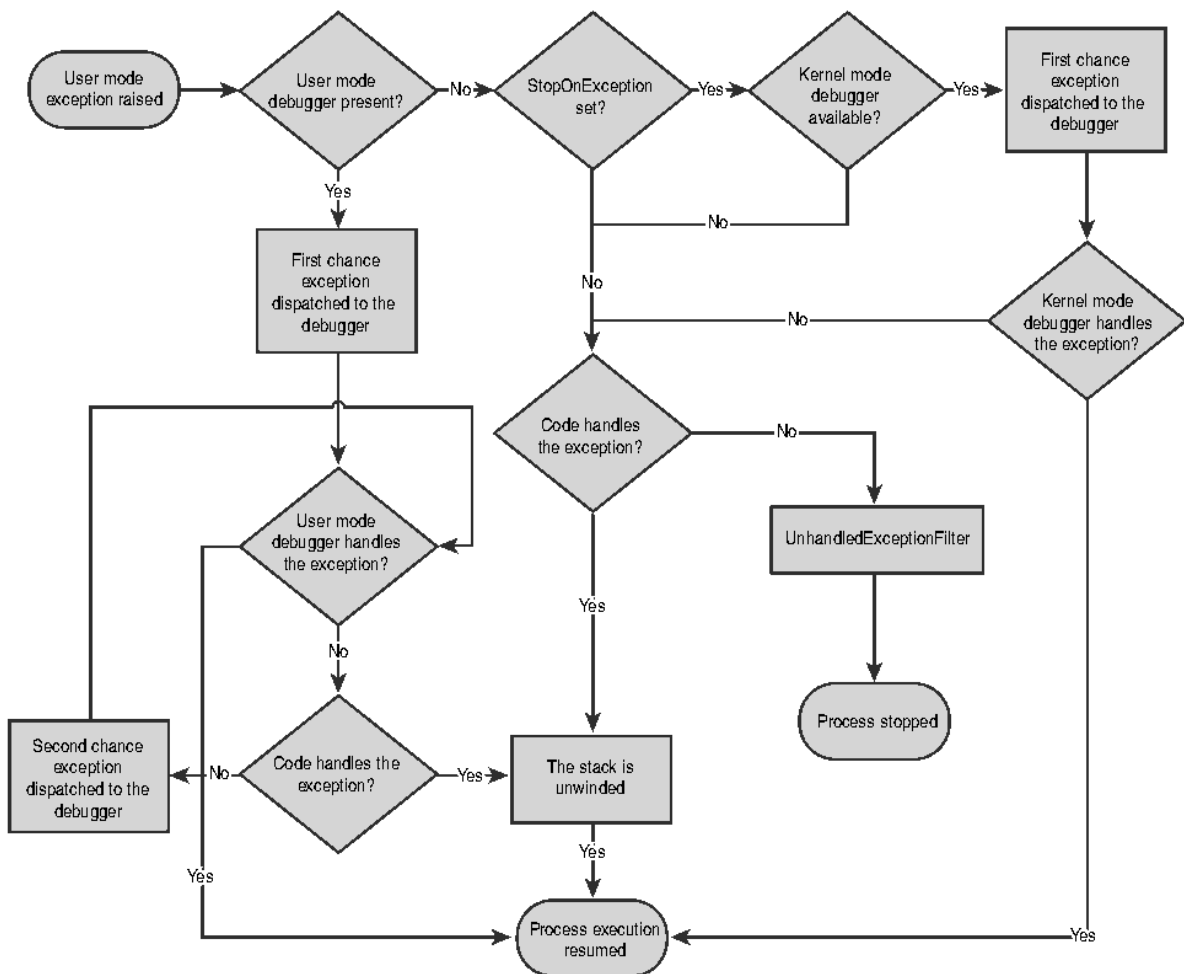


Figure 3.3 Exception dispatching logic

Рисунок 3.3. Логика диспетчеризации исключений

Следующий раздел практически показывает влияние разных конфигураций отладчика на разные исключения, используя описанную выше логику.

Исключение в разных конфигурациях отладчика

Пример 02sample.exe снова используется, чтобы проиллюстрировать логику диспетчеризации исключений пользовательского режима.

Разные параметры вызывают пути кода с разным поведением обработки исключений.

В языке C обработчики исключений создаются с помощью ключевых слов `__try/__except`, расширения Microsoft для компиляторов компании, предназначенного для генерации фильтров исключений и обработчика, требуемых операционной системой.

Этот раздел подробно описывает несколько аспектов механизма обработки исключений, реализованного операционной системой Windows.

Листинг 3.22 показывает код, выполняемый каждым параметром, описанным в подзаголовках; этот код скомпилирован в исполняемом файле 02sample.exe.

Листинг 3.22. Код, проверяющий логику диспетчеризации исключений

```

Code causing an access violation exception, exercised by option '1'
void RaiseAV()
{

```

```

_alloca(1); //Force the compiler to generate a stack frame
char* invalidAddress = 0;
*invalidAddress = 0;
}
Code causing a break point exception, exercised by option '2'
void RaiseBP()
{
_alloca(1); //Force the compiler to generate a stack frame
DebugBreak();
}
Code handling an access violation exception, exercised by option 'b'
__try
{
RaiseAV();
}
__except(EXCEPTION_EXECUTE_HANDLER)
{
}
Code handling a break point exception, exercised by option 'c'
__try
{
RaiseBP();
}
__except(EXCEPTION_EXECUTE_HANDLER)
{
}

```

Каждая функция, показанная выше, запускается в разных окружениях.

Вся важная информация, относящаяся к взаимодействию между кодом и операционной системой Windows (или к взаимодействию с отладчиками, если они подключены), подробно описана ниже. Все упражнение выполняется в предположении, что конфигурация системы не была изменена никакой программой, установленной в этой системе, особенно набором отладочных инструментов или средой разработки с возможностями отладки.

Один и тот же исполняемый файл запускается в четырех разных конфигурациях:

- Первая конфигурация не использует отладчик, что соответствует реальному пользовательскому окружению.
Мы называем ее нормальной конфигурацией.
- Во второй конфигурации к узлу подключен отладчик ядра, что часто используется на этапе тестирования программного обеспечения.
Мы называем ее конфигурацией отладчика режима ядра, или KD-конфигурацией.
- В третьей конфигурации к узлу подключен отладчик ядра и включен глобальный флаг `StopOnException`.
Мы называем ее конфигурацией KD с SOE.
- В четвертой конфигурации исполняемый файл запускается под отладчиком пользовательского режима; такая конфигурация популярна на этапе разработки.
Мы называем ее конфигурацией отладчика пользовательского режима, или UM-конфигурацией.

Необработанный исключение нарушения доступа (STATUS_ACCESS_VIOLATION)

Первый параметр генерирует самое знакомое исключение с кодом `0xC0000005`, представляющее исключение нарушения доступа, также известное как ошибка защиты.

Первая функция, описанная в листинге 3.22, должна использоваться в каждой из предыдущих конфигураций.

Поведение во всех конфигурациях выглядит так:

- Нормальная конфигурация

При отсутствии отладчика код диспетчеризации исключений проверяет все доступные фильтры на шаге 4 ранее описанного раздела "Диспетчеризация исключений".

Не найдя ни одного, код диспетчеризации исключений вызывает `kernel!UnhandledExceptionFilter`, из-за чего приложение сообщает об ошибке и завершается.

Этот процесс описан в главе 13.

- **KD-конфигурация**

При подключенном к системе отладчике ядра поведение системы не меняется, и приложение завершается так же, как в нормальной конфигурации.

- **Конфигурация KD с SOE**

В этой конфигурации код обработки исключений пересылает исключение отладчику режима ядра и ждет решения об обработке.

Система возобновляет выполнение после ввода команды `g` с кодом обработки исключения, описанным для нормальной конфигурации.

- **UM-конфигурация**

Отладчик пользовательского режима получает уведомление о встреченном исключении, поскольку обычно отладчик настроен останавливаться на исключениях первого шанса.

После ввода команды `g` код обработки исключений ищет кадровый обработчик для этого исключения, и поскольку обработчик недоступен, уведомление об исключении еще раз отправляется отладчику как исключение второго шанса.

Обработка исключения в отладчике не помогает, потому что условие, вызывающее нарушение доступа, все еще присутствует, и сбойная инструкция выполняется снова.

В результате система снова возбуждает исключение как исключение первого шанса, и цикл продолжается, пока условие не исчезнет.

Этот цикл можно увидеть в действии, запустив сбойный код под отладчиком и указав ему только уведомлять пользователя об исключениях нарушения доступа вместо ожидания пользовательского ввода:

```
c:\>windbg.exe -g -G -xn av C:\AWDBIN\WinXP.x86.chk\02sample.exe
```

Необработанное исключение точки останова (исключение STATUS_BREAKPOINT)

Как было видно в начале этой главы, исключение `STATUS_BREAKPOINT` имеет специальный смысл для отладчика, и поведение системы немного меняется по сравнению с исключением нарушения доступа.

- **Нормальная конфигурация**

Система демонстрирует то же поведение, что и при исключении нарушения доступа.

Любая инструкция процессора `int 3` (выполненная из `DebugBreak()` или оператора `assert()`) воспринимается системой и пользователем как любое другое исключение.

Вопреки тому, что мы видим в отладчике, выполнение кода не продолжается сразу после инструкции `int 3`.

- **KD-конфигурация**

Поскольку исключение характерно для процесса отладки, отладчик ядра останавливается и обрабатывает это исключение.

При продолжении выполнение возобновляется с инструкции, следующей за оператором `int 3`.

- **Конфигурация KD с SOE**

Поскольку исключение `STATUS_BREAKPOINT` уже обработано отладчиком режима ядра, флаг `StopOnException` не добавляет дальнейших изменений.

- УМ-конфигурация

Отладчик останавливается на инструкции точки останова и обрабатывает исключение.

При продолжении выполнение возобновляется с инструкции, следующей за оператором `int 3`.

Обработанное исключение нарушения доступа

Код, используемый в этом случае, похож на код, которым мы проверяли необработанные нарушения доступа, за исключением того, что он предоставляет кадровый обработчик исключения.

- Нормальная конфигурация

Как и ожидалось, исключение обработано, и код продолжает обычное выполнение после выполнения обработчика.

- KD-конфигурация

Как и ожидалось, исключение обработано, и код продолжает обычное выполнение без уведомления режима ядра.

- Конфигурация KD с SOE

В этой конфигурации механизм обработки исключений пересылает исключение отладчику режима ядра и ждет решения о продолжении.

При продолжении (после команды `g`) исключение обрабатывается в коде пользовательского режима, который продолжает обычное выполнение.

- УМ-конфигурация

Отладчик останавливается на уведомлении об исключении первого шанса согласно стандартным настройкам отладчика для обработки исключений.

При продолжении обработчик исключения обрабатывает исключение, и выполнение процесса продолжается нормально.

Обработанное исключение точки останова

Что меняется, когда исключение является специфичным для отладки, например `STATUS_BREAKPOINT` или `STATUS_SINGLE_STEP`?

Все отладчики пытаются понять и обработать такие исключения.

- Нормальная конфигурация

Как и ожидалось, исключение обработано, и код продолжает обычное выполнение.

- KD-конфигурация

Поскольку исключение специально используется в отладке, отладчик ядра останавливается и обрабатывает это исключение.

- Конфигурация KD с SOE

В этой конфигурации код обработки исключений пересылает исключение отладчику режима ядра и ждет решения по нему.

При продолжении (после команды `g`) выполнение возобновляется с инструкции, следующей за оператором `int 3`, и процесс завершается нормально.

- УМ-конфигурация

Отладчик останавливается на уведомлении об исключении первого шанса согласно стандартным настройкам обработки исключений отладчика.

При продолжении выполнение возобновляется с инструкции, следующей за оператором `int 3`, и процесс завершается нормально.

После проверки всех этих конфигураций с разными кодами исключений можно сделать несколько интересных выводов, полезных в повседневной работе.

- По умолчанию любое необработанное исключение с помощью Windows Error Reporting (WER) создает отчет о сбое, который можно использовать для посмертной отладки.
Заказчики могут централизовать такие отчеты на уровне предприятия с помощью Microsoft Corporate Error Reporting или более нового сервера Agentless Exception Monitoring.
Заказчик также может отправлять их на сайт WER для исследования разработчиками Microsoft или участвующими поставщиками программного обеспечения.
Глава 13 описывает, как независимые поставщики ПО могут участвовать в анализе отчетов WER и предоставлять решения для часто встречающихся проблем.
- Хотя пользователи любого программного решения не получают приятного опыта при встрече с необработанными исключениями, с точки зрения разработчика эти исключения предоставляют необходимую обратную связь для исправления всех дефектов ПО, присутствующих в приложениях.
Альтернативный прием, при котором все исключения скрываются путем их "обработки" независимо от типа или источника, чтобы пользователь их не видел, создает долгосрочные проблемы надежности, которые трудно диагностировать и иногда так и не исправляют, поскольку для пользователей нет "видимого" воздействия.
- На этапах разработки и тестирования отладчик ядра является очень мощным инструментом, и его следует использовать для мониторинга некоторой доли систем, участвующих в тестировании продукта, если это не конфликтует с приложением.
- Распределенные приложения, передающие ошибки из одного процесса в другой, обычно трудно отлаживать, поскольку источник исходной ошибки заранее неизвестен.
Если ошибка изначально была исключением, возбужденным в любом составляющем процессе, легко остановить выполнение системы в этой точке с помощью конфигурации KD с SOE и подходящей команды `sx` в отладчике ядра.
- Хорошие разработчики обычно проверяют состояние процесса с помощью различных техник `assert`.
К сожалению, большинство `assert` отключены в выпущенной версии продукта, которая чаще всего является целью этапа тестирования, и большая возможность убедиться, что код работает ожидаемо, оказывается упущенной.
Действительно важные `assert` можно заменить кодом, который возбуждает точку останова и сразу обрабатывает ее.
Эта точка останова заставляет код остановиться в отладчике, если он присутствует, или продолжает выполнение с небольшим снижением производительности, поскольку проверяемое условие всегда должно быть истинным.

Знание того, как система обрабатывает исключение в разных конфигурациях, позволяет разработчикам понять, почему код остановился именно там, где остановился.

Разработчики могут использовать это знание, чтобы определить стратегию обработки ошибок для своего продукта, полагаться на фильтр необработанных исключений для сбора данных о сбоях или самостоятельно обрабатывать некоторые исключения и собирать информацию из процесса.

На этапе разработки код можно инструментировать, а тестовое окружение можно настроить так, чтобы привносить ценную обратную связь в процесс разработки.

В идеале разработчики не должны менять поведение фильтра необработанных исключений и должны полагаться на механизм обратной связи WER.

МЕТОДЫ ПРОТИВОДЕЙСТВИЯ ОТЛАДКЕ Помните, что некоторые методы противодействия отладке используют механизм исключений, чтобы проверить, выполняется ли окружение без отладчиков, и затруднить отладку защищенного таким способом кода.

Исключение, возбужденное в продукте, связанном с защитой данных, управлением правами или управлением лицензиями, не всегда является тем, чем кажется.

Кадровый обработчик исключений

Как мы видели в этом разделе, механизм обработки исключений Windows довольно гибок.

Он позволяет любой функции из стека вызовов фильтровать все исключения, возбужденные при выполнении текущей функции или любой функции, вызванной из нее.

В зависимости от типа исключения или других факторов, определенных фильтром, функция может обработать исключение, исправить условие, породившее исключение, и повторить выполнение либо проигнорировать исключения.

Функция также может установить обработчик завершения, который будет вызван при возврате из текущей функции.

Этот раздел объясняет базовый механизм, используемый приложениями для поддержки механизма диспетчеризации исключений.

Понимание этого механизма полезно при отладке проблем, встречающихся в самом коде обработки исключений.

Хотя механизм, описанный в этом разделе, специфичен для архитектуры x86, он является хорошим примером для изучения того, как система работает с исключениями и как отлаживать такой код.

Системные требования к функции для участия в механизме обработки исключений минимальны.

Приложение должно предоставить обработчик исключений с четко определенной сигнатурой функции и зарегистрировать его в механизме раскрутки процесса на время выполнения функции.

Каждая регистрация представляет новый кадр исключения.

Этот обработчик вызывается операционной системой Windows, когда функция может завершить выполнение из-за исключения.

Хотя можно вручную создавать обработчики исключений, которые напрямую взаимодействуют с native-механизмом обработки исключений, для построения кадров исключений мы используем компиляторы C/C++.

На архитектурах x86 обработчики исключений организованы в односвязный список, отдельный для каждого потока и динамически изменяемый кодом, выполняющимся в этом потоке.

Когда в список нужно добавить новый обработчик, узел этого обработчика становится головой списка, которая затем сохраняется в блоке окружения потока (TEB).

Каждый узел хранит обработчик исключений для соответствующей функции, а также ссылку на следующий узел, соответствующий вызывающей стороне с обработчиком исключений.

Рисунок 3.4 иллюстрирует организацию списка.

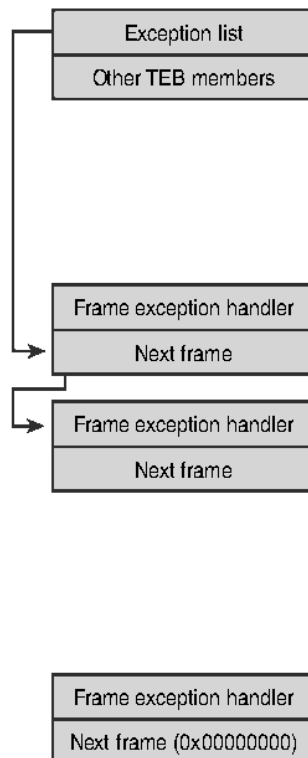


Figure 3.4 Exception handler list

Рисунок 3.4. Список обработчиков исключений

Поскольку каждая функция предоставляет не более одного обработчика исключений, длина списка не может превышать длину стека вызовов.

Большинство функций не требуют участия в логике диспетчеризации исключений и не добавляют обработчик в цепочку исключений.

Листинг 3.23 демонстрирует использование информации, описанной на рисунке 3.4: поиск головы списка обработчиков исключений и вывод всего списка исключений с помощью команды расширения `!slist`.

Команда отладчиков Windows понимает, что этот процесс неудобен, поэтому предоставила команду расширения `!exchain`, которая делает все это, а также по возможности выполняет необходимую расшифровку обработчиков функций.

Листинг 3.23 использует эти команды для исследования цепочки обработчиков исключений при остановке отладчика, вызванной в функции, запускаемой параметром `d` примера `02sample.exe`.

Листинг 3.23. Исследование списка обработчиков исключений x86

```

0:000> !teb
TEB at 7ffdf000
ExceptionList: 0006ff28
0:000> * Obtain the exception chain type information
0:000> dt nt!_NT_TIB ExceptionList
+0x000 ExceptionList : Ptr32 _EXCEPTION_REGISTRATION_RECORD
0:000> !slist $teb _EXCEPTION_REGISTRATION_RECORD 0
SLIST HEADER:
+0x000 Alignment : 700000006ff28
+0x000 Next : 6ff28
+0x004 Depth : 0
+0x006 Sequence : 7
SLIST CONTENTS:
0006ff28
+0x000 Next : 0x0006ff90 _EXCEPTION_REGISTRATION_RECORD
+0x004 Handler : 0x010020d2 _EXCEPTION_DISPOSITION
  
```



```

02sample!_except_handler4+0
0006ff90
+0x000 Next : 0x0006ffdc _EXCEPTION_REGISTRATION_RECORD
+0x004 Handler : 0x010020d2 _EXCEPTION_DISPOSITION
02sample!_except_handler4+0
0006ffdc
+0x000 Next : 0xffffffff _EXCEPTION_REGISTRATION_RECORD
+0x004 Handler : 0x77b88bf2 _EXCEPTION_DISPOSITION
ntdll!_except_handler4+0
FFFFFFFF
+0x000 Next : ???
+0x004 Handler : ???
0:000> !exchain /f
0006ff28: 02sample!_except_handler4+0 (010020d2)
0006ff90: 02sample!_except_handler4+0 (010020d2)
0006ffdc: ntdll!_except_handler4+0 (77b88bf2)
...

```

В этом случае каждая функция использует один и тот же обработчик исключений, и команда расширения `!exchain` не понимает кадр исключения и не показывает дополнительную информацию о нем.

В таких ситуациях приходится вручную декодировать кадры исключений.

Поскольку в большинстве случаев обработчики генерируются инструментами компилятора, следующий раздел углубляется в детали сгенерированного кода, используя компиляторы Microsoft C/C++ как модель.

Компилятор предоставляет эту поддержку через нестандартное расширение в виде конструкций `__try/__except` и `__try/__finally`.

Генерация кадрового обработчика исключений

Начнем с простой функции, содержащей обработчик исключений и фильтр обработчика исключений, который всегда вычисляется как `EXCEPTION_EXECUTE_HANDLER`.

Код, защищенный обработчиком исключений, обращается к недопустимой области памяти, что генерирует исключение нарушения доступа.

Исходный код этой функции показан в листинге 3.24.

Листинг 3.24. Простая функция, использующая конструкции `__try/__except`

```

void try_except()
{
    __try
    {
        *((int *) 0) = 0;
    }
    __except(ex_filter())
    {
        global = 1;
    }
}

```

Сгенерированный код этой функции можно исследовать в отладчике после запуска `02sample.exe`.

Листинг 3.26 содержит аннотированный код, соответствующий функции, показанной в листинге 3.25.

Листинг 3.25. Сгенерированный код простой функции с поддержкой `__try/__except`

```

0:000> uf 02sample!try_except
02sample!try_except:
...
01001d75 6afe push 0FFFFFFEh ;Set the block counter
01001d77 68d02a0001 push offset 02sample!_CT??_R0H+0x60 (01002ad0)
01001d7c 68d2200001 push offset 02sample!_except_handler4 (010020d2)
01001d81 64a100000000 mov eax,dword ptr fs:[00000000h] ;Retrieve the head

```

```

01001d87 50 push eax ;Save the old head
...
01001d99 8d45f0 lea eax,[ebp-10h]
01001d9c 64a300000000 mov dword ptr fs:[00000000h],eax ;Save the new head
01001da2 8965e8 mov dword ptr [ebp-18h],esp
01001da5 c745fc00000000 mov dword ptr [ebp-4],0 ;Block change
01001dac c7050000000000000000 mov dword ptr ds:[0],0
01001db6 c745fcfeffffff mov dword ptr [ebp-4],0FFFFFFEh
01001dbd eb1a jmp 02sample!try_except+0x69 (01001dd9)
02sample!try_except+0x69:
01001dd9 8b4df0 mov ecx,dword ptr [ebp-10h] ; Get old head
01001ddc 64890d00000000 mov dword ptr fs:[0],ecx ; restore old head
...
01001dea c3 ret
0:000> dc 01002ad0 l8
01002ad0 ffffffff 00000000 ffffffff 00000000 .....
01002ae0 ffffffff 01001dbf 01001dc5 00000000 .....

```

Компилятор разбивает функцию на несколько областей с разной функциональностью обработчиков и генерирует агрегатную структуру, содержащую фильтр и обработчик для каждой области.

Чтобы связать эту информацию со стандартным механизмом раскрутки, компилятор регистрирует общий обработчик в начале вызова функции и отменяет его регистрацию в конце вызова функции. Обработчик, общий для всех функций модуля, оценивает исключение с помощью функции-фильтра и вызывает пользовательский код, обрабатывающий исключение, соответствующее текущему выполняемому блоку.

Обработчик реализован в библиотеке времени выполнения компилятора, также известной как CRT.

Как общий обработчик узнает, какой блок выполняется сейчас?

Компиляторы Microsoft C/C++ на процессорах x86 используют локальный счетчик, указывающий, какая область выполняется в данный момент.

Локальный счетчик изменяется кодом, сгенерированным компилятором, когда выполнение пересекает границы области.

Обычный ассемблерный код ограничивает возможность понять код обработки исключений и преобразования, происходящие в процессе компиляции.

Чтобы уменьшить разрыв между привычным исходным кодом C/C++ и ассемблерным кодом, компилятор может сгенерировать промежуточный файл, называемый ассемблерным листингом.

Ассемблерный листинг содержит ассемблерный код, аннотированный исходным кодом и говорящими метками вместо одних только адресов.

Это часто используется, чтобы понять роль конкретной инструкции процессора в исходном коде C/C++.

Листинг 3.26 содержит ассемблерный листинг, соответствующий функции `try_except`, ранее показанной обычным ассемблерным языком.

В аннотированном коде, показанном в листинге 3.27, видно, что блок информации об исключениях, обозначенный меткой `__$sehtable$?try_except@@YGXXZ`, содержит указатели на фильтр исключения `$LN5@try_except` и на функцию обработчика исключения `$LN6@try_except`.

Общая функция обработки исключений, функция `__except_handler4`, импортированная из библиотеки `MSVCRT`, хранится в стеке сразу после блока информации об исключениях по адресу `0000c`.

Индекс области, на который ссылаются через метку `__$SEHRec$[ebp+20]`, меняется с -2, что означает нахождение функции вне любой области исключения без кода для выполнения при исключении, на 0 после начала выполнения блока `__try` по смещению `00035`.

Когда выполнение защищенной области завершается, индекс снова меняется на -2, показывая, что выполнение кода находится вне любой защищенной области.

Список обработчиков исключений обозначается через `fs:0`.

Листинг 3.26. Ассемблерный листинг, сгенерированный для функции из листинга 3.24

```

PUBLIC ?try_except@@YGXXZ ;
xdata$x SEGMENT
__sehtable$?try_except@@YGXXZ DD 0fffffffH
DD 00H
DD 0fffffd8H
DD 00H
DD 0fffffffH
DD FLAT:$LN5@try_except
DD FLAT:$LN6@try_except
xdata$x ENDS
_TEXT SEGMENT
?try_except@@YGXXZ PROC ; try_except, COMDAT
...
00005 6a fe push -2 ; ffffffffH
00007 68 00 00 00 00 push OFFSET __sehtable$?try_except@@YGXXZ
0000c 68 00 00 00 00 push OFFSET __except_handler4
00011 64 a1 00 00 00 00 mov eax, DWORD PTR fs:0
...
00029 8d 45 f0 lea eax, DWORD PTR __$SEHRec$[ebp+8]
0002c 64 a3 00 00 00 00 mov DWORD PTR fs:0, eax
00032 89 65 e8 mov DWORD PTR __$SEHRec$[ebp], esp
; 29 : __try
00035 c7 45 fc 00 00 00 00 mov DWORD PTR __$SEHRec$[ebp+20], 0
; 30 : {
; 31 : *((int *) 0) = 0;
0003c c7 05 00 00 00 00 00 00 00 mov DWORD PTR ds:0, 0
; 32 : }
00046 c7 45 fc fe ff ff ff mov DWORD PTR __$SEHRec$[ebp+20], -2 ; ffffffffH
0004d eb 1a jmp SHORT $LN4@try_except
$LN5@try_except:
$LN10@try_except:
; 33 : __except(ex_filter())
0004f e8 00 00 00 00 call ?ex_filter@@YGKXZ ; ex_filter
$LN7@try_except:
$LN9@try_except:
00054 c3 ret 0
$LN6@try_except:
00055 8b 65 e8 mov esp, DWORD PTR __$SEHRec$[ebp]
; 34 : {
; 35 : global = 1;
00058 c7 05 00 00 00 00 01 00 00 00 mov DWORD PTR ?global@@3HA, 1 ; global
; 36 : }
00062 c7 45 fc fe ff ff ff mov DWORD PTR __$SEHRec$[ebp+20], -2 ; ffffffffH
$LN4@try_except:
; 37 : }
00069 8b 4d f0 mov ecx, DWORD PTR __$SEHRec$[ebp+8]
0006c 64 89 0d 00 00 00 00 mov DWORD PTR fs:0, ecx
...
0007a c3 ret 0
?try_except@@YGXXZ ENDP ; try_except
_TEXT ENDS

```

Как мы сгенерировали этот код?

Процесс зависит от среды разработки, использованной для сборки приложения.

В среде сборки WDK процесс генерации аннотированного кода прост: файл аннотированного кода является просто еще одной целью процесса компиляции, целью с расширением .cod.

Например, файл FuncAV.cpp (содержащий код для этого раздела) можно скомпилировать в аннотированный файл, выполнив через nmake целевой файл FuncAV.cod, как показано в листинге 3.27.

Листинг 3.27. Генерация аннотированного ассемблерного файла из исходного файла

```

C:\AWD\CHAPTER2>nmake FuncAV.cod
Microsoft (R) Program Maintenance Utility Version 7.00.8882
Copyright (C) Microsoft Corp 1988-2000. All rights reserved.
cl -nologo @objfre_wxp_x86\i386\clcod.rsp /Fc /FC .\FuncAV.cpp
FuncAV.cpp

```

Метка `fs:0`, представляющая голову списка обработчиков исключений, вычисляется в адрес `fs:[0]`, первый указатель из ТEB.

Поскольку селектор `fs` имеет одно и то же значение для всех потоков, может возникнуть вопрос: что происходит в многопоточной среде и почему список исключений не повреждается, если все головы обработчиков исключений хранятся по одному и тому же адресу?

Операционная система использует только селектор `fs` для адресации информации, специфичной для потока, что обеспечивает косвенность, необходимую для доступа к разным адресам через один и тот же "дескриптор".

Хотя значение селектора остается одинаковым для всех потоков процесса, разделение потоков достигается операционной системой путем изменения дескриптора сегмента, на который указывает селектор `fs`, каждый раз, когда новый поток планируется к выполнению на процессоре.

Листинг 3.28 показывает дескриптор сегмента, соответствующий селектору `fs` со значением `0x3b`, для двух потоков одного процесса.

Столбец `base` представляет виртуальный адрес, с которого начинается ТEB.

Листинг 3.28. Блок окружения потока в двух разных потоках одного процесса

```
0:000> dg @fs
P Si Gr Pr Lo
Sel Base Limit Type l ze an es ng Flags
-----
003B 7ffdf000 00000fff Data RW Ac 3 Bg By P Nl 000004f3
0:001> dg @fs
P Si Gr Pr Lo
Sel Base Limit Type l ze an es ng Flags
-----
003B 7ffdd000 00000fff Data RW Ac 3 Bg By P Nl 000004f3
```

После этого обзора всего механизма исключений вы должны понимать, какой код выполняется, когда исключение проходит через ваши функции, и должны уметь при необходимости устанавливать точки останова в фильтрах исключений или обработчиках исключений.

В других случаях вы можете оказаться в ситуации, где исходный код правильно обрабатывает исключение, а исполняемый код - нет, и можете обнаружить, что обработчик был добавлен уже после компиляции этого исполняемого файла; теперь у вас есть средства это доказать.

Как побочный результат, исследуя голову списка обработчиков исключений, сохраненную в ТEB, мы можем выяснить, какие функции из текущего стека используют обработчики исключений. Эта информация бесценна, когда стек поврежден или недоступен, как в некоторых ситуациях отладки ядра, где стек не находится в памяти.

Обработка отладочных событий из отладчика ядра

Концепция использования отладочных событий для связи между целью отладчика и клиентом отладчика естественным образом распространяется на отладчики ядра; главное различие состоит в механизме связи между отладчиком и целью отладчика.

Протокол связи не документирован, но любознательные читатели могут увидеть часть обмена между отладчиком ядра и целью отладчика, нажав сочетание `CTRL+D` в консоли отладчика и наблюдая подробную трассировку всего протокола.

Как обсуждалось ранее, разработчики пользовательского режима редко могут получить пользу от событий отладчика ядра, поскольку полезных для них событий не так много.

Без сомнения, самое полезное из них - событие исключения `EXCEPTION_BREAKPOINT`, возбуждаемое, когда любой фрагмент кода из пользовательского режима выполняет инструкцию `int 3`, вызванную `DebugBreak()` или различными `API assert`.

Вторыми по важности являются события исключений, отправляемые, когда все исключения пользовательского режима направляются в отладчик ядра с помощью флага `StopOnException`.

Наконец, ядро Windows может отправлять уведомления, когда пользовательские модули отображаются в память.

Эта функциональность включается установкой флага `KernelSymbolLoad (kls)` в той же глобальной переменной, что и `nt!NTGlobalFlag`, с помощью утилиты `gflags.exe` или команды расширения `!gflag`.

После включения флага мы активируем уведомление, вводя команду `sxe ld:<module>` в отладчике режима ядра.

Отладчик получает уведомление, когда модуль отображается в память, что дает хорошую возможность отлаживать из режима ядра процесс, который его загружает.

Листинг 3.29 использует флаг `kls` для обнаружения первого создания процесса `notepad.exe`.

Эта возможность очень полезна для отладки модулей, загружаемых на ранних стадиях запуска Windows, или когда трудно предсказать, какой процесс загрузит интересующий модуль. Однако это уведомление не отправляется, если модуль уже кэширован в системной памяти.

Листинг 3.29. Использование флага `kls` для обнаружения отображения модуля пользовательского режима

```
kd> !gflag +kls
New NtGlobalFlag contents: 0x00040000
ksl - Enable loading of kernel debugger symbols
kd> sxe ld notepad
kd> g
nt!DebugService2+0x10:
8050b897 cc int 3
kd> k
ChildEBP RetAddr
f3b7da24 8050b8d9 nt!DebugService2+0x10
f3b7da48 805d536c nt!DbgLoadImageSymbols+0x42
f3b7da98 805d5212 nt!MiLoadUserSymbols+0x169
f3b7dad0 8057bc22 nt!MiMapViewOfImageSection+0x4b6
f3b7db38 80503a0b nt!MmMapViewOfSection+0x13c
f3b7db94 80588c21 nt!MmInitializeProcessAddressSpace+0x337
f3b7dce4 80588635 nt!PspCreateProcess+0x333
f3b7dd38 804df06b nt!NtCreateProcessEx+0x7e
f3b7dd38 7c90eb94 nt!KiFastCallEntry+0xf8
WARNING: Frame IP not in any known module. Following frames may be wrong.
0013fa88 00000000 0x7c90eb94
kd> !process -1 0
PROCESS 82f5a020 SessionId: 0 Cid: 0000 Peb: 00000000 ParentCid: 0544
DirBase: 0de15000 ObjectTable: e1b12638 HandleCount: 1.
Image: notepad.exe
```

Управление целью

После этого обзора механизмов, предоставляемых операционной системой для отладки любого выполняющегося целевого процесса, остается еще один шаг, необходимый для понимания того, как отладчик способен выполнять всю свою "магию".

Этот раздел описывает некоторые рычаги, используемые отладчиками для управления целью отладчика, и то, как каждый рычаг влияет на цель отладчика.

Как работают точки останова

Исключение с кодом STATUS_BREAKPOINT используется на протяжении всей книги, особенно в этой главе, без ясного объяснения того, как это исключение возбуждается.

Настало время объяснить, как процесс генерирует это исключение.

Набор инструкций x86 содержит специальную инструкцию `int 3`, введенную для упрощения отладки путем генерации аппаратного исключения STATUS_BREAKPOINT на процессоре, выполняющем эту инструкцию.

В ответ на исключение STATUS_BREAKPOINT процессор выполняет обработчик прерывания, зарегистрированный для вектора прерывания 3.

Обработчик прерывания преобразует аппаратное исключение в программное исключение, возбужденное по адресу, содержащему эту инструкцию.

В потоке инструкций эта инструкция представлена одним байтом со значением `0xCC`; такое представление называется кодом операции, или `opcode`.

Если отладчик недоступен, программное исключение обрабатывается как обычное исключение; иначе операционная система Windows предписывает отладчику остановиться прямо на адресе этой инструкции.

Отладчик использует `opcode 0xCC` при установке точки останова.

Чтобы установить точку останова, отладчик меняет защиту блока памяти, содержащего адрес точки останова, так, чтобы он мог записать инструкцию `int 3` по этому адресу.

Старое значение вместе с информацией о номере точки останова затем сохраняется в памяти отладчика.

Адрес точки останова должен быть адресом допустимого `opcode` в потоке инструкций, то есть всегда первым байтом машинной инструкции.

Точка останова, установленная по любому другому адресу внутри машинной инструкции, меняет смысл инструкции, не вызывая аппаратного исключения STATUS_BREAKPOINT при генерации этой инструкции.

Разумеется, запуск приложения, содержащего неправильную машинную инструкцию, опасен и непредсказуем.

Изменения в памяти не должны быть видимы пользователю, поскольку они могут повлиять на результаты дизассемблирования функций кода.

Поэтому при остановке отладчик всегда восстанавливает исходные значения памяти для каждой установленной им точки останова перед выполнением какой-либо обработки.

Независимо от приемов, используемых для скрытия точек останова, когда цели отладчика снова начинают выполняться, `opcode int 3` вставляются обратно в образ цели.

Чтобы продемонстрировать этот механизм, запустим любимую цель отладчика `notepad.exe` под отладчиком.

На начальной точке останова мы устанавливаем точку останова по любому адресу, в данном случае по начальному адресу `notepad!WinMain`, и исследуем содержимое этого адреса из другого отладчика, подключенного к тому же процессу неинтерактивно.

Такая настройка позволяет нам найти реальное содержимое памяти, принадлежащее цели отладчика.

Пока отладчик пользовательского режима ждет пользовательского ввода в командной строке, память содержит исходный поток инструкций.

При выполнении цели отладчика мы вводим `g` в окне команд интерактивного отладчика пользовательского режима, чтобы изменить память, как показано во второй части листинга 3.30.

Листинг 3.30. Исследование памяти процесса из неинвазивного отладчика

```
Before setting the breakpoint
0:000> u 010028e4
```

```

010028e4 85c0 test eax, eax
010028e6 7594 jnz 0100287c
010028e8 e8c3efffff call 010018b0
After setting the breakpoint
0:000> u 010028e4
010028e4 cc int 3
010028e5 c07594e8 shl byte ptr [ebp-0x6c], 0xe8
010028e9 c3 ret
010028ea ef out dx, eax

```

Отладчик режима ядра следует той же модели при установке точки останова, с небольшими отличиями, навязанными механизмом управления памятью операционной системы.

В операционной системе Windows большинство страниц, содержащих исполняемый код, совместно используются всеми процессами, использующими этот модуль; эта возможность применяется общими библиотеками DLL, загруженными в два разных процесса.

Когда отладчик пользовательского режима включает новую точку останова, он меняет защиту страницы с "только чтение" на "чтение-запись".

Новая страница, созданная с помощью техники Copy-On-Write (COW), становится частной страницей отлаживаемого процесса и может быть изменена без влияния на другие процессы, совместно использующие эту страницу.

Поскольку отладчик режима ядра не может создать частную страницу с помощью техники COW, он напрямую устанавливает точку останова на общей странице.

Точки останова режима ядра отражаются во всех выполняющихся процессах, совместно использующих страницу.

Более того, в зависимости от доступной в системе памяти точки останова режима ядра могут сохраняться в системной памяти после завершения выполнения отлаживаемого процесса.

Побочные эффекты трудно предсказать в реальных ситуациях отладки, поскольку управление памятью Windows сильно зависит от нагрузки на память и общей активности системы.

Тем не менее можно сделать несколько выводов о точках останова режима ядра.

- Установка точки останова на странице, совместно используемой многими процессами, приводит к остановкам во многих процессах.

Поскольку отладчик ядра обрабатывает точки останова относительно медленно, особенно через последовательные каналы, его никогда не следует использовать для часто вызываемых функций, таких как `ntdll!RtlAllocateHeap`.

Можно уменьшить число остановок отладчика, используя адрес `EPROCESS` или адрес `KTHREAD`, чтобы сузить область действия точки останова.

К сожалению, отладчик все равно получает уведомление о каждом попадании и автоматически обрабатывает точку останова для всех неподходящих процессов.

- После завершения процесса, ранее отлаживавшегося из отладчика ядра, все точки останова пользовательского режима должны быть удалены, чтобы избежать конфликтов с другими выполняющимися процессами.

Общие страницы могут оставаться в памяти неопределенное время со всеми ранее установленными точками останова, даже если процесс перезапущен.

- Когда отладчик пользовательского режима используется вместе с отладчиком режима ядра, точки останова всегда должны устанавливаться из отладчика пользовательского режима.

Иначе исключение точки останова диспетчеризуется в отладчик пользовательского режима.

Поскольку он не знает, что `int 3` является точкой останова, а не явной инструкцией `int 3`, поток выполнения нарушается.

Разумеется, поток инструкций, выполняемый после ввода `g`, оказывается полностью неправильным и, скорее всего, заканчивается длинной последовательностью исключений нарушения доступа или исключений одиночного шага в одном из отладчиков.

Как работают точки останова на доступ

Помимо стандартной инструкции точки останова, все процессоры, поддерживаемые операционной системой Windows, способны генерировать остановку при чтении, записи или выполнении с определенного адреса.

Команда `ba` использует эту функциональность процессора для реализации остановки на доступ. Эта возможность процессора управляется набором из восьми регистров (снова сосредоточимся на архитектуре x86), называемых DR0-DR7.

Использование этих регистров процессора хорошо документировано в документации производителя процессора.

Кратко: первые четыре регистра DR0-DR3, известные как регистры адресных точек останова, содержат виртуальные адреса, отслеживаемые процессором.

DR7, известный как регистр управления отладкой, содержит управляющую информацию о каждом таком адресе: длину блока, отслеживаемый тип доступа и состояние включения.

Листинг 3.31 показывает отладочные регистры до и после попадания в точку останова в отладчике режима ядра.

Листинг 3.31. Отладочные регистры на обычном процессоре

```
Before setting a breakpoint on access
kd> rM 20
dr0=00000000 dr1=00000000 dr2=00000000
dr3=00000000 dr6=ffff0fff dr7=00000400 cr4=00000699
ntdll!RtlAllocateHeap+0x5:
001b:77f57bb3 68781cf577 push 0x77f51c78
After setting a breakpoint on access (for execution)
kd> ba e1 77f57bae
kd> g
Breakpoint 0 hit
ntdll!RtlAllocateHeap:
001b:77f57bae 6808020000 push 0x208
kd> rM 20
dr0=77f57bae dr1=77f57bae dr2=00000000
dr3=00000000 dr6=ffff0fff1 dr7=00000501 cr4=00000699
ntdll!RtlAllocateHeap:
001b:77f57bae 6808020000 push 0x208
kd> .formats @dr7
Evaluate expression:
Hex: 00000501
Decimal: 1281
Octal: 0000002401
Binary: 00000000 00000000 00000101 00000001
Chars: ....
Time: Wed Dec 31 16:21:21 1969
Float: low 1.79506e-042 high 0
Double: 6.32898e-321
```

В этом случае в регистре управления отладкой установлены только два бита - бит 0 и бит 8, что означает, что включена точка останова 0.

Согласно спецификациям процессоров Intel, если нет дополнительной информации, например длины отслеживаемой точки останова или режима доступа, который нужно отслеживать, точка останова считается точкой останова на доступ при выполнении.

Как и обычные точки останова, точки останова на доступ отладчика ядра разделяются всеми процессами, выполняющимися в системе, и будут мешать любому отладчику пользовательского режима, работающему в той же системе.

Если с точкой останова столкнется отладчик пользовательского режима, не знающий причины этой остановки, такой отладчик возбудит исключение `STATUS_SINGLE_STEP`.

Трассировка процессора

Трассировка на уровне ассемблера, еще одна часто используемая возможность отладчиков, достигается с помощью native-возможностей трассировки процессора.

На процессорах x86 трассировка включается с помощью флага ловушки, обозначаемого в консоли отладчика как флаг `tf`.

Когда флаг установлен, процессор выполняет только текущую инструкцию, после чего возбуждает исключение `STATUS_SINGLE_STEP`.

Например, когда мы вводим команду `t` в консоли отладчика, отладчик устанавливает флаг ловушки в контексте потока и продолжает выполнение потока.

Когда новый контекст потока загружен и процессор возбуждает исключение `STATUS_SINGLE_STEP`, отладчик распознает исключение, сбрасывает флаг трассировки и останавливается после последней инструкции.

Такое поведение легко воспроизвести, установив флаг ловушки и разрешив цели отладчика выполняться, как показано в листинге 3.32.

В этом случае отладчик не знает о "запросе" на выполнение операции одиночного шага и просто показывает исключение в консоли.

Листинг 3.32. Имитация трассировки кода после подключения к выполняющемуся проекту

```
0:001> r tf=1
0:001> g
(608.6bc): Single step exception - code 80000004 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=7ffdf000 ebx=00000001 ecx=00000002 edx=00000003 esi=00000004 edi=00000005
eip=77f5f31f esp=0084ffd0 ebp=0084fff4 iopl=0 nv up ei pl zr na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=0038  gs=0000  efl=00000246
ntdll!DbgUiRemoteBreakin+0x2d:
77f5f31f eb07 jmp ntdll!DbgUiRemoteBreakin+0x36 (77f5f328)
```

Помимо трассировки одиночным шагом, новые процессоры постоянно расширяют возможности отладчиков, реализуя дополнительные возможности трассировки, например трассировку до следующего ветвления.

Управление состоянием потоков при живой отладке

Хотя трассировка является простым в использовании механизмом для однопоточных процессов, в многопоточных процессах она добавляет уровень непредсказуемости.

Когда задействовано несколько потоков, отладчик позволяет всем остальным потокам свободно выполняться, пока текущий поток выполняет инструкцию, через которую ожидается переход.

Если происходит переключение контекста потока, пользователь вводит `t` в отладчике, и тот попадает в другую точку останова, уже установленную в отладчике, вместо остановки на следующей инструкции.

Выполнение кода больше не следует единственному пути выполнения, что делает отслеживание одного потока выполнения, выполняющего конкретный сценарий, трудным, если не невозможным.

На самом деле нам хочется видеть в процессе один-единственный поток, чтобы управлять им знакомыми командами, а не использовать серию точек останова, ограниченных одним потоком, и тому подобное.

Чтобы уменьшить вероятность того, что несколько потоков будут выполнять одну и ту же последовательность кода, можно временно приостановить выполнение неинтересующих потоков и оставить в процессе один выполняющийся поток.

Как именно это работает?

Каждый раз, когда новое отладочное событие должно быть доставлено отладчику пользовательского режима, все выполняющиеся потоки процесса автоматически приостанавливаются ядром Windows на все время обработки события. Когда после обработки этого события отладчик решает продолжить выполнение, ядро возобновляет выполнение всех потоков процесса. У потоков, показанных в листинге 3.33, с каждым потоком связан счетчик приостановки, а также состояние Frozen/Unfrozen.

Листинг 3.33. Вывод состояния потоков

```
0:001> ~
0 Id: 1370.fc0 Suspend: 1 Teb: 7ffdf000 Unfrozen
. 1 Id: 1370.101c Suspend: 1 Teb: 7ffde000 Unfrozen
```

Счетчик приостановки потока представляет значение, распознаваемое ядром Windows и управляемое API SuspendThread и ResumeThread. Счетчиком приостановки также можно управлять из отладчика с помощью команды ~n или ~m. Поток с идентификатором <tid> можно приостановить следующей командой:

```
~<tid>n
```

Поток с идентификатором <tid> можно возобновить следующей командой:

```
~<tid>m
```

Если используются такие команды, как показано в листинге 3.34, убедитесь перед отключением отладчика от процесса, что счетчик приостановки сбалансирован числом команд возобновления. Приостановленный поток остается приостановленным навсегда. Также важно понимать побочный эффект приостановки конкретного потока для всего процесса. Например, большинство приложений с графическим интерфейсом используют один поток для получения и диспетчеризации оконных сообщений, соответствующих взаимодействиям пользователя.

Приостановка этого потока практически замораживает все приложение.

Приостановка потока, владеющего ресурсом, заставляет все остальные потоки, ожидающие тот же ресурс, блокироваться до возобновления этого потока.

Как и раньше, такое неограниченное ожидание воспринимается как зависание приложения.

Листинг 3.34. Как приостанавливать и возобновлять потоки

```
0:001> * Suspend the thread zero
0:001> ~0n
0:001> ~
0 Id: 1370.fc0 Suspend: 2 Teb: 7ffdf000 Unfrozen
. 1 Id: 1370.101c Suspend: 1 Teb: 7ffde000 Unfrozen
0:001> * Resume the thread zero
0:001> ~0m
0:001> ~
0 Id: 1370.fc0 Suspend: 1 Teb: 7ffdf000 Unfrozen
. 1 Id: 1370.101c Suspend: 1 Teb: 7ffde000 Unfrozen
```

Состояние Frozen/Unfrozen, обсуждавшееся выше, отличается от состояния приостановки, описанного в предыдущем разделе.

Состояние Frozen - чистая концепция отладчика без поддержки со стороны операционной системы Windows.

Для каждого замороженного потока отладчик запоминает это состояние и увеличивает его счетчик приостановки перед возобновлением обработки отладочного события.

Позже счетчик приостановки уменьшается при обработке нового события, поэтому счетчик выглядит неизменным.

Поток с идентификатором <tid> можно заморозить следующей командой:

```
~<tid>f
```

Поток с идентификатором <tid> можно разморозить следующей командой:

```
~<tid>u
```

Листинг 3.35 показывает пример каждой команды в действии.

Поскольку замороженный поток влияет на обычное выполнение процесса, отладчик напоминает пользователю о числе замороженных потоков каждый раз, когда обрабатывается новое событие.

Команды заморозки должны соответствовать командам разморозки так же, как команды приостановки соответствуют командам возобновления.

Достаточно интересно, что когда последний выполняющийся поток процесса заморожен, отладчик завершает целевой процесс, поскольку шансы на дальнейшую активность в этом процессе минимальны.

Листинг 3.35. Как замораживать и размораживать потоки

```
0:001> * Freeze thread number one
0:001> ~1f
0:001> * Dump thread status
0:001> ~
0 Id: 1098.1418 Suspend: 1 Teb: 7ffdf000 Unfrozen
. 1 Id: 1098.143c Suspend: 1 Teb: 7ffde000 Frozen
0:001> * Let the debugger target run
0:001> g
System 0: 1 of 2 threads are frozen
System 0: 1 of 3 threads were frozen
System 0: 1 of 3 threads are frozen
System 0: 1 of 3 threads were frozen
(1098.15fc): Break instruction exception - code 80000003 (first chance)
eax=7ffd9000 ebx=00000001 ecx=00000002
edx=00000003 esi=00000004 edi=00000005
eip=7c901230 esp=0092ffcc ebp=0092fff4
iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023
fs=0038 gs=0000 efl=00000246
ntdll!DbgBreakPoint:
7c901230 cc int 3
0:001> * Unfreeze thread number one
0:002> ~1u
0:001> * Dump thread status
0:002> ~
0 Id: 1098.1418 Suspend: 1 Teb: 7ffdf000 Unfrozen
1 Id: 1098.143c Suspend: 1 Teb: 7ffde000 Unfrozen
. 2 Id: 1098.15fc Suspend: 1 Teb: 7ffdd000 Unfrozen
```

Наконец, отладчик предлагает возможность заменить текущий выполняющийся поток любым другим потоком внутри процесса.

Это изменение временное и действует до тех пор, пока новый поток не потеряет квант выполнения из-за вытеснения, добровольной отдачи оставшейся части кванта времени или перехода в состояние ожидания.

Как видно в листинге 3.36, перед идентификатором текущего потока стоит точка (.).

Если текущий поток отличается от активного потока (потока, сгенерировавшего текущее событие), активный поток помечается знаком решетки (#) перед идентификатором потока.

Поток с идентификатором <tid> можно сделать активным следующей командой:

```
~<tid>s
```

Листинг 3.36. Изменение текущего потока

```
0:001> ~
0 Id: 3edc.1970 Suspend: 1 Teb: 7ffdf000 Unfrozen
. 1 Id: 3edc.44e8 Suspend: 1 Teb: 7ffde000 Unfrozen
0:001> ~0s
eax=0043de20 ebx=008f0507 ecx=00420000 edx=a4011de2 esi=0007fefc edi=77d491c6
eip=7c90eb94 esp=0007febc ebp=0007fed8 iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
```

```
ntdll!KiFastSystemCallRet:
7c90eb94 c3 ret
0:000> ~
. 0 Id: 3edc.1970 Suspend: 1 Teb: 7ffdf000 Unfrozen
# 1 Id: 3edc.44e8 Suspend: 1 Teb: 7ffde000 Unfrozen
```

Изменение текущего потока влияет на область действия всех команд, зависящих от текущего потока, и чрезвычайно полезно для сложных команд, таких как команда kb или команда расширения !teb.

Приостановка потока с помощью отладчика режима ядра

В настоящее время отладчик ядра не предлагает аналогичного способа изменить шаблон выполнения, например приостановить поток, возобновить поток или даже запланировать к выполнению другой поток вместо текущего.

Это недоступно по нескольким причинам: от сложности предоставления такой поддержки до безопасности подобного механизма.

Еще важнее то, что такая поддержка имеет ограниченную полезность в пространстве ядра, поскольку число потоков относительно велико.

Тем не менее эту функциональность можно имитировать с помощью уже доступной поддержки в отладчике ядра, если выполнены несколько условий.

Сценарий, требующий такой функциональности, представлен в оставшейся части этого раздела.

Предположим, что интересующий нас процесс останавливается в отладчике режима ядра в результате выполнения оператора `DebugBreak()`.

Процесс не может продолжаться после встречи с этой остановкой, и любая попытка продолжить выполнение за точкой останова завершает процесс.

Остановка часто является прямым результатом нарушения одного из инвариантов процесса, например целостности кучи или, возможно, выхода значения глобальной переменной за ожидаемый диапазон.

Виртуальное адресное пространство, содержащее подсказки об остановке, сейчас не загружено в RAM, но доступно в файле подкачки.

Команду `.pagein` можно использовать, чтобы вернуть необходимые страницы в память.

Цель отладчика должна выполняться, чтобы запланировать поток, который выполнит фактическую операцию `page-in`.

Из-за недетерминированной природы процесса `page-in` прежний поток, вызвавший остановку, может выполняться и завершить процесс.

Решение, позволяющее избежать этого сценария, - остановить сбойный поток от выполнения кода завершения, переведя его в состояние ожидания.

Пока этот поток ожидает, `.pagein` можно вызывать сколько угодно раз без страха потерять текущий сеанс живой отладки.

Поток можно легко перевести в состояние ожидания, изменив его текущий указатель инструкции и заставив поток выполнить `API kernel32!Sleep`.

Этот API принимает один параметр, представляющий длительность сна в миллисекундах.

Стек текущего выполняющегося потока нужно изменить, чтобы имитировать состояние перед вызовом стандартного API с одним параметром.

Контекст нужно изменить так, чтобы он соответствовал обновленному указателю стека, а указатель инструкции нужно обновить так, чтобы он соответствовал начальному адресу вызываемого API.

Когда поток продолжает выполнение, он переходит в режим сна на длительность, полученную из стека, как показано в листинге 3.37.

Листинг 3.37. Имитация вызова `kernel32!Sleep`

```

kd> r
eax=0040136f ebx=7ffdf000 ecx=004011d0 edx=00262649 esi=00000002 edi=00000000
eip=77f75a58 esp=0006fee8 ebp=0006fef0 iopl=0
nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=0038 gs=0000 efl=00000206
ntdll!DbgBreakPoint:
001b:77f75a58 cc int 3
kd> ed esp-4 <time>
kd> ed esp-8 .
kd> resp=@esp-8
kd> reip=kernel32!Sleep
kd> .pagein <address>;g
...

```

В течение всей длительности сна отладчик можно использовать для подкачки множества страниц без страха потерять процесс или неожиданно изменить состояние.

При необходимости в этом состоянии можно даже запустить отладчик пользовательского режима и отлаживать сбойный процесс изнутри целевой системы, если система доступна.

Независимо от метода, использованного для завершения исследования, поток возвращается в исходное место после истечения тайм-аута.

Даже если регистры, обычно сохраняемые в `__stdcall`, в этом случае сохранены, попытка продолжить выполнение процесса после этой точки опасна.

Итоги

В этой главе вы узнали, как отладчик взаимодействует с операционной системой при отладке процесса и как эффективно управлять всеми отладочными событиями и исключениями в своих интересах.

Затем вы узнали, как система реагирует на разные исключения и как использовать эту информацию в повседневной отладке.

Наконец, мы исследовали механизмы, доступные для управления состоянием потока как с помощью поддержки отладчика, так и через ручные изменения состояния процесса.

С этой информацией можно определить ясную стратегию отладки для разных ситуаций и использовать возможности отладки в своих интересах.

Глава 4. Управление файлами символов и исходного кода

Представьте на минуту, что флагманский продукт вашей компании сталкивается с проблемой на небольшой, но значимой группе систем, и вас просят решить эту проблему, используя файлы дампов памяти, присланные заказчиком. Вы загружаете дампы памяти в отладчик, чтобы выяснить, что не так. Поскольку без правильных файлов символов функциональность отладчика ограничена, необходимо найти файлы символов, соответствующие версии приложения и созданные во время сборки приложения. Если эти файлы символов найти невозможно, единственный вариант - вернуться к заказчику и предоставить оправдания вместо решений.

Управление символами доказанно экономит время инженеров, отлаживающих программные системы, и его важность нельзя недооценивать; эта экономия продолжает окупаться в течение всего срока жизни продукта. Тщательно спроектированная политика управления символами дает косвенную бизнес-ценность по сравнению с ситуативной или отсутствующей политикой. При надежной политике управления символами компания отвечает за свои продукты, своевременно устраняет проблемы и выпускает более стабильную будущую версию.

Microsoft Debugging Tools for Windows предоставляет инструменты, необходимые для настройки сервера символов, и подготавливает символы для поддержки механизма source server. Стоимость настройки сервера символов пропорциональна стоимости хранения, которая продолжает резко снижаться. В этой главе мы рассмотрим:

- как настроить и постоянно сопровождать частный сервер символов;
- как настроить и постоянно сопровождать публичный сервер символов;
- как подготовить файл символов для постоянной поддержки source server.

Все отладчики, установленные вместе с Microsoft Debugging Tools for Windows, используют эти серверы. Все версии Visual Studio .NET способны использовать сервер символов. Source server поддерживается Visual Studio 2005 Professional и Visual Studio Team Editions. Символы всегда должны пониматься всеми средствами отладки, доступными на конкретной платформе. Так инженеры могут переходить от одного инструмента к другому, будучи уверенными, что у них есть вся необходимая информация.

Управление символами для отладки

В главе 2, "Введение в отладчики", важность использования правильных файлов символов подчеркивалась неоднократно: от задания правильных символов до их проверки. Более простая отладка после выпуска продукта - вся причина внедрения сильной политики управления символами. Как общее правило, каждый двоичный файл, устанавливаемый на разные системы на срок дольше немедленного тестирования, должен иметь свой файл символов, проиндексированный на сервере символов, который проживет дольше самого двоичного файла.

Процесс управления символами начинается в момент сборки набора двоичных файлов, входящих в продукт, который будет установлен и использоваться в течение длительного времени. Если разработчики уверены, что в продукте нет ошибок, или продукт не нуждается в поддержке и следующая версия не использует текущий код, процесс может остановиться здесь. Все остальные начинают процесс подготовки созданных файлов символов к долгосрочному сопровождению.

Вместе с двоичными файлами компилятор создает связанные с ними файлы символов в формате PDB, содержащие все частные символы. Эти файлы символов содержат ссылки на все исходные файлы, использованные для сборки продукта. Каждый символ, соответствующий исполняемому адресу в двоичном файле, содержит ссылку на строку исходного кода, использованную для его создания. Большинство компаний, включая Microsoft, считают, что такая подробная информация раскрывает интеллектуальную собственность, встроенную в продукт, поэтому они предпочитают раскрывать только ее часть в виде публичных символов. Поэтому такие компании хранят оба типа файлов в двух разных местах. Частные файлы символов хранятся в защищенном месте, а публичные файлы символов обычно размещаются на общедоступном HTTP-сервере. Это позволяет пользователям приложения понять, почему приложение завершилось сбоем, когда это происходит, чего иногда достаточно, чтобы определить, что нужно сделать для устранения проблемы.

Microsoft публикует публичные символы для большинства приложений на сервере символов, расположенном по адресу <http://msdl.microsoft.com/download/symbols>.

Создание публичных символов

В этой главе мы покажем, как интегрировать управление файлами символов в процесс сборки - в данном случае в процесс, используемый для сборки файлов примеров книги. Мы начинаем с создания урезанных файлов символов, называемых публичными файлами символов, из частных файлов символов. Мы используем утилиту `binplace.exe`, установленную вместе с Windows WDK; она также помогает организовывать двоичные файлы после их сборки. Если дополнительная функциональность, предлагаемая `binplace.exe`, не нужна, для создания публичных файлов символов можно использовать инструмент `pdbcopy.exe`, поставляемый с Debugging Tools for Windows.

Следующие шаги выполняются из окна командной строки, созданного ярлыком Windows WDK. Предполагается, что другие инструменты, например инструменты отладчика, присутствуют в PATH, как требуется в листингах этой главы. В этой главе мы повторно используем исходный код и двоичный файл `03sample.exe`, представленные в предыдущей главе.

`Binplace.exe` - мощный инструмент, чрезвычайно полезный для крупных проектов. Он может запускаться в конце фазы сборки, чтобы перемещать файлы в различные места (отсюда и название `binplace`) и обрабатывать файлы символов. В этом разделе мы используем `binplace.exe`, чтобы поместить двоичные файлы в одно место и извлечь информацию публичных символов из частного символа, созданного компилятором. `Binplace.exe` использует файл инструкций обработки, где каждая строка рассматривается как инструкция, указывающая, как обрабатывать этот файл. В листинге 4.1 показано содержимое файла `placefil.txt`, используемого для постобработки наших примерных двоичных файлов.

Листинг 4.1

```
C:\>type c:\awd\placefil.txt
02sample.exe retail
03sample.exe retail
```

Команда `binplace.exe` вызывается для каждого двоичного файла, который передается команде как параметр. Имя двоичного файла используется как индекс в файле инструкций обработки. Сопоставление выполняется сравнением имени двоичного файла с именами, сохраненными в первом столбце. В нашем случае у нас есть строка для каждого EXE или DLL, за которой следует специальная строка `retail`, указывающая место размещения в выходной папке двоичных файлов.

Чтобы помочь понять все доступные параметры, справка WDK содержит несколько тем, посвященных команде `binplace.exe`: они описывают синтаксис `place`-файла, все параметры командной строки, а также все переменные среды, учитываемые `binplace.exe`. Много сведений можно найти на сайте MSDN, выполнив поиск строки `binplace` (без расширения `.exe`).

Как и у большинства инструментов командной строки, поведение `binplace.exe` зависит от переменных среды; некоторые переменные обязательны. Другие параметры передаются как аргументы командной строки. В нашем сценарии инструмент зависит от следующих параметров:

- целевое расположение двоичных файлов, предоставляемое через переменную среды `_NT386TREE`, `_NTAMD64TREE` или `_NTIA64TREE` в зависимости от платформы, на которую нацелены двоичные файлы, обрабатываемые `binplace.exe`; указанная целевая папка содержит все результирующие двоичные файлы;
- расположение `placefile.txt`, предоставляемое через переменную среды `BINPLACE_PLACEFILE`, содержит инструкции обработки для всех файлов проекта;
- целевое расположение частных файлов символов, передаваемое как аргумент ключа командной строки `-n`, представляет место, где находятся частные файлы символов;
- целевое расположение публичных файлов символов, передаваемое как аргумент ключа командной строки `-s`, представляет место, где находятся частные файлы символов;

- другие ключи командной строки, -a и -x, указывают binplace.exe удалить частные символы из публичного файла символов и удалить любые символы из самого двоичного файла;
- расположение двоичного файла, который мы собираемся обработать, передается последним параметром.

Листинг 4.2 взят из командной строки, использованной для задания этих переменных и выполнения операции bin place. В ответ binplace.exe показывает имя успешно размещенного файла. Обратите внимание, что при ошибке вывода нет.

Листинг 4.2

```
C:\> set _NT386TREE=C:\AWDBIN\WinXP.x86.chk
C:\> set BINPLACE_PLACEFILE=C:\awd\placefil.txt
C:\> binplace -a -x -s %_NT386TREE%\sym.pub -n %_NT386TREE%\sym.pri
chapter3\objchk_wxp_x86\i386\03sample.exe
binplace C:\awd\chapter3\objchk_wxp_x86\i386\03sample.exe
```

Утилита binplace.exe вызывается повторно для каждого двоичного файла. В конце целевая папка содержит все двоичные файлы, все частные файлы символов и все публичные файлы символов. Весь процесс можно автоматизировать, как видно в пакетном файле release.cmd, установленном вместе с файлами примеров. Дерево целевой папки, созданное после этой операции, выглядит примерно как в листинге 4.3.

Листинг 4.3

```
C:\AWD>tree c:\AWDBIN\WinXP.x86.chk /F/A
Folder PATH listing
Volume serial number is 00310030 B817:38E9
C:\AWDBIN\WinXP.X86.CHK
+--03sample.exe
|
| +--sym.pri
| | --retail
| | --exe
| | --03sample.pdb
|--
| --sym.pub
| | --retail
| | --exe
| | 03sample.pdb
```

Во время процесса bin-placing содержимое debug directory, хранящееся в заголовках исполняемого файла, корректируется, а исходное расположение файла символов удаляется. Debug directory можно просмотреть командой link.exe, как показано в главе 2. Листинг 4.4 показывает содержимое debug directories до операции bin place, а листинг 4.5 - после нее.

Листинг 4.4

```
C:\AWD>link -dump -headers
c:\AWD\chapter3\objchk_wxp_x86\i386\03sample.exe
Microsoft (R) COFF/PE Dumper Version 8.00.50727.220
Copyright (C) Microsoft Corporation. All rights reserved.
Dump of file c:\awd\chapter3\objchk_wxp_x86\i386\03sample.exe
...
Debug Directories
Time Type Size RVA Pointer
---- --
45A417D2 cv 49 00001810 C10 Format: RSDS, {B10B7ACC-81C5-4533-
AFE4-5AF20D9B7A09}, 1, c:\awd\chapter3\objchk_wxp_x86\i386\03sample.pdb
...
```

Листинг 4.5

```
C:\AWD>link -dump -headers c:\AWDBIN\WinXP.x86.chk\03sample.exe
Microsoft (R) COFF/PE Dumper Version 8.00.50727.220
Copyright (C) Microsoft Corporation. All rights reserved.
Dump of file c:\AWDBIN\WinXP.x86.chk\03sample.exe
```



```

...
Debug Directories
Time Type Size RVA Pointer
-----
45A417D2 cv 25 00001810 C10 Format: RSDS, {B10B7ACC-81C5-4533-
AFE4-5AF20D9B7A09}, 1, 03sample.pdb
...

```

Хранение символов в хранилище символов

После обработки каждого двоичного файла с помощью `binplace.exe` папка публичных символов содержит дерево со всеми публичными файлами символов, а папка частных символов содержит дерево со всеми частными файлами символов. Хотя кажется возможным хранить каждую версию такого дерева в отдельном месте и обращаться к его файлам при отладке любого модуля, созданного этой версией сборки, процесс получается утомительным и неэффективным. Нужно вести много учета, чтобы гарантировать, что ни один символ никогда не будет потерян. Любая группа, выполняющая ежедневные сборки на нескольких платформах, сочтет этот процесс очень трудоемким и попытается автоматизировать его. К счастью, весь процесс организации файлов символов и их обнаружения при необходимости уже автоматизирован набором инструментов и технологий, называемым `symbol server`. В этом разделе описано, как организовать символы для создания информации сервера символов.

Debugging Tools for Windows предоставляет инструмент `symstore.exe`, который сканирует папку, собирает все исполняемые модули вместе с соответствующими символами и организует их в структуру, распознаваемую клиентом сервера символов, работающим в отладчике. Файлы символов организуются на основе их имен и GUID, сохраненного после строки RSDS, показанной в листинге 4.5. Двоичные файлы индексируются на основе их имени и временной метки компиляции.

Поскольку есть две категории символов, инструмент можно использовать для создания двух хранилищ символов - одного с публичными и одного с частными файлами символов. У инструмента очень много параметров, все они хорошо описаны в справке отладчика Windows. В этом разделе мы вызываем `symstore.exe` со следующими параметрами:

- `/f` указывает папку двоичных файлов, использованную как аргумент для `binplace.exe`;
- `/s` указывает расположение хранилища символов;
- `/r` заставляет `symstore.exe` рекурсивно просканировать все файлы в папке;
- `/z` указывает, какие типы символов извлекать: `pri` означает частные символы, а `pub` - публичные символы.

Результат двукратного запуска команды, один раз для публичной папки и один раз для частной, показан в листинге 4.6. Команда выводит статистику операции, которую необходимо проанализировать на наличие ошибок. Файлы, проигнорированные в листинге 4.6, - это символы, не соответствующие требуемому типу: публичный файл символов, когда были запрошены только частные файлы символов, и наоборот.

Листинг 4.6

```

Creating public symbol store
C:\AWD>symstore.exe add /F C:\AWDBIN\WinXP.x86.chk /S
C:\AWDBIN\symstore.pub /t book /r /z pub
Finding ID... 0000000001
SYMSTORE: Number of files stored = 2
SYMSTORE: Number of errors = 0
SYMSTORE: Number of files ignored = 1
Creating private symbol store
C:\AWD>symstore.exe add /F C:\AWDBIN\WinXP.x86.chk /S
C:\AWDBIN\symstore.pri /t book /r /z pri
Finding ID... 0000000001
SYMSTORE: Number of files stored = 2

```

```
SYMSTORE: Number of errors = 0
SYMSTORE: Number of files ignored = 1
```

В результате выполнения этих команд в локальной файловой системе создаются два очень простых хранилища символов. Даже когда на сервере символов хранится только одна версия файла, при задании такого сервера отладчик автоматически выбирает правильный файл символов. После нескольких пересборок проекта легко понять, почему автоматическое управление символами настолько проще ручного учета. Вместо того чтобы хранить все файлы отдельно с использованием каких-то вручную определенных ключей, все выполняется инструментами. Процесс повторяется каждый раз, когда мы собираем продукт: один раз для каждой архитектуры процессора или набора параметров компиляции. Все файлы символов сохраняются на одном сервере символов. Структуру дерева одного из хранилищ можно изучить в листинге 4.7.

Листинг 4.7

```
C:\AWD>tree c:\AWDBIN\symstore.pri /F/A
Folder PATH listing
Volume serial number is B817-38E9
C:\AWDBIN\SYMSTORE.PRI
+--pingme.txt
|
| +--000Admin
| | 0000000001
| | 0000000002
| | 0000000003
| | history.txt
| | lastid.txt
| | server.txt
|
| \--03sample.exe
| \--45A417D214000
| \--03sample.exe
| \--refs.ptr
|
| +---45A4624314000
| | \---03sample.exe
| | \---refs.ptr
|
| +---45A4625414000
| | \---03sample.exe
| | \---refs.ptr
|
+---03sample.pdb
| +---A69EEFF7C43B400799E03BF7BCF55A9B1
| | \---03sample.pdb
| | \---refs.ptr
|
| +---B10B7ACC81C54533AFE5AF20D9B7A091
| | \---03sample.pdb
| | \---refs.ptr
|
| +---FF76A7EC166D489C943F238F76FCB32F1
| | \---03sample.pdb
| | \---refs.ptr
```

Структура частного и публичного хранилища символов идентична, но их содержимое различается. Эта простая модель организации работает для небольшого или среднего проекта, требующего разумного объема дискового пространства. Для более крупных проектов у `symstore.exe` есть различные другие параметры, позволяющие инструменту `symstore.exe` создавать более сложное хранилище, например хранилища с файлами символов, размещенными в нескольких местах, или со сжатыми файлами. Справка `symstore.exe` описывает различные параметры, поддерживаемые инструментом и применимые для создания таких сложных хранилищ.

Затем папку частных символов можно разместить на файловой шаре и использовать всеми пользователями через UNC-адрес шары, например `\\symserver\symbols`. Это UNC-расположение

становится сервером символов, используемым как путь к символам в отладчиках, следующим образом:

```
0:000> !sympath srv*\\symserver\symbols
Symbol search path srv*\\symserver\symbols
```

Каждая операция индексирования символов получает идентификатор транзакции, который можно использовать для дальнейших операций управления символами. Обычно идентификатор транзакции используется, чтобы удалить из хранилища символов все файлы символов, соответствующие промежуточным выпускам. Например, в листинге 4.8 мы используем инструмент `symstore.exe`, чтобы удалить файл, добавленный в транзакции `0000000001`, показанной в листинге 4.6.

Листинг 4.8

```
C:\AWD> symstore del /i 0000000001 /s c:\awdbin\symstore.pri
Finding ID... 0000000004
SYMSTORE: Number of references deleted = 0
SYMSTORE: Number of files/pointers deleted = 2
SYMSTORE: Number of errors = 0
```

Теперь мы можем опубликовать публичные файлы символов на интернет-сервере. Этот процесс описан в следующем разделе.

Совместное использование публичных символов на HTTP-сервере

Последний шаг - сделать публичные символы действительно публичными, предоставив их через HTTP-сервер символов. Хотя это может показаться сложной задачей, на самом деле все довольно просто. Папку хранилища публичных символов, созданную ранее, нужно добавить как виртуальный каталог на веб-сервере, хранящем символы. HTTP-сервер должен быть настроен на выдачу файлов символов как `application/octet-stream`, как показано на рисунке 4.1.

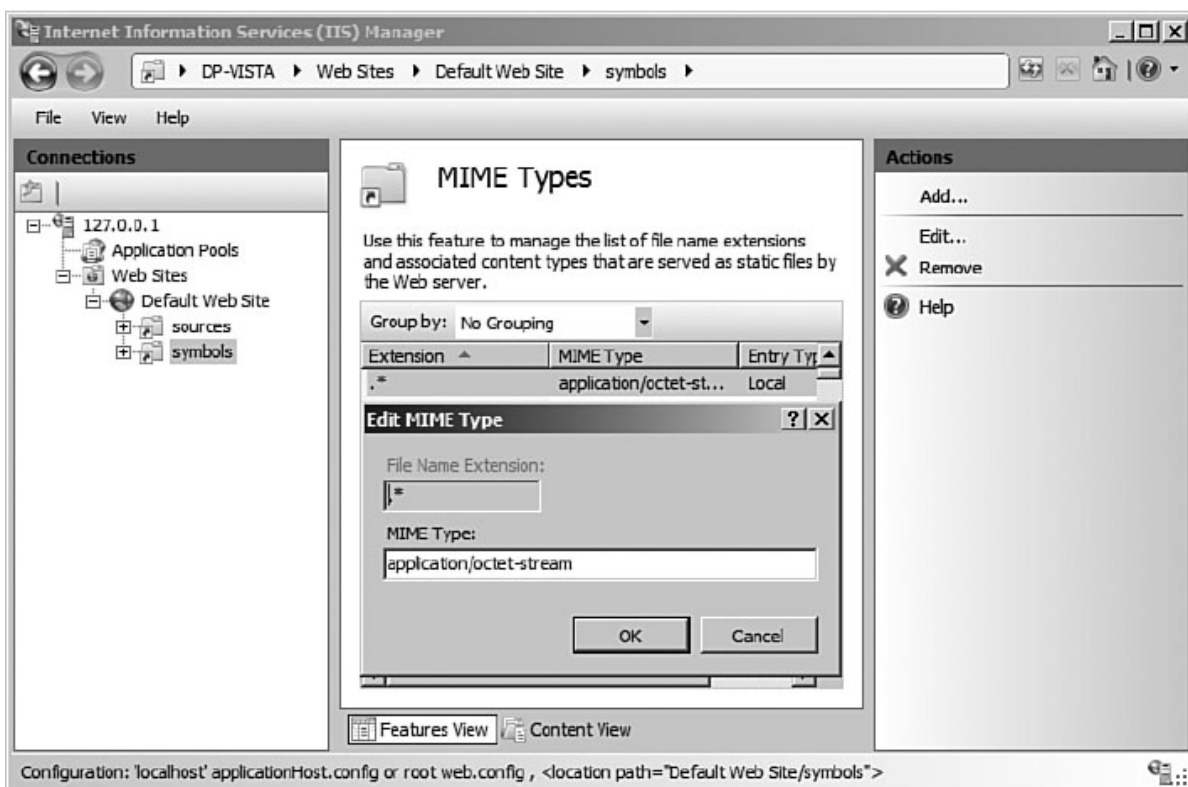


Рисунок 4.1. Настройка MIME-типа (оснастка Vista IIS)

Пошаговые инструкции доступны в документе `symhttp.doc`, установленном вместе с Debugging Tools for Windows в папке `symproxy`. Новый URL сервера, если предположить, что символы находятся в виртуальной папке `symbols`, можно использовать так:

```
0:000> !sympath srv*http://127.0.0.1/symbols  
Symbol search path is: srv*http://127.0.0.1/symbols
```

После чтения этого раздела вы знаете, какие инструменты можно использовать для автоматизации управления файлами символов с минимальными накладными расходами. Следующий раздел идет еще дальше и описывает, как подготовить файлы символов с информацией `source server`.

Управление файлами исходного кода для отладки

Хотя первичную сортировку большинства проблем можно выполнить, имея доступ только к правильным частным (или даже публичным) символам, инженеры должны также подтвердить проблему, анализируя исходные файлы. Когда рассматриваемые исходные файлы прошли через множество изменений, важно найти именно тот файл, который использовался для создания двоичного файла. Именно решение этой задачи мы покажем в этом разделе.

Если продукт не собирается и не выпускается только один раз - в таком случае у каждого двоичного файла есть единственный набор связанных с ним исходных файлов, - исходные тексты обычно управляются системой контроля версий. Существует множество вариантов: от продуктов с открытым исходным кодом, таких как Concurrent Versions System (CVS) или ее преемник Subversion (SVN), до коммерческих систем, таких как ClearCase от IBM, Visual SourceSafe от Microsoft или Perforce от одноименной компании. Debugging Tools for Windows предоставляет механизм, с помощью которого некоторые сведения, связанные с исходными файлами, сохраняются в файле символов как часть процесса сборки и используются позднее, когда соответствующий модуль загружается в отладчик.

Сбор информации об исходных файлах

Этот механизм называется Source Server и работает совместно с системой контроля версий исходного кода. Debugging Tools for Windows имеет встроенную поддержку Perforce, Visual SourceSafe и Subversion, но его можно расширить для другой системы контроля версий. В следующем разделе показано, как использовать этот механизм. Ревизии исходного кода контролируются Visual SourceSafe. Для повторного создания шагов, связанных с взаимодействием с системой контроля версий, этот раздел требует рабочего знания Visual SourceSafe. С Perforce или Subversion шаги похожи, если не проще. Процесс создания информации об исходных файлах показан на рисунке 4.2.

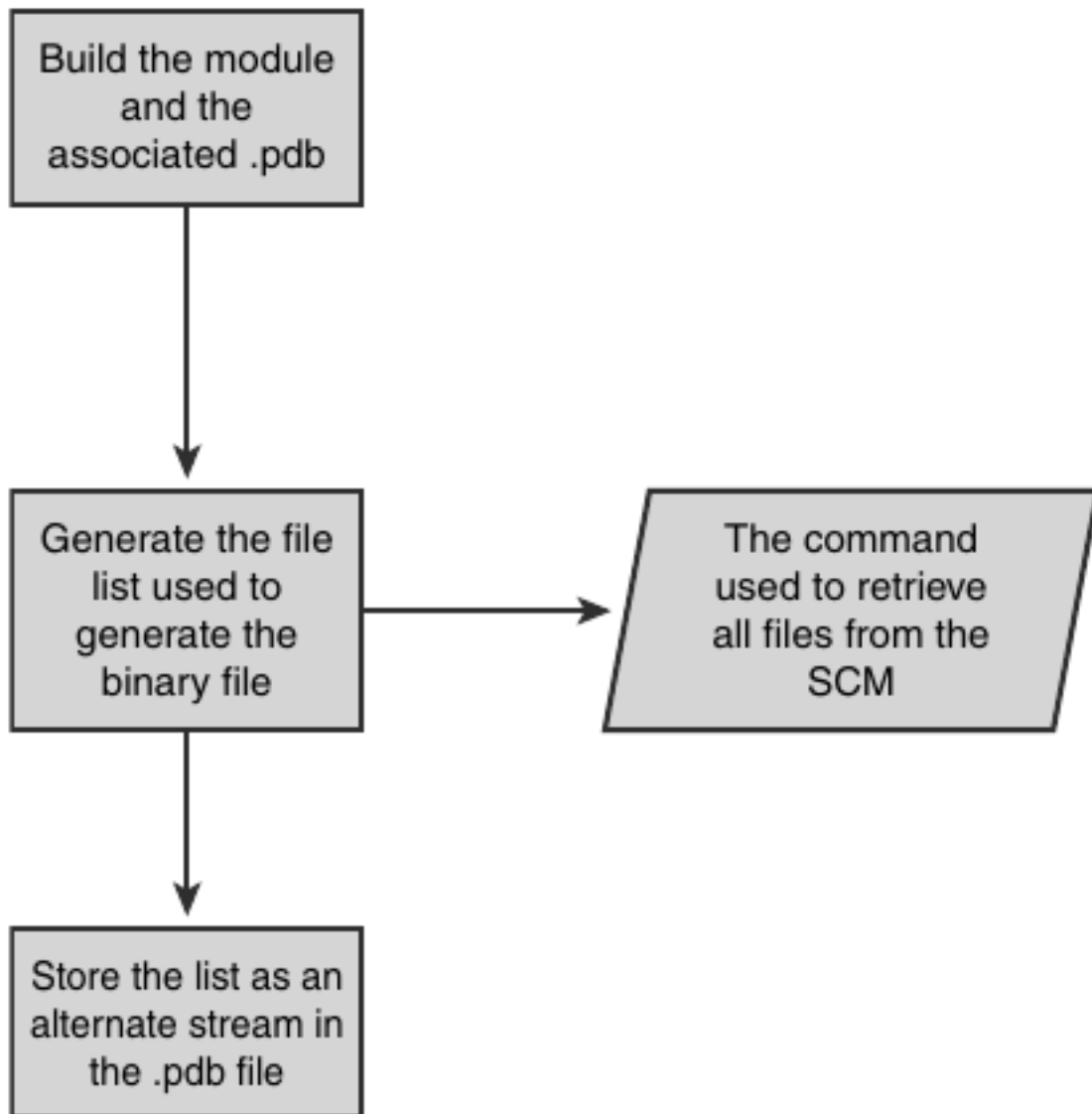


Рисунок 4.2. Процесс создания информации об исходном коде

Инструменты source server основаны на Perl, который необходимо установить до запуска процесса. В нашем случае мы использовали ActivePerl, который можно бесплатно загрузить с сайта www.ActiveState.com. Инструменты source server устанавливаются при выборе варианта SDK во время установки Debugging Tools for Windows. В установочной папке документ `sdk\srcsrv\srcsrv.doc` подробно описывает весь процесс. Расположение source server, а также расположение установки Visual SourceSafe должны присутствовать в PATH, задаваемом следующей командной строкой (зависящей от места установки):

```
C:\awd>set PATH=%PATH%;C:\Program Files\Microsoft Visual  
SourceSafe;C:\debug.x86\sdk\srcsrv
```

Следующий шаг - задать переменную среды SSDIR, указывающую на базу данных Visual SourceSafe, которая поддерживает файл проекта, как показано ниже, если предположить, что база данных хранится в папке `C:\AWD\VSS`:

```
C:\awd>set SSDIR=C:\AWD\VSS
```

Для простоты мы предполагаем, что все файлы, сохраненные в базе данных VSS, имеют структуру, похожую на структуру папок на диске.

Перед сохранением файлов символов на сервере символов необходимо обработать их, чтобы внедрить информацию об исходных файлах, которую отладчик будет использовать для извлечения файла из системы контроля версий. Этот процесс выполняется запуском инструмента индексирования source server, `ssindex.cmd`, предоставленного в папке source server. `ssindex.cmd` требует нескольких параметров, наследуемых из среды или передаваемых как аргументы команды; самые важные из них - имя системы контроля версий, в данном случае VSS, и расположение файлов символов.

Для правильной работы файл `srcsrv.ini`, расположенный в папке source server, должен быть обновлен одной строкой, содержащей расположение базы данных VSS. Левая часть знака равенства представляет имя проекта, а правая - адрес системы контроля версий. В этом случае вся строка выглядит так:

```
AWD=C:\AWD\VSS
```

При использовании VSS `ssindex.cmd` требует передать метку ревизии как параметр, поскольку ее нельзя вывести из исходных файлов. Команда выполняется из корневой папки проекта, соответствующей корневому расположению в базе данных VSS, где каждая подпапка является проектом в той же базе данных. Файлы были помечены номером ревизии с помощью инструмента командной строки `ss.exe`, предоставленного Visual SourceSafe, как в следующем листинге:

```
C:\AWD>ss cp \  
Current project is $/  
C:\AWD>ss Label  
Label for $/: VERSION1  
Comment for $/: Advanced Windows Debugging source code
```

После связывания всех файлов с вручную выбранной информацией о версии мы можем запустить команду индексирования для всех файлов, сохраненных в расположении bin place, следующим образом:

```
C:\AWD>ssindex /SYSTEM=VSS /LABEL=VERSION1 /SYMBOLS=%_NT386TREE%  
-----  
ssindex.cmd [STATUS] : Server ini file:  
d:\debug.x86\sdk\srcsrv\srcsrv.ini  
ssindex.cmd [STATUS] : Source root : C:\AWD  
ssindex.cmd [STATUS] : Symbols root : C:\AWDBIN\WINXP.X86.CHK\sym.pri  
ssindex.cmd [STATUS] : Control system : VSS  
ssindex.cmd [STATUS] : VSS Server : C:\AWD\VSS  
ssindex.cmd [STATUS] : VSS Client Root: C:\AWD  
ssindex.cmd [STATUS] : VSS Project : $/  
ssindex.cmd [STATUS] : VSS Label : VERSION1  
-----  
ssindex.cmd [STATUS] : Running... this will take some time...  
ssindex.cmd [STATUS] : Processing vssdump.exe output ...
```

Результат этого процесса можно проверить командой `srctool.exe`, которая способна показывать информацию source server, сохраненную в файле символов. Инструмент `srctool.exe` также можно использовать для извлечения необработанной информации об исходных файлах из PDB-файла и для получения исходного файла из системы контроля версий. Хорошая практика - периодически использовать этот инструмент для проверки корректности процесса индексирования исходников.

Инструмент `srctool.exe` показывает имя исходного файла, а также командную строку, необходимую для извлечения именно этого файла из системы контроля версий. Результат обработки `03sample.pdb` показан в листинге 4.9.

Листинг 4.9

```
C:\AWD>SrcTool.exe %_NT386TREE%\sym.pri\retail\exe\03sample.pdb  
[c:\awd\chapter3\spydbg.cpp] cmd: ss.exe get -  
GL"C:\AWD\AWD\chapter3\spydbg.cpp\  
VERSION1" -GF- -I-Y -W  
"$\chapter3\spydbg.cpp" -V"VERSION1"
```

```
c:\AWDBIN\WinXP.X86.chk\sym.pri\retail\exe\03sample.pdb: 1 source files are indexed -
494 are not
```

Если сбор исходных файлов завершился неудачно и предыдущий листинг пуст, `ssindex.cmd` можно запустить с параметром `/debug`, чтобы выяснить, какая часть процесса индексирования исходников дает сбой. Когда исходные файлы контролируются VSS, инструмент `vssdump.exe` также можно использовать, чтобы понять, какая метка ревизии связана с исходными файлами.

Затем инструмент `pdbstr.exe` используется для извлечения или изменения информации, сохраненной в файле символов. Например, следующая командная строка извлекает информацию `source server`, показанную в листинге 4.10. Информация `source server` хранится под именем потока `srcsrv`, которое передается как значение параметра `-s` для `pdbstr.exe`.

```
C:\>pdbstr -r -p:%_NT386TREE%\sym.pri\retail\exe\03sample.pdb -s:srcsrv
```

Листинг 4.10

```
SRCSRV: ini -----
VERSION=1
INDEXVERSION=2
VERCTRL=Visual Source Safe
DATETIME=Mon Jan 8 00:04:15 2007
SRCSRV: variables -----
SSDIR=C:\AWD\VSS
SRCSRVENV=SSDIR=%AWD%
VSSTRGDIR=%targ%\%var2%\%fnbks1%(var3%)\%var4%
VSS_EXTRACT_CMD=ss.exe get -GL"%vsstrgdir%" -GF- -I-Y -W "$/%var3%" -V"%var4%"
VSS_EXTRACT_TARGET=%targ%\%var2%\%fnbks1%(var3%)\%var4%\%fnfile%(var1%)
AWD=C:\AWD\VSS
SRCSRVTRG=%VSS_extract_target%
SRCSRVCMD=%VSS_extract_cmd%
SRCSRV: source files -----
c:\awd\chapter3\spydbg.cpp*AWD*chapter3\spydbg.cpp*VERSION1
SRCSRV: end -----
```

Использование информации об исходных файлах

Каждый файл символов, обработанный `ssindex.cmd`, содержит команды, необходимые для извлечения каждого исходного файла из системы контроля версий. Командная строка, сохраненная в файле символов и показанная в листинге 4.8, может получить файл из Visual SourceSafe.

Эта информация в первую очередь используется Debugging Tools for Windows, которые реализуют эту функциональность в `symsrv.dll`, доступной через функцию `DbgHelp SymGetSourceFile`. WinDbg использует информацию `source server`, чтобы извлекать исходный код из любой системы контроля версий. Консольные отладчики `ntsd.exe`, `cdb.exe` и `kd.exe` могут использовать только исходные файлы, сохраненные в UNC-шапе или на HTTP-сервере, организованном как `source server`, как описано в следующем разделе, "Source Server без системы контроля версий".

Механизм `source server` включается, когда путь к исходникам в отладчике содержит строку `SRV*`, заданную командой `.srcpath SRV*` в приглашении или, в случае `windbg.exe`, через пункт меню `Source symbol Path` в меню `File`. Отладчики проверяют файл символов, соответствующий текущему адресу выполнения, из которого извлекается информация об исходном коде, связанная с этим символом. Если она присутствует, информация `source server` используется для получения локальной копии исходного файла, кэшированной в папке `SRC` под папкой установки отладчика.

Как извлекается файл? Если отладчик не был настроен иначе, он напрямую выполняет команду, показанную в листинге 4.8. Это требует, чтобы система контроля версий была установлена и правильно настроена на системе, используемой для отладки. Также требуется доступ к системе контроля версий для выполнения команды, извлекающей файл, как видно на рисунке 4.3.

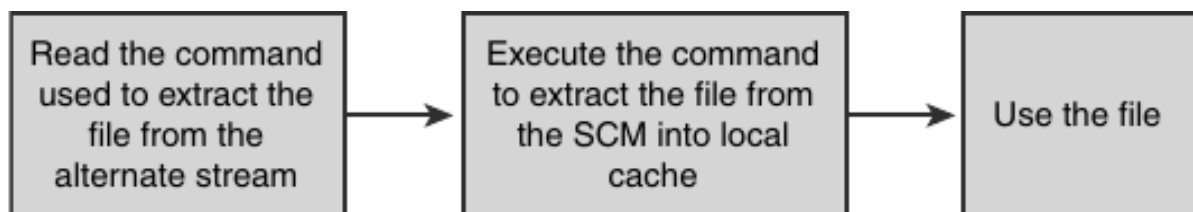


Рисунок 4.3. Использование информации об исходном коде

Хотя эти ограничения немного влияют на производительность в некоторых сценариях, особенно когда приложение отлаживается без должного доступа к системам контроля версий, они обеспечивают защиту исходного кода. Поскольку команда, используемая для извлечения файла, извлекается из файла, находящегося на сервере символов, скорее всего HTTP-сервере, отладчик запрашивает разрешение пользователя на выполнение команды.

Диалоговое окно предупреждения безопасности, показанное на рисунке 4.4, содержит командную строку, готовую к выполнению. Ее необходимо оценить перед принятием, особенно если сервер символов или происхождение PDB не являются доверенными. После кэширования исходного файла дополнительные диалоговые окна для этой версии файла не показываются, независимо от того, какие другие компоненты используют этот исходный файл.



Рисунок 4.4. Запрос WinDbg.exe перед выполнением команды системы контроля исходного кода

Source Server без системы контроля версий

Когда авторизация доступа к исходному коду не контролируется системой контроля версий, исходные файлы можно хранить на простой UNC-шаре или HTTP-сервере. Доступ к исходному коду затем ограничивается механизмом авторизации, поддерживаемым серверным хранилищем.

Доступ к HTTP-серверу можно ограничить разными механизмами, от базовой аутентификации до аутентификации по клиентскому сертификату; все они поддерживаются отладчиками. Перенос расположения исходников из системы контроля версий на HTTP-сервер можно выполнить в три шага:

1. Сначала мы извлекаем все исходные файлы из системы контроля версий, используя информацию source server, сохраненную процессом индексирования исходников, описанным ранее в разделе "Сбор информации об исходных файлах". Извлечение файлов выполняется с помощью `srctool.exe` с параметром `-x` для каждого созданного PDB-файла. Набор инструментов source server предоставляет вспомогательный пакетный файл `walk.cmd`, который может перечислить все файлы из заданной папки и передать каждое имя файла другой команде. Следующая строка выполняет `srctool.exe` для всех файлов символов, которые есть в папке публичных символов.

```
C:\>walk C:\AWDBIN\symstore.pri\*.pdb srctool -x -d:C:\AWDBIN\sources
```

Извлеченные исходники организованы подобно дереву, показанному в листинге 4.11, в структуру, позволяющую одновременно хранить несколько версий файлов в папке `sources`. Этот инструмент очень мощный: он может извлечь все исходные файлы, использованные для сборки продуктов.

Листинг 4.11

```
C:\AWD>tree c:\AWDBIN\sources /F/A
Folder PATH listing
Volume serial number is B817-38E9
C:\AWDBIN\SOURCES
+--AWD
|  \--chapter3
|  \--spydbg.cpp
|  \--VERSION1
|  spydbg.cpp
```

1. На следующем шаге мы изменяем информацию об исходных файлах, сохраненную в файлах символов. Пакетный файл `cv2http.cmd`, доступный в папке установки source server, может изменить информацию source server на выбранное расположение. Следующая строка изменяет информацию source server на HTTP-сайт книги, <http://www.advancedwindowsdebugging.com>:

```
C:\>walk C:\AWDBIN\symstore.pri\*.pdb cv2http.cmd HTTP_AWD
http://www.advancedwindowsdebugging.com/sources
```

Если желаемое расположение source server - UNC-путь или HTTPS-адрес, этот адрес заменяет URL, использованный в предыдущей командной строке. `HTTP_AWD` - простая переменная, которую в большинстве случаев можно игнорировать. Документация source server объясняет, как использовать эту переменную при необходимости.

1. На последнем шаге папка, содержащая все исходники, добавляется на HTTP-сервер как виртуальный каталог с включенным просмотром. Снимок настроек виртуальной папки показан на рисунке 4.5; он был сделан из оснастки Internet Information Services MMC, работающей в Windows Vista.

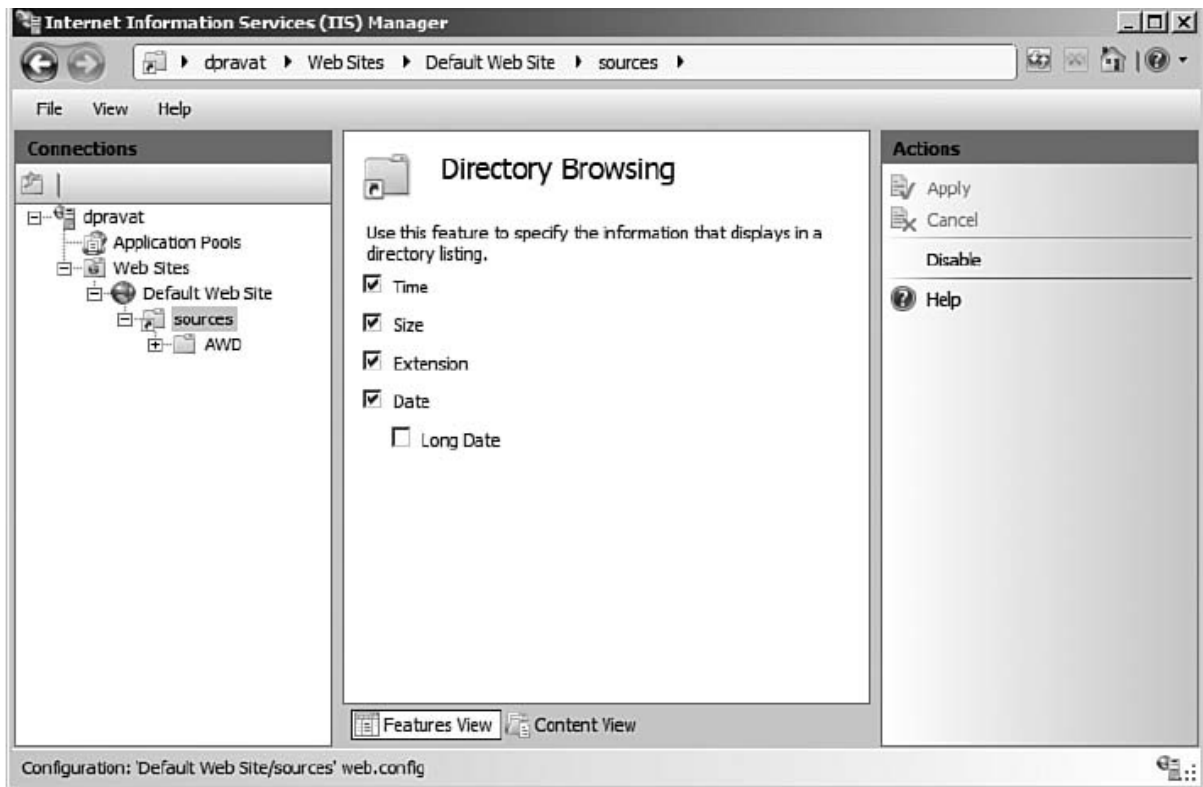


Рисунок 4.5. Виртуальная папка source server с включенным просмотром каталогов (оснастка Vista IIS)

Помните, что файлы символов, подготовленные таким образом, не содержат следов исходной системы контроля версий. Если это требуется, исходные файлы символов нужно сохранить до начала операции, описанной в этом разделе.

Резюме

Debugging Tools for Windows предоставляет дополнительные инструменты, позволяющие всем разработчикам платформы Windows управлять файлами символов и поддерживать информацию source server для своих модулей. Вариант шагов, описанных в этой главе, можно интегрировать в процесс управления важным выпуском. Эта фаза важна для обеспечения поддержки приложения.

Хотя на первый взгляд это кажется сложным, мы хотим заверить вас, что необходимые шаги тривиальны. Например, мы создали целый процесс для всех примеров книги в виде очень простого пакетного файла `release.cmd`, который делает все. Он создает двоичный файл для конкретной архитектуры процессора, использованной для запуска консоли WDK, и разделяет символы на частные и публичные, которые сохраняются в соответствующих хранилищах символов.

Частные файлы символов позднее используются для извлечения исходных файлов из системы управления версиями исходного кода. Информация source server заменяется информацией HTTP-сервера. Затем мы вручную скопировали все файлы с серверов символов и из папки source server на веб-сайт книги. Этот процесс можно легко автоматизировать или интегрировать в процесс выпуска вашего программного обеспечения.

Независимо от того, используете ли вы очень простой процесс или специализированный инструмент, объединяющий все эти шаги, процесс индексирования всех этих файлов должен быть выполнен. Глава 13, "Посмертная отладка", описывает, как интегрировать ваш продукт в систему Windows Error Reporting. Остальные главы полны сведений, которые помогут понять причину сбоя,

сообщенного через механизм WER. Без информации об исходных файлах в файлах символов мы все еще можем получить хорошую версию исходного файла из системы контроля версий. Это не идеально, но приемлемо. Без файла символов вероятность успешного исправления отчета WER падает почти до нуля. Заказчик будет сталкиваться с проблемой снова и снова, пока не выйдет следующая версия продукта. Исправит ли новая версия проблему? На этот вопрос невозможно ответить, но, скорее всего, проблема останется.

Часть II. Прикладная отладка

Глава 5. Повреждение памяти, часть I - стеки

Memory corruption - одна из самых трудноуловимых форм programming error по двум причинам. Во-первых, source of the corruption и manifestation могут быть далеко друг от друга, что затрудняет correlation cause and effect.

Во-вторых, symptoms appear under unusual conditions, поэтому consistently reproduce the error бывает трудно.

В основе memory corruption лежит ситуация, когда выполняется одно или оба следующих условия:

- выполняющийся thread пишет в memory block, которым он не владеет;
- выполняющийся thread пишет в memory block, которым он владеет, но corrupts state этого memory block.

Чтобы exemplify first condition, рассмотрим small application:

```
#include <windows.h>
#define BAD_ADDRESS 0xBAADF00D
int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    char* p = (char*)BAD_ADDRESS;
    *p='A';
    return 0;
}
```

Это небольшое application объявляет pointer на data type char и инициализирует pointer адресом, к которому у него нет access (0xBAADF00D).

Итог выполнения application - crash и появление печально известного UI Dr. Watson.

Хотя совершенно ясно, что это simple application performs an invalid memory access, in more complex systems this can be trickier to figure out.

Например, если application allocated memory blocks и делало assumptions about lifetime этих allocations, premature deletion могло бы вызвать memory corruption из-за stale pointers.

Best-case scenario при записи в memory, которой application не владеет, - это crash.

Но погодите, скажете вы, crash - это best-case scenario?

Да: для memory corruptions crash может immediately indicate, где находится source of memory corruption.

В предыдущем sample code memory, в которую выполняется запись, invalid, и происходит crash. Это хорошая новость.

Мы можем очень легко понять, почему у нас есть pointer, указывающий на invalid memory.

Однако рассмотрим scenario, в котором invalid pointer указывает на memory block, используемый другими частями application.

Symptoms в этом particular case могут быть следующими:

- Application crashes: главное отличие в том, что crash может произойти later time.
В исходном sample application выше code crashed, потому что application писало в memory, designated as invalid by operating system.
В измененном scenario, однако, application пишет в memory, которую operating system считает valid, и write разрешается without errors.
Позже application может попытаться использовать memory, в которую ошибочно была выполнена запись, и в зависимости от nature memory access может произойти crash.

- Non-crashing and unpredictable behavior: примерно так же, как previous item позволял application записать bad data в memory, принадлежащую другим частям application, net result не обязан быть crash.

Другие части application вполне могут продолжать использовать memory, куда были записаны bad data, хотя state этой memory был altered, причем обычно не в лучшую сторону.

Возьмем example.

Предположим, что у нас есть class, представляющий thread pool.

Помимо способности queue requests to thread pool, существует method, который sets flag, indicating that shutdown is in progress.

Thread pool periodically checks this flag, и если он когда-либо equals true, начинается shutdown.

Singleton instance thread pool создается и используется application.

Теперь предположим, что thread pool обслуживает 200 requests (credit card authorizations), когда thread в application ошибочно overwrites shutdown flag значением true.

Внезапно thread pool shuts down, customers начинают получать errors по credit card transactions, и начинаются phone calls.

Это classic example memory corruption, где net effect thread corrupting memory приводит к unpredictable behavior.

Поскольку thread, overwrote memory, уже нанес damage, subsequent use этой memory может (и скорее всего будет) unpredictable.

Найти source таких memory corruptions чрезвычайно трудно.

Должно быть совершенно ясно, что при столкновении с memory corruption мы хотим получить notification сразу, как только offending thread пишет в memory, которой он не владеет, а не backtrack от странного application behavior, которое может surface через несколько days после invalid memory write.

Если не повезло так, что pointer указывает на truly invalid memory и сразу вызывает access violation, большинство memory corruptions проявляются в форме странного application behavior или crashes уже после того, как memory была altered.

К счастью, with the right strategy and powerful tool set мы можем maximize efficiency при анализе potential memory corruption и force strategy "crash immediately", чтобы облегчить поиск source of memory corruption.

Процесс обнаружения memory corruption

Этот section outlines process обнаружения memory corruption.

Он включает graphical representation процесса, а также brief discussion каждого step.

Важно понимать, что finding root cause memory corruption может включать several iterations process, показанного на рисунке 5.1, depending on nature of memory corruption.

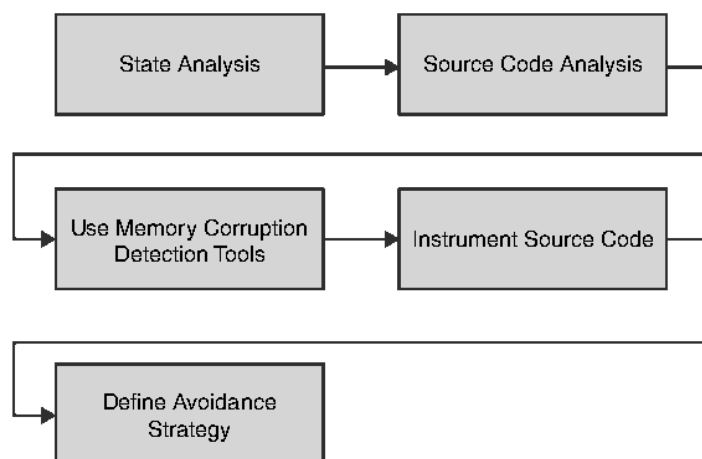


Figure 5.1 Memory corruption analysis process

Рисунок 5.1. Процесс анализа memory corruption

Шаг 1: State analysis

Самый первый step при investigation memory corruption - убедиться, что failure, который вы рассматриваете, действительно вызван memory corruption.

Этот шаг можно разбить подробнее, как показано на рисунке 5.2.

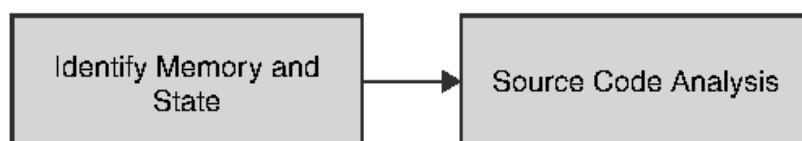


Figure 5.2 State analysis process

Рисунок 5.2. Процесс анализа состояния

Как упоминалось ранее, symptoms memory corruption делятся на две categories: crashes и noncrashing unpredictable behavior.

Этот first step требует initial analysis observed behavior через analysis state corrupted memory.

Как понять, какой state нужно analyze?

При crashes найти starting point довольно просто.

Code crashed из-за некоторого unexpected state, и code хорошо known at crash time.

Сопоставив state of memory at crash time с focused code review, можно сделать sound judgment calls об origins этого state.

"Valid", хотя и buggy, code paths могут привести к такому state.

Если это так, вы имеете дело не с memory corruption per se, а с unexpected code path, который erroneously wrote to memory.

Однако если ни один code path не позволяет memory попасть в такой state, единственное plausible explanation состоит в том, что someone overwrote that memory, и, следовательно, произошла memory corruption.

Если у вас нет crash, но вместо этого вы видите periodic strange behaviors in application, выяснить, state какой memory potentially corrupted, не так clearly, как при crashes.

Обычно при unexpected behavior вы break into debugger и начинаете с некоторого initial analysis.

Например, если clients получают error after error при попытке authorize credit cards, можно начать с investigation thread pool state (который services all credit card authorizations) и посмотреть, почему они fail.

Если вы notice, что thread pool не принимает requests, because it is shut down, нужно перейти к step 2 и source code analysis, чтобы identify "valid" code path или, если такого нет, conclude, что произошла memory corruption.

Шаг 2: Source code analysis

После того как вы identified (на step 1), что столкнулись с possible memory corruption bug, следующий step - выполнить source code analysis и посмотреть, можно ли identify root cause.

Memory corruption может происходить, когда thread пишет в memory location, которым он не владеет.

Из этого statement можно сделать очень важное observation.

Thread writes data to memory block.

Предположительно, data being written важны для этого particular thread, и если мы сможем analyze data and make sense out of it, мы сможем further narrow down scope possible suspects.

Рассмотрим example.

Code in Listing 5.1 shows very simple console-based application, которое предлагает user two choices: show application information, such as full name and version, и simulate memory corruption.

Постарайтесь не смотреть full source code, а только code, presented in Listing 5.1.

Листинг 5.1. Simple console application with simulated memory corruption

```
int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    wint_t iChar = 0 ;
    g_AppInfo = new CAppInfo(L"Simple console application", L"1.0" );
    if(!g_AppInfo)
    {
        return 1;
    }
    wprintf(L"Press: \n");
    wprintf(L" 1 To display application information\n");
    wprintf(L" 2 To simulate memory corruption\n");
    wprintf(L" 3 To exit\n");
    wprintf(L"\n\n> ");
    while((iChar=_getwche())!='3')
    {
        if(iChar == '1')
        {
            g_AppInfo->PrintAppInfo();
        }
        else if(iChar=='2')
        {
            SimulateMemoryCorruption();
            wprintf(L"\nMemory Corruption completed\n");
        }
        else
        {
            wprintf(L"\nInvalid option\n");
        }
        wprintf(L"\n\n> ");
    }
    delete g_AppInfo;
    return 0;
}
```

Source code и binary для листинга 5.1 можно найти в следующих folders:

Source code: C:\AWD\Chapter5\MemCorrupt
Binary: C:\AWDBIN\WinXP.x86.chk\05MemCorrupt.exe

Запустите application с помощью следующей command line:

C:\AWDBIN\WinXP.x86.chk\05MemCorrupt.exe

Application состоит из class, который encapsulates application-specific information: full application name and version.

Main function позволяет user вывести application information, simulate memory corruption или выйти из application.

Press:
1 For application information
2 For simulated memory corruption
3 To exit

Если нажать 1, вы увидите следующее:

```
> 1
Full application Name: Simple console application
Version: 1.0
```

Если нажать 2, вы увидите:

```
> 2
Memory Corruption completed
```

Если затем снова нажать 1, вы, что неудивительно, увидите, что application crashes.

Теперь начинается интересная part.

Как выяснить, какая часть application caused memory corruption, не выполняя step through code для option 2?

Начнем с самого простого.

Запустите application under debugger и выберите ту же sequence of choices, что и раньше.

Когда вы выбираете option 1 во второй раз, debugger должен break into debugger with access violation.

```
...
...
...
0:000> g
ModLoad: 5cb70000 5cb96000 C:\WINDOWS\system32\ShimEng.dll
Press:
1 To display application information
2 To simulate memory corruption
3 To exit
> 1
Full application Name: Simple console application
Version: 1.0
> 2
Memory Corruption completed
> 1(bdc.8d8): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=72726f43 ebx=7ffd0073 ecx=00000007 edx=7fffffff esi=00000020 edi=00000002
eip=77c43869 esp=0007fa68 ebp=0007fed8 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010202
msvcrt!_woutput+0x695:
77c43869 66833800 cmp word ptr [eax],0 ds:0023:72726f43=???
0:000> kb
ChildEBP RetAddr Args to Child
0007fed8 77c42290 77c5fca0 01001208 0007ff28 msvcrt!_woutput+0x695
0007ff1c 01001448 01001208 72726f43 00032cb0 msvcrt!wprintf+0x35
0007ff30 010013b2 00032cb0 00032cb0 7ffd0031 memcorrupt!CAAppInfo::PrintAppInfo+0x18
0007ff44 010015fa 00000001 00032bf0 00036880 05memcorrupt!wmain+0xb2
```



```
0007ffc0 7c816fd7 00011970 7c9118f1 7ffdf000 05memcorrupt!wmainCRTStartup+0x12f
0007fff0 00000000 010014cb 00000000 78746341 kernel32!BaseProcessStart+0x23
```

По stack видно, что наша main function вызывает function PrintAppInfo class CAppInfo, которая, в свою очередь, вызывает wprintf.

Если сопоставить то, что мы видим в debugger, с source code, это выглядит совершенно логично.

Следующий question - почему function wprintf failed.

Если посмотреть, что мы передаем в function из source code, мы увидим следующее:

```
VOID PrintAppInfo()
{
    wprintf(L"\nFull application Name: %s\n", m_wszAppName);
    wprintf(L"Version: %s\n", m_wszVersion);
}
```

Логично предположить, что передаваемые pointers (m_wszAppName и/или m_wszVersion) must be invalid.

Function wprintf предполагает, что переданный pointer, в нашем случае strings, представляет wide character string, terminated by NULL.

Если это assumption fails, function может crash.

Теперь обратим внимание на analysis state рассматриваемого object.

More specifically, посмотрим на state CAppInfo:

```
0:000> X 05memcorrupt!g_*
01002008 05memcorrupt!g_AppInfo = 0x00032cb0
0:000> dt CAppInfo 0x00032cb0
+0x000 m_wszAppName : 0x72726f43 -> ??
+0x004 m_wszVersion : 0x01747075 -> ??
```

Интересующие нас pointer values - это wszAppName и wszVersion.

Попробуем dump each pointer, чтобы увидеть, на что они point to:

```
0:000> dd 0x72726f43
72726f43 ???????? ???????? ???????? ????????
72726f53 ???????? ???????? ???????? ????????
72726f63 ???????? ???????? ???????? ????????
72726f73 ???????? ???????? ???????? ????????
72726f83 ???????? ???????? ???????? ????????
72726f93 ???????? ???????? ???????? ????????
72726fa3 ???????? ???????? ???????? ????????
72726fb3 ???????? ???????? ???????? ????????
0:000> dd 0x01747075
01747075 ???????? ???????? ???????? ????????
01747085 ???????? ???????? ???????? ????????
01747095 ???????? ???????? ???????? ????????
017470a5 ???????? ???????? ???????? ????????
017470b5 ???????? ???????? ???????? ????????
017470c5 ???????? ???????? ???????? ????????
017470d5 ???????? ???????? ???????? ????????
017470e5 ???????? ???????? ???????? ????????

```

Question marks указывают, что memory is not accessible.

Довольно интересно, не правда ли?

В первый раз, когда мы попросили application print out information, все worked fine.

Теперь pointers, похоже, point to inaccessible memory.

Каким-то образом contents instance CAppInfo became corrupted.

Object layout простого C++ class instance состоит из его data members, что в нашем случае включает два pointers.

Если object layout был overwritten, мы могли попасть в ситуацию, где у нас corrupt pointers.

Исходя из этого, стоит посмотреть, на что actual instance pointer points to:

```
0:000> x 05memcorrupt!g_*
01002008 05memcorrupt!g_AppInfo = 0x00032cb0
0:000> dd 0x00032cb0
```

```
00032cb0 72726f43 01747075 abababab abababab
00032cc0 00000000 00000000 00040012 001c07f2
00032cd0 00500041 00440050 00540041 003d0041
00032ce0 003a0043 0044005c 0063006f 006d0075
00032cf0 006e0065 00730074 00610020 0064006e
00032d00 00530020 00740065 00690074 0067006e
00032d10 005c0073 0061006d 00690072 0068006f
00032d20 0041005c 00700070 0069006c 00610063
```

Memory dump показывает pointer values, которые мы рассматривали before.

Вместо использования command dd можно попробовать dump out instance pointer as text:

```
0:000> da 0x00032cb0
00032cb0 "Corrupt....."
```

Это выглядит намного интереснее.

Похоже, instance pointer CAppInfo был overwritten string: "Corrupt".

Теперь можно применить code review и посмотреть, есть ли в application code, который manipulates strings with content "Corrupt".

Как вы уже suspected, при выборе option 2 (simulate memory corruption) application forcefully overwrites contents instance pointer CAppInfo string ("Corrupt").

Как понять, в какой form пытаться dump data и make sense out of it?

Четкого rule нет, есть только guidelines.

Следующие strategies хорошо работают и их стоит пробовать при analysis memory contents.

1. Используйте command dc, чтобы dump out memory contents pointer.
Command dc выводит content как double-word values, а также ASCII equivalent.
Если в output видны strings, используйте commands da или du, чтобы dump out string.
2. Используйте extension command !address, чтобы glean information about memory.
Extension command !address сообщает type of memory, например private, protection level, например read and write, state, например committed или reserved, и usage, например stack или heap memory.
3. Используйте command dds, чтобы dump out memory как double words и symbols.
Это может помочь соотнести memory с конкретным type.
4. Используйте command dpp, чтобы dereference указанный pointer и вывести double-word contents memory.
Если какой-либо double word matches symbol, symbol также отображается.
Это полезная technique, если memory, на которую указывает pointer, содержит virtual function table.
5. Используйте commands dpa и dpu, чтобы отобразить memory, на которую указывает pointer, в ASCII и Unicode formats.
6. Если memory content является small number, кратным 4, это может быть handle; можно использовать extension command !handle, чтобы вывести information about the handle.
7. Если previous steps ничего не дали, можно попробовать search entire address space for references to address memory block.

Эта technique распознавания data в corrupted memory block очень useful, когда вы пытаетесь выяснить culprit code, который corrupted memory block.

Но, again, не всегда возможно find offender с помощью этой technique.

Следующий step in process - использовать memory corruption detection tools, которые могут заметно облегчить life.

Шаг 3: используйте memory corruption detection tools

Прежде чем перейти к описанию этих tools, важно понимать, что tools не дают guarantees в отношении catching memory corruptions.

Tools merely help you catch ряд очень common memory corruption scenarios.

В зависимости от category memory corruptions, с которой вы столкнулись, доступны разные tools. Для stack-based corruptions лучший available tool - сам compiler, поскольку он может inject stack verification code in your application.

Когда речь идет о heap-based memory corruptions, лучший tool - Application Verifier; см. главу 1, "Introduction to the Tools."

Application Verifier имеет множество test settings, связанных с memory corruption, из которых можно выбирать.

Общее у обоих tools состоит в том, что они пытаются trap common memory-related programming mistakes immediately, в момент возникновения memory corruption, rather than later, когда могут появиться более troublesome side effects.

В этой главе мы рассмотрим, как compiler can aid us in stack corruptions, а Application Verifier используем при анализе heap-based corruptions в главе 6, "Memory Corruption Part II - Heaps."

Шаг 4: инструментируйте source code

Если previous steps не помогли найти culprit, вас ждет тяжелая работа.

Следующий step - собрать всю information, gathered from previous steps, и theorize about possibilities.

Когда у вас появятся несколько theories, можно instrument your code, чтобы prove them right or wrong. Instrumentation techniques варьируются от простых trace statements до tracing, supported by operating system.

Шаг 5: определите avoidance strategies

Последний и, пожалуй, самый важный шаг - взять то, что вы узнали, и define future avoidance strategy.

Avoidance strategies могут принимать форму использования tools throughout development, чтобы help catch common memory corruption problems, а также проверки, что code, который вы пишете, делает explicit steps для минимизации риска potential memory corruptions.

Остальная часть главы проходит через несколько common memory corruption scenarios и показывает, как memory corruption process можно применить, чтобы выяснить reason behind memory corruption.

Scenarios в этой главе focus on stack-based corruptions, а глава 6 focuses on heap-based corruptions.

Stack corruptions

Stack - одна из самых common и well-known data structures.

Большинство introductory classes по algorithms начинаются с изучения stack data structure.

Это действительно довольно simple and straightforward data structure, которую можно сравнить со стопкой papers.

Каждый лист, который вы кладете (push) на stack, попадает на top of the stack.

Каждый лист, который вы снимаете (pop) со stack, снимается from the top.

Таким образом, обе basic operations, performed on stack (push и pop), всегда работают с top.

Поскольку каждый лист, placed onto stack или removed from stack, проходит через top, говорят, что algorithm имеет last in first out (LIFO) semantics.

Stack применительно к executing code in Windows - это просто block of memory, assigned by operating system to running thread.

Purpose of the stack, помимо прочего, состоит в tracking function call chain: allocation local variables, parameter passing и так далее.

Каждый раз, когда выполняется function call, создается another frame и pushed on the stack.

По мере того как thread делает все больше function calls, stack grows bigger and bigger.

Рисунок 5.3 иллюстрирует анатомию стека во время вызова функции.

Далее в примерах мы увидим, как именно каждый элемент появляется в стеке, а пока рисунок 5.3 иллюстрирует общую компоновку стека при вызове функции на архитектуре x86.

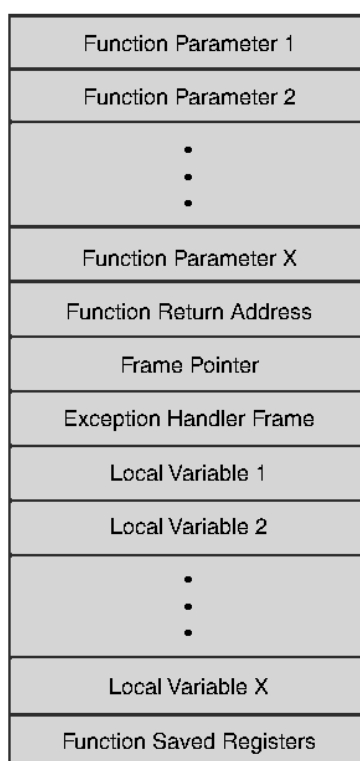


Figure 5.3 Anatomy of a call stack during a function call

Рисунок 5.3. Анатомия стека вызовов во время вызова функции

Примечание. Если вы building source code для этой главы, убедитесь, что buffer overrun checks отключены: задайте environment variable BUFFER_OVERFLOW_CHECKS в build window равной 0.

Чтобы лучше понять, how stacks work и как они can become corrupted, рассмотрим example. Application in Listing 5.2 shows starting point of a new thread, который делает several nested function calls, а также declares local variables in each function.

Листинг 5.2. Sample application, показывающее creation of a new thread

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
DWORD WINAPI ThreadProcedure(LPVOID lpParameter);
VOID ProcA();
```

```

VOID Sum(int* numArray, int iCount, int* sum);
int __cdecl wmain ()
{
    HANDLE hThread = NULL ;
    printf("Starting new thread...");
    hThread = CreateThread(NULL, 0, ThreadProcedure, NULL, 0, NULL);
    if(hThread!=NULL)
    {
        printf("success\n");
        WaitForSingleObject(hThread, INFINITE);
        CloseHandle(hThread);
    }
    return 0;
}
DWORD WINAPI ThreadProcedure(LPVOID lpParameter)
{
    ProcA();
    printf("Press any key to exit thread\n");
    _getch();
    return 0;
}
VOID ProcA()
{
    int iCount = 3;
    int iNums[] = {1,2,3};
    int iSum = 0 ;
    Sum(iNums, iCount, &iSum);
    printf("Sum is: %d\n", iSum);
}
VOID Sum(int* numArray, int iCount, int* sum)
{
    for(int i=0; i<iCount;i++)
    {
        *sum+=numArray[i];
    }
}

```

Source code и binary для листинга 5.2 можно найти в следующих folders:

```

Source code: C:\AWD\Chapter5\StackDesc
Binary: C:\AWDBIN\WinXP.x86.chk\05StackDesc.exe

```

High-level overview code in Listing 5.2 показывает main function, создающую new thread с помощью API CreateThread и задающую starting function этого thread как function named ThreadProcedure. Function ThreadProcedure также является starting point нашего stack investigation.

Согласно предыдущему discussion about stacks, каждый раз, когда thread делает function call, на stack pushed new frame, причем frame состоит из data, required to execute that function.

Является ли frame function ThreadProcedure first item on newly created thread stack?

Не совсем.

До того как наш thread вообще получит возможность execute function ThreadProcedure, operating system выполняет series of function calls as part of thread creation.

Чтобы получить представление о том, что executed, build sample application in Listing 5.2 и run it in debugger, setting breakpoint at start of function ThreadProcedure, как показано in Listing 5.3.

После ввода Go debugger останавливается на этой function, и можно посмотреть stack executing thread.

Листинг 5.3. Breaking at ThreadProcedure and dumping the stack

```

...
...
...
0:000> X 05stackdesc!*ThreadProcedure*
01001210 05stackdesc!ThreadProcedure (void *)
0:000> bp 05stackdesc!ThreadProcedure
0:000> g
ModLoad: 5cb70000 5cb96000 C:\WINDOWS\system32\ShimEng.dll
Starting new thread...success

```

```

Breakpoint 0 hit
eax=00000000 ebx=00000000 ecx=002bffb0 edx=7c90eb94 esi=00000000 edi=00030000
eip=01001210 esp=002bffb8 ebp=002bfffec iopl=0 nv up ei pl zr na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
05stackdesc!ThreadProcedure:
01001210 55 push ebp
0:001> kb
ChildEBP RetAddr Args to Child
002bffb4 7c80b683 00000000 00030000 00000000 05stackdesc!ThreadProcedure
002bfffec 00000000 01001210 00000000 00000000 kernel32!BaseThreadStart+0x37

```

Как видно, наша thread procedure фактически не является first function to execute; перед ней выполняется function, defined in kernel32.dll, named BaseThreadStart, followed by call to our thread function.

Function BaseThreadStart - это просто interceptor, defined by operating system, который invoked prior to all newly created thread executions.

Теперь, когда мы reached starting point нашего thread, рассмотрим сам stack подробнее и посмотрим, как он organized.

Как обсуждалось ранее, stack operations, такие как push и pop, работают from the top of the stack, поэтому нужно хранить pointer, который tells us, где находится top of the stack.

На x86 architectures для этой purpose используется register named esp.

Перед тем как dig in и examine actual contents of the stack, посмотрим на first few instructions нашей function.

Листинг 5.4 показывает assembly code, starting at function ThreadProcedure.

Листинг 5.4. Assembly code, starting at ThreadProcedure

```

0:000> u 05stackdesc!ThreadProcedure
05stackdesc!ThreadProcedure:
01001220 8bff mov edi,edi
01001222 55 push ebp
01001223 8bec mov ebp,esp
01001225 e826000000 call 05stackdesc!ProcA (01001250)
0100122a 68b0100001 push offset 05stackdesc!`string' (010010b0)
0100122f ff1550100001 call dword ptr [05stackdesc!_imp__printf (01001050)]
01001235 83c404 add esp,4
01001238 ff1548100001 call dword ptr [05stackdesc!_imp___getch (01001048)]

```

Перед call to ProcA, четвертой instruction from top of assembly code, выполняется несколько interesting assembly instructions.

Specifically, следующие instructions важны, когда речь идет об anatomy of a call stack:

```

01001220 8bff mov edi,edi
01001222 55 push ebp
01001223 8bec mov ebp,esp

```

Вторая instruction pushes register ebp onto the stack.

Позже мы увидим, how register ebp используется, но пока достаточно считать, что register ebp always contains base pointer to any given frame.

Поскольку base pointer нужно retain для каждого frame, он pushed onto the stack перед созданием любого new frame, то есть перед call instruction.

Следующая instruction moves stack pointer to register ebp, чтобы establish beginning of the new stack frame.

Эти три instructions form prologue of a function.

В общем случае большинство functions, которые вы встретите, следуют general outline:

- Function prologue
- Function code
- Function epilogue

Function prologue ensures, что stack properly prepared для execution нового function code.

За prologue следует actual function code, а finally function epilogue ensures, что stack restored to correct state before returning to caller.

Теперь мы находимся в point, где готовы call procedure ProcA через instruction call.

Когда выполняется instruction call, stack также updated.

Точнее, во время выполнения инструкции call адрес возврата вызова, то есть адрес следующей инструкции после вызова, помещается в стек.

Это необходимо, потому что при возврате из только что вызванной функции выполняется инструкция ret.

Инструкция ret должна вернуть управление к следующей инструкции сразу после инструкции call.

Чтобы знать, где находится эта точка, инструкция ret извлекает адрес из стека и переходит по этому адресу.

Рисунок 5.4 показывает текущее состояние стека нашего потока перед инструкцией call.

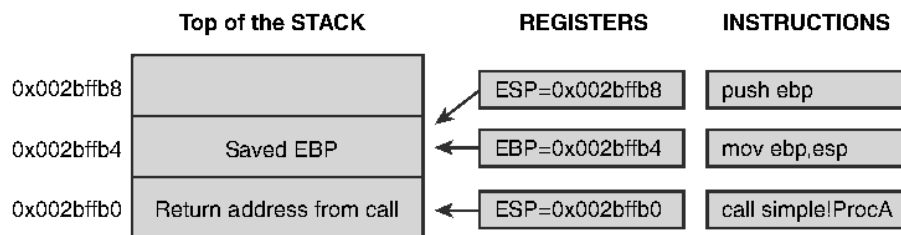


Figure 5.4 Contents of stack prior to calling ProcA

Рисунок 5.4. Содержимое стека перед вызовом ProcA

Важно отметить, что на архитектурах x86 стек растёт сверху вниз.

На рисунке 5.4 видно, как адреса стека уменьшаются в результате помещения данных в стек.

Инструкция push на x86 - это двухшаговая операция:

1. Уменьшает указатель стека (esp) на размер операнда.
2. Переносит источник (ebp на рисунке 5.4) в стек.

На рисунке 5.4 esp изначально указывал на позицию стека 0x002bffb8.

Когда выполняется инструкция push, esp сначала уменьшается на 4 байта (0x002bffb4), после чего значение ebp переносится в эту позицию стека.

Инструкция mov гарантирует, что ebp и esp указывают на одну и ту же позицию в стеке, которая также является базовой точкой нового кадра вызова.

В этот момент стек подготовлен и настроен для самой инструкции call, которая передает поток выполнения следующей вызываемой функции (ProcA).

Находясь на инструкции call, продолжаем выполнение, вводя t, чтобы войти трассировкой в следующую функцию.

Оказавшись в этой функции, дизассемблируем код всей функции, как показано в листинге 5.5.

Листинг 5.5. Дизассемблирование всей функции ProcA

```
0:000> uf 05stackdesc!ProcA
05stackdesc!ProcA:
01001250 8bff mov edi,edi
01001252 55 push ebp
01001253 8bec mov ebp,esp
01001255 83ec14 sub esp,14h
01001258 c745ec03000000 mov dword ptr [ebp-14h],3
0100125f c745f401000000 mov dword ptr [ebp-0Ch],1
01001266 c745f802000000 mov dword ptr [ebp-8],2
0100126d c745fc03000000 mov dword ptr [ebp-4],3
01001274 c745f000000000 mov dword ptr [ebp-10h],0
```

```

0100127b 8d45f0 lea eax,[ebp-10h]
0100127e 50 push eax
0100127f 8b4dec mov ecx,dword ptr [ebp-14h]
01001282 51 push ecx
01001283 8d55f4 lea edx,[ebp-0Ch]
01001286 52 push edx
01001287 e824000000 call 05stackdesc!Sum (010012b0)
0100128c 8b45f0 mov eax,dword ptr [ebp-10h]
0100128f 50 push eax
01001290 68d0100001 push offset 05stackdesc!`string' (010010d0)
01001295 ff1550100001 call dword ptr [05stackdesc!_imp__printf (01001050)]
0100129b 83c408 add esp,8
0100129e 8be5 mov esp,ebp
010012a0 5d pop ebp
010012a1 c3 ret

```

Command `uf` используется, чтобы `unassemble` entire function in one step, вместо `command u`, которая by default `unassembles` только первые eight instructions.

Первые four instructions в этой function являются частью function prologue:

```

01001250 8bff mov edi,edi
01001252 55 push ebp
01001253 8bec mov ebp,esp
01001255 83ec30 sub esp,0x14

```

Первые three instructions identical to previous frame и simply make sure, что base frame pointer и stack pointer set up properly for the frame.

Последняя instruction (`sub esp,0x14`) выглядит очень interesting.

Похоже, она subtracts 0x14 bytes, или decimal 20, from stack pointer.

Зачем выполняется это subtraction?

Она makes room for local variables.

Как видно из source code for `ProcA` в листинге 5.2, function allocates following local variables on stack:

```

int iCount = 3;
int iNums[] = {1,2,3};
int iSum = 0 ;

```

Total size этих variables:

$4 \text{ (iCount)} + 12 \text{ (iNums)} + 4 \text{ (iSum)} = 20 \text{ bytes}$

Когда мы subtract 20 bytes from stack pointer, apparent gap in stack becomes reserved for local variables, declared in function.

Рисунок 5.5 показывает содержимое стека после выполнения инструкции `sub`.

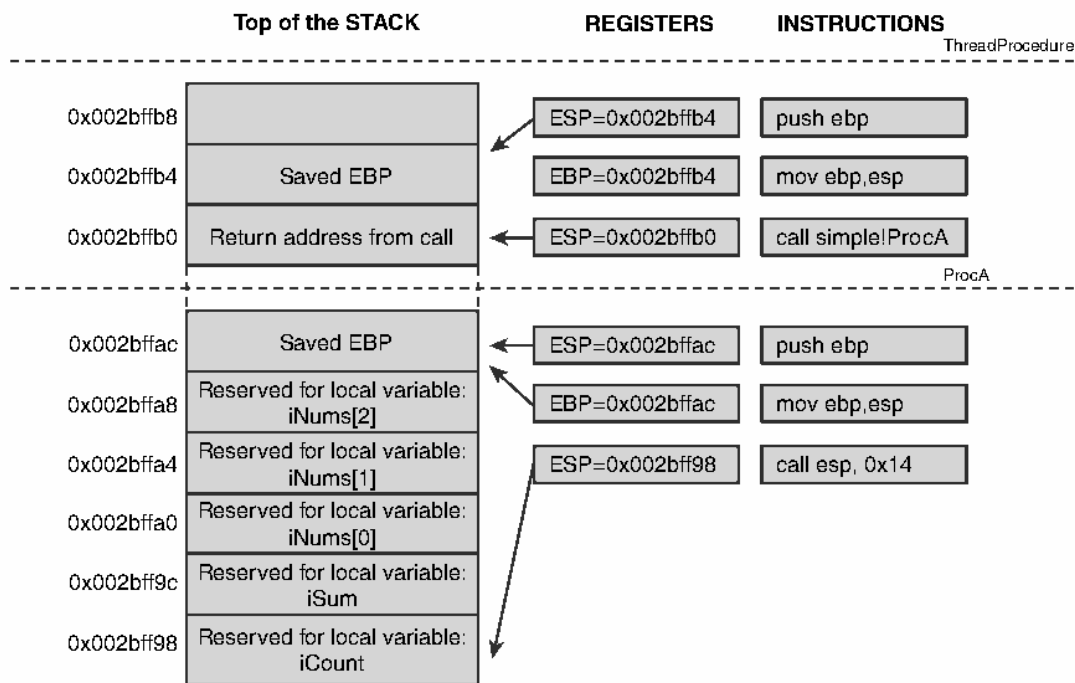


Figure 5.5 Contents of stack after reserving space for local variables in ProcA

Рисунок 5.5. Содержимое стека после резервирования места под локальные переменные в ProcA

После того как stack pointer esp adjusted, чтобы make room for local variables, next set of executed instructions initializes stack-based local variables значениями, specified in source code:

```

05stackdesc!ProcA+0x8:
01001258 c745ec03000000 mov dword ptr [ebp-14h],3
0100125f c745f401000000 mov dword ptr [ebp-0Ch],1
01001266 c745f802000000 mov dword ptr [ebp-8],2
0100126d c745fc03000000 mov dword ptr [ebp-4],3
01001274 c745f000000000 mov dword ptr [ebp-10h],0

```

Важное observation для этих instructions mov: register ebp используется with offset для reference stack location, где resides local variable.

Почему используется register ebp, а не esp?

Помните, мы говорили, что register ebp always points to beginning of call frame?

Reason в том, чтобы always have reference point, откуда можно access anything related to that frame.

By convention, register ebp используется для этой purpose.

По этой же причине всегда особо заботятся о том, чтобы store register ebp on stack before creation of new frame, чтобы его можно было safely restored, когда frame goes away, то есть function returns.

In contrast, register esp changes continually throughout execution of a function, и поэтому его было бы difficult (или at least costly) использовать как base frame pointer.

Frame Pointer Omission. Frame pointer omission - optimization technique, при которой base frame pointer register может использоваться как general-purpose register, а не как reserved base frame pointer, показанный в этой главе.

Возможность использовать base frame pointer register таким образом speeds up execution и enables compiler использовать base frame pointer register как еще один general-purpose register.

После initialization local variables идет series of instructions, которая prepares application to make another function call, как показано in Listing 5.6.

Листинг 5.6. Preparing to call the Sum function

```
0100127b 8d45f0 lea eax,[ebp-10h]
0100127e 50 push eax
0100127f 8b4dec mov ecx,dword ptr [ebp-14h]
01001282 51 push ecx
01001283 8d55f4 lea edx,[ebp-0Ch]
01001286 52 push edx
01001287 e824000000 call 05stackdesc!Sum (010012b0)
```

На первый взгляд кажется, что перед инструкцией call в стек помещается много данных. Если посмотреть на прототип функции Sum, мы увидим следующее:

```
VOID Sum(int* numArray, int iCount, int* sum);
```

В функцию передаются три параметра:

- Указатель на целочисленный массив, содержащий числа, которые нужно сложить.
- Целое число, представляющее количество элементов в массиве.
- Указатель на целое число, которое при успешном выполнении будет содержать сумму всех чисел в массиве.

Способ, которым параметры передаются из функции ThreadProc в функцию Sum, - как вы уже догадались, стек.

Каждый раз, когда инструкция call приводит к вызову функции с параметрами, вызывающая функция отвечает за помещение параметров в стек справа налево согласно стандартному соглашению о вызовах.

В нашем случае первый параметр, который должен попасть в стек, - указатель, который будет содержать сумму (sum).

Первые две инструкции в листинге 5.6 показывают, как параметр помещается в стек.

И снова видно, что регистр ebp используется для ссылки на интересующую локальную переменную.

Поскольку мы передаем указатель, используется инструкция lea (load effective address).

Остальные параметры помещаются в стек аналогичным образом; помните, справа налево.

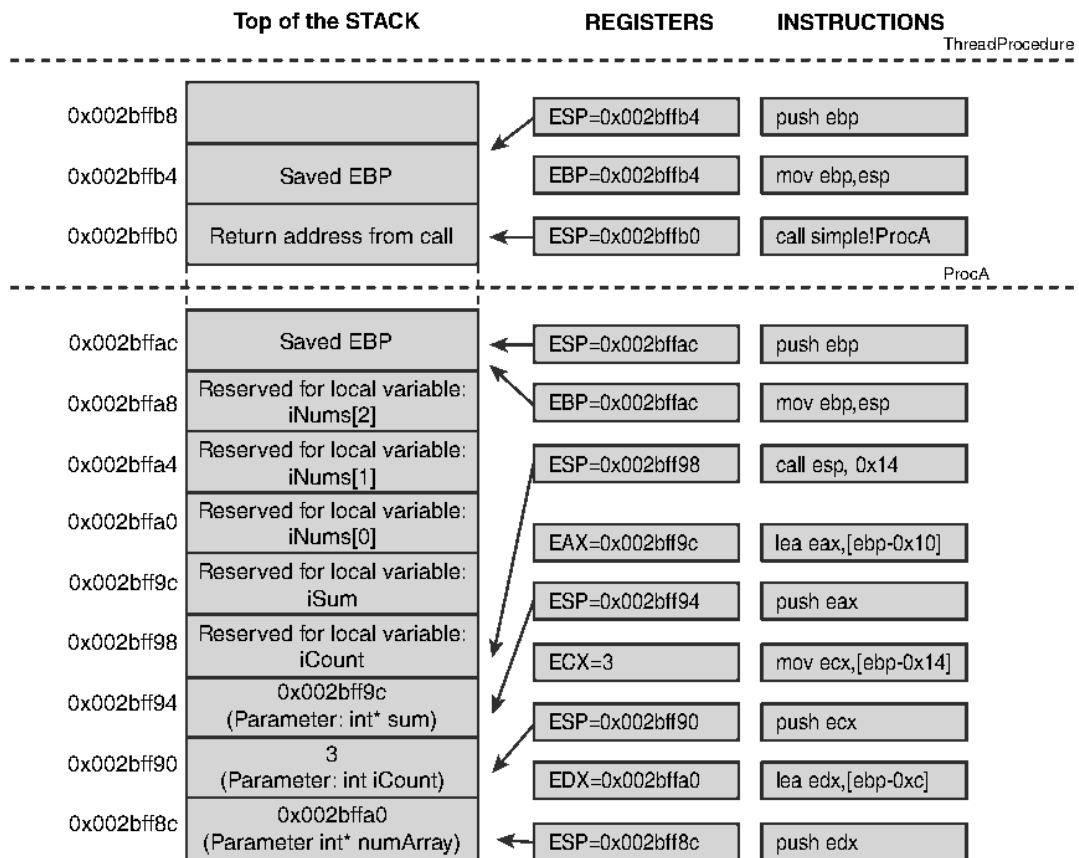


Figure 5.6 Call stack as we are ready to call the Sum function

Рисунок 5.6. Стек вызовов в момент готовности вызвать функцию Sum

Оставляю читателю упражнение: разобраться, как выглядит стек в новом кадре при вызове функции Sum.

Подсказка: поскольку параметры передаются через стек, для доступа к переданным параметрам используется смещение вместе с регистром ebp.

После того как вызов вернулся в вызывающий кадр (ProcA), указатель стека esp установлен в 0x002bff98; это также последний слот стека, использованный перед помещением параметров для вызова Sum.

Как указатель стека вернулся в эту позицию?

Ответ кроется в том, как кадр возвращается из функции; это будет видно при анализе возврата из функции ProcA.

Листинг 5.7 показывает инструкции ассемблера сразу после нашего вызова Sum.

Листинг 5.7. Инструкции после вызова Sum

```
0100128c 8b45f0 mov eax,dword ptr [ebp-10h]
0100128f 50 push eax
01001290 68d0100001 push offset 05stackdesc!`string' (010010d0)
01001295 ff1550100001 call dword ptr [05stackdesc!_imp__printf
(01001050)]
0100129b 83c408 add esp,8
0100129e 8be5 mov esp,ebp
010012a0 5d pop ebp
010012a1 c3 ret
```

Следующая инструкция call в строке 4 показывает еще один вызов, на этот раз функции printf.

Это хорошо соответствует исходному коду, поскольку он пытается вывести результат вызова `Sum`, сохраненный в `iSum`.

И снова перед вызовом функции `printf` стек настраивается для любых параметров, которые могут потребоваться во время вызова.

Точнее, передаются два параметра:

- String: "The sum is: %d\n".
- Value `iSum`.

Помните, что параметры всегда передаются справа налево, поэтому значение `iSum` первым помещается в стек.

Первые две инструкции листинга 5.7 показывают, как значение `iSum` помещается в стек.

Поскольку `iSum` - локальная переменная в кадре `ProcA`, доступ к ней выполняется через регистр `ebp` минус смещение `0x10`.

На рисунке 5.4 видно, что `ebp-0x10` индексирует локальную переменную `iSum`.

Последний параметр, который должен быть помещен в стек, - сама строка, и это видно по инструкции `push offset 05stackdesc!`string' (010010d0)`.

Чтобы проверить, что в стек действительно помещена правильная строка, можно использовать команду `da (dump ASCII)`:

```
0:001> da 0x10010d0
010010d0 "Sum is: %d."
```

Это действительно подтверждает, что передается правильная строка.

После выполнения инструкции `call` последние несколько инструкций в функции `ProcA` гарантируют, что стек восстановлен в исходное состояние, существовавшее до вызова `ProcA`, как показано в листинге 5.8.

Листинг 5.8. Восстановление стека перед возвратом из `ProcA`

```
0100129b 83c408 add esp,8
0100129e 8be5 mov esp,ebp
010012a0 5d pop ebp
010012a1 c3 ret
```

Первая инструкция прибавляет 8 к указателю стека `esp`.

В чем причина этого прибавления?

Когда функция `printf` возвращается, `esp` указывает на последний параметр, помещенный в стек при подготовке к вызову.

Помните: каждый раз, когда кадр выполняет вызов, нужно гарантировать, что стек восстановлен в состояние до вызова.

Поскольку для вызова `printf` мы поместили в стек два параметра, нужно прибавить 8 байтов к указателю стека `esp`, чтобы вернуться к состоянию до вызова: 2×4 байта равны размеру двух параметров, помещенных в стек.

После того как состояние восстановлено, мы почти готовы вернуться из функции `ProcA`.

Поскольку в функции `ProcA` мы выделили локальные переменные, регистр `esp` указывает на последнюю локальную переменную, объявленную в стеке.

При возврате из функции нужно убедиться, что регистр `esp` сброшен к значению, которое было до вызова функции `ProcA`.

Ключ к этому - вспомнить, что произошло в прологе функции `ProcA`.

Точнее, инструкция `mov ebp, esp` в прологе сохранила значение регистра `esp` в `ebp`.

Чтобы восстановить `esp`, мы просто выполняем инструкцию `mov esp, ebp`, как показано в листинге 5.8.

Рисунок 5.7 показывает текущее состояние нашего стека.

Поскольку регистр `ebp` используется как базовый указатель кадра, восстановление этого регистра так же важно, как восстановление регистра `esp`.

После возврата из функции `ProcA` мы хотим, чтобы вызывающая функция (`ThreadProcedure`) могла использовать регистр `ebp` так же, как он использовался до вызова `FuncA`.

Поскольку next item on stack - saved `ebp`, то есть frame pointer calling function, мы simply pop это value into register `ebp`.

Finally, можно issue instruction `ret`, чтобы return to calling function.

Но погодите: register `esp` (`0x002bffb0`) seems to be pointing to return address, который автоматически pushed onto stack при execution instruction `call`.

Нужно ли что-то делать with that stack location before returning?

Ответ yes and no: yes в том смысле, что return address нужен, чтобы знать where to return to, и no, потому что мы explicitly не pop it from stack.

Когда instruction `ret` executed, return address popped from stack, и control transferred to that location, so execution can resume.

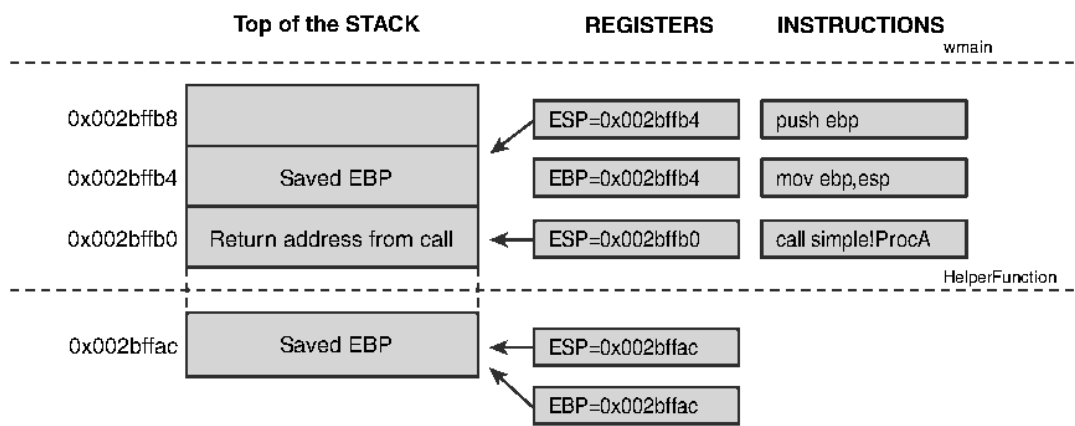


Figure 5.7 Call stack prior to returning from the `ProcA` function

Рисунок 5.7. Стек вызовов перед возвратом из функции `ProcA`

Как видите, stack - very versatile data structure, и он находится at the heart of thread execution in Windows.

Он enables applications transfer control back and forth between functions in a very structured and ordered fashion.

Поскольку compiler generates all code, handling this control transfer, managing stack, passing parameters, addressing local variables и так далее, developers обычно не слишком worry about what actually goes on behind the scenes.

For the most part, developers should not have to worry, но некоторые very frequent programming mistakes могут cause thread stack to become corrupt.

Когда это происходит, understanding how stack is managed can mean difference between successful application launch and disaster.

В следующих sections мы detail некоторые most common scenarios, которые can lead to stack corruption, и ways to apply memory corruption detection process, чтобы get to root cause.

Загадочная instruction `mov edi,edi`

Пролог функции отвечает за настройку текущего кадра.

Как мы видели, общая структура пролога функции настраивает базовый указатель кадра, помещает базовый указатель кадра в стек и резервирует место для локальных переменных.

Вот пример пролога функции `FindFirstFileExW`:

```

0:000> u kernel32!FindFirstFileExW
kernel32!FindFirstFileExW:
7c80ec7d 8bff mov edi,edi      <== Бесполезная инструкция?
7c80ec7f 55   push ebp          <== Сохранить старый базовый указатель кадра
7c80ec80 8bec mov ebp,esp       <== Настроить новый базовый указатель кадра
7c80ec82 81eccc020000 sub esp,0x2cc <== Зарезервировать место для локальных переменных
7c80ec88 837d0c01 cmp dword ptr [ebp+0xc],0x1
7c80ec8c a1cc36887c mov eax,[kernel32!__security_cookie (7c8836cc)]
7c80ec91 53   push ebx
7c80ec92 8945fc mov [ebp-0x4],eax

```

Мы еще не обсудили самую первую и загадочную инструкцию `mov edi,edi`.

Every function prologue begins with этой seemingly useless instruction.

Most of the time instruction `mov edi,edi` - это просто NOP (no operation), но under certain circumstances она может использоваться, чтобы enable hot patching.

Hot patching означает capability patch running code без необходимости сначала stop component being patched.

Этот mechanism crucial для avoiding downtime in system availability.

Basic principle состоит в том, что 2-byte instruction `mov edi,edi` можно replace by instruction `jmp`, которая execute whatever new code is required.

Поскольку это 2-byte instruction, единственная instruction `jmp`, которая actually fit, - short `jmp`, enabling jump of 127 bytes in either direction.

Обычно этого недостаточно, потому что chances are, что вы jump to locations, где existing code already located.

Чтобы bypass this limitation, нужно посмотреть на instructions, preceding instruction `mov edi,edi`:

```

0:000> u kernel32!FindFirstFileExW-9
kernel32!OpenMutexW+a6:
7c80ec74 33c0 xor eax,eax
7c80ec76 eb98 jmp kernel32!OpenMutexW+0xad (7c80ec10)
7c80ec78 90 nop
7c80ec79 90 nop
7c80ec7a 90 nop
7c80ec7b 90 nop
7c80ec7c 90 nop
kernel32!FindFirstFileExW:
7c80ec7d 8bff mov edi,edi

```

Five bytes preceding instruction `mov` - все 1-byte NOP instructions.

Replacing instruction `mov edi,edi` short jump to NOP instructions и replacing those instructions long jump, мы можем easily hot patch to location of choice.

Stack overruns

Stack overrun occurs, когда thread indiscriminately overwrites portions of its call stack, reserved for other purposes.

Это может include, но not limited to, overwriting return address for particular frame, overwriting entire frames или даже exhausting stack completely.

Net effect stack overruns ranges from crashes to unpredictable behavior и даже serious security holes.

Stack overruns стали одним из most common attack angles for malicious software, поскольку potentially позволяют attacker gain complete control of computer, on which faulty software runs.

Чтобы exemplify seriousness stack overruns, рассмотрим scenario, в котором stack overrun could result in security hole.

Seemingly innocent code in Listing 5.9 shows application, которое accepts connection string on command line и attempts to use that connection string to establish connection to data source.

Листинг 5.9. Application, copying command-line connection string

```
#include <windows.h>
#include <stdio.h>
#define MAX_CONN_LEN 30
VOID HelperFunction(WCHAR* pszConnectionString);
int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    if (argc==2)
    {
        HelperFunction(pArgs[1]);
        wprintf (L"Connection to %s established\n",pArgs[1]);
    }
    else
    {
        printf ("Please specify connection string on the command line\n");
    }
    return 0;
}
VOID HelperFunction(WCHAR* pszConnectionString)
{
    WCHAR pszCopy[MAX_CONN_LEN];
    wcsncpy(pszCopy, pszConnectionString);
    //
    // ...
    // Establish connection
    // ...
    //
}
```

Source code и binary для листинга 5.9 можно найти в следующих folders:

Source code: C:\AWD\Chapter5\Overrun
Binary: C:\AWDBIN\WinXP.x86.chk\050verrun.exe

Если запустить это application и указать несколько simple connection strings, все appears to be fine:

```
C:\AWDBIN\WinXP.x86.chk\050verrun.exe MyDataSource
Connection to MyDataSource established
C:\AWDBIN\WinXP.x86.chk\050verrun.exe MyRemoteDataSource
Connection to MyRemoteDataSource established
```

Поскольку code seems to be working fine, все в product group готовятся к ship party.

Через несколько weeks после release product support group начинает получать large number of complaints об application crashes.

Хуже того, по Internet начинают circulate rumors с claims, что application vulnerable to security exploit, позволяющему attacker inject and run arbitrary code in the process.

Чтобы troubleshoot this problem, нужно gather data from product support и посмотреть, possible ли reproduce the problem.

Более глубокий drilldown into data set, provided from support, показывает, что long connection strings seem to be culprit.

И действительно, указание следующей connection string seems to cause application crash:

```
C:\AWDBIN\WinXP.x86.chk\050verrun.exe ThisIsMyVeryExtremelySuperMagnificantConnec-
tionStringForMyDataSource
```

Согласно рисунку 5.1, first step in debugging memory corruption process - analyze state at point of crash. Запустим application under debugger и дадим ему выполняться until crash occurs, как показано in Listing 5.10.

Листинг 5.10. Crash caused by a long connection string

```
...
...
```

```

...
0:000> g
ModLoad: 5cb70000 5cb96000 C:\WINDOWS\system32\ShimEng.dll
(f80.d10): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=0007fefc ebx=7ffde000 ecx=0007ff86 edx=00034d5a esi=7c9118f1 edi=00011970
eip=00630069 esp=0007ff44 ebp=00660069 iopl=0 nv up ei pl nz na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010206
00630069 ?? ???
0:000> kb
ChildEBP RetAddr Args to Child
WARNING: Frame IP not in any known module. Following frames may be wrong.
0007ff40 00430074 006e006f 0065006e 00740063 0x630069
0007ffc0 7c816fd7 00011970 7c9118f1 7ffde000 0x430074
0007fff0 00000000 01001234 00000000 78746341 kernel32!BaseProcessStart+0x23

```

На first glance stack seems so broken, что можно было бы склониться к мысли о potential bug in debugger.

В конце концов, как мы могли cause call stack to get into a state like that?

Again, первое, что нужно сделать, - analyze some state.

Поскольку мы experiencing a crash, crucial first find out where we are crashing.

Поскольку стек вызовов, показанный командой kb, не дает аккуратного и читаемого стека, можно посмотреть на регистр eip, чтобы понять, где мы находимся в коде.

Регистр eip (instruction pointer) также называется счетчиком команд и всегда указывает на следующую инструкцию, которую предстоит выполнить.

Чтобы найти указатель инструкции, используем команду r eip:

```

0:000> r eip
eip=00630069

```

Регистр eip указывает на 0x00630069.

Дамп памяти по этому адресу дает следующее:

```

0:000> dd 00630069
00630069 ???????? ???????? ???????? ????????
00630079 ???????? ???????? ???????? ????????
00630089 ???????? ???????? ???????? ????????
00630099 ???????? ???????? ???????? ????????
006300a9 ???????? ???????? ???????? ????????
006300b9 ???????? ???????? ???????? ????????
006300c9 ???????? ???????? ???????? ????????
006300d9 ???????? ???????? ???????? ????????

```

Содержимое этой области памяти - последовательность вопросительных знаков, которые, как мы знаем, обозначают недоступную память.

Из этого простого упражнения можно выдвинуть гипотезу, что указатель инструкции, используемый процессором для управления потоком выполнения в нашем приложении, оказался поврежден.

Поскольку мы явно не управляем регистром eip, как это возможно?

Ключ к ответу - понять, как регистр eip управляется косвенно.

Мы уже знаем, что процессор автоматически обновляет регистр eip при выполнении инструкций, но что происходит, если встречается инструкция ветвления?

Из предыдущего обсуждения анатомии стека вызовов мы знаем, что при выполнении инструкции call содержимое регистра eip помещается в стек, позволяя процессору понять, где продолжить выполнение.

Когда вызываемая функция возвращается через инструкцию ret, адрес возврата извлекается из стека, eip сбрасывается на этот адрес, и выполнение продолжается оттуда.

Возможно ли, что мы каким-то образом поместили в стек неверный адрес возврата, заставив процессор продолжить выполнение с плохого адреса?

Первое побуждение снова могло бы подсказать "нет", но знание о том, что наш код действительно выполняет ветвления, делает эту теорию довольно правдоподобной.

Перезапустим приложение в отладчике и на этот раз внимательно проследим за состоянием стека. Начнем исследование прямо перед вызовом функции копирования строки в HelperFunction. Рисунок 5.8 показывает состояние стека непосредственно перед вызовом функции wcsncpy.

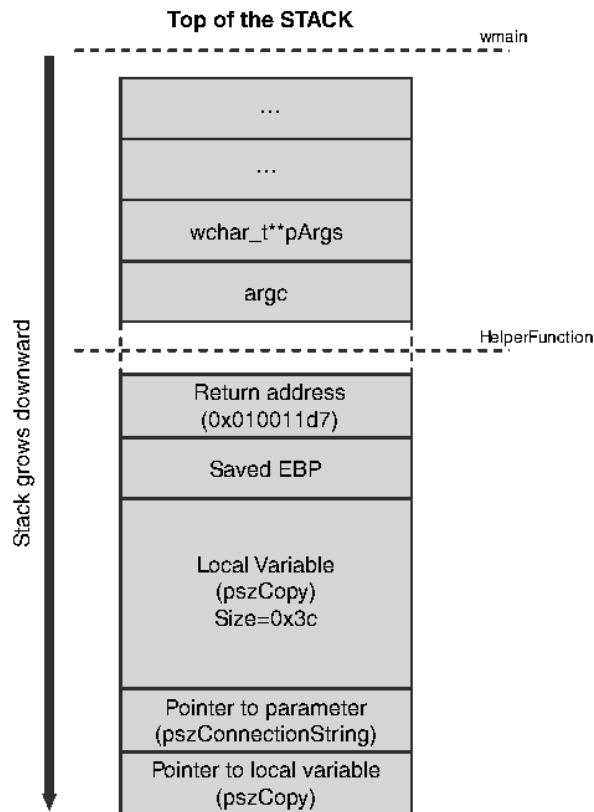


Figure 5.8 Call stack prior to calling wcsncpy from HelperFunction

Рисунок 5.8. Стек вызовов перед вызовом wcsncpy из HelperFunction

Пока стек выглядит исправным.

Теперь выполним вызов функции копирования строки с переходом через него с помощью команды р.

Мы ожидаем, что стек будет выглядеть целым, а локальная переменная pszCopy будет содержать копию строки подключения.

Выведем дамп локальной переменной и посмотрим:

```
0:000> du ebp-0x3c
0007fefc "ThisIsMyVeryExtremelySuperMagnif"
0007ff3c "icantConnectionStringForMyDataSo"
0007ff7c "urce"
```

Выглядит хорошо: содержимое ровно такое, каким мы ожидали его увидеть.

После вызова оставшиеся инструкции - это код эпилога для HelperFunction.

Переходите через инструкции, пока не дойдете до инструкции ret.

Мы знаем, что при выполнении инструкции ret следующий элемент извлекается из стека, и выполнение продолжается с извлеченного адреса.

В качестве проверки здравого смысла выведем дамп следующего элемента в стеке, чтобы увидеть, каким на самом деле является адрес возврата:

```

0:000> dd esp
0007ff3c 00630069 006e0061 00430074 006e006f
0007ff4c 0065006e 00740063 006f0069 0053006e
0007ff5c 00720074 006e0069 00460067 0072006f
0007ff6c 0079004d 00610044 00610074 006f0053
0007ff7c 00720075 00650063 7ffd0000 e4361000
0007ff8c 00000000 00000000 00000002 00034ca8
0007ff9c 00000000 00036ce0 00000000 0007ff7c
0007ffac 89e6a074 0007ffe0 01001442 010010f0
0:000> u 00630069
00630069 ?? ???
^ Memory access error in 'u 00630069'

```

Когда мы пытаемся unassemble return address on stack, получаем memory access error.

Даже without executing instruction ret можно fairly confidently сказать, что теперь мы знаем, что causes crash.

Execution instruction ret показывает, как pointer eip set to bad return address, a subsequent execution этого bad return address fails with access violation.

Поскольку мы знаем, что stack looked fine до call to string copy function, что-то during execution этой function caused stack to become corrupted.

Quick glance at source for HelperFunction показывает, что мы пытаемся make copy of connection string passed in и place it in local variable named pszCopy.

Destination string (pszCopy) declared to be 30 characters in length, а это значит, что source string, которую мы passed in, length 69 characters, will not fit.

Уважает ли wcsncpy boundaries нашей local variable?

Нет.

Фактически единственный stopping point для wcsncpy - когда она reaches null terminator in source string.

Что происходит, когда function wcsncpy passes end of local variable?

Ответ: она просто keeps copying characters.

Поскольку local variable declared on stack, function будет overwrite parts of stack, preceding allocation for local variable.

Рисунок 5.9 показывает, как выглядит стек после копирования.

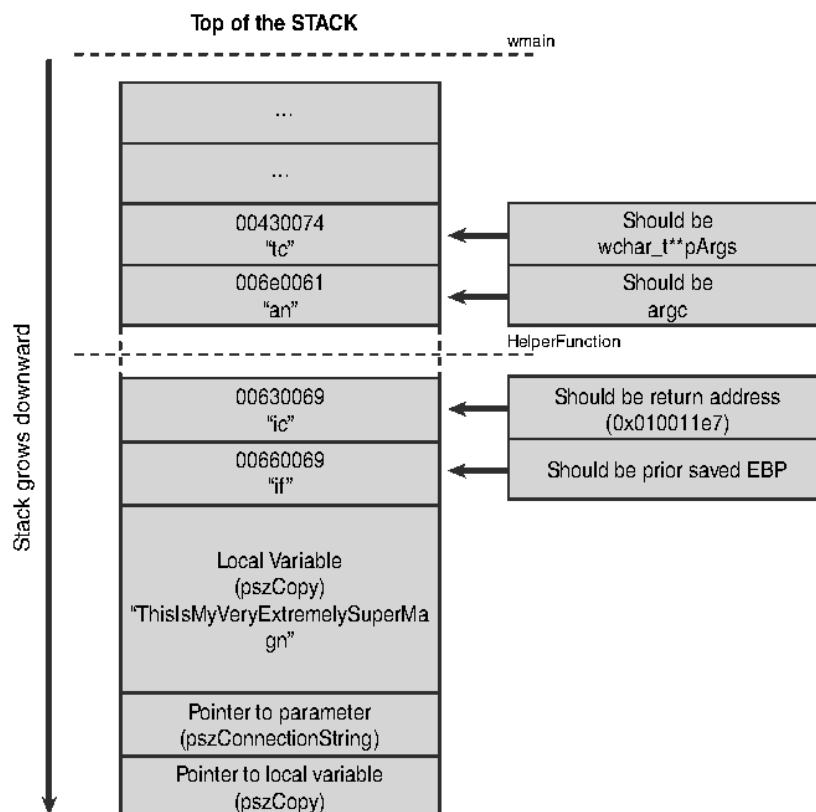


Figure 5.9 Call stack after call to `wcsncpy` from `HelperFunction`

Рисунок 5.9. Стек вызовов после вызова `wcsncpy` из `HelperFunction`

Как видно на рисунке 5.9, казалось бы простое выполнение функции копирования строки полностью повредило наш стек.

После того как string copy function reaches boundary нашей local variable `pszCopy`, она просто keeps copying string, overwriting all stack contents along the way.

More specifically, она overwrites return address, used when `HelperFunction` returns, двумя characters "ic" (`0x00630069`).

Когда processor returns from function using instruction `ret`, это value automatically popped from stack, instruction pointer `eip` set to that value, и execution resumes.

Как вы видели earlier, executing code located in erroneous location `0x00630069` causes crash, потому что location не содержит valid code.

In fact, эта location points to invalid memory.

Fix for this problem - убедиться, что мы do not copy more than allotted for in local variable.

Существуют two possible solutions depending on specification connection string.

- Если connection string может иметь variable length без upper boundaries, allocating memory on stack - wrong approach.
Without knowing size of string at compile time, невозможно allocate buffer on stack, который мог бы hold source string.
В этом case лучше allocate buffer from heap.
- Если connection string действительно limited to 30 characters, нужно make sure to respect that boundary независимо от того, насколько long string, passed in.

Good approach в этом case - использовать string copy function, позволяющую specify size of destination string, чтобы ensure, что more than 30 characters never copied to destination.

См. API StringCchCopy как excellent and safe way to achieve this.

Перед shipping update, containing fix for crashing bug in application, нужно также pay careful attention to rumors, circulating on Internet: security hole was uncovered as well, leading to machine compromise.

Мы уже выполнили большую часть investigative work, чтобы realize, что crash, который мы видели, can also lead to security hole.

Code exploits can utilize fact, что return address can be overwritten.

Если attacker could carefully construct connection string, overwriting return address on stack with address of his choosing, application executed бы code at that address и potentially let attacker take control of application.

Поскольку stack buffer overruns - такие common problems, вы можете wonder, есть ли tool, capable help detect these errors at compile time.

Ответ yes, и tool называется PREfast, part of Windows Driver Kit.

Чтобы illustrate usage of PREfast, используем тот же buffer overrun sample, shown previously.

Начните с открытия build window Windows Driver Kit (checked XP).

Перейдите в directory, containing source code for sample, и введите following:

```
C:\> prefast /filterpreset="Recommended Filters" build /ZCc
```

Эта command line запускает PREfast с recommended filters setting и performs complete build of sources in directory.

Как часть build, PREfast также analyzes code, чтобы determine, есть ли problematic code paths.

После completion process PREfast displays summary of number of defects detected.

```
-----
PREfast reported 1 defects during execution of the command.
-----
Enter PREFAST LIST to list the defect log as text within the console.
Enter PREFAST VIEW to display the defect log user interface.
```

Чтобы view defects, simply enter the following:

```
PREFAST LIST
```

Это используется при displaying defects in console.

Или введите:

```
PREFAST VIEW
```

Это используется при displaying defects in graphical user interface.

As an example, используем list feature PREfast, чтобы see what defects it detected in our source code:

```
-----
Microsoft (R) PREfast Version 8.0.86081.
Copyright (C) Microsoft Corporation. All rights reserved.
-----
Contents of defect log: C:\Documents and Settings\mariah\Application
Data\Microsoft\PDF\defects.xml
-----
c:\awd\chapter5\overrun\overrun.cpp (27): warning 6204: Possible buffer overrun in
call to 'wcscpy': use of unchecked parameter 'pszConnectionString'
FUNCTION: HelperFunction (23)
-----
```

Как видите, PREfast уведомляет нас, что в HelperFunction есть возможное переполнение буфера из-за непроверенного параметра.

PREfast содержит целый набор разных проверок, которые можно применять во время процесса сборки.

Можно использовать predefined или пользовательские фильтры, чтобы изменить набор применяемых проверок.

PREfast - невероятно полезный инструмент при сборке кода, и его настоятельно рекомендуется использовать в процессе сборки.

В конце концов, зачем тратить время на отладку проблемы, которую инструмент может автоматически указать за вас?

Асинхронные операции и указатели стека

Время жизни локальной переменной, объявленной в функции, напрямую связано с областью видимости этой функции.

Если предположить стандартное соглашение о вызовах, то при выполнении функцией кода эпилога указатель стека сбрасывается к предыдущему кадру, и любые локальные переменные считаются недействительными.

Очень распространенная ошибка программирования - делать неверные предположения о времени жизни локальных переменных и тем самым вызывать непредсказуемое поведение во время выполнения.

Чтобы продемонстрировать проблему, мы исследуем зарегистрированный сбой в консольном приложении, которое перечисляет первые два значения реестра по пути, указанному пользователем.

Базовая архитектура этого приложения относительно проста.

Пользователь задает путь в реестре, который хочет перечислить (приложение предполагает, что корневой ключ - HKEY_CURRENT_USER), а затем максимальный тайм-аут перечисления.

Затем приложение вызывает вспомогательную функцию RegEnum, которая запускает перечисление реестра асинхронно, вызывая другую вспомогательную функцию: RegEnumAsync.

Функция RegEnumAsync возвращает дескриптор, которого приложение затем ожидает в течение заданного тайм-аута.

Если происходит тайм-аут, отображается ошибка; в противном случае результат перечисления выводится на экран.

Чтобы уменьшить ненужный шум, перечисление реестра возвращает только значения типа REG_DWORD.

Перед запуском приложения обязательно импортируйте файл test.reg, входящий в комплект приложения:

```
C:\AWDBIN\WinXP.x86.chk>regedit /s test.reg
```

Пример запуска показан в листинге 5.11.

Листинг 5.11. Successful and failed registry enumeration runs

```
C:\AWDBIN\WinXP.x86.chk\05Async.exe
Enter registry key path ("quit" to quit): Test
Enter timeout for enumeration: 5000
Value 1 Name: Value1
Value 1 Data: 1
Value 2 Name: Value2
Value 2 Data: 2
Enter registry key path ("quit" to quit): Does\Not\Exist
Enter timeout for enumeration: 5000
Error enumerating DWORDS in HKEY_CURRENT_USER\Does\Not\Exist within 5000 ms!
Enter registry key path ("quit" to quit): quit
Exiting...
```

Source code и binary для листинга 5.11 можно найти в следующих folders:

```
Source code: C:\AWD\Chapter5\Async
Binary: C:\AWDBIN\WinXP.x86.chk\05Async.exe
```

Как видите, application seems to be working fine.

Valid registry paths successfully enumerate первые two DWORD values, contained within that key, a invalid registry paths generate expected errors.

Единственная remaining variable - timeout, который мы specified как 5000ms.

Когда мы try to pass меньший timeout (2000ms) for valid registry key, получаем failure:

```
C:\AWDBIN\WinXP.x86.chk\05Async.exe
Enter registry key path ("quit" to quit): Test
Enter timeout for enumeration: 2000
Timeout occurred...
Error enumerating DWORDS in HKEY_CURRENT_USER\Test within 2000 ms!
```

Failure может быть expected, поскольку enumeration registry key могла занять more than 2000ms, например during remote registry enumeration.

Unexpected здесь appearance UI Dr. Watson.

Чтобы начать investigating this problem, запускаем application under debugger.

Используя тот же registry path (Test) и timeout value (2000), debugger breaks in with access violation exception, как показано in Listing 5.12.

Листинг 5.12. Access violation after a registry enumeration timeout

```
...
...
...
0:000> g
ModLoad: 5cb70000 5cb96000 C:\WINDOWS\system32\ShimEng.dll
Enter registry key path ("quit" to quit): Test
Enter timeout for enumeration: 2000
Timeout occurred...
Error enumerating DWORDS in HKEY_CURRENT_USER\Test within 2000 ms!
(bc.eb0): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=00000000 ebx=7ffde000 ecx=7c80240f edx=7c90eb94 esi=7c9118f1 edi=00011970
eip=000380d1 esp=0007fd00 ebp=00000001 iopl=0         nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010246
000380d1 006100 add byte ptr [ecx],ah ds:0023:7c80240f=c2
0:000> kb
ChildEBP RetAddr  Args to Child
WARNING: Frame IP not in any known module. Following frames may be wrong.
0007fcfc 7c9118f1 0007fd10 01001a7a 00001770 0x380d1
0007fdcc 7c9118f1 7ffde000 00090000 0007fa18 ntdll!RtlDeleteCriticalSection+0x72
00011970 00750074 00690064 0020006f 005c0038 ntdll!RtlDeleteCriticalSection+0x72
00011970 00000000 00690064 0020006f 005c0038 0x750074
```

Stack at point of access violation выглядит really strange.

Nothing on stack trace дает indication, что executed.

Все, что у нас есть, - mysterious address (0x380d1).

Как подойти к такой проблеме, когда стек выглядит явным мусором и нет указания, что произошло или что выполнялось?

Ответ снова лежит в шаге 1 процесса работы с повреждением памяти: анализе состояния.

Хотя такой трассировки стека может показаться обескураживающей, это еще не конец света.

Чтобы получить лучшую картину происходящего в приложении, важно сделать шаг назад и усомниться в способности отладчика всегда давать правдивые ответы.

В нашем случае нам представлен стек, который выглядит совершенно бесполезным.

Отладчик дал этот стек на основе собственного процесса получения трассировок стека.

Этот процесс, с помощью которого отладчик получает трассировки стека, опирается на то, что определенные части стека остаются целыми.

Если целостность стека нарушена, отладчик наверняка даст неточные результаты.

Чтобы получить гораздо более качественную трассировку стека, нужно выполнить работу самим.

Первое, что нужно сделать, - выяснить, какая инструкция выполнялась в момент сбоя. Это очень легко сделать с помощью команды `u` в отладчике. Помните, что `еір` всегда указывает на инструкцию, которую предстоит выполнить.

```
0:000> u еір
000380d1 006100 add byte ptr [ecx],ah
000380d4 6c ins byte ptr es:[edi],dx
000380d5 007500 add byte ptr [ebp],dh
000380d8 650032 add byte ptr gs:[edx],dh
000380db 0000 add byte ptr [eax],al
000380dd 00adba0df0ad add byte ptr [ebp-520FF246h],ch
000380e3 ba0df0adba mov edx,0BAADF00Dh
000380e8 0df0adba0d or eax,0DBAADF0h
```

Из этого вывода можно сделать несколько наблюдений.

Во-первых, мы пытаемся переместить данные в позицию, на которую указывает регистр `ecx`; он указывает на адрес `0x7c80240f`.

Если дизассемблировать этот адрес, выяснится, что он на самом деле указывает на код, а не на данные как таковые.

Фактически код разрешается в `kernel32!SleepEx`:

```
0:000> u 7c80240f
kernel32!SleepEx+0x8a:
7c80240f c20800 ret 8
7c802412 8975d8 mov dword ptr [ebp-28h],esi
7c802415 c745dc00000080 mov dword ptr [ebp-24h],80000000h
7c80241c 8d45d8 lea eax,[ebp-28h]
7c80241f 8945e4 mov dword ptr [ebp-1Ch],eax
7c802422 ebbd jmp kernel32!SleepEx+0x55 (7c8023e1)
7c802424 3d01010000 cmp eax,101h
7c802429 75ca jne kernel32!SleepEx+0x70 (7c8023f5)
```

Далее, адрес, на который указывает `еір`, не попадает в диапазон адресов ни одного из загруженных сейчас модулей.

Каждый модуль, как кодовый, так и содержащий данные, загружается в процесс по начальному адресу.

Начальный адрес определяется либо самим модулем, либо операционной системой, если возникает конфликт.

В любом случае указатель инструкции почти всегда указывает на позицию внутри диапазона загрузки одного из загруженных модулей.

Диапазоны адресов модулей, загруженных в ваш процесс, очень легко определить с помощью команды `lm`:

```
0:000> lm
start      end          module name
01000000 01003000    05async (deferred)
77c10000 77c68000    msvcrt (deferred)
77dd0000 77e6b000    ADVAPI32 (deferred)
77e70000 77f01000    RPCRT4 (deferred)
7c800000 7c8f4000    kernel32 (pdb symbols)
7c900000 7c9b0000    ntdll (pdb symbols)
```

Текущая позиция `еір` (`000380d1`) не попадает ни в один из показанных диапазонов адресов.

Наконец, код по адресу `еір` выглядит неправильным.

Например, следующая инструкция выполняет OR содержимого регистра `eax` с очень интересным значением:

```
or eax,0DBAADF0h
```

Вооружившись этими наблюдениями, мы формулируем теорию: позиция стека, содержащая адрес возврата, повреждена, из-за чего процессор переходит в допустимую область памяти, содержащую недопустимый код.

Более того, мы знаем, что адрес недопустимой области памяти равен 000380d1 или находится рядом с ним.

Мы говорим "рядом", потому что процессору не особенно важно, где именно выполняется код, пока это допустимая память.

As such, если instructions, которые processor executing, benign from crashing perspective, он продолжит executing и advancing eip, пока real failure не occurs.

В нашем case мы almost certainly executing in valid memory area, albeit not the right code.

Чтобы find corruptor of our stack, нужно сделать detective work on stack itself.

Начнем by dumping out contents of stack, а затем see if we can recognize what execution flow was.

Мы уже знаем established range for our code module (05async.exe): 01000000-01003000.

Looking at stack contents, можно увидеть, находятся ли any elements on stack within that range.

If so, мы могли найти return address, который поможет construct call chain.

Листинг 5.13 shows contents of stack.

Листинг 5.13. Dumping the stack contents

```
0:000> dd esp esp+100
0007fd00 7c9118f1 0007fd10 01001a7a 00001770
0007fd10 0007ff44 0100156a 0007fd2c 00000004
0007fd20 000007d0 00000001 000007d0 00650054
0007fd30 00740073 00000000 00000000 00000000
0007fd40 00000000 00000000 00000000 00000005
0007fd50 a9b81a60 a9b81a74 89e3cc00 80543dfd
0007fd60 00000000 c0000034 888b7370 00f80084
0007fd70 e44b1738 87cd0e00 888b73d0 00000000
0007fd80 00000000 00000068 c0000034 00000000
0007fd90 00000005 a9b81adc 8056a251 888b7370
0007fda0 8056a267 a9b81b98 00000000 00000000
0007fdb0 00000000 00000000 e4657bc8 00000000
0007fdc0 00000038 00000023 00000023 00011970
0007fdd0 7c9118f1 7ffde000 00090000 0007fa18
0007fde0 01001a83 7c910570 7c810665 0000001b
0007fdf0 00000200 0007fffc 00000023 8056a267
0007fe00 8056aa94
```

Обратите внимание, что мы dump stack contents from current location all the way up to current location plus offset 100.

Поскольку stack grows downward, нужно add offset, чтобы получить good look at stack from start to finish.

100 - magic offset?

Не совсем: все depends on how much data put on stack, local variables for each frame и так далее.

Generally, offset 100 - good starting number.

Если ничего useful не находится, можно increase it and try again.

Как видите, three locations on stack fall within range of our module.

Чтобы увидеть, where in our module these locations correspond to, используем command ln:

```
0:000> ln 01001a7a
(01001a20) 05async!DisplayError+0x5a | (01001a83) 05async!wmainCRTStartup
0:000> ln 0100156a
(010014a0) 05async!wmain+0xca | (010015d0) 05async!RegEnum
0:000> ln 01001a83
(01001a83) 05async!wmainCRTStartup | (01001c0a) 05async!operator new
Exact matches:
05async!wmainCRTStartup (void)
```

Из output теперь можно hypothesize следующий call chain:

```
wmainCRTStartup -> wmain -> DisplayError
```

Чтобы reassure ourselves, смотрим source code и видим, что это definitely viable path.

Function wmain ended up calling DisplayError из-за error, occurring while calling RegEnum.

Также fairly safe assume, что error occurred because of timeout, как мы verified in sample runs. DisplayError, в свою очередь, calls API Sleep.

Теперь, когда у нас есть good idea of what is being called and why, можно continue investigation и prove our original hypothesis, что stack in fact corrupted.

Следующий logical step - посмотреть на stack before instruction ret, которая caused our instruction pointer to execute invalid code.

Если dump out contents of stack, this time with negative offset, можно получить historical perspective on execution right before return to invalid memory.

Листинг 5.14 shows dump of stack.

Листинг 5.14. Dumping the stack before the bad return

```
0:000> dd esp-8
0007fcf8 000380d0 00000002 7c9118f1 0007fd10
0007fd08 01001a7a 00001770 0007ff44 0100156a
0007fd18 0007fd2c 00000004 000007d0 00000001
0007fd28 000007d0 00650054 00740073 00000000
0007fd38 00000000 00000000 00000000 00000000
0007fd48 00000000 00000005 a8242a60 a8242a74
0007fd58 89e3cc00 80543dfd 00000000 c0000034
0007fd68 8813c708 00f80084 e44c3570 87c81800
```

При подходе снизу вверх первый интересующий элемент - адрес возврата вызова Sleep (000380d0).

Далее, как всегда, регистр ebp помещается в стек (00000002), чтобы его можно было восстановить перед возвратом.

После этих двух элементов должны следовать любые элементы, помещенные в стек API Sleep, локальные переменные или параметры.

Чтобы лучше понять, что на самом деле делает API Sleep, дизассемблируем функцию:

```
0:000> u kernel32!Sleep
kernel32!Sleep:
7c802442 8bff mov edi,edi
7c802444 55 push ebp
7c802445 8bec mov ebp,esp
7c802447 6a00 push 0
7c802449 ff7508 push dword ptr [ebp+8]
7c80244c e84bffffff call kernel32!SleepEx (7c80239c)
7c802451 5d pop ebp
7c802452 c20400 ret 4
```

Похоже, API Sleep помещает в стек еще два значения: 0 и значение тайм-аута, переданное в API Sleep через стек (ebp+0x8).

Можете заметить несоответствие?

Первые три элемента выглядят неправильными.

Мы точно знаем, что первый элемент должен быть адресом возврата, второй элемент - параметром тайм-аута (ebp+0x8), а третий элемент - 0.

Вместо этого у нас есть адрес возврата 000380d0, который не попадает в диапазон кода модуля. Далее есть значение 2 для параметра тайм-аута, хотя на самом деле должно быть 0x1770, и, наконец, последний элемент должен быть 0, явно помещенным API Sleep, а вместо этого это 7c9118f1.

Теперь мы без сомнений доказали, что происходит повреждение стека, и вся работа, проделанная для доказательства, принесет еще больше пользы, поскольку у нас есть почти вся информация, необходимая для поиска виновника.

Следующий очевидный шаг - выяснить, кто повреждает наш стек.

Поскольку мы уже знаем повреждаемую позицию стека, все, что нужно сделать перед вызовом API Sleep, - каким-то образом отслеживать все обращения к этой позиции стека.

Если бы мы могли останавливать выполнение в отладчике каждый раз, когда по этому адресу выполняется запись, потенциально можно было бы получить трассировку стека, раскрывающую источник повреждения.

К счастью, отладчик снова помогает, на этот раз командой, позволяющей установить точку останова на любой заданный адрес.

Точку останова можно настроить так, чтобы она срабатывала при каждом чтении или записи по этой области памяти либо только при записи.

Перезапустите приложение под отладчиком и установите точку останова в `DisplayError` прямо перед выполнением вызова `Sleep`.

Передайте приложению те же входные параметры, и после остановки в отладчике используйте следующую команду, чтобы установить точку останова на доступ к памяти:

```
0:000> ba w4 0006fcf0
```

Используемая команда - `ba`.

`w` означает запись, затем идет 4, указывающая размер области памяти в байтах.

Последний заданный параметр - адрес области памяти, на которой нужно останавливаться.

Помните, что указанная область памяти - это позиция адреса возврата при возврате из `SleepEx`.

Когда вы продолжите выполнение приложения, точка останова сработает почти сразу:

```
0:000> g
Breakpoint 1 hit
eax=00000043 ebx=7ffde000 ecx=77c422b0 edx=77c61b78 esi=00191ffc edi=00191fc0
eip=7c80239c esp=0007fcf8 ebp=0007fd04 iopl=0 nv up ei pl nz ac po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000212
kernel32!SleepEx:
7c80239c 6a2c push 2Ch
0:000> kb
ChildEBP RetAddr Args to Child
0007fcf4 7c802451 00001770 00000000 0007fd10 kernel32!SleepEx
0007fd04 01001a7a 00001770 0007ff44 0100156a kernel32!Sleep+0xf
0007fd10 0100156a 0007fd2c 00000004 000007d0 05async!DisplayError+0x5a
0007ff44 01001bae 00000001 00034ca8 00036c80 05async!wmain+0xca
0007ffc0 7c816fd7 00191fc0 00191ffc 7ffde000 05async!wmainCRTStartup+0x12b
0007fff0 00000000 01001a83 00000000 78746341 kernel32!BaseProcessStart+0x23
```

Это perfectly makes sense, потому что call to `SleepEx` должен store return address on stack.

Пока никакого foul play.

Continue execution, и получаем another breakpoint - на этот раз намного more interesting than last:

```
0:000> g
Breakpoint 1 hit
eax=0007fcf8 ebx=00035598 ecx=000380d0 edx=00035598 esi=00090178 edi=00000001
eip=01001a01 esp=002bff70 ebp=002bff74 iopl=0 nv up ei pl zr na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
05async!CRegValue::SetProperties+0x11:
01001a01 8b55fc mov edx,dword ptr [ebp-4] ss:0023:002bff70=0007fcf8
0:001> kb
ChildEBP RetAddr Args to Child
002bff74 0100197d 000380d0 00000002 8882ab01 05async!CRegValue::SetProperties+0x11
002bff74 7c80b683 00035598 00000001 00090178 05async!RegThreadProc+0xcd
002bffec 00000000 010018b0 00035598 00000000 kernel32!BaseThreadStart+0x37
```

На этот раз call stack показывает entirely different thread, writing to our return address location.

Quick glance at source code показывает, что каждый раз, когда registry enumeration performed via API `RegEnum`, создается new thread to handle enumeration.

In fact, если closer look at what that thread attempting to store into our return address stack location, мы видим:

```
0:001> p
eax=0007fcf8 ebx=00035598 ecx=000380d0 edx=0007fcf8 esi=00090178 edi=00000001
eip=01001a04 esp=002bff70 ebp=002bff74 iopl=0 nv up ei pl zr na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
```

```

05async!CRegValue::SetProperties+0x14:
01001a04 8b450c mov eax,dword ptr [ebp+0Ch] ss:0023:002bff80=00000002
0:001> dd 0007fcf8
0007fcf8 000380d0 00001770 00000000 0007fd10
0007fd08 01001a7a 00001770 0007ff44 0100156a
0007fd18 0007fd2c 00000004 000007d0 00000001
0007fd28 000007d0 00650054 00740073 00000000
0007fd38 00000000 00000000 00000000 00000000
0007fd48 00000000 00000005 a8242a60 a8242a74
0007fd58 89e3cc00 80543dfd 00000000 c0000034
0007fd68 87df34e8 00f80084 e3d2de08 87dff700

```

Элемент, помещенный в стек, идеально совпадает с нашим предыдущим анализом в листинге 5.14. Теперь мы определили виновника повреждения стека.

Мы закончили?

Не совсем: еще нужно разобраться, почему он записывает данные в эту позицию стека.

Как поток вообще получил указатель на нее?

Он случайно выбрал область памяти для записи?

Последний фрагмент головоломки легко поставить на место с помощью простого просмотра кода.

Если посмотреть на функцию RegThreadProc, стартовую функцию нового потока, видно, что ее параметр имеет тип CRegEnumData.

Функция, создающая этот новый поток, отвечает за передачу экземпляра этого типа потоковой функции.

В этом случае функция RegEnum отвечает за то, чтобы все было правильно настроено до создания нового потока.

Самый важный член CRegEnumData - указатель на массив типа CRegValue.

Этот член содержит результат перечисления, то есть все перечисленные значения.

После того как RegEnum вызывает RegEnumAsync, вызов немедленно возвращается, возвращая дескриптор только что созданного потока.

Функция RegEnum затем ожидает X миллисекунд, как задано в переданном параметре.

Когда ожидание завершается, операция либо закончена и мы можем отобразить результаты, либо произошел тайм-аут; в этом случае мы возвращаемся в функцию wmain, которая затем вызывает DisplayError, чтобы сообщить об ошибке.

Проблемная часть этого кода состоит в том, что функция RegEnum объявляет массив типа CRegValue в стеке и передает адрес этого массива другому потоку.

В случае тайм-аута вызов RegEnum возвращается, делая локально объявленный массив недействительным, тогда как новый поток, выполняющий перечисление значений реестра, все еще имеет указатель на него.

С этого момента каждый раз, когда новый поток записывает результат по этому указателю стека, он будет записывать в область, которая уже не считается действительной.

Как вы видели, фактическая запись не приводит к немедленному сбою, потому что позиция стека все еще считается доступной памятью.

Однако запись может вызвать нежелательные результаты, поскольку она может перезаписать память, используемую другими частями кода.

В нашем случае функция DisplayError подготавливает вызов Sleep, которая, в свою очередь, подготавливает вызов SleepEx.

Всем этим вызовам нужно пространство стека для объявления локальных переменных, передачи параметров и хранения адресов возврата.

Сочетание записи новым потоком в это пространство стека и дальнейшего использования стека приложением вызвало нарушение доступа, потому что адрес возврата был перезаписан.

Несоответствие соглашений о вызовах

Во введении к этой главе мы подробно прошли то, как стек управляется на протяжении времени жизни потока.

Пример выполнил пошаговый анализ тонкостей, связанных с вызовом функций, объявлением локальных переменных, передачей параметров, возвратом из функций и так далее.

Одна тема была намеренно оставлена в стороне - соглашения о вызовах.

Соглашение о вызовах - это не более чем контракт между вызывающей стороной функции и самой функцией.

Оно задает набор правил, с которыми должны согласиться обе стороны, чтобы вызов был выполнен правильно.

Как видно в таблице 5.1, доступно несколько разных типов соглашений о вызовах.

Основное различие между этими соглашениями состоит в том, как параметры передаются вызываемой функции и как они очищаются из стека.

Листинг 5.15 показывает небольшой пример, использующий два самых распространенных соглашения о вызовах: `__cdecl` и `__stdcall`.

Листинг 5.15. Пример использования `__cdecl` и `__stdcall`

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
void __cdecl CDeclFunction(DWORD dwParam1, DWORD dwParam2, DWORD dwParam3);
void __stdcall StdcallFunc(DWORD dwParam1, DWORD dwParam2, DWORD dwParam3);
int __cdecl wmain ()
{
    wprintf(L"Calling CDeclFunction\n");
    CDeclFunction(1,2,3);
    wprintf(L"Calling StdcallFunc\n");
    StdcallFunc(1,2,3);
    return 0;
}
void __cdecl CDeclFunction(DWORD dwParam1, DWORD dwParam2, DWORD dwParam3)
{
    wprintf(L"Inside CDeclFunction\n");
}
void StdcallFunc(DWORD dwParam1, DWORD dwParam2, DWORD dwParam3)
{
    wprintf(L"Inside StdcallFunc\n");
}
```

Исходный код и двоичный файл для листинга 5.15 можно найти в следующих папках:

Source code: C:\AWD\Chapter5\CallConv
Binary: C:\AWDBIN\WinXP.x86.chk\05Callconv.exe

Код в листинге 5.15 объявляет две вспомогательные функции, каждая со своим соглашением о вызовах.

Функция `wmain` просто вызывает каждую из этих функций.

Если запустить это приложение под отладчиком и дизассемблировать функцию `wmain`, можно сразу увидеть, чем два соглашения о вызовах отличаются друг от друга:

```
0:000> u wmain
05callconv!wmain:
01001200 8bff mov edi,edi
01001202 55 push ebp
01001203 8bec mov ebp,esp
01001205 68a8100001 push offset 05callconv!`string' (010010a8)
0100120a ff1500100001 call dword ptr [05callconv!_imp__wprintf (01001000)]
01001210 83c404 add esp,4
01001213 6a03 push 3
01001215 6a02 push 2
0:000> u
05callconv!wmain+0x17:
```

```

01001217 6a01 push 1
01001219 e832000000 call 05callconv!CDeclFunction (01001250)
0100121e 83c40c add esp,0Ch
01001221 687c100001 push offset 05callconv!`string' (0100107c)
01001226 ff1500100001 call dword ptr [05callconv!_imp__wprintf (01001000)]
0100122c 83c404 add esp,4
0100122f 6a03 push 3
01001231 6a02 push 2
0:000> u
05callconv!wmain+0x33:
01001233 6a01 push 1
01001235 e836000000 call 05callconv!StdcallFunc (01001270)
0100123a 33c0 xor eax,eax
0100123c 5d pop ebp
0100123d c3 ret

```

Когда `wmain` готовится вызвать `CDeclFunction`, она начинает с помещения параметров 3, 2 и 1 в стек; помните, они помещаются справа налево.

После этого выполняется сам вызов.

После возврата из вызова выполняется еще одна инструкция: `add esp,0Ch`.

Эта инструкция гарантирует, что указатель стека возвращается в исходную позицию, существовавшую до вызова.

Прибавление `0Ch` просто компенсирует три параметра, которые были помещены в стек перед вызовом.

Логично, что при вызове функции, объявленной с соглашением `__cdecl`, вызывающая функция отвечает за поддержание целостности стека путем корректировки указателя стека.

Если сопоставить это со следующим вызовом функции (`StdcallFunc`), видно, что параметры помещаются тем же способом, справа налево: 3, 2 и 1.

Затем выполняется инструкция `call`, но последующей очистки указателя стека не видно.

Как в этом случае поддерживается целостность стека?

Ответ: сама `StdcallFunc` отвечает за корректировку указателя стека.

Если дизассемблировать `StdcallFunc`, видим следующее:

```

0:000> u StdcallFunc
05callconv!StdcallFunc:
01001270 8bff mov edi,edi
01001272 55 push ebp
01001273 8bec mov ebp,esp
01001275 6804110001 push offset 05callconv!`string' (01001104)
0100127a ff1500100001 call dword ptr [05callconv!_imp__wprintf (01001000)]
01001280 83c404 add esp,4
01001283 5d pop ebp
01001284 c20c00 ret 0Ch

```

Последняя выполняемая инструкция - `ret`, которая передает управление вызывающей функции.

Кроме того, видно, что для инструкции `ret` указан еще один параметр: `0Ch`.

Добавление этого параметра к инструкции `ret` говорит ей скорректировать указатель стека на указанное число байтов.

В этом случае нужно скорректировать его на `0Ch` байт, что соответствует трем параметрам, переданным в функцию.

Основное различие между соглашениями о вызовах `__cdecl` и `__stdcall` - кто отвечает за очистку параметров, переданных в стек.

При использовании `__cdecl` отвечает вызывающая сторона, а при использовании `__stdcall` - вызываемая функция.

Вообще говоря, соглашение о вызовах `__stdcall` - предпочтительный способ вызова функций, потому что оно уменьшает размер сгенерированного кода.

Вместо того чтобы код очистки был разбросан по всему приложению в местах вызова функции, он создается только один раз - в вызываемой функции.

Тогда зачем вообще нужен `__cdecl`?

Соглашение о вызовах `__cdecl` нужно для поддержки списков переменной длины, очень полезной возможности C/C++.

В случаях, когда функция принимает переменное число аргументов, у вызываемой функции нет гарантированного способа узнать, сколько параметров было передано, что делает невозможной правильную очистку стека.

В таких ситуациях требуется `__cdecl`, и задача очистки стека возлагается на вызывающую сторону.

Столбец "Decoration" в таблице 5.1 показывает, как компоновщик декорирует функции, пытаясь гарантировать, что всегда вызывается правильная функция.

Таблица 5.1. Соглашения о вызовах

Соглашение о вызовах	Аргументы	Очистка стека	Декорирование
Stdcall	Стек (справа налево)	Вызываемая функция	К имени функции добавляется префикс <code>_</code> и суффикс <code>@</code> , за которым идет число байтов требуемого пространства стека
Cdecl	Стек (справа налево)	Вызываемая функция	К имени функции добавляется префикс <code>_</code>
Fastcall	Первые два аргумента (≤ 32 бит) передаются через ECX и EDI; остальные в стеке (справа налево)	Вызываемая функция	К имени функции добавляется префикс <code>@</code> и суффикс <code>@</code> , за которым идет число байтов требуемого пространства стека
Thiscall	Указатель <code>this</code> передается через регистр <code>ecx</code> ; остальные в стеке (справа налево)	Вызываемая функция	Декорирование C++

Листинг 5.16 показывает простое приложение, которое объявляет, но не определяет набор функций с разными соглашениями о вызовах.

Листинг 5.16. Объявления с разными соглашениями о вызовах

```
extern void __cdecl Func1(int iOne);
extern void __cdecl Func2(int iOne, int iTwo);
extern void __stdcall Func3(int iOne);
extern void __stdcall Func4(int iOne, int iTwo);
void __cdecl main()
{
    Func1(1);
    Func2(1,2);
    Func3(1);
    Func4(1,2);
}
```

Исходный код для листинга 5.16 можно найти в следующей папке:

Source code: C:\AWD\Chapter5\CallConv2

Если попытаться собрать это приложение, компоновщик выдаст ошибки из-за отсутствующих определений функций:

```
C:\AWD\Chapter5\CallConv2>build /ZC
BUILD: Adding /Y to COPYCMD so xcopy ops won't hang.
BUILD: Object root set to: ==> objchk_wxp_x86
BUILD: Compile and Link for i386
BUILD: Examining C:\AWD\Chapter5\CallConv2 directory for files to compile.
BUILD: Compiling (NoSync) C:\AWD\Chapter5\CallConv2 directory
Compiling - callconv2.c for i386
BUILD: Linking C:\AWD\Chapter5\CallConv2 directory
Linking Executable - objchk_wxp_x86\i386\05callconv2.exe for i386
```

```

errors in directory C:\AWD\Chapter5\CallConv2
callconv2.obj : error LNK2019: unresolved external symbol _Func4@8 referenced in
function _main
callconv2.obj : error LNK2019: unresolved external symbol _Func3@4 referenced in
function _main
callconv2.obj : error LNK2019: unresolved external symbol _Func2 referenced in function
_main
callconv2.obj : error LNK2019: unresolved external symbol _Func1 referenced in function
_main
msvcrt.lib(wcrtexe.obj) : error LNK2019: unresolved external symbol _wmain referenced
in function _wmainCRTStartup
objchk_wxp_x86\i386\05callconv2.exe : error LNK1120: 5 unresolved externals
BUILD: Done
2 files compiled
1 executable built - 6 Errors

```

Ошибки показывают имена, которые компоновщик использует при обращении к объявленным функциям.

Func1 и Func2 объявлены с `__cdecl`, и компоновщик декорирует их, добавляя подчеркивание перед именем функции.

Func3 и Func4 объявлены как `__stdcall`, поэтому они декорируются добавлением подчеркивания в начало и @ в конец, после которого идет общее число байтов всех параметров, входящих в объявление.

Func3 принимает один параметр `int` (4 байта), а Func4 принимает два параметра `int` (всего 8 байтов).

Важно отметить, что схема декорирования, используемая компоновщиком, никогда не видна разработчику при написании кода.

Это исключительно средство компоновщика.

Однако понимать схему декорирования важно, когда нужно разобраться, почему компоновщик иногда выдает ошибки, связанные с неразрешенными внешними символами.

Обычно компилятор и компоновщик работают совместно, чтобы гарантировать вызов правильной функции с правильным соглашением о вызове.

Однако иногда компоновщик не может обеспечить этот механизм за вас, и тогда нужно быть особенно внимательным, чтобы избежать несоответствий соглашений о вызове.

Взгляните на листинг 5.17, где показан код приложения, которое явно загружает DLL (`05mod.dll`) и пытается вызвать функцию `InitModule`, определенную в этой DLL.

Листинг 5.17. Приложение, которое загружает DLL и вызывает `InitModule`

```

#include <windows.h>
#include <stdio.h>
#include <conio.h>
typedef int (__cdecl *MYPROC)(DWORD dwOne, DWORD dwTwo);
VOID CallProc(MYPROC pProc);
int __cdecl wmain ()
{
    HMODULE hMod = LoadLibrary ("05mod.dll");
    if(hMod)
    {
        MYPROC pProc = (MYPROC) GetProcAddress(hMod, "InitModule");
        if(pProc)
        {
            CallProc(pProc);
        }
        else
        {
            wprintf(L"Failed to get proc address of InitModule");
        }
    }
    else
    {
        wprintf(L"Failed to load 05mod.dll.");
    }
}

```

```

return 0;
}
VOID CallProc(MYPROC pProc)
{
pProc(1,2);
}

```

Исходный код и двоичный файл для листинга 5.17 находятся в следующих папках:

```

Source code: C:\AWD\Chapter5\CallConv3\Client and
C:\AWD\Chapter5\CallConv3\Mod
Binary: C:\AWDBIN\WinXP.x86.chk\05CallConv3.exe and C:\AWDBIN\WinXP.x86.chk\05mod.dll

```

Как видите, код довольно прямолинеен.

Сначала он загружает DLL с помощью API LoadLibrary.

Если это удастся, он пытается получить адрес функции InitModule, определенной в DLL, а затем вызывает локальную вспомогательную функцию (CallProc), которая просто вызывает функцию InitModule.

Не заглядывая в реализацию InitModule, скажем только, что при вызове она выводит следующую строку:

```
In InitModule
```

Ничего особенно сложного в этом коде нет, верно?

Если запустить это простое приложение, результат может вас удивить:

```

C:\AWDBIN\WinXP.x86.chk\05CallConv3.exe
In InitModule
In InitModule

```

Строка выводится дважды.

Мало того, похоже, приложение также падает, поскольку появляется печально известный интерфейс Dr. Watson.

Запустим приложение под отладчиком и посмотрим, где в приложении происходит сбой:

```

0:000> g
ModLoad: 5cb70000 5cb96000 C:\WINDOWS\system32\ShimEng.dll
ModLoad: 00400000 00403000 C:\AWDBIN\WinXP.x86.chk\05mod.dll
In InitModule
In InitModule
(8bc.1bc): Unknown exception - code c0000096 (first chance)
(8bc.1bc): Unknown exception - code c0000096 (!!! second chance !!!)
eax=00000001 ebx=7ffdf000 ecx=77c422b0 edx=77c61b78 esi=7c9118f1 edi=00011970
eip=0007fffc5 esp=0007ff50 ebp=004010b0 iopl=0 nv up ei pl nz na po cy
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000203
0007fffc5 6f outs dx, dword ptr [esi] ds:0023:7c9118f1=3359066a
0:000> kb
ChildEBP RetAddr Args to Child
WARNING: Frame IP not in any known module. Following frames may be wrong.
0007ff7c 7c9118f1 7ffdf000 e1389408 00000000 0x7fffc5
00011970 00730069 00610075 0020006f 00740053 ntdll!RtlDeleteCriticalSection+0x72
00011970 00000000 00610075 0020006f 00740053 0x730069

```

Интересно, что стек, показанный для нарушения доступа, похоже, содержит неправильные кадры. Это поразительно похоже на наш предыдущий сеанс отладки, связанный с асинхронными операциями и указателями стека.

Как всегда, когда мы сталкиваемся с потенциальным повреждением стека, мы начинаем с изучения состояния, чтобы понять, можно ли из него извлечь какую-либо полезную информацию.

Начнем с того, что убедимся: адрес в верхнем кадре не попадает ни в один из диапазонов адресов загруженных модулей:

```

0:000> lm
start      end             module name
00400000 00403000      05mod (deferred)
01000000 01003000      05CallConv3 (deferred)
77c10000 77c68000      msvcrt (deferred)

```



```
7c800000 7c8f4000 kernel32 (deferred)
7c900000 7c9b0000 ntdll (pdb symbols)
```

Адрес 0x7ffc5 не попадает ни в один из диапазонов, показанных командой `lm`.

Далее, зная, что отладчик показывает нам неправильные результаты по стеку, мы пытаемся самостоятельно восстановить историческую картину последовательности вызовов, анализируя стек.

В листинге 5.18 показан процесс, в ходе которого мы выводим содержимое стека и пытаемся разрешить любой адрес, попадающий в наш модуль.

Листинг 5.18. Вывод стека при несоответствии соглашений о вызове

```
0:000> dd esp esp+100
0007ff50 00034cb0 00036c88 01001050 01001054
0007ff60 0007ff94 0007ff98 0007ffa0 00000000
0007ff70 0007ff9c 01001058 0100105c 00011970
0007ff80 7c9118f1 7ffdf000 e1389408 00000000
0007ff90 c0000096 00000001 00034cb0 00000000
0007ffa0 00036c88 00000000 0007ff7c 0007fb7c
0007ffb0 0007ffe0 01001486 01001118 00000000
0007ffc0 0007fff0 7c816fd7 00011970 7c9118f1
0007ffd0 7ffdf000 c0000096 0007ffc8 0007fb7c
0007ffe0 ffffffff 7c839aa8 7c816fe0 00000000
0007fff0 00000000 00000000 01001278 00000000
00080000 78746341 00000020 00000001 00002498
00080010 000000c4 00000000 00000020 00000000
00080020 00000014 00000001 00000006 00000034
00080030 00000114 00000001 00000000 00000000
00080040 00000000 00000000 00000000 00000002
00080050 00000000
0:000> ln 01001050
(01001050) 05callconv3!__xc_a | (01001054) 05callconv3!__xc_z
Exact matches:
05callconv3!__xc_a = <function> *[1]
05callconv3!__xc_a = <function> *[]
0:000> ln 01001054
(01001054) 05callconv3!__xc_z | (01001058) 05callconv3!__xi_a
Exact matches:
05callconv3!__xc_z = <function> *[1]
05callconv3!__xc_z = <function> *[]
0:000> ln 01001058
(01001058) 05callconv3!__xi_a | (0100105c) 05callconv3!__xi_z
Exact matches:
05callconv3!__xi_a = <function> *[1]
05callconv3!__xi_a = <function> *[]
0:000> ln 0100105c
(0100105c) 05callconv3!__xi_z | (0100107c) 05callconv3!`string'
Exact matches:
05callconv3!__xi_z = <function> *[1]
05callconv3!__xi_z = <function> *[]
0:000> ln 01001486
(01001486) 05callconv3!except_handler3 | (01001492) 05callconv3!controlfp
Exact matches:
0:000> ln 01001118
(01001110) 05callconv3!`string'+0x8 | (01001128) 05callconv3!_load_config_used
0:000> ln 01001278
(01001278) 05callconv3!wmainCRTStartup | (010013fe) 05callconv3!XcptFilter
Exact matches:
05callconv3!wmainCRTStartup (void)
```

Как видно из листинга 5.18, адреса, попадающие в диапазон нашего модуля, не разрешаются ни во что, что выглядело бы правильным, за исключением 01001278.

Мы даже не видим вызовов функции `InitModule`, хотя знаем, что вызывали ее.

Часто полезно вернуться к основам и заново сформулировать то, что мы наблюдаем: мы видим сбой из-за сильно поврежденного стека, причем без возможности построить историческую картину выполненных последовательностей вызовов.

Если остановиться и подумать, остается еще пространство для расследования.

Какова причина сбоя?

Да, стек сильно поврежден; но какая инструкция вызвала сбой, и можем ли мы извлечь из этого что-нибудь полезное?

Разберем инструкцию по адресу из регистра `eip` и посмотрим, что найдем:

```
0:000> u eip
0007ffc5 6f outs dx,dword ptr [esi]
0007ffc6 817c70190100f118 cmp dword ptr [eax+esi*2+19h],18F10001h
0007ffce 91 xchg eax,ecx
0007ffcf 7c00 jl 0007ffd1
0007ffd1 50 push eax
0007ffd2 fd std
0007ffd3 7f96 jg 0007ff6b
0007ffd5 0000 add byte ptr [eax],al
```

Из разобранного кода можно сделать два наблюдения.

Во-первых, последовательность инструкций явно не выглядит осмысленной.

Из этого наблюдения можно вывести новую теорию: мы выполняем код в, казалось бы, случайном участке памяти.

Чтобы убедиться, что эта теория правдоподобна, посмотрим на второе наблюдение из разобранного кода, а именно на значение самого указателя инструкций (`0007ffc5`).

Если вывести регистры в точке сбоя, мы увидим следующее:

```
0:000> r
eax=00000001 ebx=7ffdc800 ecx=77c422b0 edx=77c61b78 esi=7c9118f1 edi=00011970
eip=0007ffc5 esp=0007ff50 ebp=004010b0 iopl=0 ov up ei ng nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000a82
0007ffc5 6f outs dx,dword ptr [esi] ds:0023:7c9118f1=3359066a
```

Указатель стека и указатель инструкций кажутся подозрительно близкими друг к другу.

Это наблюдение указывает на то, что указатель инструкций каким-то образом получил значение, относящееся к области стека.

Если только мы не собирались выполнять код, расположенный в стеке, что, мягко говоря, почти никогда не является нормальным намерением, мы стали на шаг ближе к ответу.

Следующий большой вопрос таков: как мы пришли к тому, что указатель инструкций указывает на область стека?

Помните, что когда функция возвращается, мы извлекаем значение из стека и устанавливаем указатель инструкций в извлеченное значение.

Обычно это адрес возврата, но в нашем случае из-за поврежденного стека это какое-то другое значение.

Либо адрес возврата был перезаписан, либо мы каким-то образом совершенно неправильно извлекли значение из другой области стека.

Поскольку в стек может помещаться множество элементов: параметры, локальные переменные, адреса возврата, указатели кадров и так далее, почти невозможно сразу сказать, какой именно фрагмент содержимого стека был ошибочно принят за адрес возврата.

На этом этапе лучший подход - заново запустить приложение под отладчиком и внимательно следить за всеми выполняемыми вызовами функций, начиная с функции `main`.

Когда какая-либо вызванная функция возвращается, мы проверяем, какое следующее значение находится в стеке, и смотрим, можно ли сопоставить его с неправильным указателем инструкций, который у нас сейчас есть (`0007ffc5`).

Листинг 5.17 показывает, что приложение выполняет следующие вызовы функций:

- `LoadLibrary`
- `GetProcAddress`
- `CallProc`

Чтобы не тратить ценное время отладки, сосредоточимся на вызове функции CallProc, поскольку к этому моменту мы уже знаем, что именно эта функция фактически вызывает функцию InitModule, расположенную в 05mod.dll.

Мы устанавливаем точку останова на функции CallProc и пошагово доходим до вызова InitModule (eip должен указывать на 01001269).

Затем заходим внутрь вызова функции и продолжаем пошаговое выполнение, пока не дойдем до инструкции ret.

Именно здесь нужно начать смотреть внимательнее.

Когда выполняется инструкция ret, мы знаем, что адрес возврата будет извлечен из стека, а указатель инструкций будет установлен в это значение.

Выведа содержимое стека и разобрав предполагаемый адрес возврата, мы увидим следующее:

```
0:000> dd esp
0007ff24 0100126c 00000001 00000002 0007ff44
0007ff34 0100122d 004010b0 004010b0 00400000
0007ff44 0007ffc0 010013a3 00000001 00034cb0
0007ff54 00036c88 01001050 01001054 0007ff94
0007ff64 0007ff98 0007ffa0 00000000 0007ff9c
0007ff74 01001058 0100105c 00191fc0 00191ffc
0007ff84 7ffd6000 e466e840 00000000 00000000
0007ff94 00000001 00034cb0 00000000 00036c88
0:000> u 0100126c
05callconv3!CallProc+0xc:
0100126c 83c408 add esp,8
0100126f 5d pop ebp
01001270 c20400 ret 4
01001273 cc int 3
01001274 cc int 3
01001275 cc int 3
01001276 cc int 3
01001277 cc int 3
```

Информация, которую мы только что получили, полностью согласуется с ожиданиями.

Адрес возврата в стеке действительно указывает на инструкцию сразу после вызова CallProc.

Продолжая пошаговое выполнение кода, следующая инструкция ret, с которой мы сталкиваемся, относится к возврату функции CallProc в wmain:

```
0:000> p
eax=00000001 ebx=7ffd6000 ecx=77c422b0 edx=77c61b78 esi=00191ffc edi=00191fc0
eip=0100126c esp=0007ff30 ebp=0007ff30 iopl=0 nv up ei pl nz ac po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000212
05callconv3!CallProc+0xc:
0100126c 83c408 add esp,8
0:000> p
eax=00000001 ebx=7ffd6000 ecx=77c422b0 edx=77c61b78 esi=00191ffc edi=00191fc0
eip=0100126f esp=0007ff38 ebp=0007ff30 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
05callconv3!CallProc+0xf:
0100126f 5d pop ebp
0:000> p
eax=00000001 ebx=7ffd6000 ecx=77c422b0 edx=77c61b78 esi=00191ffc edi=00191fc0
eip=01001270 esp=0007ff3c ebp=004010b0 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
05callconv3!CallProc+0x10:
01001270 c20400 ret 4
```

Мы используем тот же прием, чтобы проверить, что адрес возврата, который мы собираемся извлечь из стека, является правильным:

```
0:000> dd esp
0007ff3c 004010b0 00400000 0007ffc0 010013a3
0007ff4c 00000001 00034cb0 00036c88 01001050
0007ff5c 01001054 0007ff94 0007ff98 0007ffa0
0007ff6c 00000000 0007ff9c 01001058 0100105c
0007ff7c 00191fc0 00191ffc 7ffd6000 e466e840
0007ff8c 00000000 00000000 00000001 00034cb0
```

```

0007ff9c 00000000 00036c88 00000000 0007ff7c
0007ffac 89e6904c 0007ffe0 01001486 01001118
0:000> u 004010b0
05mod!InitModule:
004010b0 8bff mov edi,edi
004010b2 55 push ebp
004010b3 8bec mov ebp,esp
004010b5 682c104000 push offset 05mod!`string' (0040102c)
004010ba ff1500104000 call dword ptr [05mod!_imp__wprintf (00401000)]
004010c0 83c404 add esp,4
004010c3 b801000000 mov eax,1
004010c8 5d pop ebp

```

На этот раз все выглядит явно неправильно.

Мы должны вернуться в `wmain`, но вместо этого адрес возврата указывает на начало функции `InitModule`.

Это определенно объясняет, почему мы видим вывод `InitModule` дважды, и, возможно, объясняет даже сам сбой.

Теперь продолжим, войдя в функцию `InitModule` и выполняя ее пошагово, пока снова не дойдем до инструкции `ret`.

В этот момент выведем содержимое стека, чтобы увидеть, куда она решит вернуться на этот раз:

```

0:000> dd esp
0007ff44 0007ffc0 010013a3 00000001 00034cb0
0007ff54 00036c88 01001050 01001054 0007ff94
0007ff64 0007ff98 0007ffa0 00000000 0007ff9c
0007ff74 01001058 0100105c 00191fc0 00191ffc
0007ff84 7ffdf000 e46e840 00000000 00000000
0007ff94 00000001 00034cb0 00000000 00036c88
0007ffa4 00000000 0007ff7c 89e6904c 0007ffe0
0007ffb4 01001486 01001118 00000000 0007fff0
0:000> u 0007ffc0
0007ffc0 f0ff07 lock inc dword ptr [edi]
0007ffc3 00d7 add bh,d1
0007ffc5 6f outs dx,dword ptr [esi]
0007ffc6 817cc01f1900fc1f cmp dword ptr [eax+eax*8+1Fh],1FFC0019h
0007ffce 1900 sbb dword ptr [eax],eax
0007ffd0 0060fd add byte ptr [eax-3],ah
0007ffd3 7ffd jg 0007ffd2
0007ffd5 3d5480c8ff cmp eax,0FFC88054h

```

Инструкция, к которой мы будем возвращаться на этот раз, находится по адресу `0007ffc0`, что точно совпадает с тем, что мы искали; если выполнить инструкцию `ret` с переходом через нее, мы окажемся в точке, где вот-вот произойдет сбой.

Пока мы трассировали эту программу, первая проблема проявилась в момент, когда функция `CallProc` собиралась вернуться.

Вместо возврата в исходную функцию `wmain` она вернулась к началу функции `InitModule`.

Давайте посмотрим на разобранную функцию `CallProc` и попробуем понять, как должен выглядеть стек на протяжении выполнения этой функции:

```

0:000> u CallProc
05callconv3!CallProc:
01001260 8bff mov edi,edi
01001262 55 push ebp
01001263 8bec mov ebp,esp
01001265 6a02 push 2
01001267 6a01 push 1
01001269 ff5508 call dword ptr [ebp+8]
0100126c 83c408 add esp,8
0100126f 5d pop ebp
0:000> u
05callconv3!CallProc+0x10:
01001270 c20400 ret 4

```

На рисунке 5.10 показано, каким мы ожидаем увидеть стек в момент, когда указатель инструкций вот-вот выполнит инструкцию `call` для `InitModule`.

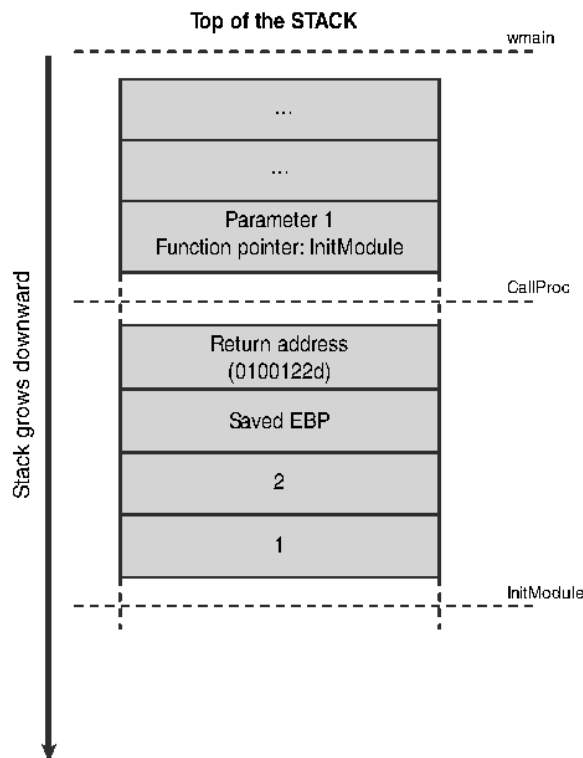


Figure 5.10 Expected stack prior to calling `InitModule`

Рисунок 5.10. Ожидаемый стек перед вызовом `InitModule`

Теперь учтем, что функция `InitModule` принимает два параметра (оба типа `DWORD`), и при возврате функции мы ожидаем, что указатель стека будет установлен на позицию в стеке перед списком параметров:

```
0:000> p
In InitModule
eax=00000001 ebx=7ffd5000 ecx=77c422b0 edx=77c61b78 esi=00191ffc edi=00191fc0
eip=0100126c esp=0007ff30 ebp=0007ff30 iopl=0 nv up ei pl nz ac po nc
cs=001b ss=0023 ds=0023 fs=003b gs=0000 efl=00000212
05callconv3!CallProc+0xc:
0100126c 83c408 add esp,8
0:000> dd esp
0007ff30 0007ff44 0100122d 004010b0 004010b0
0007ff40 00400000 0007ffc0 010013a3 00000001
0007ff50 00034cb0 00036c88 01001050 01001054
0007ff60 0007ff94 0007ff98 0007ffa0 00000000
0007ff70 0007ff9c 01001058 0100105c 00191fc0
0007ff80 00191ffc 7ffd5000 e46afdd8 00000000
0007ff90 00000000 00000001 00034cb0 00000000
0007ffa0 00036c88 00000000 0007ff7c 89e6a074
```

После возврата функции `esp` снова устанавливается на позицию в стеке перед областью параметров, что означает: вызванная функция (`InitModule`) корректно очистила стек, то есть восстановила указатель стека.

Инструкция, следующая за инструкцией `call`, такова:

```
add esp,8
```

Эта инструкция, по-видимому, прибавляет 8 байтов к указателю стека, в результате чего указатель стека фактически перескакивает через сохраненное значение `ebp` и адрес возврата, помещенные в

стек.

Чтобы вернуться к предыдущему кадру, нам ведь нужен адрес возврата?

Безусловно.

На самом деле прибавление 8 байтов к указателю стека, похоже, и является корневой причиной нашей проблемы.

Когда мы доходим до кода эпилога `CallProc`, мы в итоге извлекаем неправильное значение для `ebp`, хотя должны были извлечь сохраненное значение `ebp`, а также возвращаемся по неправильному адресу.

Неправильный адрес в данном случае - это адрес функции `InitModule`.

Причина выбора именно этого адреса состоит в том, что прибавление 8 байтов к указателю стека помещает нас в точку, где в стек был помещен параметр для `CallProc`.

Он же оказывается указателем на функцию `InitModule`.

Последняя часть головоломки - понять, почему указатель стека управляется таким неправильным образом.

Мы уже знаем, что функция `CallProc` пытается очистить стек, то есть пропускает параметры, переданные функции `InitModule`.

Очистка стека после вызовов функций необходима для сохранения целостности стека.

Однако мы также видели, что после возврата из `InitModule`, но до прибавления 8 байтов к указателю стека, указатель стека уже выглядел правильным.

То есть стек уже был очищен.

Похоже, это означает, что функция `InitModule` уже очистила стек в момент возврата.

Если разобрать функцию `InitModule`, можно увидеть, что она именно это и делает.

Не должно удивлять, что корневая причина проблемы - несоответствие соглашений о вызове.

Поскольку `InitModule` очищает стек перед возвратом, она была объявлена с соглашением о вызове `__stdcall`:

```
int __stdcall InitModule(DWORD dwOne, DWORD dwTwo)
```

тогда как клиентский код объявил указатель на функцию `InitModule` со следующей сигнатурой:

```
typedef int (__cdecl *MYPROC)(DWORD dwOne, DWORD dwTwo);
```

Несоответствие соглашений о вызове привело к серьезному повреждению нашего стека.

Системы с поддержкой NX

В предыдущем сеансе отладки мы показали, как несоответствие соглашений о вызове может заставить приложение выполнять код в стеке.

Итогом стала странная цепочка вызовов и, в конечном счете, сбой.

Эту проблему можно обобщить до выполнения кода в любой области, зарезервированной только для данных.

Авторы вредоносного ПО часто используют такую возможность, внедряя код в память, зарезервированную под данные, а затем просто переходя к этому коду и выполняя его.

Производители процессоров и программного обеспечения осознали необходимость защиты от этой проблемы, и результатом стали процессоры с поддержкой NX (No eXecute). Основная идея состоит в том, чтобы пометить области битом NX, который указывает, что в этой памяти могут храниться только данные.

Если из этой области когда-либо будет выполнен код, немедленно возникнет сбой.

Windows поддерживает системы с NX начиная с Windows XP SP2 и Windows Server 2003 SP1.

На системах с аппаратной поддержкой NX и версией Windows, поддерживающей NX, результатом выполнения кода из памяти, предназначенной только для данных, будет нарушение доступа.

Стратегии предотвращения

Как вы видели, последствия повреждений стека, подобно последствиям других типов повреждений памяти, не обязательно проявляются прямо в точке повреждения.

Вместо этого повреждение стека может довольно долго оставаться незамеченным, прежде чем произойдет реальный сбой.

Как мы уже упоминали ранее в этой главе, проще всего отследить повреждение тогда, когда мы можем поймать его в момент возникновения.

Есть несколько вариантов, позволяющих обнаруживать повреждения стека на ранних этапах процесса разработки.

Лучшая линия защиты находится в самом компиляторе, поскольку он способен встраивать в ваш код проверки целостности стека.

Чтобы включить эти проверки времени выполнения, приложение нужно собрать с правильным набором параметров.

Первый параметр компилятора, который мы обсудим, - это ключ /GS.

Хотя атаки переполнения стекового буфера существуют уже довольно давно, в последние годы они стали особенно популярны.

Большое количество вирусов использует этот вектор атаки, чтобы наносить ущерб компьютерам.

По этой причине команда компилятора Microsoft ввела механизм, который защищает стек и служит страховочной сеткой против атак переполнения буфера.

Как вы видели ранее, основная проблема переполнений стекового буфера состоит в том, что атакующий может перезаписать адрес возврата кадра и возобновить выполнение в выбранном им месте.

Если бы мы каким-то образом могли защитить адрес возврата от перезаписи, уязвимость нельзя было бы использовать.

Введение флага /GS делает шаг в сторону такой защиты: перед адресом возврата в стек помещается cookie, а когда функция возвращается, проверяется, не поврежден ли этот cookie.

Если он цел, адрес возврата не был изменен, и выполнение продолжается.

Если значение отличается, это означает, что адрес возврата, возможно, был изменен, и приложение завершается.

Чтобы получить эту дополнительную защиту, в среду сборки нужно внести следующие изменения:

- **Sources**

Файл sources должен задавать флаг компилятора /GS следующим образом:

```
USER_C_FLAGS=/GS
```

- **Окно сборки**

Переменная среды BUFFER_OVERFLOW_CHECKS должна быть равна 1.

Если посмотреть на приложение, использованное в сценарии переполнения буфера (05overrun.exe), можно увидеть, что пролог функции HelperFunction содержит несколько дополнительных шагов:

```
0:000> u 05overrun!helperfunction
05overrun!HelperFunction:
01001230 8bff mov edi,edi
01001232 55 push ebp
01001233 8bec mov ebp,esp
01001235 83ec40 sub esp,40h
01001238 a118200001 mov eax,dword ptr [05overrun!__security_cookie
(01002018)]
0100123d 8945fc mov dword ptr [ebp-4],eax
```

```
01001240 8b4508 mov eax,dword ptr [ebp+8]
01001243 50 push eax
```

Две выделенные инструкции mov показывают, как функция берет уникальный cookie и помещает его в стек в позицию перед адресом возврата.

Перед возвратом, в эпилоге функции, позиция стека, содержащая cookie (ebp-0x4), проверяется относительно исходного cookie:

```
0:000> u helperfunction+19
050verrun!HelperFunction+0x19:
01001249 1560100001 adc eax,offset 050verrun!_imp__wcsncpy (01001060)
0100124e 83c408 add esp,8
01001251 8b4dfc mov ecx,dword ptr [ebp-4]
01001254 e87e000000 call 050verrun!__security_check_cookie (010012d7)
01001259 8be5 mov esp,ebp
0100125b 5d pop ebp
0100125c c20400 ret 4
```

Вызов __security_check_cookie проверяет, остался ли cookie целым; если нет, вызов завершает процесс.

По умолчанию, если cookie был перезаписан, обработчик показывает диалоговое окно с сообщением о том, что произошло переполнение буфера.

Если вы не хотите, чтобы при неудачной проверке cookie отображалось диалоговое окно, можно предоставить собственный обработчик.

Cookie создается CRT (C runtime) во время запуска и при каждом запуске программы отличается, чтобы атакующие не знали его значение.

Есть несколько предостережений, о которых нужно помнить.

Если приложения не используют CRT, во время запуска нужно явно вызвать __security_init_cookie, чтобы гарантировать правильную инициализацию cookie.

Кроме того, приложения, которые явно вызывают инициализацию CRT, могут непреднамеренно повторно инициализировать этот cookie, что приведет к сбою проверки безопасности, поскольку cookie изменится.

Также важно отметить, что этот параметр компилятора предназначен для использования с выпускаемым кодом.

Критически важно понимать, что страховочную сетку /GS следует воспринимать именно как страховочную сетку.

Ни при каких обстоятельствах не следует полагаться на этот механизм как на полноценную защиту от атак переполнения буфера.

Следующий важный ключ компилятора - /RTC.

RTC означает RunTimeChecks.

RTC предоставляет несколько подпараметров.

- **/RTCs: проверка ошибок кадра стека во время выполнения**

Этот параметр помогает защищаться от нескольких разных повреждений стека:

- При каждом вызове функции он инициализирует все локальные переменные ненулевыми значениями, чтобы они не сохраняли старые значения от предыдущих вызовов функций.
- Он проверяет указатель стека (регистр esp), чтобы убедиться, что не возникают повреждения стека, вызванные несоответствиями соглашений о вызове.
- Он защищает от переполнений и выходов за нижнюю границу локальных переменных.

- **/RTCc: защита от потери данных**

Еще одна распространенная ошибка разработчиков - выполнять приведения между типами данных, приводящие к потере данных.

Например, приведение значения ULONG к значению BYTE может привести к потере данных.

Этот параметр компилятора отображает диалог ошибки всякий раз, когда приведение приводит к потере данных.

- **/RTCu: защита от неинициализированных переменных**

Этот параметр компилятора отображает ошибку всякий раз, когда выполняется доступ к переменной, которая еще не была инициализирована.

Неинициализированные переменные - распространенная ошибка при разработке; из-за нее ваши переменные могут получать значения, оставшиеся от предыдущих вызовов.

Эти значения могут создать массу проблем во время выполнения.

Важно отметить, что параметры компилятора /RTC предназначены для работы с отладочными сборками и поэтому не влияют на выпускаемые сборки.

Ключ /RTC предназначен исключительно для тестирования вашего кода во время разработки.

Хотя параметры компилятора дают отличный механизм поиска ошибок, связанных с повреждением стека, во время разработки, они не обеспечивают такого же уровня обнаружения, как другие инструменты.

К другим жизнеспособным, хотя и не бесплатным, вариантам относятся Rational Purify и NuMega BoundsChecker.

Резюме

Как вы видели на протяжении этой главы, приложение, страдающее от повреждения стека, может вызывать серьезные проблемы со стабильностью.

Эти проблемы обычно проявляются в виде случайных сбоев, которые в конечном счете оставляют пользователей раздраженными и уставшими.

В худшем случае повреждения стека могут даже привести к серьезным дырам в безопасности, способным скомпрометировать компьютер пользователя и сделать его уязвимым для множества разных атак.

Любому серьезному разработчику крайне важно знать причины повреждения стека и способы его анализа.

В конечном итоге разработчик должен применять методы предотвращения, чтобы обеспечить целостность стека и будущий успех своего программного обеспечения.

В этой главе мы подробно разобрали анатомию стека.

Мы также прошли через некоторые из самых распространенных форм повреждения стека, объяснили, как обнаруживать повреждение, и рассмотрели, как анализировать его и находить корневую причину.

Наконец, вы узнали, как мощные возможности компилятора могут помочь ловить повреждения стека во время разработки и даже помогать предотвращать некоторые формы повреждения стека в выпускаемом программном обеспечении.

Глава 6. Повреждение памяти, часть II - кучи

В главе 5, "Повреждение памяти, часть I - стеки", мы обсуждали, как переполнения буфера на основе стека могут вызывать серьезные проблемы безопасности в программном обеспечении и как переполнения буфера на основе стека были основным вектором атак для авторов вредоносного ПО.

Однако в последние годы стала набирать популярность другая форма атаки переполнения буфера.

Вместо того чтобы полагаться на стек для эксплуатации переполнений буфера, атакующие теперь нацеливаются на менеджер кучи Windows.

Хотя атаки безопасности на основе кучи гораздо труднее эксплуатировать, чем их стековые аналоги, их популярность продолжает быстро расти.

Помимо потенциальных уязвимостей безопасности, в этой главе обсуждается множество проблем стабильности, которые могут проявиться в приложении, когда куча используется нестандартным образом.

Хотя стек и куча управляются в Windows очень по-разному, процесс анализа проблем, связанных со стеком и кучей, одинаков.

Поэтому на протяжении этой главы мы используем тот же процесс устранения неполадок, который определили в главе 5 (см. рисунок 5.1).

Что такое куча?

Куча - это разновидность менеджера памяти, который приложение может использовать, когда ему нужно динамически выделять и освобождать память.

Обычные ситуации, требующие использования кучи, возникают тогда, когда размер требуемой памяти заранее неизвестен и когда размер памяти слишком велик, чтобы аккуратно разместиться в стеке (автоматической памяти).

Хотя куча является самым распространенным средством для динамических выделений памяти, у приложений есть и ряд других способов запрашивать память у Windows.

Память можно запрашивать у C runtime, у менеджера виртуальной памяти и даже у других форм частных менеджеров памяти.

Хотя разные менеджеры памяти можно рассматривать как отдельные сущности, внутренне они тесно связаны.

На рисунке 6.1 показан упрощенный вид поддерживаемых Windows менеджеров памяти и их зависимостей.

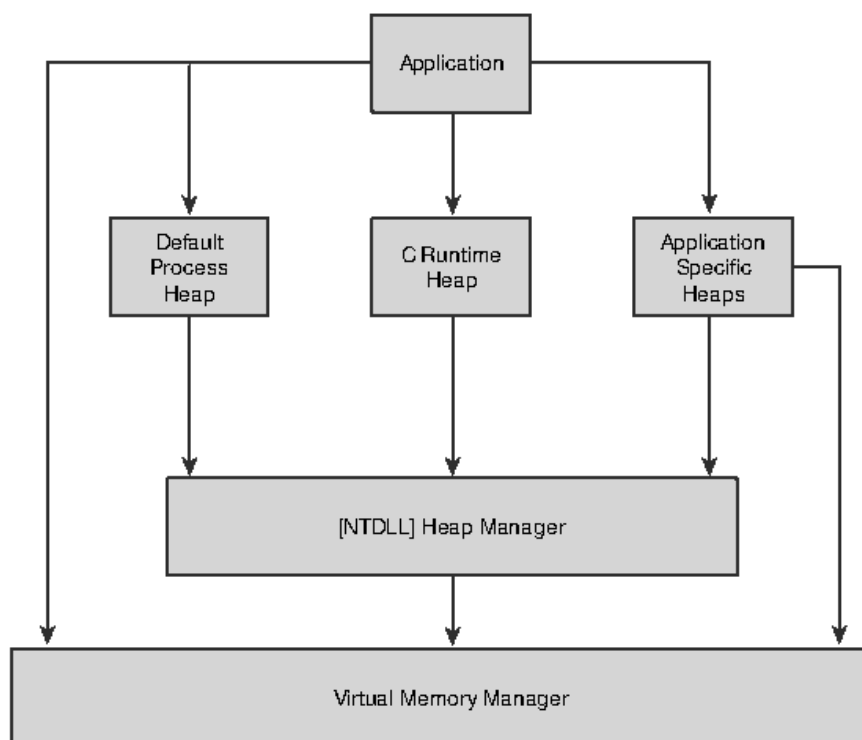


Figure 6.1 An overview of Windows memory management architecture

Рисунок 6.1. Обзор архитектуры управления памятью Windows

Как показано на рисунке 6.1, большинство высокоуровневых менеджеров памяти используют менеджер кучи Windows, который, в свою очередь, использует менеджер виртуальной памяти. Хотя высокоуровневые менеджеры памяти (и приложения, если уж на то пошло) не обязаны использовать менеджер кучи, чаще всего они делают именно это, поскольку он предоставляет прочную основу, на которой можно строить другие частные менеджеры памяти.

Из-за его популярности основное внимание в этой главе уделено менеджеру кучи Windows.

Когда процесс запускается, менеджер кучи автоматически создает новую кучу, называемую кучей процесса по умолчанию.

Хотя некоторые процессы используют кучу процесса по умолчанию, большое число процессов полагается на кучу CRT (используя семейства API `new/delete` и `malloc/free`) для всех своих потребностей в памяти.

Однако некоторые процессы создают дополнительные кучи (через API `HeapCreate`), чтобы изолировать разные компоненты, работающие в процессе.

Даже у самых простых приложений нередко бывает четыре или больше активных куч в любой момент времени.

Менеджер кучи Windows можно дополнительно разложить на части, как показано на рисунке 6.2.

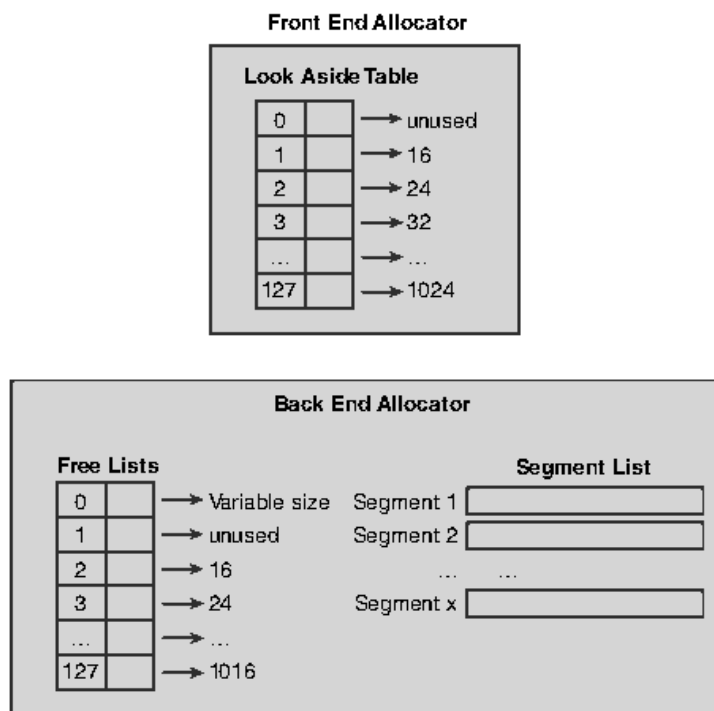


Figure 6.2 Windows heap manager

Рисунок 6.2. Менеджер кучи Windows

Front end allocator

Front end allocator - это абстрактный оптимизационный слой для back end allocator.

Разрешая разные типы front end allocator, приложения с разными потребностями в памяти могут выбирать подходящий распределитель.

Например, приложения, ожидающие коротких всплесков выделений памяти, могут предпочесть low fragmentation front end allocator, чтобы избежать фрагментации.

В Windows доступны два разных front end allocator:

- Look aside list (LAL) front end allocator
- Low fragmentation (LF) front end allocator (распределитель с низкой фрагментацией)

За исключением Windows Vista, все версии Windows по умолчанию используют LAL front end allocator.

В Windows Vista архитектурным решением стал переход на LF front end allocator по умолчанию.

Look aside list - это не более чем таблица из 128 односвязных списков.

Каждый односвязный список в таблице содержит свободные блоки кучи определенного размера, начиная с 16 байтов.

Размер каждого блока кучи включает 8 байтов метаданных блока кучи, используемых для управления блоком.

Например, если в front end allocator приходит запрос на выделение 24 байтов, front end allocator будет искать свободные блоки размером 32 байта (24 байта, запрошенные пользователем, плюс 8 байтов метаданных).

Поскольку все блоки кучи требуют 8 байтов метаданных, минимальный блок, который может быть возвращен вызывающему коду, имеет размер 16 байтов; следовательно, front end allocator не использует индекс таблицы 1, соответствующий свободным блокам размером 8 байтов.

Далее каждый индекс представляет свободные блоки кучи, размер которых равен размеру предыдущего индекса плюс 8.

Последний индекс (127) содержит свободные блоки кучи размером 1024 байта.

Когда приложение освобождает блок памяти, менеджер кучи помечает выделение как свободное и помещает его в look aside list front end allocator (в соответствующий индекс).

В следующий раз, когда запрашивается блок памяти такого размера, front end allocator проверяет, доступен ли блок памяти запрошенного размера, и если да, возвращает блок кучи пользователю. Само собой разумеется, что удовлетворение выделений через look aside list - безусловно самый быстрый способ выделять память.

Рассмотрим гипотетический пример.

Представьте, что состояние LAL выглядит так, как показано на рисунке 6.3.

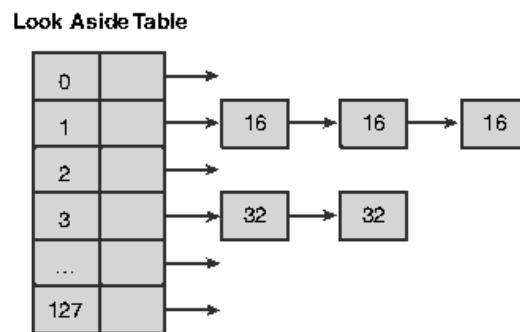


Figure 6.3 Hypothetical state of the look aside list

Рисунок 6.3. Гипотетическое состояние look aside list

LAL на рисунке 6.3 показывает, что по индексу 1 доступны три блока кучи размером 16 (из которых вызывающему коду доступны 8 байтов), а по индексу 3 - два блока размером 32 (из которых вызывающему коду доступны 24 байта).

Когда мы пытаемся выделить блок размером 24, менеджер кучи знает, что нужно смотреть на индекс 3: он добавляет 8 к запрошенному размеру блока (учитывая размер метаданных), делит на 8 и вычитает 1 (таблица с нулевой базой).

Связный список, расположенный по индексу 3, содержит два доступных блока кучи.

Менеджер кучи просто удаляет первый из списка и возвращает выделение вызывающему коду.

Если мы попробуем выделить блок размером 16, менеджер кучи заметит, что индекс, соответствующий размеру 16 ($((16+8)/8-1=2)$), является пустым списком, и поэтому выделение не может быть удовлетворено из LAL.

Запрос на выделение теперь продолжает свой путь и передается back end allocator для дальнейшей обработки.

Back End Allocator

Если front end allocator не может удовлетворить запрос на выделение, запрос попадает в back end allocator.

Подобно front end allocator, он содержит таблицу списков, обычно называемых free lists.

Единственная обязанность free list - отслеживать все свободные блоки кучи, доступные в конкретной куче.

Существует 128 free lists, где каждый список содержит свободные блоки кучи определенного размера.

Как видно на рисунке 6.2, размер, связанный с `free list[2]`, равен 16, с `free list[3]` - 24 и так далее.

`Free list[1]` не используется, потому что минимальный размер блока кучи равен 16 (8 байтов метаданных и 8 байтов, доступных пользователю).

Каждый размер, связанный со `free list`, увеличивается на 8 байтов относительно предыдущего `free list`.

Выделения, размер которых превышает максимальный размер выделения для `free list`, попадают в индекс 0 таблицы `free lists`.

`Free list[0]` по существу содержит выделения размером больше 1016 байтов и меньше предела виртуального выделения (он обсуждается позже).

Свободные блоки кучи в `free list[0]` также отсортированы по размеру (по возрастанию) для достижения максимальной эффективности.

На рисунке 6.4 показан гипотетический пример `free list`.

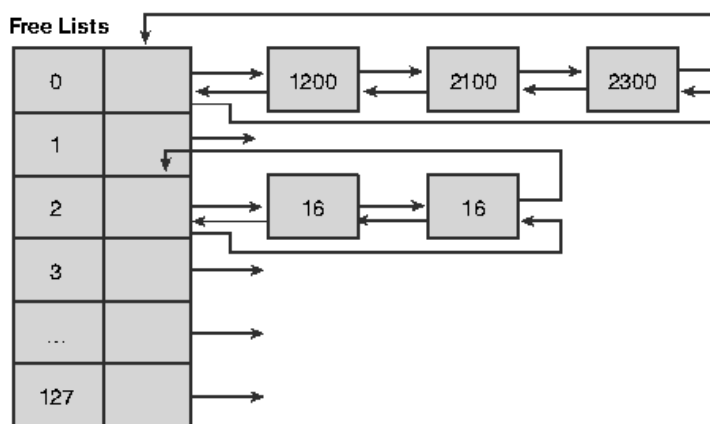


Figure 6.4 Hypothetical state of the free lists

Рисунок 6.4. Гипотетическое состояние free lists

Если в `back end allocator` приходит запрос на выделение размером 8, менеджер кучи сначала обращается к `free lists`.

Чтобы максимально повысить эффективность поиска свободных блоков кучи, менеджер кучи хранит `bitmap free list`.

`Bitmap` состоит из 128 битов, где каждый бит представляет индекс в таблице `free list`.

Если бит установлен, `free list`, соответствующий индексу `bitmap free list`, содержит свободные блоки кучи.

И наоборот, если бит не установлен, `free list` по этому индексу пуст.

На рисунке 6.5 показан `bitmap free list` для `free lists` из рисунка 6.4.

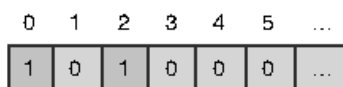


Figure 6.5 Free list bitmap

Рисунок 6.5. Bitmap free list

Менеджер кучи сопоставляет запрос на выделение заданного размера с индексом `bitmap free list`, добавляя к размеру 8 байтов (метаданные) и деля результат на 8.

Рассмотрим запрос на выделение размером 8 байтов.

Менеджер кучи знает, что индекс `bitmap free list` равен 2 $((8+8)/8)$.

На рисунке 6.5 видно, что индекс 2 bitmap free list установлен; это означает, что free list, расположенный по индексу 2 в таблице free lists, содержит свободные блоки кучи. Затем свободный блок удаляется из free list и возвращается вызывающему коду. Если удаление свободного блока кучи приводит к тому, что этот free list становится пустым, менеджер кучи также очищает bitmap free list по соответствующему индексу. Если менеджер кучи не может найти свободный блок кучи запрошенного размера, он применяет технику, известную как *splitting block*. Splitting block относится к способности менеджера кучи взять свободный блок большего размера, чем запрошено, и разделить его пополам, чтобы удовлетворить запрос на выделение меньшего размера. Например, если приходит запрос на выделение блока размером 8 (общий размер блока 16), сначала просматривается bitmap free list. Индекс, представляющий блоки размером 16, показывает, что свободные блоки недоступны. Затем менеджер кучи обнаруживает, что доступны свободные блоки размером 32. Теперь менеджер кучи удаляет блок размером 32 и делит его пополам, что дает два блока размером по 16 каждый. Один из блоков помещается во free list, представляющий блоки размером 16, а другой блок возвращается вызывающему коду. Кроме того, bitmap free list обновляется, чтобы показать, что индекс 2 теперь содержит записи свободных блоков размером 16. Результат разделения большого свободного выделения на два меньших выделения показан на рисунке 6.6.

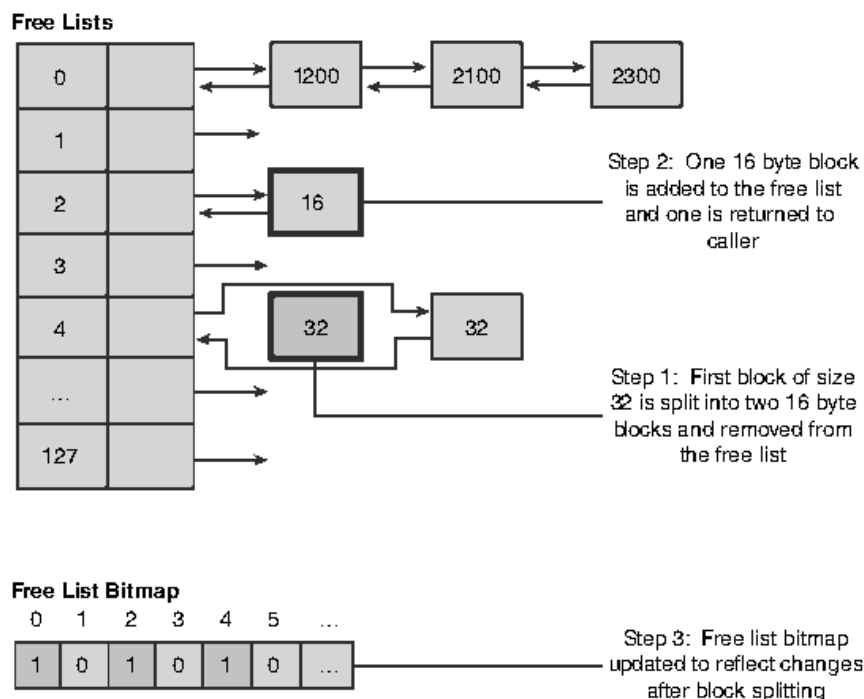


Figure 6.6 Splitting free blocks

Рисунок 6.6. Разделение свободных блоков

Как упоминалось ранее, free list по индексу 0 может содержать свободные блоки кучи размером от 1016 до 0x7FFF0 (524272) байтов.

Чтобы максимально повысить эффективность поиска свободных блоков, менеджер кучи хранит свободные блоки в отсортированном порядке (по возрастанию).

Все выделения размером больше 0x7FFF0 попадают в так называемый virtual allocation list.

Когда происходит большое выделение, менеджер кучи явно запрашивает выделение у менеджера виртуальной памяти и хранит такие выделения в virtual allocation list.

До сих пор обсуждение вращалось вокруг того, как менеджер кучи организует блоки памяти, находящиеся в его распоряжении.

Один вопрос остается без ответа: откуда менеджер кучи берет память?

В основе своей менеджер кучи использует менеджер виртуальной памяти Windows, чтобы выделять память крупными фрагментами.

Затем память преобразуется в блоки разных размеров, чтобы обслуживать запросы приложения на выделение.

Когда фрагменты виртуальной памяти исчерпываются, менеджер кучи выделяет еще один крупный фрагмент виртуальной памяти, и процесс продолжается.

Фрагменты, которые менеджер кучи запрашивает у менеджера виртуальной памяти, называются сегментами кучи.

Когда сегмент кучи создается впервые, лежащая в его основе виртуальная память в основном резервируется, и только небольшая часть фиксируется (committed).

Всякий раз, когда у менеджера кучи заканчивается зафиксированное пространство в сегменте кучи, он явно фиксирует дополнительную память и делит вновь зафиксированное пространство на блоки по мере поступления новых запросов на выделение.

На рисунке 6.7 показана базовая компоновка сегмента кучи.

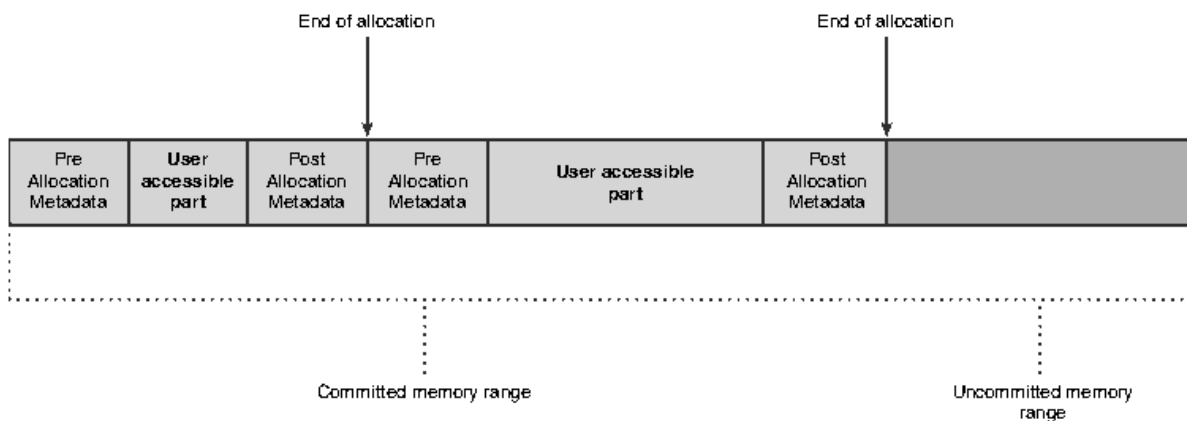


Figure 6.7 Basic layout of a heap segment

Рисунок 6.7. Базовая компоновка сегмента кучи

Сегмент, показанный на рисунке 6.7, содержит два выделения (и связанные с ними метаданные), за которыми следует диапазон незафиксированной памяти.

Если поступает еще один запрос на выделение и в free lists нет доступного свободного блока, менеджер кучи фиксирует дополнительную память из незафиксированного диапазона, создает новый блок кучи в пределах диапазона зафиксированной памяти и возвращает блок пользователю. Когда в сегменте заканчивается незафиксированное пространство, менеджер кучи создает новый сегмент.

Размер нового сегмента определяется удвоением размера предыдущего сегмента.

Если памяти мало и она не может вместить новый сегмент, менеджер кучи пытается уменьшить размер вдвое.

Если это не помогает, размер снова делится пополам до тех пор, пока операция либо не завершится успешно, либо не достигнет минимального порога размера сегмента; в этом случае вызывающему коду возвращается ошибка.

Максимальное количество сегментов, которые могут быть активны внутри кучи, равно 64.

Когда новый сегмент создан, менеджер кучи добавляет его в список, отслеживающий все сегменты, используемые в куче.

Освобождает ли менеджер кучи когда-либо память, связанную с сегментом?

Ответ таков: менеджер кучи дефксирует память по мере необходимости, но никогда не освобождает ее полностью.

То есть память остается зарезервированной.

Как показывает рисунок 6.7, каждый блок кучи в заданном сегменте имеет связанные с ним метаданные.

Метаданные используются менеджером кучи для эффективного управления блоками кучи внутри сегмента.

Содержимое метаданных зависит от состояния блока кучи.

Например, если блок кучи используется приложением, состояние блока считается busy.

И наоборот, если блок кучи не используется (то есть был освобожден приложением), состояние блока считается free.

На рисунке 6.8 показано, как метаданные структурированы в обеих ситуациях.



Figure 6.8 Structure of pre- and post-allocation metadata

Рисунок 6.8. Структура метаданных до и после выделения

Важно отметить, что блок кучи может считаться busy с точки зрения back end allocator, но при этом уже не использоваться приложением.

Причина в том, что любые блоки кучи, которые попадают в look aside list front end allocator, по-прежнему имеют состояние busy.

Два поля размера представляют размер текущего блока и размер предыдущего блока (включая метаданные).

Имея указатель на блок кучи, можно очень легко использовать два поля размера, чтобы проходить сегмент кучи вперед и назад.

Кроме того, для свободных блоков наличие размера блока в составе метаданных позволяет менеджеру кучи очень быстро индексировать правильный free list, в который нужно добавить блок. Метаданные после выделения необязательны и обычно используются отладочной кучей для дополнительной служебной информации (см. боковую заметку отладчика "Attaching Versus Running").

Поле flags указывает состояние блока кучи.

Наиболее важные значения поля flags показаны в таблице 6.1.

Таблица 6.1. Возможное состояние блока, задаваемое флагом кучи

Значение	Описание
0x01	Указывает, что выделение используется приложением или менеджером кучи.
0x04	Указывает, связан ли с блоком кучи шаблон заполнения.
0x08	Указывает, что блок кучи был выделен напрямую из менеджера виртуальной памяти.
0x10	Указывает, что это последний блок кучи перед незафиксированным диапазоном.

Вы уже видели, что происходит, когда блок кучи переходит из состояния busy в состояние free. Однако нужно обсудить еще один прием, который использует менеджер кучи.

Этот прием называется heap coalescing.

По сути, heap coalescing - это механизм, объединяющий соседние свободные блоки в один большой блок, чтобы избежать проблем фрагментации памяти.

На рисунке 6.9 показано, как работает объединение блоков кучи.

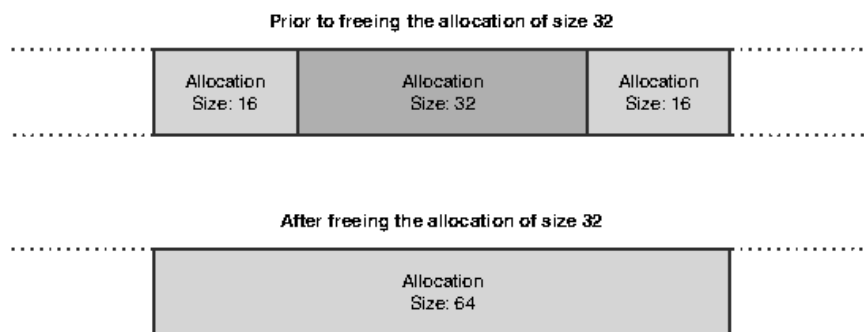


Figure 6.9 Example of heap coalescing

Рисунок 6.9. Пример объединения блоков кучи

Когда менеджеру кучи поступает запрос освободить блок кучи размером 32, он сначала проверяет, являются ли соседние блоки также свободными.

На рисунке 6.9 блок, который освобождается, окружен двумя блоками размером 16.

Вместо того чтобы передавать блок размером 32 во free lists, менеджер кучи объединяет все три блока в один (размером 64) и обновляет free lists, чтобы указать, что теперь доступен новый блок размером 64.

Менеджер кучи также заботится об удалении прежних двух блоков (размером 16) из free lists, поскольку они больше недоступны.

Само собой разумеется, что операция объединения свободных блоков дорогостоящая.

Так почему же менеджер кучи вообще этим занимается?

Основная причина объединения блоков кучи - избежать так называемой near fragmentation.

Представьте, что ваше приложение только что выполнило всплеск выделений, все очень малого размера (16 байтов).

Кроме того, допустим, что этих малых выделений было достаточно, чтобы заполнить целый сегмент.

После завершения всплеска выделений приложение освобождает все выделения.

В итоге у вас есть один сегмент кучи, полный доступных выделений размером 16 байтов.

Затем приложение пытается выделить блок памяти размером 48 байтов.

Менеджер кучи теперь пытается удовлетворить запрос на выделение из сегмента, но терпит неудачу, поскольку размеры свободных блоков слишком малы, и вынужден создать новый сегмент кучи.

Разумеется, это крайне неэффективное использование памяти.

Даже если у нас был целый сегмент свободной памяти, менеджер кучи был вынужден создать новый сегмент, чтобы удовлетворить наш немного более крупный запрос на выделение.

Heap coalescing делает все возможное, чтобы свести такие ситуации к минимуму, объединяя небольшие свободные блоки в более крупные блоки.

На этом мы завершаем обсуждение внутреннего устройства менеджера кучи.

Прежде чем перейти к практическому рассмотрению кучи, кратко подытожим изученное.

При выделении блока памяти:

1. Менеджер кучи сначала обращается к LAL front end allocator, чтобы проверить, доступен ли свободный блок памяти; если он доступен, менеджер кучи возвращает его вызывающему коду. В противном случае необходим шаг 2.
2. Проверяются free lists back end allocator:
 - Если найдено точное совпадение по размеру, флаги обновляются, чтобы указать, что блок занят; затем блок удаляется из free list и возвращается вызывающему коду.
 - Если точное совпадение по размеру найти не удастся, менеджер кучи проверяет, можно ли разделить более крупный блок на два меньших блока, удовлетворяющих запрошенному размеру выделения. Если можно, блок разделяется. У одного блока флаги обновляются до состояния busy, и он возвращается вызывающему коду. У другого блока флаги устанавливаются в состояние free, и он добавляется во free lists. Исходный блок также удаляется из free list.
3. Если free lists не могут удовлетворить запрос на выделение, менеджер кучи фиксирует дополнительную память из сегмента кучи, создает новый блок в зафиксированном диапазоне (флаги установлены в состояние busy) и возвращает блок вызывающему коду.

При освобождении блока памяти:

1. Сначала проверяется front end allocator, чтобы понять, может ли он обработать свободный блок. Если свободный блок не обрабатывается front end allocator, необходим шаг 2.
2. Менеджер кучи проверяет, есть ли соседние свободные блоки; если есть, он объединяет блоки в один большой блок следующим образом:
 - Два соседних свободных блока удаляются из free lists.
 - Новый большой блок добавляется во free list или look aside list.
 - Поле flags для нового большого блока обновляется, чтобы указать, что он свободен.
3. Если объединение выполнить нельзя, блок перемещается во free list или look aside list, а флаги обновляются до состояния free.

Теперь пришло время дополнить наше теоретическое обсуждение менеджера кучи практикой.

В листинге 6.1 показано простое приложение, которое с помощью кучи процесса по умолчанию выделяет и освобождает некоторую память.

Листинг 6.1. Простое выделение и освобождение памяти в куче процесса по умолчанию

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    BYTE* pAlloc1=NULL;
    BYTE* pAlloc2=NULL;
    HANDLE hProcessHeap=GetProcessHeap();
    pAlloc1=(BYTE*)HeapAlloc(hProcessHeap, 0, 16);
    pAlloc2=(BYTE*)HeapAlloc(hProcessHeap, 0, 1500);
    //
    // Use allocated memory
```

```
//
HeapFree(hProcessHeap, 0, pAlloc1);
HeapFree(hProcessHeap, 0, pAlloc2);
}
```

Исходный код и двоичный файл для листинга 6.1 находятся в следующих папках:

```
Source code: C:\AWD\Chapter6\BasicAlloc
Binary: C:\AWDBIN\WinXP.x86.chk\06BasicAlloc.exe
```

Запустите это приложение под отладчиком и остановитесь на функции `wmain`.

Поскольку нас интересует более подробное состояние кучи, сначала нужно выяснить, какие кучи активны в процессе.

Каждый выполняющийся процесс хранит список активных куч.

Список куч хранится в РЕВ (process environment block), который представляет собой структуру данных, содержащую огромное количество информации о процессе.

Чтобы вывести содержимое РЕВ, мы используем команду `dt`, как показано в листинге 6.2.

Листинг 6.2. Вывод содержимого РЕВ

```
0:000> dt _PEB @$peb
+0x000 InheritedAddressSpace : 0 ''
+0x001 ReadImageFileExecOptions : 0 ''
+0x002 BeingDebugged : 0x1 ''
+0x003 SpareBool : 0 ''
+0x004 Mutant : 0xffffffff
+0x008 ImageBaseAddress : 0x01000000
+0x00c Ldr : 0x00191e90 _PEB_LDR_DATA
+0x010 ProcessParameters : 0x00020000 _RTL_USER_PROCESS_PARAMETERS
+0x014 SubSystemData : (null)
+0x018 ProcessHeap : 0x00080000
+0x01c FastPebLock : 0x7c97e4c0 _RTL_CRITICAL_SECTION
+0x020 FastPebLockRoutine : 0x7c901005
+0x024 FastPebUnlockRoutine : 0x7c9010ed
+0x028 EnvironmentUpdateCount : 1
+0x02c KernelCallbackTable : (null)
+0x030 SystemReserved : [1] 0
+0x034 AtlThunkSListPtr32 : 0
+0x038 FreeList : (null)
+0x03c TlsExpansionCounter : 0
+0x040 TlsBitmap : 0x7c97e480
+0x044 TlsBitmapBits : [2] 1
+0x04c ReadOnlySharedMemoryBase : 0x7f6f0000
+0x050 ReadOnlySharedMemoryHeap : 0x7f6f0000
+0x054 ReadOnlyStaticServerData : 0x7f6f0688 -> (null)
+0x058 AnsiCodePageData : 0x7ffb0000
+0x05c OemCodePageData : 0x7ffc1000
+0x060 UnicodeCaseTableData : 0x7ffd2000
+0x064 NumberOfProcessors : 1
+0x068 NtGlobalFlag : 0
+0x070 CriticalSectionTimeout : _LARGE_INTEGER 0xffffffff`dc3cba00
+0x078 HeapSegmentReserve : 0x100000
+0x07c HeapSegmentCommit : 0x2000
+0x080 HeapDeCommitTotalFreeThreshold : 0x10000
+0x084 HeapDeCommitFreeBlockThreshold : 0x1000
+0x088 NumberOfHeaps : 3
+0x08c MaximumNumberOfHeaps : 0x10
+0x090 ProcessHeaps : 0x7c97de80 -> 0x00080000
+0x094 GdiSharedHandleTable : (null)
+0x098 ProcessStarterHelper : (null)
+0x09c GdiDCAttributeList : 0
+0x0a0 LoaderLock : 0x7c97c0d8
+0x0a4 OSMajorVersion : 5
+0x0a8 OSMinorVersion : 1
+0x0ac OSBuildNumber : 0xa28
+0x0ae OSCSDVersion : 0x200
+0x0b0 OSPlatformId : 2
+0x0b4 ImageSubsystem : 3
```

```
+0x0b8 ImageSubsystemMajorVersion : 4
+0x0bc ImageSubsystemMinorVersion : 0
```

Листинг 6.2 (продолжение)

```
+0x0c0 ImageProcessAffinityMask : 0
+0x0c4 GdiHandleBuffer : [34] 0
+0x14c PostProcessInitRoutine : (null)
+0x150 TlsExpansionBitmap : 0x7c97e478
+0x154 TlsExpansionBitmapBits : [32] 0
+0x1d4 SessionId : 0
+0x1d8 AppCompatFlags : _ULARGE_INTEGER 0x0
+0x1e0 AppCompatFlagsUser : _ULARGE_INTEGER 0x0
+0x1e8 pShimData : (null)
+0x1ec AppCompatInfo : (null)
+0x1f0 CSDVersion : _UNICODE_STRING "Service Pack 2"
+0x1f8 ActivationContextData : (null)
+0x1fc ProcessAssemblyStorageMap : (null)
+0x200 SystemDefaultActivationContextData : 0x00080000
+0x204 SystemAssemblyStorageMap : (null)
+0x208 MinimumStackCommit : 0
```

Как видите, РЕВ содержит довольно много информации, и, разбираясь в этой структуре данных, можно многое узнать о разных компонентах процесса.

В этом конкретном упражнении нас особенно интересует список куч процесса, расположенный по смещению 0x90.

Член РЕВ со списком куч - это просто массив указателей, где каждый указатель указывает на структуру данных типа `_HEAP`.

Давайте выведем массив указателей на кучи и посмотрим, что он содержит:

```
0:000> dd 0x7c97de80
7c97de80 00080000 00180000 00190000 00000000
7c97de90 00000000 00000000 00000000 00000000
7c97dea0 00000000 00000000 00000000 00000000
7c97deb0 00000000 00000000 00000000 00000000
7c97dec0 01a801a6 00020498 00000001 7c9b0000
7c97ded0 7fffd2de 00000000 00000005 00000001
7c97dee0 ffff7e77 00000000 003a0044 0057005c
7c97def0 004e0049 004f0044 00530057 0073005c
```

Дамп показывает, что в нашем процессе активны три кучи, а указатель на кучу процесса по умолчанию всегда является первым в списке.

Почему в нашем процессе больше одной кучи?

Даже самые простые приложения обычно содержат больше одной кучи.

Большинство приложений неявно используют компоненты, которые создают собственные кучи.

Отличный пример - C runtime, который создает собственную кучу во время инициализации.

Поскольку наше приложение работает с кучей процесса по умолчанию, мы сосредоточим исследование именно на этой куче.

Каждый из указателей на кучи процесса указывает на структуру данных типа `_HEAP`.

С помощью команды `dt` мы можем очень легко вывести информацию о куче процесса, как показано в листинге 6.3.

Листинг 6.3. Вывод структуры `_HEAP` для кучи процесса по умолчанию

```
0:000> dt _HEAP 00080000
+0x000 Entry : _HEAP_ENTRY
+0x008 Signature : 0xeeffeeff
+0x00c Flags : 0x50000062
+0x010 ForceFlags : 0x40000060
+0x014 VirtualMemoryThreshold : 0xfe00
+0x018 SegmentReserve : 0x100000
+0x01c SegmentCommit : 0x2000
+0x020 DeCommitFreeBlockThreshold : 0x200
+0x024 DeCommitTotalFreeThreshold : 0x2000
+0x028 TotalFreeSize : 0xcb
```

```

+0x02c MaximumAllocationSize : 0x7ffdefff
+0x030 ProcessHeapsListIndex : 1
+0x032 HeaderValidateLength : 0x608
+0x034 HeaderValidateCopy : (null)
+0x038 NextAvailableTagIndex : 0
+0x03a MaximumTagIndex : 0
+0x03c TagEntries : (null)
+0x040 UCRSegments : (null)
+0x044 UnusedUnCommittedRanges : 0x00080598 _HEAP_UNCOMMITTED_RANGE
+0x048 AlignRound : 0x17
+0x04c AlignMask : 0xffffffff
+0x050 VirtualAllocdBLOCKS : _LIST_ENTRY [ 0x80050 - 0x80050 ]
+0x058 Segments : [64] 0x00080640 _HEAP_SEGMENT
+0x158 u : __unnamed
+0x168 u2 : __unnamed
+0x16a AllocatorBackTraceIndex : 0
+0x16c NonDedicatedListLength : 1
+0x170 LargeBlocksIndex : (null)
+0x174 PseudoTagEntries : (null)
+0x178 FreeLists : [128] _LIST_ENTRY [ 0x829b0 - 0x829b0 ]
+0x578 LockVariable : 0x00080608 _HEAP_LOCK
+0x57c CommitRoutine : (null)
+0x580 FrontEndHeap : 0x00080688
+0x584 FrontHeapLockCount : 0
+0x586 FrontEndHeapType : 0x1 ''
+0x587 LastSegmentIndex : 0 ''

```

И снова видно, что структура `_HEAP` довольно велика и содержит множество сведений о куче. Для этого упражнения наиболее важные члены структуры `_HEAP` расположены по следующим смещениям:

```
+0x050 VirtualAllocdBLOCKS : _LIST_ENTRY
```

Выделения, которые больше порога размера виртуального выделения, не управляются как часть сегментов и free lists.

Вместо этого такие выделения выполняются напрямую из менеджера виртуальной памяти.

Вы отслеживаете эти выделения, храня в структуре `_HEAP` список, содержащий все виртуальные выделения.

```
+0x058 Segments : [64]
```

Поле `Segments` - это массив структур данных типа `_HEAP_SEGMENT`.

Каждый сегмент кучи содержит список записей кучи, активных внутри этого сегмента.

Позже вы увидите, как мы можем использовать эту информацию, чтобы пройти по всему сегменту кучи и найти интересные выделения.

```
+0x16c NonDedicatedListLength
```

Как упоминалось ранее, `free_list[0]` содержит выделения размером больше 1016 КБ и меньше порога виртуального выделения.

Чтобы эффективно управлять этим free list, куча хранит в этом поле число выделений в `nondedicated list`.

Эта информация может быть полезна, когда вы хотите проанализировать использование кучи и быстро увидеть, сколько ваших выделений попадает в категорию `free_list[0]` переменного размера.

```
+0x178 FreeLists : [128] _LIST_ENTRY
```

Free lists хранятся по смещению `0x178` и содержат двусвязные списки.

Каждый список содержит свободные блоки кучи определенного размера.

Чуть позже мы рассмотрим free lists подробнее.

```
+0x580 FrontEndHeap
```

Указатель, расположенный по смещению 0x580, указывает на front end allocator.

Мы знаем общую архитектуру и стратегию front end allocator, но, к сожалению, публичный пакет символов не содержит его определений, что делает глубокое исследование невозможным.

Также стоит отметить, что Microsoft оставляет за собой право изменять ранее описанные смещения между версиями Windows.

Вернемся к нашему примеру приложения: продолжим пошаговое выполнение кода в отладчике.

Первый интересующий вызов - API GetProcessHeap, который возвращает дескриптор кучи процесса по умолчанию.

Поскольку мы уже самостоятельно нашли этот дескриптор/указатель, можно проверить, что явный вызов GetProcessHeap возвращает ожидаемое значение.

После вызова регистр eax содержит 0x00080000, что соответствует нашим ожиданиям.

Далее идут два вызова API kernel32!HeapAlloc, которые пытаются выполнить выделения размером 16 и 1500.

Будут ли эти выделения удовлетворены за счет фиксации дополнительной памяти сегмента или из free lists?

Перед тем как перейти через первый вызов HeapAlloc, попробуем выяснить, где менеджер кучи найдет свободный блок кучи для удовлетворения этого выделения.

Первый шаг нашего исследования - проверить, доступны ли во free lists свободные блоки размером 16.

Чтобы проверить наличие свободных блоков, мы используем следующую команду:

```
dt _LIST_ENTRY 0x00080000+0x178+8
```

Эта команда выводит первый узел в free list, соответствующем выделениям размером 16.

0x00080000 - это адрес нашей кучи.

Мы добавляем смещение 0x178, чтобы получить начало таблицы free list.

Первая запись в таблице free list указывает на free list[0].

Поскольку наше выделение гораздо меньше порога размера free list[0], мы просто пропускаем этот free list, добавляя еще 8 байтов (размер структуры _LIST_ENTRY), что помещает нас в free list[1], представляющий свободные блоки размером 16.

```
0:000> dt _LIST_ENTRY 0x00080000+0x178+8
[ 0x80180 - 0x80180 ]
+0x000 Flink : 0x00080180 _LIST_ENTRY [ 0x80180 - 0x80180 ]
+0x004 Blink : 0x00080180 _LIST_ENTRY [ 0x80180 - 0x80180 ]
```

Помните, что free lists являются двусвязными списками; следовательно, поля Flink и Blink структуры _LIST_ENTRY - это просто указатели на следующее и предыдущее выделения.

Критически важно отметить, что указатель, указанный во free lists, на самом деле указывает на доступную пользователю часть блока кучи, а не на начало самого блока кучи.

Поэтому, если вы хотите посмотреть метаданные выделения, сначала нужно вычесть из указателя 8 байтов.

Оба этих указателя, похоже, указывают на 0x00080180, что фактически является адресом узла списка, который мы только что выводили (0x00080000+0x178+8=0x00080180).

Это означает, что free list, соответствующий выделениям размером 16, пуст.

Прежде чем предположить, что менеджер кучи должен зафиксировать дополнительную память в сегменте, вспомните, что он сделает это только в самом крайнем случае.

Поэтому менеджер кучи сначала пытается проверить, есть ли свободные блоки других размеров больше 16, которые он мог бы разделить для удовлетворения выделения.

В нашем конкретном случае free list[0] содержит свободный блок кучи:

```
0:000> dt _LIST_ENTRY 0x00080000+0x178
[ 0x82ab0 - 0x82ab0 ]
+0x000 Flink : 0x00082ab0 _LIST_ENTRY [ 0x80178 - 0x80178 ]
+0x004 Blink : 0x00082ab0 _LIST_ENTRY [ 0x80178 - 0x80178 ]
```

Член Flink указывает на расположение в блоке кучи, доступное вызывающему коду. Чтобы увидеть полный блок кучи (включая метаданные), сначала нужно вычесть 8 байтов из указателя (см. рисунок 6.8).

```
0:000> dt _HEAP_ENTRY 0x00082ab0-0x8
+0x000 Size : 0xab
+0x002 PreviousSize : 0xb
+0x000 SubSegmentCode : 0x000b00ab
+0x004 SmallTagIndex : 0xee ''
+0x005 Flags : 0x14 ''
+0x006 UnusedBytes : 0xee ''
+0x007 SegmentIndex : 0 ''
```

Важно отметить, что сообщаемый размер - это истинный размер блока кучи, деленный на гранулярность кучи.

Гранулярность кучи легко найти, взяв размер структуры _HEAP_ENTRY.

Блок кучи, размер которого сообщается как 0xab, на самом деле имеет размер 0xab*8 = 0x558 (1368) байтов.

Свободный блок кучи, на который мы смотрим, определенно выглядит достаточно большим, чтобы вместить наш запрос на выделение размером 16.

В сеансе отладки перейдите через первую инструкцию, вызывающую HeapAlloc.

Если все успешно, затем мы можем снова проверить free_list[0] и посмотреть, изменилось ли выделение, которое мы просматривали до вызова:

```
0:000> dt _LIST_ENTRY 0x00080000+0x178
[ 0x82ad8 - 0x82ad8 ]
+0x000 Flink : 0x00082ad8 _LIST_ENTRY [ 0x80178 - 0x80178 ]
+0x004 Blink : 0x00082ad8 _LIST_ENTRY [ 0x80178 - 0x80178 ]
0:000> dt _HEAP_ENTRY 0x00082ad8-0x8
+0x000 Size : 0xa6
+0x002 PreviousSize : 5
+0x000 SubSegmentCode : 0x000500a6
+0x004 SmallTagIndex : 0xee ''
+0x005 Flags : 0x14 ''
+0x006 UnusedBytes : 0xee ''
+0x007 SegmentIndex : 0 ''
```

И действительно, то, что раньше было первой записью в free_list[0], теперь изменилось.

Вместо свободного блока размером 0xab у нас теперь свободный блок размером 0xa6.

Разница в размере (0x5) объясняется тем, что наш запрос на выделение разбил более крупный свободный блок, который мы видели ранее.

Если мы выделяем 16 байтов (0x10), почему разница в размере свободного блока до и после разделения составляет только 0x5 байтов?

Ключ в том, что сообщаемый размер сначала нужно умножить на коэффициент гранулярности кучи 0x8.

Истинный размер нового свободного выделения затем равен 0x0000530 (0xa6*8), а истинная разница размеров составляет 0x28.

Из этих 0x28 байтов 0x10 - это размер нашего выделения, а оставшиеся 0x18 байтов - все метаданные, связанные с нашим блоком кучи.

Следующий вызов HeapAlloc пытается выделить память размером 1500.

Мы знаем, что свободные блоки кучи такого размера должны находиться в free_list[0].

Однако из предыдущего исследования мы также знаем, что единственный свободный блок кучи в free_list[0] слишком мал, чтобы вместить запрошенный размер.

Оказавшись в безвыходном положении, менеджер кучи теперь вынужден зафиксировать дополнительную память в сегменте кучи.

Чтобы получить более ясную картину состояния нашего сегмента кучи, полезно выполнить ручной обход сегмента.

Структура _HEAP содержит массив указателей на все сегменты, которые сейчас активны в куче.

Массив расположен по базовому адресу _HEAP плюс смещение 0x58.

```
0:000> dd 0x00080000+0x58 14
00080058 00080640 00000000 00000000 00000000
0:000> dt _HEAP_SEGMENT 0x00080640
+0x000 Entry : _HEAP_ENTRY
+0x008 Signature : 0xffeeffee
+0x00c Flags : 0
+0x010 Heap : 0x00080000 _HEAP
+0x014 LargestUnCommittedRange : 0xfd000
+0x018 BaseAddress : 0x00080000
+0x01c NumberOfPages : 0x100
+0x020 FirstEntry : 0x00080680 _HEAP_ENTRY
+0x024 LastValidEntry : 0x00180000 _HEAP_ENTRY
+0x028 NumberOfUnCommittedPages : 0xfd
+0x02c NumberOfUnCommittedRanges : 1
+0x030 UnCommittedRanges : 0x00080588 _HEAP_UNCOMMITTED_RANGE
+0x034 AllocatorBackTraceIndex : 0
+0x036 Reserved : 0
+0x038 LastEntryInSegment : 0x00082ad0 _HEAP_ENTRY
```

Структура данных _HEAP_SEGMENT содержит множество информации, которую менеджер кучи использует для эффективного управления всеми активными сегментами в куче.

При обходе сегмента наиболее полезная часть информации - поле FirstEntry, расположенное по базовому адресу сегмента плюс смещение 0x20.

Это поле представляет первый блок кучи в сегменте.

Если вывести этот блок и получить его размер, следующий блок кучи можно вывести, прибавив размер к адресу первого блока кучи.

Если продолжать этот процесс, можно пройти весь сегмент и проверить корректность каждого выделения.

```
0:000> dt _HEAP_ENTRY 0x00080680
+0x000 Size : 0x303
+0x002 PreviousSize : 8
+0x000 SubSegmentCode : 0x00080303
+0x004 SmallTagIndex : 0x9a ''
+0x005 Flags : 0x7 ''
+0x006 UnusedBytes : 0x18 ''
+0x007 SegmentIndex : 0 ''
0:000> dt _HEAP_ENTRY 0x00080680+(0x303*8)
+0x000 Size : 8
+0x002 PreviousSize : 0x303
+0x000 SubSegmentCode : 0x03030008
+0x004 SmallTagIndex : 0x99 ''
+0x005 Flags : 0x7 ''
+0x006 UnusedBytes : 0x1e ''
+0x007 SegmentIndex : 0 ''
0:000> dt _HEAP_ENTRY 0x00080680+(0x303*8)+(8*8)
+0x000 Size : 5
+0x002 PreviousSize : 8
+0x000 SubSegmentCode : 0x00080005
+0x004 SmallTagIndex : 0x91 ''
+0x005 Flags : 0x7 ''
+0x006 UnusedBytes : 0x1a ''
+0x007 SegmentIndex : 0 ''
...
...
...
+0x000 Size : 0xa6
+0x002 PreviousSize : 5
+0x000 SubSegmentCode : 0x000500a6
+0x004 SmallTagIndex : 0xee ''
+0x005 Flags : 0x14 ''
+0x006 UnusedBytes : 0xee ''
+0x007 SegmentIndex : 0 ''
```

Посмотрим, что менеджер кучи сделает с сегментом (если вообще сделает что-нибудь), попытаюсь удовлетворить запрос на выделение размером 1500 байтов.

Перейдите через вызов `HeapAlloc` и снова пройдите сегмент.
Интересующий блок кучи показан ниже.

```
+0x000 Size : 0xbf
+0x002 PreviousSize : 5
+0x000 SubSegmentCode : 0x000500bf
+0x004 SmallTagIndex : 0x10 ''
+0x005 Flags : 0x7 ''
+0x006 UnusedBytes : 0x1c ''
+0x007 SegmentIndex : 0 ''
```

До того как мы перешли через вызов `HeapAlloc`, последний блок кучи был помечен как свободный и имел размер `0ха6`.

После вызова состояние блока изменилось на `busy` с размером `0xbf` ($0xbf * 8 = 0x5f8$), что указывает: теперь этот блок используется для хранения нашего нового выделения.

Поскольку наше выделение было слишком большим, чтобы поместиться в прежний размер `0ха6`, менеджер кучи зафиксировал больше памяти в сегменте.

Зафиксировал ли он ровно столько, сколько нужно для нашего выделения?

На самом деле он зафиксировал гораздо больше и поместил оставшуюся свободную память в новый блок по адресу `0x000830с8`.

Менеджер кучи способен запрашивать у менеджера виртуальной памяти только выделения размером в страницу (4 КБ на системах x86) и возвращает остаток такого выделения во `free lists`.

Следующие несколько строк нашего приложения просто освобождают выделения, которые мы только что сделали.

Что, как мы ожидаем, сделает менеджер кучи при выполнении первого вызова `HeapFree`?

Помимо обновления состояния блока кучи на `free` и добавления его во `free lists`, мы ожидаем, что он попытается объединить блок кучи с другими окружающими свободными блоками.

Перед тем как перейти через первый вызов `HeapFree`, посмотрим на блок кучи, связанный с этим вызовом.

```
0:000> dt _HEAP_ENTRY 0x000830с8-(0xbf*8)-(0x5*8)
+0x000 Size : 5
+0x002 PreviousSize : 0xb
+0x000 SubSegmentCode : 0x000b0005
+0x004 SmallTagIndex : 0x1f ''
+0x005 Flags : 0x7 ''
+0x006 UnusedBytes : 0x18 ''
+0x007 SegmentIndex : 0 ''
0:000> dt _HEAP_ENTRY 0x000830с8-(0xbf*8)-(0x5*8)-(0xb*8)
+0x000 Size : 0xb
+0x002 PreviousSize : 5
+0x000 SubSegmentCode : 0x0005000b
+0x004 SmallTagIndex : 0 ''
+0x005 Flags : 0x7 ''
+0x006 UnusedBytes : 0x1c ''
+0x007 SegmentIndex : 0 ''
0:000> dt _HEAP_ENTRY 0x000830с8-(0xbf*8)
+0x000 Size : 0xbf
+0x002 PreviousSize : 5
+0x000 SubSegmentCode : 0x000500bf
+0x004 SmallTagIndex : 0x10 ''
+0x005 Flags : 0x7 ''
+0x006 UnusedBytes : 0x1c ''
+0x007 SegmentIndex : 0 ''
```

Состояние предыдущего и следующего блоков кучи - `busy` (`Flags=0x7`), а это означает, что менеджер кучи не может объединить память, и блок кучи просто помещается во `free lists`.

Точнее, блок кучи попадет в `free list[1]`, потому что его размер равен 16 байтам.

Проверим нашу теорию: перейдите через вызов `HeapFree` и используйте тот же механизм, что и раньше, чтобы увидеть, что произошло с блоком кучи.

```
0:000> dt _HEAP_ENTRY 0x000830c8-(0xbf*8)-(0x5*8)
+0x000 Size : 5
+0x002 PreviousSize : 0xb
+0x000 SubSegmentCode : 0x000b0005
+0x004 SmallTagIndex : 0x1f ''
+0x005 Flags : 0x4 ''
+0x006 UnusedBytes : 0x18 ''
+0x007 SegmentIndex : 0 ''
```

Как видите, состояние блока кучи действительно установлено в free, а размер остается прежним. Поскольку размер остается прежним, это служит индикатором того, что менеджер кучи не объединил блок кучи с соседними блоками. Наконец, мы проверяем, что блок попал в free list[1].

Я оставляю читателю в качестве упражнения выяснить, что происходит с сегментом и блоками кучи во время следующего вызова HeapFree.

Подсказка: помните, что размер освобождаемого блока кучи равен 1500 байтам и что состояние одного из соседних блоков установлено в free.

На этом мы завершаем обзор внутреннего устройства менеджера кучи.

Хотя понимание разных структур кучи и умение проходить их может показаться пугающей задачей, после небольшой практики все становится проще.

Прежде чем перейти к сценариям повреждения кучи, одна важная команда отладчика может помочь нам эффективнее отлаживать такие сценарии.

Эта команда расширения называется !heap и является частью расширения отладчика exts.dll. С помощью этой команды можно очень легко отобразить всю информацию о куче, которую только можно захотеть увидеть.

На самом деле вся информация, которую мы только что собирали вручную, выводится командой расширения !heap за долю секунды.

Но подождите: мы только что потратили много времени, выясняя, как анализировать кучу вручную, проходить сегменты и проверять блоки кучи.

Зачем вообще этим заниматься, если у нас есть прекрасная команда, которая делает всю работу за нас?

Как всегда, ответ кроется в том, как отладчик приходит к информации, которую он показывает.

Если состояние кучи целостно, команда расширения !heap показывает состояние кучи в аккуратной и удобоваримой форме.

Однако если состояние кучи было повреждено, уже недостаточно полагаться на команду, чтобы она сказала нам, что именно повреждено и как это произошло.

Нужно знать, как анализировать разные части кучи, чтобы приходиться к обоснованным выводам и возможным виновникам.

Подключение к процессу и запуск процесса под отладчиком

Сеанс отладки, который вы видели до сих пор, предполагал запуск процесса под отладчиком от начала до конца.

Другой вариант при отладке процессов - подключить отладчик к уже выполняющемуся процессу.

Обычно использование любого из этих подходов не меняет радикально способ отладки процесса.

Исключение из этого правила возникает при отладке проблем, связанных с кучей.

При запуске процесса под отладчиком менеджер кучи изменяет все запросы на создание новых куч и меняет флаги создания кучи, чтобы включить удобные для отладки кучи (если переменная среды _NO_DEBUG_HEAP не установлена в 1).

Для сравнения, при подключении к уже выполняющемуся процессу кучи в процессе уже были созданы с флагами создания кучи по умолчанию и не будут иметь удобных для

отладки флагов (если только приложение не установило их явно).

Флаги модификации кучи применяются ко всем кучам в процессе, включая кучу процесса по умолчанию.

Самое большое отличие при запуске процесса под отладчиком состоит в том, что блоки кучи содержат дополнительное поле шаблона заполнения после доступной пользователю части (см. рисунок 6.8).

Шаблон заполнения используется менеджером кучи для проверки целостности блока кучи во время операций с кучей.

Когда выделение завершается успешно, менеджер кучи заполняет эту область блока определенным шаблоном заполнения.

Если приложение ошибочно записывает данные за конец доступной пользователю части, оно перезаписывает все это поле шаблона заполнения или его часть.

В следующий раз, когда приложение использует это выделение в каких-либо вызовах менеджера кучи, менеджер кучи внимательно проверяет поле шаблона заполнения, чтобы убедиться, что оно не изменилось.

Если поле шаблона заполнения было перезаписано приложением, менеджер кучи немедленно прерывает выполнение в отладчике, давая вам возможность посмотреть на блок кучи и попытаться вывести, почему он был перезаписан.

Запись в любую область блока кучи за пределами фактической доступной пользователю части - серьезная ошибка, которая может разрушительно сказаться на стабильности приложения.

Повреждения кучи

Повреждения кучи, пожалуй, относятся к самым трудным для выяснения проблемам.

Процесс может повредить любую заданную кучу почти бесконечным числом способов.

Вооружившись знанием того, как работает менеджер кучи, теперь рассмотрим некоторые из самых распространенных причин повреждений кучи.

Каждый сценарий сопровождается примером исходного кода, иллюстрирующим рассматриваемый тип повреждения кучи.

Затем приводится подробный сеанс отладки, который ведет вас от исходного сбоя к источнику повреждения кучи.

По пути мы также вводим бесценные инструменты, которые можно использовать, чтобы проще добраться до корневой причины повреждения.

Использование неинициализированного состояния

Неинициализированное состояние - распространенная программная ошибка, на отслеживание которой могут уйти многочисленные часы отладки.

В основе своей неинициализированное состояние означает блок памяти, который был успешно выделен, но еще не инициализирован до состояния, в котором он считается допустимым для использования.

Блок памяти может варьироваться от простых встроенных типов данных, таких как целые числа, до сложных двоичных объектов данных.

Использование неинициализированного блока памяти приводит к непредсказуемому поведению.

В листинге 6.4 показано небольшое приложение, которое страдает от использования неинициализированной памяти.

Листинг 6.4. Приложение, использующее неинициализированную память

```

#include <windows.h>
#include <stdio.h>
#include <conio.h>
#define ARRAY_SIZE 10
BOOL InitArray(int** pPtrArray);
int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    int iRes=1;
    wprintf(L"Press any key to start...");
    _getch();
    int** pPtrArray=(int**)HeapAlloc(GetProcessHeap(),
    0,
    sizeof(int*[ARRAY_SIZE]));
    if(pPtrArray!=NULL)
    {
        InitArray(pPtrArray);
        *(pPtrArray[0])=10;
        iRes=0;
        HeapFree(GetProcessHeap(), 0, pPtrArray);
    }
    return iRes;
}
BOOL InitArray(int** pPtrArray)
{
    return FALSE ;
}

```

Исходный код и двоичный файл для листинга 6.4 находятся в следующих папках:

```

Source code: C:\AWD\Chapter6\Uninit
Binary: C:\AWDBIN\WinXP.x86.chk\06Uninit.exe

```

Код в листинге 6.4 просто выделяет массив указателей на целые числа.

Затем он вызывает функцию InitArray, которая инициализирует все элементы массива допустимыми указателями на целые числа.

После вызова приложение пытается разыменовать первый указатель и установить значение в 10.

Может ли этот код отказать?

Безусловно.

Поскольку мы не проверяем возвращаемое значение вызова InitArray, функция может не инициализировать массив.

Затем при попытке разыменовать первый элемент мы можем ошибочно взять случайный адрес.

Приложение может столкнуться с нарушением доступа, если адрес недопустим (в том смысле, что это недоступная память), или операция может завершиться успешно.

Что произойдет дальше, во многом зависит от самого случайного указателя.

Если указатель указывает на допустимый адрес, используемый где-то еще, приложение продолжит выполнение.

Однако если указатель указывает на недоступную память, приложение может упасть немедленно.

Достаточно сказать, что даже если приложение не падает сразу, память используется неправильно, и в конечном итоге приложение завершится ошибкой.

При выполнении приложения мы легко видим, что сбой действительно происходит.

Чтобы лучше понять, что именно отказывает, запустите приложение под отладчиком, как показано в листинге 6.5.

Листинг 6.5. Сбой при использовании неинициализированного указателя

```

...
...
...
0:000> g
Press any key to start...(740.5b0): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=00000000 ebx=7ffdb000 ecx=00082ab0 edx=baadf00d esi=7c9118f1 edi=00011970

```

```

eip=010011c9 esp=0006ff3c ebp=0006ff44 iopl=0 nv up ei pl zr na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010246
06uninit!wmain+0x49:
010011c9 c7020a000000 mov dword ptr [edx],0Ah ds:0023:baadf00d=???????
0:000> kb
ChildEBP RetAddr Args to Child
0007ff7c 01001413 00000001 00034ed8 00037118 06uninit!wmain+0x4b
0007ffc0 7c816fd7 00011970 7c9118f1 7ffd4000 06uninit!__wmainCRTStartup+0x102
0007fff0 00000000 01001551 00000000 78746341 kernel32!BaseProcessStart+0x23

```

Инструкция, вызывающая сбой, соответствует строке кода в нашем приложении, которая устанавливает первый элемент массива в значение 10:

```
mov dword ptr [edx],0xAh ; *(pPtrArray[0])=10;
```

Следующий логический шаг - понять, почему произошло нарушение доступа.

Поскольку мы пытаемся записать в область памяти, соответствующую первому элементу нашего массива, нарушение доступа может быть вызвано тем, что память, в которую выполняется запись, недоступна.

Вывод содержимого соответствующей памяти дает следующее:

```

0:000> dd edx
baadf00d ???????? ???????? ???????? ????????
baadf01d ???????? ???????? ???????? ????????
baadf02d ???????? ???????? ???????? ????????
baadf03d ???????? ???????? ???????? ????????
baadf04d ???????? ???????? ???????? ????????
baadf05d ???????? ???????? ???????? ????????
baadf06d ???????? ???????? ???????? ????????
baadf07d ???????? ???????? ???????? ????????

```

Указатель, находящийся в регистре edx, имеет действительно странное значение (baadf00d), указывающее на недоступную память.

Попытка разыменовать этот указатель в конечном счете и вызвала нарушение доступа.

Откуда берется это интересное значение указателя (baadf00d)?

Конечно, значение указателя настолько явно неправильное, что оно не осталось там от какого-то предыдущего выделения.

Плохой указатель, который мы видим, был явно помещен туда менеджером кучи.

Всякий раз, когда вы запускаете процесс под отладчиком, менеджер кучи автоматически инициализирует всю память шаблоном заполнения.

Конкретный шаблон заполнения зависит от состояния блока кучи.

Когда блок кучи впервые возвращается вызывающему коду, менеджер кучи заполняет доступную пользователю часть блока кучи шаблоном заполнения, состоящим из значений baadf00d.

Это указывает, что блок кучи выделен, но еще не инициализирован.

Если приложение (такое как наше) разыменует этот блок памяти, не инициализировав его сначала, оно откажет.

С другой стороны, если приложение правильно инициализирует блок памяти, выполнение продолжится.

После освобождения блока кучи менеджер кучи снова инициализирует доступную пользователю часть блока кучи, на этот раз значениями feeefeee.

И снова шаблон заполнения освобожденной памяти добавляется менеджером кучи, чтобы ловить любые обращения к блоку памяти после его освобождения.

Причиной нашего конкретного сбоя является то, что память не была инициализирована перед использованием.

Посмотрим, чем отличается выделенная память, когда приложение не запускается под отладчиком, а отладчик подключается к процессу.

Запустите приложение и, когда появится приглашение Press any key to start, подключите отладчик.

После подключения установите точку останова на инструкции, вызвавшей сбой, и выведите содержимое регистра `edx`.

```
0:000> dd edx
00080178 000830f0 000830f0 00080180 00080180
00080188 00080188 00080188 00080190 00080190
00080198 00080198 00080198 000801a0 000801a0
000801a8 000801a8 000801a8 000801b0 000801b0
000801b8 000801b8 000801b8 000801c0 000801c0
000801c8 000801c8 000801c8 000801d0 000801d0
000801d8 000801d8 000801d8 000801e0 000801e0
000801e8 000801e8 000801e8 000801f0 000801f0
```

На этот раз видно, что регистр `edx` содержит значение указателя, указывающее на доступную, хотя и неправильную, память.

Массив больше не инициализирован значениями указателей, которые при разыменовании вызывают немедленное нарушение доступа (`baadf00d`).

Фактически переход через сбойную инструкцию на этот раз завершается успешно.

Знаем ли мы происхождение значения указателя, которое только что использовали?

Совсем нет.

Это может быть любая область памяти в процессе.

Неправильное использование значения указателя может в итоге вызвать серьезные проблемы где-то еще в приложении, в путях, которые полагаются на неизменность состояния этой памяти. Если возобновить выполнение приложения, мы заметим, что нарушение доступа действительно происходит, хотя и гораздо позже в ходе выполнения.

```
0:000> g
(1a8.75c): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=0000000a ebx=00080000 ecx=00080178 edx=00000000 esi=00000002 edi=0000000f
eip=7c911404 esp=0006f77c ebp=0006f99c iopl=0         nv up ei pl nz ac po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00010212
ntdll!RtlAllocateHeap+0x6c9:
7c911404 0fb70e movzx ecx,word ptr [esi] ds:0023:00000002=????
0:000> g
(1a8.75c): Access violation - code c0000005 (!!! second chance !!!)
eax=0000000a ebx=00080000 ecx=00080178 edx=00000000 esi=00000002 edi=0000000f
eip=7c911404 esp=0006f77c ebp=0006f99c iopl=0         nv up ei pl nz ac po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000212
ntdll!RtlAllocateHeap+0x6c9:
7c911404 0fb70e movzx ecx,word ptr [esi] ds:0023:00000002=????
0:000> k
ChildEBP RetAddr
0007f9b0 7c80e323 ntdll!RtlAllocateHeap+0x6c9
0007fa24 7c80e00d kernel32!BaseComputeProcessPath+0xb3
0007fa64 7c80e655 kernel32!BaseComputeProcessDllPath+0xe3
0007faac 7c80e5ab kernel32!GetModuleHandleForUnicodeString+0x28
0007ff30 7c80e45c kernel32!BaseGetModuleHandleExW+0x18e
0007ff48 7c80b6c0 kernel32!GetModuleHandleW+0x29
0007ff54 77c39d23 kernel32!GetModuleHandleA+0x2d
0007ff60 77c39e78 msvcrt!__crtExitProcess+0x10
0007ff70 77c39e90 msvcrt!_cinit+0xee
0007ff84 01001429 msvcrt!exit+0x12
0007ffc0 7c816fd7 06uninit!__wmainCRTStartup+0x118
0007fff0 00000000 kernel32!BaseProcessStart+0x23
```

Как видите, стек, сообщающий о нарушении доступа, никак не связан с нашим собственным кодом. Все, что мы действительно знаем, - это то, что когда процесс собирается завершиться, как видно по самому нижнему кадру (`msvcrt!__crtExitProcess+0x10`), он пытается выделить память и отказывает в менеджере памяти.

Обычно нарушения доступа, происходящие в менеджере кучи, являются хорошими индикаторами того, что произошло повреждение кучи.

Обратное отслеживание источника повреждения из этой точки может быть мучительно трудным процессом, которого следует избегать любой ценой.

Из двух предыдущих примеров запусков должно быть ясно, что поймать повреждение кучи в точке возникновения гораздо предпочтительнее, чем иметь спорадические сбои в путях кода, которыми мы напрямую не владеем.

Один из способов добиться этого - запускать процесс под отладчиком и позволить менеджеру кучи использовать шаблоны заполнения, обеспечивающие некоторый уровень защиты.

Хотя менеджер кучи действительно предоставляет этот механизм, это не обязательно самый сильный уровень защиты.

Использование шаблонов заполнения требует выполнения вызова менеджера кучи, чтобы он мог проверить, что шаблон заполнения все еще допустим.

В большинстве случаев к моменту проверки ущерб уже нанесен, а сбой, вызванный менеджером кучи, все равно требует от нас двигаться назад и выяснять, что изначально вызвало отказ.

Помимо неинициализированного состояния, еще один очень распространенный сценарий, приводящий к повреждениям кучи, - это переполнение кучи.

Переполнения и выходы за нижнюю границу кучи

Во введении к этой главе мы рассмотрели внутреннее устройство менеджера кучи и то, как располагаются все блоки кучи.

Рисунок 6.8 показал, как разбивается блок кучи и какие вспомогательные метаданные хранятся для каждого блока, чтобы менеджер кучи мог управлять этим блоком.

Если дефектный фрагмент кода перезаписывает какие-либо метаданные, целостность кучи нарушается, и приложение откажет.

Самая распространенная форма перезаписи метаданных возникает тогда, когда владелец блока кучи не соблюдает границы блока.

Это явление называется `heap overrun` или, в обратном направлении, `heap underrun`.

Рассмотрим пример.

Приложение, показанное в листинге 6.6, просто создает копию строки, переданной в командной строке, и выводит эту копию.

Листинг 6.6. Приложение с переполнением блока кучи

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
#define SZ_MAX_LEN 10
WCHAR* pszCopy = NULL ;
BOOL DupString(WCHAR* psz);
int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    int iRet=0;
    if(argc==2)
    {
        printf("Press any key to start\n");
        _getch();
        DupString(pArgs[1]);
    }
    else
    {
        iRet=1;
    }
    return iRet;
}
BOOL DupString(WCHAR* psz)
{
    BOOL bRet=FALSE;
```



```

if(psz!=NULL)
{
    pszCopy=(WCHAR*) HeapAlloc(GetProcessHeap(),
    0,
    SZ_MAX_LEN*sizeof(WCHAR));
    if(pszCopy)
    {
        wcscpy(pszCopy, psz);
        wprintf(L"Copy of string: %s", pszCopy);
        HeapFree(GetProcessHeap(), 0, pszCopy);
        bRet=TRUE;
    }
}
return bRet;
}

```

Исходный код и двоичный файл для листинга 6.6 находятся в следующих папках:

```

Source code: C:\AWD\Chapter6\Overrun
Binary: C:\AWDBIN\WinXP.x86.chk\060verrun.exe

```

Когда вы запускаете это приложение с разными входными строками, вы быстро заметите, что строки размером 10 символов или меньше, похоже, работают нормально.

Как только вы превышаете предел в 10 символов, приложение падает.

Выберем следующую строку для использования в нашем сеансе отладки:

```
C:\AWDBIN\WinXP.x86.chk\060verrun.exe ThisStringShouldReproTheCrash
```

Запустите приложение и подключите отладчик, когда увидите приглашение Press any key to start.

После подключения нажмите любую клавишу, чтобы возобновить выполнение, и посмотрите, как отладчик прерывает выполнение из-за нарушения доступа.

```

...
...
...
0:001> g
(1b8.334): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=00650052 ebx=00080000 ecx=00720070 edx=00083188 esi=00083180 edi=0000000f
eip=7c91142e esp=0006f77c ebp=0006f99c iopl=0         nv up ei ng nz na po cy
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00010283
ntdll!RtlAllocateHeap+0x653:
7c91142e 8b39 mov edi,dword ptr [ecx] ds:0023:00720070=????????
0:000> k
ChildEBP RetAddr
0007f70c 7c919f5d ntdll!RtlpInsertFreeBlock+0xf3
0007f73c 7c918839 ntdll!RtlpInitializeHeapSegment+0x186
0007f780 7c911c76 ntdll!RtlpExtendHeap+0x1ca
0007f9b0 7c80e323 ntdll!RtlAllocateHeap+0x623
0007fa24 7c80e00d kernel32!BasepComputeProcessPath+0xb3
0007fa64 7c80e655 kernel32!BasepComputeProcessDllPath+0xe3
0007faac 7c80e5ab kernel32!GetModuleHandleForUnicodeString+0x28
0007ff30 7c80e45c kernel32!BasepGetModuleHandleExW+0x18e
0007ff48 7c80b6c0 kernel32!GetModuleHandleW+0x29
0007ff54 77c39d23 kernel32!GetModuleHandleA+0x2d
0007ff60 77c39e78 msvcrt!__crtExitProcess+0x10
0007ff70 77c39e90 msvcrt!_cinit+0xee
0007ff84 010014c2 msvcrt!exit+0x12
0007ffc0 7c816fd7 060verrun!__wmainCRTStartup+0x118
0007fff0 00000000 kernel32!BasepProcessStart+0x23

```

Беглый взгляд на стек показывает, что приложение находилось в процессе завершения работы, когда произошло нарушение доступа.

Согласно нашему предыдущему обсуждению, всякий раз, когда вы сталкиваетесь с нарушением доступа в коде менеджера кучи, велика вероятность, что вы имеете дело с повреждением кучи.

Единственная проблема в том, что нашего кода нигде нет в стеке.

И снова самая большая проблема с повреждениями кучи состоит в том, что сбойный код нелегко поймать в точке повреждения; вместо этого повреждение обычно проявляется позже в ходе выполнения.

Уже одно это поведение делает отслеживание источника повреждения кучи действительно трудным.

Однако, понимая, как работает менеджер кучи, мы можем выполнить предварительное исследование кучи и посмотреть, удастся ли найти какие-либо подсказки о возможных виновниках. Не зная, какая часть кучи повреждена, хорошая отправная точка - проверить, целы ли сегменты. Вместо ручного обхода сегментов мы используем команду расширения !heap, которая избавляет нас от массы изнурительной ручной работы с кучей.

Сокращенная версия вывода для кучи процесса по умолчанию показана в листинге 6.7.

Листинг 6.7. Использование !heap для проверки состояния кучи процесса по умолчанию

```
0:000> !heap -s
Heap Flags Reserv Commit Virt Free List UCR Virt Lock Fast
      (k)      (k)      (k) (k) length blocks cont. heap
-----
00080000 00000002 1024 16 16 3 1 1 0 0 L
00180000 00001002 64 24 24 15 1 1 0 0 L
00190000 00008000 64 12 12 10 1 1 0 0
00260000 00001002 64 28 28 7 1 1 0 0 L
-----
0:000> !heap -a 00080000
Index Address Name Debugging options enabled
1: 00080000
Segment at 00080000 to 00180000 (00004000 bytes committed)
Flags: 00000002
ForceFlags: 00000000
Granularity: 8 bytes
Segment Reserve: 00100000
Segment Commit: 00002000
DeCommit Block Thres: 00000200
DeCommit Total Thres: 00002000
Total Free Size: 000001d0
Max. Allocation Size: 7ffdefff
Lock Variable at: 00080608
Next TagIndex: 0000
Maximum TagIndex: 0000
Tag Entries: 00000000
```

Листинг 6.7 (продолжение)

```
0 PsuedoTag Entries: 00000000
Virtual Alloc List: 00080050
UCR FreeList: 00080598
FreeList Usage: 00000000 00000000 00000000 00000000
FreeList[ 00 ] at 00080178: 00083188 . 00083188
00083180: 003a8 . 00378 [00] - free
Unable to read nt!_HEAP_FREE_ENTRY structure at 0065004a
Segment00 at 00080640:
Flags: 00000000
Base: 00080000
First Entry: 00080680
Last Entry: 00180000
Total Pages: 00000100
Total UnCommit: 000000fc
Largest UnCommit:000fc000
UnCommitted Ranges: (1)
00084000: 000fc000
Heap entries for Segment00 in Heap 00080000
00080000: 00000 . 00640 [01] - busy (640)
00080640: 00640 . 00040 [01] - busy (40)
00080680: 00040 . 01808 [01] - busy (1800)
00081e88: 01808 . 00210 [01] - busy (208)
00082098: 00210 . 00228 [01] - busy (21a)
```

```

000822c0: 00228 . 00090 [01] - busy (84)
00082350: 00090 . 00030 [01] - busy (22)
00082380: 00030 . 00018 [01] - busy (10)
00082398: 00018 . 00068 [01] - busy (5b)
00082400: 00068 . 00230 [01] - busy (224)
00082630: 00230 . 002e0 [01] - busy (2d8)
00082910: 002e0 . 00320 [01] - busy (314)
00082c30: 00320 . 00320 [01] - busy (314)
00082f50: 00320 . 00030 [01] - busy (24)
00082f80: 00030 . 00030 [01] - busy (24)
00082fb0: 00030 . 00050 [01] - busy (40)
00083000: 00050 . 00048 [01] - busy (40)
00083048: 00048 . 00038 [01] - busy (2a)
00083080: 00038 . 00010 [01] - busy (1)
00083090: 00010 . 00050 [01] - busy (44)
000830e0: 00050 . 00018 [01] - busy (10)
000830f8: 00018 . 00068 [01] - busy (5b)
00083160: 00068 . 00020 [01] - busy (14)
00083180: 003a8 . 00378 [00]
000834f8: 00000 . 00000 [00]

```

Последняя запись кучи в сегменте обычно является свободным блоком.

Однако в листинге 6.7 в конце есть пара странных записей.

Состояние блоков кучи (0), похоже, указывает, что оба блока свободны; однако размеры блоков, похоже, не согласуются.

Посмотрим на первый свободный блок:

```
00083180: 003a8 . 00378 [00]
```

Блок кучи сообщает, что размер предыдущего блока равен 003a8, а размер текущего блока - 00378. Интересно, что предыдущий блок сообщает свой собственный размер как 0x20 байтов, что плохо согласуется.

Еще хуже, последняя свободная запись в сегменте утверждает, что и предыдущий, и текущий размеры равны 0.

Если пройти еще дальше назад по сегменту кучи, можно увидеть, что все записи кучи до 00083160 выглядят осмысленно (по крайней мере в том смысле, что метаданные записей кучи кажутся целыми).

Теперь должна начать формироваться одна из возможных теорий.

Использование блока кучи по адресу 00083160 выглядит подозрительным, и возможно, что использование этого блока кучи привело к повреждению метаданных следующего блока.

Кто выделил блок кучи по адресу 00083160?

Если присмотреться к блоку внимательнее, можно увидеть, узнаем ли мы его содержимое:

```

0:000> dd 00083160
00083160 000d0004 000c0199 00000000 00730069
00083170 00740053 00690072 0067006e 00680053
00083180 0075006f 0064006c 00650052 00720070
00083190 0054006f 00650068 00720043 00730061
000831a0 00000068 00000000 00000000 00000000
000831b0 00000000 00000000 00000000 00000000
000831c0 00000000 00000000 00000000 00000000
000831d0 00000000 00000000 00000000 00000000

```

Части блока, похоже, напоминают строку.

Если использовать команду du для блока, начиная с адреса 000830f8+0хс, мы увидим следующее:

```

0:000> du 00083160+c
0008316c "isStringShouldReproTheCrash"

```

Строка определенно выглядит знакомо.

Это та же строка (или ее часть), которую мы передали в командной строке.

Кроме того, строка, похоже, тянется до адреса 000831a0, что пересекает границу со следующим сообщенным свободным блоком по адресу 00083180.

Если вывести запись кучи по адресу 00083180, мы увидим следующее:

```
0:000> dt _HEAP_ENTRY 00083180
+0x000 Size : 0x6f
+0x002 PreviousSize : 0x75
+0x000 SubSegmentCode : 0x0075006
+0x004 SmallTagIndex : 0x6c 'l'
+0x005 Flags : 0 ''
+0x006 UnusedBytes : 0x64 'd'
+0x007 SegmentIndex : 0 ''
```

Поля текущего и предыдущего размера соответствуют части строки, которая пересекла границу предыдущего блока.

Вооружившись знанием о том, какая строка, похоже, вызвала перезапись блока кучи, мы можем перейти к ревью кода и относительно легко понять, что функция копирования строки записала больше максимально допустимого числа символов в строку назначения, вызвав перезапись следующего блока кучи.

Хотя менеджер кучи не смог обнаружить перезапись точно в момент ее возникновения, он определенно обнаружил перезапись блока кучи позже в ходе выполнения, что привело к нарушению доступа, поскольку куча находилась в несогласованном состоянии.

В предыдущем упрощенном приложении анализ кучи в точке нарушения доступа дал очень ясную картину того, что перезаписало блок кучи, а затем через ревью кода - кто был виновником. Разумеется, не всегда возможно прийти к таким выводам, просто проверяя содержимое блоков кучи.

Сложность системы может резко снизить ваши шансы на успех при использовании этого подхода.

Более того, даже если вы получите какие-то подсказки о том, что перезаписывает блоки кучи, найти виновника только через ревью кода может быть действительно трудно.

В конечном счете самый простой способ разобраться с повреждением кучи был бы в том, чтобы прервать выполнение в момент перезаписи памяти, а не после нее.

К счастью, инструмент Application Verifier предоставляет мощную возможность, позволяющую добиться такого поведения.

Параметр тестирования Application Verifier, обычно используемый при отслеживании повреждений кучи, называется Hears (также его называют pagehear).

Pagehear работает на основе окружения блоков кучи защитным слоем, который служит для изоляции блоков кучи друг от друга.

Если блок кучи перезаписан, защитный слой обнаруживает перезапись как можно ближе к источнику и прерывает выполнение, давая разработчику возможность исследовать, почему произошла перезапись.

Pagehear работает в двух разных режимах: normal pagehear и full pagehear.

Основное различие между двумя режимами - в силе защитного слоя.

Normal pagehear использует шаблоны заполнения в попытке обнаружить повреждения блоков кучи.

Использование шаблонов заполнения требует, чтобы после повреждения был сделан еще один вызов менеджера кучи, чтобы менеджер кучи получил возможность проверить целостность блока кучи (проверить шаблоны заполнения) и сообщить о любых несоответствиях.

Кроме того, normal pagehear сохраняет трассировку стека для всех выделений, что упрощает понимание того, кто выделил память.

Рисунок 6.10 показывает, как выглядит блок кучи при включенном normal pagehear.

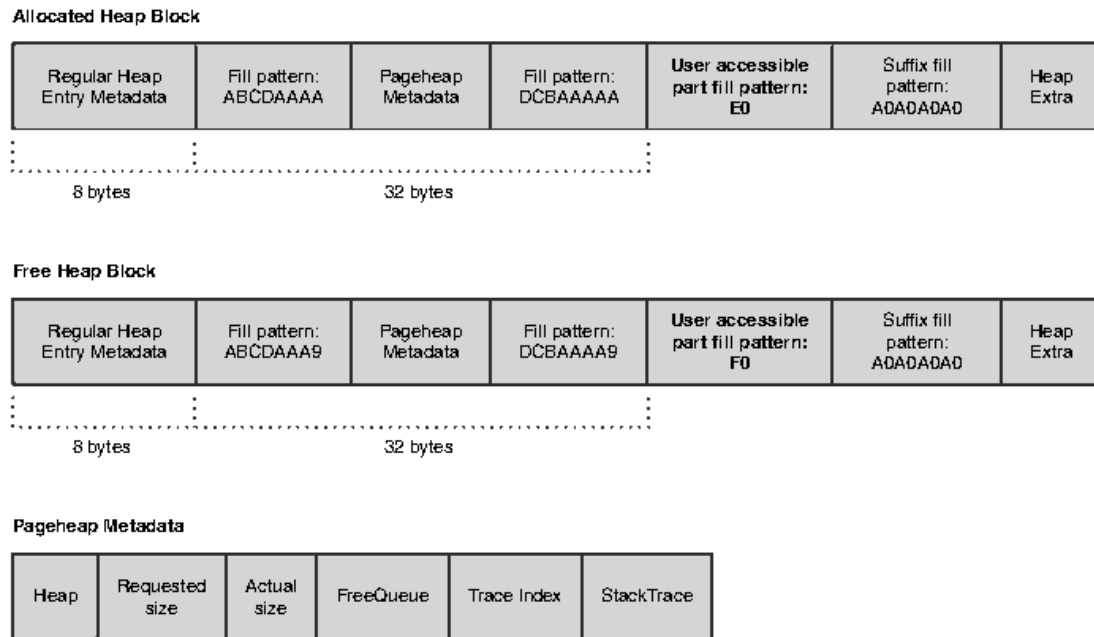


Figure 6.10 Normal page heap block layout

Рисунок 6.10. Компоновка блока normal pageheap

Основное различие между обычным блоком кучи и блоком normal page heap - добавление метаданных pageheap.

Метаданные pageheap содержат такую информацию, как запрошенный и фактический размеры блока, но, возможно, самый полезный член этих метаданных - трассировка стека.

Член stack trace позволяет разработчику получить полную трассировку стека происхождения выделения (то есть место, где оно было выделено).

Это очень помогает при рассмотрении поврежденного блока кучи, поскольку дает подсказки о том, кто является владельцем блока кучи, и позволяет сузить область ревью кода.

Представьте, что вызов HeapAlloc в листинге 6.6 вернул следующий указатель: 0019e260.

Чтобы вывести содержимое метаданных pageheap, сначала нужно вычесть из указателя 32 (0x20) байта.

```
0:000> dd 0019e4b8-0x20
0019e498 abcdaaaa 80081000 00000014 0000003c
0019e4a8 00000018 00000000 0028697c dcbaaaaa
0019e4b8 e0e0e0e0 e0e0e0e0 e0e0e0e0 e0e0e0e0
0019e4c8 e0e0e0e0 a0a0a0a0 a0a0a0a0 00000000
0019e4d8 00000000 00000000 000a0164 00001000
0019e4e8 00180178 00180178 00000000 00000000
0019e4f8 00000000 00000000 00000000 00000000
0019e508 00000000 00000000 00000000 00000000
```

Здесь мы ясно видим начальный (abcdaaaa) и конечный (dcbaaaaa) шаблоны заполнения, окружающие метаданные.

Чтобы увидеть метаданные pageheap в более удобоваримой форме, можно использовать тип данных _DPH_BLOCK_INFORMATION:

```
0:000> dt _DPH_BLOCK_INFORMATION 0019e4b8-0x20
+0x000 StartStamp :
+0x004 Heap : 0x80081000
+0x008 RequestedSize :
+0x00c ActualSize :
+0x010 FreeQueue : _LIST_ENTRY 18-0
+0x010 TraceIndex : 0x18
+0x018 StackTrace : 0x0028697c
+0x01c EndStamp :
```

Член StackTrace содержит трассировку стека выделения.

Чтобы увидеть трассировку стека, нужно использовать команду dds, которая отображает содержимое диапазона памяти в предположении, что содержимое этого диапазона является последовательностью адресов из таблицы символов.

```
0:000> dds 0x0028697c
0028697c abcdaaaa
00286980 00000001
00286984 00000006
...
...
...
0028699c 7c949d18 ntdll!RtlAllocateHeapSlowly+0x44
002869a0 7c91b298 ntdll!RtlAllocateHeap+0xe64
002869a4 01001224 06overrun!DupString+0x24
002869a8 010011eb 06overrun!wmain+0x2b
002869ac 010013a9 06overrun!wmainCRTStartup+0x12b
002869b0 7c816d4f kernel32!BaseProcessStart+0x23
002869b4 00000000
002869b8 00000000
...
...
...
```

Сокращенная версия вывода команды dds показывает нам трассировку стека кода, выполнившего выделение.

Невозможно переоценить полезность записанной базы данных трассировок стека.

Ищете ли вы повреждения кучи или утечки памяти, имея любой блок pageheap, вы очень легко можете получить трассировку стека кода, который выполнил выделение, а это, в свою очередь, позволяет сосредоточить усилия на соответствующей области кода.

Теперь посмотрим, как средство normal pageheap можно использовать для отслеживания повреждения памяти, показанного ранее в листинге 6.6.

Включите normal pageheap для приложения (см. приложение A, "Application Verifier Test Settings") и запустите процесс под отладчиком, используя ThisStringShouldReproTheCrash в качестве входных данных.

Листинг 6.8 показывает, как Application Verifier прерывает выполнение из-за поврежденного блока кучи.

Листинг 6.8. Остановка Application Verifier из-за поврежденного блока кучи

```
...
...
...
0:000> g
Press any key to start
Copy of string: ThisStringShouldReproTheCrash
=====
VERIFIER STOP 00000008 : pid 0x640: Corrupted heap block.
00081000 : Heap handle used in the call.
001A04D0 : Heap block involved in the operation.
00000014 : Size of the heap block.
00000000 : Reserved
=====
This verifier stop is not continuable. Process will be terminated
when you use the `go' debugger command.
=====
(640.6a8): Break instruction exception - code 80000003 (first chance)
eax=000001ff ebx=0040acac ecx=7c91eb05 edx=0006f949 esi=00000000 edi=000001ff
eip=7c901230 esp=0006f9dc ebp=0006fbdc iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
ntdll!DbgBreakPoint:
7c901230 cc int 3
```

Информация, представленная Application Verifier, дает нам указатель на блок кучи, который был поврежден.

Отсюда получить трассировку стека кода, выполнившего выделение, тривиально.

```
0:000> dt _DPH_BLOCK_INFORMATION 001A04D0-0x20
+0x000 StartStamp : 0xabcdaaaa
+0x004 Heap : 0x80081000
+0x008 RequestedSize : 0x14
+0x00c ActualSize : 0x3c
+0x010 FreeQueue : _LIST_ENTRY [ 0x18 - 0x0 ]
+0x010 TraceIndex : 0x18
+0x018 StackTrace : 0x0028697c
+0x01c EndStamp : 0xdcbaaaa
0:000> dds 0x0028697c
0028697c abcdaaaa
00286980 00000001
00286984 00000006
00286988 00000001
0028698c 00000014
00286990 00081000
00286994 00000000
00286998 0028699c
0028699c 7c949d18 ntdll!RtlAllocateHeapSlowly+0x44
002869a0 7c91b298 ntdll!RtlAllocateHeap+0xe64
002869a4 01001202 06overrun!DupString+0x22
002869a8 010011c1 06overrun!wmain+0x31
002869ac 0100138d 06overrun!wmainCRTStartup+0x12f
002869b0 7c816fd7 kernel32!BaseProcessStart+0x23
...
...
...
```

Зная трассировку стека, мы можем эффективно найти виновника, сузив область ревью кода.

Если сравнить подход без Application Verifier к выяснению причины сбоя процесса с подходом, где Application Verifier включен, вы быстро увидите, насколько второй эффективнее.

При использовании normal pageheap вся информация о поврежденном блоке предоставляется нам, и мы можем использовать ее для анализа блока кучи и получения трассировки стека кода, выполнившего выделение.

Хотя normal pageheap прерывает выполнение и дает всю эту полезную информацию, он все равно делает это только после того, как повреждение уже произошло, и все равно требует от нас выполнить некоторый обратный анализ, чтобы понять, почему это случилось.

Есть ли механизм, позволяющий прервать выполнение еще ближе к повреждению?

Безусловно.

Normal pageheap - лишь один из двух режимов pageheap, которые можно включить.

Другой режим известен как full pageheap.

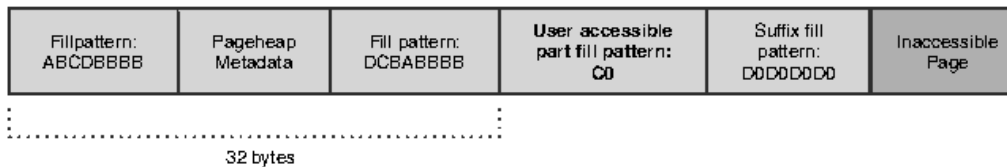
Помимо собственных уникальных шаблонов заполнения, full pageheap добавляет к каждому блоку кучи понятие guard page.

Guard page - это страница недоступной памяти, помещаемая либо в начало, либо в конец блока кучи.

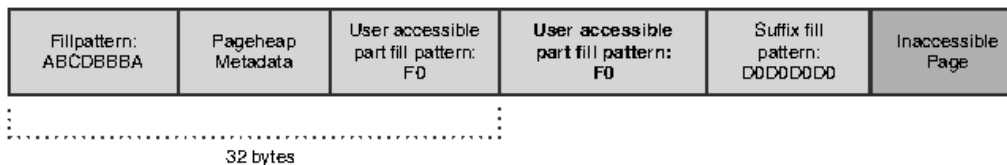
Размещение guard page в начале блока кучи защищает от heap block underruns, а размещение в конце защищает от heap overruns.

Рисунок 6.11 иллюстрирует компоновку блока full pageheap.

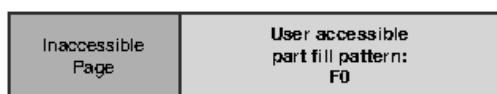
Forward Overrun: Allocated Heap Block



Forward Overrun: Free Heap Block



Backward Overrun



Pageheap Metadata

Heap	Requested size	Actual size	FreeQueue	Trace Index	StackTrace
------	-------------------	----------------	-----------	-------------	------------

Figure 6.11 Full page heap block layout

Рисунок 6.11. Компоновка блока full pageheap

Недоступная страница добавляется для защиты от переполнений или выходов за нижнюю границу блока кучи.

Если дефектный фрагмент кода записывает в недоступную страницу, это вызывает нарушение доступа, и выполнение прерывается прямо на месте.

Это позволяет нам избежать любой стратегии обратного отслеживания для выяснения происхождения повреждения.

Теперь мы можем снова запустить наше примерное приложение, на этот раз с включенным full pageheap (см. приложение А), и посмотреть, где отладчик прервет выполнение.

```
...
...
...
0:000> g
Press any key to start
(414.494): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=006f006f ebx=7ffd7000 ecx=005d5000 edx=006fefdc esi=7c9118f1 edi=00011970
eip=77c47ea2 esp=0006ff20 ebp=0006ff20 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010202
msvcrt!wcscopy+0xe:
77c47ea2 668901 mov word ptr [ecx],ax ds:0023:005d5000=????
0:000> kb
ChildEBP RetAddr Args to Child
0006ff20 01001221 005d4fe8 006fefc0 00000000 msvcrt!wcscopy+0xe
0006ff34 010011c1 006fefc0 00000000 0006ffc0 06overrun!DupString+0x41
0006ff44 0100138d 00000002 006fef98 00774f88 06overrun!wmain+0x31
0006ffc0 7c816fd7 00011970 7c9118f1 7ffd7000 06overrun!wmainCRTStartup+0x12f
0006fff0 00000000 0100125e 00000000 78746341 kernel32!BaseProcessStart+0x23
```


На этот раз нарушение доступа зафиксировано во время вызова копирования строки. Если присмотреться к блоку кучи в точке нарушения доступа, мы увидим:

```
0:000> dd 005d4fe8
005d4fe8 00680054 00730069 00740053 00690072
005d4ff8 0067006e 00680053 ???????? ????????
005d5008 ???????? ???????? ???????? ????????
005d5018 ???????? ???????? ???????? ????????
005d5028 ???????? ???????? ???????? ????????
005d5038 ???????? ???????? ???????? ????????
005d5048 ???????? ???????? ???????? ????????
005d5058 ???????? ???????? ???????? ????????
0:000> du 005d4fe8
005d4fe8 "ThisStringSh?????????????????????"
005d5028 "?????????????????????????????????"
005d5068 "?????????????????????????????????"
005d50a8 "?????????????????????????????????"
005d50e8 "?????????????????????????????????"
005d5128 "?????????????????????????????????"
005d5168 "?????????????????????????????????"
005d51a8 "?????????????????????????????????"
005d51e8 "?????????????????????????????????"
005d5228 "?????????????????????????????????"
005d5268 "?????????????????????????????????"
005d52a8 "?????????????????????????????????"
```

По дампам можно сделать два важных наблюдения:

- Строка, которую мы копируем, перезаписала suffix fill pattern блока, а также запись кучи.
- В точке нарушения доступа скопированная к этому моменту строка равна ThisStringSh, что указывает: функция копирования строки еще не закончила работу и вот-вот запишет в недоступную страницу, помещенную Application Verifier в конец блока кучи.

Включив full pageheap, мы смогли прервать выполнение в момент возникновения повреждения, а не после него.

Это может сэкономить огромное количество времени, поскольку в момент повреждения проблемный код находится прямо перед вами, и выяснить, почему произошло повреждение, становится гораздо легче.

Один из вопросов, который может возникнуть у вас в голове: "Почему бы всегда не запускать с включенным full pageheap?"

Что ж, full pageheap очень требователен к ресурсам.

Помните, что full pageheap помещает одну страницу недоступной памяти в конец (или начало) каждого выделения.

Если процесс, который вы отлаживаете, активно использует память, применение pageheap может увеличить общее потребление памяти на порядок.

Помимо переполнений блоков кучи, мы можем столкнуться и с обратным случаем: heap underruns. Хотя они встречаются не так часто, heap underruns перезаписывают часть блока кучи перед доступной пользователю частью.

Это может происходить из-за неправильной арифметики указателей, вызывающей преждевременную запись в блок кучи.

Поскольку normal pageheap защищает метаданные pageheap с помощью шаблонов заполнения, он тоже может ловить сценарии heap underrun.

Full pageheap по умолчанию помещает guard page в конец блока кучи и не будет прерывать выполнение при heap underruns.

К счастью, используя параметр backward overrun для full pageheap (см. приложение А), мы можем указать ему помещать guard page в начало выделения, а не в конец, и тем самым также ловить класс проблем underrun.

Команда расширения !heap, ранее использовавшаяся для анализа состояния кучи, также может использоваться, когда процесс выполняется под pageheap.

Используя флаг -p, мы можем сообщить команде расширения !heap, что рассматриваемая куча включена для pageheap.

Варианты, доступные для флага -p, таковы:

```
heap -p Dump all page heaps.
heap -p -h ADDR Detailed dump of page heap at ADDR.
heap -p -a ADDR Figure out what heap block is at ADDR.
heap -p -t [N] Dump N collected traces with heavy heap users.
heap -p -tc [N] Dump N traces sorted by count usage (eqv. with -t).
heap -p -ts [N] Dump N traces sorted by size.
heap -p -fi [N] Dump last N fault injection traces.
```

Например, блок кучи, возвращенный вызовом HeapAlloc в нашем примерном приложении, при использовании флагов -p и -a выглядит следующим образом:

```
0:000> !heap -p -a 005d4fe8
address 005d4fe8 found in
_DPH_HEAP_ROOT @ 81000
in busy allocation ( DPH_HEAP_BLOCK: UserAddr UserSize -
VirtAddr VirtSize)
8430c: 5d4fe8 14 -
5d4000 2000
7c91b298 ntdll!RtlAllocateHeap+0x00000e64
01001202 06overrun!DupString+0x00000022
010011c1 06overrun!wmain+0x00000031
0100138d 06overrun!wmainCRTStartup+0x0000012f
7c816fd7 kernel32!BaseProcessStart+0x00000023
```

Вывод показывает записанную трассировку стека, а также другую вспомогательную информацию, например какой шаблон заполнения используется.

Шаблоны заполнения могут давать подсказки о состоянии блока кучи (выделен он или освобожден).

Еще один полезный ключ - -t.

Ключ -t позволяет вывести часть базы данных трассировок стека, чтобы получить больше информации обо всех стеках, которые выделяли память.

Если вы отлаживаете процесс, который потребляет массу памяти, и хотите узнать, какая часть процесса отвечает за самые крупные выделения, можно использовать команду heap -p -t.

Несоответствия дескрипторов кучи

Менеджер кучи хранит список активных куч в процессе.

Кучи считаются отдельными сущностями в том смысле, что внутреннее состояние каждой кучи действительно только в контексте именно этой кучи.

Разработчики, работающие с менеджером кучи, должны быть очень внимательны и соблюдать это разделение, гарантируя, что при выделении и освобождении памяти кучи используются правильные кучи.

Это разделение открывается разработчику через использование дескрипторов кучи в вызовах API кучи.

Каждый дескриптор кучи уникально представляет конкретную кучу в списке куч процесса.

Примером является вызов API GetProcessHeap, который возвращает уникальный дескриптор кучи процесса по умолчанию.

Другой пример - вызов API HeapCreate, который возвращает уникальный дескриптор вновь созданной кучи.

Если уникальность нарушена, возникнет повреждение кучи.

Листинг 6.9 иллюстрирует приложение, которое нарушает уникальность куч.

Листинг 6.9. Приложение, нарушающее уникальность куч

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
#define MAX_SMALL_BLOCK_SIZE 20000
HANDLE hSmallHeap=0;
HANDLE hLargeHeap=0;
VOID* AllocMem(ULONG ulSize);
VOID FreeMem(VOID* pMem, ULONG ulSize);
BOOL InitHeaps();
VOID FreeHeaps();
int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    printf("Press any key to start\n");
    _getch();
    if(InitHeaps())
    {
        BYTE* pBuffer1=(BYTE*) AllocMem(20);
        BYTE* pBuffer2=(BYTE*) AllocMem(20000);
        //
        // Use allocated memory
        //
        FreeMem(pBuffer1, 20);
        FreeMem(pBuffer2, 20000);
        FreeHeaps();
    }
    printf("Done...exiting application\n");
    return 0;
}
BOOL InitHeaps()
{
    BOOL bRet=TRUE ;
    hSmallHeap = GetProcessHeap();
    hLargeHeap = HeapCreate(0, 0, 0);
    if(!hLargeHeap)
    {
        bRet=FALSE;
    }
    return bRet;
}
VOID FreeHeaps()
{
    if(hLargeHeap)
    {
        HeapDestroy(hLargeHeap);
        hLargeHeap=NULL;
    }
}
VOID* AllocMem(ULONG ulSize)
{
    VOID* pAlloc = NULL ;
    if(ulSize<MAX_SMALL_BLOCK_SIZE)
    {
        pAlloc=HeapAlloc(hSmallHeap, 0, ulSize);
    }
    else
    {
        pAlloc=HeapAlloc(hLargeHeap, 0, ulSize);
    }
    return pAlloc;
}
VOID FreeMem(VOID* pAlloc, ULONG ulSize)
{
    if(ulSize<=MAX_SMALL_BLOCK_SIZE)
    {
        HeapFree(hSmallHeap, 0, pAlloc);
    }
    else
    {
        HeapFree(hLargeHeap, 0, pAlloc);
    }
}
```

```
}  
}
```

Исходный код и двоичный файл для листинга 6.9 находятся в следующих папках:

```
Source code: C:\AWD\Chapter6\Mismatch  
Binary: C:\AWDBIN\WinXP.x86.chk\06Mismatch.exe
```

Приложение в листинге 6.9 выглядит довольно прямолинейным.

Основная функция запрашивает пару выделений с помощью вспомогательной функции AllocMem. Закончив с выделениями, она вызывает вспомогательный API FreeMem, чтобы освободить память. Вспомогательные API выделения работают с памятью либо из кучи процесса по умолчанию (если выделение меньше некоторого размера), либо из частной кучи (созданной в API InitHeaps), если размер больше порога.

Если запустить приложение, мы увидим, что оно успешно завершает выполнение:

```
C:\AWDBIN\WinXP.x86.chk\06Mismatch.exe  
Press any key to start  
Done...exiting application
```

У нас может возникнуть соблазн заключить, что приложение работает как ожидается, и подписать его как готовое.

Однако прежде чем делать это, используем Application Verifier, включим для приложения full pageheap и запустим его снова.

На этот раз приложение так и не завершилось.

Фактически, судя по появившемуся диалогу сбоя, похоже, у нас crash.

Чтобы получить больше информации о сбое, запустим приложение под отладчиком:

```
...  
...  
...  
0:000> g  
Press any key to start  
(118.3c8): Access violation - code c0000005 (first chance)  
First chance exceptions are reported before any exception handling.  
This exception may be expected and handled.  
eax=0006fc54 ebx=00000000 ecx=0211b000 edx=0211b008 esi=021161e0 edi=021161e0  
eip=7c96893a esp=0006fbec ebp=0006fc20 iopl=0 nv up ei ng nz ac po cy  
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010293  
ntdll!RtlpDphIsNormalHeapBlock+0x81:  
7c96893a 8039a0 cmp byte ptr [ecx],0A0h ds:0023:0211b000=??  
0:000> kb  
ChildEBP RetAddr Args to Child  
0006fc20 7c96ac47 00081000 021161e0 0006fc54 ntdll!RtlpDphIsNormalHeapBlock+0x81  
0006fc44 7c96ae5a 00081000 01000002 00000007 ntdll!RtlpDphNormalHeapFree+0x1e  
0006fc94 7c96defb 00080000 01000002 021161e0 ntdll!RtlpDebugPageHeapFree+0x79  
0006fd08 7c94a5d0 00080000 01000002 021161e0 ntdll!RtlDebugFreeHeap+0x2c  
0006fdf0 7c9268ad 00080000 01000002 021161e0 ntdll!RtlFreeHeapSlowly+0x37  
0006fec0 003ab9eb 00080000 00000000 021161e0 ntdll!RtlFreeHeap+0xf9  
0006ff18 010012cf 00080000 00000000 021161e0 vfbasics!AVrfrRtlFreeHeap+0x16b  
0006ff2c 010011d3 021161e0 00004e20 021161e0 06mismatch!FreeMem+0x1f  
0006ff44 01001416 00000001 02060fd8 020daf80 06mismatch!wmain+0x53  
0006ffc0 7c816fd7 00011970 7c9118f1 7ffdc000 06mismatch!wmainCRTStartup+0x12f  
0006fff0 00000000 010012e7 00000000 78746341 kernel32!BaseProcessStart+0x23
```

По трассировке стека видно, что наше приложение пыталось освободить блок памяти, когда менеджер кучи получил нарушение доступа.

Чтобы выяснить, какое из двух выделений памяти мы освобождали, разберем функцию 06mismatch!wmain и посмотрим, какой из вызовов соответствует адресу 06mismatch!wmain+0x55.

```
0:000> u 06mismatch!wmain+0x53-10  
06mismatch!wmain+0x43:  
010011c3 0000 add byte ptr [eax],al  
010011c5 68204e0000 push 4E20h  
010011ca 8b4df8 mov ecx,dword ptr [ebp-8]  
010011cd 51 push ecx
```

```

010011ce e8dd000000 call 06mismatch!FreeMem (010012b0)
010011d3 e858000000 call 06mismatch!FreeHeaps (01001230)
010011d8 688c100001 push offset 06mismatch!`string' (0100108c)
010011dd ff1550100001 call dword ptr [06mismatch!_imp__printf (01001050)]

```

Поскольку вызов перед 06mismatch!FreeHeaps - это FreeMem, мы знаем, что проблему вызывает последний вызов FreeMem в нашем коде.

Теперь можно применить ревью кода и посмотреть, нет ли чего-нибудь неправильного.

Из листинга 6.9 видно, что функция FreeMem освобождает память либо в куче процесса по умолчанию, либо в частной куче.

Кроме того, похоже, решение зависит от размера блока.

Если размер блока меньше или равен 20 КБ, используется куча процесса по умолчанию.

В противном случае используется частная куча.

Наше выделение было ровно 20 КБ, а это означает, что функция FreeMem попыталась освободить память из кучи процесса по умолчанию.

Правильно ли это?

Один простой способ выяснить это - вывести метаданные блока pageheap, внутри которых содержится дескриптор владеющей кучи:

```

0:000> dt _DPH_BLOCK_INFORMATION 021161e0-0x20
+0x000 StartStamp : 0xabcdabbbb
+0x004 Heap : 0x02111000
+0x008 RequestedSize : 0x4e20
+0x00c ActualSize : 0x5000
+0x010 FreeQueue : _LIST_ENTRY [ 0x21 - 0x0 ]
+0x010 TraceIndex : 0x21
+0x018 StackTrace : 0x00287510
+0x01c EndStamp : 0xdcababbbb

```

Владеющая куча для этого блока кучи - 0x02111000.

Далее выясним, чему равна куча процесса по умолчанию:

```

0:000> x 06mismatch!hSmallHeap
01002008 06mismatch!hSmallHeap = 0x00080000

```

Две кучи не совпадают, и по сути мы освобождаем блок памяти, принадлежащий куче 0x02111000, в куче 0x00080000.

Это также причина, по которой Application Verifier прервал выполнение: несоответствие куч вызывает серьезные проблемы стабильности.

Вооружившись знанием причины остановки, теперь должно быть довольно просто понять, почему наше приложение перепутало две кучи.

Поскольку мы полагаемся на размер, чтобы определить, в каких кучах выделять и освобождать память, мы быстро видим, что функция AllocMem использует следующее условие:

```

if(ulSize<MAX_SMALL_BLOCK_SIZE)
{
    pAlloc=HeapAlloc(hSmallHeap, 0, ulSize);
}

```

тогда как функция FreeMem использует:

```

if(ulSize<=MAX_SMALL_BLOCK_SIZE)
{
    HeapFree(hSmallHeap, 0, pAlloc);
}

```

Условие выделения проверяет, что размер выделения меньше порога, тогда как условие освобождения проверяет, что он меньше или равен порогу.

Следовательно, при освобождении выделения размером 20 КБ память неправильно освобождается из кучи процесса по умолчанию.

Помимо умения анализировать проблемы несоответствия куч и докапываться до их сути, из нашего упражнения можно извлечь еще один очень важный урок: никогда не предполагайте, что приложение работает правильно только потому, что при обычном неинструментированном запуске не сообщается об ошибках.

Как вы уже видели, проблемы повреждения кучи не всегда проявляются во время тестов, выполняемых без какой-либо помощи отладки.

Проблемы проявляются только тогда, когда подключен отладчик и включен Application Verifier.

Причина проста.

При запуске без отладчика и без Application Verifier повреждение кучи все равно происходит, но может не успеть проявиться в виде нарушения доступа.

Допустим, тест проходит сценарии А, В и С, а повреждение кучи происходит в сценарии С.

После повреждения кучи приложение завершается без каких-либо признаков повреждения, и вы считаете, что все работает правильно.

После поставки приложения клиенту он запускает те же сценарии, но в другом порядке: С, В и А.

Первым выполняется сценарий С, немедленно вызывая повреждение кучи, но приложение не завершается; вместо этого оно продолжает выполнять сценарии В и А, создавая гораздо большее окно, в котором повреждение кучи может реально повлиять на приложение.

Повторное использование кучи после освобождения

После переполнений кучи повторное использование кучи после освобождения является вторым по распространенности источником повреждений кучи.

Как вы уже видели, после освобождения блока кучи диспетчер кучи помещает его в свободные списки (или в look aside list).

С этого момента он считается недопустимым для использования приложением.

Если приложение использует свободный блок каким-либо образом, в каком-либо виде или форме, состояние блока в свободном списке, скорее всего, будет повреждено, и приложение аварийно завершится.

Прежде чем рассматривать практические примеры повторного использования кучи после освобождения, давайте еще раз разберем процесс удаления.

На рисунке 6.12 показан гипотетический пример сегмента кучи.

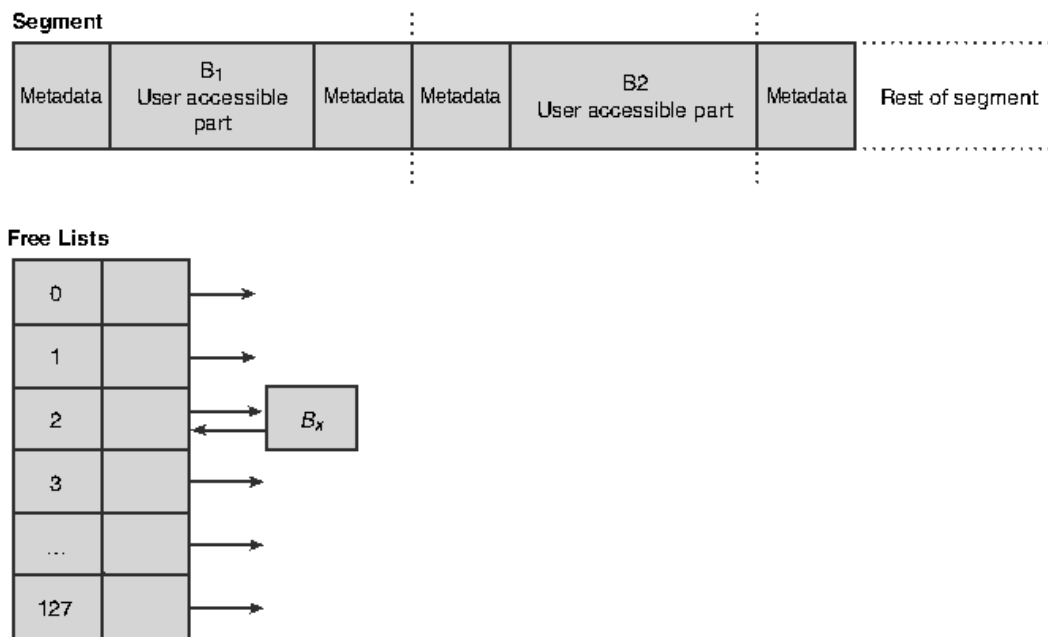


Figure 6.12 Hypothetical example of a heap segment

Рисунок 6.12. Гипотетический пример сегмента кучи

Сегмент состоит из двух занятых блоков (B₁ и B₂), доступная пользователю часть которых окружена связанными с ними метаданными.

Кроме того, свободный список содержит один свободный блок (B_x) размером 16.

Если приложение освобождает блок B₁, диспетчер кучи прежде всего проверяет, можно ли объединить блок с какими-либо соседними свободными блоками.

Поскольку соседних свободных блоков нет, диспетчер кучи просто обновляет состояние блока (поле флагов в метаданных) на свободное и обновляет соответствующий свободный список, чтобы включить в него B₁.

Важно отметить, что свободный список состоит из прямой ссылки (FLINK) и обратной ссылки (BLINK), каждая из которых указывает на следующий и предыдущий свободный блок в списке.

Являются ли указатели FLINK и BLINK частью отдельно выделенного узла свободного списка?

Не совсем: по причинам эффективности при освобождении блока структура существующего свободного блока меняется.

Точнее, доступная пользователю часть блока кучи перезаписывается диспетчером кучи указателями FLINK и BLINK, каждый из которых указывает на следующий и предыдущий свободный блок в свободном списке.

В нашем гипотетическом примере на рисунке 6.12 блок B₁ вставляется в начало свободного списка, соответствующего размеру 16.

Доступная пользователю часть B₁ заменяется на FLINK, указывающий на B_x, и BLINK, указывающий на начало списка (на самого себя).

Существующий свободный блок B_x также обновляется: его BLINK указывает на B₁.

На рисунке 6.13 показана получившаяся раскладка после освобождения блока B₁.

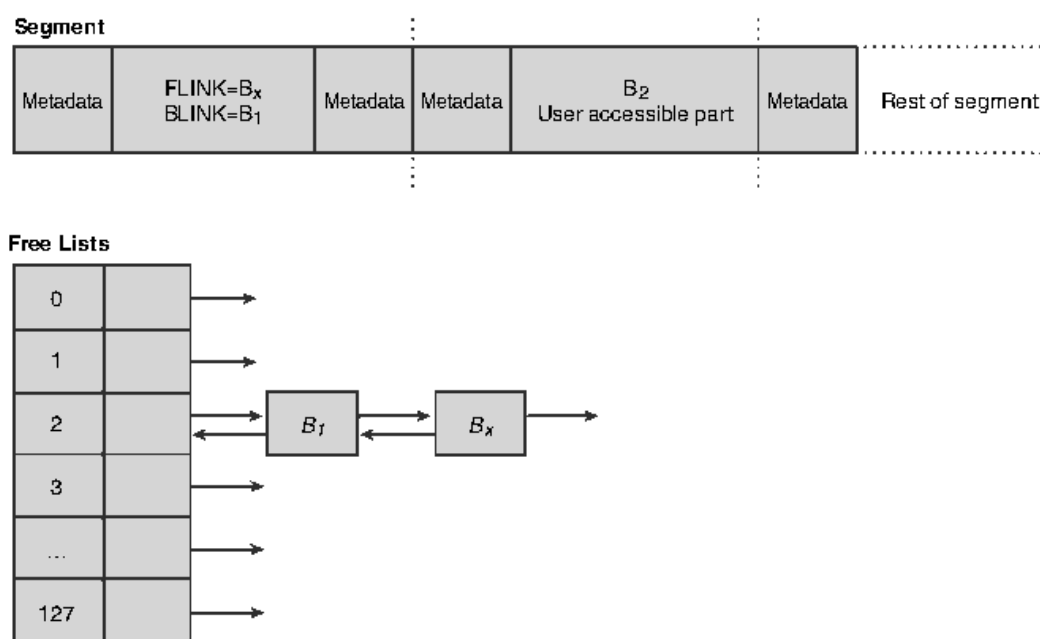


Figure 6.13 Heap segment and free lists after freeing B₁

Рисунок 6.13. Сегмент кучи и свободные списки после освобождения B₁

Далее, когда приложение освобождает блок B₂, диспетчер кучи находит соседний свободный блок (B₁) и объединяет оба блока в один большой свободный блок.

В рамках процесса объединения диспетчер кучи должен удалить блок B₁ из свободного списка, поскольку он больше не существует, и добавить новый более крупный блок в соответствующий ему свободный список.

Доступная пользователю часть получившегося большого блока теперь содержит указатели FLINK и BLINK, которые обновляются в соответствии с состоянием свободного списка.

До сих пор мы предполагали, что все освобожденные блоки кучи попадают в свободные списки back end allocator.

Хотя верно, что некоторые свободные блоки идут прямо в свободные списки, часть выделений может в итоге попасть в look aside list front end allocator.

Когда блок кучи попадает в look aside list, основные отличия можно увидеть в метаданных блока кучи:

- У блоков кучи, которые попадают в look aside list, бит состояния установлен в busy (в отличие от free в свободных списках).
- Look aside list является односвязным списком (в отличие от двусвязных свободных списков), и поэтому действительным считается только указатель FLINK.

Самый важный аспект освобождения памяти, связанный с повторным использованием кучи после освобождения, состоит в том, что структура блока кучи меняется после его освобождения.

Доступная пользователю часть блока кучи теперь используется для внутреннего учета, чтобы поддерживать свободные списки в актуальном состоянии.

Если приложение перезаписывает какое-либо содержимое (полагая, что блок все еще занят), указатели FLINK и BLINK повреждаются, и структурная целостность свободного списка нарушается. Итоговым результатом, скорее всего, станет аварийное завершение где-то позже, когда диспетчер кучи попытается манипулировать свободным списком (обычно во время очередного вызова выделения или освобождения).

В листинге 6.10 показан пример приложения, которое выделяет блок памяти, а затем дважды освобождает этот блок.

Листинг 6.10. Двойное освобождение блока кучи

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>

int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    printf("Press any key to start\n");
    _getch();

    BYTE* pByte=(BYTE*) HeapAlloc(GetProcessHeap(), 0, 10);
    (*pByte)=10;
    HeapFree(GetProcessHeap(), 0, pByte);
    HeapFree(GetProcessHeap(), 0, pByte);

    printf("Done...exiting application\n");

    return 0;
}
```

Исходный код и двоичный файл для листинга 6.10 можно найти в следующих папках:

Source code: C:\AWD\Chapter6\Db1Free
Binary: C:\AWDBIN\WinXP.x86.chk\06Db1Free.exe

Запуск приложения не приводит к ошибкам:

C:\AWDBIN\WinXP.x86.chk\06Db1Free.exe

Чтобы убедиться, что не происходит ничего необычного, запустим приложение под отладчиком и дойдем до первого выделения кучи.

```
0:001> u wmain
06db1free!wmain:
01001180 55 push ebp
01001181 8bec mov ebp,esp
01001183 51 push ecx
01001184 68a8100001 push offset 06db1free!`string' (010010a8)
01001189 ff1548100001 call dword ptr [06db1free!_imp__printf (01001048)]
0100118f 83c404 add esp,4
01001192 ff1550100001 call dword ptr [06db1free!_imp__getch (01001050)]
01001198 6a0a push 0Ah
0:001> u
06db1free!wmain+0x1a:
0100119a 6a00 push 0
0100119c ff1508100001 call dword ptr [06db1free!_imp__GetProcessHeap
(01001008)]
010011a2 50 push eax
010011a3 ff1500100001 call dword ptr [06db1free!_imp__HeapAlloc (01001000)]
010011a9 8945fc mov dword ptr [ebp-4],eax
010011ac 8b45fc mov eax,dword ptr [ebp-4]
010011af c6000a mov byte ptr [eax],0Ah
010011b2 8b4dfc mov ecx,dword ptr [ebp-4]
0:001> g 010011a9
eax=000830c0 ebx=7ffde000 ecx=7c9106eb edx=00080608 esi=01c7078e edi=83485b7a
eip=010011a9 esp=0006ff40 ebp=0006ff44 iopl=0 nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000246
06db1free!wmain+0x29:
010011a9 8945fc mov dword ptr [ebp-4],eax
ss:0023:0006ff40={msvcrt!__winitenv (77c61a40)}
```

Регистр `eax` теперь содержит указатель на только что выделенный блок памяти:

```
0:000> dt _HEAP_ENTRY 000830c0-0x8
+0x000 Size : 3
+0x002 PreviousSize : 3
+0x000 SubSegmentCode : 0x00030003
```

```
+0x004 SmallTagIndex : 0x21 '!'
+0x005 Flags : 0x1 ''
+0x006 UnusedBytes : 0xe ''
+0x007 SegmentIndex : 0 ''
```

Ничего необычного, похоже, нет: поля размера выглядят разумными, а поле флагов указывает, что блок занят.

Теперь продолжим выполнение за первый вызов HeapFree и выведем тот же блок кучи.

```
0:000> dt _HEAP_ENTRY 000830c0-0x8
+0x000 Size : 3
+0x002 PreviousSize : 3
+0x000 SubSegmentCode : 0x00030003
+0x004 SmallTagIndex : 0x21 '!'
+0x005 Flags : 0x1 ''
+0x006 UnusedBytes : 0xe ''
+0x007 SegmentIndex : 0 ''
```

Даже после освобождения блока метаданные выглядят идентично.

В поле флагов даже по-прежнему установлен бит busy, что указывает, что блок не освобожден.

Ключевой момент здесь - помнить, что при освобождении блока кучи он может попасть в одно из двух мест: look aside list или свободные списки.

Когда блок кучи помещается в look aside list, состояние блока кучи сохраняется как busy.

В свободных списках, однако, состояние устанавливается в free.

В нашей конкретной операции освобождения блок, похоже, попал в look aside list.

Когда блок помещается в look aside list, первая часть доступной пользователю области блока перезаписывается указателем FLINK, который указывает на следующий доступный блок в look aside list.

Доступная пользователю часть нашего блока выглядит следующим образом:

```
0:000> dd 000830c0
000830c0 00000000 00080178 00000000 00000000
000830d0 000301e6 00001000 00080178 00080178
000830e0 00000000 00000000 00000000 00000000
000830f0 00000000 00000000 00000000 00000000
00083100 00000000 00000000 00000000 00000000
00083110 00000000 00000000 00000000 00000000
00083120 00000000 00000000 00000000 00000000
00083130 00000000 00000000 00000000 00000000
```

Как видите, указатель FLINK в нашем случае равен NULL, что означает, что это первый свободный блок кучи.

Далее продолжим выполнение до момента сразу после второго вызова HeapFree (для того же самого блока).

Снова посмотрим на состояние блока кучи:

```
0:000> dt _HEAP_ENTRY 000830c0-0x8
+0x000 Size : 3
+0x002 PreviousSize : 3
+0x000 SubSegmentCode : 0x00030003
+0x004 SmallTagIndex : 0x21 '!'
+0x005 Flags : 0x1 ''
+0x006 UnusedBytes : 0xe ''
+0x007 SegmentIndex : 0 ''
```

В метаданных, похоже, ничего не изменилось.

Блок все еще занят, и поля размера, похоже, не изменились.

Выведем доступную пользователю часть и посмотрим на указатель FLINK:

```
0:000> dd 000830c0
000830c0 000830c0 00080178 00000000 00000000
000830d0 000301e6 00001000 00080178 00080178
000830e0 00000000 00000000 00000000 00000000
000830f0 00000000 00000000 00000000 00000000
```

```
00083100 00000000 00000000 00000000 00000000
00083110 00000000 00000000 00000000 00000000
00083120 00000000 00000000 00000000 00000000
00083130 00000000 00000000 00000000 00000000
```

На этот раз FLINK указывает на другой свободный блок кучи, доступная пользователю часть которого начинается по адресу 000830c0.

Блок, соответствующий адресу 000830c0, - тот же самый блок, который мы освободили в первый раз.

Двойным освобождением мы, по сути, поместили look aside list в кольцевую ссылку.

Последствием этого может стать бесконечный цикл в диспетчере кучи, когда последующие операции с кучей заставят диспетчер кучи пройти по свободному списку с кольцевой ссылкой.

Если в этот момент возобновить выполнение, мы заметим, что приложение завершает работу. Почему оно завершилось без сбоя в коде кучи?

Чтобы кольцевая ссылка look aside list проявилась, должен быть выполнен еще один вызов диспетчера кучи, который заставил бы его пройти по списку и попасть на кольцевую ссылку. Наше приложение закончилось после второго вызова HeapFree, и диспетчер кучи так и не получил возможности аварийно завершиться.

Хотя сбой не проявился в тех нескольких запусках, которые мы выполнили, это все равно повреждение кучи, и его нужно исправлять.

Повреждение блока кучи в look aside list (или в свободных списках) может вызвать серьезные проблемы в приложении.

Как и предыдущие типы повреждений кучи, проблемы двойного освобождения обычно проявляются в форме сбоев после повреждения, когда диспетчеру кучи нужно пройти по look aside list (или свободному списку).

Можно ли и в этом случае использовать Application Verifier, чтобы поймать проблему в момент ее возникновения?

Та же настройка тестирования куч, которую мы использовали на протяжении всей главы, также делает лучшую попытку поймать проблемы двойного освобождения.

Помечая блоки кучи определенным способом, Application Verifier способен ловить проблемы двойного освобождения в момент их возникновения и прерывать выполнение, позволяя разработчику внимательнее посмотреть на код, который пытается освободить блок во второй раз. Включим full pageheap для нашего приложения и повторно запустим его под отладчиком.

Сразу же вы увидите нарушение доступа первого шанса со следующей трассировкой стека:

```
0:000> kb
ChildEBP RetAddr Args to Child
0007fcc4 7c96ac47 00091000 005e4ff0 0007fcf8 ntdll!RtlpDphIsNormalHeapBlock+0x1c
0007fce8 7c96ae5a 00091000 01000002 00000000 ntdll!RtlpDphNormalHeapFree+0x1e
0007fd38 7c96defb 00090000 01000002 005e4ff0 ntdll!RtlpDebugPageHeapFree+0x79
0007fdac 7c94a5d0 00090000 01000002 005e4ff0 ntdll!RtlDebugFreeHeap+0x2c
0007fe94 7c9268ad 00090000 01000002 005e4ff0 ntdll!RtlFreeHeapSlowly+0x37
0007ff64 0100128a 00090000 00000000 005e4ff0 ntdll!RtlFreeHeap+0xf9
0007ff7c 01001406 00000001 0070cfd8 0079ef68 06Db1Free!wmain+0x5a
0007ffc0 7c816fd7 00011970 7c9118f1 7ffd7000 06Db1Free!__wmainCRTStartup+0x102
0007fff0 00000000 01001544 00000000 78746341 kernel32!BaseProcessStart+0x23
```

Судя по стеку, видно, что наша функция wmain выполняет второй вызов HeapFree, который в итоге приводит к нарушению доступа глубоко в коде диспетчера кучи.

Всякий раз, когда эта настройка тестирования включена и вы сталкиваетесь со сбоем во время вызова HeapFree, первое, что следует проверить, - не освобождается ли блок кучи дважды.

Поскольку блок кучи при освобождении может попасть в look aside list (его состояние может по-прежнему быть busy, хотя с точки зрения диспетчера кучи он считается свободным), лучший способ выяснить, действительно ли он свободен, - использовать команду !heap -p -a <heap block>.

Помните, что эта команда выводит подробную информацию о блоке pageheap, включая трассировку стека кода выделения или освобождения. Найдите адрес блока кучи, который мы освобождаем дважды (согласно предыдущей трассировке стека), и выполните для него команду расширения !heap:

```
0:000> !heap -p -a 005d4ff0
address 005d4ff0 found in
_DPH_HEAP_ROOT @ 81000
in free-ed allocation ( DPH_HEAP_BLOCK: VirtAddr VirtSize)
8430c: 5d4000 2000
7c9268ad ntdll!RtlFreeHeap+0x000000f9
010011c5 06db1free!wmain+0x00000045
0100131b 06db1free!wmainCRTStartup+0x0000012f
7c816fd7 kernel32!BaseProcessStart+0x00000023
```

Как видно из вывода, состояние блока кучи - свободный.

Кроме того, стек показывает последнюю операцию, выполненную над блоком кучи, то есть первый выполненный вызов освобождения.

Показанная трассировка стека хорошо соответствует нашему первому вызову HeapFree в функции wmain.

Если возобновить выполнение приложения, мы заметим еще несколько нарушений доступа первого шанса, пока наконец не получим останов Application Verifier:

```
0:000> g
(1d4.6d4): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=0006fc7c ebx=00081000 ecx=00000008 edx=00000000 esi=005d4fd0 edi=0006fc4c
eip=7c969a1d esp=0006fc40 ebp=0006fc8c iopl=0 nv up ei pl nz na po cy
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010203
ntdll!RtlpDphReportCorruptedBlock+0x25:
7c969a1d f3a5 rep movs dword ptr es:[edi],dword ptr [esi]
es:0023:0006fc4c=00000000 ds:0023:005d4fd0=???????
0:000> g
(1d4.6d4): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=0006fc20 ebx=00000000 ecx=005d4ff0 edx=00000000 esi=00000000 edi=00000000
eip=7c968a84 esp=0006fc08 ebp=0006fc30 iopl=0 nv up ei pl zr na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010246
ntdll!RtlpDphGetBlockSizeFromCorruptedBlock+0x13:
7c968a84 8b41e0 mov eax,dword ptr [ecx-20h] ds:0023:005d4fd0=???????
0:000> g
=====
VERIFIER STOP 00000008 : pid 0x1D4: Corrupted heap block.
00081000 : Heap handle used in the call.
005D4FF0 : Heap block involved in the operation.
00000000 : Size of the heap block.
00000000 : Reserved
=====
This verifier stop is not continuable. Process will be terminated
when you use the `go' debugger command.
=====
(1d4.6d4): Break instruction exception - code 80000003 (first chance)
eax=000001ff ebx=0040acac ecx=7c91eb05 edx=0006f959 esi=00000000 edi=000001ff
eip=7c901230 esp=0006f9ec ebp=0006fbec iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
ntdll!DbgBreakPoint:
7c901230 cc int 3
```

Показанный останов Application Verifier последнего шанса дает базовую информацию о поврежденном блоке кучи.

Если в этот момент возобновить выполнение, приложение просто завершится, поскольку это невозможный останов.

На этом мы завершаем обсуждение проблем, связанных с двойным освобождением памяти.

Как вы видели, лучший инструмент для обнаружения проблем двойного освобождения - использовать настройку тестирования куч (full pageheap), доступную в Application Verifier. Она не только сообщает о текущей проблеме, но и прерывает выполнение в той точке, где проблема действительно возникла, а не на стадии после повреждения, благодаря чему гораздо проще понять, почему блок кучи был поврежден.

Использование full pageheap дает максимально сильный уровень защиты, доступный для проблем, связанных с памятью в целом.

Механизм, с помощью которого full pageheap способен обеспечивать эту защиту, заключается в отделении метаданных блока кучи от самого блока кучи.

В сценарии без full pageheap метаданные, связанные с блоком кучи, являются частью самого блока кучи.

Если приложение ошибается всего на несколько байтов, оно очень легко может перезаписать метаданные, повредив блок кучи и затруднив диспетчеру кучи немедленное сообщение о проблеме.

В отличие от этого, при использовании full pageheap метаданные хранятся во вторичной структуре данных с односторонней ссылкой на настоящий блок кучи.

Благодаря использованию односторонней ссылки неисправному коду почти невозможно повредить метаданные блока кучи, и поэтому full pageheap почти всегда можно доверять в том, что он содержит неповрежденную информацию.

Именно отделение метаданных от фактического блока кучи дает full pageheap возможность обеспечивать сильное обнаружение повреждений кучи.

Резюме

Повреждение кучи - серьезная ошибка, которая может вызвать хаос в вашем приложении.

Единичное повреждение на один байт из-за ошибки на единицу может заставить приложение демонстрировать самые разные странные виды поведения.

Приложение может аварийно завершаться, может вести себя непредсказуемо или даже входить в бесконечные циклы.

Хуже того, итоговый результат повреждения кучи обычно не проявляется до тех пор, пока повреждение уже не произошло, что чрезвычайно затрудняет выяснение источника повреждения кучи.

Чтобы эффективно отслеживать повреждения кучи, нужно твердое понимание внутреннего устройства диспетчера кучи.

В первой части главы обсуждались низкоуровневые подробности работы диспетчера кучи.

Мы рассмотрели, как блок кучи проходит через различные уровни диспетчера кучи и как состояние и структура блока меняются при переходе от выделенного состояния к освобожденному.

Мы также рассмотрели некоторые из самых распространенных форм повреждений кучи (неинициализированное состояние, переполнения кучи вперед и назад, несоответствующие дескрипторы кучи и повторное использование кучи после освобождения) и то, как вручную анализировать кучу в точке сбоя, чтобы выяснить источник повреждения.

Кроме того, мы обсудили, как Application Verifier (pageheap) можно использовать для прерывания выполнения ближе к источнику повреждения, что значительно облегчает выяснение виновника.

Как показывают некоторые примеры в этой главе, повреждения кучи могут остаться незамеченными во время тестирования программного обеспечения и проявиться только на компьютере клиента при запуске в другой среде и при других условиях.

Постоянное использование Application Verifier (pageheap) является необходимым условием для того, чтобы повреждения кучи обнаруживались до поставки программного обеспечения и чтобы избежать дорогостоящих проблем на стороне клиента.

Глава 7. Безопасность

За сравнительно короткий период времени отношение к безопасности программного обеспечения резко изменилось - как с точки зрения разработчика, так и с точки зрения пользователя.

Много лет назад компьютеры в основном были отключенными друг от друга устройствами, а автономные носители, главным образом гибкие диски, были основным источником проблем компьютерной безопасности.

Большую проблему в то время представляли вирусы.

Сегодня почти каждая проблема компьютерной безопасности может эксплуатироваться удаленно из-за высокой степени связности.

Старые операционные системы, такие как Windows 95, не предоставляли поддержки для защиты объектов, хранящихся на локальном компьютере.

Появление кодовой базы Windows NT на потребительских рынках сделало защищенное, C2-совместимое ядро доступным потребителям.

Сегодня потребительские версии операционной системы Windows, а именно Windows XP Home и Windows Vista Home, управляют доступом к каждому объекту, и поэтому возрастает вероятность столкнуться с отказом в доступе.

Еще один толчок исходит от сообщества безопасности, которое рекомендует всегда запускать процесс от имени пользователя с наименьшими привилегиями.

В этом случае хост-компьютер изолируется от уязвимостей безопасности, которые могут существовать в приложениях.

Насколько реально запускать приложение не от имени администратора?

Возможно, это осуществимо для некоторых приложений, спроектированных с учетом безопасности, тогда как большинство из них все еще будет пытаться обратиться к расположению реестра или файловой системы, зарезервированному только для администраторов.

Будем надеяться, что безопасность объектов станет первоклассной опорой разработки.

Эта глава предоставляет информацию, необходимую для начала пути к успешному пониманию и исправлению проблем безопасности программного обеспечения.

Глава сосредоточена прежде всего на шагах, выполняемых тогда, когда допустимая операция завершается успехом или отказом, и не описывает неожиданное поведение кода из-за дефектов кода (переполнения буфера, целочисленного переполнения, выхода за границы буфера), которое сегодня эксплуатируется вирусами, поскольку эта тема очень хорошо освещена в нескольких справочных книгах.

В этой главе мы рассмотрим следующее:

- Основы безопасности Windows и то, как в действительности работает Windows Security. Мы суммируем ключевую информацию, необходимую для понимания проблем, связанных с безопасностью.
- Как исследовать различные элементы безопасности с помощью расширений отладчика. Этот раздел вводит несколько команд расширений, необходимых для отладки аспектов безопасности.
- Как объединить методики и информацию, представленные в книге до этого момента, чтобы решать проблемы, вызванные неожиданными ограничениями безопасности.

Обзор безопасности Windows

Любой защищаемый объект Windows, который может быть представлен дескриптором этого объекта, имеет связанную с ним информацию безопасности и защищается стандартными механизмами безопасности Windows.

Модель безопасности Windows использует три концепции безопасности:

- Discretionary access control list (DACL): описывает, какой принципал может использовать объект и каким образом.
- Идентичность пользователя: также известна как принципал.
- Security Reference Monitor (SRM): использует доступную информацию, чтобы ограничивать доступ к защищаемому им объекту.

DACL, связанные с защищаемыми объектами Windows, управляются самим создателем объекта. DACL является компонентом другой структуры, известной как дескриптор безопасности, то есть небольшого фрагмента информации, хранящегося вместе с объектом в защищенном хранилище. Дескриптор безопасности извлекается из защищенного хранилища и используется каждый раз, когда к объекту обращается новый принципал.

Например, дескрипторы безопасности файлов хранятся в файловой системе NTFS, дескрипторы безопасности разделов реестра хранятся в кустах реестра, тогда как у объектов ядра дескрипторы безопасности хранятся в адресном пространстве ядра.

Windows SRM выполняется в адресном пространстве ядра, изолированном от кода пользовательского режима.

Большинство защищаемых объектов создается и управляется компонентами ядра, которые используют разделение адресных пространств, чтобы защищать связанный дескриптор безопасности от компонентов пользовательского режима.

Поскольку компоненты пользовательского режима не могут использовать ядро для реализации собственных брокеров защищаемых объектов, несколько компонентов Windows реализуют специальные модели безопасности, применяя идеи, похожие на механизмы безопасности Windows.

Специальный брокер объектов должен принудительно обеспечивать механизм доступа к своему объекту.

Иными словами, при проектировании брокера защищаемых объектов нужно убедиться, что к этому объекту нельзя обратиться с помощью какого-либо другого механизма.

В таких случаях брокер объектов берет на себя роль SRM и управляет дескрипторами безопасности объектов собственными способами.

Чтобы обеспечить функциональную согласованность с остальной операционной системой и использовать те же элементы пользовательского интерфейса в параметрах безопасности, брокер объектов, скорее всего, будет использовать те же структуры данных, что и Windows SRM.

Другим ключевым компонентом в управлении доступом является принципал безопасности, создаваемый и удостоверяемый операционной системой.

Принципал безопасности хранится в маркере доступа, который объединяет список групповых принципалов безопасности, членом которых является данный принципал, список специальных привилегий, предоставленных операционной системой, а также другую информацию, используемую различными компонентами системы.

Доступ к объекту представлен набором битов, каждый из которых представляет право (специфичное для природы объекта), которое может быть предоставлено принципалу или запрещено для него.

В следующем разделе описываются все структуры безопасности, относящиеся к отладке приложений Windows, и приводятся различные методы их исследования.

Читатели, знакомые с этими концепциями, могут пропустить этот раздел.

Во всех примерах используются три новые команды расширений: !sd, !token и !sid, доступные в расширении по умолчанию, загружаемом отладчиками.

В этой главе используется 07sample.exe, исходный код и двоичный файл которого находятся в следующих папках:

```
Source code: C:\AWD\Chapter7
Binary: C:\AWDBIN\WinXP.x86.chk\07sample.exe
```

Поскольку ошибки безопасности часто встречаются в распределенных приложениях, в этой главе также используется пример, созданный для главы 8, "Interprocess Communication".

Он состоит из клиентского приложения 08cli.exe, библиотеки 08comps.dll, содержащей проxy-stub-код, и серверного приложения 08comsrv.exe.

08comsrv.exe нужно зарегистрировать с помощью командной строки 08comsrv.exe /RegServer, а 08comps.dll нужно зарегистрировать с помощью командной строки regsvr32 08comps.dll.

Исходный код и двоичные файлы находятся в следующих папках:

```
Source code: C:\AWD\Chapter8
Binaries: C:\AWDBIN\WinXP.x86.chk\08cli.exe, 08comps.dll, and
08comsrv.exe.
```

Идентификатор безопасности

Идентификатор безопасности, также известный как SID, является одной из базовых концепций, используемых в Windows Security.

SID идентифицирует принципал или атрибут, уникальный относительно области идентификаторов, доступных в операционной системе, использующей этот SID.

SID представлен простой структурой, объявленной в заголовочном файле winnt.h, как показано в листинге 7.1.

Листинг 7.1. Структуры SID_IDENTIFIER_AUTHORITY и SID

```
typedef struct _SID_IDENTIFIER_AUTHORITY {
    BYTE Value[6];
} SID_IDENTIFIER_AUTHORITY;

typedef struct _SID {
    BYTE Revision;
    BYTE SubAuthorityCount;
    SID_IDENTIFIER_AUTHORITY IdentifierAuthority;
    DWORD SubAuthority[1];
} SID;
```

Структура SID является структурой переменной длины, содержащей переменное число записей SubAuthority, предназначенных для представления любого принципала.

SID группируются на основе IdentifierAuthority.

Размещение SID в памяти тривиально, легко понимается компьютером, но трудно интерпретируется человеком.

В технической документации SID представлены строками вида S-R-I-S-S-S-...-S, где R - уровень ревизии, I идентифицирует полномочный орган, управляющий SID, а S - один или несколько относительных идентификаторов подчиненных полномочий, управляемых этим органом.

У SID Windows поле Revision установлено в 1, и они могут иметь до шести подчиненных полномочий.

У Windows IdentifierAuthority равен пяти: {0, 0, 0, 0, 0, 5}.

Например, Local System, идентифицируемая как S-1-5-18, представлена в памяти последовательностью байтов, показанной в следующем листинге (разделенной на несколько строк, соответствующих каждому компоненту SID):


```

0:000> db 000840c8 Lc
000840c8 01
01
00 00 00 00 00 05-
12 00 00 00 .....

```

Первая строка представляет ревизию SID, вторая строка - количество элементов RID, за ней следует идентификатор полномочного органа Windows, а последняя строка содержит RID. Эта структура данных интерпретируется и преобразуется в строковый формат S-... командой расширения !sid следующим образом:

```

0:000> !sid 000840c8
SID is: S-1-5-18

```

Список управления доступом

Следующая фундаментальная структура, встречающаяся при отладке проблем безопасности Windows, - это access control entry (ACE).

ACE указывает, какие права предоставлены принципалу, идентифицированному его SID, над объектом, защищаемым этой ACE.

Набор упорядоченных ACE образует Access Control List (ACL), который управляет правами доступа к базовому объекту для всех принципалов.

Структурно каждая ACE имеет общий ACE_HEADER, за которым следуют данные, специфичные для конкретной ACE: старый прием языка C для реализации полиморфизма объектов.

Все типы ACE очень хорошо документированы в MSDN, а также в заголовочном файле winnt.h.

Текущий раздел описывает только ACCESS_ALLOWED_ACE, поскольку это наиболее часто используемая структура.

Все остальные типы ACE похожи и также могут быть найдены в заголовочном файле winnt.h.

Заголовок структуры ACE объявлен следующим образом:

```

typedef struct _ACE_HEADER {
    BYTE AceType;
    BYTE AceFlags;
    WORD AceSize;
} ACE_HEADER;

typedef struct _ACCESS_ALLOWED_ACE {
    ACE_HEADER Header;
    ACCESS_MASK Mask;
    DWORD SidStart;
} ACCESS_ALLOWED_ACE;

```

Поле AceType идентифицирует тип структуры, следующей за ACE_HEADER.

Обычная практика состоит в приведении универсальной структуры ACE_HEADER к конкретному типу ACE, например ACCESS_ALLOWED_ACE, в зависимости от значения поля AceType.

Поле Mask имеет тип DWORD и объединяет все права, предоставленные этой ACE.

Каждый бит имеет смысл, представленный в таблице 7.1.

Из этой таблицы только младшие 21 бит являются фактическими правами, используемыми как таковые в ACE; все остальные биты используются в других контекстах, где требуется маска доступа.

Таблица 7.1. Значение битов маски доступа

Биты	Значение
31	Generic Read
30	Generic Write
29	Generic Execute

Биты	Значение
28	Generic All
25 to 27	Reserved
24	SACL access
21 to 23	Not defined
20	Synchronize
19	Write Owner
18	Write DAC
17	Read DAC
16	Delete
0 to 15	Object specific rights

Структура ACL объявлена в заголовочном файле `winnt.h` следующим образом:

```
typedef struct _ACL {
    BYTE AclRevision;
    BYTE Sbz1;
    WORD AclSize;
    WORD AceCount;
    WORD Sbz2;
} ACL;
```

В настоящем ACL после этой структуры следует переменное число ACE (как указано в `AceCount`), использующее непрерывную область памяти размером `AclSize` байт.

В настоящее время все ACL, используемые в операционной системе Windows, имеют ревизию, равную 2.

ACL можно легко декодировать с помощью команды расширения `!acl`, как показано ниже:

```
0:000> !acl 000840ac
ACL is:
ACL is: ->AclRevision: 0x2
ACL is: ->Sbz1 : 0x0
ACL is: ->AclSize : 0x1c
ACL is: ->AceCount : 0x1
ACL is: ->Sbz2 : 0x0
ACL is: ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
ACL is: ->Ace[0]: ->AceFlags: 0x0
ACL is: ->Ace[0]: ->AceSize: 0x14
ACL is: ->Ace[0]: ->Mask : 0x00120089
ACL is: ->Ace[0]: ->SID: S-1-1-0
```

Дескриптор безопасности

Все структуры, которые мы видели до сих пор, агрегируются в структуре дескриптора безопасности (SD), определенной в заголовочном файле `winnt.h`, как показано здесь:

```
typedef WORD SECURITY_DESCRIPTOR_CONTROL;

typedef struct _SECURITY_DESCRIPTOR {
    BYTE Revision;
    BYTE Sbz1;
    SECURITY_DESCRIPTOR_CONTROL Control;
    PSID Owner;
    PSID Group;
    PACL Sacl;
    PACL Dacl;
} SECURITY_DESCRIPTOR;
```

Ревизия, используемая операционными системами Windows, установлена в 1.

Поле Control описывает содержимое дескриптора безопасности, например указывает, содержит ли дескриптор безопасности DACL (когда установлен флаг SE_DACL_PRESENT) или SACL, и многое другое.

Все указатели, используемые внутри дескриптора безопасности, следует рассматривать как смещения от базового адреса дескриптора безопасности, когда в поле Control установлен бит SE_SELF_RELATIVE; в противном случае адреса являются абсолютными.

Чтобы понять, как эти структуры размещены в памяти, мы используем исполняемый файл 07sample.exe с параметром 0, который задействует API, связанные с дескрипторами безопасности. Исходный код, показанный в листинге 7.2, создает дескриптор безопасности из строки с использованием security descriptor definition language (SDDL).

Права пользователя, обращающегося к объекту, защищенному этим дескриптором безопасности, получают с помощью API advapi32!AccessCheck.

Листинг 7.2. Создание дескриптора безопасности и вызов AccessCheck

```
void Sample0()
{
    LPWSTR stringSD = L"O:SYG:BAD:(A;;;FR;;;S-1-1-0)";
    PSECURITY_DESCRIPTOR sd = NULL;
    ...
    if (FALSE == ConvertStringSecurityDescriptorToSecurityDescriptor(
        stringSD, SDDL_REVISION_1, &sd, NULL))
    { ... }
    ImpersonateSelf(SecurityIdentification);
    STOP_ON_DEBUGGER;
    HANDLE hToken=NULL;
    if (!OpenThreadToken(
        GetCurrentThread(), TOKEN_QUERY, TRUE, &hToken))
    { ... }
    RevertToSelf();
    ...
    if (FALSE == AccessCheck(
        sd,
        hToken,
        MAXIMUM_ALLOWED,
        &rightsMapping,
        privileges,&privilegesSize ,
        &grantedAccess,
        &grantedAccessStatus))
    {
        TRACE(L"AccessCheck failed ");
    }
    ...
}
```

Распространенные источники дескрипторов безопасности

Адрес дескриптора безопасности часто доступен в закрытых символах.

Когда закрытые символы недоступны, дескриптор безопасности, используемый для проверок доступа, можно обнаружить как первый параметр API advapi32!AccessCheck.

В следующем разделе интерпретируется параметр, доступный в стеке после учета соглашения о вызовах, используемого API (__stdcall в данном случае).

Объявление функции выглядит следующим образом:

```
WINADVAPI BOOL WINAPI AccessCheck (
    IN PSECURITY_DESCRIPTOR pSecurityDescriptor,
    IN HANDLE ClientToken,
    IN DWORD DesiredAccess,
    IN PGENERIC_MAPPING GenericMapping,
    OUT PPRIVILEGE_SET PrivilegeSet,
```

```
IN LPDWORD PrivilegeSetLength,
OUT LPDWORD GrantedAccess,
OUT LPBOOL AccessStatus );
```

Мы запускаем приложение 07sample.exe под отладчиком пользовательского режима, например windbg.exe, и устанавливаем точку останова по адресу API.

Затем дескриптор безопасности выводится байт за байтом в листинге 7.3.

Листинг 7.3. Ручной вывод дескриптора безопасности

```
0:000> k2
ChildEBP RetAddr
0006fe9c 0100204e ADVAPI32!AccessCheck
0006ff00 01001f33 07sample!Sample0+0x10e
0:000> dc @esp L4
0006fea0 0100204e 00084098 000007bc 02000000 N ...@.....
0:000> db 00084098 L4c
00084098 01 00 04 80 30 00 00 00-3c 00 00 00 00 00 00 00 ....0...<.....
000840a8 14 00 00 00 02 00 1c 00-01 00 00 00 00 14 00 .....
000840b8 89 00 12 00 01 01 00 00-00 00 00 01 00 00 00 .....
000840c8 01 01 00 00 00 00 05-12 00 00 00 01 02 00 00 .....
000840d8 00 00 00 05 20 00 00 00-20 02 00 00 .... ..
```

Хотя весь дескриптор безопасности можно расшифровать вручную, лучший вариант - использовать предоставленную команду расширения !sd.

Результат ее использования показан в листинге 7.4.

Листинг 7.4. !sd

```
kd> !sd 00084098
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
          SE_DACL_PRESENT
          SE_SELF_RELATIVE
->Owner : S-1-5-18
->Group : S-1-5-32-544
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x1c
->Dacl : ->AceCount : 0x1
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x14
->Dacl : ->Ace[0]: ->Mask : 0x00120089
->Dacl : ->Ace[0]: ->SID: S-1-1-0
->Sacl : is NULL
```

SID и ACL, введенные в предыдущих разделах, являются частью этого дескриптора безопасности. Адреса этих структур относительно адресу дескриптора безопасности, и их можно легко извлечь, когда расширение не работает из-за несоответствия символов.

Маркер доступа

Дескриптор безопасности полезен только в том случае, если мы можем надежно идентифицировать принципал, запрашивающий доступ к защищенному объекту, который охраняется дескриптором безопасности.

Идентичность принципала, а также все предоставленные ему привилегии инкапсулированы в структуру ядра, называемую маркером доступа.

Маркер доступа используется компонентами пользовательского режима через дескриптор маркера.

Такие маркеры доступа можно исследовать с помощью команды расширения `!token`, которая принимает в качестве аргумента либо адрес маркера доступа, как обычно используется в отладчиках режима ядра, либо дескриптор маркера, как используется в отладчиках пользовательского режима.

Если расширение используется без аргумента, оно отображает маркер доступа олицетворения потока, если он присутствует; в противном случае используется маркер процесса.

В листинге 7.5 мы используем маркер, переданный функции `advapi32!AccessCheck` в листинге 7.3. Поскольку мы используем параметр `-n`, команда расширения разрешает имя, связанное с каждым SID (показано в скобках после SID).

Листинг 7.5. `!token`

```
0:000> * Displays the information for token handle 0x7bc
0:000> !token 7bc -n
TS Session ID: 0
User: S-1-5-21-1060284298-2111687655-1957994488-1003 (User: XP-SP2\TestAdmin)
Groups:
00 S-1-5-21-1060284298-2111687655-1957994488-513 (Group: XP-SP2\None)
Attributes - Mandatory Default Enabled
01 S-1-1-0 (Well Known Group: localhost\Everyone)
Attributes - Mandatory Default Enabled
02 S-1-5-32-544 (Alias: BUILTIN\Administrators)
Attributes - Mandatory Default Enabled Owner
03 S-1-5-32-545 (Alias: BUILTIN\Users)
Attributes - Mandatory Default Enabled
04 S-1-5-4 (Well Known Group: NT AUTHORITY\INTERACTIVE)
Attributes - Mandatory Default Enabled
05 S-1-5-11 (Well Known Group: NT AUTHORITY\Authenticated Users)
Attributes - Mandatory Default Enabled
06 S-1-5-5-0-35778 (no name mapped)
Attributes - Mandatory Default Enabled LogonId
07 S-1-2-0 (Well Known Group: localhost\LOCAL)
Attributes - Mandatory Default Enabled
Primary Group: S-1-5-21-1060284298-2111687655-1957994488-513 (Group: XP-SP2\None)
Privs:
00 0x0000000017 SeChangeNotifyPrivilege Attributes - Enabled Default
01 0x0000000008 SeSecurityPrivilege Attributes -
...
17 0x0000000009 SeTakeOwnershipPrivilege Attributes -
18 0x000000001e SeCreateGlobalPrivilege Attributes - Enabled Default
19 0x000000001d SeImpersonatePrivilege Attributes - Enabled Default
Auth ID: 0:1c3a8
Impersonation Level: Identification
TokenType: Impersonation
```

Внимательно посмотрев на все SID в этом маркере, мы можем сгруппировать их в групповые принципалы безопасности, пользовательские принципалы и идентификаторы, например `LogonId`. Концепция SID очень гибка, поскольку это всего лишь уникальный идентификатор, используемый для представления разных сущностей, подобных тем, что показаны в таблице 7.2.

Таблица 7.2. Типы SID

Тип SID	Примеры значений SID
Идентичность пользователя	S-1-5-21-1060284298-2111687655-1957994488-1003
Идентичность группы	S-1-5-21-1060284298-2111687655-1957994488-513
Источник входа	S-1-5-4 (интерактивный)
Пользовательский сеанс	S-1-5-5-0-35778
Атрибуты	S-1-2-0 (локальный)

Некоторые SID, используемые как атрибуты или как абстрактное членство в группах и встречающиеся повсюду, называются `well-known SID`.

Таблица 7.3 содержит краткий список самых распространенных SID.

MSDN, как авторитетный источник информации, содержит самый актуальный список well-known SID, используемых в операционных системах Windows.

Таблица 7.3. Распространенные well-known SID

Значение SID	Использование SID
S-1-1-0	Специальный SID, представляющий группу безопасности Everyone
S-1-5-18	Специальный SID, представляющий учетную запись LocalSystem
S-1-5-19	Специальный SID, представляющий учетную запись LocalService
S-1-5-20	Специальный SID, представляющий учетную запись NetworkService
S-1-5-6	Пользователь вошел как служба
S-1-5-2	Пользователь вошел через сеть
S-1-5-3	Пользователь вошел как пакетная учетная запись
S-1-5-4	Пользователь вошел интерактивно
S-1-5-5-X-Y	Идентифицирует пользовательский сеанс

Расширение показывает список SID, представляющих идентичность принципа маркера и группы безопасности, частью которых является этот принципал.

После этого расширение показывает список привилегий, предоставленных этому пользователю; некоторые из них включены.

Информация маркера устанавливается каждый раз, когда пользователь входит в систему, и остается неизменной на протяжении времени жизни сеанса входа.

Привилегии могут включаться или отключаться приложением, а также могут удаляться, но не добавляться в маркер.

Один и тот же принципал, прошедший проверку подлинности на разных системах, получает разную информацию маркера, членство в группах или предоставленные ему привилегии.

Взаимодействие этих концепций можно проиллюстрировать аналогией из реальной жизни.

Маркер доступа - это паспорт, используемый путешественниками, или принципами, для идентификации себя на разных границах.

Дескриптор безопасности представляет иммиграционное законодательство, используемое иммиграционным офицером посещаемой страны, которое описывает права и требования путешественника на основе страны происхождения.

Вся информация в паспорте, например страна происхождения или отметки, полученные в разных консульствах, может быть сопоставлена членству в группах маркера и привилегиям.

Иммиграционный агент, аналог кода, выполняющего проверку доступа, доверяет издателю паспорта - в данном случае операционной системе - и уверен (чего сложнее добиться в реальной жизни), что паспорт не подделан.

В зависимости от иммиграционного закона (дескриптора безопасности) путешественнику разрешается или запрещается право посетить страну (получить доступ к объекту).

В реальной жизни нет страны без иммиграционной политики, и программное обеспечение как минимум столь же защищено: каждый объект защищается дескриптором безопасности.

В реальной жизни управление документами, удостоверяющими личность, иммиграционными правилами и визами выполняется в узких кругах под строгим контролем.

Чтобы достичь такого же уровня доверия в операционных системах Windows, управление маркерами доступа выполняется исключительно компонентами trusted computing base, известными

как TCB.

Каждый компонент, работающий в TCB, считается доверенным операционной системой и, неявно, каждым пользователем системы безопасности.

Оставшаяся часть этой главы использует предыдущую информацию для исследования или решения различных случаев, в которых безопасность играет важную роль.

Источник информации безопасности

Чтобы безопасно ориентироваться в обширной области безопасности, инженерам нужны подсказки о том, где искать информацию безопасности и чего ожидать, когда они ее найдут.

Маркеры доступа

Где хранятся маркеры доступа и как их можно найти?

Операционная система Windows принудительно назначает первичный маркер доступа каждому процессу в системе.

Этот маркер идентифицирует принципал, создавший сеанс входа, в котором размещен процесс, и по умолчанию используется для всего доступа к объектам.

Адрес первичного маркера доступа доступен в структуре `nt!EPROCESS`, соответствующей каждому процессу.

Маркеры доступа процессов можно отображать как из отладчиков пользовательского режима, так и из отладчиков режима ядра с помощью команды расширения `!token`.

В отладчике пользовательского режима первичный маркер доступа автоматически отображается командой расширения `!token`, если текущий поток не олицетворяет другого пользователя.

В отладчике режима ядра адрес первичного маркера доступа является частью базовой информации о процессе, отображаемой командой расширения `!process`, как показано в листинге 7.6.

В листинге предполагается, что примерный процесс выполняется в системе.

Листинг 7.6. Первичный маркер доступа в выводе `!process`

```
kd> * The option 1 displays process basic information (Token, Stats)
kd> !process 0 1 07sample.exe
PROCESS 81136930 SessionId: 0 Cid: 045c Peb: 7ffd8000 ParentCid: 030c
DirBase: 0ae64000 ObjectTable: e13e5d38 HandleCount: 18.
Image: 07sample.exe
VadRoot 811eaa90 Vads 24 Clone 0 Private 50. Modified 0. Locked 0.
DeviceMap e164c948
Token e1424030
ElapsedTime 00:46:16.327
...
kd> * Token field contains the address of the primary access token
```

В клиент-серверном приложении операционная система Windows сильно полагается на олицетворение.

Олицетворение - это гибкий механизм, с помощью которого поток использует маркер доступа, отличный от первичного маркера доступа, для доступа ко всем объектам из этого потока.

Объект потока, представленный в ядре структурой `nt!ETHREAD`, содержит ссылку на маркер доступа олицетворения.

Базовая команда расширения `!thread` отображает явное сообщение, когда поток выполняет олицетворение, указывая маркер олицетворения и уровень олицетворения.

В листинге 7.7 используется основной поток `07sample.exe` сразу после возврата функции `ImpersonateSelf`.

Листинг 7.7. Маркер олицетворения потока

```
Using the kernel mode debugger
kd> * Displays the thread, referred by kernel thread object
kd> !thread ffad3020
THREAD ffad3020 Cid 045c.03f0 Teb: 7ffdf000 Win32Thread: 00000000 RUNNING on
processor 0
Impersonation token: e1424568 (Level Identification)
...
kd> * Token field contains the address of the impersonation token

Using the user mode debugger
0:000> !token -n
TS Session ID: 0
User: S-1-5-21-1060284298-2111687655-1957994488-1003 (User: XP-SP1\TestAdmin)
...
```

Когда поток не выполняет олицетворение, состояние олицетворения явно показано в дампе в листинге 7.8.

Все потоки в системе начинают свою жизнь в этом состоянии независимо от состояния олицетворения потока, который их создает.

Листинг 7.8. Поток без олицетворения

```
Using the kernel mode debugger
kd> !thread ffad3020
THREAD ffad3020 Cid 045c.03f0 Teb: 7ffdf000 Win32Thread: 00000000 RUNNING on
processor 0
Not impersonating
...
kd> * Token field is missing. The thread is in Not impersonating state

Using the user mode debugger
0:000> !token
Thread is not impersonating. Using process token
...
```

Наконец, маркеры доступа доступны как результат различных вызовов API, создающих или возвращающих дескрипторы маркеров доступа.

Если значение дескриптора известно, либо из вывода API, либо другими методами, такие маркеры доступа можно исследовать, как показано в листинге 7.5.

Когда поток олицетворяет маркер доступа, каждый native API использует эту идентичность для выполнения необходимых проверок доступа.

Если поток не олицетворен, вместо этого для каждой проверки доступа должен использоваться маркер доступа процесса, за одним заметным исключением.

В случае API `advapi32!OpenThreadToken` разработчик может выбрать эту идентичность между первичным маркером доступа процесса и маркером доступа олицетворения с помощью параметра `OpenAsSelf`.

Однако мы считаем, что любой маркер доступа всегда должен быть доступен процессу, который его использует.

Приложение пользовательского режима получает маркер доступа, используемый Security Reference Monitor, вызывая API `advapi32!OpenThreadToken` или `advapi32!OpenProcessToken`.

Те же API используются расширением пользовательского режима `exts.dll` при реализации команды расширения `!token`.

Когда команда расширения `!token` в отладчике пользовательского режима показывает отсутствие состояния олицетворения для потока, этот вывод следует воспринимать с осторожностью.

Расширение всегда возвращается к первичному маркеру, когда ему не удастся получить информацию об олицетворении, как мы покажем позже в разделах о `!token`.

Дескрипторы безопасности

Где хранятся дескрипторы безопасности?

Мы знаем, что все объекты защищаются присоединенным дескриптором безопасности, хранящимся в различных местах.

Все объекты ядра содержат общую структуру заголовка, предшествующую настоящему адресу памяти объекта.

Структура заголовка, называемая `_OBJECT_HEADER`, содержит, наряду со счетчиками ссылок и типом объекта, указатель на дескриптор безопасности, защищающий объект.

В листинге 7.9 мы используем другой выполняющийся экземпляр `02sample.exe`.

Объект процесса используется как начальная точка для получения заголовка объекта, содержащего указатель на дескриптор безопасности, защищающий этот объект.

Листинг 7.9. Указатель на дескриптор безопасности в `_OBJECT_HEADER`

```
kd> !process 0 0 07sample.exe
PROCESS ffbbc818 SessionId: 0 Cid: 01c4 Peb: 7ffde000 ParentCid: 00ac
DirBase: 0232e000 ObjectTable: e1112e10 HandleCount: 8.
Image: 07sample.exe
kd> !object ffbbc818
Object: ffbbc818 Type: (812ee900) Process
ObjectHeader: ffbbc800
HandleCount: 2 PointerCount: 7
kd> dt _OBJECT_HEADER ffbbc800
+0x000 PointerCount : 7
+0x004 HandleCount : 2
+0x004 NextToFree : 0x00000002
+0x008 Type : 0x812ee900 _OBJECT_TYPE
+0x00c NameInfoOffset : 0 ''
+0x00d HandleInfoOffset : 0 ''
+0x00e QuotaInfoOffset : 0 ''
+0x00f Flags : 0x20 ''
+0x010 ObjectCreateInfo : 0x812ca8e8 _OBJECT_CREATE_INFORMATION
+0x010 QuotaBlockCharged : 0x812ca8e8
+0x014 SecurityDescriptor : 0xe198bb92
+0x018 Body : _QUAD
```

Заголовок содержит псевдоуказатель на дескриптор безопасности объекта.

Псевдоуказатель использует последние три бита для хранения информации состояния, не связанной с адресом дескриптора безопасности.

Это возможно благодаря выравниванию памяти, используемому дескрипторами безопасности.

После маскирования младших значащих битов адрес указывает на допустимый дескриптор безопасности, который можно отобразить командой расширения `!sd`, как показано в листинге 7.10.

Листинг 7.10. Отображение дескриптора безопасности объекта

```
kd> !sd 0xe198bb92 & 0xFFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
          SE_DACL_PRESENT
          SE_SELF_RELATIVE
->Owner : S-1-5-21-1060284298-2111687655-1957994488-1003
->Group : S-1-5-21-1060284298-2111687655-1957994488-513
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x40
->Dacl : ->AceCount : 0x2
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x24
->Dacl : ->Ace[0]: ->Mask : 0x001f0fff
->Dacl : ->Ace[0]: ->SID: S-1-5-21-1060284298-2111687655-1957994488-1003
```

```

->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x14
->Dacl : ->Ace[1]: ->Mask : 0x001f0fff
->Dacl : ->Ace[1]: ->SID: S-1-5-18
->Sacl : is NULL

```

Поскольку адрес дескриптора безопасности хранится прямо перед адресом объекта, для упрощения операции получения дескриптора безопасности объекта все необходимые шаги можно объединить в одну строку следующим образом:

```
!sd poi(<object_address>-4) & FFFFFFFF8
```

Не все объекты, доступные в данный момент в памяти ядра, имеют дескриптор безопасности, к которому можно обратиться методом, описанным в листинге 7.10.

Постоянные объекты ядра, такие как файлы или разделы реестра, хранят дескриптор безопасности во вторичном хранилище и управляют доступом безопасности через собственный механизм.

Если мы смотрим на объект раздела реестра, можно увидеть, что его дескриптор безопасности равен NULL, что не позволяет нам статически исследовать дескриптор безопасности.

Чтобы продемонстрировать этот случай, мы использовали параметр 4 в примере, который открывает несколько разделов реестра.

Листинг 7.11. Раздел реестра с недоступным дескриптором безопасности в заголовке объекта

```

kd> k4
ChildEBP RetAddr
0006ff00 01001f33 07sample!Sample4Get+0x45
0006ff18 01001e48 07sample!AppInfo::Loop+0xb3
0006ff7c 01002aa6 07sample!wmain+0xa8
0006ffc0 7c816fd7 07sample!__wmainCRTStartup+0x102
kd> dv *key
softwareKey = 0x000007f4
bookKey = 0x77c2ed0e
kd> !handle 7f4
processor number 0, process ffb8c818
PROCESS ffb8c818 SessionId: 0 Cid: 01c4 Peb: 7ffde000 ParentCid: 00ac
DirBase: 0232e000 ObjectTable: e1112e10 HandleCount: 9.
Image: 07sample.exe
Handle table at e122f000 with 9 Entries in use
07f4: Object: e18cce60 GrantedAccess: 00020019 Entry: e122ffe8
Object: e18cce60 Type: (812e4e70) Key
ObjectHeader: e18cce48
HandleCount: 1 PointerCount: 1
Directory Object: 00000000 Name: \REGISTRY\MACHINE\SOFTWARE
kd> dt _OBJECT_HEADER e18cce48
+0x000 PointerCount : 1
+0x004 HandleCount : 1
+0x004 NextToFree : 0x00000001
+0x008 Type : 0x812e4e70 _OBJECT_TYPE
+0x00c NameInfoOffset : 0 ''
+0x00d HandleInfoOffset : 0 ''
+0x00e QuotaInfoOffset : 0 ''
+0x00f Flags : 0 ''
+0x010 ObjectCreateInfo : 0x812ca8e8 _OBJECT_CREATE_INFORMATION
+0x010 QuotaBlockCharged : 0x812ca8e8
+0x014 SecurityDescriptor : (null)
+0x018 Body : _QUAD

```

Когда дескриптор безопасности нелегко доступен для исследования, его значение можно проверить в тот момент, когда брокер объекта выполняет проверку доступа.

Все остальные компоненты пользовательского режима, предоставляющие объекты, не управляемые ядром (например, Service Control Manager), также используют собственный механизм управления своими дескрипторами безопасности.

Как выполняется проверка безопасности?

Чтобы обеспечить согласованные правила доступа между компонентами Windows, ядро реализует набор API безопасности с сигнатурой, опубликованной в заголовочном файле `ntddk.h`.

Центральной функцией является функция ядра `SeAccessCheck`, используемая компонентами пользовательского режима через API `advapi32!AccessCheck`.

`SeAccessCheck` принимает в качестве параметров дескриптор безопасности, маркер доступа (в параметре `SubjectSecurityContext`) и запрошенный доступ.

```
BOOLEAN SeAccessCheck (
    IN PSECURITY_DESCRIPTOR SecurityDescriptor,
    IN PSECURITY_SUBJECT_CONTEXT SubjectSecurityContext,
    IN BOOLEAN SubjectContextLocked,
    IN ACCESS_MASK DesiredAccess,
    IN ACCESS_MASK PreviouslyGrantedAccess,
    OUT PPRIVILEGE_SET *Privileges OPTIONAL,
    IN PGENERIC_MAPPING GenericMapping,
    IN KPROCESSOR_MODE AccessMode,
    OUT PACCESS_MASK GrantedAccess,
    OUT PNTSTATUS AccessStatus);
```

Доступ, предоставленный кодом пользовательского режима, можно легко определить в отладчике, исследуя возвращаемое значение и выходные параметры, заполненные API `advapi32!AccessCheck`.

Доступ, предоставленный кодом режима ядра, можно определить, исследуя возврат из API ядра `SeAccessCheck`.

Чтобы выявлять проблемы доступа, вызванные неправильными параметрами безопасности для различных файлов и разделов реестра, мы также можем использовать средства трассировки, такие как Process Monitor, бесплатно предоставляемые Microsoft.

Распространение идентичности в клиент-серверных приложениях

Большинство приложений использует первичный маркер доступа для всех операций.

Клиент-серверные приложения часто используют модель олицетворения, в которой сервер выполняет большинство, если не все, запросы клиента в контексте маркера доступа олицетворения, полученного от этого клиента.

Маркер доступа олицетворения распространяется специальной функциональностью, предоставляемой инфраструктурой межпроцессного взаимодействия, используемой для поддержки клиент-серверного разговора.

Функции олицетворения, такие как `ntdll!NtImpersonateClientOfPort`, предоставляемая механизмом взаимодействия LPC; `rpcrt4!RpcImpersonateClient`, реализованная инфраструктурой RPC; и `advapi32!ImpersonateNamedPipeClient`, реализованная перенаправителем файловой системы, олицетворяют вызывающий поток с маркером доступа клиента, использованным для вызова сервера через соответствующие средства.

В некоторых случаях учетные данные пользователя доступны на стороне сервера, особенно в веб-приложениях, и сервер создает маркер доступа, напрямую вызывая `advapi32!LogonUser(Ex)W`.

Каждый протокол использует собственный механизм для распространения идентичности клиента. Когда клиент и сервер находятся на разных системах, для распространения информации безопасности в клиент-серверных приложениях может использоваться Security Server Provider Interface (SSPI).

`rpcrt4!RpcImpersonateClient` - это специальная "proxy"-функция, которая делегирует запрос олицетворения базовому механизму связи, используемому RPC для данного соединения.

Когда RPC используется для взаимодействия между двумя процессами, находящимися в одной системе, вызов использует функции LPC для достижения результата.

Когда клиент выполняется на другой системе, отличной от сервера, RPC использует либо функциональность перенаправителя файловой системы в случае удаленных вызовов с использованием транспортной безопасности, либо функциональность SSPI в подавляющем большинстве случаев.

Удаленная проверка подлинности и Security Support Provider Interface

У клиента есть набор учетных данных, которые должны быть представлены серверу.

Эти учетные данные используются для представления принципа клиента в системе сервера.

SSPI используется для проверки удаленных учетных данных через различные поставщики безопасности, такие как проверка подлинности NTLM, доменная проверка подлинности Kerberos или проверка подлинности по клиентскому сертификату.

Чтобы пройти проверку подлинности на удаленной системе, клиент начинает последовательность вызовов, передавая набор учетных данных в API secur32!InitializeSecurityContextW.

Непрозрачный blob данных, полученный в результате этого вызова, отправляется по сетевому протоколу серверу.

Сервер берет этот blob и передает его API secur32!AcceptSecurityContext, который генерирует еще один непрозрачный блок данных и сообщает серверу, завершена ли проверка подлинности.

Если нет, сгенерированный сервером блок затем отправляется клиенту, который использует его как параметр для еще одного вызова secur32!InitializeSecurityContextW.

Получившийся blob данных отправляется обратно серверу, и процесс повторяется несколько раз, пока пакет безопасности, используемый для проверки подлинности, не сможет проверить учетные данные.

Когда обмен сообщениями завершен, сервер вызывает secur32!ImpersonateSecurityContext с последним blob данных, чтобы олицетворить клиента.

Эту последовательность вызовов часто называют последовательностью ISC/ASC.

В главе 8 показано, как такая удаленная проверка подлинности выглядит "на проводе".

Листинг 7.12 захвачен из серверного процесса до того, как удаленный клиент устанавливает соединение с сервером.

Код возврата каждого вызова secur32!AcceptSecurityContext является важной подсказкой о том, как продвигается ISC/ASC, а каждая ошибка, обнаруженная соответствующим пакетом проверки подлинности, является отличной подсказкой для понимания того, почему удаленная проверка подлинности терпит неудачу, когда это происходит, - подсказкой, часто теряемой высокоуровневым API, использующим SSPI.

Листинг 7.12. Точки останова на AcceptSecurityContext и ImpersonateSecurityContext

```
0:009> bp Secur32!AcceptSecurityContext
0:009> bp Secur32!ImpersonateSecurityContext
0:003> g
...
Breakpoint 0 hit
eax=0009be20 ebx=00000000 ecx=0009722c edx=76f9d1e0 esi=00097220 edi=000000a6
eip=76f949ba esp=005bfe68 ebp=005bfea8 iopl=0 nv up ei pl nz na pe nc
Secur32!AcceptSecurityContext:
76f949ba 55 push ebp
0:003> k
ChildEBP RetAddr
005bfe64 78023b9f Secur32!AcceptSecurityContext
005bfea8 78023b22 RPCRT4!SECURITY_CONTEXT::AcceptThirdLeg+0x3e
005bff18 78004aed RPCRT4!OSF_SCONNECTION::ProcessReceiveComplete+0x595
005bff28 78001848 RPCRT4!ProcessConnectionServerReceivedEvent+0x20
0:003> * Third Leg is a concept used in NTLM authentication
```

```

0:003> g
Breakpoint 1 hit
eax=76f9d1e0 ebx=005bf83c ecx=0009722c edx=75867028 esi=000971e0 edi=005bf848
eip=76f95099 esp=005bf75c ebp=005bf768 iopl=0 nv up ei pl nz na pe nc
Secur32!ImpersonateSecurityContext:
76f95099 55 push ebp
0:003> k
ChildEBP RetAddr
005bf758 7802372a Secur32!ImpersonateSecurityContext
005bf768 78023701 RPCRT4!SECURITY_CONTEXT::ImpersonateClient+0x39
005bf770 78004443 RPCRT4!OSF_SCONNECTION::ImpersonateClient+0x3b
005bf778 75852a8f RPCRT4!RpcImpersonateClient+0x64
0:003> * The RPCImpersonateClient function uses the SSPI function

```

После успешного выполнения всех показанных ранее функций вызывающий поток затем олицетворяет маркер доступа олицетворения клиента. Возврат из API secur32!ImpersonateSecurityContext - отличное место для установки точек останова при исследовании безопасности после того, как сервер выполняет функцию олицетворения:

```

0:003> gu
eax=00000000 ebx=005bf83c ecx=c000023c edx=7ffe0304 esi=000971e0 edi=005bf848
eip=7802372a esp=005bf764 ebp=005bf768 iopl=0 nv up ei pl zr na po nc
RPCRT4!SECURITY_CONTEXT::ImpersonateClient+0x39:
7802372a 85c0 test eax,eax

```

После проверки кода возврата, который согласно MSDN указывает на успешное олицетворение, проверьте маркер доступа олицетворения потока с помощью команды расширения !token, как показано в листинге 7.13.

Листинг 7.13. Проверка маркера после олицетворения

```

0:003> !token -n
TS Session ID: 0
User: S-1-5-21-1060284298-2111687655-1957994488-1003 (User: XP-SP1\TestAdmin)
...
Auth ID: 0:2780c
Impersonation Level: Impersonation
TokenType: Impersonation

```

После олицетворения поток может вернуться в состояние без олицетворения с помощью функции отката, обычно соответствующей методу олицетворения; обе функции находятся в MSDN на одной странице.

Другая распространенная функция олицетворения - advapi32!SetThreadToken, используемая, когда сервер уже имеет дескриптор маркера доступа клиента, полученный другими средствами. Это часто применяется, когда сервер хранит кэш маркеров доступа и управляет их использованием.

advapi32!ImpersonateSelf - еще один API, используемый в ситуации, когда потоку нужно использовать маркер, похожий на первичный доступ, но с другим членством в группах или списком включенных привилегий.

Уровень олицетворения

Еще один интересный компонент маркера доступа, как было видно раньше, - его ImpersonationLevel.

Уровень олицетворения - это ограничение, накладываемое клиентом на использование маркера доступа сервером, и это ограничение принудительно обеспечивается операционной системой. Поток, олицетворяющий маркер доступа на уровне олицетворения ниже SecurityImpersonation, неспособен получить какой-либо защищенный ресурс в системе, где выполняется серверный процесс.

Чтобы показать важность уровня олицетворения, пример в листинге 7.14 выполняет несколько вызовов API `GetComputerNameEx`, олицетворяя первичный маркер доступа на разных уровнях олицетворения.

Эту функцию можно задействовать с помощью параметра 1 в `07sample.exe`.

Листинг 7.14. Вызовы `GetComputerNameEx` при разных уровнях олицетворения

```
void Sample1()
{
    WCHAR computerName[MAX_PATH];
    DWORD arrayLength = MAX_PATH;
    BOOL retCode = TRUE;
    ImpersonateSelf(SecurityAnonymous);
    retCode = GetComputerNameEx(ComputerNameNetBIOS, computerName, &arrayLength);
    RevertToSelf();
    ...
    ImpersonateSelf(SecurityDelegation);
    retCode = GetComputerNameEx(ComputerNameNetBIOS, computerName, &arrayLength);
    RevertToSelf();
    if (retCode != TRUE)
    {
        TRACE(L"GetComputerName fails with token @ SecurityDelegation.");
    }
}
```

Следующий вывод показывает результаты выполнения, которое завершается неудачей, когда уровень олицетворения установлен в `SecurityAnonymous` или `SecurityIdentify`:

```
GetComputerName fails with token @ SecurityAnonymous.Last error = 1346
GetComputerName fails with token @ SecurityIdentification.Last error = 1346
```

Быстрый взгляд в заголовочный файл `winerror.h` показывает, что ошибка 1346L является ошибкой `ERROR_BAD_IMPERSONATION_LEVEL`.

Код ошибки также можно расшифровать с помощью командной строки `net helpmsg <error>` или команды расширения `!error`.

Проверки безопасности на границах системы

Сегодня даже более простые приложения имеют сложные взаимодействия с компонентами операционной системы, выполняющимися в различных контекстах.

Например, когда вы тестируете приложение в ограниченном контексте безопасности, приложение не может открыть файл или записать ошибки в Event Log.

Как начать его отладку?

В следующем разделе мы оцениваем несколько распространенных сценариев, вызванных проверками безопасности или встречающихся в простых приложениях либо компонентах операционной системы, с целью создать отладочный каркас, который можно применять в других контекстах.

Перед началом нужно понять базовые ворота безопасности, используемые операционной системой.

В Windows много границ безопасности, определенных и принудительно обеспечиваемых операционной системой, и каждый переход внутрь этих границ безопасности или наружу из них подвергается проверкам безопасности.

Мы можем легко определить распространенные границы, такие как файловая система, реестр Windows, адресное пространство каждого процесса, адресное пространство ядра, тогда как другие, например рабочий стол Windows, не столь очевидны.

Машина является физической границей безопасности, но также является и логической границей безопасности.

В результате каждый API потенциально может проверять идентичность вызывающего кода и отклонять вызов согласно политике безопасности, реализованной в этом API.

Успешный подход к исследованию отказов безопасности требует хорошего знания каждого API, чего трудно, если вообще возможно, достичь без доступа к исходному коду и большого времени, потраченного на понимание этого кода.

В действительности только разработчики API понимают код на уровне, позволяющем эффективно точно указать проблему.

Поскольку знать детали каждого API непрактично, каков минимум, необходимый для успешного исследования проблем безопасности?

Разработчикам нужно минимальное понимание используемой подсистемы и мест, где проверки безопасности, скорее всего, выполняются при использовании API этой подсистемы.

Им также нужно знать, как исследовать результаты этих проверок.

Если выполнение кода не вызывает другой процесс, код режима ядра будет единственным диспетчером ресурсов, отказывающим в доступе к ресурсам.

Обратите внимание, что многие Win32 API взаимодействуют с разными процессами, чтобы реализовать свою функциональность.

Когда выполнение кода продолжается в другом процессе, ворота доступа, через которые оно должно пройти, становятся практически бесконечными, потому что этот вызов может породить множество процессов и даже множество систем.

Например, базовая трехуровневая система с общей архитектурой, показанной на рисунке 7.1, использующая веб-сервер на front end, любое промежуточное ПО на среднем уровне и базу данных на back end, имеет много потенциальных точек отказа, связанных с безопасностью.

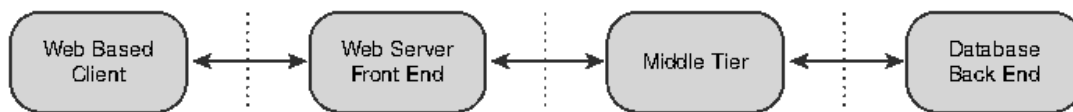


Figure 7.1 Typical multitier application

Рисунок 7.1. Типичное многоуровневое приложение

На рисунке 7.1 каждый прямоугольник может выполняться на одной или нескольких системах, соединенных разными механизмами связи.

Каждая часть, участвующая в этой архитектуре, может проверять идентичность пользователя и отклонять вызов.

В следующем разделе исследуются несколько сценариев отказов, встречающихся в распределенных средах, где существует много возможностей для ошибок.

Исследование отказов безопасности

Сеансы отладки, показанные в этом разделе и встречающиеся в разных системах, всегда запускаются ошибками отказа в доступе.

Иногда отказ в доступе нормален и ожидаем.

В других случаях ошибки нормальны, но неожиданны даже в правильно настроенной системе.

Тем не менее отлаживать отказ в правильно настроенной системе гораздо проще, чем в неверно настроенной системе, как показано в последнем сценарии отладки в этом разделе.

Первые несколько примеров - это классические ресурсы ядра с отказом доступа, за которыми следуют более сложные распределенные сценарии, использующие DCOM как инфраструктуру связи.

Локальные отказы безопасности

Неожиданные отказы различных API являются одним из крупнейших источников раздражения при разработке программного обеспечения, особенно когда отказ полностью противоречит ожиданиям или опыту разработчика.

Попытка понять, почему такой API терпит неудачу, всегда оказывается сложной задачей - более сложной, чем должна быть, особенно когда отказ неожиданный.

Распространенный случай отказа встречается, когда процессы выполняются под учетной записью NetworkService, идентифицированной S-1-5-20, или под учетной записью LocalService, идентифицированной S-1-5-19.

Пример в этом разделе основан на реальной ситуации, но был встречен во время экспериментов с побочными эффектами вызова advapi32!ImpersonateSelf процессом, выполняющимся под учетной записью NetworkService.

Чтобы сэкономить время, мы решили использовать один из временных процессов, выполняющихся под этой учетной записью, и подключили отладчик к процессу, выполняющемуся под этой идентичностью, которую можно определить с помощью Task Manager.

В потоке, используемом отладчиком для вызова kernel32!DebugBreak, мы меняем указатель инструкций на адрес advapi32!ImpersonateSelf и заполняем параметры в стеке.

Команды, изменяющие контекст, показаны в первой части листинга 7.15.

После выполнения API advapi32!ImpersonateSelf мы используем команду расширения !token, чтобы выяснить поток олицетворения потока.

Команда расширения !token указывает, что поток не выполняет олицетворение.

Последняя ошибка указывает, что API завершился с совершенно неожиданной ошибкой отказа в доступе.

Как понять, почему этот вызов функции завершился неудачей?

Листинг 7.15. Неудачный вызов ImpersonateSelf из процесса NetworkService

```
0:008> |
. 0 id: 650 attach name: C:\WINDOWS\System32\wbem\wmiprvse.exe
0:008> * set the instruction pointer to the advapi32!ImpersonateSelf
0:008> r $ip=advapi32!ImpersonateSelf
0:008> * enter the argument to the API
0:008> ed esp+4 2
0:008> gu
eax=00000000 ebx=00000001 ecx=00000005 edx=00000015 esi=00000004 edi=00000005
eip=7c9507a8 esp=00a9ffd4 ebp=00a9fff4 iopl=0 nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000246
ntdll!DbgUiRemoteBreakin+0x2d:
7c9507a8 eb11 jmp ntdll!DbgUiRemoteBreakin+0x40 (7c9507bb)
0:008> !token
Thread is not impersonating. Using process token...
Error 0xc0000022 getting thread token !token command failed
0:008> ~.
. 8 Id: 650.334 Suspend: 1 Teb: 7ffd7000 Unfrozen
Start: ntdll!DbgUiRemoteBreakin (7c95077b)
Priority: 0 Priority class: 32
0:008> !gle
LastErrorMessage: (Win32) 0x5 (5) - Access is denied.
LastStatusValue: (NTSTATUS) 0xc0000022 - {Access Denied} A process has requested
access to an object, but has not been granted those access rights.
```

В качестве попутного замечания интересно отметить, что одна и та же логическая ошибка имеет несколько кодов ошибок в зависимости от подсистемы, которая ее использует.

Например, однозначная ошибка отказа в доступе может иметь разные значения, как показано в таблице 7.4.

Таблица 7.4. Представления ошибки отказа в доступе

Компонент	Определено в	Символическое имя	Значение
Windows NT Kernel	winnt.h	STATUS_ACCESS_DENIED	((NTSTATUS)0xC0000022L)
Ntdll.dll	winnt.h	STATUS_ACCESS_DENIED	((NTSTATUS)0xC0000022L)
Win32 APIs	winerror.h	ERROR_ACCESS_DENIED	5L
COM APIs	winerror.h	E_ACCESSDENIED	_HRESULT_TYPEDEF_(0x80070005L)
RPC APIs	winerror.h	RPC_E_ACCESS_DENIED	_HRESULT_TYPEDEF_(0x8001011BL)

При отладке этого сценария мы поняли, что команда расширения !token также завершается с ошибкой отказа в доступе, но, по-видимому, результат корректен.

Причину этого отказа мы исследуем позже в разделе "!token Extension Command Failure".

Нужно сосредоточиться на настоящей проблеме: выяснить, почему функция `advapi32!ImpersonateSelf` завершается неудачей.

Первый шаг - понять, что `advapi32!ImpersonateSelf` делает под капотом.

Согласно объяснению, найденному в MSDN, API создает маркер доступа олицетворения, дублируя первичный маркер доступа на запрошенном уровне олицетворения, и устанавливает его на текущий поток.

В псевдокоде функциональность API выглядит следующим образом:

```
ImpersonateSelf(ImpersonationLevel)
{
    processHandle = OpenCurrentProcess()
    processToken = OpenProcessToken(processHandle, TOKEN_DUPLICATE);
    newToken = DuplicateToken(processToken, ImpersonationLevel)
    SetThreadToken(newToken)
}
```

Каждый шаг из показанного ранее псевдокода подвергается как минимум одной проверке безопасности, поскольку все участвующие объекты защищены ядром Windows.

Чтобы первый шаг завершился успешно, объект процесса должен предоставить право `PROCESS_QUERY_INFORMATION` пользователю, выполняющему вызов, - в данном случае учетной записи `NetworkService`.

Затем первичный маркер доступа должен предоставить вызывающему пользователю право `TOKEN_DUPLICATE` в своем дескрипторе безопасности.

Последний шаг требует, чтобы пользователь имел права `THREAD_SET_THREAD_TOKEN` на объект потока.

Эта очень простая функция проверяет три дескриптора безопасности:

- Дескриптор безопасности объекта процесса.
- Дескриптор безопасности первичного маркера.
- Дескриптор безопасности объекта потока.

Поскольку поток ни в какой момент не выполняет олицетворение, все вызовы выполняются в контексте первичного маркера, то есть учетной записи `NetworkService`, которая должна иметь доступ с указанными выше конкретными правами в соответствующих дескрипторах безопасности.

Прежде чем искать другие причины этого отказа, нужно исследовать каждый дескриптор безопасности, участвующий в операции, и понять, какие права предоставлены пользователю. Самый простой способ проверить их - запустить отладчик режима ядра в локальном режиме и исследовать каждый объект.

Начнем с объекта процесса, идентификатор которого был получен в листинге 7.15.

Дескриптор безопасности объекта процесса исследуется в листинге 7.16.

Листинг 7.16. Дескриптор безопасности объекта процесса

```

lkd> !process 650 1
Searching for Process with Cid == 650
PROCESS ffacccc8 SessionId: 0 Cid: 0650 Peb: 7ffd5000 ParentCid: 02d0
DirBase: 0b233000 ObjectTable: e120ddc0 HandleCount: 164.
Image: wmiprvse.exe
VadRoot 811c2790 Vads 102 Clone 0 Private 416. Modified 0. Locked 1.
DeviceMap e15f04a8
Token e1b3db20
...
lkd> !sd poi(ffacccc8-4)&FFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
                SE_DACL_PRESENT
                SE_SELF_RELATIVE
->Owner : S-1-5-20
->Group : S-1-5-20
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x58
->Dacl : ->AceCount : 0x3
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x18
->Dacl : ->Ace[0]: ->Mask : 0x001f0fff
->Dacl : ->Ace[0]: ->SID: S-1-5-20
->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x20
->Dacl : ->Ace[1]: ->Mask : 0x00100201
->Dacl : ->Ace[1]: ->SID: S-1-5-5-0-32366
->Dacl : ->Ace[2]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[2]: ->AceFlags: 0x0
->Dacl : ->Ace[2]: ->AceSize: 0x18
->Dacl : ->Ace[2]: ->Mask : 0x00100201
->Dacl : ->Ace[2]: ->SID: S-1-5-18
->Sacl : is NULL

```

Интерпретируя биты доступа в маске доступа, используемой для пользователя S-1-5-20, мы заключаем, что NetworkService имеет полные права на объект процесса, как и ожидалось. Первичный маркер доступа, полученный в предыдущем листинге, является еще одним объектом, участвующим в реализации API, и защищен своим дескриптором безопасности, как показано в листинге 7.17.

Листинг 7.17. Дескриптор безопасности первичного маркера доступа

```

lkd> !sd poi(e1b3db20-4)&FFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
                SE_DACL_PRESENT
                SE_SELF_RELATIVE
->Owner : S-1-5-20
->Group : S-1-5-20
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x30
->Dacl : ->AceCount : 0x2
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x14
->Dacl : ->Ace[0]: ->Mask : 0x000f01ff
->Dacl : ->Ace[0]: ->SID: S-1-5-18
->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x14

```

```

->Dacl : ->Ace[1]: ->Mask : 0x000f01ff
->Dacl : ->Ace[1]: ->SID: S-1-5-20
->Sacl : is NULL

```

Как и раньше, интерпретируя биты доступа в маске доступа, используемой для пользователя S-1-5-20, мы заключаем, что NetworkService имеет полные права на первичный маркер доступа, как и ожидалось.

Сам поток является последним объектом ядра, участвующим в операции, и подчиняется тем же правилам, которые управляют безопасностью Windows.

Следуя тем же шагам, дескриптор безопасности вызывающего потока можно легко получить. Но сначала нужно идентифицировать объект ядра, представляющий отказавший поток: мы сопоставляем идентификатор потока 0650.0334 из отладчика пользовательского режима со структурой KTHREAD в отладчике режима ядра.

Идентификатор процесса и идентификатор потока были известны из сеанса отладчика пользовательского режима, в котором наблюдался этот отказ.

Листинг 7.18. Дескриптор безопасности отказавшего потока

```

lkd> * List all threads running inside the process with 0x0650 PID
lkd> !process 0n1616 4
Searching for Process with Cid == 650
PROCESS ffacccc8 SessionId: 0 Cid: 0650 Peb: 7ffd5000 ParentCid: 02d0
DirBase: 0b233000 ObjectTable: e120ddc0 HandleCount: 164.
Image: wmiprvse.exe
THREAD fface088 Cid 0650.0658 Teb: 7ffdf000 Win32Thread: e1226650 WAIT
THREAD 8125b020 Cid 0650.04dc Teb: 7ffde000 Win32Thread: 00000000 WAIT
THREAD ffadb100 Cid 0650.064c Teb: 7ffdd000 Win32Thread: e1345138 WAIT
THREAD ffb25408 Cid 0650.0654 Teb: 7ffdc000 Win32Thread: 00000000 WAIT
THREAD 811c6b30 Cid 0650.03b4 Teb: 7ffdb000 Win32Thread: e1b4ebf0 WAIT
THREAD ffb47b18 Cid 0650.05f4 Teb: 7ffda000 Win32Thread: e13482b0 WAIT
THREAD 811c2da8 Cid 0650.05f8 Teb: 7ffd9000 Win32Thread: 00000000 WAIT
THREAD ffacaaa0 Cid 0650.0570 Teb: 7ffd8000 Win32Thread: 00000000 WAIT
THREAD ffb2a020 Cid 0650.0334 Teb: 7ffd7000 Win32Thread: 00000000 WAIT
lkd> *Inspecting the security descriptor protecting this kernel object
kd> !sd poi(ffb2a020-4)&FFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
            SE_DACL_PRESENT
            SE_SELF_RELATIVE
->Owner : S-1-5-32-544
->Group : S-1-5-21-1060284298-2111687655-1957994488-513
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x34
->Dacl : ->AceCount : 0x2
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x18
->Dacl : ->Ace[0]: ->Mask : 0x001f03ff
->Dacl : ->Ace[0]: ->SID: S-1-5-32-544
->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x14
->Dacl : ->Ace[1]: ->Mask : 0x001f03ff
->Dacl : ->Ace[1]: ->SID: S-1-5-18
->Sacl : is NULL

```

Удивительно, но NetworkService не имеет доступа к объекту потока.

Изучив его, мы видим, что только пользователи локальной группы администраторов, идентифицированной S-1-5-32-544, и учетная запись LocalSystem, идентифицированная S-1-5-18, могут изменить маркер олицетворения потока, что объясняет отказ API.

В таких случаях мы часто смотрим на похожие объекты, чтобы понять отличие и построить теорию, объясняющую отказ.

Мы выбираем другой поток в том же процессе с адресом, показанным в листинге 7.18.

Дескрипторы безопасности, показанные в листинге 7.18 и листинге 7.19, отличаются только одной ACE: отказавший поток предоставляет все права S-1-5-32-544, тогда как нормальный поток предоставляет те же права S-1-5-20.

Листинг 7.19. Дескриптор безопасности нормального потока

```
kd> !sd poi(ffacaaa0-4)&FFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
           SE_DACL_PRESENT
           SE_SELF_RELATIVE
->Owner : S-1-5-20
->Group : S-1-5-20
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x30
->Dacl : ->AceCount : 0x2
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x14
->Dacl : ->Ace[0]: ->Mask : 0x001f03ff
->Dacl : ->Ace[0]: ->SID: S-1-5-18
->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x14
->Dacl : ->Ace[1]: ->Mask : 0x001f03ff
->Dacl : ->Ace[1]: ->SID: S-1-5-20
->Sacl : is NULL
```

Это можно объяснить, поняв, как дескриптор безопасности был изначально назначен объекту потока.

Оказывается, этот поток был создан процессом, выполнявшимся под идентичностью локального администратора, и к потоку был применен дескриптор безопасности по умолчанию.

Поток был создан в целевом процессе отладчиком с помощью `kernel32!CreateRemoteThread`, когда сам отладчик выполнялся под учетной записью локального администратора.

Хотя этот пример выглядит неестественным, такое вполне может произойти в любом приложении. Важно помнить о сложности каждого API и последствиях его вызова при олицетворении пользователя, отличного от пользователя первичного маркера.

Следующий раздел, "Security Problems During Deferred Initialization", описывает другие ситуации, создаваемые похожими обстоятельствами.

Проблемы безопасности во время отложенной инициализации

Методика ленивой инициализации откладывает инициализацию дорогих объектов настолько, насколько возможно, с целью улучшить время запуска и уменьшить объем памяти до использования компонента.

Чтобы достичь еще большей масштабируемости, проектировщики компонентов даже деинициализируют компонент после периода затухания, определенного как часть исходного проекта.

Они полагаются на методику ленивой инициализации, чтобы вернуть компонент к жизни, когда он понадобится.

В клиент-серверном приложении фаза ленивой инициализации запускается запросом клиента и подчиняется всем правилам безопасности, принудительно обеспечиваемым операционной системой.

Все компоненты, участвующие в ленивой инициализации, могут играть роль в процессе и должны рассматриваться очень осторожно.

Маркер олицетворения потока и его уровень олицетворения, а также потенциальное олицетворение потока могут повлиять на общую функциональность системы или внести тонкие функциональные ошибки, которые трудно найти.

Пример имитирует олицетворение, создавая и олицетворяя маркер доступа, представляющий обычного пользователя.

Пользователь с именем Test1 и паролем TestUser1 должен быть создан вручную перед запуском примера и удален, когда пример больше не используется.

Давайте проанализируем следующий код, который имеет несколько целей.

Он создает новый раздел в HKLM\Software, кэширует маркер процесса для дальнейшего использования и создает событие ядра, используемое для синхронизации доступа к тем же глобальным объектам.

Этот код можно задействовать с помощью параметра 2 в 07sample.exe.

Мы используем эту функцию для имитации побочного эффекта ее выполнения во время олицетворения.

Такой тип функциональности часто встречается в функциях инициализации служб.

Листинг 7.20. Отложенная инициализация под олицетворением

```
void LazyInitialization()
{
    HKEY softwareKey = NULL;
    LONG retCode = RegOpenKeyEx(HKEY_LOCAL_MACHINE, L"Software", 0, MAXIMUM_ALLOWED,
        &softwareKey);
    ...
    HKEY bookKey = NULL;
    retCode = RegCreateKey(bookKey, L"Advanced Windows Debugging", &bookKey);
    ...
    RegCloseKey(bookKey);
    RegCloseKey(softwareKey);
    BOOL otherCode = ImpersonateSelf(SecurityImpersonation);
    ...
    HANDLE threadToken = NULL;
    otherCode = OpenThreadToken(GetCurrentThread(), TOKEN_QUERY, FALSE, &threadToken);
    ...
    if (threadToken) CloseHandle(threadToken);
    HANDLE event = CreateEvent(NULL, FALSE, FALSE, L"07sample");
    CloseHandle(event);
    HANDLE threadTokenAsSelf = NULL;
    otherCode = OpenThreadToken(GetCurrentThread(), TOKEN_QUERY | TOKEN_IMPERSONATE,
        TRUE, &threadTokenAsSelf);
    ...
    RevertToSelf();
    otherCode = ImpersonateLoggedOnUser(threadTokenAsSelf);
    ...
    if (threadTokenAsSelf) CloseHandle(threadTokenAsSelf);
    RevertToSelf();
}
```

Поскольку тесты продукта хороши и в этом коде нет очевидных ошибок, код включается в продукт и затем выпускается.

Вскоре после этого клиент сообщает, что приложение завершается с одной из следующих ошибок в файле журнала, которые пример выводит на экран следующим образом:

```
RegCreateKeyW failed.Last error = 6
ImpersonateSelf failed.Last error = 5
OpenThreadToken failed.Last error = 5
```

Наряду с уже известным кодом отказа в доступе 5 мы видим неожиданную ошибку недопустимого дескриптора 6, поступающую от API реестра.

Сопоставив все места, где раздел используется или создается, мы выясняем, что сбойный код находится в пути ленивой инициализации.

Он запускается запросом клиента, который выполняется в потоке запроса клиента, пока поток олицетворяет пользователя.

Мы смоделировали олицетворение в простом клиентском приложении, войдя под конкретным пользователем, олицетворив его и вызвав функцию `LazyInitialization`, как показано ниже:

```
void Sample2()
{
    HANDLE userToken = NULL;
    BOOL retCode = LogonUser(L"Test1", NULL, L"TestUser1", LOGON32_LOGON_INTERACTIVE,
        LOGON32_PROVIDER_DEFAULT, &userToken);
    ...
    ImpersonateLoggedOnUser(userToken);
    LazyInitialization();
    RevertToSelf();
    CloseHandle(userToken);
}
```

Поскольку ревью кода не выявляет источник отказа, мы запустим этот код под отладчиком пользовательского режима, чтобы полностью понять, что идет не так.

Сразу после выполнения первой строки отказа, то есть `API advapi32!RegCreateKey`, мы исследуем значение дескриптора, переданное как первый параметр, с помощью команды расширения `!handle`.

Мы выбираем этот параметр, потому что API реестра возвращает ошибку "invalid handle".

```
0:000> !handle poi(softwareKey) 7
Handle 58
Type Key
Attributes 0
GrantedAccess 0x20019:
ReadControl
QueryValue,EnumSubKey,Notify
HandleCount 2
PointerCount 3
Name \REGISTRY\MACHINE\SOFTWARE
0:000> * The !handle command decodes the rights granted to the caller
```

Мы замечаем, что API реестра не предоставил `softwareKey` права на создание нового раздела. Диспетчер безопасности предоставляет права на объекты при открытии объекта на основе его дескриптора безопасности и запрошенной маски доступа.

Предоставленный доступ, сохраненный в таблице дескрипторов вместе с дескриптором, проверяется каждой операцией, использующей этот дескриптор, на допустимость.

Маска доступа, связанная с дескриптором, отображается командой расширения `!handle`, как показано в предыдущем листинге.

В этом случае раздел был открыт во время олицетворения пользователя с низкими привилегиями.

Снова читая код, мы видим, что запрошенная маска, использованная для открытия раздела реестра, равна `MAXIMUM_ALLOWED`, удобному определению маски доступа, которым пользуются все. Возможно, у разработчика не было времени или желания выяснять необходимые права, и он не хотел оправдывать использование `GENERIC_ALL`.

Система действительно возвращает то, что запрашивает код, но предоставленный доступ отличается от того, что имел в виду разработчик.

В качестве попутного замечания: `MAXIMUM_ALLOWED` следует использовать только для пробного определения разрешенного доступа к объекту.

Использование его где-либо еще - дефект кода, ожидающий момента, чтобы проявиться.

После того как мы нашли один дефект, нас ждут еще две ошибки.

Возвращаясь к журналу трассировки, видим, что `advapi32!ImpersonateSelf` завершается отказом в доступе.

Как обсуждалось в предыдущем разделе "Local Security Failures", сначала нужно понять операцию и определить безопасность всех компонентов, участвующих в ней.

Теперь ясно, что `advapi32!ImpersonateSelf` открывает дескриптор процесса, дублирует первичный маркер доступа и устанавливает его на вызывающий поток.

Мы устанавливаем точку останова на `advapi32!ImpersonateSelf` в отладчике пользовательского режима, но продолжаем исследование с помощью отладчика режима ядра, пока отладчик пользовательского режима остановлен в точке останова.

Начнем с проверки информации безопасности объекта процесса, как показано в листинге 7.21.

Листинг 7.21. Дескриптор безопасности процесса во время отказа ленивой инициализации

```
lkd> !process 0 1 07Sample.exe
PROCESS ffb36020 SessionId: 0 Cid: 0784 Peb: 7ffde000 ParentCid: 0284
DirBase: 0a257000 ObjectTable: e183bbb0 HandleCount: 22.
Image: 07sample.exe
VadRoot ffa7c978 Vads 33 Clone 0 Private 66. Modified 0. Locked 0.
DeviceMap e1798128
Token e196a3f0
...
lkd> !process 0 2 07sample.exe
PROCESS ffb36020 SessionId: 0 Cid: 0784 Peb: 7ffde000 ParentCid: 0284
DirBase: 0a257000 ObjectTable: e183bbb0 HandleCount: 22.
Image: 07sample.exe
THREAD 82f408a8 Cid 0784.04f8 Teb: 7ffdf000 Win32Thread: e17a5d28 WAIT
: (Executive) KernelMode Non-Alertable
SuspendCount 1
f3ad77d4 SynchronizationEvent
lkd> !sd poi(ffb36020-4)&FFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
                SE_DACL_PRESENT
                SE_SELF_RELATIVE
->Owner : S-1-5-32-544
->Group : S-1-5-21-1060284298-2111687655-1957994488-513
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x34
->Dacl : ->AceCount : 0x2
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x18
->Dacl : ->Ace[0]: ->Mask : 0x001f0fff
->Dacl : ->Ace[0]: ->SID: S-1-5-32-544
->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x14
->Dacl : ->Ace[1]: ->Mask : 0x001f0fff
->Dacl : ->Ace[1]: ->SID: S-1-5-18
->Sacl : is NULL
```

Наш поток олицетворяет маркер доступа, полученный из вызова `advapi32!LogonUserExW`, представляющий пользователя `Test1`, который не является членом какой-либо группы, способной открыть дескриптор процесса для доступа, запрошенного `advapi32!ImpersonateSelf`.

В листинге 7.22 команда расширения `!thread` используется для получения маркера доступа олицетворения, который передается как параметр команде расширения `!token`.

Адрес объекта потока получен из листинга 7.21.

Листинг 7.22. Маркер олицетворения пользователя `Test1`

```
lkd> !thread 82f408a8
THREAD 82f408a8 Cid 0784.07a4 Teb: 7ffdd000 Win32Thread: e189aeb0 WAIT: (Executive)
```

```

KernelMode Non-Alertable
SuspendCount 1
f70687d4 SynchronizationEvent
Impersonation token: e13fee28 (Level Impersonation)
Owning Process ffb36020 Image: 07sample.exe
kd> !token e13fee28
TS Session ID: 0
User: S-1-5-21-1060284298-2111687655-1957994488-1006
Groups:
00 S-1-5-21-1060284298-2111687655-1957994488-513
Attributes - Mandatory Default Enabled
01 S-1-1-0
Attributes - Mandatory Default Enabled
02 S-1-5-32-545
Attributes - Mandatory Default Enabled
03 S-1-5-5-0-1757850
Attributes - Mandatory Default Enabled LogonId
04 S-1-2-0
Attributes - Mandatory Default Enabled
05 S-1-5-4
Attributes - Mandatory Default Enabled
06 S-1-5-11
Attributes - Mandatory Default Enabled
Primary Group: S-1-5-21-1060284298-2111687655-1957994488-513
Privs:
00 0x00000017 SeChangeNotifyPrivilege Attributes - Enabled Default
01 0x00000013 SeShutdownPrivilege Attributes -
02 0x00000019 SeUndockPrivilege Attributes -
Auth ID: 0:1ad29b
Impersonation Level: Impersonation
TokenType: Impersonation

```

Когда еще один дефект кода понят, пора сосредоточиться на последнем, похожем на невозможность открыть объект процесса.

Однако у этой функции есть еще одна проблема.

Следующая строка в примере кода создает именованное событие, которое, исходя из безопасности по умолчанию, предоставляет олицетворяемому пользователю Test1 полный доступ к нему.

Если тот же пользователь может запускать собственный код в системе, где выполняется служебный код с этой проблемой, он может манипулировать событием, принадлежащим службе.

Это проблема безопасности.

Поскольку приложение не задает явный дескриптор безопасности для вновь созданного события, система назначает дескриптор, сгенерированный с помощью механизма безопасности по умолчанию.

Сгенерированный дескриптор безопасности предоставляет полный доступ принципалу, представленному олицетворенным маркером доступа.

В той же функции, используя отладчик пользовательского режима, мы можем остановиться после создания события ядра, чтобы исследовать его дескриптор безопасности.

Мы ищем адрес события ядра для дескриптора события, полученного в отладчике пользовательского режима.

Дескриптор события 0x7a8 используется как параметр команды расширения !handle вместе с идентификатором процесса.

В листинге 7.23 мы извлекаем дескриптор безопасности события тем же методом, что и для любых других объектов ядра.

Листинг 7.23. Дескриптор безопасности события, созданного под олицетворением

```

kd> !handle 7a8 7 784
processor number 0, process 00000784
Searching for Process with Cid == 784
PROCESS ffb36020 SessionId: 0 Cid: 0784 Peb: 7ffde000 ParentCid: 0284
DirBase: 0a257000 ObjectTable: e183bbb0 HandleCount: 23.
Image: 07sample.exe

```



```

Handle table at e1910000 with 23 Entries in use
07a8: Object: ffb47ff0 GrantedAccess: 001f0003 Entry: e1910f50
Object: ffb47ff0 Type: (812ed320) Event
ObjectHeader: ffb47fd8
HandleCount: 1 PointerCount: 2
Directory Object: e171d128 Name: 07sample
kd> !sd poi(ffb47ff0-4)&FFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
                SE_DACL_PRESENT
                SE_SELF_RELATIVE
->Owner : S-1-5-21-1060284298-2111687655-1957994488-1006
->Group : S-1-5-21-1060284298-2111687655-1957994488-513
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x40
->Dacl : ->AceCount : 0x2
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x24
->Dacl : ->Ace[0]: ->Mask : 0x001f0003
->Dacl : ->Ace[0]: ->SID: S-1-5-21-1060284298-2111687655-1957994488-1006
->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x14
->Dacl : ->Ace[1]: ->Mask : 0x001f0003
->Dacl : ->Ace[1]: ->SID: S-1-5-18
->Sacl : is NULL

```

Показанные ранее сценарии могут выглядеть незнакомыми для разработчиков, которые не пишут службы, не используют олицетворение или не вызывают Win32 API напрямую.

Но с развитием Web Services в корпоративной разработке программного обеспечения переход к службам с олицетворением становится обычным делом.

Кроме того, сложные библиотеки с тяжелым кодом инициализации, отложенным до первого использования, которые, скорее всего, применяются внутри сложного распределенного приложения, являются идеальной средой для проблем того типа, который исследовался в этом разделе.

Потенциальные последствия олицетворения для безопасности

При создании служб, принимающих клиентские запросы, нужно помнить, как олицетворение потока влияет на компонент, используемый во время запроса к службе.

Даже если служба не олицетворяет пользователя перед использованием компонентов, каждый компонент потенциально может олицетворить вызывающего.

В таких случаях мы должны быть знакомы с поведением каждого компонента и использовать эту информацию при принятии решения об использовании этого компонента.

Это справедливо для компонентов, выполняющихся внутри служб, поддерживающих источники олицетворения, таких как приложение ASP.NET, Web services, RPC или DCOM-серверы.

Эта возможность олицетворения ограничена только потоком, диспетчеризованным в результате вызова клиента.

При вызове внешнего компонента разработчик должен понимать последствия, которые эта потенциальная возможность олицетворения может иметь для вызова компонента, и при необходимости удалить ее, используя специальные методики для каждого источника олицетворения, когда это возможно, либо делегировать выполнение новому потоку, который больше не подвержен этой возможности.

Ошибки Distributed COM

Как вы видели в таблице 7.4, ошибка отказа в доступе может принимать несколько значений в зависимости от компонента, который выводит ошибку наружу.

Мы искали в Интернете ошибку 0x80070005, выдаваемую DCOM, и нашли более 7000 страниц с вопросами и обходными путями.

Мы также искали десятичную форму ошибки и получили еще 1500 результатов.

Ошибки отказа в доступе DCOM трудно исследовать из-за присущей любым распределенным системам сложности.

Мы ожидаем увидеть похожий уровень сложности в распределенных приложениях, построенных поверх других инфраструктур.

Ошибки отказа в доступе выдаются, когда DCOM-клиент не имеет права активировать сервер, когда клиенту не разрешено вызывать сервер, когда компоненты зарегистрированы неправильно и когда инфраструктура сталкивается с ошибкой отказа в доступе.

Активация DCOM является хорошим примером систем пользовательского режима, использующих специальные проверки доступа.

DCOM хранит дескрипторы безопасности активации и доступа в реестре.

Все следующие сценарии часто встречаются в операциях, выполняемых в правильно настроенных системах.

В конце этой главы мы диагностируем систему, конфигурация которой была ошибочно изменена и в которой отказывает большинство операций DCOM, интересный сценарий от начала до конца.

Все сценарии выполняются в операционной системе Windows XP SP2.

Проверки активации DCOM

Наивный подход к отладке отказа взаимодействия путем пошаговой трассировки клиентского кода имеет минимальные шансы на успех и его следует избегать.

Поскольку активация DCOM по сути является распределенным процессом, ее следует исследовать с использованием модели, описанной в главе 8, в разделе "Breaking the Call Path".

Используя эту модель, мы сначала идентифицируем процесс, размещающий двоичный файл, который вернул исходную ошибку, а затем пытаемся выяснить детали отказа.

Чтобы использовать модель, нужно подробнее понять путь вызова запроса активации, который мы описываем в этом разделе.

Рисунок 7.2 иллюстрирует все процессы, участвующие в активации DCOM.

Каждый прямоугольник представляет границу безопасности, а длинная вертикальная серая линия представляет границу системы.

Клиент активирует удаленный COM-объект, взаимодействуя с локальным интерфейсом активации DCOM, реализованным службой RPCSS Server, которая при необходимости делегирует запрос активации удаленной службе RPCSS Server.

Удаленная служба RPCSS запускает процесс, размещающий сервер; она ждет, пока серверный процесс регистрируется как DCOM-сервер, и наконец вызывает процесс, чтобы получить интерфейс, запрошенный вызывающим.

Уже по одному взгляду на все шесть границ процессов, одна из которых также является границей машины, легко увидеть, сколько компонентов должны работать в полной гармонии, чтобы активация стала возможной.

В стандартной корпоративной среде каждая служба RPCSS Server также может взаимодействовать с контроллером домена.

Чтобы уменьшить сложность диаграммы, соединение с контроллером домена было опущено.

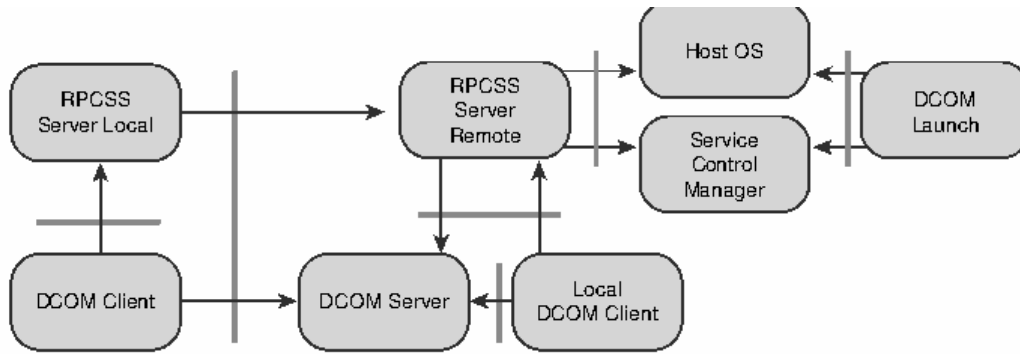


Figure 7.2 Processes participating in DCOM activation

Рисунок 7.2. Процессы, участвующие в активации DCOM

Согласно рисунку 7.2, активация включает клиентский процесс, службу RPCSS и службу DcomLaunch на стороне сервера, а также серверный процесс.

В случае локальной активации взаимодействие между RPCSS на стороне клиента и RPCSS на стороне сервера является сокращенным путем.

Мы начинаем с идентификации процессов, участвующих в пути активации, и создаем мысленную диаграмму отношений между ними.

Для этого отлично подходит инструмент `tlist.exe`, установленный вместе с Debugging Tools for Windows.

В листинге 7.24 мы используем `tlist.exe`, чтобы найти идентификаторы процессов служб DcomLaunch и RpcSs на стороне сервера.

Листинг 7.24. Поиск служб DcomLaunch и RpcSs

```
c:\>tlist -s
0 System Process
4 System
300 smss.exe
432 csrss.exe Title:
464 winlogon.exe
548 services.exe Svcs: Eventlog,PlugPlay
560 lsass.exe Svcs: PolicyAgent,ProtectedStorage,SamSs
716 svchost.exe Svcs: DcomLaunch,TermService
768 svchost.exe Svcs: RpcSs
```

После идентификации процесса, используемого путем выполнения, самый быстрый способ отладки - предположить, что вызов активации достигает последнего процесса в цепочке вызовов, подключить отладчик пользовательского режима к последнему процессу в пути и остановить выполнение процесса, а затем снова выполнить отказавший клиентский вызов.

Если клиент не зависает, путь вызова не достигает процесса, который в данный момент остановлен в отладнике, и мы можем отсоединить отладчик, введя команду `qd`.

Мы повторяем процесс выше по пути вызова, пока клиент не зависнет в вызове активации.

В этой точке мы можем использовать этот процесс, чтобы определить, какие учетные данные использует клиент, какие другие параметры DCOM действуют во время вызова, и так далее. Чем лучше мы понимаем клиентскую среду во время вызова, тем проще создать возможный сценарий для каждого отказа, продемонстрировать его достоверность и двигаться дальше.

Этот раздел описывает все места в пути активации, полезные для оценки хода активации, и объясняет, как интерпретировать информацию, доступную в этих точках.

Путь активации можно задействовать с помощью параметра `0` примера `08cli.exe`.

Удаленные клиенты сталкиваются с первыми воротами безопасности, когда система проходит проверку подлинности на удаленной системе.

Ход выполнения можно отслеживать, исследуя коды возврата SSPI, как описано в разделе "Remote Authentication and Security Support Provider Interface".

Запрос проверки подлинности SSPI обрабатывается кодом службы RPCSS.

После успешной удаленной проверки подлинности (локальные клиенты уже прошли проверку подлинности операционной системой) код активации использует маркер олицетворения, представляющий клиента, чтобы выполнить различные проверки с помощью `advapi32!AccessCheck` в службе RPCSS, выполняющейся на сервере.

В рамках активации служба RPCSS выполняет несколько проверок, каждая из которых имеет свою роль.

В листинге 7.25 показана первая проверка, которая проверяет, имеет ли вызывающий право доступа к серверу с использованием протокола DCOM.

Мы подключаем отладчик к службе RPCSS и устанавливаем точку останова на `ADVAPI32!AccessCheck`, как показано в следующем листинге:

Листинг 7.25. Первая проверка `AccessCheck`, выполняемая RPCSS

```
0:007> bp ADVAPI32!AccessCheck;g
Breakpoint 0 hit
eax=007dfce4 ebx=00000000 ecx=007dfcf8 edx=007dfd08 esi=00000001 edi=00000000
eip=77dd7c11 esp=007dfcb8 ebp=007dfd10 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000206
ADVAPI32!AccessCheck:
77dd7c11 8bff mov edi,edi
0:007> k
ChildEBP RetAddr
007dfcb4 76a822a6 ADVAPI32!AccessCheck
007dfd10 76a824f6 rpcss!CheckForAccess+0x81
007dfd5c 77e7a2c1 rpcss!LocalInterfaceOnlySecCallback+0xb9
007dfdb4 77e7c767 RPCRT4!RPC_INTERFACE::CheckSecurityIfNecessary+0x6f
007dfdcc 77e7bcc9 RPCRT4!LRPC_SBINDING::CheckSecurity+0x4f
007dfdfc 77e7bb6a RPCRT4!LRPC_SCALL::DealWithRequestMessage+0x194
007dfe20 77e76784 RPCRT4!LRPC_ADDRESS::DealWithLRPCRequest+0x16d
007dff80 77e76c22 RPCRT4!LRPC_ADDRESS::ReceiveLotsaCalls+0x28f
007dff88 77e76a3b RPCRT4!RecvLotsaCallsWrapper+0xd
007dff8a 77e76c0a RPCRT4!BaseCachedThreadRoutine+0x79
007dff8b 7c80b50b RPCRT4!ThreadStartRoutine+0x1a
007dffec 00000000 kernel32!BaseThreadStart+0x37
0:007> * !sd extension fails; we grab the ACL directly from the SD
0:007>!acl poi(@esp+4)+poi(poi(@esp+4)+10)
ACL is:
ACL is: ->AclRevision: 0x2
ACL is: ->Sbz1 : 0x0
ACL is: ->AclSize : 0x30
ACL is: ->AceCount : 0x2
ACL is: ->Sbz2 : 0x0
ACL is: ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
ACL is: ->Ace[0]: ->AceFlags: 0x0
ACL is: ->Ace[0]: ->AceSize: 0x14
ACL is: ->Ace[0]: ->Mask : 0x00000003
ACL is: ->Ace[0]: ->SID: S-1-5-7
ACL is: ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
ACL is: ->Ace[1]: ->AceFlags: 0x0
ACL is: ->Ace[1]: ->AceSize: 0x14
ACL is: ->Ace[1]: ->Mask : 0x00000007
ACL is: ->Ace[1]: ->SID: S-1-1-0
```

Эта первая проверка определяет, может ли пользователь пройти ограничения безопасности, наложенные на машину DCOM-сервера, показанные на рисунке 7.3.

Страница конфигурации безопасности Component Services запускается с помощью командной строки `dcomcnfg.exe`.

Из оснастки MMC Component Services можно настроить все параметры безопасности, используемые в DCOM.

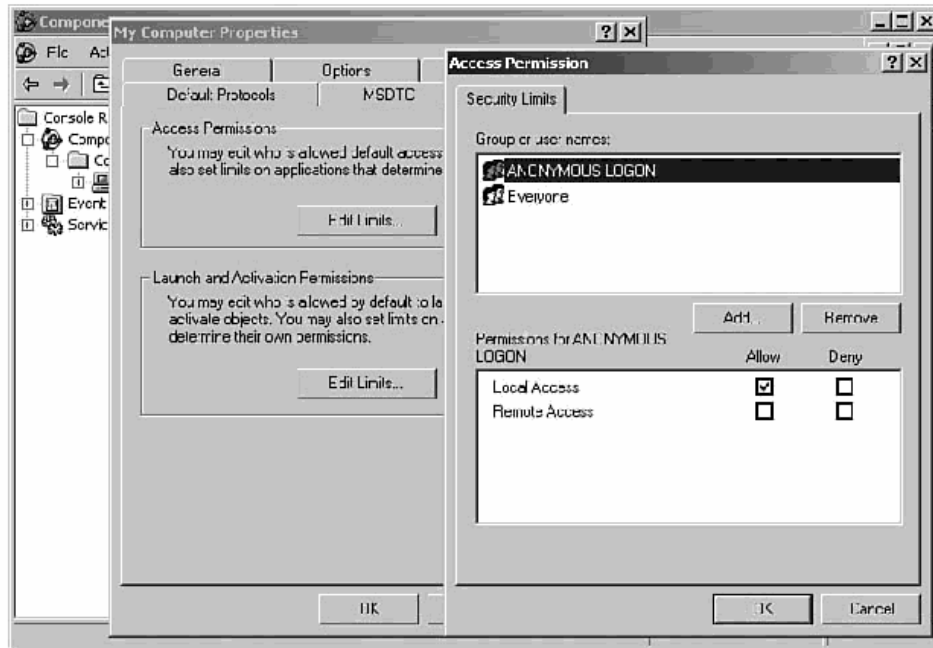


Figure 7.3 Access permission limits configuration page, differentiating between local and remote clients

Рисунок 7.3. Страница конфигурации пределов разрешений доступа, различающая локальных и удаленных клиентов

После прохождения первой проверки система проверяет, имеет ли пользователь право активировать любой DCOM-сервер в системе.

В листинге 7.26 показана вторая проверка доступа, выполняемая относительно другого дескриптора безопасности.

Листинг 7.26. Вторая проверка AccessCheck для пределов запуска и активации

```
0:007> g
Breakpoint 0 hit
eax=007dfce4 ebx=00000000 ecx=007dfcf8 edx=007dfd08 esi=00000001 edi=00000000
eip=77dd7c11 esp=007dfcb8 ebp=007dfd10 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000202
ADVAPI32!AccessCheck:
77dd7c11 8bff mov edi,edi
0:007> k
ChildEBP RetAddr
007dfcb4 76a822a6 ADVAPI32!AccessCheck
007dfd10 76a8c2e4 rpcss!CheckForAccess+0x81
007dfd5c 77e7a2c1 rpcss!LocalInterfaceOnlySecCallback+0x138
007dfdb4 77e7c767 RPCRT4!RPC_INTERFACE::CheckSecurityIfNecessary+0x6f
007dfdcc 77e7bcc9 RPCRT4!LRPC_SBINDING::CheckSecurity+0x4f
007dfdfc 77e7bb6a RPCRT4!LRPC_SCALL::DealWithRequestMessage+0x194
007dfe20 77e76784 RPCRT4!LRPC_ADDRESS::DealWithLRPCRequest+0x16d
007dff80 77e76c22 RPCRT4!LRPC_ADDRESS::ReceiveLotsaCalls+0x28f
007dff88 77e76a3b RPCRT4!RecvLotsaCallsWrapper+0xd
007dfffa8 77e76c0a RPCRT4!BaseCachedThreadRoutine+0x79
007dfffb4 7c80b50b RPCRT4!ThreadStartRoutine+0x1a
007dffec 00000000 kernel32!BaseThreadStart+0x37
0:007> !acl poi(@esp+4)+poi(poi(@esp+4)+10)
ACL is:
ACL is: ->AclRevision: 0x2
```

```

ACL is: ->Sbz1 : 0x0
ACL is: ->AclSize : 0x34
ACL is: ->AceCount : 0x2
ACL is: ->Sbz2 : 0x0
ACL is: ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
ACL is: ->Ace[0]: ->AceFlags: 0x0
ACL is: ->Ace[0]: ->AceSize: 0x18
ACL is: ->Ace[0]: ->Mask : 0x0000001f
ACL is: ->Ace[0]: ->SID: S-1-5-32-544
ACL is: ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
ACL is: ->Ace[1]: ->AceFlags: 0x0
ACL is: ->Ace[1]: ->AceSize: 0x14
ACL is: ->Ace[1]: ->Mask : 0x0000000b
ACL is: ->Ace[1]: ->SID: S-1-1-0

```

Дескриптор безопасности, используемый во второй проверке, также является общесистемным пределом безопасности, накладываемым на запуск и активацию всех DCOM-серверов. Он управляется другой страницей конфигурации безопасности, показанной на рисунке 7.4, также входящей в конфигурацию DCOM.

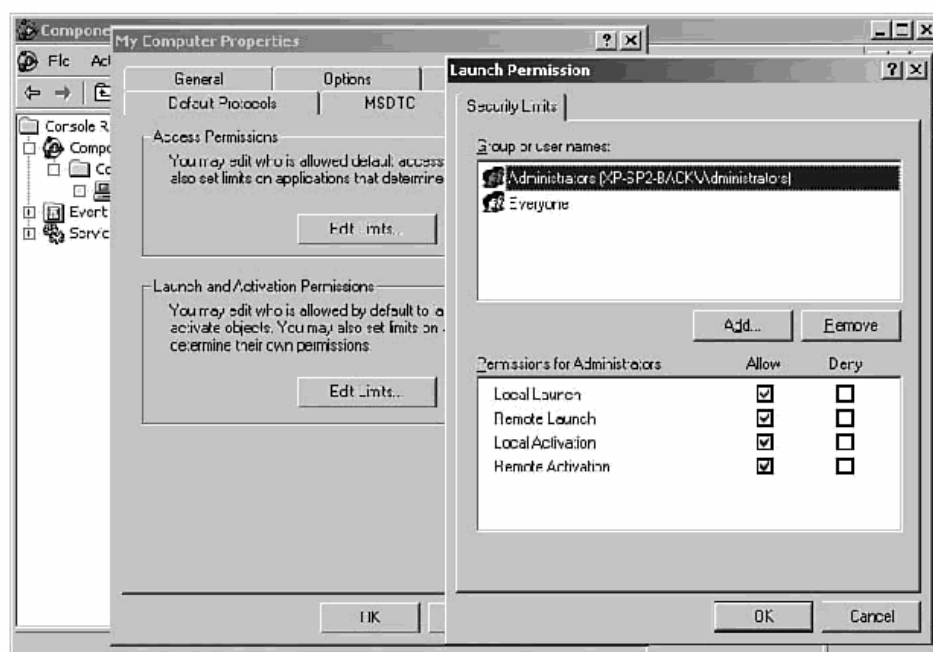


Figure 7.4 Launch Permission limits configuration page, differentiating between local and remote clients

Рисунок 7.4. Страница конфигурации пределов разрешений запуска, различающая локальных и удаленных клиентов

После успешного завершения этих двух начальных проверок, не специфичных для запрошенного компонента, сервер RPCSS читает из реестра информацию, относящуюся к компоненту. Ограничения компонента наконец проверяются RPCSS, как показано в листинге 7.27.

Листинг 7.27. Проверка специфичного для компонента AccessCheck, выполняемая RPCSS

```

0:007> g
Breakpoint 0 hit
eax=007df59c ebx=0009ade0 ecx=007df5b0 edx=007df5c0 esi=00000001 edi=00000000
eip=77dd7c11 esp=007df570 ebp=007df5c8 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000206
ADVAPI32!AccessCheck:
77dd7c11 8bff mov edi,edi
0:007> k
ChildEBP RetAddr

```

```

007df56c 76a822a6 ADVAPI32!AccessCheck
007df5c8 76a8c0cd rpcss!CheckForAccess+0x81
007df5f4 76a8e5fb rpcss!CClsidData::LaunchOrActivationAllowed+0x155
007df65c 76a8e4ab rpcss!Activation+0x1fb
007df6b8 76a91e12 rpcss!ActivateFromProperties+0x213
007df6c8 76a91e66 rpcss!SCmActivator::CreateInstance+0x10
007df708 76a91e7b rpcss!ActivationPropertiesIn::DelegateCreateInstance+0xf7
007df754 76a8c1d7 rpcss!ActivateFromPropertiesPreamble+0x4c1
007df79c 76a91de7 rpcss!PerformScmStage+0xbb
007df8b0 77e79dc9 rpcss!SCmActivatorCreateInstance+0x97
007df8e0 77ef321a RPCRT4!Invoke+0x30
007dfcf8 77ef36ee RPCRT4!NdrStubCall2+0x297
007dfd14 77e7988c RPCRT4!NdrServerCall2+0x19
007dfd48 77e797f1 RPCRT4!DispatchToStubInC+0x38
007dfd9c 77e7971d RPCRT4!RPC_INTERFACE::DispatchToStubWorker+0x113
007dfd0c 77e7bd0d RPCRT4!RPC_INTERFACE::DispatchToStub+0x84
007dfd9c 77e7bd0d RPCRT4!RPC_INTERFACE::DispatchToStub+0x84
007dfd9c 77e7bd0d RPCRT4!RPC_INTERFACE::DispatchToStub+0x84
007dfdfc 77e7bb6a RPCRT4!LRPC_SCALL::DealWithRequestMessage+0x2db
007dfe20 77e76784 RPCRT4!LRPC_ADDRESS::DealWithLRPCRequest+0x16d
007dff80 77e76c22 RPCRT4!LRPC_ADDRESS::ReceiveLotsaCalls+0x28f
007dff88 77e76a3b RPCRT4!RecvLotsaCallsWrapper+0xd
0:007> !acl poi(@esp+4)+poi(poi(@esp+4)+10)
ACL is:
ACL is: ->AclRevision: 0x2
ACL is: ->Sbz1 : 0x0
ACL is: ->AclSize : 0x50
ACL is: ->AceCount : 0x3
ACL is: ->Sbz2 : 0x0
ACL is: ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
ACL is: ->Ace[0]: ->AceFlags: 0x0
ACL is: ->Ace[0]: ->AceSize: 0x18
ACL is: ->Ace[0]: ->Mask : 0x00000001
ACL is: ->Ace[0]: ->SID: S-1-5-18
ACL is: ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
ACL is: ->Ace[1]: ->AceFlags: 0x0
ACL is: ->Ace[1]: ->AceSize: 0x18
ACL is: ->Ace[1]: ->Mask : 0x00000001
ACL is: ->Ace[1]: ->SID: S-1-5-4
ACL is: ->Ace[2]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
ACL is: ->Ace[2]: ->AceFlags: 0x0
ACL is: ->Ace[2]: ->AceSize: 0x18
ACL is: ->Ace[2]: ->Mask : 0x00000001
ACL is: ->Ace[2]: ->SID: S-1-5-32-544

```

Эта проверка доступа, последняя, выполняемая службой RPCSS до попытки запустить СОМ-сервер, реализующий запрошенный объект, управляется страницей конфигурации безопасности, специфичной для компонента, показанной на рисунке 7.5.

Страница конфигурации позволяет администратору выбрать между пользовательским дескриптором безопасности и дескриптором безопасности по умолчанию, используемым для всех компонентов.

Страница конфигурации, специфичная для сервера, отображается после выбора сервера SRV в узле DCOM Config.

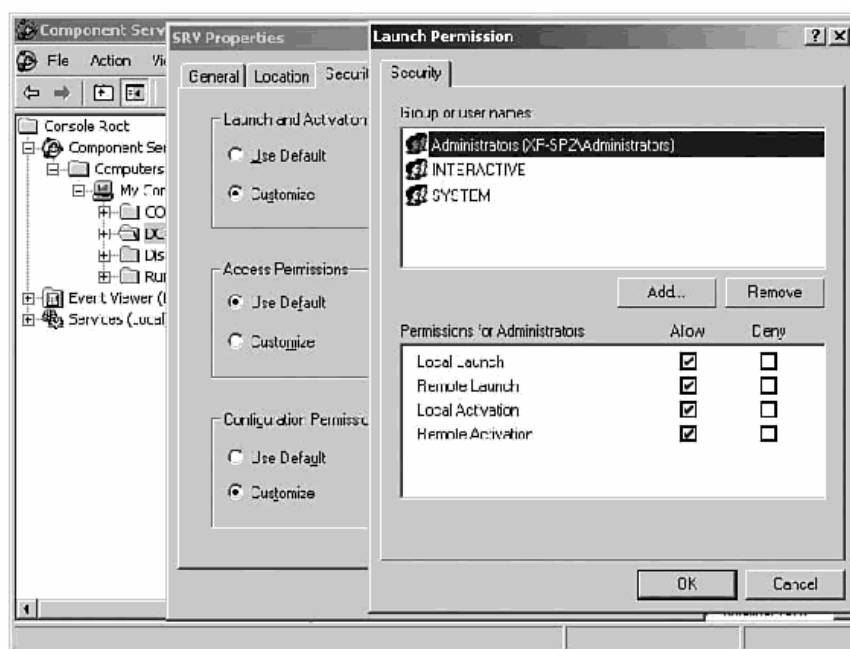


Figure 7.5 Component specific Launch Permission page, equal to the default Launch Permission

Рисунок 7.5. Страница Launch Permission, специфичная для компонента и равная разрешению Launch Permission по умолчанию

Дескриптор, показанный на рисунке 7.5, имеет то же значение, что и разрешение Launch Permission по умолчанию.

Легко заметить, насколько ограничительным является этот дескриптор безопасности.

Чтобы поддерживать обычных пользователей, он разрешает все активации, исходящие из интерактивного сеанса.

В то же время активация завершается неудачей для всех пользователей без административных прав, вошедших через сетевую проверку подлинности, проверку подлинности службы или пакетный вход.

Например, код, который пытается активировать COM-сервер из приложения ASP.NET, настроенного для выполнения под учетной записью NetworkService, завершается отказом в доступе, если компонент не переопределяет разрешение запуска по умолчанию.

Предположим, что начальные ворота пройдены; запрос активации отправляется службе DcomLaunch, другой службе, играющей роль в процессе активации.

До Windows XP SP2 эта функциональность службы была частью службы RPCSS.

Служба DcomLaunch похожим образом повторно проверяет разрешение, специфичное для компонента.

Каждый процесс, порождаемый DCOM Service Control Manager, проходит через еще одни общие ворота, реализованные API ADVAPI32!CreateProcessAsUserW, который вызывается службой DcomLaunch.

Точка останова на этой функции дает идеальное место для понимания командной строки сервера и идентичности, под которой он будет выполняться, как показано в листинге 7.28.

Мы можем интерпретировать параметры из стека после учета соглашения о вызовах функции.

Мы подключаем отладчик к службе DcomLaunch и устанавливаем точку останова на ADVAPI32!CreateProcessAsUserW, как показано в следующем листинге.

Листинг 7.28. Командная строка и первичный маркер при запуске DCOM-сервера

```
0:010> bp ADVAPI32!CreateProcessAsUserW;g
Breakpoint 0 hit
eax=00000000 ebx=00000410 ecx=0000038c edx=00aff71c esi=00000000 edi=000c2b48
eip=77df7775 esp=00aff690 ebp=00aff7dc iopl=0         nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00000246
ADVAPI32!CreateProcessAsUserW:
77df7775 8bff mov edi,edi
0:010> k
ChildEBP RetAddr
00aff68c 76a93acd ADVAPI32!CreateProcessAsUserW
00aff7dc 76a93849 rpcss!CClsidData::PrivilegedLaunchActivatorServer+0x39d
00aff858 77e79dc9 rpcss!_LaunchActivatorServer+0xbc
00aff8b4 77ef321a RPCRT4!Invoke+0x30
...
0:010> * According to MSDN, the command line is the 3rd parameter
0:010> du poi(@esp+c)
000c2750 ""C:\awdbin\WinXP.x86.chk\08comsr"
000c2790 "v.exe" -Embedding"
0:010> * According to MSDN, the primary token is the 1st parameter
0:010> !token poi(@esp+4) -n
TS Session ID: 0
User: S-1-5-21-1060284298-2111687655-1957994488-1003 (User: XP-SP2\TestAdmin)
Groups:
00 S-1-5-21-1060284298-2111687655-1957994488-513 (Group: XP-SP2\None)
Attributes - Mandatory Default Enabled
...
...
TokenType: Primary
```

Если активация дошла до этой точки, но ей не удастся создать процесс, отказ активации сводится к отказу запуска процесса в этом пользовательском контексте.

Отказы могут быть вызваны множеством факторов, но чаще всего пользователь, обозначенный маркером, не имеет доступа к файлам серверного процесса.

Среду для пользователя можно смоделировать с помощью команды `runas.exe`, а запуск процесса следует исследовать отдельно.

Если сервер реализован как Windows Service, DcomLaunch использует SCM API для запуска службы. Эти API идеально подходят для исследования возможных ошибок, возвращаемых в ответ на запуск службы.

Если сервер уже выполняется и поддерживает множественные активации, путь активации даже не достигает этого процесса; он завершается в RPCSS.

Почти в конце этого пути активации, когда серверный процесс уже запущен и выполняется, RPCSS выполняет финальный вызов в сервер, чтобы создать экземпляр, запрошенный клиентом. Вызов выполняется во время олицетворения пользователя, сделавшего исходный вызов, и обрабатывается COM-сервером как любой другой вызов, подчиняясь всем ограничениям, накладываемым доступом вызовов, который обсуждается далее.

Проверки доступа вызовов DCOM

Поскольку инфраструктура DCOM обрабатывает все клиентские вызовы до их диспетчеризации в серверный код, она создает ворота безопасности, которые должны быть пройдены клиентом до того, как сервер выполнит этот запрос.

Эти ворота безопасности можно инициализировать явно, вызвав API `ole32!CoInitializeSecurity` со следующей сигнатурой:

```
HRESULT CoInitializeSecurity(
    PSECURITY_DESCRIPTOR pVoid,
    LONG cAuthSvc,
    SOLE_AUTHENTICATION_SERVICE * asAuthSvc,
```

```

void * pReserved1,
DWORD dwAuthnLevel,
DWORD dwImpLevel,
SOLE_AUTHENTICATION_LIST * pAuthList,
DWORD dwCapabilities,
void * pReserved3
);

```

Второй параметр функции представляет минимальный принимаемый уровень проверки подлинности входящего вызова.

Первый параметр API является полиморфным и может быть дескриптором безопасности Windows, значением NULL, строкой AppID или указателем на объект, реализующий интерфейс IAccessControl.

На практике этот параметр часто равен NULL и редко является явным дескриптором безопасности. Значение NULL в сочетании с флагом EOAC_APPID в dwCapabilities указывает, что инфраструктура DCOM должна загрузить дескриптор безопасности из настроек разрешений доступа, связанных с серверным приложением.

Когда EOAC_APPID отсутствует, дескриптор безопасности, используемый инфраструктурой DCOM, разрешает всем выполнять вызовы в сервер, что не рекомендуется.

На рисунке 7.6 показано, как настроить разрешение доступа для входящих вызовов в сервер SRV.

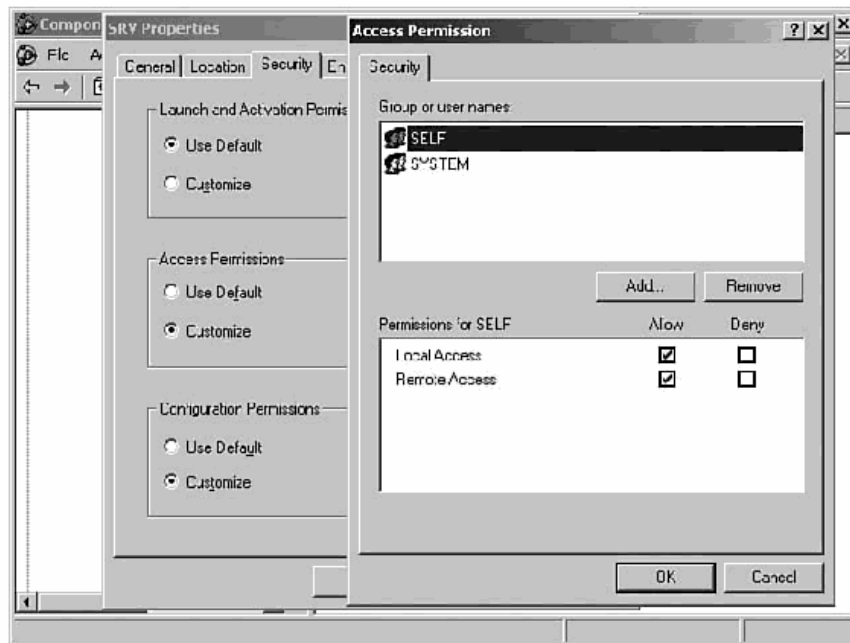


Figure 7.6 Component-specific Access Permission page, equal to the default Access Permission

Рисунок 7.6. Страница Access Permission, специфичная для компонента и равная разрешению Access Permission по умолчанию

Если приложение явно не вызывает API `ole32!CoInitializeSecurity`, DCOM делает это от имени приложения перед экспортом первого интерфейса.

Параметры по умолчанию, используемые в этом случае, - NULL для дескриптора безопасности с флагом EOAC_APPID в параметре dwCapabilities.

NOTE Сервер безопаснее, если он не инициализирует безопасность DCOM, чем если он инициализирует ее с более слабым ограничением, как в следующем примере:

```
CoInitializeSecurity( NULL, -1, NULL, NULL, RPC_C_AUTHN_LEVEL_DEFAULT,  
    RPC_C_IMP_LEVEL_IDENTIFY, NULL, EOAC_NONE , NULL );
```

API `ole32!CoInitializeSecurity` сохраняет переданные аргументы в глобальных переменных, расположенных внутри `ole32.dll`, с символическими именами, похожими на имена аргументов. Такие значения можно интерпретировать согласно их смыслу, описанному на странице справки, связанной с API, который их инициализирует.

Их полные имена показаны ниже:

```
0:000> x ole32!g*  
...  
772bb20c OLE32!gSecDesc = <no type information>  
...  
772bb208 OLE32!gAuthnLevel = <no type information>  
...  
772bbf70 OLE32!gImpLevel = <no type information>  
...  
772bb05c OLE32!gCapabilities = <no type information>
```

После того как мы знаем, что вызовы выполняются в серверный процесс, эти переменные можно исследовать в любой момент, чтобы обнаружить источник ошибки отказа в доступе.

Инфраструктура DCOM олицетворяет каждый вызов, извлекает маркер олицетворения и выполняет проверку доступа относительно дескриптора безопасности, сохраненного в `OLE32!gSecDesc`.

Маркер олицетворения, используемый для выполнения вызова, доступен до вызова функции проверки доступа.

Точка останова на этой функции также позволяет проверить результаты проверки доступа.

Инфраструктура DCOM использует либо `advapi32!AccessCheck`, либо `advapi32!AccessCheckByType` в зависимости от версии операционной системы.

В листинге 7.29 исследуется идентичность перед выполнением проверки доступа.

Листинг 7.29. Идентичность перед проверкой доступа входящего DCOM-вызова

```
0:001> k  
ChildEBP RetAddr  
007efc34 77525505 ADVAPI32!AccessCheckByType  
007efc8c 775448c2 ole32!CallAccessCheck+0x9c  
007efcec 775387a9 ole32!CheckAcl+0x73  
007efd08 77532fe7 ole32!CheckAccess+0x88  
007efd5c 77e7a2c1 ole32!ORPCInterfaceSecCallback+0x178  
007efdb4 77e7c767 RPCRT4!RPC_INTERFACE::CheckSecurityIfNecessary+0x6f  
007efdcc 77e7bcc9 RPCRT4!LRPC_SBINDING::CheckSecurity+0x4f  
007efdfc 77e7bb6a RPCRT4!LRPC_SCALL::DealWithRequestMessage+0x194  
007efe20 77e76784 RPCRT4!LRPC_ADDRESS::DealWithLRPCRequest+0x16d  
007eff80 77e76c22 RPCRT4!LRPC_ADDRESS::ReceiveLotsaCalls+0x28f  
007eff88 77e76a3b RPCRT4!RecvLotsaCallsWrapper+0xd  
007effa8 77e76c0a RPCRT4!BaseCachedThreadRoutine+0x79  
007effb4 7c80b50b RPCRT4!ThreadStartRoutine+0x1a  
007effec 00000000 kernel32!BaseThreadStart+0x37  
0:001> !token poi(@esp+c)  
TS Session ID: 0  
User: S-1-5-21-1060284298-2111687655-1957994488-1003  
Groups:  
00 S-1-5-21-1060284298-2111687655-1957994488-513  
Attributes - Mandatory Default Enabled  
...  
Impersonation Level: Identification  
TokenType: Impersonation
```

Маркер олицетворения - не единственная причина, по которой инфраструктура DCOM отклоняет некоторые вызовы.

Все удаленные вызовы имеют связанный уровень проверки подлинности, который может варьироваться от `RPC_C_AUTHN_LEVEL_NONE`, где проверка подлинности клиента вообще отсутствует, до `RPC_C_AUTHN_LEVEL_PKT_PRIVACY`, где идентичность клиента проверяется при каждом вызове, а данные шифруются.

Инфраструктура DCOM на стороне сервера отклоняет все вызовы, выполненные на уровне проверки подлинности ниже значения, переданного в `ole32!CoInitializeSecurity`, которое хранится в глобальной переменной `OLE32!gAuthnLevel`.

Уровень проверки подлинности не имеет смысла для вызовов между локальными процессами, поскольку эти вызовы выполняются на уровне `RPC_C_AUTHN_LEVEL_PKT_PRIVACY`, гарантированном ядром Windows.

Листинг 7.29 взят из проверки доступа, выполняемой перед диспетчеризацией клиентского вызова в серверный код, будь то обычные вызовы или вызов активации.

Маркер олицетворения, предоставленный клиентским приложением, имеет уровень `ImpersonationIdentify` и может вызвать большие проблемы, если сервер не полностью инициализирован.

Это один из потенциальных маркеров доступа олицетворения с огромными ограничениями, если он в итоге используется в глобальной инициализации, как описано в предыдущем разделе "Security Problems During Deferred Initialization".

Хотя реализовывать полноценный DCOM-сервер встречается не очень часто, все эти ограничения обычно встречаются при написании клиентского кода, использующего асинхронные callback-парадигмы.

Каждый раз, когда клиентский код передает callback-интерфейс, который должен вызываться извне клиентского процесса, базовая инфраструктура запускает DCOM-сервер, и применяются все проверки и настройки.

В этом случае клиентский код берет на себя роль сервера и выполняет все проверки доступа, описанные в этом разделе.

Начиная с Windows XP SP2, инфраструктура DCOM предоставляет журналирование нескольких отказов, встречающихся в обычной работе, с использованием NT Event Log, когда в реестре установлены следующие ключи:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Ole\CallFailureLoggingLevel
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Ole\ActivationFailureLoggingLevel
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Ole\InvalidSecurityDescriptorLoggingLevel
```

NOTE Поскольку RPCSS является базовой службой, часто используемой инфраструктурой DCOM, любая точка останова, установленная в службе, срабатывает очень часто, и источник вызова нужно проверять, чтобы не тратить время на трассировку несвязанных вызовов активации. Кроме того, каждый раз, когда один из системных процессов прерывается под отладчиком, функциональность машины нарушается.

Отказ команды расширения !token

В разделе "Local Security Failures" попытка исследовать маркер олицетворения с помощью команды расширения `!token` завершилась отказом в доступе.

Хотя исправить расширение невозможно, полезно понять причину отказа и методологию, использованную для ее выяснения.

Первым шагом должно быть понимание логического пути выполнения, приводящего к этой ошибке. Следующий шаг - подтвердить путь выполнения с помощью отладчика, установив точки останова в основных точках пути выполнения.

Как описано в главе 2, "Introduction to the Debuggers", в ответ на команду расширения !token отладчик выполняет метод с именем token, реализованный в одной библиотеке расширений (в данном случае exts.dll).

Поскольку расширение выполняется внутри отладчика, необходимо подключить новый отладчик к отладчику, выполняющему расширение.

"Отладчик отладчика" можно легко запустить, введя команду .dbgdbg в командной строке, или запустить его из командной строки, что обычно используется при разработке расширений.

Поскольку маркер олицетворения и первичный маркер защищены ядром, API, обеспечивающие доступ к этим маркерам, являются правильным местом для перехвата вызовов расширения.

Расширение использует недокументированные API, предоставляемые ntdll.dll, функционально похожие на документированные API из advapi32.dll.

Мы узнаем это, устанавливая в "отладчике отладчика" точки останова на все API, реализующие функции с похожими именами:

```
0:000> x *!*OpenProcessToken*
77dd7753 ADVAPI32!OpenProcessToken = <no type information>
77dd1364 ADVAPI32!_imp__NtOpenProcessToken = <no type information>
77e71350 RPCRT4!_imp__OpenProcessToken = <no type information>
7c801434 kernel32!_imp__NtOpenProcessToken = <no type information>
7c90dd90 ntdll!NtOpenProcessToken = <no type information>
7c90dda5 ntdll!NtOpenProcessTokenEx = <no type information>
...
0:000> bp ntdll!NtOpenProcessToken
0:000> bp ntdll!NtOpenThreadToken
0:000> g
```

После повторного вызова команды расширения !token в отладчике выполнение останавливается в "отладчике отладчика".

Каждый API возвращает ошибку отказа в доступе, объясняя ошибку, отображенную расширением. В листинге 7.30 показано, как выполнить текущую функцию после срабатывания точки останова и где искать код ошибки.

Листинг 7.30. Перехват отказов NtOpenThreadToken и NtOpenProcessToken

```
0:000> g
Breakpoint 1 hit
eax=000007a4 ebx=7ffda000 ecx=00000000 edx=0007dc78 esi=00000000 edi=0007dd04
eip=7c90de0e esp=0007dc5c ebp=0007dc80 iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
ntdll!NtOpenThreadToken:
7c90de0e b881000000 mov eax,0x81
0:000> * Execute the current function, OpenThreadToken and return
0:000> gu
eax=c0000022 ebx=7ffda000 ecx=0007dc58 edx=7c90eb94 esi=00000000 edi=0007dd04
eip=01936cf8 esp=0007dc70 ebp=0007dc80 iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
exts!tls+0xbb8:
01936cf8 8945f4 mov [ebp-0xc],eax ss:0023:0007dc74=00000000
0:000> * Notice the NT_STATUS access denied error in eax register
0:000> g
Breakpoint 0 hit
eax=00000000 ebx=7ffda000 ecx=0007dc78 edx=0000079c esi=00000000 edi=0007dd04
eip=7c90dd90 esp=0007dc60 ebp=0007dc80 iopl=0 nv up ei pl nz ac pe nc
ntdll!NtOpenProcessToken:
7c90dd90 b87b000000 mov eax,0x7b
0:000> * Execute the current function, OpenProcessToken
0:000> gu
eax=c0000022 ebx=7ffda000 ecx=0007dc58 edx=7c90eb94 esi=00000000 edi=0007dd04
eip=01936cf8 esp=0007dc70 ebp=0007dc80 iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
exts!tls+0xbb8:
01936cf8 8945f4 mov [ebp-0xc],eax ss:0023:0007dc74=00000000
0:000> * Notice the NT_STATUS access denied error in eax register
```

Поскольку нет простого способа идентифицировать дескрипторы безопасности, защищающие ресурсы, участвующие в этом отказе, мы запускаем отладчик ядра, чтобы исследовать дескрипторы безопасности маркеров доступа и маркеры доступа, используемые вызывающим кодом.

Поскольку полноценный сеанс отладчика ядра доступен не всегда, локального отладчика ядра достаточно.

Исследование, показанное в листинге 7.31, сосредоточено на первичном маркере, который открывается API `ntdll!NtOpenProcessToken`.

Листинг 7.31. Первичный маркер процесса `wmiprvse.exe`

```
lkd> * Finding the token used by the process executing wmiprvse.exe
lkd> !process 0 1 wmiprvse.exe
PROCESS 81a71da0 SessionId: 0 Cid: 03f4 Peb: 7ffd8000 ParentCid: 0320
DirBase: 0a848000 ObjectTable: e21f59c8 HandleCount: 159.
Image: wmiprvse.exe
VadRoot 8203e5b0 Vads 109 Clone 0 Private 377. Modified 89. Locked 0.
DeviceMap e1881148
Token e18b2a68
...
lkd> * Displaying the token information
lkd> !token e18b2a68 -n
_TOKEN e18b2a68
TS Session ID: 0
User: S-1-5-20 (Well Known Group: NT AUTHORITY\NETWORK SERVICE)
Groups:
00 S-1-5-20 (Well Known Group: NT AUTHORITY\NETWORK SERVICE)
Attributes - Mandatory Default Enabled
...
Impersonation Level: Impersonation
TokenType: Primary
Source: Advapi TokenFlags: 0x81 ( Token in use )
Token ID: 34e00f ParentToken ID: 0
Modified ID: (0, 34de7a)
RestrictedSidCount: 0 RestrictedSids: 00000000
```

Поскольку отладчик всегда имеет полный доступ к целевому процессу отладчика, единственной причиной отказа доступа при открытии первичного маркера может быть дескриптор безопасности первичного маркера.

В листинге 7.32 показан дескриптор безопасности, защищающий маркер, полученный в предыдущем листинге.

Листинг 7.32. Дескриптор безопасности первичного маркера

```
lkd> !sd poi(e18b2a68-4) & FFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
          SE_DACL_PRESENT
          SE_SELF_RELATIVE
->Owner : S-1-5-20
->Group : S-1-5-20
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x30
->Dacl : ->AceCount : 0x2
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x14
->Dacl : ->Ace[0]: ->Mask : 0x000f01ff
->Dacl : ->Ace[0]: ->SID: S-1-5-18
->Dacl : ->Ace[1]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[1]: ->AceFlags: 0x0
->Dacl : ->Ace[1]: ->AceSize: 0x14
->Dacl : ->Ace[1]: ->Mask : 0x000f01ff
```

```
->Dacl : ->Ace[1]: ->SID: S-1-5-20
->Sacl : is NULL
```

Дескриптор безопасности первичного маркера не позволяет системным администраторам получить дескриптор к нему.

Поскольку отладчик выполняется под принципом администратора, отличным от LocalSystem или NetworkService, первичный маркер недоступен команде расширения !token.

Отказ при открытии маркера олицетворения вызван похожей несовместимостью между объектом потока и учетной записью администратора, под которой выполняется отладчик.

Отказ активации DCOM в Windows XP SP2 после установки приложения

Последний пример отладки выполняется на ранее здоровой системе под управлением Windows XP SP2, которая начинает вести себя странно после перезагрузки, запрошенной установкой приложения.

Система не может активировать ни один DCOM-сервер, что затрагивает большинство административных MMC-оснасток.

Даже после включения всех настроек трассировки DCOM, описанных ранее в разделе "DCOM Call Access Checks", нет ясного сообщения, указывающего на корневую причину проблемы.

Мы начинаем отладку, используя ранее обсуждавшуюся модель остановки каждого процесса, являющегося частью пути активации, в отладчике при повторной попытке клиентской активации.

Первый процесс снизу пути вызова, при остановке которого клиент зависает, - это процесс, размещающий службу DcomLaunch.

Пока эта служба остановлена в отладчике, никакие процессы, являющиеся частью пути активации, а именно клиент, выполняющий вызов активации, процесс, размещающий службу RPCSS, и процесс, размещающий DcomLaunch, не изменяются и могут быть исследованы.

Мы ожидаем, что в это время в клиентском процессе будет как минимум один поток с вызовом API ole32!CocreateInstanceEx в стеке.

Поэтому мы подключаем отладчик пользовательского режима к клиентскому процессу и выводим стек для всех потоков.

Стек клиентской активации, доступный в листинге 7.33, показывает поток, ожидающий ответа на локальный RPC-вызов, о чем говорит наличие rpcrt4!LRPC_CALL в стеке.

Ожидание и видимое зависание клиента вызваны остановками отладчика в процессе, размещающем службу DcomLaunch.

Листинг 7.33. Клиентский поток, ожидающий ответ RPC

```
0:001> ~0 k
ChildEBP RetAddr
0013de30 7c90e3ed ntdll!KiFastSystemCallRet
0013de34 77e7c968 ntdll!NtRequestWaitReplyPort+0xc
0013de80 77e7a716 RPCRT4!LRPC_CCALL::SendReceive+0x228
...
0013e4f0 77545fc8 ole32!CRpcResolver::CreateInstance+0x13d
0013e73c 7752f4f5 ole32!CClientContextActivator::CreateInstance+0xfa
0013e77c 7752f33a ole32!ActivationPropertiesIn::DelegateCreateInstance+0xf7
0013ef2c 77526000 ole32!ICoCreateInstanceEx+0x3c9
0013ef54 77525fcf ole32!CComActivator::DoCreateInstance+0x28
0013ef78 74ef18c1 ole32!CoCreateInstanceEx+0x1e
...
```

Поскольку ошибка, возвращаемая клиенту, всегда была ошибкой отказа в доступе, следующий логический шаг - идентифицировать принципал, под которым выполняются вызывающие потоки.

Как и раньше, мы используем команду расширения !token, чтобы получить текущий маркер доступа олицетворения потока.

Поскольку команда расширения действует на текущий поток, первый шаг делает поток ноль активным потоком.

Листинг 7.34. Маркер клиента, выполняющего активацию

```
0:001> ~0s
0:000> !token -n
Thread is not impersonating. Using process token...
TS Session ID: 0
User: S-1-5-21-1060284298-2111687655-1957994488-1003 (User: XP-SP2-BACK\TestAdmin)
Groups:
00 S-1-5-21-1060284298-2111687655-1957994488-513 (Group: XP-SP2-BACK\None)
Attributes - Mandatory Default Enabled
01 S-1-1-0 (Well Known Group: localhost\Everyone)
Attributes - Mandatory Default Enabled
02 S-1-5-32-544 (Alias: BUILTIN\Administrators)
Attributes - Mandatory Default Enabled Owner
...
Auth ID: 0:45550
Impersonation Level: Anonymous
TokenType: Primary
```

Поток не выполняет олицетворение; следовательно, он использует первичный маркер, представляющий локального администратора, достаточно мощного, чтобы сделать почти все на этой системе.

Мы возвращаемся к процессу, размещающему службу DcomLaunch, чтобы понять, что именно отказывает внутри этого процесса.

Как видно из листинга 7.34, почти каждый DCOM-вызов пытается получить маркер доступа олицетворения, представляющий вызывающего, прежде чем выполнять работу от имени клиента, используя базовые функции олицетворения протокола.

Следовательно, нужно понять, какая конкретная идентичность выполняет вызов, установив точку останова на `rpcrt4!RpcImpersonateClient` и проверив олицетворение потока при возврате, как в листинге 7.35.

Листинг 7.35. Отказ `RpcImpersonateClient` в `DcomLaunch`

```
0:019> bp RPCRT4!RpcImpersonateClient "g @$ra"
0:019> g
eax=00000005 ebx=000c0b78 ecx=0065f7b4 edx=7c90eb94 esi=00000000 edi=0065f854
eip=76a822fc esp=0065f7dc ebp=0065f7f0 iopl=0 nv up ei ng nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000286
rpcss!LookupOrCreateTokenForRPCClient+0x24:
76a822fc 8b1d2014a876 mov ebx,[rpcss!_imp__GetCurrentThread
(76a81420)]{kernel32!GetCurrentThread (7c809919)} ds:0023:76a81420=7c809919
0:003> k
ChildEBP RetAddr
0065f7f0 76a95dad rpcss!LookupOrCreateTokenForRPCClient+0x24
0065f858 77e79dc9 rpcss!_LaunchActivatorServer+0x55
0065f8b4 77ef321a RPCRT4!Invoke+0x30
...
0:003> !token
Thread is not impersonating. Using process token...
TS Session ID: 0
User: S-1-5-18
Groups:
00 S-1-5-32-544
Attributes - Default Enabled Owner
01 S-1-1-0
Attributes - Mandatory Default Enabled
02 S-1-5-11
Attributes - Mandatory Default Enabled
Primary Group: S-1-5-18
...
Auth ID: 0:3e7
Impersonation Level: Anonymous
TokenType: Primary
0:003> reax
```



```

eax=00000005
0:003> !error 5
Error code: (Win32) 0x5 (5) - Access is denied.

```

После попытки олицетворения поток все еще не выполняет олицетворение, поскольку API завершился отказом в доступе.

Пора посмотреть путь выполнения ближе к клиенту, в процессе, размещающем службу RPCSS, и идентифицировать поток, выполняющий этот вызов.

Быстрый просмотр потоков показывает поток из листинга 7.36 с ожидающим RPC-вызовом. Однако получить олицетворение потока невозможно по причинам, описанным в предыдущем разделе.

Листинг 7.36. Поток RPCSS с ожидающим вызовом

```

0:008>k
ChildEBP RetAddr
0099f528 7c90e9c0 ntdll!KiFastSystemCallRet
0099f52c 7c8025db ntdll!NtWaitForSingleObject+0xc
0099f590 7c802542 kernel32!WaitForSingleObjectEx+0xa8
0099f5a4 76a92fad kernel32!WaitForSingleObject+0x12
0099f608 76a92a4a rpcss!CClsidData::ServerLaunchMutex+0x0c
0099f65c 76a8e4ab rpcss!Activation+0x384
0099f6b8 76a91e12 rpcss!ActivateFromProperties+0x213
0099f6c8 76a91e66 rpcss!CScmActivator::CreateInstance+0x10
0099f708 76a91e7b rpcss!ActivationPropertiesIn::DelegateCreateInstance+0xf7
0099f754 76a8c1d7 rpcss!ActivateFromPropertiesPreamble+0x4c1
0099f79c 76a91de7 rpcss!PerformScmStage+0xbb
0099f8b0 77e79dc9 rpcss!SCMAActivatorCreateInstance+0x97
0099f8e0 77ef321a RPCRT4!Invoke+0x30
...
0099fdfc 77e7bb6a RPCRT4!LRPC_SCALL::DealWithRequestMessage+0x2db
0099fe20 77e76784 RPCRT4!LRPC_ADDRESS::DealWithLRPCRequest+0x16d
0:008> !token
Thread is not impersonating. Using process token...
Error 0xc0000022 getting thread token

```

Чтобы получить маркер олицетворения, мы используем методику, представленную в предыдущем разделе "!"token Extension Command Failure", с помощью отладчика режима ядра в локальном режиме.

Результат этого шага показан в листинге 7.37.

Листинг 7.37. Маркер олицетворения проблемного потока RPCSS

```

lkd> !thread 815aada8
THREAD 815aada8 Cid 035c.0fac Teb: 7ffd4000 Win32Thread: 00000000 WAIT: (Suspended)
KernelMode Non-Alertable
SuspendCount 1
FreezeCount 1
815aaf44 Semaphore Limit 0x2
Waiting for reply to LPC MessageId 00015a17:
Current LPC port e1dc2480
Impersonation token: e23ce530 (Level Identification)
Owning Process 8217a520 Image: svchost.exe
Wait Start TickCount 657309 Elapsed Ticks: 1362
Context Switch Count 570
UserTime 00:00:00.0000
KernelTime 00:00:00.0020
Start Address kernel32!BaseThreadStartThunk (0x7c810856)

```

Маркер олицетворения в этом потоке на уровне SecurityIdentification является настоящей причиной отказа в службе DcomLaunch Server, поскольку маркер на этом уровне не может распространяться в последовательный удаленный процесс.

Это полностью противоречит исходному маркеру доступа вызывающего или намерениям клиентского кода.

Это больше похоже на проблему с механизмом олицетворения, используемым службой RPCSS Server.

После некоторого исследования на сайте Microsoft MSDN мы нашли ссылку на новую привилегию, добавленную в Windows Server 2003 и позднее в Windows XP SP2, с именем SeImpersonatePrivilege, которая влияет на уровень олицетворения, получаемый после олицетворения маркера доступа клиента.

Кроме того, в Local Security Policy, показанной на рисунке 7.7, видно, что SeImpersonatePrivilege не предоставлена идентичности NetworkService; таким образом, увиденная ранее ошибка нормальна.

Предоставление этой привилегии учетной записи SERVICES, включающей NetworkService, и перезапуск системы восстанавливают функциональность системы.

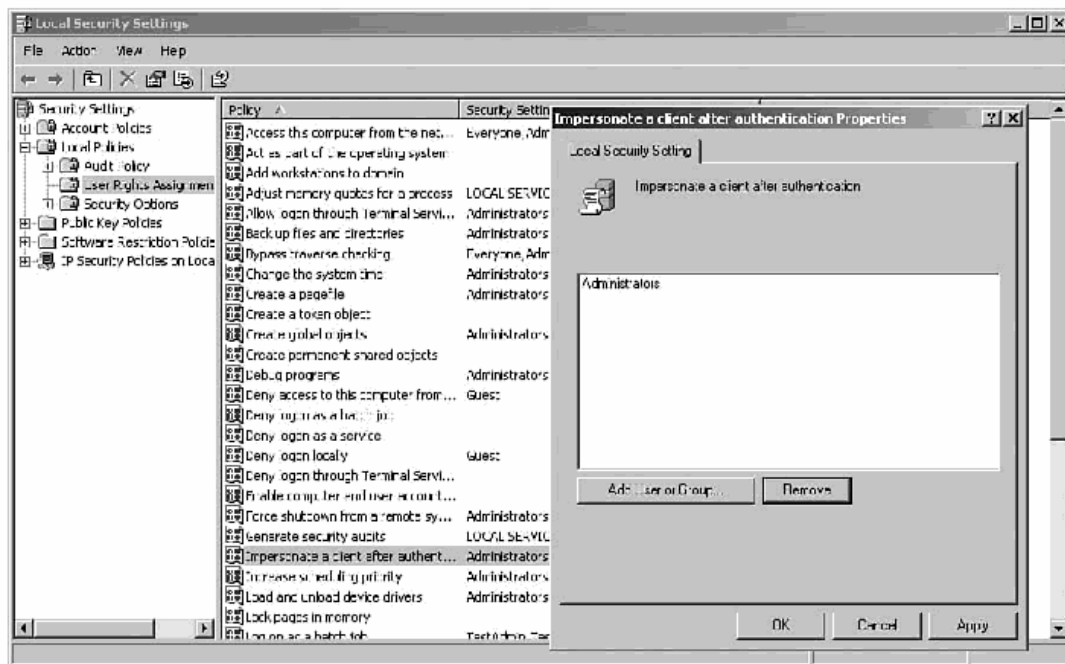


Figure 7.7 Impersonate client assignment

Рисунок 7.7. Назначение Impersonate client

Исследование отказов безопасности с помощью средств трассировки

Общая причина случаев отказа в доступе, представленных до сих пор в этой главе, - несовместимость между принципом, пытающимся обратиться к объекту, и дескриптором безопасности, защищающим этот объект.

Кроме того, довольно легко понять, какие части участвуют в операции, а информация безопасности легко доступна из отладчиков Windows.

На другом конце спектра находятся случаи отказа в доступе в сложных приложениях с относительно неизвестной архитектурой, где ошибки возникают главным образом при обращении к защищенным ресурсам за их границей безопасности.

В таких случаях исследование следует начинать с различных средств трассировки, чтобы понять, к каким ресурсам выполняется доступ, как выполняется доступ и в каком порядке выполняется

доступ.

Process Monitor - один из таких инструментов, показывающий в реальном времени файловую активность и активность реестра на локальной системе.

Когда приложение взаимодействует с другими компьютерными системами, трассировка сети является лучшим способом обнаружить сетевую активность и ошибку отказа в доступе, с которой сталкивается приложение.

В следующей главе используется инструмент сетевого мониторинга для наблюдения за поведением удаленного приложения.

Все обращения к файловой системе и реестру, выполненные в разделе "DCOM Activation Checks", легко трассируются.

Например, операции доступа к файлам и их результаты явно отображаются инструментом Process Monitor, как показано на рисунке 7.8, после скрытия реестра и активности процессов.

В этом случае дескриптор безопасности, защищающий файл образа сервера, был вручную изменен, чтобы запретить доступ локальным администраторам.

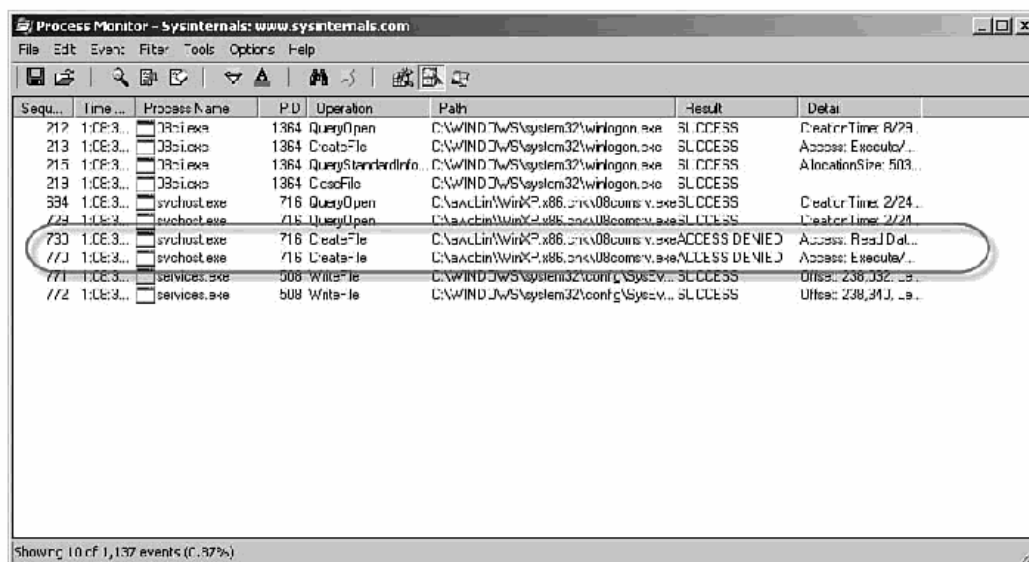


Figure 7.8 Detecting file errors using the tracing tools

Рисунок 7.8. Обнаружение файловых ошибок с помощью средств трассировки

На рисунке 7.8 легко увидеть, как процесс svchost.exe, размещающий DcomLaunch, пытается открыть файл образа серверного процесса и получает ошибки отказа в доступе.

Эта трассировка может выявить и другие ошибки доступа к файлам, а также другие ошибки, встречающиеся сервером после запуска процесса.

На рисунке 7.9 показаны ошибки, встреченные серверным процессом при попытке доступа к нескольким разделам реестра.

Пути реестра нужно сопоставлять с доступной информацией о компоненте, чтобы понять, что пошло не так.

Обычно мы фильтруем активность по имени исполняемого файла или по пути доступных объектов.

Ошибки, показанные на рисунке 7.9, вызваны неправильной регистрацией proxy-stub-модуля, используемого приложением при обращении к одному интерфейсу.

Вооружившись этой информацией и общим представлением об инфраструктуре, очень легко найти решение: повторно зарегистрировать proxy-stub на системе, размещающей серверный процесс.

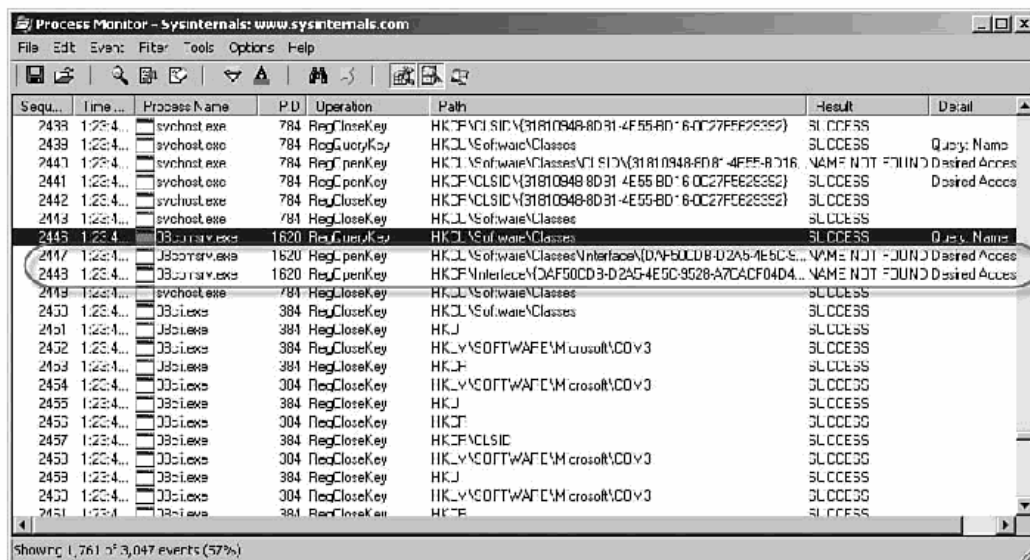


Figure 7.9 Detecting registry errors using the tracing tools

Рисунок 7.9. Обнаружение ошибок реестра с помощью средств трассировки

Резюме

В этой главе вы изучили базовый механизм, используемый операционной системой для управления доступом к различным ресурсам, механизм, используемый для идентификации принципалов, и способ исследования каждого из этих элементов с помощью отладчиков Windows. Кроме того, вы узнали, где хранится информация безопасности и как она распространяется от одного процесса к другому или от одной системы к другой.

Затем вы использовали эти знания, чтобы понять несколько ошибок отказа в доступе, встречающихся в приложениях: от очень простой ошибки отказа в доступе "внутри процесса" до сложных случаев, связанных с Distributed COM.

Используя те же инструменты и похожие эвристики, теперь вы можете справиться с любым отказом безопасности, встреченным в процессе разработки или на этапе развертывания.

Глава 8. Межпроцессное взаимодействие

Много лет назад программные компоненты в основном работали изолированно, почти не взаимодействуя друг с другом. Ограниченное взаимодействие выполнялось с помощью собственных механизмов, редко используемых несколькими компонентами, - механизмов, основанных на операциях файловой системы или сетевых протоколах, таких как IP или UDP. Понимать взаимодействие между компонентами могли лишь люди, знавшие детали приложения.

Сегодня повсеместная клиент-серверная архитектура изменила программный ландшафт даже для простых приложений. Если приложения MS-DOS когда-то записывали данные непосредственно в буфер видеопамати, чтобы обновить видимое состояние приложения, то современные компоненты Windows выполняют вызовы системных API, чтобы состояние приложения было обновлено. Под системным API Windows вызывает процесс, отвечающий за управление всеми окнами, используя один из процессов взаимодействия, описанных в этой главе. Другое приложение записывает

событие в Event Log, что приводит к межпроцессному вызову службы, отвечающей за управление Event Log.

Современные решения все чаще используют несколько систем, работающих в нескольких процессах. Одни применяют этот механизм для отказоустойчивости или изоляции безопасности, другие - просто для достижения уровней масштабируемости, недоступных однопроцессным системам. Незнание того, как ориентироваться в этой сложной инфраструктуре, ставит инженеров в странное положение: у них есть все знания для решения бизнес-задачи, которую решает программная система, но они не могут быстро обнаружить проблему, потому что весь процесс межпроцессного взаимодействия мешает легко понять ее настоящую причину.

Эта глава дает инструменты и сведения, необходимые для успешного исследования проблем в связанных программных средах, то есть проблем, затрагивающих более одного процесса или более одного компьютера. Мы сосредоточимся на нескольких примитивах взаимодействия и введем несколько новых инструментов. В этой главе вы получите ответы на несколько базовых вопросов о клиент-серверном приложении, например:

- Когда клиентский вызов завершается ошибкой, как найти место и причину этого отказа?
- Когда сервер не отвечает предсказуемым образом и его нужно отлаживать, какой поток, процесс и система отвечают за блокировку вызова?
- Когда сервер вызывается с недопустимыми параметрами, как определить клиента, который вызвал этот метод сервера?

Мы используем новую команду расширения `!lpc`, доступную в расширениях отладчиков Windows, загружаемых по умолчанию. Пример этой главы - распределенное COM-приложение, состоящее из клиентского приложения `08cli.exe`; динамической библиотеки `08comps.dll`, содержащей `проxy-stub`-код взаимодействия; и серверного приложения `08comsrv.exe`. Исходный код и двоичные файлы находятся в следующих папках:

```
Source code: C:\AWD\Chapter8
Binary: C:\AWDBIN\winXP.x86.chk\08cli.exe, 08comps.dll, and
08comsrv.exe.
```

Механизмы взаимодействия

Современные операционные системы Windows, такие как Windows XP и Windows Server 2003, имеют встроенную поддержку нескольких протоколов взаимодействия. Протоколы транспортного уровня, такие как IP с установлением соединения или датаграммный UDP, можно напрямую использовать для простых форм межпроцессного взаимодействия. Однако у приложений могут быть сложные требования, например надежное или защищенное взаимодействие, которые нужно выполнить с минимальным объемом кода. Кроме того, взаимодействие между системами с разной архитектурой, например между системой с 64-битной процессорной архитектурой и системой с 32-битной процессорной архитектурой, должно работать прозрачно. Сообщения, которыми обмениваются разнородные системы, должны быть независимы от типа процессора, операционной системы и характеристик компилятора.

В таких случаях разработчики выбирают протоколы взаимодействия сеансового уровня, реализующие все эти требования. DCE Remote Procedure Call (DCE/RPC) - один из таких протоколов, удовлетворяющих перечисленным требованиям. RPC используется для реализации привычной парадигмы взаимодействия "вызов-ответ" между компонентами, находящимися в разных процессах или физических системах. Среда выполнения RPC предоставляет механизмы, необходимые для маршалинга и демаршалинга сообщений, передаваемых между клиентским и серверным процессом, которые используются для реализации парадигмы "вызов-ответ". Реализация протокола RPC от Microsoft, называемая MSRPC, может использовать любой протокол сеансового

уровня или более низкого уровня, доступный между клиентом и сервером, включая TCP/IP, Named Pipe или HTTP. Неудивительно, что большинство административных инструментов в операционной системе Windows используют MSRPC для взаимодействия с серверами, которыми они управляют.

С появлением практик объектно-ориентированного программирования разработчики стали искать протоколы взаимодействия, облегчающие эти практики. Microsoft создала инфраструктуру Distributed Common Object Model (DCOM) поверх инфраструктуры MSRPC. В качестве добавочной ценности к MSRPC инфраструктура DCOM предоставляет возможность активировать, использовать и уничтожать объекты, реализующие несколько интерфейсов. Временем жизни объектов DCOM явно управляет клиентское приложение. Случайно отсоединенные объекты периодически возвращаются распределенным сборщиком мусора DCOM.

Объекты DCOM можно создавать практически на любом языке программирования, и их можно использовать из любого языка или инструмента, способного с ними работать. Более новые языки программирования, основанные на среде выполнения .NET, могут прозрачно взаимодействовать с объектами DCOM, представляя эти объекты DCOM как объекты .NET.

Взаимодействие между двумя процессами, работающими на одном физическом хосте, нативно поддерживается ядром Windows в форме Local Procedure Call (LPC). MSRPC, использующий LPC, часто называют Local RPC или LRPC. На рисунке 8.1 показаны связи между различными протоколами взаимодействия, доступными в операционной системе Windows, что помогает понять весь стек протоколов, полезный при отладке межпроцессного взаимодействия.

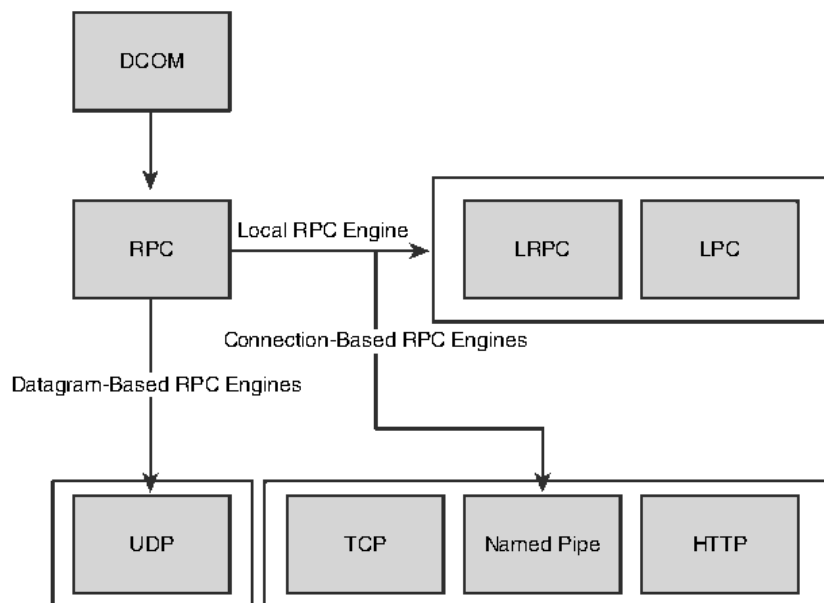


Figure 8.1 Relationship between various communication protocols available in Windows operating systems

Рисунок 8.1. Связь между различными протоколами взаимодействия, доступными в операционных системах Windows

Большинство приемов, используемых при отладке конкретного протокола, применяются и для отладки любого протокола, производного от него или использующего его как основу взаимодействия. Например, чтобы отлаживать взаимодействие между двумя процессами, использующими DCOM, разработчик также должен отлаживать LRPC-взаимодействие между клиентским и серверным процессом.

Устранение неполадок локального взаимодействия

Важность локального взаимодействия между различными процессами нельзя игнорировать. Объекты автоматизации, которые предоставляются или используются всеми сложными приложениями, управляются последовательностью DCOM-вызовов к объектам, реализованным различными серверами. Велика вероятность, что рано или поздно инженер либо будет предоставлять службу, либо будет использовать службу, предоставленную чужими компонентами. Когда клиент и сервер работают в разных процессах, вызовы не всегда выполняются ожидаемым образом. Клиент может передать неверные аргументы, например контекст безопасности. Аналогично сервер может обрабатывать запрос гораздо дольше ожидаемого. В таких случаях инженер вынужден отлаживать взаимодействие между этими процессами.

К счастью, взаимодействие между локальными компонентами обычно выполняется с использованием протоколов, построенных вокруг протокола LPC. Владение этим базовым протоколом, которому посвящен этот раздел, существенно важно для отладки операционной системы Windows. Протокол LPC удовлетворяет набору противоречивых требований, которые трудно выполнить при локальном взаимодействии с помощью других протоколов:

- Канал взаимодействия между клиентом и сервером защищен; никакой другой процесс, кроме ядра Windows, не может наблюдать, перехватывать или изменять сообщения, которыми обмениваются клиент и сервер.
- Взаимодействие между клиентом и сервером оптимизировано для производительности.
- Синхронное взаимодействие между клиентом и сервером полностью трассируемо; в любой момент процесса взаимодействия клиент знает, какой серверный поток выполняет запрос, а сервер знает, какой клиент сделал запрос. Кроме того, чтобы включить такую трассировку, не нужно ничего менять в системе или добавлять специальную инструментализацию. Это очень важный аспект отладки работающих систем, и он показывает, что протокол создавался с учетом возможности отладки.

Однако не все локальное взаимодействие выигрывает от возможностей LPC, поскольку существуют отдельные случаи, когда локальное взаимодействие выполняется нетрадиционными способами. Например, два процесса могут отправлять друг другу оконные сообщения, могут использовать MSRPC поверх сетевого протокола или даже напрямую использовать протокол транспортного уровня. Раздел "Устранение неполадок удаленного взаимодействия" посвящен отладке взаимодействия, использующего RPC поверх сетевых протоколов. LPC-взаимодействие отлаживается с помощью отладчика режима ядра, подключенного к системе или работающего в локальном режиме.

Предпосылки LPC

Несмотря на то что протокол не документирован Microsoft, существует множество справочных материалов, помогающих сформировать достаточно хорошее понимание этого протокола и уверенно отлаживать его. История LPC восходит к первым дням операционной системы Windows NT, когда клиент-серверная архитектура, использованная в ядре операционной системы, потребовала нового протокола взаимодействия, соответствующего строгим требованиям производительности. Протокол LPC поддерживается набором API, реализованных непосредственно ядром Windows и предоставленных коду пользовательского режима рядом функций внутри `ntdll.dll`, имеющих форму `ntdll!Nt[operation]Port`.

Чтобы понимать, как используется протокол, инженеры должны иметь базовое представление о его поведении. Основное взаимодействие проходит несколько важных шагов:

1. Сервер иницирует протокол, создавая именованный порт вызовом API `ntdll!NtCreatePort`. Этот порт называется портом подключения.
2. Сервер слушает этот порт подключения на предмет новых запросов взаимодействия с помощью API `ntdll!NtListenPort`. У сервера всегда должен быть поток, ожидающий на порту подключения.
3. Клиент иницирует новое соединение, отправляя серверу запрос подключения с помощью API `ntdll!NtConnectPort`. Запрос отправляется в порт, созданный на шаге 1.
4. Сервер проверяет запрос подключения и, исходя из своих политик, принимает соединение с помощью API `ntdll!NtAcceptConnectPort`, за которым следует вызов `ntdll!NtCompleteConnectPort`.
5. После установки соединения и клиент, и сервер получают объект коммуникационного порта, который можно использовать для фактического взаимодействия.
6. Сервер запускает цикл, выделенный для порта подключения: в нем он получает новое сообщение, обрабатывает его и отвечает клиенту, например с помощью API `ntdll!NtReplyWaitReceivePort`.
7. Клиент использует `ntdll!NtRequestWaitReplyPort`, чтобы отправить новый запрос серверу, и ожидает, пока сервер его обработает. Шаги 6 и 7 повторяются на протяжении всего диалога между клиентом и сервером.

Каждое сообщение, которым обмениваются клиент и сервер, имеет уникальный идентификатор `DWORD`, хранящийся в структуре `KTHREAD`, представляющей клиентский и серверный поток. Этот идентификатор используется для отслеживания пути вызова в отладчике режима ядра с помощью команды расширения `!lpc`.

Отладка LPC-взаимодействия

Каждый поток, участвующий в LPC-диалоге, хранит ссылку на сообщение, которое сейчас обрабатывается этим потоком. Эта ссылка выводится каждый раз при отображении информации о потоке. Иными словами, когда клиентский поток ожидает обработки LPC-запроса, идентификатор сообщения, соответствующий текущему запросу, доступен после выполнения команды расширения `!thread`. Аналогично, если серверный поток обрабатывает сообщение, идентификатор сообщения выводится командой расширения `!thread`. С помощью команды расширения `!lpc` всю информацию о клиентском порте подключения, серверном порте подключения, серверном коммуникационном порте и серверном процессе получают из информации, связанной с сообщением.

Чтобы показать, как использовать эту возможность, рассмотрим вызов, сделанный клиентом `08CLI.EXE` в метод `ICalculator::SlowSum`, реализованный сервером `08COMSRV.EXE`, который не возвращается за приемлемое время. В листинге 8.1 показан результат выполнения команды расширения `!thread` в отладчике режима ядра на клиентском потоке, инициировавшем запрос.

Листинг 8.1. Клиентский поток, ожидающий завершения LPC-запроса

```
kd> !thread ffb10020
THREAD ffb10020 Cid 05b4.04f8 Teb: 7ffdd000 Win32Thread: e16e5eb0 WAIT: (WnLpcReply) UserMode Non-Alertable
ffb10214 Semaphore Limit 0x1
Waiting for reply to LPC MessageId 00004f99:
Current LPC port e138cd98
Not impersonating
DeviceMap e1a60398
Owning Process ffaa62f0 Image: 08cli.exe
Wait Start TickCount 563720 Ticks: 1391 (0:00:00:13.930)
Context Switch Count 98 LargeStack
UserTime 00:00:00.0000
KernelTime 00:00:00.0530
```



```

Start Address kernel32!BaseProcessStartThunk (0x7c810867)
Win32 Start Address 08CLII!ILT+1385(_wmainCRTStartup) (0x0042c56e)
Stack Init f6c05000 Current f6c04c50 Base f6c05000 Limit f6c01000 Call 0
Priority 8 BasePriority 8 PriorityDecrement 0 DecrementCount 16
ChildEBP RetAddr Args to Child
f6c04c68 804dc6a6 ffb10090 ffb10020 804dc6f2 nt!KiSwapContext+0x2e
f6c04c74 804dc6f2 ffb10214 ffb101e8 ffb10020 nt!KiSwapThread+0x46
f6c04c9c 805788ef 00000001 00000011 e100da01 nt!KeWaitForSingleObject+0x1c2
f6c04d50 804df06b 000006e0 0015c2b8 0015c2b8 nt!NtRequestWaitReplyPort+0x63d
...

```

Состояние потока, содержащего LPC-информацию, явно расшифровано в третьей строке информации о потоке, показанной в листинге 8.1. Сообщение можно передать команде расширения `!lpc`, чтобы извлечь связанную с ним информацию, как показано в листинге 8.2. В этом случае команда использована для вывода информации о сообщении в форме `!lpc message <message_id>`.

Листинг 8.2. Использование расширения `!lpc` для получения информации о сообщении

```

kd> !lpc message 00004f99
Searching message 4f99 in threads ...
Server thread ffab41c0 is working on message 4f99
Client thread ffb10020 waiting a reply from 4f99
Searching thread ffb10020 in port rundown queues ...
Server communication port 0xe111b878
Handles: 1 References: 1
The LpcDataInfoChainHead queue is empty
Connected port: 0xe138cd98 Server connection port: 0xe14684f0
Client communication port 0xe138cd98
Handles: 1 References: 2
The LpcDataInfoChainHead queue is empty
Server connection port e14684f0 Name: OLE0D6120B10F36435E84795A344064
Handles: 1 References: 9
Server process : ffab3530 (08comsrv.exe)
Queue semaphore : 8124a248
Semaphore state 0 (0x0)
The message queue is empty
The LpcDataInfoChainHead queue is empty
Done.

```

Команда расширения извлекает доступную информацию о клиент-серверном взаимодействии. В выводе команды можно найти информацию о серверном процессе, включая имя его образа, имя порта подключения, а также дополнительные сведения, например длину очереди сообщений. Очередь содержит сообщения, ожидающие обслуживания процессом: сообщения, полученные как на порту подключения, так и на подключенном порту. В листинге 8.3 показан случай, когда серверный процесс остановлен в отладчике, а запросы подключения накапливаются на порту подключения. Адрес порта используется как аргумент команды расширения `!lpc port <port_id>`.

Листинг 8.3. Использование расширения `!lpc` для получения информации о порте

```

kd> !lpc port e13f6878
Server connection port e13f6878 Name: OLE9D3C2AF8298042C9A8D0FACAE0FA
Handles: 1 References: 10
Server process : ffb52020 (08comsrv.exe)
Queue semaphore : 8124f3d0
Semaphore state 2 (0x2)
Messages in queue:
0000 e13f8528 - Busy Id=00006dcd From: 0348.077c Context=80020000
[e13f6888 . e160a858]
Length=0044002c Type=00380001 (LPC_REQUEST)
Data: 00008701 00040342 00007801 000007f4 8f62e1ae 2ee99a5d
0000 e160a858 - Busy Id=00006f23 From: 0348.07f0 Context=80020000
[e13f8528 . e13f6888]
Length=0044002c Type=00380001 (LPC_REQUEST)
Data: 00005b01 00040342 00007801 000007f4 8f62e1ae 2ee99a5d
The message queue contains 2 messages
The LpcDataInfoChainHead queue is empty

```

Еще одна удобная возможность команды расширения `!lpc` - извлечение LPC-информации из потока, переданного как параметр, с помощью следующего синтаксиса: `!lpc thread <threadid>`. Если идентификатор потока опущен, команда расширения выводит всю LPC-активность, происходящую в системе на момент выполнения, как показано в листинге 8.4.

Листинг 8.4. Использование расширения `!lpc` для получения всей LPC-активности в системе

```
kd> !lpc thread
Searching message 0 in threads ...
Server thread 8118b7b8 is working on message 5ee
Client thread 81129da8 waiting a reply from 88f
Server thread 81271020 is working on message 1968
Server thread 8112c168 is working on message 47c7
Server thread 81130c98 is working on message 2f35
Server thread ffb952c8 is working on message 47c4
Server thread 8120fda8 is working on message 5fe
Server thread ffbc1c18 is working on message 887
Server thread ffbc7f0 is working on message 888
Server thread ffbc17f0 is working on message 88f
Server thread 81122768 is working on message 47ca
Server thread 811323b0 is working on message b6c
Server thread 81134568 is working on message 2fd1
Server thread 81206020 is working on message 4b3
Server thread 81211c58 is working on message 4943
Client thread ffb40da8 waiting a reply from f83
Server thread 8125d020 is working on message 26ff
Server thread ffb42da8 is working on message f83
Server thread ffb06a60 is working on message 2fff
Server thread ffaba020 is working on message 4d1c
Server thread ffb096c0 is working on message 29a5
Server thread ffab1020 is working on message 4e7c
Server thread ffab41c0 is working on message 4f99
Client thread ffb10020 waiting a reply from 4f99
Done.
```

Примечание. Впечатляет, сколько потоков взаимодействуют друг с другом в каждый момент времени даже на простаивающей машине.

Возможности отладки протокола LPC замечательны. Клиентский поток заблокирован, пока серверный поток обрабатывает сообщение, и это легко обнаружить, исследуя структуры ядра с помощью команды расширения `!lpc`. Зная эти методы, несложно расширить область отладки за пределы одного процесса, как это делалось на протяжении книги, до всей машины. Например, сценарии из главы о синхронизации, посвященные обнаружению взаимоблокировок внутри одного процесса, можно распространить на группу процессов, взаимодействующих с помощью протоколов на основе LPC.

Единственная оговорка во всем этом заключается в том, что LPC-информация доступна только из отладчика режима ядра. В более новых операционных системах, таких как Windows XP или Windows 2003, это не должно быть проблемой, потому что очень легко запустить отладчик ядра в локальном режиме и использовать его параллельно с другими отладчиками. Глава 2, "Введение в отладчики", служит хорошей справкой для ситуаций, в которых несколько отладчиков должны использоваться одновременно.

Но поскольку протокол LPC не документирован, он не используется напрямую за пределами ядра операционной системы Windows. За несколькими исключениями (системные API Windows, использующие LPC напрямую), разработчик сталкивается с протоколом LPC косвенно, через протокол LRPC или другие протоколы, построенные поверх него. Локальный вызов DCOM - один из таких протоколов, и именно ему посвящен следующий раздел.

Отладка локального взаимодействия DCOM и MSRPC

В наиболее распространенном сценарии клиент выполняет вызов в сервер, который не возвращается за разумное время. Первый шаг исследования - определить проблемный клиентский поток, ожидающий ответа сервера. Следующий шаг - определить серверный процесс и поток, обрабатывающий соответствующий вызов, если такой есть, и выяснить состояние этого потока. Поток может, например, ожидать другой объект ядра или пользовательский ввод.

Чтобы показать этот прием, мы снова используем клиент-серверный пример. Пример вызывает сервер синхронно в многопоточном COM-apartment, что напрямую отображается на синхронное LPC-взаимодействие. Пока серверный код ждет перед отправкой ответа, клиент зависает, что дает прекрасную возможность для отладки.

Мы запускаем 08CLI.EXE под отладчиком и даем ему свободно выполняться несколько секунд, чтобы завершилась последовательность инициализации. Окно времени, когда взаимодействие не отслеживается, несущественно, поскольку приложение будет ждать в зависшем состоянии гораздо дольше. В этом случае мы понимаем, что вызов ICalculator::SlowSum выполняется чрезвычайно медленно без какого-либо объяснения (кроме имени метода интерфейса). Следующий шаг - вывести стеки всех потоков и определить те потоки, в которых видна LRPC-активность. В листинге 8.5 видно первый поток, у которого в стеке есть метод объекта rpcrt4!LRPC_CCALL. В свою очередь, этот метод напрямую использует LPC API. Функция LPC, используемая в этом случае, ntdll!NtRequestWaitReplyPort, является хорошим индикатором вызова, инициированного клиентом. Клиент делает запрос серверу и ожидает ответа на LPC-порту. Этот прием работает только для синхронного RPC.

Листинг 8.5. Запуск клиента и вывод частичного стека вызовов для каждого потока

```
C:\windbg 08CLI.EXE
...
0:003> * The client has been running freely for a few seconds before stopping it
0:003> ~* k2
0 Id: 5b4.4f8 Suspend: 1 Teb: 7ffdd000 Unfrozen
ChildEBP RetAddr
0012f6e4 7c90e3ed ntdll!KiFastSystemCallRet
0012f6e8 77e7cc55 ntdll!NtRequestWaitReplyPort+0xc
0012f734 77e7aae6 RPCRT4!LRPC_CCALL::SendReceive+0x228
1 Id: 5b4.1d0 Suspend: 1 Teb: 7ffdc000 Unfrozen
ChildEBP RetAddr
00e9fe18 7c90e399 ntdll!KiFastSystemCallRet
00e9fe1c 77e76703 ntdll!NtReplyWaitReceivePortEx+0xc
00e9ff80 77e76c1b RPCRT4!LRPC_ADDRESS::ReceiveLotsaCalls+0xf4
2 Id: 5b4.278 Suspend: 1 Teb: 7ffdb000 Unfrozen
ChildEBP RetAddr
00b0ff1c 7c90d85c ntdll!KiFastSystemCallRet
00b0ff20 7c8023ed ntdll!NtDelayExecution+0xc
00b0ff78 7c802451 kernel32!SleepEx+0x61
# 3 Id: 5b4.bd0 Suspend: 1 Teb: 7ffdb000 Unfrozen
ChildEBP RetAddr
00b6ffc8 7c9507a8 ntdll!DbgBreakPoint
00b6ffff 00000000 ntdll!DbgUiRemoteBreakin+0x2d
```

Примечание. Соглашение об именовании объектов CCALL хорошо указывает на протокол, используемый для межпроцессного взаимодействия. LRPC_CCALL - клиентская сторона, способная обрабатывать локальные вызовы через LPC; OSF_CCALL указывает на взаимодействие с использованием протокола с установлением соединения, такого как TCP/IP или именованные каналы; а DG_CCALL указывает на взаимодействие с использованием датаграммного протокола, такого как UDP. Связь между этими протоколами показана на рисунке 8.1.

Исследование полного стека ранее определенного потока помогает точно определить, какой вызов функции зависает и какие уровни участвуют в обработке этого вызова. В случае, показанном в листинге 8.6, клиентский вызов использует DCOM, на что указывает применение методов из

ole32.dll, которая, в свою очередь, использует RPC и в конечном итоге LPC для диспетчеризации вызова серверу.

Листинг 8.6. Типичный стек клиентов, использующих DCOM поверх LRPC

```
0:003> ~0k
ChildEBP RetAddr
0012f6e4 7c90e3ed ntdll!KiFastSystemCallRet
0012f6e8 77e7cc55 ntdll!NtRequestWaitReplyPort+0xc
0012f734 77e7aae6 RPCRT4!LRPC_CCALL::SendReceive+0x228
0012f740 776016bf RPCRT4!I_RpcSendReceive+0x24
0012f75c 776011b6 ole32!ThreadSendReceive+0xf5
0012f778 7760109a ole32!CRpcChannelBuffer::SwitchAptAndDispatchCall+0x13d
0012f858 7751047c ole32!CRpcChannelBuffer::SendReceive2+0xb9
0012f8c4 77510414 ole32!CAptRpcChnl::SendReceive+0xab
0012f918 77ef3db5 ole32!CCtxComChnl::SendReceive+0x113
0012f934 77ef3ead RPCRT4!NdrProxySendReceive+0x43
0012fd10 77ef3e42 RPCRT4!NdrClientCall2+0x1fa
0012fd30 77e8a433 RPCRT4!ObjectStublessClient+0x8b
0012fd40 0042ea5b RPCRT4!ObjectStubless+0xf
0012fe48 0042e7ae 08CLI!MTAClientCall+0x7b
0012ff54 0042f902 08CLI!wmain+0xae
0012ffb8 0042f6bd 08CLI!wmainCRTStartup+0x252
0012ffc0 7c816fd7 08CLI!wmainCRTStartup+0xd
0012fff0 00000000 kernel32!BaseProcessStart+0x23
```

Даже если соответствующий клиентский поток определен, полезно понять, почему второй поток ожидает незавершенный LPC-вызов со стеком, показанным в листинге 8.7. Функция LPC, используемая в этом случае, ntdll!NtReplyWaitReceivePort, указывает на серверный поток, ожидающий получения нового запроса операции. Хотя может показаться немного странным, что у каждого DCOM-клиента также есть серверная роль, в начале главы мы говорили, что DCOM добавляет к стеку RPC дополнительные возможности, например распределенную сборку мусора. Этот поток является частью всего механизма, характерного для данного клиентского процесса. Клиентский процесс получает уведомление в этом потоке, когда сервер исчезает, и очищает все структуры, связанные с этим сервером.

Листинг 8.7. Типичный стек серверного потока, ожидающего новый запрос в DCOM поверх LRPC

```
0:003> ~1k
ChildEBP RetAddr
00e9fe18 7c90e399 ntdll!KiFastSystemCallRet
00e9fe1c 77e76703 ntdll!NtReplyWaitReceivePortEx+0xc
00e9ff80 77e76c1b RPCRT4!LRPC_ADDRESS::ReceiveLotsaCalls+0xf4
00e9ffb8 77e76a3d RPCRT4!RecvLotsaCallsWrapper+0xd
00e9ffa8 77e76c03 RPCRT4!BaseCachedThreadRoutine+0x79
00e9ffb4 7c80b683 RPCRT4!ThreadStartRoutine+0x1a
00e9ffec 00000000 kernel32!BaseThreadStart+0x37
```

Примечание. Как и соглашение об именовании объектов CCALL, соглашение об именовании объектов ADDRESS хорошо указывает на протокол, который слушает процесс. LRPC_ADDRESS - серверная сторона, ожидающая обработки локальных вызовов через LPC; OSF_ADDRESS указывает, что сервер ожидает на протоколах с установлением соединения, таких как TCP/IP или именованные каналы; а DG_CCALL указывает, что сервер ожидает на датаграммном протоколе, таком как UDP. Связь между этими протоколами показана на рисунке 8.1.

На этом этапе есть несколько способов выяснить, какой серверный поток обрабатывает клиентские запросы. Первый метод использует возможности отладки LPC для отслеживания обрабатываемого сообщения; для него требуется отладчик режима ядра. На следующем шаге инженер подключает к системе отладчик режима ядра или использует его внутри системы в локальном режиме, как описано в главе 2. Остальные шаги этого раздела выполняются из отладчика режима ядра.

LRPC-вызовы также можно отслеживать теми же методами, которые используются для отслеживания удаленных вызовов, то есть методами, использующими RPC troubleshooting state information. Этот метод описан в разделе "Устранение неполадок удаленного взаимодействия", и его без проблем можно использовать для LRPC-взаимодействия.

Еще один вариант - интерпретировать информацию, уже доступную в клиентском потоке, и извлечь сведения о сервере из структур MSRPC, используемых при выполнении вызова. К сожалению, этот метод невозможен с использованием публичных символов. Он также требует глубокого знания внутренних структур, хранящихся внутри MSRPC. Это наименее привлекательный метод для разработчиков, у которых нет доступа к приватным символам rpcrt4.dll.

Тот же экземпляр процесса 08cli.exe, запущенный в листинге 8.5, исследуется с помощью отладчика режима ядра. Мы используем команду расширения !process, чтобы вывести все потоки процесса, как показано в листинге 8.8.

Листинг 8.8. Вывод сводной информации о потоках

```
kd> !process 5b4 4
Searching for Process with Cid == 5b4
PROCESS ffaa62f0 SessionId: 0 Cid: 05b4 Peb: 7ffde000 ParentCid: 00d8
DirBase: 0a5d0000 ObjectTable: e10a97d0 HandleCount: 70.
Image: 08cli.exe
THREAD ffb10020 Cid 05b4.04f8 Teb: 7ffdd000 Win32Thread: e16e5eb0 WAIT
THREAD ffa6d698 Cid 05b4.01d0 Teb: 7ffdc000 Win32Thread: 00000000 WAIT
THREAD ffa6ada8 Cid 05b4.0278 Teb: 7ffdb000 Win32Thread: 00000000 WAIT
```

Помимо идентификатора процесса, нам известен идентификатор клиентского потока, который сопоставляется со всеми потоками из листинга 8.8, чтобы получить адрес структуры ETHREAD потока. Затем эта структура используется с командой расширения !thread, чтобы подтвердить корректность потока и получить LPC-информацию, как показано в листинге 8.9.

Листинг 8.9. Вывод информации о потоке ядра

```
kd> !thread ffb10020
THREAD ffb10020 Cid 05b4.04f8 Teb: 7ffdd000 Win32Thread: e16e5eb0 WAIT:
(WrLpcReply) UserMode Non-Alertable
ffb10214 Semaphore Limit 0x1
Waiting for reply to LPC MessageId 00004f99:
Current LPC port e138cd98
Not impersonating
DeviceMap e1a60398
Owning Process ffaa62f0 Image: 08cli.exe
Wait Start TickCount 563720 Ticks: 1391 (0:00:00:13.930)
Context Switch Count 98 LargeStack
UserTime 00:00:00.0000
KernelTime 00:00:00.0530
Start Address kernel32!BaseProcessStartThunk (0x7c810867)
Win32 Start Address 08CLI!ILT+1385(_wmainCRTStartup) (0x0042c56e)
Stack Init f6c05000 Current f6c04c50 Base f6c05000 Limit f6c01000 Call 0
Priority 8 BasePriority 8 PriorityDecrement 0 DecrementCount 16
ChildEBP RetAddr Args to Child
f6c04c68 804dc6a6 ffb10090 ffb10020 804dc6f2 nt!KiSwapContext+0x2e
6c04c74 804dc6f2 ffb10214 ffb101e8 ffb10020 nt!KiSwapThread+0x46
f6c04c9c 805788ef 00000001 00000011 e100da01 nt!KeWaitForSingleObject+0x1c2
f6c04d50 804df06b 000000e0 0015c2b8 0015c2b8 nt!NtRequestWaitReplyPort+0x63d
f6c04d50 7c90eb94 000000e0 0015c2b8 0015c2b8 nt!KiFastCallEntry+0xf8
(TrapFrame @ f6c04d64)
0012f6e4 7c90e3ed 77e7c968 000000e0 0015c2b8 ntdll!KiFastSystemCallRet
0012f6e8 77e7c968 000000e0 0015c2b8 0015c2b8 ntdll!NtRequestWaitReplyPort+0xc
0012f734 77e7a716 0015c2f0 0012f75c 776009c0 RPCRT4!LRPC_CCALL::SendReceive+0x228
0012f740 776009c0 0016149c 0015ecc0 0012f840 RPCRT4!I_RpcSendReceive+0x24
...
0012fe48 0042e7ae a2b35800 01c6e05c 7ffde000 08CLI!MTAClientCall+0x7b
0012ff54 0042f902 00000002 00372e20 00372ea0 08CLI!wmain+0xae
0012ffb8 0042f6bd 0012fff0 7c816d4f a2b35800 08CLI!wmainCRTStartup+0x252
0012ffc0 7c816d4f a2b35800 01c6e05c 7ffde000 08CLI!wmainCRTStartup+0xd
0012fff0 00000000 0042c56e 00000000 78746341 kernel32!BaseProcessStart+0x23
```

Информация о потоке содержит состояние этого потока, расшифрованное как WAIT: (WtLpcReply), а также LPC-сообщение, для которого ожидается ответ. Затем информация о сообщении используется, чтобы выяснить, какой серверный поток удерживает выполнение клиента, как показано в листинге 8.10.

Листинг 8.10. Получение дополнительной информации о LPC-сообщении

```
kd> !lpc message 00004f99
Searching message 4f99 in threads ...
Server thread ffab41c0 is working on message 4f99
Client thread ffb10020 waiting a reply from 4f99
Searching thread ffb10020 in port rundown queues ...
Server communication port 0xe111b878
Handles: 1 References: 1
The LpcDataInfoChainHead queue is empty
Connected port: 0xe138cd98 Server connection port: 0xe14684f0
Client communication port 0xe138cd98
Handles: 1 References: 2
The LpcDataInfoChainHead queue is empty
Server connection port e14684f0 Name: OLE0D6120B10F36435E84795A344064
Handles: 1 References: 9
Server process : ffab3530 (08comsrv.exe)
Queue semaphore : 8124a248
Semaphore state 0 (0x0)
The message queue is empty
The LpcDataInfoChainHead queue is empty
Done.
```

Если она присутствует, вторая строка листинга 8.10 показывает, какой поток обрабатывает клиентский запрос. В сильно нагруженной системе возможно, что серверный поток, обрабатывающий LPC-сообщение, не будет найден. В таком случае разработчику нужно понять, почему ни один из серверных потоков не забирает сообщение.

С помощью команды расширения `!thread` можно узнать все остальное о серверном процессе и потоке, активно обслуживающем запрос. Эту информацию можно использовать для дальнейшей отладки, при желании - с помощью отладчика пользовательского режима. В этом разделе отладка продолжается с использованием отладчика режима ядра. В листинге 8.11 показан результат вывода информации о серверном потоке после переключения представления отладчика на серверный процесс и перезагрузки символов пользовательского режима.

Листинг 8.11. Серверный поток, обрабатывающий LPC-сообщение

```
kd> .thread /p /r ffab41c0
Implicit thread is now ffab41c0
Implicit process is now ffab3530
.cache forcedecodeuser done
Loading User Symbols
.....
kd> !thread ffab1020
THREAD ffab1020 Cid 036c.06e0 Teb: 7ffdc000 Win32Thread: 00000000 WAIT: (DelayExecution) UserMode Non-Alertable
ffab1110 NotificationTimer
Not impersonating
DeviceMap e1a60398
Owning Process ffab3530 Image: 08comsrv.exe
Wait Start TickCount 550275 Ticks: 15038 (0:00:02:30.596)
Context Switch Count 8
UserTime 00:00:00.0010
KernelTime 00:00:00.0020
Start Address kernel32!BaseThreadStartThunk (0x7c810856)
LPC Server thread working on message Id 4f99
Stack Init f73c1000 Current f73c0cbc Base f73c1000 Limit f73be000 Call 0
Priority 9 BasePriority 8 PriorityDecrement 0 DecrementCount 0
Kernel stack not resident.
ChildEBP RetAddr Args to Child
f73c0cd4 804dc6a6 ffab10d8 ffab1020 804dc5cb nt!KiSwapContext+0x2e
```

```

f73c0ce0 804dc5cb f73c0d64 00e5f428 00e5f448 nt!KiSwapThread+0x46
f73c0d0c 8056603f 00000001 00000000 f73c0d2c nt!KeDelayExecutionThread+0x1c9
f73c0d54 804df06b 00000000 00e5f448 00e5f470 nt!NtDelayExecution+0x87
f73c0d54 7c90eb94 00000000 00e5f448 00e5f470 nt!KiFastCallEntry+0xf8
(TrapFrame @ f73c0d64)
00e5f414 7c90d85c 7c8023ed 00000000 00e5f448 ntdll!KiFastSystemCallRet
00e5f418 7c8023ed 00000000 00e5f448 00e5f558 ntdll!NtDelayExecution+0xc
00e5f470 7c802451 000927c0 00000000 00e5f558 kernel32!SleepEx+0x61
00e5f480 0043ad9b 000927c0 00e5f55c 00e5f58c kernel32!Sleep+0xf
00e5f558 77e79dc9 0092267c 00000001 00000002 SRV!CCalculator::SumSlow+0x2b
00e5f57c 77ef321a 0043857c 00e5f590 00000004 RPCRT4!Invoke+0x30
...
00e5fdfc 77e7bb6a 001625f0 00159360 00165630 RPCRT4!LRPC_SCALL::DealWithRequestMes-
sage+0x2cd
00e5fe20 77e76784 0015939c 00e5fe38 00165630 RPCRT4!LRPC_ADDRESS::DealWithLRPCRe-
quest+0x16d
...
00e5ffec 00000000 77e76bf0 0015e5e8 00000000 kernel32!BaseThreadStart+0x37

```

Теперь совершенно ясно, почему серверному потоку нужно так много времени, чтобы сложить несколько чисел: один из авторов примера намеренно оставил вызов функции `kernel32!Sleep` для целей отладки.

Имперсонация локальных вызовов DCOM и LRPC

Имперсонация - фундаментальная концепция, используемая в современных версиях операционной системы Windows. Она позволяет конкретному потоку выполнять все операции в контексте безопасности, отличном от контекста процесса, которому принадлежит поток. Имперсонацию можно включать или отключать по требованию, устанавливая или сбрасывая токен имперсонации в потоке.

Но что происходит с точки зрения безопасности, когда клиентский поток делает вызов в сервер с использованием протокола LPC? Клиент может указать, какой токен имперсонации должен быть представлен серверу, и ядро сохраняет эту информацию в серверном потоке. Когда сервер имперсонирует клиента с помощью RPC-функции `rpcrt4!RpcImpersonateClient` или DCOM-функции `ole32!CoImpersonateClient`, имперсонация выполняется другой LPC-функцией, называемой `ntdll!NtImpersonateClientOfPort`. Эта функция использует информацию об имперсонации, сохраненную ядром Windows в потоке в момент передачи сообщения серверу.

Из отладчика пользовательского режима информацию об имперсонации можно проверить только после того, как сервер выполнит вызов одной из функций имперсонации, проверив токен, установленный в данный момент в потоке; этот метод часто использовался в главе 7, "Безопасность". Из отладчика режима ядра это сделать гораздо проще: информация всегда присутствует в серверном потоке как указатель на `_PS_IMPERSONATION_INFORMATION`, сохраненный в члене `ImpersonationInfo` структуры потока `_ETHREAD`. Вместе с токеном имперсонации хранятся инструкции о том, как имперсонировать клиента. В случае, показанном в листинге 8.12, любая имперсонация приводит к токену на уровне идентификации.

Листинг 8.12. Чтение `ImpersonationInfo`, сохраненного в серверном потоке

```

kd> dt _ETHREAD ffab1020 ImpersonationInfo
+0x20c ImpersonationInfo : 0xe1269038 _PS_IMPERSONATION_INFORMATION
kd> dt 0xe1269038 _PS_IMPERSONATION_INFORMATION
+0x000 Token : 0xe1acba08
+0x004 CopyOnOpen : 0 ''
+0x005 EffectiveOnly : 0 ''
+0x008 ImpersonationLevel : 1 ( SecurityIdentification )

```

Информация в этом разделе помогает при отладке простого сценария, использующего локальные вызовы LRPC или DCOM. Более сложные сценарии, такие как активация DCOM, с точки зрения

отладки являются лишь комбинацией вызовов, и с ними можно справляться, следуя тем же простым шагам, показанным ранее.

Устранение неполадок удаленного взаимодействия

MS RPC расширяет реализацию RPC, предоставляя платформно-специфичные модели безопасности и добавляя поддержку LPC-взаимодействия. Хотя локальное взаимодействие имеет отличную поддержку отладки, удаленному взаимодействию этих средств не хватает. В этом разделе мы изучим возможность, доступную разработчикам для компенсации отсутствующей здесь поддержки отладки.

Один вариант - собрать все знания, необходимые для отладки основных сценариев, в умное расширение, способное интерпретировать все внутренние структуры и связи между различными структурами. Расширение могло бы показывать эту информацию в легко понятной форме и автоматизировать весь процесс обнаружения пути вызова. К сожалению, такого расширения сейчас нет.

Чтобы ответить на эти сложности, команда RPC ввела специальный метод отладки взаимодействия между клиентом и сервером, использующий дополнительную трассировочную информацию, называемую RPC Troubleshooting State Information. Этот метод описан в следующем разделе.

Использование RPC Troubleshooting State Information

Поскольку это единственный доступный сегодня метод, в оставшейся части раздела мы сосредоточимся на нем. Так как информация хранится в ячейках, используемых только для целей отладки, метод, который их использует, также называется RPC cell debugging, или cell debugging. Первая часть этого раздела описывает, как управлять поведением среды выполнения RPC в части поддержания информации о состоянии; вторая подробно объясняет, где эта информация хранится и как к ней получить доступ; третья описывает инструменты, доступные для ее фильтрации и отображения. Последняя часть использует эти инструменты для решения реального сценария.

Обратите внимание, что cell debugging доступна начиная с Windows XP и Windows 2003.

Настройка Cell Debugging

Cell debugging - это метод инструментализации, используемый средой выполнения RPC для записи RPC-активности. Состоянием включения инструментализации, а также уровнем инструментализации можно управлять с помощью системного административного шаблона, доступного в оснастке Group Policy. Оснастку можно запустить командой `gpedit.msc`, или ее можно добавить в существующую консоль оснасток, выбрав автономную оснастку "Group Policy Object Editor", нацеленную на локальный компьютер. Независимо от способа запуска политика, управляющего поведением Remote Procedure Call, находится в Administrative Templates раздела System, относящихся к конфигурации компьютера, как показано на рисунке 8.2.

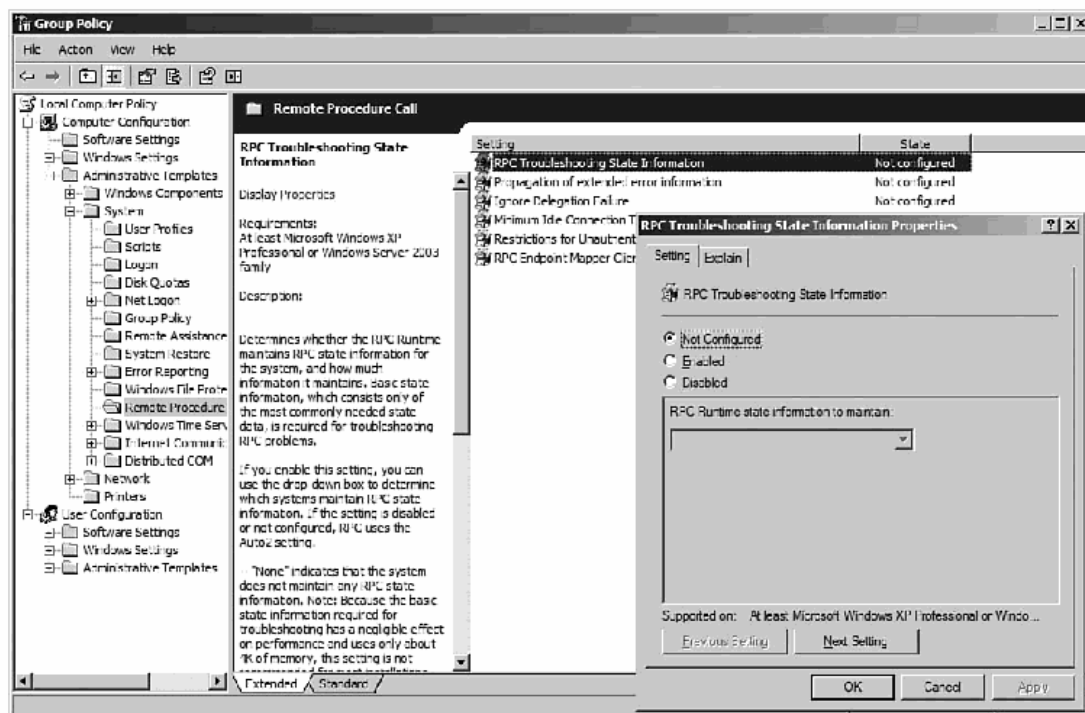


Figure 8.2 Enabling the RPC troubleshooting state information

Рисунок 8.2. Включение RPC troubleshooting state information

RPC Troubleshooting State Information управляется включенным состоянием, которое может иметь пять разных значений:

- Состояние None: предписывает среде выполнения RPC не собирать никакой информации о своей активности.
- Состояние Auto1: предписывает среде выполнения RPC собирать базовую информацию о своей активности.
- Состояние Auto2: предписывает среде выполнения RPC собирать базовую информацию о своей активности только на системах с более чем 128 МБ RAM. На сервере это политика по умолчанию, и прямое следствие состоит в том, что большинство, если не все, серверы имеют базовую информацию обо всех RPC-вызовах.
- Состояние Server: предписывает среде выполнения RPC собирать базовую информацию о своей активности независимо от конфигурации системы.
- Состояние Full: предписывает среде выполнения RPC собирать полную информацию о своей активности независимо от конфигурации системы.

После анализа всех вариантов, доступных для настройки RPC Troubleshooting Information, становится ясно, что существует всего три способа ее настройки: отсутствие информации, только серверная информация или полная информация. На серверной системе вариант Auto1 эквивалентен Auto2 и варианту Server для всех систем с объемом RAM более 128 МБ. На клиентских системах вариант Auto1 эквивалентен варианту Server на всех системах с объемом RAM более 64 МБ.

С практической точки зрения серверные системы, такие как Windows Server 2003, всегда предварительно настроены на сбор базовой информации, тогда как клиентские системы, такие как Windows XP, по умолчанию никогда не настроены. Чтобы использовать возможность cell debugging на клиентских системах, ее нужно включить в варианте Server или Full, в зависимости от потребностей отладки. Заявляется, что трассировка легковесна, и ее всегда можно включить в состояние Server даже на клиентской системе, если достаточно памяти.

После изменения политики RPC troubleshooting state систему нужно перезагрузить, прежде чем политика вступит в силу. Когда система снова запущена, среда выполнения RPC записывает информацию о своей активности в каждом процессе, использующем RPC, и обновляет все изменения состояния.

Информация Cell Debugging

После включения RPC Troubleshooting Information среда выполнения RPC создает необходимые структуры для хранения генерируемой ею информации. При первом взгляде новый список объектов, созданный после этого в системе, показывает множество объектов section с именами, производными от идентификаторов процессов. Снимок этих дескрипторов, сделанный с помощью инструмента Process Explorer, показан на рисунке 8.3. В диалоговом окне поиска Process Explorer, отображаемом через меню Find, мы вводим строку section, чтобы найти все объекты типа section. На рисунке 8.3 показан отсортированный результат на системе, где работает 08cli.exe.

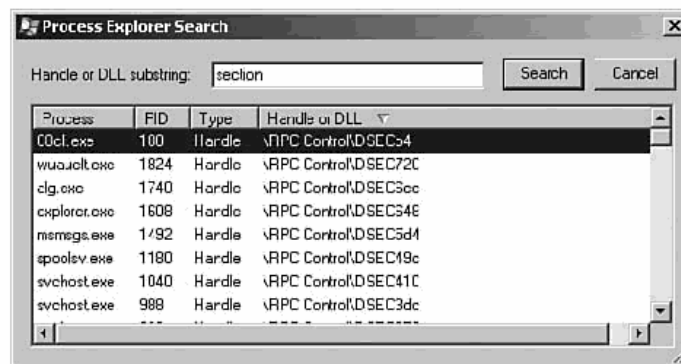


Figure 8.3 Debug cell sections

Рисунок 8.3. Секции debug cell

Секции troubleshooting state на рисунке 8.3 доступны любому процессу, работающему в локальной системе; это очень важный аспект при отладке приложений, охватывающих несколько процессов. Более того, поскольку информация troubleshooting state не принадлежит конкретному процессу и не требует сложного механизма для ее получения или обновления, мы можем использовать инфраструктуру трассировки даже тогда, когда система находится в действительно плохом состоянии. Каждый объект section содержит несколько ячеек; каждая ячейка содержит информацию о том, как конкретный элемент создается и поддерживается:

- Для каждой новой конечной точки, созданной в процессе, в секцию RPC troubleshooting state этого процесса добавляется новая ячейка, содержащая информацию о конечной точке.
- Для каждого нового потока, созданного инфраструктурой RPC, в секцию RPC troubleshooting state процесса добавляется новая ячейка, содержащая информацию о потоке. Эта ячейка обновляется каждый раз при изменении состояния потока, и временная метка изменения обновляется.
- Каждый раз, когда сервер обрабатывает новое соединение или коммуникационный запрос, инфраструктура RPC создает ячейку, представляющую серверную информацию, относящуюся к этому вызову.
- Для каждого запроса, инициированного клиентом, создается новая ячейка, представляющая клиентскую информацию, относящуюся к этому вызову. Эта ячейка создается только тогда, когда политика RPC Troubleshooting Information установлена в режим Full. Клиентскую информацию, созданную таким образом, мы используем в разделе "Получение информации о клиентском вызове".

Следующий раздел описывает инструменты, используемые для извлечения и фильтрации информации, хранящейся в этих секциях troubleshooting state. Он также показывает, как интерпретировать и сопоставлять информацию cell debugging, чтобы решить имеющуюся проблему.

Доступ к информации Cell Debugging

К информации ячеек можно получить доступ с помощью автономного инструмента `dbgrpc.exe`, расположенного в каталоге, где установлены отладчики. В качестве альтернативы расширение отладчика `rpcexts.dll`, устанавливаемое по умолчанию вместе с Debugging Tools for Windows, содержит несколько команд расширения для управления информацией troubleshooting state. Хотя расширение полезно для исследования проблемы внутри отладчика, инструмент командной строки может обрабатывать информацию с удаленной машины, вызывая RPC-интерфейс, предоставляемый инфраструктурой RPC на этой машине, при условии, что вызывающий пользователь является администратором удаленной системы. Параметры командной строки и команда расширения отладчика похожи и будут представлены рядом. Поскольку информация, используемая расширением отладчика, доступна из всех процессов, расширение работает из любого отладчика пользовательского режима, запущенного в системе. Отладчик, используемый в этом разделе, присоединен к клиентскому или серверному процессу.

Примечание. Расширение `rpcexts.dll` реализует несколько команд расширения, которым требуется доступ к приватным символам. Поскольку у нас нет доступа к приватным символам, эти команды не обсуждаются. Кроме того, расширение не загружается по умолчанию, поэтому команды расширения, или по крайней мере первое использование команды расширения, нужно предварять именем расширения `rpcexts`.

Получение текущей временной метки

Команда расширения `!rpctime` показывает время, прошедшее с запуска системы, в формате `<seconds>.<milliseconds>`, как показано в листинге 8.13. Временная привязка, используемая во всей инфраструктуре трассировки, полезна для понимания временных отношений между событиями ячеек. Временная метка выводится из системного времени и увеличивается даже тогда, когда процесс остановлен в отладчике пользовательского режима.

Листинг 8.13. Использование `!rpctime` для получения текущей временной метки, применяемой инфраструктурой troubleshooting

```
0:003> !rpctime
Current time is: 002960.857 (0x000b90.359)
```

Получение информации о конечных точках

Команда расширения `!getendpointinfo`, используемая без аргументов, выводит все конечные точки, предоставляемые всеми процессами на системе, где работает отладчик. Вывод команды содержит пять столбцов в следующем порядке:

- PID: идентификатор серверного процесса, размещающего конечную точку.
- CELL ID: идентификатор ячейки относительно PID процесса, идентифицирующий информационную ячейку.
- ST: состояние конечной точки, показывающее, активна ли конечная точка (состояние равно единице) или она была удалена.

- PROTSEQ: имя протокола.
- ENDPOINT: имя конечной точки.

В листинге 8.14 показан пример результата из системы под управлением Windows XP SP2 без установленного дополнительного программного обеспечения. Вывод можно использовать, чтобы выяснить, какой процесс владеет какими конечными точками и какие протоколы включены в каждом процессе. Имена протоколов самодокументируемы, и они принудительно задают формат имени конечной точки; протокол TCP может иметь только числовые конечные точки, тогда как NMP имеет имя, начинающееся с \pipe\, и так далее. Очень длинные имена конечных точек могут быть усечены до размера, допустимого ячейкой.

Как наблюдение: все LRPC-конечные точки с именем, начинающимся с OLE, используются инфраструктурой DCOM для процессов в клиентской или серверной роли.

Листинг 8.14. Использование !getendpointinfo для вывода всех конечных точек, известных RPC

```
0:005> !getendpointinfo
Searching for endpoint info ...
PID CELL ID ST PROTSEQ ENDPOINT
-----
...
038c 0000.0001 01 LRPC dhcpcsvc
038c 0000.0003 01 LRPC wzcsvc
038c 0000.0005 01 LRPC OLEA0BD1FB22E8E4CB3AED9EA46E
038c 0000.0009 01 NMP \PIPE\atsvc
038c 0000.000d 01 LRPC AudioSrv
038c 0000.0010 01 NMP \PIPE\wkssvc
038c 0000.0013 01 NMP \pipe\keysvc
038c 0000.0014 01 LRPC keysvc
038c 0000.0016 01 LRPC SECLOGON
038c 0000.0017 01 NMP \pipe\trkwws
038c 0000.0018 01 LRPC trkwws
038c 0000.001a 01 NMP \PIPE\srsvsc
038c 0000.0025 01 NMP \PIPE\browser
038c 0000.0026 01 LRPC senssvc
038c 0000.0028 01 NMP \PIPE\W32TIME
...
0240 0000.0001 01 LRPC OLE9D488805CBAA4A479CDD8DCD0
05cc 0000.0001 01 LRPC OLE9A35F92EE10245499B5520104
06a0 0000.0001 01 LRPC OLE71BE2F37F98B4AE5B9E13F5C2
0078 0000.0001 01 LRPC OLECF2A0CC062794FA78A63DA9A5
0388 0000.0001 01 LRPC OLE73A51130EAF4D5AB504E5597
```

Ту же информацию можно получить с помощью автономного инструмента dbgrpc.exe через следующую командную строку:

```
C:\>dbgrpc -e
```

Когда мы сосредоточены на конкретной конечной точке, за командой может следовать имя конечной точки, как в листинге 8.15. Имя конечной точки действует как фильтр для команды расширения !getendpointinfo.

Листинг 8.15. Использование !getendpointinfo для вывода всех конечных точек, известных RPC

```
0:003> !getendpointinfo \PIPE\W32TIME
Searching for endpoint info ...
PID CELL ID ST PROTSEQ ENDPOINT
-----
038c 0000.0028 01 NMP \PIPE\W32TIME
```

Альтернатива командной строки для получения той же информации передает конечную точку как параметр переключателю -E, как показано ниже:

```
C:\>dbgrpc -e -E \PIPE\W32TIME
```

Получение информации о потоках

Каждый процесс с активными RPC-конечными точками должен слушать все зарегистрированные конечные точки с помощью одного или нескольких потоков, входящих в пул RPC-потоков, которым управляет среда выполнения RPC. Команда расширения !getthreadinfo выводит все информационные ячейки потоков в следующем формате:

- PID: идентификатор сервера, размещающего поток.
- CELL ID: идентификатор ячейки относительно PID процесса, идентифицирующий информационную ячейку.
- ST: состояние потока, показывающее, простаивает ли поток или он был передан серверному коду.
- TID: идентификатор Win32-потока.
- ENDPOINT: ячейка, содержащая дополнительную информацию о конечной точке, которую слушает поток.
- LASTIME: временная метка последнего изменения состояния потока.

Команда принимает идентификатор процесса как параметр, как показано в листинге 8.16, где целевой процесс имеет идентификатор 0x038c.

Листинг 8.16. Использование !getthreadinfo для вывода всех потоков из RPC thread pool

```
0:005> !getthreadinfo 038c
Searching for thread info ...
PID CELL ID ST TID ENDPOINT LASTTIME
-----
038c 0000.0004 03 000004a8 0000.0003 0009237f
038c 0000.0006 02 000004b4 <IOCP> 009124dd
038c 0000.0007 03 000004cc 0000.0005 00958d5d
038c 0000.000a 02 000004c8 <IOCP> 00ac3dc1
038c 0000.000b 03 0000052c 0000.0001 008d7e51
038c 0000.000e 03 0000050c 0000.000d 001320ce
038c 0000.001d 03 00000650 0000.0026 00af1978
038c 0000.0020 03 00000794 0000.0016 000a9d76
038c 0000.0023 03 00000090 <IOCP> 00abc898
038c 0000.0024 03 00000790 0000.0018 000a9d76
038c 0000.0027 03 00000688 0000.0026 00af1978
038c 0000.002c 03 0000078c 0000.0014 000a9d76
038c 0000.002e 03 000007dc 0000.0026 00af196e
```

Информация о конечной точке. Столбец ячейки не всегда содержит информацию о ячейке конечной точки, как в случае потоков с идентификаторами b4b, 4c8 и 90. В этих случаях поле ENDPOINT заменено строкой <IOCP>, указывающей, что соответствующие потоки ожидают на портах завершения ввода-вывода, связанных с несколькими конечными точками.

Альтернатива командной строки для получения той же информации передает идентификатор процесса как параметр переключателю -t, как показано ниже:

```
C:\> dbgRPC.exe -t -P 38c
```

Вывод можно отфильтровать дополнительно, добавив идентификатор потока к списку аргументов команды. Например, листинг 8.17 содержит вывод команды, фильтрующей конкретный поток с идентификатором 0x4a8, работающий в процессе 38c.

Листинг 8.17. Использование !getthreadinfo для получения RPC-информации о конкретном потоке

```
0:005> !getthreadinfo 038c 000004a8
Searching for thread info ...
PID CELL ID ST TID ENDPOINT LASTTIME
-----
038c 0000.0004 03 000004a8 0000.0003 0009237f
```

Альтернативный способ получить ту же информацию - передать идентификатор потока как параметр переключателю -T, как в следующей строке:

```
C:\> dbgRPC.exe -t -P 38c -T 4a8
```

Получение информации о вызовах

Одна из самых важных частей инструментализации хранится в ячейках call info. Чтобы понять, какая информация там хранится, приведем немного сведений о том, как работает среда выполнения RPC. Подобно протоколу LRPC, описанному в первом разделе этой главы, среда выполнения RPC слушает все конечные точки на предмет запросов соединения и создает объект соединения, отвечающий за управление каждым новым соединением. Серверный код, отвечающий за обработку соединения, позже обрабатывает все запросы вызовов на соединении, создавая другой временный объект, в общем виде называемый объектом SCALL (конкретнее, вызов может обслуживаться классом LRPC_SCALL, OSF_SCALL или DG_SCALL), в зависимости от протокола, обслуживающего это соединение, созданного для диспетчеризации конкретного вызова. Каждый объект соединения и объект вызова имеет одну связанную ячейку в списке, возвращаемом командой расширения !getcallinfo, как показано в листинге 8.18.

Полный листинг содержит обычные поля - процесс, размещающий этот объект, идентификатор ячейки, время последнего обновления и состояние ячейки, - а также специфичные для объекта ячейки в следующем формате:

- PID: идентификатор серверного процесса, обрабатывающего вызов.
- CELL ID: идентификатор ячейки относительно PID процесса, идентифицирующий информационную ячейку.
- ST: состояние потока, показывающее, активен ли вызов или он был завершен.
- PNO: номер процедуры из RPC-интерфейса, к которой выполняется или выполнялся вызов, также известный как opnum.
- IFSTART: первые 32 бита Interface Identifier или IID, к которому выполняется или выполнялся вызов.
- THRDCELL: идентификатор ячейки потока, содержащей подробную информацию о потоке, который обрабатывает или обрабатывал вызов.
- CALLFLAG: комбинация флагов, связанных с вызовом, хорошо расшифровываемая командой расширения !getdbgcell.
- CALLID: идентификатор вызова, который можно использовать, чтобы связать информационную ячейку вызова с информационной ячейкой клиента.
- CONN/CLN: информация о клиентском соединении. Для LRPC-вызовов этот столбец содержит идентификатор процесса, за которым следует идентификатор потока. Вызовы протоколов с установлением соединения хранят в этом столбце идентификатор ячейки, содержащей дополнительную информацию о соединении, использованном для этого вызова.

Листинг 8.18. Использование !getcallinfo для получения информации о вызовах, поддерживаемой сервером

```
0:005> !getcallinfo
Searching for call info ...
PID CELL ID ST PNO IFSTART THRDCELL CALLFLAG CALLID LASTTIME CONN/CLN
-----
021c 0000.000e 00 009 00000134 0000.000d 00000009 00000001 0014142a 0348.047c
...
038c 0000.001e 00 003 00000132 0000.0029 00000008 00000000 0004a91f 0348.0628
038c 0000.001f 00 000 d674a233 0000.001d 00000009 00000000 00afb51f 038c.0720
038c 0000.0021 00 004 00000132 0000.000a 00000009 00000000 00870be6 0348.05c0
038c 0000.002a 00 005 fdd384cc 0000.0006 00000009 00000000 0003d03f 0740.0750
```

```

038c 0000.002f 00 000 629b9f66 0000.0027 00000009 00000000 0004ad58 021c.00ec
038c 0000.0030 00 007 3faf4738 0000.000e 00000009 00000000 0004c874 021c.00cc
038c 0000.0032 00 009 06bba54a 0000.0027 00000009 004f0044 000521b9 01fc.0208
038c 0000.0037 00 005 00000134 0000.0039 00000009 00000003 00059385 05cc.04c0
038c 0000.003a 00 003 609b9557 0000.0039 00000009 00000004 00059335 05cc.04c0
038c 0000.003b 00 000 63fbe424 0000.0027 00000009 00000000 00afe977 0460.0474
0460 0000.0007 02 009 4b112204 0000.0006 00000009 00000000 0005a9a9 038c.07e8
...
0388 0000.0005 02 004 daf50cdb 0000.0003 00000009 0078006f 008a8023 0078.03ac

```

Альтернатива командной строки для получения той же информации использует переключатель -с, как показано здесь:

```
C:\>dbgrrpc -c
```

Поскольку список вызовов становится очень большим на production-серверах, рекомендуется фильтровать эту информацию. Расширение принимает идентификатор вызова, первые 32 бита UUID интерфейса, номер процедуры и идентификатор процесса, обрабатывающего вызовы, как параметры фильтрации. У каждого параметра фильтра есть необязательное значение, описанное в справке команды. В листинге 8.19 используются значения по умолчанию для всех параметров, кроме идентификатора процесса, чтобы получить ячейки вызовов, доступные в процессе с идентификатором 0x38c.

Листинг 8.19. Использование !getcallinfo для фильтрации информации о вызовах до конкретного процесса

```

0:005> !getcallinfo 0 0 FFFF 38c
Searching for call info ...
PID CELL ID ST PNO IFSTART THRDCELL CALLFLAG CALLID LASTTIME CONN/CLN
-----
038c 0000.000c 00 000 0a74ef1c 0000.0006 00000009 00000006 008a6272 038c.04e4
038c 0000.0012 00 00b 3faf4738 0000.000e 00000009 004f0044 000e17b0 06a0.0780
038c 0000.001b 00 00b 3faf4738 0000.000e 00000009 004f0044 0005467f 0240.0314
038c 0000.001e 00 003 00000132 0000.0029 00000008 00000000 0004a91f 0348.0628
038c 0000.001f 00 000 d674a233 0000.001d 00000009 00000000 00afb51f 038c.0720
038c 0000.0021 00 004 00000132 0000.000a 00000009 00000000 00870be6 0348.05c0
038c 0000.002a 00 005 fdd384cc 0000.0006 00000009 00000000 0003d03f 0740.0750
038c 0000.002f 00 000 629b9f66 0000.0027 00000009 00000000 0004ad58 021c.00ec
038c 0000.0030 00 007 3faf4738 0000.000e 00000009 00000000 0004c874 021c.00cc
038c 0000.0032 00 009 06bba54a 0000.0027 00000009 004f0044 000521b9 01fc.0208
038c 0000.0037 00 005 00000134 0000.0039 00000009 00000003 00059385 05cc.04c0
038c 0000.003a 00 003 609b9557 0000.0039 00000009 00000004 00059335 05cc.04c0
038c 0000.003b 00 000 63fbe424 0000.0027 00000009 00000000 00afe977 0460.0474

```

Альтернатива командной строки для получения той же информации использует параметр -с, как показано здесь:

```
C:\>dbgrrpc -c -P 38c
```

Получение полной информации о ячейке

Теперь пора глубже заглянуть в каждую ячейку, чтобы расшифровать информацию, не объясненную и не показанную в листинге 8.19. Команда расширения !getdbgcell понимает все типы ячеек и может корректно их расшифровывать. Процесс и идентификатор ячейки, используемые как параметры в листинге 8.20, взяты из каждой ячейки, полученной после перечисления ячеек, как показано в листинге 8.19.

Листинг 8.20. Использование !getdbgcell для получения информации о ячейке, поддерживаемой сервером

```

0:005> * Obtaining information about a call cell
0:005> !getdbgcell 038c 0000.000c
Getting cell info ...
Call

```

```

Status: Allocated
Procedure Number: 0
Interface UUID start (first DWORD only): A74EF1C
Call ID: 0x6 (6)
Servicing thread identifier: 0x0.6
Call Flags: cached, LRPC
Last update time (in seconds since boot):9069.170 (0x236D.AA)
Caller (PID/TID) is: 38c.4e4 (908.1252)
0:005> * Obtaining information about an endpoint cell obtained in Listing 8.14
0:005> !getdbgcell 038c 0000.0028
Getting cell info ...
Endpoint
Status: Active
Protocol Sequence: NMP
Endpoint name: \PIPE\W32TIME

```

Альтернатива командной строки для получения той же информации использует переключатель -l, за которым следует информация о ячейке, как показано ниже:

```
C:\>dbgrpc -l -P 38c -L 0000.000c
```

Получение информации о клиентском вызове

Когда политика RPC Troubleshooting State Information установлена в режим Full, информационную ячейку клиентского вызова, записанную средой выполнения RPC, можно перечислить с помощью команды расширения !getclientcallinfo, используя те же параметры, что и у команды расширения !getcallinfo (см. листинг 8.21).

Вывод команды содержит обычные поля - идентификатор клиентского процесса, идентификатор ячейки, время последнего обновления и состояние ячейки, - а также специфичные для объекта ячейки в следующем формате:

- PID: идентификатор клиентского процесса, породившего вызов.
- CELL ID: идентификатор ячейки относительно PID процесса, идентифицирующий информационную ячейку.
- PNO: номер процедуры из RPC-интерфейса, к которой выполняется или выполнялся вызов, также известный как opnum.
- IFSTART: первые 32 бита Interface Identifier или IID, к которому выполняется или выполнялся вызов.
- TIDNUMBER: идентификатор ячейки, содержащей подробную информацию о потоке, инициировавшем вызов.
- CALLID: идентификатор вызова, который можно использовать для сопоставления информационной ячейки вызова с информационной ячейкой клиента.
- LASTIME: временная метка последнего обновления ячейки.
- PS: комбинация флагов, связанных с вызовом, которую можно расшифровать командой расширения !getdbgcell.
- CLTNUMER: идентификатор ячейки целевого вызова, содержащей дополнительную информацию о сервере, обрабатывающем вызов.
- ENDPOINT: имя серверной конечной точки, обслуживающей этот вызов.

Листинг 8.21. Использование !getclientcallinfo для получения информации о вызовах, поддерживаемой клиентом

```

0:005> !getclientcallinfo
Searching for call info ...
PID CELL ID PNO IFSTART TIDNUMBER CALLID LASTIME PS CLTNUMBER ENDPOINT
-----
038c 0000.003f 0009 4b112204 0000.0000 ffffffff 0005a9a9 09 0000.0040 LRPC00000460
0078 0000.0003 0004 daf50cdb 0000.0000 ffffffff 008a8023 09 0000.0004 OLE73A51130E

```


Альтернатива командной строки для получения той же информации использует переключатель -a, как показано ниже:

```
C:\>dbprpc -a
```

Вся эта информация о состоянии может использоваться в нескольких простых сценариях, где вы научитесь сопоставлять ее, чтобы быстрее прийти к решению.

Использование информации Cell Debugging

Как и в локальных клиент-серверных сценариях, при отладке удаленных клиент-серверных сценариев нам часто приходится идти по пути выполнения от клиентского процесса до обработки вызова на стороне сервера. В этом разделе для отслеживания пути выполнения используется RPC Troubleshooting State Information, собранная средой выполнения RPC при обработке вызова.

В этом примере клиентский процесс 08cli.exe выполняет синхронный DCOM-вызов в удаленный сервер, который завершается дольше ожидаемого. В данном конкретном случае клиентская и серверная системы имеют фиксированные TCP/IP-адреса, соответственно 192.168.0.105 и 192.168.0.104. Обе системы являются членами одной рабочей группы, и список пользователей на клиенте и сервере идентичен, что позволяет клиенту аутентифицироваться на нашем сервере с помощью pass-through-аутентификации. На клиентской системе политика RPC Troubleshooting State Information установлена в режим Full, тогда как на сервере политика установлена в режим Server. Клиент запускается следующей командной строкой:

```
C:\>08cli.exe server:192.168.0.104
```

Процесс отладки начинается внутри клиентского процесса, где мы определили поток, ожидающий завершения вызова. В листинге 8.22 показан стек нулевого потока, ожидающего RPC-вызов.

Листинг 8.22. Типичный клиентский стек, ожидающий удаленный вызов, сделанный с использованием протокола с установлением соединения

```
0:003> ~0k50
ChildEBP RetAddr
0012f450 7c90e9c0 ntdll!KiFastSystemCallRet
0012f454 7c8025cb ntdll!NtWaitForSingleObject+0xc
0012f4b8 77e80acb kernel32!WaitForSingleObjectEx+0xa8
0012f4d4 77e80a81 RPCRT4!UTIL_WaitForSyncIO+0x20
0012f4f8 77eeb7ba RPCRT4!UTIL_GetOverlappedResultEx+0x1d
0012f52c 77e8520d RPCRT4!WS_SyncRecv+0xca
0012f54c 77e80e8d RPCRT4!OSF_CCONNECTION::TransSendReceive+0x9d
0012f5c8 77e80e0d RPCRT4!OSF_CCONNECTION::SendFragment+0x226
0012f620 77e80c6f RPCRT4!OSF_CCALL::SendNextFragment+0x1d2
...
0012fccc 0042ead1 RPCRT4!ObjectStubless+0xf
0012fe48 0042e846 08CLI!MTAClientCall+0xc1
0012ff54 00430692 08CLI!wmain+0xb6
0012ffb8 0043044d 08CLI!wmainCRTStartup+0x252
0012ffc0 7c816fd7 08CLI!wmainCRTStartup+0xd
0012fff0 00000000 kernel32!BaseProcessStart+0x23
0:003> |
. 0 id: 63c create name: 08cli.exe
```

Мы собираем всю клиентскую информацию, доступную о конкретном потоке, с помощью команды расширения !getclientcallinfo. Поскольку на клиентской системе не так много RPC-активности, можно использовать команду без фильтра. В листинге 8.23 столбец PID сопоставляется с идентификатором клиентского процесса, чтобы получить идентификатор ячейки вызова.

Листинг 8.23. Перечисление всех информационных ячеек клиентских вызовов

```
0:002> !rpcexts.getclientcallinfo
Searching for call info ...
```

```
PID CELL ID PNO IFSTART TIDNUMBER CALLID LASTTIME PS CLTNUMBER ENDPOINT
-----
055c 0000.005b 0009 4b112204 0000.0000 ffffffff 0010a534 09 0000.005c LRPC00000384
0590 0000.0006 0009 4b112204 0000.0000 ffffffff 0000e745 09 0000.0007 LRPC00000384
063c 0000.0003 0004 daf50cdb 0000.0000 00000001 004464bb 07 0000.0004 1359
```

В листинге 8.24 информация о вызове расшифровывается командой расширения `!getdbgcell`. Номер процедуры показан в третьей строке (4 означает, что клиент вызвал второй метод DCOM-интерфейса, в котором стандартный интерфейс IUnknown использует первые три слота процедур), целевая конечная точка показана в восьмой строке, а ячейка, содержащая дополнительную информацию о цели вызова, показана в седьмой строке.

Листинг 8.24. Получение дополнительных деталей из информации клиентской ячейки

```
0:002> !getdbgcell 063c 0000.0003
Getting cell info ...
Client call info
Procedure number: 4
Interface UUID start (first DWORD only): DAF50CDB
Call ID: 0x1 (1)
Calling thread identifier: 0x0.0
Call target identifier: 0x0.4
Call target endpoint: 1359
```

Поскольку мы не знаем, какая система обрабатывает вызов, мы расшифровываем и используем идентификатор ячейки цели вызова, как показано в листинге 8.25. Текущая временная метка полезна для понимания того, как давно начался этот вызов: в данном случае `004752s - 004482s = 270s`, то есть почти пять минут.

Листинг 8.25. Получение дополнительных деталей о цели вызова

```
0:002> !getdbgcell 063c 0000.0004
Getting cell info ...
Call target info
Protocol Sequence: TCP
Last update time (in seconds since boot):4482.235 (0x1182.EB)
Target server is: 192.168.0.104
0:002> !rpctime
Current time is: 004752.183 (0x001290.0b7)
```

Примечание. Когда клиентская информация недоступна (например, когда она не включена), можно использовать инструмент `netstat.exe`, чтобы получить часть информации, необходимой для поиска сервера. В этом случае мы используем текущий процесс 1596 (`0x63c`), чтобы определить TCP-соединение взаимодействия с серверной системой. Соединение содержит как адрес сервера, так и номер порта, использованный для соединения.

```
C:\>netstat -o
Active Connections
...
TCP XP-SP2:1734 192.168.0.104:1359 ESTABLISHED 1596
```

После нахождения адреса серверной системы и информации о конечной точке соединения отладка продолжается на сервере. Первый шаг - выяснить, какой процесс владеет конечной точкой, используемой клиентским процессом, с помощью либо инструмента `dbgrrpc.exe`, либо системного инструмента `netstat.exe`. После определения серверного процесса мы присоединяем к нему отладчик и определяем ожидающие вызовы; этот процесс показан в листинге 8.26. Идентификатор процесса, полученный из `dbgrrpc.exe`, нужно преобразовать из шестнадцатеричного вида в десятичный перед использованием как параметра параметра командной строки отладчика -p.

Листинг 8.26. Получение информации о вызове из информации о конечной точке

```
C:\>dbgrrpc.exe -e -E 1359
Searching for endpoint info ...
PID CELL ID ST PROTSEQ ENDPOINT
-----
```

```

058c 0000.0006 01 TCP 1359
C:\>windgg -p 1420
...
0:007> !getcallinfo 0 0 FFFF 58c
Searching for call info ...
PID CELL ID ST PNO IFSTART THRDCELL CALLFLAG CALLID LASTTIME CONN/CLN
-----
058c 0000.0003 00 004 00000132 0000.0005 00000009 00000000 007b30d4 0338.05d4
058c 0000.0004 00 009 00000134 0000.0006 00000009 00000001 0080b279 0338.0710
058c 0000.000a 02 004 daf50cdb 0000.0008 00000001 00000001 007b34c8 0000.0009

```

Активные вызовы из этого списка находятся в состоянии (столбец ST), отличном от нуля. Затем мы сосредоточиваемся на потоке, обрабатывающем эти вызовы. Идентификатор ячейки потока доступен в столбце THRDCELL. Последний столбец указывает идентификатор ячейки объекта соединения, содержащей дополнительные свойства соединения, например уровень аутентификации, службу аутентификации, использованную для этого вызова, и исходный IP-адрес, как показано в листинге 8.27.

Листинг 8.27. Исследование информационной ячейки потока и объекта соединения

```

0:000> !getdbgcell 058c 0000.0008
Getting cell info ...
Thread
Status: Dispatched
Thread ID: 0x760 (1888)
Thread is an IO completion thread
Last update time (in seconds since boot): 8074.440 (0x1F8A.1B8)
0:000> !getdbgcell 058c 0000.0009
Getting cell info ...
Connection
Connection flags: Exclusive
Authentication Level: Connect
Authentication Service: NTLM
Last Transmit Fragment Size: 144 (0x4CBBA4)
Endpoint for the connection: 0x0.6
Last send time (in seconds since boot): ): 8013.920 (0x1F4D.398)
Last receive time (in seconds since boot): ): 8074.440 (0x1F8A.1B8)
Getting endpoint info ...
Caller is(IPv4): 192.168.0.105

```

Мы используем идентификатор серверного потока, выполняющего запрос, чтобы получить стек выполнения, как показано в листинге 8.28. Неудивительно, что поток выполняет долгую операцию сна, как вы видели в начале этой главы.

Листинг 8.28. Стек вызовов серверного потока

```

0:000> ~[760]k
ChildEBP RetAddr
010ef458 7c90d85c ntdll!KiFastSystemCallRet
010ef45c 7c8023ed ntdll!NtDelayExecution+0xc
010ef4b4 7c802451 kernel32!SleepEx+0x61
010ef4c4 0043ad9b kernel32!Sleep+0xf
010ef59c 77e79dc9 SRV!CCalculator::SumSlow+0x2b
010ef5c0 77ef321a RPCRT4!Invoke+0x30
010ef9cc 77ef3bf3 RPCRT4!NdrStubCall2+0x297
...
010efdcd 77e8a067 RPCRT4!RPC_INTERFACE::DispatchToStub+0x84
010efe00 77eac1f4 RPCRT4!RPC_INTERFACE::DispatchToStubWithObject+0xc0

```

Информацию ячеек можно использовать для решения других сценариев, связанных с RPC-взаимодействием, комбинируя приемы, объясненные в этом разделе. Поскольку RPC troubleshooting state information доступна глобально в системе, доступ к ней из инструмента командной строки не создает накладных расходов, что делает ее подходящей даже для различных сценариев мониторинга, используемых на этапе разработки продукта.

Анализ сетевого трафика

В электронной инженерии цепи диагностируют, анализируя сигналы, циркулирующие внутри проблемных устройств, с помощью различного испытательного оборудования, от простых скалярных измерителей до сложных анализаторов данных. Поскольку сетевой трафик - это не более чем электрический сигнал в электронной цепи, приемы устранения неполадок, используемые в электронной инженерии, можно применять к устранению неполадок сетевого взаимодействия. Вопрос в том, какое измерительное устройство дает наибольшую ценность?

Хотя производители оборудования используют сложные инструменты для измерения электрических характеристик сетевых устройств, мы можем предполагать, что аппаратный уровень полностью работоспособен. Нас интересует только мониторинг логических данных, текущих по проводам. Мы можем читать и анализировать данные, передаваемые между компьютерами туда и обратно, с помощью инструментов анализа протоколов (также известных как packet sniffer tools).

В этом разделе мы используем сетевой анализатор Ethereal - очень мощный, но простой в использовании инструмент, доступный по лицензии GNU General Public License. Инструмент можно настроить на полный захват трафика, входящего в систему, где он работает, и выходящего из нее. Этого достаточно для анализа проблем, затрагивающих только наблюдаемую систему. В качестве альтернативы инструмент можно настроить на захват всего трафика, принимаемого Network Interface Card (NIC), подключенной к системе, независимо от адреса источника или назначения. Этот режим, называемый promiscuous capture mode, требует поддержки NIC. Promiscuous capture mode помогает решать проблемы, затрагивающие несколько систем, обменивающихся сообщениями в этой сети. Захват управляется из диалогового окна Capture Interfaces, вызываемого выбором пункта Interface в меню Capture. Диалоговое окно, показанное на рисунке 8.4, отображает статистику реального времени для каждой сетевой карты и позволяет начать захват на любой из них. Режим захвата, используемый для каждой NIC, можно изменить щелчком по соответствующей кнопке Prepare.

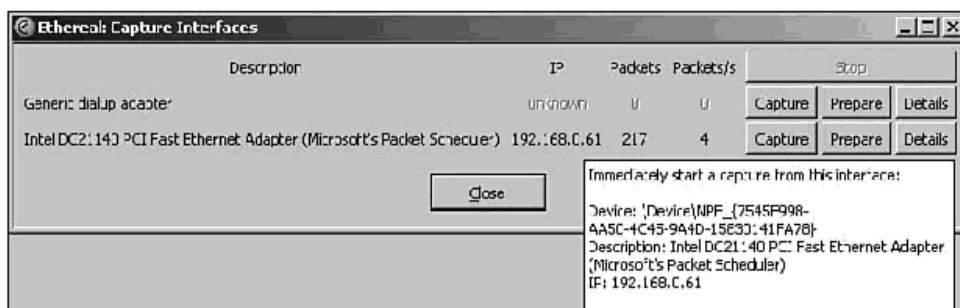


Figure 8.4 Capture Interface dialog box used to start capturing the traffic

Рисунок 8.4. Диалоговое окно Capture Interface, используемое для запуска захвата трафика

Независимо от способа захвата сетевого трафика, файлы захвата затем можно постобработать различными парсерами; трафик можно фильтровать или анализировать позже. Даже если вы не знакомы с некоторыми протоколами, встреченными в трафике, декодирование, выполненное инструментом, служит хорошим ориентиром для дальнейшего анализа или для прояснения решения.

Когда протокол, реализованный конкретным приложением, неизвестен, файлы захвата из корректно работающей установки можно использовать как эталон при анализе проблемного сценария. В этом случае пользователь сосредоточивается на понимании различий между файлами захвата неисправной системы и эталонными файлами захвата. Инструменты packet sniffer также

можно использовать для изучения поведения системы или проверки того, соответствует ли функциональность системы ее спецификации. На вопросы вроде "Зашифрован ли сетевой трафик?" или "Насколько болтлив протокол?" гораздо быстрее ответить анализом трафика, чем ревью кода реализации системы.

Ethereal показывает пакеты в упорядоченном списке, содержащем номер пакета в текущем файле захвата, захваченное время, адрес исходной NIC, адрес целевой NIC, имя протокола и дополнительную информацию, декодированную из пакета. В отдельном окне каждый пакет, интерпретированный dissector-ами, отображается как структура данных. Поскольку dissector-ы вызываются для иерархической интерпретации пакетов, базовая информация всегда декодируется. Если для протоколов более высокого уровня нет dissector-а, эта часть пакета показывается как массив байтов. Когда протокол имеет состояние и текущий пакет зависит от предыдущих пакетов, не захваченных в текущем файле, пакет нельзя декодировать полностью, и информация представляется в формате более базового уровня. Ethereal также показывает простой дамп содержимого пакета, очень полезный для быстрого визуального просмотра содержимого пакета.

Файлы захвата, используемые в этом разделе, от 08capture1.cap до 08capture4.cap, доступны в папке C:\AWDBIN\LOGS в пакете загрузки, содержащем двоичные файлы примеров.

Трасса успешной активации DCOM

В этом разделе анализируются пакеты, которыми обмениваются две системы, настроенные в рабочей группе, когда клиент вызывает DCOM-метод, реализованный сервером, используя пример кода этой главы. На рисунке 8.5 показан трафик Ethereal, захваченный в этом случае после удаления дополнительного трафика в сети, размещающей системы. Как и в предыдущем разделе, сервер имеет адрес 192.168.0.104, а клиент использует адрес 192.168.0.105. Сетевой трафик, иллюстрирующий это, был захвачен в файле 08capture1.cap.

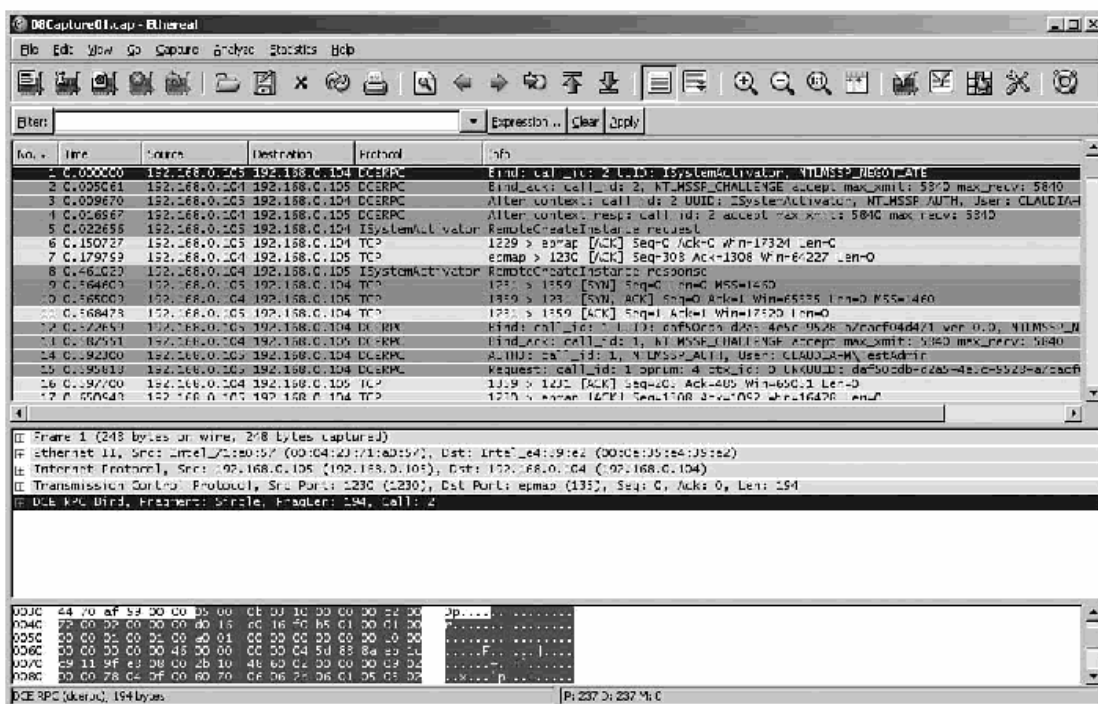


Figure 8.5 Packets exchanged during a DCOM activation followed by a long-running call

Рисунок 8.5. Пакеты, которыми обмениваются при активации DCOM с последующим долгим вызовом

Итак, что представляют собой все пакеты, которыми обменивается это очень простое приложение? Роли пакетов интерпретируются следующим образом:

- Frame 1: клиент отправляет сообщение Bind, чтобы привязать интерфейс ISystemActivator, идентифицированный декодером с помощью GUID {000001A0-0000-0000-C000-000000000046}. Этот пакет также содержит сообщение согласования безопасности. Это сообщение отправляется по существующему TCP/IP-соединению к порту DCOM SCM, установленному до начала операции захвата.
- Frame 2: сервер подтверждает Bind пакетом Bind_ack. Этот пакет также содержит сообщение NTLM challenge, потому что это единственный общий механизм аутентификации, принимаемый и сервером, и клиентом.
- Frame 3: клиент отвечает на challenge сообщением Alter_context, используя информацию, полученную из учетных данных пользователя TestAdmin.
- Frame 4: сервер проверяет идентичность вызывающего и подтверждает ее сообщением Alter_context_resp. Интерфейс готов к использованию.
- Frame 5: клиент вызывает RemoteCreateInstance, передавая серверный CLSID как параметр (текущий декодер не разбирает эту информацию), в данном случае {31810948-8D81-4E55-BD16-0C27F5629392}.
- Frame 8: сервер возвращает указатель интерфейса запрошенного объекта вместе с данными, необходимыми для подключения к этому экземпляру объекта (информацией, известной как object exporter identifier, или OXID). Возвращенный OXID содержит строку RPC binding для object exporter.
- Frames 9, 10, 11: клиент подключается к object exporter, управляющему интерфейсом, возвращенным процессом активации.
- Frames 12, 13, 14: клиент привязывается к интерфейсу ICalculator и аутентифицирует пользователя, аналогично процессу, описанному для frames 2-4.

- Frame 15: клиент вызывает `IClculator::SlowSum`, что можно идентифицировать по IID интерфейса и номеру метода, или орнум.
- Frames 41-46: каждые две минуты выполняется вызов `IOXIDResolver::ComplexPing` от клиента к серверу, используемый для уведомления сервера, что клиент все еще работает.
- Frame 233: сервер возвращает результаты операции, инициированной в frame 15.
- Frames 234-235: клиент получает интерфейс `IRemUnknown2`, используя текущее соединение с серверным объектом.
- Frames 234-235: клиент выполняет `IRemUnknown2::RemRelease` для интерфейса, полученного в frame 235.

Трасса неудачной активации DCOM

Поскольку инструменты мониторинга сети чаще всего используются для устранения неполадок, важно знать, насколько эффективен этот метод для обнаружения проблем в сетевом взаимодействии. Какие проблемы можно обнаружить таким способом? В этом разделе используется файл, захватывающий ошибку удаленной активации DCOM, довольно распространенный сбой.

Трафик, захваченный в случае отказа, показывает отклонение от потока взаимодействия, характерного для успешной активации. Различия могут очень быстро привести к наиболее вероятной проблеме. На рисунке 8.6 показано содержимое файла `08capture02.cap`, содержащего всю активность, ведущую к отказу.

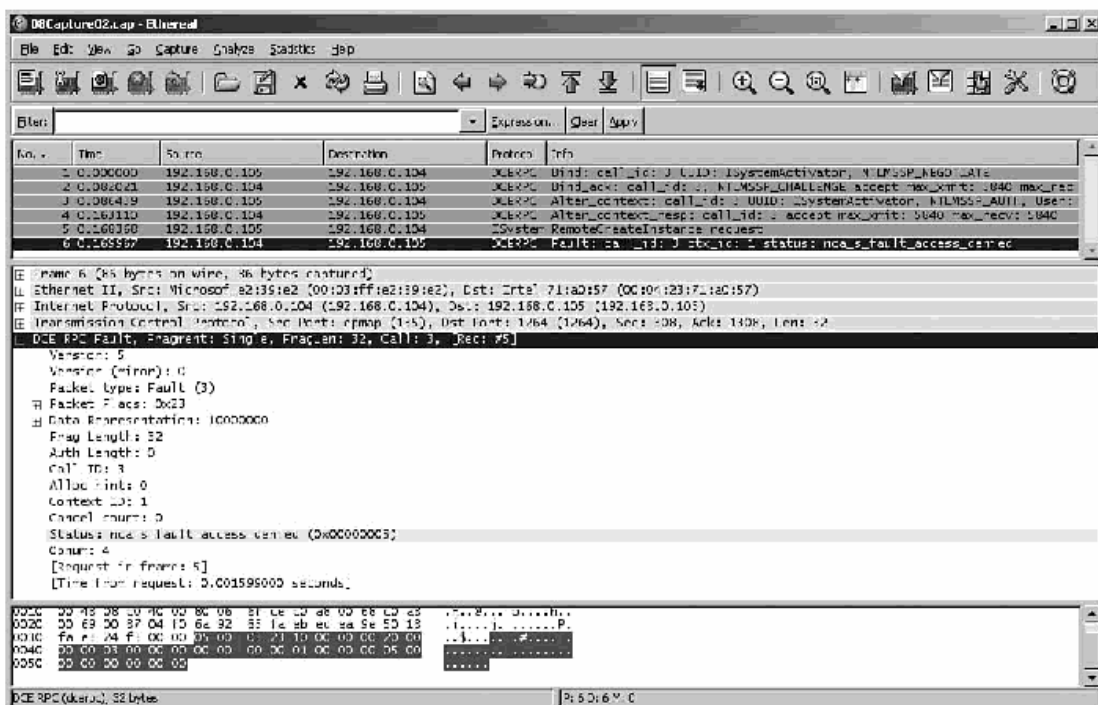


Figure 8.6 Packets captured during a failed DCOM activation

Рисунок 8.6. Пакеты, захваченные во время неудачной активации DCOM

Первые несколько пакетов играют роли, похожие на роли из предыдущего раздела, тогда как последний пакет активации полностью отличается. Интерпретация пакетов следующая:

- Frame 1: клиент отправляет запрос bind к интерфейсу `ISystemActivator`, и он также содержит сообщение согласования безопасности, как описано выше.

- Frame 2: сервер подтверждает bind пакетом Bind_ack.
- Frame 3: клиент отвечает на challenge сообщением Alter_context, используя информацию, связанную с именем пользователя TestAdmin, например пароль.
- Frame 4: сервер проверяет идентичность вызывающего сообщением Alter_context_resp. Интерфейс готов к использованию.
- Frame 5: клиент вызывает RemoteCreateInstance.
- Frame 6: сервер отклоняет активацию, и результат отправляется клиенту как fault frame, содержащий код ошибки access denied 0x00000005, аккуратно извлеченный инструментом из кадра ошибки.

Имя пользователя, использованное для этого запроса активации, ясно видно во frame 3. Поскольку frame 4 показывает, что учетные данные пользователя были приняты сервером, проблема активации в этом случае сводится к проблеме авторизации, специфичной для этого пользователя. С опытом, полученным в главе 7, относительно легко продолжить расследование и точно определить источник проблемы.

Трасса неудачной активации DCOM из-за фильтрации firewall

В последнее время ландшафт сетевой безопасности сместился в сторону ограничения входящего сетевого доступа с целью минимизации поверхности атаки. Начиная с Windows XP Service Pack 2 сетевой firewall встроен в операционную систему и включен по умолчанию. Большинство OEM-систем также поставляются с предустановленными другими firewall-продуктами. Хотя каждый firewall предоставляет механизм журналирования отклоненных запросов, гораздо проще использовать инструменты сетевой трассировки для обнаружения проблем взаимодействия благодаря единообразному интерфейсу, независимому от установленного firewall-продукта. Кроме того, расследование можно легко выполнить без внесения изменений в конфигурацию затронутой системы.

Файл 08capture03.cap, показанный на рисунке 8.7, иллюстрирует случай, когда firewall блокирует некоторые, но не все входящие запросы к системе.

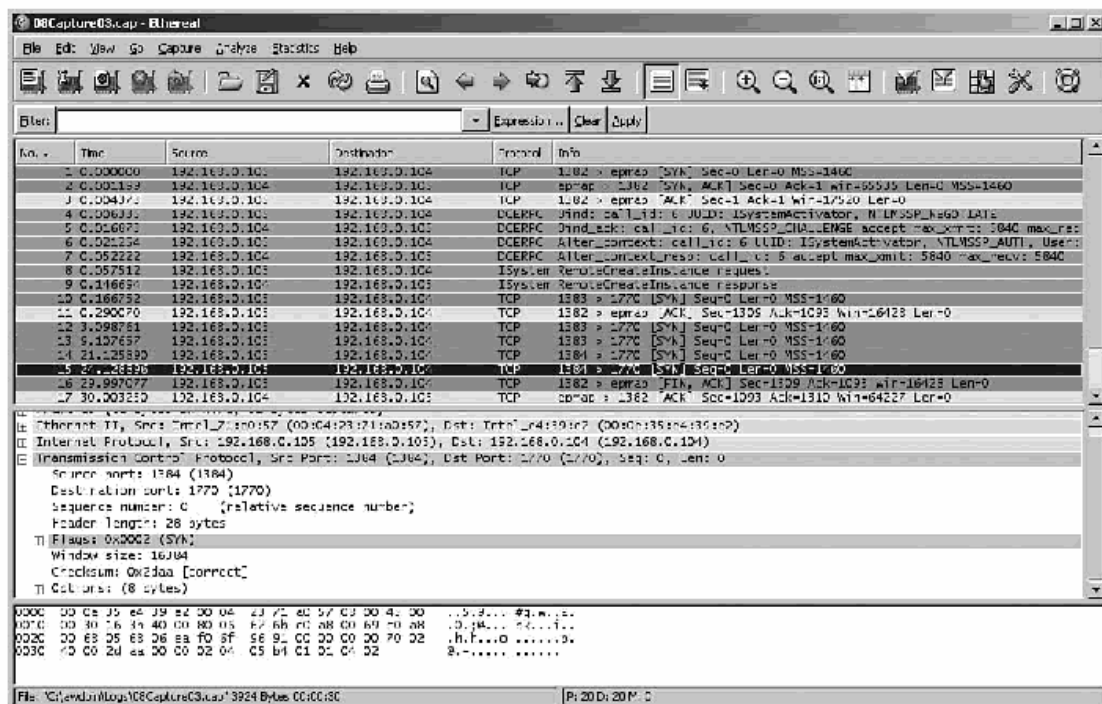


Рисунок 8.7. Пакеты, захваченные во время активации DCOM, заблокированной firewall

Роли пакетов интерпретируются следующим образом:

- Frame 1 to Frame 9: клиент активирует интерфейс, реализованный сервером, в данном случае ICalculator, тем же способом, что и в первой трассе, показанной в "Трасса успешной активации DCOM". Сервер возвращает маршированный интерфейс вместе с RPC binding information, необходимой для подключения к нему. В этом случае конечная точка - TCP-порт 1770.
- Frame 10 and beyond: клиент пытается установить TCP-соединение с сервером на порту 1770, как показано последовательностью SYS frames, но от сервера нет ответа. Клиент несколько раз безуспешно пытается установить соединение. В итоге вызов активации возвращает отказ в клиентском процессе.

В этом случае firewall пропускает трафик к порту endpoint mapper 135, но блокирует трафик к портам, динамически открытым в серверном процессе. С точки зрения клиентского кода запрос активации DCOM завершается ошибкой 0x800706ba. Когда firewall блокирует весь трафик в системе, даже начальное соединение с портом endpoint mapper завершается неудачей, как показано на рисунке 8.8. Кадры, иллюстрируемые в этом примере, находятся в файле 08capture04.cap.

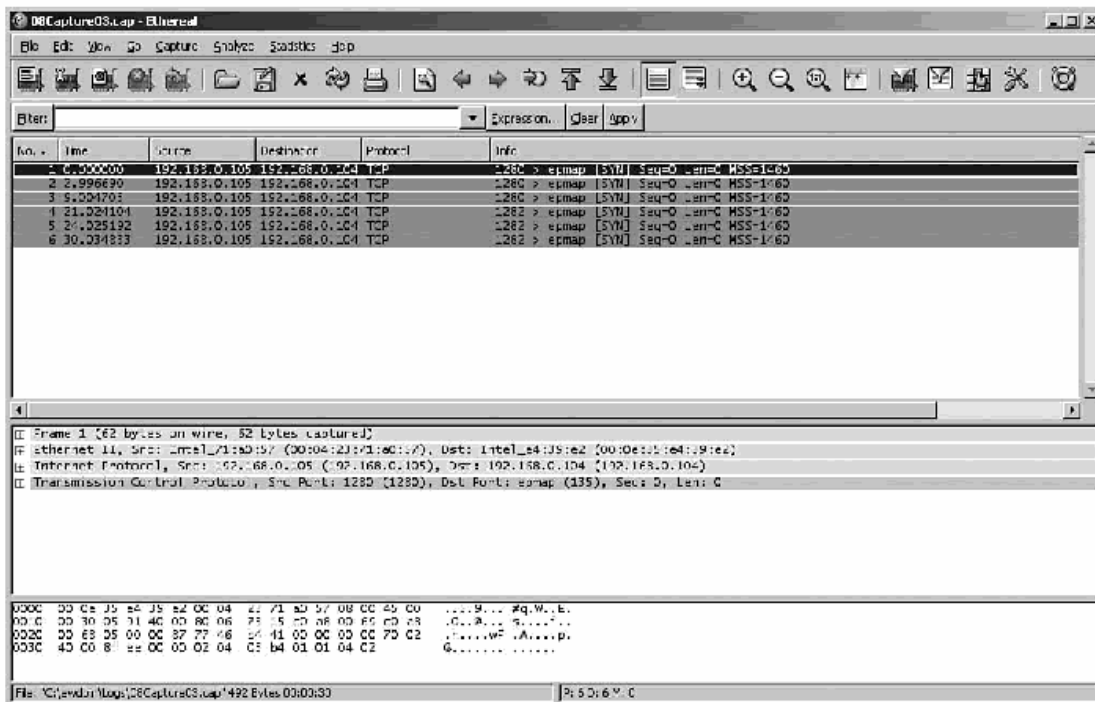


Figure 8.8 Packets captured using a DCOM activation attempt blocked completely by a firewall

Рисунок 8.8. Пакеты, захваченные при попытке активации DCOM, полностью заблокированной firewall

Другие сетевые протоколы

Другие протоколы взаимодействия можно анализировать теми же инструментами и по той же модели. Даже если вы не знакомы с активностью на проводе, создаваемой высокоуровневыми API-вызовами, распространенные сетевые протоколы обычно декодируются инструментами сетевого анализа. Для таких протоколов относительно легко найти связь между API-вызовом и соответствующей сетевой активностью.

Когда вы проектируете новый протокол, для принятия протокола было бы полезно предоставить собственный интерпретатор протокола, используемый внутри инструментов сетевого анализа. Так инструменты смогут декодировать все взаимодействие между системами. На рисунке 8.9 показан захват трафика как результат открытия реестра на удаленной машине. В этом случае первый декодер протокола декодирует TCP-трафик, следующий в стеке декодирует SMB-запросы, а еще один декодирует протокол MSRPC, построенный на взаимодействии через именованные каналы. Поскольку операции удаленного реестра довольно распространены, еще один декодер протокола интерпретирует MSRPC-трафик, созданный API удаленного реестра.

В файле захвата 08capture05.cap легко получить общее представление о сообщениях, которыми обмениваются клиент и сервер. Например, последовательность аутентификации легко распознается в frames 8-13, тогда как frames 18 и 21 содержат RPC-вызовы, сделанные с использованием протокола SMB.

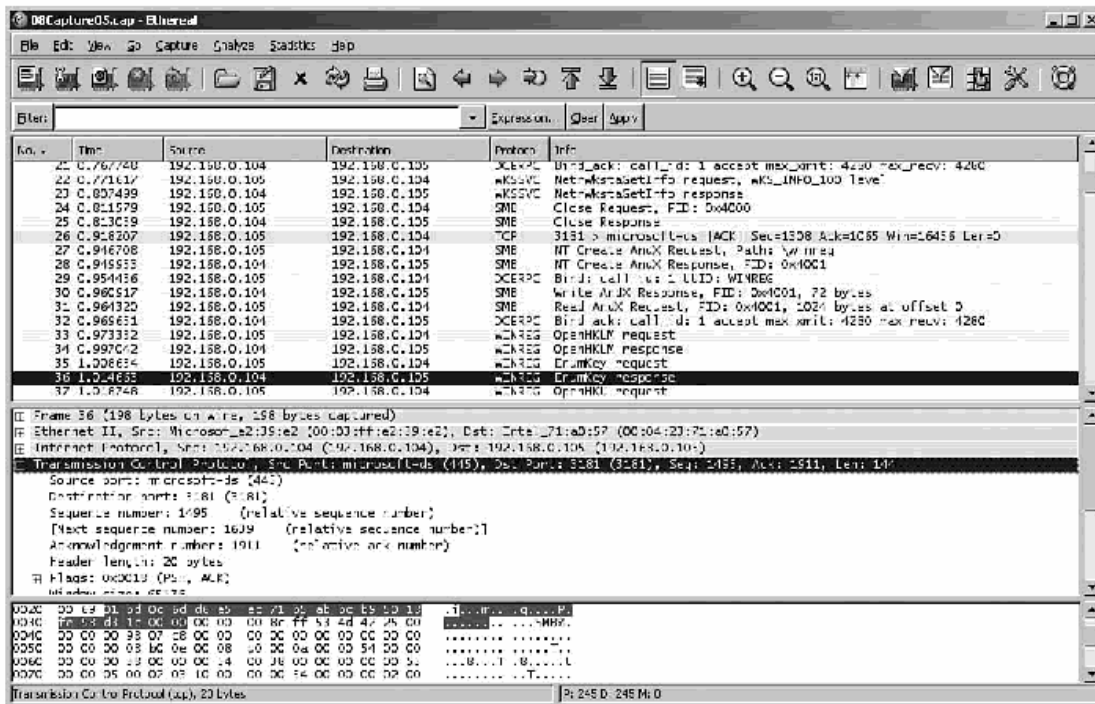


Figure 8.9 Packets containinga remote registry operations

Рисунок 8.9. Пакеты, содержащие операции удаленного реестра

В других случаях клиент и сервер соединены сложными сетевыми устройствами, например решениями балансировки нагрузки, и сетевая трассировка является единственным способом определить настоящую причину проблемы. Когда пакет теряется в трафике, сетевая активность, захваченная на клиентской сети, сравнивается с трафиком в серверной сети, чтобы доказать несоответствие.

Разрыв пути вызова

Предыдущий метод анализа сетевого трафика чрезвычайно эффективен для понимания того, что правильно или неправильно во взаимодействии между двумя компьютерами. К сожалению, один пакет на проводе может быть результатом очень сложной операции, часто затрагивающей более одного процесса. Любой сложный путь выполнения скрывает фактический источник ошибки, затрудняя определение процесса, в котором ошибка действительно происходит, и, следовательно, отладку проблемы.

Какой самый эффективный способ исследовать такую проблему? Один метод - визуализировать поток вызовов как цепь, начинающуюся в пространстве клиента, проходящую через несколько уровней взаимодействия и выходящую как серверный запрос в серверном процессе, как показано на рисунке 8.10. Кроме того, сервер может решить использовать службы, предоставленные еще одним сервером, прежде чем вернуть информацию клиенту, и цепь расширяется к следующему серверу. Затем обратный путь используется для возврата результатов в синхронных вызовах.

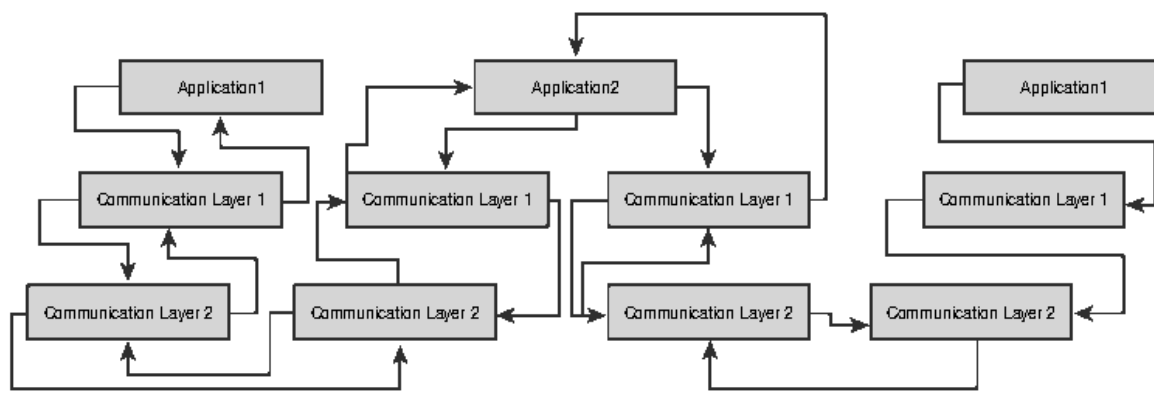


Figure 8.10 Sample execution path

Рисунок 8.10. Пример пути выполнения

Мы хотели бы провести аналогию между устранением неполадок сложного межпроцессного взаимодействия и электронной цепью, чтобы понять, что можно заимствовать из последней области. В электронных цепях есть различные выводы, наружу выдающие сигналы, важные для корректной работы печатной платы; они называются *test points*. Чтобы устранить неполадки печатной платы, инженер начинает где-то рядом с ее выходом и постепенно движется к входу цепи, чтобы локализовать неисправный участок. Иногда он будет прыгать между входом и выходом, чтобы локализовать участок, который получает правильный сигнал, но не генерирует ожидаемый ответ, однако большая часть расследования идет строго назад.

Этот шаблон можно успешно использовать при устранении неполадок распределенных системных решений, где ошибка возникает где-то посередине, и мы не знаем где. Ситуация похожа на цепь, когда выходной сигнал отличается от ожидаемого ответа на входной сигнал. Любую ошибку, происходящую в любом из процессов, используемых распределенной системой, можно рассматривать как короткое замыкание в большой цепи, которое не дает сообщениям течь глубже в систему. Вместо *test points*, недоступных в программном обеспечении, можно использовать отладчики Windows. Когда один компонент, являющийся частью потока взаимодействия, остановлен в отладчике пользовательского режима, вся операция, инициированная клиентом, не может продолжаться и зависает. Это подтверждает, что компонент играет активную роль в функциональном участке системы. В этом случае компонент, расположенный ближе к концу цепочки, скорее всего, и поднимает ошибку.

К этой проблеме подходят, предполагая, что весь сценарий работает, и начинают устранение неполадок с "низа" стека вызовов. Остановите последний процесс цепочки вызовов в отладчике (Application 3 на рисунке 8.10) и повторно выполните всю операцию. Если операция возвращается с тем же отказом, этот процесс не является источником отказа, потому что он даже не был вызван, и мы поднимаемся выше по стеку (Application 3 в данном случае) и повторяем процедуру. Когда вызов не возвращается, ошибку нужно искать в этом процессе, используя методики отладки, специфичные для однопроцессного сценария.

Для асинхронного или основанного на сообщениях взаимодействия процедуру нужно адаптировать к потоку сообщений внутри распределенной системы.

Примечание. Неудивительно, что отладка распределенного приложения трудоемка: поверх простой в использовании высокоуровневой библиотеки мы должны знать ее внутреннюю реализацию и системные вызовы, которые она использует.

Дополнительная техническая информация

Отладка межпроцессного взаимодействия - это эвристический процесс анализа информации из нескольких источников, чтобы понять отлаживаемую проблему. Этот раздел описывает, где перехватывать процесс удаленной аутентификации и как настроить инфраструктуру RPC на отправку дополнительной информации для каждой ошибки, встреченной при обработке сообщения. Последние два инструмента отображают информацию о различных интерфейсах, опрашивая базу данных endpoint mapper.

Удаленная аутентификация

В предыдущей главе вы узнали, как удаленные клиенты аутентифицируются на сервере с помощью SSPI-вызовов. стек вызовов потока, выполняющего вызов, часто показывает механизм аутентификации, используемый клиентом. В следующем примере клиент использует NTLM-аутентификацию, что видно по ее трехшаговому протоколу. Пример, показанный в листинге 8.29, взят из службы RPCSS, принимающей удаленный вызов активации. Сетевую активность, показанную на рисунке 8.5, можно сопоставить с SSPI-вызовами. Первый `secure32!AcceptSecurityContext` выполняется с данными, полученными из frame 4, а второй вызов - с данными, полученными из frame 6.

Листинг 8.29. Серверные точки останова, встреченные при использовании SSPI

```
0:009> bp Secur32!AcceptSecurityContext
0:009> bp Secur32!ImpersonateSecurityContext
0:009> g
Breakpoint 0 hit
eax=0009be20 ebx=00200a03 ecx=76f9d1e0 edx=0009722c esi=000971e0 edi=000af088
\eip=76f949ba esp=005bfd14 ebp=005bfd50 iopl=0 nv up ei pl nz na pe nc
Secur32!AcceptSecurityContext:
76f949ba 55 push ebp
0:003> * The first call to AcceptSecurityContext
0:003> k
ChildEBP RetAddr
005bfd10 780239bc Secur32!AcceptSecurityContext
005bfd50 7802389c RPCRT4!SECURITY_CONTEXT::AcceptFirstTime+0xd7
005bfeac 78010000 RPCRT4!OSF_SCONNECTION::AssociationRequested+0x3b8
...
0:003> g
Breakpoint 0 hit
eax=0009be20 ebx=00000000 ecx=0009722c edx=76f9d1e0 esi=00097220 edi=000000a6
eip=76f949ba esp=005bfe68 ebp=005bfea8 iopl=0 nv up ei pl nz na pe nc
Secur32!AcceptSecurityContext:
76f949ba 55 push ebp
0:003> * The second call to AcceptSecurityContext
0:003> k
ChildEBP RetAddr
005bfe64 78023b9f Secur32!AcceptSecurityContext
005bfea8 78023b22 RPCRT4!SECURITY_CONTEXT::AcceptThirdLeg+0x3e
005bff18 78004aed RPCRT4!OSF_SCONNECTION::ProcessReceiveComplete+0x595
005bff28 78001848 RPCRT4!ProcessConnectionServerReceivedEvent+0x20
...
0:003> g
Breakpoint 1 hit
eax=76f9d1e0 ebx=005bf83c ecx=0009722c edx=75867028 esi=000971e0 edi=005bf848
eip=76f95099 esp=005bf75c ebp=005bf768 iopl=0 nv up ei pl nz na pe nc
Secur32!ImpersonateSecurityContext:
76f95099 55 push ebp
0:003> * The identity of the client is available at the end of the call
0:003> k
ChildEBP RetAddr
005bf758 7802372a Secur32!ImpersonateSecurityContext
005bf768 78023701 RPCRT4!SECURITY_CONTEXT::ImpersonateClient+0x39
005bf770 78004443 RPCRT4!OSF_SCONNECTION::ImpersonateClient+0x3b
```

005bf778 75852a8f RPCRT4!RpcImpersonateClient+0x64
....

RPC Extended Error Information

Компоненты, использующие протоколы на основе RPC, могут получить пользу от расширенной информации, доступной в протоколе и управляемой системной политикой "Propagation of Extended Error Information". Политику, управляющую распространением информации об ошибках, можно найти в узле Administrative Templates раздела System, относящихся к конфигурации компьютера, как показано на рисунке 8.11.

Политику можно выборочно включить для интересующих нас процессов или для всех процессов. Информацию об ошибках, передаваемую по сети, затем можно анализировать инструментами packet sniffer. Приложения могут использовать эту информацию об ошибках при столкновении с ошибками, если такая информация доступна. Даже самый простой подход с журналированием этой расширенной информации помогает процессу отладки приложения.

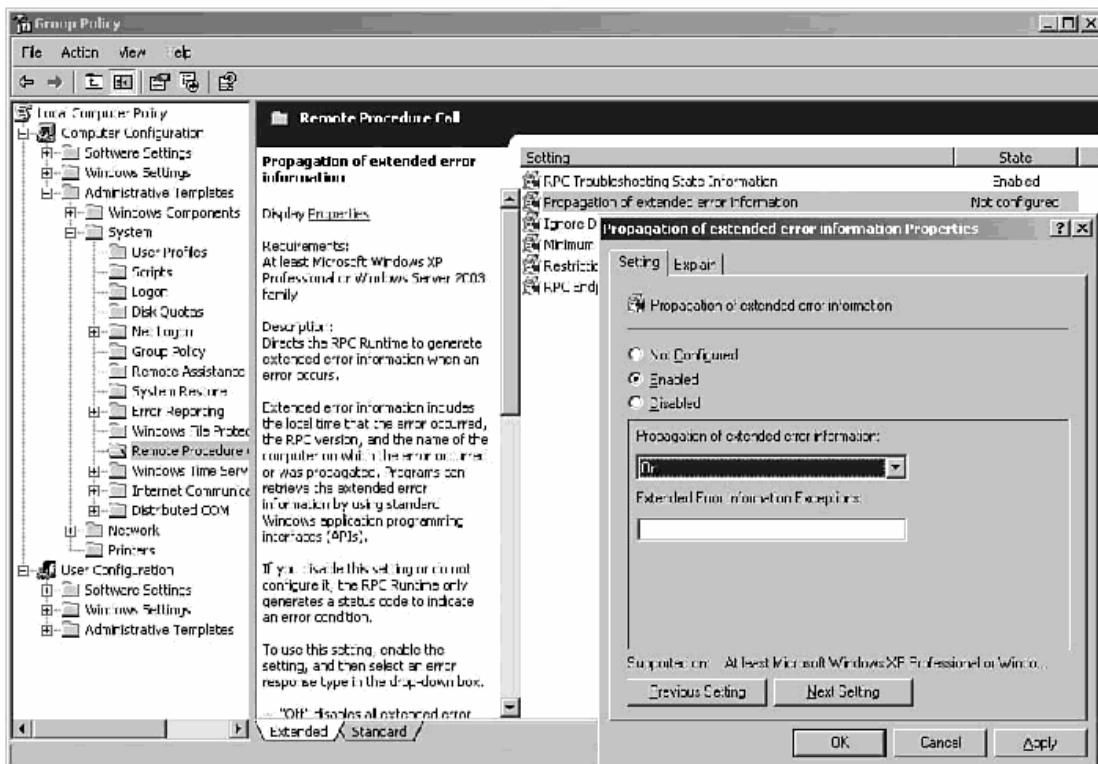


Figure 8.11 Enabling RPC Propagation of Extended Error Information

Рисунок 8.11. Включение RPC Propagation of Extended Error Information

Другие инструменты

При анализе RPC-отказов должен быть быстрый способ ответить на вопрос: "Зарегистрирован этот интерфейс или нет?" Два инструмента, используемые для такого поиска, - `rpcdump.exe` и `ifids.exe`, доступные как бесплатные загрузки от компании BindView и легко обнаруживаемые с помощью поисковой системы Интернета. Программа `Ifids.exe` выводит интерфейсы, зарегистрированные в endpoint mapper, связанном с конкретной конечной точкой. Использование и вывод инструмента довольно просты, как показано в листинге 8.30.

Листинг 8.30. Вывод всех интерфейсов, зарегистрированных на конечной точке \\PIPE\\winreg в локальной системе

```
C:\>ifids -p ncacn_np -e \\PIPE\\winreg \\.
Interfaces: 7
c8cb7687-e6d3-11d2-a958-00c04f682e16 v1.0
338cd001-2244-31f1-aaaa-900038001003 v1.0
4b112204-0e19-11d3-b42b-0000f81feb9f v1.0
00000134-0000-0000-c000-000000000046 v0.0
18f70770-8e64-11cf-9af1-0020af6e72f4 v0.0
00000131-0000-0000-c000-000000000046 v0.0
00000143-0000-0000-c000-000000000046 v0.0
```

`rpcdump.exe` выполняет функциональность `ifids.exe` для каждой конечной точки, зарегистрированной в системе. В листинге 8.31 показан упрощенный вывод, сгенерированный при запуске на системе Windows XP SP2. Список зарегистрированных интерфейсов огромен и зависит от конфигурации системы.

Листинг 8.31. Вывод всех интерфейсов, зарегистрированных в локальной системе, идентифицированной как \\.

```
C:\>rpcdump.exe \\.
IfId: 906b0ce0-c70b-1067-b317-00dd010662da version 1.0
Annotation:
UUID: 705bd495-44aa-4b4d-8e8d-1927d9dd9e8c
Binding: ncalrpc:[LRPC00000fc4.00000001]
IfId: 3c4728c5-f0ab-448b-bda1-6ce01eb0a6d5 version 1.0
Annotation: DHCP Client LRPC Endpoint
UUID: 00000000-0000-0000-0000-000000000000
Binding: ncalrpc:[dhcpcsvc]
...
IfId: 4b112204-0e19-11d3-b42b-0000f81feb9f version 1.0
Annotation:
UUID: 00000000-0000-0000-0000-000000000000
Binding: ncacn_np:\\\\XP-SP2-BACK[\\PIPE\\winreg]
```

Резюме

В этой главе мы сосредоточились на устранении неполадок распределенных служб с помощью различных инструментов и приемов, чтобы найти логический путь выполнения в клиент-серверном приложении. Вы узнали о важности диагностических возможностей, встроенных в протокол взаимодействия, а также о том, как использовать их при отладке защищенных приложений Windows.

Хотя общего рецепта нет, комбинацию этих приемов можно применять практически в любой ситуации. Хорошее общее понимание конкретной распределенной системы и лежащих в основе протоколов взаимодействия является предварительным условием успешного устранения неполадок, но это также путь к созданию более качественных систем в будущем. Эта глава также демонстрирует пользу использования устоявшихся протоколов взаимодействия, поддерживаемых программной индустрией множеством инструментов.

Глава 9. Утечки ресурсов

Без сомнения, утечки ресурсов - один из основных источников проблем, способных приводить к нестабильности программного обеспечения. Одной "маленькой" утечки ресурса достаточно, чтобы крупным корпорациям пришлось перезапускать критически важные приложения и службы (а в худших случаях - всю систему) и в процессе терять тысячи, а иногда и сотни тысяч долларов. Компании-разработчики не могут позволить себе игнорировать такие проблемы, как утечки памяти. Когда эти проблемы проявляются во время тестирования, на их устранение нужно выделять серьезное время и усилия. Конечно, одни утечки ресурсов отследить сложнее, чем другие, но вопроса о том, нужно ли их исправлять, возникать не должно. Вооружившись правильным ходом мысли и набором бесценных инструментов, разработчик может довольно быстро отследить проблемы такого типа. В этой главе обсуждаются эти мыслительные шаблоны и инструменты, позволяющие разработчикам эффективно находить утечки ресурсов.

Что такое ресурс?

В Windows ресурс - это любая сущность, занимающая место в системе. Место в данном случае определяется как физическая или виртуальная память. Примеры таких сущностей включают дескрипторы, различные формы выделений памяти и COM-объекты. Хотя многие из этих конструкций действительно сводятся к выделению памяти, способы, которыми разработчик получает и освобождает управление этими ресурсами, различаются. Например, выделение массива символов с помощью оператора `new` с последующим забытым освобождением через `delete[]` вызывает утечку памяти. (Размер утечки памяти прямо пропорционален количеству символов.) Точно так же создание экземпляра COM-объекта с помощью `CoCreateInstance` и забытый вызов освобождения тоже вызывают утечку памяти (и потенциально другие формы утечек, в зависимости от того, какие ресурсы, в свою очередь, выделяет COM-объект). Во многих случаях серьезность утечки ресурса прямо пропорциональна уровню абстракции, с которым вы работаете. Как в случае с COM-объектом: он может агрегировать другие COM-объекты, которые агрегируют другие COM-объекты, и так далее. Самый важный аспект при отладке проблем утечек ресурсов - то, как ресурс приобретается и освобождается.

Чтобы эффективно отлаживать утечки ресурсов, сначала нужно уметь анализировать проблему, стоящую перед вами. При утечках ресурсов не работает подход "сесть и начать случайную отладку" в надежде наткнуться на подсказку, которая выдаст источник проблемы. Нет, почти так же, как детектив должен собирать и упорядочивать улики и версии, это должен делать и разработчик. Часто версии оказываются неверными, и вы снова оказываетесь у доски, ищите другие версии о потенциально виновном коде. Полностью понимая систематический мыслительный процесс анализа утечки ресурса, вы сможете браться за любую утечку ресурса, будь то дескриптор, память или COM-объект. Чтобы помочь разработчику, работающему с проблемами утечек ресурсов, есть также набор инструментов, которые окажутся бесценными при проверке ваших версий.

Эта глава проведет вас по пути обнаружения корневой причины потерянных битов. В ней обсуждается ход мысли, стоящий за вашей работой битового детектива, а также подробно объясняются инструменты, имеющиеся в вашем распоряжении и облегчающие эту работу. В качестве тематических примеров мы используем два разных типа ресурсов:

- Дескрипторы
- Обычные выделения памяти

Далее мы рассмотрим процесс выявления и устранения утечки ресурса с высоты 30 000 футов, а затем начнем углубляться в детали.

Высокоуровневый процесс

Процесс устранения утечки ресурса в вашем коде показан на рисунке 9.1. В этом разделе мы подробно рассмотрим каждую часть процесса.

Шаг 1. Выявить потенциальные утечки ресурсов

Первый шаг в процессе работы с утечкой ресурса - убедить себя, что то, что вы видите, действительно является утечкой. Многие приложения включают внутренние кэши, которые заполняются во время высокой нагрузки и затем освобождаются в состоянии простоя, что приводит к ложному срабатыванию. Другим ложным срабатыванием может быть наблюдаемое общее увеличение использования памяти, которое не обязательно означает, что ваше приложение течет. Все хорошие расследования начинаются с основ, и поэтому первым шагом должно быть выявление потенциально текущих ресурсов. Это достигается тщательным анализом состояния машины, с особым вниманием к аномально большим объемам одного или нескольких типов ресурсов. Только после подтверждения этого можно уверенно переходить к стадии диагностики. Существует несколько разных инструментов, позволяющих анализировать состояние системы. Самый базовый инструмент (часть Windows) - Task Manager (CTRL+SHIFT+ESC или `taskman.exe`). С помощью Task Manager вы получаете глобальный обзор потребления ресурсов системы, а также более детальный вид для каждого работающего процесса, как показано на рисунке 9.2.

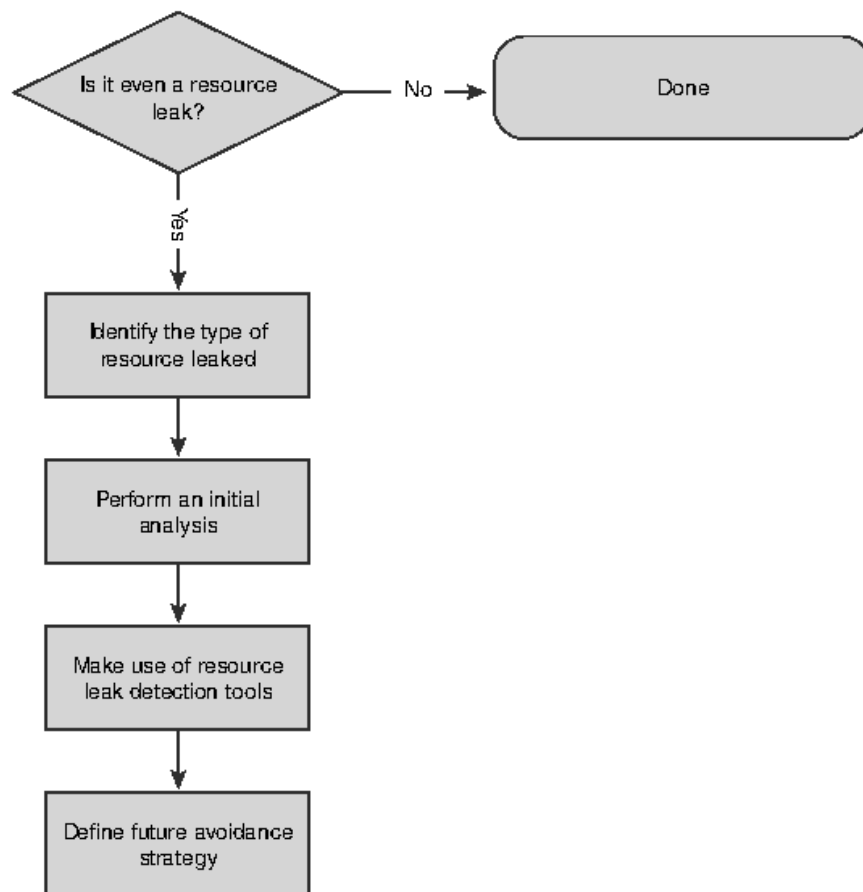


Figure 9.1 Resource leak analysis process

Рисунок 9.1. Процесс анализа утечки ресурсов

Task Manager можно настроить на показ различных типов данных процесса. Если исследуемый процесс показывает необычно высокий объем использования ресурсов, велика вероятность, что вы видите утечку ресурса.

На этом первый шаг процесса завершен. Вы определили большой объем ресурсов, потребляемых предполагаемым процессом, с помощью Task Manager, и пора переходить к стадии диагностики.

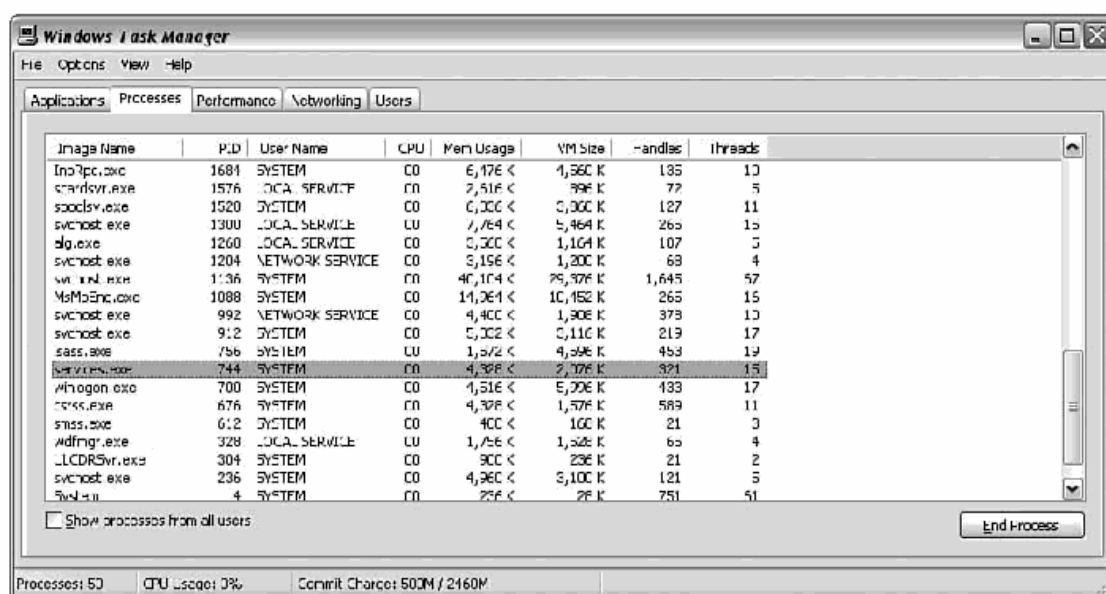


Figure 9.2 Detailed process view using Task Manager

Рисунок 9.2. Подробный вид процессов в Task Manager

Шаг 2. Что течет?

Следующий критически важный шаг - выяснить, какой тип ресурса течет в приложении. На шаге 1 мы уже затронули, как Task Manager может показывать полезные данные для любого процесса, работающего в системе. Доступные параметры можно настроить, открыв Task Manager (CTRL+SHIFT+ESC), затем выбрав View, Choose Columns. Это открывает диалог Select Columns на рисунке 9.3.

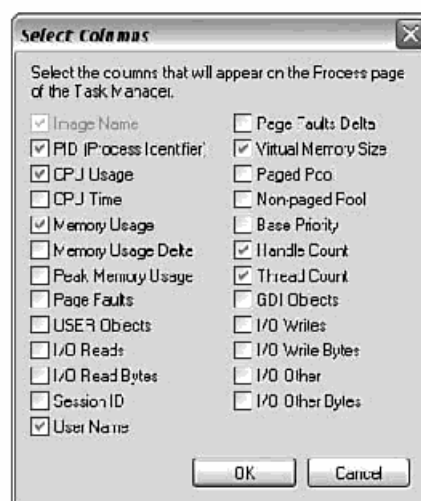


Figure 9.3 Customizing the process data in Task Manager

Рисунок 9.3. Настройка данных процесса в Task Manager

Столбцы, наиболее применимые к утечкам ресурсов:

- Memory Usage (размер working set)

- Memory Usage Delta
- Peak Memory Usage
- Virtual Memory Size
- Handle Count
- Thread Count
- GDI Objects (если приложение использует возможности UI) и USER Objects

После включения интересных столбцов Task Manager покажет эти данные как новые столбцы в представлении Processes.

Еще один отличный инструмент, который можно использовать для отслеживания утечек ресурсов, - Performance Monitor (Start, Run: `perfmon.exe`). Дополнительное преимущество Performance Monitor состоит в том, что он включает массу счетчиков, связанных с памятью, которые можно использовать для отслеживания утечек во времени.

Шаг 3. Первичный анализ

Предположим, что шаг 2 показал: ваш процесс использует большое количество дескрипторов (больше, чем должен). Следующий шаг - выполнить первичный анализ. Поскольку вы, вероятно, знакомы с кодом, который анализируете, отличная отправная точка - посмотреть пути кода, связанные с дескрипторами. Удивительно, сколько утечек ресурсов можно выявить, просто выполнив несколько базовых шагов и внимательно просмотрев код, работающий с соответствующим ресурсом. Что на самом деле происходит, из-за чего использование ресурса вообще начинает расти? Если у вас есть ответ на этот вопрос, можно начать с ревью путей кода во время этих операций или пройти их в отладчике, внимательно отслеживая использование этих конкретных ресурсов. После того как вы определили, где ресурс открывается, найти отсутствующее закрытие ресурса довольно просто. Поздравляем! Вы только что выявили и исправили утечку ресурса с очень низкими затратами.

К сожалению, не все решения проблем утечек ресурсов так же тривиальны, как простой просмотр кода, и иногда таким способом невозможно найти источник утечки. Для этого существует несколько причин:

- Проблема воспроизводится не всегда. Если отлаживаемая утечка ресурса происходит редко (даже при тех же шагах воспроизведения), очень сложно сузить область кода, где она может возникать.
- Утечка ресурса обнаружена на production-сервере, который заказчик не может позволить себе оставить "простаивать" во время отладки. Еще хуже, во многих случаях ограничения и проблемы связности не позволяют инженерам даже получить доступ к серверам.
- Часто стресс-тестирование приложения или службы дает очень недетерминированные результаты, и утечки приходится отлаживать на сервере, который интенсивно использовался и уже утек огромный объем ресурсов.

Если вы оказались в любой из ранее описанных ситуаций, ваша задача только что стала сложнее. Но не бойтесь: большое количество инструментов может помочь выявлять и устранять утечки ресурсов, которые иначе было бы невозможно или, как минимум, очень дорого разобрать простым ревью кода.

Шаг 4. Инструменты обнаружения утечек

Допустим, вы разработали службу, и она готова к включению в ночной стрессовый прогон. По самому определению стресс-тестового кода ваша служба будет десять часов подряд получать тысячи параллельных и разных запросов, как допустимых, так и недопустимых. Получив уведомление, что стресс-тестирование начнется сегодня ночью, вы в конце дня уходите домой, ожидая худшего. Утром публикуется отчет: "Падений нет, НО к концу стрессового прогона потребление памяти и число дескрипторов службы взлетели до небес". На статусной встрече команда руководства смотрит на вас в ожидании ответов.

В такой ситуации лучший курс действий - снять полные дампы текущего процесса (см. главу 13, "Postmortem Debugging") и попросить тестовую команду воспроизвести утечку ресурса (то есть снова запустить стресс-тестирование ночью). Перед началом нового стрессового прогона включите один или несколько инструментов обнаружения утечек, которые позволят гораздо эффективнее отследить проблему. Пока утечка воспроизводится, можно анализировать дампы, созданные ранее (см. главу 13). Если команда опасается оставить эту конкретную утечку ресурса в надежде снова ее воспроизвести, скажите им, что без инструментов обнаружения утечек расследование может занять недели. Правда, иногда именно столько времени уходит на postmortem-решение утечки ресурса без инструментов. Если они все равно хотят, чтобы вы отлаживали проблему без инструментов обнаружения утечек, есть механизмы, которые могут немного облегчить вам жизнь.

Выбор включаемых инструментов полностью зависит от того, какой ресурс течет. В таблице 9.1 представлены наиболее распространенные варианты.

Таблица 9.1. Инструменты обнаружения утечек ресурсов

Name	Resource Leaked	Download
htrace	Handles	Debugging Tools for Windows
UMDH	Heap Memory	Windows 2003 Server Resource Kit
LEAKDIA G	Various forms of memory allocators	ftp://ftp.microsoft.com/PSS/Tools/Developer%20Support%20Tools/LeakDiag/LeakDiag125.msi

Основная идея всех этих инструментов состоит в том, что, включая их, вы сообщаете Windows: вы хотите отслеживать все приобретения и освобождения ресурсов. Windows, в свою очередь, отвечает перехватом вызовов соответствующих API приобретения/освобождения ресурса и создает базу данных всех трасс стеков, которые приобретали и освобождали конкретный тип ресурса. Некоторые из этих инструментов (например, UMDH) запрашивают базу данных по всем вызовам, приводящим к выделению памяти кучи, и анализируют результаты, чтобы сформировать отчет о потенциально утекшей памяти.

После того как вы определили виновную трассу стека, поиск утечки ресурса становится гораздо более простой задачей (хотя и не тривиальной).

Обратите внимание, что некоторым из этих инструментов для корректной работы требуется поддержка Windows, и поэтому пользователь должен включить запись трасс стеков в операционной системе. Вы увидите эти инструменты в действии в последующих частях главы.

Шаг 5. Определить стратегию предотвращения в будущем

На этом этапе вы определили, что есть утечка ресурса, выполнили первичный анализ, запустили необходимые инструменты обнаружения утечек и наконец выявили и исправили виновный код. Следующий шаг, возможно самый важный, - убедиться, что только что обнаруженная проблема не повторится; лучший способ сделать это - определить стратегию предотвращения в будущем для конкретной проблемы. Как бы нам ни хотелось думать, что мы не совершаем одну и ту же ошибку дважды, это случается, и случается часто. Используя наши повседневные инструменты, мы можем убрать часть человеческой ошибки из кода и позволить системе "автоматически" обрабатывать ее.

Воспроизводимость утечек ресурсов

Воспроизводимость утечек ресурсов может принимать несколько разных форм. Три основные категории воспроизведения:

- Последовательные и полностью воспроизводимые
- Спорадические и воспроизводимые большую часть времени
- Спорадические и воспроизводимые очень редко

Последовательные и полностью воспроизводимые утечки ресурсов обычно встречаются во время разработки при запуске unit tests или набора автоматизированных тестов. Такие утечки ресурсов обычно проявляются каждый раз при запуске теста. Более того, запуск того же теста с теми же входными данными воспроизводит ту же утечку ресурса. Как оказывается, такие типы утечек также проще всего исследовать.

Спорадические утечки ресурсов, воспроизводимые большую часть времени, могут позволить роскошь включить инструменты обнаружения утечек и подождать несколько дней, пока утечка снова произойдет. Это предполагает, что заказчик готов ждать очередного проявления проблемы. Если он не готов, сценарий превращается в третью категорию проблем и одновременно в самую трудную форму утечек ресурсов.

Если утечка ресурса воспроизводится достаточно редко, не всегда реально просто сказать заказчику: включите инструменты обнаружения утечек и подождите. Заказчики, запускающие ваше приложение или службу на production-машинах, могут неохотно устанавливать утилиты и инструменты, не входящие в состав операционной системы. Кроме того, некоторые инструменты обнаружения утечек замедляют обработку и потребляют больше памяти, чем желательно. В таких случаях в вашем распоряжении остаются только два варианта: либо запросить разрешение на отладку их серверов (что почти никогда не дают), либо выполнить postmortem debugging. Postmortem подразумевает снятие снимка процесса и анализ снимка памяти на другой машине. (Подробнее о postmortem debugging см. главу 13.) Поскольку до запуска процесса никакие инструменты обнаружения утечек не работали, теперь вам предстоит искать утечку ресурса, просто анализируя состояние процесса. Такие задачи могут оказаться пугающими и заставить даже лучших инженеров усомниться в своих способностях. В следующих разделах вы увидите конкретные примеры утечек ресурсов и способы их анализа. Каждый раздел описывает конкретный тип утечки ресурса. Важно понимать, что хотя мы рассматриваем лишь несколько возможных утечек ресурсов, описанный пятишаговый процесс анализа утечек ресурсов можно применять к любому типу утечки ресурса.

Утечки дескрипторов

Ядро Windows определяет набор типов объектов, нативных для операционной системы Windows. Примеры таких типов объектов - файловые объекты, объекты процессов и объекты потоков. Каждый тип объекта имеет связанный набор свойств и API, работающих с этим конкретным типом объекта. Например, рассмотрим файловый объект. У файлового объекта есть набор атрибутов, определяющих, является ли файл скрытым, видимым, системным и так далее. Чтобы выполнять работу с типом объекта, нужно использовать связанный набор API. Например, Win32 API `CreateFile` позволяет создать или открыть файловый объект. Хотя ядро Windows в основном реализовано на C, свойства и функции типов объектов можно рассматривать как способ реализации инкапсуляции с помощью C. Сами типы объектов напрямую не раскрываются; вместо этого разработчик должен управлять типами объектов через C API, тем самым скрывая детали типа и позволяя внутреннему устройству типа со временем меняться. Кроме того, модель инкапсуляции способствует более надежной разработке, поскольку инкапсулированные данные никогда не изменяются вызывающим кодом напрямую, что снижает риск неправильного использования данных объекта. Большинство API предоставляется коду пользовательского режима через Win32 API. На рисунке 9.4 показана высокоуровневая архитектура дескрипторов.

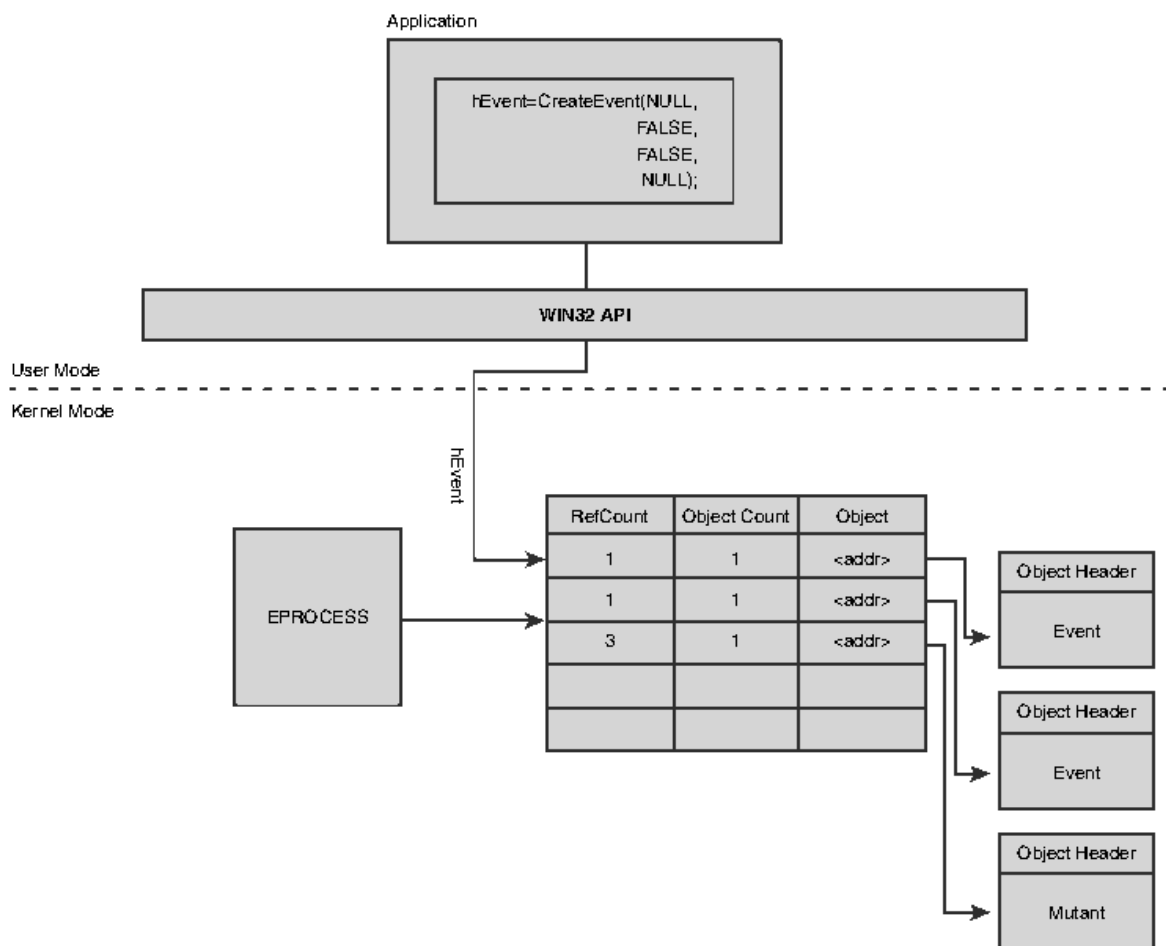


Figure 9.4 Windows handle architecture overview

Рисунок 9.4. Обзор архитектуры дескрипторов Windows

Чтобы работать с типами объектов, сначала нужно создать экземпляр этого типа объекта. Возьмем файл в качестве примера. API `CreateFile` позволяет создать или открыть существующий файл. Под капотом API `CreateFile` вызывает ядро, создает экземпляр типа файлового объекта и

возвращает клиенту получившийся дескриптор. Дескриптор - это то, что клиент использует для ссылки на недавно созданный экземпляр в ядре. Если вы хотите выполнять другие операции с новым файлом, при обращении к правильному экземпляру объекта всегда нужно использовать дескриптор. Когда работа с конкретным файлом закончена, нужно закрыть дескриптор, чтобы Windows могла правильно уменьшить счетчик ссылок на экземпляр и освободить память, когда счетчик ссылок достигнет нуля.

Самый важный вывод состоит в том, что после создания экземпляра типа объекта и возврата дескриптора вызывающему коду потребляется память для хранения данных этого экземпляра. Само собой разумеется, что если вы забудете закрыть дескриптор, память никогда не будет освобождена, и вы получите то, что обычно называют утечкой дескриптора. Разные инструменты показывают количество дескрипторов разных типов. Самый простой и удобный инструмент - Task Manager, позволяющий увидеть количество дескрипторов для конкретного процесса.

Поскольку процесс пользовательского режима использует дескриптор как связь с экземпляром объекта в режиме ядра, где хранится эта связь дескриптора в процессе пользовательского режима? Ответ: она хранится в таблице дескрипторов процесса. Рисунок 9.4 показывает таблицу дескрипторов, содержащуюся внутри процесса пользовательского режима. Он иллюстрирует связь между экземплярами объектов режима ядра и процессом пользовательского режима. В действительности таблица дескрипторов процесса фактически хранится в режиме ядра. Каждый процесс представлен в Windows экземпляром объекта, и каждый такой объект имеет связанную таблицу дескрипторов. Любой дескриптор в процессе пользовательского режима на самом деле является всего лишь индексом в таблице дескрипторов процесса. Каждая строка таблицы содержит указатель на экземпляр объекта режима ядра, маску доступа и флаги. Маска доступа определяет, какой доступ был запрошен при первом создании дескриптора. Например, в случае файлов процесс мог открыть файл для чтения, что было бы отражено в маске доступа.

Когда процесс завершается, Windows закрывает все дескрипторы в таблице дескрипторов этого процесса, чтобы гарантировать отсутствие утечек экземпляров режима ядра. Хотя Windows и закрывает все дескрипторы, открытые процессом при выходе, это не приглашение к небрежному кодированию. Определить время жизни приложения иногда непросто. Будет ли оно работать несколько минут или несколько месяцев до завершения? Иногда это действительно трудно сказать, и полагаться на выход процесса для очистки ресурсов - плохая практика программирования.

Приложение с утечкой

Прежде чем переходить к анализу приложения с утечкой, важно понять, как приложение работает, а также шаги, из-за которых утечка проявляется. Вы можете удивиться, зачем упоминать настолько очевидную вещь. Однако в реальности нам часто приходится исправлять код других инженеров, и важно получить хороший обзор перед началом работы. Чтобы проиллюстрировать пример приложения с утечкой, мы используем службу, предоставляющую функцию, которая позволяет клиентам читать текстовые файлы с серверной машины и возвращать содержимое этих файлов. Для простоты функция предоставлена как статическая библиотека. Тестовое приложение показывает приглашение, позволяющее пользователю ввести имя интересующего его файла и нажать Enter. Затем выполняется вызов службы, после чего отображаются первые 1023 символа этого файла.

Пример запуска приложения показан ниже:

```
C:\AWDBIN\WinXP.x86.chk\09Basichleak.exe
Client application console menu
=====
Enter filename to read > c:\boot.ini
Scheduled request successfully
Data read:
```



```
[boot loader]
timeout=30
default=multi(0)disk(0)rdisk(0)partition(1)\WINDOWS
[operating systems]
multi(0)disk(0)rdisk(0)partition(1)\WINDOWS="Microsoft Windows XP Professional"
/fastdetect /NoExecute=OptIn
multi(0)disk(0)rdisk(0)partition(1)\WINXP="Microsoft Windows XP Professional" /fast-
detect
&
Enter filename to read > _
```

Исходный код и двоичный файл приложения находятся в следующих папках:

```
Source code: C:\AWD\Chapter9\BasicHLeak\Client and
C:\AWD\Chapter9\BasicHLeak\Server
Binary: C:\AWDBIN\WinXP.x86.chk\09Basichleak.exe
```

Теперь, когда у вас есть хорошее понимание архитектуры кода вместе с утверждением QA-отдела, что существует утечка дескрипторов, мы начинаем следовать пятишаговому процессу анализа утечек ресурсов. Поскольку мы знаем, что ищем утечки дескрипторов, шаги 1 и 2 объединяются.

Шаги 1 и 2. Это вообще утечка дескрипторов?

Как всегда, первый шаг исследования потенциальной утечки ресурса - подтвердить, что она действительно есть. Утечки дескрипторов легко обнаружить с помощью Task Manager. По умолчанию Task Manager не показывает количество дескрипторов для конкретного процесса. Это можно включить, щелкнув вкладку Process, затем выбрав подменю View и Select Columns. Появится диалоговое окно с множеством параметров, которые Task Manager способен отображать. Установите флажок Handle Count и нажмите ОК. Теперь вкладка Processes отображает дополнительный столбец, показывающий количество дескрипторов, открытых у каждого процесса. Давайте попробуем это с нашим предположительно текущим приложением. Приложение с утечкой находится здесь:

```
C:\AWDBIN\WinXP.x86.chk\09Basichleak.exe
```

После запуска приложения появляется приглашение ввести имя файла. Начните с ввода допустимого имени файла (обязательно с полным путем) и нажмите Enter. Вывод покажет первые 1023 символа содержимого файла, затем появится новое приглашение для имени файла. Теперь самое время открыть Task Manager и посмотреть процесс 09Basichleak.exe на вкладке Processes. Конкретнее, нужно посмотреть столбец Handles и увидеть, что он показывает. Похоже, сейчас у процесса открыто 13 дескрипторов, как показано на рисунке 9.5.

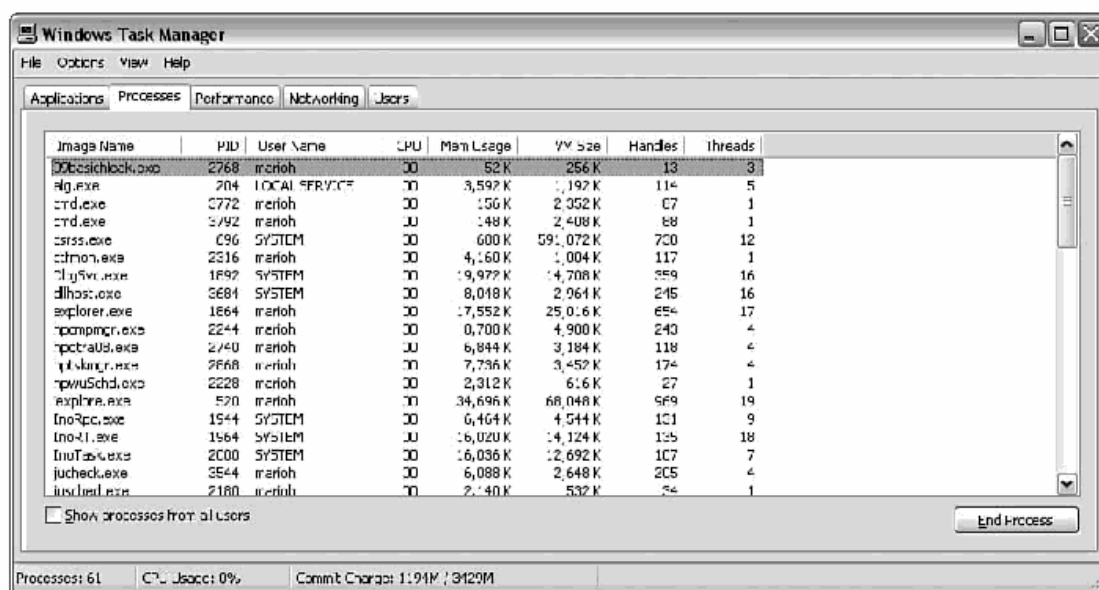


Figure 9.5 Handle count of the 09Basichleak.exe process after reading the first file

Рисунок 9.5. Число дескрипторов процесса 09Basichleak.exe после чтения первого файла

Мы вводим еще одно имя файла и нажимаем Enter. Снова проверяем число дескрипторов: теперь оно равно 14. Действительно, пока это выглядит нехорошо. Мы продолжаем процесс открытия файлов примерно дюжину раз, и, как и ожидалось, количество дескрипторов, похоже, увеличивается на единицу каждый раз, когда запрос выполнен. Рисунок 9.6 показывает количество дескрипторов, открытых процессом 09Basichleak.exe после выполнения запроса чтения файла 12 раз.

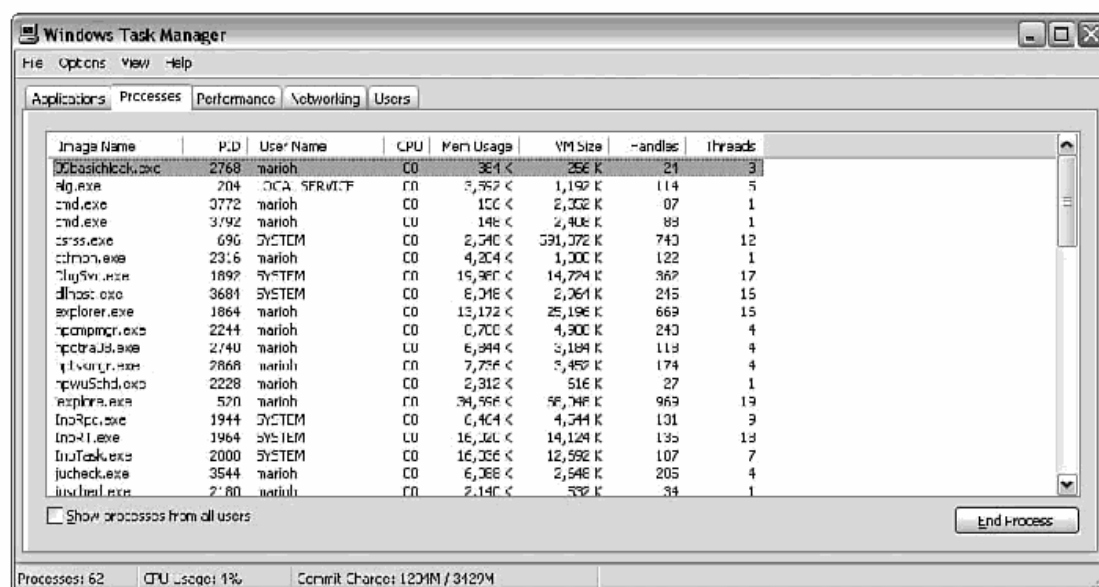


Figure 9.6 Handle count of the 09Basichleak.exe process after 12 files have been read

Рисунок 9.6. Число дескрипторов процесса 09Basichleak.exe после чтения 12 файлов

Иногда число дескрипторов может расти из-за кэширования, но после того как приложение некоторое время простояло без нагрузки, мы все равно не видим, чтобы число дескрипторов уменьшилось. На этом этапе можно довольно уверенно сказать, что мы видим приложение, в

котором текут дескрипторы.

Шаг 3. Первичный анализ

Поскольку дескриптор непрозрачен и может представлять любое количество типов объектов, как сузить проблему? Если бы мы могли определить, с каким типом объекта связан дескриптор, это могло бы дать лучшую подсказку об источнике утечки. Например, если все предыдущие дескрипторы являются дескрипторами потоков, мы могли бы сосредоточить усилия на соответствующих частях кода. К сожалению, Task Manager не всегда дает такую информацию, и приходится переходить к более мощному диагностическому инструменту.

Отличный инструмент для этого, Process Explorer, доступен бесплатно на www.microsoft.com. Process Explorer умеет показывать много полезной информации о работающих процессах, включая различные дескрипторы и связанные с ними типы. Стоит потратить время и поэкспериментировать с этим инструментом: у него отличные возможности исследования. Рисунок 9.7 показывает Process Explorer, запущенный для нашего приложения с утечкой.

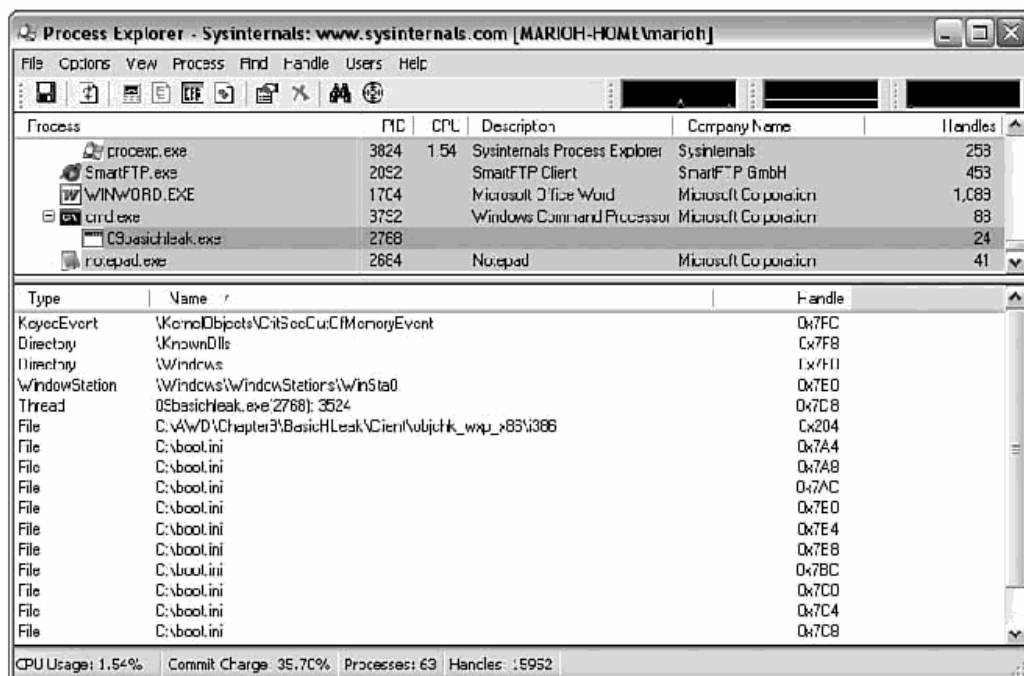


Figure 9.7 View of 09BasicHLeak.exe using Process Explorer

Рисунок 9.7. Вид 09BasicHLeak.exe в Process Explorer

Как видно, этот инструмент гораздо мощнее Task Manager потому, что способен отображать разные типы дескрипторов, открытых в процессе. Но на этом полезные возможности не заканчиваются: он также показывает имя открытого дескриптора. В нашем конкретном запуске мы снова и снова открывали один и тот же файл (BOOT.INI), и это ясно видно в UI Process Explorer. Количество файловых дескрипторов с именем BOOT.INI соответствует числу раз, когда мы открывали этот файл. Было бы справедливо сказать, что на этом этапе мы подтвердили: утечка действительно есть, и конкретный текущий дескриптор - файловый дескриптор. Поскольку мы точно знаем, какой тип дескриптора течет, и похоже, что можем воспроизводить это на каждой итерации выполняемой команды, первый шаг - пройти по пути кода ровно так, как это происходит при выполнении операции. Используемое тестовое приложение делает следующий вызов:

```

CHAR szFiledata[1024];
BOOL bRet=CServer::GetTextFileContents(hCompletionEvent,
                                       pFileName,
                                       szFiledata, 1024 );

if(bRet==FALSE)
{
    printf("\nFailed to read file\n");
}
else
{
    printf("\nScheduled request successfully\n");
    WaitForSingleObject(hCompletionEvent, INFINITE);
    printf("\nData read:\n");
    printf("%s\n", szFiledata);
}

```

hCompletionEvent - дескриптор события, которое мы создали. Мы используем это событие как механизм уведомления: сервер может установить его в сигнальное состояние, когда операция завершена. Это позволяет нам выполнять дополнительную работу, пока служба делает свою работу. pFileName в нашем случае - имя файла, введенное в командной строке (BOOT.INI), а szFileData - строковый буфер, выделенный на стеке, который содержит первые 1024 символа содержимого файла. Последний параметр, 1024, просто указывает количество символов, которое наш буфер способен хранить.

Пока ничто в нашем коде не указывает, что мы являемся причиной утечки файлового дескриптора. Дескриптор у нас есть, но это дескриптор события, который, согласно Process Explorer, не течет. Мы продолжаем расследование, рассматривая серверную реализацию GetTextFileContents:

```

BOOL CServer::GetTextFileContents(HANDLE hEvent,
                                  PWSTR pszFileName,
                                  PSTR pBuffer,
                                  DWORD dwBufferLen)
{
    BOOL bRet=FALSE;
    if(hEvent!=NULL && pszFileName!=NULL && pBuffer!=NULL && dwBufferLen!=0)
    {
        WorkerData* pWorkerData=new WorkerData;
        if(pWorkerData!= NULL)
        {
            pWorkerData->dwBufferLen=dwBufferLen;
            pWorkerData->pBuffer=pBuffer;
            pWorkerData->pszFileName=pszFileName;
            pWorkerData->hCompletionHandle=hEvent;
            bRet=QueueUserWorkItem(RequestWorker,
                                  (LPVOID) pWorkerData,
                                  WT_EXECUTEONBEHALF);

            if(!bRet)
            {
                delete pWorkerData;
            }
        }
    }
    return bRet;
}

```

Беглый взгляд на эту функцию не проясняет, где открывается файловый дескриптор. Более внимательный взгляд показывает, что мы используем QueueUserWorkItem с callback-функцией RequestWorker. Win32 API QueueUserWorkItem позволяет приложению поставить рабочий элемент в нативный пул потоков Windows. Это означает, что приложение предоставляет callback-функцию, которую операционная система вызывает с помощью одного из собственных потоков. Это выглядит логично, поскольку вызывающее службу приложение, согласно контракту, должно передать службе дескриптор события, которое будет установлено в сигнальное состояние после завершения запроса. На основе этой информации мы продолжаем расследование, рассматривая функцию RequestWorker:

```

DWORD WINAPI CServer::RequestWorker(LPVOID lpParameter)
{
    DWORD dwRet=0;
    WorkerData* pWorkerData=(WorkerData*) lpParameter;
    HANDLE hFile=CreateFile(pWorkerData->pszFileName,
                           FILE_READ_DATA,
                           FILE_SHARE_READ,
                           NULL,
                           OPEN_EXISTING,
                           FILE_ATTRIBUTE_NORMAL,
                           NULL);
    if(hFile!=INVALID_HANDLE_VALUE)
    {
        DWORD dwBytesRead=0;
        BOOL bRet=ReadFile(hFile,
                           (LPVOID) pWorkerData->pBuffer,
                           (pWorkerData->dwBufferLen-1),
                           &dwBytesRead,
                           NULL);

        if(bRet==TRUE)
        {
            dwRet=1;
        }
    }
    SetEvent(pWorkerData->hCompletionHandle);
    delete pWorkerData;
    return dwRet;
}

```

Теперь, кажется, мы куда-то продвинулись. Эта функция работает с файлами, судя по вызову API `CreateFile`, а также по API `ReadFile`. API `CreateFile` возвращает дескриптор открытого файла, сохраненный в локальной переменной `hFile`. Предполагая, что ошибок не происходит, код продолжает работу вызовом API `ReadFile`. После чтения файла дескриптор события, переданный вызывающим кодом, устанавливается в сигнальное состояние (чтобы указать, что операция завершена), и функция возвращается. Важно отметить, что когда мы говорим, что функция возвращается, она возвращается в пул потоков Windows. На этом этапе должно быть ясно, что мы упустили критически важный ингредиент этой функции. Мы открыли файл, что вернуло файловый дескриптор, но забыли закрыть файловый дескриптор перед возвратом. Каждый раз при выполнении запроса мы теряем один файловый дескриптор. Решение этой проблемы - добавить вызов `CloseHandle` (только если файл был успешно открыт) перед возвратом из функции. Также стоит отметить: когда вы находите утечку, всегда полезно, но часто упускается, посмотреть вокруг того же участка кода - возможно, рядом прячутся другие утечки.

Вы прошли пятишаговый процесс обнаружения утечек и сумели найти утечку уже на шаге 3. Найти утечку так рано в процессе очень недорого. К сожалению, не всегда бывает так, что проблема полностью воспроизводима и утечка возникает при каждой операции. Давайте слегка изменим наш код и покажем, как такие проблемы могут проявляться, а также как их отслеживать.

Более сложное приложение

Если вы можете отследить утечку дескриптора, зная только тип текущего дескриптора, считайте, что вам повезло (или что вы очень сильный code reviewer). Чаще всего требуется дополнительная диагностика.

Предыдущий пример показывает, как можно анализировать довольно простую утечку дескриптора и что можно сделать, чтобы докопаться до сути. Теперь пора рассмотреть еще одно приложение с утечкой, но с добавленной сложностью. Ключевое различие между предыдущим приложением с утечкой и этим состоит в том, что оно больше не теряет дескрипторы систематически; вместо этого возникновение утечки спорадическое и на первый взгляд случайное. Базовая архитектура та же:

есть клиентское приложение и серверное приложение (для простоты реализованное как статическая библиотека). Сервер предоставляет набор функций, позволяющих клиенту получать информацию, связанную с безопасностью вызывающего, например количество привилегий токена, количество групп и security identifier (SID). Клиентское приложение называется 09h1eak.exe и находится здесь:

```
Source code: C:\AWD\Chapter9\HLeak\Client and
C:\AWD\Chapter9\HLeak\Server
Binary: C:\AWDBIN\WinXP.x86.chk\09h1eak.exe
```

Двоичный файл 09h1eak.exe допускает следующие аргументы командной строки:

```
C:\AWDBIN\WinXP.x86.chk\09h1eak.exe /t:<num_threads> /i:<iterations_per_thread>
/s:<sleep_time_per_iteration>
```

/t:<num_threads> задает количество параллельных потоков, которые клиент использует при вызове операций на сервере.

/i:<iterations_per_thread> задает количество операций, которые будет выполнять каждый поток.

/s:<sleep_time_per_iteration> задает количество секунд ожидания между операциями в каждом потоке.

Снова, ради простоты, клиентское стресс-приложение напрямую линкуется со статической библиотекой, представляющей сервер. Начнем с однократного запуска приложения, указав, что нам нужно 5 потоков, 5 итераций на поток и 0 секунд ожидания:

```
C:\AWDBIN\WinXP.x86.chk\09h1eak.exe /t:5 /i:5 /s:0
```

Дайте приложению выполниться и одновременно наблюдайте потребление дескрипторов в Process Explorer. Рисунок 9.8 показывает результат запуска в представлении Process Explorer. Как видно, число дескрипторов выросло примерно с 8 в начале запуска приложения до 13 (не беспокойтесь, если ваше число дескрипторов отличается; это часть упражнения) в конце запуска. Нехороший признак. Теперь запустим его снова с теми же параметрами. Рисунок 9.9 показывает результаты в представлении Process Explorer.

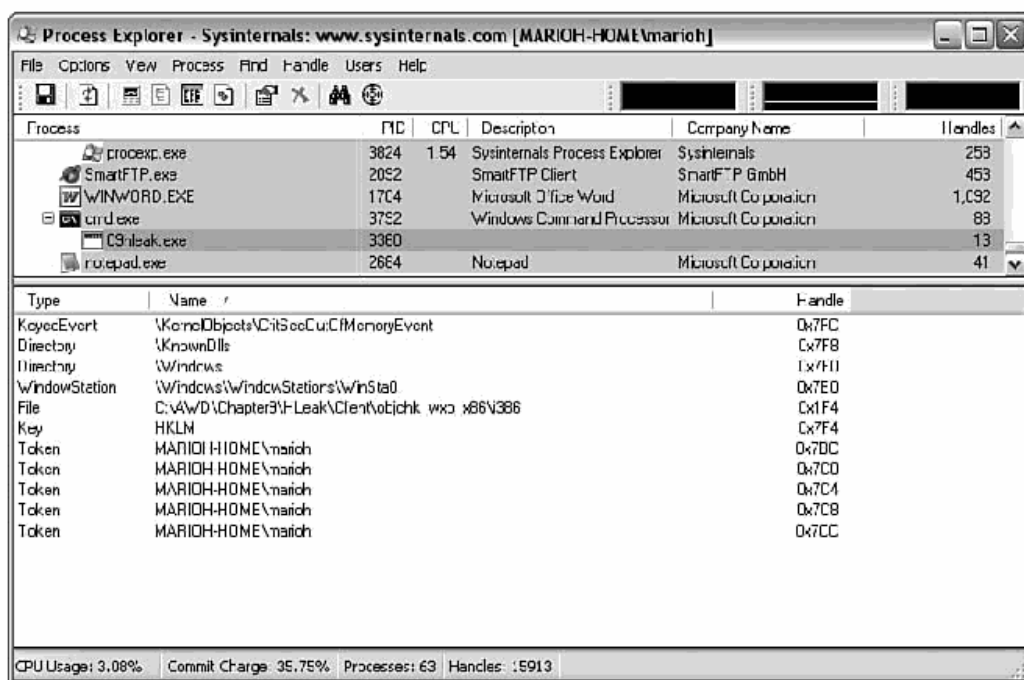


Figure 9.8 Handle consumption in the first run of 09h1eak.exe

Рисунок 9.8. Потребление дескрипторов при первом запуске 09h1eak.exe

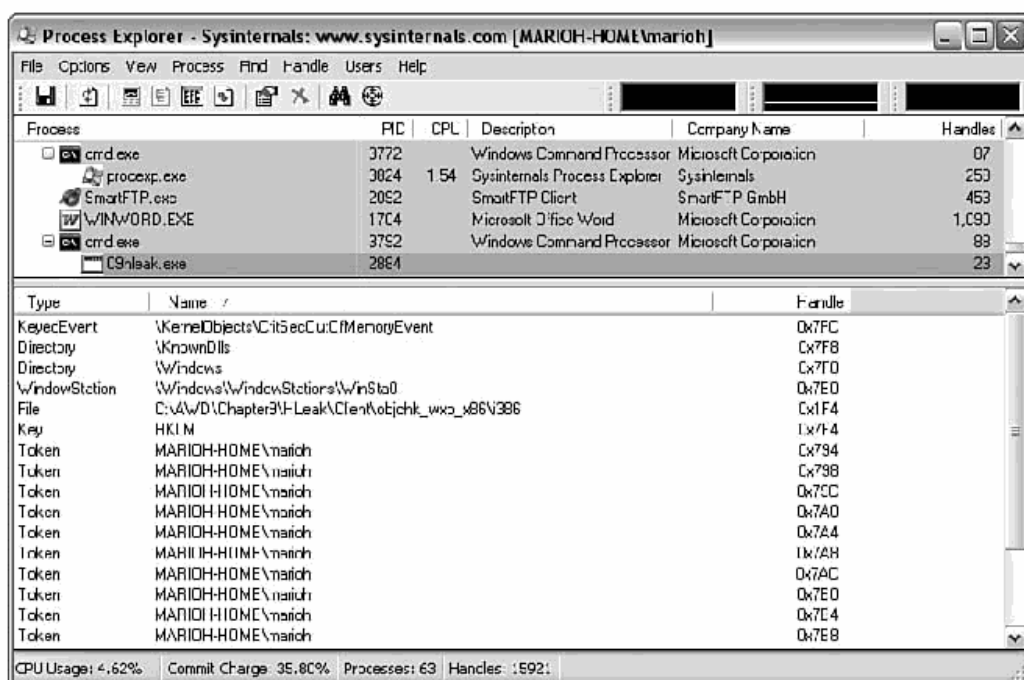


Figure 9.9 Handle consumption in the second run of 09h1eak.exe

Рисунок 9.9. Потребление дескрипторов при втором запуске 09h1eak.exe

На этот раз мы получили 23 дескриптора, хотя использовали тот же ввод. Если продолжать запускать приложение, вы заметите, что у утечки нет настоящего шаблона. Единственное наблюдение, которое, похоже, остается верным: если увеличить количество потоков и итераций, мы видим более крупную утечку. Например, при запуске следующей командной строкой:

```
C:\AWDBIN\WinXP.x86.chk\09h1eak.exe /t:20 /i:10 /s:0
```

число дескрипторов резко растет, как показано на рисунке 9.10.

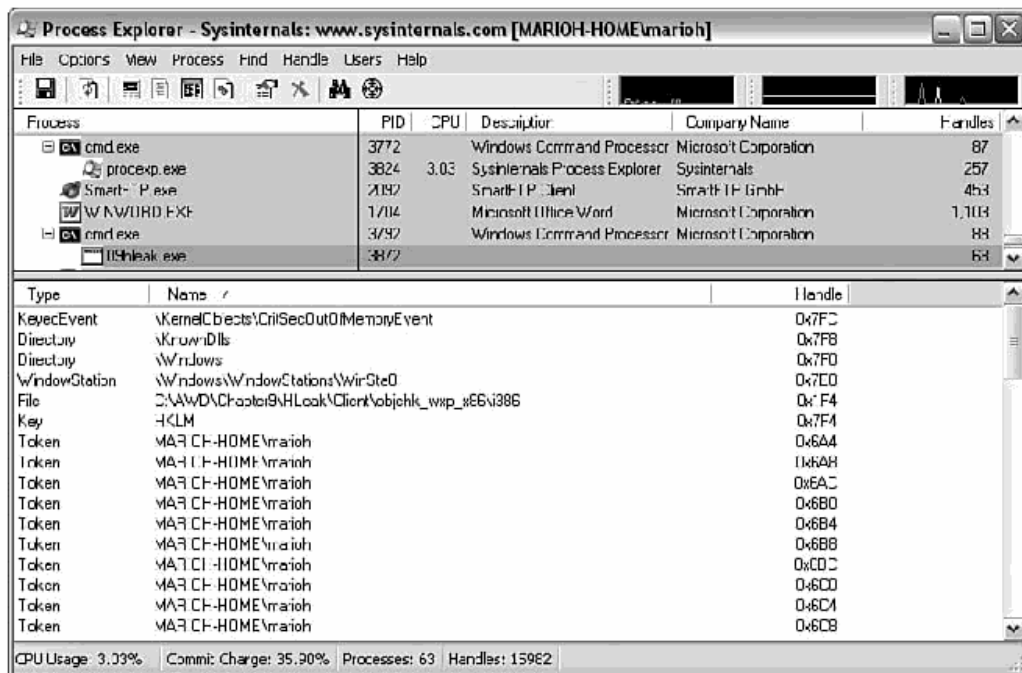


Figure 9.10 Handle consumption of 09h leak.exe with a large number of threads and iterations

Рисунок 9.10. Потребление дескрипторов 09h leak.exe с большим количеством потоков и итераций

Мы знаем, что клиентское приложение использует сервер в многопоточном режиме и вызывает различные функции сервера. Из представления Process Explorer мы также видим, что, похоже, текут дескрипторы token. Как отследить такой тип спорадической утечки дескрипторов? Ответ - шаг 4 в нашем процессе обнаружения утечек: использование инструментов обнаружения утечек.

Для этого упражнения постарайтесь не смотреть код заранее. Мы собираемся показать некоторые из самых важных инструментов отслеживания таких непредсказуемых проблем. Под непредсказуемыми мы понимаем утечки, которые не воспроизводятся стабильно и которые нельзя разумно отследить простым ревью кода. Мы пропустим шаги 1-3 нашего пятишагового процесса обнаружения утечек, потому что уже знаем, что есть утечка дескрипторов. Мы также предполагаем, что ее нельзя легко заметить простым ревью кода.

Шаг 4. Использование инструментов обнаружения утечек ресурсов

Итак, перед нами спорадическая и, по-видимому, случайная утечка дескрипторов. Хотя это может казаться сценарием конца света, есть и хорошие новости. Как ни странно, хорошая новость в том, что утечка, похоже, проявляется при каждом запуске приложения; она просто не воспроизводится с одинаковым количеством утекших дескрипторов. Почему это хорошая новость? Потому что это прекрасная возможность задействовать чрезвычайно мощную команду расширения !htrace, которая поможет обнаружить, где происходит утечка.

Htrace означает handle trace, и базовая идея команды состоит в том, чтобы включить в операционной системе отслеживание всех вызовов (со связанными трассами стеков), приводящих к открытию и закрытию дескрипторов. Когда утечка выявлена, можно использовать команду

расширения !htrace, чтобы вывести все трассы стеков в отладчике. После отображения всех трасс стеков отслеживать спорадические утечки дескрипторов становится гораздо проще. Посмотрим доступные параметры команды расширения !htrace.

Сначала запустите наше приложение с утечкой (с теми же параметрами командной строки, что и раньше):

```
C:\AWDBIN\WinXP.x86.chk\09hleak.exe /t:20 /i:10 /s:0
Press any key to start stress application...
```

Перед началом фактического воспроизведения утечки присоедините отладчик к только что созданному процессу, задайте путь к символам и введите !htrace -?:

```
0:001> !htrace -?
!htrace [handle [max_traces]]
!htrace -enable [max_traces]
!htrace -disable
!htrace -snapshot
!htrace -diff
```

Первое, что интересно в этом help-тексте, - параметр -enable. Запись всех трасс стеков для вызовов открытия и закрытия дескрипторов не является возможностью самой команды расширения !htrace; это возможность операционной системы. !htrace лишь говорит операционной системе включить трассировку стеков для данного процесса, прежде чем ее можно будет использовать. Это можно сделать с помощью команды -enable. Более того, если попытаться использовать другую команду расширения !htrace до включения трассировки стеков, вы получите следующую ошибку:

```
0:001> !htrace -snapshot
Handle tracing is not enabled for this process. Use "!htrace -enable" to enable it.
```

Включите трассировку стеков, как показано здесь:

```
0:001> !htrace -enable
Handle tracing enabled.
Handle tracing information snapshot successfully taken.
```

Как видно, переключатель -enable выполняет операцию из двух шагов. Во-первых, он включает трассировку стеков, а во-вторых, снимает снимок текущего состояния процесса относительно дескрипторов (на что указывает вторая строка вывода). Как только трассировка стеков включена, Windows начинает записывать все вызовы, приводящие к созданию и удалению дескрипторов. В следующий раз, когда вы снимете снимок (используя параметр -snapshot), команда расширения !htrace запросит у операционной системы все трассы стеков, приводящие к созданию и удалению дескрипторов, и отобразит их. Если дать приложению поработать еще немного (возможно, потеряв еще несколько дескрипторов), прервать его и снять еще один снимок, он снова покажет все трассы стеков, показанные ранее, плюс любые дополнительные дескрипторы, созданные или удаленные с момента предыдущего снимка. Систематически выполняя это, можно сравнивать снимки и видеть, какие части вашего кода создавали и/или удаляли дескрипторы, или, что интереснее, какие части создавали дескрипторы, но не закрывали их (что может быть виновником утечки).

Вернемся к нашему приложению с утечкой. Поскольку мы только что запустили процесс и включили трассировку стеков, дадим процессу завершиться. После завершения можно использовать команду расширения !htrace, чтобы получить список всех стеков, которые создавали и удаляли дескрипторы на протяжении жизни процесса. Поскольку даже "небольшие" процессы обычно создают и удаляют довольно большое количество дескрипторов, следующий пример показывает только фрагменты вывода. Также помните, что наше приложение с утечкой теряет дескрипторы очень спорадически в том смысле, что ни один запуск не гарантированно потеряет то же количество дескрипторов даже при тех же входных данных. Поэтому вывод, который вы увидите в своей отладочной сессии, скорее всего, будет отличаться от приведенного здесь.

```
...
...
```

```

...
0:001> !htrace -enable
Handle tracing enabled.
Handle tracing information snapshot successfully taken.
0:001> g
(d3c.18c): Break instruction exception - code 80000003 (first chance)
eax=7ffdd000 ebx=00000001 ecx=00000002 edx=00000003 esi=00000004 edi=00000005
eip=7c901230 esp=0028ffcc ebp=0028fff4 iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=0038 gs=0000 efl=00000246
ntdll!DbgBreakPoint:
7c901230 cc int 3
0:001> !htrace
-----
Handle = 0x0000078C - CLOSE
Thread ID = 0x00000410, Process ID = 0x00000D3C
0x0100176A: 09h!leak!wmain+0x0000027A
0x01001933: 09h!leak!wmainCRTStartup+0x0000012B
0x7C816FD7: kernel32!BaseProcessStart+0x00000023
-----
Handle = 0x00000798 - CLOSE
Thread ID = 0x00000410, Process ID = 0x00000D3C
0x0100176A: 09h!leak!wmain+0x0000027A
0x01001933: 09h!leak!wmainCRTStartup+0x0000012B
0x7C816FD7: kernel32!BaseProcessStart+0x00000023
-----
<output truncated>
...
...
...
-----
Handle = 0x00000480 - CLOSE
Thread ID = 0x00000C04, Process ID = 0x00000D3C
0x01001E1E: 09h!leak!CServer::GetGroupCount+0x000000BE
0x01001499: 09h!leak!ThreadWorker+0x000000E9
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
Handle = 0x00000480 - OPEN
Thread ID = 0x00000C04, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001D81: 09h!leak!CServer::GetGroupCount+0x00000021
0x01001499: 09h!leak!ThreadWorker+0x000000E9
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
<output truncated>
...
...
...
-----
Parsed 0x191 stack traces.
Dumped 0x191 stack traces.

```

Вывод команды расширения !htrace состоит из двух основных частей:

- Список всех записанных трасс стеков
- Сводный раздел ближе к концу

Сводный раздел показывает, сколько трасс стеков было разобрано и сколько было выведено в отладчик. Внимательно рассмотрим раздел трассы стека, соответствующий дескриптору 0x480.

```

Handle = 0x00000480 - OPEN
Thread ID = 0x00000C04, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001D81: 09h!leak!CServer::GetGroupCount+0x00000021
0x01001499: 09h!leak!ThreadWorker+0x000000E9
0x7C80B683: kernel32!BaseThreadStart+0x00000037

```

Каждая записанная трасса стека состоит из заголовка и самой трассы стека. Заголовок содержит следующую информацию:

- Значение дескриптора, представленное как `Handle = <value>`. В нашем примере значение дескриптора - `0x00000480`.
- Рядом со значением дескриптора указан тип выполненной операции. Это может быть `OPEN` или `CLOSE`. Наш конкретный пример показывает `OPEN`, что означает: показанная трасса стека является трассой, открывшей дескриптор.
- `Thread ID` и `Process ID`, представленные как `Thread ID = <value>` и `Process ID = <value>`. Эти значения показывают, какому потоку принадлежит трасса стека, а также идентификатор процесса. Можно склониться к мысли, что идентификатор процесса занимает место зря, поскольку дескрипторы относительно к процессу, а значит, идентификатор процесса должен совпадать с текущим процессом. Обычно это верно, но, как мы покажем позже, бывают случаи, когда другие процессы могут внедрить дескриптор в ваш процесс; в таком случае идентификатор процесса будет другим.

Трасса стека выглядит так:

```
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001D81: 09h!leak!CServer::GetGroupCount+0x00000021
0x01001499: 09h!leak!ThreadWorker+0x000000E9
0x7C80B683: kernel32!BaseThreadStart+0x00000037
```

Судя по трассе стека, похоже, что `09h!leak.exe` породил новый поток. (Подсказка - кадр `kernel32!BaseThreadStart`.) Главная точка входа потока - `ThreadWorker`, который вызывает серверную функцию `GetGroupCount`, а она, в свою очередь, вызывает `GetToken`. Похоже, именно функция `GetToken` в сервере открыла этот дескриптор. Число рядом с каждым кадром трассы стека - адрес возврата для соответствующего кадра.

Теперь, когда мы определили трассу стека, приведшую к открытию дескриптора, должна существовать соответствующая трасса стека, закрывающая конкретный дескриптор (`0x00000480`). Самый простой способ найти эту информацию - поискать значение дескриптора в выводе.

```
Handle = 0x00000480 - CLOSE
Thread ID = 0x00000C04, Process ID = 0x00000D3C
0x01001E1E: 09h!leak!CServer::GetGroupCount+0x000000BE
0x01001499: 09h!leak!ThreadWorker+0x000000E9
0x7C80B683: kernel32!BaseThreadStart+0x00000037
```

Трасса стека выглядит вполне осмысленно. `Thread ID` совпадают, и сами трассы стеков имеют смысл (`GetGroupCount` изначально вызвал `GetToken`, который открыл дескриптор. Затем `GetGroupCount` закрыл дескриптор). Должно быть ясно, что ключ к поиску текущих трасс стеков - найти те, которые открыли дескрипторы, но не имеют связанной трассы закрытия. Это может быть утомительным упражнением, потому что требует проверять каждый открытый дескриптор на наличие соответствующего закрытия в выводе, который может занимать много страниц. К счастью, команда расширения `!htrase` приходит на помощь. Можно использовать параметр `-diff` в `!htrase`, чтобы она сделала всю эту работу за вас. По сути, она сопоставляет все пути, приведшие к созданию и удалению дескрипторов (с момента последнего снимка), и сообщает только о трассах стеков, у которых нет связанного стека удаления. Попробуем.

```
0:001> !htrase -diff
Handle tracing information snapshot successfully taken.
0x191 new stack traces since the previous snapshot.
Ignoring handles that were already closed...
Outstanding handles opened since the previous snapshot:
-----
Handle = 0x000004D0 - OPEN
Thread ID = 0x000001B0, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
Handle = 0x000004E0 - OPEN
```

```
Thread ID = 0x00000E64, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
```

```
Handle = 0x000004E4 - OPEN
Thread ID = 0x000002D0, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
```

```
Handle = 0x000004EC - OPEN
Thread ID = 0x000001B0, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
```

```
Handle = 0x000004F0 - OPEN
Thread ID = 0x00000C04, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
```

```
Handle = 0x00000504 - OPEN
Thread ID = 0x00000E64, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
```

```
Handle = 0x00000508 - OPEN
Thread ID = 0x000002D0, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
```

```
Handle = 0x0000050C - OPEN
Thread ID = 0x00000D18, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
```

<output truncated>

```
...
...
...
-----
```

```
Handle = 0x00000754 - OPEN
Thread ID = 0x00000EA0, Process ID = 0x00000D3C
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
-----
```

Displayed 0x29 stack traces for outstanding handles opened since the previous snapshot.

Интересно, не правда ли? Команда показала 0x29 трасс стеков, у которых нет связанных вызовов закрытия дескриптора. Еще интереснее то, что все эти трассы стеков выглядят почти одинаково:

```
0x01001E85: 09h!leak!CServer::GetToken+0x00000055
0x01001B91: 09h!leak!CServer::GetSID+0x00000021
0x0100141B: 09h!leak!ThreadWorker+0x0000006B
0x7C80B683: kernel32!BaseThreadStart+0x00000037
```

Серверная функция GetSID вызывает GetToken, которая открывает дескриптор, но связанного вызова закрытия нет. Теперь самое время перейти к ревью кода. Посмотрев функцию GetSID в серверном коде, мы видим следующее:

```
PSID CServer::GetSID()
{
    PSID pSid = NULL;
    HANDLE hToken = INVALID_HANDLE_VALUE;
    hToken = GetToken();
    if(hToken!=INVALID_HANDLE_VALUE)
    {
        DWORD dwNeeded=0;
        BOOL bRes=GetTokenInformation(hToken,
                                     TokenUser,
                                     NULL,
                                     0,
                                     &dwNeeded
                                     );
        if(bRes==FALSE && GetLastError()==ERROR_INSUFFICIENT_BUFFER)
        {
            TOKEN_USER* pBuffer=reinterpret_cast<TOKEN_USER*>(new BYTE[dwNeeded]);
            if(pBuffer!=NULL)
            {
                BOOL bRes=GetTokenInformation(hToken,
                                             TokenUser,
                                             (LPVOID)pBuffer,
                                             dwNeeded,
                                             &dwNeeded
                                             );

                if(bRes==TRUE)
                {
                    DWORD dwSidLen=GetLengthSid(pBuffer->User.Sid);
                    pSid=static_cast<PSID>(new BYTE[dwSidLen]);
                    if(pSid!=NULL)
                    {
                        if(CopySid(dwSidLen, pSid, pBuffer->User.Sid)==FALSE)
                        {
                            delete pSid;
                            pSid=NULL;
                        }
                    }
                }
            }
            delete pBuffer;
        }
        return pSid;
    }
}
```

Строка, выделенная в книге полужирным, возвращает token в серверную функцию GetSID. Возвращенный дескриптор находится на стеке и не передается наружу из функции. Более того, в функции GetSID, похоже, вообще нет вызова CloseHandle, что по сути приводит к утечке дескриптора.

Как мы видели, когда дело касается спорадических утечек дескрипторов, которые нелегко отследить одним только ревью кода, команда расширения !htrace оказывает бесценную помощь. Она способна показывать аккуратные и чистые трассы стеков, включая дельты разных запусков. Общая стратегия использования !htrace такова:

1. Перед началом фактического воспроизведения утечки включить трассировку дескрипторов (с помощью !htrace -enable).
2. Запустить воспроизведение и дать процессу потерять дескрипторы.
3. Использовать !htrace -diff, чтобы найти виновные стеки.

Повторение шагов 1-3 даст достаточно информации, чтобы сузить проблему в коде и найти утечку с помощью ревью кода.

Только что описанный механизм трассировки дескрипторов работает чрезвычайно хорошо при поиске утечек дескрипторов. Однако есть одна оговорка, о которой нужно помнить. Трассировка дескрипторов использует массив для отслеживания всех дескрипторов. Если массив исчерпан, старые записи в массиве заменяются новыми. Фактически это означает, что чем дольше вы работаете с включенной трассировкой дескрипторов, тем выше вероятность повторного использования отдельных слотов массива; следовательно, информация о старых и потенциально утекших дескрипторах теряется. Лучший подход при использовании механизма трассировки дескрипторов - сузить проблему до достаточно небольшого и быстро воспроизводимого сценария, чтобы гарантировать, что массив трассировки дескрипторов не будет переиспользован.

Внедрение дескрипторов и !htrace

Как обсуждалось ранее, дескрипторы экземпляров объектов ядра относительноны к процессу и хранятся в таблице дескрипторов процесса. Поэтому дескриптор из процесса А нельзя использовать в процессе В, потому что такого дескриптора нет в его таблице дескрипторов. Может возникнуть соблазн заключить, что все дескрипторы в любом процессе открыты самим этим процессом. В большинстве случаев это верно, но, как всегда, есть исключения. Процесс может открыть дескриптор и внедрить его в другой процесс, если у внедряющего процесса есть нужные права доступа. Когда это происходит и внедренный дескриптор не закрывается целевым процессом, возникает утечка дескриптора. Еще хуже: команда расширения !htrace выдает довольно странную трассу стека для такого конкретного дескриптора.

Рассмотрим пример. В этом сценарии есть два консольных приложения. Первое консольное приложение называется 09target.exe и является стандартным консольным приложением со следующим кодом:

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>

int __cdecl wmain (int argc, wchar_t* pArgs[])
{
    printf("Waiting for handles...\n");
    printf("Press any key to exit application...\n");
    _getch();
    return 1;
}
```

Исходный код и двоичный файл приложения находятся в следующих папках:

```
Source code: C:\AWD\Chapter9\HInject\Target
Binary: C:\AWDBIN\WinXP.x86.chk\09htarget.exe
```

Как видно, этот код делает очень мало. Он просто простаивает и ждет, пока пользователь нажмет любую клавишу, после чего завершается. Чтобы проиллюстрировать устранение неполадок внедрения дескрипторов, исходный код другого участвующего процесса не показан. Просто запустите приложение (09hsource.exe).

```
C:\AWDBIN\WinXP.x86.chk\09hsource.exe
Enter process ID to inject handle into: _
```

С помощью Task Manager найдите идентификатор целевого процесса (обратите внимание на число дескрипторов) и введите его в приглашение 09hsource.exe. Когда приложение закончит свою работу, оно снова покажет то же приглашение. Снова откройте Task Manager, и вы увидите, что число дескрипторов увеличилось на единицу. Введите тот же идентификатор процесса еще раз и проверьте Task Manager. Снова вы увидите, что число дескрипторов у 09htarget.exe выросло на единицу. Продолжайте итерации, и вы увидите, что при каждом прохождении итерации приложения 09hsource.exe число дескрипторов в процессе 09htarget.exe увеличивается на единицу.

Кроме того, используя технику !htrace, описанную ранее, мы выводим все трассы стеков для процесса 09htarget.exe и замечаем, что действительно есть несколько трасс стеков, указывающих на утечки дескрипторы. Странная часть состоит в том, что трасса стека выглядит очень запутанной. Вот пример трассы стека, сообщенной !htrace в процессе 09htarget.exe:

```
Handle = 0x000007D8 - OPEN
Thread ID = 0x00000854, Process ID = 0x0000093C
0x01001363: 09htarget!XcptFilter+0x00000009
0x010014D3: 09htarget!_NULL_IMPORT_DESCRIPTOR+0x000000CB
0x7C816FD7: kernel32!BaseProcessStart+0x00000023
```

Помимо того что сама трасса стека не имеет особого смысла для нашего приложения 09htarget.exe, идентификатор процесса тоже, похоже, не имеет смысла. Фактически идентификатор процесса, указанный в трассе стека, не соответствует идентификатору процесса 09htarget.exe. С помощью Task Manager можно быстро сопоставить сообщенный процесс и найти процесс, о котором сообщает !htrace. Неудивительно, что это идентификатор процесса 09hsource.exe. Возвращаясь к нашему систематическому подходу к обнаружению утечек, можно уверенно перечислить следующие наблюдения:

- Целевой процесс теряет дескрипторы. Более того, целевой процесс теряет дескрипторы, за которые он не отвечает.
- Судя по трассам стеков, выданным !htrace в процессе 09htarget.exe, исходным процессом дескриптора является 09hsource.exe.

Самая большая проблема в выяснении происхождения дескриптора состоит в том, что имеющийся стек, похоже, не имеет смысла. Кадры стека указывают на места в нашем двоичном файле, которые, кажется, не лежат в допустимом пути кода. Остановимся и переосмыслим сценарий в целом. Исходный процесс - 09hsource.exe, и мы ожидали бы увидеть трассу стека того, как дескриптор был получен в этом процессе, при использовании !htrace. Единственная проблема в том, что мы присоединили отладчик к процессу 09htarget.exe, и полученный стек выглядит странно. Он выглядит странно только потому, что отладчик пытается разрешить адреса кадров вызова в контексте 09htarget.exe, но на самом деле адреса кадров вызова надежны только в контексте процесса 09hsource.exe. (В конце концов, именно этот процесс действительно открыл дескриптор.) Если попытаться разрешить адреса кадров вызова в контексте процесса 09hsource.exe, мы должны получить настоящую трассу стека.

Давайте возьмем трассу стека, которая не имела смысла, и попробуем. Присоедините отладчик к процессу 09hsource.exe, прервите выполнение и разрешите каждый адрес, указанный в трассе стека. Мы используем команду !n, чтобы разрешить адрес в соответствующее символическое имя:

```
...
...
...
0:001> !n 0x7C816FD7
(7c816fb4) kernel32!BaseProcessStart+0x23 | (7c816ff1) kernel32!CsrBasepNlsGe-
tUserInfo
0:001> !n 0x010014D3
(010013a8) 09HSource!wmainCRTStartup+0x12b | (0100152e) 09HSource!XcptFilter
0:001> !n 0x01001363
(010012c0) 09HSource!wmain+0xa3 | (010013a8) 09HSource!wmainCRTStartup
```

Разрешение адресов в возможные символы дает следующий потенциальный стек вызовов:

```
09HSource!wmain+0xa3
09HSource!wmainCRTStartup+0x12b
kernel32!BaseProcessStart+0x23
```

Это выглядит вполне разумно. Функция BaseProcessStart в kernel32 вызывает функцию wmainCRTStartup в нашем процессе 09hsource.exe, за которой следует вызов фактической функции wmain. Пока ничто не указывает, что мы открыли дескриптор и внедрили его в целевой

процесс. Ключ здесь - посмотреть на верх стека:

```
09HSource!wmain+0xa3
```

Этот кадр выполняет вызов другой функции. Если дизассемблировать эту функцию по указанному смещению, мы увидим следующее:

```
0:001> u 09HSource!wmain+0xa3
09HSource!wmain+0xa3:
01001363 8945f0 mov [ebp-0x10],eax
01001366 837df000 cmp dword ptr [ebp-0x10],0x0
0100136a 7515 jnz 09HSource!wmain+0xc1 (01001381)
0100136c ff151c100001 call dword ptr [09HSource!_imp__GetLastError (0100101c)]
01001372 50 push eax
01001373 68ac100001 push 0x10010ac
01001378 ff156c100001 call dword ptr [09HSource!_imp__printf (0100106c)]
0100137e 83c408 add esp,0x8
```

Ничто в этом дизассемблированном коде, похоже, не указывает на вызов функции, которая открыла бы новый дескриптор и внедрила его. Вспомните из главы 5, "Повреждение памяти, часть I - стеки", что адрес, указанный в трассе стека, - это адрес, на который указывает регистр EIP, и он также является адресом сразу после инструкции CALL. Дизассемблируем снова, но на этот раз вычтем несколько байтов:

```
0:001> u 09HSource!wmain+0xa3-11
09HSource!wmain+0x92:
01001352 8b55ec mov edx,[ebp-0x14]
01001355 52 push edx
01001356 ff1510100001 call dword ptr [09HSource!_imp__GetCurrentProcess (01001010)]
0100135c 50 push eax
0100135d ff1518100001 call dword ptr [09HSource!_imp__DuplicateHandle (01001018)]
01001363 8945f0 mov [ebp-0x10],eax
01001366 837df000 cmp dword ptr [ebp-0x10],0x0
0100136a 7515 jnz 09HSource!wmain+0xc1 (01001381)
```

Теперь мы куда-то продвинулись. Инструкция перед текущим указателем инструкций действительно является инструкцией CALL. Более того, инструкция CALL указывает на вызов API DuplicateHandle. Если посмотреть DuplicateHandle в MSDN, видно, что API позволяет не только дублировать существующий дескриптор в текущем процессе, но и дублировать его в другой процесс. Теперь тривиально исследовать параметры, переданные API DuplicateHandle, и увидеть, что мы действительно указываем идентификатор процесса 09htarget.exe.

Шаг 5. Определить стратегию предотвращения утечек дескрипторов в будущем

И последнее, но не менее важное: мы всегда должны убедиться, что извлекли урок из опыта, чтобы не совершать одни и те же ошибки дважды. Один отличный способ гарантировать, что дескрипторы не теряются, - применять конструкцию автоматического приобретения/освобождения. Очень похожая на auto pointers, эта конструкция позволяет приобрести дескриптор в любой области видимости и автоматически освободить его, когда auto-конструкция выходит из области видимости. В нашем серверном примере функцию GetSID можно было бы изменить примерно так, чтобы использовать конструкцию auto handle:

```
PSID CServer::GetSID()
{
    PSID pSid = NULL;
    HANDLE hToken = INVALID_HANDLE_VALUE;
    hToken = GetToken();
    AutoHandle autoHandle(hToken);
    ...
    ...
}
```


Класс `AutoHandle` принимает владение указанным дескриптором и закрывает его при выходе из области видимости. Расширение класса `AutoHandle` следующей функциональностью делает его еще более гибким:

- Перегрузка оператора присваивания позволила бы писать код вроде `AutoHandle autoHandle=GetToken();`.
- Перегрузка оператора приведения, позволяющая приводить `AutoHandle` к `HANDLE`, упрощает доступ к нижележащему `HANDLE`.
- Снятие владения нижележащим `HANDLE` в случаях, когда дескриптор должен быть передан наружу из текущей области видимости.

Это пример очень эффективного способа гарантировать, что дескрипторы корректно закрываются при выходе из области видимости. Без сомнения, существует много способов и альтернатив, позволяющих убедиться, что мы правильно используем инструменты и код для очистки дескрипторов. Какой выбрать - полностью зависит от ваших личных предпочтений и стиля кодирования.

Утечки памяти

Тогда как предыдущий раздел сосредоточен на утечках дескрипторов, этот раздел обсуждает более обычные утечки памяти. Под обычными я имею в виду утечки памяти, возникающие при непосредственном выделении и работе с памятью с помощью любых конструкций выделения памяти (таких как `new` и `HeapAlloc`). Прежде чем углубиться и посмотреть, как анализировать утечки памяти, начнем с краткого обзора того, как память управляется в Windows.

Менеджер памяти в Windows можно разбить на несколько уровней, как показано на рисунке 9.11.

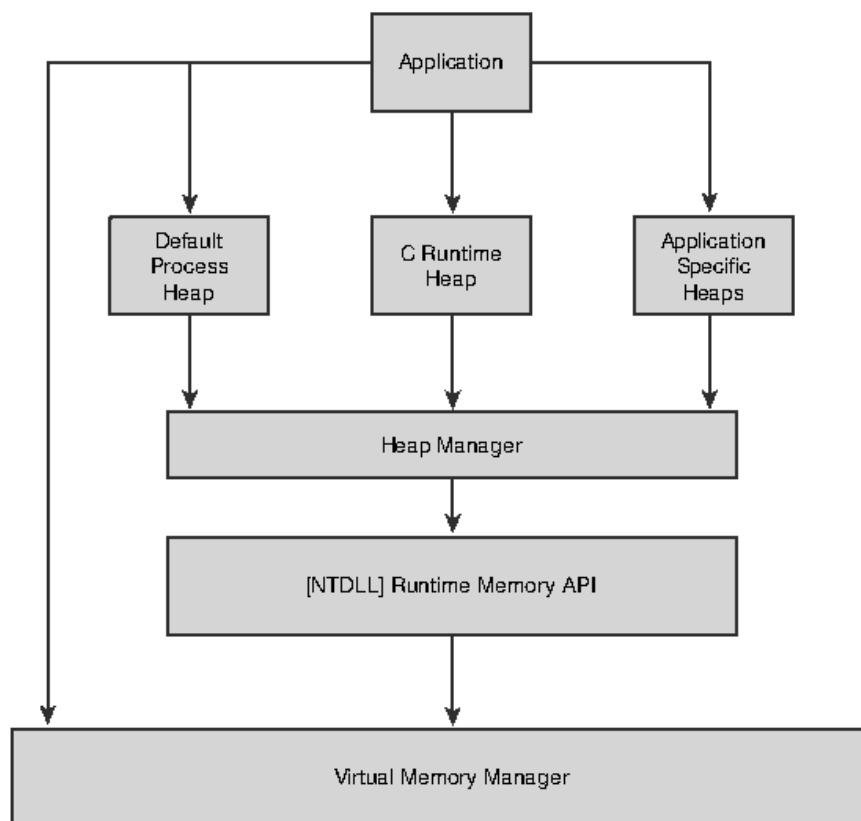


Figure 9.11 Windows memory architecture overview

Рисунок 9.11. Обзор архитектуры памяти Windows

Нижний и самый низкоуровневый компонент - Virtual Memory Manager (VMM). VMM является последней остановкой для всех запросов, связанных с памятью, в системе и работает с памятью в гораздо более низкоуровневой форме, чем привыкли разработчики приложений. VMM работает на основе крупных фрагментов памяти (страниц). Чтобы сделать выделения памяти небольшого размера более эффективными, Windows размещает поверх VMM слой абстракции. Этот слой абстракции называется менеджером кучи и состоит из API, который разработчики приложений могут использовать для очень простого выделения памяти. Кучу лучше всего рассматривать как слой изоляции, позволяющий приложениям создавать отдельные арены памяти внутри своего адресного пространства и работать с этими аренами (или кучами) изолированно. Конечно, приложения не обязаны создавать одну или несколько куч, прежде чем смогут начать манипулировать памятью. Вместо этого Windows делает очень логичное предположение, что любому приложению, вероятно, понадобится использовать хотя бы немного памяти, и создает кучу процесса по умолчанию при первом создании процесса. Начальный зарезервированный размер кучи процесса по умолчанию равен 1 МБ и растет по мере необходимости.

Как видно из рисунка 9.11, между VMM и менеджером кучи существует еще один слой, называемый Runtime Memory API. Это очень тонкий слой, который просто перенаправляет вызовы вниз к VMM. Например, менеджер кучи предоставляет API HeapAlloc, который на самом деле является лишь перенаправлением к нижележащему API RtlAllocateHeap, который, в свою очередь, вызывает VMM.

Поверх менеджера кучи находится слой приложения, который использует одну или несколько куч при выделении памяти. Приложение может выбрать либо кучу процесса по умолчанию, либо частные кучи (явно созданные приложением). Довольно часто приложения используют несколько куч, сами об этом не зная. Например, использование C runtime (такого как malloc или new) приводит

к выделению памяти в куче C runtime (созданной во время инициализации C runtime).

Предупреждение: при работе с несколькими кучами нужно быть внимательным. Поскольку несколько куч рассматриваются изолированно, выделение памяти из одной кучи и удаление этой памяти в другой куче является неопределенным поведением (см. главу 6, "Повреждение памяти, часть II - кучи").

Когда процесс собирается завершиться, Windows освобождает всю память, связанную с этим процессом, и уничтожает все активные кучи.

Теперь, когда вы понимаете, как память управляется в Windows, рассмотрим пример приложения, которое теряет память, и посмотрим, как можно проанализировать утечку памяти и в конечном итоге исправить ее.

Простая утечка памяти

В разделе "Утечки дескрипторов" мы использовали клиент-серверную парадигму, чтобы проиллюстрировать сценарий утечки дескриптора. Снова обратимся к тому же коду (слегка измененному), чтобы проиллюстрировать пример утечки памяти. Сервер позволяет клиентам выполнять любые из следующих вызовов:

- GetSID: получает SID токена вызывающего (токена потока или процесса).
- GetPrivilegeCount: получает количество привилегий токена вызывающего (токена потока или процесса).
- GetGroupCount: получает количество групп токена вызывающего (токена потока или процесса).

Клиентское приложение (09basicmleak.exe) порождает несколько потоков и случайно выбирает операцию для выполнения.

Исходный код и двоичный файл приложения находятся в следующих папках:

```
Source code: C:\AWD\Chapter9\BasicMLeak\Client and  
C:\AWD\Chapter9\BasicMLeak\Server  
Binary: C:\AWDBIN\WinXP.x86.chk\09basicmleak.exe
```

Согласно первоначальным отчетам, приложение, по-видимому, показывает рост использования памяти, и теперь перед нами стоит задача исправить эту потенциальную утечку. Начнем с первых двух шагов процесса анализа утечек ресурсов.

Шаги 1 и 2. Это вообще утечка, и что течет?

При использовании Task Manager потребление памяти может быть немного сложнее определить. Основная причина - способ, которым Task Manager сообщает потребление памяти процессами, по сравнению, скажем, с дескрипторами. Начнем с открытия Task Manager и выбора столбцов Memory Usage и Virtual Memory, которые сообщают, сколько памяти потребляет процесс.

Затем запустите процесс 09basicmleak.exe с 5 потоками и 50 итерациями на поток (0 времени сна), используя следующую командную строку:

```
C:\AWDBIN\WinXP.x86.chk\09basicmleak.exe /t:10 /i:50 /s:0
```

Перед фактическим запуском приложения откройте Task Manager, найдите 09basicmleak.exe и запишите столбцы Mem Usage и VM Size, как показано на рисунке 9.12.

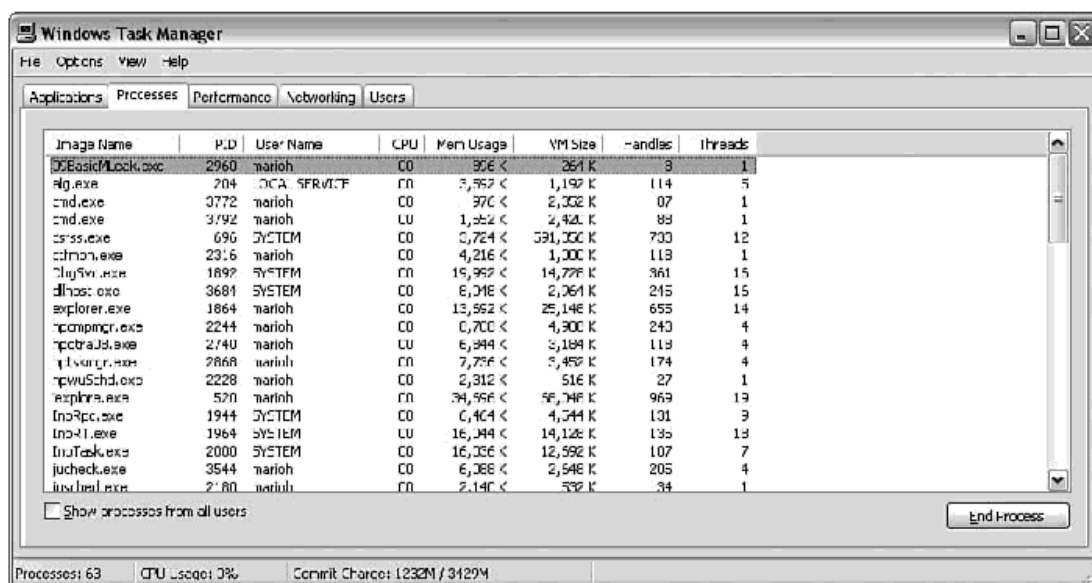


Figure 9.12 Initial memory consumption of 09basicmleak.exe

Рисунок 9.12. Начальное потребление памяти 09basicmleak.exe

Из рисунка 9.12 видно, что до запуска любых тестов процесс 09basicmleak.exe использует 896 КБ памяти и 264 КБ виртуальной памяти. Virtual memory показывает, сколько памяти процесс использует в целом (как находящейся в физической памяти, так и вне ее), тогда как столбец Mem Usage показывает, сколько физической памяти потребляет процесс (также известной как working set процесса). Обычно лучший индикатор утечек памяти - увеличение размера виртуальной памяти, а не колебания размера working set.

Теперь позволим процессу 09basicmleak.exe выполниться до завершения и посмотрим, что произошло с потреблением памяти.

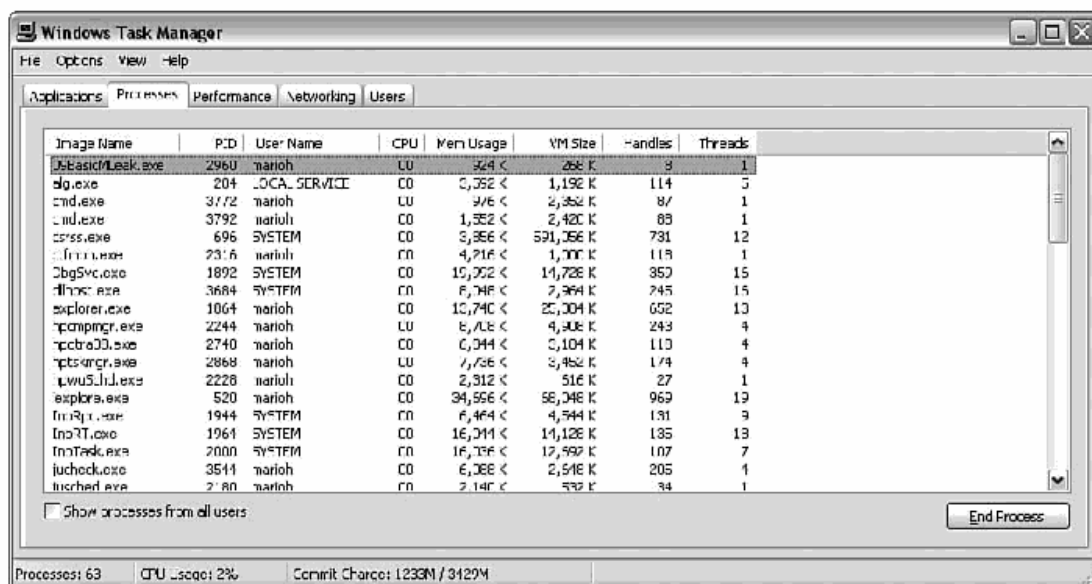


Figure 9.13 Memory consumption of 09basicmleak.exe after a complete run

Рисунок 9.13. Потребление памяти 09basicmleak.exe после полного запуска

Как видно из рисунка 9.13, увеличились и размер working set, и размер виртуальной памяти. Нехороший признак. Увеличение количества потоков и количества итераций на поток дает результат в таблице 9.2.

Таблица 9.2. Измерения роста памяти

Threads	Iterations	Memory (Kb)	Virtual Memory (Kb)
10	200	948	292
10	200	944	288
10	200	944	288
20	300	956	300
20	300	964	308
20	300	956	300

Судя по таблице 9.2, версия о потенциальной утечке памяти теперь подтверждается. Кроме того, утечка памяти не постоянна при одном и том же количестве потоков и итераций на поток. По природе это похоже на сценарий утечки дескрипторов, показанный ранее. Вместо перехода к шагу 3 мы предполагаем, что утечку памяти дорого отслеживать через ревью кода, поэтому сразу переходим к шагу 4: используем инструменты обнаружения утечек. Поиск утечек дескрипторов оказался гораздо проще с невероятно ценной командой расширения !htrace. Есть ли что-то похожее для утечек памяти? Безусловно! Инструмент, который спасет положение, называется UMDH.

Корректировки размера Working Set

Размер working set любого процесса постоянно корректируется Windows. Эти корректировки происходят из-за изменений системной нагрузки и приоритетов процессов. При запуске предыдущего сценария утечки памяти вы можете обнаружить, что сообщаемое потребление памяти немного отличается от показанного здесь. Это действительно ожидаемо. Утечка памяти спорадическая и, скорее всего, не дает дважды совершенно одинаковый объем утечки. Хотя небольшие различия в потреблении памяти увидеть можно, больших различий быть точно не должно. Если они есть, причина может быть в минимизации окна командной строки во время просмотра потребления ресурсов. Когда вы сворачиваете окно командной строки, Windows автоматически предполагает, что это окно должно быть отправлено в фон (то есть не используется), и поэтому урезает working set любого консольного приложения, которое сейчас выполняется в контексте этого командного окна. Уменьшая объем физической памяти, используемой командной оболочкой, Windows может отдать эту память другим приложениям, которым она теперь может понадобиться.

Шаг 4. Использование инструментов обнаружения утечек

Существует несколько инструментов, помогающих эффективно отслеживать утечки памяти. В следующих разделах мы обсудим несколько наиболее часто используемых инструментов.

UMDH

UMDH - это инструмент, входящий в состав установки Debugging Tools for Windows. Основная идея инструмента очень похожа на команду расширения !hrtrace. Мы начинаем с того, что просто говорим операционной системе сохранять трассы стеков для всех вызовов, приводящих к выделению памяти. Мы снимаем снимок использования памяти до начала выполнения приложения, а когда воспроизведение завершено, снимаем еще один снимок и сравниваем результаты. Это дает все трассы стеков, которые еще не были освобождены, и мы можем перейти к более тактическому ревью кода, чтобы найти виновника.

Сначала нужно включить трассы стеков для выделений памяти. Для этого мы используем инструмент gflags и включаем для 09basicleak.exe параметр Create user mode stack trace database. Подробнее о включении инструментирования через gflags см. главу 1, "Введение в инструменты".

Когда вы включили gflags для приложения 09basicleak.exe, запустите 09basicleak.exe следующей командной строкой:

```
C:\AWDBIN\WinXP.x86.chk\09basicleak.exe /t:10 /i:200 /s:0
Press any key to start stress application...
```

Перед запуском фактического воспроизведения нужно запустить UMDH, чтобы снять начальный снимок. UMDH можно запускать в трех режимах:

- Режим 1: создает дампы выделений кучи, сгруппированных по трассам стеков. Этот режим говорит UMDH создать дампы всех выделений кучи. Для этого режима существует несколько параметров, и большинство из них говорят сами за себя. Наибольший интерес представляют следующие:
 - -p сообщает UMDH, для какого идентификатора процесса записывать трассы стеков.
 - -l печатает сведения о файле и номере строки как часть трасс стеков.
- Режим 2: сравнивает два дампа выделений кучи, созданных в режиме 1. Это очень удобный способ анализа дампов. Вместо ручного обхода двух журналов мы можем позволить UMDH сделать всю работу по отчету о различиях.
- Режим 3: этот режим является сокращенным способом использования режимов 1 и 2.

Чтобы проиллюстрировать использование UMDH, мы покажем, как применять режимы 1 и 2, а не сокращенный режим.

Еще одно заключительное замечание об UMDH перед началом. Как и большинству инструментов обнаружения утечек, для получения хороших трасс стеков инструменту нужно сообщить, где искать символы. Это требуется, чтобы инструмент мог разрешать фреймы в символьную информацию. UMDH ожидает, что путь к символам задан в переменной окружения _NT_SYMBOL_PATH.

```
set _NT_SYMBOL_PATH=<path to your symbol store>
```

Подробнее о символах см. главы 2 и 4.

Теперь найдите идентификатор процесса недавно запущенного экземпляра 09basicleak.exe и введите в командной строке следующее. (UMDH находится в корневой папке каталога установки отладчика.)

```
UMDH.exe -p:<process ID> > firstsnap.txt
```

Запустите приложение до завершения и снимите еще один снимок:

```
UMDH.exe -p:<process ID> > secondsnap.txt
```

Теперь, когда у нас есть оба файла журнала, выполните следующую команду, чтобы сказать UMDH сравнить два журнала и перенаправить различие в новый файл diff.txt:

```
UMDH.exe -v firstsnap.txt secondsnap.txt > diff.txt
```

Теперь у нас есть файл diff.txt, который должен указать источник утекших выделений. Откроем diff.txt и посмотрим внимательно:

```
//
// Each log entry has the following syntax:
//
// + BYTES_DELTA (NEW_BYTES - OLD_BYTES) NEW_COUNT allocs BackTrace TRACEID
// + COUNT_DELTA (NEW_COUNT - OLD_COUNT) BackTrace TRACEID allocations
// ... stack trace ...
//
// where:
//
// BYTES_DELTA - increase in bytes between before and after log
// NEW_BYTES - bytes in after log
// OLD_BYTES - bytes in before log
// COUNT_DELTA - increase in allocations between before and after log
// NEW_COUNT - number of allocations in after log
// OLD_COUNT - number of allocations in before log
// TRACEID - decimal index of the stack trace in the trace database
// (can be used to search for allocation instances in the original
// UMDH logs).
//
+ d482 ( d482 - 0) 2a0 allocs BackTrace00081
+ 2a0 ( 2a0 - 0) BackTrace00081 allocations
ntdll!RtlAllocateHeap+00001292
09basicmleak!CServer::GetSID+00000115
09basicmleak!ThreadWorker+0000006E
kernel32!BaseThreadStart+0000003A
+ bca ( 2f28 - 235e) 2 allocs BackTrace00066
ntdll!RtlAllocateHeap+00001292
kernel32!LocalAlloc+00000081
ADVAPI32!AppmgmtInitialize+00000023
ADVAPI32!DllInitialize+00000105
ntdll!LdrpRunInitializeRoutines+000004D7
ntdll!LdrpInitializeProcess+00001BB6
ntdll!LdrpInitialize+0000018F
ntdll!KiUserApcDispatch+00000015
kernel32!BaseProcessStart+00000000
- be ( 3970 - 3a2e) 65 allocs BackTrace00068
ntdll!RtlAllocateHeap+00001292
msvcrt!malloc+00000060
msvcrt!malloc_crt+0000002A
msvcrt!_mbtow_envron+0000005E
msvcrt!_wgetmainargs+00000079
09basicmleak!wmainCRTStartup+0000013C
kernel32!BaseProcessStart+00000029
- be ( 2c30 - 2cee) 2 allocs BackTrace00072
ntdll!RtlAllocateHeap+00001292
msvcrt!malloc+00000060
msvcrt!malloc_crt+0000002A
msvcrt!stbuf+00000073
msvcrt!printf+00000045
09basicmleak!wmain+0000003E
09basicmleak!wmainCRTStartup+00000171
kernel32!BaseProcessStart+00000029
Total increase == ded0
```

Первая часть файла содержит очень полезную и подробную справочную информацию о формате файла. Что особенно приятно в UMDH: он сортирует перечисленные трассы стеков по размеру и количеству выделений. Трассы стеков с самым большим размером и самым большим количеством выделений находятся в начале файла. Разберем первую трассу стека:

```
+ d482 ( d482 - 0) 2a0 allocs BackTrace00081
+ 2a0 ( 2a0 - 0) BackTrace00081 allocations
ntdll!RtlAllocateHeap+00001292
09basicmleak!CServer::GetSID+00000115
09basicmleak!ThreadWorker+0000006E
kernel32!BaseThreadStart+0000003A
```

Первая строка сообщает, что у нас есть чистое увеличение на 482 байта из-за выделений, выполненных показанной трассой стека. Она также сообщает, что эта конкретная трасса стека была вызвана 240 раз, что привело к 240 выделениям. Также показан TRACEID этой конкретной трассы стека. Это может быть полезно, когда нужно сопоставить определенные трассы стеков в исходных файлах снимков. Вторая строка сообщает о чистом увеличении количества выделений из-за этой трассы стека. В нашем случае видно, что произошло 240 выделений. Наконец, у нас есть сама трасса стека, самая интересная часть информации. Первый фрейм (kernel32!BaseThreadStart) - функция, с которой все потоки начинают выполнение. Вторым фреймом входит в функцию ThreadWorker приложения 09basicmleak.exe. Это вполне логично, потому что приложение 09basicmleak.exe порождает потоки, которые, в свою очередь, вызывают сервер. Третий фрейм входит в серверную функцию GetSID, которая, в свою очередь, вызывает AllocateHeap. Похоже, сервер выделяет память, но не освобождает ее. Если посмотреть код GetSID, ясно, что он действительно выделяет память для SID, но никогда не освобождает ее. Может возникнуть соблазн немедленно исправить это вызовом free в функции GetSID, но будет ли это правильным исправлением? Более тщательный анализ показывает, что сервер выделил память для SID, но передает SID обратно клиенту, ожидая, что клиент его освободит. Посмотрев клиентский код, мы быстро видим, что клиент забыл это сделать. Решение - просто добавить соответствующий вызов free, и утечка исчезнет. Остальные трассы стеков в файле журнала показывают довольно стандартные стеки, которые не являются утечками. Помните, приложение все еще работает (хотя и готово завершиться), а выделения, сделанные операционной системой, освобождаются только при выходе процесса. Например, вторая трасса стека показывает, что загрузчик Windows выделил память во время инициализации DLL.

```
+ bca ( 2f28 - 235e) 2 allocs BackTrace00066
ntdll!RtlAllocateHeap+00001292
kernel32!LocalAlloc+00000081
ADVAPI32!AppmgmtInitialize+00000023
ADVAPI32!DllInitialize+00000105
ntdll!LdrpRunInitializeRoutines+000004D7
ntdll!LdrpInitializeProcess+00001BB6
ntdll!LdrpInitialize+0000018F
ntdll!KiUserApcDispatch+00000015
kernel32!BaseProcessStart+00000000
```

Это выделение не является тем, за что отвечали мы, и эту трассу стека можно спокойно отбросить.

UMDH - довольно мощный инструмент для поиска утечек памяти. Однако у него есть ограничения. Точнее, UMDH лучше всего работает с кодом, оптимизированным без FPO. Начиная с Windows XP SP2, весь код операционной системы компилируется с отключенными FPO-оптимизациями, так что это не должно быть большой проблемой. Другой недостаток состоит в том, что UMDH работает только с диспетчером кучи Windows по умолчанию. Пользовательские аллокаторы (например C runtime) отслеживаются UMDH не очень хорошо. Чтобы обойти эти недостатки, был создан еще один инструмент, называемый LeakDiag; его мы рассмотрим дальше.

UMDH и BSTR

BSTR по существу представляет собой не более чем COM-совместимую строку (инкапсулирующую как длину строки, так и содержимое). В большинстве случаев, когда мы используем COM-интерфейсы, принимающие строки на вход, эти строки будут иметь тип BSTR. Выделение BSTR с помощью API SysAlloc и забытое освобождение приводит к утечке памяти. Такие утечки памяти не гарантированно будут пойманы UMDH. Более того, большую часть времени трассы стеков, показываемые UMDH, не имеют смысла и могут увести вас по длинному и дорогому ложному пути. OLE кэширует BSTR, чтобы избежать постоянных обращений к менеджеру памяти. Поэтому если выделить BSTR, освободить его, а затем позднее выделить еще один BSTR и забыть освободить уже его, UMDH сообщит исходную, не утекающую трассу стека для выделения. Если вы когда-либо окажетесь в ситуации, где подозреваете утечку BSTR, к счастью, есть способ отключить кэширование. Перед запуском приложения установите переменную окружения OANOCACHE=1, и кэширование будет отключено. Если вы анализируете службу (запущенную не из конкретной командной оболочки), переменную окружения можно задать в глобальной системной таблице окружения.

LeakDiag

LeakDiag позволяет отслеживать многочисленные выделения, поступающие из источников, отличных от диспетчера кучи Windows по умолчанию. Например, если приложение напрямую вызывает API VirtualAlloc и забывает освободить память, UMDH об этом не сообщит; однако LeakDiag покажет такую утечку. Кроме того, LeakDiag не требует включать запись трасс стеков через gflags. Вместо этого LeakDiag использует технологию Microsoft Detours для перехвата вызовов указанных аллокаторов памяти.

LeakDiag можно запускать в двух разных режимах. Первый режим - через командную строку, второй - через UI. Здесь будет показан первый.

Запуск LeakDiag - двухшаговый процесс:

1. Выбор целевого процесса. Это просто сообщает LeakDiag, для какого процесса он должен перехватывать выделения памяти, а также какой аллокатор нужно перехватывать:

```
ldcmd.exe /p <processed> /start /a 2
```

Параметр /a выбирает конкретный интересующий вас аллокатор. В предыдущем примере 2 относится к NT Heap Allocator. Начиная с версии 1.25 поддерживаются следующие аллокаторы:

- Virtual Allocator (VirtualAlloc)
- NT Heap Allocator (HeapAlloc) [DEFAULT]
- MPHeap Allocator (MPHeap)
- COM Allocator (CoTask)
- COM Private Allocator (PrivateMemAlloc)
- C Runtime Allocator (msvcrt new)

1. Создание файлов журнала. Всякий раз, когда нужно создать файл журнала для выбранного целевого процесса, используйте переключатель /dump. Например:

```
ldcmd.exe /p <processed> /dump /a 2
```

Предыдущий пример создает файл журнала и сохраняет его в папку журналов по умолчанию. Папка журналов по умолчанию - это папка Logs в пути установки LeakDiag.

Вернемся к нашему приложению 09basicleak.exe и используем LeakDiag, чтобы найти ту же утечку памяти, которую видели ранее. Запустите приложение 09basicleak.exe следующей

командной строкой:

```
C:\AWDBIN\WinXP.x86.chk\09basicmleak.exe /t:10 /i:200 /s:0
```

Затем найдите идентификатор процесса только что запущенного экземпляра 09basicmleak.exe и выполните следующую команду:

```
C:\LeakDiag\Logs>c:\LeakDiag\ldcmd.exe /p 3028 /start /a 2
Sent Start Tracing command for pID 3832
Allocator 1: TRACING OFF
Allocator 2: TRACING ON
Allocator 3: TRACING OFF
Allocator 4: TRACING OFF
Allocator 5: TRACING OFF
Allocator 6: TRACING OFF
```

Не забывайте указывать идентификатор процесса, соответствующий вашему запуску. Команда /start отправляет процессу сигнал начать перехват вызовов выделения, а /a 2 говорит перехватывать все выделения из аллокатора кучи.

Следующий шаг состоит в выгрузке всех трасс стеков выделений. Перед выполнением команды dump с LeakDiag необходимо убедиться, что путь к символам задан правильно. В отличие от UMDH, LeakDiag не учитывает переменную окружения _NT_SYMBOL_PATH; вместо этого он опирается на значение реестра, хранящееся в следующем ключе:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\LeakDiag
```

Значение реестра называется SymPath, и его нужно установить в каталог, содержащий символы. После задания пути к символам продолжите выполнение 09basicmleak.exe и перед выходом введите следующую команду:

```
C:\LeakDiag\Logs>c:\LeakDiag\ldcmd.exe /p 3028 /dump /a 2
Sent Dump Log command for pID 3832
Allocator 1: TRACING OFF
Allocator 2: TRACING ON
Allocator 3: TRACING OFF
Allocator 4: TRACING OFF
Allocator 5: TRACING OFF
Allocator 6: TRACING OFF
```

На этот раз мы использовали переключатель /dump, чтобы сказать LeakDiag создать файл журнала всех выделений, собранных в процессе. Фактическое имя файла журнала представляет собой объединение различных атрибутов файла (например имени файла, даты запуска и т. д.). Если в каталоге много файлов журнала, лучший способ найти нужный - просто посмотреть на дату и время файла. В этом конкретном запуске имя файла, соответствующее запуску, было таким:

```
05/31/2005 03:54 PM 12,631 09basicmleak_2296_WindowsHeapAllocator_050531-155419_sess_01.xml
```

Как видно по расширению xml, файл журнала хранится в формате XML. Хороший способ просмотреть XML-файл - загрузить его в Internet Explorer. Вы также увидите, что файл журнала поставляется с соответствующей схемой и имеет довольно большой размер. Вместо того чтобы приводить здесь весь файл журнала, мы сосредоточимся только на самых важных частях, а именно на том, как представлены трассы стеков. Если вы хотите увидеть весь файл журнала, его можно найти в следующем расположении:

```
C:\AWDBIN\WinXP.x86.chk\09basicmleak_2296_WindowsHeapAllocator_050531-155419_sess_01.xml
```

Общая структура файла журнала выглядит примерно так:

```
<?xml version="1.0" ?>
<logdata>
<xs:schema id="logdata" xmlns:xs="http://www.w3.org/2001/XMLSchema"
xmlns:msdata="urn:schemas-microsoft-com:xml-msdata"> </xs:schema>
<LEAKS ver="1.31.03.0915"> </LEAKS>
```

```
<SUMMARY_INFO> </SUMMARY_INFO>
</logdata>
```

В то время как раздел схемы подробно описывает структуру XML-данных, раздел LEAKS описывает историю выделений во время запуска приложения, а итоговый раздел показывает такую информацию, как настройки LeakDiag, загруженные модули, общая статистика памяти и т. д. Самый интересный раздел - LEAKS. Раскрытие раздела LEAKS показывает ряд разделов STACK, каждый из которых подробно описывает выделения, сделанные на протяжении жизни приложения. Если посмотреть на первую трассу стека, получим:

```
<STACK numallocs="0710" size="028" totalsize="019880" totalalloccount="0710"
  totalallocsize="019880">
  <STACKSTATS>
  <SIZESTAT size="028" numallocs="0710" type="N/A" />
  <HEAPSTAT handle="80000" numallocs="0710" />
  </STACKSTATS>
  <FRAME num="0" dll="09basicleak.exe" function="CServer__GetSID" offset="0xC8"
  filename="c:\zone\pwd\cd\code\resleak\memleak\scenario1\server\srv.cpp" line="42"
  addr="0x1001C38" />
  <FRAME num="1" dll="09basicleak.exe" function="ThreadWorker" offset="0x6B"
  filename="c:\zone\pwd\cd\code\resleak\memleak\scenario1\client\client.cpp" line="36"
  addr="0x100142B" />
  <FRAME num="2" dll="kernel32.dll" function="BaseThreadStart" offset="0x37"
  filename="" line="" addr="0x7C80B50B" />
  <STACKID>008E5C88</STACKID>
</STACK>
```

Хотя файл журнала представлен в XML, он дает результаты, очень похожие на журналы UMDH. Атрибуты элемента STACK дают такую информацию, как количество выделений из трассы стека, размер каждого выделения и, наконец, общий размер. HEAPSTAT показывает, в какой куче было сделано выделение. Последняя часть - список фреймов, составляющих трассу стека. Как видно, самый нижний фрейм - функция BaseThreadStart из kernel32, вызывающая функцию ThreadWorker, которая вызывает серверную функцию GetSID, забывающую освободить выделенную память. Хотя это по существу та же утечка, которую мы обнаружили с помощью UMDH, должно быть ясно, что LeakDiag может пригодиться при работе с утечками, которые происходят не из диспетчера кучи по умолчанию.

Команда расширения !address

Команда расширения !address очень удобна, когда нужно быстро понять, где в действительности находится память в вашем процессе. Команда выдает статистику, такую как использование областей памяти кучами, стеком, свободной памятью и т. д. Чтобы увидеть это самостоятельно, запустите notepad.exe под отладчиком и выполните команду !address. Первая часть вывода дает более подробный взгляд на использование памяти, а ближе к концу вывода вы увидите сводку.

```
...
...
----- Usage SUMMARY -----
TotSize Pct(Tots) Pct(Busy) Usage
001d4000 : 0.09% 10.59% : RegionUsageIsVAD
7eeab000 : 99.16% 0.00% : RegionUsageFree
00e0d000 : 0.69% 81.36% : RegionUsageImage
00040000 : 0.01% 1.45% : RegionUsageStack
00001000 : 0.00% 0.02% : RegionUsageTeb
00120000 : 0.05% 6.51% : RegionUsageHeap
00000000 : 0.00% 0.00% : RegionUsagePageHeap
00001000 : 0.00% 0.02% : RegionUsagePeb
00001000 : 0.00% 0.02% : RegionUsageProcessParameters
00001000 : 0.00% 0.02% : RegionUsageEnvironmentBlock
Tot: 7fff0000 Busy: 01145000
...
```

```
...
Largest free region: Base 01014000 - Size 71fec000
```

Это может оказаться весьма полезным, если вы пытаетесь понять, какой инструмент использовать для дальнейшего поиска утечки. Например, если вы видите большой рост использования памяти, связанный с утечкой, но не видите существенного роста при просмотре `RegionUsageHeap`, весьма вероятно, что выделения происходят из активности памяти, не связанной с кучей (например из вызовов `VirtualAlloc`). Это экономит драгоценное время, которое иначе было бы потрачено на запуск UMDH (он отслеживает только выделения кучи), и позволяет сосредоточиться на более подходящем инструменте, таком как LeakDiag.

Наш пример - простой сервер для иллюстрации, но представьте чрезвычайно сложный сервер, который весь день нагружался клиентскими запросами и теряет память. С чего начать поиск без инструментов? Во многих случаях UMDH или LeakDiag могут быть ответом в таких ситуациях. Но постойте, скажете вы: UMDH и LeakDiag предполагают, что у нас есть доступ к системе и мы можем запустить эти инструменты. А как быть в ситуациях, когда вы просто получаете дампы памяти процесса с утечкой и должны проанализировать утечку *postmortem*? В этом случае runtime-инструменты обнаружения утечек не подходят. К счастью, в отладчике есть несколько мощных команд, позволяющих выполнять довольно впечатляющий анализ утечек.

Команда расширения !heap

Команда расширения `!heap` входит в расширение отладчика `exts.dll` и является чрезвычайно мощной командой, позволяющей пользователям глубоко изучать потребление памяти процессом. Например, команда расширения `!heap` способна искать в адресном пространстве утекшие блоки, выполнять пользовательские поиски по всем кучам, выдавать подробные трассы стеков выделений, устанавливать точки останова в менеджере кучи и многое другое. В этом разделе мы используем измененную версию приложения `09basicmleak.exe`, применявшегося в предыдущем разделе. Клиентский код почти идентичен, за исключением возвращаемого типа. Вместо возврата сырой структуры `SID` сервер возвращает указатель на экземпляр класса `CIdentity`. Экземпляр класса `CIdentity` просто оборачивает структуру `SID` в более удобную для программиста форму.

```
class CIdentity
{
public:
    virtual BOOL GetUsername(WCHAR** pUserName) { return FALSE; }
    virtual BOOL GetDomain(WCHAR** pUserName) { return FALSE; }
protected:
    CIdentity(PVOID pIdentityBlob):m_pIdentityBlob(pIdentityBlob){};
    virtual ~CIdentity(){};
    PVOID GetBlob() { return m_pIdentityBlob; }
    PVOID m_pIdentityBlob;
};
```

Общая идея состоит в том, чтобы класс `CIdentity` хранил сырые данные, представляющие идентичность, и предоставлял набор виртуальных функций, которые могут интерпретировать эти данные. Например, виртуальная функция `GetUserName` возвращает имя пользователя идентичности. Когда появляется новая идентичность, от класса `CIdentity` нужно наследовать подкласс и переопределить соответствующие функции. Главная мысль здесь в том, что клиент всегда работает с экземплярами класса `CIdentity`, тем самым абстрагируясь от конкретики той базовой идентичности, которая используется в данный момент. Это прекрасный пример широко используемой техники, называемой полиморфизмом: один интерфейс, несколько реализаций. В данном конкретном случае у нас есть класс `CSID`, наследующийся от `CIdentity` для представления обычного идентификатора безопасности, используемого в Windows. Ради простоты класс `CSID` полагается на реализацию функций по умолчанию (возвращающую `FALSE`).

Клиентский код немного изменился, чтобы работать с экземплярами CIdentity вместо ранее использовавшихся сырых структур SID.

Как вы уже, вероятно, догадались, при запуске приложения сообщается об утечке памяти. Посмотрим, как можно использовать команду расширения !heap для анализа проблемы.

Статистика кучи

Очень полезный прием при изучении утечек ресурсов - всегда получить хорошее представление об общем потреблении памяти процессом, который течет. Сюда входят такие сведения, как объем потребляемой памяти, а также информация о том, какой куче принадлежит эта память. Команда расширения !heap позволяет подробно посмотреть сводку куч процесса.

Сразу погрузимся в наше текущее приложение.

Исходный код и двоичный файл приложения находятся в следующих папках:

```
Source code: C:\AWD\Chapter9\MemLeak\Client and
C:\AWD\Chapter9\MemLeak\Server
Binary: C:\AWDBIN\WinXP.x86.chk\09memleak.exe
```

Запустите клиентское приложение следующей командой:

```
C:\AWDBIN\WinXP.x86.chk\09memleak.exe /t:64 /i:1000 /s:0
```

После завершения выполнения присоедините отладчик к процессу и используйте команду расширения !heap, чтобы вывести сводку всех куч процесса.

```
0:001> !heap -s
Heap Flags Reserv Commit Virt Free List UCR Virt Lock Fast
(k) (k) (k) (k) length blocks cont. heap
-----
00090000 00000002 1024 20 20 3 1 1 0 0 L
00190000 00001002 64 24 24 15 1 1 0 0 L
001a0000 00008000 64 12 12 10 1 1 0 0
00030000 00001002 3136 1232 1232 8 3 1 0 0 L
-----
```

Переключатель -s предоставляет базовую информацию о каждой куче процесса. Наиболее важные данные применительно к утечкам ресурсов таковы:

- Heap: адрес кучи.
- Flags: флаги, связанные с каждой кучей. Позже мы покажем гораздо более читаемый способ идентифицировать флаги.
- Reserv (k): объем памяти, зарезервированный для данной кучи.
- Commit (k): объем памяти, закоммиченный для данной кучи.
- Virt (k): объем виртуальной памяти для данной кучи.

Обзор куч всегда является хорошей отправной точкой при поиске утечек памяти, потому что он разбивает активность по каждой куче процесса. Из всех куч больше всего памяти использует куча с идентификатором 00030000. Точнее, объем зарезервированной памяти для этой кучи составляет 3136 КБ, а объем закоммиченной памяти - 1232 КБ. Столкнувшись с этой информацией, мы начнем расследование именно с кучи 00030000. Хотя такой обзор позволяет нацелить поиск утечки, он не дает более конкретной информации по куче. Например, было бы действительно полезно получить список всех выделений конкретной кучи. К счастью, команда расширения !heap позволяет получить такую информацию. Та же команда, но с указанием конкретного адреса кучи, дает нужные результаты.

```
0:001> !heap -s 00030000
Walking the heap 00030000 ...
0: Heap 00030000
```

```

Flags 00001002 - HEAP_GROWABLE
Reserved 3136 (k)
Committed 1232 (k)
Virtual bytes 1232 (k)
Free space 8 (k)
External fragmentation 0% (3 free blocks)
Virtual address fragmentation 0% (1 uncommitted ranges)
Virtual blocks 0
Lock contention 0
Segments 3
Lookaside heap 00030688
Default heap Front heap Unused bytes
Range (bytes) Busy Free Busy Free Total Average
-----
0 - 1024 43604 1 0 0 438533 10
1024 - 2048 2 0 0 0 8 4
2048 - 3072 1 3 0 0 8 8
4096 - 5120 1 0 0 0 8 8
6144 - 7168 1 0 0 0 8 8
-----
Total 43609 4 0 0 438565 10

```

Дополнительная информация включает человекочитаемые флаги кучи, сведения о фрагментации кучи и количество неиспользованных байтов.

Это дает нам немного больше информации, но, конечно, все еще недостаточно, чтобы понять, что может течь в этой куче. Если бы мы могли использовать команду расширения !heap для получения еще более подробной информации, например сведений о каждом выделении, сделанном в куче, мы могли бы приблизиться к отслеживанию утечки. Команда расширения !heap действительно предоставляет такую функциональность через переключатель -a. Будьте предупреждены: переключатель -a выполняет исчерпывающий дамп рассматриваемой кучи. Обычно это может занять несколько секунд или даже минут, и в итоге у вас может оказаться столько информации, что не хватит буфера консоли для ее хранения. Как правило, лучше всего открыть файл журнала командой .logopen <filename>, выполнить команду расширения !heap и затем закрыть журнал командой .logclose. После этого можно просто открыть файл журнала и продолжить анализ. В нашем примере файл журнала находится здесь:

```
C:\AWDBIN\WinXP.x86.chk\heaplog.txt
```

Файл журнала разделен на две секции:

- Общая информация об указанной куче.
- Список одного или нескольких сегментов с базовой информацией, за которой следуют все блоки кучи, видимые в сегменте. Перечисленные блоки кучи могут использоваться, а могут и не использоваться, как вы увидите далее.

Первая часть, общая информация о куче, является надмножеством данных, получаемых командой расширения !heap -s, как объяснялось ранее. Самая важная часть файла журнала - вторая: подробная информация о сегментах. После начального обзора сегмента отображается длинный список сведений о блоках кучи. Каждая строка вывода организована следующим образом:

```

<heap block address>: <previous size> . <size> [<Flags>] - <status> (user allocation size),
<debug flags>

```

- Heap block address: адрес блока кучи. Обратите внимание, что адрес блока кучи не совпадает с фактическим адресом указателя пользовательского режима. Оказывается, адрес, показываемый командой расширения !heap, - это адрес самого блока, а не содержимого (то есть пользовательского выделения) выделения. Первые 8 байт структуры блока содержат метаданные блока кучи (такие как размер и флаги), хранимые Windows для управления блоком кучи. После этой информации находятся фактические данные, которые нас интересуют. Если бы мы хотели вывести содержимое пользовательских данных, находящихся в этом блоке, нужно было бы добавить 8 байт к адресу блока.

- **Previous size:** размер предыдущего блока кучи. Размер указывается в единицах гранулярности выделения, а не в размере пользовательских данных.
- **Size:** размер выделенного блока. Важно отметить, что указанный размер не совпадает с размером, который пользователь указал при выделении. Причина проста: менеджер кучи выделяет память на основе размеров гранулярности выделения.
- **Flags:** состояние блока кучи. Примеры состояния - свободный блок кучи и занятый блок кучи.
- **Status:** поле состояния сообщает, свободен блок или занят. Когда блок занят, выделение активно; когда он свободен, он доступен для использования. Когда речь идет об утечках памяти, нас обычно интересуют только занятые выделения.
- **User allocation size:** возможно, самая полезная часть данных применительно к утечкам памяти. Она сообщает размер пользовательского выделения, ставшего причиной выделения. С этой информацией мы можем сопоставить размер с различными выделениями, которые делаем в приложении, и посмотреть, есть ли совпадения.
- **Debug Flags:** флаги блока кучи сообщают, какой тип поддержки отладки кучи включен. Например, **tail fill** говорит, что конец блока кучи заполнен хорошо известным шаблоном.

Имея эту информацию, как на самом деле искать утечку? Ключевое слово - терпение. Обычно применяемая стратегия состоит в том, чтобы найти шаблон в перечисленных блоках. Чаще всего вы пытаетесь найти большое количество блоков с одинаковым размером пользовательского выделения. Обычно это хороший индикатор того, что они потенциально являются утечками блоками. В нашем файле журнала, спустя несколько страниц после первого списка сегмента (segment 00), мы видим следующее:

```
0003a4f0: 00028 . 00010 [01] - busy (8)
0003a500: 00010 . 00028 [01] - busy (1c)
0003a528: 00028 . 00010 [01] - busy (8)
0003a538: 00010 . 00028 [01] - busy (1c)
0003a560: 00028 . 00010 [01] - busy (8)
0003a570: 00010 . 00028 [01] - busy (1c)
0003a598: 00028 . 00010 [01] - busy (8)
0003a5a8: 00010 . 00028 [01] - busy (1c)
0003a5d0: 00028 . 00010 [01] - busy (8)
0003a5e0: 00010 . 00028 [01] - busy (1c)
0003a608: 00028 . 00010 [01] - busy (8)
0003a618: 00010 . 00028 [01] - busy (1c)
0003a640: 00028 . 00010 [01] - busy (8)
0003a650: 00010 . 00028 [01] - busy (1c)
0003a678: 00028 . 00010 [01] - busy (8)
0003a688: 00010 . 00028 [01] - busy (1c)
0003a6b0: 00028 . 00010 [01] - busy (8)
0003a6c0: 00010 . 00028 [01] - busy (1c)
0003a6e8: 00028 . 00010 [01] - busy (8)
0003a6f8: 00010 . 00028 [01] - busy (1c)
0003a720: 00028 . 00010 [01] - busy (8)
0003a730: 00010 . 00028 [01] - busy (1c)
0003a758: 00028 . 00010 [01] - busy (8)
0003a768: 00010 . 00028 [01] - busy (1c)
0003a790: 00028 . 00010 [01] - busy (8)
0003a7a0: 00010 . 00028 [01] - busy (1c)
0003a7c8: 00028 . 00010 [01] - busy (8)
0003a7d8: 00010 . 00028 [01] - busy (1c)
0003a800: 00028 . 00010 [01] - busy (8)
0003a810: 00010 . 00028 [01] - busy (1c)
0003a838: 00028 . 00010 [01] - busy (8)
0003a848: 00010 . 00028 [01] - busy (1c)
```

Похоже, выделены массы блоков пользовательских размеров 8 и 1с. Фактически выборочная проверка случайных блоков в файле журнала дает довольно большое количество таких выделенных блоков. Учитывая, что выполнение завершено и приложение вот-вот завершится, велика вероятность, что мы обнаружили утечку памяти.

На этом этапе мы на полпути. Следующий шаг - выяснить, что эти блоки на самом деле содержат. Если бы мы теряли память, было бы разумно ожидать, что внутри этих блоков находятся данные, связанные с нашим приложением. Сложная и иногда долгая часть - понять, что именно содержат блоки. Посмотрим память одного из этих блоков.

```
0:001> dd 0003a7c8+0x8
0003a7d0 010012bc 0003a7a8 00020005 000c01e4
0003a7e0 00000501 05000000 00000015 42f831d9
0003a7f0 125f5219 2b3be507 000003ec 00000000
0003a800 00050002 0008011f 010012bc 0003a7e0
0003a810 00020005 000c011d 00000501 05000000
0003a820 00000015 42f831d9 125f5219 2b3be507
0003a830 000003ec 00000000 00050002 00080118
0003a840 010012bc 0003a818 00020005 000c0116
```

Похожи ли первые два DWORD на что-нибудь? С этого момента все сводится к попытке распознать в данных что-то, применимое к вашему приложению. Попробуйте использовать разные варианты команд дампа, чтобы найти что-нибудь осмысленное. Например, команды `da` или `du` позволяют вывести конкретный указатель как строку. Если это не сработает, можно попробовать разрешить содержимое выделения. Например, применив команду `ln` к первому DWORD, вы получите следующий результат:

```
0:001> ln 010012bc
(010012bc) 09memleak!CSID::`vftable' | (010012c8)
09memleak!CIdentity::`vftable'
Exact matches:
```

Это уже слишком хорошо, чтобы быть совпадением. Наше тестовое приложение определенно работает с классами типа `CIdentity`. И, как мы уже знаем, `CSID` - класс, производный от `CIdentity`. Поскольку таблицы виртуальных функций обычно идут первыми в раскладке объекта, мы можем выдвинуть гипотезу:

- Судя по шаблону выделения в выводе команды расширения `!heap`, велика вероятность, что один из этих блоков кучи утек.
- Более того, просматривая содержимое блока кучи вокруг, мы видим, что оно содержит таблицы виртуальных функций объектов, с которыми мы работаем.

Иногда попытка распознать содержимое утекших блоков кучи может быть пугающей задачей. К счастью, после некоторого опыта анализа утечек памяти вы научитесь распознавать определенные категории данных просто с помощью команды `dd`.

Поиск в куче

Прежде чем сделать вывод, что это действительно утечка (помните: кэширование может заставлять объекты оставаться рядом даже после завершения их использования), нужно проверить теорию. Если эти потенциально утекшие блоки использовались бы (возможно, кэшировались), где-то в памяти также должна была бы быть ссылка, указывающая на этот блок кучи. Если ссылок нет, это означает, что у нас определенно есть утечка. И снова команда расширения `!heap` предоставляет средство это выяснить. С помощью переключателей `-x` и `-v` мы можем попросить команду расширения `!heap` просканировать все пространство памяти процесса на наличие заданного адреса. В нашем примере поиск адреса `0003a7d0` (помните: адрес блока + `0x8` дает пользовательское выделение) дает следующее:

```
0:001> !heap -x -v 0003a7d0
Entry User Heap Segment Size PrevSize Unused Flags
-----
0003a7c8 0003a7d0 00030000 00030640 10 28 8 busy
Search VM for address range 0003a7c8 - 0003a7d7 :
```


Поиск дал нулевой результат. Как говорилось ранее, если на текущий выделенный блок кучи нигде в памяти нет ссылки, мы можем уверенно сказать, что этот блок утекает. Поскольку мы знаем (по анализу кода), что работаем с экземпляром класса CIdentity (CSID наследуется от этого класса), теперь можно перейти к ревью соответствующих участков кода. Начиная с клиентского кода, видно, что функция ThreadWorker использует класс CIdentity.

```
if(dwOperation==GETSID)
{
    CIdentity* pSid=serverInst.GetSID();
    if(pSid==NULL)
    {
        printf("Failed to get SID!\n");
    }
    else
    {
        printf(".");
    }
}
```

Клиент вызывает серверную функцию GetSID, которая возвращает экземпляр класса CIdentity. Поскольку в клиентском коде мы не выделяли место, сервер, должно быть, отвечал за выделение и, в случае успеха, передал его обратно клиенту. Но кто отвечает за удаление? В данном случае ответ таков: это ответственность клиента. Мы также довольно быстро можем сказать, что код не справился с этой ответственностью и не удаляет память. Исправление простое: если сервер успешно отработал и передал обратно экземпляр, мы добавляем соответствующий вызов delete, когда закончили работу с экземпляром.

После внесения исправления, повторного запуска приложения и прохождения той же процедуры с командой расширения !heap, мы замечаем, что все блоки кучи с размером пользовательского выделения 0x8 теперь исчезли. Интересно, что все сообщенные утекшие блоки размера 0x8 исчезли, но выделения размера 0x1c все еще остаются. Пора внимательнее посмотреть на первую утечку, которую мы выявили и исправили, - класс CIdentity:

```
class CIdentity
{
public:
    virtual BOOL GetUsername(WCHAR** pUserName) { return FALSE; }
    virtual BOOL GetDomain(WCHAR** pUserName) { return FALSE; }
protected:
    CIdentity(PVOID pIdentityBlob):m_pIdentityBlob(pIdentityBlob){};
    virtual ~CIdentity(){};
    PVOID GetBlob() { return m_pIdentityBlob; }
    PVOID m_pIdentityBlob;
};
```

Если экземпляр этого класса выделен, обычная раскладка выделения будет содержать указатель на таблицу виртуальных функций (поскольку класс содержит виртуальные функции) и все члены-данные. Единственный член-данные в этом классе - указатель на VOID (pIdentityBlob). Поскольку оба члена являются указателями (по 4 байта каждый на 32-битных машинах), общий размер объекта должен быть 0x8. Это совпадает с утекшими блоками пользовательского размера 0x8, которые мы видели, но что насчет утекших блоков размера 0x1c? Ответ довольно прост. Мы уже определили, что теряли экземпляры конкретного класса. Следовательно, если вы теряете экземпляр класса, это означает, что деструктор никогда не будет вызван. Довольно распространенная практика - удалять любые инкапсулированные данные в деструкторе класса. Поэтому, если вы теряете экземпляр, вы также теряете любые данные, содержащиеся внутри этого класса. Единственный член-данные в классе CIdentity - это PVOID, который, как мы все знаем, не является чем-то, что можно удалить напрямую. Эти наблюдения, в сочетании с наличием виртуальных функций, подразумевают, что может быть задействован производный класс. Посмотрим на серверную реализацию GetSID:

```

CIdentity* CServer::GetSID()
{
    PSID pSid = NULL;
    HANDLE hToken = INVALID_HANDLE_VALUE;
    hToken = GetToken();
    if(hToken!=INVALID_HANDLE_VALUE)
    {
        DWORD dwNeeded=0;
        BOOL bRes=GetTokenInformation(hToken,
            TokenUser,
            NULL,
            0,
            &dwNeeded
        );
        if(bRes==FALSE && GetLastError()==ERROR_INSUFFICIENT_BUFFER)
        {
            TOKEN_USER* pBuffer=(TOKEN_USER*) new BYTE[dwNeeded];
            if(pBuffer!=NULL)
            {
                BOOL bRes=GetTokenInformation(hToken,
                    TokenUser,
                    (LPVOID)pBuffer,
                    dwNeeded,
                    &dwNeeded
                );
                if(bRes==TRUE)
                {
                    DWORD dwSidLen=GetLengthSid(pBuffer->User.Sid);
                    pSid=(PSID) new BYTE[dwSidLen];
                    if(pSid!=NULL)
                    {
                        if(CopySid(dwSidLen, pSid, pBuffer->User.Sid)==FALSE)
                        {
                            delete[] pSid;
                            pSid=NULL;
                        }
                    }
                }
                delete[] pBuffer;
            }
        }
        CloseHandle(hToken);
    }
    CSID* pIdentity=NULL ;
    if(pSid!=NULL)
    {
        pIdentity=new CSID(pSid);
        if(pIdentity==NULL)
        {
            delete pSid;
        }
    }
    return (CIdentity*) pIdentity;
}

```

Приведенный код поразительно похож на функцию GetSID, использованную ранее в этой главе. Высокоуровневый обзор показывает, что сервер пытается получить токен вызывающего (потока или процесса) и извлечь SID из токена. В рамках извлечения этого SID он выделяет память для хранения SID (локальная переменная pSid). В конце функции серверный код выделяет экземпляр класса CSID (который наследуется от CIdentity) и передает этот указатель SID в конструктор класса. Затем конструктор передает владение этой выделенной памятью классу (сохраняет указатель в члене-данных PVOID класса CIdentity). Логично, что теперь именно класс отвечает за освобождение памяти, связанной с указателем PVOID, но если посмотреть код класса CSID, он не освобождает выделенную память. Исправление в этом сценарии - добавить в деструктор класса CSID код, освобождающий память, за которую он только что взял ответственность.

Теперь, если мы снова запускаем приложение и проходим тот же процесс с командой расширения !heap, видно, что все ранее утекшие выделения теперь корректно удаляются и не показываются как занятые выделения в выводе.

Всякий раз, когда вы работаете с утекшими экземплярами какого-либо инкапсулирующего конструктора (например класса), также необходимо внимательно посмотреть на сам класс и убедиться, что он освобождает ресурс, который приобрел. Забыть освободить все ресурсы, инкапсулированные внутри конкретного класса, - довольно распространенная программная ошибка.

Обнаружение утечек

Дамп всех блоков кучи и систематический поиск потенциально утекших блоков с помощью возможностей поиска требуют усилий и могут быть очень дорогими. К счастью, команда расширения !heap объединяет эти шаги в один с помощью переключателя -l. Переключатель -l говорит команде расширения !heap использовать алгоритм сборки мусора для обнаружения всех активных выделений, на которые нигде в процессе нет ссылок. Следующий отладочный вывод показывает запуск команды расширения !heap -l на нашем текущем приложении (частичный вывод).

```
0:001> !heap -l
Heap 00090000
Heap 00190000
Heap 001a0000
Heap 00030000
Scanning VM ...
Entry User Heap Segment Size PrevSize Unused Flags
-----
...
...
...
012904e8 012904f0 00030000 01280000 28 10 c busy
01290510 01290518 00030000 01280000 10 28 8 busy
01290520 01290528 00030000 01280000 28 10 c busy
01290548 01290550 00030000 01280000 10 28 8 busy
01290558 01290560 00030000 01280000 28 10 c busy
01290580 01290588 00030000 01280000 10 28 8 busy
01290590 01290598 00030000 01280000 28 10 c busy
012905b8 012905c0 00030000 01280000 10 28 8 busy
012905c8 012905d0 00030000 01280000 28 10 c busy
012905f0 012905f8 00030000 01280000 10 28 8 busy
01290600 01290608 00030000 01280000 28 10 c busy
01290628 01290630 00030000 01280000 10 28 8 busy
01290638 01290640 00030000 01280000 28 10 c busy
01290660 01290668 00030000 01280000 10 28 8 busy
01290670 01290678 00030000 01280000 28 10 c busy
01290698 012906a0 00030000 01280000 10 28 8 busy
012906a8 012906b0 00030000 01280000 28 10 c busy
012906d0 012906d8 00030000 01280000 10 28 8 busy
012906e0 012906e8 00030000 01280000 28 10 c busy
01290708 01290710 00030000 01280000 10 28 8 busy
01290718 01290720 00030000 01280000 28 10 c busy
...
...
...
42710 leaks detected.
```

Результаты команды расширения !heap показывают огромное количество выделений с размерами блоков 28 и 10. (Обратите внимание, что это размеры блоков кучи, а не размеры пользовательских выделений.) Кроме того, последняя строка вывода сообщает, сколько утечек было обнаружено. В данном случае найдено 42710 утекших блоков. Это чрезвычайно полезная возможность команды расширения !heap, потому что она устраняет необходимость выполнять большой объем ручного поиска.

Pageheap

Предыдущий пример показал утечку, которую было довольно легко заметить, анализируя состояние кучи и выполняя ревью кода. Иногда не очевидно, что именно течет. В таких случаях, после того как вы определили потенциального виновника утечки, было бы полезно увидеть, какая трасса стека изначально сделала выделение. Если бы она у нас была, мы могли бы точно узнать, что делал код и что он выделял. Команда расширения !heap может работать совместно с возможностями Windows по записи трасс стеков. Чтобы использовать эту возможность, обязательно включите трассировку стеков с помощью Application Verifier (см. главу 1). Применимые переключатели команды расширения !heap - -p и -a. -p сообщает команде расширения !heap, что запрашивается информация pageheap, а переключатель -a позволяет указать адрес, для которого нужно увидеть трассу стека. В предыдущем разделе адресом, который мы считали текущим, был 0026ab88. Выполните следующую команду, и вы увидите исходную трассу стека для этого выделения:

```
0:001> !heap -p -a 0003a7c8
address 0003a7c8 found in
_HEAP @ 30000
in HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
3a7a8: 0007 : N/A [N/A] - 3a7b0 (1c) - (busy)
Trace: 003c
7c96d6dc ntdll!RtlDebugAllocateHeap+0x000000e1
7c949d18 ntdll!RtlAllocateHeapSlowly+0x00000044
7c91b298 ntdll!RtlAllocateHeap+0x000000e4
77c2c3c9 msvcrt!_heap_alloc+0x000000e0
77c2c3e7 msvcrt!_nh_malloc+0x00000013
77c29cd4 msvcrt!operator new+0x0000000f
1001c52 09memleak!CServer::GetSID+0x000000d2
100143b 09memleak!ThreadWorker+0x0000006b
7c80b683 kernel32!BaseThreadStart+0x00000037
```

Мы видим не только общую информацию об утекшем адресе (например, в какой куче он находится и идентификатор трассы), но и полную трассу стека кода, сделавшего выделение. С этого места ревью кода и поиск виновного участка становятся тривиальным упражнением. Само собой разумеется, включение трассировки стеков - вариант -p -a команды расширения !heap - экономит невероятное количество времени.

Другие приемы команды расширения кучи

Если посмотреть справку по команде расширения !heap (набрав !heap -?), вы заметите некоторые команды, перечисленные там, но не документированные в документации отладчика. Точнее, следующие команды позволяют выполнять полезную фильтрацию и поиск по куче. Начнем с команды фильтрации. Команда фильтрации позволяет сообщить отладчику, что вас интересуют только выделения, соответствующие конкретному размеру (или диапазону). Синтаксис команды таков:

```
!heap -flt s SIZE
```

где SIZE - интересующий вас размер. Если же вы не знаете точный размер, а знаете диапазон, можно использовать следующий синтаксис:

```
!heap -flt r SIZEBEGIN SIZEEND
```

где SIZEBEGIN - начальный размер, а SIZEEND - конечный размер диапазона.

И снова используем наше текущее 09memleak.exe. Из предыдущего расследования мы знаем, что размеры утекших блоков - 0x8 и 0x1c. Запустите приложение 09memleak.exe следующей командой:

```
C:\AWDBIN\WinXP.x86.chk\09memleak.exe /t:64 /i:1000 /s:0
```

После завершения выполнения присоедините отладчик к процессу и выполните команду расширения !heap.

```
0:001> !heap -flt s 0x1c
_HEAP @ 90000
_HEAP @ 190000
_HEAP @ 1a0000
_HEAP @ 30000
HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
33768: 0007 : N/A [N/A] - 33770 (1c) - (busy)
35ae0: 0008 : N/A [N/A] - 35ae8 (1c) - (busy)
37ec8: 0007 : N/A [N/A] - 37ed0 (1c) - (busy)
37f40: 0007 : N/A [N/A] - 37f48 (1c) - (busy)
37f78: 0007 : N/A [N/A] - 37f80 (1c) - (busy)
37ff0: 0007 : N/A [N/A] - 37ff8 (1c) - (busy)
38028: 0007 : N/A [N/A] - 38030 (1c) - (busy)
380a0: 0007 : N/A [N/A] - 380a8 (1c) - (busy)
380d8: 0007 : N/A [N/A] - 380e0 (1c) - (busy)
38150: 0007 : N/A [N/A] - 38158 (1c) - (busy)
38188: 0007 : N/A [N/A] - 38190 (1c) - (busy)
38200: 0007 : N/A [N/A] - 38208 (1c) - (busy)
38238: 0007 : N/A [N/A] - 38240 (1c) - (busy)
382b0: 0007 : N/A [N/A] - 382b8 (1c) - (busy)
382e8: 0007 : N/A [N/A] - 382f0 (1c) - (busy)
38360: 0007 : N/A [N/A] - 38368 (1c) - (busy)
38398: 0007 : N/A [N/A] - 383a0 (1c) - (busy)
38410: 0007 : N/A [N/A] - 38418 (1c) - (busy)
...
...
...
```

Результат команды расширения !heap аккуратно показывает все блоки кучи размера 0x1c и связанную информацию о блоках. Состояние busy указывает, что блок сейчас используется. Хотя эта информация весьма полезна для поиска всех блоков с конкретным размером, перед нами все еще остается задача выяснить, что эти блоки на самом деле содержат. Как вы, возможно, уже догадались, команда расширения !heap приходит на помощь. В сочетании с переключателями -p и -h команда расширения !heap выгружает все блоки кучи и пытается разрешить первый DWORD. Следующий отладочный вывод показывает результат выполнения команды heap -p -h для кучи, которая предположительно течет.

```
0:001> !heap -p -h 00030000
_HEAP @ 30000
_HEAP_LOOKASIDE @ 30688
_HEAP_SEGMENT @ 30640
CommittedRange @ 30680
HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
* 30680: 0303 : N/A [N/A] - 30688 (1800) - (busy)
31e98: 0014 : N/A [N/A] - 31ea0 (88) - (busy)
31f38: 0093 : N/A [N/A] - 31f40 (480) - (busy)
323d0: 0103 : N/A [N/A] - 323d8 (800) - (busy)
msvcrt!_iob
32be8: 0007 : N/A [N/A] - 32bf0 (20) - (busy)
32c20: 000b : N/A [N/A] - 32c28 (3a) - (busy)
32c78: 000a : N/A [N/A] - 32c80 (32) - (busy)
32cc8: 0008 : N/A [N/A] - 32cd0 (26) - (busy)
32d08: 000a : N/A [N/A] - 32d10 (34) - (busy)
32d58: 000a : N/A [N/A] - 32d60 (38) - (busy)
32da8: 0009 : N/A [N/A] - 32db0 (2e) - (busy)
32df0: 000a : N/A [N/A] - 32df8 (36) - (busy)
32e40: 000b : N/A [N/A] - 32e48 (3a) - (busy)
32e98: 000b : N/A [N/A] - 32ea0 (32) - (busy)
32ef0: 0011 : N/A [N/A] - 32ef8 (70) - (busy)
32f78: 0010 : N/A [N/A] - 32f80 (62) - (busy)
```

```

32ff8: 0008 : N/A [N/A] - 33000 (28) - (busy)
33038: 0004 : N/A [N/A] - 33040 (8) - (busy)
09memleak!CSID::~`vftable'
33058: 000c : N/A [N/A] - 33060 (48) - (busy)

```

Как видно, отображается много выделений с размерами 1с и 8. Еще интереснее то, что все выделения размера 8 имеют связанную с ними дополнительную информацию. Точнее, они показывают следующее:

```

33038: 0004 : N/A [N/A] - 33040 (8) - (busy)
09memleak!CSID::~`vftable'

```

Это одна из действительно приятных возможностей использования команды расширения !heap с переключателем -p. Когда встречается блок кучи, команда расширения !heap пытается разрешить первый DWORD этого блока. В нашем случае он аккуратно разрешается в таблицу виртуальных функций CSID (как мы обнаружили ранее).

Следующая команда, которую мы рассмотрим, - -srch. Синтаксис команды выглядит так:

```

!heap -srch [-b|-w|-d|-q] PATTERN
It scans all heap allocations and it searches
for the given pattern.
The size of the pattern can be specified.

```

Команда -srch позволяет искать конкретные шаблоны во всех выделениях кучи. Это может быть действительно удобно, если у нас есть идея (или внутреннее ощущение), что именно может течь. Допустим, мы хотим проверить, не теряют ли какие-либо утекающие блоки в процессе 09memleak.exe экземпляры CSID. Первое, что нужно сделать, - узнать адрес нашей таблицы виртуальных функций. Это можно сделать с помощью команды X (см. главу 2), которая позволяет разрешить символьное имя в одном или нескольких модулях:

```

0:001> X 09memleak!CSID*
01001e20 09memleak!CSID::~`scalar deleting destructor' (void)
01001e60 09memleak!CSID::~~CSID (void)
01001d40 09memleak!CSID::~CSID (void *)
010012bc 09memleak!CSID::~`vftable' = <no type information>

```

* используется как wildcard. Таблица виртуальных функций - последняя показанная запись с адресом 010012bc. Теперь этот адрес можно использовать как часть команды -srch:

```

0:001> !heap -srch 010012bc
_HEAP @ 30000
in HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
34f18: 0002 : N/A [N/A] - 34f20 (8) - (busy)
09memleak!CSID::~`vftable'
_HEAP @ 30000
in HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
3ace0: 0002 : N/A [N/A] - 3ace8 (8) - (busy)
09memleak!CSID::~`vftable'
_HEAP @ 30000
in HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
3ad18: 0002 : N/A [N/A] - 3ad20 (8) - (busy)
09memleak!CSID::~`vftable'
_HEAP @ 30000
in HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
3ad50: 0002 : N/A [N/A] - 3ad58 (8) - (busy)
09memleak!CSID::~`vftable'
_HEAP @ 30000
in HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
3ad88: 0002 : N/A [N/A] - 3ad90 (8) - (busy)
09memleak!CSID::~`vftable'
_HEAP @ 30000
in HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state
3adc0: 0002 : N/A [N/A] - 3adc8 (8) - (busy)
09memleak!CSID::~`vftable'
_HEAP @ 30000
in HEAP_ENTRY: Size : Prev Flags - UserPtr UserSize - state

```

```
3adf8: 0002 : N/A [N/A] - 3ae00 (8) - (busy)
09memleak!CSID::`vftable'
```

Судя по чрезмерному количеству таблиц виртуальных функций CSID, оставшихся в конце запуска приложения, это хороший признак того, что что-то забыло удалить экземпляры класса CSID.

Последняя команда - `-stat` со следующим синтаксисом:

```
!heap -stat [-h HANDLE [-grp A|B|S [MaxDisplay]]]
This command calculates usage statistics on all the heaps (sorting by committed
bytes) or on the given heap.
The -grp A|B|C options specify a Group-By criteria.
-grp A groups by Allocation Size
-grp B groups by Block count
-grp S groups by Total Size for each allocation size
If HANDLE is 0, it iterates over all the heaps.
```

Команда `-stat` выдает очень полезную статистику использования одной или нескольких куч, группируя вывод по размеру выделения (пользовательскому размеру), количеству блоков с этим размером, общему размеру всех блоков с этим размером и, наконец, проценту занятых сейчас блоков. По умолчанию `-stat` сортирует по самым большим итоговым значениям. Поскольку мы знаем, какая куча, скорее всего, течет в нашем процессе (00030000), выбираем ее для дальнейшего анализа:

```
0:001> !heap -stat -h 00030000
heap @ 00030000
group-by: TOTSIZE max-display: 20
size #blocks total ( %) (percent of total busy bytes)
1c 52c2 - 90d38 (75.89)
8 52c2 - 29610 (21.68)
1000 1 - 1000 (0.52)
800 1 - 800 (0.26)
480 1 - 480 (0.15)
318 1 - 318 (0.10)
164 2 - 2c8 (0.09)
220 1 - 220 (0.07)
58 6 - 210 (0.07)
54 6 - 1f8 (0.06)
18c 1 - 18c (0.05)
62 4 - 188 (0.05)
32 7 - 15e (0.04)
2a 8 - 150 (0.04)
2c 7 - 134 (0.04)
4c 4 - 130 (0.04)
64 3 - 12c (0.04)
5e 3 - 11a (0.04)
88 2 - 110 (0.03)
5a 3 - 10e (0.03)
```

Это может быстро дать обзор того, на какие выделения нужно смотреть в первую очередь. В данном случае выделения размера 0x1c составляют 75,89% всего использования кучи. Подпереключатель `-grp` дает гибкость группировать информацию разными способами:

- `-grp A`: группирует вывод по размеру выделения, показывая самые большие выделения первыми. Верхнее выделение может быть самым большим одиночным выделением, но, скорее всего, не будет самым большим потребителем кучи.
- `-grp B`: группирует вывод по количеству блоков, показывая выделения с самым большим количеством блоков первыми. Если вы действительно смотрите на утекшее выделение, обычно главный кандидат в категории количества блоков будет соответствовать размеру блока, который у вас течет.
- `-grp S`: группирует по общему размеру. Это настройка по умолчанию.

Невозможно переоценить мощь, заложенную в команду расширения `!heap`. Она позволяет увидеть практически все, что вы хотели бы увидеть об активности кучи. В качестве бонуса возможности поиска экономят много времени при поиске виновных утекших объектов. С этим мощным

инструментом определенно стоит поэкспериментировать.

Шаг 5. Стратегии предотвращения на будущее

Умение пользоваться всеми этими мощными инструментами спасает жизнь, когда нужно отслеживать утечки памяти. Но мы хотели бы использовать их как можно реже, чтобы экономить время и нервы во время разработки. Примерно так же, как мы делали с утечками дескрипторов, сейчас самое время сесть и подумать, что мы можем сделать в будущем, чтобы не забывать удалять память, когда работа с ней завершена. И снова чрезвычайно полезный прием - использовать `auto`-конструкцию, которая автоматически удаляет память, когда переменная выходит из области видимости. Фактически эта идея считалась настолько полезной, что была включена в стандартную библиотеку шаблонов (`auto_ptr`). Сегодня доступно множество разных вариантов `auto`-конструкций. Некоторые делают не больше, чем `delete` в конце области видимости (как `auto_ptr`), а некоторые делают сложные вещи (например подсчет ссылок). Суть в том, что при работе с памятью нужно использовать `auto`-конструкции как можно шире. Если подходящей для ваших нужд нет - напишите ее. Это будет стоить потраченного времени. Помимо простого забывания освободить память, в коде могут пойти не так и другие вещи. Например, код, который не является `exception-safe`, очень легко может вызвать утечки. Вот простой пример:

```
void myfunc()
{
    BYTE* ptr = new BYTE[255];
    SomeFunc();
    delete[] ptr;
}
```

Если функция `SomeFunc` выбрасывает исключения (которые могут быть, а могут и не быть перехвачены выше вас), эта функция определенно утечет. Точнее, она утечет на 255 байт памяти. Если бы мы использовали `auto_ptr`, была бы гарантия, что утечки не будет - даже при наличии исключений, потому что раскрутка стека гарантирует очистку всех локальных объектов (то есть выделенных на стеке) при выходе из функции.

Еще одна возможность - перегрузить API выделения, используемые в вашем приложении. Это позволяет перехватывать все вызовы выделения памяти, тем самым давая вам `hooks` ко всем выделениям памяти, выполняемым вашими приложениями. Затем `hooks` выделений можно использовать для отслеживания выделений памяти, симуляции сбоев выделения памяти и многого другого.

Итоги

Утечки ресурсов - одна из главных причин нестабильности программного обеспечения, и поэтому в любом программном продукте их следует рассматривать как ошибки высокого приоритета. В этой главе мы объяснили общий процесс обнаружения утечек и два разных типа утечек ресурсов (утечки дескрипторов и утечки памяти), а также связанные с ними инструменты, которые существенно упрощают жизнь при отслеживании и исправлении утечек. Мы описали, как использовать `UMDH`, `LeakDiag` и ряд чрезвычайно мощных команд расширения (`!htrace` и `!heap`) для более эффективного поиска утечек ресурсов. Кроме того, мы представили некоторые (но определенно не все) способы заставить инструменты, которыми мы пользуемся каждый день (например компилятор), снижать нагрузку, связанную со случайно забытым освобождением ресурса после завершения работы с ним. `Auto`-конструкция - очень популярный и мощный механизм уменьшения количества утечек ресурсов в вашем ПО.

Вооружившись знанием общего процесса обнаружения утечек ресурсов, а также хорошим пониманием самых фундаментальных типов ресурсов, вы сможете справиться с любым типом утечки ресурсов.

Глава 10. Синхронизация

В этой главе мы подробно рассмотрим некоторые очень распространенные проблемы синхронизации, а также способы их устранения и максимально эффективного поиска первопричины. Глава начинается с объяснения базовых примитивов синхронизации, доступных в Windows, а затем переходит к ряду практических сценариев отладки, демонстрирующих распространенные проблемы синхронизации и то, как использовать отладчики для поиска первопричины.

Основы синхронизации

Операционная система Windows является вытесняющей и многопоточной операционной системой. Многопоточность означает способность выполнять любое количество потоков параллельно. Если система является однопроцессорной машиной, Windows создает иллюзию параллельного выполнения потоков, позволяя каждому потоку выполняться короткий промежуток времени (известный как квант времени). После исчерпания этого кванта времени поток переводится в состояние ожидания, а процессор переключается на другой поток (это называется переключением контекста), и так далее. На многопроцессорной машине два или более потоков способны выполняться одновременно (по одному потоку на физический процессор).

Поскольку система вытесняющая, все активные потоки в системе должны быть способны уступить управление процессором другому потоку в любой момент времени. Учитывая, что операционная система может забрать управление у потока, разработчики должны заботиться о том, чтобы поток всегда находился в состоянии, из которого управление можно безопасно забрать.

Если бы все приложения были однопоточными или если бы все потоки выполнялись изолированно, синхронизация не была бы проблемой. Увы, ради эффективности зависимая многопоточность сегодня является нормой, и она же является источником множества ошибок в приложениях. Зависимая многопоточность возникает, когда два или более потоков должны работать совместно, чтобы выполнить задачу. Выполнение кода для заданной задачи может, например, быть разбито между одним или несколькими потоками (с общими ресурсами или без них), и поэтому потокам нужно "общаться" друг с другом относительно порядка выполнения потоков. Это общение называется синхронизацией потоков и имеет решающее значение для любого многопоточного приложения.

Чтобы синхронизировать потоки, Windows предоставляет набор примитивов синхронизации.

Событие

Событие - это примитив режима ядра, доступный в пользовательском режиме через непрозрачный дескриптор. Событие является объектом синхронизации, который может принимать одно из двух состояний: сигнальное или несигнальное. Когда событие переходит из несигнального состояния в сигнальное (указывая, что произошло конкретное событие), поток, ожидающий этот объект события, будет пробужден и сможет продолжить выполнение. Объекты событий очень часто используются для синхронизации потока выполнения кода между несколькими потоками. Например, Win32 API `ReadFile` может читать данные асинхронно, если передать указатель на структуру `OVERLAPPED`. Рисунок 10.1 иллюстрирует поток событий.

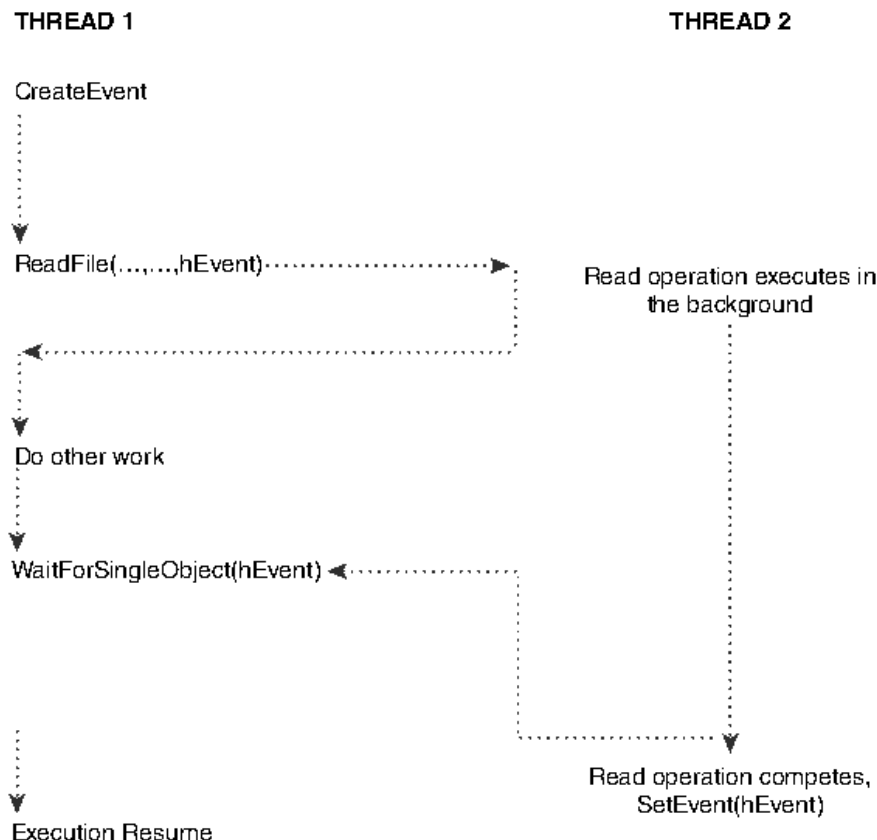


Figure 10.1 Asynchronous `ReadFile` API flow

Рисунок 10.1. Асинхронный поток API `ReadFile`

Часть структуры `OVERLAPPED` - это дескриптор события, который передает вызывающий код. Поскольку наличие параметра `OVERLAPPED` указывает, что операция является асинхронной, `ReadFile` немедленно возвращает управление вызывающему коду и обрабатывает операцию чтения в фоне. После этого вызывающий код свободен выполнять другую работу. Когда вызывающий код готов получить результаты операции чтения, он просто ожидает (с помощью API `WaitForSingleObject`), пока состояние события станет сигнальным. Когда фоновая операция чтения успешно завершается, событие переводится в сигнальное состояние, тем самым пробуждая вызывающий поток и позволяя продолжить выполнение.

Существует две формы объектов событий: `manual reset` и `auto reset`. Ключевое различие между ними состоит в том, что происходит, когда событие получает сигнал. В случае события `manual reset` объект события остается в сигнальном состоянии до явного сброса, тем самым позволяя освободить любое количество потоков, ожидающих объект события. В отличие от этого, событие `auto reset` позволяет освободить только один ожидающий поток, прежде чем автоматически

сбрасывается в несигнальное состояние. Если потоков, ожидающих событие, нет, событие остается в сигнальном состоянии до тех пор, пока первый поток не попытается ожидать это событие.

В пользовательском режиме объект события представлен непрозрачным дескриптором базового объекта ядра. Поэтому в пользовательском режиме невозможно посмотреть, как объект дескриптора разложен в памяти. Однако существует команда расширения, позволяющая получить некоторые сведения о конкретном дескрипторе. Эта команда расширения называется !handle. Чтобы увидеть, как работает команда расширения !handle, присоедините отладчик к экземпляру notepad.exe и выполните команду !handle. Листинг 10.1 показывает сокращенный вывод команды расширения !handle. (Обратите внимание, что вывод может выглядеть иначе в зависимости от состояния Notepad в момент выполнения команды.)

Листинг 10.1

```
0:001> !handle
Handle 74
Type File
Handle 3c8
Type Section
Handle 3cc
Type Mutant
Handle 3d8
Type Mutant
Handle 3dc
Type Mutant
Handle 3e0
Type Mutant
Handle 3e4
Type Mutant
Handle 3e8
Type Mutant
Handle 3f0
Type Section
Handle 42c
Type Key
Handle 438
Type Section
Handle 43c
Type Port
Handle 47c
Type Event
37 Handles
...
...
...
Type Count
None 1
Event 5
Section 4
File 4
Port 2
Directory 3
Mutant 6
WindowStation 2
Semaphore 5
Key 4
Desktop 1
```

Как видно, команда расширения !handle (без параметров) выгружает все дескрипторы, открытые в процессе, с сокращенной информацией. Чтобы получить более подробные сведения о конкретном дескрипторе, нужно добавить значение дескриптора к команде расширения !handle, а затем значение, представляющее глубину отображаемой информации. Значение f дает наиболее исчерпывающую информацию. Возьмем дескриптор 47c (событие) в качестве примера (см. листинг 10.2).

Листинг 10.2

```
0:001> !handle 47c f
Handle 614
Type Event
Attributes 0
GrantedAccess 0x1f0003:
Delete,ReadControl,WriteDac,WriteOwner,Synch
QueryState,ModifyState
HandleCount 2
PointerCount 4
Name <none>
Object Specific Information
Event Type Auto Reset
Event is Set
```

Листинг 10.2 показывает тип дескриптора, его атрибуты, предоставленный доступ, счетчики дескрипторов и т. д. Он также дает сведения о типе события (auto reset), а также о состоянии события, которое в данном конкретном случае оказалось установленным. Еще одна интересная часть информации - имя события (задано как <none>). При создании события его можно именовать, тем самым позволяя использовать событие между процессами, а не только внутри одного процесса. Два или более процесса договариваются об имени события, и при попытке открыть событие с этим конкретным именем оно либо будет создано, если это первый вызов, либо у существующего события просто будет увеличен счетчик ссылок.

Критическая секция

Критические секции чаще всего используются для защиты общих ресурсов между потоками, гарантируя эксклюзивный доступ (то есть только один поток способен получить доступ к ресурсу). Чтобы проиллюстрировать использование критической секции, представьте следующий фрагмент псевдокода:

```
1. Enter Critical Section
...
...
2. Access Shared Resource
...
...
3. Leave Critical Section
```

Кроме того, представьте два потока (T1 и T2), которые оба выполняют приведенный код, пытаясь получить доступ к общему ресурсу. Предположим, что T1 первым доходит до шага 1. Первое, что происходит, когда T1 пытается войти в критическую секцию, - он проверяет, доступна ли критическая секция (то есть что никакой другой поток сейчас не находится внутри критической секции). Поскольку это так, T1 входит в критическую секцию и начинает обращаться к общему ресурсу на шаге 2. Теперь происходит переключение контекста, и T2 получает возможность выполняться, доходит до шага 1 и пытается войти в критическую секцию. Поскольку T1 уже владеет критической секцией, T2 получает указание ждать в точке входа в критическую секцию, пока T1 не покинет ее. Происходит еще одно переключение контекста, и T1 завершает работу, выполняя шаг 3 и покидая критическую секцию. При следующем переключении контекста T2 входит в критическую секцию, и выполнение продолжается.

Способ, которым поток ожидает доступности критической секции, различается между однопроцессорными и многопроцессорными машинами. На однопроцессорных машинах поток действительно переходит в эффективное состояние ожидания (переход в ядро), тогда как на многопроцессорных машинах поток может попытаться вращаться X раз в надежде, что критическая секция станет доступной во время этого вращения. Это делается, чтобы избежать затрат на переход в состояние ожидания, требующий перехода в ядро и переключения контекста.

Рассмотрим подробнее раскладку критической секции в памяти. Базовая структура данных критической секции - `RTL_CRITICAL_SECTION`, и ее можно посмотреть с помощью команды `dt`:

```
0:001> dt RTL_CRITICAL_SECTION
+0x000 DebugInfo : Ptr32 _RTL_CRITICAL_SECTION_DEBUG
+0x004 LockCount : Int4B
+0x008 RecursionCount : Int4B
+0x00c OwningThread : Ptr32 Void
+0x010 LockSemaphore : Ptr32 Void
+0x014 SpinCount : Uint4B
```

Отдельные поля структуры `RTL_CRITICAL_SECTION` подробнее описаны ниже:

- **DebugInfo**: поле `DebugInfo` - сопутствующая структура, выделенная системой; она содержит набор расширенной информации о критической секции (обсуждается далее).
- **LockCount**: это поле показывает, сколько потоков ожидают приобретения критической секции. По умолчанию оно инициализируется значением -1, которое указывает, что критическая секция не была приобретена. Значение 0 или больше указывает, что она была приобретена. Чтобы узнать, сколько других потоков ожидают критическую секцию, можно использовать следующую формулу:

$$\text{Number of waiting threads} = \text{LockCount} - \text{RecursionCount} + 1$$

В Windows 2003 Server SP1 и более поздних версиях это поле было изменено на битовое поле, чтобы устранить очень распространенную проблему критических секций, известную как проблема `lock convoy`. Позже в этой главе мы подробнее рассмотрим, что такое `lock convoy` и как его обнаружить.

- **RecursionCount**: поток может приобрести критическую секцию более одного раза. Это поле показывает, сколько раз один и тот же поток приобрел критическую секцию. По умолчанию значение этого поля равно 0, что указывает на отсутствие потока, владеющего критической секцией.
- **OwningThread**: если критическая секция была приобретена, это поле содержит ID потока, который ее приобрел.
- **LockSemaphore**: это поле на самом деле содержит дескриптор события `auto-reset`, а не семафор. Его основное назначение - указывать, когда критическая секция свободна и готова к приобретению. Событие создается всякий раз, когда делается попытка приобрести критическую секцию, уже приобретенную другим потоком. Чтобы избежать утечки дескриптора, критически важно вызвать `API DeleteCriticalSection` после завершения работы с критической секцией.
- **SpinCount**: это поле используется только в многопроцессорных системах. Если поток уже владеет критической секцией, а другой поток пытается ее приобрести, этот поток перейдет в состояние ожидания до освобождения критической секции. Переход в это состояние ожидания требует перехода в ядро, а это дорогая операция. Чтобы попытаться устранить этот переход в многопроцессорных системах, поток вместо немедленного перехода в состояние ожидания вращается `SpinCount` раз, на каждой итерации пытаясь приобрести критическую секцию, что повышает производительность в случаях, когда критическая секция вот-вот должна быть освобождена. По умолчанию это значение равно 0, но его можно изменить с помощью `API InitializeCriticalSectionAndSpinCount`.

Теперь рассмотрим поле `DebugInfo` подробнее.

```
0:001> dt RTL_CRITICAL_SECTION_DEBUG
+0x000 Type : Uint2B
+0x002 CreatorBackTraceIndex : Uint2B
+0x004 CriticalSection : Ptr32 _RTL_CRITICAL_SECTION
+0x008 ProcessLocksList : _LIST_ENTRY
+0x010 EntryCount : Uint4B
+0x014 ContentionCount : Uint4B
+0x018 Flags : Uint4B
+0x01c Spare : Uint4B
```

Различные части DebugInfo объяснены ниже:

- Type: это поле не используется (по умолчанию 0).
- CreatorBackTraceIndex: если расширенное инструментирование было включено запуском gflags, это поле содержит индекс, использованный при сборе информации трасс стеков.
- CriticalSection: это поле содержит указатель на критическую секцию, связанную с этой структурой, по сути позволяя пройти назад от отладочной структуры к критической секции.
- ProcessLocksList: для любого заданного процесса операционная система поддерживает список всех активных критических секций в этом процессе. Это поле представляет узел в этом списке и содержит прямой и обратный указатели. Указатели FLINK и BLINK этого узла можно использовать для обхода списка критических секций процесса.
- EntryCount: это поле увеличивается всякий раз, когда поток переходит в состояние ожидания, пытаясь приобрести уже занятую критическую секцию.
- ContentionCount: это поле увеличивается всякий раз, когда поток переходит в состояние ожидания, пытаясь приобрести уже занятую критическую секцию.
- Flags: это поле не используется.
- Spare: это поле не используется.

Важно отметить, что хотя RTL_CRITICAL_SECTION_DEBUG, похоже, содержит в основном отладочную информацию, в ранних версиях Windows она требуется, чтобы критическая секция считалась пригодной к использованию. Фактически, если операционная система не может выделить память для этой структуры во время инициализации, API завершится ошибкой. В Windows Server 2003 SP1 и выше отладочная информация больше не нужна для функционирования критической секции. Об этом различии важно помнить во время отладки, потому что поле DebugInfo, равное NULL, может заставить вас подумать, что критическая секция находится в плохом и непригодном состоянии.

Вместо ручного обхода списка критических секций, поддерживаемого операционной системой, можно использовать команду расширения !cs. Листинг 10.3 показывает сокращенный пример на недавно запущенном экземпляре notepad.exe.

Листинг 10.3

```
0:000> !cs
-----
DebugInfo = 0x7c97c420
Critical section = 0x7c97c0a0 (ntdll!RtlCriticalSectionLock+0x0)
NOT LOCKED
LockSemaphore = 0x0
SpinCount = 0x00000000
-----
DebugInfo = 0x7c97c440
Critical section = 0x7c97c080 (ntdll!DeferredCriticalSection+0x0)
NOT LOCKED
LockSemaphore = 0x0
SpinCount = 0x00000000
-----
DebugInfo = 0x7c97c100
Critical section = 0x7c97c0d8 (ntdll!LdrpLoaderLock+0x0)
LOCKED
LockCount = 0x0
OwningThread = 0x000000b48
RecursionCount = 0x1
LockSemaphore = 0x0
SpinCount = 0x00000000
...
...
...
```

Как видно из листинга 10.3, отображаемая информация - это просто урезанная версия фактической структуры критической секции, которую мы рассматривали ранее. Если критическая секция приобретена, дополнительно показываются поля LockCount, OwningThread и RecursionCount.

Команду расширения !cs также можно использовать для отображения информации об одной критической секции, добавив адрес критической секции к команде, как показано ниже.

```
0:000> !cs 0x7c97c0d8
-----
Critical section = 0x7c97c0d8 (ntdll!LdrpLoaderLock+0x0)
DebugInfo = 0x7c97c100
LOCKED
LockCount = 0x0
OwningThread = 0x00000b48
RecursionCount = 0x1
LockSemaphore = 0x0
SpinCount = 0x00000000
```

Одно предупреждение об API EnterCriticalSection в Windows 2000: он может выбросить исключение нехватки памяти при низком объеме доступной памяти. Помните, что критическая секция использует событие для выполнения своей работы, и это событие может быть инициализировано внутри API EnterCriticalSection. Если в системе мало памяти, будет выброшено исключение. Если вы хотите, чтобы критические секции надежно работали в Windows 2000, следует использовать API InitializeCriticalSectionAndSpinCount, который выделяет событие во время инициализации критической секции и не выбрасывает исключений при последующем использовании.

Мьютекс

Мьютекс - это конструкция синхронизации режима ядра, которую можно использовать для синхронизации потоков как внутри процесса, так и между несколькими процессами (если при создании мьютекс именуется). В общем случае, если все ваши задачи синхронизации находятся внутри одного процесса, следует использовать критическую секцию. Если же нужно синхронизироваться между процессами, правильный подход - именованный мьютекс. Поскольку мьютекс является конструкцией режима ядра, код пользовательского режима обращается к мьютексу через непрозрачное значение дескриптора. Чтобы получить больше информации о мьютексе во время отладки в пользовательском режиме, можно использовать команду расширения !handle. Присоедините отладчик к экземпляру Notepad и введите !handle, как показано в сокращенном выводе в листинге 10.4.

Листинг 10.4

```
0:001> !handle
Handle c
Type File
Handle 368
Type Section
Handle 36c
Type Mutant
Handle 3a4
Type Section
Handle 3a8
Type Mutant
Handle 3b4
Type Mutant
Handle 3b8
Type Mutant
Handle 3bc
Type Mutant
...
...
...
39 Handles
Type Count
Event 5
Section 5
```

```
File 4
Port 2
Directory 3
Mutant 7
WindowStation 2
Semaphore 5
Key 4
Desktop 1
KeyedEvent 1
```

Первое важное, что нужно заметить в листинге 10.4: отладчик называет мьютекс mutant, и листинг показывает, что открыто семь mutants. Чтобы получить расширенную информацию для любого заданного mutant, можно выполнить команду расширения !handle, значение дескриптора и число, указывающее объем отображаемой информации:

```
0:001> !handle 3b4 f
Handle 3b4
Type Mutant
Attributes 0
GrantedAccess 0x1f0001:
Delete,ReadControl,WriteDac,WriteOwner,Synch
QueryState
HandleCount 22
PointerCount 24
Name \BaseNamedObjects\CTF.TMD.MutexDefaultS-1-5-21-1123561945-308236825-
725345543-1004
Object Specific Information
Mutex is Free
```

Помимо общих полей информации об объекте ядра, описанных в разделе "Событие" этой главы, объектно-специфичная информация показывает, свободен мьютекс или занят. (В нашем случае он свободен.) Если выгрузить расширенную информацию для всех mutants в экземпляре Notepad, вы также увидите, что большинство из них являются именованными mutants, что указывает на возможность доступа к этому mutant из других процессов.

Мьютекс считается abandoned, если поток, который сейчас владеет им, завершается без освобождения мьютекса, тем самым не позволяя другим потокам приобрести его. Операционная система обнаруживает этот сценарий и автоматически переводит мьютекс в сигнальное состояние, позволяя ожидающим потокам приобрести мьютекс. В таком сценарии, когда поток пробуждается, чтобы приобрести мьютекс, API ожидания возвращает код состояния (WAIT_ABANDONED), тем самым сигнализируя ожидающему потоку, что мьютекс был abandoned. Обычно такая ситуация указывает на ошибку в коде, и сценарий следует расследовать.

Семафор

Семафор - это объект синхронизации режима ядра, доступный из пользовательского режима. Он похож на критическую секцию и мьютекс в том смысле, что позволяет эксклюзивный доступ к ресурсу. Однако главное различие в том, что семафор использует подсчет ресурсов, тем самым позволяя X потокам получить доступ к ресурсу. Пример использования семафора - система с четырьмя USB-портами, к которым обращается некоторый код. Поскольку USB-портов четыре, мы хотим разрешить четырем потокам одновременно использовать один из доступных USB-портов. Для этого мы создали бы семафор с максимальным счетчиком ресурсов 4. Когда потоки пытаются приобрести семафор, проверяется счетчик ссылок (инициализированный значением 4): больше ли он 0; если да, приобретение разрешается, а счетчик ссылок уменьшается. Когда счетчик ссылок достигает 0, поток, пытающийся приобрести семафор, будет усыплен до тех пор, пока другой поток не освободит семафор и счетчик ссылок не увеличится. Как и с событиями и мьютексами, в отладчике используется команда расширения `!handle`, чтобы получить расширенную информацию о семафоре. Присоедините отладчик к экземпляру Notepad и перечислите все дескрипторы в процессе. Найдите дескриптор, представляющий семафор, и выгрузите расширенную информацию:

```
0:001> !handle 7f4 f
Handle 7f4
Type Semaphore
Attributes 0
GrantedAccess 0x100003:
Synch
QueryState,ModifyState
HandleCount 2
PointerCount 3
Name <none>
Object Specific Information
Semaphore Count 0
Semaphore Limit 2147483647
```

Объектно-специфичный вывод показывает состояние семафора, включая текущий счетчик семафора.

Высокоуровневый процесс

Процесс разрешения проблемы синхронизации в вашем коде показан на рисунке 10.2.

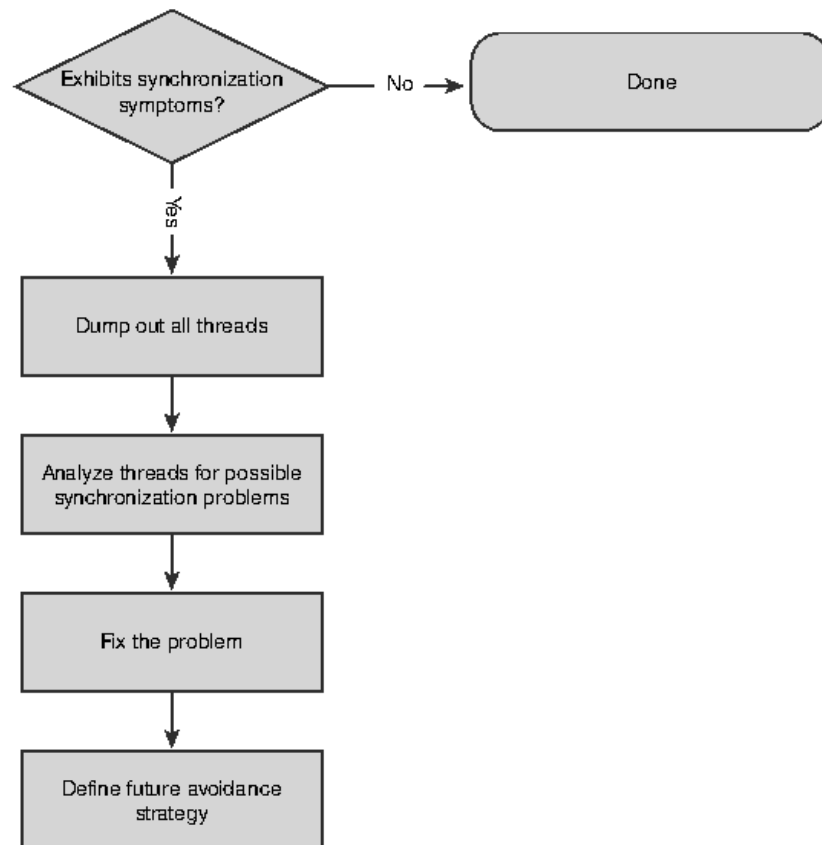


Figure 10.2 Synchronization Problem Analysis Process

Рисунок 10.2. Процесс анализа проблемы синхронизации

Этот процесс подробнее рассматривается в следующих разделах.

Шаг 1. Распознать симптомы

Первый шаг в анализе возможной проблемы синхронизации - научиться распознавать симптомы. Хотя невозможно перечислить все разные симптомы, которые могут проявиться, вполне возможно перечислить их подавляющее большинство. Базовая предпосылка проблемы синхронизации и соответствующего симптома состоит в том, что прогресс приложения остановился. Это может происходить на легко распознаваемом уровне, например когда все приложение кажется зависшим и не отвечает, или при выполнении конкретных задач в приложении. Хороший индикатор "зависшего" приложения - использование CPU приложением во время выполнения задачи, про которую вы знаете, что она должна вызывать рост потребления CPU. Использование CPU легко отслеживать с помощью Task Manager (CTRL+SHIFT+ESC). Например, если ваше приложение использует 0% CPU, вычисляя число π до 100 000-го десятичного знака, вполне возможно, что приложение зависло.

Еще один распространенный симптом зависания - CPU резко вырос в использовании, но обработка не завершается в ожидаемые временные пределы. По сути приложение находится в "зависшем" состоянии, но вместо того чтобы зависнуть из-за того, что два или более потоков ожидают друг друга в эффективном состоянии ожидания, эти же потоки могут не продвигаться вперед из-за агрессивного вращения, тем самым резко увеличивая использование CPU.

Если приложение демонстрирует симптомы отсутствия прогресса, следует перейти к следующему шагу процесса.

Шаг 2. Выгрузить все потоки

Итак, теперь у вас есть приложение, которое отказывается продвигаться по текущей задаче. Вы довольно уверены, что имеете дело с проблемой синхронизации. Что делать дальше? Такие ситуации требуют внимательнее посмотреть на процесс и понять, можно ли выявить проблемы. Поскольку проблемы такого типа обычно включают два или более потока, которые не были правильно синхронизированы, первый шаг - присоединить отладчик и перечислить все потоки вместе с их трассами стеков. Просмотр потоков и их стеков может дать подсказки, на чем сосредоточить усилия и где может быть проблема. Самый простой способ выгрузить все потоки и трассы стеков - использовать команду `~*kb`. Листинг 10.5 показывает вывод команды, выполненной на недавно запущенном экземпляре `notepad.exe`.

Листинг 10.5

```
0:001> ~*kb
0 Id: ea4.e9c Suspend: 1 Teb: 7ffdf000 Unfrozen
ChildEBP RetAddr Args to Child
0007feb8 77d491be 77d491f1 0007fefc 00000000 ntdll!KiFastSystemCallRet
0007fed8 01002a1b 0007fefc 00000000 00000000 USER32!NtUserGetMessage+0xc
0007ff1c 01007511 01000000 00000000 00bd0ffb notepad!WinMain+0xe5
0007ffc0 7c816fd7 00090000 0007fa0c 7ffd5000 notepad!WinMainCRTStartup+0x174
0007fff0 00000000 0100739d 00000000 78746341 kernel32!BaseProcessStart+0x23
# 1 Id: ea4.974 Suspend: 1 Teb: 7ffde000 Unfrozen
ChildEBP RetAddr Args to Child
02d5ffc8 7c9507a8 00000005 00000004 00000001 ntdll!DbgBreakPoint
02d5fff4 00000000 00000000 00000000 00000000 ntdll!DbgUiRemoteBreakin+0x2d
```

Как видно из листинга 10.5, в процессе активно только два потока. Первый поток, похоже, является основным message pump. (Второй фрейм `USER32!NtUserGetMessage+0xc` - подсказка.) Второй поток - это поток прерывания отладчика, и на самом деле он не имеет отношения к самому приложению, поскольку такой тип потока вы всегда будете видеть, когда отладчик прерывает выполнение.

После выгрузки всех потоков пора посмотреть, проявляют ли какие-либо из них признаки проблем синхронизации.

Шаг 3. Проанализировать потоки на возможные проблемы синхронизации

К проблемам синхронизации может приводить множество сценариев. Этот шаг выявляет проблемные потоки. Сам процесс анализа потоков мы отложим до раздела "Сценарии синхронизации" этой главы, где рассмотрим ряд распространенных проблем синхронизации.

Очень распространенный индикатор неправильных техник синхронизации - ситуация, когда два или более потока ждут, пока друг друга освободят некоторый примитив синхронизации, но ни один из потоков не готов освободить его, пока другой поток не сделает это первым. Ключ к выявлению такого сценария - понять, что значит для потока "ждать". Поток может перейти в состояние ожидания множеством разных техник. Однако чаще всего поток использует один из двух способов:

- Пытается приобрести примитив синхронизации с помощью собственного API этого примитива. Отличный пример - попытка войти в критическую секцию (с помощью API `EnterCriticalSection`). Обычная трасса стека, в которой поток пытается войти в критическую секцию, но не может, выглядит так:

```

1 Id: 25c.6e0 Suspend: 1 Teb: 7ffde000 Unfrozen
ChildEBP RetAddr Args to Child
007eff18 7c90e9c0 7c91901b 000007f4 00000000 ntdll!KiFastSystemCallRet
007eff1c 7c91901b 000007f4 00000000 00000000 ntdll!NtWaitForSingleObject+0xc
007effa4 7c90104b 00002008 01001144 01002008 ntdll!RtlpWaitForCriticalSection+
0x132
007effac 01001144 01002008 7c80b683 00000000
ntdll!RtlEnterCriticalSection+0x46
007effb4 7c80b683 00000000 00081000 005cadf8 simple!ThreadProc+0xb
007effec 00000000 01001139 00000000 00000000 kernel32!BaseThreadStart+0x37

```

- Если для примитива синхронизации нет конкретного API (как в случае всех примитивов синхронизации режима ядра), наиболее часто используемые API ожидания - это `WaitForSingleObject(/Ex)` или `WaitForMultipleObjects(/Ex)`. Эти API принимают один или несколько дескрипторов примитивов синхронизации режима ядра. Обычная трасса стека в таком сценарии выглядит так:

```

1 Id: f38.b60 Suspend: 1 Teb: 7ffdd000 Unfrozen
ChildEBP RetAddr Args to Child
007eff2c 7c90e9c0 7c8025cb 000007e8 00000000 ntdll!KiFastSystemCallRet
007eff30 7c8025cb 000007e8 00000000 00000000 ntdll!NtWaitForSingleObject+0xc
007eff94 7c802532 000007e8 ffffffff 00000000
kernel32!WaitForSingleObjectEx+0xa8
007effa8 01001147 000007e8 ffffffff 7c80b683 kernel32!WaitForSingleObject+0x12
007effb4 7c80b683 00000000 00081000 005cadf8 simple!ThreadProc+0xe
007effec 00000000 01001139 00000000 00000000 kernel32!BaseThreadStart+0x37

```

Если какие-либо интересующие потоки находятся в этом состоянии ожидания, следующий шаг - посмотреть, какие из потоков потенциально не продвигаются из-за проблемы синхронизации.

Если потоков в состоянии ожидания нет или если потоки, которые находились в состоянии ожидания, работали нормально, следующее, что нужно искать, - вращающиеся потоки. Существует несколько разных способов выяснить, какие потоки (если такие есть) вращаются. Самый базовый включает установку точек останова в подозрительных потоках в разных местах цепочки вызовов и просмотр того, когда и как срабатывает точка останова и при каких условиях поток в итоге вращается. Это может быть довольно трудоемкой работой, и, к счастью, есть команда расширения, упрощающая жизнь. Команда расширения `!runaway` перечисляет все потоки в соответствии с тем, сколько времени они провели в выполнении. Пример команды расширения `!runaway` показан ниже:

```

0:001> !runaway
User Mode Time
Thread Time
0:9d8 0 days 0:00:00.046
1:e48 0 days 0:00:00.000

```

Показанное для каждого потока время - это время, которое поток провел в пользовательском режиме. Из вывода видно, что поток с ID 9d8 использовал больше всего времени пользовательского режима. Если нужна более подробная разбивка времени, которое поток провел в выполнении, можно выполнить команду `runaway` и передать ей параметр `flags`. Параметр 7 дает наиболее подробную информацию.

```

0:001> !runaway 7
User Mode Time
Thread Time
0:9d8 0 days 0:00:00.046
1:e48 0 days 0:00:00.000
Kernel Mode Time
Thread Time
0:9d8 0 days 0:00:00.031
1:e48 0 days 0:00:00.000
Elapsed Time
Thread Time
0:9d8 0 days 0:00:38.000
1:e48 0 days 0:00:36.734

```

Время пользовательского режима и режима ядра указывает, сколько времени каждый поток выполнялся в соответствующем контексте. Прошедшее время показывает, как долго поток выполнялся с момента запуска (независимо от того, использовал ли поток CPU).

В сценарии, где приложение зависло из-за вращения одного или нескольких потоков, эта команда может очень быстро показать, какие потоки вращаются.

После выявления виновных потоков и обнаружения источника проблемы следующий шаг процесса - найти подходящее исправление.

Шаг 4. Исправить проблему

В зависимости от природы проблемы синхронизации может существовать несколько разных исправлений. Например, deadlock исправляется обеспечением того, что условие, при котором возникает deadlock, устранено; orphaned locks исправляются обеспечением того, что за всеми приобретениями блокировок следуют соответствующие вызовы освобождения; а lock convoys можно исправить переработкой кода для устранения спорных блокировок. Различные техники исправления проблем синхронизации мы подробнее рассмотрим в разделе "Сценарии синхронизации" этой главы.

Последний шаг процесса - определить стратегию предотвращения на будущее и убедиться, что диагностированная и исправленная проблема больше не возникнет.

Шаг 5. Определить стратегию предотвращения на будущее

Как специалисты по информатике, мы часто гордимся своей способностью управлять сложностью. Но не менее важно, чем разобраться со сложными проблемами, - убедиться, что похожие проблемы не появятся снова в будущем. Этот финальный шаг процесса берет то, что мы узнали при исследовании проблемы синхронизации, и формирует план того, как избегать похожих проблем в будущем. Это может выражаться в изменении инженерных практик или в использовании инструментов, способных обнаруживать проблемы на ранних этапах разработки.

Сценарии синхронизации

Эта часть главы показывает примеры некоторых наиболее распространенных форм проблем синхронизации. Она показывает, как использовать отладчики для эффективного поиска проблемы, чтобы найти первопричину, а также какие инструменты и техники можно использовать для предотвращения похожих проблем в будущем.

Базовый deadlock

Deadlock, возможно, является самой распространенной и раздражающей проблемой, с которой сталкиваются разработчики при написании многопоточных приложений. По сути deadlock возникает, когда два или более потока удерживают защищенные ресурсы и отказываются отпускать эти ресурсы, пока другие не отпустят свои. Поскольку ни один из потоков не готов освободить свои защищенные ресурсы, в конечном итоге ни один из потоков никогда не продвинется вперед. Они просто сидят и ждут, пока другие сделают ход, и возникает deadlock. Deadlock может возникнуть огромным количеством разных способов, и мы рассмотрим некоторые распространенные варианты на протяжении этой главы. Однако прежде чем перейти к более сложным случаям, мы проиллюстрируем простой и смоделированный сценарий deadlock. Это даст вам хорошее представление о том, как deadlock выглядит в отладчике и какие команды можно использовать, чтобы разобраться в его сути.

Пример приложения, используемый для иллюстрации deadlock, довольно прост и показан в листинге 10.6.

Листинг 10.6

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
CRITICAL_SECTION cs_DB1;
CRITICAL_SECTION cs_DB2;
DWORD WINAPI ThreadProc( LPVOID lpParam )
{
    EnterCriticalSection(&cs_DB1);
    //
    // Do work on stack
    //
    printf("Updating database 1\n");
    Sleep(3000);
    EnterCriticalSection(&cs_DB2);
    printf("Updating database 2\n");
    LeaveCriticalSection(&cs_DB2);
    LeaveCriticalSection(&cs_DB1);
    return 1;
}
void __cdecl main ( )
{
    HANDLE hThread=NULL;
    DWORD dwId=0;
    InitializeCriticalSection(&cs_DB1);
    InitializeCriticalSection(&cs_DB2);
    hThread = CreateThread(NULL, 0, ThreadProc, NULL, 0, &dwId);
    if(hThread)
    {
        Sleep(2000);
        EnterCriticalSection(&cs_DB2);
        printf("Updating database 2\n");
        Sleep(2000);
        EnterCriticalSection(&cs_DB1);
        printf("Updating database 1\n");
        LeaveCriticalSection(&cs_DB1);
        LeaveCriticalSection(&cs_DB2);
        WaitForSingleObject(hThread, INFINITE);
        CloseHandle(hThread);
    }
    DeleteCriticalSection(&cs_DB1);
    DeleteCriticalSection(&cs_DB2);
}
```

Исходный код и двоичный файл для листинга 10.6 находятся в следующих папках:

Source code: C:\AWD\Chapter10\DeadLock
Binary: C:\AWDBIN\WinXP.x86.chk\10DeadLock.exe

Приложение является многопоточным (два потока) и использует две разные базы данных. Каждому потоку нужен доступ к обеим базам данных, чтобы выполнить свою работу. Поскольку нижележащие API доступа к базам данных не являются thread safe, приложение использует две критические секции, каждая из которых защищает одну базу данных. Чтобы не загрязнять пример кода, код, использующий базы данных, моделируется простым усыплением потока на некоторое количество миллисекунд. Когда вы запустите это приложение, вы быстро увидите, что оно никогда не завершается. В этом простом приложении вы, возможно, уже заметили проблему в коде, но давайте присоединим отладчик к работающему экземпляру и посмотрим, что на самом деле происходит. Первый шаг - выгрузить все потоки, сейчас выполняющиеся в процессе, как показано в листинге 10.7.

Листинг 10.7

```
0:002> ~*kb
0 Id: 978.960 Suspend: 1 Teb: 7ffde000 Unfrozen
ChildEBP RetAddr Args to Child
0006fe9c 7c90e9c0 7c91901b 000007dc 00000000 ntdll!KiFastSystemCallRet
0006fea0 7c91901b 000007dc 00000000 00000000 ntdll!NtWaitForSingleObject+0xc
0006ff28 7c90104b 00002008 01001286 01002008 ntdll!RtlpWaitForCriticalSection+0x132
0006ff30 01001286 01002008 000007e8 00001210 ntdll!RtlEnterCriticalSection+0x46
0006ff44 01001406 00000001 00264e20 00263328 10DeadLock!main+0x86
0006ffc0 7c816fd7 00090000 0007fa0c 7ffdf000 10DeadLock!mainCRTStartup+0x12f
0006fff0 00000000 010012d7 00000000 78746341 kernel32!BaseProcessStart+0x23
1 Id: 978.1210 Suspend: 1 Teb: 7ffdd000 Unfrozen
ChildEBP RetAddr Args to Child
002bfff14 7c90e9c0 7c91901b 000007f4 00000000 ntdll!KiFastSystemCallRet
002bfff18 7c91901b 000007f4 00000000 00000000 ntdll!NtWaitForSingleObject+0xc
002bffa0 7c90104b 00002020 010011c4 01002020 ntdll!RtlpWaitForCriticalSection+0x132
002bffa8 010011c4 01002020 002bfffec 7c80b683 ntdll!RtlEnterCriticalSection+0x46
002bffb4 7c80b683 00000000 000822c8 02080028 10DeadLock!ThreadProc+0x34
002bfffec 00000000 01001190 00000000 00000000 kernel32!BaseThreadStart+0x37
# 2 Id: 978.1074 Suspend: 1 Teb: 7ffdc000 Unfrozen
ChildEBP RetAddr Args to Child
002ffffb4 7c80b683 00000000 00919920 0308f7dc kernel32!CtrlRoutine+0xbd
002ffffec 00000000 7c875280 00000000 00000000 kernel32!BaseThreadStart+0x37
```

Хотя пример прост, потоки, показанные в листинге 10.7, иллюстрируют очень распространенную технику распознавания deadlock. Два (или более) потока каждый ждут приобретения разных критических секций, но ничего не происходит. Чтобы проверить, что наше предположение о deadlock действительно верно, нужно внимательнее посмотреть на задействованные критические секции. Как найти адрес критической секции для каждого потока? По стекам видно, что наш код вызывает API RtlEnterCriticalSection. Это выглядит немного странно, учитывая, что наш код использует API EnterCriticalSection. Здесь происходит следующее: EnterCriticalSection (определенная в kernel32.dll) является просто API-forwarder к RtlEnterCriticalSection (определенной в ntdll.dll). Вы легко можете определить, какие API являются forwarders, с помощью линкера, как показано ниже (сокращенный вывод):

```
C:\>link /dump /exports %SystemRoot%\system32\kernel32.dll
Microsoft (R) COFF/PE Dumper Version 7.10.4035
Copyright (C) Microsoft Corporation. All rights reserved.
Dump of file kernel32.dll
File Type: DLL
Section contains the following exports for KERNEL32.dll
00000000 characteristics
44AB7FD3 time date stamp Wed Jul 05 02:01:07 2006
0.00 version
1 ordinal base
949 number of functions
949 number of names
ordinal hint RVA name
1 0 0000A644 ActivateActCtx
2 1 000354ED AddAtomA
3 2 000326C1 AddAtomW
4 3 00070CBF AddConsoleAliasA
```

```

5 4 00070C81 AddConsoleAliasW
6 5 00058F26 AddLocalAlternateComputerNameA
7 6 00058E0A AddLocalAlternateComputerNameW
8 7 0002BF01 AddRefActCtx
...
...
...
151 96 EnterCriticalSection (forwarded to NTDLL.RtlEnterCriticalSection)
152 97 00038211 EnumCalendarInfoA
153 98 00075749 EnumCalendarInfoExA
...
...
...

```

Поскольку API EnterCriticalSection является всего лишь forwarder, можно также разумно ожидать, что параметры forwarded-функции будут идентичными. Точнее, первый параметр RtlEnterCriticalSection должен быть адресом критической секции. В листинге 10.7 видно, что первый поток вызывает RtlEnterCriticalSection, а первый переданный параметр - 01001286. Теперь можно использовать команду расширения !cs, чтобы выгрузить поля критической секции:

```

0:002> !cs 01002008
-----
Critical section = 0x01002008 (10DeadLock!cs_DB1+0x0)
DebugInfo = 0x7c97c8c0
LOCKED
LockCount = 0x1
OwningThread = 0x00001210
RecursionCount = 0x1
LockSemaphore = 0x7DC
SpinCount = 0x00000000

```

В структуре данных критической секции существует много информации, которую мы можем использовать. Помимо очевидного факта, что критическая секция находится в состоянии LOCKED, первое поле, Critical section, дает нам символьную ссылку на критическую секцию (в нашем случае 10DeadLock!cs_DB1+0x0). Это немедленно сообщает, что рассматриваемая критическая секция относится к первой базе данных приложения. Мы также очень быстро видим, какой поток сейчас удерживает критическую секцию, а именно поток 0x1210. Но какой поток имеет ID 0x1210? Верхняя строка каждой трассы стека показывает несколько разных ID. В листинге 10.7 видно, что верх стека содержит следующее:

```

Id: 978.960 Suspend: 1 Teb: 7ffde000 Unfrozen

```

Два числа служат для идентификации двух сущностей: первое число идентифицирует процесс, второе - поток. Поэтому первый поток в нашем процессе имеет ID 0x960. Используя тот же механизм, мы легко видим, что поток, соответствующий ID потока 0x1210, - наш второй поток. Используя тот же механизм выгрузки критической секции, которую пытается приобрести второй поток, видим следующее:

```

0:002> !cs 01002020
-----
Critical section = 0x01002020 (10DeadLock!cs_DB2+0x0)
DebugInfo = 0x7c97c8e0
LOCKED
LockCount = 0x1
OwningThread = 0x00000960
RecursionCount = 0x1
LockSemaphore = 0x7F4
SpinCount = 0x00000000

```

И снова видно, что рассматриваемая блокировка - блокировка нашей второй базы данных, а владеющий поток - поток с ID 0x960 (который также является нашим первым потоком). Картина теперь должна быть ясной. Первый поток в нашем процессе удерживает блокировку второй базы данных, ожидая, пока станет доступна блокировка первой базы данных. Второй поток удерживает критическую секцию, связанную с блокировкой первой базы данных, ожидая доступности второй.

Итоговый результат - приложение в deadlock.

Теперь, когда мы знаем, какие потоки на чем заблокировались, финальный шаг - выполнить анализ исходного кода, чтобы попытаться разорвать deadlock. Из нашего простого примера должно быть вполне очевидно, почему произошел deadlock и как его можно разорвать.

К сожалению, для разработчиков, которые хотят эффективнее отслеживать deadlocks, доступно не так много помощи (в форме инструментов). В основном это вопрос способности обнаружить deadlock и вручную использовать отладчик для получения информации, необходимой для его разрешения. Существует одна потенциальная техника, позволяющая получить уведомление, если отладчик считает, что может происходить deadlock. Эта техника представлена значением реестра, расположенным здесь:

```
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\CriticalSection-Timeout
```

Это значение задает количество секунд (по умолчанию это число секунд в 30 днях), в течение которых критическая секция может удерживаться до того, как отладчик выведет предупреждение. Для вступления изменений в силу после изменения этой настройки требуется перезагрузка системы. Попробуем это на нашем примере приложения. Начните с изменения значения на 60 (1 минута), перезагрузите систему и снова запустите приложение под отладчиком. Примерно через 60 секунд вы должны увидеть следующий отладочный вывод:

```
0:000> g
Updating database 1
Updating database 2
RTL: Enter Critical Section Timeout (60 secs) 0
RTL: Pid.Tid e9c.eb8, owner tid ea0 Critical Section 01002020 - ContentionCount == 1
RTL: Re-Waiting
RTL: Enter Critical Section Timeout (60 secs) 0
RTL: Pid.Tid e9c.ea0, owner tid eb8 Critical Section 01002008 - ContentionCount == 1
RTL: Re-Waiting
```

Система дает критической секции еще один шанс, повторно ожидая после истечения периода времени. В конечном итоге, однако, отладчик прервет выполнение и покажет следующее:

```
Possible Deadlock in ntdll!RtlInitializeSListHead Lock 10DeadLock!cs_DB2 (01002020)
Possible Deadlock in ntdll!RtlInitializeSListHead Lock 10DeadLock!cs_DB2 (01002020)
!!! second chance !!!
eax=002bff38 ebx=00000000 ecx=7c910833 edx=000000d8 esi=01002020 edi=7c97c140
eip=7c942426 esp=002bff2c ebp=002bffa0 iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
ntdll!RtlInitializeSListHead+0x9416:
7c942426 68ca24947c push 0x7c9424ca
```

Если в вашем приложении есть очень конкретные времена удержания блокировок, эту технику можно использовать для обнаружения возможных deadlocks.

Хотя мы показали deadlock в контексте критической секции, важно отметить, что любой примитив синхронизации может привести к deadlock при неправильном использовании.

Сценарий orphaned critical section 1: исключения

Писать корректно ведущий себя код при наличии исключений может быть пугающей задачей, особенно в сочетании с многопоточностью. Хотя большинство Win32 API не выбрасывают исключений, исключения все же есть, и при их использовании нужно соблюдать осторожность. В этом разделе мы рассмотрим сценарий с приложением, использующим C++ исключения. Код приложения показан в листинге 10.8.

Листинг 10.8

```

#include <windows.h>
#include <stdio.h>
#include <conio.h>
CRITICAL_SECTION g_cs;
class InvalidParameterException{};
VOID Call3rdPartyCode(LPVOID lpParam)
{
    if(lpParam==NULL)
    {
        throw InvalidParameterException();
    }
    //
    // Do some work
    //
}
DWORD WINAPI ThreadProc(LPVOID lpParam)
{
    DWORD dwRet=1;
    try
    {
        EnterCriticalSection(&g_cs);
        Call3rdPartyCode(lpParam);
        LeaveCriticalSection(&g_cs);
    }
    catch(...)
    {
        // Error occurred
        dwRet=0;
    }
    return dwRet;
}
void __cdecl main ( )
{
    DWORD dwId=0;
    HANDLE hThread=NULL;
    InitializeCriticalSection(&g_cs);
    hThread = CreateThread(NULL, 0, ThreadProc, NULL, 0, &dwId);
    if(hThread)
    {
        Sleep(500);
        printf("Acquiring critical section\n");
        EnterCriticalSection(&g_cs);
        //
        // Do some work
        //
        printf("Leaving critical section\n");
        LeaveCriticalSection(&g_cs);
        WaitForSingleObject(hThread, INFINITE);
        CloseHandle(hThread);
    }
    DeleteCriticalSection(&g_cs);
}

```

Исходный код и двоичный файл для листинга 10.8 находятся в следующих папках:

```

Source code: C:\AWD\Chapter10\Exceptions
Binary: C:\AWDBIN\WinXP.x86.chk\10Exception.exe

```

Как видно, код в листинге 10.8 довольно прямолинеен. Главный поток начинает с инициализации глобальной критической секции, после чего создает новый поток. После успешного создания потока он пытается войти в глобальную критическую секцию, чтобы выполнить некоторую работу. Завершив работу, он покидает критическую секцию и ожидает завершения рабочего потока.

Задача рабочего потока - вызвать некоторый сторонний код (возможно, динамически загружаемую DLL) под защитой глобальной критической секции. Код также пытается быть exception-safe, оборачивая вызов в оператор try/catch, пытаясь поймать все выброшенные исключения, а затем возвращает код ошибки, если исключение было выброшено.

Хотя приложение спроектировано очень плохо, оно тем не менее иллюстрирует очень распространенную проблему. Прежде чем углубиться во все разные проблемы приложения, запустим его и посмотрим на конечный результат:

```
C:\AWDBIN\WinXP.x86.chk\10Exception.exe
Acquiring critical section
```

Все, что, по-видимому, происходит, - приложение зависает при попытке приобрести критическую секцию. Посмотрим на состояние потоков процесса, присоединив отладчик к процессу и выгрузив все потоки, как показано в листинге 10.9.

Листинг 10.9

```
0:001> ~*kb
0 Id: 424.ba8 Suspend: 1 Teb: 7ffdf000 Unfrozen
ChildEBP RetAddr Args to Child
0007fe9c 7c90e9c0 7c91901b 000007e8 00000000 ntdll!KiFastSystemCallRet
0007fea0 7c91901b 000007e8 00000000 7c97c140 ntdll!NtWaitForSingleObject+0xc
0007ff28 7c90104b 00002038 010012e5 01002038 ntdll!RtlpWaitForCriticalSection+0x132
0007ff30 010012e5 01002038 000007f4 0000020c ntdll!RtlEnterCriticalSection+0x46
0007ff44 01001459 00000001 00034e50 00033338 10Exception!main+0x65
0007ffc0 7c816fd7 00090000 0007fa0c 7ffd8000 10Exception!mainCRTStartup+0x12f
0007fff0 00000000 0100132a 00000000 78746341 kernel32!BaseProcessStart+0x23
# 1 Id: 424.dfc Suspend: 1 Teb: 7ffde000 Unfrozen
ChildEBP RetAddr Args to Child
002bffb4 7c80b683 00000000 00000000 00000000 kernel32!CtrlRoutine+0xbd
002bfffec 00000000 7c875280 00000000 00000000 kernel32!BaseThreadStart+0x37
```

Листинг 10.9 указывает, что главный поток ожидает приобретения критической секции, но не может этого сделать. Чтобы выяснить почему, нужно выгрузить рассматриваемую критическую секцию и посмотреть, какую информацию она может дать:

```
0:001> !cs 01002038
-----
Critical section = 0x01002038 (10Exception!g_cs+0x0)
DebugInfo = 0x7c97c8c0
LOCKED
LockCount = 0x1
OwningThread = 0x0000020c
RecursionCount = 0x1
LockSemaphore = 0x7E8
SpinCount = 0x00000000
```

Критическая секция находится в заблокированном состоянии, а владеющий поток - 0x20c. Поскольку ни один из потоков в нашем процессе не имеет ID 0x20c, логично предположить, что в какой-то момент этот загадочный поток приобрел критическую секцию, но так и не освободил ее. Единственный другой поток - наш рабочий поток; может ли он быть виновником? Возможно, поскольку он определенно пытается приобрести критическую секцию:

```
EnterCriticalSection(&g_cs);
Call3rdPartyCode(lpParam);
LeaveCriticalSection(&g_cs);
```

Единственная проблема в том, что он также корректно покидает критическую секцию после вызова стороннего кода; следовательно, критическая секция должна быть в хорошем состоянии. Хотя это может быть нашим первым впечатлением, более внимательный взгляд на код обнаруживает, казалось бы, невинный блок try/catch, окружающий код критической секции. По сути, поскольку поток вызывает некоторый сторонний код, он хочет сделать все возможное, чтобы защититься от кода, выбрасывающего исключение любого типа; поэтому он пытается поймать все исключения, которые могут из него выйти. Хотя можно поспорить, что это не лучший способ защиты, на горизонте маячит еще более серьезная проблема. Что на самом деле происходит, если сторонний код выбрасывает исключение? Выполняется фильтр catch-all, где он устанавливает возвращаемое значение, указывающее на ошибку, а затем поток просто завершается. Покинули ли мы критическую секцию, которая была приобретена перед вызовом стороннего кода? Совершенно нет!

Единственный код, который выполняется в этом сценарии, - фильтр catch, и в итоге мы получаем orphaned critical section. Я оставляю читателю в качестве упражнения проверить в отладчике, что именно это и происходит. Подсказка: вспомните кажущийся недопустимым ID потока, которым владеет критическая секция с точки зрения главного потока.

Теперь мы знаем, почему приложение зависло: рабочий поток оставил критическую секцию orphaned во время своей работы. Помимо плохой попытки защититься, ловя все исключения, возникает вопрос удержания блокировки при вызове некоторого стороннего кода. Безопасно ли это делать? В общем случае ответ - нет. Поскольку вы не представляете, что может сделать этот сторонний код, удержание критической секции во время его вызова может привести к другим разрушительным проблемам. Представьте, что сторонний код пытается вызвать обратно некоторый API, который требует приобретения той же критической секции. Если API не спроектирован для обработки reentry, возникает deadlock. Как общее практическое правило, разработчики должны проявлять крайнюю осторожность при работе со сторонним кодом и при попытке защитить свой код от всех разных неприятностей, которые могут из этого следовать.

Допустим, разработчик, написавший код в листинге 10.9, твердо против каких-либо существенных изменений. Можно ли сделать что-то, чтобы он хотя бы вел себя немного лучше? Безусловно! Разработчик может убедиться, что критическая секция корректно освобождается даже при наличии исключений, гарантировав ее освобождение в коде обработки исключений. Если приложение становится все сложнее, разбрасывание вызовов LeaveCriticalSection повсюду превращается в кошмар. Один возможный способ эффективнее устранить необходимость разбрасывать эти вызовы - позволить самому языку протянуть руку помощи. Помните, что когда исключение выбрасывается (и не перехватывается), компилятор генерирует код раскрутки стека, позволяющий исключению распространяться вниз по трассе стека. Если мы можем каким-то образом внедрить процесс освобождения критической секции в эту раскрутку стека, мы можем гарантировать, что она будет освобождена. В рамках раскрутки стека компилятор генерирует код, очищающий любые переменные, определенные внутри области видимости. Поскольку критическая секция - просто структура без связанного кода, компилятор ничего с ней не делает. Однако мы можем заставить его что-то сделать, обернув функциональность критической секции в класс, время жизни которого тесно связано со временем жизни критической секции. Листинг 10.10 показывает базовый пример класса критической секции.

Листинг 10.10

```
class CCriticalSection
{
public:
    CCriticalSection(CRITICAL_SECTION* pCs)
    {
        m_pCs = pCs;
        if(m_pCs)
        {
            EnterCriticalSection(m_pCs);
        }
    }
    ~CCriticalSection()
    {
        if(m_pCs)
        {
            LeaveCriticalSection(m_pCs);
        }
    }
private:
    CRITICAL_SECTION* m_pCs;
};
```

Класс CCriticalSection в листинге 10.10 - довольно минималистичная версия класса auto-critical section, и его легко расширить для предоставления более продвинутой функциональности, но он

служит иллюстрацией силы оборачивания входа в критические секции и выхода из них в более автоматизированную форму. Чтобы показать, как можно использовать этот класс, функцию ThreadProc теперь можно переписать следующим образом:

```
DWORD WINAPI ThreadProc(LPVOID lpParam)
{
    DWORD dwRet=1;
    CCriticalSection cs(&g_cs);
    try
    {
        //EnterCriticalSection(&g_cs);
        Call3rdPartyCode(lpParam);
        //LeaveCriticalSection(&g_cs);
    }
    catch(...)
    {
        // Error occurred
        dwRet=0;
    }
    return dwRet;
}
```

Как видно, явные вызовы входа в критическую секцию и выхода из нее теперь заменены экземпляром класса CCriticalSection, инициализированным интересующей критической секцией. Во время конструирования выполняется вход в критическую секцию, а во время разрушения - выход из нее. При наличии исключений это работает прекрасно, поскольку компилятор гарантирует, что любые локальные переменные будут уничтожены как часть раскрутки стека.

Помимо использования самого языка, Application Verifier предоставляет целый набор тестов критических секций, которые можно использовать, включив настройку теста Basics. Запустите Application Verifier и включите настройку Basics для предыдущего примера приложения (до использования auto-класса CCriticalSection). После включения запустите приложение под отладчиком и посмотрите на результат:

```
0:000> g
ModLoad: 5cb70000 5cb96000 C:\WINDOWS\system32\ShimEng.dll
(8fc.4bc): C++ EH exception - code e06d7363 (first chance)
=====
VERIFIER STOP 00000200 : pid 0x8FC: Thread cannot own a critical section.
000004BC : Thread ID.
01002038 : Critical section address.
7C97CC00 : Critical section debug information address.
00000000 : Critical section initialization stack trace.
=====
This verifier stop is continuable.
After debugging it use `go' to continue.
=====
(8fc.4bc): Break instruction exception - code 80000003 (first chance)
eax=00000000 ebx=003c5638 ecx=7c91eb05 edx=00e6fa9f esi=00000000 edi=000004bc
eip=7c901230 esp=00e6fb58 ebp=00e6fd24 iopl=0 nv up ei pl nz na pe nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000202
ntdll!DbgBreakPoint:
7c901230 cc int 3
```

Когда поток собирается завершиться, Application Verifier отслеживает все заблокированные критические секции, которыми владеет этот поток, и прерывает выполнение, если такие найдены. Это позволяет разработчику внимательнее посмотреть, почему поток, владеющий критической секцией, завершается без ее освобождения. Хорошая сторона включения Application Verifier в том, что он ловит множество проблем синхронизации в момент их возникновения, а не после, делая отладку намного, намного проще.

Сценарий orphaned critical section 2: завершение потока

Одно из самых типичных требований многопоточного приложения - возможность очень эффективно завершить любой заданный рабочий поток. Причиной может быть отмена пользователем конкретного запроса, который обслуживал рабочий поток, или завершение приложения, при котором нужно быстро остановить все рабочие потоки процесса. Один очень простой и соблазнительный способ сделать это - вызвать API `TerminateThread`, который мгновенно останавливает поток. Если все потоки независимы друг от друга (то есть не разделяют данные и выполняют очень простую работу), вреда быть не должно, верно? Ответ: возможно. Все зависит от того, какая работа выполняется в рабочем потоке. Завершение даже самого простого потока может привести к разрушительным результатам. Чтобы проиллюстрировать это, приложение, показанное в листинге 10.11, создает один рабочий поток, который сидит в плотном цикле, выделяя память, используя эту память и освобождая ее. Затем главный поток завершает рабочий поток (с помощью API `TerminateThread`) и выполняет некоторую финальную работу перед завершением.

Листинг 10.11

```
#include <windows.h>
#include <stdio.h>
#include <conio.h>
DWORD WINAPI ThreadProc( LPVOID lpParam )
{
    BYTE* pData=NULL;
    for(;;)
    {
        pData=(BYTE*) HeapAlloc(GetProcessHeap(), 0, 10000);
        //
        // Use memory
        //
        HeapFree(GetProcessHeap(), 0, pData);
    }
    printf("Exiting thread\n");
    return 1;
}
void __cdecl main ( )
{
    DWORD dwId=0;
    HANDLE hThread=NULL;
    hThread = CreateThread(NULL, 0, ThreadProc, NULL, 0, &dwId);
    if(hThread)
    {
        BYTE* pData=NULL;
        Sleep(500);
        printf("Terminating worker thread...\n");
        TerminateThread(hThread, 0);
        pData = (BYTE*) HeapAlloc(GetProcessHeap(), 0, 10000);
        if(pData)
        {
            //
            // Use memory
            //
            HeapFree(GetProcessHeap(), 0, pData);
        }
        WaitForSingleObject(hThread, INFINITE);
        CloseHandle(hThread);
    }
}
```

Исходный код и двоичный файл для листинга 10.11 находятся в следующих папках:

Source code: C:\AWD\Chapter10\TerminateThread
Binary: C:\AWDBIN\WinXP.x86.chk\10TermThread.exe

Выполните это приложение следующей командной строкой:

C:\AWDBIN\WinXP.x86.chk\10TermThread.exe

Обратите внимание, что приложение зависит от таймингов. При запуске приложения вы можете обнаружить, что оно успешно завершилось. Если это произошло, перезапустите приложение несколько раз, и в конечном итоге увидите, что приложение входит в состояние, в котором никогда не завершается и просто выглядит зависшим. Рассмотрим подробнее, присоединив отладчик к процессу и выгрузив все потоки и связанные стеки, как показано в листинге 10.12.

Листинг 10.12

```
...
...
...
0:001> ~*kb
0 Id: 404.9dc Suspend: 1 Teb: 7ffdd000 Unfrozen
ChildEBP RetAddr Args to Child
0006fc64 7c90e9c0 7c91901b 000007f4 00000000 ntdll!KiFastSystemCallRet
0006fc68 7c91901b 000007f4 00000000 00000000 ntdll!NtWaitForSingleObject+0xc
0006fcf0 7c90104b 00080608 7c911320 00080608 ntdll!RtlpWaitForCriticalSection+0x132
0006fcf8 7c911320 00080608 00090000 0007fa0c ntdll!RtlEnterCriticalSection+0x46
0006ff24 01001236 00080000 00000000 00002710 ntdll!RtlAllocateHeap+0x2f0
0006fff4 010013a2 00000001 00263b50 00262bb0 10TermThread!main+0x76
0006fffc 7c816fd7 00090000 0007fa0c 7ffde000 10TermThread!mainCRTStartup+0x12f
0006ffff 00000000 01001273 00000000 78746341 kernel32!BaseProcessStart+0x23
# 1 Id: 404.870 Suspend: 1 Teb: 7ffdc000 Unfrozen
ChildEBP RetAddr Args to Child
002fffb4 7c80b683 00000000 77d4882a 00000000 kernel32!CtrlRoutine+0xbd
002fffec 00000000 7c875280 00000000 00000000 kernel32!BaseThreadStart+0x37
```

Первый поток в листинге 10.12 определенно выглядит интересно, поскольку, похоже, ожидает критическую секцию. Поскольку в нашем приложении мы не используем никаких критических секций, вы можете спросить, почему поток вообще ожидает какую-то из них. Чтобы ответить на этот вполне разумный вопрос, нужно получить базовую информацию о потоке. Это один из наших потоков? Если да, что делает поток? По самым нижним фреймам потока видно, что это действительно наш главный поток и что он пытается выделить память из кучи. Это хорошо совпадает с нашим исходным листингом 10.11. Поднимаясь вверх по стеку, мы видим, что функция `RtlAllocateHeap` пытается войти в критическую секцию, что в конечном итоге приводит к вызову `WaitForSingleObject`. На этом этапе безопасное предположение состоит в том, что критической секцией уже владеет другой поток, но, как всегда, нужно проверить теории. Чтобы получить более подробную информацию о рассматриваемой критической секции, сначала нужно найти ее адрес. Используем команду расширения `!cs` с этим адресом и посмотрим, похожа ли она на хорошую критическую секцию:

```
0:001> !cs 00080608
-----
Critical section = 0x00080608 (+0x80608)
DebugInfo = 0x7c97c500
LOCKED
LockCount = 0x1
OwningThread = 0x00000224
RecursionCount = 0x1
LockSemaphore = 0x7F4
SpinCount = 0x00000000
```

Строка `LOCKED` указывает, что критическая секция удерживается. Значение поля `OwningThread`, равное `0x224`, указывает, что критическую секцию удерживает поток с ID `0x224`. Поле `LockSemaphore` имеет значение `0x7F4`. Поскольку мы знаем, что `LockSemaphore` соответствует дескриптору события, можно использовать команду расширения `!handle`, чтобы убедиться, что это событие:

```
0:001> !handle 0x7F4
Handle 7f4
Type Event
```

На этом этапе мы можем уверенно сказать, что работаем с тем, что похоже на неповрежденную критическую секцию, но главный вопрос остается: какой поток удерживает эту критическую секцию? В нашем процессе выполняются только два потока, и ни один из них не имеет ID потока 0x224. Из листинга 10.11 мы знаем, что главный поток завершил рабочий поток, поэтому вполне разумно предположить, что наш рабочий поток имел ID 0x224. Чтобы проверить это предположение, можно перезапустить приложение под отладчиком, установить точку останова в начале процедуры рабочего потока (ThreadProc) и продолжить выполнение. После срабатывания точки останова мы проверяем ID потока и продолжаем выполнение, пока зависание не проявится снова. Затем выгружаем критическую секцию и убеждаемся, что ID владеющего потока совпадает с ID рабочего потока (см. листинг 10.13).

Листинг 10.13

```
0:000> bp 10TermThread!ThreadProc
0:000> g
Breakpoint 0 hit
eax=00000000 ebx=00000000 ecx=002bffb0 edx=7c90eb94 esi=000822c8 edi=00680066
eip=01001180 esp=002bffb8 ebp=002bfffec iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
10TermThread!ThreadProc:
01001180 8bff mov edi,edi
0:001> ~
0 Id: ea8.ea0 Suspend: 1 Teb: 7ffdf000 Unfrozen
. 1 Id: ea8.e68 Suspend: 1 Teb: 7ffde000 Unfrozen
0:001> g
Terminating worker thread...
(ea8.fa4): Control-C exception - code 40010005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=002ffff38 ebx=00000000 ecx=002ffff04 edx=7c90eb94 esi=00000000 edi=00000002
eip=7c87533d esp=002fff2c ebp=002ffffb4 iopl=0 nv up ei pl zr na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
kernel32!CtrlRoutine+0xbd:
7c87533d 834dfcff or dword ptr [ebp-0x4],0xffffffff ss:0023:002ffffb0=00000000
0:001> ~*kb
0 Id: ea8.ea0 Suspend: 1 Teb: 7ffdf000 Unfrozen
ChildEBP RetAddr Args to Child
0006fc64 7c90e9c0 7c91901b 000007f4 00000000 ntdll!KiFastSystemCallRet
0006fc68 7c91901b 000007f4 00000000 00000000 ntdll!NtWaitForSingleObject+0xc
0006fcf0 7c90104b 00080608 7c911320 00080608 ntdll!RtlpWaitForCriticalSection+0x132
0006fcf8 7c911320 00080608 00090000 0007fa0c ntdll!RtlEnterCriticalSection+0x46
0006ff24 01001236 00080000 00000000 00002710 ntdll!RtlAllocateHeap+0x2f0
0006ff44 010013a2 00000001 00263b78 00262bb0 10TermThread!main+0x76
0006ff60 7c816fd7 00090000 0007fa0c 7ffdf900 10TermThread!mainCRTStartup+0x12f
0006ff60 00000000 01001273 00000000 78746341 kernel32!BaseProcessStart+0x23
# 1 Id: ea8.fa4 Suspend: 1 Teb: 7ffde000 Unfrozen
ChildEBP RetAddr Args to Child
002ffffb4 7c80b683 00000000 0000000d 0199fe64 kernel32!CtrlRoutine+0xbd
002ffffec 00000000 7c875280 00000000 00000000 kernel32!BaseThreadStart+0x37
0:001> !cs 00080608
-----
Critical section = 0x00080608 (+0x80608)
DebugInfo = 0x7c97c500
LOCKED
LockCount = 0x1
OwningThread = 0x00000e68
RecursionCount = 0x1
LockSemaphore = 0x7f4
SpinCount = 0x00000000
```

Как видно из отладочного вывода в листинге 10.13, ID рабочего потока - e68. Критическая секция, удерживаемая бесконечно, также указывает, что поток, владеющий критической секцией, - e68. Поскольку рабочий поток был завершен до того, как получил шанс освободить критическую секцию, в итоге мы получаем то, что известно как orphaned critical section.

Этот сценарий служит иллюстрацией того, что разработчики должны быть крайне осторожны при использовании Win32 API. Пример кода, который мы только что отладили, не содержал ни одной критической секции, но нам все равно удалось остановить его намертво, заблокировавшись на критической секции. Оказывается, критическая секция принадлежит менеджеру кучи, и когда главный поток завершил рабочий поток, рабочий поток находился в середине выделения или освобождения памяти. Обычно это заставляет менеджер кучи приобретать критическую секцию. Когда рабочий поток завершился, менеджер кучи так и не получил шанс покинуть критическую секцию. Хотя плотный цикл выделения/освобождения памяти в рабочем потоке является преувеличенным фрагментом кода, он служит иллюстрацией важности понимания ваших API, а также ясно показывает сеанс отладки очень распространенной проблемы. Как общее практическое правило, никогда не следует вызывать `TerminateThread`, если вы точно не знаете, что делает поток.

Чтобы убедиться, что мы помним об опасностях вызова `TerminateThread`, можно использовать Application Verifier. Включив настройку теста `DangerousAPIs` в ветке `Miscellaneous` для нашего приложения, мы можем быть уверены, что Application Verifier перехватит все вызовы API `TerminateThread`. Включение этой настройки теста для нашего примера приложения и запуск под отладчиком дают следующее:

```
0:000> g
Terminating worker thread...
=====
VERIFIER STOP 00000100 : pid 0x9F8: Dangerous call to TerminateThread.
00000B28 : Thread ID for the caller of TerminateThread.
00000000 : Not used.
00000000 : Not used.
00000000 : Not used.
=====
This verifier stop is continuable.
After debugging it use `go` to continue.
=====
(9f8.b28): Break instruction exception - code 80000003 (first chance)
eax=00000000 ebx=003b5470 ecx=7c91eb05 edx=0006fc15 esi=00000000 edi=00000b28
eip=7c901230 esp=0006fd04 ebp=0006fed0 iopl=0         nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000202
ntdll!DbgBreakPoint:
7c901230 cc int 3
```

Как видно из вывода, отладчик прерывает выполнение сразу после обнаружения вызова `TerminateThread` и также показывает предупреждающее сообщение. Кроме того, отображается ID потока-нарушителя.

Осведомленность о DllMain

Dynamic Link Libraries (DLL) являются фундаментальными конструкциями в операционной системе Windows. На высоком уровне DLL - это файл, содержащий исполняемый код и/или данные, которые могут использовать приложения. По сути это очень похоже на статическую библиотеку, но вместо того чтобы заставлять приложение компоноваться со статической библиотекой, приложение свободно использовать DLL динамически. DLL могут загружаться во время загрузки приложения или во время выполнения, чтобы приложение могло выбирать, какой код/данные использовать, в зависимости от того, что выполняется в приложении. Приложение может использовать DLL двумя способами:

- **Implicit linking:** при неявной компоновке приложение, использующее DLL, компоуется с `import library`, предоставленной автором DLL. Затем DLL автоматически загружается при загрузке приложения. Функции, экспортируемые DLL, можно вызывать в приложении так, как если бы они были частью приложения.

- Explicit linking: при явной компоновке приложение должно явно вызывать функции загрузки (LoadLibrary), выгрузки (FreeLibrary) и использования функций DLL (GetProcAddress). Затем функции вызываются через указатели на функции.

Чтобы DLL могла инициализировать и деинициализировать себя, из DLL может экспортироваться специальная функция с именем DllMain. Прототип функции DllMain выглядит так:

```
BOOL WINAPI DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved);
```

Каждый аргумент обсуждается ниже:

- hinstDLL: дескриптор модуля DLL.
- fdwReason: функция DllMain может вызываться по нескольким причинам. Она вызывается, когда DLL загружается в адресное пространство приложения (DLL_PROCESS_ATTACH), когда в приложении создается поток (DLL_THREAD_ATTACH), и, наоборот, когда DLL отсоединяется от процесса (DLL_PROCESS_DETACH) или поток чисто завершается (DLL_THREAD_DETACH).
- lpvReserved: не используется.

Если DLL относительно проста и экспортирует набор функций, которые не зависят от состояния, реализацию DllMain можно опустить. Однако чаще всего реализация предоставляется, поскольку DLL нужно инициализировать себя (например, инициализировать состояние вроде thread local storage). Разработчики, реализующие DllMain, должны знать о нескольких ограничениях реализации. Вместо простого перечисления всех ограничений рассмотрим самую распространенную проблему, встречающуюся при реализации этой функции.

DLL, которую мы используем для иллюстрации этой проблемы (листинг 10.14), - простая DLL, выполняющая некоторую инициализацию DLL. Вместо выполнения ее в главном потоке код в DllMain создает новый поток, отвечающий за инициализацию. Главный поток просто ожидает завершения рабочего потока и возвращается из функции DllMain.

Листинг 10.14

```
#include <windows.h>
#include <stdio.h>
DWORD WINAPI InitDllProc( LPVOID lpParam )
{
    LoadLibrary("xmlprov.dll");
    return 1;
}
BOOL WINAPI DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)
{
    BOOL bRet=FALSE;
    switch(fdwReason)
    {
    case DLL_PROCESS_ATTACH:
    {
        DWORD dwId=0;
        HANDLE hThread=NULL;
        hThread = CreateThread(NULL, 0, InitDllProc, NULL, 0, &dwId);
        if(hThread)
        {
            WaitForSingleObject(hThread, INFINITE);
            CloseHandle(hThread);
            bRet=TRUE;
        }
    }
    break;
    }
    return bRet;
}
```

Исходный код и двоичные файлы для листинга 10.14 находятся в следующих папках:

```
Source code: C:\AWD\Chapter10\DllMain\App and
C:\AWD\Chapter10\DllMain\ModDll
```

```
Binary: C:\AWDBIN\WinXP.x86.chk\10dllmain.exe and  
C:\AWDBIN\WinXP.x86.chk\10moddll.dll
```

Приложение, использующее эту DLL, просто загружает DLL с помощью LoadLibrary. Чтобы запустить приложение, используйте следующую командную строку:

```
C:\AWDBIN\WinXP.x86.chk\10dllmain.exe
```

Учитывая код этого приложения, оно должно завершиться относительно быстро, но вы заметите, что приложение просто зависает. Чтобы лучше понять, что происходит, найдите ID процесса 10dllmain.exe и присоедините к нему отладчик. Результат присоединения отладчика показан в листинге 10.15.

Листинг 10.15

```
...  
...  
ModLoad: 01000000 01003000 C:\AWDBIN\WinXP.x86.chk\10dllmain.exe  
ModLoad: 7c900000 7c9b0000 C:\WINDOWS\system32\ntdll.dll  
ModLoad: 7c800000 7c8f4000 C:\WINDOWS\system32\kernel32.dll  
ModLoad: 77c10000 77c68000 C:\WINDOWS\system32\msvcrt.dll  
ModLoad: 00400000 00404000 C:\AWDBIN\WinXP.x86.chk\10moddll.dll  
Break-in sent, waiting 30 seconds...
```

Вы сразу заметите, что отладчик просто сидит на месте и не прерывает процесс. Последняя строка в листинге 10.15, похоже, подразумевает, что отладчик попытался отправить команду break-in, но не может получить ответ от отлаживаемого процесса. Если подождать рекомендуемые 30 секунд, отладчик в конце концов прервется со следующим уведомлением:

```
WARNING: Break-in timed out, suspending.  
This is usually caused by another thread holding the loader lock  
(498.27c): Wake debugger - code 80000007 (first chance)  
eax=0006f8b0 ebx=00400000 ecx=ffffffff edx=000000d8 esi=000007f4 edi=00000000  
eip=7c90eb94 esp=0006f770 ebp=0006f7d4 iopl=0 nv up ei pl zr na po nc  
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000246  
*** ERROR: Symbol file could not be found. Defaulted to export symbols for C:\WIN-  
DOWS\system32\ntdll.dll -  
ntdll!KiFastSystemCallRet:  
7c90eb94 c3 ret
```

Предупреждение сообщает, что может удерживаться loader lock. Что здесь на самом деле происходит? Поведение отладчика кажется очень странным (он не прерывается и жалуется на loader lock, которого мы никогда не касались в своем коде). Если обратиться к нашему надежному процессу устранения неполадок, следующий шаг - выгрузить все потоки и посмотреть, можем ли мы понять причину зависания. Листинг 10.16 показывает потоки нашего процесса.

Листинг 10.16

```
0:000> ~*kb  
. 0 Id: 498.27c Suspend: 1 Teb: 7ffdd000 Unfrozen  
ChildEBP RetAddr Args to Child  
0006f76c 7c90e9c0 7c8025cb 000007f4 00000000 ntdll!KiFastSystemCallRet  
0006f770 7c8025cb 000007f4 00000000 00000000 ntdll!NtWaitForSingleObject+0xc  
0006f7d4 7c802532 000007f4 ffffffff 00000000 kernel32!WaitForSingleObjectEx+0xa8  
0006f7e8 0040113a 000007f4 ffffffff 00000001 kernel32!WaitForSingleObject+0x12  
0006f804 00401251 00400000 00000001 00000000 10modDll!DllMain+0x4a  
0006f824 7c9011a7 00400000 00000001 00000000 10modDll!_DllMainCRTStartup+0x52  
0006f844 7c91cbab 004011ff 00400000 00000001 ntdll!LdrpCallInitRoutine+0x14  
0006f94c 7c916178 00000000 c0150008 00000000 ntdll!LdrpRunInitializeRoutines+0x344  
0006fbf8 7c9162da 00000000 00083338 0006feec ntdll!LdrpLoadDll+0x3e5  
0006feaf 7c801bb9 00083338 0006feec 0006fecc ntdll!LdrLoadDll+0x230  
0006ff08 7c801d6e 7ffddc00 00000000 00000000 kernel32!LoadLibraryExW+0x18e  
0006ff1c 7c801da4 0100107c 00000000 00000000 kernel32!LoadLibraryExA+0x1f  
0006ff38 01001160 0100107c 0006ffc0 01001298 kernel32!LoadLibraryA+0x94  
0006ffa4 01001298 00000001 00263b08 00262b78 10dllmain!main+0x10  
0006ffc0 7c816fd7 5a99c54d 01c6d533 7ffde000 10dllmain!mainCRTStartup+0x12f  
0006fff0 00000000 01001169 00000000 78746341 kernel32!BaseProcessStart+0x23  
1 Id: 498.f4 Suspend: 1 Teb: 7ffdc000 Unfrozen
```

```

ChildEBP RetAddr Args to Child
002bfc10 7c90e9c0 7c91901b 000007e8 00000000 ntdll!KiFastSystemCallRet
002bfc14 7c91901b 000007e8 00000000 00000000 ntdll!NtWaitForSingleObject+0xc
002bfc9c 7c90104b 0197c0d8 7c927357 7c97c0d8 ntdll!RtlpWaitForCriticalSection+0x132
002bfca4 7c927357 7c97c0d8 002bfd30 020a0018 ntdll!RtlEnterCriticalSection+0x46
002bfd1c 7c90eac7 002bfd30 7c900000 00000000 ntdll!_LdrpInitialize+0xf0
00000000 00000000 00000000 00000000 00000000 ntdll!KiUserApcDispatcher+0x7
2 Id: 498.f90 Suspend: 1 Teb: 7ffdb000 Unfrozen
ChildEBP RetAddr Args to Child
002cfc0c 7c90e9c0 7c91901b 000007e8 00000000 ntdll!KiFastSystemCallRet
002cfc10 7c91901b 000007e8 00000000 00000000 ntdll!NtWaitForSingleObject+0xc
002cfc98 7c90104b 0197c0d8 7c927357 7c97c0d8 ntdll!RtlpWaitForCriticalSection+0x132
002cfca0 7c927357 7c97c0d8 002cfd2c 00000004 ntdll!RtlEnterCriticalSection+0x46
002cfd18 7c90eac7 002cfd2c 7c900000 00000000 ntdll!_LdrpInitialize+0xf0
00000000 00000000 00000000 00000000 00000000 ntdll!KiUserApcDispatcher+0x7

```

Первый поток в листинге 10.16 вполне понятен. Это главный поток нашего приложения, где мы вызываем LoadLibrary для нашей примерной DLL. Вызов проходит через множество системных вызовов, прежде чем, как и ожидалось, попасть в нашу реализацию DllMain. Функция DllMain наконец вызывает WaitForSingleObject, чтобы дождаться завершения нашего рабочего потока. Однако это ожидание, похоже, никогда не удовлетворяется, что подразумевает: рабочий поток либо не выполнялся, либо завис. Судя по другим потокам процесса, мы не видим следов рабочего потока; следовательно, делаем вывод, что он никогда не стартовал. Пока что сеанс отладки не кажется осмысленным. Последние два потока тоже выглядят очень странно. У нас есть два потока, которые пытаются войти в критическую секцию. Кроме того, самые нижние фреймы этих потоков выглядят немного странно:

```

002cfca0 7c927357 7c97c0d8 002cfd2c 00000004 ntdll!RtlEnterCriticalSection+0x46
002cfd18 7c90eac7 002cfd2c 7c900000 00000000 ntdll!_LdrpInitialize+0xf0
00000000 00000000 00000000 00000000 00000000 ntdll!KiUserApcDispatcher+0x7

```

Если выгрузить критическую секцию, которая приобретается, видно, что она называется loader lock:

```

0:000> !cs 7c97c0d8
-----
Critical section = 0x7c97c0d8 (ntdll!LdrpLoaderLock+0x0)
DebugInfo = 0x7c97c100
LOCKED
LockCount = 0x3
OwningThread = 0x0000027c
RecursionCount = 0x2
LockSemaphore = 0x7E8
SpinCount = 0x00000000

```

Критическая секция находится в заблокированном состоянии, а поток, владеющий критической секцией, имеет ID потока 0x27c. Поток, соответствующий этому ID, оказывается нашим главным потоком, который сейчас инициализирует загруженную нами DLL.

Почему два потока ожидают освобождения loader lock? Более того, почему стеки потоков выглядят так странно? Обычно поток пользовательского режима начинается с фрейма, похожего на следующий:

```
kernel32!BaseThreadStart
```

В нашем случае начальная точка потока, похоже, находится во фрейме, похожем на:

```
ntdll!KiUserApcDispatcher
```

Мы видим пример того, как поток пользовательского режима на самом деле появляется в Windows. Рисунок 10.3 иллюстрирует высокоуровневый процесс создания нового потока. Обратите внимание, что многие детали, связанные с созданием потоков, в значительной степени недокументированы, и поэтому рисунок 10.3 следует рассматривать как близкое высокоуровневое приближение.

Когда поток создается в Windows, он не начинает "автоматически" выполняться из функции kernel32!BaseThreadStart; вместо этого он всегда начинает жизнь как user mode APC

(Asynchronous Procedure Call). На высоком уровне, когда Windows получает уведомление о создании нового потока, user mode APC ставится в APC-очередь нового потока. Диспетчер APC доставляет APC новому потоку, который в ответ выполняет набор инициализационной работы перед переходом в состояние, которое мы обычно связываем с потоком пользовательского режима (то есть видим `kernel32!BaseThreadStart` в трассе стека). Именно это мы и видим с нашими двумя потоками. Они все еще находятся в APC-состоянии, выполняя инициализационную работу.

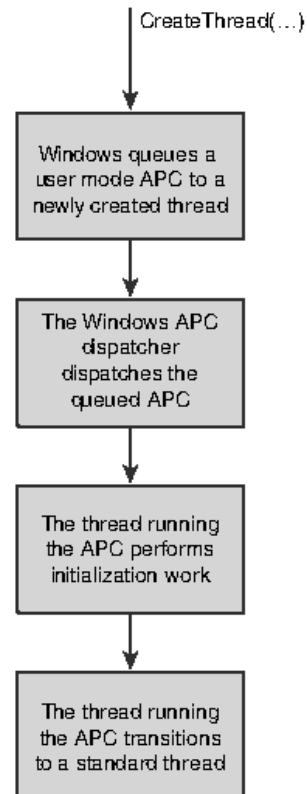


Figure 10.3 High-level overview of the thread creation process

Рисунок 10.3. Высокоуровневый обзор процесса создания потока

Теперь это объясняет, почему по крайней мере один из наших потоков находится в APC-состоянии. Похоже, он выполняет инициализационную работу и застревает на попытке приобрести loader lock. Прежде чем идти по пути расследования того, почему он не может приобрести эту блокировку, следует проявить должную осмотрительность (помните: чем больше вы знаете обо всех потоках процесса, тем выше шанс добраться до первопричины того, что идет не так); нам также нужно выяснить, почему финальный третий поток застрял в APC-состоянии. В конце концов, в нашем приложении должно быть два потока: (1) главный поток и (2) рабочий поток. Вспомните предыдущее обсуждение высокоуровневого процесса и то, что при прерывании в отладчик в нашем процессе всегда будет "debugger" thread, поток, отвечающий за выдачу команды debug break. Так вот, финальный поток в нашем процессе - именно этот поток отладчика. Хотя, что интересно, он тоже выглядит неправильно. Трасса стека, которую мы ожидаем увидеть, должна содержать фрейм debug break. Вместо того чтобы отмахнуться от этого потока как от "не нашего", попробуем понять, откуда он взялся. Отладчик, который пытается прервать целевой процесс, может сделать это одним из двух способов:

- Инжектировать remote thread в целевой процесс. Этот поток затем отвечает за выдачу debug break, в конечном итоге заставляя целевой процесс остановить выполнение. После того как remote thread был инжектирован отладчиком, отладчик просто ожидает завершения потока.

Однако время ожидания, заданное отладчиком, не бесконечно, поскольку некоторые ситуации могут помешать созданию remote thread. Точнее, время ожидания равно 30 секундам.

- Если remote thread не завершил создание через 30 секунд, отладчик выбирает другой подход и просто приостанавливает все потоки процесса и прерывает выполнение.

Теперь должно начать становиться понятно, почему мы видим третий поток. Третий поток на самом деле является remote thread, инжектированным отладчиком, чтобы попытаться остановить выполнение. Поскольку поток застрял в APC-режиме, инициализируя себя (очень похоже на наш рабочий поток), вывод, который показал отладчик (Break-in sent, waiting for 30 seconds... и WARNING: Break-in timed out, suspending), тоже имеет смысл. Итак, похоже, что каждый поток, кроме главного, застревает очень рано в APC-режиме из-за этого загадочного loader lock, который создает хаос в нашем процессе, заставляя все остановиться. Единственный оставшийся вопрос - что такое loader lock и почему наш главный поток, похоже, удерживает его. Ранее мы обсуждали общую концепцию DLL и то, как DllMain можно использовать для инициализации DLL. Чтобы Windows поддерживала процесс загрузки и выгрузки DLL структурно целостным, она использует loader lock для сериализации любого доступа к функциям DllMain. Это делается, чтобы предотвратить множество проблем, связанных с параллельным выполнением функций DllMain. Windows приобретает блокировку перед вызовом любой функции DllMain и затем освобождает блокировку, когда DllMain завершает выполнение.

Теперь у нас есть все элементы, чтобы объяснить, что на самом деле происходит. Главный поток выполняет код в DllMain, что косвенно означает, что loader lock удерживается. Тем временем рабочий поток пытается начать выполнение, но застревает при попытке вызвать DllMain (с DLL_THREAD_ATTACH), и в итоге мы получаем deadlock.

Как вы видели, реализовывать DllMain может быть довольно непросто. Помимо deadlocks, могут проявляться и другие проблемы, например access violations, если порядок загрузки не соблюдается. Как общее практическое правило, DllMain должна выполнять как можно меньше работы (lazy initialization) и быстро возвращаться. Что значит "мало работы"? Хороший вопрос. Нашу реализацию можно считать малой, потому что она не делает ничего, кроме порождения потока, который пытается загрузить другую DLL. Microsoft написала whitepaper по вопросу реализаций DLL, и его можно найти по следующему адресу:

https://www.microsoft.com/whdc/driver/kernel/DLL_bestprac.mspx

Можно принять и другие меры предосторожности, чтобы не попасть в проблемную ситуацию при реализации DllMain. Прежде всего, нужно знать API, которые вы вызываете из DllMain. Даже если некоторые API выглядят достаточно безобидно, в конечном итоге они могут вызвать API, который создаст проблемы. К сожалению, фиксированного списка опасных API, которых следует избегать, нет; однако надежда есть. Она приходит в форме Application Verifier. Application Verifier способен обнаруживать многие наиболее распространенные проблемы, встречающиеся в реализациях DllMain. Нужно использовать настройку теста Basics в Application Verifier. Если включить эту настройку для нашего примера приложения и снова запустить приложение под отладчиком, вы увидите вывод, похожий на следующий:

```
0:000> g
ModLoad: 00e30000 00e34000 C:\AWDBIN\WinXp.x86.chk\10moddll.dll
=====
VERIFIER STOP 00000304 : pid 0x5f4: Waiting on a thread handle in DllMain.
00000780 : Thread handle.
00000000 : Not used.
00000000 : Not used.
00000000 : Not used.
=====
This verifier stop is continuable.
After debugging it use `go' to continue.
=====
(5f4.5fc): Break instruction exception - code 80000003 (first chance)
```

```
eax=00000000 ebx=003b5c78 ecx=7c91eb05 edx=0006f263 esi=00000000 edi=00000780
eip=7c901230 esp=0006f36c ebp=0006f538 iopl=0  nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000202
ntdll!DbgBreakPoint:
7c901230 cc int 3
```

Отладчик прервался, как только обнаружил вызов `WaitForSingleObject` в функции `DllMain`. Он также предоставляет некоторую вспомогательную информацию, например дескриптор потока, которого ожидают.

У `Application Verifier` есть масса действительно потрясающих настроек тестов, и его следует широко использовать во время разработки. Поверьте: позволив `Application Verifier` ловить эти проблемы на раннем этапе, вы сэкономите массу времени на отладку.

Соперничество за блокировку

`Lock contention` - интересный сценарий, который может резко повлиять на производительность и масштабируемость вашего приложения. `Lock contention` означает ситуацию, в которой большое количество потоков соревнуется за одну блокировку. В зависимости от того, сколько работы выполняется в защищенной области кода, количество потоков, ожидающих приобретения блокировки, может становиться все больше и больше. До `Windows 2003 Server SP1` критические секции использовали `fair semantics`; `fair` означает, что соблюдалась семантика `First-In-First-Out`, по сути гарантирующая сохранение порядка блокировки критической секции. Поток, который ждал дольше всех, также был следующим в очереди на приобретение критической секции. Чтобы `fair locking scheme` работала, выход из критической секции должен включать передачу владения от владеющего потока к следующему потоку в очереди на приобретение. Передача владения увеличивает время, которое владеющий поток проводит в критической секции, на порядок. Например, представьте, что фрагмент кода, защищенный критической секцией, делает не больше, чем увеличивает счетчик ссылок:

```
EnterCriticalSection(&cs);
refCount++;
LeaveCriticalSection(&cs);
```

Время, проведенное в критической секции, относительно мало, потому что увеличение `DWORD` на единицу - довольно дешевая операция. Кроме того, представьте, что множество потоков ожидало эту критическую секцию. Теперь, когда владеющий поток покидает критическую секцию, он должен соблюдать `fair semantics` блокировки и передать владение следующему потоку в очереди. Объем работы, необходимый для передачи владения, довольно велик по сравнению с объемом работы, фактически выполненной внутри критической секции (инкремент переменной). Потенциально нужно разбудить другой поток, что приводит к переключению контекста до того, как следующий поток приобретет критическую секцию. Добавленное время, необходимое для соблюдения `fair locking semantics`, приводит к довольно большой накладной нагрузке, в течение которой еще больше потоков получают возможность встать в очередь. Рисунок 10.4 иллюстрирует работу, необходимую потоку для входа, выполнения защищенной области кода и выхода из критической секции.

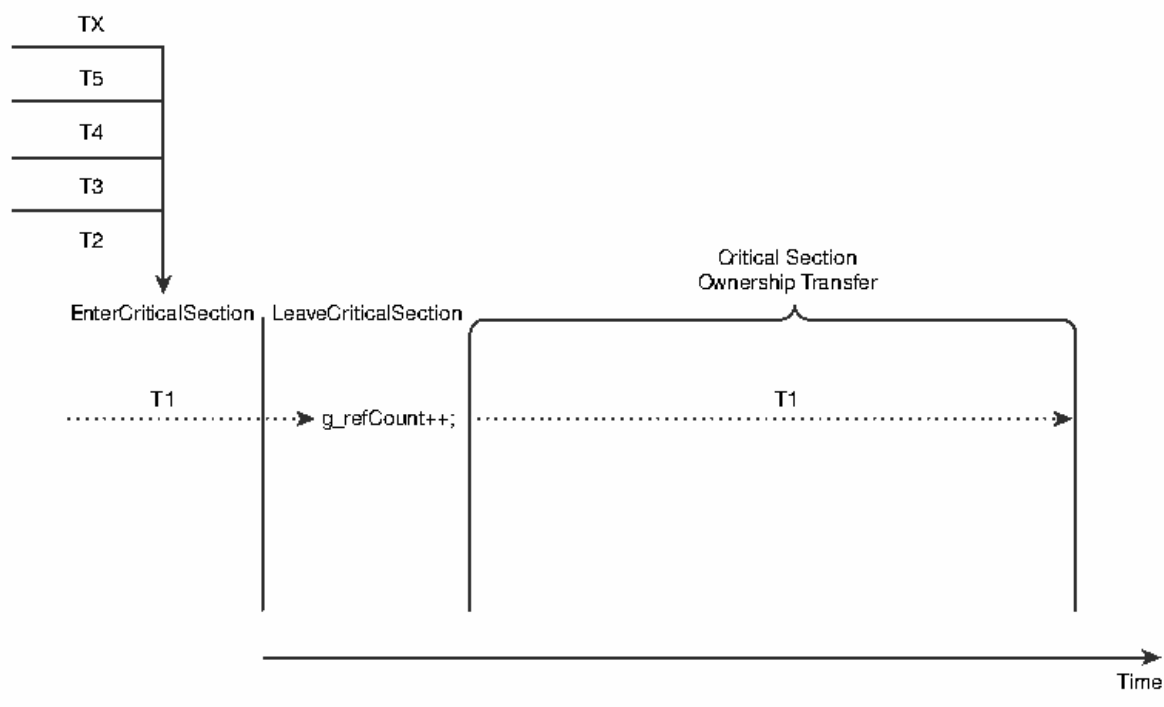


Figure 10.4 Processing involved with entering and leaving a critical section

Рисунок 10.4. Обработка при входе в критическую секцию и выходе из нее

Рисунок 10.4 показывает, как очень небольшая область кода защищена критической секцией и что происходит, когда первый поток (T1) входит в критическую секцию, выполняет простой оператор инкремента и покидает критическую секцию. Как видно, время, потраченное на выполнение соответствующего инкремента переменной счетчика ссылок, намного меньше времени, потраченного на передачу владения критической секцией.

Иллюстрируемая проблема - входящая скорость потоков, пытающихся приобрести критическую секцию, больше времени, необходимого для входа, обработки и выхода из критической секции (включая работу, необходимую для передачи владения), - известна как *lock convoy*. Lock convoys могут резко снижать производительность приложения из-за чрезмерного объема работы, необходимого для управления *fair critical section*.

Как распознать, действительно ли происходит проблема *lock convoy*? На высоком уровне есть пара симптомов, на которые стоит обратить внимание:

- Чрезмерное переключение контекста. Помните, что для честной передачи владения критической секцией другой критической секции она должна разбудить новый поток. Сам акт пробуждения потока может привести к переключению контекста.
- Деградация производительности на пути кода, включающем схему блокировки.

Если вы подозреваете, что *lock convoy* может быть проблемой в вашем приложении, следующий шаг - внимательно посмотреть на приложение с помощью отладчиков и убедиться, что именно это и происходит. Листинг 10.17 показывает пример приложения, страдающего от проблемы *lock convoy*.

Листинг 10.17

```
#include <windows.h>
#include <stdio.h>
CRITICAL_SECTION g_cs;
DWORD g_refCount;
```



```

DWORD WINAPI ThreadProc( LPVOID lpParam )
{
while(true)
{
EnterCriticalSection(&g_cs);
g_refCount++;
LeaveCriticalSection(&g_cs);
}
return 1;
}
void __cdecl main ( )
{
DWORD dwId=0;
HANDLE hThread[MAXIMUM_WAIT_OBJECTS];
DWORD dwRet=0;
g_refCount=0;
InitializeCriticalSection(&g_cs);
for(int i=0; i<MAXIMUM_WAIT_OBJECTS; i++)
{
hThread[i] = CreateThread(NULL, 0, ThreadProc, NULL, 0, &dwId);
if(!hThread[i])
{
for(int j=0;j<i;j++)
{
CloseHandle(hThread[j]);
}
return;
}
}
WaitForMultipleObjects(MAXIMUM_WAIT_OBJECTS, hThread, TRUE, INFINITE);
for(int i=0; i<MAXIMUM_WAIT_OBJECTS; i++)
{
CloseHandle(hThread[i]);
}
}

```

Исходный код и двоичный файл для листинга 10.17 находятся в следующих папках:

```

Source code: C:\AWD\Chapter10\LockConvoy
Binary: C:\AWDBIN\WinXP.x86.chk\10lockconv.exe

```

Код в листинге 10.17 довольно тривиален. Он создает некоторое количество потоков, где каждый поток сидит в плотном цикле, увеличивая переменную счетчика ссылок. Инкремент счетчика ссылок защищен критической секцией. Если запустить это приложение под отладчиком, видно, что оно содержит большое количество потоков, большинство из которых ожидают приобретения критической секции, защищающей счетчик ссылок. Обратите внимание: это приложение нужно запускать на версии Windows до Windows 2003 Server SP1.

```

61 Id: a48.a28 Suspend: 1 Teb: 7ff79000 Unfrozen
ChildEBP RetAddr Args to Child
0128ff14 7c90e9c0 7c91901b 0000073c 00000000 ntdll!KiFastSystemCallRet
0128ff18 7c91901b 0000073c 00000000 7c97c140 ntdll!NtWaitForSingleObject+0xc
0128ffa0 7c90104b 00002008 01001169 01002008 ntdll!RtlpWaitForCriticalSection+0x132
0128ffa8 01001169 01002008 0128ffec 7c80b683 ntdll!RtlEnterCriticalSection+0x46
0128ffb4 7c80b683 00000000 7c910833 00000000 10LockConv!ThreadProc+0x19
0128ffec 00000000 01001150 00000000 00000000 kernel32!BaseThreadStart+0x37

```

Пока все хорошо. Мы действительно ожидаем, что большое количество потоков будет ожидать эту весьма спорную блокировку. Теперь попробуем выяснить, какой поток владеет критической секцией. Помните, первый параметр RtlEnterCriticalSection - это адрес рассматриваемой критической секции.

```

0:065> !cs 01002008
-----
Critical section = 0x01002008 (10LockConv!g_cs+0x0)
DebugInfo = 0x7c97c8c0
LOCKED
LockCount = 0x1C

```

```
OwningThread = 0x00000000
RecursionCount = 0x0
LockSemaphore = 0x73C
SpinCount = 0x00000000
```

Поле LockCount выглядит корректно (0x1C) и указывает, что критическая секция заблокирована и что большое количество потоков ожидает ее приобретения. Поле OwningThread, с другой стороны, выглядит неверно. Оно подразумевает, что критическую секцию удерживает поток с ID потока 0. Поток с ID 0 никогда не будет существовать в процессе. Что происходит? То, что вы видите, по сути является характерным признаком происходящего lock convoy. Помните, что в ситуации lock convoy владеющий поток передает владение критической секцией при выходе из нее. В это время критическая секция все еще считается заблокированной, но код освобождения очищает поле OwningThread, пока новый поток не войдет в критическую секцию; в этот момент оно корректно сбрасывается.

Поскольку lock convoys могут вызывать серьезные проблемы в вашем коде, какие есть техники смягчения? К сожалению, техники смягчения для этой проблемы найти непросто. Лучший ответ - просто перепроектировать приложение и убрать сильно спорную блокировку. К сожалению, это не всегда возможно. Еще одно возможное смягчение - полагаться на spin counts при попытке приобрести критическую секцию. Эта техника может резко уменьшить время, необходимое для передачи владения (поскольку переключение контекста не требуется), и convoy может быть разрушен. К сожалению, этот подход работает только на многопроцессорных машинах.

Проблема lock convoys была признана достаточно важной, чтобы команда Windows решила кардинально изменить работу критических секций в Windows 2003 Server SP1. Поскольку фундаментальная проблема lock convoys состоит в том, что владеющий поток должен передать владение следующему потоку в очереди (чтобы соблюсти базовую цель дизайна fair lock), Microsoft решила изменить дизайн критической секции и убрать требование fairness. Если поток не должен передавать владение, после освобождения критической секции будет потрачено минимальное время, тем самым устраняя проблему lock convoy. Побочный эффект удаления fairness состоит в том, что поток, пробужденный для входа в критическую секцию, должен конкурировать с совершенно новыми входящими потоками, также пытающимися за нее соревноваться, что потенциально приводит к starvation потока. Тем не менее проблема lock convoy была признана достаточно важной и перевесила потенциальный thread starvation. Чтобы поддержать это изменение, нужно было изменить структуру критической секции, и согласованный подход состоял в использовании поля LockCount для добавления дополнительной информации, требуемой для unfair lock. Точнее, LockCount изменился, как показано на рисунке 10.5.

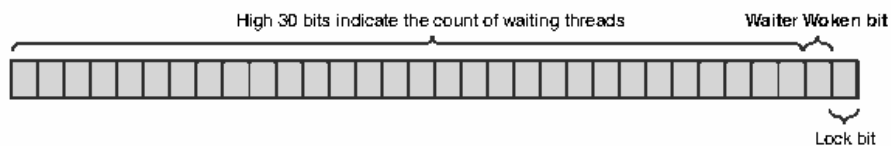


Figure 10.5 The LockCount field of the new unfair critical sections

Рисунок 10.5. Поле LockCount новых unfair critical sections

Все биты в новом поле LockCount теперь также хранятся в инвертированном состоянии, поэтому при просмотре поля LockCount в отладчике важно перевернуть биты, прежде чем делать предположения о его состоянии. Как показывает рисунок 10.5, старшие 30 бит поля LockCount довольно прямолинейны и содержат количество ожидающих потоков. Бит 1 - это бит Waiter Woken, и он указывает, просигнализировал ли поток, освободивший критическую секцию, событие для пробуждения другого потока. Если бит установлен, это служит указанием, что другой поток, покидающий критическую секцию, не должен устанавливать его тоже. Младший значащий бит - это

Lock bit, и он указывает, находится ли критическая секция в заблокированном состоянии.

Рассмотрим несколько примеров (см. листинги 10.18, 10.19 и 10.20) того, как выглядит поле LockCount у критических секций в разных состояниях. (Помните: вы должны работать как минимум на Windows 2003 Server SP1.)

Листинг 10.18

```
Old Critical Section
+0x000 DebugInfo : 0x7c97c8c0
+0x004 LockCount : -1
+0x008 RecursionCount : 0
+0x00c OwningThread : (null)
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
New Critical Section
+0x000 DebugInfo : 0x77ca5be0
+0x004 LockCount : -1
+0x008 RecursionCount : 0
+0x00c OwningThread : (null)
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
```

Листинг 10.19

```
Old Critical Section
+0x000 DebugInfo : 0x7c97c8c0
+0x004 LockCount : 0
+0x008 RecursionCount : 1
+0x00c OwningThread : 0x00000b04
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
New Critical Section
+0x000 DebugInfo : 0x77ca5be0
+0x004 LockCount : -2
+0x008 RecursionCount : 1
+0x00c OwningThread : 0x00000134
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
```

Листинг 10.20

```
Old Critical Section
+0x000 DebugInfo : 0x7c97c8c0
+0x004 LockCount : 1
+0x008 RecursionCount : 1
+0x00c OwningThread : 0x00000b04
+0x010 LockSemaphore : 0x000007f4
+0x014 SpinCount : 0
New Critical Section
+0x000 DebugInfo : 0x77ca5be0
+0x004 LockCount : -6
+0x008 RecursionCount : 1
+0x00c OwningThread : 0x00000134
+0x010 LockSemaphore : 0x00000010
+0x014 SpinCount : 0
```

Чтобы проще понять поле LockCount новых критических секций, посмотрим на бинарную разбивку поля LockCount критической секции в листинге 10.20. Чтобы получить бинарное представление поля LockCount, используем команду dyd:

```
0:003> dyd 01002008+0x4
3 2 1 0
10987654 32109876 54321098 76543210
-----
0100200c 11111111 11111111 11111111 11111010 ffffffff
01002010 00000000 00000000 00000000 00000001 00000001
01002014 00000000 00000000 00000001 11111000 000001f8
01002018 00000000 00000000 00000000 00010000 00000010
0100201c 00000000 00000000 00000000 00000000 00000000
```

```
01002020 00000000 00000000 00000000 00000000 00000000
01002024 00000000 00000000 00000000 00000000 00000000
01002028 00000000 00000000 00000000 00000000 00000000
```

Первое, что нужно помнить: все биты инвертированы, поэтому перед попыткой выяснить состояние блокировки нужно снова перевернуть биты:

```
00000000 00000000 00000000 00000101
```

Lock Bit (бит 0) равен 1, что означает, что критическая секция заблокирована. Бит Waiter Woken (бит 1) равен 0, что означает, что ожидающие еще не были пробуждены. Оставшиеся 30 бит - это количество ожидающих, в нашем случае 1. К счастью, команда расширения !cs была обновлена, чтобы распознавать это изменение на затронутых платформах и отображать информацию более дружелюбно. Например, если использовать ту же критическую секцию, что и выше, как параметр команды расширения !cs, мы увидим следующее:

```
0:003> !cs 01002008
-----
Critical section = 0x01002008 (10LockConv!g_cs+0x0)
DebugInfo = 0x77965be0
LOCKED
LockCount = 0x1
WaiterWoken = No
OwningThread = 0x000001f8
RecursionCount = 0x1
LockSemaphore = 0x10
SpinCount = 0x00000000
```

Критическая секция находится в состоянии LOCKED, имеет одного ожидающего, и ни один ожидающий не был пробужден.

Прямое использование полей критической секции

Поскольку структура критической секции напрямую доступна разработчикам, бывали случаи, когда разработчики делали предположения о значении полей. Один такой пример - поле LockCount, где код приложения использует поле LockCount напрямую, чтобы посмотреть, свободна критическая секция или удерживается. Само собой разумеется, что смотреть на поля критической секции напрямую, а не через API критической секции, - большая ошибка. Хотя сама структура документирована, Microsoft оставляет за собой право менять эту структуру в любой форме, которую сочтет необходимой. Если у вас есть код, напрямую манипулирующий полями критической секции, вы рискуете получить приложение, которое не работает между версиями Windows.

Управление критическими секциями

Правильное управление критическими секциями так же важно, как управление любым другим видом ресурса. Неправильное управление может приводить к разрушительным ситуациям, которые чрезвычайно трудно и долго отлаживать. В этой части главы мы рассмотрим некоторые наиболее распространенные проблемы управления критическими секциями.

Четыре самых распространенных типа проблем при управлении критическими секциями:

- использование критической секции до инициализации;
- использование критической секции после удаления;
- чрезмерное освобождение критической секции;
- недостаточное освобождение критической секции.

Чтобы понять первую проблему, использование критической секции до инициализации, нужно сначала уметь определить, как выглядит правильно инициализированная критическая секция. Листинг 10.21 показывает содержимое инициализированной критической секции.

Листинг 10.21

```
+0x000 DebugInfo : 0x7c97c8c0
+0x000 Type : 0
+0x002 CreatorBackTraceIndex : 0
+0x004 CriticalSection : 0x0007ff2c
+0x008 ProcessLocksList : _LIST_ENTRY [ 0x7c97c0c8 - 0x7c97c8a8 ]
+0x010 EntryCount : 0
+0x014 ContentionCount : 0
+0x018 Spare : [2] 0
+0x004 LockCount : -1
+0x008 RecursionCount : 0
+0x00c OwningThread : (null)
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
```

Все поля критической секции в листинге 10.21 имеют правильные значения по умолчанию, а поле `CriticalSection` структуры `DebugInfo` указывает обратно на фактическую структуру критической секции.

Теперь представьте, что поток сумел вызвать `EnterCriticalSection` до того, как критическая секция была инициализирована. Итоговый результат, скорее всего, - `access violation` со следующей трассой стека:

```
ChildEBP RetAddr Args to Child
0007fe98 7c94243c c0000000 00090000 0007fa0c ntdll!RtlRaiseStatus+0x26
0007ff18 7c90104b 0007ff2c 01001152 0007ff2c ntdll!RtlpWaitForCriticalSection+0x204
0007ff20 01001152 0007ff2c 0007ff50 77c3ac60 ntdll!RtlEnterCriticalSection+0x46
0007ff44 010012a0 00000001 00034e50 00033338 simple!main+0x12
0007ffc0 7c816fd7 00090000 0007fa0c 7ffde000 simple!mainCRTStartup+0x12f
0007fff0 00000000 01001171 00000000 78746341 kernel32!BaseProcessStart+0x23
```

Если вы встретите похожую трассу стека, сначала нужно проверить структурную целостность критической секции и попытаться понять, находится ли критическая секция в хорошем состоянии.

```
0:000> dt CRITICAL_SECTION 0007ff2c
+0x000 DebugInfo : 0x0007ff50
+0x004 LockCount : 2009312352
+0x008 RecursionCount : 216660
+0x00c OwningThread : 0x0007ff4c
+0x010 LockSemaphore : 0x00090000
+0x014 SpinCount : 0x7fa0c
```

Похоже, это недопустимая критическая секция. Все поля критической секции имеют значения, которые, кажется, не имеют смысла. Обычно, когда вы имеете дело с критической секцией, которая еще не была инициализирована, вы увидите, казалось бы, случайные значения, и почти каждое поле будет неверным.

Вторая форма неправильного управления критическими секциями - использование критической секции после ее удаления. И снова важно знать, как на самом деле выглядит критическая секция, которая была удалена:

```
+0x000 DebugInfo : (null)
+0x004 LockCount : 0
+0x008 RecursionCount : 0
+0x00c OwningThread : (null)
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
```

Неудивительно, что все поля критической секции были инициализированы значением 0 (или `null`). Теперь, если поток пытается приобрести критическую секцию, вы увидите `access violation` почти так же, как в сценарии использования до инициализации:

```
ChildEBP RetAddr Args to Child
0007ff18 7c90104b 0007ff2c 01001170 0007ff2c ntdll!RtlpWaitForCriticalSection+0x8c
0007ff20 01001170 0007ff2c 00000000 00000001 ntdll!RtlEnterCriticalSection+0x46
0007ff44 010012aa 00000001 00034e50 00033338 simple!main+0x30
0007ffc0 7c816fd7 00090000 0007fa0c 7ffdd000 simple!mainCRTStartup+0x12f
0007fff0 00000000 0100117b 00000000 78746341 kernel32!BaseProcessStart+0x23
```

Последние две проблемы, которые мы рассмотрим, относятся к категориям недостаточного и чрезмерного освобождения критических секций. Обе эти проблемы обычно проявляются как зависания приложения. Рассмотрим первую проблему - недостаточное освобождение критических секций. Обычно эту проблему легко заметить, потому что у вас будет некоторое количество потоков, ожидающих приобретения критической секции, со следующей трассой стека:

```
1 Id: 748.d14 Suspend: 1 Teb: 7ffdd000 Unfrozen
ChildEBP RetAddr Args to Child
002bfff14 7c90e9c0 7c91901b 000007e8 00000000 ntdll!KiFastSystemCallRet
002bfff18 7c91901b 000007e8 00000000 7c97c140 ntdll!NtWaitForSingleObject+0xc
002bffa0 7c90104b 00002008 010011e0 01002008 ntdll!RtlpWaitForCriticalSection+0x132
002bffa8 010011e0 01002008 002bfffec 7c80b683 ntdll!RtlEnterCriticalSection+0x46
...
...
```

Дальше у нас есть вся необходимая информация в критической секции (например, владеющий поток), чтобы пройти назад и выяснить, почему критическая секция никогда не была освобождена.

Аналогично, чрезмерное освобождение критической секции также приводит к зависанию.

Представьте, что приложение выполняет следующий простой фрагмент кода:

```
InitializeCriticalSection(&g_cs);
EnterCriticalSection(&g_cs);
LeaveCriticalSection(&g_cs);
LeaveCriticalSection(&g_cs);
EnterCriticalSection(&g_cs);
```

После первого вызова EnterCriticalSection мы ожидаем, что состояние критической секции будет похоже на следующее:

```
+0x000 DebugInfo : 0x7c97c8c0
+0x004 LockCount : 0
+0x008 RecursionCount : 1
+0x00c OwningThread : 0x00000afc
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
```

Затем приложение покидает критическую секцию, и мы ожидаем, что состояние критической секции выглядит так:

```
+0x000 DebugInfo : 0x7c97c8c0
+0x004 LockCount : -1
+0x008 RecursionCount : 0
+0x00c OwningThread : (null)
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
```

Следующий вызов выхода из критической секции вызывает overrelease, и критическая секция выглядит так:

```
+0x000 DebugInfo : 0x7c97c8c0
+0x004 LockCount : -2
+0x008 RecursionCount : -1
+0x00c OwningThread : (null)
+0x010 LockSemaphore : (null)
+0x014 SpinCount : 0
```

Поскольку поле LockCount равно не -1 (что означает незаблокированную критическую секцию), а -2, следующий вызов EnterCriticalSection приводит к зависанию, потому что Windows считает, что критическая секция фактически заблокирована:

```
0 Id: 4bc.afc Suspend: 1 Teb: 7ffdf000 Unfrozen
ChildEBP RetAddr Args to Child
0007fe98 7c90e9c0 7c91901b 000007f4 00000000 ntdll!KiFastSystemCallRet
0007fe9c 7c91901b 000007f4 00000000 7c97c140 ntdll!NtWaitForSingleObject+0xc
0007ff24 7c90104b 00002008 0100118d 01002008 ntdll!RtlpWaitForCriticalSection+0x132
0007ff2c 0100118d 01002008 0007ff4c 00000000 ntdll!RtlEnterCriticalSection+0x46
0007ff44 010012c7 00000001 00034e50 00033338 simple!main+0x4d
0007ffc0 7c816fd7 00090000 0007fa0c 7ffdf000 simple!mainCRTStartup+0x12f
0007fff0 00000000 01001198 00000000 78746341 kernel32!BaseProcessStart+0x23
```

В качестве последнего шага можно выгрузить критическую секцию и посмотреть, что с ней произошло, пока мы пытались войти в нее:

```
+0x000 DebugInfo : 0x7c97c8c0
+0x004 LockCount : -1
+0x008 RecursionCount : -1
+0x00c OwningThread : (null)
+0x010 LockSemaphore : 0x000007f4
+0x014 SpinCount : 0
```

Поле LockCount теперь было увеличено на 1 (-1), что имеет смысл, потому что мы только что попытались войти в критическую секцию.

Как общее практическое правило, если вы видите зависание на критической секции с отрицательным lock count и владеющим потоком, равным null, следует рассматривать источником проблемы чрезмерно освобожденную критическую секцию.

Существует еще много способов, которыми критическая секция может перейти в невосстановимое состояние из-за неправильного управления. Другие такие способы включают повреждение критической секции, освобождение памяти, содержащей активные критические секции, выгрузку DLL с активной критической секцией и многое другое. Вместо того чтобы выполнять много обратного отслеживания после проявления проблемы, было бы хорошо ловить некоторые из этих проблем в момент, когда виновный поток ведет себя неправильно, а не когда потоки-жертвы либо падают, либо зависают. К счастью, наш старый добрый друг Application Verifier снова приходит на помощь. В настройке теста Basics есть настройка Locks, которая выполняет целый набор тестов во время работы приложения. Она поймает все: от использования неинициализированной критической секции до освобождения фрагмента памяти, содержащего активную и еще не освобожденную критическую секцию. Мы настоятельно рекомендуем включать эту настройку теста всякий раз, когда ваше приложение содержит код синхронизации, потому что она ловит виновный код в действии, а не как жертву.

Итоги

Многопоточность может казаться тривиальным упражнением в программировании. В конце концов, все довольно просто: вы запускаете некоторое количество потоков и заставляете их работать параллельно, чтобы выполнить задачу. Как вы видели на протяжении главы, область concurrency и синхронизации далека от тривиальности. Можно даже утверждать, что это одна из наиболее подверженных ошибкам областей. Нужно проявлять крайнюю осторожность, чтобы убедиться, что все потоки живут и работают вместе гармонично. Небольшие ошибки в этой логике могут иметь существенные и разрушительные последствия.

В этой главе мы рассмотрели несколько очень распространенных ошибок, совершаемых при работе с многопоточными приложениями и синхронизацией. Мы начали с краткого обзора разных примитивов синхронизации, доступных в Windows, и высокоуровневого процесса устранения проблемы синхронизации. Были показаны несколько сценариев, таких как deadlocks, orphaned critical sections, осведомленность об API и управление критическими секциями, а также то, как искать первопричину с помощью доступных отладчиков и инструментов. Превентивные меры, такие

как использование Application Verifier, показывают, как находить первопричину раньше в процессе разработки, тем самым уменьшая объем времени, затрачиваемого на отладку сложных проблем синхронизации. Кроме того, были показаны техники с использованием языка C++, позволяющие снизить риск столкновения с проблемами синхронизации.

Часть III. Расширенные темы

Глава 11. Написание собственных расширений отладчика

Как вы видели в подробных примерах, показанных до сих пор, системные отладчики и инструменты, доступные разработчикам Windows, предоставляют широкий набор развитых и мощных приемов для работы с большим спектром проблем. Тем не менее бывают моменты, когда некоторые аспекты сеансов отладки становятся очень повторяющимися и склонными к ошибкам. Примеры включают вывод собственных структур данных или поиск параметров безопасности у объектов ядра. К счастью, команда Microsoft, отвечающая за отладчики, осознала болезненность этих задач и ввела понятие расширений отладчика. На самом высоком уровне расширение отладчика - это компонент, позволяющий автоматизировать повторяющиеся и сложные задачи для более эффективной отладки. В этой главе объясняются основы расширений отладчика и рассматривается анатомия расширения отладчика. Затем приводится полный пошаговый разбор реализации расширения отладчика, автоматизирующего процесс вывода содержимого типа данных "двоичное дерево".

Введение в расширения отладчика

Предыдущие главы показывают различные полезные сеансы отладки и связанные с ними команды расширений. Среди примеров - `!sid`, `!token` и `!sd`. Откуда берутся эти команды расширений? Реализованы ли они в самом отладчике? Большинство отладочных команд, которые вы видели до сих пор, реализованы как собственные расширения отладчика. Debugging Tools for Windows включает основной набор расширений отладчика, содержащий команды, наиболее часто используемые во время сеансов отладки. В таблице 11.1 перечислены доступные основные расширения отладчика.

Таблица 11.1

Расширение	Имя расширения	Описание
General Extensions	Ext.dll	Общие команды расширений, такие как <code>error</code> , <code>cxr</code> , <code>std_map</code> .
User Mode Extensions	Ntsdexts.dll	Команды расширений, часто используемые при отладке пользовательского режима, такие как <code>runaway</code> , <code>critsec</code> , <code>threadtoken</code> .
RPC extensions	rpcexts.dll	Команды расширений для отладки RPC, такие как <code>authinfo</code> , <code>getcallinfo</code> , <code>rpcheap</code> .

В самой базовой форме расширение отладчика - это не более чем DLL, экспортирующая набор точек входа. Каждая точка входа вызывается отладочным движком, когда пользователь вводит команду (с тем же именем, что и точка входа) в сеансе отладки. В качестве примера рассмотрим следующую команду:

```
0:000> !ext.error 0
Error code: (Win32) 0 (0) - The operation completed successfully.
```

Команда указывает, что команда `error` является частью расширения `ext.DLL` и что эту команду нужно выполнить. В этом случае `ext.DLL` экспортирует соответствующую функцию `error`,

вызываемую в результате выполнения приведенной выше команды. Сама команда просто печатает текстовое представление кода ошибки Windows 0 в окно консоли отладчика.

Изучение расширений отладчика. Используя инструмент SDK с именем `dumpbin.exe`, можно очень легко получить список всех экспортируемых функций из любой DLL расширения (включая перечисленные в таблице 11.1). Например, запуск `dumpbin.exe` для `exts.dll` дает следующий результат:

```
C:\>dumpbin /EXPORTS exts.dll
Microsoft (R) COFF/PE Dumper Version 8.00.50727.42
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
Dump of file exts.dll
```

```
File Type: DLL
```

```
Section contains the following exports for exts.dll
```

```
00000000 characteristics
44B0109E time date stamp Sat Jul 08 13:07:58 2006
0.00 version
1 ordinal base
32 number of functions
32 number of names
```

```
ordinal hint RVA name
2 0 0002D8F0 DebugExtensionInitialize
3 1 0002D9B0 DebugExtensionNotify
4 2 0002DB80 DebugExtensionUninitialize
1 3 000446E0 _EFN_GetEnvironmentVariable
5 4 0003FE50 acl
6 5 0002EC90 atom
7 6 00024750 avrf
8 7 00021D50 bitcount
9 8 000235A0 cs
10 9 0002ED70 decodeptr
11 A 0002B250 dlls
12 B 0002C5E0 dlltree
13 C 0002ED70 encodeptr
14 D 00044890 envvar
15 E 0002E000 gflag
16 F 0001C850 heap
17 10 0002DE80 help
18 11 0002E530 kuser
19 12 000399E0 mui
20 13 00045960 peb
21 14 0003C3D0 psr
22 15 0003D920 rebase
23 16 0003F4B0 sd
24 17 00047260 shipassert
25 18 0003FD70 sid
26 19 00040700 slist
27 1A 00043EC0 stl
28 1B 00045F70 teb
29 1C 00047090 tls
30 1D 00049240 token
31 1E 00050100 tp
32 1F 00019A40 udeadlock
```

```
Summary
```

```
5000 .data
6000 .reloc
1000 .rsrc
54000 .text
```

Пример расширения отладчика

До сих пор вы видели примеры небольших, но очень полезных расширений. Чтобы показать реальную мощь расширений отладчика, предположим, что вы разработали простую версию компонента двоичного дерева, позволяющего разработчикам добавлять элементы в двоичное дерево и обходить дерево. (Ради краткости другие распространенные операции с двоичным деревом опущены.) Двоичное дерево также ограничено хранением целых чисел. Код двоичного дерева показан в листинге 11.1 и довольно понятен сам по себе.

Листинг 11.1

```
// bstree.h
#ifndef __BSTREE_H
#define __BSTREE_H
#include <windows.h>
#include <strsafe.h>
class CBinaryTree
{
public:
    CBinaryTree(CHAR* pszDescription);
    BOOL Add(int num);
    VOID Traverse();
private:
    class _TreeNode
    {
    public:
        _TreeNode(int num)
        {
            pLeftChild = NULL; pRightChild = NULL; data=num;
        }
        int data;
        _TreeNode* pLeftChild;
        _TreeNode* pRightChild;
    };
    BOOL AddNode(_TreeNode* item,
        _TreeNode* current,
        DWORD dwRecursiveCount);
    VOID TraverseTree(_TreeNode* current);
    _TreeNode* m_pHead;
    DWORD m_dwDepth;
    DWORD m_dwNumNodes;
};
#endif
// bstree.cpp
#include "bstree.h"
#include <stdio.h>
CBinaryTree::CBinaryTree () : m_pHead(NULL), m_dwDepth(0), m_dwNumNodes(0)
{
}
BOOL CBinaryTree::Add(int num)
{
    BOOL bRet=FALSE;
    _TreeNode* item=new _TreeNode(num);
    if(item!=NULL)
    {
        bRet=AddNode(item, m_pHead, 1);
        if(bRet==TRUE)
        {
            m_dwNumNodes++;
        }
    }
    return bRet;
}
VOID CBinaryTree::Traverse()
{
    TraverseTree(m_pHead);
}
VOID CBinaryTree::TraverseTree(_TreeNode* current)
```

```

{
if(current==NULL)
{
return;
}
//
// Simple left->right subtree traversal
//
TraverseTree(current->pLeftChild);
printf("Item: %d\n", current->data);
return;
}
BOOL CBinaryTree::AddNode(_TreeNode* item,
_TreeNode* current,
DWORD dwRecursiveCount)
{
BOOL bRet=FALSE;
//
// Empty tree, initialize the head pointer
//
if(current==NULL)
{
m_pHead=item;
m_dwDepth=dwRecursiveCount;
return TRUE;
}
else
{
if(item->data>current->data)
{
if(current->pRightChild==NULL)
{
current->pRightChild=item;
m_dwDepth=(m_dwDepth<dwRecursiveCount+1) ?
dwRecursiveCount+1 : m_dwDepth;
bRet=TRUE;
}
else
{
return AddNode(item,
current->pRightChild,
dwRecursiveCount+1);
}
}
else if(item->data<current->data)
{
if(current->pLeftChild==NULL)
{
current->pLeftChild=item;
m_dwDepth=(m_dwDepth<dwRecursiveCount+1) ?
dwRecursiveCount+1 : m_dwDepth;
bRet=TRUE;
}
else
{
return AddNode(item,
current->pLeftChild,
dwRecursiveCount+1);
}
}
}
return bRet ;
}

```

Исходный код и двоичный файл двоичного дерева находятся в следующих папках:

Source code: C:\AWD\Chapter11\bstree
Binary: C:\AWDBIN\WinXP.x86.chk\bstree.exe

Кроме того, было написано простое тестовое приложение (bstree.exe), использующее двоичное дерево. Исходный код приложения bstree.exe показан в листинге 11.2.

Листинг 11.2

```
#include "bstree.h"
void __cdecl main ( )
{
    CBinaryTree* tree = new CBinaryTree();
    if ( tree != NULL )
    {
        tree->Add (10);
        tree->Add (5);
        tree->Add (15);
        tree->Add (7);
        tree->Add (13);
        tree->Traverse ();
    }
}
```

Вывод одного запуска приложения выглядит так:

```
Item: 5
Item: 10
```

Судя по выводу, в приложении что-то идет не так. Код добавил в двоичное дерево пять узлов, но при обходе дерева отображаются только два. Как именно вы продолжили бы этот сеанс отладки? На самом деле в этом коде могли пойти не так только две крупные части:

- добавление новых узлов в дерево;
- код обхода.

Поскольку проблема, по-видимому, возникает во время обхода и отображения содержимого дерева, хорошая отправная точка - установить точку останова в функции Traverse и проверить целостность дерева с помощью команды dt.

```
0:000> dt this
Local var @ 0x6ff1c Type CBinaryTree*
0x00262c30
+0x000 m_pHead : 0x00262c50
```

Как видно, у членов данных CBinaryTree нет ничего сложного. Он просто содержит указатель (m_pHead) на первый узел дерева, и на первый взгляд указатель кажется вполне разумным (то есть не равен null и находится в допустимом диапазоне). Поскольку у нас есть только указатель на корневой узел дерева, как вывести содержимое в отладчике? Хорошая новость в том, что это возможно; плохая - это утомительно. Как и в большинстве реализаций двоичного дерева, головной указатель указывает на тип данных, который обычно содержит элемент данных в узле, а также левый дочерний/правый дочерний элемент, которые тоже являются указателями на тот же тип данных. Посмотрев на исходный код, мы видим, что указатель m_pHead имеет тип _TreeNode, показанный ниже:

```
class _TreeNode
{
public:
    _TreeNode (int num) { pLeftChild=NULL; pRightChild=NULL; data=num; }
    int data;
    _TreeNode* pLeftChild;
    _TreeNode* pRightChild;
};
```

Чтобы увидеть содержимое указателя m_pHead, мы снова используем команду dt для указателя m_pHead:

```
0:000> dt bstree!CBinaryTree::_TreeNode 0x00262c50
+0x000 data : 10
+0x004 pLeftChild : 0x00262c78
+0x008 pRightChild : 0x00262ca0
```

Поле data установлено в 10 (что соответствует первой вставке в клиентском коде), и все указатели выглядят разумно. Прежде чем продолжить ручной обход всего дерева в отладчике, нужно посмотреть, каким мы ожидаем увидеть дерево. Вот вставки, выполненные в клиентском коде:

```
tree->Add(10);
tree->Add(5);
tree->Add(15);
tree->Add(7);
tree->Add(13);
```

Эти вставки дают дерево, показанное на рисунке 11.1.

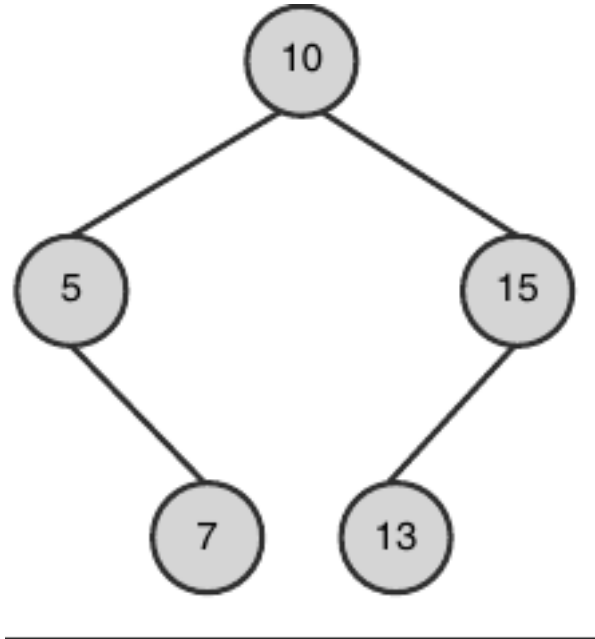


Рисунок 11.1. Представление дерева после вставки 10, 5, 15, 7 и 13

Если вывести левый дочерний указатель, мы разумно ожидали бы увидеть _TreeNode с элементом данных, равным 5, левым дочерним указателем, равным null, и правым дочерним указателем, содержащим еще один допустимый указатель:

```
0:000> dt bstree!CBinaryTree::_TreeNode 0x00262c78
+0x000 data : 5
+0x004 pLeftChild : (null)
+0x008 pRightChild : 0x00262cc8
```

Продолжая этот процесс для всего дерева, получаем следующий сеанс отладки:

```
0:000> dt bstree!CBinaryTree::_TreeNode 0x00262cc8
+0x000 data : 7
+0x004 pLeftChild : (null)
+0x008 pRightChild : (null)
0:000> dt bstree!CBinaryTree::_TreeNode 0x00262ca0
+0x000 data : 15
+0x004 pLeftChild : 0x00262cf0
+0x008 pRightChild : (null)
0:000> dt bstree!CBinaryTree::_TreeNode 0x00262cf0
+0x000 data : 13
+0x004 pLeftChild : (null)
+0x008 pRightChild : (null)
```

На этом этапе мы проверили целостность дерева (элементы данных были корректны, а все указатели успешно разыменовывались), что означает: функция Add дерева, похоже, работает правильно, и что-то должно быть не так с функцией Traverse. Я оставлю читателю в качестве упражнения просмотр кода функции Traverse и поиск ошибки.

Довольно много отладочной работы, не так ли? И это только для дерева размером 5. Представьте, что этот компонент использовался в системе под высокой нагрузкой: мы могли бы получить тысячи записей. Гораздо лучше было бы сообщить отладчику, как интерпретировать эту структуру данных, просто выполнив команду `tree` с головным указателем, и позволить отладчику автоматически обойти экземпляр двоичного дерева.

Это типичный сценарий, в котором можно использовать мощь расширений отладчика для автоматизации утомительного процесса вывода содержимого нашего типа данных дерева. Создав собственное расширение отладчика, мы можем реализовать команду, принимающую указатель на корень дерева и отображающую его содержимое. Это может сэкономить часы ручных отладочных шагов. Прежде чем перейти к техническим деталям реализации расширения отладчика, нужно рассмотреть и понять две разные модели расширений отладчика, которые предоставляют отладчики, а затем решить, какая лучше подойдет для нашего сценария.

Модели расширений отладчика

Основная идея собственного расширения отладчика - позволить пользователям автоматизировать сеансы отладки, вводя конкретные команды, реализованные в расширении отладчика. Как это на самом деле работает? На высоком уровне это четырехэтапный процесс, показанный на рисунке 11.2.

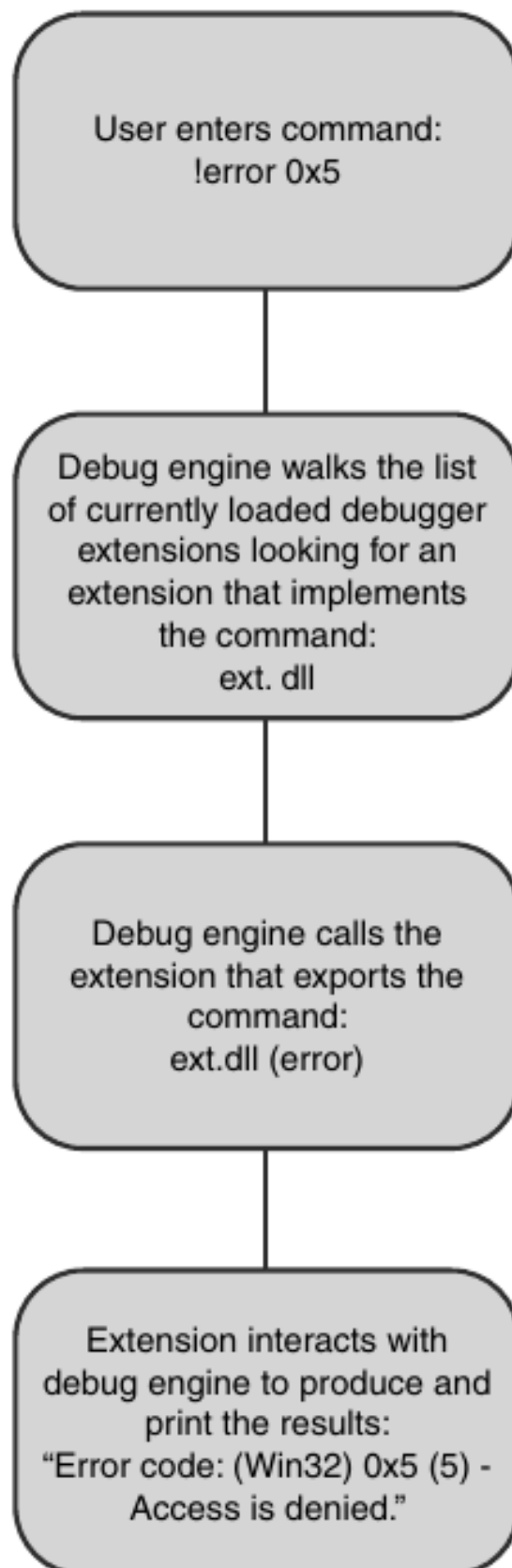


Рисунок 11.2. Обзор процесса расширений отладчика

Самая критически важная часть расширения отладчика - то, как оно взаимодействует с основным отладочным движком (шаг 4). Существуют три разные модели, определяющие это взаимодействие. Первая модель, называемая WDbgExt, является исходной моделью и существует уже довольно

давно. Взаимодействие между расширением отладчика и основным отладочным движком выполняется через обычные C API. Поскольку она основана на C, она предоставляет довольно безболезненную модель программирования, но также несколько ограничена в поддерживаемых взаимодействиях. Таблица 11.2 показывает API, поддерживаемые в настоящий момент в модели WDbgExts.

Таблица 11.2

Функция WDbgExts	Описание
GetContext	Возвращает контекст отлаживаемого процесса.
SetContext	Задаёт контекст отлаживаемого процесса.
CheckControlC	Проверяет, нажал ли пользователь CTRL+C. Полезно при реализации прерываемых команд.
GetDebuggerData	Получает данные отладчика.
Disasm	Выполняет дизассемблирование.
Dprintf	Печатает строку в командное окно отладчика.
GetExpression	Возвращает значение выражения.
Ioctl	Точка входа для процедур расширения отладчика ядра.
GetKdContext	Возвращает общее число процессоров и номер текущего процессора.
ReadMemory	Читает из памяти.
WriteMemory	Записывает в память.
ReadMsR	Читает содержимое model-specific register на Pentium.
WriteMsR	Записывает в model-specific register на Pentium.
ReadPhysical	Читает из физической памяти.
WritePhysical	Записывает в физическую память.
StackTrace	Возвращает трассировку стека отлаживаемого процесса.
GetSymbol	Находит символ.
GetSetSympath	Получает или задаёт путь символов.
ReadControlSpace	Читает processor-specific control space.
ReadControlSpace64	Читает processor-specific control space (64-разрядный).
ReadIoSpace	Читает из системных I/O-адресов.
ReadIoSpace64	Читает из системных I/O-адресов (64-разрядный).
ReadIoSpaceEx	Помимо чтения I/O-адресов, читает bus I/O.
ReadIoSpaceEx64	Помимо чтения I/O-адресов, читает bus I/O (64-разрядный).
WriteIoSpace	Записывает в системные I/O-адреса.
WriteIoSpace64	Записывает в системные I/O-адреса (64-разрядный).
WriteIoSpaceEx	Помимо записи в I/O-адреса, записывает в bus I/O.
WriteIoSpaceEx64	Помимо записи в I/O-адреса, записывает в bus I/O (64-разрядный).
SetThreadForOperation	Задаёт поток, используемый для следующей трассировки стека.
SetThreadForOperation64	Задаёт поток, используемый для следующей трассировки стека (64-разрядный).

Несколько callback-функций также позволяют отладочному движку запросить у расширения отладчика некоторые базовые сведения (например, версию), а также вызвать процедуры инициализации, выполняемые отладочным движком.

- `ExtensionApiVersion`: возвращает версию расширения.
- `CheckVersion`: проверяет соответствие версий отладочного движка и расширения.
- `WinDbgExtensionDllInit`: процедура инициализации для расширений WinDbg.

Более новая и мощная модель называется DbgEng и основана на COM (Component Object Model). Реализации расширений отладчика получают указатели COM-интерфейсов, которые можно использовать для взаимодействия с основным отладочным движком. Указатели интерфейсов передаются каждой функции, экспортируемой в DLL расширения. Модель COM-интерфейсов предоставляет большой набор API и позволяет очень тонко управлять взаимодействием между расширением и отладочным движком. Более подробные сведения о доступных функциях каждого интерфейса и подробные описания см. в документации SDK (`debugext.chm`).

Множество интерфейсов и связанных функций, доступных для модели DbgEng, существенно больше и мощнее, чем у ее более старой родственницы (WDbgExts), но программировать против нее немного сложнее. Чтобы сократить разрыв между моделями WDbgExt и DbgEng, была введена третья модель, называемая EngExtCpp. Модель EngExtCpp - это C++-библиотека расширений, построенная поверх DbgEng. Она предоставляет абстракцию модели DbgEng, значительно упрощающую программирование против нее.

Важно отметить, что эти модели не взаимоисключающие. Возможен гибридный подход, и часто он полезен. В одном и том же расширении отладчика можно представить использование C-style API модели WdbgExts для более простых задач и одновременно использование мощности модели DbgEng для более продвинутых команд. Чтобы сделать задачу интересной и образовательной, в собственном расширении отладчика, которое мы собираемся реализовать, мы покажем гибридную модель.

Требования к нашему примерному расширению

Как и для любого компонента, который мы разрабатываем, важно определить точные требования. Начнем путь реализации с добросовестного выделения времени на точное описание того, чего мы ожидаем от расширения отладчика.

Наша DLL расширения называется `sysexts.dll` и экспортирует следующие команды. (Пока не беспокойтесь о конкретных API; они будут объяснены позже.)

- `Help`

Описание: команда `help` отображает синтаксис всех доступных команд в расширении отладчика, а также краткое описание и пример использования команды.

Пример:

```
0:000> !sysexts.help
SYSYEXTS.DLL commands:
Help = Shows this help
DumpTree <address> = Dumps out the contents of a CBinaryTree.
Address must point to the root node of the tree.
```

API отладочного движка: единственный API отладочного движка, требуемый в этой команде, - `dprintf`, позволяющий печатать информацию в консольное окно отладки (часть модели WdbgExts).

- `DumpTree`

Описание: команда DumpTree обходит переданный корневой указатель CBinaryTree и выводит его содержимое. Обход будет выполняться стандартным алгоритмом симметричного (in-order) обхода дерева.

Пример:

```
0:000> !sysexts.dumptree 0x00262c40
** Node **
Data: 5
Left child pointer: 0x0
Right child pointer: 0x262cb8
** Node **
Data: 7
Left child pointer: 0x0
Right child pointer: 0x0
0:000>
```

API отладочного движка: основной требуемый API отладочного движка - `IDataSpaces::ReadVirtual`, позволяющий расширению читать виртуальную память.

- Поддержка собственного форматирования наших объектов, когда пользователь вводит `dt <our-object> <address>`

Описание: выводит расширенное представление содержимого CBinaryTree. Дополнительная информация включает корневой узел дерева.

Пример:

```
0:000> dt CBinaryTree 0x00262c20
Root Node [Ptr]: 0x262D48
Root Node data: 10
Root Node Left Child [Ptr]: 0x262D70
Root Node Right Child [Ptr]: 0x262D98
+0x000 m_pHead : 0x00262d48
+0x004 m_dwDepth : 3
+0x008 m_dwNumNodes : 5
+0x00c m_szDescription : [256] "Sample binary tree"
```

API отладочного движка: реализовать функцию `KnownStructOutput`.

- Поддержка отмены

Описание: поддержка отмены выполняющейся команды. Все расширения отладчика, которые потенциально могут выполнять команду долгое время, должны включать логику отмены. Обычно отмена выполняется, когда пользователь нажимает CTRL+C во время выполнения команды. Хотя включать такую поддержку не обязательно, нет ничего более раздражающего, чем ждать несколько минут завершения команды, ошибочно введенной пользователем. Поскольку наша команда `DumpTree` потенциально может выполняться довольно долго (если глубина дерева велика), мы включим поддержку отмены в реализацию.

API отладочного движка: `API CheckControlC` (часть модели `Wdbgexts`).

Теперь, когда у вас есть ясное понимание требований к нашему расширению отладчика, можно начать реализацию.

Необходимые заголовочные файлы и организация кода

Чтобы писать собственные расширения отладчика, необходимо выполнить custom-установку `Debugging Tools for Windows` и выбрать параметр SDK. На рисунке 11.3 показана структура каталогов папки SDK после установки.

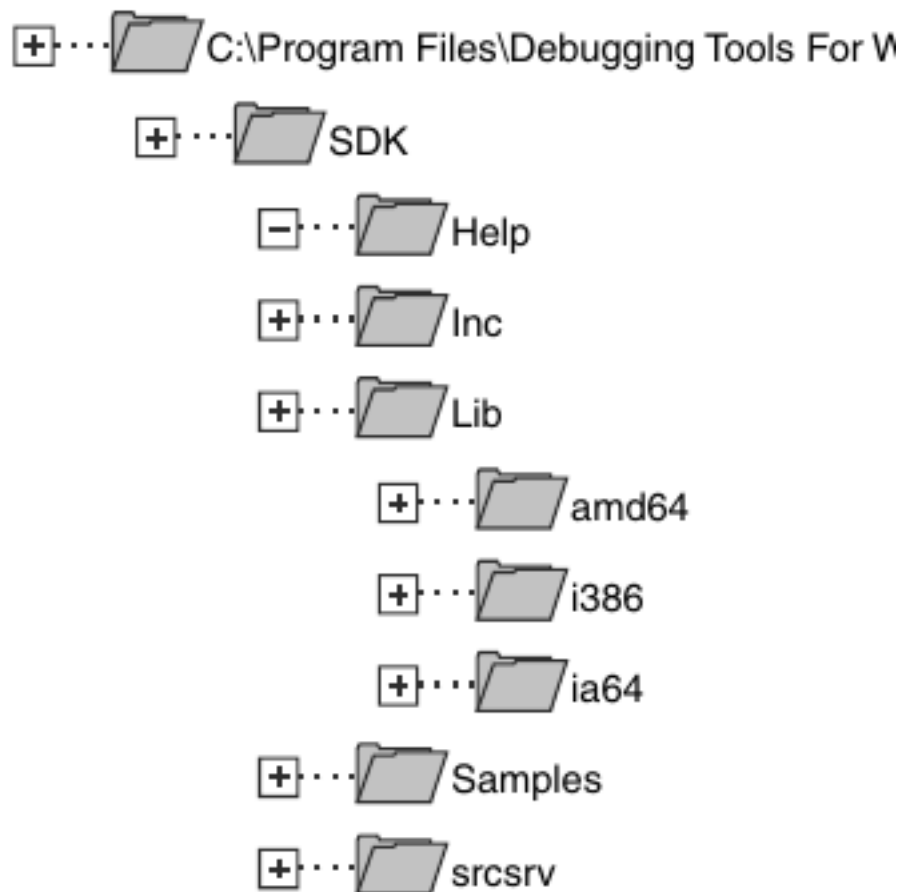


Рисунок 11.3. Структура каталогов SDK

Как и в случае любого нового SDK, с которым вы сталкиваетесь, первое, что нужно выяснить, - какие заголовочные файлы требуются для программирования против него. Варианты заголовочных файлов, которые можно использовать, показаны в таблице 11.3.

Таблица 11.3

Заголовочный файл	Описание
dbgeng.h	Включается всеми расширениями отладчика, использующими модель DbgEng.
wdbgexts.h	Включается всеми расширениями отладчика, использующими модель WDbgExts.
engextcpp.hpp	Включается всеми расширениями отладчика, использующими модель EngExtCpp.

Если расширение отладчика использует только одну конкретную модель, достаточно включить один из приведенных выше файлов. Поскольку наша реализация является гибридной моделью, мы включаем оба файла.

Далее нужно решить вопрос организации кода. Расширение отладчика, которое мы будем строить, устроено так, как описано в таблице 11.4.

Таблица 11.4

Исходный файл	Описание
Sources	Файл, требуемый средой сборки WDK. Этот файл подробно описывает, как собирается расширение отладчика (см. "Компиляция и компоновка").

Исходный файл	Описание
sysexts.cpp	Этот файл содержит реализацию всех команд расширения отладчика (включая код инициализации и деинициализации).
sysexts.h	Это заголовочный файл, сопровождающий основной файл sysexts.cpp.
sysexts.def	Это файл, на который ссылается sources и который подробно описывает, какие функции в DLL расширения экспортируются.

В большинстве случаев организация кода, определенная в таблице 11.4, вполне достаточна. Если расширение отладчика растет в размере, а число реализованных команд становится все больше и больше, можно рассмотреть разбиение исходного кода расширения на отдельные файлы, чтобы упростить сопровождение.

Исходный код и двоичный файл примерного расширения находятся в следующих папках:

Source code: C:\AWD\Chapter11\sysexts
Binary: C:\AWDBIN\WinXP.x86.chk\sysexts.dll

Начнем с файла sysexts.h:

```
#pragma once
#include <windows.h>
#define KDEXT_64BIT
#define DBG_COMMAND_EXCEPTION ((DWORD)0x40010009L)
#include <wdbgexts.h>
#include <dbgeng.h>
WINDBG_EXTENSION_APIS64 ExtensionApis;
```

Помимо включения заголовочных файлов debug SDK, как обсуждалось ранее, мы включаем windows.h, чтобы получить доступ к большинству объявлений Windows API. Единственные странные части этого заголовочного файла - следующие загадочные определения:

```
#define KDEXT_64BIT
#define DBG_COMMAND_EXCEPTION ((DWORD)0x40010009L)
WINDBG_EXTENSION_APIS64 ExtensionApis;
```

Для первых двух определений (KDEXT_64BIT и DBG_COMMAND_EXCEPTION), если реализуется гибридная модель (как в нашем случае), включение обоих отладочных заголовочных файлов требует приведенных выше определений до включения заголовочного файла wdbgexts.h. Без этих определений возникнут ошибки сборки. Если вы реализуете негибридную модель, это определение не требуется.

Третье определение также связано с правильной инициализацией модели wdbgexts. Чтобы расширение могло использовать WDbgExts API, ему нужно определить глобальную переменную, гарантирующую возможность использования WDbgExts API. Глобальная переменная должна быть инициализирована до использования расширения отладчика, и обычно это делается в процедуре инициализации расширения. (Пример этого вы увидите в следующем разделе "Инициализация расширения".)

Последний важный файл перед погружением в реализации команд - sysexts.def:

```
EXPORTS
help
dumptree
DebugExtensionInitialize
DebugExtensionUninitialize
KnownStructOutput
```

Этот файл просто сообщает среде сборки, какие функции в нашей DLL расширения отладчика должны считаться публично экспортируемыми. Если бы мы добавляли больше команд, имя

функции также нужно было бы добавить в этот файл. Мы начинаем реализацию с кода инициализации и деинициализации расширения.

Инициализация расширения

Когда пользователь вводит команду `.load` в отладчике, отладочный движок отвечает загрузкой DLL расширения и выполнением последовательности вызовов функций расширения отладчика, позволяя расширению выполнить подготовительную работу перед приемом команд. Аналогично, когда расширение отладчика выгружается, отладочный движок вызывает экспортированную функцию в DLL расширения, чтобы дать ей возможность выполнить очистку. Во время сеанса отладки отладочный движок выполняет любое число вызовов функций DLL расширения в зависимости от действия, выполняемого в отладчике. Процесс загрузки и выгрузки DLL расширения и соответствующие вызовы функций показаны на рисунке 11.4.

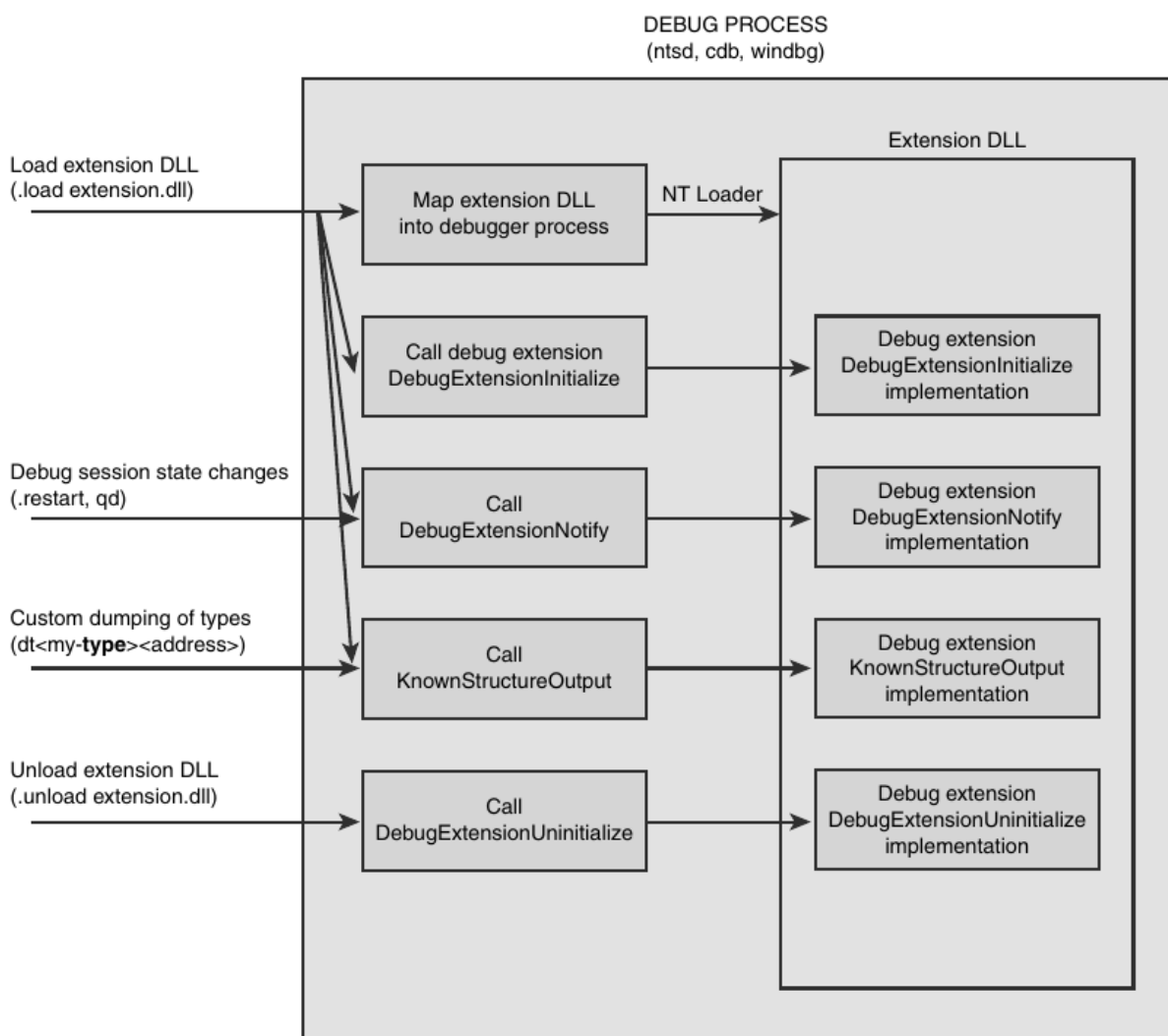


Рисунок 11.4. Процесс загрузки и выгрузки DLL расширения

Когда отладочный движок загружает расширение в ответ на команду `.load`, он сначала отображает DLL в адресное пространство отладчика, а затем вызывает функцию `DebugExtensionInitialize`, которая должна экспортироваться из собственной DLL расширения. Точная сигнатура функции показана ниже.

```
HRESULT CALLBACK  
DebugExtensionInitialize(OUT PULONG Version, OUT PULONG Flags);
```

Критически важно, чтобы расширение отладчика экспортировало именно эту функцию, иначе расширение не загрузится. Основная цель этой функции - дать расширению отладчика возможность выполнить инициализацию, которая может потребоваться будущим командам (например, инициализировать другие отладочные интерфейсы, выделить ресурсы и т. д.). В случае нашего расширения основная работа, необходимая в функции инициализации, следующая:

- инициализировать информацию о версии;
- инициализировать требуемые COM-интерфейсы;
- инициализировать информацию о типах;
- инициализировать данные расширения WinDbg.

Первое, что должно сделать наше расширение, - инициализировать информацию о версии.

Инициализация информации о версии

Для расширения отладчика важно передать информацию о своей версии отладочному движку. Функция `DebugExtensionInitialize`, вызываемая отладочным движком при загрузке расширения, предоставляет механизм для сообщения этой информации. Сигнатура функции передает два параметра, один из которых можно использовать для этой цели.

- `Version`

Структура версии должна быть заполнена DLL расширения во время инициализации, чтобы указать, какая версия расширения отладчика загружается. Старшие 16 бит этого параметра `ULONG` представляют `major`-версию, а младшие 16 бит - `minor`-версию. Проще всего построить версию с помощью вспомогательного макроса `DEBUG_EXTENSION_VERSION`:

```
*Version = DEBUG_EXTENSION_VERSION(Major, Minor)
```

- `Flags`

Параметр `flags` зарезервирован и всегда должен устанавливаться в 0.

Один важный момент: когда вызывается эта функция, не гарантируется наличие активного сеанса отладки, и функция не должна предполагать, что информация о сеансе доступна.

Инициализация COM-интерфейсов

Следующая задача, которую должен решить код инициализации, - инициализировать COM-интерфейсы отладочного движка, чтобы мы могли эффективно взаимодействовать с отладочным движком и получать структуры данных, необходимые для выполнения наших команд. Для вывода двоичного дерева мы будем использовать следующие COM-интерфейсы:

- `IDebugClient`
- `IDebugControl`
- `IDebugDataSpaces`
- `IDebugSymbols`
- `IDebugSymbols3`

Самый первый COM-интерфейс, к которому нам нужен доступ, - `IDebugClient`. Этот интерфейс нужен, чтобы получить доступ к структуре, требуемой моделью `WDbgExts` для правильной работы, а также чтобы запрашивать все остальные интерфейсы. Чтобы получить указатель интерфейса на COM-интерфейс `IDebugClient`, нужно вызвать API `DebugCreate`:

```
HRESULT DebugCreate(IN REFIID InterfaceId,OUT PVOID* Interface);
```

Идентификатор интерфейса можно легко получить с помощью расширения компилятора __uuidof, ожидающего объявление COM-интерфейса как часть выражения. Далее можно использовать интерфейс IDebugClient, чтобы запросить остальные COM-интерфейсы, которые мы будем использовать позже.

Код, инициализирующий COM-интерфейсы, показан в листинге 11.3.

Листинг 11.3

```
//
// Initialize required COM interface pointers
//
if(FAILED(hRes=DebugCreate(__uuidof(IDebugClient),
(void**) &pDebugClient)))
{
dprintf( "Failed to get required COM interface\n");
return hRes;
}
if(FAILED(hRes=pDebugClient->QueryInterface(__uuidof(IDebugControl),
(void**) &pDebugControl)))
{
dprintf( "Failed to get required COM interface\n");
ReleaseComPointers();
return hRes;
}
if(FAILED((hRes=pDebugClient->QueryInterface(__uuidof(IDebugDataSpaces),
(void**) &pDataSpaces))))
{
dprintf( "Failed to get required COM interface\n");
ReleaseComPointers();
return hRes;
}
if(FAILED(hRes=pDebugClient->QueryInterface(__uuidof(IDebugSymbols),
(void**) &pSymbols)))
{
dprintf( "Failed to get required COM interface\n");
ReleaseComPointers();
return hRes;
}
if(FAILED(hRes=pDebugClient->QueryInterface(__uuidof(IDebugSymbols3),
(void**) &pSymbols3)))
{
dprintf( "Failed to get required COM interface\n");
ReleaseComPointers();
return hRes;
}
}
```


Инициализация информации о типах

Следующая часть кода инициализации получает информацию о типах двоичного дерева. Чтобы понять, зачем нужен этот шаг, нужно понимать, как расширение отладчика получает данные, связанные с двоичным деревом. Когда пользователь вводит команду `dumptree`, он указывает адрес, соответствующий адресу корневого узла дерева. Задача расширения отладчика - прочитать X байт из этого адреса и интерпретировать прочитанные байты так, будто они представляют двоичное дерево. Одно возможное решение - включить в расширение отладчика заголовочный файл двоичного дерева и использовать его как полноценный тип; однако у такого подхода есть большой недостаток. Расширение отладчика жестко типизировано под конкретную версию двоичного дерева, и если реализация двоичного дерева изменится в будущей версии (например, будут добавлены члены данных), потребуется перекомпилировать расширение для работы с новым типом двоичного дерева и хранить разные версии расширения. Чтобы обойти этот недостаток, отладочный движок включает набор API, позволяющих расширению запрашивать информацию о типах во время выполнения, тем самым минимизируя зависимость от типа на этапе компиляции. Эти API чрезвычайно мощны и позволяют выполнять глубокий анализ типов. В нашем расширении нам нужно знать тип узла двоичного дерева. Более конкретно, нужно найти смещение каждого члена данных (правого дочернего указателя, левого дочернего указателя и члена данных), чтобы знать, откуда начинать чтение данных относительно адреса, указанного пользователем. Размещение узла двоичного дерева в памяти выглядит так:

```
+0x000 data : Int4B
+0x004 pLeftChild : Ptr32 CBinaryTree::_TreeNode
+0x008 pRightChild : Ptr32 CBinaryTree::_TreeNode
```

Каждое число с префиксом + представляет смещение соответствующего члена в типе. Шаги, необходимые для программного определения смещений, следующие:

1. Получить модуль с помощью `IDebugSymbols::GetModuleByModuleName`, чтобы использовать его на последующих шагах. Интересующий модуль - тот, где определен тип. В нашем случае это модуль `bstree`, содержащий тип `CBinaryTree::_TreeNode`.
2. Получить идентификатор интересующего нас типа (`CBinaryTree::_TreeNode`) с помощью API `IDebugSymbols3::GetTypeId`. Результатом вызова API является идентификатор типа, который можно использовать на последующих шагах.
3. Получить смещение каждого члена с помощью API `IDebugSymbols::GetFieldOffset`. Полученные смещения затем будут использоваться расширением для чтения информации о типе.

Код, инициализирующий информацию о типах, показан в листинге 11.4.

Листинг 11.4

```
//
// Initialize type information
//
if(FAILED(hRes=pSymbols->GetModuleByModuleName("bstree",
0,
NULL,
&pBase)))
{
dprintf("Failed to get module information for bstree.exe\n");
ReleaseComPointers();
return hRes;
}
if(FAILED(hRes=pSymbols3->GetTypeId(pBase,
"CBinaryTree::_TreeNode",
&pNodeIndex)))
{
dprintf("Failed to get type id\n");
ReleaseComPointers();
return hRes;
}
```

```

}
if(FAILED(hRes=pSymbols->GetTypeSize(pBase,
pNodeIndex,
&pSize)))
{
dprintf("Failed to get type size\n");
ReleaseComPointers();
return hRes;
}
if(FAILED(hRes=pSymbols->GetFieldOffset(pBase,
pNodeIndex,
"pLeftChild",
&ulLeftOffset)))
{
dprintf("Failed to get left child offset\n");
ReleaseComPointers();
return hRes;
}
if(FAILED(hRes=pSymbols->GetFieldOffset(pBase,
pNodeIndex,
"pRightChild",
&ulRightOffset)))
{
dprintf("Failed to get right child offset\n");
ReleaseComPointers();
return hRes;
}
if(FAILED(hRes=pSymbols->GetFieldOffset(pBase,
pNodeIndex,
"data",
&ulDataOffset)))
{
dprintf("Failed to get data offset\n");
ReleaseComPointers();
return hRes;
}
}

```

Инициализация данных расширения WinDbg

Последняя задача расширения - инициализировать данные расширения WinDbg, необходимые при работе с моделью расширений WinDbg. Для этой цели можно использовать `IDebugControl::GetWinDbgExtensionApis64`; он просто принимает указатель на экземпляр `WINDBG_EXTENSION_APIS64`.

Код, инициализирующий эту информацию, показан в листинге 11.5.

Листинг 11.5

```

//
// Initialize WinDbg extension data
//
ExtensionApis.nSize=sizeof(ExtensionApis);
hRes=pDebugControl->GetWinDbgExtensionApis64(&ExtensionApis);

```

Изменения состояния сеанса

После успешного выполнения функции инициализации отладочный движок вызывает функцию `DebugExtensionNotify`, экспортируемую DLL расширения. Эта функция позволяет отладочному движку уведомлять расширение отладчика о любых изменениях состояния сеанса. Если сеанс становится активным или неактивным, расширение отладчика должно отметить это, если ему нужно приостанавливать отдельные части своего кода. Аналогично, если сеанс отладки становится доступным или недоступным, это также следует записать. Сигнатура функции показана ниже.

```
void CALLBACK
DebugExtensionNotify ( IN ULONG Notify, IN ULONG64 Argument )
```

Отладочный движок вызывает эту функцию (в DLL расширения) и указывает причину уведомления (см. SDK) в параметре `Notify`, а для `Argument` передает 0 (зарезервировано). Различные уведомления перечислены в таблице 11.5.

Таблица 11.5

Состояние	Описание
DEBUG_NOTIFY_SESSION_ACTIVE	Активен сеанс отладки. Сеанс не обязательно остановлен.
DEBUG_NOTIFY_SESSION_INACTIVE	Активного сеанса отладки нет.
DEBUG_NOTIFY_SESSION_ACESSIBLE	Сеанс отладки остановился и теперь доступен.
DEBUG_NOTIFY_SESSION_INACCESSIBLE	Сеанс отладки начал выполняться и теперь недоступен.

KnownStructOutput

Следующий вызов функции, выполняемый отладочным движком во время загрузки, направлен в `KnownStructOutput`. Отладчики Windows имеют удобную возможность, позволяющую расширениям отладчика поддерживать собственное отображение типа данных в одной строке (например, когда пользователь выполняет команду `dt`). Вместо того чтобы вызывать каждое расширение для каждой выполненной команды `dt`, отладочный движок при первой загрузке DLL вызывает функцию `KnownStructOutput` и спрашивает у расширения имена типов данных, для которых оно поддерживает собственное форматирование. В случае загрузки ответственность расширения отладчика - вернуть список собственных типов данных, если они поддерживаются. Если в отладчик когда-либо будет отправлен вызов `dt`, отладочный движок знает, какую версию `KnownStructOutput` какого расширения вызывать, на основе накопленного списка, возвращенного во время загрузки. Пример реализации `KnownStructOutput` мы покажем далее в главе.

Деинициализация расширения

Финальный вызов функции отладочный движок выполняет, когда пользователь выгружает DLL расширения (с помощью команды `.unload`); в этот момент отладочный движок вызывает `DebugExtensionUninitialize` в расширении отладчика. Сигнатура функции показана ниже.

```
void CALLBACK DebugExtensionUninitialize(void);
```

Расширение отладчика должно предоставлять эту функцию, если ему нужно выполнить какую-либо очистку при выгрузке расширения. Пример очистки - освобождение любых глобальных ресурсов, полученных во время инициализации (например, освобождение глобальных COM-экземпляров или памяти).

Поскольку код инициализации нашего расширения отладчика получил COM-интерфейсы, важно уменьшить счетчик ссылок для всех указателей интерфейсов, чтобы избежать утечки:

```
extern "C" void CALLBACK DebugExtensionUninitialize(void)
{
    ReleaseComPointers();
    return;
}
VOID ReleaseComPointers()
{
    if(pDebugClient) pDebugClient->Release();
    if(pDebugControl) pDebugControl->Release();
}
```

```

if(pSymbols) pSymbols->Release();
if(pSymbols3) pSymbols3->Release();
if(pDataSpaces) pDataSpaces->Release();
}

```

Реализация команды Help

Теперь, когда код инициализации и деинициализации уже разобран, можно перейти к более интересному коду собственно команд, которые мы хотим реализовать.

Все собственные отладочные команды должны соответствовать следующей сигнатуре и состоять только из строчных букв:

```

HRESULT CALLBACK
(* PDEBUG_EXTENSION_CALL)
(
    IN IDebugClient* Client,
    IN OPTIONAL PCSTR Args
) ;

```

PDEBUG_EXTENSION_CALL - это сама функция, и она должна называться в соответствии с реализуемой командой расширения отладчика. Реализация команды help нашего расширения показана ниже:

```

// This function displays help for the debugger extension
HRESULT CALLBACK help ( PDEBUG_CLIENT Client, PCSTR Args )
{
    dprintf ("SYSYEXTS.DLL commands:\n"
    "help = Shows this help\n"
    "\tdumptree <address>=Dumps out the contents of a CBinaryTree. Address
    must point to the root node of the tree"
    );
    return S_OK ;
}

```

Как видно, функция сводится к простому использованию dprintf для печати справки расширения отладчика. Функция dprintf по сути является эквивалентом printf для расширений отладчика.

Чтобы проверить расширение, запустим произвольное приложение под отладчиком. (Обычно я использую notepad.exe.) Когда вы остановились на точке останова в отладчике, введите следующую команду:

```
.load C:\AWDBIN\WinXP.x86.chk\sysexts.dll
```

В этот момент DLL расширения загружена в адресное пространство отладчика, и вызывается функция инициализации. Если ошибок нет, перед вами снова появляется отладочный prompt. Введите !sysexts.help, чтобы вызвать команду help, и вы увидите следующее:

```

0:000> !sysexts.help
SYSYEXTS.DLL commands:
Help = Shows this help
dumptree <address> = Dumps out the contents of a CBinaryTree. Address must
point to the root node of the tree

```

Если при загрузке расширения отладчика или выполнении команды возникает ошибка, например выбрасывается исключение, ошибка будет показана как общее исключение в отладочном выводе, но не приведет к аварийному завершению процесса отладки. Отладочный движок оборачивает все вызовы в собственное расширение отладчика обработчиком исключений, чтобы неисправное расширение не могло обрушить весь сеанс отладки. При этом если проблема достаточно серьезна (например, запись в память, не принадлежащую расширению), есть вероятность, что сеанс отладки в какой-то момент все равно завершится неудачей. Если вы подозреваете, что расширение работает неправильно, нужно отладить его и выяснить, что не так, прежде чем разворачивать

расширение. Интересный вопрос: как именно отлаживать расширение? Хотя ответ может показаться несколько странным, он совершенно логичен: просто отлаживайте отладчик. Подключите отладчик к отладчику, и когда произойдет сбой, он остановится так же, как в любом другом сеансе отладки.

Теперь можно перейти к реализации немного более сложной команды - вывода содержимого нашего двоичного дерева.

Реализация команды `dumptree`

Основная идея команды `dumptree` - дать возможность легко выводить содержимое двоичного дерева типа `CBinaryTree`. Сам синтаксис команды показан ниже.

```
!sysexts.dumptree <address to root pointer of the binary tree>
```

Сигнатура функции и пустое тело выглядят следующим образом:

```
// This function dumps the CBinaryTree received in the Args
HRESULT CALLBACK dumptree(PDEBUG_CLIENT Client, PCSTR Args)
{
    HRESULT hRes=S_OK;
    return hRes;
}
```

Прежде чем слишком углубляться в детали реализации, опишем общую стратегию команды `dumptree`. Алгоритм, используемый этой командой, будет таким:

1. Извлечь и проверить адрес, переданный пользователем.
2. Прочитать корневой узел дерева из памяти на основе указанного адреса.
3. Вызвать рекурсивную функцию `Traverse` для симметричного (in-order) обхода, которая отображает элементы узлов, пока дерево не будет исчерпано.

Нам нужен адрес корневого узла дерева. Он указывается пользователем при вводе команды в сеансе отладки, например:

```
!sysexts.dumptree 00181eb4
```

При взгляде на предыдущую строку должно возникнуть несколько вопросов. Как получить доступ к указанному адресу? Как убедиться, что пользователь ввел допустимый адрес? Эти вопросы подводят нас к первой важной части реализации сложных команд: извлечению и проверке пользовательского ввода.

Параметры, которые пользователь вводит как часть команды, передаются в вашу функцию через параметр `Args`. Параметр `Args` по сути является строкой, содержащей всю только что введенную командную строку за вычетом имени самой команды. Если пользователь ввел следующую команду:

```
!sysexts.dumptree 00181eb4
```

параметр `Args` будет содержать:

```
00181eb4
```

Поскольку наша цель - читать память по указанному адресу, нельзя принимать этот параметр как тип `PCSTR` (указатель на строку); в идеале нужно преобразовать это значение в указатель, который можно передать API доступа к памяти. API, позволяющий этого добиться, - `GetExpression`, объявленный так:

```
ULONG_PTR GetExpression ( PCSTR expression )
```

При вызове API `GetExpression` принимает переданный параметр и пытается вычислить его в `ULONG_PTR`. Если вычисление успешно, API возвращает `ULONG_PTR`. Затем этот указатель можно

использовать при вызове API доступа к памяти. Если API завершается неудачно, возвращается null.

Использование GetExpression в реализации dumptree показано ниже.

```
HRESULT CALLBACK dumptree(PDEBUG_CLIENT pClient, PCSTR szArgs)
{
    HRESULT hRes=S_OK;
    ULONG_PTR pAddress=(ULONG_PTR) GetExpression(szArgs);
    if(!pAddress)
    {
        dprintf("Invalid head pointer address specified: %s\n", szArgs);
        return E_FAIL;
    }
    InOrderTraversal(pAddress);
    return hRes;
}
```

Предыдущий код довольно прямолинеен. Мы просто передаем параметр szArgs в API GetExpression. Если возвращаемое значение не равно null, продолжаем; иначе печатаем сообщение в окно отладочной консоли о том, что указан недопустимый адрес, и возвращаем ошибку.

Теперь, когда мы извлекли и проверили адрес, указанный как часть команды dumptree, можно продолжить, вызвав вспомогательную функцию InOrderTraversal, которая выполняет основную работу при обходе и отображении информации об узлах дерева.

Функция InOrderTraversal рекурсивно обходит дерево и печатает информацию об узлах дерева с помощью вспомогательной функции GetNodeValues. GetNodeValues выполняет интересную работу - фактически читает содержимое памяти, связанное с узлом дерева. Реализация GetNodeValues показана здесь.

```
HRESULT GetNodeValues(ULONG64 pNode,
    ULONG* pulData,
    ULONG* pulLeft,
    ULONG* pulRight)
{
    if(FAILED(hRes=pDataSpaces->ReadVirtual((ULONG_PTR) pNode+ulDataOffset,
        pulData,
        sizeof(ULONG),
        NULL)))
    {
        dprintf("Failed to read memory at address: 0x%X\n",
            pNode+ulDataOffset);
        return E_FAIL;
    }
    if(FAILED(hRes=pDataSpaces->ReadVirtual((ULONG_PTR) pNode+ulLeftOffset,
        pulLeft,
        sizeof(ULONG),
        NULL)))
    {
        dprintf("Failed to read memory at address: 0x%X\n",
            pNode+ulLeftOffset);
        return E_FAIL;
    }
    if(FAILED(hRes=pDataSpaces->ReadVirtual((ULONG_PTR) pNode+ulRightOffset,
        pulRight,
        sizeof(ULONG),
        NULL)))
    {
        dprintf("Failed to read memory at address: 0x%X\n",
            pNode+ulRightOffset);
        return E_FAIL;
    }
    return S_OK;
}
```

На высоком уровне функция читает память, связанную с каждым членом данных, используя API `IDataSpaces::ReadVirtual`. Обратите внимание, что мы используем ранее инициализированные смещения, чтобы получить правильные члены данных. API `ReadVirtual` определен следующим образом. Он позволяет читать память из виртуального адресного пространства цели отладчика:

```
HRESULT IDataSpaces::ReadVirtual (
    IN ULONG64 Offset,
    OUT PVOID Buffer,
    IN ULONG BufferSize,
    OUT OPTIONAL PULONG BytesRead
) ;
```

Параметр `Offset` - адрес, из которого нужно читать, параметр `Buffer` при успешном чтении содержит прочитанные данные, параметр `BufferSize` указывает размер переданного буфера данных, а `BytesRead` по завершении содержит фактическое число байтов, прочитанных API. Если функция успешна, возвращается `S_OK`.

Последняя вспомогательная функция в нашем расширении - `PrintNode`; она просто печатает данные, левый дочерний указатель и правый дочерний указатель узла дерева.

Теперь можно использовать расширение, чтобы очень просто и удобно выводить деревья произвольного размера. Давайте попробуем. Запустите приложение `bstree.exe` под отладчиком и установите точку останова в функции `Traverse`. Получите головной указатель (как показано ранее) и вызовите команду `dumptree`:

```
...
...
...
0:000> .load C:\AWDBIN\WinXP.x86.chk\sysexts.dll
0:000> dv
this = 0x00262c20
0:000> dt this
Local var @ 0x6ff1c Type CBinaryTree*
0x00262c20
+0x000 m_pHead : 0x00262c40
+0x004 m_dwDepth : 3
+0x008 m_dwNumNodes : 5
0:000> !sysexts.dumptree 0x00262c40
** Node **
Data: 5
Left child pointer: 0x0
Right child pointer: 0x262cb8
** Node **
Data: 7
Left child pointer: 0x0
Right child pointer: 0x0
** Node **
Data: 10
Left child pointer: 0x262c68
Right child pointer: 0x262c90
** Node **
Data: 13
Left child pointer: 0x0
Right child pointer: 0x0
** Node **
Data: 15
Left child pointer: 0x262ce0
Right child pointer: 0x0
```

Обратите внимание, что пример использования команды `dumptree` не включает вывод функции `KnownStructOutput`, которую мы обсудим в следующем разделе.

Несколько замечаний о команде `dumptree`. Если вы когда-либо столкнетесь с поврежденным деревом (то есть один из указателей в узлах указывает на недопустимый адрес), команда `dumptree` зафиксирует это ошибкой в соответствии с нашим кодом. Обычно это отличное указание на поврежденное дерево. Дальнейшую отладку следует начинать с этого места (см. главу 6,

"Повреждение памяти, часть II - кучи").

Реализация функции KnownStructOutput

Функция KnownStructOutput - удобный способ расширить функциональность обычной команды dt. По умолчанию команда dt отображает содержимое указанного типа данных (если доступно достаточно символьной информации). Если бы мы вывели экземпляр данных CBinaryTree::_TreeNode с помощью команды dt, получили бы следующее:

```
0:000> dt CBinaryTree::_TreeNode
+0x000 data : Int4B
+0x004 pLeftChild : Ptr32 CBinaryTree::_TreeNode
+0x008 pRightChild : Ptr32 CBinaryTree::_TreeNode
```

Было бы удобно настроить вывод команды dt при применении к типу данных CBinaryTree::_TreeNode. Реализация функции KnownStructOutput позволяет добавить эту дополнительную информацию без особого труда.

Прототип функции KnownStructOutput показан ниже.

```
HRESULT CALLBACK KnownStructOutput (
    IN ULONG Flag,
    IN ULONG64 Address,
    IN PSTR StructName,
    OUT PSTR Buffer,
    IN OUT PULONG BufferSize
) ;
```

Эта функция имеет двойственное поведение: в зависимости от переданного флага она либо возвращает имена поддерживаемых типов, либо результат вычисления заданного адреса для конкретного типа. Возможные флаги показаны в таблице 11.6.

Таблица 11.6

Флаг	Описание
DEBUG_KNOWN_STRUCT_GET_NAMES	Передается функции, когда отладочный движок впервые загружает расширения отладчика. Реализация расширения должна вернуть список имен структур, для которых оно поддерживает собственный вывод информации.
DEBUG_KNOWN_STRUCT_GET_SINGLE_LINE_OUTPUT	Передается функции, когда расширение отладчика должно выполнить фактическое вычисление.
DEBUG_KNOWN_STRUCT_SUPPRESS_TYPE_NAME	Передается функции для запроса, хочет ли она, чтобы имя типа печаталось автоматически.

В нашем случае имя типа данных, для которого мы хотим предоставить дополнительную информацию, - структура CBinaryTree::_TreeNode. Каркас кода функции KnownStructOutput показан ниже.

```
HRESULT CALLBACK KnownStructOutput (
    IN ULONG Flag,
    IN ULONG64 Address,
    IN PSTR StructName,
    OUT PSTR Buffer,
    IN OUT PULONG BufferSize
)
{
    HRESULT hRes=E_FAIL;
    if(Flag==DEBUG_KNOWN_STRUCT_GET_NAMES)
    {
        //
        // Return the list of names of supported data types
        //
    }
```



```

}
else if(Flag==DEBUG_KNOWN_STRUCT_GET_SINGLE_LINE_OUTPUT)
{
//
// Return the result of evaluating a data type
//
}
else if(Flag==DEBUG_KNOWN_STRUCT_SUPPRESS_TYPE_NAME)
{
//
// Return S_OK to indicate that we want the type name automatically
// printed.
//
return S_OK;
}
else
{
dprintf("KnownStructOutput called with invalid flags\n");
}
return hRes;
}

```

Начнем с объяснения того, как вернуть список имен структур данных, которые мы поддерживаем. Список имен возвращается через OUT-параметр `Buffer`, имеющий тип `PSTR`. Критически важно убедиться, что в `Buffer` достаточно места для успешной записи всех имен типов данных. Размер буфера передается через параметр `BufferSize`. Вот наша реализация получения имен типов данных:

```

if(Flag==DEBUG_KNOWN_STRUCT_GET_NAMES)
{
if ((*BufferSize)<strlen(SYSEXTS_KNOWNSTRUCT_1)+2)
{
// Not enough buffer available, return S_FALSE
(*BufferSize)=strlen(SYSEXTS_KNOWNSTRUCT_1)+2;
hRes=S_FALSE;
}
else
{
hRes=StringCchPrintfA(Buffer,
(*BufferSize)-2,
"%s\\0",
SYSEXTS_KNOWNSTRUCT_1);
if (FAILED(hRes))
{
dprintf ("Failed to copy the data type name into buffer\n");
}
}
}
}

```

`SYSEXTS_KNOWNSTRUCT_1` определен в файле `sysexts.cpp` следующим образом:

```

#define SYSEXTS_KNOWNSTRUCT_1 "CBinaryTree::_TreeNode"

```

Обратите внимание, что сначала мы проверяем, меньше ли длина имени типа данных, чем переданный размер буфера. (+2 учитывает оба нулевых терминатора.) Если размер меньше, мы продолжаем, заполняя имя и возвращаясь. Если места недостаточно, мы помещаем требуемую длину в параметр `BufferSize` и возвращаем `S_FALSE`, что указывает отладочному движку выделить больший буфер указанного размера и попробовать вызвать функцию снова.

Если бы мы поддерживали собственное форматирование нескольких типов данных, мы поместили бы каждое имя в буфер, разделив их `NULL`-терминаторами и завершив всю строку еще одним нулевым терминатором.

Теперь перейдем к реализации вычислителя. В этом случае отладочный движок передает нам флаг со значением `DEBUG_KNOWN_STRUCT_GET_SINGLE_LINE_OUTPUT` и адрес структуры данных. Адрес можно передать нашей вспомогательной функции `GetNodeValues`, определенной ранее; она

читает содержимое узла по указанному адресу и возвращает значения узла. После успешного чтения значений узла можно отформатировать вывод и вернуть его отладочному движку. Обратите внимание, что мы особенно следим за тем, чтобы не превысить размер буфера, явно проверяя, поместятся ли данные:

```
else if(Flag==DEBUG_KNOWN_STRUCT_GET_SINGLE_LINE_OUTPUT)
{
    ULONG ulData=0;
    ULONG ulLeft=0;
    ULONG ulRight=0;
    if(FAILED(hRes=GetNodeValues(Address, &ulData, &ulLeft, &ulRight)))
    {
        return hRes;
    }
    DWORD dwLen=_scprintf(SYSEXTS_KNOWNSTRUCT_OUT,
        ulData,
        ulLeft,
        ulRight)+1;
    if(dwLen>(*BufferSize))
    {
        dprintf("KnownStructOutput unable to fit return data into buffer\n");
        hRes=E_FAIL;
        return hRes;
    }
    hRes=StringCchPrintfA(Buffer,
        dwLen,
        SYSEXTS_KNOWNSTRUCT_OUT,
        ulData,
        ulLeft,
        ulRight);
    if(FAILED(hRes))
    {
        dprintf ("KnownStructOutput unable to write data into buffer\n");
    }
}
```

Теперь мы реализовали расширение стандартной команды dt, которое позволяет видеть содержимое корневого узла в дополнение к данным, выводимым командой dt. Давайте попробуем. Запустите bstree.exe под отладчиком, установите точку останова в методе traverse и выполните команду dt для экземпляра корневого узла CBinaryTree:

```
0:000> dt CBinaryTree::_TreeNode 0x000369c8
Binary tree node contents
Data: 10
Left Child Pointer: 0x36fb0
Right Child Pointer: 0x36fd8
+0x000 data : 10
+0x004 pLeftChild : 0x00036fb0
Binary tree node contents
Data: 5
Left Child Pointer: 0x0
Right Child Pointer: 0x37000
+0x008 pRightChild : 0x00036fd8
Binary tree node contents
Data: 15
Left Child Pointer: 0x37028
Right Child Pointer: 0x0
```

Реализация отмены команд

Как и в любой потенциально долгой задаче, возможность отменить команду становится важной. Представьте, что мы попытались вывести экземпляр `CBinaryTree` с тысячами и тысячами узлов и захотели отменить команду на полпути. В текущем виде наша команда `dumptree` не отменяема, что может раздражать пользователей этого расширения отладчика. Как добавить поддержку отмены? Ответ на самом деле довольно прост. API (часть модели `WDbgExts`) позволяет расширениям отладчика проверять нажатие `CTRL+C` пользователем:

```
ULONG CheckControlC (VOID);
```

Вызывая этот API через регулярные интервалы в долгих командах, мы можем убедиться, что уважаем желание пользователя отменить выполнение. Помимо добросовестного соблюдения запросов `CTRL+C` от пользователя, это дает дополнительную ценность: позволяет пользователю отменить команду `dumptree`, которая потенциально никогда бы не завершилась (за исключением переполнения стека из-за рекурсивной природы реализации). Если по какой-либо причине в реализации `CBinaryTree` проявится ошибка, из-за которой дерево начнет указывать само на себя, мы потенциально можем рекурсировать бесконечно.

В нашем случае команда `dumptree` вызывает рекурсивную функцию, которая продолжает печатать узлы, пока дерево не будет полностью исчерпано. Если поместить эту проверку перед началом каждого рекурсивного вызова, можно просто завершить рекурсию и вернуть управление пользователю:

```
VOID InOrderTraversal(CBinaryTree::_TreeNode* pNode, IDebugDataSpaces* pDataSpaces)
{
    HRESULT hRes;
    if(CheckControlC()==TRUE)
    {
        dprintf("Control C hit, canceling command\n");
        return;
    }
    //
    // Rest of the InOrderTraversal function code
    //
}
```

Если немного изменить клиентский код `CBinaryTree`, чтобы добавить 1000 узлов в экземпляр дерева, запустить его под отладчиком, а затем выполнить команду `dumptree`, можно нажать `CTRL+C` для отмены команды:

```
0:000> !sysexts.dumptree 0x262D48
** Node **
Data: 0
Left child pointer: 0x0
Right child pointer: 0x262d70
** Node **
Data: 1
Left child pointer: 0x0
Right child pointer: 0x262d98
** Node **
Data: 2
Left child pointer: 0x0
...
...
...
...
** Node **
Data: 156
Left child pointer: 0x0
Right child pointer: 0x265f38
** Node **
Data: 157
Left child pointer: 0x0
```

```
Right child pointer: 0x265f60
Control C hit, canceling command
0:000>
```

Примечание о символах. Как описано в главе 4, "Управление файлами символов и исходного кода", во время отладки по сути используются два разных типа файлов символов: публичные символы (также известные как *stripped symbols*) и частные символы. Главное различие между этими двумя файлами символов - объем информации, доступной разработчику, отлаживающему приложение. Публичные символы предоставляют очень ограниченную символьную информацию (что делает отладку довольно трудной), а частные символы содержат полную символьную информацию. Любой разработчик предпочел бы частные символы для отладки приложения. Проблема возникает при выпуске продукта. Обычно компании разрешают доступ (через сервер символов) к ограниченным публичным символам, чтобы обеспечить хотя бы базовую форму отладки, но углубленная отладка становится очень сложной из-за отсутствующей символьной информации, например информации о типах. Собственные расширения отладчика могут помочь в ситуациях, когда вы не хотите записывать CD с частными символами каждый раз, когда нужно отладить проблему за пределами площадки. Рассмотрим следующий сценарий, в котором нужно отладить *CBinaryTree*, имея только публичные символы. Попытка вывести содержимое отлаживаемого экземпляра *CBinaryTree* завершится неудачно, потому что публичные символы не предоставляют члены типа данных как часть своей символьной информации. Без частных символов и без знания точного размещения структуры *CBinaryTree* что делать? Можно развернуть расширение *sysexts.dll* на этой машине и использовать расширенную информацию, доступную при вводе *dt*. Механизм поддержки расширенной информации о типах в вашем расширении отладчика - очень полезный прием, позволяющий избегать ситуаций, когда доступна только ограниченная символьная информация.

Версионирование

Как и в любом другом компоненте, версионирование важно и в расширениях отладчика. На протяжении жизни приложения структуры данных меняют форматы, и расширения отладчика придется дорабатывать, чтобы они работали с новыми структурами данных и версиями приложений. Чтобы учесть будущие изменения, придется создать новую версию расширения отладчика (с новым именем, например *sysexts20.dll*) и использовать ее во время отладки. Ведение полного списка таких расширений отладчика стоит потраченного времени. Можно даже представить написание команды расширения отладчика, которая запрашивает глобальную переменную версии приложения и загружает правильную версию расширения отладчика.

После установки отладчиков Windows вы заметите, что в публичных отладчиках Windows версионирование обрабатывается путем размещения DLL расширений в каталогах, соответствующих каждому выпуску компонента. Например, посмотрев на расширение отладчика *userexts.dll*, можно увидеть, что оно существует в следующих подкаталогах пути установки отладчиков Windows по умолчанию:

```
C:\Program Files\Debugging Tools for Windows>dir usere* /s
Volume in drive C has no label.
Volume Serial Number is 688E-2C02

Directory of C:\Program Files\Debugging Tools for Windows\nt4chk

04/16/2004 08:59 AM 98,845 userexts.dll
1 File(s) 98,845 bytes
```

Directory of C:\Program Files\Debugging Tools for Windows\nt4fre

04/16/2004 08:59 AM 99,357 userexts.dll
1 File(s) 99,357 bytes

Directory of C:\Program Files\Debugging Tools for Windows\w2kchk

04/16/2004 08:59 AM 171,037 userexts.dll
1 File(s) 171,037 bytes

Directory of C:\Program Files\Debugging Tools for Windows\w2kfre

04/16/2004 08:59 AM 170,013 userexts.dll
1 File(s) 170,013 bytes

В зависимости от версии операционной системы, которую вы отлаживаете, нужно загрузить правильную версию расширения отладчика userexts.dll. Версию операционной системы можно узнать, выполнив отладочную команду !version.

Сборка расширения отладчика

Чтобы скомпилировать DLL расширения, сначала нужно убедиться, что в консоль сборки DDK добавлены правильные переменные среды. Обычно я добавляю следующие переменные среды прямо в общесистемные параметры среды, чтобы не беспокоиться об их задании в разных консолях:

```
DBGSDK_INC_PATH=C:\PROGRA~1\debugg~1\sdk\inc  
DBGSDK_LIB_PATH=C:\PROGRA~1\debugg~1\sdk\lib\*
```

Убедитесь, что пути для переменных среды соответствуют структуре каталогов на вашей машине (а также платформенно-специфическим подкаталогам). Нужно внести небольшую корректировку в файл sources и указать дополнительные пути, а также убедиться, что мы компонуемся с dbgeng.lib:

```
TARGETLIBS= \  
$(CRT_LIB_PATH)\MSVCRT.lib \  
$(SDK_LIB_PATH)\kernel32.lib \  
$(DBGSDK_LIB_PATH)\dbgeng.lib \  
  
INCLUDES= $(DDK_INC_PATH); \  
$(DBGSDK_INC_PATH); \
```

Оба этих раздела могут содержать больше библиотек или путей include в зависимости от точной природы расширения отладчика, но для нашего собственного расширения достаточно того, что показано в этом листинге. После загрузки и установки исходного кода с веб-страницы книги перейдите в папку C:\AWD\Chapter11\sysexts и введите:

```
build /ZCc
```

При успешной сборке результирующий sysexts.dll помещается в выходной каталог. Имя выходного каталога зависит от типа открытой среды сборки (checked или free), а также от операционной системы. В окне консоли WinXP checked build (x86) двоичный файл помещается в:

```
objchk_wxp_x86\i386\sysexts.dll
```

Теперь можно запустить отладчик, загрузить расширение и выполнить любую из только что реализованных команд.

Резюме

Как вы видели в этой главе, понятие собственных расширений отладчика позволяет разработчику автоматизировать процесс отладки, разрабатывая собственные команды для улучшения сеансов отладки. Это действительно отличный инструмент, который должен быть в арсенале каждого системного разработчика. Хотя мы лишь слегка затронули поверхность расширений отладчика, стоит потратить время на изучение документации SDK и знакомство с большим числом API, доступных для разработки расширений. Команда отладчиков Microsoft приложила большие усилия, чтобы сделать debug API глубоким и открыть практически неограниченные возможности по расширению стандартного опыта отладки.

Глава 12. 64-битная отладка

С появлением 64-битных процессоров необходимость понимать, как эти процессоры работают и как отлаживать 64-битные приложения, стала критически важной. Эта глава описывает основы 64-битной отладки. Она построена как дополнение ко всем предыдущим главам, где листинги отладчика были сняты с систем, работавших под 32-битной версией операционной системы. В этой главе мы выделяем различия в поведении, когда хост-системы имеют 64-битную архитектуру. Порядок изложения соответствует порядку, в котором понятия вводились в книге.

Глава сосредоточена на 64-битной архитектуре, представленной AMD в 2004 году. Практически все компьютерные системы, продающиеся сегодня, способны запускать 64-битные операционные системы. Windows Vista - первая пользовательская операционная система, одновременно доступная в 64-битной и 32-битной архитектурах. Вопрос уже не в том, выпустят ли независимые поставщики ПО собственные 64-битные приложения. Это лишь вопрос времени, когда они выпустят 64-битные версии своих приложений.

В этой главе повторно используются несколько примеров, представленных в предыдущих главах. Двоичные файлы, нацеленные на Windows Vista x64, доступны в папках C:\AWDBIN\winLH.AMD64.chk и C:\AWDBIN\winLH.AMD64.fre. Если система Windows Vista x64 недоступна, в папке C:\AWDBIN\Dumps предоставлено несколько дампов памяти.

64-битные системы Microsoft

Microsoft официально вышла на арену 64-битных операционных систем с Microsoft Windows XP 64-Bit Edition, которая могла работать на системах с процессорами Intel Itanium. Как следует из названия, она была основана на кодовой базе Windows XP и была выпущена публично в 2002 году, вскоре после выпуска Windows XP 2001 года (32-Bit Edition).

Год спустя Microsoft выпустила Windows Server 2003 Enterprise Edition для систем на базе Itanium, способную поддерживать рабочие нагрузки, требуемые самыми высокими уровнями надежности, доступности и масштабируемости, предоставляемыми системами на базе Intel Itanium.

В том же году Advanced Micro Devices раскрыла новую 64-битную процессорную архитектуру, спроектированную как расширение текущей процессорной архитектуры x86. Новая архитектура процессора также была принята Intel и продвигалась под названием Intel Extended Memory 64 Technology, а позднее - Intel 64 Architecture. Поскольку различия между реализациями минимальны, эта архитектура также известна как x86-64. Microsoft использует название x64 для обозначения операционной системы, поддерживающей такие архитектуры.

В 2005 году Microsoft выпустила первую версию клиентской и серверной операционной системы с поддержкой x64-систем: Windows XP Professional x64 Edition и Windows Server 2003 SP1 x64 Edition соответственно. Обе версии имеют одинаковый номер сборки, что указывает на их построение из одной кодовой базы, и обладают возможностями, идентичными 32-битной версии Windows 2003 SP1.

В последнем квартале 2006 года Microsoft выпустила Windows Vista, следующую версию клиентских операционных систем Windows с поддержкой процессоров x86 и x64. Начиная с выпуска Windows Vista клиентская операционная система больше не поддерживает процессоры Intel Itanium.

Поскольку доступность систем на базе Intel Itanium относительно ограничена по сравнению с x64-системами, эта глава сосредоточена только на архитектуре x64. Различие между клиентской и серверной операционной системой с точки зрения отладки относительно невелико, поэтому далее мы будем использовать термин Windows x64 для обозначения таких систем. В этой главе мы также используем термин Windows x86 как общее обозначение операционной системы, построенной для 32-битной архитектуры.

Обзор операционной системы

В чем именно состоит различие между операционными системами Windows x64 и Windows x86? Без полного анализа обеих реализаций трудно понять все различия между ними. Настоятельно рекомендуется изучить книгу *Microsoft Windows Internals, 4th Edition* (2004, Mark E. Russinovich и David A. Solomon, Redmond, WA: Microsoft Press), которая отлично справляется с подробным анализом каждого компонента операционной системы. Эта глава дает очень краткий анализ различий и того, как они влияют на отладку.

Самый большой выигрыш от перехода на Windows x64 связан со способностью процессора обращаться к 64-битному плоскому виртуальному адресному пространству. Даже если 64-битный указатель может адресовать 16 эксабайт, текущая реализация Windows x64 ограничивается 16 терабайтами виртуального адресного пространства, поровну разделенного между адресами пользовательского режима и режима ядра. Это все равно огромное улучшение по сравнению с ограничением в 4 гигабайта, накладываемым архитектурой x86.

Чтобы эффективно поддерживать такое виртуальное адресное пространство, Windows x64 способна адресовать до одного терабайта физической памяти и дополнительно 512 терабайт в файле подкачки. Ограничение физической памяти выбрано с учетом аппаратных возможностей, доступных сегодня и в ближайшем будущем. В будущем это ограничение может быть снято, чтобы соответствовать развитию оборудования.

Как компания Microsoft несет огромную ответственность за сохранение обратной совместимости с существующим программным обеспечением, работающим на предприятиях, при каждой новой версии операционной системы. Но насколько Windows x64 совместима с текущими приложениями? Windows x64 может запускать программные приложения, скомпилированные нативно - то есть нацеленные на архитектуру x64, - а также программные приложения, нацеленные на традиционную Windows x86. Однако операционная система накладывает некоторые ограничения, которые должны быть ясно поняты и описаны далее.

Ядро Windows x64 выполняет строго 64-битный код и не может загружать модули, в общем случае драйверы устройств, предназначенные для операционных систем Windows x86. На практике это ограничение не затрагивает массовых пользователей, поскольку большинство поставщиков оборудования либо уже предоставляют, либо находятся в процессе предоставления 64-битных драйверов для своих устройств.

Все самостоятельные приложения, которые не зависят ни от чего, кроме компонентов операционной системы, работают одинаково после преобразования из 32-битной цели в 64-битную цель. Все ключи реестра и все расположения системных файлов остаются в Windows x64 неизменными по сравнению с Windows x86. Некоторых может удивить, что большинство системных двоичных файлов все еще хранится в папке %SystemRoot%\system32, даже если это чистые 64-битные двоичные файлы. Все приложения по-прежнему устанавливаются в папку %SystemDrive%\Program Files, как и в Windows x86.

Если нативное 64-битное приложение зависит от компонента, не поставляемого в составе операционной системы, и использует его, установщик приложения должен убедиться, что в системе доступны компоненты, скомпилированные как 64-битные. В текущей Windows x64 невозможно загрузить 32-битный двоичный файл в процесс, выполняющий 64-битное приложение. Например, приложение воспроизведения мультимедиа, нацеленное на Windows x64 и загружающее различные фильтры DirectShow, должно найти все такие фильтры, скомпилированные как 64-битные двоичные файлы.

Даже если большинство компаний перевыпустит свои приложения в виде 64-битных версий, большая часть существующих приложений все еще нацелена только на 32 бита. Для предприятий с тысячами 32-битных приложений непрактично рассматривать Windows x64, если им придется менять или заново покупать все эти приложения. К счастью, они поддерживаются с помощью подсистемы эмуляции Windows-On-Windows, известной как WOW64. Процесс, выполняющийся в режиме эмуляции WOW64, во всех практических отношениях является 32-битным процессом, как и его 64-битный аналог, и не способен загружать компоненты, нацеленные на 64-битную архитектуру.

Поскольку двоичные файлы разных архитектур нельзя смешивать в одном процессе, поставщики ПО должны понимать рынок своих приложений и соответственно выбирать целевую архитектуру. В зависимости от приложения существует несколько ситуаций, которые нужно обрабатывать по-разному.

Разработчики, поставляющие компоненты для Windows x64, также должны понимать, кто является их целевой аудиторией, и удостовериться, что архитектура компонента соответствует архитектуре приложения, которое его использует. Нередко один и тот же компонент доступен в обеих архитектурах в одной системе. Библиотеки счетчиков производительности Windows - хороший пример применения такой модели.

На другой стороне находятся автономные приложения, расширяемые через подключаемую архитектуру, например интернет-браузеры, поддерживающие различные надстройки. В такой ситуации разработчики должны понимать доступность подключаемых модулей и, возможно, выбрать поставку приложения в обеих архитектурах, пока переход к Windows x64 не будет завершен. Такой подход был выбран разработчиками Internet Explorer, предоставившими 32-битное и 64-битное приложение. В качестве альтернативы приложение может оставаться в 32-битной архитектуре до тех пор, пока весь рынок не перейдет к 64-битным вычислениям. Это решение было выбрано для Windows Media Player, поставляемого с Windows XP Professional x64.

Нативные x64-процессы работают бок о бок с другими приложениями, выполняющимися внутри WOW64-процессов, и все процессы совместно используют одни и те же глобальные ресурсы. Когда приложение является службой, оно может существовать в единственной форме - либо как 64-битное, либо как 32-битное приложение. Например, сервер не может запустить 64-битный и 32-битный экземпляры веб-службы, которые используют один и тот же порт и возвращают одни и те же веб-страницы. Когда такая служба имеет подключаемую архитектуру, разработчики службы определяют, какие подключаемые модули могут загружаться в их системах, а также создадут механизм поддержки выбранного режима. Можно иметь службу, расширяемую подключаемыми модулями, нацеленными и на 32-битную, и на 64-битную архитектуру, загружая их в хост-процессы, специфичные для архитектуры.

Хотя все это может звучать сложно, в действительности все не так плохо. Большинство проблем встречается тогда, когда обратная совместимость обязательна, а приложения имеют множество зависимых компонентов.

В следующем разделе мы кратко разберем, как работает система WOW64, чтобы понять последствия для отладки унаследованных приложений.

32-битное приложение, выполняющееся в WOW64

Каждое 32-битное приложение запускается в Windows x64 как обычный 64-битный процесс. Поскольку все указатели, используемые в приложении, а также структуры, объявленные операционной системой и используемые в приложении, ограничены 32 битами, приложение не может выполняться напрямую. Операционная система сначала загружает несколько вспомогательных динамически подключаемых библиотек, реализующих систему WOW64:

- `ntdll.dll`: 64-битная библиотека, которая связывает любой процесс пользовательского режима с функциями входа в ядро. Эта библиотека та же, что нативно используется любым приложением в Windows x64.
- `WOW64.dll`: 64-битная библиотека, выполняющая необходимое преобразование, или *thunking*, входных параметров и возвращаемых значений между 32-битными модулями и API ядра, предоставленными 64-битной версией `ntdll.dll`.
- `WOW64Win.dll`: 64-битная библиотека, выполняющая *thunking* для 32-битных модулей, вызывающих функции `win2k.sys` как часть API сообщений Windows.
- `WOW64Cpu.dll`: 64-битная библиотека, способная эмулировать набор инструкций x86. В Windows x64 эта библиотека ничего не делает, потому что процессор способен нативно выполнять поток инструкций x86.

В этот момент с точки зрения операционной системы процесс считается созданным; событие начальной точки останова процесса отправляется отладчику, даже если приложение еще не подготовлено к выполнению. Следующим шагом подсистема WOW64 загружает `ntdll32.dll`, а также остальные 32-битные динамически подключаемые библиотеки, от которых зависит приложение. `ntdll32.dll` - это 32-битная библиотека, предоставляющая функциональность, сходную с `ntdll.dll`. В отличие от `ntdll.dll`, которая переключает выполнение процессора из пользовательского режима в режим ядра, `ntdll32.dll` выполняет только переход между 32-битным кодом и системой WOW64, оставаясь в пользовательском режиме.

Внутри WOW64-процесса каждый поток может выполнять код в 64-битном режиме, называемом длинным режимом выполнения, или код в 32-битном режиме, называемом режимом совместимости. Режим выполнения определяется флагом `L (long)`, хранящимся в дескрипторе сегмента, связанном с текущим селектором кода, на который ссылается регистр `cs`. Чтобы переключить режим выполнения, поток должен выполнить дальний вызов через шлюз вызова, нацеленный на сегмент кода с противоположным режимом выполнения. Приложения, нацеленные на 64 бита, называются нативными приложениями, а приложения, нацеленные на 32 бита, называются WOW64-приложениями.

В листинге 12.1 показан стек потока, принадлежащего 32-битному процессу, который выполняет код в режиме совместимости внутри библиотеки эмулятора CPU `WOW64cpu.dll`. Стек взят из нативного отладчика, запускающего в режиме эмуляции WOW64 32-битную версию приложения `02sample.exe`, использованного в главе 2, "Introduction to the Debuggers".

Листинг 12.1. Нативный стек в WOW64-процессе

```
0:000> k8
Child-SP RetAddr Call Site
00000000`0006e508 00000000`78be6866 ntdll!NtTerminateProcess+0xa
```

```

00000000`0006e510 00000000`78b83c7d wow64!Wow64SystemServiceEx+0xd6
00000000`0006edd0 00000000`78be6a5a wow64cpu!ServiceNoTurbo+0x28
00000000`0006ee60 00000000`78be5e0d wow64!RunCpuSimulation+0xa
00000000`0006ee90 00000000`78ed8501 wow64!Wow64LdrpInitialize+0x2ed
00000000`0006f6c0 00000000`78ed6416 ntdll!LdrpInitializeProcess+0x17d9
00000000`0006f9d0 00000000`78ef3925 ntdll!LdrpInitialize+0x18f
00000000`0006fab0 00000000`77d59640 ntdll!KiUserApcDispatch+0x15

```

Но как WOW64-приложения могут работать бок о бок с нативными приложениями, когда и те и другие зависят от одних и тех же библиотек? Разделение достигается созданием виртуальной среды WOW64, похожей на среду, используемую нативными приложениями. Большинство системных ресурсов совместно используется нативной средой и средой WOW64, а API Windows возвращают одно и то же значение независимо от того, является вызывающее приложение нативным или WOW64-приложением. Когда WOW64-приложение обращается к ресурсам, не общим для обеих сред, оно перенаправляется в виртуальную среду WOW64 - этот механизм называется виртуализацией.

Один компонент этой виртуальной среды состоит из всех ключей реестра, относящихся к конфигурации программного обеспечения и обычно изменяемых при установке нового приложения в систему. Виртуализованные ключи реестра включают COM-регистрации, доступные как на уровне всей системы, так и для конкретных пользователей. На рисунке 12.1 используется regedit.exe, нативное приложение, чтобы показать несколько ключей реестра, затронутых перенаправлением реестра.

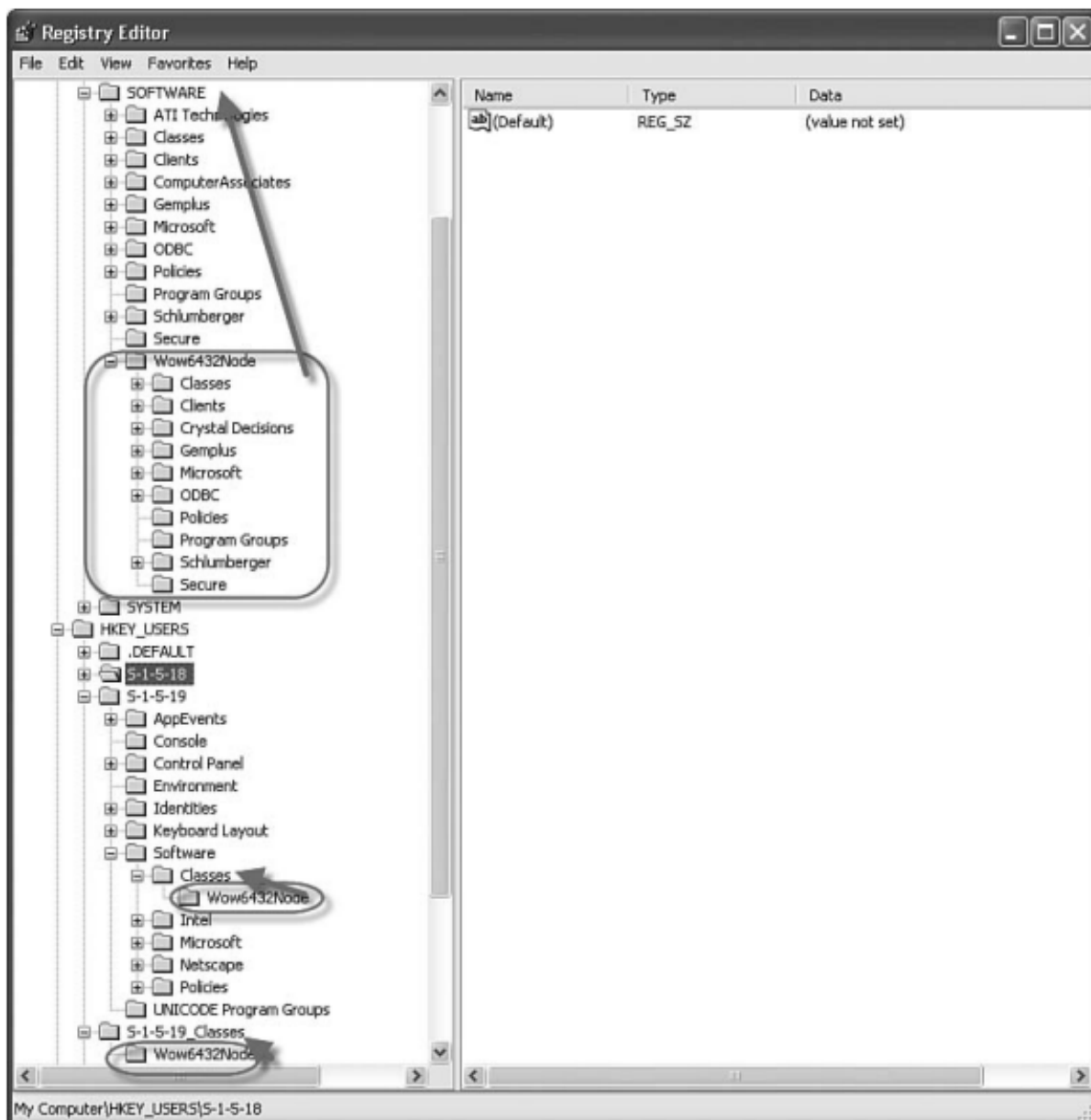


Рисунок 12.1. Реестры, затронутые перенаправлением реестра

WOW64-приложение, к которому применяется виртуализация, видит содержимое ключа HKLM\Software с рисунка 12.1 замененным ключом HKLM\Software\Wow6432Node, подобно всем другим ключам, подлежащим виртуализации. Каждая операция с реестром может управлять перенаправлением с помощью флагов KEY_WOW64_64KEY или KEY_WOW64_32KEY.

Аналогично большинство файлов в папке %SystemRoot%\system32 виртуализуется для WOW64-приложений. Когда приложение пытается прочитать файл из папки %SystemRoot%\system32, вместо этого оно получает файл из папки %SystemRoot%\sysWOW64. При необходимости приложение может отключить перенаправление файлов с помощью API Wow64DisableWow64FsRedirection.

Побочный эффект перенаправления файлов. Как побочный эффект перенаправления файлов, WOW64-приложения не могут получить доступ ни к одному файлу из реальной папки %SystemRoot%\system32 и ее подпапок. Если WOW64-приложение запускает исполняемый файл, предоставленный операционной системой, оно будет искать файл в %SystemRoot%\SysWOW64 и ее подпапках. Именно поэтому большинство .exe-файлов

дублируется в папке %SystemRoot%\SysWOW64, даже если это не имеет смысла. При необходимости 32-битный исполняемый файл является просто запускателем для 64-битного аналога.

Поскольку приложения также могут использовать несколько переменных среды, чтобы узнать о своем контексте выполнения, несколько переменных изменены так, чтобы отражать виртуальную среду. Некоторые переменные среды добавлены только в нативную среду, тогда как другие присутствуют только в среде WOW64. В таблице 12.1 перечислены все переменные, специфичные для Windows x64 или зависящие от режима выполнения.

Таблица 12.1. Переменные среды в Windows x64

Имя переменной	В WOW64-среде раскрывается в	В нативной среде раскрывается в
CommonProgramFile(x86)	Новая в Windows x64, указывает на %ProgramFiles(x86)%\Common Files	
CommonProgramFile	%CommonProgramFile(x86)%	%ProgramFiles%\Common Files
CommonProgramW6432	Нативная %CommonProgramFile%	N/A
PROCESSOR_ARCHITECTURE	X86	AMD64 или IA64
PROCESSOR_ARCHITECTUREW6432	Нативная %PROCESSOR_ARCHITECTURE%	N/A
ProgramFiles(x86)	Новая в Windows x64, указывает на %SystemDrive%\Program Files (x86)	
ProgramFiles	%ProgramFiles(x86)%	%SystemDrive%\Program Files
ProgramW6432	Нативная %ProgramFiles%	N/A

Один очень распространенный вопрос, на который приходится отвечать при исследовании проблем на платформе Windows x64, - является ли процесс нативным процессом или WOW64-процессом. Ответ определяет, как отлаживать процесс и как исследовать размещающую его среду. Когда есть возможность получить доступ к GUI-консоли, WOW64-приложения легко распознать в Task Manager по суффиксу *32 после имени процесса. Например, процесс cmd.exe, показанный на рисунке 12.2, выполняется как WOW64-процесс.



Рисунок 12.2. 32-битное приложение, выполняющееся в Windows x64

Альтернативный механизм обнаружения, более подходящий для пользователей отладчика, состоит в определении модулей, загруженных в процесс. WOW64-приложение загружает большинство модулей из папки %systemroot%\SysWOW64. Поскольку все такие модули нацелены на 32-битную архитектуру, приложения с любым из этих загруженных модулей должны быть WOW64-процессами. Кроме того, переменные среды, используемые соответствующим процессом, можно проверить на значения, специфичные для среды WOW64.

После этого краткого обзора нескольких новых механизмов, применяемых в Windows x64, нужно понять, какое влияние эти механизмы оказывают при отладке приложений. Остальная часть этой главы сосредоточена на изменениях, накладываемых архитектурой Windows x64.

Изменения Windows x64

В этом разделе мы пересматриваем все предыдущие главы, наблюдая за основными различиями при отладке соответствующих сценариев. Это не исчерпывающее освещение, а введение в наиболее важные изменения, за которыми следует следить при отладке приложений Windows x64. Мы рекомендуем обращаться к документации отладчика, когда конкретная отладочная ситуация требует дополнительной информации.

Каждый подраздел имеет то же название, что и соответствующая предыдущая глава, и пытается следовать той же организации, что и сама глава. Главы, охватывающие темы, не имеющие различий между Windows x86 и Windows x64, пропущены.

По материалам главы 1: Введение в инструменты

Не все инструменты из нашего набора пригодны в каждой ситуации, особенно в среде, которая не предполагалась при создании инструмента. Некоторые инструменты работают с ограничениями, тогда как другие не работают совсем. Этот раздел анализирует влияние Windows x64 на инструменты, представленные в главе 1, "Introduction to the Tools".

Leak Diagnostic Tool

Инструмент Leak Diagnostic можно использовать в Windows x64 с некоторыми ограничениями. Он устанавливается корректно, но не способен работать с процессами, выполняющимися в нативном режиме. Хорошая новость в том, что он прекрасно работает, когда нацелен на процессы, выполняющие приложения в режиме эмуляции WOW64. Мы надеемся, что это ограничение исчезнет в будущем, поскольку новые библиотеки Microsoft Detours, на которых основан инструмент, могут перехватывать API на 64-битных платформах.

Debugging Tools for Windows

Debugging Tools for Windows доступны как нативная версия, нацеленная на Windows x64. Хотя можно отлаживать нативные 64-битные процессы с помощью WOW64-инструмента отладки, для лучшего результата рекомендуется использовать 64-битный отладчик. Все инструменты, установленные вместе с Debugging Tools for Windows, работают в Windows x64.

Команда отладчика рекомендует использовать 32-битный отладчик, выполняющийся в режиме эмуляции WOW64, при live-отладке - как в пользовательском режиме, так и в режиме ядра, - если целью отладки является Windows NT или Windows 2000. Та же рекомендация действительна для отладки crash dump, созданных на этих платформах. 64-битный процесс пользовательского режима всегда должен отлаживаться с помощью нативных 64-битных отладчиков. WOW64-процессы можно отлаживать как нативными, так и WOW64-отладчиками, но последние обеспечивают более простой опыт для тех, кто привык к 32-битной отладке.

Все отладчики могут взаимозаменяемо использоваться в других ситуациях, не упомянутых выше. Полную рекомендацию см. в справке отладчика (раздел справки "Choosing a 32-bit or 64-bit Debugger Package").

В идеале в системе, используемой для отладки, должны быть установлены бок о бок оба отладчика - 64-битная версия и 32-битная версия. Затем правильная версия выбирается на основе архитектуры цели отладки. В этой главе предполагается, что 64-битная версия Debugging Tools for Windows установлена в папку `c:\debug.x64`, а 32-битная версия - в папку `c:\debug.x86`.

Microsoft Application Verifier

Microsoft Application Verifier доступен как нативное приложение, нацеленное на Windows x64. Поскольку в Windows x64 можно также установить 32-битную версию, иногда приходится выбирать между нативной версией инструмента и WOW64-версией. Чтобы понять, какую версию следует использовать, нужно сначала понять, как работает Microsoft Application Verifier.

У каждого приложения, наблюдаемого с помощью Microsoft Application Verifier, есть связанный ключ реестра, идентифицируемый именем исполняемого файла и содержащий параметры verifier для него. Этот ключ создается, когда приложение настраивается через инструмент Microsoft Application Verifier. Ключ, хранящий все параметры verifier, имеет следующий вид:

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\<Image File Name>

Но этот ключ реестра подлежит перенаправлению реестра, как описано в разделе "32-битное приложение, выполняющееся в WOW64". В результате 64-битная версия Microsoft Application Verifier изменяет нативный реестр, но ничего не делает с виртуализованным узлом реестра WOW64, даже когда настраиваемое приложение является 32-битным. Инфраструктура verifier загружает в проверяемый процесс две дополнительные библиотеки, архитектура которых должна совпадать с архитектурой целевого процесса.

На практике это означает, что для тестирования 64-битных и 32-битных приложений должны быть установлены обе версии verifier. Application Verifier, соответствующий архитектуре интересующего нас процесса, должен запускаться из командной строки одной из следующих команд:

- %systemroot%\system32\appverif.exe для 64-битных приложений
- %systemroot%\syswow64\appverif.exe для 32-битных приложений

Например, первая командная строка может использоваться для включения Application Verifier для 64-битной версии notepad.exe, тогда как вторая включит его для 32-битной версии notepad.exe.

Ethereal

Для захвата сетевого трафика Ethereal использует драйвер ядра, который в настоящее время доступен только для 32-битной архитектуры. Поскольку Windows x64 не может загружать 32-битные драйверы, инструмент не может захватывать сетевой трафик. Однако само ПО устанавливается в Windows x64.

Хотя Ethereal не может захватывать пакеты, проходящие через системные интерфейсы, его можно использовать для визуализации и интерпретации сохраненных файлов захвата. Поскольку Ethereal может захватывать сетевой трафик в promiscuous capture mode, легко использовать другую систему Windows x32 для выполнения захвата и переместить файл для дальнейшего исследования на систему Windows x64, для которой этот захват был выполнен.

Другие инструменты

Все остальные инструменты работают на платформе Windows x64 без проблем.

По материалам главы 2: Введение в отладчики

Как объяснялось в предыдущем разделе, и 32-битная, и 64-битная версии отладчиков могут использоваться для отладки файлов дампов или для выполнения отладки в режиме ядра. Решение использовать тот или другой отладчик в основном зависит от архитектуры пользовательских расширений, доступных для процессов, подлежащих отладке. Live-отладку следует выполнять с помощью отладчика, соответствующего архитектуре цели отладки: 64-битный отладчик для нативного процесса Windows x64 и 32-битный отладчик для WOW64-процессов. В настоящее время 32-битный отладчик лучше работает с WOW64-процессом, потому что он полностью скрывает 64-битный аспект WOW64-процесса. Поскольку опыт, полученный при отладке Windows x86, полностью применим при отладке WOW64-процесса, этот раздел сосредоточен на использовании нативного отладчика как для нативных, так и для WOW64-процессов. Такая конфигурация похожа на использование отладчика режима ядра для отладки процессов, выполняющихся в системе Windows x64.

Базовая задача в отладчиках

64-битные указатели и значения регистров, используемые архитектурой Windows x64, представляют собой крупное изменение режима взаимодействия с отладчиком. В отладчике 64-битные значения представлены как два 32-битных числа, иногда разделенные символом grave accent (` ). Например, `0x80000000`00000000` представляет то же число, что и `0x8000000000000000`, то есть наименьшее 64-битное знаковое целое. Это значение эквивалентно `0x80000000`` на процессорах x86.

Каковы текущие значения регистров?

Поскольку указатели были расширены до 64-битных значений, все остальные регистры также были расширены до 64 бит. Имена 64-битных регистров изменились и начинаются с буквы `r` вместо буквы `e`. Поэтому 32-битный процессорный регистр `eax` был заменен 64-битным регистром `rax`. Мнемоника `eax` все еще действительна и обозначает младшие 32 бита нативного регистра `rax`.

Число регистров общего назначения увеличилось еще на восемь регистров общего назначения, обозначаемых от `r8` до `r15`. В листинге 12.2 показан результат выполнения команды `r` с настройками по умолчанию.

Листинг 12.2. 64-битные регистры общего назначения x86

```
0:000> r
rax=0000000000000000 rbx=0000000000000000 rcx=0000734f67090000
rdx=0000000078ba0002 rsi=0000000000000000 rdi=0000000000000003
rip=0000000078ef3320 rsp=0000000000006f6b8 rbp=00000000000080000
r8=0000000000000002 r9=0000000000000000 r10=0000000000000018
r11=0000000078c108a0 r12=0000000000000000 r13=000000007efdf000
r14=0000000000200000 r15=0000000078ec0000
iopl=0 nv up ei pl nz na pe nc
cs=0033  ss=002b  ds=002b  es=002b  fs=0053  gs=002b  efl=00000202
```

Как вы видели в предыдущих главах, взаимодействие между модулями или между модулями и операционной системой возможно потому, что существует четко определенное соглашение о вызовах: как передавать аргументы вызываемой функции и как получать результаты операции. Соглашение о вызовах определяет, какие регистры сохраняются между вызовами функций, как регистры используются для передачи аргументов вызываемой функции и как возвращать результаты функции.

32-битные компиляторы используют несколько соглашений о вызовах, каждое со своими сильными и слабыми сторонами. В Windows x64 существует единое соглашение о вызовах, имеющее похожий шаблон с соглашением `__fastcall`, используемым 32-битными компиляторами. В соглашении о вызовах Windows x64 назначения регистров следующие:

- `rcx`: содержит первый параметр, переданный функции. Например, вызов метода объекта в C++ передает адрес объекта, или указатель `this`, в регистр `rcx`, подобно соглашению `__fastcall`.
- `rdx`: содержит второй параметр, переданный функции.
- `r8`: содержит третий параметр, переданный функции.
- `r9`: содержит четвертый параметр, переданный функции.
- `rax`: содержит результат вызова функции.

При использовании 64-битного соглашения о вызовах большинство функций получает свои аргументы через регистры, а поскольку на 64-битной архитектуре регистров больше, регистр указателя стека не так изменчив. В результате регистр `rsp` часто используется для адресации локальных переменных, уменьшая использование регистра адреса на основе стекового кадра, `rbp`,

который в большинстве функций становится регистром общего назначения. Функции с более чем четырьмя параметрами используют стек для всех параметров, начиная с пятого. Эти параметры сохраняются в стеке справа налево, причем самый правый параметр сохраняется по самому высокому адресу в стеке.

Соглашение о вызовах не требует сохранять регистры передачи аргументов rcx, rdx, r8 и r9, которые могут использоваться как scratch-регистры. Еще два регистра, r10 и r11, также могут использоваться вызываемой функцией как scratch-регистры. Все остальные регистры сохраняют значение между вызовами функций. Соглашение о вызовах подробнее рассматривается в следующем разделе "Каков текущий стек вызовов?"

Специфические команды WOW64

При отладке процесса, выполняющего приложение в системе WOW64, возникают несколько уникальных ситуаций, не встречающихся при отладке процессов, выполняющихся в длинном режиме. Например, как нужно интерпретировать стек, отображенный отладчиком в листинге 12.1?

Возьмем WOW64-приложение, которое вот-вот завершится, запущенное под нативным 64-битным отладчиком с помощью следующей командной строки:

```
c:\debug.x64\ntsd.exe -g c:\AWDBIN\WinXP.x86.chk\02sample.exe
```

Стек показан в листинге 12.1, но в нем нет следов отлаживаемого процесса, и он выглядит неправильно. Выполнение процесса скрыто под вызовом функции `WOW64cpu!ServiceNoTurbo`. Информацию стека за этой функцией можно получить после использования новой команды `.effmach`. Команда `.effmach <x86|AMD64|IA64|. >` изменяет режим, используемый отладчиком для интерпретации цели отладки. Команду можно использовать для переключения представления отладчика туда и обратно между 32-битной архитектурой и нативной архитектурой. Аргумент точка (.) изменяет текущую архитектуру на нативную.

В листинге 12.3 показан истинный стек того же процесса после изменения архитектурного представления на 32-битное. Приглашение отладчика изменяется на `0:000:x86>`, чтобы показать переход из нативного режима.

Листинг 12.3. Истинный 32-битный стек в WOW64-процессе, скрытый в листинге 12.1

```
0:000>.effmach x86
0:000:x86> k
ChildEBP RetAddr
0014fed0 7d4e80c7 ntdll32!NtTerminateProcess+0x12
0014ff0c 7d4e8072 kernel32!_ExitProcess+0x4b
0014ff20 77bcade4 kernel32!ExitProcess+0x14
0014ff2c 77bcaefb msvcrt!__crtExitProcess+0x32
0014ff5c 77bcaf52 msvcrt!_cinit+0xd2
0014ff70 00401a4f msvcrt!exit+0x11
0014ffc0 7d4e992a 02sample!mainCRTStartup+0x144
0014fff0 00000000 kernel32!BaseProcessStart+0x28
```

В режиме эмуляции WOW64 каждый поток имеет два разных контекста: нативный контекст, выполняющийся в длинном режиме (используется кодом эмулятора WOW64), и 32-битный контекст, видимый, когда контекст выполнения потока переключается в режим совместимости. Поскольку код, выполняющийся в этих контекстах, обращается к блоку окружения потока, содержащему указатели на другие структуры, WOW64-процесс содержит два ТЕВ для каждого потока, выполняющего 32-битный код. Такое дублирование происходит и с блоком окружения процесса: существует один экземпляр для нативного кода и другой экземпляр для кода, выполняющегося в режиме совместимости внутри WOW64-процесса. По той же причине стек, используемый каждым потоком, дублируется для длинного режима выполнения и режима совместимости.

Вся эта информация возвращается командой расширения `!straddr`, реализованной в `wow64exts.dll`, которая устанавливается с 64-битной версией Debugging Tools for Windows. Команда расширения `!straddr` возвращает адрес текущего блока окружения потока, адрес блока окружения процесса и стек потока, используемые, когда процессор работает в длинном режиме и режиме совместимости, как показано в листинге 12.4.

Листинг 12.4. Получение структур WOW64

```
0:000:x86> !wow64exts.straddr
Address of important WOW64 structures:
TEB64: 0x7efdb000
TEB32: 0x7efdd000
PEB64: 0x7efdf000
PEB32: 0x7efde000
STACK64: BASE: 0x70000 LIMIT: 0x6b000 DEALLOC: 0x30000
STACK32: BASE: 0x150000 LIMIT: 0x14e000 DEALLOC: 0x110000
```

Адреса этих структур можно использовать как входные параметры для других команд расширений, как вы увидите далее в этой главе. Большинство команд или команд расширений знают о текущем режиме выполнения и работают предсказуемо, когда тип входного параметра соответствует режиму выполнения.

Какой код процессор выполняет сейчас?

В Windows x86 ответ становится совершенно ясным после просмотра потока инструкций из участка памяти, адресуемого псевдорегистром указателя инструкций `$ip`. В Windows x64 перед ответом на этот вопрос нужно также учитывать режим выполнения процессора. В листинге 12.5 показан поток инструкций, интерпретированный как 64-битный код, а затем как 32-битный код. Хотя большинство инструкций использует одни и те же `opcodes` в 32-битном и 64-битном мире, интерпретация различается в зависимости от режима выполнения процессора.

Листинг 12.5. Дизассемблирование кода зависит от режима выполнения процессора

```
0:000> .effmach AMD64
Effective machine: x64 (AMD64)
0:000> u ntdll!NtTerminateProcess l4
ntdll!NtTerminateProcess:
00000000`78ef1520 4c8bd1 mov r10,rcx
00000000`78ef1523 b829000000 mov eax,29h
00000000`78ef1528 0f05 syscall
00000000`78ef152a c3 ret
0:000> .effmach x86
Effective machine: x86 compatible (x86)
0:000:x86> u ntdll!NtTerminateProcess l4
ntdll!NtTerminateProcess:
00000000`78ef1520 4c dec esp
00000000`78ef1521 8bd1 mov edx,ecx
00000000`78ef1523 b829000000 mov eax,29h
00000000`78ef1528 0f05 syscall
```

Каков текущий стек вызовов?

В общем случае текущий стек легко получить с помощью любой формы команды `k`. Поскольку бывают ситуации, когда указатель стека неверен или локальные переменные нельзя получить стандартными командами, знание того, как стек используется процессорами x86-64, является требованием для понимания реальной проблемы.

Стек растет в сторону меньших адресов, и перед вызовом другой функции он должен быть выровнен по 16 байтам. В разделе "Каковы текущие значения регистров?" вы узнали, что первые

четыре параметра всегда передаются через регистры (rcx, rdx, r8 и r9), тогда как остальные передаются в стеке. Чтобы понять управление стеком, мы используем функцию Function5, показанную в листинге 12.6, которая принимает пять 64-битных параметров и вызывает другую функцию с тем же числом параметров. Вызываемая функция объявлена как внешняя, чтобы компилятор не встроил ее inline.

Листинг 12.6. Функция с пятью параметрами, вызывающая другую функцию с пятью параметрами

```
extern int CalledFunction5(int a,int b,int c,int d,int e);
int Function5(int a, int b, int c, int d, int e)
{
    CalledFunction5(a,b,c,d,e);
    return 5;
}
```

В листинге 12.7 показан ассемблерный код, сгенерированный из исходного кода листинга 12.6 в среде Windows Vista Free x64 Build Environment, установленной WDK. Он сгенерирован с помощью команды `nmake param64.cod`.

Листинг 12.7. Ассемблерный код, представляющий функцию Function5

```
; COMDAT ?Function5@@YAHNNNNNN@Z
_TEXT SEGMENT
a$ = 64
b$ = 72
c$ = 80
d$ = 88
e$ = 96
?Function5@@YAHNNNNNN@Z PROC NEAR ; Function5, COMDAT
; 64 : {
$LN3:
00000 48 83 ec 38 sub rsp, 56 ; 00000038H
; 65 : CalledFunction5(a,b,c,d,e);
00004 8b 44 24 60 mov eax, DWORD PTR e$[rsp]
00008 89 44 24 20 mov DWORD PTR [rsp+32], eax
0000c e8 00 00 00 00 call ?CalledFunction5@@YAHNNNNNN@Z ; CalledFunction5
; 66 : return 5;
00011 b8 05 00 00 00 mov eax, 5
; 67 : }
00016 48 83 c4 38 add rsp, 56 ; 00000038H
0001a c3 ret 0
?Function5@@YAHNNNNNN@Z ENDP ; Function5
_TEXT ENDS
```

Этот сгенерированный ассемблерный код начинается со смещений, соответствующих всем входным параметрам, которые можно добавить к стековому регистру `rsp` внутри функции, чтобы найти адрес параметра в стеке. Эти смещения вычисляются в предположении, что все параметры передаются через стек, причем самый правый параметр расположен по самому высокому адресу.

Смещения уже учитывают первый оператор в функции, который уменьшает значение указателя стека: `sub rsp, 56`. Например, смещение, используемое для доступа к последнему параметру с именем `e`, равно `0n96`, то есть на `0n40` больше значения уменьшения стека, использованного в первой инструкции. В начале вызова функции смещение для параметра `e` было `0n40`, то есть тем смещением, которое использовалось бы, если бы все параметры передавались через стек.

Соглашение о вызовах требует, чтобы вызывающая сторона выделяла стек для всех параметров, переданных регистрами, так, как если бы они передавались через стек. Область стека, выделенная для параметров, переданных регистрами, вызывающей стороной не используется и не инициализируется, но может использоваться вызываемой функцией как временное хранилище. Эти места временного хранения обычно используются для сохранения входных параметров, если какие-либо регистры нужны для других целей, например для вызова другой функции.

В случае, когда функция использует временные переменные, стек корректируется, чтобы предоставить место таким переменным, которые будут расположены в верхней части области стека, используемой функцией. Хранилище, выделенное перед вызовом другой функции, похоже на пространство, используемое для временных переменных. В этом конкретном случае значение, используемое для корректировки стека на входе, больше пространства, требуемого для этих пяти параметров, чтобы сохранить выравнивание стека по 16 байтам. Этот зазор виден на рисунке 12.3.

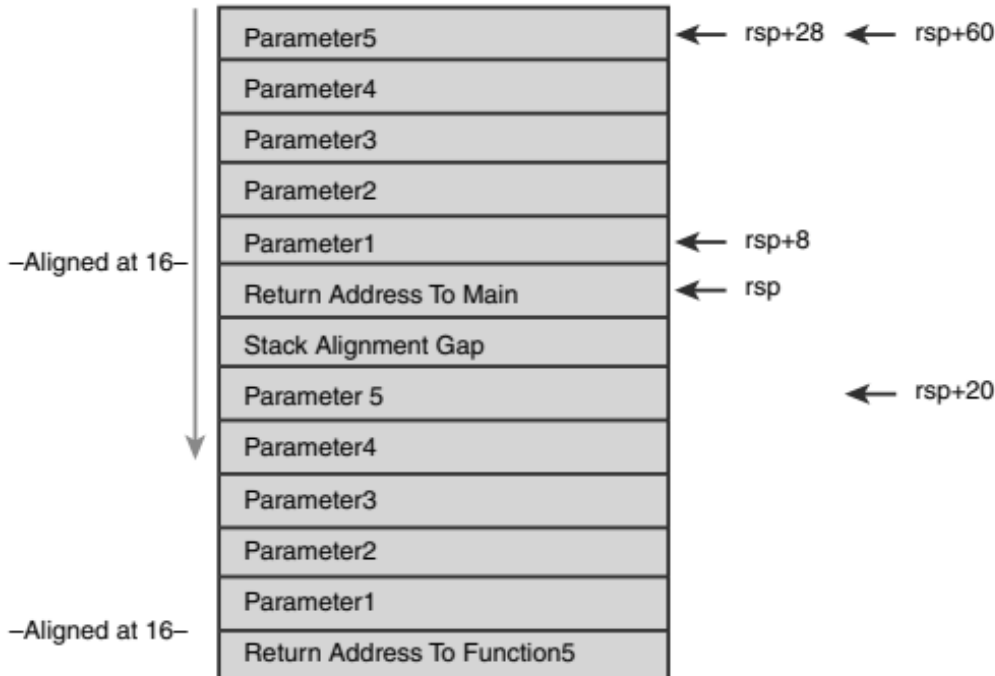


Рисунок 12.3. Использование стека вызова x86-64

Самый правый столбец содержит шестнадцатеричные адреса относительно значения `rsp`, используемого в `Function5`. Следующий столбец слева показывает шестнадцатеричный адрес относительно значения `rsp` в начале `Function5`, до корректировки.

До сих пор выводы основывались на анализе сгенерированного кода, который должен быть проверен в отладчике. Пример можно запустить под отладчиком с помощью следующей командной строки, запускающей 64-битную версию примера, представленного в главе 2, под 64-битным отладчиком:

```
C:\>C:\Debug.x64\windbg C:\awdbin\WinLH.AMD64.fre\02sample.exe
```

В этом процессе мы устанавливаем точку останова на функцию `02sample.exe!CalledFunction5`, которая срабатывает, когда мы выбираем пункт 5 из меню. В месте точки останова описанные выше предположения можно проверить, исследовав стек. В листинге 12.8 показаны аргументы функций для каждой функции из стека вызовов. Может оказаться неожиданным, что большинство параметров неверно. Первые четыре параметра из `Function5` или `CalledFunction5` являются случайными значениями, тогда как последний имеет правильное значение.

Листинг 12.8. Параметры функций в стеке вызовов

```
0:000> kP
Child-SP RetAddr Call Site
00000000`000afe98 00000000`00401973 02sample!CalledFunction5(
int64 a = 4202800,
int64 b = 8793941677150,
int64 c = 0,
int64 d = 4202020,
int64 e = 5)
00000000`000afea0 00000000`0040171a 02sample!Function5(
int64 a = 3684256,
```

```

int64 b = 0,
int64 c = 128094338278281250,
int64 d = 3684318,
int64 e = 5)+0x13
00000000`000afee0 00000000`00401cf1 02sample!main(
unsigned long argc = 0,
char ** argv = 0x00000000`00000000)+0x12a
00000000`000aff20 00000000`77d5966c 02sample!mainCRTStartup(void)+0x171
00000000`000aff80 00000000`00000000 kernel32!BaseProcessStart+0x2c

```

Почему значения неверны? Это неприятная обратная сторона данного соглашения о вызовах в сочетании с символьной информацией, доступной отладчику. Отладчик показывает значения, сохраненные в области стека, соответствующей аргументам, переданным через регистры, а эти значения случайны. Кроме того, если один входной параметр используется в функции для чего-то еще и больше не нужен в оставшейся части этой функции, значение параметра нигде не сохраняется. После этой точки отладчик не может восстановить этот параметр. Если он где-то был сохранен, нужно вручную найти это место и прочитать его значение.

Например, когда Function5 в листинге 12.8 начинает выполнение, первый параметр передается в регистр rcx. Когда эта функция вызывает CalledFunction5, тот же регистр rcx должен быть заполнен первым параметром, переданным этой функции. Если rcx не был сохранен раньше и не используется после возврата из CalledFunction5, компилятор не генерирует код для его сохранения. Его значение теряется прямо перед вызовом CalledFunction5.

Ситуация немного лучше для кода, скомпилированного без оптимизаций и запущенного с помощью ярлыка checked build WDK. В checked-сборках префикс каждой функции содержит специальный код для сохранения всех входных параметров, даже если ни один из них не используется.

Checked-версия CalledFunction5 выглядит намного, намного лучше с точки зрения отладки, как видно в листинге 12.9.

Листинг 12.9. Дизассемблированная неоптимизированная функция

```

0:000> uf .
02sample!CalledFunction5:
00000000`00401340 4c894c2420 mov qword ptr [rsp+20h],r9
00000000`00401345 4c89442418 mov qword ptr [rsp+18h],r8
00000000`0040134a 4889542410 mov qword ptr [rsp+10h],rdx
00000000`0040134f 48894c2408 mov qword ptr [rsp+8],rcx
00000000`00401354 b805000000 mov eax,5
00000000`00401359 c3 ret

```

Поддержка, полученная в сгенерированной checked-сборке от компилятора C/C++, удобна только в процессе разработки. Клиенты каждого приложения ожидают, что поставляемый им код будет сильно оптимизирован. Все дампы памяти, полученные через цикл обратной связи Windows Error Reporting, описанный в главе 13, "Postmortem Debugging", обычно будут оптимизированы, без такой поддержки компилятора.

Каковы значения локальных переменных?

Команда отображения переменных dv показывает хранилище, содержащее локальные переменные. Для входных параметров местом хранения является область передачи параметров в стеке, которая не заполняется во free- или оптимизированных сборках. Поскольку dv обычно показывает случайные значения для входных переменных, процесс ручного обнаружения использует наши знания о правилах генерации ассемблерного кода, сам ассемблерный код, а иногда и удачу, чтобы найти правильное значение. В листинге 12.10 показана локальная переменная, отображенная в использованной ранее остановке отладчика, на входе в CalledFunction5.

Листинг 12.10. Локальная переменная

```
0:000> dv /V
00000000`000afea0 @rsp+0x08 a = 4202800
00000000`000afea8 @rsp+0x10 b = 8793941677150
00000000`000afeb0 @rsp+0x18 c = 0
00000000`000afeb8 @rsp+0x20 d = 4202020
00000000`000afec0 @rsp+0x28 e = 5
```

Какую логику нужно использовать, чтобы найти правильные значения? Если отладчик не остановлен точно в начале функции и входные переменные не имеют правильных значений из регистра, использованного для передачи параметра, пользователь должен спросить себя, имеют ли значения, отображенные `dv`, смысл. Когда параметр выглядит неправильно, реальное значение нужно найти, просматривая ассемблерный код в поисках места, если оно есть, где значение ранее было сохранено. Затем это значение проверяется на осмысленность. Если нет, процесс повторяется, пока не будет найдено правильное значение.

При необходимости поиск исходного значения должен продолжаться в вызывающей функции текущих функций. В некоторых случаях исходное значение никогда не находится, но в большинстве случаев поиск успешен. Недопустимо останавливать расследование только потому, что команда `dv` не способна показать правильную информацию, и делать вывод, что сеанс отладчика поврежден.

Существует несколько способов искать вхождение имени регистра в одной из ассемблерных инструкций дизассемблированной функции. Отладчик поддерживает поиск по ассемблеру с помощью команды `round (#)`, которая принимает как параметры строковый шаблон для поиска, начальный адрес и длину блока, в котором нужно искать этот шаблон. Последовательную команду `round (#)` можно повторять без параметров, в результате чего поиск продолжается в блоке памяти, следующем за последним просмотренным блоком и смежном с ним. Следующий листинг показывает использование команды и ее вывод при поиске регистра `ebp`, начиная с начала функции `kernel32!CreateProcessW`, когда отладчик находится в режиме `x86`:

```
0:000> # ebp kernel32!CreateProcessW
kernel32!CreateProcessW+0x2:
771a1d29 55 push ebp
0:000>
```

Как просмотреть память процесса?

Знакомые команды, используемые для дампа содержимого памяти, работают без изменений, когда процессор выполняется в длинном режиме. Команды, зависящие от размера указателя в 32 бита, должны быть заменены 64-битным эквивалентом или командами, учитывающими размер указателя. Например, вместо команды `dds` нужно использовать более новую команду `dqs` или, что еще лучше, команду `dps`. Следующий листинг показывает, как вывести аргументы командной строки с помощью команды `dpa`, которая заменяет команду `dda`, использовавшуюся на 32-битной архитектуре.

```
0:000> dpa poi(argv) 12
00000000`003837a0 00000000`003837b8 "WinLH.AMD64.fre\02sample.exe"
```

Другие исследовательские команды

Блок окружения процесса является важной частью информации, описывающей текущий отлаживаемый процесс. Процессы, выполняющие приложения в режиме эмуляции WOW64, имеют два блока окружения процесса: один связан с нативным процессом, а другой предоставляется эмулятором коду, выполняющему 32-битный код. По умолчанию команда расширения !peb показывает нативный блок окружения процесса, сохраненный по виртуальному адресу, на который указывает псевдорегистр \$peb. Для процессов, выполняющихся в системе WOW64, команда расширения !peb пытается найти соответствующий 32-битный блок окружения процесса и требует символ структуры wow64!TEB32. В настоящее время этот символ недоступен в публичном файле символов WOW64.pdb, и команда останавливает обработку на нативном РЕВ процесса, как показано в листинге 12.11.

Листинг 12.11. РЕВ WOW64-приложения

```
0:000:x86> !peb
*****
***
***
*** Your debugger is not using the correct symbols
***
*** In order for this command to work properly, your symbol path
*** must point to .pdb files that have full type information.
***
*** Certain .pdb files (such as the public OS symbols) do not
*** contain the required information. Contact the group that
*** provided you with these symbols if you need this command to
*** work.
***
*** Type referenced: wow64!TEB32
***
*****
PEB at 000000007efdf000
InheritedAddressSpace: No
ReadImageFileExecOptions: No
BeingDebugged: Yes
ImageBaseAddress: 000000000400000
Ldr 0000000078fa7ea0
Ldr.Initialized: Yes
Ldr.InInitializationOrderModuleList: 000000000082e50 . 000000000082f90
Ldr.InLoadOrderModuleList: 000000000082d60 . 000000000083210
Ldr.InMemoryOrderModuleList: 000000000082d70 . 000000000083220
Base TimeStamp Module
400000 453bf190 Oct 22 15:32:48 2006 C:\awdbin\WinLH.AMD64.fre\02sample.exe
78ec0000 42438b79 Mar 24 19:54:33 2005 D:\WINDOWS\system32\ntdll.dll
78be0000 42438b79 Mar 24 19:54:33 2005 D:\WINDOWS\system32\wow64.dll
78b90000 42438b79 Mar 24 19:54:33 2005 D:\WINDOWS\system32\wow64win.dll
78b80000 42438b7a Mar 24 19:54:34 2005 D:\WINDOWS\system32\wow64cpu.dll
SubSystemData: 0000000000000000
ProcessHeap: 0000000000800000
ProcessParameters: 0000000000200000
WindowTitle: 'C:\awdbin\WinLH.AMD64.fre\02sample.exe'
ImageFile: 'C:\awdbin\WinLH.AMD64.fre\02sample.exe'
CommandLine: 'C:\awdbin\WinLH.AMD64.fre\02sample.exe'
DllPath: 'C:\awdbin\WinLH.AMD64.fre\02sample.exe;D:\WINDOWS\system32;D:\WIN-
DOWS\system;D:\WINDOWS;. ; D:\WINDOWS\system32;D:\WINDOWS;D:\WINDOWS\System32\Wbem'
Environment: 000000000010000
=::::\
=D:= C:\awdbin\WinLH.AMD64.fre
=ExitCode=00000000
ALLUSERSPROFILE=D:\Documents and Settings\All Users
APPDATA=D:\Documents and Settings\Administrator\Application Data
CLIENTNAME=Console
CommonProgramFiles=D:\Program Files (x86)\Common Files
CommonProgramFiles(x86)=D:\Program Files (x86)\Common Files
CommonProgramW6432=D:\Program Files\Common Files
...
```

```

USERNAME=Administrator
USERPROFILE=D:\Documents and Settings\Administrator
windir=D:\WINDOWS

```

Эту проблему можно обойти, используя команду расширения `!wow64exts.straddr`, чтобы получить адрес 32-битного PEВ и отобразить его с помощью команды `dt`. Адрес интерпретируется как 32-битный PEВ с использованием структуры, определенной в WOW64-версии `ntdll.dll`, называемой `ntdll132.dll`, как показано в листинге 12.12.

Листинг 12.12. PEВ WOW64-приложения с использованием команды `dt`

```

0:000:x86> !wow64exts.straddr
Address of important WOW64 structures:
TEB64: 0x7efdb000
TEB32: 0x7efdd000
PEB64: 0x7efdf000
PEB32: 0x7efde000
STACK64: BASE: 0x70000 LIMIT: 0x6b000 DEALLOC: 0x30000
STACK32: BASE: 0x150000 LIMIT: 0x14e000 DEALLOC: 0x110000
0:000:x86> dt ntdll132!_PEB 0x7efde000
+0x000 InheritedAddressSpace : 0 ''
+0x001 ReadImageFileExecOptions : 0 ''
+0x002 BeingDebugged : 0x1 ''
+0x003 BitField : 0 ''
+0x003 ImageUsesLargePages : 0y0
+0x003 SpareBits : 0y0000000 (0)
+0x004 Mutant : 0xffffffff`ffffffff
+0x008 ImageBaseAddress : 0x00000000`00400000
+0x00c Ldr : 0x00000000`7d6a01e0 _PEB_LDR_DATA
+0x010 ProcessParameters : 0x00000000`00100000 _RTL_USER_PROCESS_PARAMETERS
...
+0x210 FlsListHead : _LIST_ENTRY [ 0x152f28 - 0x152f28 ]
+0x218 FlsBitmap : 0x00000000`7d6a2048
+0x21c FlsBitmapBits : [4] 3
+0x22c FlsHighIndex : 1

```

Из сырой структуры `_PEB` можно извлечь ту же информацию, что и команда расширения `!peb`, проходя по структурам, на которые ссылается `_PEB`.

Блоки окружения потоков являются еще одними полезными структурами, которые можно получить с помощью команды расширения `!teb`. Команда расширения `!teb` страдает от тех же недостатков, что и команда расширения `!peb`, когда действует на поток, выполняющий код через эмулятор WOW64. В листинге 12.13 показан типичный результат, полученный от команды расширения `!teb` в этом случае, а также то, какую структуру нужно использовать как параметр команды `dt`, чтобы визуализировать структуру `_TEB`. Адрес структуры блока окружения потока был получен из команды расширения `!WOW64exts.straddr`, показанной в листинге 12.12.

Листинг 12.13. ТЕВ потоков WOW64 с использованием команды расширения `!teb` и команд `dt`

```

0:000:x86> !teb
Wow64 TEB32 at 000000007efdd000
*****
***
***
*** Your debugger is not using the correct symbols
***
*** In order for this command to work properly, your symbol path
*** must point to .pdb files that have full type information.
***
*** Certain .pdb files (such as the public OS symbols) do not
*** contain the required information. Contact the group that
*** provided you with these symbols if you need this command to
*** work.
***
***
*** Type referenced: wow64!_TEB32
***
*****
error InitTypeRead( wow64!_TEB32 )...

```



```

Wow64 TEB at 000000007efdb000
ExceptionList: 000000007efdd000
StackBase: 0000000000070000
StackLimit: 000000000006b000
SubSystemTib: 0000000000000000
FiberData: 00000000000001e00
ArbitraryUserPointer: 0000000000152c50
Self: 000000007efdb000
EnvironmentPointer: 0000000000000000
ClientId: 000000000000a4c . 0000000000000364
RpcHandle: 0000000000000000
Tls Storage: 0000000000000000
PEB Address: 000000007efdf000
LastErrorValue: 0
LastStatusValue: 0
Count Owned Locks: 0
HardErrorMode: 0
0:000:x86> dt ntdll32!_TEB 0x7efdd000
+0x000 NtTib : _NT_TIB
+0x01c EnvironmentPointer : (null)
+0x020 ClientId : _CLIENT_ID
+0x028 ActiveRpcHandle : (null)
+0x02c ThreadLocalStoragePointer : (null)
+0x030 ProcessEnvironmentBlock : 0x00000000`7efde000 _PEB
...
+0xf9c IsImpersonating : 0
+0xfa0 NlsCache : (null)
+0xfa4 pShimData : (null)
+0xfa8 HeapVirtualAffinity : 0
+0xfac CurrentTransactionHandle : (null)
+0xfb0 ActiveFrame : (null)
+0xfb4 FlsData : (null)
+0xfb8 SafeThunkCall : 0 ''
+0xfb9 BooleanSpare : [3] ""

```

Сценарии отладки

В главе 2 описан механизм, используемый для запуска приложений под отладчиком, с применением Image File Execution Option, известного как IFEO. Поскольку этот ключ находится в узле, подлежащем перенаправлению реестра, нужно внимательно изменять тот куст реестра, который используется архитектурой целевого процесса. Вместо прямого изменения реестра мы предлагаем использовать gflags, инструмент, устанавливаемый как часть Debugging Tools for Windows.

64-битная версия gflags.exe, установленная 64-битной версией Debugging Tools for Windows, включает IFEO для 64-битного процесса. Когда целевой процесс выполняется в режиме эмуляции WOW64, изменение должно выполняться с помощью 32-битной версии gflags.exe. Следующая командная строка включает IFEO-отладчик для 32-битной версии notepad.exe.

```
C:\>debug.x86\gflags /p /enable notepad.exe /debug "c:\debug.x86\ntsd.exe -g -G"
```

Следующая строка делает то же самое для 64-битной версии notepad.exe. Особое внимание нужно уделить отладчику, используемому как параметр ключа IFEO. Если отладчик является 32-битным, он не сможет отлаживать 64-битное приложение или, когда приложение находится в перенаправляемой папке вроде %SystemRoot%\system32, всегда будет загружать приложение из папки %SystemRoot%\syswow64.

```
C:\>debug.x64\gflags /p /enable notepad.exe /debug "c:\debug.x64\ntsd.exe -g -G"
```

По материалам главы 3: Отладчики без покровов

Глава 3, "Debuggers Uncovered", сосредоточена на объяснении того, как работают отладчики, исходя из убеждения, что лучшее знание инструментов повышает продуктивность и минимизирует риск быть удивленным их побочными эффектами.

Порядок событий отладчика

03sample.exe, использованный в главе 3, без проблем работал в Windows x64, когда параметр командной строки указывал на 64-битное приложение, но не мог запустить какие-либо 32-битные приложения в режиме эмуляции WOW64. Поскольку приложение выводит все в консоль, понять причины нетрудно. При запуске приложения операционная система сгенерировала исключение с новым кодом исключения 0x4000001C. Поиск в заголовках, установленных с WDK, показывает, что новый код исключения представляет startup-исключение для WOW64-приложения, сгенерированное после запуска процесса, который его размещает.

```
C:\WINDDK\6000\inc>findstr /is 4000001f *.h
api\ntstatus.h:#define STATUS_WX86_BREAKPOINT ((NTSTATUS)0x4000001FL)
```

Поскольку эта точка останова генерируется операционной системой каждый раз, когда приложение запускается под отладчиком, пример главы должен обрабатывать ее автоматически. После добавления кода обработки 03sample.exe может запускать любое приложение независимо от его архитектуры. Вывод, сгенерированный выполнением 32-битной версии хсору.exe, показан в листинге 12.14. Большинство неинтересных событий, генерируемых загрузкой 32-битных DLL, удалено, чтобы сделать листинг короче и понятнее.

Листинг 12.14. События отладчика, сгенерированные выполнением WOW64-процесса (хсору.exe)

```
D:\AWDBIN\WinLH.AMD64.fre>03sample.exe d:\WINDOWS\SysWOW64\хсору.exe
PID.TID=2264.3016
CreateProcess PID=2264
PID.TID=2264.3016
ImageName ntdll.dll
PID.TID=2264.3016
ImageName D:\WINDOWS\system32\wow64.dll
PID.TID=2264.3016
ImageName D:\WINDOWS\system32\wow64win.dll
PID.TID=2264.3016
ImageName D:\WINDOWS\system32\wow64cpu.dll
PID.TID=2264.3016
new ExceptionEvent:
Code = 80000003
FirstChance = 1
PID.TID=8d8.bc8
ImageName NOT_AN_IMAGE
PID.TID=8d8.bc8
UnloadDll @0000000077D40000
PID.TID=8d8.bc8
ImageName NOT_AN_IMAGE
PID.TID=8d8.bc8
UnloadDll @000000007D4C0000
PID.TID=8d8.bc8
ImageName D:\WINDOWS\SysWOW64\ntdll32.dll
PID.TID=8d8.bc8
ImageName NOT_AN_IMAGE
PID.TID=8d8.bc8
UnloadDll @0000000077D40000
PID.TID=8d8.bc8
ImageName NOT_AN_IMAGE
PID.TID=8d8.bc8
UnloadDll @0000000078C30000
```

```

PID.TID=8d8.bc8
ImageName D:\WINDOWS\syswow64\kernel32.dll
...
PID.TID=8d8.bc8
ImageName D:\WINDOWS\syswow64\MPR.dll
PID.TID=8d8.bc8
new ExceptionEvent:
Code = 4000001f
FirstChance = 1
PID.TID=8d8.bc8
ImageName d:\WINDOWS\SysWOW64\ShimEng.dll
PID.TID=8d8.bc8
ImageName d:\WINDOWS\SysWOW64\apphelp.dll
PID.TID=8d8.bc8
UnloadDll @0000000071AF0000
PID.TID=8d8.bc8
UnloadDll @0000000075E60000
Invalid number of parameters
0 File(s) copied
PID.TID=8d8.bc8
ExitProcess ExitCode=4

```

Порядок событий очень важен, но относительно предсказуем после понимания деталей реализации WOW64. Сразу после события создания процесса система загружает ntdll.dll и DLL, используемые для реализации слоя эмуляции WOW64. После загрузки всех DLL эмуляции процесс считается запущенным, и операционная система генерирует событие начальной точки останова.

До этого момента ни одна из DLL, требуемых приложением, не загружена в процесс. Следующий шаг загружает все библиотеки, от которых зависит приложение, начиная с ntdll32.dll. В конце этого процесса операционная система генерирует вторую начальную точку останова, означающую запуск 32-битного приложения, выполняющегося в режиме эмуляции WOW64. Эта начальная точка останова имеет специфический код исключения 0x4000001f.

После этого приложение выполняется нормально. Любая инструкция точки останова, встреченная в приложении, например связанные с assert, генерирует исключение с тем же кодом исключения 0x4000001f.

Механизм диспетчеризации исключений в 64-битных операционных системах

Windows x64 предлагает тот же мощный механизм диспетчеризации исключений, который используется во всех операционных системах Windows. Механизм был специально спроектирован для 64-битной архитектуры и опирается более чем на десять лет опыта реализации на процессорах x86. Структуры исключений скорректированы так, чтобы содержать информацию, специфичную для новых процессоров, и работать в новой модели.

Модель исключений в Windows x64

Один способ понять новую модель исключений - увидеть ее в действии и сравнить с моделью, поддерживаемой Windows x86. Функция из следующего листинга использовалась в главе 3 для объяснения модели исключений в Windows x86.

```

void try_except()
{
    __try
    {
        *((int *) 0) = 0;
    }
    __except(ex_filter())
    {
        global = 1;
    }
}

```

```
}
}
```

В 32-битной архитектуре механизм обработки исключений предполагает, что с каждой функцией связан счетчик блоков, который обновляется каждый раз, когда выполнение кода переходит в этот блок. В листинге 12.15 показан ассемблерный листинг, сгенерированный для функции `try_except`, показанной выше.

Листинг 12.15. Код обработки исключений для очень простой функции (`tryexcept` в `02sample.exe`)

```
PUBLIC ?try_except@@YAXXZ ; try_except
EXTRN __C_specific_handler:PROC
; COMDAT pdata
pdata SEGMENT
$pdata$?try_except@@YAXXZ DD imagere1 $LN9
DD imagere1 $LN9+32
DD imagere1 $unwind$?try_except@@YAXXZ
pdata ENDS
; COMDAT pdata
pdata SEGMENT
$pdata$?filt$0@?0??try_except@@YAXXZ@4HA DD imagere1 ?filt$0@?0??try_except@@YAXXZ@4HA
DD imagere1 ?filt$0@?0??try_except@@YAXXZ@4HA+29
DD imagere1 $unwind$?filt$0@?0??try_except@@YAXXZ@4HA
pdata ENDS
; COMDAT xdata
xdata SEGMENT
$unwind$?filt$0@?0??try_except@@YAXXZ@4HA DD 020f01H
DD 0500b320fH
xdata ENDS
; COMDAT xdata
xdata SEGMENT
$unwind$?try_except@@YAXXZ DD 010409H
DD 04204H
DD imagere1 __C_specific_handler
DD 01H
DD imagere1 $LN9+4
DD imagere1 $LN9+17
DD imagere1 ?filt$0@?0??try_except@@YAXXZ@4HA
DD imagere1 $LN9+17
; Function compile flags: /OdtP
xdata ENDS
; COMDAT ?try_except@@YAXXZ
_TEXT SEGMENT
?try_except@@YAXXZ PROC ; try_except, COMDAT
; 28 : {
$LN9:
00000 48 83 ec 28 sub rsp, 40 ; 00000028H
; 29 : __try
; 30 : {
; 31 : *((int *) 0) = 0;
00004 c7 04 25 00 00 00 00 00 00 00 mov DWORD PTR ds:0, 0
; 32 : }
0000f eb 0a jmp SHORT $LN4@try_except
$LN6@try_except:
; 33 : __except(ex_filter())
; 34 : {
; 35 : global = 1;
00011 c7 05 00 00 00 00 01 00 00 00 mov DWORD PTR ?global@@3HA, 1
$LN4@try_except:
; 36 : }
; 37 : }
0001b 48 83 c4 28 add rsp, 40 ; 00000028H
0001f c3 ret 0
?try_except@@YAXXZ ENDP ; try_except
; Function compile flags: /OdtP
?filt$0@?0??try_except@@YAXXZ@4HA PROC ; `try_except':`1':`filt$0
00020 48 89 4c 24 08 mov QWORD PTR [rsp+8], rcx
00025 48 89 54 24 10 mov QWORD PTR [rsp+16], rdx
0002a 55 push rbp
```

```

0002b 48 83 ec 20 sub rsp, 32 ; 00000020H
0002f 48 8b ea mov rbp, rdx
$LN5@filt$0:
; 33 : __except(ex_filter())
00032 e8 00 00 00 00 call ?ex_filter@@YAKXZ ; rrex_filter
$LN7@filt$0:
00037 48 83 c4 20 add rsp, 32 ; 00000020H
0003b 5d pop rbp
0003c c3 ret 0
?filt$0@?0??try_except@@YAXXZ@4HA ENDP ; `try_except'::`1'::filt$0
_TEXT ENDS

```

Код функции, показанный курсивом в листинге 12.15, максимально чист. В нем нет учетного кода исключений, и только один безусловный переход в строке 32 указывает, что код использует исключения. В остальном в функции нет условного оператора.

Как операционная система узнает, что выполнять, когда исключение генерируется в защищенном блоке? Первая часть листинга, выделенная жирным курсивом, отвечает на этот вопрос. Компилятор генерирует блок информации о функции, содержащий общую информацию о каждой функции, а также о каждом блоке, защищенном обработчиком исключений. Каждый блок идентифицируется диапазоном адресов, покрывающим возможные значения указателя инструкций во время выполнения этого блока. Текущее значение указателя инструкций используется механизмом раскрутки исключений, реализованным в Windows x64, чтобы найти защищенный блок, содержащий фильтр исключений и соответствующий обработчик исключений.

В листинге 12.15 метка \$pdata\$?try_except@@YAXXZ содержит информацию для диспетчера исключений, которая должна использоваться, когда функция выполняет код между адресами LN9# и LN9#+32 относительно начала модуля. В этом случае единственный защищенный блок сохранен под меткой \$unwind\$?try_except@@YAXXZ#.

Блок идентифицируется диапазоном адресов и содержит адрес фильтра исключений и расположение продолжения кода после обработки исключения, как показано в следующем листинге:

```

DD @imagerel($LN9#+4)
DD @imagerel($LN9#+17)
DD @imagerel(?filt$0@?0??try_except@@YAXXZ@4HA#)
DD @imagerel($LN9#+17)

```

В конце листинга несколько сгенерированных компилятором функций используются для вызова фильтров блоков. Фильтры идентифицируются автоматически сгенерированными символами, имеющими следующую форму:

```
`Function Name'::`Function Number'::filt$FilterNumber
```

Сгенерированный код чрезвычайно эффективен, поскольку содержит только бизнес-логику, полностью игнорируя исключительные пути кода. Компоновщик может переместить весь код и всю информацию, связанную с исключениями, в отдельную секцию модуля и уплотнить весь код функций в другой секции. Итоговый модуль чрезвычайно эффективен с точки зрения памяти, поскольку максимально использует процессорный кэш. Должно выполняться только одно условие: код не должен генерировать исключения. Иными словами, исключение действительно должно быть исключением, а не ожидаемой ошибкой, которую можно возвращать снова и снова.

Когда код генерирует другое исключение изнутри обработчика, Windows x64 начинает новую диспетчеризацию исключения, требующую выделения нового контекста из стека. Это может довольно легко израсходовать доступный стек и представляет проблему, которую нужно явно учитывать в высоконадежных приложениях.

Типичный код исключений C++, демонстрирующий эту проблему, показан в следующем фрагменте. Мы рекомендуем удалять любые такие конструкции из коммерческого кода и заменять их другими шаблонами очистки, например шаблоном Resource Acquisition Is Initialization.

```

catch(...)
{
//Cleanup
throw();
}

```

По материалам главы 5: Повреждения памяти, часть I - стеки

Глава 5, "Memory Corruptions Part I - Stacks", полностью зависит от архитектуры процессора. Текущая глава как раз посвящена новой процессорной архитектуре, и большую часть времени она рассматривает различия между архитектурами x86 и x64. Большинство ранее представленных сценариев отладки действительно после учета соглашения о вызовах, используемого Windows x64 и представленного в разделе "Каков текущий стек вызовов?"

По материалам главы 6: Повреждения памяти, часть II - кучи

Глава 6, "Memory Corruptions Part II - Heaps", использует различные команды и приемы, зависящие от архитектуры процессора x86. Как и в предыдущем разделе, информации, представленной до сих пор, должно быть достаточно для исследования большинства сценариев, описанных в этой главе.

Сценарии в главе 6 сильно опираются на структуры кучи, которые в Windows x64 немного отличаются. Все структуры, содержащие указатели и размеры, скорректированы так, чтобы хранить 64-битные значения. В качестве примера следующий листинг показывает структуру `_HEAP_ENTRY` для 64-битных куч.

```

0:000> dt ntdll!_HEAP_ENTRY 000000000080000
+0x000 PreviousBlockPrivateData : (null)
+0x008 Size : 0xc5
+0x00a PreviousSize : 0
+0x00c SmallTagIndex : 0xb9 ''
+0x00d Flags : 0x1 ''
+0x00e UnusedBytes : 0 ''
+0x00f SegmentIndex : 0 ''
+0x008 CompactHeader : 0x1b9`000000c5

```

Обратите внимание, что процессы, выполняющие приложения в WOW64, имеют два типа куч. Нативный процесс содержит по меньшей мере одну кучу, кучу процесса, организованную как 64-битная куча, тогда как само приложение может иметь несколько куч, организованных как 32-битные кучи.

Команда расширения `!heap`, использованная без параметров, показывает список куч, созданных в процессе, исследуя блоки окружения процесса. Как и команда расширения `!peb`, команда расширения `!heap` не способна найти 32-битный РЕВ и не может показать кучи в WOW64-приложениях при использовании нативного отладчика.

Особое внимание нужно уделить включению инструмента Application Verifier, как объяснено в разделе "Глава 1 - Введение в инструменты", чтобы получить желаемые результаты.

По материалам главы 7: Безопасность

Все концепции безопасности, используемые в Windows x86, применимы в Windows x64 без изменений. Ничего не изменилось ни в способе идентификации субъектов безопасности операционной системой, ни в способе защиты каждого объекта. Управление доступом выполняется с использованием тех же механизмов, что и в Windows x86, но принимает аргументы, соответствующие размеру указателя архитектуры.

В листинге 12.16 показано типичное использование наиболее часто применяемой команды в области безопасности, которая отличается в Windows x64. В этом листинге, снятом из отладчика режима ядра, подключенного к системе Windows x64, мы исследуем дескриптор безопасности, защищающий процесс winlogon.exe, а также токен, под которым выполняется процесс.

Листинг 12.16. Информация безопасности в Windows x64

```
1: kd> !process 0 1 winlogon.exe
PROCESS fffffadfe74010f0
SessionId: 0 Cid: 02dc Peb: 7fffffd4000 ParentCid: 0268
DirBase: 351de000 ObjectTable: fffffa8000b74f90 HandleCount: 691.
Image: winlogon.exe
VadRoot fffffadfe73780f0 Vads 179 Clone 0 Private 3088. Modified 2148. Locked 0.
DeviceMap fffffa80000027d0
Token fffffa8000742060
ElapsedTime 2 Days 12:02:23.296
UserTime 00:00:00.546
KernelTime 00:00:00.906
...
CommitCharge 3666
1: kd> !object fffffadfe74010f0
Object: fffffadfe74010f0 Type: (fffffadfe7afcd40) Process
ObjectHeader: fffffadfe74010c0
HandleCount: 13 PointerCount: 363
1: kd> dt nt!_OBJECT_HEADER fffffadfe74010c0
+0x000 PointerCount : 363
+0x008 HandleCount : 13
+0x008 NextToFree : 0x00000000`0000000d
+0x010 Type : 0xfffffadf`e7afcd40 _OBJECT_TYPE
+0x018 NameInfoOffset : 0 ''
+0x019 HandleInfoOffset : 0 ''
+0x01a QuotaInfoOffset : 0 ''
+0x01b Flags : 0x20 ''
+0x020 ObjectCreateInfo : 0xfffff800`011d2f00 _OBJECT_CREATE_INFORMATION
+0x020 QuotaBlockCharged : 0xfffff800`011d2f00
+0x028 SecurityDescriptor : 0xfffffa80`000037d6
+0x030 Body : _QUAD
1: kd> * The address of the security descriptor
1: kd> !sd (poi(fffffadfe74010f0-8)&0xffffffff`ffffffff0)
1: kd> !sd (0xfffffa80`000037d6&0xffffffff`ffffffff0)
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8004
SE_DACL_PRESENT
SE_SELF_RELATIVE
->Owner : S-1-5-32-544
->Group : S-1-5-18
->Dacl :
...
->Dacl : ->Ace[1]: ->SID: S-1-5-32-544
->Sacl : is NULL
1: kd> !token fffffa8000742060
_TOKEN fffffa8000742060
TS Session ID: 0
User: S-1-5-18
Groups:
00 S-1-5-32-544
Attributes - Default Enabled Owner
...
```

RestrictedSidCount: 0 RestrictedSids: 0000000000000000
OriginatingLogonSession: 0

Помимо размера указателя, интересно также искать расположение дескриптора безопасности, управляющего доступом к объектам ядра. Как и в Windows x86, у указателя есть младшие значащие биты - в этом случае четыре, поскольку указатели выровнены по шестнадцати байтам, - используемые для чего-то еще, и эти биты нужно замаскировать перед использованием указателя.

В некоторых ситуациях дескриптор безопасности, связанный с ключом реестра или файлом, подлежащим перенаправлению WOW64, отличается между нативной частью и частью WOW64. В таких случаях пользователь наблюдает асимметричное поведение одного и того же приложения в зависимости от используемой версии. После идентификации компонента и среды, проявляющей проблему, расследование безопасности выполняется с использованием приемов, похожих на описанные в главе 7, "Security".

По материалам главы 8: Межпроцессное взаимодействие

LPC-взаимодействие между 64-битными процессами, выполняющими 64-битные приложения, и процессами, выполняющими приложения в режиме эмуляции WOW64, выполняется ядром Windows и не зависит от архитектуры процесса. Однако когда данные передаются между процессами без преобразования, приложение должно учитывать проблемы маршалинга. Смещения членов структуры, размеры членов и выравнивание членов различаются между этими архитектурами, если только структуры не были тщательно спроектированы как независимые от архитектуры.

Какие есть варианты исправления проблем маршалинга? Безусловно, лучший вариант - использовать существующие компоненты, способные без проблем маршализовать данные между архитектурами. Например, Remote Procedure Call (RPC), а также более новые коммуникационные стеки, такие как SOAP, были изобретены именно для преодоления этой проблемы. В эти протоколы вложены годы работы и опыта, которые нужно использовать, если это возможно. Если переход к 64 битам страдает от проблем в этой области, это хорошая возможность перейти от самодельного механизма коммуникации к протоколам, используемым во всей индустрии.

DCOM-взаимодействие также может показывать различие между нативными процессами и процессами, эмулируемыми WOW64. DCOM-клиент вызывает API `ole32!CoCreateInstance`, чтобы получить указатель на серверный объект, реализующий запрошенный интерфейс. DCOM использует реестр как основное хранилище регистрационной информации и загружает соответствующие модули из файловой системы.

Поскольку DCOM-регистрация подлежит перенаправлению реестра, серверный компонент должен быть правильно зарегистрирован для архитектуры, выполняющей вызов. Иными словами, установка компонента должна убедиться, что записи реестра и файлы видимы их клиенту. Внепроцессные DCOM-серверы подчиняются другому механизму, называемому registry reflector.

Registry reflector поддерживает синхронизацию 64-битного представления и 32-битного представления нескольких ключей реестра, включая регистрацию внепроцессных DCOM-серверов. Registry reflector использует политику "побеждает последний записавший", когда синхронизирует представление реестра. В результате после регистрации 32-битного DCOM-серверного приложения его регистрация автоматически появляется в 64-битном представлении реестра. 64-битный клиент создаст экземпляр сервера и будет использовать его, то есть 32-битное приложение, выполняющееся в среде эмуляции WOW64.

Коммуникация возможна, поскольку внепроцессные серверы изолированы от своих клиентов и могут взаимодействовать в любой комбинации архитектур: 32-битный клиент вызывает 64-битный

или 32-битный сервер, а 64-битный клиент вызывает 32-битный или 64-битный сервер. Несмотря на изоляцию процессов, клиент-серверные архитектуры могут требовать загрузки proxy-stub-модулей в свои процессы. Поскольку proxy-stub-модули должны соответствовать архитектуре размещающего процесса, система должна содержать как 64-битную версию, так и 32-битную версию этих модулей, чтобы обеспечить коммуникацию между архитектурами. Когда зарегистрированный proxy-stub правильной архитектуры не найден, коммуникация завершается неудачей.

По материалам главы 11: Написание собственных расширений отладчика

Есть ли причина перекомпилировать расширение, чтобы оно работало в 64-битных отладчиках? В большинстве случаев, например при отладке live-целей ядра или отладке дампов памяти, существующее расширение прекрасно работает внутри 32-битного отладчика. Для live-отладки нативных приложений пользовательского режима рекомендуется использовать только 64-битную версию отладчиков, которые не способны загружать 32-битные расширения. Решение вложить некоторое время в создание 64-битной версии того же расширения зависит от потенциального использования расширения.

Резюме

После прочтения этой главы, описывающей, чего ожидать при отладке 64-битных приложений, любому инженеру должно быть относительно безопасно перейти из зоны комфорта, предоставляемой Windows x86, в мир Windows x64. 64-битный код оптимизирован для производительности, из-за чего инструментам бывает трудно предоставлять пользователю надежную информацию. Поэтому инженеры должны быть готовы вручную интерпретировать стек приложения, использовать информацию из этой главы и решать стоящую перед ними проблему. Поскольку все инструменты используют одни и те же файлы символов, одни и те же исходные файлы и в основном одни и те же концепции, для получения максимальной продуктивности нужно выбрать правильный инструмент.

Глава 13. Postmortem-отладка

На протяжении книги мы рассмотрели довольно много мощных инструментов, доступных разработчикам при устранении проблемного кода. Конечная цель - убедиться, что эти инструменты встроены в процесс разработки для обеспечения высокого качества. Эти инструменты являются прекрасными автоматизированными способами поиска ошибок, но они не дают абсолютных гарантий, что приложение будет свободно от ошибок на момент поставки.

Неизбежно проблемы проявляются в приложении уже после его поставки. Эти проблемы возникают в самые неподходящие моменты - главным образом, когда приложение использует заказчик. В зависимости от серьезности ошибка может либо иметь разрушительные последствия для заказчика, либо быть просто раздражающей помехой. В любом случае можно ожидать звонка от расстроенного заказчика с вопросом, почему приложение работает неправильно. Чтобы исправить ситуацию и разобраться с проблемой, один вариант - попросить у заказчика удаленный доступ к нужному компьютеру. Хотя иногда это может быть осуществимо, заказчики обычно относятся к этому неодобрительно, и во многих случаях ответ будет отрицательным. Причины отказа в удаленном доступе к машине различаются, но обычно они могут быть следующими:

- Среда или политика заказчика не допускает входящих соединений.
- Удаленная отладка требует присоединения отладчика к одному или нескольким процессам и подразумевает простой. Если процесс работает на критически важном сервере, заказчики неохотно согласятся на downtime.
- Отладка процесса в пользовательском или kernel mode означает, что разработчики получают полный доступ к состоянию машины, включая содержимое памяти. Для некоторых заказчиков это может быть проблемой приватности.

Если заказчик отказывается предоставить live-доступ к машине, демонстрирующей проблему, а локально воспроизвести проблему невозможно, можно ли вообще отладить проблему? Ответ - да, и этот процесс называется postmortem-отладкой. На высоком уровне postmortem-отладка включает следующие шаги:

1. Спровоцировать возникновение сбоя.
2. Снять снимок состояния системы в точке сбоя (или даже до и после, в зависимости от типа сбоя).
3. Отправить снимок инженерам для дальнейшего анализа.

В этой главе мы рассмотрим разные способы генерации снимков (также известных как dump files), разные доступные типы дампов и способы их анализа. Мы также рассмотрим два очень мощных сервиса агрегации dump-файлов, известные как Windows Error Reporting и Corporate Error Reporting.

Начнем с некоторых фундаментальных тем, связанных с dump-файлами.

Основы dump-файлов

Как мы уже упоминали, dump-файл - это out-of-band-представление состояния заданного процесса. Основная цель генерации dump-файла - анализировать сбой приложения без необходимости live-отладочного доступа к компьютеру, где проявляются сбои. После генерации dump-файла его можно отправить соответствующему инженеру, который затем может анализировать сбой без доступа к неисправной машине. Вместо этого он просто загружает dump-файл на своем компьютере и анализирует сбой с помощью postmortem-возможностей отладчиков. Какую информацию содержит dump-файл? Это полностью зависит от того, как был сгенерирован dump-файл. Есть две категории dump-файлов:

- full dumps;
- mini dumps.

Full dump file содержит все пространство памяти процесса: исполняемый образ, таблицу дескрипторов и другую информацию, используемую отладчиком. При использовании full dump file нет способа настроить объем собираемых данных. Однако full dump file можно преобразовать в mini dump file с помощью отладчиков.

Содержимое mini dump files переменное и может настраиваться генератором dump-файла, в зависимости от используемого генератора. Информация, содержащаяся внутри mini dump file, варьируется от информации о конкретном потоке до исчерпывающего описания процесса, для которого создается дамп. Как бы странно это ни звучало, самый большой mini dump file фактически будет содержать больше отладочной информации, чем full dump file. Поэтому эта глава сосредоточена на конструкции mini dump file.

Существует ряд инструментов, которые генерируют dump-файлы, как показано в таблице 13.1.

Таблица 13.1. Инструменты генерации dump-файлов

Name	Description
Windows Debuggers	Отладчики Windows могут генерировать дампы разных размеров и дают полный контроль над процессом генерации dump-файла.
ADPlus	ADPlus - инструмент, входящий в Debugging Tools for Windows. Он действует как монитор процесса, способный генерировать dump-файлы при возникновении crash или hang. Кроме того, у него есть механизм уведомлений, который может уведомлять пользователя о crash.
Windows Error Reporting	Windows Error Reporting - сервис Microsoft, позволяющий заказчикам регистрироваться на live-сайте отчетов об ошибках. Каждый раз, когда crash происходит в одном из приложений, принадлежащих конкретному заказчику, отчет об ошибке отправляется с падающей машины на сайт Windows Error Reporting. Информация о crash (включая dump file) может быть получена из WER-сервиса и проанализирована заказчиком postmortem.
Corporate Error Reporting	Corporate Error Reporting работает на той же основе, что и Windows Error Reporting, но на уровне корпоративного сервера, вместо отправки информации о crash через Интернет. Заказчики могут настроить Corporate Error Server и отправлять crash-информацию на этот сервер. Впоследствии crash-информация может быть отправлена на сайт Windows Error Reporting.

В этом разделе мы рассмотрим, как генерировать dump-файлы с помощью Windows Debuggers и ADPlus. Механизмы Windows Error Reporting и Corporate Error Reporting обсуждаются позже в этой главе.

Чтобы лучше проиллюстрировать процесс генерации dump-файла, мы используем простое приложение, которое выделяет память в куче, пишет в эту память, а затем падает. Листинг 13.1 показывает код примерного приложения.

Листинг 13.1

```
void __cdecl wmain ( )
{
    WCHAR* pszTitle=L"Advanced Windows Debugging";
    wprintf(L"Press any key to start\n");
    WCHAR* pBuffer=(WCHAR*) new WCHAR[wcslen(pszTitle)+1];
    if(pBuffer)
    {
        StringCchCopy(pBuffer, wcslen(pszTitle)+1, pszTitle);
        wprintf(L"Title: %s\n", pBuffer);
        pBuffer=NULL;
        *pBuffer='\0';
    }
    else
    {
        wprintf(L"Failed to allocate memory\n");
    }
    wprintf(L"Press any key to end\n");
    __getch();
}
```

Источник crash должен быть довольно очевиден. После копирования строки в недавно выделенный буфер кучи указатель строки сбрасывается в null и затем используется при null-терминации буфера, что приводит к crash. Начнем с иллюстрации того, как использовать отладчики для генерации dump-файла.

Генерация dump-файлов с помощью отладчиков

Как отмечалось ранее, приложение, которое мы будем использовать, показано в листинге 13.1. Скомпилированную версию можно найти здесь:

```
Source code: C:\AWD\Chapter13
Binary: C:\AWDBIN\WinXP.x86.chk\awdscenario1.exe
```

Запустите приложение под отладчиком и продолжайте выполнение, пока не произойдет crash.

```
....
ModLoad: 5cb70000 5cb96000 C:\WINDOWS\system32\ShimEng.dll
Press any key to start
Title: Advanced Windows Debugging
(9d8.f0c): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=00000000 ebx=7ffd5000 ecx=77c418bf edx=77c61b78 esi=7c9118f1 edi=00011970
eip=0100127e esp=0007ff1c ebp=0007ff44 iopl=0         nv up ei pl nz na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010202
awdscenario1!wmain+0xbe:
0100127e c60000 mov byte ptr [eax],0 ds:0023:00000000=??
0:000> kb
ChildEBP RetAddr  Args to Child
0007ff44 01001495 00000001 00032bf0 00036890 awdscenario1!wmain+0xbe
0007ffc0 7c816fd7 00011970 7c9118f1 7ffd5000 awdscenario1!wmainCRTStartup+0x12f
0007fff0 00000000 01001366 00000000 78746341 kernel32!BaseProcessStart+0x23
```

Неудивительно, что мы падаем из-за second-chance access violation. В этот момент мы хотим сгенерировать dump-файл для дальнейшего postmortem-анализа. Самый большой вопрос при генерации dump-файлов - сколько информации включать. Как общее практическое правило, чем больше состояния сохранено в dump-файле, тем больше информации будет в вашем распоряжении при postmortem-отладке. Самый большой ограничивающий фактор - размер dump-файла. Вы можете оказаться в средах, где получение огромного dump-файла с высокозащищенного сервера неосуществимо, и вам придется работать с урезанной версией.

Средство генерации dump-файла - команда `.dump`. Параметр `.dump /m` указывает отладчику, что нужно сгенерировать mini dump file. Кроме того, команда `.dump /m` может принимать ряд других параметров, подробно описанных в таблице 13.2.

Таблица 13.2. Параметры команды `.dump`

Option	Description
a	Генерирует complete mini dump со всеми включенными параметрами. Включает полные данные памяти, данные информации о дескрипторах, информацию о модулях, базовую информацию о памяти и информацию о потоках. Эквивалентно использованию <code>/mfFhnt</code> .
f	Генерирует mini dump, содержащий все доступные и закоммиченные страницы целевого процесса.
F	Генерирует mini dump, включающий всю необходимую базовую информацию о памяти, чтобы отладчик мог реконструировать все виртуальное адресное пространство.
h	Генерирует mini dump, содержащий информацию о дескрипторах.
u	Генерирует mini dump, включающий информацию о выгруженных модулях. Обратите внимание, что это доступно только в Windows Server 2003.
t	Генерирует mini dump, включающий информацию о временах потоков. Thread time information включает время создания, а также время user mode и kernel mode.

Option	Description
i	Генерирует mini dump, включающий secondary memory information. Secondary memory - любая память (плюс небольшая область вокруг нее), на которую ссылается stack pointer или backing store.
p	Генерирует mini dump, включающий process и thread environment blocks.
w	Генерирует mini dump, включающий все закоммированные read-write private pages.
d	Генерирует mini dump, включающий все image data segments.
c	Генерирует mini dump, включающий все image code segments.
r	Генерирует mini dump, подходящий для сценариев, где важна приватность. Этот параметр стирает (заменяет нулями) любую информацию, не нужную для воссоздания стека (включая локальные переменные).
R	Генерирует mini dump, подходящий для сценариев, где важна приватность. Этот параметр удаляет полные пути к модулям из mini dump, тем самым обеспечивая приватность структуры каталогов пользователя.

Помимо разных параметров, управляющих содержимым dump-файла, необходимо указать имя dump-файла. Когда dump-файл генерируется без указания полного пути, по умолчанию он создается в каталоге, из которого был запущен отладчик. Следующий пример показывает, как сгенерировать dump-файл с полной информацией памяти, используя полный путь.

```
.dump /mf c:\dumpfile.dmp
```

Выполним приведенную выше команду .dump на нашем падающем приложении:

```
0:000> .dump /mf dumpfile.dmp
Creating dumpfile.dmp - mini user dump
Dump successfully written
```

После того как отладчик завершит генерацию dump-файла, у вас должен появиться файл размером примерно 3 МБ. Чтобы использовать dump-файл, загрузите его в новый экземпляр отладчика с помощью переключателя /z. Например, чтобы загрузить только что созданный dump-файл, используйте следующую команду.

```
c:\>windbg -z dumpfile.dmp
```

После загрузки dump-файла отладчиком вы увидите следующий отладочный вывод:

```
...
...
Loading Dump File [C:\dumpfile.dmp]
User Mini Dump File with Full Memory: Only application data is available
Windows XP Version 2600 (Service Pack 2) UP Free x86 compatible
Product: WinNt, suite: SingleUserTS Personal
Debug session time: Thu Nov 16 17:59:25.000 2006 (GMT-8)
System Uptime: 0 days 8:45:08.239
Process Uptime: 0 days 0:01:35.000
...
This dump file has an exception of interest stored in it.
The stored exception information can be accessed via .ecxr.
(9d8.f0c): Access violation - code c0000005 (first/second chance not available)
eax=00000000 ebx=7ffd5000 ecx=77c418bf edx=77c61b78 esi=7c9118f1 edi=00011970
eip=0100127e esp=0007ff1c ebp=0007ff44 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010202
awdscenario1!wmain+0xbe:
0100127e c60000 mov byte ptr [eax],0 ds:0023:00000000=??
```

Ближе к началу отладочного вывода вы заметите, что отладчик дает базовую информацию о загружаемом dump-файле. Информация включает расположение dump-файла и тип dump-файла, а также то, какая информация доступна. Следующая интересная часть информации находится ближе к концу вывода отладчика, где отображается причина fault (access violation). Вооружившись этим dump-файлом, теперь можно отлаживать этот сбой на любой машине без физического доступа к неисправной машине. Позже в этой главе мы подробно обсудим, как работает postmortem-анализ.

При явной генерации dump-файла с помощью отладчика одна из трудностей состоит в том, что отладчик должен быть присоединен к неисправному процессу в правильный момент. Хотя это может не казаться большим препятствием, подумайте о сценариях, где crash воспроизводится лишь время от времени, и возможность присоединить отладчик упущена. Было бы удобно сказать Windows использовать отладчики для генерации dump-файла каждый раз, когда процесс падает. К счастью, такой механизм существует и обычно называется настройкой postmortem debugger. По умолчанию Windows использует Dr. Watson (обсуждается позже в этой главе) как postmortem debugger. Dr. Watson генерирует dump-файл каждый раз, когда процесс падает, и дает пользователю возможность отправить dump-файл в Microsoft для дальнейшего анализа. Используемый postmortem debugger можно изменить с помощью командных строк, показанных в таблице 13.3.

Таблица 13.3. Настройка postmortem debugger

Command Line	AeDebug\Debugger Registry Value	Description
Windbg -I	windbg.exe -p %ld -e %ld -g	Изменяет postmortem debugger на WinDbg. Обратите внимание, что -I должен быть заглавным.
cdb -iae	cdb.exe -p %ld -e %ld -g	Изменяет postmortem debugger на cdb.
ntsd -iae	ntsd.exe -p %ld -e %ld -g	Изменяет postmortem debugger на ntsd.
Drwtsn32 -i	drwtsn32 -p %ld -e %ld -g	Изменяет postmortem debugger на Dr. Watson.

Что происходит за кулисами, когда выполняются командные строки из таблицы 13.3? Ответ довольно прост. Они изменяют несколько значений реестра, на которые Windows смотрит при обнаружении crash процесса. Путь реестра, используемый для настройки postmortem debugger, показан ниже.

```
HKKEY_LOCAL_MACHINE\Software\Microsoft\Windows NT\CurrentVersion\AeDebug
```

Ключ AeDebug содержит два значения, критически важных для настройки postmortem debugger.

Auto

Значение реестра Auto определяет, нужно ли отображать message box при каждом crash, когда настроен postmortem debugger. Если значение равно 0, message box отображается; наоборот, если оно равно 1, message box не отображается. Отображаемое message box - печально известное "Application X has encountered a problem and needs to close." В этом message box пользователь может выбрать отладку неисправного приложения, отправку отчета об ошибке или пропуск отправки отчета об ошибке. Рисунок 13.1 показывает пример message box, когда awdscenario1.exe падает, а ntsd настроен как postmortem debugger.

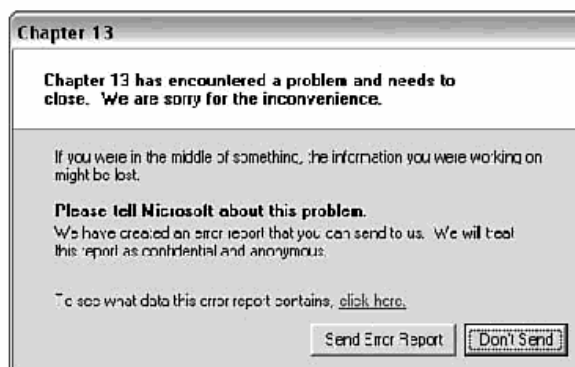


Figure 13.1 Example of the Windows fault message when awdscenario1.exe crashes

Рисунок 13.1. Пример сообщения Windows о сбое при падении awdscenario1.exe

Debugger

Значение реестра Debugger определяет, какой postmortem debugger должен вызываться при каждом fault процесса. Это значение можно либо редактировать вручную, чтобы указать postmortem-инструмент, либо заполнить автоматически отладчиками, если настроить любой из них как postmortem debugger. Как показано в таблице 13.3, за исключением имени инструмента, остальная часть командной строки выглядит так:

```
-p %ld -e %ld -g
```

Часть `-p %ld` указывает, к какому процессу должен присоединиться отладчик, а переключатель `-e %ld` задает событие, которое сигнализируется, когда процесс attach завершен. Наконец, переключатель `-g` инструктирует отладчик игнорировать стандартную первую точку останова, которая всегда возникает при присоединении отладчика к процессу. Что если вы хотите указать дополнительные параметры? Nttd и cdb опционально позволяют указать key string при запуске с переключателем `-iaec`. Key string позволяет добавить дополнительные параметры к значению реестра Debugger даже при использовании собственного процесса регистрации отладчика как postmortem. Например, чтобы настроить nttd с отладкой строк исходного кода, используйте следующую командную строку:

```
nttd -iaec -lines
```

Если проверить значение реестра Debugger, вы увидите, что переключатель `-lines` был добавлен.

```
nttd.exe -p %ld -e %ld -g -lines
```

Хотя отладчики обычно предоставляют более чем достаточно мощности и гибкости при генерации dump-файлов, можно использовать еще один дополнительный инструмент. Этот инструмент называется ADPlus.

Генерация dump-файлов с помощью ADPlus

ADPlus - инструмент, который отслеживает и автоматизирует генерацию dump-файлов для одного или нескольких неисправных процессов и способен уведомлять пользователя или компьютер при возникновении crashes. ADPlus - command-line-driven script, и Microsoft настоятельно рекомендует запускать ADPlus под интерпретатором `cscript.exe`. Фактически, если интерпретатор скриптов по умолчанию задан чем-то отличным от `cscript.exe`, появляется диалог, спрашивающий, хотите ли вы изменить интерпретатор на `cscript.exe`. Помимо параметров командной строки, ADPlus может работать на основе конфигурационных файлов. Конфигурационные файлы дают более детальный контроль над operational flow ADPlus.

ADPlus может работать в одном из двух режимов:

- Hang mode используется для устранения неполадок процессов, демонстрирующих симптомы зависания (например, отсутствие прогресса или 100% CPU utilization). ADPlus должен быть запущен после того, как процесс или процессы для мониторинга уже зависли.
- Crash mode используется для устранения неполадок процессов, демонстрирующих crash-поведение. ADPlus должен быть запущен до того, как процесс упадет.

Используем crash mode как пример того, как применять ADPlus для генерации dump-файла для `awdscenario1.exe`. Начните с запуска приложения `awdscenario1.exe`:

```
C:\AWDBIN\WinXP.x86.chk\awdscenario1.exe
```

Перед нажатием любой клавиши для продолжения выполнения запустите следующую командную строку:

```
C:\>adplus.vbs -crash -pn awdscenario1.exe -y  
SRV*c:\Symbols*http://msdl.microsoft.com/download/symbols
```

Переключатель `-crash` переводит ADPlus в crash mode, переключатель `-pn` сообщает ADPlus имя процесса для мониторинга, а `-y` задает путь к символам, используемый на протяжении выполнения ADPlus. Прелесть использования переключателя `-pn` в том, что он может мониторить любое количество экземпляров процесса с заданным именем.

После завершения выполнения ADPlus поместит результирующие log-файлы в каталог пути установки Windows debuggers. Имя каталога принимает следующую структуру:

```
<runtime>_Mode__Date_<date of run>__Time_<time of run>
```

Например, когда ADPlus завершил выполнение, был создан следующий каталог:

```
C:\Program Files\Debugging Tools for Windows\Crash_Mode__Date_10-21-2006__Time_10-31-17AM
```

Обратите внимание, что путь по умолчанию можно изменить с помощью переключателя `-o`.

В предыдущем каталоге есть несколько файлов, но самые важные - файлы `*.dmp`, которые содержат всю dump-информацию из запуска. Как видно, собирается несколько dump-файлов. Почему у нас больше одного dump-файла на один crash? ADPlus автоматизирует процесс сбора dump-файлов и генерирует dump-файлы при возникновении определенных предустановленных условий во время выполнения. Имя dump-файла дает необходимые подсказки, чтобы понять причину, по которой dump-файл был сгенерирован. Например, в нашем предыдущем запуске ADPlus сгенерировал следующие dump-файлы:

```
PID-2728__AWDSCENARIO1.EXE__1st_chance_AccessViolation__mini_0BA8_2006-10-21_10-31-30-671_0AA8.dmp  
PID-2728__AWDSCENARIO1.EXE__1st_chance_Process_Shut_Down__full_0BA8_2006-10-21_10-31-30-718_0AA8.dmp  
PID-2728__AWDSCENARIO1.EXE__2nd_chance_AccessViolation__full_0BA8_2006-10-21_10-31-30-687_0AA8.dmp
```


ADPlus сгенерировал mini dump file при возникновении first-chance access violation, затем full dump file при возникновении second-chance access, и наконец также создал full dump file при завершении процесса. Нужны ли нам все эти дампы для нашего конкретного запуска? Нет, в нашем конкретном случае самый интересный dump-файл - second-chance access violation. Однако бывают ситуации, где периодическая генерация dump-файлов может быть очень полезной, поскольку она дает историческую перспективу систематического ухудшения состояния процесса. ADPlus даже предлагает мощный способ для пользователя настроить, как часто и при каких условиях должна собираться информация, по сути предоставляя scripting front end для отладчиков. Подробнее о scripting-возможностях ADPlus можно узнать в документации отладчика. Важно отметить, что ADPlus не делает ничего "магического" через свой scripting engine. Он просто берет удобный для пользователя способ указания директив отладчика и переводит их в чистые автоматизированные команды отладчика. Увидеть, как удобная конфигурация фактически переводится в команды отладчика, можно, посмотрев каталог CDBScripts, находящийся в том же каталоге, что и dump-файлы. В нашем примере каталог CDBScripts содержит файл PID-2728__AWDSCENARIO1.EXE.cfg, который содержит все команды отладчика, использованные в этом сеансе ADPlus.

Последний важный момент об ADPlus - как контролировать, какой тип dump-файла генерируется при возникновении fault. Это поведение контролируют четыре переключателя командной строки:

- -FullOnFirst: заставляет ADPlus генерировать full dump file при возникновении first-chance exception.
- -MiniOnSecond: заставляет ADPlus генерировать mini dump file при возникновении second-chance exception.
- -NoDumpOnFirst: говорит ADPlus не генерировать mini dump file при возникновении first-chance exception. Это может быть очень полезно, поскольку приложения иногда генерируют first-chance exceptions, которые корректно обрабатываются.
- -NoDumpOnSecond: говорит ADPlus не генерировать mini dump file при возникновении second-chance exception.

ADPlus - удобный, мощный и гибкий инструмент для мониторинга и сбора данных из неисправных процессов. В этом разделе мы рассмотрели основы инструмента, и определенно стоит потратить время на дальнейшее изучение других мощных возможностей, таких как scripting capabilities и определение пользовательских обработчиков исключений, позволяющих генерировать dump-файлы при возникновении custom exceptions.

Помимо создания user mode dump-файлов, возможны и kernel mode dump-файлы. Следующий раздел иллюстрирует, как генерировать kernel mode dump-файлы.

Создание kernel dumps

Как и с user mode debuggers, в качестве target image можно использовать любой kernel memory dump. Эта функциональность в основном используется разработчиками kernel drivers для postmortem-исследования driver failures. Иногда и user mode developers могут получить пользу от kernel dumps, например когда система является критически важной production system и мы не можем оправдать downtime; или когда проблема сообщена заказчиком, а process dump недостаточен.

Как упоминалось ранее, crash dump file - это дамп состояния машины на момент серьезного системного исключения. Этот dump-файл можно анализировать с помощью debugging tools, загрузив его в отладчик, чтобы получить информацию о состоянии машины и в конечном итоге прийти к причине crash.

Перед созданием crash dump нужно включить и настроить создание дампов. Чтобы настроить эти параметры, перейдите в Control Panel, щелкните значок System, выберите вкладку Advanced, перейдите в область Startup and Recovery и щелкните Settings, как показано на рисунке 13.2. Желаемый размер dump-файла нужно выбрать в списке Write Debugging Information. Для любого конкретного crash может быть создан только один dump-файл. Различие между разными dump-файлами - в размере. Complete Memory Dump самый большой и содержит больше всего информации, Kernel Memory Dump несколько меньше, а Small Memory Dump имеет размер всего 64 КБ. Содержимое этих трех видов kernel mode crash dump-файлов следующее:

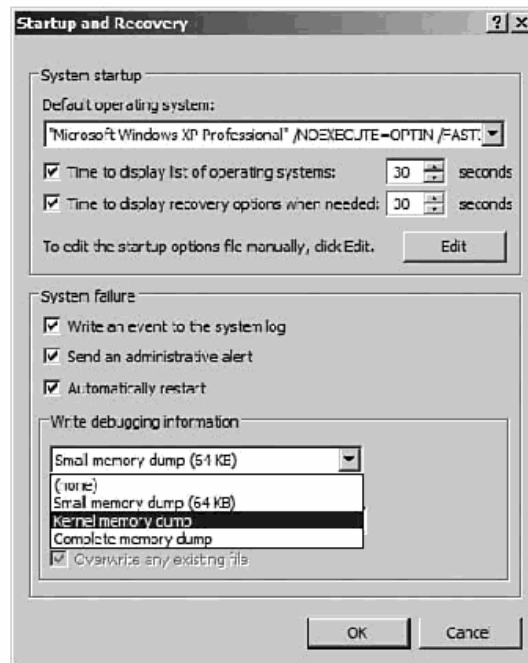


Figure 13.2 Configuring an automatic kernel dump

Рисунок 13.2. Настройка автоматического kernel dump

- Complete Memory Dump: этот dump-файл содержит всю физическую память машины на момент fault. Этот dump-файл требует page file на boot drive размером как минимум не меньше основной системной памяти: он должен быть способен вместить файл, размер которого равен всему вашему RAM плюс один мегабайт.
- Kernel Memory: этот тип dump содержит всю память, используемую kernel на момент crash. Такой dump-файл значительно меньше, чем Complete Memory Dump. Обычно dump-файл будет примерно равен одной трети размера физической памяти системы. Этот dump-файл не включает невыделенную память или любую память, выделенную user mode applications. Он включает только память, выделенную Windows kernel и hardware abstraction layer (HAL), а также память, выделенную kernel mode drivers и другим kernel mode programs.
- Small Memory dump (Minidump): это намного меньший файл, чем два других вида kernel-mode crash dumps. Его размер ровно 64 КБ, и он требует только 64 КБ пространства page file на boot drive. Такой dump-файл может быть полезен, когда пространство сильно ограничено. Однако из-за ограниченного объема включенной информации ошибки, не вызванные напрямую потоком, выполнявшимся в момент crash, могут не быть обнаружены при анализе этого файла.

Small memory dump file включает следующее:

- Bug check message и параметры, а также другие blue screen data.
- Processor context (PRCB) для процессора, который упал.

- Process information и kernel context (EPROCESS) для процесса, который упал.
- Thread information и kernel context (ETHREAD) для потока, который упал.
- Kernel-mode call stack для потока, который упал. Если он длиннее 16 КБ, будут включены только верхние 16 КБ.
- Список загруженных драйверов.

В Windows XP и Windows Server 2003 также включаются следующие элементы:

- Список загруженных модулей и выгруженных модулей.
- Debugger data block. Он содержит базовую отладочную информацию о системе.
- Любые дополнительные страницы памяти, которые Windows определяет как полезные для отладки сбоев. Сюда входят страницы данных, на которые указывали регистры в момент crash, и другие страницы, специально запрошенные faulting component.
- (Только Itanium processor) Backing store.
- (Только Windows Server 2003) Тип Windows, например "Professional" или "Server".

В некоторых ситуациях система не упала; следовательно, crash dump не был создан. Чтобы отлаживать такие случаи, может быть полезно вручную создать crash dump file. На большинстве систем это можно сделать с помощью PS2-клавиатуры. Для включения ручного создания crash dump требуется дополнительная конфигурация. Помимо задания размера и расположения dump-файла, нужно включить keyboard-initiated crash в реестре. В ключе реестра HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\i8042prt\Parameters создайте значение с именем CrashOnCtrlScroll и задайте его равным REG_DWORD 0x1 (или любому ненулевому значению). После перезагрузки системы можно сгенерировать crash, удерживая самый правый клавишу CTRL и дважды нажав клавишу SCROLL LOCK. Это заставляет систему вызвать KeBugCheck() с bug check code 0xE2 (MANUALLY_INITIATED_CRASH).

Распространенная проблема, встречающаяся в сеансах kernel debugger, состоит в том, что некоторые страницы не загружены в физическую память. Поскольку любой paged-out memory address не содержится в dump, в общем случае предпочтительно отлаживать live machine вместо использования kernel dumps. Если kernel debugger включен в системе, kernel debugger получает приглашение расследовать crash. Если kernel debugger не подключен, система просто зависает, ожидая подключения kernel debugger.

Можно сгенерировать явный dump с помощью встроенной команды dump, .dump [option] <filename>, как показано в листинге 13.2. Вся dump-информация передается через communication pipe между debugger и host, и отладчик предупреждает, если запрошенный dump велик для канала связи между kernel debugger host и kernel debugger target. (Например, захват full memory dump из системы с 256 МБ памяти должен передать весь этот объем через канал, что требует многих часов через serial connection.) В течение этого времени debugger target system неработоспособна.

Листинг 13.2

```
kd> * .dump without parameters creates a mini kernel dump file
kd> .dump %temp%\mini.dmp
Creating c:\temp\mini.dmp - mini kernel dump
Dump successfully written
kd> * f parameter generates a full but large dump file
kd> .dump /f %temp%\full.dmp
Creating a full kernel dump over the COM port is a VERY VERY slow operation.
This command may take many HOURS to complete. Ctrl-C if you want to terminate the command.
Creating c:\temp\full.dmp - Full kernel dump
Percent written 0
Failed reading target physical memory at 35000, NTSTATUS 0xC000013A
```

Теперь, когда мы увидели, как генерировать dump-файлы, пора проиллюстрировать, как dump-файлы можно использовать в процессе устранения неполадок.

Использование dump-файлов

Теперь, когда вам передали один или несколько dump-файлов и поручили расследование, чтобы найти root cause неисправного процесса, что на самом деле можно сделать с помощью этих dump-файлов? Можно ли выгружать память, смотреть handles или выполнять код пошагово? Помните, что dump-файл - это просто статический снимок состояния процесса. Поэтому устанавливать breakpoints и пошагово выполнять код невозможно. Использование dump-файлов лучше всего рассматривать как manual debugging. Под manual мы имеем в виду, что, просто глядя на состояние приложения, вам придется вручную строить теории о том, какой код выполнялся и привел приложение в это состояние. Должно быть очевидно, что восстановление выполнения кода через анализ состояния - гораздо более сложная задача, чем live debug session. Тем не менее множество debugger commands, которые преобразуют состояние приложения в более удобоваримую форму, продолжают работать и при использовании dump-файлов; и в большинстве случаев, если набраться терпения, root cause можно найти.

Прежде чем мы подробнее рассмотрим dump-файлы, сгенерированные в предыдущем разделе, нужно поднять один критически важный вопрос: symbol files. Dump-файлы не содержат символической информации, и при анализе dump-файла критически важно, чтобы symbol files были доступны. Это может показаться достаточно простой задачей. Когда приложение выпускается, symbols архивируются; когда нужно анализировать dump, вы просто указываете debugger на архивированные symbols. Такая стратегия хорошо работает, если когда-либо поставлялась только одна версия приложения, но, увы, приложения развиваются и обычно проходят через несколько версий до вывода из эксплуатации. Кроме того, приложения часто патчатся, а это означает, что отдельные компоненты приложения обновляются, усложняя управление версиями symbol files. Чтобы решить эти проблемы, настоятельно рекомендуется настроить symbol server, чтобы symbols архивировались и были доступны простым и организованным способом. Подробную информацию о настройке symbol server см. в главе 4, "Managing Symbol and Source Files". Начнем наш анализ dump-файла с очень базового сценария - access violation.

Анализ dump-файла: Access Violation

В предыдущем разделе мы сгенерировали dump-файл faulting application, и теперь перед нами стоит задача найти root cause, используя только dump-файл. Сгенерированный dump-файл можно найти в следующем расположении:

```
C:\AWDBIN\Dumps\dumpfile.dmp
```

Чтобы использовать dump-файл, мы должны сообщить debugger, что хотим анализировать dump-файл, применив switch -z.

```
C:> windbg -z C:\AWDBIN\Dumps\dumpfile.dmp
```

Когда debugger запустился, первая важная информация - это вывод access violation.

```
This dump file has an exception of interest stored in it.
The stored exception information can be accessed via .ecxr.
(9d8.f0c): Access violation - code c0000005 (first/second chance not available)
eax=00000000 ebx=7ffd5000 ecx=77c418bf edx=77c61b78 esi=7c9118f1 edi=00011970
eip=0100127e esp=0007ff1c ebp=0007ff44 iopl=0         nv up ei pl zr na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010202
awdscenario1!wmain+0xbe:
0100127e c60000 mov byte ptr [eax],0 ds:0023:00000000=??
```

Это сразу сообщает нам, что исследуемый dump-файл был создан из-за access violation. Следующий логичный шаг - вывести stack trace для offending thread и посмотреть, какие операции выполнялись, когда произошло access violation.

```

0:000> kb
ChildEBP RetAddr Args to Child
0007fff44 01001495 00000001 00032bf0 00036890 awdscenario1!wmain+0xbe
0007ffc0 7c816fd7 00011970 7c9118f1 7ffd5000 awdscenario1!wmainCRTStartup+0x12f
0007fff0 00000000 01001366 00000000 78746341 kernel32!BaseProcessStart+0x23

```

Судя по stack trace, видно, что наше приложение вызвало access violation в своей main function. Чтобы получить дополнительные детали, мы дизассемблируем верхний frame и смотрим, что выполнялось в момент access violation.

```

0:000> u awdscenario1!wmain+0xbe
awdscenario1!wmain+0xbe:
0100127e c60000 mov byte ptr [eax],0
01001281 eb0e jmp awdscenario1!wmain+0xd1 (01001291)
01001283 68a4100001 push offset awdscenario1!`string' (010010a4)
01001288 ff1544100001 call dword ptr [awdscenario1!_imp__printf (01001044)]
0100128e 83c404 add esp,4
01001291 688c100001 push offset awdscenario1!`string' (0100108c)
01001296 ff1544100001 call dword ptr [awdscenario1!_imp__printf (01001044)]
0100129c 83c404 add esp,4

```

Offending-инструкция mov, похоже, перемещает значение 0 по адресу, заданному регистром eax. Чтобы выяснить, что содержит регистр eax, используем команду r.

```

0:000> r
eax=00000000 ebx=7ffd5000 ecx=77c418bf edx=77c61b78 esi=7c9118f1 edi=00011970
eip=0100127e esp=0007ff1c ebp=0007ff44 iopl=0 nv up ei pl nz na po nc
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00010202
awdscenario1!wmain+0xbe:
0100127e c60000 mov byte ptr [eax],0 ds:0023:00000000=??

```

Регистр eax содержит значение 0, а это означает, что offending-инструкция mov пыталась сохранить значение 0 в memory location 0. Достаточно сказать, что мы все знаем ожидаемый результат такой инструкции - access violation. Глядя на код в листинге 13.1, мы также видим, что он идеально совпадает с debug session. Более конкретно, строка кода, вызвавшая access violation, - *pBuffer='\\0';, а значение pointer pBuffer равно null.

Теперь, когда мы увидели, как исследовать простое access violation с помощью postmortem-возможностей debugger, рассмотрим немного более сложный сценарий, связанный с handle leak.

Анализ dump-файла: Handle Leaks

Crashes - только одна категория проблем, которые можно анализировать с помощью dump-файлов. Довольно часто в приложения просачиваются resource leaks, и их тоже может понадобиться анализировать через механизм dump-файлов. В этом разделе мы используем sample application из главы 9, "Resource Leaks", который находится здесь:

```

Source code: C:\AWD\Chapter9\HLeak\Client
Binary: C:\AWDBIN\WinXP.x86.chk\09HLeak.exe

```

Мы запускаем приложение под debugger с 20 threads, 200 iterations на thread и sleep time 0 между iterations.

```
C:\AWDBIN\WinXP.x86.chk\09HLeak.exe /t:20 /i:200 /s:0
```

С помощью task manager видно резкое увеличение использования handles. Перед тем как приложение завершится, прервите выполнение в debugger и сгенерируйте dump-файл с помощью switch /mf (full memory).

```

...
...
...

```

```

0:000> .dump /mf c:\dumpfile.dmp
Creating c:\dumpfile.dmp - mini user dump
Dump successfully written

```

После записи dump-файла выйдите из текущего экземпляра debugger, запустите новый экземпляр с только что созданным dump-файлом (или, как вариант, с dump-файлом, расположенным в C:\AWDBIN\Dumps\dumpfile2.dmp) и выполните extension command !handle, чтобы получить представление об использовании handles в приложении.

```

0:000> !handle
ERROR: !handle: extension exception 0x80004002.
"Unable to read handle information"

```

Почему extension command !handle возвращает ошибку? Dump-файл был сгенерирован со switch full memory dump (/mf) и должен содержать всю необходимую информацию. Оказывается, чтобы включить handle information в dump-файл, full memory dump недостаточно. Dump-файлы, которым нужно включать handle information, также должны добавлять h к switch. Снова запустим приложение под debugger и выполним следующую команду .dump перед выходом из приложения.

```

.dump /mfh c:\dumpfile3.dmp

```

Затем загружаем вновь сгенерированный dump-файл в debugger и снова пробуем использовать extension command !handle.

```

0:000> !handle
Handle 00000004
Type Token
Handle 00000008
Type Token
Handle 0000000c
Type File
Handle 00000010
Type Token
Handle 00000014
Type Token
Handle 00000018
Type Token
Handle 0000001c
Type Token
Handle 00000020
Type Token
Handle 00000024
Type Token
...
...
...
738 Handles
Type Count
Event 2
File 1
Port 1
Directory 2
WindowStation 1
Key 1
Token 1440
KeyedEvent 1

```

На этот раз у нас есть полный доступ к handle information в приложении. Мы даже можем получить подробную информацию о любом из перечисленных handles с помощью команды !handle <handle value>. По handle statistics внизу вывода !handle видно, что имеется аномально большое количество token handles. Из главы 9 мы также знаем, что отличный способ выяснить allocation sources для handles - использовать extension command !htrace. Включение htrace для dump-файла кажется лучшим направлением.

```

00> !htrace -enable
ERROR: Cannot enable handle tracing, status code 0xc0000008.

```

И снова кажется, что мы натолкнулись на препятствие. htrase возвращает error code при запуске на dump-файле. Плохая новость в том, что htrase не работает для dump-файлов, и поэтому получить удобные stack traces для каждого открытого handle невозможно. Поскольку htrase information по своей природе статична, один обходной путь для этой проблемы - запустить htrase на live machine и направить все результаты в log file. Dump-файл можно создать с соответствующей handle information, которую затем можно использовать для сопоставления всех open handles со stack traces в log file. Попробуем это на нашем leaky application и посмотрим, что можно узнать из log и dump files. Начните с запуска leaky application под debugger. Задайте symbol path, включите htrase и откройте log file. Возобновите приложение, а перед выходом выполните команду !htrase -diff, чтобы получить stack traces leaked handles, после чего закройте log file. Наконец, сгенерируйте full memory dump с handle information.

```
...
0:000> !htrase -enable
Handle tracing enabled.
Handle tracing information snapshot successfully taken.
...
...
0:000> .logopen c:\handles.log
Opened log file 'c:\handles.log'
...
...
0:000> !htrase -diff
Handle tracing information snapshot successfully taken.
0x1000 new stack traces since the previous snapshot.
Ignoring handles that were already closed...
Outstanding handles opened since the previous snapshot:
-----
Handle = 0x00001274 - OPEN
Thread ID = 0x000000e4, Process ID = 0x00000eb8
0x01002939: 09h!leak!CServer::GetToken+0x00000049
0x01002651: 09h!leak!CServer::GetSID+0x00000021
0x010015f1: 09h!leak!ThreadWorker+0x00000081
0x7c80b683: kernel32!BaseThreadStart+0x00000037
-----
Handle = 0x00001270 - OPEN
Thread ID = 0x00000fb8, Process ID = 0x00000eb8
0x01002939: 09h!leak!CServer::GetToken+0x00000049
0x01002651: 09h!leak!CServer::GetSID+0x00000021
0x010015f1: 09h!leak!ThreadWorker+0x00000081
0x7c80b683: kernel32!BaseThreadStart+0x00000037
-----
Handle = 0x0000126c - OPEN
Thread ID = 0x00000d68, Process ID = 0x00000eb8
0x01002939: 09h!leak!CServer::GetToken+0x00000049
0x01002651: 09h!leak!CServer::GetSID+0x00000021
0x010015f1: 09h!leak!ThreadWorker+0x00000081
0x7c80b683: kernel32!BaseThreadStart+0x00000037
-----
Handle = 0x00001244 - OPEN
Thread ID = 0x00000df0, Process ID = 0x00000eb8
0x01002939: 09h!leak!CServer::GetToken+0x00000049
0x01002651: 09h!leak!CServer::GetSID+0x00000021
0x010015f1: 09h!leak!ThreadWorker+0x00000081
0x7c80b683: kernel32!BaseThreadStart+0x00000037
-----
Handle = 0x0000120c - OPEN
Thread ID = 0x00000b78, Process ID = 0x00000eb8
0x01002939: 09h!leak!CServer::GetToken+0x00000049
0x01002651: 09h!leak!CServer::GetSID+0x00000021
0x010015f1: 09h!leak!ThreadWorker+0x00000081
0x7c80b683: kernel32!BaseThreadStart+0x00000037
-----
Handle = 0x000011d8 - OPEN
Thread ID = 0x00000bdc, Process ID = 0x00000eb8
```

```

0x01002939: 09h!CServer::GetToken+0x00000049
0x01002651: 09h!CServer::GetSID+0x00000021
0x010015f1: 09h!ThreadWorker+0x00000081
0x7c80b683: kernel32!BaseThreadStart+0x00000037
-----
Handle = 0x000011ac - OPEN
Thread ID = 0x000009b0, Process ID = 0x00000eb8
0x01002939: 09h!CServer::GetToken+0x00000049
0x01002651: 09h!CServer::GetSID+0x00000021
0x010015f1: 09h!ThreadWorker+0x00000081
0x7c80b683: kernel32!BaseThreadStart+0x00000037
...
...
...
-----
Displayed 0x349 stack traces for outstanding handles opened since the previous snapshot.
0:000> .logclose
Closing open log file c:\handles.log
0:000> .dump /mfh c:\dumpfile4.dmp
Creating c:\dumpfile4.dmp - mini user dump
Dump successfully written

```

Затем мы загружаем только что созданный dump-файл (или используем dump-файл C:\AWDBIN\Dumps\dumpfile4.dmp) в debugger и выполняем extension command !handle, которая точно показывает, какие handles все еще открыты в процессе. Один из handles, перечисленных как все еще открытый в dumpfile4.dmp, - это:

```

Handle 00001490
Type Token

```

Чтобы найти соответствующий stack trace, который открыл этот handle, смотрим в файл htrace.log и ищем 00001490.

```

Handle = 0x00001490 - OPEN
Thread ID = 0x000002ac, Process ID = 0x00000eb8
0x01002939: 09h!CServer::GetToken+0x00000049
0x01002651: 09h!CServer::GetSID+0x00000021
0x010015f1: 09h!ThreadWorker+0x00000081
0x7c80b683: kernel32!BaseThreadStart+0x00000037

```

Показанный выше stack trace соответствует stack trace, который привел к открытию leaked handle. Отсюда целевой code review раскрывает причину leaked handle. Используя несколько простых корреляционных и исследовательских приемов, мы можем прийти к root cause leaked handles.

В этом разделе главы мы рассмотрели, как использовать dump-файлы, чтобы прийти к root cause конкретной проблемы. Генерация правильного типа dump-файла также имеет первостепенную важность. Без правильной информации, включенной в dump-файл, критически важные недостающие части могут означать разницу между успешной и неудачной debug session. Как общее практическое правило, всегда следует стремиться получить dump-файл с максимально возможным количеством информации, а лучший dump switch для этого - /ma.

Важность postmortem-отладки невозможно переоценить. Часто получить доступ к live debug session невозможно, и наличие инструментов и знаний для правильного анализа bug постфактум критически важно. Фактически postmortem-отладка с использованием dump-файлов настолько важна, что Microsoft построила вокруг нее целый online service, известный как Windows Error Reporting service.

Windows Error Reporting

Любой, кто пользовался Windows, хотя бы раз встречал message box, показанный на рисунке 13.3.

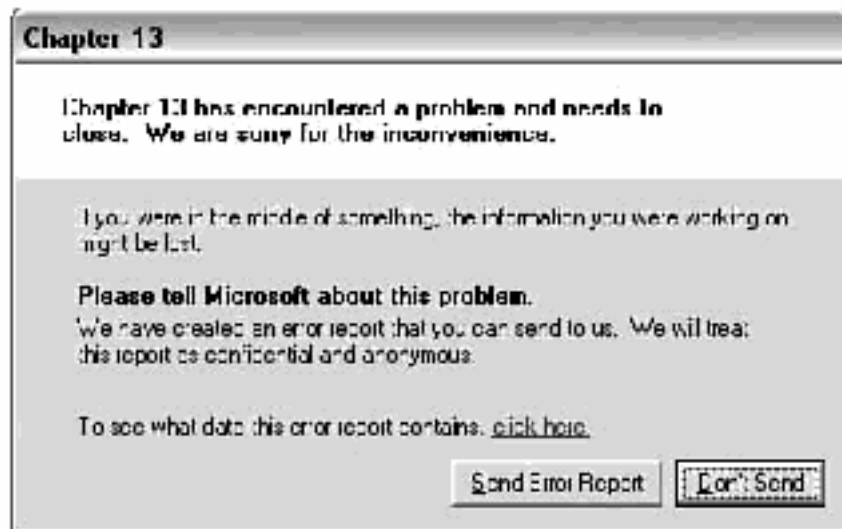


Рисунок 13.3. Message box Dr. Watson

Рисунок 13.3 иллюстрирует характерный UI технологии, известной как Dr. Watson. Когда пользователю показывается это message box, он может отправить error report в Microsoft. Если пользователь выбирает отправку error report, отчет загружается по защищенному каналу (HTTPS) в database Microsoft, где он классифицируется (или bucketized) и сохраняется для последующего анализа. Неудивительно, что информация, отправляемая как часть error report, включает dump file, который помогает разработчикам, разбирающим проблему, найти root cause. Приложения, которые могут участвовать в Windows Error Reporting, не ограничены продуктами Microsoft. Любой crashing process в Windows будет частью того же механизма. Однако, чтобы получить доступ к error reports, соответствующим вашим приложениям, вы должны сначала зарегистрироваться в Windows Error Reporting Service. В этом разделе мы рассмотрим, как работает Windows Error Reporting, что отправляется как часть error report, как зарегистрироваться в Windows Error Reporting database и как запрашивать service для получения error reports.

Dr. Watson

Dr. Watson используется в Windows для генерации error reporting information, которая может быть отправлена в Windows Error Reporting service. Windows настраивает Dr. Watson как default postmortem debugger. Когда происходит application crash, Dr. Watson перехватывает этот crash и генерирует error report, о котором уведомляется пользователь и который пользователь может при желании загрузить в Microsoft. Поведение Dr. Watson при сборе ошибок можно настраивать, запуская drwtsn32.exe. Рисунок 13.4 показывает configuration dialog box Dr. Watson.

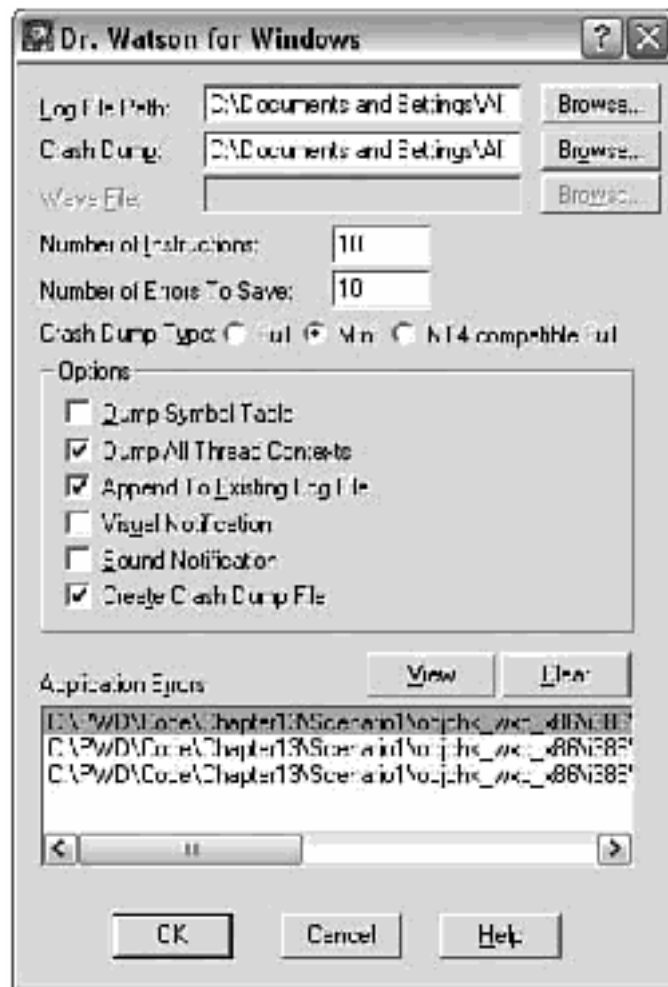


Рисунок 13.4. Configuration dialog box Dr. Watson

Далее обсуждается каждая из настраиваемых options.

Log File Path

Log file path задает путь, по которому будет храниться log file Dr. Watson. По умолчанию log file хранится здесь:

Documents and Settings\All Users\Application Data\Microsoft\Dr Watson

Если вы изменяете этот путь, убедитесь, что указанный path предоставляет всем users write access.

Crash Dump

Crash dump задает path к crash dump files, которые генерируются, когда application faults. Default file path:

Documents and Settings\All Users\Application Data\Microsoft\Dr Watson\user.dmp

Если вы изменяете этот путь, убедитесь, что указанный path предоставляет всем users write access.

Number of Instructions

Задает number of instructions, которые Dr. Watson будет дизассемблировать до и после текущего program counter для каждого thread. По умолчанию количество instructions для дизассемблирования равно 10.

Number of Errors to Save

Задает number of errors, которые будут сохранены в log file. По умолчанию number of errors, попадающих в log, равен 10.

Crash Dump Type

Задает тип crash dump file, который генерируется Dr. Watson. По умолчанию генерируется mini dump file, содержащий только registers, stack и portions of memory.

Options

Таблица 13.4 описывает все options, которые можно настраивать.

Таблица 13.4

Option	Description
Dump Symbol Table	Dump symbol table для каждого module. Обратите внимание, что эта option может делать log files очень большими. По умолчанию эта option не включена.
Dump All Thread Contexts	Выгружает state для каждого thread в faulting application. По умолчанию эта option не включена, и Dr. Watson записывает только state faulting thread.
Append to Existing Log File	Задает, должна ли log information добавляться в existing log file или для каждого нового application fault должен создаваться новый log file. По умолчанию эта option включена.
Visual Notification	Задает, должен ли Dr. Watson выдавать visual notifications о faulting applications. Если включено, показывается dialog box с кнопкой ОК. По умолчанию эта option выключена.
Sound Notification	Задает, должен ли Dr. Watson выдавать audible notifications о faulting applications. Если включено, можно указать WAV file. По умолчанию эта option выключена.
Create Crash Dump File	Задает, должен ли Dr. Watson генерировать dump file для faulting application. По умолчанию эта option включена и требует, чтобы filename был указан в поле Crash Dump.

Application Errors

Option application errors отображает все errors из различных collections Dr. Watson. Возьмем sample awdscenario1.exe, использованный ранее в главе, и посмотрим, какой тип конкретной информации собирает Dr. Watson. Сначала убедитесь, что Dr. Watson выбран как postmortem debugger, выполнив следующее:

```
drwtsn32.exe -i
```

Затем запустите приложение awdscenario1.exe (использованное ранее в главе) с помощью:

```
C:\AWDBIN\WinXp.x86.chk\awdscenario1.exe
```

Когда отображается message box Dr. Watson, нажмите **Don't Send**. Чтобы посмотреть log file, сгенерированный Dr. Watson для этого crash, используем следующий path:

```
Documents and Settings\All Users\Application Data\Microsoft\Dr Watson\drwtsn32.log
```

Log file является plaintext, и для его открытия можно использовать любой text reader или editor, например Notepad.

Log file организован в один или несколько application exception sections, по одному section на fault, где каждый section начинается со следующей строки:

```
Application exception
```

Каждая application exception представляет faulting application и содержит следующую информацию:

- App: полный path к faulting application, за которым следует process identifier.
- When: date and time fault.
- Exception number: exception number и textual representation exception.

В нашем sample run awdscenario1.exe application exception section log file сообщает следующее (если в системе было несколько process crashes, возможно, придется искать crash awdscenario1.exe):

```
Application exception occurred:
App: C:\AWDBIN\WinXP.x86.chk\awdscenario1.exe (pid=2276)
When: 10/19/2006 @ 08:12:41.687
Exception number: c0000005 (access violation)
```

Каждая application exception section далее разбивается на следующие дополнительные категории.

System Information

Section system information содержит огромное количество сведений о system, на которой выполнялось faulting application. Наш sample run дал следующий section system information:

```
*----> System Information <----*
Computer Name: MARIOH-HOME
User Name: marioh
Terminal Session Id: 0
Number of Processors: 1
Processor Type: x86 Family 15 Model 15 Stepping 0
Windows Version: 5.1
Current Build: 2600
Service Pack: 2
Current Type: Uniprocessor Free
Registered Organization: The High-tech Avenue
Registered Owner: Mario Hewardt
```

Как видите, information вроде processor count, processor type и Windows version входит в этот список и может быть очень полезна при анализе проблемы postmortem. Представьте, что конкретный crash происходил только при запуске на Windows XP SP2. Error logs для всех crashes ясно указывают, что crash произошел на Windows XP SP2, а это помогает developer сузить scope проблемы.

Task List

Task list содержит список процессов, выполнявшихся в системе в момент failure. Этот список очень похож на вывод знакомой команды `tasklist.exe`.

Module List

Section module list содержит все modules (включая address range, занимаемый каждым module), которые были загружены в address space faulting process в момент failure. Для нашего sample run module list состоит из следующего:

```
*----> Module List <----*
(0000000001000000 - 0000000001003000: C:\AWDBIN\WinXP.x86.chk\awdscenario1.exe
(0000000007639000 - 000000000763ad000: C:\WINDOWS\system32\IMM32.DLL
(00000000077b4000 - 00000000077b62000: C:\WINDOWS\system32\Apphelp.dll
(00000000077c0000 - 00000000077c08000: C:\WINDOWS\system32\VERSION.dll
(00000000077c1000 - 00000000077c68000: C:\WINDOWS\system32\msvcrt.dll
(00000000077d4000 - 00000000077dd000: C:\WINDOWS\system32\USER32.dll
(00000000077dd000 - 00000000077e6b000: C:\WINDOWS\system32\ADVAPI32.dll
(00000000077e7000 - 00000000077f01000: C:\WINDOWS\system32\RPCRT4.dll
(00000000077f1000 - 00000000077f57000: C:\WINDOWS\system32\GDI32.dll
(00000000077f6000 - 00000000077fd6000: C:\WINDOWS\system32\SHLWAPI.dll
(0000000007800000 - 00000000078f4000: C:\WINDOWS\system32\kernel32.dll
(0000000007c90000 - 0000000007c9b000: C:\WINDOWS\system32\ntdll.dll
```

State Dump for Thread ID X

State dump для faulty thread включает состояние processor registers, а также unassembled code до и после faulting instruction. Кроме того, assembly code в этом section имеет одну annotation, `FAULT ->`,

которая помогает точно указать location failure.

```
*----> State Dump for Thread Id 0xb6c <----*
eax=00000000 ebx=7ffdf000 ecx=77c418bf edx=77c61b78 esi=01c709ea edi=fcead396
eip=0100127e esp=0007ff1c ebp=0007ff44 iopl=0 nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000202
function: awdscenario1!wmain
01001259 8b4dfc mov ecx,[ebp-0x4]
0100125c 51 push ecx
0100125d e84e000000 call awdscenario1!StringCchCopyA (010012b0)
01001262 8b55fc mov edx,[ebp-0x4]
01001265 52 push edx
01001266 68c0100001 push 0x10010c0
0100126b ff1544100001 call dword ptr [awdscenario1!_imp_printf (01001044)]
01001271 83c408 add esp,0x8
01001274 c745fc00000000 mov dword ptr [ebp-0x4],0x0
0100127b 8b45fc mov eax,[ebp-0x4]
FAULT ->0100127e c60000 mov byte ptr [eax],0x0 ds:0023:00000000=??
01001281 eb0e jmp awdscenario1!wmain+0xd1 (01001291)
01001283 68a4100001 push 0x10010a4
01001288 ff1544100001 call dword ptr [awdscenario1!_imp_printf (01001044)]
0100128e 83c404 add esp,0x4
01001291 688c100001 push 0x100108c
01001296 ff1544100001 call dword ptr [awdscenario1!_imp_printf (01001044)]
0100129c 83c404 add esp,0x4
0100129f ff154c100001 call dword ptr [awdscenario1!_imp_getch (0100104c)]
010012a5 33c0 xor eax,eax
010012a7 8be5 mov esp,ebp
```

Stack Back Trace

Section stack back trace содержит stack trace faulting thread. Обратите внимание, что stack back trace может содержать (и, скорее всего, будет содержать) неправильные frames, поскольку symbolic information обычно недоступна. Например, frame 3 в следующем выводе показывает неправильный frame для function в kernel32.dll. Сам факт сообщения неправильных frames не является проблемой, потому что correct symbols можно загрузить при postmortem-анализе fault с помощью debuggers.

```
*----> Stack Back Trace <----*
*** ERROR: Symbol file could not be found. Defaulted to export symbols for C:\WINDOWS\
system32\kernel32.dll -
WARNING: Stack unwind information not available. Following frames may be wrong.
ChildEBP RetAddr Args to Child
0006ff44 01001495 00000001 00262430 00264260 awdscenario1!wmain+0xbe (FPO: [Non-Fpo])
0006ff00 7c816fd7 20c88c32 01c7a3a2 7ffdf000 awdscenario1!wmainCRTStartup+0x12f (FPO: [Non-
Fpo])
0007fff0 00000000 0100131d 00000000 78746341 kernel32!RegisterWaitForInputIdle+0x49
```

Raw Stack Dump

Section raw stack dump содержит raw dump stack в момент failure.

```
*----> Raw Stack Dump <----*
00000000007ff1c 1a 00 00 00 00 00 00 00 - a5 10 00 01 bf 10 00 01 .....
00000000007ff2c 1a 00 00 00 c0 45 03 00 - a5 10 00 01 bf 10 00 01 .....E.....
00000000007ff3c a4 10 00 01 00 00 00 00 - c0 ff 07 00 48 14 00 01 .....H...
00000000007ff4c 01 00 00 00 70 3b 03 00 - 28 45 03 00 50 10 00 01 ....p;..(E..P...
00000000007ff5c 54 10 00 01 94 ff 07 00 - 98 ff 07 00 a0 ff 07 00 T.....
00000000007ff6c 00 00 00 00 9c ff 07 00 - 58 10 00 01 5c 10 00 01 .....X...\.
00000000007ff7c a4 5b 64 2d 8f f3 c6 01 - 00 c0 fd 7f 00 dc 31 e4 .[d-.....1.
00000000007ff8c 00 00 00 00 05 00 00 c0 - 01 00 00 00 70 3b 03 00 .....p;...
00000000007ff9c 00 00 00 00 28 45 03 00 - 00 00 00 00 7c ff 07 00 ....(E.....|...
00000000007ffac 40 fb 07 00 e0 ff 07 00 - 2e 15 00 01 c0 10 00 01 @.....
00000000007ffbc 00 00 00 00 f0 ff 07 00 - d7 6f 81 7c a4 5b 64 2d .....o.|.[d-
00000000007ffcc 8f f3 c6 01 00 c0 fd 7f - 05 00 00 c0 c8 ff 07 00 .....
00000000007ffdc 40 fb 07 00 ff ff ff ff - a8 9a 83 7c e0 6f 81 7c @.....|.o.|
00000000007ffec 00 00 00 00 00 00 00 00 - 00 00 00 00 1d 13 00 01 .....
00000000007fffc 00 00 00 00 41 63 74 78 - 20 00 00 00 01 00 00 00 ....Actx .....
00000000008000c 98 24 00 00 c4 00 00 00 - 00 00 00 00 20 00 00 00 .$.
00000000008001c 00 00 00 00 14 00 00 00 - 01 00 00 00 06 00 00 00 .....
```

```

00000000008002c 34 00 00 00 14 01 00 00 - 01 00 00 00 00 00 00 00 4.....
00000000008003c 00 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00 .....
00000000008004c 02 00 00 00 00 00 00 00 - 00 00 00 00 00 00 00 00 .....

```

В дополнение к богатству information, записанной в log file Dr. Watson, dump file, пожалуй, является самой важной частью information в процессе postmortem analysis. Dump file расположен в том же directory, что и log file, и называется user.dmp (если configuration не была изменена). Используем dump file для нашего запуска awdscenario1.exe и загрузим его в debugger (не забывайте использовать switch -z).

```

...
...
...
This dump file has an exception of interest stored in it.
The stored exception information can be accessed via .ecxr.
(bb0.b6c): Access violation - code c0000005 (first/second chance not available)
eax=00000000 ebx=7ffdf000 ecx=77c418bf edx=77c61b78 esi=01c709ea edi=fcead396
eip=0100127e esp=0007ff1c ebp=0007ff44 iopl=0         nv up ei pl nz na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00000202
awdscenario1!wmain+0xbe:
0100127e c60000 mov byte ptr [eax],0 ds:0023:00000000=??
0:000> kb
ChildEBP RetAddr  Args to Child
0007ff44 01001495 00000001 00032c38 000372c8 awdscenario1!wmain+0xbe
0007ff40 7c816fd7 fcead396 01c709ea 7ffdf000 awdscenario1!wmainCRTStartup+0x12f
0007fff0 00000000 01001366 00000000 00000000 kernel32!BaseProcessStart+0x23

```

Обратите внимание, что frame 3 в stack trace теперь показывает correct symbolic name. С этого момента анализ dump file идентичен упражнению, показанному ранее в главе.

Можно ли выключить Dr. Watson? Конечно. Самый простой способ выключить Dr. Watson - щелкнуть правой кнопкой My Computer, выбрать Properties, а затем вкладку Advanced. Рисунок 13.5 показывает отображаемый dialog.



Рисунок 13.5. Управление Windows Error Reporting

Как видно из рисунка 13.5, доступны две основные options:

- Disable error reporting: эта option отключает error reporting, но все еще дает возможность получать уведомления о critical errors.
- Enable error reporting: вы можете управлять тем, какие типы error reports включить, выбирая errors, связанные с Windows operating system, и/или Program errors. Если выбрана категория Programs,

можно дополнительно выбрать, по каким programs отправлять reports, нажав кнопку Choose Programs. Процесс выбора конкретных applications довольно прост и понятен сам по себе.

Важный вывод из этого section состоит в том, как работает Dr. Watson, как он configured, что он collects и где хранятся resulting data.

Хотя полезно иметь возможность настраивать и debug проблемы локально, сила Windows Error Reporting исходит из способности загружать error information в Microsoft и затем извлекать error information структурированным образом. Следующий section обсуждает детали Windows Error Reporting, включая enrollment process, configuration process, feedback loops и многое другое.

Архитектура Windows Error Reporting

Windows Error Reporting - это failure data aggregation service, который позволяет Microsoft и ISVs легко получать доступ к failure data, связанной с их applications. Рисунок 13.6 показывает high-level operational flow WER service.

Две основные сущности участвуют в рисунке 13.6:

- Computers, выполняющие applications, в которых проявляются проблемы и которые загружают error reports в WER.
- ISV, который отслеживает failures, связанные со своими applications и reported to WER.

Предположим, что некая machine где-то в мире выполняет application (показанное как Process X на рисунке 13.6), произведенное компанией AWD. Допустим также, что application crashes, и пользователю, столкнувшемуся с crash, показывается UI Dr. Watson с вопросом, хочет ли он отправить error report в Microsoft. Пользователь выбирает отправку, и error report отправляется по защищенному каналу (HTTPS) в WER service. WER service, в свою очередь, организует полученную error information по категориям (известным как buckets) и сохраняет error information. Чтобы использовать error reports, пользователь из компании AWD запрашивает WER service на предмет crashes, связанных с его application, и получает reported error information. Если AWD выбирает это сделать, они теперь могут исправить проблему и предоставить response, чтобы в следующий раз, когда пользователь столкнется с тем же crash, Dr. Watson показал ему этот response. Response может быть представлен в форме fix или другой полезной information.

Как видите, WER service - невероятно мощный механизм, который обеспечивает secure aggregation of error reporting information, которую ISVs могут запрашивать, чтобы активно оценивать health своих applications. Кроме того, ISVs могут предоставлять responses для известных problems и интегрировать responses в WER feedback loop, что делает применение responses при их наличии очень простым для customers.

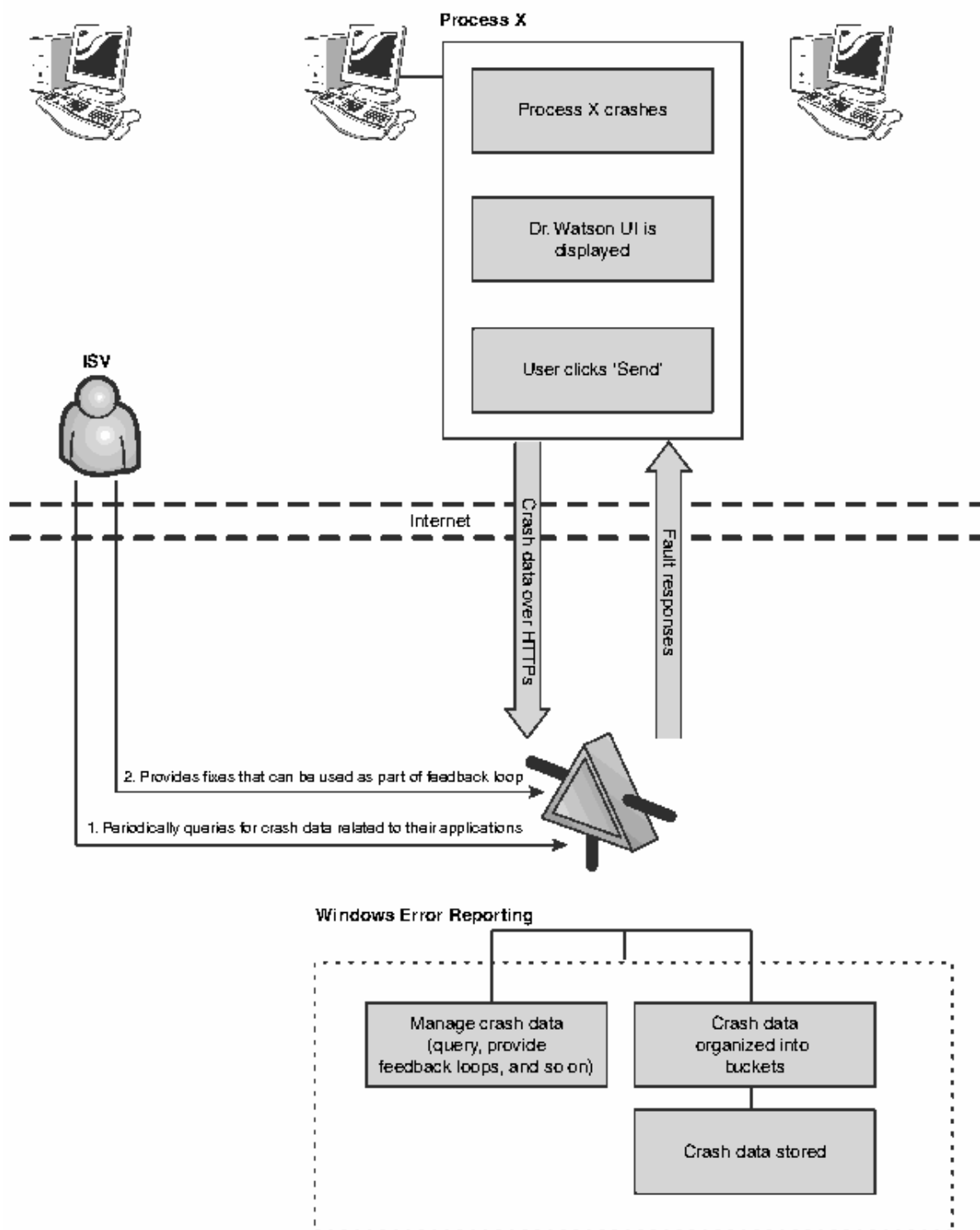


Рисунок 13.6. High-level overview of WER functionality

Важность отправки error information

Когда UI Dr. Watson появляется и сообщает вам об application crash, вы можете спросить себя: зачем вообще отправлять это? Будет ли с этим когда-нибудь что-то сделано? На самом деле Microsoft относится к error reporting очень серьезно. В конце концов, именно поэтому этот невероятный service вообще был реализован. Error reporting data активно мониторятся и распределяются внутри компании по соответствующим product groups. Когда fix найден и готов к выпуску (обычно через Microsoft Update), users могут легко применить fix. Другими словами, вы -

пользователь - напрямую влияете на видимость bugs; поэтому всегда следует загружать error reports, чтобы Microsoft или другие ISVs получили возможность проанализировать проблему и предоставить fix.

На протяжении оставшегося обсуждения WER мы будем использовать приложение awdscenario1.exe, применявшееся в предыдущей части главы, чтобы практически проиллюстрировать процесс использования WER.

Первый шаг в использовании WER - enrollment; он описан далее.

Регистрация в Windows Error Reporting

Чтобы участвовать в WER, нужно завершить enrollment process. Enrollment process разбивается на два шага:

- Создание user account.
- Создание company account.

Чтобы начать enrollment process, перейдите по следующему URL:

<https://winqual.microsoft.com/SignUp/>

После загрузки страницы вам будет показана account creation page, как на рисунке 13.7.

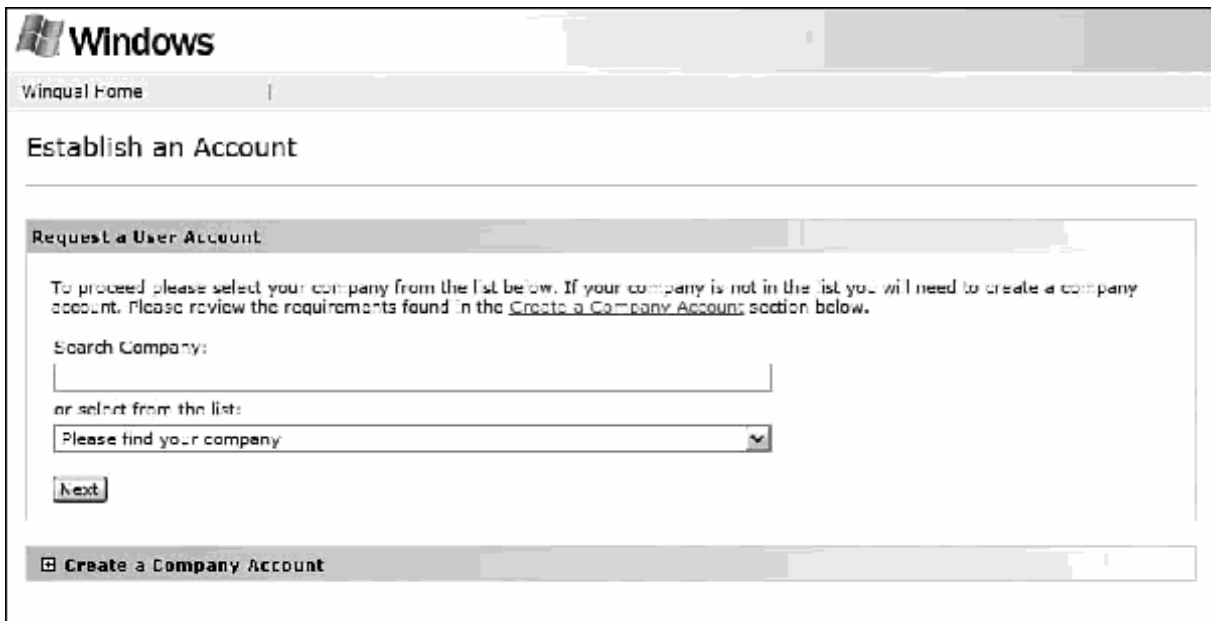


Рисунок 13.7. Первая страница процесса регистрации WER

Чтобы создать user account, сначала нужно иметь company account. Если вы уже создали company account, можно либо найти этот account поиском, либо выбрать его в drop-down list. Нажатие кнопки Next затем переводит вас на account creation page. Поскольку company account мы еще не создали, нажмите expand button рядом с Create a Company Account, чтобы начать процесс создания company account, как показано на рисунке 13.8.

Рисунок 13.8. Создание company account

Создание company account включает три шага.

1. Сгенерировать code signed файл Winqual.exe. Чтобы компания могла участвовать в WER, Microsoft требует, чтобы компания была способна безопасно и уникально идентифицировать себя. Это достигается с помощью digital code signing certificate Class 3, который можно приобрести у VeriSign (<http://www.verisign.com/products-services/security-services/code-signing/index.html>). После получения code-signing certificate нужно подписать файл Winqual.exe этим certificate и загрузить его в Microsoft для verification.
2. Предоставить billing information. Хотя Microsoft не взимает плату с компаний за большую часть WER functionality, некоторые features WER стоят денег, и поэтому Microsoft требует ввести billing information.
3. Последняя часть процесса - предоставить contact data, создав user account, который вы будете использовать для доступа к company account.

Начнем с шага 1 (code signing файла Winqual.exe). Как упоминалось, по соображениям security Microsoft требует, чтобы все WER company accounts идентифицировались с помощью digital code signing certificate Class 3. Остальные sections о WER предполагают, что вы приобрели code signing certificate у VeriSign. Первый шаг - загрузить binary, который нужно подписать, из Microsoft. Используйте следующий URL, чтобы начать download Winqual.exe:

<https://winqual.microsoft.com/signup/winqual.exe>

Сохраните этот file на hard drive в C:\Sign. Далее нужно получить code signing tools, необходимые для signing binaries. URL для download code signing tools:

<https://winqual.microsoft.com/signup/signcode.zip>

Сохраните этот file и извлеките signcode.zip в C:\Sign. В результате извлечения zip file у вас теперь должны быть два files:

- Readme.rtf: этот file содержит instructions о том, как code sign binary с помощью code-signing tools. Он также содержит password, который нужно использовать при извлечении файла signcode.exe (password protected), также расположенного в zip file.
- Signcode.exe: это application, которое мы будем использовать для code sign файла Winqual.exe.

Извлеките файл signcode.exe (не забудьте ввести password, найденный в readme file, при извлечении) в то же расположение, что и файл Winqual.exe (C:\Sign). Также обязательно скопируйте code-signing certificate file (extension .spc) и private key (extension .pvk) в то же расположение. Используйте следующую command line, чтобы подписать файл Winqual.exe:

```
C:\Sign>signcode.exe /spc myCert.spc /v myKey.pvk -t
http://timestamp.verisign.com/scripts/timestamp.dll winqual.exe
Succeeded
```

Нужно заменить `mycert.spc` и `mykey.pvk` именами ваших certificate и private key files. Во время signing process вас попросят ввести private key password. Введите password, предоставленный вам VeriSign при покупке certificate. Если signing succeeds, отображается message Succeeded. Если возникает error, убедитесь, что вы правильно ввели имена certificate и private key files и что они находятся в том же directory, что и binary `signcode.exe`.

Следующий шаг в enrollment process - взять newly signed файл `Winqual.exe` и загрузить его в Microsoft для verification purposes. Продолжая со страницы, показанной на рисунке 13.8, нажмите кнопку Next. Следующая page позволяет загрузить ваш signed файл `Winqual.exe`, как показано на рисунке 13.9.

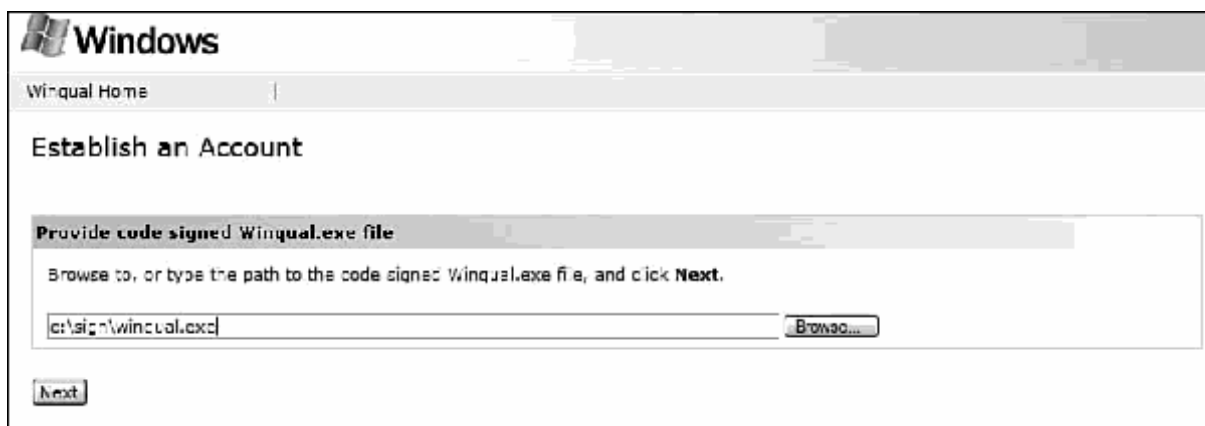


Рисунок 13.9. Upload signed Winqual.exe binary

Просто введите path к signed binary `Winqual.exe` и нажмите Next, чтобы загрузить file. Следующая page в process - Billing information page, показанная на рисунке 13.10.

Windows

Winqual Home

Establish an Account

Billing Address

Address Line 1:

Address Line 2:

City:

State/Province:

ZIP/Postal Code:

Country/Region:

E-mail Address:

Phone Number:

Fax Number:

Default Purchase Order Number:

Bold fields are required.

Рисунок 13.10. Billing information page

Как упоминалось ранее, большинство features WER бесплатны, но за некоторые взимается fee. Billing information используется, если customer применяет WER services, которые стоят money.

Введите billing information вашей company (bold fields обязательны) и нажмите кнопку Next, которая перенесет вас на account (profile) creation page, показанную на рисунке 13.11.

Windows

Winqual Home

Establish an Account

Profile

Please make a note of your User Name and Password you will need to sign in if your account request is approved.

User Name:

Password:

Confirm Password:

Secret Question:

Secret Answer:

Full Name:

Work E-mail Address:

Work Phone Number:

Work Fax Number:

Next

Bold fields are required.

Password Requirements:
A password must:

- be at least 8 characters long and no longer than 16 characters
- have at least 1 lower-case alphabetic character
- have at least 1 upper-case alphabetic character
- have at least 1 number
- have at least 1 punctuation character/symbol
- have at least 1 non-alpha (number or a punctuation/symbol) within the 2nd to 5th character

Рисунок 13.11. Profile information page

Поля User Name и Password представляют logon information, которую вы будете использовать при доступе к WER site. Заполните всю information и обратите особое внимание на strong password requirements, перечисленные внизу. Эти password requirements важны для обеспечения security error information вашей company. После заполнения всей information нажмите кнопку Next, и вы попадете на page, указывающую, что account успешно создан, как показано на рисунке 13.12.



Рисунок 13.12. Account creation succeeded

Финальные шаги, которые нужно завершить, прежде чем мы сможем получить доступ к WER, включают настройку permissions и signing legal agreements. Начнем с управления permissions. Нажмите link Manage Permissions, чтобы перейти на permissions page, показанную на рисунке 13.13.

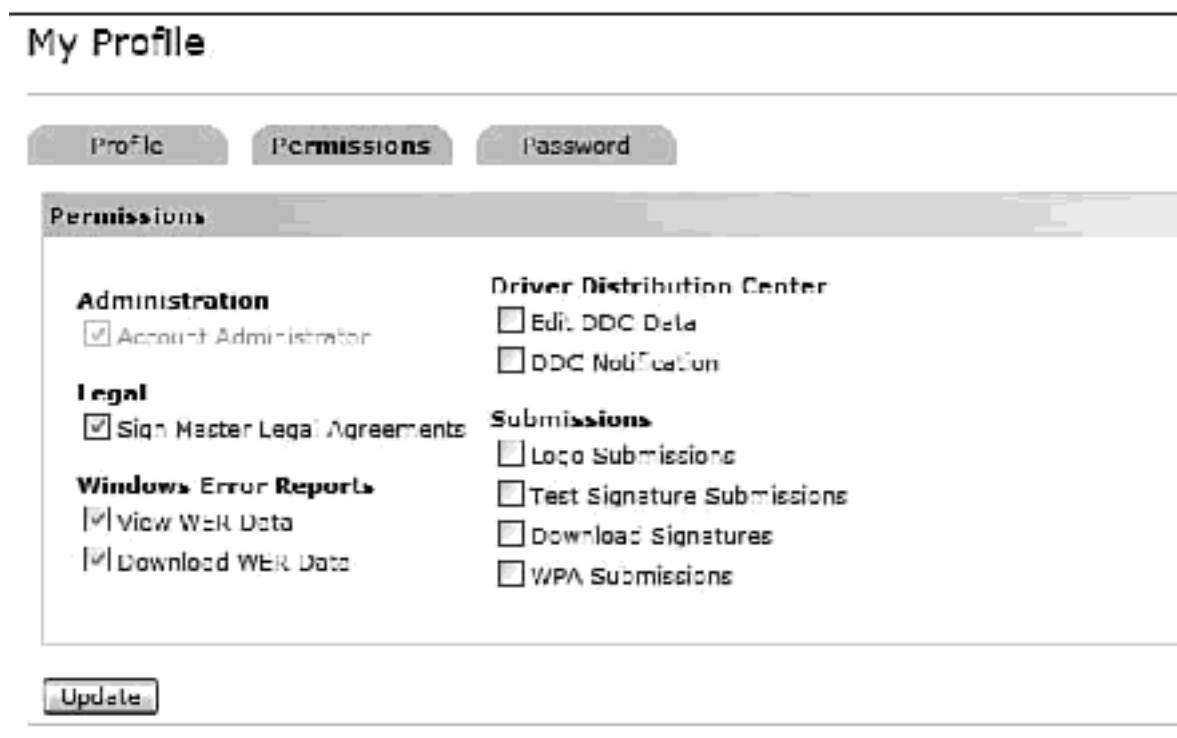


Рисунок 13.13. Manage permissions

Убедитесь, что check boxes Sign Master Legal Agreements, View WER Data и Download WER Data включены для вашего account, и нажмите кнопку Update. Обратите внимание, что в WER можно иметь несколько user accounts, связанных с одним company account. Это может быть очень полезно, если нужно предоставить разным users разные levels of access (как показано на рисунке 13.13). Например, одному user можно предоставить доступ к error reports, а другому - доступ к signing legal agreements.

Затем мы возвращаемся на page, показанную на рисунке 13.12, чтобы завершить process, подписав legal agreements, required by Microsoft. К сожалению, на момент написания link Sign Legal Agreements был недоступен, и нам нужно использовать альтернативный URL:

<https://winqual.microsoft.com/member/LAC/DocumentDetails.aspx?id=420&type=0>

URL переводит вас на Windows Error Reporting legal agreement page. Внимательно прочитайте всю представленную information и, если выбираете ассепт, заполните information внизу последней page, чтобы подписать agreement. Если вы хотите copy agreement для своих records, можно ввести company information в form и распечатать copy.

Signup process теперь complete, и вы можете получить доступ ко всему range WER features, войдя в account по следующему URL:

<https://winqual.microsoft.com/default.aspx>

Навигация по WER Web Site

Когда вы входите на WER Web site, вам показывается page с recent Winqual announcements. Слева от announcements находится pane, позволяющая переходить в разные части site. Три основных section navigation pane:

- Windows Logo Programs
- Windows Error Reports
- Driver Distribution Center

В этой главе мы рассматриваем только section Windows Error Reports на Web site и, более конкретно, Software portion WER. Рисунок 13.14 иллюстрирует options, доступные в menu Software.

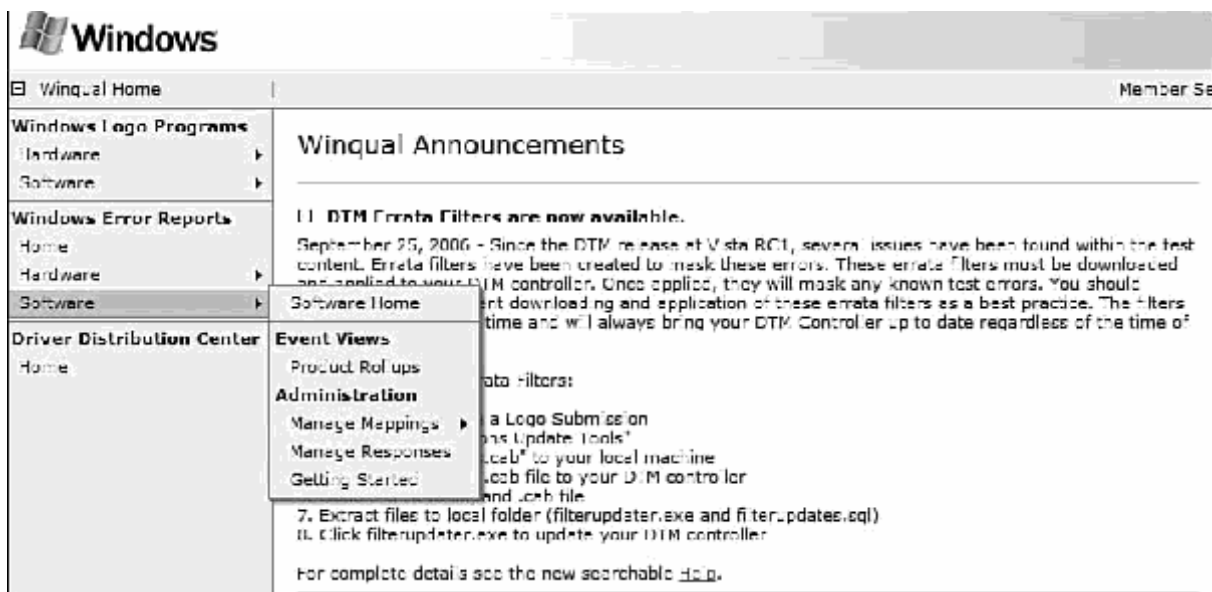


Рисунок 13.14. WER Software options

Option Product Rollups в категории Event Views показывает view, организующий error reports по product name и version. Рисунок 13.15 показывает пример Product Rollup page.

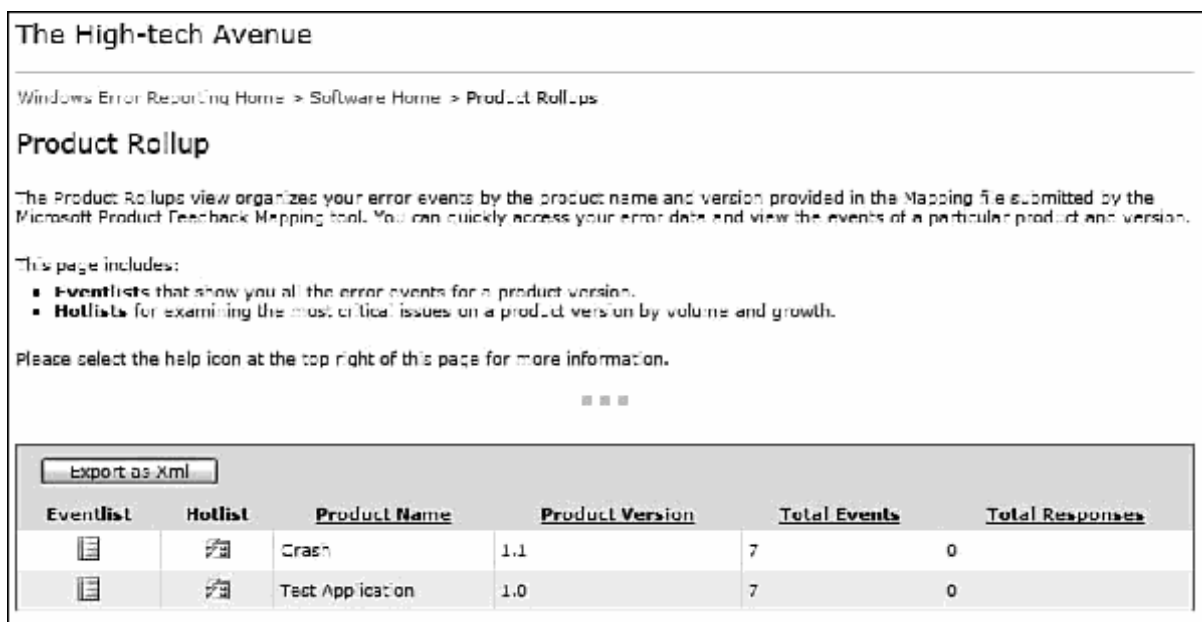


Рисунок 13.15. Пример Product Rollup page

Рисунок 13.15 показывает два registered products: Crash и Test Application. У каждого product есть две columns, позволяющие углубиться в любые events (crashes или другие), которые могли быть reported для applications:

- Eventlist: icon Eventlist переводит на page, подробно перечисляющую полный list events, произошедших в application.
- Hotlist: icon Hotlist переводит на page, подробно показывающую biggest hitters для application за последние 90 days.

Следующий menu item - категория Administrative. Она содержит следующие options:

- Manage Mappings: эта option позволяет сопоставлять binaries с products, чтобы WER знал, какие binaries относятся к какому product. Позже в этой главе мы покажем, как создать mapping file.
- Manage Responses: эта option позволяет определять responses на common problems, reported by customers, и, по сути, создавать feedback loop, который может содержать что угодно - от informative messages до fixes. Позже в этой главе мы рассмотрим, как generate response.
- Getting Started: option Getting Started переводит на help page.

Теперь, когда мы познакомились с general layout WER site, пора сопоставить binaries нашего product с конкретным product, чтобы WER знал, какой binary относится к какому product.

Сопоставление binaries с products

После доступа к account нужно убедиться, что любая error information, reported для ваших applications, routed to your company account. Когда error report отправляется в WER service, service должен знать, что именно в application идентифицирует его как принадлежащее конкретной company. Ключевой ингредиент в этом mapping process - имя application. Поэтому companies, регистрирующиеся в WER, должны сообщить service имена applications (включая все binaries), связанных с их company. Затем mapping information передается на WER site с помощью XML file, который WER service понимает. Чтобы customers не составляли этот mapping XML file вручную, на WER site есть tool с именем Microsoft Product Feedback Mapping Tool. Tool можно найти по следующему URL:

<http://www.microsoft.com/downloads/details.aspx?FamilyId=4333E2A2-5EA6-4878-BBE5-60C3DBABC170&displaylang=en>

После установки запустите tool из Start, Programs, Microsoft Product Feedback Mapping Tool, и вам будет показан wizard, который проведет вас через mapping process. Первая page wizard показана на рисунке 13.16.

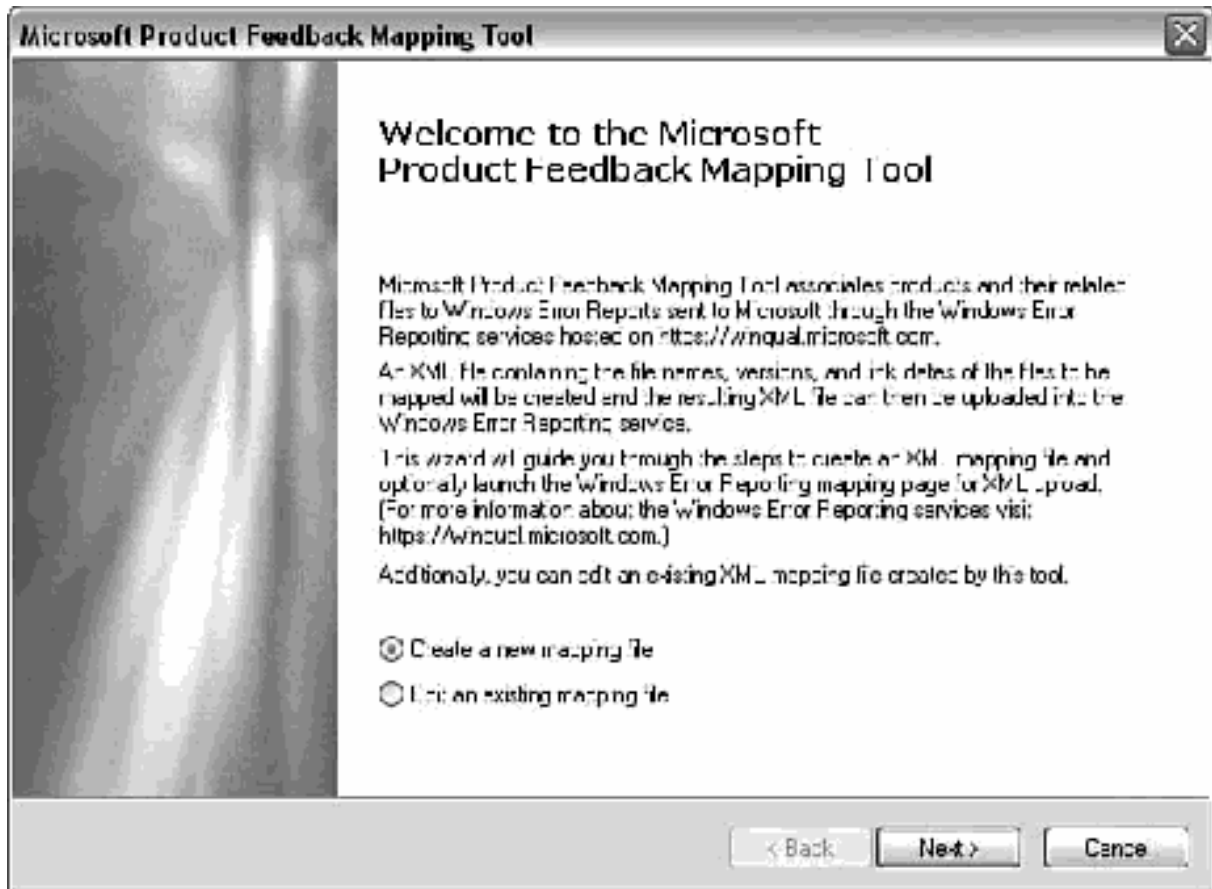


Рисунок 13.16. Microsoft Product Feedback Mapping Tool

Чтобы проиллюстрировать процесс настройки mapping file для WER, мы используем приложение awdscenario1.exe, применявшееся ранее в главе. Убедитесь, что выбрано Create a New Mapping File, и нажмите Next. Рисунок 13.17 показывает page Gathering Product Mapping Information.

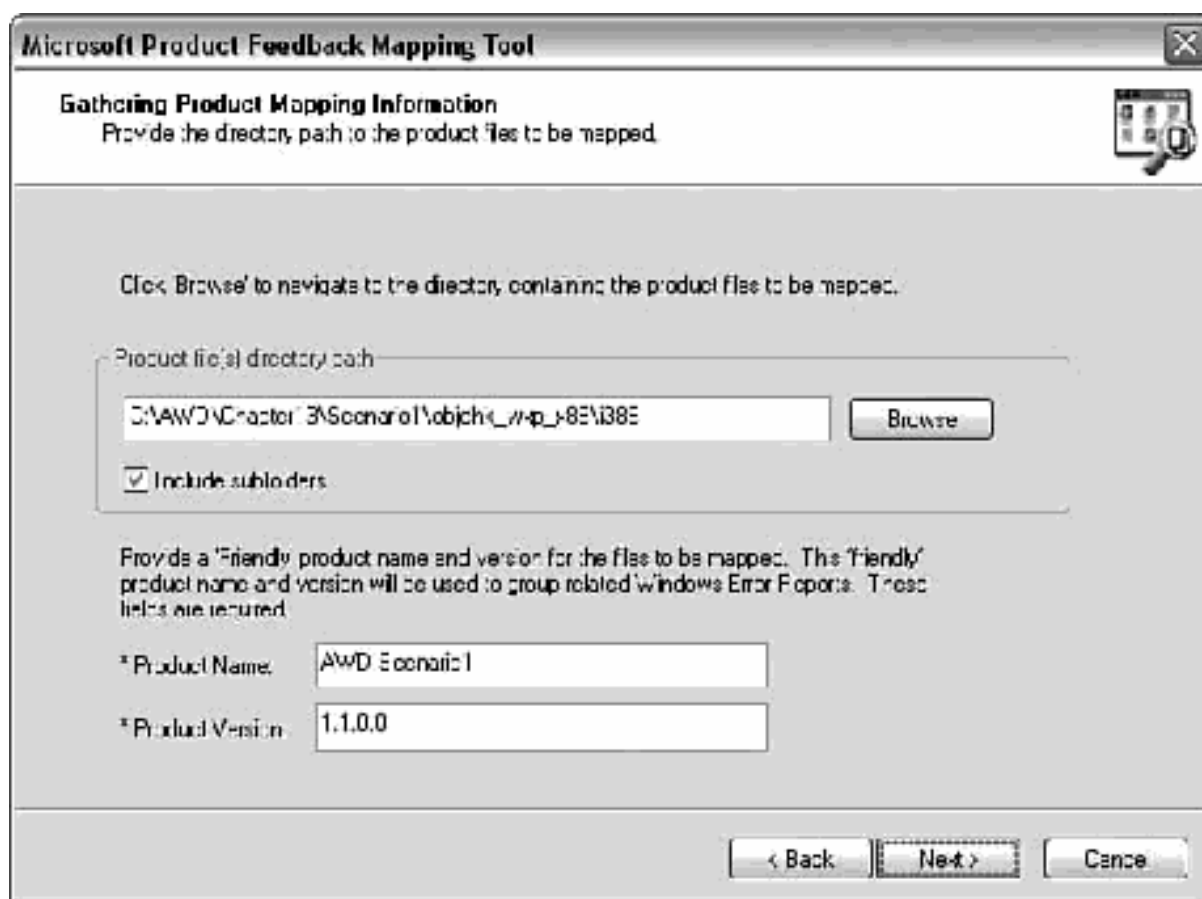


Рисунок 13.17. Gathering Product Mapping Information page

Options, показанные на рисунке 13.17, объясняются далее. Убедитесь, что вводите information так, как показано на figure.

- Product File(s) Directory Path: задает directory path к application binaries, которые вы хотите map.
- Product Name: задает имя product, с которым должны быть связаны binaries. Обратите внимание, что product name - это просто friendly name, используемое на WER site, чтобы users могли эффективнее группировать и искать error information.
- Product Version: задает product version, с которой должны быть связаны binaries. Обратите внимание, что product version - это просто friendly version, используемая на WER site, чтобы users могли эффективнее группировать и искать error information.

Когда вся information введена, нажмите Next, а затем еще раз Next. Wizard теперь просит указать filename для mapping file, который он собирается сгенерировать. Введите следующий path для map file и нажмите Next.

C:\testmap.xml

Рисунок 13.18 показывает последний шаг process, который позволяет загрузить mapping file на WER site.



Рисунок 13.18. Uploading mapping file to WER

Убедитесь, что check box отмечен, и нажмите Finish. Wizard теперь запускает browser и показывает File Upload page, как показано на рисунке 13.19.

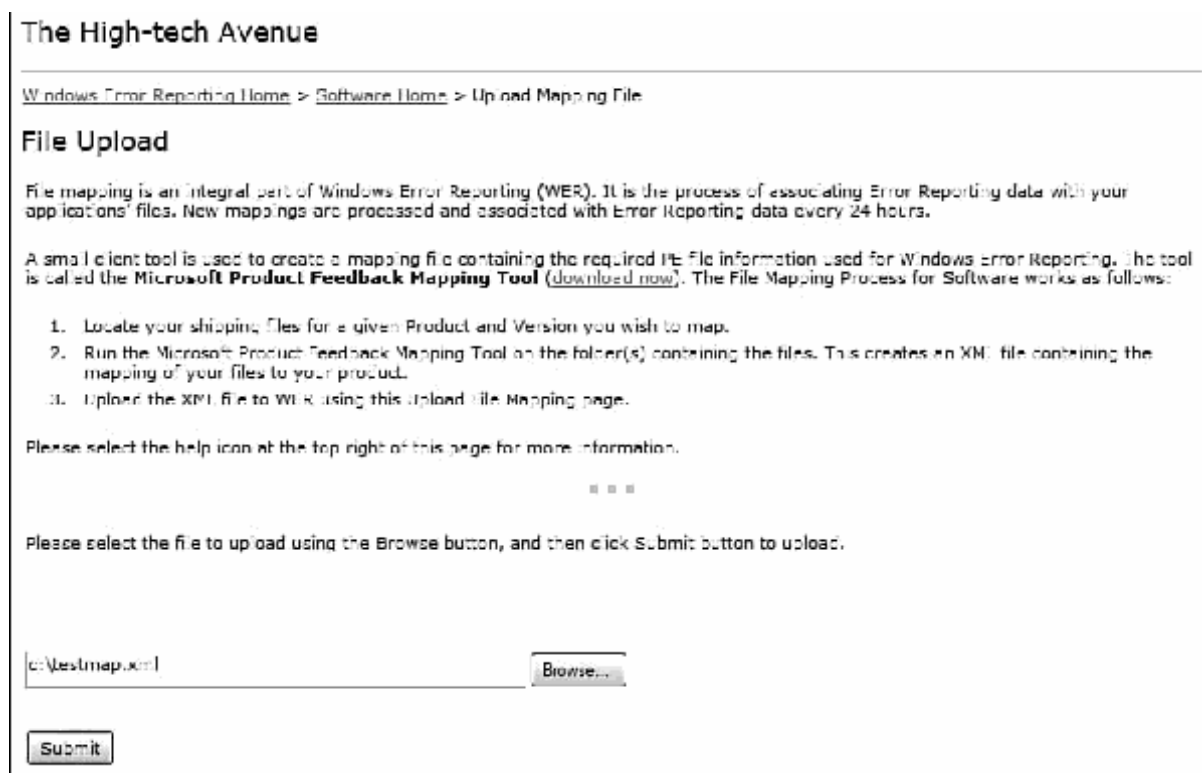


Рисунок 13.19. File Upload page

Введите path к map file, который мы только что создали, и нажмите Submit. После successful upload file-mapping process и upload завершены. Если у вас больше одного product, нужно пройти весь mapping process снова - один раз для каждого product.

Вернувшись на основной WER site, можно управлять product и file mappings, выбрав options Software и Manage Mappings в left navigation pane. Можно выбрать управление product и file mappings, а также upload mapping file. Например, выбор link File Mapping после upload mapping file показан на рисунке 13.20.

The screenshot shows a web page titled 'The High-tech Avenue' with a breadcrumb trail: 'Windows Error Reporting Home > Software Home > Manage File Mappings'. The main heading is 'Manage File Mappings'. Below it, there is explanatory text about file mapping and a list of two bullet points. At the bottom, there is a table with columns: Product Mappings, File Name, File Version, Link Date, Map Date, Mapped By, and Mapped By E-mail. The table contains one row with the file 'awdscenario1.exe'.

Product Mappings	File Name	File Version	Link Date	Map Date	Mapped By	Mapped By E-mail
	awdscenario1.exe	1.1.0.0	11/17/2006 1:45:46 AM	11/16/2006 5:50:02 PM	marion	marion@advancedwindowsdebugging.com

Рисунок 13.20. WER file mappings

Из рисунка 13.20 видно, что у нас есть одно file mapping, где filename - awdscenario1.exe с конкретными attributes (например, link date и map date), а также administrative information, например кто создал mapping и его email address.

Теперь, когда мы создали product и file mapping, пора посмотреть на report generation aspects WER. Мы рассмотрим, как можно generate reports по error information, отправленной customers, а также глубже погрузиться в каждый error report (например, crash dumps).

Запрос Windows Error Reporting Service

Теперь, когда мы создали account и mapped наш binary awdscenario1.exe на product, пора посмотреть, как query WER для uploaded error reports. Запустим наше приложение awdscenario1.exe несколько раз и, когда оно crashes, скажем Dr. Watson upload error information на WER site. Обратите внимание, что между моментом, когда user uploads report, и моментом, когда он становится available to view, есть time delay. Average delay обычно составляет около 7 days.

После того как error reports загружены и стали available для вас, вы увидите table of products на Product Rollup page, как показано на рисунке 13.21.

The High-tech Avenue

Windows Error Reporting Home > Software Home > Product Rollups

Product Rollup

The Product Rollups view organizes your error events by the product name and version provided in the Mapping file submitted by the Microsoft Product Feedback Mapping tool. You can quickly access your error data and view the events of a particular product and version.

This page includes:

- **Eventlists** that show you all the error events for a product version.
- **Hotlists** for examining the most critical issues on a product version by volume and growth.

Please select the help icon at the top right of this page for more information.

■ ■ ■

Export as Xml					
Eventlist	Hotlist	Product Name	Product Version	Total Events	Total Responses
		AWD Scenario1	1.1.0.0	1	0

Рисунок 13.21. Product Rollup with error events

Рисунок 13.21 показывает product, который мы mapped (AWD Scenario1), а также total number of events, которые были reported. Кроме того, columns Eventlist и Hotlist содержат icons, которые отображают все events, произошедшие для этого particular product, а также top error events за последние 90 days. Hotlist - удобный способ определить top issues с product. Рисунок 13.22 иллюстрирует Event List page, отображаемую при щелчке icon Eventlist.

Event List for product: AWD Scenario1 1.1.0.0

The Event List page displays events for a specified product or search criteria. This page is useful for reviewing the complete list of events for a given product or search criteria.

Please select the help icon at the top right of this page for more information.

■ ■ ■

Show filter										
Export as Xml										
Event ID	Cabs	Responses	Total Hits	Avg. Hits	Growth Percent	Event Type	Application Name	Application Version	Module Name	Module Version
318048162			22	11.00		Crash32	awdscenario1.exe	1.1.0.0	awdscenario1.exe	1.1.0.0

Рисунок 13.22. Event list for AWD Scenario1

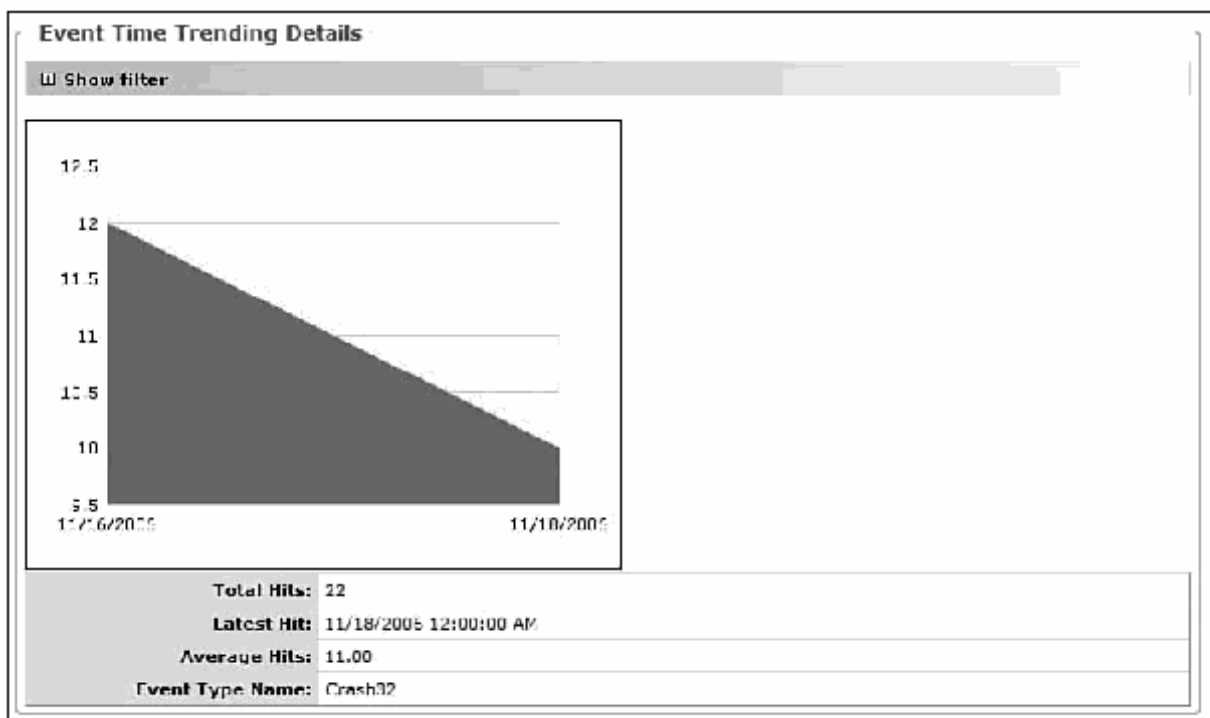
Event List page содержит table, в которой каждая row представляет unique error event. На рисунке 13.22 видно, что есть только один event с total hit count 22. Table также показывает, какой type of event вызвал report; в нашем случае event type - Crash32, что просто означает, что event произошел из-за crash. Если щелкнуть event ID, вы увидите breakdown information, связанной с этим particular event. Event Details page разбита на три main sections:

- **Event Signature:** поскольку один product может иметь multiple events, связанных с ним, каждый event должен быть made unique. Разные части information, делающие event unique, - это application name и version, module name и version, а также offset into the module, из-за которого event occurred. Как видно на рисунке 13.23, offset into module awdscenario1.exe, вызвавший crash, равен 4734.
- **Event Time Trending Details:** graph, отображаемый в section Event Time Trending Details, показывает, как event проявлялся over time. На рисунке 13.23 видно, что наш event резко вырос

16 November, а затем gradually decreased in frequency over time.

- Platform Details: последний section показывает platform details для конкретного event. Он показывает operating system breakout, а также language breakout. Этот section критически важен, когда нужно identify problems, происходящие только при определенных configurations, и может дать clues, например что event происходит только на non-English versions product.

Event Signature	
Application Name	awdsscenario1.exe
Application Version	1.1.0.0
Module Name	awdsscenario1.exe
Module Version	1.1.0.0
Offset	4/34
Exception Code	00000004
Cab Status	Cabs requested



Platform Details	
Operating System Breakout:	Windows XP SP2 >95%
Language Breakout:	English - United States >99%

Рисунок 13.23. Event details for AWD Scenario1

Последняя важная column в table, проиллюстрированной на рисунке 13.22, - это column cabs. Щелчок по icon дает list of cabs, доступных для event. Cab - это не более чем conglomerate of files, представляющих event information (one cab per upload), отправленную users, которые решили upload information в Microsoft. Один из самых критически важных files в cab - dump file, сгенерированный в point of failure. Этот dump file можно использовать при postmortem-debugging проблемы, как объяснялось ранее. Точный list of files в cab file подробнее обсуждается в section "Corporate Error Reporting" этой главы.

Теперь, когда мы рассмотрели различные pieces of information, доступные через WER Web site, от high-level overview events до более detailed drilldown с использованием information, которую customer

uploaded to Microsoft, мы обратим внимание на последний critical step в process - как предоставлять responses customers после того, как issue understood.

Предоставление responses

Чтобы предоставить response customers по particular event, нужно перейти на Event Details page (проиллюстрированную на рисунке 13.23). Если response для event еще не был recorded, самый верхний section page содержит options для регистрации response, как показано на рисунке 13.24.



Event Details for Event ID: 348048162 and Event Type: Crash32

No Response Registered!

You can register a response for users who encounter this event in the future. A response can be linked by this event, application or module.

Register a response for (Choose One): Read More!

☐ Event

☐ Application

☐ Module

Рисунок 13.24. Event response options

Responses можно регистрировать на трех разных levels:

- Event: обычно используется, когда fix очень изолирован и не будет включен в product update.
- Application: предоставление response на application level позволяет создать rules-based response, который увидят все users с particular version вашего application. Response может быть в форме update, например новой version.
- Module: предоставление response на module level позволяет создать rules-based response, который увидят ваши users с particular version вашего module. Response может быть в форме update, например новой version.

Для нашего конкретного scenario мы выберем event-based response registration. Выберите radio button Event и нажмите Register Response. Следующий шаг - заполнить details о event response. Следующая information required, прежде чем response может быть registered:

- Products: введите name of the product в поле Products.
- URL of Solution/Info: введите URL response. URL должен указывать на page со всей required information для этого particular response.
- Response Template: можно выбрать predefined template для response или использовать собственный custom template. Examples predefined templates включают System Does Not Meet Minimum Requirements, Product Upgrade, Upgrade to New Version и другие. В зависимости от выбранного template в drop-down preview field будет меняться.
- Response Template Preview: показывает preview того, какая information будет включена в response.
- Additional Information: введите любую additional information, которую хотите включить в response.

Когда вся information заполнена, продолжайте регистрацию response, и вы будете redirected на Response Management page, где перечислены все responses, которые вы registered. Обратите внимание, что newly registered response не вступает в effect немедленно, а проходит approval process, обработка которого занимает несколько days. Response Management page также позволяет manage все созданные responses. Можно просматривать responses in detail, вносить changes и удалять individual responses, которые больше не applicable.

Как этот response будет представлен user? В следующий раз, когда user столкнется с failure, с которым связан response, он увидит dialog, показанный на рисунке 13.25.

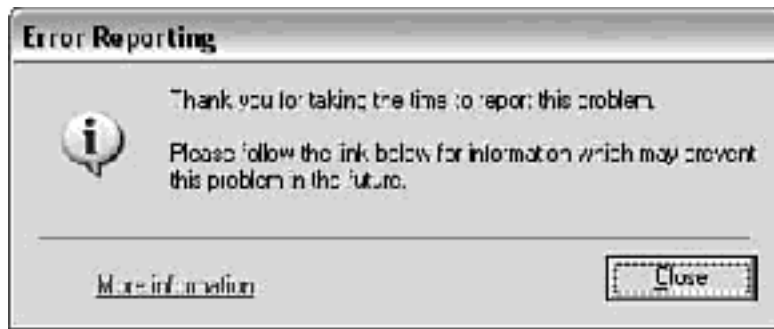


Рисунок 13.25. User experience при application failure, у которого есть response

Если user щелкнет link More Information, он будет переведен на Web page, содержащую response для particular failure с associated information о том, как resolve the problem. В нашем случае он routed на page, registered как часть response: <http://www.advancedwindowsdebugging.com>.

Как видите, WER - невероятно мощный service, который позволяет monitor, насколько хорошо ваше application behaves in the real world. Возможность для customers отправлять error information, которую вы можете analyze и для которой можете create response, - невероятная technology, облегчающая pain, через которую customers проходят при столкновении с software problems.

Corporate Error Reporting

Windows Error Reporting - отличная technology для оценки health вашего application в real world. Если когда-нибудь возникает problem, можно подготовить fix и доставить его на computer customer с помощью response system WER. Ключ WER - его способность отправлять error information, которая включает general fault information, а также mini dump file процесса, который crashed. Хотя способность перехватывать и отправлять эту error information в Microsoft является ключевым enabler WER, она также удерживает некоторых ISVs от использования системы. Более конкретно, некоторые ISVs не хотят, чтобы error information отправлялась в Microsoft, из-за sensitive nature данных, которые могут быть отправлены как часть error information. Один пример - banking application, содержащее sensitive information о customer accounts. Из-за строгих требований некоторых ISVs Microsoft создала то, что известно как Corporate Error Reporting (CER). CER позволяет ISV включить error reporting across the company и инструктирует каждую machine отправлять information на file share вместо отправки information в Microsoft. Затем file share можно periodically query, чтобы искать любые problems, которые могли surfaced.

В этой последней части главы мы рассмотрим, как настроить CER, а также как data uploaded and stored на file share.

Настройка Corporate Error Reporting

Group Policy (GP) - это technology в Windows, обеспечивающая centralized management computers and users в Active Directory environment. CER является частью group policy management capabilities. GP нужно использовать, чтобы включить CER либо locally (на одной machine), либо across all company machines. Все examples в этой части главы выполняются locally (то есть настройка CER на одной machine), но те же principles применяются при pushing down policy на все machines в company. Обратите внимание, что GP недоступна для Windows Home Edition, и при использовании GP нужна Windows XP Professional Edition или выше.

Чтобы включить CER на specific machine, нужно использовать Group Policy edit tool: перейдите в Start, Run и введите gpedit.msc. Это открывает GP Microsoft Management Console (MMC) snap-in, показанный на рисунке 13.26.

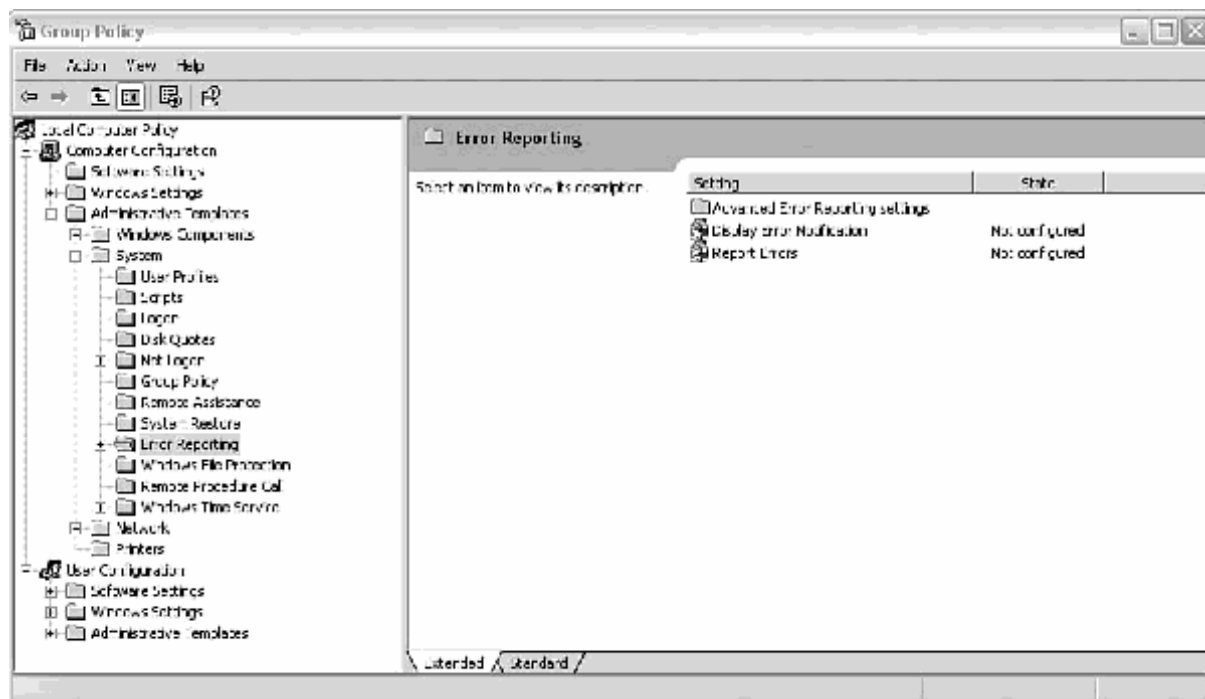


Рисунок 13.26. Group Policy settings for CER

В navigation pane слева доступ к CER settings выполняется через следующие nodes: Local Computer Policy, Computer Configuration, Administrative Templates, System, Error Reporting. После выбора можно видеть available options справа. Начнем с option Display Error Notification, показанной на рисунке 13.27. Убедитесь, что settings включены для каждого из configuration dialogs, как показано на следующих figures.

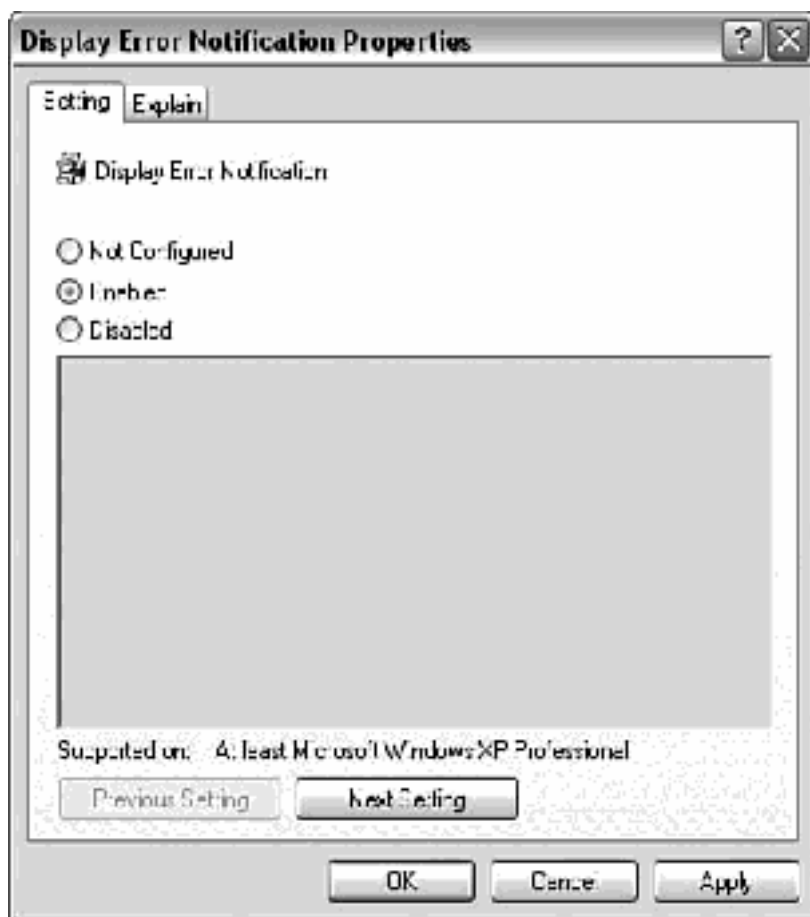


Рисунок 13.27. Display Error Notification settings

Option display error notification позволяет управлять тем, увидит ли user, столкнувшийся с error, UI, дающий ему выбор report the error. Если эта option включена, UI, появляющийся у user, дает user доступ к error information. Если, кроме того, configure error reporting включен, UI также позволит отправить error report.

Если display error notification выключена, user не будет показан UI. Это, однако, не означает, что error reports не отправляются. Будет ли отправлен error report, также зависит от setting configure error reporting.

Следующее setting - Configure Error Reporting, показанное на рисунке 13.28, и оно составляет основную часть CER configuration.

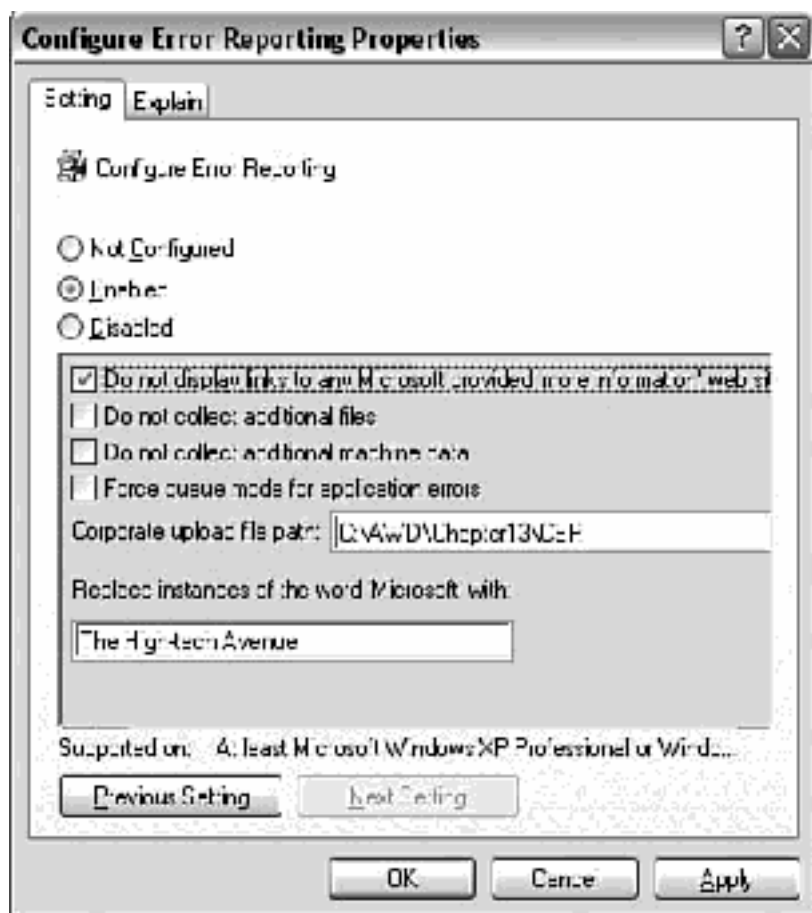


Рисунок 13.28. Configure Error Reporting settings

UI Configure Error Reporting settings позволяет управлять поведением error reporting при encounter errors. Highest-level settings - enable, disable или not configure this setting - можно выбирать через radio buttons. Если вы выбираете enable error reporting, следующие options становятся configurable:

- Do Not Display Links to Any Microsoft Provided "More Information" Web Sites: этот check box позволяет отключить Microsoft links в error UI, отображаемом user при error. Например, по умолчанию UI содержит link на Microsoft data collection policies.
- Do Not Collect Additional Files: если этот check box отмечен, error reports не будут collect additional information (files) вместе с error report.
- Do Not Collect Additional Machine Data: если этот check box отмечен, additional machine data не будет collected вместе с error report.
- Force Queue Mode for Application Errors: если этот check box отмечен, user не сможет immediately send error reports; вместо этого все error reports будут queued, чтобы при следующем logon administrator на machine был показан list of error reports, и administrator мог выбрать, какие error reports отправить.
- Corporate File Path: это field включает CER. Введите UNC path, который хотите использовать для CER. Все error reports будут stored в этом location, и administrator сможет выбрать, какие error reports send. Убедитесь, что все machines, reporting errors, имеют write access к этому location.
- Replace Instances of the Word "Microsoft" With: это позволяет customize error dialogs, отображаемые users, заменяя слово Microsoft чем-то другим. Например, если я введу The High-tech Avenue и столкнусь с crash, я увижу error dialog, показанный на рисунке 13.29. Как видите, sentence, которая раньше говорила "Please tell Microsoft about this problem", теперь заменена на "Please tell The High-tech Avenue about this problem."

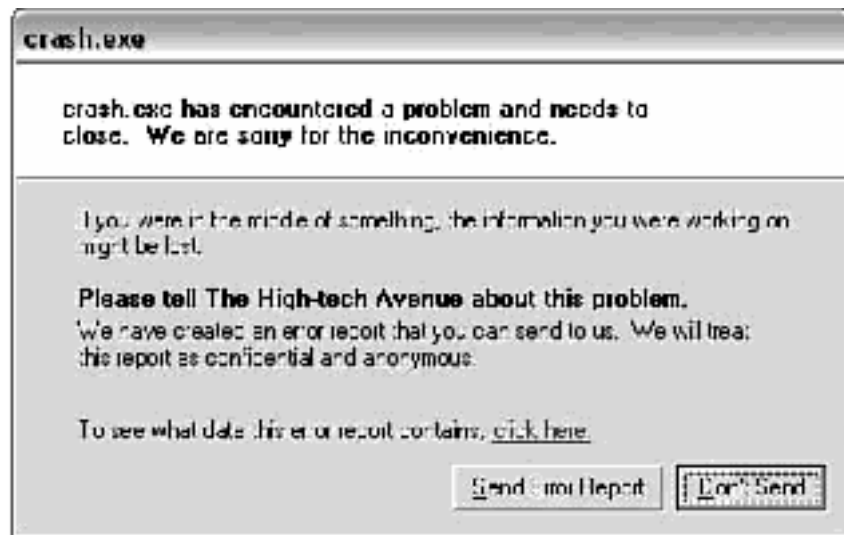


Рисунок 13.29. Example of customized error reporting UI

На рисунке 13.26 final CER configuration settings, которые мы еще не обсуждали, - это Advanced Error Reporting settings. Эти configuration settings summarized далее.

- Default Application Reporting: это setting управляет тем, какие applications участвуют в error reporting infrastructure. Например, можно настроить эту option так, чтобы все Windows applications были включены в error reporting или чтобы были включены только Windows components.
- List of Applications to Always Report Errors On: это setting позволяет управлять list of applications, которые всегда должны включаться в error reporting, независимо от того, что задает default application reporting.
- List of Applications to Never Report Errors On: это setting позволяет управлять list of applications, которые никогда не должны включаться в error reporting.
- Report Operating System Errors: это setting управляет тем, должны ли operating system components быть частью error reporting.
- Report Unplanned Shutdown Events: это setting управляет тем, должны ли unplanned shutdown events быть частью error reporting.

Теперь, когда мы configured and enabled CER через error reporting settings, пора посмотреть на practical example того, как error information для faulty application появляется в directory, который мы указали на рисунке 13.28. Помните, что мы запускаем faulty application на той же machine, на которой только что configured error reporting, чтобы simulate domain-enforced group policy.

Reporting errors с помощью Corporate Error Reporting

Теперь, когда мы successfully enabled CER, пора посмотреть, что actually happens, когда application crashes. Мы используем нашего надежного старого знакомого - приложение awdscenario1.exe, которое использовали в предыдущих частях главы. Запустите application на той же machine, которую мы использовали для configuration CER, по следующему path:

C:\AWDBIN\WinXP.x86.chk\awdscenario1.exe

Когда вам будет показан error UI Dr. Watson, нажмите кнопку Send. Далее происходит следующее: вместо отправки error report в global Windows Error Reporting service, error report будет stored in the path, specified при configuration CER. В нашем случае мы configured path:

C:\AWD\Chapter13\CER

Чтобы error information из multiple sources не перезаписывала друг друга, CER организует files под error reporting folder. При первой генерации error report в folder создаются две primary folders:

- Cabs: folder Cabs содержит actual error information, например dump files и associated error information.
- Counts: folder Counts содержит hit count для каждого fault. Hit count - чрезвычайно полезная информация, поскольку позволяет сосредоточить efforts на faults с highest hit count. File, stored in Counts folder, - это text file, содержащий два counts. Первый count показывает, сколько Cabs было gathered для этого fault, а второй count показывает, сколько hits было recorded для particular fault.

Под каждой из предыдущих folders создается еще одна nested folder hierarchy, uniquely identifying particular fault. Uniqueness исходит из следующих properties:

- Image Name: name of the image, вызвавшего fault. В нашем example созданная folder называлась бы awdscenario1.exe.
- Image Version: version number image, causing the fault. В нашем случае image version number - 1.1.0.0.
- Module Name: name of the module, causing the fault. В нашем example у нас нет separate module (скажем, DLL), поэтому выбранное name - просто image name.
- Module Version: version number module, causing the fault. В нашем случае image version совпадает с module version number 1.1.0.0.
- Offset: offset in the module, из-за которого fault occurred (обычно register eip). В нашем example offset был 0000127E.

Рисунок 13.30 иллюстрирует folder hierarchy после запуска binary awdscenario1.exe.

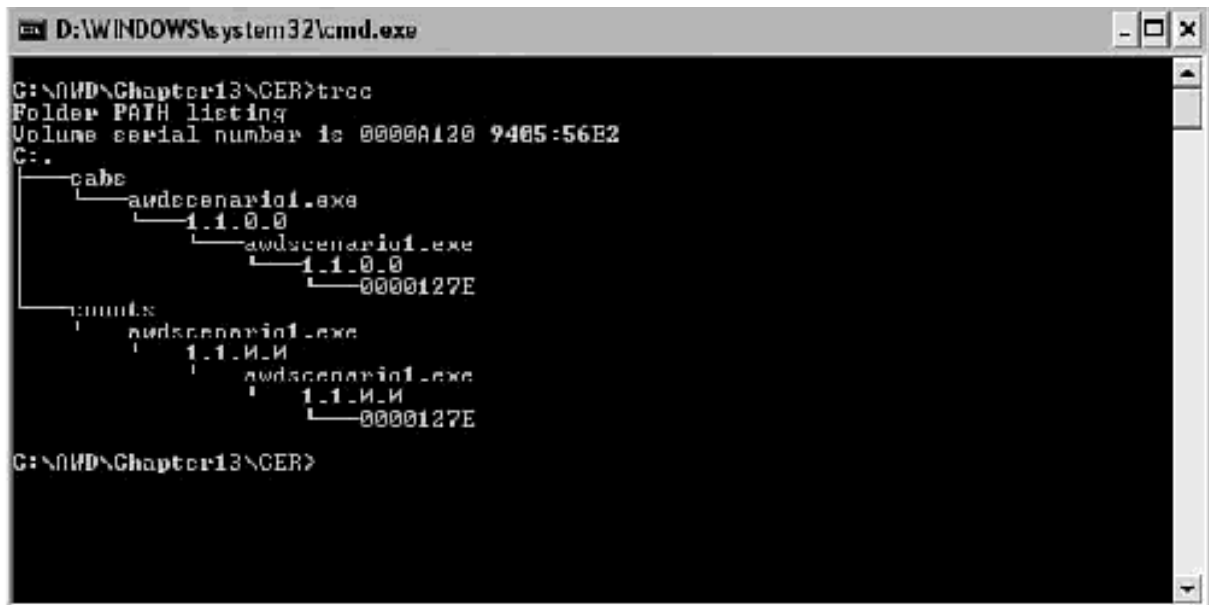


Рисунок 13.30. CER folder hierarchy after running awdscenario1.exe

В leaf folder любого данного fault's Cab directory находится compressed cab file. Name cab file выбирается random, чтобы multiple failures, originating from same faults, не overwrote each other's error data. Uncompressing cab file дает три files:

- Mini dump file: mini dump file представляет state of the process в момент, когда fault happened. Ранее в главе мы обсуждали, как использовать dump file, чтобы получить больше information о причинах fault.
- Version.txt: file Version.txt содержит version operating system, на которой выполнялся process.

- App compat file: app compat file - это XML file, содержащий list of attributes о process, который failed, например file description, check sum и другое.

Извлеките files из CAB file, сгенерированного для нашего sample run, и attach debugger к dump file.
(Не забудьте использовать switch -z.)

```
...
...
...
This dump file has an exception of interest stored in it.
The stored exception information can be accessed via .ecxr.
(f0f0f0f0.6fc): Access violation - code c0000005 (first/second chance not available)
eax=007b0000 ebx=0006ddc4 ecx=00001000 edx=7c90eb94 esi=00000000 edi=7ffdd000
eip=7c90eb94 esp=0006ddc4 ebp=0006de60 iopl=0         nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=0000  efl=00000246
Unable to load image D:\WINDOWS\system32\ntdll.dll, Win32 error 2
*** WARNING: Unable to verify timestamp for ntdll.dll
*** ERROR: Module load completed but symbols could not be loaded for ntdll.dll
ntdll+0xeb94:
7c90eb94 c3 ret
0:000> .ecxr
eax=00000000 ebx=7ffdd000 ecx=77c418bf edx=77c61b78 esi=01c709ef edi=a32cec54
eip=0100127e esp=0006ff1c ebp=0006ff44 iopl=0         nv up ei pl nz na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00010202
awdscenario1!wmain+0xbe:
0100127e c60000 mov byte ptr [eax],0 ds:0023:00000000=??
0:000> kb
*** Stack trace for last set context - .thread/.cxr resets it
ChildEBP RetAddr Args to Child
0006ff44 01001495 00000001 00263798 00264160 awdscenario1!wmain+0xbe
0006ffc0 7c816d4f a32cec54 01c709ef 7ffdd000 awdscenario1!wmainCRTStartup+0x12f
0006fff0 00000000 01001366 00000000 00000000 kernel32!BaseProcessStart+0x23
0:000> u awdscenario1!wmain+0xbe
awdscenario1!wmain+0xbe:
0100127e c60000 mov byte ptr [eax],0
01001281 eb0e jmp awdscenario1!wmain+0xd1 (01001291)
01001283 68a410001 push offset awdscenario1!`string' (010010a4)
01001288 ff154410001 call dword ptr [awdscenario1!_imp__printf (01001044)]
0100128e 83c404 add esp,4
01001291 688c10001 push offset awdscenario1!`string' (0100108c)
01001296 ff154410001 call dword ptr [awdscenario1!_imp__printf (01001044)]
0100129c 83c404 add esp,4
```

Как видите, debug session идентична предыдущим debug sessions на том же binary.

На этом завершается наше обсуждение Corporate Error Reporting. Как показано, CER - мощный mechanism, который можно включить через group policy. Он позволяет companies хранить все error reports locally для дальнейшего analysis или, как вариант, отправлять selected error reports в Microsoft, depending on configuration CER.

Summary

Postmortem debugging - критически важный аспект работы software engineer. После поставки application customers обычно очень трудно troubleshoot problems. Наличие knowledge и ability respond quickly, accurately и с минимальной pain для customer - ключ к тому, чтобы company могла effectively manage customer complaints.

В этой главе мы обсудили reasons, почему иногда нужно debug problem postmortem. Мы рассмотрели, какой type of debug information required, чтобы postmortem debugging работала, и какие tools можно использовать для сбора этой information. После того как information оказывается у нас, мы также обсудили, как debugger можно использовать для анализа debug information и нахождения source of the problem.

Был подробно рассмотрен powerful service Windows Error Reporting, который дает возможность monitor health вашего application in the real world и даже получать доступ к error information, например crash dumps, для каждой particular problem, с которой может сталкиваться ваше application, а также предоставлять response to the problem.

Также обсуждался Corporate Error Reporting, который позволяет collect error information и store it on a centralized file share перед отправкой error reports в Microsoft.

Глава 14. Мощные инструменты

На протяжении книги вы видели, как различные инструменты и отладчики работают вместе, облегчая процесс поиска и устранения неполадок в программном обеспечении. Мы обсудили широкий круг проблем: повреждения памяти, проблемы, связанные с безопасностью, утечки ресурсов, проблемы синхронизации и многое другое. Каждый раз, сталкиваясь с проблемой, мы выполняли базовый исследовательский анализ, прежде чем погрузиться глубже в подробный сеанс отладки. Чем больше вы отлаживаете определенные категории проблем, тем сильнее начальный процесс расследования начинает напоминать общий процесс. Разве не было бы удобно автоматизировать весь начальный анализ или его части и сосредоточить усилия на использовании результатов анализа для поиска первопричины? К счастью, существуют свободно доступные инструменты, которые позволяют делать именно это. В этой главе мы обсудим два таких мощных инструмента:

- Debug Diagnostics Tool: Debug Diagnostic Tool автоматизирует процесс анализа процессов, которые аварийно завершаются, зависают или дают утечки.
- Команда расширения !analyze: Debugging Tools for Windows поставляется с набором команд отладчика и команд расширений, помогающих в процессе поиска неполадок. Одна из этих команд называется analyze; она автоматизирует начальный исследовательский процесс.

Debug Diagnostic Tool

Debug Diagnostic Tool (также известный как DebugDiag) изначально был разработан как инструмент для анализа и отладки проблем Internet Information Server (IIS), таких как сбои, зависания и утечки памяти. Хотя инструмент позиционировался как ориентированный на IIS, он был спроектирован так, чтобы анализировать любой процесс, и быстро стал популярным как средство общего поиска неполадок. Сила DebugDiag проявляется в удобном пользовательском интерфейсе, прекрасных возможностях анализа (через сценарии анализа) и модели расширяемости, поддерживаемой крупной объектной моделью, которая упрощает написание собственных сценариев анализа.

Инструмент DebugDiag состоит из четырех основных компонентов.

- Service: процесс службы отладчика (dbgsvc.exe) - рабочая лошадка DebugDiag. В его обязанности входит управление отлаживаемыми процессами, мониторинг производительности, включение обнаружения утечек памяти и многое другое.
- Host: хост отладчика (dbgghost.exe) - отладочный движок DebugDiag. Он позволяет выполнять живую отладку процессов с помощью автоматизированных сценариев и посмертную отладку.
- User Interface: пользовательский интерфейс инструмента DebugDiag.
- Leak tracker: компонент leak tracker отвечает за всю работу, связанную с отслеживанием утечек памяти. Он реализован как DLL (leaktrack.dll) и внедряется в отслеживаемый процесс, чтобы отслеживать выделения памяти и связанные с ними стеки вызовов.

В DebugDiag существуют три основных сценария применения (также называемые правилами). Каждое правило сосредоточено на конкретной категории проблем.

- Crash: правило crash позволяет анализировать сбой процесса.
- Hang: правило hang позволяет анализировать зависший процесс IIS.
- Leaks: правило leaks позволяет анализировать утекшие ресурсы, такие как память и/или дескрипторы.

Инструмент (версия января 2006 года) можно найти по следующему адресу:

<http://www.microsoft.com/downloads/details.aspx?familyid=9BFA49BC-376B-4A54-95AA-73C9156706E7&displaylang=en>

Установка инструмента проста, и если вы не хотите выбирать конкретные устанавливаемые инструменты, достаточно типовой установки (она устанавливает все инструменты). DebugDiag можно запустить через Start, All Programs, IIS Diagnostics, Debug Diagnostics Tool, Debug Diagnostics Tool 1.0.

При первом запуске DebugDiag появляется мастер, позволяющий выбрать интересующее правило. Можно выбрать правило crash, hang или leaks. Если в этот момент вы не хотите выбирать правило, щелчок по Cancel приведет вас в главное окно DebugDiag. В главном окне по умолчанию выбрана вкладка Rules, и в нем отображается список активных правил (изначально пустой). Каждый раз, когда добавляется новое правило, оно также добавляется в список правил. Доступны еще две вкладки: Advanced Analysis и Processes. Вкладка Advanced Analysis содержит список доступных сценариев анализатора, а также список файлов данных (файлов дампов), над которыми можно выполнить сценарий анализа. Далее в этой главе вы подробнее увидите, как используется вкладка Advanced Analysis. Вкладка Processes просто перечисляет все процессы, выполняющиеся в системе, вместе с соответствующими атрибутами (такими как идентификатор процесса, имя процесса, идентичность процесса и многое другое).

Давайте рассмотрим одно из доступных правил совместно с ошибочным приложением и посмотрим, как DebugDiag может облегчить нашу жизнь при анализе приложения.

Анализ утечки памяти или дескрипторов

Чтобы показать, как DebugDiag можно использовать для анализа приложения с утечкой, мы воспользуемся сценарием из главы 9, "Утечки ресурсов".

Source code: C:\AWD\Chapter9\BasicMLeak\Client
Binary: C:\AWDBIN\WinXP.x86.chk\09BasicMLeak.exe

Аргументы командной строки, которые мы будем использовать для тестового приложения, следующие:

```
09BasicMLeak.exe /t:50 /i:2000 /s:0
```

Аргументы командной строки указывают приложению работать с 50 потоками, 2000 итерациями на поток и нулевым временем сна между итерациями. При запуске этого приложения вы быстро заметите, что потребление памяти растет, но не возвращается обратно вниз (даже когда приложение вот-вот завершится). Перезапустите приложение и, когда появится сообщение Press any key to start stress application, запустите DebugDiag. В мастере выберите переключатель Memory and Handle Leak и нажмите Next. Теперь мастер позволяет выбрать процесс, который нужно анализировать. Выберите процесс 09BasicMLeak.exe и нажмите Next. Следующая страница мастера позволяет настроить, как должно выполняться отслеживание утечек памяти и дескрипторов. Доступны следующие параметры.

- Warm-Up Time: этот параметр позволяет указать, когда должно начаться отслеживание памяти. По умолчанию отслеживание памяти начинается сразу после активации правила. Чтобы указать ограничение по времени (в минутах), после которого должно начаться отслеживание памяти, снимите флажок и задайте предел времени в поле Edit.
- Tracking Time: этот параметр позволяет управлять временным окном, в течение которого должно выполняться отслеживание. По умолчанию этот параметр установлен в 60 минут.
- Auto-Create Crash Rule: если этот флажок установлен, DebugDiag создает файл дампа при неожиданном завершении процесса.
- Auto-Unload: если этот флажок выбран, DebugDiag автоматически выгружает leaktrack.dll по завершении отслеживания памяти.

Для этого упражнения оставьте значения по умолчанию без изменений и нажмите кнопку Next. Последняя страница мастера позволяет назвать новое правило и указать расположение, где должны храниться файлы дампов. Назовем правило Stress и оставим расположение файлов дампов по умолчанию. Наконец, можно нажать Finish, чтобы сохранить и активировать правило. Главное окно правил теперь отображает два правила (правило leak и правило crash). Продолжите выполнение примерного приложения и дайте ему завершиться. Теперь можно вернуться в главное окно DebugDiag и нажать кнопку Analyze; откроется браузер с результатами отслеживания памяти. Отчет об отслеживании памяти разделен на два основных раздела: Analysis Summary и Analysis Details.

Раздел Analysis Summary содержит таблицу со сводкой проблем, найденных во время анализа. Он показывает сведения обо всех оставшихся выделениях, включая размер каждого выделения, а также модуль, ответственный за выделение. Для каждой найденной проблемы последний столбец таблицы также дает рекомендации по действиям, которые можно предпринять для решения проблемы. Рисунок 14.1 иллюстрирует Analysis Summary для исследуемого примерного запуска.

Analysis Summary		
Type	Description	Recommendation
Warning	09BasicMLeak.exe is responsible for 984.38 KBytes worth of outstanding allocations. The following are the top 2 memory consuming functions: 09BasicMLeak!CServer::GetSID+bd: 984.38 KBytes worth of outstanding allocations.	If this is unexpected, please contact the vendor of this module, Advanced Windows Debugging , for further assistance with this issue.
Warning	ntdll.dll is responsible for 446 Bytes worth of outstanding allocations. The following are the top 2 memory consuming functions: ntdll!LdrpAllocateDataTableEntry+33: 160 Bytes worth of outstanding allocations. ntdll!RtlpAllocateDebugInfo+49: 160 Bytes worth of outstanding allocations.	If this is unexpected, please contact the vendor of this module, Microsoft Corporation , for further assistance with this issue.
Information	DebugDiag did not detect any known native heap(unmanaged) problems in 09BasicMLeak_PID_3372_Date_05_30_2007_Time_05_27_19PM_kernel32! ExitProcess.dmp using the current set of scripts.	

Рисунок 14.1. Сводка анализа запуска 09BasicMLeak.exe

Как видно, 09BasicMLeak.exe отвечает примерно за 980 Кбайт утекшей памяти. Более конкретно, функция GetSID, похоже, является источником выделения. Одна строка также, по-видимому, подразумевает, что ntdll.dll отвечает за 446 байт оставшихся выделений. Обычно строки, показывающие системные DLL, не являются утечками памяти - скорее это выделения, которые еще не успели быть освобождены. Сводный раздел отчета может быть весьма полезен, когда нужно получить обзор различных компонентов процесса и соответствующей активности выделений.

Чтобы получить более подробную картину, обратитесь к разделу analysis details. Этот раздел содержит несколько подразделов, каждый из которых подробно описывает конкретные сведения, связанные с памятью.

- Если при завершении процесса был запрошен файл дампа, первый раздел содержит сводку информации из файла дампа.

- Virtual Memory Analysis: этот раздел содержит сведения об активности виртуальной памяти в анализируемом процессе. Раздел включает такие сведения, как детали виртуальной памяти, загруженные модули и потоки.
- Heap Analysis: раздел Heap Analysis содержит верхнеуровневый обзор использования кучи, а также подробную разбивку статистики кучи для каждой кучи.
- Leak Analysis: раздел Leak Analysis содержит результат анализа утечек, выполненного DebugDiag. Он показывает верхнеуровневый обзор активности памяти, а также активность отдельных куч с разбивкой по модулям.

Давайте посмотрим на раздел Leak Analysis примерного приложения, которое мы только что запустили. Первая интересующая часть - обзорный раздел, показанный на рисунке 14.2.

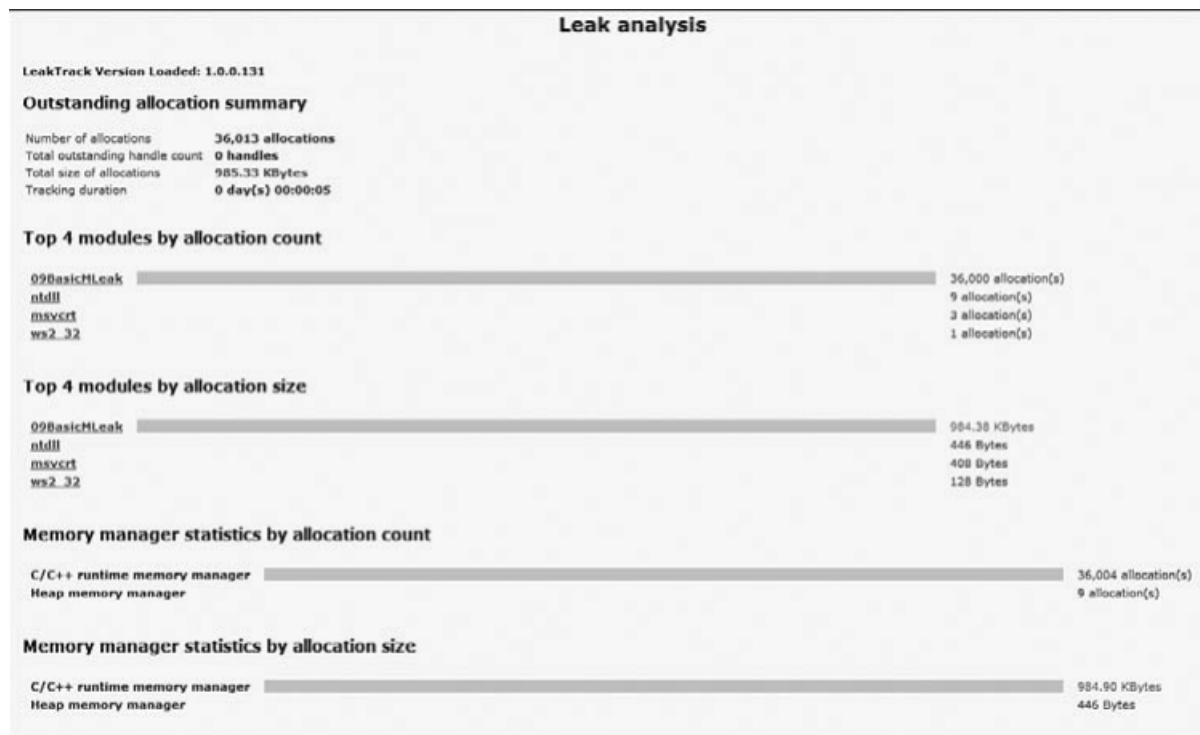


Рисунок 14.2. Обзор анализа утечек

Как видно из рисунка 14.2, 09BasicMLeak.exe - модуль, который является крупнейшим потребителем выделений: всего 36000 выделений и 984,38 Кбайт общей памяти. Эта информация позволяет быстро получить сведения, необходимые для сосредоточения усилий по поиску неполадки в правильном участке кода. За обзорным разделом следует подробное углубление в активность каждого модуля. Поскольку из обзорного раздела мы уже знаем, что 09BasicMLeak.exe выглядит подозрительно, начнем с просмотра раздела модуля 09BasicMLeak.exe, показанного на рисунке 14.3.

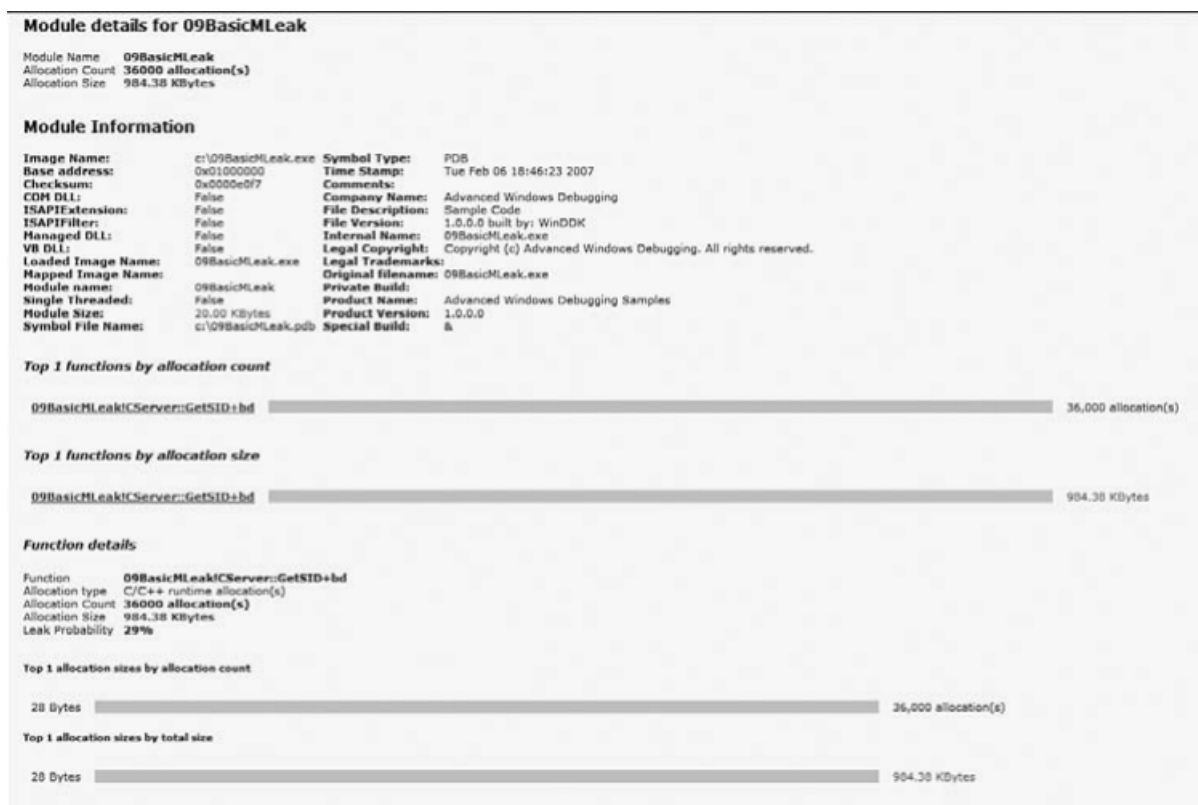


Рисунок 14.3. Информация о модуле 09BasicMLeak.exe

Раздел Module начинается со сводки информации о модуле, такой как имя образа, имя модуля, размер модуля и многое другое. После сводки модуля идет разбивка функций в модуле с наибольшей активностью выделений. В нашем случае видно, что функция GetSID отвечает как за наибольшее количество выделений, так и за наибольший размер выделений. Последняя часть информации - раздел Function Details, который подробно описывает функции с наибольшим числом выделений и дает разбивку разных выделений, сделанных в функции. Например, рисунок 14.3 показывает, что функция GetSID отвечала за 36000 выделений размером 28 байт каждое.

Как вы видели, DebugDiag предлагает действительно впечатляющие возможности анализа, избавляющие разработчика от необходимости вручную выполнять дорогостоящие отладочные шаги. Кроме того, DebugDiag показывает результат анализа в форме, которую легко читать и воспринимать. Доступны еще два правила: правила анализа crash и hang. Мы настоятельно рекомендуем изучить эти правила, поскольку они предоставляют информацию, похожую на информацию правил утечек памяти и дескрипторов. Оба правила поддерживаются мощными сценариями анализа. Однако большой вопрос в том, можно ли расширить этот инструмент и разработать новые собственные сценарии анализа для сценариев, которые не покрывают существующие сценарии? Ответ - да, и для этого используется модель расширяемости инструмента, к которой мы сейчас перейдем.

Написание собственных сценариев анализа

Помимо предоставления интересных и мощных сценариев анализа из коробки, DebugDiag также открывает мощный доступ к сценариям с богатой объектной моделью, позволяющей разработчикам писать собственные сценарии анализа на любимом языке сценариев (например, VBScript или JScript). Объектная модель очень обширна и охватывает подавляющее большинство отладочных объектов, которые могут понадобиться разработчику. Более подробные сведения о полной объектной модели см. в документации DebugDiag.

В этой части главы мы покажем, как возможности сценариев можно использовать для написания собственного сценария анализа, который выводит все заблокированные критические секции в файле дампа.

Каждый сценарий начинается с раздела метаданных, который сообщает движку анализа используемый язык сценария и его категорию, а также описание. Раздел метаданных очень похож на заголовок ASP, как показано в листинге 14.1.

Листинг 14.1

```
<%@ Language = VBScript %>
<%@ Category = Sample analysis script %>
<%@ Description = Example of a custom analysis script %>
<% script code goes here %>
```

Каждый элемент метаданных (за исключением элемента Language) представляет информацию, которая будет отображаться на вкладке Advanced Analysis в пользовательском интерфейсе DebugDiag.

Цель нашего примерного сценария - вывести все заблокированные критические секции в файле дампа. Идеальный объект для этой цели - объект CritSecInfo, который предоставляет коллекцию критических секций со всеми заблокированными критическими секциями в файле дампа. Объект CritSecInfo имеет свойство Count, а также свойство Item (среди прочих), позволяющие пройти по всем заблокированным критическим секциям. В листинге 14.2 показан код, необходимый для доступа к критическим секциям.

Листинг 14.2

```
Set DataFiles = Manager.DataFiles
For each DataFile in DataFiles
Manager.Write "<B>Analyzing dump file " & DataFile & "</B><BR>"
Set Debugger = Manager.GetDebugger(DataFile)
Set CritSecInfo = Debugger.CritSecInfo
CritSecCount = CritSecInfo.Count
if CritSecCount = 0 Then
Manager.Write "No locked critical sections found<BR>"
else
For i = 0 to CritSecCount-1
Set DbgCritSec = CritSecInfo.Item(i)
Manager.Write "The owner of critsec: " & DbgCritSec.Address & " is thread: "
& DbgCritSec.OwnerThreadID & "<BR>"
Next
End If
Set Debugger = Nothing
Next
```

Все сценарии анализа DebugDiag изначально имеют доступ к объекту Manager. Объект Manager позволяет сценарию, например, сообщать результаты анализа, а также получать доступ к файлам данных (то есть файлам дампов), которые нужно анализировать. Первое, что нужно сделать, - получить доступ ко всем файлам дампов, которые пользователь выбрал для запуска сценария. Свойство DumpFiles объекта Manager возвращает коллекцию выбранных пользователем файлов дампов. Поскольку мы хотим анализировать каждый файл дампа, указанный пользователем, далее мы проходим по возвращенной коллекции файлов дампов и получаем объект Debugger для каждого файла дампа в коллекции. Объект Debugger позволяет сценарию получить доступ к множеству сведений о файле дампа. (Список свойств см. в документации DebugDiag.) В нашем конкретном случае нас интересуют критические секции файла дампа, и поэтому мы используем свойство CritSecInfo объекта Debugger. Свойство CritSecInfo возвращает экземпляр объекта CritSecInfo, который далее можно использовать для поиска всех критических секций, загруженных для данного файла дампа. Объект CritSecInfo имеет свойство Count, возвращающее количество заблокированных критических секций в файле дампа. После получения количества можно использовать свойство Item, чтобы получить доступ к каждой заблокированной

критической секции. Свойство `Item` возвращает экземпляр объекта `DbgCritSec`, который содержит свойства критической секции, такие как адрес критической секции, идентификатор владеющего потока, счетчик вращения, состояние и многое другое. В нашем примерном сценарии в листинге 14.2 мы использовали свойства `Address` и `OwnerThreadID`, чтобы напечатать адрес и владельца критической секции.

Как вы заметили в листинге 14.2, метод `Write` объекта `Manager` позволяет записывать данные в отчет, подготавливаемый для пользователя. Поскольку отчеты отображаются с помощью браузера, мы включаем правильные HTML-теги, чтобы информация была правильно отформатирована (то есть с использованием горизонтальных разрывов строк).

После написания сценария сохраните его с расширением `.asp` и поместите в папку `Scripts` пути установки `DebugDiag`. Например, если диск установки - `C:\`, поместите файл сценария в следующий каталог:

```
C:\Program Files\IIS Resources\DebugDiag\Scripts
```

После сохранения сценария можно запустить `DebugDiag`, щелкнуть вкладку `Advanced Analysis` и выбрать новый мощный сценарий анализа, который только что был создан. Обратите внимание, что можно выбрать несколько файлов дампов, и новый сценарий анализа критических секций по очереди проанализирует каждый из выбранных файлов.

Возможности `DebugDiag` по написанию собственных сценариев и богатая объектная модель позволяют инженерам создавать сложные и мощные сценарии посмертного анализа, которые могут резко сократить время, затрачиваемое на анализ проблемы. Без этого инструмента разработчику пришлось бы либо вручную выполнять все шаги, которые выполняет сценарий (в каждом сеансе отладки), либо, как вариант, написать собственное расширение отладчика, выполняющее ту же работу, что и сценарий. Нет нужды говорить, что в главе 11, "Написание собственных расширений отладчика", вы уже видели: написание собственных расширений отладки - несколько более сложное занятие. Важно отметить, что хотя объектная модель, предоставляемая `DebugDiag`, довольно полна, она не охватывает 100% возможностей, и иногда необходимо написать расширение отладки, которое затем можно вызвать из сценария `DebugDiag`.

Команда расширения `!analyze`

Команда расширения `analyze` была изобретена как способ автоматизации анализа сбоев с конечной целью автоматического анализа сбоев, обнаружения и назначения известных проблем. Со временем широта и область применения команды расширения `!analyze` значительно выросли, и теперь она способна выполнять автоматический анализ большого диапазона трудных проблем. Примеры таких проблем включают повреждение стека, сбои `Application Verifier` и многое другое. Одна из замечательных возможностей команды расширения `!analyze` - способность назначать сбои на основе результатов анализа. Например, можно сказать команде расширения `!analyze`, что любые сбои, происходящие внутри конкретного модуля, должны назначаться определенному владельцу. Это может быть весьма удобным механизмом, когда сбои автоматически анализируются с помощью команды расширения `!analyze`, и он может сократить объем ручного времени, затрачиваемого на начальный анализ и назначение сбоев. В этой части главы мы посмотрим, как команду расширения `!analyze` можно использовать для анализа сбоя в одном из приложений, использованных в главе 10, "Синхронизация". Кроме того, мы покажем, как механизм автоматического назначения сбоев может помочь сократить время, затрачиваемое на анализ и последующий процесс.

Ошибочное приложение

В главе 10 мы потратили некоторое время на исследование базовой взаимной блокировки. Начальный процесс анализа включал вывод всех потоков и, для каждого из них, поиск любого поведения блокировок, которое могло привести к возникновению deadlock. После выявления потенциально виновных потоков выполнялось углубленное исследование каждой конструкции блокировки, чтобы выяснить, есть ли между потоками связь, которая могла вызвать deadlock. В этом разделе мы посмотрим на тот же сценарий deadlock и увидим, как команда расширения `!analyze` помогает повысить эффективность начального анализа. Исследуемый двоичный файл находится в следующем месте:

```
Source code: C:\AWD\Chapter10\Deadlock
Binary: C:\AWDBIN\winXP.x86.chk\10DeadLock.exe
```

Чтобы начать процесс автоматического анализа, запустите `10DeadLock.exe` под отладчиком и дайте ему выполняться, пока не возникнет deadlock. Когда приложение заблокируется, прервитесь в отладчик и выведите все потоки. Далее нужно попытаться определить поток, который, по нашему мнению, завис, и переключить текущий поток на зависший поток. В нашем сценарии поток 0 может быть потенциальным виновником (это видно по попытке войти в критическую секцию). После переключения на потенциально виновный поток выполните команду расширения `!analyze`, как показано здесь:

```
!analyze -v -hang
```

Ключ `-v` заставляет команду расширения `!analyze` выводить результаты в подробном режиме, а ключ `-hang` сообщает команде расширения `!analyze`, что она должна выполнить анализ зависания. Результат выполнения команды - большой объем информации, которую команда смогла экстраполировать на основе общего состояния процесса. Давайте подробнее рассмотрим вывод команды.

Результаты анализа

Листинг 14.3 показывает вывод команды `analyze -hang -v`, которую мы только что выполнили.

Листинг 14.3

```
0:000> !analyze -hang -v
*****
* *
* Exception Analysis *
* *
*****
FAULTING_IP:
kernel32!CtrlRoutine+bd
7c87533d 834dfcff or dword ptr [ebp-4],0FFFFFFFFh
EXCEPTION_RECORD: ffffffff -- (.exr ffffffff)
ExceptionAddress: 7c87533d (kernel32!CtrlRoutine+0x00000bd)
ExceptionCode: 40010005 (Control-C exception)
ExceptionFlags: 00000000
NumberParameters: 0
FAULTING_THREAD: 00000d08
BUGCHECK_STR: HANG
PROCESS_NAME: 10DeadLock.exe
ERROR_CODE: (NTSTATUS) 0xcffffff - <Unable to get error code text>
CRITICAL_SECTION: 01003320 (!cs -s 01003320)
BLOCKING_THREAD: 00000d08
DERIVED_WAIT_CHAIN:
Dl Eid Cid WaitType
-----
x 0 85c.d08 Critical Section ->
x 1 85c.8f4 Critical Section -^
WAIT_CHAIN_COMMAND: ~0s;k;;~1s;k;;
```

```

DEFAULT_BUCKET_ID: APPLICATION_HANG_DEADLOCK
PRIMARY_PROBLEM_CLASS: APPLICATION_HANG_DEADLOCK
LAST_CONTROL_TRANSFER: from 7c90e9c0 to 7c90eb94
STACK_TEXT:
0007fed4 7c90e9c0 7c91901b 000007dc 00000000 ntdll!KiFastSystemCallRet
0007fed8 7c91901b 000007dc 00000000 7c97c140 ntdll!NtWaitForSingleObject+0xc
0007ff60 7c90104b 00003320 0100137d 01003320 ntdll!RtlpWaitForCriticalSection+0x132
0007ff68 0100137d 01003320 000007f4 000008f4 ntdll!RtlEnterCriticalSection+0x46
0007ff7c 0100153b 00000001 00032470 00032ce0 10DeadLock!main+0xad
0007ffc0 7c816fd7 00011970 7c9118f1 7ffd6000 10DeadLock!__mainCRTStartup+0x102
0007fff0 00000000 01001679 00000000 78746341 kernel32!BaseProcessStart+0x23
FOLLOWUP_IP:
10DeadLock!main+ad
0100137d 6808110001 push offset 10DeadLock!`string' (01001108)
SYMBOL_STACK_INDEX: 4
FOLLOWUP_NAME: MachineOwner
MODULE_NAME: 10DeadLock
IMAGE_NAME: 10DeadLock.exe
DEBUG_FLR_IMAGE_TIMESTAMP: 45c93d82
SYMBOL_NAME: 10DeadLock!main+ad
STACK_COMMAND: ~0s ; kb
FAILURE_BUCKET_ID: HANG_10DeadLock!main+ad
BUCKET_ID: HANG_10DeadLock!main+ad
Followup: MachineOwner
-----

```

Разбор и подробное описание вывода приведены в следующем списке.

- **FAULTING_IP**

FAULTING_IP показывает значение указателя инструкции в момент возникновения сбоя. В нашем случае указатель инструкции, считающийся сбойным, связан с тем, что мы прервались в отладчик, когда был обнаружен deadlock (`kernel32!CtrlRoutine`).

- **EXCEPTION_RECORD**

EXCEPTION_RECORD дает более подробные сведения о произошедшем исключении в виде записи исключения. Из листинга 14.3 видно, что у нас нет записи исключения, сохраненной для этого сбоя (ffffffff). Если бы запись исключения была доступна, можно было бы использовать команду `exr` для отображения ее содержимого.

- **ExceptionAddress**

ExceptionAddress показывает адрес, где произошло исключение. В нашем сценарии адрес исключения - `7c87533d`, что соответствует функции `control routine` в `kernel32.dll` (`kernel32!CtrlRoutine`).

- **ExceptionCode**

ExceptionCode сообщает точное исключение, вызвавшее сбой. В нашем сценарии код исключения - `40010005`, что соответствует исключению `Control-C`. Этот адрес исключения записан из-за прерывания в отладчик с помощью `Control-C`.

- **ExceptionFlags**

ExceptionFlags показывает флаги, связанные с исключением. В нашем случае флаг установлен в `00000000`.

- **NumberParameters**

NumberParameters сообщает количество параметров, связанных с исключением. В нашем случае количество параметров равно 0.

- **FAULTING_THREAD**

FAULTING_THREAD сообщает, какой поток вызвал возникновение сбоя. Из вывода в листинге 14.3 видно, что сбойный поток - `00000d08`. Если найти поток `00000d08` в списке потоков, можно увидеть,

что сбойный поток соответствует одному из потоков, ожидающих доступности критической секции.

- BUGCHECK_STR

BUGCHECK_STR представляет текстовое описание произошедшего "bugcheck". Обратите внимание, что имя этого поля может вводить в заблуждение, поскольку мы находимся в пользовательском режиме, а bugcheck обычно относится только к сбоям режима ядра. Несмотря на то что различие между пользовательским режимом и режимом ядра не отражено в имени этого поля, причине сбоя все равно можно доверять. В нашем сценарии причина сбоя - HANG.

- PROCESS_NAME

PROCESS_NAME сообщает имя процесса, в котором проявляется сбой (10DeadLock.exe).

- ERROR_CODE

ERROR_CODE сообщает код ошибки (NTSTATUS), вызвавший сбой.

- CRITICAL_SECTION

CRITICAL_SECTION показывает адрес критической секции, найденной и проанализированной в начальном потоке. Поскольку нашим начальным потоком был 0, показанный адрес критической секции - 01003320.

- BLOCKING_THREAD

BLOCKING_THREAD дает ID потока, который блокируется на критической секции, показанной в поле CRITICAL_SECTION. В нашем случае поток, блокирующийся на критической секции по адресу 01003320, - 00000d08.

- DERIVED_WAIT_CHAIN

DERIVED_WAIT_CHAIN находится в центре возможностей анализа зависимостей команды расширения !analyze. Когда команда была запущена, она попыталась вывести, как выглядит цепочка ожидания для каждого из потоков процесса, связанных с критической секцией, указанной в поле CRITICAL_SECTION. Давайте подробнее посмотрим на вывод.

```
Dl Eid Cid WaitType
- - - - -
x 0 85c.d08 Critical Section -->
x 1 85c.8f4 Critical Section --^
```

Из этого вывода видно, что у нас есть два потока, связанные с критической секцией. Самый интересный вопрос - каково состояние этой критической секции (то есть заблокирована ли она и какой поток ею владеет). В конце каждой отображенной строки потока находится серия символов, обозначающих состояние критической секции по отношению к потоку. Символы включают:

- -->: поток ожидает критическую секцию;
- --^: поток владеет критической секцией.

Из этого вывода можно очень быстро увидеть, что поток с ID d08 ожидает критическую секцию, которой владеет поток 8f4.

- WAIT_CHAIN_COMMAND

Это поле показывает команды, которые можно использовать для дальнейшего анализа цепочки ожидания проблемной блокировки. В нашем сценарии команды для выполнения:

```
~0s;k;;~1s;k;;
```

- DEFAULT_BUCKET_ID

DEFAULT_BUCKET_ID подробно указывает общую категорию сбоев, к которой относится данный конкретный сбой. В нашем сценарии сбой был определен как APPLICATION_HANG_DEADLOCK.

- PRIMARY_PROBLEM_CLASS

PRIMARY_PROBLEM_CLASS указывает основной класс проблем, в который был отнесен сбой. И снова основной класс проблемы для нашего сценария - APPLICATION_HANG_DEADLOCK.

- LAST_CONTROL_TRANSFER

LAST_CONTROL_TRANSFER показывает два последних вызова функций, выполненных в стеке. Листинг 14.3 показывает следующие адреса:

```
from 7c90e9c0 to 7c90eb94
```

Если использовать команду `ln` для этих адресов, можно увидеть, что последний вызов функции в нашем сценарии был из `ntdll!NtWaitForSingleObject` в `ntdll!KiFastSystemCallRet`.

- STACK_TEXT

STACK_TEXT показывает полную трассировку стека потока. В нашем случае она показывает трассировку стека потока 0.

- FOLLOWUP_IP

FOLLOWUP_IP показывает инструкцию, которая, скорее всего, вызвала сбой. Из вывода в листинге 14.3 видно, что вызов `EnterCriticalSection` вызвал проблемное зависание в коде.

- SYMBOL_STACK_INDEX

SYMBOL_STACK_INDEX соответствует кадру в трассировке стека потока, который вызвал сбой. В нашем сценарии `symbol-stack-index` равен 4, что соответствует кадру стека, вызвавшему сбой:

```
10DeadLock!main
```

- FOLLOWUP_NAME

FOLLOWUP_NAME показывает, кто должен заниматься этим конкретным сбоем. Как видно, последующим ответственным в нашем сценарии указан `MachineOwner`, что обычно используется, когда для конкретного сбоя не найдена директива follow-up. Позже в этой главе вы увидите, как можно управлять содержимым этого поля.

- MODULE_NAME

Это поле просто указывает модуль, в котором проявляется сбой (`10DeadLock`).

- IMAGE_NAME

IMAGE_NAME указывает имя образа, в котором произошел сбой (`10DeadLock.exe`).

- DEBUG_FLR_IMAGE_TIMESTAMP

Это поле показывает временную метку образа, в котором произошел сбой (`45c93d82`).

- SYMBOL_NAME

Это поле показывает символическое имя места, где произошел сбой (`10DeadLock!main`).

- STACK_COMMAND

STACK_COMMAND показывает команды, которые были выполнены для получения трассировки стека сбойного потока. В нашем случае последовательность команд была:

```
~0s ; kb
```

- FAILURE_BUCKET_ID и BUCKET_ID

Поля `bucket ID` показывают конкретную категорию проблем, к которой относится сбой. Эта информация может использоваться командой расширения `!analyze`, чтобы определить, какие дополнительные сведения отображать в результате анализа.

- FOLLOWUP

FOLLOWUP показывает наиболее подходящих владельцев для данного конкретного сбоя. Если владелец не найден, отображается значение по умолчанию MachineOwner.

Как вы видели, выполнение команды расширения !analyze дает много информации, на ручной сбор которой в противном случае могло бы уйти немало времени. Помимо всей этой ценной информации, еще одна важная возможность команды расширения !analyze - способность назначать владельцев конкретным сбоям. Далее мы посмотрим, как настроить эту возможность.

Последующее назначение сбоя

В предыдущем разделе мы рассмотрели результаты запуска команды расширения !analyze для приложения, в котором возник deadlock. Часть информации, полученной от команды расширения !analyze, представляла собой контакт follow-up для данного конкретного сбоя. Поскольку analyze не знал, кому мы хотим назначить именно эту проблему, он просто использовал значение по умолчанию MachineOwner. Теперь посмотрим, как настроить это поведение под собственные нужды.

Для этого сценария предположим, что владельца приложения 10DeadLock.exe зовут John Anderson (e-mail: johna@fictionous.com). Мы хотим связать имя John с любыми сбоями, происходящими в образе 10DeadLock.exe. Такая связь задается через файл triage.ini, расположенный в папке triage пути установки отладчика. По умолчанию файл triage.ini содержит множество сопоставлений, которые либо игнорируют указанные сбои, либо назначают сбои владельцу по умолчанию (например, MachineOwner). Давайте посмотрим пример из файла triage.ini:

```
nt!Zw*=ignore
```

Эта строка показывает, что любые сбои, происходящие в модуле nt и начинающиеся с букв Zw, должны игнорироваться, и команда расширения !analyze не будет отображать владельца follow-up. Как видно, файл triage.ini поддерживает указание подстановочных знаков, чтобы упростить назначение диапазона проблем конкретной сущности.

Добавим в файл triage.ini строку, связывающую John Anderson как владельца 10DeadLock.exe:

```
10DeadLock!*=John Anderson (johna@fictionous.com)
```

Используя только подстановочный символ *, мы можем сообщить команде расширения !analyze, что все сбои, происходящие в модуле scenario1, нужно назначать John. Давайте снова запустим приложение под отладчиком и выполним анализ зависания, чтобы убедиться, что сбой действительно назначается John. Листинг 14.4 показывает сокращенную версию результата команды расширения !analyze.

Листинг 14.4

```
0:000> !analyze -v -hang
*****
* *
* Exception Analysis *
* *
*****
FAULTING_IP:
kernel32!CtrlRoutine+bd
7c87533d 834dfcff or dword ptr [ebp-4],0FFFFFFFFh
...
...
...
FOLLOWUP_NAME: John Anderson (johna@fictionous.com)
...
.
..
```

...
Followup: John Anderson (johna@fictionous.com)

Как видно, follow-up теперь изменился с MachineOwner на John Anderson, что облегчает человеку, просматривающему анализ, назначение сбоя правильному владельцу.

Возможность указать владельца невероятно полезна, когда у вас есть сложная система, в которой большое количество разработчиков работает со множеством разных образов и модулей. Во время тестирования, всякий раз когда где-то в системе возникает сбой, команду расширения `!analyze` можно использовать, чтобы извлечь информацию об источнике проблемы, а также сразу определить, к кому обращаться по поводу этого сбоя. Можно даже представить полностью автоматизированный процесс уведомления, который автоматически разбирает вывод команды расширения `!analyze` и отправляет электронное письмо follow-up, указанному в результате.

Резюме

В этой главе мы исследовали два мощных инструмента, которые могут резко сократить объем времени, затрачиваемого на начальный анализ сбоев. Инструмент Debug Diagnostics представил механизм, с помощью которого можно автоматизировать обнаружение утечек памяти, анализ сбоев и анализ зависаний, а также писать собственные сценарии анализа. Результат анализа состоит из хорошо отформатированного и легко воспринимаемого отчета, в котором вся релевантная информация категоризирована и отображена как в текстовой, так и в графической форме.

Другой инструмент, рассмотренный в этой главе, представлен командой расширения под названием `!analyze`. Команда расширения `!analyze` обладает сильными аналитическими возможностями. Со временем она созрела до способности анализировать большое количество трудных проблем и выполнять автоматическое назначение обнаруженных проблем.

Глава 15. Основы Windows Vista

Большинство приложений без дополнительных усилий выигрывают от всех улучшений Windows Vista. Большинство заказчиков, использующих эти приложения, рады улучшениям и стремятся обновить все компьютеры до Windows Vista. Однако некоторые приложения, запущенные под Windows Vista, аварийно завершаются через случайные промежутки времени, и заказчик требует новую версию. Поставщик должен исследовать первопричину проблемы, найти решение и предоставить Vista-обновление для своего приложения.

В предыдущих версиях Windows расследование аварийного завершения можно было выполнить за разумное время. В Windows Vista это может занять значительно больше времени. Почему так? Хотя большинство приложений отлично работает в новой среде, улучшения безопасности и надежности Windows Vista влияют на их функциональность. Система стала более устойчивой ценой совместимости приложений. Операционная система Windows Vista менее терпима к ошибкам памяти приложений и накладывает более строгие ограничения безопасности на системные ресурсы. Несколько знакомых системных механизмов, переработанных для Windows Vista, требуют времени на освоение.

Эта глава проходит по всем сценариям, описанным в книге, - сценариям, которые мы считаем часто используемыми при отладке приложений Windows, - с целью выяснить, что изменилось в Windows Vista. Цель главы не в том, чтобы стать исчерпывающим списком изменений Vista; она ограничена изменениями, имеющими прямые последствия для процесса отладки.

Следующие разделы, каждый из которых соответствует предыдущей главе, идут в той же последовательности, что и главы книги. Каждый подзаголовок примерно следует организации соответствующей главы и сосредоточен только на отличиях от предыдущих версий операционной системы. Главы, не представленные здесь, не имеют существенных отличий от Windows XP Service Pack 2, и можно считать, что ничего специфичного не изменилось. Однако возможно увидеть слегка иное поведение, вызванное улучшениями безопасности.

В этой главе повторно используются несколько примеров, представленных в предыдущих главах. Двоичные файлы, нацеленные на Windows Vista x64, доступны в папке C:\AWDBIN\winLH.AMD64.chk. Двоичные файлы, нацеленные на Windows XP, доступны в папке C:\AWDBIN\winXP.x86.chk.

По материалам главы 1: Введение в инструменты

Каждый раз, когда в базовую архитектуру любой операционной системы вносятся существенные изменения, совместимость приложений становится проблемой. Приложения, которые раньше работали, теперь могут потребовать изменений. Хотя Microsoft тратит невероятное количество времени и ресурсов на то, чтобы приложения продолжали работать с каждой версией операционной системы, иногда достичь стопроцентной совместимости невозможно или нежелательно. Инструменты, используемые на протяжении этой книги, не являются исключением.

Хотя большинство инструментов работает, некоторые пока не работают в Windows Vista. Таблица 15.1 показывает все инструменты, использованные в книге, вместе с их состоянием совместимости с Vista.

Таблица 15.1. Совместимость инструментов с Windows Vista

Инструмент	Совместим с Vista	Альтернативный инструмент
WDK	Да	
LeakDiag	Нет	UMDH
UMDH	Да	
Application Verifier	Да	
Global Flags	Да	
Process Monitor	Да	
Ethereal	Да	
DebugDiag	Нет	Нет
Debugging Tools for Windows	Да	

Для каждого инструмента, помеченного в таблице 15.1 как неработающий, столбец "Альтернативный инструмент" указывает предложение другого инструмента, который можно использовать вместо него. Улучшения безопасности Windows Vista влияют на все инструменты, и возможное изменение поведения должно быть хорошо понятно, как показано на примере в разделе "Виртуализация реестра и файлов".

Журналы отладки Windows

Windows Vista приносит огромные улучшения в область управляемости, особенно в управление событиями, создаваемыми различными компонентами, которые можно использовать при отладке приложений. Новая система Event Log является центральным хранилищем для всех журналов компонентов, известных как diagnostic logs или debug logs. Журналы отладки отображаются в Event Viewer после изменения настроек журналов с помощью пункта меню Show Analytic and Debug в меню View. Несколько каналов по умолчанию отключены и могут быть включены с помощью правой панели.

Даже если некоторые каналы содержат информацию, полезную только разработчикам компонентов, большинство из них содержит сведения, полезные для интуитивного понимания состояния машины. Один такой журнал - Diagnostic-Performance Operational, содержащий ясные сообщения о различных измерениях производительности, как показано на рисунке 15.1.

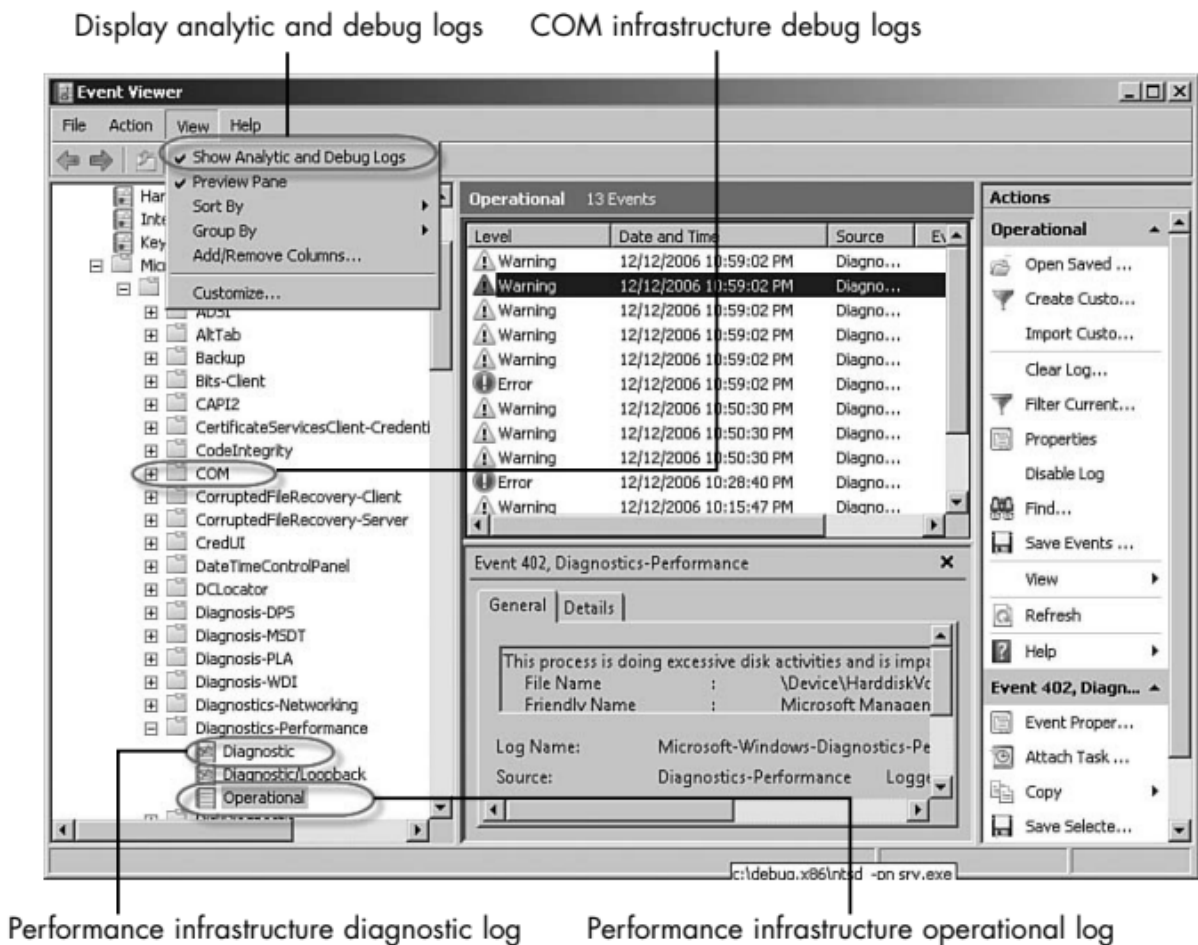


Figure 15.1 Operational, analytic, and debug logs stored in Windows Vista Event Log

Рисунок 15.1. Операционные, аналитические и отладочные журналы, сохраненные в Windows Vista Event Log

По материалам главы 2: Введение в отладчики

Перед тем как решить, нужен ли этот раздел, мы попробовали сценарии, описанные в главе 2, "Introduction to the Debuggers", во всех возможных конфигурациях. Немногие проблемы, встреченные при установлении сеанса отладки, обсуждаются в этом разделе.

Побочные эффекты User Access Control

В начале мы использовали отладчики пользовательского режима, чтобы запускать новые процессы или подключаться к уже существующим. Отладчик без проблем подключается к большинству выполняющихся процессов, пока целевой процесс не является системным процессом. В таких случаях отладчик выдает ошибку, как показано в листинге 15.1.

Листинг 15.1. Отказ подключения к привилегированному процессу

```
C:\>cdb -pn winlogon.exe
Microsoft (R) Windows Debugger Version 6.6.0007.5
Copyright (c) Microsoft Corporation. All rights reserved.
Cannot debug pid 564, Win32 error 5
"Access is denied."
Debuggee initialization failed, Win32 error 5
"Access is denied."
```

Отладчик не может подключиться к привилегированному процессу и возвращает ошибку доступа, даже если учетная запись, запускающая отладчик, входит в локальную группу администраторов. Поскольку ошибка выглядит вызванной изменениями безопасности Windows Vista, быстрый просмотр новых возможностей безопасности направил расследование к побочному эффекту функции User Account Control (UAC). Функция UAC в Windows Vista защищает компьютер от несанкционированных изменений, добиваясь этого несколькими средствами. В данном случае UAC преобразовала мощную учетную запись администратора в учетную запись пользователя с низкими привилегиями, которая не может использовать членство в группе администраторов. Дополнительные сведения доступны в разделе, посвященном изменениям безопасности Windows Vista.

На следующем этапе мы запустили несколько процессов под отладчиком пользовательского режима. Большинство процессов запускалось без проблем, но некоторые не запускались. Сбойные процессы делают это с сообщением об ошибке *The requested operation requires elevation*, как в листинге 15.2. Эта ошибка возникает для всех процессов, показывающих запрос повышения UAC.

Листинг 15.2. Запрос повышения при запуске процесса

```
C:\>cdb mmc.exe
Microsoft (R) Windows Debugger Version 6.6.0007.5
Copyright (c) Microsoft Corporation. All rights reserved.
CommandLine: mmc.exe
Cannot execute 'mmc.exe', Win32 error 740
"The requested operation requires elevation."
Debuggee initialization failed, Win32 error 740
"The requested operation requires elevation."
```

Далее мы запускаем консоль в режиме повышенных привилегий, который имеет права, похожие на права учетной записи администратора на платформе более раннего уровня. Любой исполняемый файл может запускаться с полными административными привилегиями после выбора пункта *Run as Administrator* из контекстного меню ярлыка приложения. На рисунке 15.2 показаны параметры, доступные для запуска ярлыка.



Рисунок 15.2. Запуск командной строки с полными административными правами

Из повышенной командной строки отладчик пользовательского режима работает ожидаемо. Он может подключаться к любому выполняющемуся процессу в системе и запускать любое приложение.

Мы используем эти две командные строки - одну, выполняющуюся как UAC-пользователь, и одну, выполняющуюся как повышенный пользователь, - чтобы протестировать отладчик режима ядра. Отладчик режима ядра, работающий в локальном режиме, доказал свою полезность при исследовании проблем прав доступа, deadlock или отслеживании серии LPC-вызовов. При запуске из обычной командной строки отладчик ожидаемо завершается с ошибкой, как показано в листинге 15.3.

Листинг 15.3. Локальная отладка ядра из обычной командной строки

```
C:\>kd -kl
Microsoft (R) Windows Debugger Version 6.6.0007.5
Copyright (c) Microsoft Corporation. All rights reserved.
The system does not support local kernel debugging.
Local kernel debugging requires Windows XP, Administrative
privileges, and is not supported by WOW64.
Only a single local kernel debugging session can run at a time.
Debuggee initialization failed, HRESULT 0x80004001
"Not implemented"
```

Не должно быть сюрпризом, что операция завершается неудачей, поскольку отладчик режима ядра в локальном режиме может причинить системе неограниченный ущерб. Однако после попытки выполнить ту же операцию из повышенной командной строки мы были немного удивлены, что инструмент завершается ошибкой доступа, показанной в листинге 15.4.

Листинг 15.4. Отказ локального отладчика ядра из повышенной командной строки

```
C:\>kd -kl
Microsoft (R) Windows Debugger Version 6.6.0007.5
Copyright (c) Microsoft Corporation. All rights reserved.
Debugger can't get KD version information, Win32 error 5
```

Чтобы проверить конфигурацию режима ядра, мы используем инструмент `kdbgctrl.exe`, установленный вместе с `Debugging Tools for Windows`. Этот инструмент можно использовать, чтобы узнать текущее состояние отладчика ядра и изменить его при необходимости. Вывод команды сообщает нам, что отладчик режима ядра отключен, как показано в следующем листинге:

```
C:\>kdbgctrl.exe -c
Kernel debugger is disabled
```

В Windows Vista, как и в предыдущих версиях операционных систем Windows, отладчик ядра должен быть включен во время загрузки, чтобы стать активным. Следующий раздел описывает, как настроить состояние отладчика ядра при следующей загрузке.

Включение отладчика режима ядра

Глава 2 использует `bootcfg.exe` как предпочтительный инструмент для внесения изменений в `boot.ini`, который управляет состоянием отладчика режима ядра. Однако в Windows Vista этого инструмента нет, потому что вся система загрузчика изменилась, и конфигурация загрузки переместилась из `boot.ini` в другое системное хранилище, называемое Boot Configuration Data (BCD). Хранилищем BCD можно управлять с помощью командной строки `bcdedit.exe`. При использовании без параметров командной строки команда отображает все BCD-объекты из хранилища BCD со всеми параметрами конфигурации, или BCD-элементами, как показано в листинге 15.5.

Листинг 15.5. Просмотр Boot Configuration Data

```
C:\>bcdedit
Windows Boot Manager
-----
identifier {bootmgr}
device partition=C:
description Windows Boot Manager
locale en-US
inherit {globalsettings}
default {current}
resumeobject {b4e45951-b979-11db-b123-967f83be464e}
displayorder {current}
{9556a9b0-7a61-11db-935f-fbbbd349454f}
toolsdisplayorder {memdiag}
timeout 3
Windows Boot Loader
-----
identifier {current}
device partition=C:
path \Windows\system32\winload.exe
description Microsoft Windows Vista
locale en-US
inherit {bootloadersettings}
osdevice partition=C:
systemroot \Windows
resumeobject {b4e45951-b979-11db-b123-967f83be464e}
nx OptIn
```



```
debug Yes
nx OptIn
```

Конфигурацию можно изменить, передав новые значения как параметры в командной строке. По умолчанию все изменения влияют на BCD-объект, соответствующий текущей выполняющейся конфигурации. Изменения можно нацелить на другой BCD-объект, указав его в командной строке. Листинг 15.6 использует `bcdedit.exe`, чтобы включить отладчик режима ядра на последовательном порту COM1 со скоростью 115200 бод.

Листинг 15.6. Включение отладчика режима ядра через `bcdedit`

```
C:\>bcdedit /debug {current} on
The operation completed successfully.
C:\debug.x86>bcdedit /dbgsettings SERIAL DEBUGPORT:1 BAUDRATE:115200
The operation completed successfully.
```

Хранилище BCD, как и инструмент командной строки `bcdedit.exe`, очень хорошо документированы на веб-сайте Microsoft.

После включения отладчика режима ядра и перезапуска системы отладчик ядра работает ожидаемо как в локальном режиме, так и при подключении к хост-системе с помощью физического кабеля. Переходя к следующей главе, мы с приятным удивлением видим, что пример, использованный в главе 3, "Debuggers Uncovered", работает ожидаемо после учета ограничений УАС и использования повышенной командной строки при необходимости. Механизм обработки необработанных исключений значительно улучшен в Windows Vista и подробно описан в разделе, посвященном главе 13, "Postmortem Debugging".

Address Space Layout Randomization

В некоторых случаях инженеры делают предположения об адресах, содержащих различные элементы, например об адресе глобальной переменной или адресе функции. Предположение может быть таким простым, как заметка с несколькими командами, использующими виртуальные адреса для установки точек останова. Такие "командные заметки" часто используются инженерами, чтобы быстрее вводить набор команд.

В Windows Vista функция Address Space Layout Randomization (ASLR) ставит это предположение под сомнение. При ASLR система игнорирует предпочтительный адрес загрузки и загружает динамические библиотеки по разным адресам при каждом запуске системы. Листинги 15.7 и 15.8 показывают фрагмент окружения процесса для примера `08cli.exe` до и после перезапуска системы.

Листинг 15.7. Фрагмент окружения процесса до перезапуска

```
0:000> !peb
...
Base TimeStamp Module
400000 4529f49a Oct 09 00:04:58 2006 c:\AWDBIN\WinXP.x86.chk\08cli.exe
77570000 4549bdc9 Nov 02 01:43:37 2006 C:\Windows\system32\ntdll.dll
76d70000 4549bd80 Nov 02 01:42:24 2006 C:\Windows\system32\kernel32.dll
76e50000 4549bd92 Nov 02 01:42:42 2006 C:\Windows\system32\ole32.dll
...
```

После перезапуска системы то же приложение загружает все системные динамически подключаемые библиотеки по другим адресам.

Листинг 15.8. Фрагмент окружения процесса после перезапуска

```
0:003> !peb
...
Base TimeStamp Module
400000 4529f49a Oct 09 00:04:58 2006 c:\AWDBIN\WinXP.x86.chk\08cli.exe
```

```
774b0000 4549bdc9 Nov 02 01:43:37 2006 C:\Windows\system32\ntdll.dll
75f40000 4549bd80 Nov 02 01:42:24 2006 C:\Windows\system32\kernel32.dll
760a0000 4549bd92 Nov 02 01:42:42 2006 C:\Windows\system32\ole32.dll
...
```

Образ процесса загружен по тому же адресу, потому что он был скомпонован для Windows XP без параметра /dynamicbase, то есть параметра linker, управляющего поведением ASLR.

По материалам главы 6: Повреждения памяти, часть II - кучи

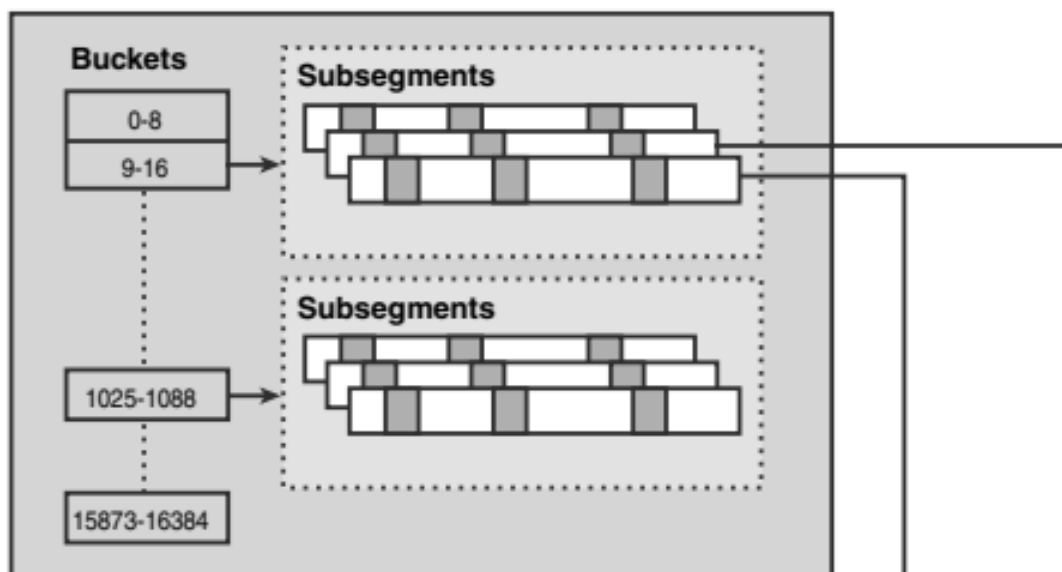
Менеджер кучи в Windows Vista прошел через серьезные изменения. Хотя предыдущие версии менеджера кучи работали хорошо, быстрое развитие вычислительного оборудования привело к новым требованиям к менеджеру кучи. По мере развития операционной системы три крупнейших вызова для менеджера кучи относились к безопасности, производительности и повышенной устойчивости к плохо ведущим себя приложениям.

В прежних версиях Windows атаки безопасности становились все более изощренными и в итоге привели к атакам на основе кучи. Хотя они встречались не так часто, как другие типы атак, например атаки на основе стека, и были гораздо сложнее по природе, инженеры Microsoft решили учесть этот очень важный вектор атак в Windows Vista. Другая причина переработки менеджера кучи была связана с эволюцией оборудования. С исторической точки зрения компьютеры прошли путь от нескольких мегабайт памяти (или килобайт, если уйти достаточно далеко назад) до гигабайтов и даже терабайтов RAM. Кроме того, с появлением многопроцессорных систем как стандартных конфигураций машин требования к менеджеру кучи изменились: он должен был хорошо работать на таких системах. Наконец, последняя категория улучшений относилась к повышенной устойчивости к плохо ведущим себя приложениям. Неправильное управление памятью кучи постоянно было источником проблем в приложениях, и в Windows Vista менеджер кучи теперь имеет встроенное раннее обнаружение нескольких разных типов распространенных повреждений кучи.

Чтобы удовлетворить новые требования, Windows Vista представила обновленную версию front-end allocator, называемую low-fragmentation front end. Low-fragmentation heap была частью Windows начиная с Windows Server 2003 и Windows XP, но теперь она значительно улучшена и включена по умолчанию в Windows Vista.

Рисунок 15.3 иллюстрирует архитектуру low-fragmentation heap.

Front End Allocator



Back End Allocator

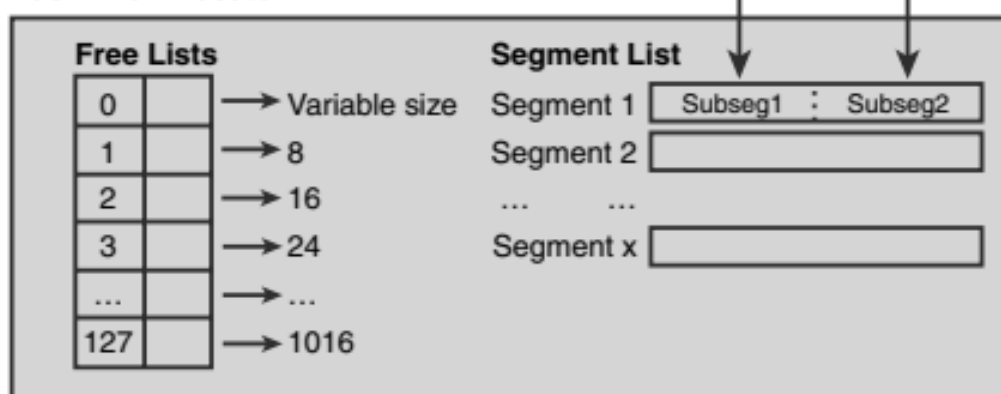


Рисунок 15.3. Архитектура low-fragmentation heap

Основная идея low-fragmentation heap состоит в том, чтобы уменьшить объем фрагментации, который мог возникать со старым look aside list front end. До Windows Vista свободные и занятые блоки в любом заданном сегменте могли чередоваться так, что не оставалось свободных блоков с достаточным количеством непрерывной памяти для удовлетворения выделения. Рисунок 15.4 иллюстрирует гипотетический пример сегмента кучи, страдающего от фрагментации.



Рисунок 15.4. Фрагментация в сегменте кучи

Сегмент кучи содержит пять занятых блоков кучи размером 32 байта и четыре свободных блока кучи размером 16 байт. Если запрошено выделение 32 байт, оно не будет удовлетворено из этого сегмента, потому что менеджер кучи не способен найти 32 байта доступной непрерывной памяти, даже если всего доступно больше 32 байт свободной памяти.

Low-fragmentation front end allocator в Windows Vista решает эту проблему, поддерживая подсегменты внутри каждого сегмента кучи. Каждый подсегмент может содержать только блоки заранее заданного размера. Например, рисунок 15.3 показывает, что выделения размером 9-16 байт имеют набор подсегментов, где каждый подсегмент размещен в сегментах кучи back-end allocator.

Поскольку каждый подсегмент занимает committed memory из сегмента back-end heap и управляется low-fragmentation front end, как управляются блоки кучи? Скажем, блок кучи размером 16 байт освобождается из одного из подсегментов. Кто знает, что блок освобожден? Помимо того, что сам блок кучи содержит флаги, указывающие состояние блока кучи - свободен или занят, - low-fragmentation front end также хранит односвязный список, связанный с каждым подсегментом. Этот односвязный список действует как свободный список для подсегмента. Когда свободный блок кучи становится доступен, он помещается в список, и если затем приходит запрос выделения для блока кучи такого размера, первая запись удаляется из списка и возвращается вызывающей стороне.

Свободный список каждого подсегмента доступен через заголовок подсегмента, содержащий некоторые метаданные, включая указатель на свободный список. Подсегменты создаются, когда запросы выделения приходят на front end. Когда подсегмент создается, менеджер кучи решает начальный размер подсегмента, то есть число слотов для предварительного выделения, на основе истории использования кучи в приложении или, если истории нет, начального размера по умолчанию. Если подсегмент когда-либо заполняется, менеджер кучи может создать другой подсегмент для размещения выделений.

Разметка блока кучи в low-fragmentation heap показана на рисунке 15.5.

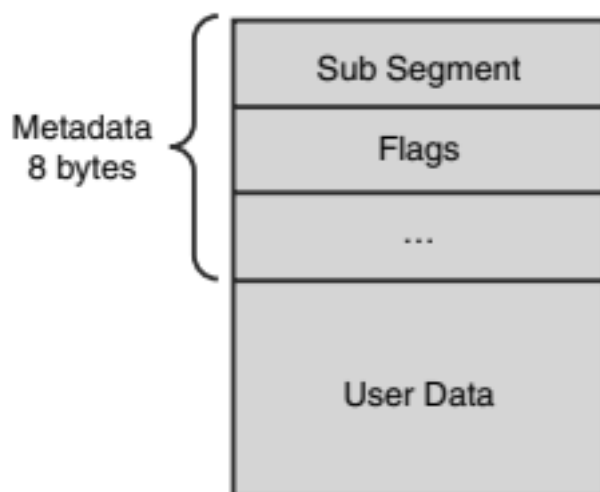


Рисунок 15.5. Разметка блока кучи для low-fragmentation heap

На высоком уровне основное различие между старыми блоками кучи и блоками low-fragmentation heap состоит в ссылке на подсегмент, частью которого является блок кучи. Кроме того, важно отметить, что метаданные блока кучи теперь "randomized". Следствие рандомизации метаданных состоит в том, что вывод сырого содержимого метаданных, например с помощью команды `dc`, больше не дает вывода, который можно легко понять. К счастью, команда расширения `!heap` обновлена так, чтобы учитывать рандомизацию, и перед отображением содержимого блока кучи команда расширения `!heap` декодирует метаданные и показывает результаты. Чтобы отображать блоки low-fragmentation heap с помощью команды расширения `!heap`, нужно использовать ключ `-i`. Например, если блок low-fragmentation heap находится по адресу `0x10b25de0`, вывод команды расширения `!heap` будет следующим:

```
0:028> !heap -i 10b25de0
Detailed information for block entry 10b25de0
Assumed heap : 0x00100000 (Use !heap -i NewHeapHandle to change)
Header content : 0x3C67E5EE 0xC2006898
Block flags : 0x1 LFH (busy )
Total block size : 0xe units (0x70 bytes)
Requested size : 0x58 bytes (unused 0x18 bytes)
Subsegment : 0x05f4dc58
7710dab4: ntdll!RtlAllocateHeap+0x0000021d
76eedcaf: ole32!CGIPTable::LazyMarshalGIPEntry+0x00000045
76eedef2: ole32!CRemoteUnknown::DoCallback+0x0000007a
76bb6dfe: RPCRT4!Invoke+0x0000002a
76c303ef: RPCRT4!NdrStubCall2+0x0000027b
76c321e2: RPCRT4!CStdStubBuffer_Invoke+0x000000a0
76fd92a0: ole32!SyncStubInvoke+0x0000003c
76fd923a: ole32!StubInvoke+0x000000b9
76f025c2: ole32!CCtxComChnl::ContextInvoke+0x000000fa
76f024d3: ole32!MTAInvoke+0x0000001a
76f03403: ole32!STAInvoke+0x00000046
76fd9162: ole32!AppInvoke+0x000000aa
76fd8fdf: ole32!ComInvokeWithLockAndIPID+0x0000032c
76fd9696: ole32!ComInvoke+0x000000c5
76f032d9: ole32!ThreadDispatch+0x00000023
76f0339e: ole32!ThreadWndProc+0x000000f5
77021a10: USER32!InternalCallWinProc+0x00000023
77021ae8: USER32!UserCallWinProcCheckWow+0x0000014b
77022a47: USER32!DispatchMessageWorker+0x00000322
77022a98: USER32!DispatchMessageW+0x0000000f
6f4ce5db: IEFRAME!CTabWindow::_TabWindowThreadProc+0x00000189
767d3833: kernel32!BaseThreadInitThunk+0x0000000e
770da9bd: ntdll!_RtlUserThreadStart+0x00000023
```

Low Fragmentation Heap и привязка к процессору. В главе 6, "Memory Corruptions - Part II", мы описали, как работает look aside front end allocator. По мере того как

многопроцессорные системы становились все более распространенными, look aside front end начал испытывать падение производительности в виде межпроцессорной синхронизации. Если несколько процессоров обращались к look aside front end, все запрашивая выделение одного и того же размера, доступ к look aside front end должен был синхронизироваться на уровне шины, что приводило к штрафу производительности. Чтобы решить эту проблему производительности, low-fragmentation front end ввел понятие привязки подсегмента к процессору. Low-fragmentation front end хранит массив подсегментов конкретных размеров на каждый процессор, чтобы избежать конкуренции при манипуляциях с кучей. Когда несколько процессоров обращаются к блокам кучи, содержащимся в этих подсегментах, блоки кучи управляются изолированно; поэтому вероятность того, что несколько процессоров одновременно обращаются к одним и тем же расположениям памяти и cache lines, уменьшается.

Чтобы показать, как отладчики можно использовать для отображения информации о low-fragmentation heap, мы используем команду расширения !heap с ключами -s и -a. Вывод команды расширения !heap, запущенной на экземпляре Internet Explorer 7.0, можно найти в следующем расположении:

```
C:\AWDBIN\Logs\heap.log
```

В подробном разделе сегмента файла журнала вы найдете список блоков кучи, доступных в сегменте. Некоторые блоки кучи разных размеров перемежаются, тогда как некоторые блоки одинакового размера идут подряд. Когда вывод показывает список подряд идущих блоков кучи одинакового размера, начало списка также показывает подсегмент, частью которого являются эти блоки, включая метаданные. Пример показан здесь:

```
...
...
...
092b95b0 092b95b8 00c50000 09280000 208 208 8 busy
092b97b8 092b97c0 00c50000 09280000 208 208 8 busy
092b99c0 092b99c8 00c50000 09280000 208 208 8 busy
092b9bc8 092b9bd0 00c50000 09280000 208 208 8 busy
092b9dd0 092b9dd8 00c50000 09280000 208 208 8 busy
092b9fd8 092b9fe0 00c50000 09280000 208 208 8 busy
092ba1e0 092ba1e8 00c50000 09280000 208 208 8 busy
092ba3e8 092ba3f0 00c50000 09280000 208 208 8 busy
092ba5f0 092ba5f8 00c50000 09280000 208 208 8 busy
092ba7f8 092ba800 00c50000 09280000 208 208 8 busy
Sub-segment 092dbcd8
User blocks: 0x092baa08
Block size: 0x208
Block count: 15
Free blocks: 0
Size index: 48
Affinity index: 0
Lock mask: 0x1
Flags: 0x0
092baa18 092baa20 00c50000 092dbcd8 208 - 8 LFH;busy
092bac20 092bac28 00c50000 092dbcd8 208 - 8 LFH;busy
092bae28 092bae30 00c50000 092dbcd8 208 - 8 LFH;busy
092bb030 092bb038 00c50000 092dbcd8 208 - 8 LFH;busy
092bb238 092bb240 00c50000 092dbcd8 208 - 8 LFH;busy
092bb440 092bb448 00c50000 092dbcd8 208 - 8 LFH;busy
092bb648 092bb650 00c50000 092dbcd8 208 - 8 LFH;busy
092bb850 092bb858 00c50000 092dbcd8 208 - 8 LFH;busy
092bba58 092bba60 00c50000 092dbcd8 208 - 8 LFH;busy
092bbc60 092bbc68 00c50000 092dbcd8 208 - 8 LFH;busy
092bbe68 092bbe70 00c50000 092dbcd8 208 - 8 LFH;busy
092bc070 092bc078 00c50000 092dbcd8 208 - 8 LFH;busy
092bc278 092bc280 00c50000 092dbcd8 208 - 8 LFH;busy
092bc480 092bc488 00c50000 092dbcd8 208 - 8 LFH;busy
092bc688 092bc690 00c50000 092dbcd8 208 - 8 LFH;busy
...
```

...

Предыдущие метаданные показывают подsegment с 15 блоками размером 0x208 байт. За метаданными следует список блоков кучи в этом подsegmente. Как видно, каждый блок имеет размер 0x208 байт и помечен LFH, что указывает, что это действительно блок low-fragmentation heap.

Как видно, менеджер кучи Windows Vista прошел через крупное обновление. Проблемы, связанные с безопасностью и производительностью, были значительно улучшены новым low-fragmentation front end allocator. Команда расширения !heap обновлена для бесшовной работы с новыми структурами кучи, что упрощает глубокое исследование кучи. По мере того как все больше приложений входит в пространство Vista, понимание того, как работает новая куча, будет становиться все более важным для эффективной отладки приложений, выполняющихся в Windows Vista.

По материалам главы 7: Безопасность

Без сомнения, Windows Vista представила беспрецедентные изменения в параметрах безопасности, доступных пользователю. Модель безопасности, определенная в Windows NT, была перенесена и расширена для поддержки новых возможностей, но осталась полностью обратно совместимой. Новые параметры безопасности можно включать и отключать по мере необходимости.

Настройки Windows Vista по умолчанию должны соответствовать ожиданиям большинства приложений, потому что эти настройки являются результатом бесчисленных часов тестирования совместимости приложений. В редких случаях, когда приложение работает не так, как ожидалось, инженеры, разрабатывающие Vista-совместимое приложение, должны знать существенные изменения в этой области, способы их идентификации и их влияние на приложение. Этот раздел ограничен существенными изменениями, видимыми инженеру, отлаживающему приложение с проблемами безопасности.

Самый заметный аспект новых параметров безопасности - функция User Account Control, уже встречавшаяся в предыдущих разделах. Чтобы понять эту функцию, необходимо понять лежащие в ее основе понятия.

Каждый выполняющийся процесс работает на одном из уровней целостности, показанных в таблице 15.2, причем уровень определяется на основе множества фактов. Уровень целостности идентифицируется SID, как и любой другой principal в Windows Vista. Уровни целостности используют новую authority идентификатора со значением {0, 0, 0, 0, 0, 16}. Относительные идентификаторы (RID), соответствующие этим уровням целостности, начиная с низких уровней целостности, следующие: 0x1000, 0x2000, 0x3000 и 0x4000.

Таблица 15.2. Уровни целостности

Имя уровня целостности	Идентификатор уровня целостности	Примеры приложений, выполняющихся на уровне
System	S-1-16-16384	Svchost.exe, Winlogon.exe, Windows Kernel
High	S-1-16-12288	mmc.exe, setup.exe
Medium	S-1-16-8196	explorer.exe, ieuser.exe
Low	S-1-16-4096	ieexplore.exe

Какова цель каждого уровня целостности и как система решает, какой уровень целостности выбрать для каждого процесса? Некоторые факторы, используемые при такой оценке, следующие:

- Windows Vista всегда выполняет процесс служб Windows и код ядра на системном уровне целостности.
- Инструменты, запрашивающие административные привилегии, специально распознаются операционной системой по их манифесту приложения. Пример манифеста, запрашивающего высокий уровень целостности, показан в листинге 15.9. Система также использует другие эвристики для определения требуемого уровня целостности. Например, любое приложение с именем `setup.exe` всегда запускается на высоком уровне целостности.
- Приложения, используемые для повседневных задач, такие как текстовые процессоры, электронные таблицы или почтовое ПО, выполняются на среднем уровне целостности. Один способ смотреть на процессы, выполняющиеся на среднем уровне целостности, - считать их процессами, запущенными обычным пользователем, без административных привилегий.
- Чтобы предотвратить распространение вредоносного ПО из Интернета, Internet Explorer работает на низком уровне целостности. В настоящее время Internet Explorer - единственное приложение, использующее самый низкий уровень целостности.

Листинг 15.9. Манифест, запрашивающий высокий уровень целостности

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">
<assemblyIdentity version="1.0.0.0" processorArchitecture="X86" name="Application"
type="win32"/>
<description>Application descriptions</description>
<trustInfo xmlns="urn:schemas-microsoft-com:asm.v3">
<security>
<requestedPrivileges>
<requestedExecutionLevel level="requireAdministrator" uiAccess="false"/>
</requestedPrivileges>
</security>
</trustInfo>
</assembly>
```

Процессы группируются по уровню целостности, и взаимодействие между двумя процессами, выполняющимися на разных уровнях целостности, контролируется операционной системой. Объекты ядра, созданные процессами, работающими на одном уровне целостности, по умолчанию не могут быть доступны процессам, выполняющимся на более низком уровне целостности.

Кроме того, функция User Interface Privilege Isolation (UIPI) контролирует поток сообщений Windows между процессами, выполняющимися на разных уровнях целостности. По умолчанию процесс может отправлять сообщения Windows только процессам, выполняющимся на том же или более низком уровне целостности.

Оба ограничения можно обойти разными приемами. Создатель объекта может скорректировать дескриптор безопасности, защищающий объекты ядра, чтобы сделать их доступными процессам, выполняющимся на более низких уровнях целостности.

Приложение может потребовать обхода UIPI, установив флаг `uiAccess` в `true` в манифесте приложения. Такое приложение должно быть подписано Microsoft Authenticode и находиться в защищенном расположении, например `%systemdrive%\Program Files` или `%systemdrive%\Windows\System32`, чтобы запускаться правильно.

User Access Control

User Access Control (UAC) меняет то, как члены локальных групп администраторов запускают свои приложения. UAC можно включить с помощью оснастки Local Security Policy, как показано на рисунке 15.6.

В Windows XP члены локальных администраторов запускают все свои приложения в своем контексте, имея полные административные привилегии. Любая уязвимость в любом из приложений, используемых администраторами, может быть эксплуатирована с серьезными последствиями для всей системы. Эксплуатируемая уязвимость может позволить тихую установку других компонентов в систему, тихое изменение разрешений реестра и файлов или выполнение административных задач.

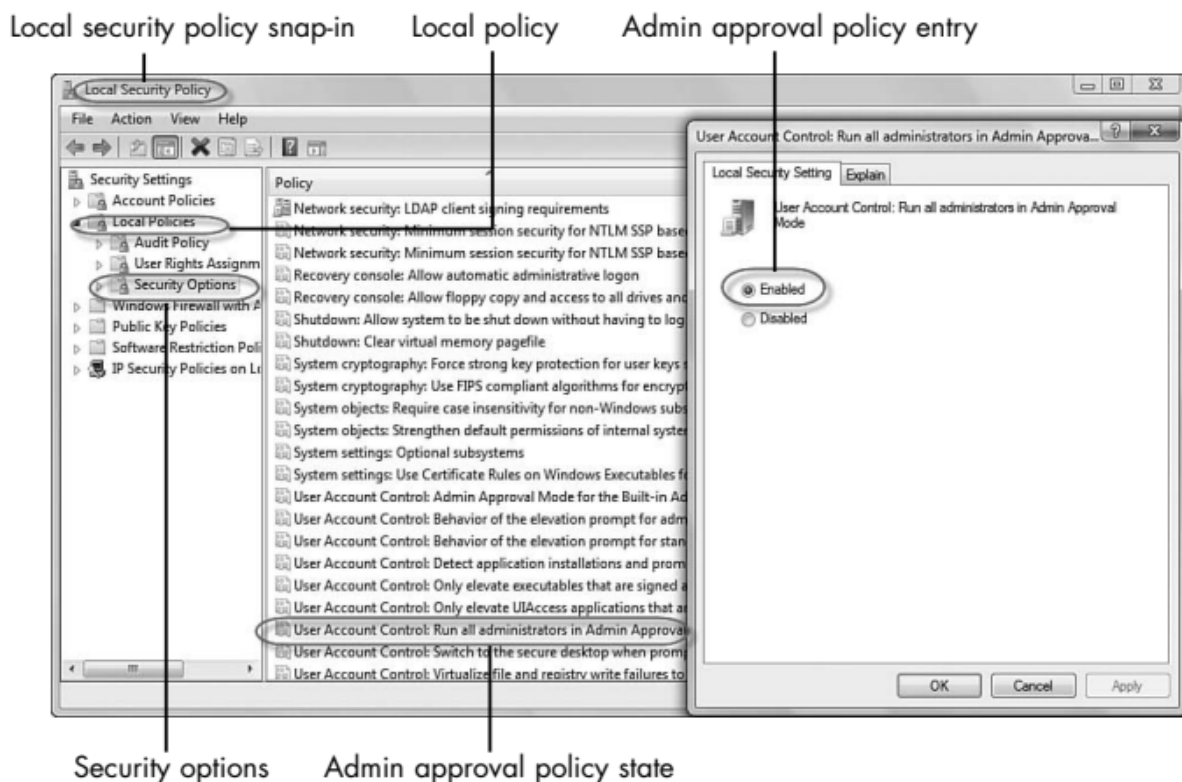


Рисунок 15.6. Изменение настройки User Access Control

В Windows Vista члены локальных администраторов запускают shell-процесс на среднем уровне целостности, когда политика UAC настроена на выполнение всех администраторов в Admin approval mode, и на высоком уровне целостности, когда политика отключена.

Если администратор не запускает приложение в административном режиме, как показано на рисунке 15.2, каждое новое приложение, запущенное из shell, наследует уровень целостности shell. Если система обнаруживает, что приложению нужны дополнительные привилегии, как определено в манифесте приложения или установлено системными эвристиками, заданными в локальной политике безопасности, она запрашивает у администратора согласие в диалоговом окне, похожем на показанное на рисунке 15.7.

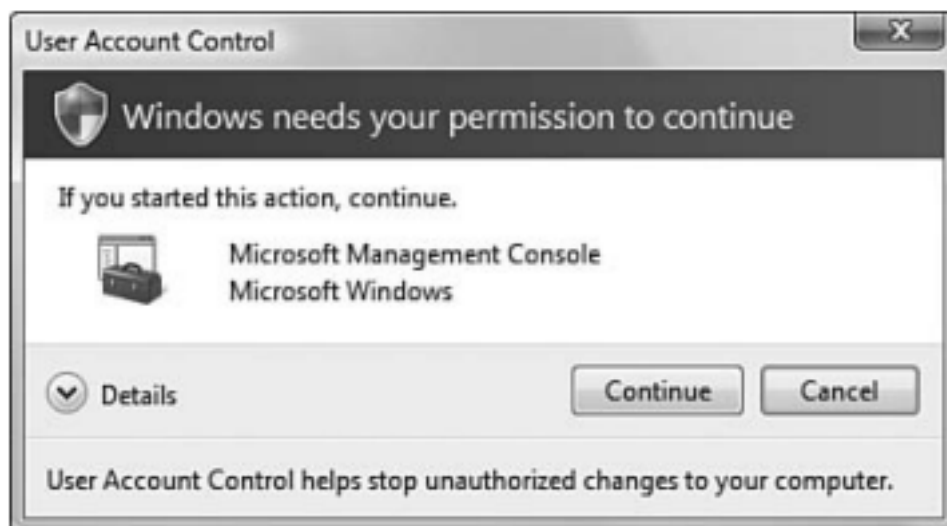


Рисунок 15.7. Запрос согласия на повышение

После выбора кнопки Continue система запускает приложение с полными административными привилегиями, подобно предыдущим версиям операционной системы. Поведение согласия управляется из той же оснастки, которая использовалась для включения UAC.

UAC в отладчике

При исследовании сбоев безопасности, связанных с UAC, мы начинаем с понимания роли всех участников сбоя. Как и любой другой сбой безопасности, сбой UAC следует тому же шаблону: одна сущность запрашивает доступ к объекту, защищенному gatekeeper, который использует дескриптор безопасности объекта, чтобы отказать в доступе к этому объекту. Листинг 15.10 не является результатом отладки случая сбоя. Он отображает security token, используемый процессом, выполняющим системный процесс, и дескриптор безопасности, защищающий объект процесса, иллюстрируя, как mandatory integrity level применяется к объекту ядра.

Листинг 15.10. Токен и дескриптор безопасности процесса системного уровня

```
kd> * Use !process extension command to get the process token
kd> !process 0 1 wininit.exe
PROCESS 82e59888 SessionId: 0 Cid: 01cc Peb: 7ffd5000 ParentCid: 0198
DirBase: 11d85000 ObjectTable: 8c65ff38 HandleCount: 93.
Image: wininit.exe
VadRoot 82efb180 Vads 53 Clone 0 Private 227. Modified 156. Locked 2.
DeviceMap 84403128
Token 8c6651d8
...
CommitCharge 294
kd> * Use !token extension command to dump token information.
kd> * Integrity level is the last group in the token
kd> !token 8c6651d8
_TOKEN 8c6651d8
TS Session ID: 0
User: S-1-5-18
Groups:
00 S-1-5-32-544
Attributes - Default Enabled Owner
01 S-1-1-0
Attributes - Mandatory Default Enabled
02 S-1-5-11
Attributes - Mandatory Default Enabled
03 S-1-16-16384
Attributes - GroupIntegrity GroupIntegrityEnabled
Primary Group: S-1-5-18
```

```

Privs:
03 0x00000003 SeAssignPrimaryTokenPrivilege Attributes -
...
35 0x00000023 SeCreateSymbolicLinkPrivilege Attributes - Enabled Default
Authentication ID: (0,3e7)
Impersonation Level: Anonymous
TokenType: Primary
Source: *SYSTEM* TokenFlags: 0x800 ( Token in use )
Token ID: 6c01 ParentToken ID: 0
Modified ID: (0, 6a7a)
RestrictedSidCount: 0 RestrictedSids: 00000000
OriginatingLogonSession: 0
kd> * What is the security of system-integrity level objects?
kd> * The process object is a perfect example to illustrate it
kd> !sd poi(82e59888-4)&FFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8814
SE_DACL_PRESENT
SE_SACL_PRESENT
SE_SACL_AUTO_INHERITED
SE_SELF_RELATIVE
->Owner : S-1-5-32-544
->Group : S-1-5-18
->Dacl :
->Dacl : ->AclRevision: 0x2
->Dacl : ->Sbz1 : 0x0
->Dacl : ->AclSize : 0x3c
->Dacl : ->AceCount : 0x2
->Dacl : ->Sbz2 : 0x0
->Dacl : ->Ace[0]: ->AceType: ACCESS_ALLOWED_ACE_TYPE
->Dacl : ->Ace[0]: ->AceFlags: 0x0
->Dacl : ->Ace[0]: ->AceSize: 0x14
->Dacl : ->Ace[0]: ->Mask : 0x001fffff
->Dacl : ->Ace[0]: ->SID: S-1-5-18
...
->Sacl :
->Sacl : ->AclRevision: 0x2
->Sacl : ->Sbz1 : 0x0
->Sacl : ->AclSize : 0x1c
->Sacl : ->AceCount : 0x1
->Sacl : ->Sbz2 : 0x0
->Sacl : ->Ace[0]: ->AceType: SYSTEM_MANDATORY_LABEL_ACE_TYPE
->Sacl : ->Ace[0]: ->AceFlags: 0x0
->Sacl : ->Ace[0]: ->AceSize: 0x14
->Sacl : ->Ace[0]: ->Mask : 0x00000003
->Sacl : ->Ace[0]: ->SID: S-1-16-16384

```

Уровень целостности присутствует в токене процесса как специальная группа: третья группа имеет значение SID уровня system integrity S-1-16-16384. Каждый объект, который должен быть доступен только пользователям, выполняющимся на конкретном уровне целостности, кодирует такой доступ как ACE-запись в SACL-части дескриптора безопасности объекта. В отличие от проверок доступа, основанных на членстве в группах, где разрешение оценивается путем проверки каждого SID из токена против соответствующего SID из дескриптора безопасности, mandatory system integrity level проверяется с учетом относительного порядка mandatory integrity levels. Уровень целостности SYSTEM самый высокий, за ним следуют HIGH, MEDIUM и LOW. Когда токен, используемый для доступа к ресурсу, выполняется на более низком уровне, чем уровень целостности объекта, флаги SYSTEM_MANDATORY_LABEL_ACE_TYPE управляют дальнейшим доступом. В листинге 15.10 флаги интерпретируются следующим образом:

```
0x03 = SYSTEM_MANDATORY_LABEL_NO_WRITE_UP | SYSTEM_MANDATORY_LABEL_NO_READ_UP
```

В этом случае объект недоступен ни одному пользователю в системе, кроме principal LocalSystem. Листинг 15.11 показывает тот же анализ для процесса, выполняющегося под учетной записью, входящей в локальную группу администраторов, при включенной политике UAC.

Листинг 15.11. Токен и дескриптор безопасности процесса с UAC

```
kd> !process 0 1 explorer.exe
PROCESS 901674f0 SessionId: 1 Cid: 0c04 Peb: 7ffd7000 ParentCid: 09cc
DirBase: 0a53e000 ObjectTable: 8f55b850 HandleCount: 599.
Image: explorer.exe
VadRoot 82ec6428 Vads 337 Clone 0 Private 3112. Modified 27217. Locked 0.
DeviceMap 927fb4f8
Token 9256d890
...
kd> !token 9256d890
_TOKEN 9256d890
TS Session ID: 0x1
User: S-1-5-21-2084298851-3655559499-3523964647-1000
Groups:
00 S-1-5-21-2084298851-3655559499-3523964647-513
Attributes - Mandatory Default Enabled
01 S-1-1-0
Attributes - Mandatory Default Enabled
02 S-1-5-32-544
Attributes - DenyOnly
...
10 S-1-16-8192
Attributes - GroupIntegrity GroupIntegrityEnabled
Primary Group: S-1-5-21-2084298851-3655559499-3523964647-513
Privs:
19 0x00000013 SeShutdownPrivilege Attributes -
23 0x00000017 SeChangeNotifyPrivilege Attributes - Enabled Default
25 0x00000019 SeUndockPrivilege Attributes -
33 0x00000021 SeIncreaseWorkingSetPrivilege Attributes -
34 0x00000022 SeTimeZonePrivilege Attributes -
Authentication ID: (0,9ec99)
Impersonation Level: Anonymous
TokenType: Primary
Source: User32 TokenFlags: 0xa00 ( Token in use )
Token ID: a2c7b ParentToken ID: 9ec9c
Modified ID: (0, 173021)
RestrictedSidCount: 0 RestrictedSids: 00000000
OriginatingLogonSession: 3e7
kd> !sd poi(901674f0-4) & 0xFFFFFFFF8
->Revision: 0x1
->Sbz1 : 0x0
->Control : 0x8814
SE_DACL_PRESENT
SE_SACL_PRESENT
SE_SACL_AUTO_INHERITED
SE_SELF_RELATIVE
->Owner : S-1-5-21-2084298851-3655559499-3523964647-1000
->Group : S-1-5-21-2084298851-3655559499-3523964647-513
...
->Sacl :
->Sacl : ->AclRevision: 0x2
->Sacl : ->Sbz1 : 0x0
->Sacl : ->AclSize : 0x1c
->Sacl : ->AceCount : 0x1
->Sacl : ->Sbz2 : 0x0
->Sacl : ->Ace[0]: ->AceType: SYSTEM_MANDATORY_LABEL_ACE_TYPE
->Sacl : ->Ace[0]: ->AceFlags: 0x0
->Sacl : ->Ace[0]: ->AceSize: 0x14
->Sacl : ->Ace[0]: ->Mask : 0x00000003
->Sacl : ->Ace[0]: ->SID: S-1-16-8192
```

Токен процесса explorer в этом случае имеет средний уровень целостности, и токен процесса ограничен уровнем целостности, равным medium или более высоким. Кроме того, третий SID в токене процесса является restricted SID, играющим роль отказа в доступе к ресурсам, доступным только этому restricted SID, в данном случае локальным администраторам, представленным S-1-5-32-544. Важно упомянуть, что концепция restricted SID не нова. Restricted SID используются на платформах более раннего уровня приложениями, применяющими restricted tokens.

Уровень целостности может применяться, и применяется, к ресурсам, управляемым другими сущностями, например файловой системой или ключами реестра. Уровень целостности не отображается в обычных местах, используемых для изменения безопасности таких ресурсов. Операционная система устанавливает новый инструмент, знающий об уровнях целостности, `icacls.exe`, который заменяет `cacls.exe`. `icacls.exe` отображает или изменяет уровень целостности любого файла с помощью параметра `setintegritylevel` и желаемого уровня целостности. Однако инструмент не позволяет тонко настраивать доступ, разрешенный пользователям с более низкой целостностью, и всегда использует флаг `SYSTEM_MANDATORY_LABEL_NO_WRITE_UP`. Листинг 15.12 проверяет безопасность одного из файлов примера, изменяет его уровень целостности и повторно проверяет его дескриптор безопасности. Новый mandatory integrity level установлен в medium, no write up.

Листинг 15.12. Проверка и изменение уровня целостности файла

```
C:\>icacls c:\AWDBIN\WinXP.x86.chk\01sample.exe
c:\AWDBIN\WinXP.x86.chk\01sample.exe BUILTIN\Administrators:(I)(F)
NT AUTHORITY\SYSTEM:(I)(F)
BUILTIN\Users:(I)(RX)
C:\>icacls c:\AWDBIN\WinXP.x86.chk\01sample.exe /setintegritylevel M
processed file: c:\AWDBIN\WinXP.x86.chk\01sample.exe
Successfully processed 1 files; Failed processing 0 files
C:\>icacls c:\AWDBIN\WinXP.x86.chk\01sample.exe
c:\AWDBIN\WinXP.x86.chk\01sample.exe BUILTIN\Administrators:(I)(F)
NT AUTHORITY\SYSTEM:(I)(F)
BUILTIN\Users:(I)(RX)
Mandatory Label\Medium Mandatory Level:(NW)
Successfully processed 1 files; Failed processing 0 files
```

Виртуализация реестра и файлов

Этот раздел рассматривает более тонкую проблему, встреченную при запуске нетривиального приложения в Windows Vista. Пример, использованный в главе 7, "Security", демонстрирует, как приложение может давать сбой после переноса в новую операционную систему. У примера есть вариант сохранения некоторых данных в двух защищенных расположениях - ключе реестра `HKEY_LOCAL_MACHINE\Software` и папке `C:\Program Files`, - и другой вариант чтения этих данных. Такой сценарий типичен для любого приложения, использующего глобальные настройки, настраиваемые через MMC snap-in. Приложение создает конфигурационную информацию нажатием клавиши 4 и проверяет конфигурацию нажатием клавиши 5.

Следующие два листинга имитируют описанную ситуацию. Листинг 15.13 показывает приложение, которому в Windows XP требовались административные привилегии, а теперь оно безупречно работает в контексте UAC.

Листинг 15.13. Создание и чтение конфигурации в контексте UAC

```
c:\>c:\AWDBIN\WinXP.x86.chk\07sample.exe
Press:
0 To use Security Descriptor APIs
1 To run Impersonation test
2 To run Default SD test
3 To run MAXIMUM_ALLOWED test
4 To set application configuration information
5 To read application configuration information
x To exit
> Creating application settings ...
Registry key HKEY_LOCAL_MACHINE\Software\Advanced Windows Debugging created
Create file C:\Program Files\sample4.test
Creating application done.
> Retrieving application settings ...
Registry key HKEY_LOCAL_MACHINE\Software\Advanced Windows Debugging found
```

```
Create file C:\Program Files\sample4.test found
Retrieving application done.
```

Листинг 15.14 имитирует конфигурацию приложения, функциональность которой обычно размещается внутри MMC snap-in. Поскольку MMC.EXE запрашивает повышение при запуске, приложение, использованное в листинге 15.14, также было запущено из повышенной командной строки, полученной, как показано на рисунке 15.2.

Листинг 15.14. Чтение конфигурации из повышенного контекста

```
c:\>c:\AWDBIN\WinXP.x86.chk\07sample.exe
Press:
0 To use Security Descriptor APIs
1 To run Impersonation test
2 To run Default SD test
3 To run MAXIMUM_ALLOWED test
4 To set application configuration information
5 To read application configuration information
x To exit
> Retrieving application settings ...
RegOpenKeyW failed.Last error = 2
CreateFile failed with>Last error = 2
Retrieving application done.
```

Даже если приложение может найти конфигурацию, пока оно работает в контексте UAC, в листинге 15.14 приложение не способно получить какую-либо информацию из глобального хранилища. Функция виртуализации реестра и файлов, добавленная в Windows Vista, чтобы на ограниченное время исправить приложения, требующие административных привилегий, вызывает ошибку, встреченную в этом случае. Эта функция включена по умолчанию, но ее можно отключить с помощью оснастки Local Security Policy, используемой для управления поведением UAC, как видно на рисунке 15.6. Не рекомендуется отключать виртуализацию, потому что после этого все существующие виртуальные файлы или ключи реестра теряются.

Виртуализованные файлы хранятся в профиле пользователя, в структуре папок, похожей на системную структуру папок, расположенной в %USERPROFILE%\AppData\Local\VirtualStore. Это дерево папок выглядит следующим образом:

```
C:\>tree %USERPROFILE%\AppData\Local\VirtualStore /a
Folder PATH listing for volume New Volume
Volume serial number is 5C50-0187
C:\USERS\DANIEL\APPDATA\LOCAL\VIRTUALSTORE
+--Program Files
| +--Common Files
| |--SmartFTP Client 2.0
| |--Minidump
|--Windows
|--Debug
--UserMode
```

Виртуализованные реестры хранятся в текущем профиле пользователя в структуре реестра, похожей на иерархию реестра, под ключом HKEY_CURRENT_USER\Software\Classes\VirtualStore. Инструмент командной строки reg.exe удобен для поиска расположения ключа через командную строку, как в следующем листинге:

```
C:\>reg query HKCU /s /k /f "Advanced Debugging"
HKEY_CURRENT_USER\Software\Classes\VirtualStore\MACHINE\SOFTWARE\Advanced Debugging
End of search: 1 match(es) found.
```

Поскольку приложения, выполняющиеся в повышенном режиме, не подлежат виртуализации, они будут иметь другое представление системы по сравнению с приложением, выполняющимся под UAC. Виртуализация Windows Vista влияет на все инструменты, использующие реестр Windows как хранилище конфигурации. Инструмент должен выполняться повышенно или нет, в зависимости от необходимости. Например, чтобы включить Application Verifier для MMC snap-in, нужно запустить Application Verifier повышенно, потому что MMC.EXE всегда выполняется повышенно. Иначе

конфигурационный инструмент обновит виртуальный ключ реестра, который не используется, когда приложение запускается в повышенном режиме. Состояние виртуализации видно в Windows Task Manager в столбце Virtualization на вкладке Processes, как показано на рисунке 15.8. Столбец можно включить через меню View, затем команду Select columns.

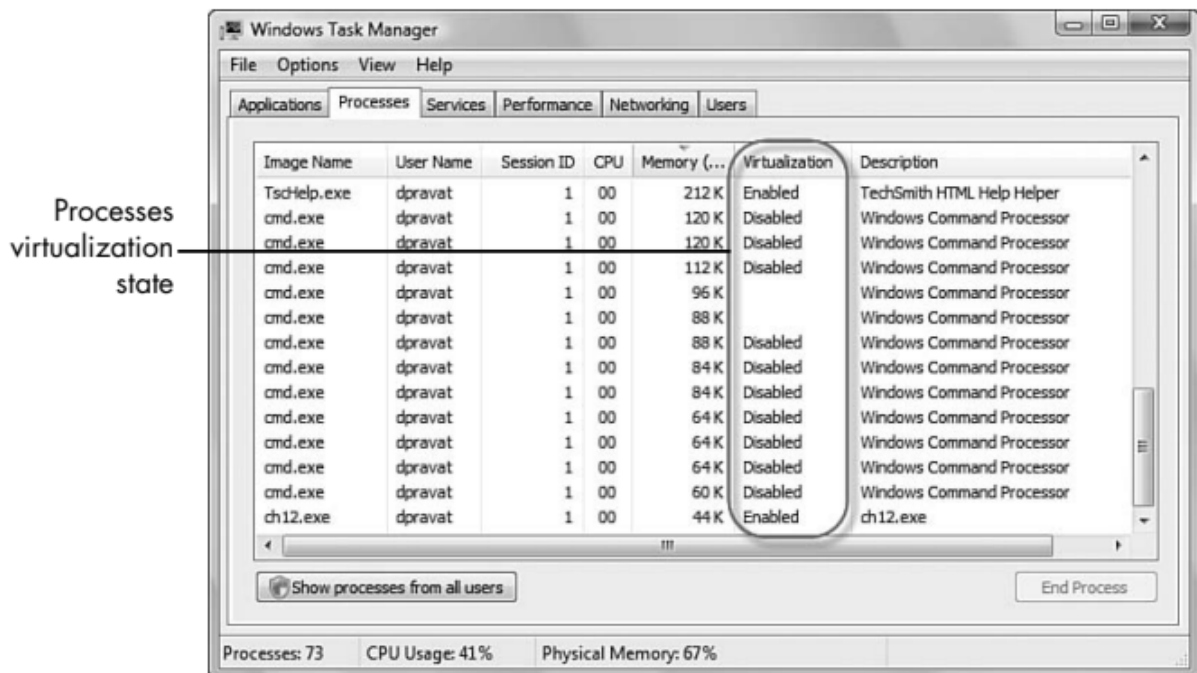


Рисунок 15.8. Просмотр виртуализации процессов

Приложения, такие как `cmd.exe` или `reg.exe`, не подлежат виртуализации, потому что их манифест приложения указывает `requestedExecutionLevel`, как показано в листинге 15.9.

Нативные приложения x86-64 не виртуализуются и будут завершаться сбоем под учетной записью UAC так же, как они завершались сбоем в Windows XP при выполнении под стандартной учетной записью пользователя. Это служит мотивацией для написания приложений, которым для правильной работы не требуются административные привилегии. Такое принуждение имеет минимальное влияние на совместимость приложений, поскольку число приложений x86-64 все еще невелико.

Почему приложения не работают под стандартной учетной записью пользователя?

Некоторые приложения записывают данные в расположения, защищенные стандартными настройками безопасности операционной системы. Чтобы записывать в эти расположения, приложения должны выполняться под учетной записью локального администратора. Приложения, хранящие свои настраиваемые параметры в папке установки или в глобальном кусте реестра, таком как `HKEY_LOCAL_MACHINE`, являются типичными случаями сбоев.

Самый простой способ найти такие сбои - тестировать приложение под Application Verifier, как описано в главе 1, "Introduction to the Tools". Другой простой инструмент для оценки такого поведения приложения - Microsoft Standard User Analyzer, доступный для бесплатной загрузки на веб-сайте Microsoft.

Исправление приложения на этапе разработки является самым быстрым и эффективным решением таких проблем. Как положительный побочный эффект приложение становится на один шаг ближе к возможности получить логотип Certified for Windows Vista.

Чтобы найти все процессы, не соответствующие стандартам сертификации Windows Vista, простая проверка виртуальных расположений файлов и реестра, скорее всего, выявит их.

Помимо изменений безопасности, сделанных с намерением создать "самую безопасную версию Windows на данный момент", Windows Vista представила другие изменения безопасности, предназначенные для предотвращения вмешательства в так называемые *protected processes*. *Protected processes*, обычно приложения-хосты воспроизведения мультимедиа, предоставляют более высокую степень защиты от вмешательства, особенно когда используют медиа, защищенные технологиями DRM. Хотя изменения, поддерживающие *protected process*, реализованы глубоко в ядре, они проявляются в невозможности отлаживать такие процессы. В Microsoft MSDN есть несколько whitepapers, описывающих *protected process* и будущее разработок в этой области.

В этом разделе вы узнали, что стоит за изменениями безопасности, реализованными в Windows Vista, и какой эффект эти изменения оказывают на программные приложения. Кроме того, вы узнали механизмы извлечения подробной информации из сбойной системы, когда сбой вызван этими изменениями безопасности.

По материалам главы 8: Межпроцессное взаимодействие

Изменения, внесенные Windows Vista в область межпроцессного взаимодействия, ограничены границами одной физической системы. Поскольку компьютеры под управлением Windows Vista должны бесшовно работать в неоднородной сети, поведение, наблюдаемое по сети, похоже на поведение предыдущих операционных систем, и все приемы, основанные на интерпретации сетевого трафика, по-прежнему применимы. Однако модель взаимодействия между различными компонентами внутри одной физической системы изменилась. Протокол *Lightweight Procedure Call* (LPC) был заменен новым протоколом *Advanced Lightweight Procedure Call* (ALPC), который является основой для других протоколов.

Изменения протоколов RPC и DCOM

Все протоколы, построенные поверх ALPC, были изменены, чтобы воспользоваться новыми коммуникационными парадигмами. DCOM и RPC также были улучшены в области надежности и управляемости.

Чтобы улучшить возможности отладки RPC, команда разработчиков ввела два канала трассировки. В канале *Microsoft-Windows-Rpcss-EndpointMapper/Debug* инфраструктура RPC записывает каждый интерфейс, зарегистрированный или снятый с регистрации в *EndpointMapper*. Канал *Microsoft-Windows-RPC/EEInfo* собирает ошибку, сгенерированную инфраструктурой RPC, как описано в главе 8, в разделе "RPC Extended Error Information".

Как часть улучшений управляемости, DCOM имеет канал трассировки *Microsoft-Windows-COM/Analytic*, который может захватывать события, генерируемые инфраструктурой. Однако на момент написания не было доступно информации о том, как включить этот канал или как интерпретировать события, генерируемые в этом журнале.

По материалам главы 9: Утечки ресурсов

В главе 9 мы обсуждали несколько разных инструментов и стратегий для поиска утечек ресурсов. Обсуждались два типа утечек ресурсов: handles и memory. Handles, с точки зрения отладки, ведут себя в Windows Vista так же, то есть команда расширения !htracе полностью функциональна, тогда как heap memory значительно изменилась для улучшения производительности и безопасности. Даже несмотря на то, что менеджер кучи изменился, некоторые инструменты обнаружения утечек все еще можно использовать для поиска утечек памяти. Два основных инструмента, обсуждавшихся в главе 9, - LeakDiag и UMDH. К сожалению, на момент написания LeakDiag не работает в Windows Vista, хотя UMDH можно использовать как альтернативу.

По материалам главы 10: Синхронизация

В главе 10, "Synchronization", мы обсуждали наиболее часто используемые примитивы синхронизации потоков, доступные в Windows. Critical sections, events, mutants и так далее уже довольно давно являются частью операционной системы Windows. С появлением Windows Vista возможности синхронизации были расширены и включили набор новых возможностей. В частности, были представлены следующие новые примитивы синхронизации:

- Slim reader/writer lock
- Condition variables
- One-time initialization
- Improved thread pool

В этой части главы мы кратко рассмотрим новые возможности синхронизации, доступные в Windows Vista.

Slim Reader/Writer Lock

Reader/writer lock - это синхронизационная конструкция, обычно используемая при организации конкурентного доступа к общему ресурсу, где разрешено множество одновременных читателей, но писатели получают эксклюзивное владение ресурсом. Хотя доступ на чтение/запись к общему ресурсу можно легко решить с помощью одного из существующих примитивов синхронизации, например critical section или mutex, преимущество reader/writer lock проявляется, когда есть довольно большое число читателей и относительно мало писателей. В таком сценарии использование critical section может быть неоптимальным, потому что critical section разрешает только эксклюзивный доступ к общему ресурсу независимо от того, являются ли все потоки, пытающиеся получить доступ к ресурсу, потоками-читателями. Новый reader/writer lock различает доступ reader lock и writer lock, вводя два режима блокировки:

- **Exclusive:** предоставляет эксклюзивный доступ к общему ресурсу. Пока поток имеет эксклюзивный доступ к lock, никакой другой поток не может получить доступ к ресурсу. Потоки-писатели обычно запрашивают эксклюзивный доступ.
- **Shared:** предоставляет общий доступ к ресурсу, позволяя нескольким одновременным потокам читать данные, защищенные lock. Потоки-читатели обычно запрашивают общий доступ.

Как и для любой синхронизационной конструкции, fairness является важным аспектом. Важно отметить, что новый reader/writer lock в Windows Vista не соблюдает fairness. Таблица 15.3 перечисляет новые reader/write locks в Windows Vista.

Таблица 15.3. API slim reader/writer lock

API	Описание
InitializeSRWLock	Инициализирует simple reader/writer lock. Структура данных, используемая для представления reader writer lock, имеет тип SRWLOCK.
AcquireSRWLockExclusive	Захватывает reader/writer lock в exclusive mode, указывая, что другим потокам не разрешен доступ к lock. Примером может быть writer, получающий эксклюзивный доступ к lock.
AcquireSRWLockShared	Захватывает reader/writer lock в shared mode, указывая, что другим потокам разрешен доступ к lock. Примером может быть reader, получающий shared access к lock.
ReleaseSRWLockExclusive	Освобождает reader/writer lock, ранее захваченный в exclusive mode.
ReleaseSRWLockShared	Освобождает reader/writer lock, ранее захваченный в shared mode.

Отладка reader/writer locks может оказаться сложной задачей главным образом из-за ограниченного количества bookkeeping, связанного с lock. Определение SRWLOCK показано ниже:

```
typedef struct _RTL_SRWLOCK {
    PVOID Ptr;
}
```

Как видно, содержимое lock минимально и содержит только указатель на тип VOID. Сравните это определение с critical section, которая содержит массу информации, такую как owning thread, lock count, recursion count и так далее. Даже несмотря на ограниченность информации, доступной с reader/writer lock, процесс отладки похож на отладку других синхронизационных конструкций.

Condition Variables

Condition variable - новая синхронизационная конструкция, представленная в Windows Vista. Condition variable всегда работает совместно либо с critical section, либо со slim reader/writer lock. Основная идея condition variable состоит в том, что она позволяет потокам ждать наступления определенного условия. Ожидание выполняется как атомарная операция из трех шагов:

1. Освободить lock, связанный с condition variable.
2. Перейти в состояние ожидания.
3. После пробуждения повторно захватить lock, связанный с condition variable.

Функции, связанные с condition variables, показаны в таблице 15.4.

Таблица 15.4. API condition variables

API	Описание
InitializeConditionVariable	Инициализирует condition variable.
SleepConditionVariableCS	Ожидает condition variable и освобождает указанную critical section как атомарную операцию.
SleepConditionVariableSRW	Ожидает condition variable и освобождает указанный slim reader/writer lock как атомарную операцию.
WakeAllConditionVariable	Пробуждает все потоки, ожидающие указанную condition variable.
WakeConditionVariable	Пробуждает один поток, ожидающий указанную condition variable.

Подобно новому slim reader/writer lock, определение condition variable, тип CONDITION_VARIABLE, относительно простое по природе, как показано ниже:

```
typedef struct _RTL_CONDITION_VARIABLE {
    PVOID Ptr;
}
```

One-Time Initialization

Коду часто необходимо выполнить одноразовую инициализацию. В однопоточной среде это обычно несложная задача, но в многопоточной среде обеспечение того, что объект создается только один раз, требует некоторой работы. Чтобы упростить жизнь разработчикам, которым нужна одноразовая инициализация в многопоточной среде, Windows Vista представила API, помогающие снять болезненность с кода инициализации. Таблица 15.5 показывает функции, связанные с one-time initialization.

Таблица 15.5. API one-time initialization

API	Описание
InitOnceBeginInitialize	Начинает one-time initialization.
InitOnceComplete	Завершает one-time initialization.
InitOnceExecuteOnce	Выполняет указанную функцию инициализации один раз. Никакие другие потоки, указывающие ту же структуру one-time initialization, не могут выполняться, пока выполняется текущий поток.
InitOnceInitialize	Инициализирует структуру one-time initialization.

Фундаментальный тип данных за one-time initialization - структура INIT_ONCE, показанная ниже:

```
typedef RTL_RUN_ONCE INIT_ONCE;
typedef union _RTL_RUN_ONCE {
    PVOID Ptr;
}
```

Improved Thread Pool

Thread pool в Windows Vista прошел через крупное обновление. Фактически thread pool был переписан с нуля, чтобы сделать его надежнее и встроенно предоставить многие возможности, отсутствовавшие в предыдущих версиях Windows. Вот несколько примеров новых возможностей, добавленных в thread pool Windows Vista:

- Несколько pools на процесс
- Отмена работы
- Автоматическая очистка, включая освобождение locks
- Настраиваемое максимальное и минимальное число потоков

По материалам главы 11: Написание собственных расширений отладчика

Если вы не пишете собственное расширение отладчика, которому требуется специфичная функциональность Windows Vista, ничего особенного делать не нужно. Однако если вы пишете расширение, специфичное для Windows Vista, нужно убедиться, что build window, в котором вы выбираете сборку расширения, является Windows Vista build window. Это можно сделать, установив WDK и выбрав Windows Vista and Windows Longhorn Server build window из пункта меню Windows Driver Kits в Start menu.

В целом рекомендуется версионировать расширения отладчика в соответствии с версией операционной системы, для которой они работают. Если посмотреть на расширения отладчика, поставляемые с Debugging Tools for Windows, вы увидите несколько подкаталогов, каждый из которых представляет конкретную версию операционной системы. В каждом из подкаталогов можно найти расширения отладчика, требующие именно эту версию Windows. Если же вы разрабатываете расширение, которое не зависит от какой-либо конкретной версии Windows, можно просто собрать одно расширение, и оно будет работать в нескольких версиях Windows. Единственная оговорка такого подхода - нужно убедиться, что расширение собрано для lowest common denominator, чтобы оно загружалось в более поздних версиях Windows.

По материалам главы 13: Посмертная отладка

Postmortem debugging - важная часть работы инженера. В версиях до Windows Vista фундаментальным компонентом, обеспечивающим сбор postmortem data, был Dr. Watson. Каждый раз, когда происходил crash или hang, Dr. Watson вызывался для сбора postmortem data в виде log file и dump file.

В Windows Vista Dr. Watson был отправлен в отставку в пользу более надежной архитектуры. Раньше механизм, через который вызывался Dr. Watson, включал unhandled exception filter, оборачивавший все потоки, выполняющиеся внутри процесса. Когда поток давал fault и выполнялся exception filter, он запускал процесс Dr. Watson через API CreateProcess, чтобы выполнить error reporting. Этот подход работал хорошо, но существовали ситуации, где возникали осложнения. Примером такого осложнения был случай, когда стек потока был исчерпан или разрушен. В такой ситуации exception filter мог выполняться корректно или нет, в зависимости от состояния стека. Чтобы сделать error reporting в Windows Vista более надежным, архитектура сбора postmortem data была изменена.

Рисунок 15.9 иллюстрирует новую postmortem architecture в Windows Vista.

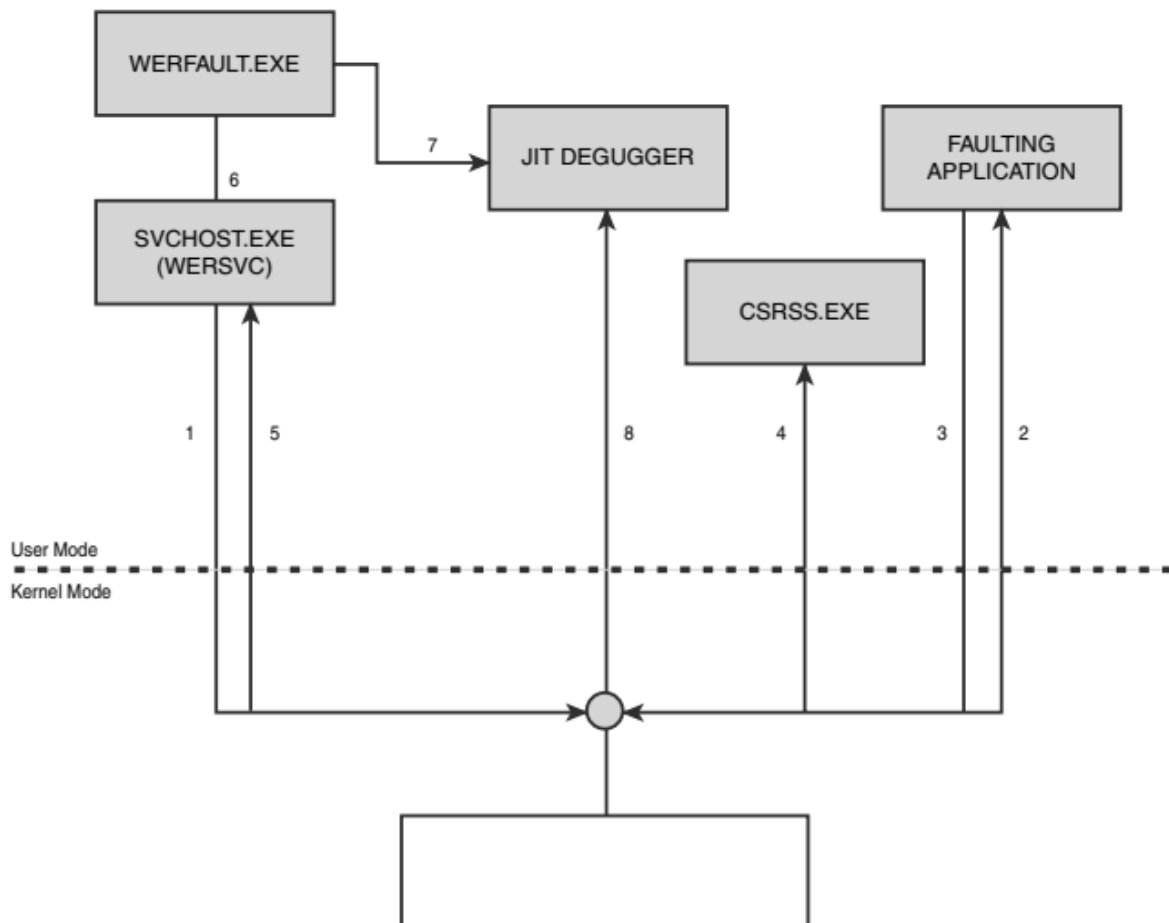


Рисунок 15.9. Postmortem-архитектура Windows Vista

Когда Windows Vista загружается, новая служба с именем WERSVC загружается в service host и регистрирует порт в ядре (шаг 1). Порт служит каналом связи между ядром и сбоями в пользовательском режиме. Когда fault возникает в процессе пользовательского режима, ядро - первый компонент, который видит исключение. Поскольку приложение пользовательского режима должно иметь возможность обработать first-chance exceptions, ядро просто направляет исключение в сбойный процесс пользовательского режима (шаг 2). Если процесс пользовательского режима не может обработать исключение, то есть это unhandled exception, блок try/catch вокруг потока пользовательского режима просто возвращает EXCEPTION_CONTINUE_SEARCH обратно ядру, указывая, что исключение было необработанным (шаг 3), а ядру следует продолжить поиск других компонентов, которые могут быть готовы обработать исключение.

Ядро снова пересылает исключение всем связанным exception ports для сбойного процесса. Для Win32-процессов все исключения направляются процессу CSRSS (подсистема Win32), который, в свою очередь, возвращает EXCEPTION_CONTINUE_SEARCH (шаг 4). В этот момент ядро вызывает службу WERSVC через error port на шаге 5 со связанной информацией о сбое, такой как process identifier, thread identifier и exception parameters. Компонент WERSVC, в свою очередь, вызывает процесс WERFAULT.EXE (шаг 6), который проверяет, должен ли он вызвать Just In Time (JIT) debugger. Если JIT debugger включен, процесс WERFAULT.EXE запускает отладчик и подключает его к сбойному процессу (шаг 7). Затем процесс WERFAULT.EXE возвращает статус в kernel mode, указывая, что JIT debugger был запущен, и ядро диспетчеризует исключение отладчику (шаг 8). Если JIT debugger не был включен, пользователю показывается error dialog, позволяющий отправить error report в Microsoft или отладить fault.

Как видно, благодаря отделению и делегированию обязанностей по обработке ошибок от сбойного процесса пользовательского режима в kernel mode мы больше не зависим от сбоев

пользовательского режима, таких как исчерпание стека или разрушение стека, которые могут привести к сбою логики обработки ошибок. Эта новая архитектура и процесс позволяют выполнять очень надежное error reporting, что раньше было невозможно.

Создание файлов дампа

В главе 13 мы обсуждали различные способы создания файлов дампа. Windows Vista предоставляет еще один удобный механизм через Task Manager. Щелчок правой кнопкой по процессу на вкладке Process в Task Manager позволяет выбрать пункт контекстного меню Create Dump File. Создание файла дампа с помощью этого варианта создает mini dump file с полной информацией памяти.

Еще одно важное изменение в Windows Vista - то, как технология error-reporting сохраняет dump files локально на машине. До Windows Vista Dr. Watson по умолчанию сохранял созданные dump files на локальной машине. Эти dump files могли быть доступны любому, кто хотел отладить конкретный dump file. В Windows Vista Dr. Watson был заменен более надежным и устойчивым механизмом error-reporting. Как часть этого изменения, dump files, созданные на машине, по умолчанию не сохраняются на локальной машине. Чтобы изменить поведение по умолчанию, можно установить параметр реестра ForceQueue в 1, что заставляет все dump files ставиться в локальную очередь перед отправкой в Microsoft. Значение реестра ForceQueue расположено по следующему пути реестра:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\Windows Error Reporting
```

После установки значения реестра ForceQueue в 1 все созданные dump files будут храниться в следующем расположении:

```
Processes running in system context or elevated:  
%ALLUSERSPROFILE%\Microsoft\Windows\WER\ [ReportQueue | ReportArchive]
```

```
All other processes:  
%LOCALAPPDATA%\Microsoft\Windows\WER\ [ReportQueue | ReportArchive]
```

Помимо повышенной надежности, Windows Vista представила набор новых API, которые позволяют приложениям делать гораздо больше, чем просто запускать стандартный механизм сбора ошибок, то есть отправку dump file. Точнее, приложения теперь могут программно отправлять пользовательские данные в дополнение к dump file. Данные могут иметь форму пользовательских log files, расширенных memory dumps и многого другого. Таблица 15.6 подробно описывает новые API.

Таблица 15.6. API Windows Error Reporting

Имя API	Описание
WerAddExcludedApplication	Исключает указанное приложение из error reporting.
WerGetFlags	Получает error-reporting settings для указанного процесса.
WerRegisterFile	Добавляет файл в набор файлов, которые могут быть добавлены к reports, создаваемым для текущего процесса. Это очень полезный механизм для добавления пользовательских файлов в процесс error-reporting.
WerRegisterMemoryBlock	По умолчанию error reporting собирает только определенные типы информации памяти. Иногда процессам необходимо расширить эту информацию, добавив memory dumps пользовательских областей памяти. Этот API позволяет вызывающей стороне указать начальный адрес и размер области памяти, которую нужно включить в report.

Имя API	Описание
WerRemoveExcludedApplication	Удаляет приложение из списка исключенных из error reporting.
WerReportAddDump	Позволяет вызывающей стороне управлять тем, что добавляется в dump file, создаваемый во время error reporting.
WerReportAddFile	Добавляет файл в report, создаваемый для fault.
WerReportCloseHandle	Закрывает указанный report.
WerReportCreate	Создает новый report.
WerReportSetParameter	Устанавливает параметры report.
WerReportSetUIOption	Позволяет вызывающей стороне настроить UI, отображаемый для конкретного report.
WerReportSubmit	Отправляет указанный report.
WerSetFlags	Устанавливает error-reporting settings для текущего процесса.
WerUnregisterFile	Удаляет файл из набора файлов, добавляемых к reports, создаваемым для текущего процесса.
WerUnregisterMemoryBlock	Удаляет memory block, ранее зарегистрированный с помощью API WerRegisterMemoryBlock.

Добавленные возможности надежности, а также новая функциональность делают Windows Vista чрезвычайно мощной платформой, когда речь идет о диагностике и postmortem analysis.

Резюме

Windows Vista представляет последнее обновление семейства клиентских операционных систем Windows. Существует невероятное богатство новых функций: от впечатляющих новых приложений до значительно расширенных возможностей надежности, устойчивости и безопасности. Не должно удивлять, что Windows Vista представляет собой большую веху в истории операционных систем Windows и позволяет разработчикам ПО использовать все новые и интересные возможности при разработке программ.

На протяжении книги мы обсуждали конкретные сценарии отладки, а в этой главе подробно рассмотрели каждый из этих сценариев и различия, встречающиеся при работе в Windows Vista в том, что относится к отладке. Обсуждение началось с того, как инструменты работают под Windows Vista, а затем показало конкретные сценарии отладки, связанные с конкретной технологией, такой как безопасность, менеджер кучи, межпроцессное взаимодействие и другие.

Приложение А. Настройки тестов Application Verifier

На протяжении книги мы использовали Application Verifier, чтобы находить проблемы на ранней стадии разработки, когда их исправление еще не стоило бы дорого. Помимо использованных нами test settings, Application Verifier включает множество дополнительных настроек тестов, которые можно применять в зависимости от текущей проблемы. Это приложение служит справочником по доступным test settings.

Exceptions

Название этой настройки теста на самом деле довольно вводит в заблуждение. Хотя можно подумать, что этот stop относится ко всем видам проблем, связанных с исключениями, внутри него есть только один stop. Конкретно это stop, позволяющий прервать выполнение в отладчике сразу после возникновения first-chance access violation exception. Это позволяет поймать любые first-chance access violations, которые приложение могло непреднамеренно перехватить.

Handles

Handles используются как модель абстракции для базовых объектов операционной системы. Одна из самых распространенных проблем при работе с handles - передача неправильного значения handle в различные API, которые их принимают. Эта настройка теста проверяет разные проблемы handles, как показано в таблице A.1. По умолчанию все stop-коды, связанные с handles, вызывают break в отладчике при срабатывании.

Таблица A.1. Stop-коды проверки handles

Stop code	Тест	Описание
00000300	Invalid handles	Заставляет Application Verifier остановить выполнение при передаче недопустимого значения handle в системные routines.
00000301	Invalid Thread local storage (TLS) index	Чтобы получить данные thread local storage (TLS), нужно передать значение TLS index, соответствующее расположению хранения данных. Если указан недопустимый TLS index, Application Verifier останавливает выполнение.
00000302	Invalid parameters in calls to WaitForMultipleObjects	Когда выполняется вызов WaitForMultipleObjects, вызывающая сторона должна указать массив handles, которых нужно ожидать, а также число handles в массиве. Если любой из этих параметров равен null или 0, Application Verifier останавливает выполнение.
00000303	null handle values	Когда вызов system routine, принимающей handle как входной параметр, выполняется с null handle value, отладчик останавливает выполнение.
00000304	DllMain concurrency problems	Если поток, который сейчас выполняет код в DllMain, ожидает handle другого потока, Application Verifier останавливает выполнение. Ошибка состоит в том, что единственный способ сигнализировать ожидающему потоку - завершение выполняющегося потока. Когда выполняющийся поток завершается, он пытается вызвать функцию DllMain с сообщением DLL_THREAD_DETACH. Для этого он должен захватить DLL loader lock. Захват завершается неудачей, потому что первый поток уже владеет lock. Итоговый результат - deadlock.

Stop code	Тест	Описание
00000305	Invalid handle types	Этот stop срабатывает, когда код вызывает API, ожидающий handle конкретного типа, но вместо этого передает handle неправильного типа. Например, вызов SetEvent с handle на semaphore вызывает этот stop.

Heaps

Настройка теста Heaps содержит действительно мощную instrumentation, упрощающую жизнь при попытке разобраться с heap-related problems. Помимо широкого набора verifier stops, настройка heaps включает instrumentation на уровне отдельного heap block, в том числе heap block fill patterns, heap block guard pages и stack tracing. Возможности stack tracing чрезвычайно полезны, поскольку дают историческое представление через stack trace recording обо всех allocations и de-allocations, выполненных в процессе.

Свойства, являющиеся частью настройки теста heaps, весьма мощные и обширные. Рисунок А.1 показывает свойства, доступные при щелчке правой кнопкой по настройке теста heaps и выборе пункта меню Properties.

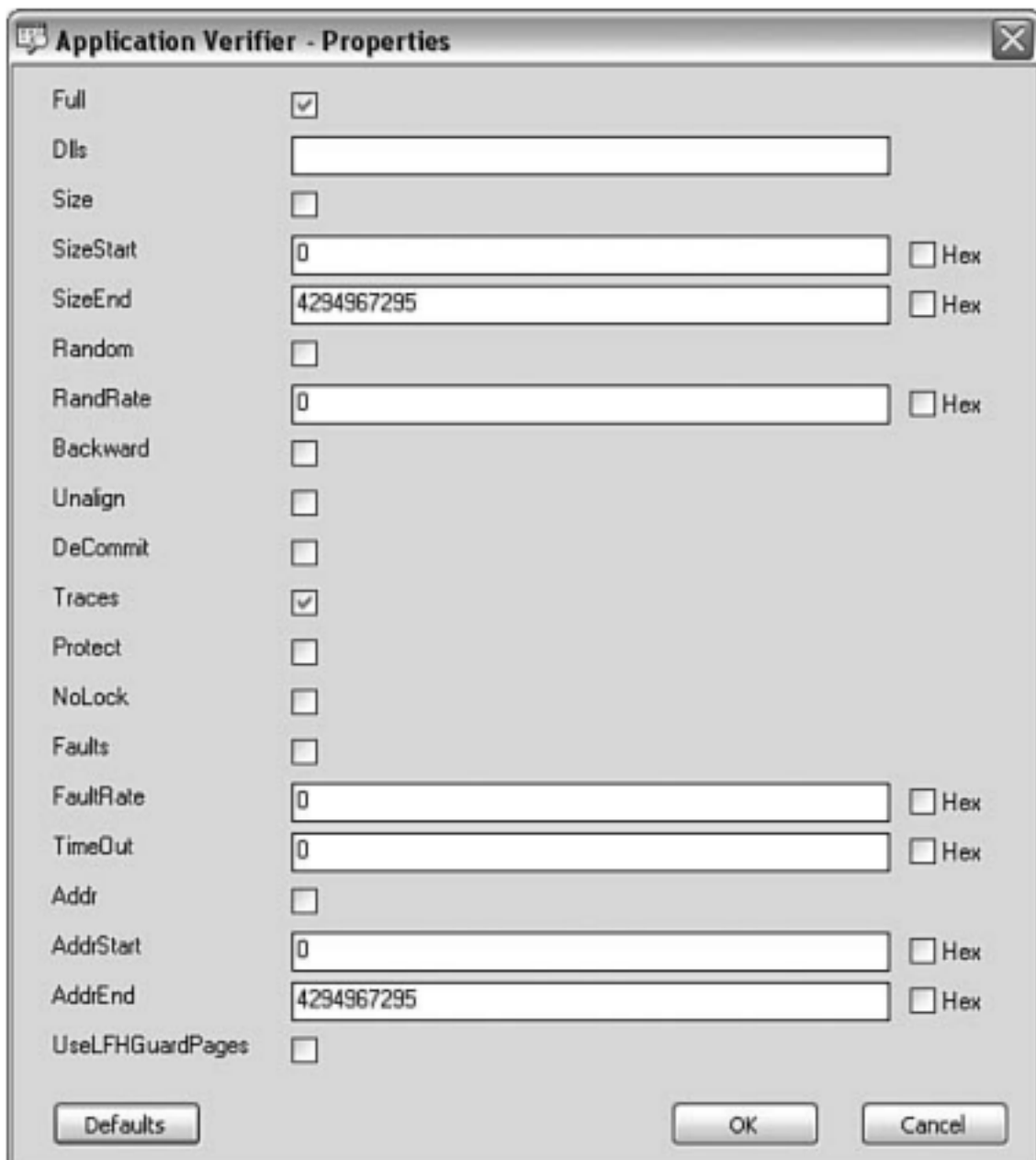


Рисунок А.1. Окно свойств Heaps

Каждый настраиваемый параметр обсуждается в следующем списке:

- **Full**

Heap instrumentation попадает в два режима:

- **Normal pageheap:** этот вариант pageheap является облегченной версией full heap instrumentation и не обнаруживает heap-related problems так своевременно, как full pageheap. Преимущество использования этого режима состоит в том, что он работает быстрее и требует меньше ресурсов. Чтобы работать с normal pageheap, снимите флажок Full в окне Properties.
- **Full pageheap:** full pageheap - король heap instrumentation. Он способен обнаруживать heap-related problems в момент возникновения, а не postmortem, и обычно облегчает отладку по сравнению с normal pageheap. Недостаток full pageheap в том, что он требует довольно много ресурсов, памяти, и поэтому работает медленнее normal pageheap. Чтобы выбрать вариант full pageheap, установите флажок Full в окне Properties.

Далее в этом разделе мы обсудим, как работают и normal, и full pageheap.

- **Dlls**

Поле Dlls позволяет указать, какие DLL должны участвовать в heap tests. Укажите имя DLL, включая расширение, а если нужно больше одной DLL, разделяйте каждую DLL пробелом.

- **Size**

Флажок Size позволяет включить тестирование allocations конкретного размера или диапазона размеров. Если вы устанавливаете этот флажок, нужно также заполнить поля SizeStart и SizeEnd.

- **SizeStart**

Если вы выбрали тестирование allocations конкретных размеров, нужно ввести начало интересующего диапазона размеров.

- **SizeEnd**

Если вы выбрали тестирование allocations конкретных размеров, можно ввести конец интересующего диапазона размеров.

- **Random**

Этот флажок позволяет выбрать, вводить ли случайный фактор при выборе instrumentation model для allocations. Например, установка этого флажка позволяет указать, что некоторые allocations должны выполняться на normal pageheap, а некоторые - на full pageheap. Если вы устанавливаете этот флажок, нужно также указать random rate в поле RandRate.

- **RandRate**

Поле RandRate должно быть заполнено, если вы также установили флажок Random. Введите в это поле значение от 0 до 100, чтобы указать вероятность выполнения allocations на normal pageheap по сравнению с full pageheap.

- **Backward**

Обычно full pageheap защищает от buffer overflows, размещая guard pages в конце каждого heap block. Однако также можно обнаруживать buffer underrun problems, установив флажок Backward.

- **Unalign**

Установка флажка Unalign создает allocations, не выровненные по гранулярности выделения менеджера кучи. Все heap-based allocations выполняются на уровне гранулярности heap manager в зависимости от архитектуры, на которой вы работаете: 8 байт на x86 и 16 байт на x64. Например, если вы выделили 5 байт из кучи, возвращенное вам выделение в действительности имеет размер 8 байт. Оставшиеся 3 байта просто используются как padding, чтобы выровнять allocation на 8-байтную гранулярность выделения. Флажок Unalign удобен при ловле heap block overruns. Поскольку padding был добавлен, перезапись этих padding bytes не вызовет немедленный break в отладчике, потому что guard page не было на месте, чтобы поймать это. Чтобы такие overruns проявлялись, флажок Unalign возвращает allocations, не дополненные padding до гранулярности выделения heap manager. Обратите внимание, что некоторые компоненты требуют, чтобы allocations были выровнены по гранулярностям выделения heap manager, и не будут работать с установленным флажком Unalign. Хороший пример - RPC.

- **DeCommit**

Флажок DeCommit, если установлен, decommits guard pages, когда они больше не используются, что снижает потребление памяти.

- **Traces**

Установка флажка Traces заставляет test setting собирать stack traces для allocations и deallocations. Этот флажок включен по умолчанию, и большую часть времени его следует

оставлять включенным, поскольку наличие stack traces при отладке heap-related problems может спасти жизнь.

- **Protect**

Если этот флажок установлен, test setting защищает внутренние структуры кучи от повреждения. Это можно использовать для обнаружения случайных повреждений.

- **NoLock**

Если этот флажок установлен, test setting не будет выполнять critical section verification.

- **Faults**

Если этот флажок выбран, test setting включает heap fault injection. Если вы устанавливаете этот флажок, нужно также заполнить поля FaultRate и TimeOut.

- **FaultRate**

Если флажок Faults был установлен, нужно указать вероятность возникновения fault в heap manager, задав значение от 1 до 10 000.

- **TimeOut**

Если флажок Faults был установлен, можно изменить время в миллисекундах во время инициализации процесса, когда fault injection не происходит. Это почти всегда требуется, поскольку fault injection во время инициализации процесса может помешать процессу запуститься.

- **Addr**

Этот флажок позволяет управлять диапазоном адресов, в котором работает test setting. Если вы устанавливаете этот флажок, нужно также указать поля AddrStart и AddrEnd.

- **AddrStart**

Поле AddrStart указывает начальный адрес диапазона, в котором работает test setting.

- **AddrEnd**

Поле AddrEnd указывает конечный адрес диапазона, в котором работает test setting.

- **UseLFHGuardPages**

Этот флажок, если установлен, использует guard pages на low-fragmentation heap. Low-fragmentation heap - относительно новая куча, представленная в Windows 2003 Server и включенная по умолчанию в Windows Vista.

Прежде чем перейти дальше и обсудить различные stops, доступные как часть настройки heap test, нужно подробнее рассмотреть различие между normal и full pageheap.

Основная идея normal pageheap setting состоит в попытке поймать некоторые категории heap-related problems с помощью fill pattern на heap blocks. Когда выполняется heap allocation, heap manager заполняет все выделение fill pattern E0. Если выделенная память не инициализирована должным образом и на нее ссылаются, скорее всего возникнет access violation, поскольку приложение пытается использовать содержимое выделенной памяти, заполненной E0. Например, если allocation должен содержать допустимый указатель, но не содержит его из-за неудачной инициализации, приложение может попытаться dereference E0E0, что не является допустимым адресом, и мы увидим access violation. Затем, когда allocation освобождается, heap manager заполняет allocation fill pattern F0. Любая попытка использовать этот allocation после освобождения, скорее всего, также вызовет access violation.

Оба этих сценария - использование памяти без инициализации и использование памяти после освобождения - очень распространенные проблемы, ежедневно мучающие разработчиков.

Последнее замечание о fill patterns: помимо заполнения allocation заранее заданными шаблонами, Application Verifier добавляет prefix и postfix к каждому allocated block с заранее заданным шаблоном. В некоторых обстоятельствах это помогает обнаруживать heap under- и overrun problems. Допустим, у вас есть приложение, выделившее 1 КБ памяти. Приложение заполняет эту memory allocation тем, что считает нужным, и иногда заполняет более 1 КБ памяти, фактически приводя к сценарию, в котором allocated block слишком мал для хранения данных. Если приложение выходит за конец allocation на несколько байт, оно может сгенерировать fault при попытке использовать эту недопустимую память. Это никоим образом не гарантирует поимку всех случаев invalid heap usage, потому что можно писать в произвольные memory locations, но некоторые случаи оно ловит. Общая структура fill patterns, используемых этой test setting, показана на рисунке A.2.



Рисунок A.2. Fill patterns, используемые normal pageheap

Различные части блока на рисунке A.2 показаны в таблице A.2.

Таблица A.2. Конкретные fill patterns на основе инициализированных или освобожденных heap blocks

Часть блока	Fill pattern
Block Header Start Fill Pattern	Если allocation не инициализирован, заполнен 0xABCDAAAA; если allocation освобожден, заполнен 0xABCDAAA9.
Block Header	None
Block Header End Fill Pattern	Если allocation не инициализирован, заполнен 0xDCBAAAAA; если allocation освобожден, заполнен 0xDCBAAA9.
User Allocation Fill Pattern	Если allocation не инициализирован, заполнен 0xE0; если allocation освобожден, заполнен 0xF0.
Post Allocation Fill Pattern	0xA0A0A0A0

В момент allocation heap manager заполняет memory allocation правильным fill pattern. Использование памяти, содержащей fill patterns, может завершиться сбоем или не завершиться им, в зависимости от того, как используется память. Если приложение dereferences эти fill patterns, возникает access violation. Если оно просто перезаписывает fill patterns какими-то данными, но не разменовывает эти данные, возникает Application Verifier stop. Однако Application Verifier stop возникает только в момент deallocation, потому что это единственная возможность Application Verifier проверить целостность fill patterns.

Хотя хорошо, что он может проверять целостность heap block с помощью fill pattern checks, большую часть времени повреждение уже произошло, и бывает трудно вернуться назад и выяснить code path, вызвавший overwrite. К счастью, существует другой тип pageheap, известный как full pageheap, который гораздо агрессивнее и вызывает access violation в момент неправильного использования, тем самым локализуя код в точке, где он ведет себя неправильно.

Full pageheap работает, применяя так называемые guard pages, защищающие рассматриваемый allocation. Каждый раз, когда выполняется allocation, страница недоступной памяти добавляется в конец allocation, из-за чего любой доступ к памяти за концом allocation немедленно завершается сбоем. Аналогично guard page также может быть помещена в начало allocation, чтобы поймать любые записи в память, происходящие перед фактическим allocation. Кроме того, когда memory block освобождается, он не возвращается системе немедленно; вместо этого он некоторое время сохраняется в pool защищенной памяти, позволяя системе ловить любой код, который может

попытаться обратиться к allocation после его удаления.

Не должно удивлять, что использование этой test setting имеет большую цену. Цена состоит в дополнительной памяти, из-за guard pages, используемой для каждого allocation, выполненного приложением. Фактически pressure на память настолько высок, что рекомендуется иметь по меньшей мере 256 МБ RAM и 1 ГБ pagefile, чтобы эффективно использовать этот тест. Тем не менее, когда речь идет о memory-related problems, это самая исчерпывающая test setting, какую можно представить, и она ловит множество проблем на ранней стадии. Помимо guard pages, full pageheap использует fill patterns. Разметка fill pattern показана на рисунке A.3, а фактические fill patterns показаны в таблице A.3.

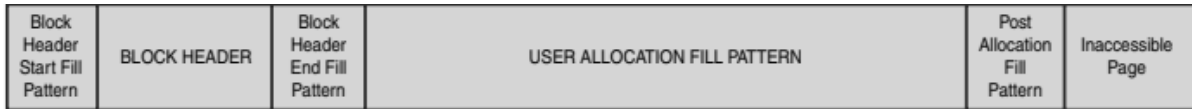


Рисунок A.3. Fill patterns, используемые full pageheap

Таблица A.3. Fill patterns для full pageheap

Часть блока	Fill pattern
Block Header Start Fill Pattern	Если allocated, заполнен 0xABCD8BBB; если allocation освобожден, заполнен 0xABCD8BBA.
Block Header	None
Block Header End Fill Pattern	Если allocated, заполнен 0xDCB8BBBB; если allocation освобожден, заполнен 0xDCB8BBBA.
User Allocation Fill Pattern	Если allocation не инициализирован, заполнен 0xC0; если allocation освобожден, заполнен 0xF0.
Post Allocation Fill Pattern	0xD0D0D0D0

Самое большое преимущество этой test setting по сравнению с предыдущей состоит в том, что guard pages гарантируют возникновение access violation в момент использования памяти, а не только при выделении и освобождении памяти. Это значительно упрощает поиск memory-related problems, потому что вы ловите код в действии, а не postmortem.

Настройка heap test содержит несколько stop-кодов, которые отслеживаются во время выполнения процесса. Таблица A.4 подробно описывает разные stop-коды. По умолчанию все stop-коды, связанные с heap, вызывают break в отладчике при срабатывании.

Таблица A.4. Stop-коды heap test setting

Stop code	Тест	Описание
00000001	Unknown error	Этот stop срабатывает, когда произошла heap error, которую нельзя классифицировать никаким другим образом.
00000002	Buffer overrun	В случаях, когда включен full pageheap, этот stop срабатывает, когда приложение пишет за конец buffer и попадает в guard page.
00000003	Unserialized heap	Heaps, которые не serialized, не должны использоваться concurrent threads. Этот stop срабатывает, если два или больше потоков одновременно обращаются к nonserialized heap.

Stop code	Тест	Описание
00000004	Large allocations	Этот stop срабатывает, если размер allocation, указанный API HeapAlloc и HeapReAlloc, неоправданно велик.
00000005	Invalid heap handles	Этот stop срабатывает, если недопустимый handle указан одному из heap API.
00000006	Mismatched heaps	Этот stop срабатывает, если блок памяти, выделенный в одной heap, освобождается в другой heap.
00000007	Over freed blocks	Этот stop срабатывает, если heap block освобождается дважды.
00000008	Generic heap corruption	Эта ошибка вызвана heap corruption, которую нельзя категоризировать.
00000009	Destroying the default process heap	Этот stop срабатывает, если приложение пытается уничтожить default process heap.
0000000A	Heap manager access violation	Этот stop срабатывает, если код heap manager генерирует access violation.
0000000B	Generic heap corruption	Этот stop срабатывает, если источник heap corruption не определен. Это может случиться при передаче адреса, указывающего на недоступную память, в heap-free API.
0000000C	Generic heap corruption	Этот stop срабатывает, если источник heap corruption не определен. Это может случиться при передаче адреса, указывающего на недоступную память, в heap-free API. Также это может произойти, если вы дважды освобождаете память.
0000000D	Use after delete	Этот stop срабатывает, если блок памяти был освобожден, а затем в него была выполнена запись.
0000000E	Use after delete	Этот stop срабатывает, если блок памяти был освобожден, а затем в него была выполнена запись. По сравнению со stop code 0000000E, этот stop-код обычно происходит в normal pageheap mode, который использует fill patterns для обнаружения heap problems.
0000000F	Buffer overrun	Этот stop работает, когда buffer overrun возникает в normal pageheap mode, через проверку fill patterns, применяемых normal pageheap.
00000010	Buffer underrun	Этот stop срабатывает, когда происходит buffer underrun и start stamp заголовка heap block поврежден.
00000011	Buffer underrun	Этот stop срабатывает, когда происходит buffer underrun и end stamp заголовка heap block поврежден.
00000012	Buffer underrun	Этот stop срабатывает, когда происходит buffer underrun и prefix заголовка heap block поврежден.
00000013	First chance access violation	Этот stop срабатывает, когда происходит first-chance access violation.

Stop code	Тест	Описание
00000014	Process heaps	Этот stop срабатывает, когда вызов GetProcessHeaps заставляет heap manager обнаружить несогласованности в структурах heap manager.

Full Pageheap или Normal Pageheap?

Test setting pageheap, использующая только fill patterns, также называется normal pageheap, тогда как ее более мощный родственник, full pageheap, использует guard pages, чтобы ловить programming mistakes в момент их возникновения. Имея эти два варианта, когда следует выбирать один вместо другого? Full pageheap имеет гораздо более высокий шанс поймать проблемы и значительно упрощает их отслеживание. С другой стороны, он использует так много памяти, что система может стать достаточно медленной, чтобы некоторые проблемы, например synchronization-related problems, даже не получили шанса проявиться. Обычно хорошая стратегия - всегда иметь normal pageheap во время разработки, а в регулярных контрольных точках включать более исчерпывающую настройку для получения покрытия. Чтобы уменьшить pressure от более исчерпывающей test setting, можно также сузить источник потенциально виновного memory code, настроив тестирование, например, на основе конкретной DLL или размера allocation.

Locks

Операционная система Windows является вытесняющей операционной системой, что по сути означает: любой поток, выполняющийся в системе, должен быть готов уступить управление процессором другому потоку.

Это создает очень распространенную дилемму для приложений, в которых одновременно выполняется больше одного потока.

Дилемма заключается в том, что любые ресурсы, совместно используемые потоками, должны строго контролироваться, чтобы избежать ситуации, когда несколько потоков одновременно обращаются к ресурсу: читают его и записывают в него.

Процесс управления доступом к ресурсу называется synchronization.

Один очень распространенный прием, используемый приложениями, состоит в применении встроенного synchronization object, называемого critical section.

Critical section гарантирует, что только один поток за раз может обращаться к любому ресурсу, защищенному этой critical section.

Перед обращением к shared resource поток входит в critical section, использует ресурс, а затем освобождает critical section.

Если другой поток пытается войти в critical section, пока в нее уже вошел другой поток, этот поток блокируется на critical section до тех пор, пока она не будет освобождена; после этого поток пробуждается и получает возможность войти в нее.

Перед использованием critical section должна быть инициализирована; в этот момент операционная система выделяет память, необходимую для успешной инициализации объекта.

Когда critical section больше не нужна, ее необходимо удалить, чтобы избежать memory leaks.

Поток, вошедший в critical section, может повторно входить в нее любое число раз, пока для каждого входа существует соответствующий вызов освобождения critical section.

Само собой разумеется, critical sections очень часто используются неправильно и вызывают проблемы, от memory leaks до deadlocks, просто из-за несоблюдения правильного protocol.

Настройка Locks test предоставляет набор проверок, относящихся к critical sections, и может сэкономить ценное время, которое иначе было бы потрачено на отладку concurrency problems. Таблица A.5 перечисляет все tests, предоставляемые этой test setting. По умолчанию все stop-коды, связанные с locks, вызывают break в отладчике при срабатывании.

Таблица A.5. Stop-коды Locks test setting

Stop code	Тест	Описание
00000200	Thread state does not allow holding critical sections	Этот stop срабатывает, когда поток завершен, приостановлен или находится в состоянии, в котором он не может удерживать critical section.
00000201	Active critical sections when unloading DLL	Если при выгрузке DLL обнаруживается active critical section, Application Verifier останавливает выполнение.
00000202	Freeing of a heap block that contains an active critical section	Когда heap allocation освобождается и содержит active critical section, Application Verifier останавливает выполнение. Этот test предотвращает потенциальную утечку critical section.
00000203	Initialize only once	Critical section может быть инициализирована только один раз. Инициализация critical section больше одного раза является undefined behavior и представляет ошибки в коде.
00000204	Forgetting to free the critical section	Чтобы правильно удалить или освободить critical section, необходимо использовать API DeleteCriticalSection. Этот stop срабатывает, если block of memory, содержащий critical section, был освобожден, но critical section не была удалена с помощью API DeleteCriticalSection.
00000205	Invalid DebugInfo pointer	Этот stop срабатывает, если поле DebugInfo critical section указывает на освобожденную память.
00000206	Verify correctness of owner thread ID	Каждый раз, когда выполняется вход в critical section, записывается thread ID. Если thread ID недопустим в текущем execution context, Application Verifier останавливает выполнение.
00000207	Verify correctness of the recursion count	Потоки, входящие в critical section больше одного раза, увеличивают recursion count critical section. Если recursion count недопустим в текущем execution context, Application Verifier останавливает выполнение.
00000208	Delete without initialization	Если critical section удаляется без инициализации, Application Verifier останавливает выполнение.
00000209	Release count matches	Если поток освобождает critical section больше раз, чем входил в нее, Application Verifier останавливает выполнение.

Stop code	Тест	Описание
00000210	Use prior to initialization	Если critical section используется без инициализации или используется после удаления, Application Verifier останавливает выполнение.
00000211	Reinitialization	Если critical section повторно инициализируется текущим потоком, Application Verifier останавливает выполнение.
00000212	Invalid deletion of critical section (VirtualFree)	Когда block of memory, содержащий active critical section, освобождается с помощью VirtualFree, но critical section не была освобождена через API DeleteCriticalSection, срабатывает этот stop.
00000213	Invalid deletion of critical section (UnmapViewOfFile)	Когда block of memory, содержащий active critical section, отображение которого снимается с помощью API UnmapViewOfFile, но critical section не была освобождена через API DeleteCriticalSection, срабатывает этот stop.
00000214	Leaving an unowned critical section	Этот stop срабатывает, когда поток вызывает LeaveCriticalSection для critical section, которой он не владеет.
00000215	Private locks usage	Этот stop срабатывает, если поток пытается войти в private critical section, определенную в другой DLL.

Все tests, перечисленные в таблице A.5, являются распространенными programming errors, с которыми можно столкнуться при работе с critical sections.

Без этой test setting большинство таких ошибок проявляется как memory leaks, dead locks, live locks и тому подобное, и их выяснение может привести к часам утомительной отладки.

Эта test setting ловит большинство проблем в точке их возникновения, а не в случайных точках postmortem.

Всякий раз, когда вы пишете код, которому нужно управлять synchronization между несколькими потоками, эту test setting следует включать.

Memory

Настройка memory test ловит многочисленные programming mistakes, связанные с работой с памятью.

Проблемы вроде передачи недопустимых параметров в memory API(s), неудачной инициализации объектов и выгрузки модулей покрываются широким набором stop-кодов.

Таблица A.6 подробно описывает все stop-коды, доступные для этой test setting.

По умолчанию все stop-коды, связанные с memory, вызывают break в отладчике при срабатывании.

Таблица A.6. Stop-коды Memory test setting

Stop code	Тест	Описание
00000600	VirtualFree or Dll unload	Этот stop срабатывает, когда Application Verifier обнаруживает вызов VirtualFree с недопустимым адресом или выгрузку DLL с недопустимым start address.

Stop code	Тест	Описание
00000601	VirtualAlloc	Этот stop срабатывает, когда Application Verifier обнаруживает вызов VirtualAlloc с недопустимым start address или размером allocation.
00000602	MapViewOfFile	Этот stop срабатывает, когда Application Verifier обнаруживает вызов MapViewOfFile с недопустимым base address или размером mapping.
00000603	IsBadXXXPtr	Этот stop срабатывает, когда Application Verifier обнаруживает вызов IsBadXXXPtr с недопустимым адресом.
00000604	IsBadXXXPtr	Этот stop срабатывает, когда Application Verifier обнаруживает вызов IsBadXXXPtr с memory location, которая освобождена.
00000605	IsBadXXXPtr	Этот stop срабатывает, когда Application Verifier обнаруживает вызов IsBadXXXPtr с memory address, содержащим guard page. Это может быть особенно плохо в случаях, когда адрес указывает на stack.
00000606	IsBadXXXPtr	Этот stop срабатывает, когда Application Verifier обнаруживает вызов IsBadXXXPtr с null address.
00000607	IsBadXXXPtr	Этот stop срабатывает, когда Application Verifier обнаруживает вызов IsBadXXXPtr с недопустимым адресом или размером memory buffer.
00000608	Dll unload	Этот stop срабатывает, когда DLL выгружается с недопустимым start address.
00000609	VirtualFree	Этот stop срабатывает, когда вызов VirtualFree используется для адреса, который фактически является частью stack текущего потока.
0000060A	VirtualFree	Этот stop срабатывает, когда VirtualFree вызывается с неправильным значением параметра FreeType.
0000060B	VirtualFree	Этот stop срабатывает, когда VirtualFree вызывается с адресом, который уже был освобожден.
0000060C	VirtualFree	Этот stop срабатывает, когда VirtualFree вызывается с MEM_RELEASE, а параметр size больше 0.
0000060D	DllMain exceptions	Этот stop срабатывает, когда реализация DllMain возбуждает exception.
0000060E	Thread exceptions	Этот stop срабатывает, когда поток возбуждает exception.
0000060F	IsBadXXXPtr	Этот stop срабатывает, когда вызов IsBadXXXPtr приводит к выбрасыванию exception.

Stop code	Тест	Описание
00000610	VirtualFree	Этот stop срабатывает, когда VirtualFree вызывается с MEM_RESET и null в качестве первого параметра.
00000612	HeapFree	Этот stop срабатывает, когда выполняется вызов HeapFree с указателем, который является частью stack текущего потока.
00000613	UnmapViewOfFile	Этот stop срабатывает, когда выполняется вызов UnmapViewOfFile для block of memory, который является частью stack текущего потока.
00000614	Incorrect address	Этот stop срабатывает, если в определенные API(s) передается недопустимый адрес. Например, передача null в RtlInitializeResource некорректна.
00000615	Incorrect address	Этот stop срабатывает, если в определенные API(s) передается недопустимый адрес. Например, передача null в EnterCriticalSection некорректна.
00000616	Non executable code	Этот stop срабатывает, когда приложение пытается выполнить code из nonexecutable memory address.
00000617	Failed buffer initialization	Этот stop срабатывает, когда при инициализации output parameter, переданного в Win32 или CRT API(s), возбуждается exception. Обычно это указывает, что связанный параметр size некорректен.
00000618	HeapSize	Этот stop срабатывает при вызове API HeapSize с адресом block of memory, который уже был освобожден.
00000619	VirtualFree	Этот stop срабатывает, когда выполняется вызов VirtualFree с недопустимым base address. Недопустимый в этом случае означает, что указанный base address не совпадает с base address, возвращенным соответствующими API(s) VirtualAlloc или VirtualAllocEx.
0000061A	UnmapViewOfFile	Этот stop срабатывает, когда выполняется вызов UnmapViewOfFile с недопустимым base address. Недопустимый в этом случае означает, что указанный base address не совпадает с base address, возвращенным соответствующими API(s) MapViewOfFile или MapViewOfFileEx.
0000061B	Thread pool thread exceptions	Этот stop срабатывает, когда callback function для thread pool thread возбуждает exception.
0000061C	Nonexecutable code	Этот stop срабатывает, когда приложение пытается выполнить code из nonexecutable memory address.

Stop code	Тест	Описание
0000061D	Executable heap	Этот stop срабатывает, когда приложение пытается создать executable heap.
0000061E	Executable memory	Этот stop срабатывает, когда приложение пытается выделить executable memory.

Настройка memory test имеет одно настраиваемое свойство, называемое ExecWritePage. Оно позволяет управлять тем, должна ли test setting проверять, создает или выделяет ли приложение executable and writable memory.

ThreadPool

Нативный Windows thread pool - мощный и удобный компонент, который можно использовать, когда приложению требуется несколько потоков.

Вместо того чтобы создавать собственный thread pool и обеспечивать его оптимальную работу, можно позволить операционной системе принимать решения о том, когда и как создавать потоки за вас.

В конце концов, именно операционная система обладает наибольшим знанием для принятия этих сложных решений.

Однако использование thread pool несет большую ответственность.

Существовало множество трудно отслеживаемых bugs, возникших из-за неправильного использования thread pool.

Например, изменение состояния потока и отсутствие сброса этого состояния перед возвратом потока в thread pool может привести к серьезным проблемам.

Эта test setting ловит многие из таких проблем и уведомляет вас о потенциальных проблемах в коде.

Таблица A.7 подробно описывает stops, доступные для этой test setting.

По умолчанию все stop-коды, связанные с thread pool, вызывают break в отладчике при срабатывании.

Таблица A.7. Stop-коды ThreadPool test setting

Stop code	Тест	Описание
00000700	Changed thread priority	Если приложение изменяет priority потока и не сбрасывает ее перед передачей потока обратно в thread pool, срабатывает этот stop.
00000701	Changed thread affinity	Если приложение изменяет affinity потока и не сбрасывает ее перед передачей потока обратно в thread pool, срабатывает этот stop.
00000702	Unprocessed messages	Если перед возвратом в thread pool у потока остаются какие-либо unprocessed messages, срабатывает этот stop.
00000703	Active windows	Если при возврате в thread pool у потока есть какие-либо живые и active windows, срабатывает этот stop.

Stop code	Тест	Описание
00000704	Exit or termination of thread	Если для thread pool thread вызывается ExitThread или TerminateThread, срабатывает этот stop.
00000705	Impersonation	Если thread pool thread impersonates другого пользователя, но не отменяет impersonation перед возвратом потока в thread pool, срабатывает этот stop.
00000706	Dedicated thread	В некоторых ситуациях определенные API(s) должны вызываться на persistent или dedicated thread, например API RegNotifyChangeKeyValue. Если вы не запросили выдачу dedicated thread из thread pool, срабатывает этот stop.
00000707	Transaction handle	Если приложение забывает сбросить transaction handle перед возвратом потока в thread pool, срабатывает этот stop.
00000708	COM initialization	При инициализации COM в потоке необходимо позаботиться об uninitialize COM перед завершением потока. Этот stop срабатывает, если есть несбалансированные вызовы COM initialize и uninitialize.

Настройка ThreadPool test также имеет одно связанное свойство с именем AsyncCheck, которое позволяет verifier проверять asynchronous calls, требующие persistent thread.

Чтобы включить эту настройку, укажите 1.

По умолчанию это свойство установлено в 0 (off).

TLS

Thread local storage (TLS) - распространенный механизм, используемый для выделения local thread-specific data.

Чтобы выделить TLS data, приложение использует семейство TLS API(s).

Чтобы обеспечить правильное использование этих API(s), эта test setting перехватывает вызовы API(s) и останавливает выполнение, если встречается недопустимое использование.

Таблица A.8 подробно описывает stops, входящие в эту test setting.

По умолчанию все stop-коды, связанные с TLS, вызывают break в отладчике при срабатывании.

Таблица A.8. Stop-коды TLS test setting

Stop code	Тест	Описание
00000350	TlsIndex leak	Если DLL, выделившая TLS index, выгружается без освобождения index, этот stop срабатывает, чтобы избежать TLS index leaks.
00000351	Corrupted TLS state	Если data structure, используемая для хранения TLS slots, повреждается, срабатывает этот stop. Скорее всего, причиной является случайная memory corruption.

Stop code	Тест	Описание
00000352	Invalid TLS index used	Если приложение использует TLS index, считающийся недопустимым, срабатывает этот stop. Распространенный источник этой проблемы - использование TLS index, который уже был освобожден.

FilePaths

Приложения, использующие system file paths, например получающие windows directory или system directory, могут сталкиваться с проблемами, связанными с использованием неправильных путей.

Эта test setting проверяет, что system paths, полученные через Win32 APIs, корректны.

API, которые перехватывает эта test setting, перечислены в таблице A.9.

По умолчанию все stop-коды, связанные с file paths, не вызывают break в отладчике при срабатывании, а записывают stop code в log file вместе с offending stack trace.

Таблица A.9. Stop-коды FilePaths test setting

Stop code	Тест	Описание
00002400	Common program path	Этот stop срабатывает, если приложение использовало common Program directory path, который не был получен с помощью правильных API(s).
00002401	Common Start menu directory path	Этот stop срабатывает, если приложение использовало common Start menu directory path, который не был получен с помощью правильных API(s).
00002402	Programs directory path	Этот stop срабатывает, если приложение использовало program's directory path, который не был получен с помощью правильных API(s).
00002404	Start Menu directory path	Этот stop срабатывает, если приложение использовало Start Menu Directory Path, который не был получен с помощью правильных API(s).
00002405	Windows Temp path	Этот stop срабатывает, если приложение использовало Windows Temp path, который не был получен с помощью правильных API(s).
00002406	Windows directory path	Этот stop срабатывает, если приложение использовало Windows directory path, который не был получен с помощью правильных API(s).
00002407	System Windows directory path	Этот stop срабатывает, если приложение использовало System Windows directory path, который не был получен с помощью правильных API(s).
00002408	System directory path	Этот stop срабатывает, если приложение использовало System directory path, который не был получен с помощью правильных API(s).

Stop code	Тест	Описание
0000240A	My Documents directory path	Этот stop срабатывает, если приложение использовало My Documents directory path, который не был получен с помощью правильных API(s).

HighVersionLie

Очень распространенная programming mistake, приводящая к сломанным приложениям, состоит в зависимости от hard-coded Windows version number.

Слишком часто программисты, которым для работы кода требуется конкретная версия Windows, проверяют именно эту версию и только ее.

Итоговый результат такого подхода в том, что когда выходит новая версия Windows, приложение просто перестает работать, потому что не учло увеличение version number, хотя само приложение могло бы работать совершенно нормально.

Чтобы предотвращать подобные ситуации, эта test setting позволяет разработчику приложения тестировать свой код относительно "future" versions of Windows.

Это работает так: Application Verifier просто перехватывает вызовы API GetVersion и GetVersionEx и возвращает version, которая по умолчанию превосходит самую последнюю версию Windows.

Например, запуск простого test application, вызывающего GetVersionEx, на Windows XP Professional Service Pack 2 дает следующие результаты:

```
Local var @ 0x7fea4 Type _OSVERSIONINFOEXA
+0x000 dwOSVersionInfoSize : 0x9c
+0x004 dwMajorVersion : 5
+0x008 dwMinorVersion : 1
+0x00c dwBuildNumber : 0xa28
+0x010 dwPlatformId : 2
+0x014 szCSDVersion : [128] "Service Pack 2"
+0x094 wServicePackMajor : 2
+0x096 wServicePackMinor : 0
+0x098 wSuiteMask : 0x300
+0x09a wProductType : 0x1 ''
+0x09b wReserved : 0 ''
```

Здесь dwMajorVersion (5) и dwMinorVersion (1) указывают на Windows XP (список всех Windows versions см. в GetVersionEx в MSDN).

Включение HighVersionLie test и повторный запуск приложения дают следующие результаты:

```
Local var @ 0x7fea4 Type _OSVERSIONINFOEXA
+0x000 dwOSVersionInfoSize : 0x9c
+0x004 dwMajorVersion : 7
+0x008 dwMinorVersion : 3
+0x00c dwBuildNumber : 0xe10
+0x010 dwPlatformId : 2
+0x014 szCSDVersion : [128] ""
+0x094 wServicePackMajor : 2
+0x096 wServicePackMinor : 0
+0x098 wSuiteMask : 0x300
+0x09a wProductType : 0x1 ''
+0x09b wReserved : 0 ''
```

На этот раз major и minor versions являются несуществующими Windows versions, и приложение должно обработать эту ситуацию соответствующим образом.

По умолчанию Application Verifier использует 7 как Major version, 3 как minor version и 0xe10 как build number.

Эти settings и другие можно очень легко настроить, изменив свойства test setting.

Таблица А.10 объясняет доступные свойства.

Таблица А.10. Свойства HighVersionLie test setting

Property	Description
Major version	HighVersionLie использует значение этого свойства как major version.
Minor version	HighVersionLie использует значение этого свойства как minor version.
Build number	HighVersionLie использует значение этого свойства как build number.
Service pack major	HighVersionLie использует значение этого свойства как major service pack version.
Service pack minor	HighVersionLie использует значение этого свойства как minor service pack version.
Suite mask	HighVersionLie использует значение этого свойства как suite mask.
Product type	HighVersionLie использует значение этого свойства как product type.
CSD version	HighVersionLie использует значение этого свойства как CSD version.

Как видно, помимо изменения major, minor и build numbers, можно изменить множество других options.

Обычно standard settings достаточно хороши, поскольку большую часть времени нужно убедиться, что приложение не сломается с новыми releases of Windows.

Stop-коды, доступные для этой test setting, все являются placeholders и служат для указания, что test setting была включена.

InteractiveServices

Services обычно выполняются под account с высокими privileges.

Поэтому любые security compromises обычно оказывают гораздо более сильное влияние на систему по сравнению с атаками на процессы, выполняющиеся под менее привилегированными accounts.

Атаки могут, например, выполняться с использованием graphical user interface subsystem Windows. Этот тип атаки также известен как shatter attack.

Один распространенный механизм состоит в том, что менее привилегированный attacking process отправляет определенные Windows messages более привилегированному приложению и получает unprivileged access.

Например, это может быть information disclosure или elevation of privilege.

Services разрешено отправлять user interface elements другим interactive applications или просто самим выполняться как interactive service.

Поскольку в таких типах services необходимо проявлять особую осторожность, чтобы избежать exploits, эта test setting позволяет пользователю быстро понять, является ли service interactive.

Она также показывает, отправляет ли service user interface elements, например Windows messages, другим приложениям.

Таблица А.11 подробно описывает stops, входящие в эту test setting.

По умолчанию все stop-коды для этой test setting записывают результаты test в log file и включают stack trace.

Таблица A.11. Stop-коды InteractiveServices test setting

Stop code	Тест	Описание
00002800	Interactive service creation	Этот stop срабатывает, когда приложение создает interactive service с помощью API CreateService. Interactive services могут быть security risk.
00002801	Local system interactive service	Приложение выполнялось как interactive service под local system account, создавая security risk.
00002802	Usage of MessageBox API	Использование API MessageBox для отправки notifications на interactive desktop может быть security threat.
00002803	Usage of MessageBoxEx API	Использование API MessageBoxEx для отправки notifications на interactive desktop может быть security threat.
00002804	Interactive Desktop	Приложение вызвало OpenWindowStation и OpenDesktop, чтобы отправить UI messages на interactive desktop. Это может быть security risk.
00002805	Interactive Desktop	Приложение вызвало OpenDesktop, чтобы отправить UI messages на interactive desktop. Это может быть security risk.
00002806	Interactive Desktop	Приложение вызвало OpenWindowStation и OpenInputDesktop, чтобы отправить UI messages на interactive desktop. Это может быть security risk.
00002807	Interactive Desktop function pointer	Приложение попыталось получить address функции OpenDesktop в попытке отправить ей UI messages.

KernelModeDriverInstall

В Windows drivers выполняются в kernel mode; следовательно, они имеют доступ к system-critical data.

Когда drivers дают сбой, это может значительно повлиять на систему и, в худшем случае, привести систему к blue screen.

Чтобы убедиться, что drivers устанавливаются через правильные channels, разработчики должны использовать корректные API(s).

Эта test setting гарантирует, что applications используют правильные API(s) при установке drivers.

Таблица A.12 подробно описывает verifier stops, входящие в эту test setting.

По умолчанию все stop-коды, связанные с kernel mode driver install, не вызывают break в отладчике при срабатывании; вместо этого stop code записывается в log file вместе с offending stack trace.

Таблица A.12. Stop-коды KernelModeDriverInstall test setting

Stop code	Тест	Описание
00002305	Driver install	CreateServiceA был вызван для установки kernel mode driver.
00002306	File system filter driver	CreateServiceW был вызван для установки file system filter driver.

Stop code	Тест	Описание
00002307	File system filter driver	CreateServiceA был вызван для установки file system filter driver.
00002308	Driver install	CreateServiceW был вызван для установки kernel mode driver.
00002309	Driver install API	Приложение установило driver без использования правильного API (CreateService).
0000230A	File system driver install	Приложение установило file system driver без использования правильного API (CreateService).
0000230B	Driver install API	Приложение установило driver без использования правильного API (CreateService), и image path не был указан.
0000230C	File system driver install	Приложение установило file system driver без использования правильного API (CreateService), и image path не был указан.
0000230D	Driver install API	Приложение установило driver без использования правильного API (CreateService), и service type не был указан.

Low Resource Simulation

Чтобы обеспечить robustness приложения, приложение должно быть способно правильно работать не только при normal system load, но и тогда, когда resources становятся дефицитными.

Канонический пример этого - проверка того, что приложение ведет себя правильно, когда системе, в которой оно выполняется, не хватает памяти.

В таких обстоятельствах одна из проблем обеспечения robustness приложения состоит в simulation environment, в которой resources ограничены.

Настройка Low Resource Simulation test помогает тестировать такие scenarios, позволяя определить условия, при которых low resource faults должны возникать во время выполнения.

Эта test setting не имеет определенных для нее stops; вместо этого она позволяет изменять ряд параметров test setting.

Таблица A.13 подробно описывает разные параметры.

Таблица A.13. Свойства Low Resource Simulation test setting

Property	Description
Include	Управляет тем, в каких DLLs должны возникать faults. DLLs следует указывать без paths и по одной на строку. * можно использовать, чтобы указать, что все modules должны быть включены в fault injection.
Exclude	Список DLLs, которые нужно исключить из fault injection. DLLs следует указывать по одной на строку.
Timeout	Указывает, сколько milliseconds должно пройти при process startup до начала fault injection.
WAIT	Число от 0 до 100, указывающее вероятность того, что WaitForXXX API(s) завершатся с ошибкой из-за fault injection.

Property	Description
HEAP_ALLOC	Число от 0 до 100, указывающее вероятность того, что HEAP_ALLOC API(s) завершатся с ошибкой из-за fault injection.
VIRTUAL_ALLOC	Число от 0 до 100, указывающее вероятность того, что VIRTUAL_ALLOC API(s) завершатся с ошибкой из-за fault injection.
REGISTRY	Число от 0 до 100, указывающее вероятность того, что REGISTRY API(s) завершатся с ошибкой из-за fault injection.
FILE	Число от 0 до 100, указывающее вероятность того, что FILE API(s) завершатся с ошибкой из-за fault injection.
EVENT	Число от 0 до 100, указывающее вероятность того, что EVENT API(s) завершатся с ошибкой из-за fault injection.
MAP_VIEW	Число от 0 до 100, указывающее вероятность того, что MAP_VIEW API(s) завершатся с ошибкой из-за fault injection.
OLE_ALLOC	Число от 0 до 100, указывающее вероятность того, что OLE_ALLOC API(s) завершатся с ошибкой из-за fault injection.
STACKS	Если установлено в TRUE, указывает, что feature роста stack должна быть отключена для simulation low resources. Если установлено в FALSE, stack growth происходит.

LuaPriv

Настройка LuaPriv test расшифровывается как Limited User Account Predictor и служит двум целям:

- Предсказывает, способно ли приложение выполняться с reduced privilege set, например как normal user.
- Определяет возможные проблемы, когда приложение выполняется с reduced privilege set, то есть как normal user.

Таблица A.14 перечисляет все stop-коды, определенные этой test setting.

По умолчанию все stop-коды, связанные с limited user account, не вызывают break в отладчике при срабатывании.

Вместо этого stop code записывается в log file вместе с offending stack trace.

Таблица A.14. Stop-коды LuaPriv test setting

Stop code	Тест	Описание
00003300	Name canonicalization failure	Этот stop срабатывает, когда Application Verifier пытается canonize имя object, но не может этого сделать.
00003301	Unable to find canonical name	Этот stop срабатывает, когда Application Verifier не может найти canonical name object.
00003302	Failure to open object	Этот stop срабатывает, когда Application Verifier не может получить information about an object из-за ошибок при попытке открыть object.

Stop code	Тест	Описание
00003303	HKEY_CURRENT_USER	Этот stop срабатывает, когда Application Verifier не может интерпретировать registry hive HKEY_CURRENT_USER. В результате registry keys, расположенные под этим registry hive, должны быть помечены как restrictive.
00003304	USERPROFILE	Этот stop срабатывает, если environment variable USERPROFILE не найдена. Это может заставить Application Verifier считать, что profile текущего пользователя недоступен, и поэтому перечислить некоторые files and directories как restrictive.
00003305	Safe object	Этот stop срабатывает из-за того, что Application Verifier считает object safe.
00003306	Listed object namespace	Этот stop срабатывает в Application Verifier, когда встречается object в listed namespace. Этот namespace недоступен для записи standard users.
00003307	No namespace	Object был создан без указания namespace, из-за чего он создается в global namespace, который недоступен для записи standard users.
00003308	File/directory location	Этот stop создается, когда Application Verifier пытается locate file or directory, но не может этого сделать.
00003309	Failure to open parent object	Application Verifier не может открыть parent object, пытаясь определить, смогут ли standard users создавать child objects.
0000330A	Running as administrator	Application Verifier обнаруживает, что приложение выполнялось as an administrator. Это просто informative information, позволяющая убедиться, что scenarios, проверяемые запуском as a standard user, не выполняются непреднамеренно as administrators.
0000330B	Failed SID conversion	Application Verifier не может преобразовать human-readable form security identifier (SID) во внутреннюю Windows form.
0000330C	GetTokenInformation	Приложение вызывает API GetTokenInformation, запрашивая listed class of information. Хотя это может работать as a standard user, обычно это подразумевает, что приложение должно выполняться as an administrator.
0000330D	Privilege does not exist	Проверяемая privilege не существует в версии Windows, на которой выполняется приложение.
0000330E	Privilege lookup failure	Application Verifier не может выполнить lookup указанной privilege через LUID.
0000330F	Successful privilege request	Приложение запрашивает и получает privilege, которая не предоставляется standard users.

Stop code	Тест	Описание
00003310	Failed privilege request	Приложение не может запросить privilege, которая не предоставляется standard users.
00003311	Successful access to listed privilege	Приложение запускается как standard user с доступом к listed privilege.
00003312	Failed access to the listed privilege	Приложение запускается как user без доступа к listed privilege, хотя privilege lookup был successful.
00003313	Listed privilege	Приложение запускается как user без доступа к listed privilege.
00003314	Failed registry query	Application Verifier пытается query registry value, но безуспешно.
00003315	INI file registry mapping	Приложение использует INI file, mapped to a registry key, a syntax INI file неизвестен.
00003316	Failed profile access	Приложение пытается получить access к заданному profile, но терпит failure.
00003317	Granted object access	Приложению предоставлен access к specified object. Standard users должны иметь возможность access specified object.
00003318	Granted object access	Приложению был предоставлен access к specified object. Standard users могут быть не способны access specified object.
00003319	Failed security descriptor conversion	Application Verifier пытается преобразовать security descriptor в human-readable form, но не может этого сделать.
0000331A	Object access failure	Приложение запрашивает access к object, и ему отказано.
0000331B	Object access	Приложение запрашивает access к object, который grants access только trusted users.
0000331C	Object access	Приложение запрашивает access к object, который grants access только trusted users. За этим message всегда следуют другие messages.
0000331D	Object access	Приложение запрашивает access к object, который grants access только privileged users и owners of the object.
0000331E	Object access	Приложение запрашивает access к object, который grants access по меньшей мере одному nonprivileged user.
0000331F	Object access	Приложение запрашивает access к object, который does not grant access anyone.
00003320	Security descriptor analysis	Application Verifier пытается analyze security descriptor object, разбивая его на отдельные pieces, но не может этого сделать.

Stop code	Тест	Описание
00003321	Security descriptor analysis	Application Verifier пытается analyze ACL в discretionary access list security descriptor object, но не может этого сделать.
00003322	Object access	Приложение запрашивает maximum allowed access к object, и Application Verifier решает analyze application так, как будто все object privileges фактически required.
00003323	Object access	Приложение запрашивает maximum allowed access к object. Application Verifier пытается определить, какие privileges фактически required by the application, но не может этого сделать.
00003324	Unknown permissions	Приложение запросило permissions, неизвестные Application Verifier.
00003325	Profiles	Приложению предоставлен access к profile.
00003326	Security Identifiers	Приложение queries Windows, является ли listed security identifier частью user's token. Эта operation обычно restricted to users in the listed group.
00003327	Security Identifiers	Приложение queries Windows, является ли listed security identifier (SID) частью user's token, хотя SID не identifiable.
00003328	Profiles	Приложение вызвало API WriteProfile с input, который может fail при запуске as a standard user.

DangerousAPIs

Многие API(s) в Win32 чрезвычайно powerful.

Они powerful настолько, что при использовании API(s) необходимо проявлять большую осторожность, чтобы избежать problems, которые могут возникнуть при их использовании.

Эта test setting проверяет вызовы этих dangerous API(s).

Таблица A.15 подробно описывает stops, доступные для этой test setting.

По умолчанию все stop-коды, связанные с dangerous API, вызывают break в отладчике при срабатывании.

Таблица A.15. Stop-коды DangerousAPIs test setting

Stop code	Тест	Описание
00000100	TerminateThread	Этот stop срабатывает, если приложение завершает thread с помощью API TerminateThread. TerminateThread с высокой вероятностью вызывает problems, например deadlocks и data corruption.

Stop code	Тест	Описание
00000101	Stack commit size	Этот stop срабатывает, если stack size приложения установлен так, что stack growth не допускается в случаях, когда могут возникать stack overflow exceptions, например при low memory conditions.
00000102	ExitProcess	Этот stop срабатывает, если приложение завершает process с несколькими running threads с помощью API ExitProcess. ExitProcess internally uses API TerminateThread, что может вызывать stability problems; см. stop code 00000100.
00000103	LoadLibrary	Этот stop срабатывает, если приложение вызывает LoadLibrary в функции DllMain. Вызов этого API из функции DllMain может приводить к crashes или deadlocks.
00000104	FreeLibrary	Этот stop срабатывает, если приложение вызывает FreeLibrary в функции DllMain. Вызов этого API из функции DllMain может приводить к crashes или deadlocks.

Настройка DangerousAPIs test имеет одно свойство, DllMainCheck, которое можно изменить. Если это свойство установлено в TRUE, test setting ловит любые вызовы из DllMain к API LoadLibrary или FreeLibrary.

DirtyStacks

Uninitialized stack variables являются распространенным источником bugs.

Это особенно верно для uninitialized stack pointers.

Чтобы гарантировать обнаружение таких bugs, эта test setting периодически заполняет unused portion of the stack определенным pattern.

Когда приложение пытается использовать uninitialized variable, это может привести к access violation из-за обращения к memory, initialized with a fill pattern, который считается invalid memory access.

Эта test setting приводит к выбрасыванию exceptions, а не к hard stop.

TimeRollOver

При работе с API(s) GetTickCount и TimeGetTime критически важно, чтобы developer обрабатывал случаи, когда происходит rollover.

Однако для тестирования rollover cases должна существовать возможность быстро вызвать rollover.

Эта test setting позволяет управлять тем, когда должен произойти rollover.

Для этой test setting не существует actual stops.

Вместо этого существует свойство test setting с именем Delay, которое можно установить, чтобы указать number of seconds задержки перед rollover.

PrintAPI and PrintDriver

При работе с Print API(s) могут проявляться многочисленные problems.

И PrintAPI test setting, и PrintDriver test setting включают набор tests, обеспечивающих правильное использование API(s).